

Лекции по Алгоритми

работна версия, 2021 година

Минко Марков

Copyright © 2014 – 2021 Минко Марков (Minko Markov)
All rights reserved.
Maple is a trademark of Waterloo Maple Inc.

Съдържание

1 Въведение в алгоритмите. 1
1.1 Алгоритъм . . . 1
1.1.1 Опит за дефиниция . . . 1
1.1.2 Изчислителни задачи . . . 4
1.1.2.1 Фундамент . . . 4
1.1.2.2 Видове изчислителни задачи . . . 7
1.1.2.3 Как се описват изчислителните задачи . . . 8
1.1.3 Алгоритми и изчислителни задачи . . . 9
1.1.4 Елементарни инструкции . . . 11
1.1.5 Knuth за алгоритмите . . . 14
1.1.6 Изчислителни методи. Итератори. . . . 17
1.2 Евклидовият алгоритъм . . . 19
1.2.1 Оригиналният Евклидов алгоритъм . . . 20
1.2.2 Съвременни варианти на Евклидовия алгоритъм. . . . 22
1.3 Как описваме алгоритми . . . 22
1.3.1 Псевдокод . . . 22
1.3.2 Други видове описания на алгоритми . . . 23

2 Анализ на алгоритми. 26
2.1 Анализ на коректността . . . 27
2.1.1 Доказателство за финитност . . . 28
2.1.2 Верификация на рекурсивни алгоритми: по индукция . . . 31
2.1.2.1 Нарастване с единица . . . 31
2.1.2.2 Максимум на несортиран масив, рекурсивен . . . 32
2.1.2.3 Максимум на битоничен масив . . . 33
2.1.2.4 Евклидовият алгоритъм в рекурсивен вариант . . . 35
2.1.3 Верификация на итеративни алгоритми: с инвариант . . . 37
2.1.3.1 Максимум на несортиран масив, итеративен . . . 42
2.1.3.2 Странен алгоритъм за сумиране на елементите на масив . . . 43
2.1.3.3 Евклидовият алгоритъм в итеративен вариант . . . 46
2.1.3.4 Alg2SUM . . . 47
2.1.3.5 Alg3SUMcount . . . 48
2.1.3.6 Една алгоритъм върху масиви . . . 54
2.2 Анализ на сложността . . . 56
2.2.1 Големина на входа . . . 57
2.2.2 Въведение в сложността по време . . . 58
2.2.3 Определение на сложността по време . . . 63
2.2.4 Анализ на сложността по памет . . . 67
2.2.5 Проблеми със сложността по време, до които води нашият модел . . . 67

2.3 Асимптотични нотации . . . 68
2.3.1 Опит за намиране на точната сложност по време на Selection Sort и Insertion Sort . . . 68
2.3.2 Отказ от точна оценка и търсене на приблизителна оценка на сложността 71
2.3.3 Асимптотична нотация Тета . . . 73
2.3.4 Други асимптотични нотации: O-голямо, Омега-голямо, o-малко, омега-малко . . . 77
2.3.5 Релации $\asymp$, $\preceq$, $\prec$, $\succeq$ и $\succ$ . . . 81
2.3.6 Релациите $\preceq$ и $\succeq$ като преднаредби . . . 91
2.3.7 Релациите на асимптотични сравнения при $\mathfrak{n} \in \mathbb{N}^+$ . . . 97
2.3.8 Асимптотични еквивалентности на често срещани функции . . . 99
2.3.9 Асимптотичните нотации и релациите на асимптотични сравнения . . . 109
2.3.10 Асимптотични сравнения на функции на повече от една променлива . 109
2.3.11 Релация $\sim$: друга релация на асимптотична еквивалентност . . . 109
2.4 Примери за изчисляване на сложност по време на итеративни алгоритми . . . 110
2.4.1 Алгоритми с единичен цикъл . . . 110
2.4.2 Алгоритми с вложени цикли . . . 111
2.4.3 Двоично търсене, итеративен вариант . . . 115
2.5 Винаги ли е лоша високата сложност . . . 118

3 Сортиране. Елементарни сортиращи алгоритми. 122
3.1 Обща дефиниция . . . 122
3.2 Защо сортирането е важно . . . 123
3.2.1 Двоично търсене . . . 124
3.2.2 Най-близки елементи . . . 127
3.2.3 Уникалност на елементите . . . 128
3.2.4 Мода . . . 129
3.2.5 Медиана . . . 133
3.2.6 2SUM, 3SUM, kSUM . . . 134
3.3 Анализ на сортиращи алгоритми. Стабилност. . . . 137
3.4 Елементарни Сортиращи Алгоритми . . . 139
3.4.1 Insertion Sort . . . 139
3.4.1.1 Коректност на Insertion Sort . . . 139
3.4.1.2 Сложност по време на Insertion Sort . . . 145
3.4.1.3 Сложност по памет на Insertion Sort . . . 145
3.4.1.4 Стабилност на Insertion Sort . . . 145
3.4.2 Selection Sort . . . 146
3.4.2.1 Коректност на Selection Sort . . . 146
3.4.2.2 Сложност по време на Selection Sort . . . 147
3.4.2.3 Сложност по памет на Selection Sort . . . 147
3.4.2.4 Стабилност на Selection Sort . . . 148

4 Двоична Пирамида. Heapsort. Приоритетна Опашка. 149
4.1 Двоични дървета и пирамиди . . . 149
4.1.1 Попълнено двоично дърво . . . 149
4.1.2 Адреси в попълнено двоично дърво . . . 156
4.1.3 Двоична пирамида . . . 160
4.1.4 Реализиране на пирамида чрез масив . . . 161
4.1.5 Подпирамида . . . 164

4.2 Построяване на пирамида . . . 166
4.2.1 Наивно построяване на пирамида . . . 166
4.2.2 Бързо построяване на пирамида: алгоритъм Build Heap . . . 170
4.2.2.1 Предварителни обяснения . . . 170
4.2.2.2 Неформално въвеждане на Heapify . . . 174
4.2.2.3 Heapify в итеративен вариант . . . 174
4.2.2.4 Heapify в рекурсивен вариант . . . 176
4.2.2.5 Алгоритъм Build Heap . . . 179
4.3 Сортиращ алгоритъм Heapsort . . . 180
4.4 Приоритетни опашки . . . 182
4.4.1 Абстрактен Тип Данни (АТД) . . . 182
4.4.2 Приоритетна опашка: вид АТД . . . 182
4.4.3 Реализация на приоритетни опашки с двоични пирамиди . . . 184
4.4.4 Функцията Increase-Key . . . 185

5 Рекурсивни Алгоритми, Разделяй-и-Владей и Рекурентни Уравнения. 188
5.1 Рекурсивни алгоритми и Разделяй-и-Владей . . . 188
5.2 Рекурентни уравнения . . . 190
5.2.1 Определение на рекурентно уравнение . . . 190
5.2.2 Обобщения на "рекурентно уравнение" . . . 192
5.2.3 Рекурентни уравнения и рекурсивни алгоритми . . . 193
5.2.4 Класификации на рекурентните уравнения . . . 194
5.2.5 Винаги ли променливата в рекурентно уравнение е целочислена . . . 197
5.2.6 Решаване на рекурентни уравнения . . . 199
5.2.7 Рекурентни уравнения в изследването на алгоритми . . . 201
5.3 Методи за решаване на рекурентни уравнения . . . 202
5.3.1 Чрез развиване . . . 202
5.3.2 Чрез дървото на рекурсията . . . 214
5.3.3 По индукция . . . 218
5.3.4 Чрез Мастър теоремата (MT) . . . 228
5.3.5 Чрез теоремата на Akra-Bazzi . . . 240
5.3.6 Чрез метода с характеристичното уравнение . . . 244
5.4 Примери за ефикасни Разделяй-и-Владей алгоритми . . . 267
5.4.1 Най-близки елементи, 2D . . . 267
5.4.2 Избор на елемент по големина . . . 281

6 Сортиращи алгоритми Mergesort и Quicksort. 288
6.1 Mergesort . . . 288
6.2 Quicksort . . . 295
6.2.1 Имплементация на фазата **Разделяй** чрез Partition–Hoare . . . 296
6.2.2 Имплементация на фазата **Разделяй** чрез Partition–Lomuto . . . 299
6.2.3 Самият Quicksort . . . 303
6.2.4 Сложност по време на Quicksort . . . 304

7 Долни граници върху сложност на задачи. 308
7.1 Долни граници на сложността на изчислителни задачи . . . 308
7.2 Дървета за вземане на решения . . . 310
7.2.1 Задачите The Balance Puzzle и The Twelve-Coin Puzzle . . . 310
7.2.2 Долна граница $\Omega(n \lg n)$ за Сортиране . . . 314

7.2.3 Долна граница $\Omega(n \lg n)$ за Уникалност на елементи . . . 320
7.2.4 Долна граница $\Omega(\lg n)$ за Търсене . . . 324
7.2.5 Лампи и ключове . . . 325
7.3 Редукции . . . 330
7.3.1 Долна граница $\Omega(n \lg n)$ за Най-близка двойка елементи . . . 330
7.3.2 Долна граница $\Omega(n \lg n)$ за Мода . . . 331
7.3.3 Долна граница $\Omega(n \lg n)$ за 2SUM във версия за броене . . . 332
7.3.4 Относителна долна граница: Degeneracy Testing е 3SUM-hard . . . 333
7.4 Аргументиране чрез противник (Adversary argument) . . . 335
7.4.1 Неформално обяснение . . . 335
7.4.2 Формално определение . . . 341
7.4.3 Примери за доказване на долни граници чрез противник . . . 342
7.4.3.1 $n-1$ за намиране на максимален елемент . . . 343
7.4.3.2 $\lceil \frac{3n}{2} \rceil - 2$ за намиране на максимален и минимален елемент . . . 345
7.4.3.3 $\lceil \frac{3n-3}{2} \rceil$ за намиране на медиана . . . 349
7.4.3.4 $n - 2 + \lceil \lg n \rceil$ за намиране на втори максимален елемент . . . 349
7.4.3.5 $2n-1$ за търсене в сортирана по редове и колони, $n \times n$ матрица 350

8 Сортиране в линейно време. Counting Sort. Radix Sort. 357
8.1 Сортиране в линейно време . . . 357
8.2 Counting Sort . . . 357
8.3 Radix Sort . . . 362

9 Въведение в алгоритмите върху графи. Обхождания на графи. 365
9.1 Рекапитулация на графите . . . 365
9.2 Обхождания на графи . . . 369
9.3 Обща схема за обхождане . . . 371
9.4 BFS . . . 372
9.4.1 Псевдокод на BFS от един стартов връх . . . 374
9.4.2 Псевдокод на BFS, обхождащ целия граф . . . 379
9.4.3 Приложения на BFS . . . 379
9.5 DFS . . . 380
9.5.1 Неформално въведение и сравнение с BFS . . . 380
9.5.2 Псевдокод на DFS . . . 382
9.5.3 Свойства на DFS . . . 383
9.5.4 Приложения на DFS . . . 387

10 Топосортиране, силно св. комп., сразващи върхове, мостове 390
10.1 Фундамент . . . 390
10.2 Алгоритъм на Tarjan за топологическо сортиране . . . 393
10.3 Алгоритъм на Kahn за топологическо сортиране . . . 395
10.4 Алгоритъм за намиране на силно свързаните компоненти . . . 398
10.5 Алгоритми за намиране на сразващите върхове и на мостовете . . . 403
10.5.1 Фундамент . . . 403
10.5.2 Алгоритъм за ефикасно намиране на сразващите върхове . . . 406
10.5.3 Алгоритъм за за ефикасно намиране на мостовете . . . 410

11 Минимални покриващи дървета. Алгоритми на Prim и Kruskal. 412
11.1 Фундамент . . . 412
11.2 Алчни алгоритми . . . 418
11.3 Алгоритъм на Prim . . . 420
11.3.1 Базов вариант на алгоритъма на Prim . . . 421
11.3.2 Изтънчен вариант на алгоритъма на Prim . . . 423
11.4 Алгоритъм на Kruskal . . . 426
11.5 Union-Find структури данни . . . 429
11.5.1 Два крайни подхода, които не вършат работа . . . 429
11.5.2 Реализация на разбиване чрез ориентирана гора . . . 431

12 Най-къси пътища. Dijkstra, Bellman-Ford, Floyd-Warshall. 440
12.1 Фундамент . . . 440
12.1.1 Базови дефиниции . . . 440
12.1.2 Пак за отрицателните тегла . . . 447
12.1.3 Релаксация . . . 450
12.2 Алгоритъмът на Dijkstra в два варианта . . . 452
12.3 Най-къси пътища в дагове . . . 454
12.4 Алгоритъмът на Bellman-Ford . . . 456
12.5 Най-къс път за всяка двойка върхове . . . 456
12.5.1 Алгоритъм, реализиращ матрично умножение . . . 457
12.5.2 Алгоритъмът на Floyd-Warshall . . . 463

13 Динамично Програмиране. 467
13.1 Въведение . . . 467
13.1.1 Задачата за опашката пред касата . . . 467
13.1.2 Числата на Fibonacci . . . 469
13.2 Фундамент . . . 470
13.3 Задачи за броене на комбинаторни структури . . . 473
13.3.1 Биномни коефициенти . . . 473
13.3.2 Числа на Stirling от втори род . . . 477
13.3.3 Числа на Stirling от първи род . . . 478
13.3.4 Брой на целочислени разбивания . . . 480
13.4 Задачи за оптимални скобувания на стринг (триъгълна таблица) . . . 484
13.4.1 Matrix-Chain Multiplication . . . 485
13.4.2 Minimum Weight Triangulation . . . 496
13.4.3 Факторизация при неасоциативно умножение . . . 503
13.4.4 Деривация в контекстно-свободна граматика . . . 506
13.5 Задачи за оптимални подмножества . . . 507
13.5.1 Set Partition . . . 507
13.5.2 2 Equal Sum Subsets . . . 512
13.5.3 Задачата за раницата (Knapsack) . . . 514
13.5.3.1 Unbounded Knapsack . . . 515
13.5.3.2 0-1 Knapsack . . . 516
13.5.3.3 Bounded Knapsack . . . 517
13.6 Задачи за планиране (Scheduling) . . . 519
13.6.1 Задачата за безработния актьор (Interval Scheduling) . . . 520
13.6.1.1 Алчно решение . . . 520
13.6.1.2 Решение по схемата Динамично Програмиране . . . 526

13.7 Longest Increasing Subsequence . . . 529
13.7.1 Дин. Прогр. със сложност $\Theta(n^2)$ . . . 530
13.7.2 Дин. Прогр. със сложност $\Theta(n \lg n)$ . . . 531
13.7.3 Patiencesort . . . 538
13.7.4 Building Bridges . . . 538
13.8 **NP**-трудни задачи върху дървета . . . 538
13.8.1 Minimum Vertex Cover on trees . . . 539
13.8.2 Minimum Dominating Set on trees . . . 544
13.9 Игри . . . 550
13.9.1 The Daisy Petals Game . . . 552
13.9.2 The Coins in a Line Game . . . 556
13.10 Мемоизация . . . 561
13.11 За границите на възможностите на динамичното програмиране . . . 563

14 Алгоритмична неподатливост. NP-пълнота и NP-трудност. 566

15 SAT и теоремата на Cook. 567

16 Основни NP-пълни задачи. 568

17 Заобикаляне на неподатливостта: апроксимиране и параметризиране. 569

Благодарности 570

Библиография 571

Списък на фигурите

1.1 Dixit algorismi. . . . 2
1.2 Блок-схема на Евклидовия алгоритъм. . . . 24

2.1 Видове сложност по време. . . . 66
2.2 Графиките на n^2, $3n^2$ и $(2 + \sin n)n^2$. . . . 74
2.3 Графиките на n, $n^{1+\sin n}$, 1 и n^2. . . . 85
2.4 Графиките на $2^{2^{\lfloor n \rfloor}}$ и $2^{2^{\lceil n \rceil}}$. . . . 86
2.5 Диаграма на преднаредба. . . . 93
2.6 Фактор-релация на пълна преднаредба. . . . 95
2.7 Преднаредба, която не е пълна. . . . 96
2.8 Фактор-релация на преднаредба, която не е пълна. . . . 97
2.9 Графиките на n и $n^{1+\sin 2\pi n}$. . . . 98
2.10 $\ln n!$ е голямата сума на Darboux за $\ln(x)$ за $x \in [0,n]$. . . . 100
2.11 $\int_1^n \frac{1}{x} dx$ като площ на район. . . . 105
2.12 $\sum_{i=1}^{n-1} \frac{1}{i}$ като площ на район. . . . 106
2.13 $\sum_{i=2}^{n} \frac{1}{i}$ като площ на район. . . . 107

3.1 Блок-схема на Compute Mode. . . . 131

4.1 Попълнено двоично дърво съгласно Определение 31. . . . 151
4.2 Височините на върховете в попълнено двоично дърво T. . . . 155
4.3 Изтриване на листата на попълнено двоично дърво. . . . 155
4.4 Пирамида с 26 върха. . . . 165
4.5 Линеаризацията на пирамида от Фигура 4.4. . . . 165
4.6 Листата на пирамидата от Фигура 4.4. . . . 165
4.7 Подпирамидата $A[\![3]\!]$. . . . 165
4.8 Листата на $A[\![3]\!]$ в червено. . . . 165
4.9 Почти-пирамидата, получена от пирамидата от Фигура 4.4 с един добавен връх. 167
4.10 Илюстрация на работата на Naive Build Heap. . . . 167
4.11 Понякога по-изгодно коренът да бъде разменян с детето с по-малкия ключ. . . 171
4.12 Ако разменяме връх с корена на детето с по-голям ключ . . . 172
4.13 След 4 размени на връх с детето с по-голям ключ получаваме пирамида. . . . 172
4.14 След 2 размени на връх с детето с по-малък ключ получаваме пирамида. . . . 173
4.15 Може размените на връх с по-малкото дете да доведе до $\Omega(n)$ размени. . . . 173
4.16 Пирамидата, получена от дървото на Фигура 4.15 с $\Omega(n)$ размени. . . . 173
4.17 АТД приоритетна опашка като черна кутия. . . . 183
4.18 Попълнено дърво с 14 върха и техните адреси. . . . 186

5.1 Дървото на рекурсията, отговарящо на $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$. . . . 214
5.2 Схема на дървото на рекурсията, отговарящо на $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$. . . . 215

5.3 Друга схема на дървото при $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$. 216
5.4 Дърво на рекурсията при $T(n) = 2T(n-1) + 1$. 217
5.5 Дърво на рекурсията при $T(n) = T(n-1) + 2T(n-2) + 2$. 218
5.6 MT: дърво на рекурсията при $T(n) = aT\left(\frac{n}{b}\right) + f(n)$. 230
5.7 Точките, които ще ползваме за пример в Най-близка Двойка, 2D. 269
5.8 Бисекторът ℓ. 272
5.9 Дегенеративни случаи. ℓ си остава бисектор. 273
5.10 2d-лентата. 275
5.11 2d-лентата в примера от Фигура 5.7. 276
5.12 Възможните други краища на къса прекосяваща отсечка са малко. 278
5.13 Прозорецът върху примера от Фигура 5.7. 278
5.14 Прозорецът може да обхваща най-много осем точки. 279
5.15 Диаграма на Hasse на данните след ред 9 на Select. 285
5.16 Диаграма на Hasse на данните след ред 11 на Select. 285

7.1 Схема за измервания за The Balance Puzzle. 311
7.2 Схема от измервания за The Twelve-coin Puzzle. 312
7.3 Половината от дървото на сравненията на Insertion Sort върху $\langle a_1, a_2, a_3 \rangle$. . 316
7.4 Дървото на сравненията на Insertion Sort върху $\langle a_1, a_2, a_3 \rangle$. 317
7.5 Да-пример на 3SUM поражда Да-пример на Degeneracy Testing. 335
7.6 Аргументация с противник: два алгоритъма. 341
7.7 Диаграма на състоянията и преходите при сравняване на числа. 347

9.1 Обхождане на дърво от BFS и от DFS. 382
9.2 Ориентиран граф със сдвоени ребра и неориентиран граф. 386
9.3 Различен брой ребра назад. 388

10.1 Ориентиран граф и неговият фактор-граф. 399
10.2 SCC-dummy при различни наредби на върховете. 399
10.3 Транспонираният граф на графа от Фигура 10.1. 402
10.4 Срязващ връх и срязващо ребро. 404
10.5 Разцепване на връх. 405
10.6 Блокове в графи. 406

11.1 Граф и неговите покриващи дървета. 414
11.2 Прехвърляне на връх от границата в дървото при Prim. 422
11.3 Два варианта за Union. 432
11.4 Union by rank: правилен и неправилен. 434

13.1 Минимално върхово покриване в графа на Petersen. 539
13.2 Редуване на слоеве не дава минимално върхово покриване на дърво. 541
13.3 Намиране на максимална антиклика в дърво с 22 върха. 543
13.4 Минимално доминиращо множество в графа на Petersen. 544

Списък на таблиците

7.1 Стратегията на противника при намиране максимум и минимум. 348

9.1 Сложностите на действията върху графи при двете представяния. 368

13.1 Таблица за изчисляване на биномни коефициенти. 474
13.2 Таблица за изчисляване на числа на Stirling от втори род. 478
13.3 Таблица за изчисляване на числа на Stirling от първи род. 480
13.4 Таблица за изчисляване на броеве на целочислени разбивания. 483
13.5 Печеливши и губещи целочислени разбивания. 555

Списък на допълненията

1 За произхода на думата “алгоритъм” . . . 2
2 Не може ли изч. задача да е функция $\Pi : \mathcal{I} \to 2^{\mathcal{S}}$? . . . 5
3 Функциите са повече от алгоритмите! . . . 10
4 Изчислителни модели . . . 11
5 On-line и off-line алгоритми . . . 14
6 Алгоритмичната нерешимост на Стоп задачата . . . 16
7 Понятие за рандомизиран алгоритъм . . . 17
8 За разликата между *ефективност* и *ефикасност* . . . 26
9 Когато коректността не е стопроцентова. . . . 27
10 Не-финитност и частични функции . . . 28
11 Други ресурси освен времето и паметта . . . 57
12 Принцип на Landauer . . . 61
13 Колко са класовете на екв. на входа при сортиране . . . 64
14 Друга интерпретация на появата на Θ вляво . . . 76
15 Нотациите O и o в анализа . . . 80
16 За преднаредбите . . . 92
17 Апроксимация на Stirling . . . 99
18 Коректност на двоичното търсене . . . 125
19 Коректността на Compute Mode . . . 129
20 Неформално за доказателствата за коректност . . . 139
21 За броя на пирамидите на n върха . . . 186
22 Няколко рекурентни уравнения чрез развиване . . . 207
23 Няколко рекурентни уравнения по индукция . . . 223
24 Решение на $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n}$ . . . 231
25 Условието за регулярност в MT влече **Случай 3** . . . 233
26 Ограничена MT следва от метода с хар-чното у-ние . . . 235
27 Едно разширение на Теорема 34 . . . 238
28 Доказателство на теоремата на Akra-Bazzi . . . 241
29 Извеждане на решението на линейните рек. у-ния . . . 246
30 Когато има комплексни характеристични корени . . . 265
31 Как да изчисляваме индексите на елементите на B . . . 270
32 Бързо намиране на броя на инверсиите в масив. . . . 293
33 За избора на pivot в Quicksort. . . . 295
34 Долни граници, задачи и изчислителни модели . . . 309
35 Decision trees са изчислителни модели . . . 319
36 Thinking outside the box – добро или лошо? . . . 326
37 $f(n) \neq \Omega(g(n))$ не влече $f(n) = o(g(n))$ . . . 330
38 Морски шах с дявола . . . 337
39 Разликата между BFS и DFS, илюстрирана с шах . . . 381

40 Произход на “топологическо сортиране”. . . . 392
41 Команда `tsort`. . . . 392
42 За броя на покриващите дървета 413
43 Частична наредба $\sqsubseteq$ върху разбиванията 428
44 Генерирането на всички най-къси пътища е неефикасно. 443
45 За структурата на най-дългите пътища 447
46 Намиране на арбитражи чрез отрицателни цикли 448
47 Колко са булевите вектори без съседни единици 468
48 Не всеки Разделяй-и-Владей води до Дин. Прогр. 471
49 За произхода на “Динамично Програмиране” 472
50 Имплементацията на `binomial(n,k)` в Maple(TM) 474
51 За числата на Catalan 487
52 Броят на триангулациите на изпъкнал n-ъгълник 497
53 Още за псевдополиномиалната сложност 511
54 Алчността на алчните алгоритми: глупава или умна 522
55 Максимална антиклика върху интервални графи 524

Списък на теоремите

1 Стоп задачата е алгоритмично нерешима . . . 16
2 Голяма Теорема на Fermat . . . 29
3 Коректността на алгоритъма Нарастване с единица . . . 32
4 Коректността на алгоритъма Max Unsorted, rec . . . 33
5 Коректността на алгоритъма Max Bitonic . . . 34
6 Коректността на алгоритъма Euclid, rec . . . 37
7 Коректността на алгоритъм за сумата от елементите на масив . . . 38
8 Коректността на алгоритъма Max Unsorted, iter . . . 42
9 Коректността на Alg2SUM . . . 47
10 Коректността на Alg3SUMcount . . . 49
11 Коректността на алгоритъма AlgDist . . . 55
12 Различните възможности при асимптотично сравняване на функции . . . 84
13 Повдигането на степен запазва отношението $\asymp$ . . . 87
14 Водещото събираемо определя асимптотиката . . . 87
15 Експонирането запазва отношението $\prec$ . . . 88
16 Логаритмуването запазва отношението $\asymp$ . . . 89
17 Полиномиалното нарастване изпреварва полилогаритмичното . . . 90
18 Експоненциалното нарастване изпреварва полиномиалното . . . 91
19 Апроксимация на Stirling, слаба версия . . . 99
20 Апроксимация на Stirling . . . 99
21 Апроксимация на Stirling, трета версия . . . 99
22 Асимптотиката на $\lg n!$ . . . 104
23 Асимптотиката на $\sum_{k=1}^{n} k \lg k$ . . . 104
24 Асимптотиката на $\binom{n}{n/2}$ . . . 105
25 Асимптотиката на H_n . . . 105
26 Сложността на двоичното търсене е логаритмична . . . 118
27 Коректност на BinSearchIter . . . 125
28 Коректност на BinSearchRec . . . 126
29 Коректност на Insertion Sort . . . 144
30 Броят на върховете с височина k в попълнено дърво. . . . 156
31 Коректност на наивното построяване на пирамида . . . 169
32 Коректност на бързото построяване на пирамида . . . 179
33 Теорема за решение на клас рекурентни уравнения (Theorem 2.1 в [2]) . . . 228
34 Мастър теорема (Theorem 4.1 от [21, стр. 94]) . . . 229
35 Разширение на MT . . . 238
36 Теорема на Akra-Bazzi (Theorem 1 от [63]) . . . 240
37 Решение на линейно хомогенно рекурентно уравнение . . . 245
38 Решение на линейно нехомогенно рекурентно уравнение . . . 246
39 Множество решения е фундаментално тстк дет. на Casorati е ненулева . . . 253

40 Theorem 2.18 от [23, стр. 72]: съществуване на фунд. множество от решения . . 254
41 Принцип на суперпозицията 254
42 Фундаментално множество от решения и конкретно решение 255
43 Решенията на хомогенно лин. рек. у-ние образуват линейно пространство . . . 256
44 Theorem 2.23 от [23, стр. 77]: фунд. м-во от решения при кратни корени 259
45 Разликата м-у две реш. на нехомогенното е реш. на хомогенното у-ние 261
46 Theorem 2.40 от [23, стр. 84]: общото решение на нехомогенното уравнение . . 262
47 Листата на дървото на вземане на решения имат по $\leqslant 1$ пермутация 321
48 Задачата Degeneracy Testing е 3SUM-hard 334
49 Counting Sort е стабилен сортиращ алгоритъм. 361
50 Коректност на BFS (Theorem 22.5 в [21, стр. 599]) 377
51 Скобова структура на интервалите при DFS 383
52 Theorem 22.10 в [21, стр. 610] 387
53 Граф е цикличен тстк DFS открива ребро назад 388
54 НДУ за съществуване на топологическо сортиране 390
55 Поне един източник и поне един сифон в даг 390
56 Коректност на топологическото сортиране на Tarjan 394
57 Коректност на топологическото сортиране на Kahn 396
58 The Matrix-Tree Theorem, [87, 9.8 Theorem, стр. 141] 414
59 Corollary 9.10-b на стр. 142 в [87] 415
60 Proposition 1.5 на стр. 4 в [87] 415
61 стр. 204 в [3] 416
62 стр. 204 в [3] 416
63 Theorem 23.1 от [21, стр. 627] 418
64 Коректност на МПД Prim2 424
65 Максимална височина на дърво след Union by rank 437
66 Най-къс път се състои от най-къси подпътища. 444
67 Коректност на алгоритъма на Dijkstra 453
68 Най-ранно приключване е оптимално за Interval Scheduling 521
69 Обосновка на бързия алг. за Longest Increasing Sequence 536

Списък на нотациите

1 Фактор-релацията $\mathbf{R}/\simeq$ 94
2 “H_n” означава n-тата парциална сума на хармоничния ред 105
3 “$\mathsf{SC}(G)$” означава релацията на силна свързаност над $V(G)$ 366
4 “adj[x]” означава списъка на съседство на връх x 367
5 “$u \overset{p}{\rightsquigarrow} v$” означава “$p$ е ориентиран път от u до v” 440
6 “$\left\{ {n \atop k} \right\}$”означава числото на Stirling от втори род n-подмножество-k 477
7 “$\left[{n \atop k} \right]$”означава числото на Stirling от първи род n-цикъл-k 478
8 “$\left| {n \atop k} \right|$”означава броя на целочислените разбивания на n на k части 481

Лекция 1

Въведение в алгоритмите.

Резюме: Въвеждаме понятията изчислителна задача и алгоритъм. Дискутираме елементарните инструкции, от които са съставени алгоритмите. Даваме като пример Евклидовия алгоритъм: най-старият известен истински алгоритъм. Разглеждаме различните начини да бъдат описвани алгоритми.

1.1 Алгоритъм

1.1.1 Опит за дефиниция

Понятието *алгоритъм* е първично. За него няма общоприета прецизна формална дефиниция, също както няма дефиниция на *множество*. Задълбочена дискусия на тема дефиниция на алгоритъм има в статията на Blass и Gurevich [14]. Steve Skiena [86, стр. 3] казва:

> What is an algorithm? An algorithm is a procedure to accomplish a specific task. An algorithm is the idea behind any reasonable computer program.
>
> To be interesting, an algorithm must solve a general, well-specified *problem*. An algorithmic problem is specified by describing the complete set of *instances* it must work on and of its output after running on one of these instances. This distinction, between a problem and an instance of a problem, is fundamental.

Често в литературата на тема алгоритми се казва, че алгоритъм е *механична процедура*. Добре известен е следният цитат от статия на Edward Forrest Moore [68]:

> The methods given in this paper require no foresight or ingenuity, and hence deserve to be called algorithms. They would be especially suited for use in machine, either a special-purpose or a general-purpose digital computer.

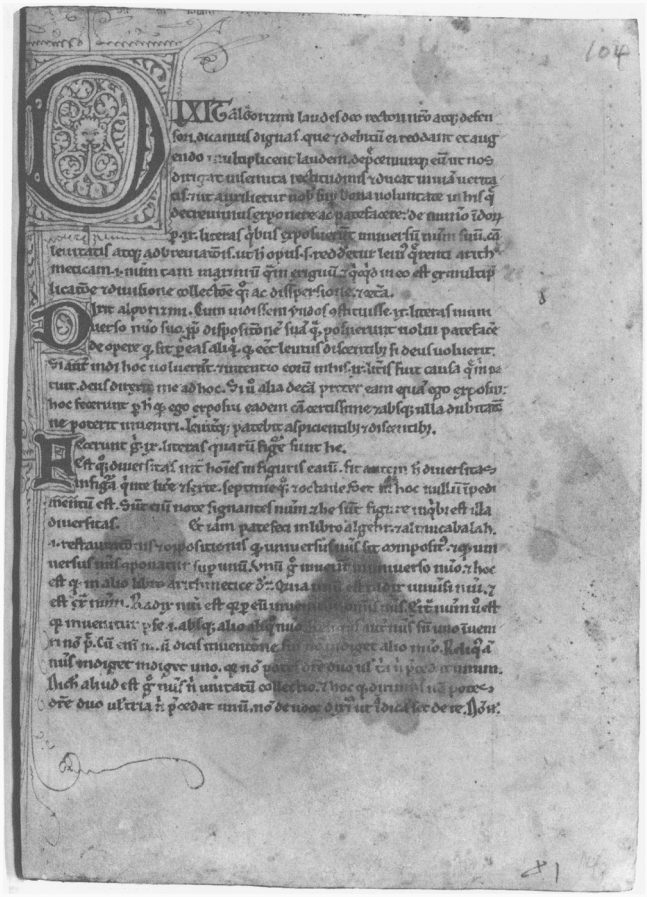
"Механична процедура" е, точно както в цитата от Moore, действия, чието извършване не изисква човешка досетливост и могат да бъдат извършени от машина. Преди ерата на цифровите компютри такива машини са били механични и хората са си представяли (нещо като) часовникови механизми, които ги извършват. Днес вместо "механична процедура" бихме казали "алгоритъм"[†], защото механичната имплементация не е съществена – процедурата може да се имплементира върху електрическа, електронна или квантово-фотонна машина [91]. Съществено е, че процедурата в никой момент не използва човешкия талант да "вижда" решение чрез прозрение, а работата ѝ се реализира от машина, базирана единствено върху законите на физиката.

[†]Естествено, няма да се опитваме да дефинираме "алгоритъм" чрез "алгоритъм".

Допълнение 1: За произхода на думата "алгоритъм"

Подробно разглеждане на историята ключовия термин *алгоритъм* има в първия том от култовата книга The Art of Computer Programming на Donald Knuth [54, стр. 1–2]. Терминът е поевропейчен вариант на името на великия персийски учен Мохамад ибн Муса ал-Хорезми (около 780 г. – около 850 г. сл. Хр.) В англоезичната литература името се пише—когато целта е записът да отразява максимално точно оригиналното произнасяне—Muḥammad ibn Mūsā al-Khwārizmī. Буквално, името означава "Мухамад, син на Муса, Хорезмиецът". Частицата "ал-" на арабски е определителен член. Хорезм е древен град в Централна Азия, който днес се намира в Узбекистан и се нарича Хива. "Ал-Хорезми" означава "който е от Хорезм", накратко "хорезмиецът". Името предполага, че Мухамад е роден в Хорезм, макар че това не е сигурно. Това, което днес се смята от историците за сигурна истина е, че ал-Хорезми живее и работи в в двора на халифа в Багдад по време на златния период на арабската култура, в периода около 813–830 г. сл. Хр. Ал-Хорезми е автор на трактат, който се смята за основоположен за алгебрата, на трактати по география и астрономия, но от алгоритмична гледна точка, важният му принос е трактат върху създадената няколко века по-рано в Индия десетична позиционна бройна система, използваща нулата като пълноправна цифра наред с другите девет цифри. За съжаление, оригиналът на този текст изглежда загубен във времето. Предполага се, че се е казвал "За изчислението с индийски цифри". До нас е достигнал превод на латински от 12 век, който по всяка вероятност е правен от друг превод на латински, който е правен от копие на арабски на оригиналния трактат на ал-Хорезми. Черно-бяло факсимиле на първата страница от въпросния превод на латински от 12 век (който се съхранява в библиотеката на Кеймбриджкия Университет) е показано на Фигура 1.1. Много подробна информация за този превод на трактата ал-Хорезми, както и негов превод на английски, има в статията на Crossley и Henry [44], свободно достъпна онлайн на *sciencedirect.com*.

Фигура 1.1 : Първата страница на превода на латински на трактата на ал-Хорезми за изчислението с цифри. Първите думи са “Dixit algorismi”.



Както се вижда, трактатът започва с думите “Dixit algorismi”. На латински това означава, “Така каза Алгоризми”, където “Алгоризми” е неправилно написаното име на ал-Хорезми. “Dixit algorismi” е и началото на втория параграф.

Оттук в западноевропейските езици се появява думата “algorism”, която означава процедура за работа с числа, написани в индо-арабската бройна система. По онова време, а именно късното средновековие, в Европа все още използват римската не-позиционна бройна система. Предимствата на индо-арабската позиционна бройна система са очевидни за всеки непредубеден човек, но минават няколко века, преди тя да се наложи. По времето, когато двете бройни системи се ползват паралелно, смятащите хора в Европа са разделени на две: абакистите, които ползват римската бройна система и *абака* (вид устройство за смятане), и алгористите, които ползват индо-арабската бройна система. Правилата за работа с индо-арабската система са се казвали “алгоризми”. Имало е нескрит антагонизъм между тези две групировки [88], докато индо-арабската система не се е наложила напълно.

Много по-късно думата algorism се трансформира в *algorithm* и смисълът ѝ става това, което днес разбираме под “алгоритъм”.

1.1.2 Изчислителни задачи

1.1.2.1 Фундамент

Понятието *изчислителна задача*, на английски *computational problem*, е ключово в теорията на алгоритмите. Авторът на тези лекционни записки е на мнение, че често се бърка "алгоритъм" и "изчислителна задача", а това са фундаментално различни неща и разликата между тях трябва да е кристално ясна. Да си припомним цитата на Skiena на стр. 1: "This distinction, between a problem and an instance of a problem, is fundamental.".

Всяка изчислителна задача се характеризира със своите *примери*, на английски *instances*, като на всеки пример съответства едно или повече *решения*, на английски *solutions*. Като прост пример (тази поява на думата "пример" има съвсем различен смисъл от този в предното изречение!) да разгледаме задачата Сортиране, една от базовите изчислителни задачи, с която започва всяко въведение в алгоритмите. Тази задача е разгледана много подробно в Лекция 3. В основния вариант на задачата, примерите на Сортиране са наредените $\mathtt{n}$-орки от цели числа, за $\mathtt{n} \geqslant 1$, а за всяка $\mathtt{n}$-орка решението е уникалната пермутация на тези $\mathtt{n}$ числа в ненамаляващ ред. В този вариант решението за всеки пример е едно единствено. Ако $\mathtt{n} = 5$ и примерът е

$$(2, -3, 0, 4, 0)$$

то решението е

$$(-3, 0, 0, 2, 4)$$

В по-усложнен вариант на задачата, примерите са наредени $\mathtt{n}$-орки от наредени $\mathtt{k}$-орки от цели числа, за някое $\mathtt{k} \geqslant 2$. Във всяка от тези $\mathtt{k}$-орки едно число, да кажем първото, е *ключ* (на английски *key*), и решението е всяка линейна наредба (може пак да кажем пермутация) на $\mathtt{k}$-орките, в която ключовете се появяват в ненамаляващ ред. В този вариант може да има повече от едно решение. Ако $\mathtt{k} = 2$, $\mathtt{n} = 5$ и примерът е

$$((2, 5), (-3, 6), (0, 2), (4, 4), (0, -10))$$

то двете решения са

$$((-3, 6), (0, -10), (0, 2), (2, 5), (4, 4))$$
$$((-3, 6), (0, 2), (0, -10), (2, 5), (4, 4))$$

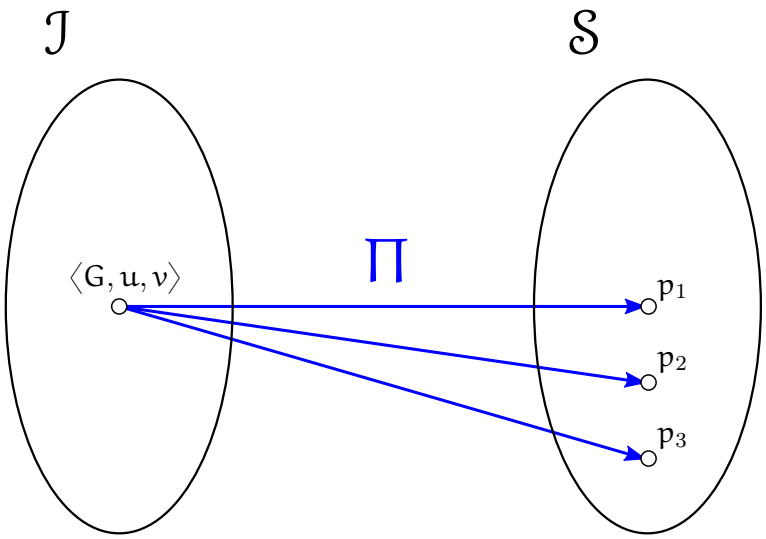
Професор Красимир Манев използва на български термина "масова задача" [65] наместо "изчислителна задача". Идеята е, че задачата е интересна, ако има много примери, което я прави масова. Колко много? По принцип всяка интересна изчислителна задача има безброй примери. Освен това, както ще видим в Лекция 2, общоприетата теория за анализ на алгоритми не е приложима, ако задачата има краен брой примери. И така, разглеждаме само задачи с безкрайно много примери. Обаче **изброимо** безкрайно. Дори ако казваме, че примерите на задачата съдържат реални числа, което—строго формално—влече неизброима безкрайност, ние знаем, че всъщност числата са рационални, а "реални" е просто начин на изразяване.

За разлика от "алгоритъм", за който няма формална дефиниция, "изчислителна задача" има формална дефиниция.

Определение 1: Изчислителна задача

Изчислителна задача Π е двуместна релация $\Pi \subseteq \mathcal{I} \times \mathcal{S}$, където $\mathcal{I}$ е безкрайното множество от примерите, а $\mathcal{S}$ е безкрайното или крайно множество от решенията.

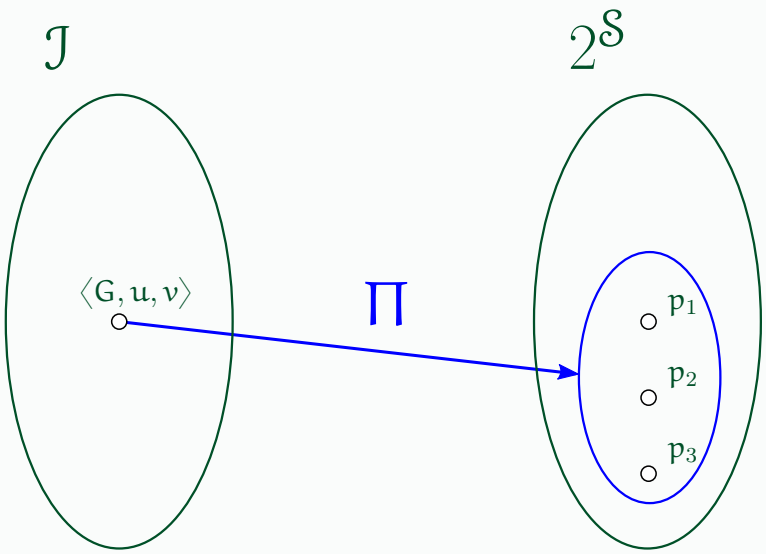
Следната фигура илюстрира нашата представа за изчислителната задача ПЪТ В ГРАФ като релация: всеки пример е наредена тройка от граф и два негови върха, кодомейнът е множеството от пътищата в G, и за даден пример $\langle \mathsf{G}, u, v \rangle$, той е в релация с всеки път p, чиито крайни върхове са именно u и v. Това, че в G има точно три пътя p_1, p_2 и p_3 от u до v изразяваме, казвайки, че $(\langle \mathsf{G}, u, v \rangle, p_1) \in \Pi$, $(\langle \mathsf{G}, u, v \rangle, p_2) \in \Pi$, $(\langle \mathsf{G}, u, v \rangle, p_3) \in \Pi$ и в Π няма други наредени двойки с първи елемент $\langle \mathsf{G}, u, v \rangle$. Ако си представим Π като изображение, по отношение на $\langle \mathsf{G}, u, v \rangle$ нещата изглеждат така:



Очевидно е, че в общия случай изображението не е функция. Само в случай, че всеки пример има точно едно решение можем да кажем, че Π е функция $\Pi : \mathcal{I} \to \mathcal{S}$.

Допълнение 2: Не може ли изч. задача да е функция $\Pi : \mathcal{I} \to 2^{\mathcal{S}}$?

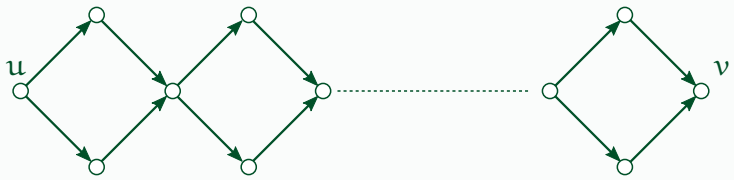
Човек може да се изкуши да дефинира "изчислителна задача" не като релация с първи домейн примерите и втори домейн, решенията, а като функция с домейн примерите и кодомейн степенното множество на множеството от решенията. По този начин може успешно да изрази това, че един пример има множество от решения. Това, че няма решение, може да се изрази, изобразявайки в празното множество. Продължавайки с примера горе, със същия успех може да кажем, че функцията Π изобразява $\langle \mathsf{G}, u, v \rangle$ в множеството $\{p_1, p_2, p_3\}$.



Но това е само на пръв поглед. Има две причини да не искаме да правим това.

1. Може да искаме да смятаме за решение **кой да е** от пътищата p_1, p_2 и p_3, а **не непременно съвкупността** от всички тях.

2. Дори в простия пример с пътищата, те може да са прекалено много. Лесно може да измислим граф и два върха, такива че всички пътища от единия до другия са експоненциално много в размера на графа[a].



   Очевидно е, че ако настояваме решението да е съвкупността от пътищата, а индивидуалните пътища не са решения, то никой алгоритъм няма да може да генерира решение дори за примери със скромни размери по простата причина, че големината на решението е фантастично голяма и е напълно извън възможностите на всеки реален компютър (вижте Допълнение 12).

Поради това е много по-разумно да дефинираме "изчислителна задача" като релация, а не като функция.

[a] Същата конструкция се ползва и Допълнение 44.

1.1.2.2 Видове изчислителни задачи

В този курс ще разгледаме три вида изчислителни задачи: задачи за разпознаване, задачи за търсене и оптимизационни задачи.

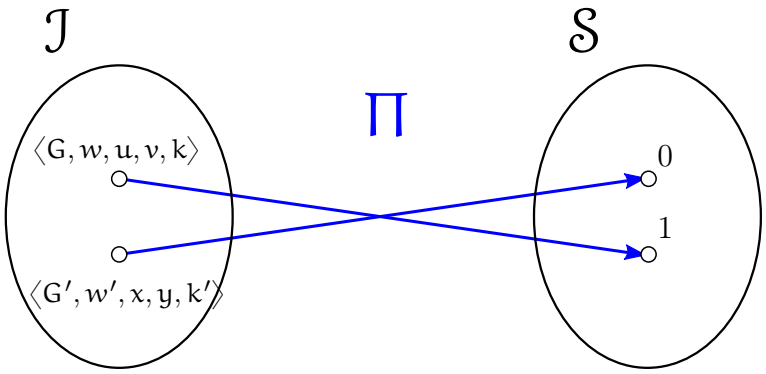
Определение 2: Задачи за разпознаване

Задача за разпознаване е всяка изчислителна задача, за която множеството от решенията е $\{\text{TRUE}, \text{FALSE}\}$.

Терминът "задача за разпознаване" на български е въведен от професор Манев [65]. На английски е *decision problem*. Ето примери за такива задачи:

- дали дадено число е просто,
- дали дадена програма, написана на даден език за програмиране, е синтактично коректна,
- дали даден граф е свързан.
- при даден ориентиран граф G с дадена тегловна функция w, дадени два върха u и v от графа и дадено число k, дали в G има път от u до v с тегло не по-голямо от k.

Следва илюстрация на задачата за разпознаване, дали в графа G с тегловна функция w има път от един даден връх до друг даден връх с тегло не повече от някаква дадена стойност. Графите и тегловните функции не са дадени, но си представете, че в G с тегловна функция w има път от u до v с тегло $\leqslant k$, докато в G' с тегловна функция w' няма път от x до y с тегло $\leqslant k'$.



Очевидно всяка задача за разпознаване е функция, понеже няма смисъл от задача, в която решението за даден пример е хем истина, хем лъжа.

Ако Π е задача за разпознаване, тези нейни примери, за които отговорът е TRUE (още може да пишем "Да", или "1"), се наричат Да-*примерите*. А тези нейни примери, за които отговорът е FALSE (още може да пишем "Не", или "0"), се наричат Не-*примерите*.

За илюстрация да разгледаме задачата за разпознаване PRIMALITY. Нейните примери са числата от $\mathbb{N}^+\backslash\{1\}$, а въпросът е дали дадено число е просто. Числото 5 е Да-пример, числото 55 е Не-пример, и така нататък.

Определение 3: Задачи за търсене

Задача за търсене е изчислителна задача, в която за даден пример има нула, едно или повече решения. Иначе казано, за всеки пример $x \in \mathcal{I}$ има множество решения $Y \subseteq \mathcal{S}$, такива че $\forall y \in Y : (x, y) \in \Pi$, където Π е задачата; множеството Y може и да е празното множество. Може да се иска кой да е елемент от Y или индикация, че Y е празното множество; а може да се иска самото Y.

Терминът на английски е *search problem*. Такава задача вече видяхме: при даден ориентиран граф G и два негови върха u и v да се намери път от u до v в G или да се индикира, че такъв път няма.

Определение 4: Оптимизационни задачи

Оптимизационна задача е всяка задача, в която всеки пример се асоциира с множество от *допустими решения* (на английски, *feasible solutions*), което може да е и празно. Дефинирана е *целева функция* (на английски, *objective function*), която изобразява всяко допустимо решение в число, наречено *цена* или *тегло*. В общия случай, това число е реално[a]. Дефинирана е и *цел*, която е или минимизация, или максимизация. Множеството от *оптималните решения* е подмножеството на допустимите решения, върху чиито елементи целевата функция има минимална стойност, ако целта е минимизация, или максимална стойност, ако целта е максимизация.
При дадена оптимизационна задача се иска или цената на едно оптимално решение, или едно оптимално решение (самото решение, не просто цената му), или броят на оптималните решения, или всички оптимални решения, или индикация, че даденият пример няма решения (множеството от решенията е празното множество).

[a] Като имаме предвид, че всъщност числата са рационални, а "реални" е само начин на изразяване.

Оптимизационните задачи се явяват усложнение на задачите за търсене. В учебника Complexity and Approximations [6, стр. 22] се казва, че цената не е само върху решенията y, а върху наредените двойки (x, y), където x е пример, а y е решение за него. Ако приемем това, то приемаме, че едно и също решение y може да има различна цена в контекста на различни примери, чието решение се явява.

Пример за оптимизационна задача е добре познатата ни задача Най-къс път в граф: при даден ориентиран граф G, тегловна функция w върху ребрата и два върха s и t, да се намери път с минимална цена от s до t. Всеки пример е наредена четворка (G, w, s, t), множеството от допустимите решения е множеството от всички пътища от s до t, цената на решение е сумата от теглата на пътя, а целта е минимизация. Цената на решение не зависи от контекста: претеглената дължина на път е една и съща за всеки граф, който го съдържа.

1.1.2.3 Как се описват изчислителните задачи

Имената на изчислителните задачи се пишат с малки главни букви. Забележете огромната разлика между, да кажем, "Сортиране" и "сортиране":

- "Сортиране" е множество от наредени двойки от наредени n-орки.
- "сортиране" е отглаголно съществително, означаващо вид действие.

В никакъв случай не може да кажем, че "Сортиране" е отглаголно съществително или "сортиране" е множество от наредени двойки.

Въвеждаме понятието *общ пример на изчислителна задача*, като общият пример е описание, от което получаваме представа за всеки конкретен пример. На английски терминът е *generic instance*. Аналогично, *общо решение на изчислителна задача* е описание, от което получаваме представа за решението за всеки конкретен пример. За краткост ще казваме "пример" наместо "общ пример" и "решение" наместо "общо решение". Всяка изчислителна задача се определя напълно от примера и решението. Ето как описваме изчислителните задачи:

Изч. Задача: Сортиране

пример: вектор $A = (a_1, a_2, \ldots, a_n)$ от естествени числа
решение: пермутация $(a'_1, a'_2, \ldots, a'_n)$ на числата от A, такава че $a'_1 \leqslant a'_2 \leqslant \ldots \leqslant a'_n$

В частния случай на задачите за разпознаване говорим не за решение, а за *въпрос* (с отговор ДА или НЕ). Ето пример:

Изч. Задача: Свързаност на граф

пример: Неориентиран граф $G = (V, E)$.
въпрос: Дали G е свързан?

Всяка оптимизационна задача може да бъде превърната изкуствено в задача за разпознаване, като към примера добавим число и въпросът е дали обектът от примера има параметър, не по-голям или не по-малък от това число, съответно за минимизационни и максимизационни задачи. Ето как изглежда задачата за най-къс път във вариант за разпознаване.

Изч. Задача: Най-къс път в графи, разпознаване

пример: Ориентиран тегловен граф $G = (V, E, w)$; връх $s \in V$; връх $t \in V$; число $k \in \mathbb{N}$.
въпрос: Има ли в $G = (V, E, w)$ ориентиран път с начало s, край t и тегло $\leqslant k$?

На пръв поглед, този вариант на задачата за най-къс път е безполезен. На практика бихме искали да знаем поне точната цена на най-къс път, ако не и самия път. Но, както ще видим в някои следващи лекции, задачите за разпознаване имат голямо значение в теорията на изчислителната сложност и има смисъл да разглеждаме оптимизационни задачи, изкуствено модифицирани като задачи за разпознаване.

1.1.3 Алгоритми и изчислителни задачи

Както вече казахме, изчислителна задача е нещо съвсем различно от алгоритъм. Изчислителната задача е релацията или функцията, която трябва да бъде реализирана. Съществуват много алгоритми, които решават дадена изчислителна задача. Всеки от тези алгоритми е *реализация* на въпросната релация или функция.

"Реализация на функция" е ясно какво е. "Реализация на релация R" нещо, което при даден елемент x от първия домейн конструира елемент y от втория домейн, такъв че xRy. Строго погледнато, всеки детерминиран алгоритъм реализира именно (частична) функция, защото при даден вход изходът е строго определен и не повече от един, но за момента си позволяваме да кажем "релация или функция".

Допълнение 3: Функциите са повече от алгоритмите!

Заслужава да се отбележи, че въпросните функции са повече от алгоритмите в теоретико-множествения смисъл на по-голяма безкрайна кардиналност. Алгоритмите, както ще видим, са безкрайно много, но *изброимо* безкрайно. Функциите, от друга страна, са *неизброимо* безкрайно много.
Сега ще покажем с диагоналния метод, че дори само множеството от функции от вида $f : \mathbb{N} \to \{0, 1\}$ е неизброимо безкрайно. Да допуснем, че функциите от този вид може да бъдат изброени като $f_0, f_1, f_2, \ldots$.

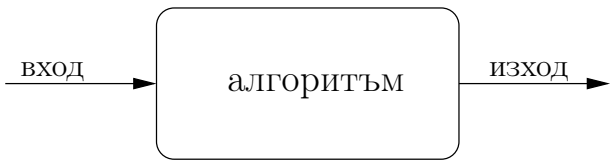
Да разгледаме редицата $f_0(0), f_1(1), f_2(2), \ldots$. Това е някаква функция с домейн $\mathbb{N}$ и кодомейн $\{0, 1\}$. Да дефинираме нотацията $\overline{f_k(k)}$ по следния начин: за всяко $k \in \mathbb{N}$:

$$\overline{f_k(k)} = \begin{cases} 1, & \text{ако } f_k(k) = 0 \\ 0, & \text{ако } f_k(k) = 1 \end{cases}$$

Сега да разгледаме редицата $\overline{f_0(0)}, \overline{f_1(1)}, \overline{f_2(2)}, \ldots$, която също е редица от нули и единици, което означава, функция с домейн $\mathbb{N}$ и кодомейн $\{0, 1\}$. Но функцията $\overline{f_0(0)}, \overline{f_1(1)}, \overline{f_2(2)}, \ldots$ се различава, **за всяко** $j \in \mathbb{N}$, от f_j върху поне една стойност, а именно j. Следователно, $\overline{f_0(0)}, \overline{f_1(1)}, \overline{f_2(2)}, \ldots$ не е нито една от функциите в наредбата. Щом функциите от вида $f : \mathbb{N} \to \{0, 1\}$ са неизброимо безкрайно много, то и функциите от вида $f : \mathbb{N} \to \mathbb{N}$ също са неизброимо безкрайно много. Следва, че възможните функции са повече от алгоритмите в същия смисъл, в който реалните числа са повече от естествените. Това означава, че само безкрайно малко подмножество от тези функции имат алгоритми!

Алгоритъмът е преобразувател на информация, той има *вход* и *изход*, като на входа "влиза" някой от примерите, а на изхода "излиза" решение за него. При всяко конкретно пускане на алгоритъма, входът е някой конкретен пример, а изходът, някое конкретно решение. Когато разглеждаме алгоритъма по принцип, входът е общият пример, а изходът, общото решение.

Използвайки термините "вход" наместо "пример" и "изход" наместо "решение", подчертаваме, че гледаме на алгоритъма като на процес, който преобразува информация, а не като на статично съответствие между примери и решения. Тази фигура илюстрира интуитивното ни разбиране за алгоритъм:



От ключово значение е това, че се интересуваме не просто от дефиницията на функцията, изобразяваща входа в изхода, а от **реализацията** на функцията. Тоест, интересуваме от механизма, който е **вътре в кутията**. Повече за въпросната реализация ще кажем в следващата подсекция. Тук само ще повторим следното; то не може да се нарече "определение", а е просто разяснение.

Разяснение 1: Задачи и алгоритми

При дадена изчислителна задача, която е функция или релация, изобразяваща примерите в решенията, *алгоритъм за тази задача* е всяка реализация на тази функция или релация на някакво приемливо ниво на детайлност.

Допълнение 4: Изчислителни модели

Подробностите по реализацията на функцията-изчислителна задача са предмет на *изчислителния модел* (на английски, *computational model* или *model of computation*), който сме приели. В тези лекционни записки няма да разглеждаме изчислителни модели експлицитно. Изчислителният модел, който ние възприемаме, е близък до машина с произволен достъп (на английски е RAM, идва от *Random Access Machine*, да не се бърка с *Random Access Memory*!). За повече информация за изчислителните модели вижте [78]. RAM моделът там се въвежда на страница 19.

1.1.4 Елементарни инструкции

И така, алгоритъм е реализация на някаква функция. Тази реализацията се състои от крайна последователност от елементарни инструкции. **Това** е съдържанието на кутията, за която вече стана дума – последователност от елементарни инструкции. Какви инструкции са допустими? Няма общоприета схема за това, какви са елементарните инструкции, но е очевидно, че ако допуснем съвсем произволни инструкции, ще обезсмислим конструирането на алгоритми. Например, ако допуснем елементарна инструкция, която сортира $\mathtt{n}$ числа за произволно $\mathtt{n}$, въпросът за конструиране на сортиращ алгоритъм е "решен" – алгоритъмът се състои от тази единствена инструкция. Такова решение би било напълно безполезно на практика. Полезните, смислените елементарни инструкции са тези, които могат да бъдат реализирани **физически** така, че да се изпълняват за време, близко до единица (един машинен такт), върху реален компютърен процесор. Например,

> *събери числата, намиращи се в променливите* $\mathtt{a}$ *и* $\mathtt{b}$*, и запиши резултата в* $\mathtt{a}$

и

> *сравни числените променливи* $\mathtt{a}$ *и* $\mathtt{b}$ *и ако* $\mathtt{a}$ *е по-малко или равно на* $\mathtt{b}$*, премини към изпълнение на инструкция номер* 45*, в противен случай премини към изпълнение на инструкция* 99

са смислени елементарни инструкции. С много примери нататък ще илюстрираме ясно какви инструкции и какво ниво на детайлизация имаме предвид. Засега ще кажем, че инструкциите се делят на императивни, условни, инструкции за извиквания на други алгоритми и входно-изходни.

Смятаме, че информацията на нашия алгоритъм—а именно информацията на входа, информацията на изхода и междинната информация по време на работа на алгоритъма—е структурирана в променливи. Всяко късче информация е част от някаква променлива. Има съставни променливи, например масиви, и атомарни (елементарни) променливи, например целите числа. Това, разбира се, е абстракция по отношение на работата на истински компютър. Всеки истински компютър работи не директно с числа или букви, а с *представяне*, или

кодиране, на числа (или букви). Тоест, със стрингове над някаква азбука—най-често азбуката е $\{0, 1\}$—които може да кодират числа. За целите на този курс е много удобно да избегнем разглеждането на кодиранията и да смятаме, че на най-ниското ниво имаме обекти като числа, с които се работи директно. Елементарните типове данни са `boolean`, `integer`, `real`, `char`, `pointer` и `vertex` (на граф). В това разделение има голяма доза условност, например върховете на графа може да се идентифицират с естествени числа, булевите стойности да са 0 и не-нула и т. н.

Приемаме, че елементарните инструкции се изпълняват в една стъпка независимо от големината на операндите. Например, събирането и умножаването на числа стават за една стъпка *независимо от големината на числата*. От практическа гледна точка, разбира се, това е нереалистично допускане по отношение на много големите числа. За всеки реален компютър има числа, които са достатъчно големи, за да не се побират (по-точно, тяхното представяне да не се побира) в една машинна дума. Нещо повече, всеки реален компютър може да работи само с крайно подмножество на безкрайното множество на целите числа; независимо от избраното кодиране на числата, само безкрайно малка част от всички тях са достъпни за него. Но за целите на курса ще приемаме, че всяко число може да се представи върху нашата абстрактна машина и че основните операции върху кои да е две числа стават за единица време. Това допускане ни позволява да се съсредоточим върху основните аспекти на алгоритмите, а и днешните компютри са толкова мощни, че допускането не е нереалистично за типично възникващите на практика данни.

Изпълнението на елементарните инструции става в дискретно време (стъпки). Както казахме, инструкциите на алгоритъма са подредени и са краен брой. В изложението нататък ще смятаме, че са номерирани. Започвайки от първата инструкция на алгоритъма, във всеки следващ момент (стъпка) се изпълнява точно една инструкция[†]. Коя ще е следващата инструкция след инструкция `i` се определя така:

- Ако инструкция `i` е императивна или входно-изходна, но не е инструкция за край и не е последната инструкция, то след нея се изпълнява инструкция $\texttt{i} + 1$.
- Ако инструкция `i` е условна, например

  `if` ⟨някакъв израз с булева интерпретация⟩ `then`

  или е начало на цикъл, например

  `while` ⟨някакъв израз с булева интерпретация⟩ `do`

  или

  `for` ⟨някакъв израз с булева интерпретация⟩ `do`

  то следващата инструкция, която ще се изпълни, зависи от конкретния алгоритъм и стойностите на променливите в момента.
- Ако инструкция `i` е безусловна инструкция за преход

  `goto k`

  то след нея се изпълнява инструкция `k` (допускаме, че `k` е валиден номер на инструкция).

[†]С други думи, няма паралелизъм.

- Ако инструкция `i` е инструкция за викане на друг алгоритъм, да кажем на алгоритъм AlgX, има две възможности. Ще ги наречем условно "прагматичната" и "теоретичната".

  - Прагматичната възможност е да мислим, че AlgX присъства като последователност от номерирани инструкции и следващата инструкция, която ще се изпълни, е първата инструкция на AlgX. Това е програмистката гледна точка: ако от някаква функция викаме друга функция, кодът на другата функция трябва да присъства и да е достъпен, и след извикването започваме да изпълняваме нея. В такъв случай, последната изпълнена инструкция на AlgX няма да е последната изпълнена инструкция на алгоритъма, с който започнахме – след последната инструкция на AlgX ще се изпълни инструкция $\mathtt{i}+1$ от викащия алгоритъм (а ако няма такава, то това ще е краят на изпълнението).
  - Теоретичната възможност е точно обратното нещо: мислим, че AlgX не присъства като конкретни инструкции. Извиквайки AlgX, **някак си, няма значение как,** получаваме ефекта, който очакваме от него. Например, ако AlgX е сортиращ алгоритъм, то ефектът от викането на $\textsc{AlgX}(\mathtt{a}_1, \ldots, \mathtt{a}_n)$ е, че числата се оказват сортирани и толкова. С други думи, гледаме на AlgX като на черна кутия. Това е абстракция и като всяка абстракция позволява да игнорираме детайли, които смятаме за маловажни (конкретиката на AlgX), и да се фокусираме върху това, което смятаме за важно (конкретиката на викащия алгоритъм).

    При това можем да отчитаме времето за изпълнение на AlgX[†], но може да отидем още по-далече в абстракцията и да мислим, че AlgX се изпълнява **мигновено**, тоест за една единица от дискретното време. Ако дори не отчитаме времето за изпълнение на AlgX, казваме, че AlgX е *оракул*. В реалния свят оракули няма, но това също е полезна абстракция.

    След всички тези обяснения: ако гледаме на AlgX като на черна кутия, то следващата инструкция след инструкция `i` от викащия алгоритъм ще бъде, разбира се, инструкция $\mathtt{i}+1$ от викащия алгоритъм, ако има такава; а ако няма такава, целият алгоритъм ще приключи.

- Ако инструкция `i` е за край на алгоритъма, след нея не се изпълнява нищо. Има две възможности за това: тази инструкция да е последната в алгоритъма, без да е част от цикъл, или да бъде специална инструкция за прекратяване на работата, наречена например `exit` или `halt`.

По принцип не е невъзможно алгоритъмът да се състои в изпълнението на първата инструкция, последвано от втората, третата и така нататък до последната, но това би бил един твърде прост и скучен алгоритъм. Тъй като имаме условни инструкции и възможност за преминаване към изпълнение на произволна инструкция, можем да реализираме цикли. Има изискване за финитност: всеки алгоритъм трябва да завършва изпълнението си за краен брой стъпки върху **всеки** възможен вход. Вече обяснихме при какви условия алгоритъмът завършва своята работа. Има два начина алгоритъм да не завърши:

- Може да "зацикли", което означава да влезе повторно в конфигурация, в която вече е бил. *Конфигурация* е съвкупността от стойностите на всички използвани променливи,

[†]Въпреки че в тази възможност не гледаме на AlgX като на последователност от инструкции, ние може да знаем някакви долни граници за задачата, която AlgX решава, и да искаме да отчитаме това време като част от общото време за изпълнение на целия алгоритъм

тоест състоянието на паметта. Прост пример за фрагмент от алгоритъм, написан на синтаксиса на C, който гарантира зацикляне, е

```
for (;;);
```

- Може, без да влиза никога повторно в конфигурация, в която вече е бил, да мени някои променливи неограничено, например

```
for (i = 0, j = 1; i < 2; j ++);
```

Теоретично говорейки, върху истински компютър вторият сценарий не може да се реализира, тъй като всеки истински компютър има само крайна памет и краен, макар и невъобразимо голям, брой възможни състояния[†], така че ако бъде оставен да работи достатъчно дълго, той неизбежно ще влезе в състояние, в което вече е бил. В посочения пример, паметта, която реализира променливата `j`, рано или късно ще се препълни и `j` пак ще стане единица.

От практическа гледна точка вторият сценарий лесно може да се реализира върху истински компютър, защото, ако времето за повторно влизане в дадена конфигурация е по-голямо от човешки живот или дори живота на звездите във Вселената, то спокойно можем да смятаме, че до повторение на конфигурация няма да се стигне.

1.1.5 Knuth за алгоритмите

Knuth [54, стр. 6] изброява пет ключови свойства, които трябва да има всеки алгоритъм

Финитност. Всеки алгоритъм, за всеки свой вход, трябва да приключи работата си за краен брой стъпки. Процедура, която има всички свойства на алгоритъм, може би с изключение на свойството финитност, се нарича *изчислителен метод*.

Дефинитност. Всяка стъпка на алгоритъма трябва да бъде дефинирана прецизно и недвусмислено. Такава недвусмисленост имат алгоритмите, описани на някакъв конкретен език за програмиране. Ако описваме алгоритъм на естествен език има риск от двусмислие – различни читатели може да тълкуват по различен начин дадени инструкции.

Вход. Всеки алгоритъм има нула или повече *входни данни*, които са зададени преди началото на изпълнението или се задават динамично по време на изпълнението. Тези данни се вземат от някакво определено множество[‡].

Допълнение 5: On-line и off-line алгоритми

Някои автори, например Karp [49], категоризират алгоритмите на

- on-line алгоритми (още се пише “online”), при които целият вход може да не е наличен при стартиране на алгоритъма и съответно има възможност алгоритъмът да получава още и още входни данни, докато работи, и

[†]Тук се твърди имплицитно, че всеки реален компютър е детерминиран краен автомат – броят на състоянията, макар и фантастично голям, е ограничен от две-на-степен-обема-на-цялата-памет, и преходът от едно състояние към друго е напълно детерминиран. Това е така само ако компютърът е изолиран от околния свят. Ако обаче компютърът ползва input от околния свят, който input е случаен, то не може да твърдим с абсолютна сигурност, че е *детерминиран* краен автомат. Input от околния свят може да е нещо, което идва по мрежата, или нещо, въведено от потребителя.

[‡]Това множество е множеството от примерите на съответната изчислителна задача.

- off-line алгоритми (още се пише “offline”), при които целият вход е наличен в момента на стартирана на алгоритъма, като по време на работата входът не се мени.

Очевидно off-line алгоритмите са частен случай на on-line алгоритмите. Както видяхме, разбиранията на Knuth позволяват on-line алгоритми.
Off-line алгоритмите са, в някакъв смисъл, затворени или капсуловани, понеже за тях не съществува външен свят. Те връщат резултата си във външния свят, но по време на работата си са капсуловани. В контраст на това, on-line алгоритмите (може) да получават информация от външния свят, докато работят.
Алгоритмите в този курс са off-line без изключение. В учебника [21] има задачи за конструиране на алгоритми, които получават заявки и връщат резултати (като това трябва да става бързо) – това са примери за on-line алгоритми.

Изход. Всеки алгоритъм има една или повече *изходни данни*, които са в определена релация[†] с входните данни.

Ефективност. Всеки алгоритъм трябва да е ефективен в смисъл, че всяка от неговите елементарни инструкции, наречени от Knuth “operations”, трябва да може принципно да бъде извършена в *крайно време* от човек с лист и молив.

Моделът с човека с лист и молив е именно модел. Това е идеализация, която не отчита ресурсите, необходими за целта: време, количество хартия (памет), количество моливи и така нататък. По отношение на ефективността, игнорираме ресурсите и допускаме, че човекът може да се занимава произволно дълго (но крайно!) време с тази операция без умора, без грешки, без да остарява и умира, без да му свършва хартията или молива, и се интересуваме дали при тези допускания операцията може да бъде завършена за крайно време.

Не всяка операция е ефективна. Добре известно е, че има алгоритмично нерешими задачи. Всяка от тези задачи може да се използва за конструирането на операция, която не е ефективна. Пример за не-ефективна операция е:

> Ако $\mathtt{P}(\mathtt{x})$ терминира, то ...

където $\mathtt{P}$ е произволна програма, а $\mathtt{x}$ е произволен неин вход. Тази операция, или елементарна инструкция в нашата терминология, не е ефективна, защото задачата дали дадена програма терминира върху даден вход е алгоритмично нерешима (вж. Допълнение 6).

Knuth дава друг пример за не-ефективна инструкция, слагайки в условна инструкция булево условие, чийто отговор е същият като на някаква нерешена от никого до момента математическа хипотеза. Ето пример за такава не-ефективна инструкция (примерът на Knuth е друг, но идеята е същата):

> Ако съществува четно число, по-голямо от 2, което не е сума на две прости числа, то ...

Не-ефективността тук идва от това, че досега никой не е доказал или опровергал хипотезата на Goldbach[‡]. В съвременната формулировка, тя звучи по следния начин.

[†] Става дума за релацията, която е изчислителната задача.

[‡] Повече информация за хипотезата на Goldbach има в [92]. Оригиналното писмо на Goldbach до Euler, изказващо хипотезата, може да бъде видяно на *този сайт*.

Хипотеза 1: Goldbach

Всяко четно число, по-голямо от 2, е сума на две прости числа.

Докато хипотезата на Goldbach остава недоказана и неопровергана, никоя условна инструкция, чието булево условие зависи от истинността на тази хипотеза, не може да бъде смятана за ефективна.

Допълнение 6: Алгоритмичната нерешимост на Стоп задачата

Alan Turing е доказал [90], че няма алгоритъм, който решава следната задача.

Изч. Задача: Стоп задача

пример: Компютърна програма $\mathfrak{P}$ и неин вход $\mathfrak{I}$
въпрос: Дали $\mathfrak{P}$ с вход $\mathfrak{I}$ терминира?

Следва опростено доказателство, че Стоп задачата е алгоритмично нерешима. За по-задълбочено разглеждане, добър източник е, например, книгата на Sipser [85].

Теорема 1: Стоп задачата е алгоритмично нерешима

Стоп задачата е алгоритмично нерешима.

Доказателство: Да допуснем противното. Тогава съществува програма $\mathfrak{Q}$, чийто вход е наредена двойка от компютърна програма $\mathfrak{P}$ и неин вход $\mathfrak{I}$[a], като $\mathfrak{Q}(\mathfrak{P},\mathfrak{I})$ връща True, ако $\mathfrak{P}$ с вход $\mathfrak{I}$ терминира, и False в противен случай. Да дефинираме програма $\mathfrak{S}(\mathfrak{P})$ по следния начин: $\mathfrak{S}(\mathfrak{P}) = \mathfrak{Q}(\mathfrak{P},\mathfrak{P})$. Забележете, че $\mathfrak{S}(\mathfrak{P})$ се състои от един единствен ред:

```
return Q(P,P)
```

Дефинираме още една програма $\mathfrak{T}(\mathfrak{P})$ по следния начин:

```
if S(P) then loop forever
else return TRUE
```

Да анализираме $\mathfrak{T}(\mathfrak{T})$. Ако $\mathfrak{T}(\mathfrak{T})$ не терминира, трябва да е вярно, че $\mathfrak{S}(\mathfrak{T})$ връща True. Но тогава трябва да е вярно, че $\mathfrak{Q}(\mathfrak{T},\mathfrak{T})$ връща True. Тогава трябва да е вярно, че $\mathfrak{T}$ терминира с вход $\mathfrak{T}$. Това е противоречие.
Тогава да допуснем, че $\mathfrak{T}(\mathfrak{T})$ терминира. Тогава тя връща True. Тогава трябва да е вярно, че $\mathfrak{S}(\mathfrak{T})$ връща False. Тогава $\mathfrak{Q}(\mathfrak{T},\mathfrak{T})$ връща False. Тогава $\mathfrak{T}$ не терминира с вход $\mathfrak{T}$. Но това също е противоречие. □

[a] Всъщност, входът на $\mathfrak{Q}$ е **кодирането** на $\mathfrak{P}$ и **кодирането** на нейния вход $\mathfrak{I}$.

Базирайки се на казаното от Knuth, има два начина дадена елементарна инструкция да е неподходяща за инструкция на алгоритъм: тя може да е лошо дефинирана или не-ефективна. Примери за лошо дефинирани инструкции са

Ако красотата ще спаси света, то ...

и

Ако Моцарт е по-добър композитор от Бах, то ...

Това са субективни твърдения и мнения, които нямат нищо общо с формалната математическа прецизност на алгоритмичните инструкции. Примери за не-ефективни инструкции вече дадохме: условните инструкции, базирани на булево условие, съдържащо алгоритмично нерешима задача, или недоказана и неопровергана хипотеза. Подчертаваме, че въпросните два начина са принципно различни.

Има и трети начин една елементарна инструкция да е неподходяща: да е не-ефикасна, което ще рече, допускането, че тя може да бъде изпълнена за единица време, да е прекалено нереалистично. За разликата между ефективно и ефикасно, вижте Допълнение 8. Пример за такава инструкция е

```
if isprime(n) then ...
```

където `n` е естествено число, а `isprime(n)` е TRUE тогава и само тогава, когато `n` е просто число. Доколкото е известно в момента, задачата дали дадено число е просто или не, е *достатъчно алгоритмично трудна*, за да не може да твърдим, че се извършва в единица време. Следователно, `isprime(n)` не може да е елементарна инструкция. В контраст на това,

```
if iseven(n) then ...
```

може да бъде елементарна инструкция, защото тестването за четност е алгоритмично тривиална задача и можем да мислим, че се извършва за единица време.

Към петте свойства на алгоритмите, посочени от Knuth, ще добавим още едно:

Детерминираност. За всеки алгоритъм A и за всеки негов вход x, работата на A с вход x е е напълно определена от x: изходът зависи единствено от входа и от нищо друго. Нещо повече, броят на изпълнените стъпки, както и тяхната последователност, зависи единствено от входа и от нищо друго. Колкото пъти да пускаме A с вход x, изходът ще е един и същи, както и последователността от номерата на последователно изпълнените инструкции ще е една и съща.

Допълнение 7: Понятие за рандомизиран алгоритъм

Съществуват и *рандомизирани алгоритми*, за които се предполага, че имат достъп до източник на случайни битове. При тях както изходът, така и последователността от изпълнените инструкции, зависят не само от входа, а и от прочетените от алгоритъма случайни битове. Рандомизираните алгоритми очевидно не притежават свойството детерминираност.
В този курс няма да разглеждаме рандомизирани алгоритми, така че всички алгоритми ще са детерминирани.

1.1.6 Изчислителни методи. Итератори.

Както видяхме в Подсекция 1.1.5, процедура, която може да не терминира, е изчислителен метод. Въпреки че такава процедура не може да е алгоритъм и оттам не може да е обект на изучаване в този курс, понякога се налага да разглеждаме изчислителни методи. Самият

Knuth [54, стр. 6] посочва като важен пример на нетерминиращ алгоритъм *реактивен процес*, който (изчислителен процес) непрекъснато реагира на външната среда.

Важен пример за изчислителен метод е *итератор*. Авторът на лекционните записки зае термина "итератор" от книгата "Chaos and Fractals: New Frontiers of Science" [75]. Обикновено в компютърните науки под "итератор" се разбира *средство за обхождане на някакви агрегирани данни*, примерно списъци. В [75, стр. 17] под "итератор" разбират нещо съвсем различно:

> We will use the terms iterator, feedback and dynamic law synonymously. ... The same operation is carried out repeatedly, the output of one iteration being the input for the next one.

Итератор, в този смисъл, е изчислителен метод с обратна връзка (feedback), който има вход и изход, като типът данни на входа и на изхода е еднакъв, най-често реално число. Този метод стартира от даден вход, после изходът се подава като нов вход, изходът от него се подава като нов вход, и така нататък до безкрай.

Ето формално определение.

Определение 5: Итератор

Итератор наричаме изчислителен метод от вида

$$n \mapsto f(n)$$

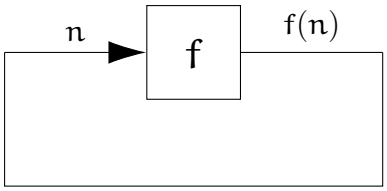
където n е реално число, а f е реална функция. Това описва безкраен изчислителен метод, който започва с n, от него получава $f(n)$, от него получава $f(f(n))$, и така нататък, без край.

$\boxed{n \mapsto f(n)}$ се чете "n се изобразява във $f(n)$", но тук се има предвид нещо повече от функцията f. Тук функцията е участник в процес с обратна връзка, в който нейният изход и́ се подава като вход, отново и отново.

Въпросният итератор може да се опише и така:

$$n \mapsto f(n) \mapsto f(f(n)) \mapsto f(f(f(n))) \mapsto \cdots$$

А може да се опише картинно и така, при което става съвсем ясно защо говорим за обратна връзка:



Естествено, тази рисунка има смисъл само при допускане за наличие на време (дискретно или континуално), инак всички стойности, генерирани от итератора, трябва да са заедно върху "жицата".

Въпреки че така описаният изчислителен процес е безкраен, ако итераторът веднъж повтори стойност, тоест достигне n, за което $f(n) = n$, той ще "залепне" в тази стойност и няма да се промени повече. Такава стойност се нарича *фиксирана точка*, на английски *fixed point*, на функцията (и на итератора).

По-общо, ако итераторът повтори стойност, той ще я повтаря отново и отново, без край. По-подробно казано, ако за някакво n' итераторът генерира редица

$$n' \mapsto n_1 \mapsto n_2 \mapsto \cdots \mapsto n_k \mapsto n'$$

то той ще генерира отново и отново тази редица, без край. Ако n' е хронологически първата повторена стойност и тя не се съдържа в $\{n_1, n_2, \ldots, n_k\}$, казваме, че редицата

$$\langle n', n_1, n_2, \ldots, n_k, n' \rangle$$

е *орбита* за итератора, като дължината на орбитата е $k + 1$. Очевидно фиксирана точка задава орбита с дължина 1.

Ако си представим итератора като граф—само че безкраен граф—върху множеството, върху което е дефиниран, то орбита отговаря на прост цикъл в този граф, а фиксирана точка отговаря на примка в него.

1.2 Евклидовият алгоритъм

Въпреки че терминът "алгоритъм" идва от името на човек, живял през IX век, алгоритми са били известни и използвани много преди това. Най-старият известен нетривиален алгоритъм е *Евклидовият алгоритъм*. Евклид е един от най-великите умове на античния свят, математик, известен като *баща на геометрията*. Неговата поредица от книги, наречена *Елементи*, е в основата на курсовете по класическа геометрия и до днес. В седмата книга от *Елементи* [35] е описана процедура за изчисляване на най-големия общ делител на две цели положителни числа. Тази процедура има всички характеристики на алгоритъм според Knuth (вж. Подсекция 1.1.5), поради което я наричаме "алгоритъм". Въпросният алгоритъм се съдържа в Задача 1 [35, стр. 296] и Задача 2 [35, стр. 298].

1.2.1 Оригиналният Евклидов алгоритъм

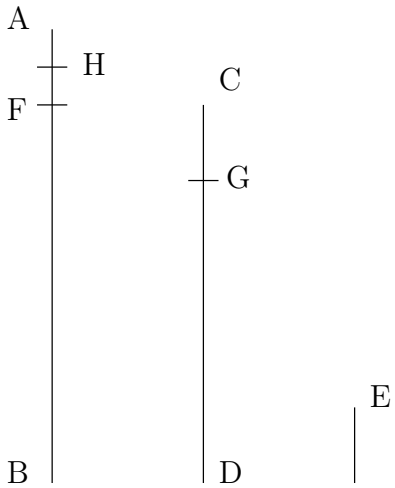
Задача 1: Кога две числа са взаимно прости

Дадени са две числа, като по-малкото бива изваждано последователно от по-голямото. Ако числото, което се получава накрая, никога не дели по-голямото[a], и най-накрая остане единица, оригиналните числа са взаимно прости.

[a] Има се предвид, в процеса на изваждане.

Аргументация

Нека по-малкото от две неравни числа AB и CD[†] бива изваждано последователно от по-голямото и нека получаваното число никога не дели това преди него, докато накрая се получи единица; твърдя, че AB и CD са взаимно прости, тоест, само единицата ги дели и двете.



Защото, ако AB и CD не са взаимно прости, някакво число ги дели[‡]. Нека ги дели някакво число и нека то да е E; нека CD, което дели BF, оставя някакво FA, по-малко от себе си.

Нека AF, което дели DG, оставя по-малко от себе си GC, и нека GC, което дели FH, оставя единица HA.

Но тъй като E дели CD, и CD дели BF, то E дели BF.

Но то също така дели цялото BA; следователно, то дели и остатъка AF.

Но AF дели DG; затова E дели и DG.

Но то също така дели и цялото DC, затова дели и остатъка CG.

Но CG дели FH; затова E също дели и FH.

Но то дели и цялото FA, затова то още дели и остатъка, а именно единицата AH, въпреки че то[§] е число. А това е невъзможно.

Затова никое число не дели числата AB и CD; затова те са взаимно прости. □

[†] Древните гърци са мислели за числата като за дължини на отсечки.

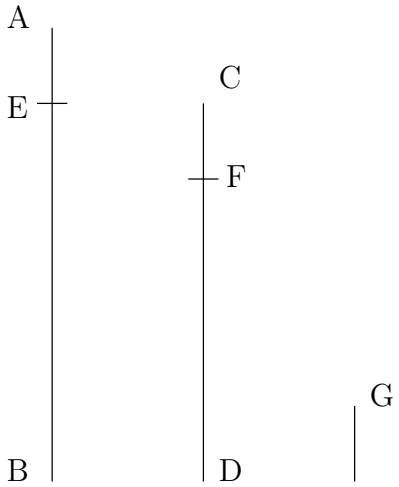
[‡] Древните гърци не са смятали единицата за число, а за нулата дори не са имали понятие.

[§] Става дума отново за E.

Задача 2: Как се намира най-голям общ делител

Дадени са две числа, които не са взаимно прости, да се намери техният най-голям общ делител.

Аргументация



Нека AB и CD са две дадени числа, които не са взаимно прости.

Иска се да се намери най-големият общ делител на AB, CD.

Ако CD дели AB—тогава то дели и себе си—CD е общ делител на AB и CD.

И е очевидно, че то освен това е най-големият общ делител; защото никое число, по-голямо от CD, не дели CD.

Но ако CD не дели AB, тогава по-малкото от числата AB, CD, ако бъде изваждано последователно от по-голямото, то ще остане число, което дели това преди себе си.

Единица няма да остане; инак, AB, CD ще се окажат взаимно прости (виж Задача 1), което противоречи на хипотезата.

Затова ще остане число, което дели това преди него.

Нека CD, което дели BE, оставя EA, което е по-малко от него самото, което дели DF, оставяйки FC, което е по-малко от него самото[†]*, и нека CF дели AE.*

Тъй като CF дели AE, и AE дели DF, следва, че CF дели DF.

Но то дели освен това себе си, затова дели цялото CD.

Но CD дели BE, следователно CF също така дели BE.

Но то[‡] *също така дели EA, затова то също така дели цялото BA.*

Но то също така дели CD, затова CF дели AB, CD.

Затова CF е общ делител на AB, CD.

Сега аз казвам, че това число е най-голямото такова.

Защото, ако CF не е най-големият общ делител на AB, CD, някое число, по-голямо от CF ще дели числата AB, CD.

Нека едно такова число е G.

Понеже G дели CD, тъй като CD дели BE, G също така дели и BE.

Но то също дели и цялото BA; затова то дели и остатъка AE.

Но AE дели DF; затова и G дели DF.

Но то също дели и цялото DC; затова то дели и остатъка CF, тоест, по-голямото дели по-малкото: което е невъзможно.

Затова никое число, което е по-голямо от CF, не дели числата AB, CD.

Затова CF е най-големият общ делител на AB, CD. □

Този текст ни звучи непривично заради особената нотация и конвенции. Очевидно по Евклидово време не са имали съвременната идея за индукция; той дори не ползва някаква форма на "и така нататък" при аргументацията, а разглежда трикратно прилагане на конструкцията на вземане на (това, което днес бихме нарекли) остатъци при деление и смята, че това е

[†]Вече се има предвид от EA.

[‡]CF

достатъчно убедително. В Задача 1, трикратното прилагане е:

AB и CD дават остатък FA → остатък GC → остатък HA, равен на единица

Въпреки необичайното за нас изразяване, тези две задачи от седмата книга на Евклид съдържат истинско описание на алгоритъм, напълно недвусмислено, с аргументация за привършване на процедурата и за коректност на получения резултат. Анализ на сложността по време, естествено, няма. В десетата книга на Евклид се дискутира същата идея, но върху числа-отсечки, които днес бихме нарекли "реални"; в Евклидовата терминология, това са отсечки, чието отношение на дължините не е рационално число (*incommensurables* в английския превод). Изключително задълбочена дискусия за Евклидовия алгоритъм има във втория том от поредицата на Knuth [55].

1.2.2 Съвременни варианти на Евклидовия алгоритъм.

Съвременната формулировка на Евклидовия алгоритъм ползва не последователни изваждания, а значително по-бързото изчисляване на остатък при деление (което може да се имплементира чрез изваждания-докато-е-възможно, но това би била много тромава от днешна гледна точка имплементация). Ето две възможни реализации на този алгоритъм: итеративна и рекурсивна.

```
EUCLID, ITERATIVE(a, b ∈ ℕ⁺, a ⩾ b)
1  while a mod b > 0 do
2      r ← a mod b
3      a ← b
4      b ← r
5  return b
```

```
EUCLID, REC(a, b ∈ ℕ, a ⩾ b, a > 0)
1  if b = 0
2      return a
3  else
4      return EUCLID, REC(b, a mod b)
```

В Глава 2 ще докажем коректността на Евклидовия алгоритъм.

1.3 Как описваме алгоритми

1.3.1 Псевдокод

Реализациите на Евклидовия алгоритъм от Подсекция 1.2.2 са добра илюстрация за използването на *псевдокод*. Псевдокод е езикът, който ползваме за описание на алгоритми. Това не е истински език за програмиране, а измислен от нас език, който е пределно изчистен от ненужни детайли. С псевдокод не можем да направим синтактична грешка – понеже няма общоприети правила за псевдокод, синтаксисът си го измисляме ние, пишейки псевдокода. Псевдокодът, който ще използваме в тези лекции, е почти същият като псевдокода в учебника на Cormen, Leiserson, Rivest и Stein [21]. Той прилича далечно на синтаксиса на езика за програмиране Pascal. Читател, свикнал със синтаксиса на езиците C или Java лесно ще чете псевдокода, ако има предвид няколко негови особености.

- ползваме стрелка наляво "←" за присвояване (на английски, *assignment*), а не ":=" като на Pascal или "=" като на C.
- ползваме знак за равенство "=" за сравняване на обекти, също като на Pascal, а не "==" като на C.

- Масивите се индексират от едно, а не от нула. С други думи, по подразбиране, началният елемент на масива A е $A[1]$, а не $A[0]$. $A[0]$ може да се използва само ако масивът е дефиниран като, например, $A[0, \ldots, n]$.
- Цикъл с постусловие **repeat**–**until** се "върти", докато постусловието е лъжа; при първото достигане на постусловието, в което то е TRUE, изпълнението на цикъла се прекратява и управлението минава към следващата инструкция, ако има такава. Така е в Pascal. В C и Java, постусловията са изградени по обратната конвенция: следващо изпълнение има ако и само ако постусловието е TRUE.
- Не се използват ограничители като "{" и "}" за означаване на блокове от код, било в цикъл, било в условна инструкция. Блоковете код се маркират с различен отстъп от лявата страна. Това е като в езика за програмиране Python. Само ако алгоритъмът е много дълъг и окото на читателя може да се объркa, ще ползваме "{" и "}" за означаване на блокове от код.

Писането на псевдокод има потенциален недостатък. Свободата при писането на псевдокода създава опасност да започнем да пишем елементарни инструкции, които:

- не са ефикасни,
- или не са ефективни,
- или дори не са добре дефинирани.

1.3.2 Други видове описания на алгоритми

Вече видяхме един начин за описване на алгоритми: в псевдокод. Алгоритми може да се описват и на естествен човешки език, например описанията на Евклид в аргументациите на Задача 1 и Задача 2. Но естественият език е подходящ само за описание на прости алгоритми или за описание на високо ниво, без детайлите. Ако трябва да опишем сложен алгоритъм на човешки език, и то в детайли, неизбежно ще започнем да използваме някаква смесица от естествен език и псевдокод. Колкото по-сложно и детайлно е описанието, толкова по-вероятно е това да бъде неясно[†]. Защо тогава да не ползваме пределно изчистения и недвусмислен псевдокод поначало?

Алгоритми могат да се описват и на истински езици за програмиране като C или Java. В този случай не правим разлика между описание на алгоритъм и неговата програмна реализация – програмната реализация **е** алгоритъмът. Този начин за описание на алгоритми има съществен недостатък: колкото и да е изчистен езикът за програмиране, програмата, написана на него, съдържа детайли, които са ирелевантни за алгоритъма и пречат да се разбере идеята. Да си припомним цитата на Skiena: "An algorithm is the idea behind any reasonable computer program" (на стр. 1). Ако вярваме в това, то има смисъл да правим разлика между описанието на идеята и на конкретна програма, имплементираща идеята. Разбира се, това не трябва да се абсолютизира. Прост алгоритъм като Евклидовият можеше да бъде описан на чисто C и описанието щеше да е толкова ясно, колкото е псевдокодът. Но колкото по-изтънчен и труден за възприемане е алгоритъмът, толкова повече се виждат предимствата на описанието в псевдокод пред описанието на C. Най-малкото, ако пишем на псевдокод, винаги можем да изберем на какво ниво на детайлизация да е описанието и можем да опишем практически всеки алгоритъм съвсем кратко.

[†]Опитайте се да опишете на чист български език алгоритъм, в който има сложни if-then-else условия с многократна вложеност.

И накрая, алгоритми може да се описват чрез *блок-схеми*[†]. Фигура 1.2 показва блок-схема на Евклидовия алгоритъм (алгоритъм EUCLID, ITERATIVE от Подсекция 1.2.2). Блок-схемата е диаграма, изобразяваща ориентиран граф с четири вида върхове:

- връх-начало и връх-край, които се рисуват със заоблени правоъгълници,
- върхове-условия, които се рисуват с ромбове, в които са написани условията,
- върхове-императивни инструкции, в които са написани императивните инструкции,
- може да има и четвърти вид върхове, да ги наречем *ветрила*, които не съответстват на инструкции, а са нещо като точки, в които може да дойде изпълнението от повече от едно места.

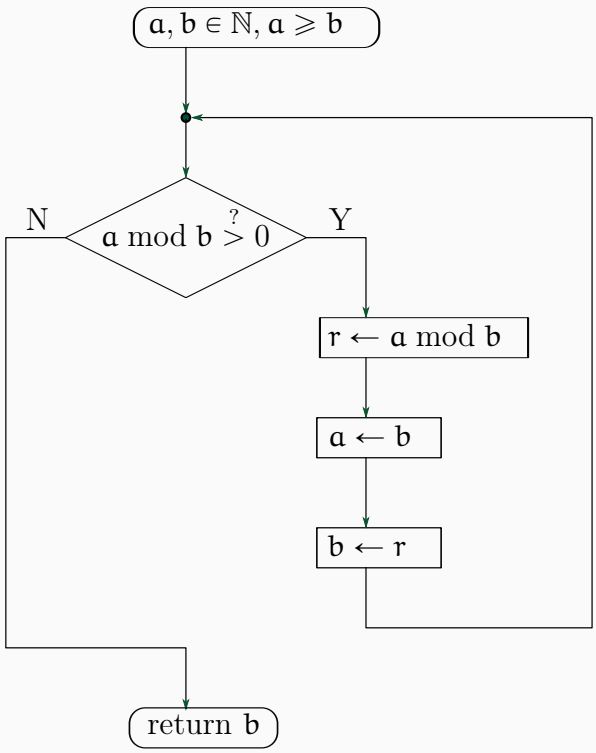
Стрелките показват как върви изпълнението, което по правило се изобразява отгоре надолу. Върхът-начало има степен на входа нула, защото от него започва изпълнието, и степен на изхода единица. Върховете-условия и върховете-императивни инструкции, както и върхът-край, имат степен на входа единица. Върхът-край има степен на изхода нула, върховете-условия има степен на изхода две, а върховете-императивни инструкции имат степен на изхода едно. Ако в някой връх може да се "влезе" по повече от един начина[‡], преди него слагаме връх от тип ветрило, който има повече от един входове и само един изход. Връх-ветрило рисуваме с дебела точка.

Ориентираните ребра показват изпълнението на алгоритъма. Както казахме, върховете-условия имат степен на изхода две, като едното от излизащите ребра е маркирано с Y, а другото, с N, които очевидно идват от Yes и No и показват къде отива изпълнението на алгоритъма, ако условието е съответно истина или лъжа.

[†]На английски "блок-схема" е *flowchart*.

[‡]Например на Фигура 1.2 в условието "$\mathtt{a} \bmod \mathtt{b} \overset{?}{>} 0$" може да се влезе по два начина: "отгоре" и "отдолу".

Фигура 1.2 : Блок-схема на Евклидовия алгоритъм.



Описанието с блок-схеми е удачно за прости алгоритми. На практика много често алгоритми се описват с блок-схеми, ако описанието е предназначено за хора, които не са специалисти по алгоритми (съответно е безсмислено да им се дава псевдокод) и освен това алгоритмите са прости, като често дори нямат цикли[†]. Пример за прост алгоритъм без цикли, който трябва да бъде разчетен от човек, който не е специалист по алгоритми, е алгоритъм за това какво да правим, ако автомобилът ни отказва да запали. Ето един *такъв алгоритъм*. Но за сложни алгоритми, особено ако описанието е върху няколко страници, блок-схемите не са подходящи.

[†]Алгоритъмът да няма цикли е същото като графът на блок-схемата да е ацикличен, тоест стрелките да вървят само отгоре надолу.

Лекция 2

Анализ на алгоритми.

Резюме: Разглеждаме различните аспекти на анализа на алгоритмите: анализ на коректността и анализ на сложността, който на свой ред се разбива на анализ по време и анализ по памет. Въвеждаме големина на входа на алгоритъм и видовете сложност като функции от големината на входа. Извеждаме точни изрази за сложността на конкретни прости алгоритми. Показваме необходимостта от по-малко прецизна, по-груба мярка за сложността на алгоритмите. Въвеждаме петте общоприети асимптотични нотации. Разискваме дали високата сложност е непременно лошо нещо.

Да бъде *анализиран* даден алгоритъм означава да бъде доказано, че той е коректен, и ако е коректен, да бъде изчислено колко ресурси ползва—за всеки възможен вход—като ресурсите са основно време и памет. Това са различни аспекти на анализа. Първият аспект—коректността—е свързан с понятието *ефективност*, а вторият аспект е свързан с понятието *ефикасност*.

Допълнение 8: За разликата между *ефективност* и *ефикасност*

В стандартния български език е прието *ефективност* и *ефикасност* да се смятат за синоними, но на английски има значителна разлика между съответните *effective* и *efficient*. Въпросът дали даден алгоритъм е *effective* има отговор или Да, или Не; ако алгоритъмът върши работата, която е определена от заданието (изчислителната задача), то той е **ефект**ивен, понеже неговата работа дава желания **ефект**; в противен случай той не е ефективен. При ефикасността въпросът не е дали, а до колко. Ефикасността е свързана с ресурсите, които алгоритъмът ползва.
При тази езикова конвенция, "ефективен" е свързан с резултата, а "ефикасен", с пътя, по който достигаме до резултата. Ако зададем въпроса дали даден алгоритъм е ефикасен и очакваме отговор или Да, или Не, то ние сме задали един безсмислен въпрос[a]. Обаче има смисъл да твърдим, че един алгоритъм е по-ефикасен от друг алгоритъм по отношение на някакъв ресурс, ако първият ползва по-малко от този ресурс от втория за всички входове; например, ако винаги работи по-бързо. От друга страна, сравняването на ефективността на два алгоритъма е безсмислено, ако и двата са коректни – в такъв случай и двата са ефективни, и това е всичко по отношение на ефективността.

Терминологичната разлика между "ефективен" и "ефикасен" е много полезна на практика и ние ще я спазваме.

[a] Освен, разбира се, ако предварително не сме се разбрали, че "ефикасен" означава нещо количествено определено. В теорията на алгоритмите съществува конвенция, според която "ефикасен" означава "работещ в полиномиално време". При наличие на такава конвенция, въпросът дали даден алгоритъм

е ефикасен с отговор или Да, или Не, е смислен. При липса на такава конвенция обаче, ефикасността не се описва с Да или Не.

Допълнение 9: Когато коректността не е стопроцентова.

Съществуват сложни изчислителни задачи, за които са приемливи алгоритми, които не винаги работят коректно, но все пак са полезни при условие, че можем да ограничим количествено броя на входовете, върху които алгоритъмът греши. Ако броят на входовете, върху които алгоритъмът не работи коректно, е пренебрежимо малък спрямо броя на всички входове, то този алгоритъм може да е интересен и полезен при условие, че не разполагаме с други алгоритми за тази задача или разполагаме, но те са прекалено бавни. В такъв случай, ефективността не е булева величина Да/Не, а нещо по-сложно, например положително реално число.
В този курс такива задачи и алгоритми няма да разглеждаме, така че ефективността за нас ще е именно булева. Ако някакъв алгоритъм не работи коректно дори за един единствен вход, ще казваме, че не е коректен.

И така, анализирайки един алгоритъм, първо ще доказваме, че е ефективен, тоест коректен, и след това ще изследваме колко е ефикасен, тоест какви ресурси ползва. Когато разглеждаме сложността на **даден алгоритъм за дадена задача**, искаме алгоритъмът да е колко е възможно по-ефикасен (естествено, бивайки ефективен). Когато разглеждаме сложността **на задачата**, фактът, че за нея няма бърз алгоритъм може да бъде от голям теоретичен интерес, но може и да е полезен на практика, както ще стане ясно в Подсекция 2.5.

2.1 Анализ на коректността

Анализът на коректността е по-важният вид анализ в смисъл, че ако се окаже, че даден алгоритъм не е коректен, това е краят на анализа му. Колко бързо работи и колко памет ползва некоректен алгоритъм, не ни интересува.

Нека е даден алгоритъм A, който решава някаква изчислителна задача Π. Както казахме вече, можем да мислим за Π като за релация от множеството на конкретните примери в множеството на конкретните решения, а алгоритъмът A е една възможна реализация на тази релация чрез някакви основни елементи-инструкции. Анализът на коректността е доказателство, че наистина A реализира именно **тази** релация, а не някаква друга. Ще ползваме главно две техники, за да доказваме коректност на алгоритми: чрез индукция (виж страница 31), когато става дума за рекурсивни алгоритми, и чрез *инвариант на цикъл* (виж страница 37), когато става дума за итеративни алгоритми.

Определение 6: Верификация

Верификация, на английски *verification*, означава формално доказване на коректност.

Произходът на термина е от латинските думи *veritas*, което означава "истина", и *facio*, което означава "правя". За целите на тези лекции имаме предвид верификации на алгоритми. "Ще верифицираме алгоритъма A" означава "ще докажем формално коректността на алгоритъма A".

2.1.1 Доказателство за финитност

Важна част от доказателството за коректност е доказателството за финитност (вж. Подсекция 1.1.5). С други думи, че алгоритъмът завършва работата си върху всеки вход.

Допълнение 10: Не-финитност и частични функции

Както вече казахме много пъти, алгоритъмът е реализация на някаква релация с домейн примерите и кодомейн, решенията, която релация отговаря еднозначно на изчислителната задача. За простота тук ще смятаме, че релацията е функция. При липса на финитност, според терминологията на Knuth (вж. Подсекция 1.1.5), казваме "изчислителен метод" вместо "алгоритъм".

Свойствата финитност и нефинитност може да бъдат описани елегантно в терминологията на функциите: при наличие на финитност говорим за алгоритъм, който реализира **тотална** функция, а при липса на финитност говорим за изчислителен метод, който реализира **частична** функция[a]. В контекста на теорията на алгоритмите ние се интересуваме от реализацията на изображението (частична или тотална функция) чрез елементарни инструкции. Има смисъл да кажем, че даден елемент x от домейна се изобразява в елемент от кодомейна ако и само ако реализацията от елементарни инструкции, стартирайки с вход x, терминира; а ако не терминира, то x не се изобразява в кодомейна. Тогава можем да кажем, че алгоритмите реализират тотални функции, докато изчислителните методи реализират по-общите частични функции.

В този контекст, да докажем финитността на алгоритъма е да докажем, че той е реализация на тотална функция. Повече информация за съответствието между терминиращи и не непременно терминиращи процедури, от една страна, и тотални функции и частични функции, от друга страна, има в книгата на Manna [66].

[a] Да си припомним, че "частична функция" е обобщение на "тотална функция", при което може да има елементи от домейна, на които не съответстват елементи-изображения от кодомейна.

Доказателството за финитност често се пренебрегва, защото бива считано за очевидно. В много случаи наистина е очевидно, но не винаги е така. Може да бъде изключително трудно да се отговори дали даден алгоритъм винаги завършва. Както е показано в Допълнение 6, **не съществува** алгоритъм, който по дадено кодиране на **произволен** друг алгоритъм плюс вход да определи дали той (другият) ще завърши работата си върху този вход, или не. Алгоритмичната нерешимост на СТОП ЗАДАЧАТА обрича на провал всеки опит да подходим алгоритмично към анализа на финитността на произволен алгоритъм. Можем да докажем—ако успеем!—финитността на **даден** алгоритъм, но нямаме алгоритмичен начин да правим това за **всеки** алгоритъм.

Нещо повече. Дори за конкретен даден алгоритъм, да анализираме неговата финитност е същото като да решим някаква математическа задача. Верността на математическо твърдение е **кодирана** в това, дали алгоритъмът терминира винаги, или не! Ще разгледаме три примера за това.

Първи пример: хипотезата на Goldbach Следният пример беше предложен на автора от Георги Георгиев. Хипотезата на Goldbach (вж. Хипотеза 1) може да бъде използвана, за да се конструира алгоритъм, чиято финитност зависи директно от това, дали хипотезата е истина.

```
DUMMY–GOLDBACH(∅)
 1  n ← 2
 2  do
 3     n ← n + 2
 4     more ← FALSE
 5     for i ← 2 to n/2
 6        if isprime(i) and isprime(n − i)
 7           more ← TRUE
 8  while more
```

Очевидно DUMMY–GOLDBACH терминира ако и само ако Хипотезата на Goldbach е невярна. Тъй като засега не знаем дали хипотезата е вярна, не можем да кажем и дали този алгоритъм терминира, или не.

Втори пример: голямата теорема на Fermat

Теорема 2: Голяма Теорема на Fermat

Уравнението $a^n + b^n = c^n$ няма решения в цели положителни числа за $n \geqslant 3$.

Голяма Теорема на Fermat има свой алгоритмичен аналог – да докажем, че следният код, написан на C, не терминира, е същото като да докажем, че тя е вярна:

```
k = 3;
for (;;) {
  for(a = 1; a <= k; a ++)
   for(b = 1; b <= k; b ++)
    for(c = 1; c <= k; c ++)
     for(n = 3; n <= k; n ++)
      if (pow(a,n) + pow(b,n) == pow(c,n))
       exit();
  k++; }
```

Голямата теорема на Fermat е доказана—за разлика от Хипотезата на Goldbach—така че ние знаем, че тази процедура не терминира и поради това не е алгоритъм. Но тази теорема е **изключително трудна** за доказване. Доказателството ѝ е било непреодолимо предизвикателство в продължение на стотици години. През 90те години на 20 век най-накрая Andrew Wiles направи доказателство [95], чието прочитане (с разбиране ...) остава разбираемо за сравнително малко хора. За подробности вижте *тази онлайн, свободно достъпна, книга на Nigel Boston*.

Трети пример: хипотезата на Collatz

Хипотеза 2: Collatz

Нека $C : \mathbb{N}^+ \to \mathbb{N}^+$ е дефинирана така:

$$C(n) = \begin{cases} \frac{n}{2}, & \text{ако } n \text{ е четно,} \\ 3n+1, & \text{ако } n \text{ е нечетно.} \end{cases}$$

Нека $C^{(0)}(n) = n$ и $C^{(k+1)}(n) = C(C^{(k)}(n))$, за всяко $n \in \mathbb{N}^+$ и всяко $k \in \mathbb{N}$. Тогава,

$$\forall n\, \exists m : C^{(m)}(n) = 1$$

Хипотезата казва, че за итераторът (вижте Подсекция 1.1.6) $\boxed{n \mapsto C(n)}$ достига 1 от всяко цяло положително n. Например, ако започнем с 14, ще минем през следните стойности, докато стигнем до единицата:

$$14 \mapsto 7 \mapsto 22 \mapsto 11 \mapsto 34 \mapsto 17 \mapsto 52 \mapsto 26 \mapsto 13 \mapsto 40 \mapsto 20 \mapsto 10 \mapsto 5 \mapsto 1$$

Въпреки значителните усилия на много математици в продължение на много години, хипотезата остава именно хипотеза – нито е намерен контрапример, нито има доказателство. За повече информация вижте статията на Belaga и Mignotte [7] и огромната библиография от статии за хипотезата на Collatz [60], [61].

Това, че $\langle 1, 4, 2, 1 \rangle$ е орбита за този итератор, е очевидно. Ако игнорираме четните числа, което спокойно можем да направим тук—всяко четно число при достатъчно много деления на 2 дава нечетно число—имаме право да кажем, че 1 е фиксирана точка за итератора на Collatz. Ерго, ако хипотезата е вярна, то итераторът на Collatz влиза в орбита $\langle 1, 4, 2, 1 \rangle$, стартирайки от кое да е цяло положително число. Ако хипотезата е вярна и игнорираме четните числа, то итераторът влиза във фиксирана точка 1, стартирайки от кое да е цяло положително число.

Следният алгоритъм имплементира идеята за финитност, необходимо и достатъчно условие за която е хипотезата на Collatz да е истина.

```
DUMMY–COLLATZ(n ∈ ℕ⁺)
1  while n ≠ 1 do
2      if iseven(n)
3          n ← n/2
4      else
5          n ← 3n + 1
```

Очевидно DUMMY-COLLATZ терминира за всяко n ако и само ако Хипотеза 2 е истина.

Наблюдение 2

Доказателството за финитност, което е само един аспект от доказателството за коректност на даден алгоритъм, може да бъде изключително трудно и да изисква дълбоки познания по математика.

Забележка

Алгоритмите, които разглеждаме в този курс, са значително по-прости от посочените в тази подсекция и доказателствата за тяхното терминиране по правило са тривиални. За типичния алгоритъм в курса ние дори не споменаваме това, че той терминира, когато правим анализ на коректността му – няма смисъл да подчертаваме очевидното.

2.1.2 Верификация на рекурсивни алгоритми: по индукция

Ще говорим подробно за рекурсивни алгоритми в Глава 5.1. Ние, разбира се, знаем какво е рекурсивен алгоритъм: това е алгоритъм, който вика веднъж или повече пъти себе си върху по-малък вход, като задължително има адекватни "спирачки на рекурсията", които гарантират финитността.

Формално и прецизно доказателство за коректност на рекурсивен A алгоритъм са прави по индукция. Базата на доказателството отговаря на спирачките на рекурсията – проверяваме, че за всеки вход x с големина, която задейства "спирачка", $A(x)$ връща точно това, което очакваме. Следва същинската част на доказателството. Разглеждаме вход w с големина, която не задейства "спирачка". Това значи, че $A(w)$ прави едно или повече викания върху входове с по-малки големини. Допускаме, че всяко от тези викания връща точно това, което очакваме—това е индуктивното предположение—и въз основа на допускането доказваме, че $A(w)$ връща точно това, което очакваме.

Да разгледаме няколко примера.

2.1.2.1 Нарастване с единица

Това е елементарен алгоритъм, който получава число на входа и връща това число, инкрементирано с единица. На практика никой не инкрементира числа по този начин; би било пълна безсмислица да го изпълняваме, при положение, че разполагаме с механизъм да добавяме единица (ред 5), така че може да заменим целия алгоритъм с $\boxed{\textbf{return } n+1}$. Нещо повече, всеки процесор има инструкции за събиране, които са изключително бързи, така че и викане на процедура за добавяне на единица е безсмислено на практика. Алгоритъмът е стриктно с учебна цел.

Забележете, че индукцията тук не е по големината на входа—както е в почти всички примери, които ще видим в тези лекции—а по стойността на входа. Причината за това отклонение от правилото е, че входът е число (вижте дискусията в Подсекция 2.2.1 за входове, състоящи се от едно число или константен брой числа).

Нарастване с единица(n: естествено число)
```
1  if n = 0
2      return 1
3  else
4      if n mod 2 = 0
5          return n + 1
6      else
7          return 2 × Нарастване с единица(⌊n/2⌋)
```

Теорема 3: Коректността на алгоритъма Нарастване с единица

Алгоритъмът Нарастване с единица връща $n+1$ за всеки вход n.

Доказателство:
Със силна индукция по n.

База. $n = 0$. В такъв случай булевият израз на ред 1 е истина и алгоритъмът връща $0+1=1$ чрез присвояването на ред 2. ✓

Индуктивно Предположение. Нека за всяко $m < n$ алгоритъмът връща $m+1$.

Индуктивна стъпка. Да разгледаме работата на Нарастване с единица върху вход $n \geqslant 1$. Първо да допуснем, че n е четно. Тогава булевият израз на ред 4 е истина, следователно ред 5 се изпълнява и алгоритъмът връща $n+1$. ✓

Сега да допуснем, че n е нечетно. Булевият израз на ред 4 е лъжа, следователно ред 7 се изпълнява и алгоритъмът връща $2 \times$ Нарастване с единица $\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$. Тъй като $\left\lfloor \frac{n}{2} \right\rfloor < n$, може да приложем индуктивното предположение. Съгласно него, Нарастване с единица $\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$ връща $\left\lfloor \frac{n}{2} \right\rfloor + 1$. Тъй като n е нечетно, $n = 2k+1$ за някое $k \in \mathbb{N}$, и изходът е

$$2 \times \left(\left\lfloor \frac{2k+1}{2} \right\rfloor + 1\right) = 2 \times \left(\left\lfloor k + \frac{1}{2} \right\rfloor + 1\right) = 2 \times (k+1) = 2k+2 = n+1 \qquad ✓$$

Това е и краят на доказателството. □

В това доказателство използва едно неявно допускане: итераторът (вижте Подсекция 1.1.6) $n \mapsto \left\lfloor \frac{n}{2} \right\rfloor$ достига 0, започвайки от произволно естествено число. Това е вярно и може лесно да се докаже по индукция, ако разбием $\mathbb{N}$ на подходящи подмножества $S_0 = \{0\}$, $S_1 = \{1\}$, $S_2 = \{2,3\}$, $S_3 = \{4,5,6,7\}$, и така нататък, такива че въпросният итератор изобразява S_k в S_{k-1}, и извършим индукцията по индекса на тези множества, но отгоре надолу. Този факт ни позволява да сме убедени, че базата $n = 0$ е достатъчна за рекурсията в Нарастване с единица.

2.1.2.2 Максимум на несортиран масив, рекурсивен

Даден е произволен масив от n числа. Търси се максималният елемент на масива. Има елементарен итеративен алгоритъм за тази задача, описан и верифициран в Подподсекция 2.1.3.1. Тук ще разгледаме рекурсивен алгоритъм за задачата. С учебна цел, базата е за $n = 0$ и съответният максимум е $-\infty$; на практика базата би била за $n = 1$ и съответният максимум би бил $A[1]$.

```
Max Unsorted, rec(A[1,2,...,n]: array of integers)
1  if n = 0
2      return −∞
3  else
4      max ← Max Unsorted, rec(A[1,2,...,n−1])
5      if A[n] > max
6          max ← A[n]
7      return max
```

Теорема 4: Коректността на алгоритъма Max Unsorted, rec

Ако $A[1,\ldots,n]$ е произволен масив от числа, то Max Unsorted, rec$(A[1,\ldots,n])$ връща максималния елемент на $A[1,\ldots,n]$.

Доказателство: По индукция по n. Забележете, че това е и индукция по големината на входа, защото в тази задача n е и големината на входа, за разлика от алгоритъма Нарастване с единица на стр. 31, в който индукцията пак е по n, но големината на входа винаги е единица.

Базовият случай е $n = 0$. От една страна, масивът е празен, така че максимумът му е $-\infty$, която е неутралният елемент на операцията максимум. От друга страна, условието на ред 1 е истина и алгоритъмът връща $-\infty$ на ред 2. ✓

Да допуснем, че за някое $n-1$, такова че $n-1 \geqslant 0$, Max Unsorted, rec$(A[1,\ldots,n-1])$ връща максималния елемент на $A[1,\ldots,n-1]$. Разглеждаме викането Max Unsorted, rec$(A[1,\ldots,n])$. Тъй като $n > 0$, условието на ред 1 е лъжа и изпълнението отива на ред 4. Съгласно индуктивното предположение, променливата *max* получава стойността на максимума на $A[1,\ldots,n-1]$). Следните две възможности са изчерпателни:

- $A[n]$ е по-голям от всеки елемент на $A[1,\ldots,n-1]$. Прилагаме индуктивното предположение и заключаваме, че $A[n] >$ *max*. Условието на ред 5 е истина и изпълнението отива на ред 6. Там на *max* се присвоява $A[n]$, което, при текущите допускания, е максимумът на $A[1,\ldots,n]$. После изпълнението отива на ред 7.
- Не е вярно, че $A[n]$ е по-голям от всеки елемент на $A[1,\ldots,n-1]$. Тогава максимумът на $A[1,\ldots,n]$ е максимумът на $A[1,\ldots,n-1]$. Прилагаме индуктивното предположение и заключаваме, че на ред 4, на *max* се присвоява максимума на $A[1,\ldots,n]$. Условието на ред 5 е лъжа и изпълнението отива на ред 7.

Във всеки от тези случаи, на ред 7, променливата *max* съдържа максимума на $A[1,\ldots,n]$. Заключаваме, че алгоритъмът връща максимума на $A[1,\ldots,n]$. □

2.1.2.3 Максимум на битоничен масив

Неформално, масив се нарича *битоничен*, на английски *bitonic*, ако започва като строго растящ, след което става строго намаляващ. Формално, ако $A[1,\ldots,n]$ е числен масив, то A е битоничен, ако съществува индекс k, такъв че $1 \leqslant k \leqslant n$ и

$$A[1] < A[2] < \cdots < A[k]$$
$$A[k+1] > A[k+2] > \cdots > A[n]$$

Забележете, че нарастващата част е празна, в случай, че $k = 1$; тогава първата верига от неравенства съдържа нула неравенства и е тривиално истина. Аналогично, намаляващата част е празна, ако $k = n$.

Дефиницията не казва нищо за взаимните големини на $A[k]$ и $A[k+1]$.

- Може $A[k] < A[k+1]$. В този случай нарастващата част е $A[1,\ldots,k+1]$, а намаляващата е $A[k+1,\ldots,n]$, като те имат общ елемент $A[k+1]$.
- Може $A[k] > A[k+1]$. В този случай нарастващата част е $A[1,\ldots,k]$, а намаляващата е $A[k,\ldots,n]$, като те имат общ елемент $A[k]$.

- Може $A[k] = A[k+1]$. В този случай нарастващата част е $A[1, \ldots, k]$, а намаляващата е $A[k+1, \ldots, n]$, като те нямат общ елемент.

Следователно, или има точно един, има точно два максимални елемента, които са съседи в масива. Всеки от следните масиви е битоничен:

$[1, 2, 3, 4, 9]$
$[9, 4, 3, 2, 1]$
$[1, 2, 3, 9, 1]$
$[1, 2, 3, 9, 9, 1]$

Повече от два максимални елемента обаче не може да има. Освен това, ако $A[k] = A[k+1]$, със сигурност това е максималният елемент.

MAX BITONIC($A[1, \ldots, n]$, ℓ, h; изпълнено е $1 \leqslant \ell \leqslant h \leqslant n$)
1 **if** $\ell = h$
2 **return** $A[\ell]$
3 mid $\leftarrow \left\lfloor \frac{\ell+h}{2} \right\rfloor$
4 **if** $A[\text{mid}] < A[\text{mid}+1]$
5 **return** MAX BITONIC(A, mid $+ 1$, h)
6 **else if** $A[\text{mid}] > A[\text{mid}+1]$
7 **return** MAX BITONIC(A, ℓ, mid)
8 **else**
9 **return** $A[\text{mid}]$

Теорема 5: Коректността на алгоритъма MAX BITONIC

Ако $A[1, \ldots, n]$ е битоничен и $1 \leqslant \ell \leqslant h \leqslant n$, то MAX BITONIC$(A, \ell, h)$ връща максималния елемент на подмасива $A[\ell, \ldots, h]$.

Доказателство:
Управляваш параметър за рекурсията е разликата $h - \ell$. Това си заслужава да се повтори: въпреки че има два числени аргумента, а именно ℓ и h, рекурсията е "едномерна"[†], понеже е по разликата $h - \ell$; рекурсивно викане се прави с по-малка съответна разлика и "спирачката" на рекурсията се "задейства" от тази разлика.

Базовият случай е $h - \ell = 0$. Той "задейства спирачката" на рекурсията, като условието на ред 1 е истина и алгоритъмът връща $A[\ell]$ на ред 2. Това е коректно, понеже единственият елемент е максимален.

Да разгледаме стойности на ℓ и h, които не "задействат спирачката" на рекурсията. С други думи, $\ell < h$. Индуктивното предположение е, че за всички h' и ℓ', такива че $1 \leqslant \ell' \leqslant h' \leqslant n$ и $h' - \ell' < h - \ell$, викането на MAX BITONIC(A, ℓ', h') връща максимума на $A[\ell', \ldots, h']$.

Изпълнението отива на ред 3, където mid става $\left\lfloor \frac{\ell+h}{2} \right\rfloor$. Преди да продължим с доказателството, да се убедим, че елементите $A[\text{mid}]$ и $A[\text{mid}+1]$ се намират в масива $A[\ell, \ldots, h]$; с други думи, че няма опит за достъп извън подмасива от входа.

- Първо, $\ell \leqslant \text{mid}$, защото $\ell \leqslant \left\lfloor \frac{\ell+h}{2} \right\rfloor$ при $\ell < h$, като равенство е възможно.

[†]"Двумерна" рекурсия е, примерно, пресмятането на биномния коефициент съгласно $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

- Второ, $\text{mid} < \mathtt{h}$, защото

$$\begin{aligned}
\left\lfloor \frac{\ell + \mathtt{h}}{2} \right\rfloor &< \mathtt{h} \\
\left\lfloor \frac{\ell + \mathtt{h}}{2} - \mathtt{h} \right\rfloor &< 0 \\
\left\lfloor \frac{\ell - \mathtt{h}}{2} \right\rfloor &< 0 \quad // \text{ нека } \mathtt{h} = \mathtt{l} + \mathtt{k}, \text{ където } \mathtt{k} > 0 \\
\left\lfloor \frac{-\mathtt{k}}{2} \right\rfloor &< 0
\end{aligned}$$

  като последното е вярно (почти) директно от дефиницията на $\lfloor\ \rfloor$.

И така, $\mathtt{A}[\text{mid}]$ и $\mathtt{A}[\text{mid} + 1]$ се намират в масива $\mathtt{A}[\ell, \ldots, \mathtt{h}]$.

Следните възможности са изчерпателни:

- $\mathtt{A}[\text{mid}] < \mathtt{A}[\text{mid} + 1]$. В такъв случай нито един от $\mathtt{A}[\ell]$, ..., $\mathtt{A}[\text{mid}]$ не е максимален. Тогава максималният елемент се намира в подмасива $\mathtt{A}[\text{mid} + 1, \ldots, \mathtt{h}]$. Но $\mathtt{h} - (\text{mid} + 1) < \mathtt{h} - \ell$ и можем да приложим индуктивното предположение, от което следва, че рекурсивното викане на ред 5 ще върне максимума на $\mathtt{A}[\text{mid} + 1, \ldots, \mathtt{h}]$. Както вече установихме, той е и максимумът на $\mathtt{A}[\ell, \ldots, \mathtt{h}]$.

- $\mathtt{A}[\text{mid}] > \mathtt{A}[\text{mid}+1]$. В такъв случай нито един от $\mathtt{A}[\text{mid}+1]$, ..., $\mathtt{A}[\mathtt{h}]$ не е максимален. Тогава максималният елемент се намира в подмасива $\mathtt{A}[\ell, \ldots, \text{mid}]$. Но $\text{mid} - \ell < \mathtt{h} - \ell$ и можем да приложим индуктивното предположение, от което следва, че рекурсивното викане на ред 7 ще върне максимума на $\mathtt{A}[\ell, \ldots, \text{mid}]$. Както вече установихме, той е и максимумът на $\mathtt{A}[\ell, \ldots, \mathtt{h}]$.

- $\mathtt{A}[\text{mid}] = \mathtt{A}[\text{mid} + 1]$. В такъв случай имаме двоен максимум $\mathtt{A}[\text{mid}] = \mathtt{A}[\text{mid} + 1]$, и алгоритъмът връща точно тази стойност на ред 9. □

За пълнота на доказателството е добре да споменем изрично следния факт, който е тривиално следствие на Теорема 5.

Следствие 1: Коректността на началното викане на MAX BITONIC

MAX BITONIC$(\mathtt{A}, 1, \mathtt{n})$ връща максималния елемент на масива $\mathtt{A}$.

2.1.2.4 Евклидовият алгоритъм в рекурсивен вариант

Тъй като става дума за *най-голям общ делител*, на английски *greatest common divisor*, накратко *gcd*, нека за пълнота дефинираме това добре познато понятие.

Определение 7: $\gcd(\mathtt{a}, \mathtt{b})$

Нека $\mathtt{a}$ и $\mathtt{b}$ са естествени числа, които не са и двете нули. Тогава $\gcd(\mathtt{a}, \mathtt{b})$ е най-голямото естествено число, което дели и $\mathtt{a}$, и $\mathtt{b}$.

Следствие 2

Ако $a \in \mathbb{N}^+$, то $\gcd(a, 0) = a$.

Съгласно [34, (3.21) на стр. 82]:

Определение 8: mod: остатък при целочислено деление

Нека $a \in \mathbb{N}$ и $b \in \mathbb{N}^+$. Тогава

$$a \bmod b = a - b \left\lfloor \frac{a}{b} \right\rfloor$$

Лема 1

Всеки общ делител на a и b също така дели и $a \bmod b$.

Доказателство: Нека c е общ делител на a и b. Тогава, за някакви цели положителни p и q, $a = cp$ и $b = cq$. Но $a \bmod b = a - b \left\lfloor \frac{a}{b} \right\rfloor$ от Определение 8, така че $a \bmod b = cp - cq \left\lfloor \frac{a}{b} \right\rfloor = ck$ за някое естествено k. Ерго, c дели $a \bmod b$. □

Следната лема е ключова за верификацията на Евклидовия алгоритъм.

Лема 2

Нека $a, b \in \mathbb{N}$ и $a \geqslant b$, $a > 0$. Тогава

$$\gcd(a, b) = \begin{cases} a, & \text{ако } b = 0 \\ \gcd(b, a \bmod b), & \text{в противен случай} \end{cases}$$

□

Доказателство: Ако $a > 0$ и $b = 0$, то $\gcd(a, b) = a$ от Следствие 2. Нека $a \geqslant b > 0$.

Ако b дели a, то $a \bmod b = 0$, така че "$\gcd(b, a \bmod b)$" става "$\gcd(b, 0)$", а $\gcd(b, 0) = b$ от Следствие 2. От друга страна, $\gcd(a, b) = b$ в този случай, така че твърдението пак е вярно.

Нека b не дели a. Нека $a \bmod b = c$, като $c > 0$. Тогава $c = a - bk$, за някое цяло k. Ако някое цяло d дели както a, така и b, то d дели и $a - bk$; тоест, d дели c и b. От друга страна, ако d дели c и b, то d дели и $c + bk$, което е a; тоест, d дели a. Ерго, d е общ делител на c и b тстк d е общ делител на a и b. Ерго, множеството от делителите на c и b точно съвпада с множеството от делителите на a и b. Щом имат едни и същи делители, в частност най-големият им делител е един и същи. □

Ето отново рекурсивната версия на Евклидовия алгоритъм от Подсекция 1.2.2.

```
EUCLID, REC(a, b ∈ ℕ, a ⩾ b, a > 0)
1  if b = 0
2      return a
3  else
4      return EUCLID, REC(b, a mod b)
```

Теорема 6: Коректността на алгоритъма EUCLID, REC

Алгоритъмът EUCLID, REC(a, b) връща $\gcd(a, b)$ за всички естествени a и b, такива че $a \geqslant b$.

Доказателство: Рекурсията е двумерна, в смисъл, че има два аргумента и началното условие е едномерното $b = 0$. Приемаме за очевидно, че за всяко $a, b \in \mathbb{N}$, такива че $a \geqslant b$:

- итераторът $\langle a, b\rangle \mapsto \langle b, a \bmod b\rangle$ винаги достига до $\langle x, 0\rangle$ за някое $x > 0$,
- итераторът $\langle a, b\rangle \mapsto \langle b, a \bmod b\rangle$ изобразява наредената двойка $\langle a, b\rangle$ в наредена двойка $\langle p, q\rangle$, която е по-малка в смисъл, че $p \leqslant a$ и $q < b$.

Второто наблюдение ни дава право да правим доказателството със силна индукция по наредените двойки $\langle a, b\rangle$.

База. В базовия случай разглеждаме наредените двойки $\langle a, 0\rangle$ за $a > 0$. Тогава условието на ред 1 е истина и изпълнението отива на ред 2 и връща a. Но по дефиниция, $\gcd(a, 0) = a$, ако $a > 0$. ✓

Индуктивно Предположение. Нека за някакви естествени a и b, такива че $a \geqslant b > 0$, за всяка наредена двойка $\langle p, q\rangle$, такава че $p \leqslant a$ и $q < b$, EUCLID, REC(p, q) връща $\gcd(p, q)$.

Индуктивна стъпка. Разглеждаме EUCLID, REC(a, b). Щом $b > 0$, условието на ред 1 е лъжа и изпълнението отива на ред 4, където се прави викане на EUCLID, REC$(b, a \bmod b)$. Но $b \leqslant a$ и $a \bmod b < b$, ерго, съгласно индуктивното предположение, викането на EUCLID, REC$(b, a \bmod b)$ връща $\gcd(b, a \bmod b)$. От Лема 2 знаем, че $\gcd(a, b) = \gcd(b, a \bmod b)$, ако $b > 0$. Следователно, това, което EUCLID, REC(a, b) връща на ред 4, е $\gcd(a, b)$, което и трябваше да покажем. □

2.1.3 Верификация на итеративни алгоритми: с инвариант

Доказателствата за коректност с инвариант на цикъла по същество също са доказателства по индукция, но с индукцията не е по големината на входа, а по броя на "завъртанията" на цикъла.

Инвариантът е **едноместен предикат**, свързан с цикъла. Променливата на предиката е броят на достиганията на дадена инструкция на алгоритъма. По правило, тази инструкция е инструкцията, която съдържа условието за край на цикъла. Да кажем, че условието за край на цикъла е на ред ℓ. Въпросният предикат трябва да е:

- верен първия път, когато изпълнението на алгоритъма е на ред ℓ,
- верността му трябва да се запазва по време на всяко изпълнение, откъдето идва и името "инвариант", и
- в момента на напускане на цикъла (такъв момент настъпва неизбежно, щом алгоритъмът завършва) неговата вярност трябва да влече директно това, което искаме да докажем за алгоритъма.

Тъй като алгоритъмът завършва работата си рано или късно, то броят на пътите, в които изпълнението е на ред ℓ, е **краен**. Ако ред ℓ се изпълнява точно m пъти, то индукцията е върху крайното множество $\{1, 2, , \ldots, m\}$

Това, какъв предикат използваме в доказателството, е ключово. Ако не успеем да измислим адекватен предикат, няма да имаме доказателство, колкото и добре да разбираме техниката с инвариант по принцип. Най-общо, предикатът се формулира така:

Инвариант

При k-тото достигане на ред ℓ на алгоритъм НЯКАКЪВ АЛГОРИТЪМ е изпълнено *«някакво-твърдение-формулирано-чрез-*k*»*.

Ако предикатът е $P(k)$, то доказателството следва позната схема на доказателства по индукция върху **краен** домейн (който е индуктивно дефинирано множество):

$$P(1) \wedge \forall k_{1 \leqslant k < m}(P(k) \to P(k+1)) \vdash \forall k_{1 \leqslant k \leqslant m}(P(k))$$

По отношение на някои алгоритми предикатът може да се дефинира по-просто: директно върху управляващата променлива на цикъла. Това не винаги е възможно, но, ако е възможно, доказателството става по-елегантно. Нека цикълът е **for**-цикъл с инкрементиране на управляващата променлива с единица:

for $i \leftarrow n_1$ **to** n_2
  «тяло-на-цикъла»

за някакви фиксирани n_1 и n_2 и освен това i не бива променяна вътре в тялото на цикъла. Тогава може да вземем за домейн на предиката множеството $\{n_1, \ldots, n_2 + 1\}$[†] и да дефинираме предиката директно като $P(i)$, където i е управляващата променлива на цикъла. Но при по-сложни цикли, примерно **while**-цикли, в които управляващата променлива се мени в тялото на цикъла по някакъв сложен начин, по-удачно е предикатът да е върху броя пъти на достигането на реда с условието.

Пример за верификация на итеративен алгоритъм с инвариант Това е може би най-простият итеративен алгоритъм. Ще докажем коректността му с инвариант.

```
FIND SUM(A[1, 2, ..., n]: array of integers)
1  sum ← 0
2  i ← 1
3  while i ⩽ n do
4      sum ← sum + A[i]
5      i ← i + 1
6  return sum
```

Теорема 7: Коректността на алгоритъм за сумата от елементите на масив

FIND SUM връща сумата от елементите на входния масив $A[1, 2, \ldots, n]$.

Доказателство: Доказателството на теоремата се основава на следното помощно твърдение, което ще докажем първо.

[†]Това е множеството от стойностите, които управляващата променлива взема. Забележете, че най-голямата от тях е $n_2 + 1$, а не n_2, понеже при последното достигане на реда с условието—когато условието вече не е изпълнено и тялото на цикъла няма да се изпълнява повече—управляващата променлива е именно $n_2 + 1$.

Инвариант

Всеки път, когато изпълнението на FIND SUM е на ред 3, променливата *sum* съдържа сумата от елементите на подмасива $A[1,\ldots,i-1]$.

Доказателството на инварианта е по индукция по броя на достиганията на ред 3.

База. Да разгледаме първото достигане на ред 3. От една страна, подмасивът $A[1,\ldots,i-1]$ в този момент е празен, защото i съдържа стойност 1 заради присвояването на ред 2, следователно $A[1,\ldots,i-1]$ всъщност е $A[1,\ldots,0]$, а това е празният подмасив. Сумата от елементите на празния масив е 0, защото 0 е неутралният елемент на операцията събиране. Следователно, математически погледнато, сумата от елементите на $A[1,\ldots,i-1]$ е 0. От друга страна, променливата *sum* съдържа 0 заради присвояването на ред 2. ✓

Индуктивно предположение. Да допуснем, че твърдението е вярно при някое достигане на ред 3, което не е последното. И така, *sum* съдържа сумата от елементите на подмасива $A[1,\ldots,i-1]$.

Преди да продължим с доказателството, подчертаваме, че би било грешка да изпуснем уточнението "което не е последното", защото искаме да се убедим, че инвариантът се запазва при преминаване през тялото на цикъла, а след последното достигане на ред 3, повече преминаване през тялото цикъла **няма**.

Индуктивна стъпка. След присвояването на ред 4, прилагайки и индуктивното предположение, заключаваме, че *sum* съдържа сумата от елементите на подмасива $A[1,\ldots,i-1,i]$. Но след това на ред 5, променливата i се инкрементира. Изразено чрез новото i, заключението става " *sum* съдържа сумата от елементите на подмасива $A[1,\ldots,i-1]$". Но това е точно същото нещо, което инвариантът казва. Ерго, инвариантът се запазва.

С това доказателството на инварианта приключи. Да се върнем към доказателството на теоремата.

Следствие от инварианта. FIND SUM очевидно терминира, така че някое достигане на ред 3 е последно. Да разгледаме моментът, в който ред 3 се изпълнява за последен път. Тогава i съдържа стойността $n+1$. Да заместим i с $n+1$ в инварианта. Получаваме буквално "променливата *sum* съдържа сумата от елементите на подмасива $A[1,\ldots,(n+1)-1]$.". По-просто казано, *sum* съдържа сумата от елементите на масива $A[1,\ldots,n]$.

Доказателството е почти готово. Забелязваме, че веднага след това се изпълнява ред 6. FIND SUM връща *sum*, която променлива съдържа сумата от елементите на входния масив. Следователно, FIND SUM връща сумата от елементите на входния масив, което и трябваше да покажем. □

В следващите доказателства за коректност с инвариант няма да правим следствие от инварианта, както сторихме току-що, а в доказателството ще добавяме още една фаза, наречена **Терминация**. Освен това, за краткост ще сливаме фазите **Индуктивно предположение** и **Индуктивна стъпка** в една единствена фаза, наречена **Поддръжка**. Въпросните доказателства имат следната структура съгласно учебника CLR [21]. Инвариантът е едноместен предикат, да го наречем $P(k)$, който е свързан с този ред от алгоритъма, който съдържа условието, управляващо цикъла. В най-общия случай, променливата k е броят на достиганията на въпросния ред от началото на изпълнението, включително и сегашното достигане. Общата схема на доказателството, както стана ясно от изложението досега, е:

- **База** Показваме, че $P(1)$ е истина при първото достигане на въпросния ред.

- **Поддръжка** Допускайки, че $P(k)$ е в сила за някое достигане, което не е последното, показваме, че $P(k+1)$ е в сила при следващото достигане.
- **Терминация** Разглеждаме момента, в който изпълнението е на въпросния ред от алгоритъма за последен път. Нека k' е броят на всички достигания до въпросния ред, включително и последното (текущото). Заместваики k с k' в инварианта, получаваме точно това твърдение за алгоритъма, което ни трябва.

Важен частен случай, който вече дискутирахме горе, е следният. Да кажем, че цикълът е **for**-цикъл от вида (ℓ е номерът на съответния ред в алгоритъма):

ℓ: **for** $i \leftarrow a$ to b

a и b са константи, а променливата i не се променя в тялото на цикъла. Тогава за простота променливата на предиката е управляващата променлива на цикъла, тоест предикатът е $P(i)$. Общата схема на доказателството в този частен случай е:

- Показваме, че $P(a)$ е вярно при първото достигане на ред ℓ.
- Допускайки, че $P(i)$ е вярно при някое достигане на ред ℓ, което не е последното, (което значи, че $i \leqslant b$), доказваме $P(i+1)$ при следващото достигане на ред ℓ.
- Разглеждаме момента, в който доказателството е на ред ℓ за последен път. Тогава трябва i да съдържа $b+1$. Заместваики i с $b+1$, получаваме получаваме точно това твърдение за алгоритъма, което ни трябва.

Дебело подчертаваме следните две неща.

- Първо, може да има много начини да се мине през тялото на цикъла в зависимост от стойностите както на контролната променлива, така и на останалите променливи. Например, може тялото на цикъла да има сложна вложена **if-else-if** структура. Доказателството трябва да следва всеки възможен път на изпълнение, тоест всяка възможност за преминаване през тази структура.

  Ето пример. Нека алгоритъмът има променливи-естествени числа a, b и t и тялото на цикъла има такава структура:

```
if i mod 5 = 1 then
    if t is a perfect square then
        a ← b + 1
    else if t < 200 then
        a ← b + 2
    else
        a ← b + 3
else if i mod 5 = 2 then
        b ← b^2
else
        b ← b − t
```

Пълно доказателство трябва да разглежда всички начини да се "мине" през тези случаи и подслучаи, а това зависи от делимостта на $\mathtt{i}$ на 5 и освен това, в случай, че остатъкът е 1, от стойността на $\mathtt{t}$.

Ето друг пример. Ако тялото на цикъла има $\mathtt{k}$ на брой последователни **if-else** блока:

if *«булево условие 1»* **then**
    «някакви императивни инструкции»
else
    «някакви императивни инструкции»
if *«булево условие 2»* **then**
    «някакви императивни инструкции»
else
    «някакви императивни инструкции»
...
if *«булево условие k»* **then**
    «някакви императивни инструкции»
else
    «някакви императивни инструкции»

то от най-общи комбинаторни съображения има $2^{\mathtt{k}}$ начина, по които изпълнението може да мине през всички тези **if-else**. Доказателството, което разглежда всеки от тези $2^{\mathtt{k}}$ начина, може да е изключително дълго.

- Въпросният предикат $\mathtt{P}$ трябва не просто да е верен, а освен това да ни върши работа за доказателството. Не всеки верен предикат ни върши работа. Като пример, да разгледаме следния алгоритъм.

```
AlgY(A[1,2,...n]: масив от цели числа)
  1   for i ← 1 to n
  2       A[i] ← i
```

Да разгледаме невалидно доказателство, че това е сортиращ алгоритъм (какъвто той не е), което се основава на следния инвариант.

Инвариант

Всеки път, когато изпълнението на AlgY е на ред 1, подмасивът $\mathtt{A}[1,\ldots,\mathtt{i}-1]$ е сортиран.

Този инвариант верен ли е? Очевидно да, и доказването му е тривиално. AlgY сортиращ алгоритъм ли е? Очевидно не, защото той унищожава входните данни; това, че реализира сортиран масив, а именно $[1,2,\ldots,\mathtt{n}]$, не е достатъчно, за да е сортиращ. Защо верният инвариант не доказва коректността на AlgY? Защото, за да бъде един алгоритъм сортиращ, той трябва не просто да реализира сортиран масив, а трябва да реализира сортиран масив, който **се състои точно от входните елементи**. Още веднъж: инвариантът трябва да е не просто верен, а и полезен за целта на доказателството.

Наблюдение 3

Не всеки верен инвариант е полезен за това, което искаме да докажем.

Да разгледаме още няколко примера за доказателство на коректност чрез инвариант.

2.1.3.1 Максимум на несортиран масив, итеративен

На практика никой не би инициализирал max с $-\infty$; на практика човек би инициализирал max с $A[1]$ и i с 2. Следната имплементация е с учебна цел.

```
Max Unsorted, iter(A[1,2,...,n]: array of integers)
1  max ← −∞
2  i ← 1
3  while i ⩽ n do
4      if A[i] > max
5          max ← A[i]
6      i ← i + 1
7  return max
```

Теорема 8: Коректността на алгоритъма Max Unsorted, iter

Нека $A[1,\ldots,n]$ е произволен масив от числа. Max Unsorted, iter(A) връща максимума на A.

Доказателство: Заслужава си да се отбележи, че това доказателство ползва неявно факта, че Max Unsorted, iter не променя входния масив. Това е напълно очевидно: в нито една от четирите инструкции за присвояване (редове 1, 2, 5 и 6) не се присвоява стойност на елемент на A. Този факт облекчава доказателството, защото позволява да говорим просто за "масива A"; независимо от това дали става дума за входа, или за A по време на работата, или за A в края на алгоритъма, това е **един и същи масив**. В контраст на това, итеративната версия на Евклидовия алгоритъм от Подподсекция 2.1.3.3, Insertion Sort от Подсекция 3.4.1 и много други алгоритми в тези лекции менят входа и при тях се налага да въвеждаме специални означения за входните данни за даден момент от изпълнението. Примерно,

- в доказателството за коректност на Евклидовия алгоритъм използваме $\boxed{a'}$ и $\boxed{b'}$, за да означим съдържанието съответно на a и b преди началото на алгоритъма,
- в доказателството за коректност на Insertion Sort, в Лема 20 използваме означението $\boxed{A'}$ за съдържанието на масива A в самото начало на едно изпълнение на външния цикъл, а в Теорема 29 използваме означението $\boxed{A''}$ за съдържанието на A самото начало на алгоритъма.

Доказателството на Теорема 8 се основава на следния инвариант.

Инвариант

Всеки път, когато изпълнението на Max Unsorted, iter е на ред 3, променливата max съдържа стойността на максималния елемент в подмасива $A[1,\ldots,i-1]$.

База. Да разгледаме първото достигане на ред 3. От една страна, подмасивът $A[1,\ldots,i-1]$ е празен, защото i съдържа стойност 1 заради присвояването на ред 1, следователно $A[1,\ldots,i-1]$ всъщност е $A[1,\ldots,0]$, а това е празният подмасив. Максимумът на празния масив е $-\infty$, защото $-\infty$ е неутралният елемент на операцията максимум. От друга страна, променливата *max* съдържа $-\infty$ заради присвояването на ред 2. ✓

Поддръжка. Да допуснем, че твърдението е вярно при някое достигане на ред 3, което не е последното. Следните две възможности са изчерпателни по отношение на текущото i.

- $A[i]$ е по-голям от всеки елемент в $A[1,\ldots,i-1]$. Съгласно индуктивното предположение, *max* съдържа стойността на максимален елемент в $A[1,\ldots,i-1]$. Тогава *max* $< A[i]$. Следователно, условието на ред 4 е истина и изпълнението отива на ред 5, където *max* получава стойността на $A[i]$. И така, *max* съдържа максимума на $A[1,\ldots,i]$. Тогава обаче i се инкрементира с единица (ред 6). Написано чрез новото i, твърдението става "*max* съдържа максимума на $A[1,\ldots,i-1]$". Виждаме, че инвариантът се запазва при следващото достигане на ред 3.

- Не е вярно, че $A[i]$ е по-голям от всеки елемент в $A[1,\ldots,i-1]$. Тогава $A[i]$ е по-малък или равен на максимума на $A[1,\ldots,i-1]$, който на свой ред е равен на *max* съгласно индуктивното предположение. Заключаваме, че *max* съдържа максимума на $A[1,\ldots,i]$. В този случай, *max* $\not< A[i]$. Следователно, условието на ред 4 е лъжа и присвояването на ред 5 не се случва. После i се инкрементира с единица (ред 6). Написано чрез новото i, твърдението става "*max* съдържа максимума на $A[1,\ldots,i-1]$". Виждаме, че инвариантът се запазва при следващото достигане на ред 3.

Терминация. При последното достигане на ред 3 е вярно, че $i = n+1$. Заместваме i с $n+1$ в инварианта и получаваме "променливата *max* съдържа стойността на максималния елемент в подмасива $A[1,\ldots,(n+1)-1]$". По-просто казано, "променливата *max* съдържа стойността на максималния елемент в подмасива $A[1,\ldots,n]$". Но на ред 7, алгоритъмът връща именно *max*. Заключаваме, че MAX UNSORTED, ITER(A) връща максимума на A. □

2.1.3.2 Странен алгоритъм за сумиране на елементите на масив

Този алгоритъм нарочно е описан на C, а не на псевдокод.

```
1 int A[n];
2 int strangesum(int n) {
3   int i, s = 0;
4   for(i = 0; i < n; i ++) {
5     if (i%2 == 0)
6       s += A[i/2];
7     else
8       s += A[n - 1 - i/2]; }
9   return s; }
```

Трябва да се докаже, че функцията `strangesum` връща сумата на елементите на масива $A[0,1,\ldots,n-1]$. Известно е, че целочисленото деление `i/2` връща $\lfloor \frac{i}{2} \rfloor$.

Инвариант

При всяко достигане на ред 4:

$$s = \sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j]$$

База При първото достигане на ред 4, s съдържа 0 заради присвояването на ред 3. От друга страна, $s = \sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor + 1}^{n-1} A[j] = 0 + 0 = 0$, защото i е 0, така че $\left\lfloor \frac{0+1}{2} \right\rfloor = 0$ и $\left\lfloor \frac{0}{2} \right\rfloor = 0$, което на свой ред означава, че множествата $\{0, \ldots, \left\lfloor \frac{i+1}{2} \right\rfloor - 1\} = \{0, \ldots, -1\}$ и $\{n - \left\lfloor \frac{i}{2} \right\rfloor, \ldots, n-1\} = \{n-0, \ldots, n-1\}$ са празни. ✓

Поддръжка Нека твърдението е вярно при някое достигане, което не е последното.

Случай 1: i е четно Условието на ред 5 е истина и изпълнението достига до ред 6. Преди присвояването на ред 6, имаме $s = \sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j]$ от предположението. След присвояването, имаме

$$\begin{aligned} s &= \left(\sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] \right) + A\left[\left\lfloor \frac{i}{2} \right\rfloor\right] \\ &= \left(\sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] \right) + A\left[\left\lfloor \frac{i}{2} \right\rfloor\right] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] \end{aligned} \tag{2.1}$$

Тъй като i е четно, $i+1$ е нечетно. Лесно се вижда, че $\left\lfloor \frac{i+1}{2} \right\rfloor = \left\lfloor \frac{i}{2} \right\rfloor$, следователно (2.1) е равно на:

$$\left(\sum_{j=0}^{\left\lfloor \frac{i}{2} \right\rfloor - 1} A[j] \right) + A\left[\left\lfloor \frac{i}{2} \right\rfloor\right] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] =$$

$$\left(\sum_{j=0}^{\left\lfloor \frac{i}{2} \right\rfloor} A[j] \right) - A\left[\left\lfloor \frac{i}{2} \right\rfloor\right] + A\left[\left\lfloor \frac{i}{2} \right\rfloor\right] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] =$$

$$\sum_{j=0}^{\left\lfloor \frac{(i+1)+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i+1}{2} \right\rfloor}^{n-1} A[j] \tag{2.2}$$

При следващото достигане на ред 4, i се инкрементира с единица. Очевидно, спрямо новата стойност на i, израз (2.2) е:

$$\sum_{j=0}^{\left\lfloor \frac{i+1}{2} - 1 \right\rfloor} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j]$$

Виждаме, че инвариантът се запазва.

Случай 2: i е нечетно Условието на ред 5 е лъжа и изпълнението достига до ред 8. Преди присвояването на ред 8, имаме $s = \sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j]$ от предположението. След присвояването, имаме

$$\begin{aligned} s &= \left(\sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] \right) + A\left[n - 1 - \left\lfloor \frac{i}{2} \right\rfloor \right] \\ &= \sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \left(\left(\sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] \right) + A\left[n - 1 - \left\lfloor \frac{i}{2} \right\rfloor \right] \right) \\ &= \sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-1-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] \end{aligned} \tag{2.3}$$

При нечетно i, в сила е равенството $\left\lfloor \frac{i+1}{2} \right\rfloor = \left\lfloor \frac{i+2}{2} \right\rfloor$. Освен това, за всяко i е вярно, че $n - 1 - \left\lfloor \frac{i}{2} \right\rfloor = n - 1 + \left\lceil -\frac{i}{2} \right\rceil = n + \left\lceil -1 - \frac{i}{2} \right\rceil = n - \left\lfloor 1 + \frac{i}{2} \right\rfloor = n - \left\lfloor \frac{i+2}{2} \right\rfloor$. При нечетно i, последният израз е равен на $n - \left\lfloor \frac{i+1}{2} \right\rfloor$. Следователно, (2.3) е равно на:

$$\sum_{j=0}^{\left\lfloor \frac{(i+1)+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i+1}{2} \right\rfloor}^{n-1} A[j] \tag{2.4}$$

При следващото достигане на ред 4, i се инкрементира с единица. Спрямо новата стойност на i, (2.4) е:

$$\sum_{j=0}^{\left\lfloor \frac{i+1}{2} - 1 \right\rfloor} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j]$$

Виждаме, че инвариантът се запазва.

Терминация При последното достигане на ред 4, в сила е

$$\begin{aligned} i &= n \\ s &= \sum_{j=0}^{\left\lfloor \frac{i+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{i}{2} \right\rfloor}^{n-1} A[j] \end{aligned}$$

Следователно,

$$s = \sum_{j=0}^{\left\lfloor \frac{n+1}{2} \right\rfloor - 1} A[j] + \sum_{j=n-\left\lfloor \frac{n}{2} \right\rfloor}^{n-1} A[j] \tag{2.5}$$

Ще покажем, че $\left\lfloor \frac{n+1}{2} \right\rfloor - 1$ и $n - \left\lfloor \frac{n}{2} \right\rfloor$ са съседни стойности, нарастващи в този ред, за всяко

естествено n. Първо да допуснем, че е n е четно, тоест $n = 2k$ за някое естествено k. Тогава

$$\left\lfloor \frac{n+1}{2} \right\rfloor - 1 = \left\lfloor \frac{2k+1}{2} \right\rfloor - 1 = \left\lfloor \frac{2k}{2} \right\rfloor - 1 = k - 1$$

а

$$n - \left\lfloor \frac{n}{2} \right\rfloor = 2k - \left\lfloor \frac{2k}{2} \right\rfloor = 2k - k = k$$

Сега да допуснем, че n е нечетно, тоест $n = 2k + 1$ за някое естествено k. Тогава

$$\left\lfloor \frac{n+1}{2} \right\rfloor - 1 = \left\lfloor \frac{2k+2}{2} \right\rfloor - 1 = k + 1 - 1 = k$$

а

$$n - \left\lfloor \frac{n}{2} \right\rfloor = 2k + 1 - \left\lfloor \frac{2k+1}{2} \right\rfloor = 2k + 1 - \left\lfloor \frac{2k}{2} \right\rfloor = 2k + 1 - k = k + 1$$

Щом $\left\lfloor \frac{n+1}{2} \right\rfloor - 1$ и $n - \left\lfloor \frac{n}{2} \right\rfloor$ са съседни стойности, нарастващи в този ред, то интервалите

$$\left[0, 1, \ldots, \left\lfloor \frac{n+1}{2} \right\rfloor - 1\right] \text{ и } \left[n - \left\lfloor \frac{n}{2} \right\rfloor, n - \left\lfloor \frac{n}{2} \right\rfloor + 1, \ldots, n - 1\right]$$

са разбиване на интервала $[0, 1, \ldots, n-1]$. Следователно, (2.5) е равно на

$$s = \sum_{j=0}^{n-1} A[j]$$

2.1.3.3 Евклидовият алгоритъм в итеративен вариант

Ето отново итеративната версия на Евклидовия алгоритъм от Подсекция 1.2.2.

```
EUCLID, ITERATIVE(a, b ∈ ℕ⁺, a ⩾ b)
1  while a mod b > 0 do
2      r ← a mod b
3      a ← b
4      b ← r
5  return b
```

И неговото доказателство се основава на Лема 2.

Инвариант

Нека a' и b' означават съответно входните a и b в алгоритъм EUCLID, ITERATIVE. При всяко достигане на ред 1 е вярно, че $\gcd(a, b) = \gcd(a', b')$. Освен това, $b > 0$.

База. При първото достигане на ред 1, $a' = a$ и $b' = b$, така че $\gcd(a, b) = \gcd(a', b')$ е тривиално вярно. Това, че $b > 0$ следва от факта, че $b = b'$, а b' е положително число. ✓

Поддръжка. Да допуснем, че твърдението е вярно за някое достигане на ред 1, което не е последното. Да означим с a_1 и b_1 текущите стойности на a и b. Да означим с a_2 и b_2 стойностите на a и b при следващото достигане на ред 1. Допускането е, че $\gcd(a_1, b_1) = \gcd(a', b')$и $b_1 > 0$.

Щом достигането на реда не е последното, то $a_1 \bmod b_1 > 0$; тоест, b_1 не дели a_1. От Лема 2 и факта, че $b_1 > 0$ следва, че $\gcd(a_1, b_1) = \gcd(b_1, a_1 \bmod b_1)$. Но очевидно присвояванията на редове 2, 3 и 4 имат следния ефект: $a_2 = b_1$ и $b_2 = a_1 \bmod b_1$. Показахме, че $\gcd(a_1, b_1) = \gcd(a_2, b_2)$. Щом $\gcd(a_1, b_1) = \gcd(a', b')$ (от допускането) и $\gcd(a_1, b_1) = \gcd(a_2, b_2)$, то $\gcd(a', b') = \gcd(a_2, b_2)$. ✓

Терминиране. Да разгледаме последното достигане на ред 1. Да наречем съответно $\widehat{a}$ и $\widehat{b}$ текущите стойности на a и b. От инварианта знаем, че $\widehat{b} > 0$. Щом условието на ред 1 е лъжа, очевидно $\widehat{a} \bmod \widehat{b} = 0$. От Определение 8 следва, че $\widehat{a} = \widehat{b} \left\lfloor \frac{\widehat{a}}{\widehat{b}} \right\rfloor$. Тогава $\widehat{b}$ дели $\widehat{a}$. Освен това $\widehat{a} > 0$, така че $\gcd(\widehat{a}, \widehat{b}) = \widehat{b}$. От инварианта знаем, че $\gcd(a', b') = \gcd(\widehat{a}, \widehat{b})$. Веднага следва, че $\gcd(a', b') = \widehat{b}$. Ерго, която алгоритъмът на ред 5 връща $\gcd(a', b')$. □

2.1.3.4 Alg2SUM

Изчислителната задача 2SUM се споменава в Подсекция 3.2.6 и Подсекция 7.3.3. Тук само ще разгледаме алгоритъм за нея и ще докажем коректността му.

```
Alg2SUM(A[1,...,n]: масив от цели числа, такъв че A[1] ⩽ A[2] ⩽ ··· ⩽ A[n]; m ∈ ℤ)
1  i ← 1, j ← n
2  while i < j do
3      if A[i] + A[j] = m
4          return 1
5      else if A[i] + A[j] < m
6          i++
7      else
8          j--
9  return 0
```

За удобство да дефинираме *диада* като всяко двуелементно мултимножество $\{A[p], A[q]\}_M$, такова че $1 \leqslant p, q \leqslant n$, $p \neq q$ и $A[p] + A[q] = m$.

Теорема 9: Коректността на Alg2SUM

Нека входният масив на Alg2SUM е сортиран ненамаляващо. Алгоритъмът връща 1, ако във входния масив има поне една диада, а в противен случай връща 0.

Доказателство: Следното твърдение е инвариант за **while**-цикълът (редове 2–8).

Инвариант

Всеки път, когато изпълнението е на ред 2, всички диади в $A[1, \ldots, n]$ се намират в подмасива $A[i, \ldots, j]$.

База. Да разгледаме първото достигане на ред 2. Тогава i е 1, j е n, така че твърдението става "всички диади в $A[1, \ldots, n]$ се намират в подмасива $A[1, \ldots, n]$", което е тривиално вярно. ✓

Поддръжка. Да допуснем, че твърдението е вярно при някое достигане на ред 2, което не е последното. Щом не е последното, вярно е, че $A[i] + A[j] \neq m$, инак следващо достигане на ред 2 няма да има, защото условието на ред 3 би било истина и алгоритъмът би терминирал през ред 4. И така, $A[i] + A[j] \neq m$. Следните случаи са взаимно изключващи се и изчерпателни.

- $A[i] + A[j] < m$. От една страна е вярно, че $A[i] + A[j'] < m$ за всяко $j' < j$; това следва веднага от факта, че масивът е сортиран и релацията $\leqslant$ е транзитивна. С други думи, $A[i]$ не може да е участник в диада нито с $A[j]$, нито с $A[j-1]$, нито с $A[j-2]$, и така нататък. Но тогава всички диади в $A[i, \ldots, j]$ се намират в $A[i+1, \ldots, j]$. Тъй като по допускане всички диади в целия $A[1, \ldots, n]$ са в $A[i, \ldots, j]$, следва, че всички диади в $A[1, \ldots, n]$ се намират в $A[i+1, \ldots, j]$.

  Изпълнението отива на ред 6, където i бива инкрементирано. Спрямо новото i, твърдението става "всички диади в $A[1, \ldots, n]$ се намират в $A[i, \ldots, j]$". Виждаме, че инвариантът се запазва.

- $A[i] + A[j] > m$. От една страна е вярно, че $A[i'] + A[j] > m$ за всяко $i' > i$; това следва веднага от факта, че масивът е сортиран и релацията $\geqslant$ е транзитивна. С други думи, $A[j]$ не може да е участник в диада нито с $A[i]$, нито с $A[i+1]$, нито с $A[i+2]$, и така нататък. Но тогава всички диади в $A[i, \ldots, j]$ се намират в $A[i, \ldots, j-1]$. Тъй като по допускане всички диади в целия $A[1, \ldots, n]$ са в $A[i, \ldots, j]$, следва, че всички диади в $A[1, \ldots, n]$ се намират в $A[i, \ldots, j-1]$.

  Изпълнението отива на ред 8, където j бива декрементирано. Спрямо новото j, твърдението става "всички диади в $A[1, \ldots, n]$ се намират в $A[i, \ldots, j]$". Виждаме, че инвариантът се запазва.

Терминация. Да разгледаме последното достигане на ред 2. Точно едно от следните две е вярно.

- $i < j$ и $A[i] + A[j] = m$. Изпълнението отива на ред 3, условието е истина и на ред 4 алгоритъмът връща 1. Доказахме верността на теоремата в този случай: масивът съдържа поне една диада, а алгоритъмът връща единица. ✓.

- $\neg(i < j)$, тоест, $i \geqslant j$. Но инвариантът казва, че всички диади се намират в $A[i, \ldots, j]$. Ако $i \geqslant j$, подмасивът $A[i, \ldots, j]$ има по-малко от два елемента. Тогава той не съдържа нито една диада; по дефиниция, всяка диада се състои от точно два различни елемента от масива, така че в $A[i, \ldots, j]$ "няма място" за диада. Изпълнението отива на ред 9, където алгоритъмът връща 0. Доказахме верността на теоремата в този случай: масивът не съдържа нито една диада, а алгоритъмът връща нула. ✓. □

2.1.3.5 Alg3SUMCount

За формалната дефиниция и практическото значение на задачата 3SUM вижте Подсекция 3.2.6. Тук само ще разгледаме алгоритъм за нея, наречен Alg3SUMCount, и ще докажем коректността му. За разлика от Alg2SUM, който просто индикира дали има или няма диади, Alg3SUMCount връща броя на триадите (дефиницията на "триада" е очевидната). Както ще видим, това прави доказателството за коректност значително по-трудно.

В алгоритъма Alg2SUM, числото m, което трябва да е сумата на всяка диада, е част от входа, докато в алгоритъма Alg3SUMCount сумата на триада е фиксирана на 0. Тази разлика между Alg2SUM и Alg3SUMCount е незначителна.

```
Alg3SUMCount(A[1,...,n]: сортиран масив от две по две различни цели числа)
 1  c ← 0
 2  for i ← 1 to n − 2
 3      p ← i + 1, q ← n
 4      d ← 0
 5      while p < q do
 6          if A[i] + A[p] + A[q] = 0
 7              d++
 8              p++, q--
 9          else if A[i] + A[p] + A[q] > 0
10              q--
11          else
12              p++
13      c ← c + d
14  return c
```

За удобство да дефинираме *триада* като всяка триелементна редица[†] $\langle A[x], A[y], A[z]\rangle$, такава че $1 \leqslant x < y < z \leqslant n$ и $A[x] + A[y] + A[z] = 0$.

Теорема 10: Коректността на Alg3SUMCount

За всеки входен масив $A[1,\ldots,n]$, който е сортиран и няма повтарящи се елементи, Alg3SUMCount$(A[1,\ldots,n])$ връща броя на триадите в $A[1,\ldots,n]$.

Доказателство: Alg3SUMCount е първият итеративен алгоритъм с вложеност на цикли измежду алгоритмите, които верифицираме. Наблюдателният читател сигурно е забелязал, че може да се каже, че Alg3SUMCount за всяко i пуска вариант на Alg2SUM, който присвоява на променливата d броя на диадите в $A[i+1,\ldots,n]$, сумиращи се до $-A[i]$, и променливата c акумулира тези бройки. Но верификацията на Alg3SUMCount ще бъде направена формално и от първи принципи – все едно, че не знаем за Alg2SUM.

Доказателството ще бъде съобразено с факта, че алгоритъмът има **for**-цикъл (редове 2–13), който на свой ред съдържа **while**-цикъл (редове 5–12). Инвариантът, от който следва директно Теорема 10, е за външния цикъл. Това е смислено: разгледан на най-високо ниво, алгоритъмът се състои от инициализация $c \leftarrow 0$, следвана от **for**-цикъла, следван от връщане на c. Ако успеем да докажем смислен и полезен предикат с променлива i за всяко достигане на ред 2, от този предикат ще получим верността на теоремата, замествайки i с $n+1$. Този предикат няма нужда да казва каквото и да е за **while**-цикъла! Достатъчно е предикатът да казва нещо за броя на триадите и елемента i. Когато доказваме този предикат, ние трябва да отчетем ефекта от изпълнението на вътрешния цикъл, но това касае само **доказателството** на предиката, а не неговата **формулировка**. Ето една възможност за въпросния инвариант.

Инвариант 1: Верификацията на Alg3SUMCount

Всеки път, когато изпълнението е на ред 2, променливата c съдържа броя на триадите, чийто минимален елемент се намира в подмасива $A[1,\ldots,i-1]$.

Това е напълно смислен предикат: тъй като всяка триада се състои от различни числа, тя има точно един минимален елемент и множеството от триадите може да бъде разбито по

[†]По-удобно за доказателството е триадите да са подредени обекти, тоест редици, а не множества.

позицията на минималния елемент. Предикатът не казва нищо за вътрешния цикъл – това не е необходимо засега. Ще се замислим за вътрешния цикъл, когато се наложи.

База: При първото достигане на ред 2, `c` съдържа 0, а `i` съдържа 1. Инвариантът става "0 е броят на триадите, чийто минимален елемент се намира в подмасива $A[1,\ldots,1-1]$". Но подмасивът $A[1,\ldots,1-1]$ е празният масив $A[1,\ldots,0]$, така че всъщност инвариантът става "Броят на триадите, чийто минимален елемент се намира в празния масив, е нула". Това вярно ли е? Да, разбира се, понеже множеството от триадите, чиито минимален елемент е в празния масив, е празното множество, а мощността на празното множество е нула. ✓

Поддръжка: Да допуснем, че твърдението е вярно за някое достигане на ред 2, което не е последното. Ще докажем, че инвариантът се запазва след изпълнението на тялото на **for**-цикъла. На ред 3, `p` се инициализира с $\mathtt{i}+1$, а `q` се инициализира с `n`. На ред 4, `d` се инициализира с 0. Следва изпълнението на **while**-цикъла.

Предишните итеративни алгоритми, които верифицирахме, имаха единствен цикъл. При някои, например FIND SUM на стр. 38, преминаването през тялото на цикъла е "линейно" в смисъл, че се първо изпълнява една точно определена инструкция, след това друга точно определена инструкция, и така нататък, докато тялото бъде изпълнено. При други като MAX UNSORTED, ITER на стр. 42 преминаването е по-сложно, защото тялото съдържа **if**, така че може да се изпълни една последователност или друга последователност от инструкции в зависимост от верността на булевото условие на **if**-а. Това прави доказателството по-дълго, но то остава с **фиксирана дължина**. Докато при сегашния алгоритъм ALG3SUMCOUNT ние не можем да направим доказателство (за поддръжката на инварианта на **for**-а) с фиксирана дължина, ако се опитаме да "разгънем" изпълнението на **while**-а, защото **не знаем точно колко пъти ще се изпълни while-а**. Така че се налага да осмислим работата на **while**-а, да разберем какъв ефект има изпълнението му—става дума не за едно "завъртане" на **while**-а, а за цялото му изпълнение в рамките на едно изпълнение на **for**-а—после да формулираме нов инвариант, който касае **while**-а, и да докажем този нов инвариант. Този нов инвариант е напълно различен от инварианта за **for**-цикъла! Когато го докажем, във фазата **Терминация** от неговото доказателство ще получим твърдение, което описва ефекта от работата на **while**-а. Разполагайки с това твърдение, можем да се върнем към фазата **Поддръжка** от доказателството на инварианта за **for**-цикъла.

Ето едно помощно твърдение за ефекта от работата на вътрешния цикъл. То е по отношение на **едно единствено** изпълнение на външния цикъл. Това е много важно! В крайна сметка, ние искаме да докажем фазата **Поддръжка** за работата на външния цикъл, а за тази цел има смисъл да разгледаме всички изпълнения на вътрешния цикъл от **едно единствено** изпълнение на външния цикъл.

Лема 3: За ефекта от изпълнението на вътр. цикъл на ALG3SUMCOUNT

Когато изпълнението достигне ред 13, променливата `d` съдържа броя на триадите, чиито минимален елемент е $A[\mathtt{i}]$.

Доказателство: Ще докажем Лема 3 чрез следния инвариант.

Инвариант

В рамките на едно изпълнение на **for**-цикъла (редове 2–13), всеки път, когато изпълнението е на ред 5:

① за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, p-1\}$ тстк $k \in \{q+1, \ldots, n\}$.

② d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, за които е вярно, че $j \in \{i+1, \ldots, p-1\}$ и $k \in \{q+1, \ldots, n\}$.

Доказателство: Доказателството ползва следните очевидни твърдения. Нека $A[1, \ldots, n]$ е сортиран, без повтарящи се елементи, и нека $1 \leqslant i < j < k \leqslant n$.

Факт1 Ако $A[i] + A[j] + A[k] \leqslant 0$, то $A[i] + A[j] + A[k'] < 0$ за всяко $k' < k$.

Факт2 Ако $A[i] + A[j] + A[k] \geqslant 0$, то $A[i] + A[j'] + A[k] > 0$ за всяко $j' > j$.

Факт3 Ако $\langle A[i], A[j], A[k]\rangle$ е триада, то $\langle A[i], A[j], A[k']\rangle$ не е триада за никое $k' < k$ и $\langle A[i], A[j'], A[k]\rangle$ не е триада за никое $j' > j$.

Факт3 е очевидно следствие на **Факт1** и **Факт2**.

База: $p = i + 1$ и $q = n$ заради присвояванията на ред 3. Тогава множествата $\{i+1, \ldots, p-1\}$ и $\{q+1, \ldots, n\}$ са съответно $\{i+1, \ldots, i\}$ и $\{n+1, \ldots, n\}$, така че и двете са празни. Тогава ① е вярно в празния смисъл.

Да видим ②. От една страна, $d = 0$ заради присвояването на ред 4. От друга страна, $\{i+1, \ldots, i\}$ и $\{n+1, \ldots, n\}$ са празни, така че твърдението става "d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, където $j \in \varnothing$ и $k \in \varnothing$." Но има точно нула триади, в които средният елемент е от празното множество (алтернативно, в които максималният елемент е от празното множество). Виждаме, че и ② е изпълнено. ✓

Поддръжка: Нека ① и ② са в сила за някое достигане на ред 5, което не е последното. Да кажем, че в това е моментът t от дискретното време на алгоритъма. Следните три възможности са взаимно изключващи се и са изчерпателни:

- $\langle A[i], A[p], A[q]\rangle$ е триада. Първо да се убедим, че ① се запазва. От **Факт3** знаем, че $\langle A[i], A[p], A[q]\rangle$ е единствената триада, съдържаща $A[i]$ и $A[p]$, и е единствената триада, съдържаща $A[i]$ и $A[q]$. От това и от индуктивното предположение следва, че за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, p\}$ тстк $k \in \{q, \ldots, n\}$. След инкрементирането на p и декрементирането на q на ред 8, по отношение на новите стойности на p и q, е вярно, че за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, p-1\}$ тстк $k \in \{q-1, \ldots, n\}$. Наистина, ① се запазва. ✓

  Сега да се убедим, че ② се запазва. Пак разглеждаме момента t. Щом $\langle A[i], A[p], A[q]\rangle$ е единствената триада, съдържаща $A[i]$ и $A[p]$, и е единствената триада, съдържаща $A[i]$ и $A[q]$, то броят на триадите $\langle A[i], A[j], A[k]\rangle$, където $j \in \{i+1, \ldots, p\}$ и $k \in \{q, \ldots, n\}$, е с единица по-голям от броя на триадите $\langle A[i], A[j], A[k]\rangle$, където $j \in \{i+1, \ldots, p-1\}$ и $k \in \{q+1, \ldots, n\}$. От ② знаем, че d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, където $j \in \{i+1, \ldots, p-1\}$ и $k \in \{q+1, \ldots, n\}$. Тогава броят на триадите $\langle A[i], A[j], A[k]\rangle$, където $j \in \{i+1$

$, \ldots, p\}$ и $k \in \{q, \ldots, n\}$, е $d+1$. След инкрементирането на d на ред 7 е вярно, че d е броят на триадите $\langle A[i], A[j], A[k]\rangle$, където $j \in \{i+1, \ldots, p\}$ и $k \in \{q$ $, \ldots, n\}$, е $d+1$. След инкрементирането на p и декрементирането на q на ред 8, по отношение на новите стойности на p и q, е вярно, че d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, където $j \in \{i+1, \ldots, p-1\}$ и $k \in \{q+1$ $, \ldots, n\}$. Виждаме, че ② се запазва. ✓

- $A[i] + A[p] + A[q] > 0$. Да се убедим, че ① се запазва. От **Факт2** знаем, че $A[i]$ и $A[q]$ не образуват триада нито с $A[p+1]$, нито с $A[p+2]$, и така нататък, нито с $A[q-1]$. Заедно с индуктивното предположение ①, това влече, че за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, p-1\}$ тстк $k \in \{q, \ldots, n\}$. От друга страна, условието на ред 9 е истина и на ред 10, q бива декрементирано. Спрямо новата стойност на q е вярно, че за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, p-1\}$ тстк $k \in \{q+1, \ldots, n\}$. Виждаме, че ① се запазва. ✓

  Да се убедим, че ② се запазва. Пак разглеждаме момента t. Припомняме си, че $A[i]$ и $A[q]$ не образуват триада нито с $A[p+1]$, нито с $A[p+2]$, и така нататък, нито с $A[q-1]$. Заедно с индуктивното предположение ②, това влече, че d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, за които е вярно, че $j \in \{i+1$ $, \ldots, p-1\}$ и $k \in \{q, \ldots, n\}$. На ред 10, q бива декрементирано. Спрямо новата стойност на q е вярно, че d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, за които е вярно, че $j \in \{i+1, \ldots, p-1\}$ и $k \in \{q+1, \ldots, n\}$. Виждаме, че ② се запазва. ✓

- $A[i] + A[p] + A[q] < 0$. Да се убедим, че ① се запазва. От **Факт1** знаем, че $A[i]$ и $A[p]$ не образуват триада нито с $A[q-1]$, нито с $A[q-2]$, и така нататък, нито с $A[p+1]$. Заедно с индуктивното предположение ①, това влече, че за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, p\}$ тстк $k \in \{q+1, \ldots, n\}$. От друга страна, условието на ред 9 е лъжа и на ред 12, p бива инкрементирано. Спрямо новата стойност на p е вярно, че за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, p-1\}$ тстк $k \in \{q+1, \ldots, n\}$. Виждаме, че ① се запазва. ✓

  Да се убедим, че ② се запазва. Пак разглеждаме момента t. Припомняме си, че $A[i]$ и $A[p]$ не образуват триада нито с $A[q-1]$, нито с $A[q-2]$, и така нататък, нито с $A[p+1]$. Заедно с индуктивното предположение ②, това влече, че d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, за които е вярно, че $j \in \{i+1$ $, \ldots, p\}$ и $k \in \{q+1, \ldots, n\}$. На ред 12, p бива инкрементирано. Спрямо новата стойност на p е вярно, че d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, за които е вярно, че $j \in \{i+1, \ldots, p-1\}$ и $k \in \{q+1, \ldots, n\}$. Виждаме, че ② се запазва. ✓

Терминация: Изпълнението е на ред 5 за последен път. Нека $\tilde{p}$ и $\tilde{q}$ са стойностите съответно на p и q в този момент. Щом е на ред 5 за последен път, вярно е, че $\neg(\tilde{p} < \tilde{q})$, което е същото като $\tilde{p} \geqslant \tilde{q}$. Сега ще се убедим, че има точно две възможности за големините на $\tilde{p}$ и $\tilde{q}$:

Възможност1: $\tilde{q} = \tilde{p}$.

Възможност2: $\tilde{q} = \tilde{p} - 1$.

Нека $\hat{p}$ и $\hat{q}$ са съответно стойностите на променливите p и q при предното достигане на ред 5. Сегашните $\tilde{p}$ и $\tilde{q}$ са получени от $\hat{p}$ и $\hat{q}$ по точно един от следните три начина.

- Последното изпълнение на тялото на **while**-цикъла е достигнало ред 8. Тогава $\tilde{p} = \hat{p} + 1$ и $\tilde{q} = \hat{q} - 1$. Това означава, че при последното минаване през тялото на **while**-цикъла е била открита триада $\langle A[i], A[\hat{p}], A[\hat{q}]\rangle$ и двата индекса p и q са "мръднали": p нагоре и q надолу. Следните два случая са изчерпателни.
  - $\hat{p} = \hat{q} - 1$. Тогава $\tilde{q} = \tilde{p} - 1$. Това е **Възможност2**.
  - $\hat{p} = \hat{q} - 2$. Тогава $\tilde{q} = \tilde{p}$. Това е **Възможност1**.
- Последното изпълнение на тялото на **while**-цикъла е достигнало ред 10. Тогава $\tilde{p} = \hat{p}$ и $\tilde{q} = \hat{q} - 1$. Това означава, че при последното минаване през тялото на **while**-цикъла не е била открита триада и само q е "мръднал", а p е "останал на място". Тогава $\tilde{q} = \tilde{p}$. Това е **Възможност1**.
- Последното изпълнение на тялото на **while**-цикъла е достигнало ред 12. Тогава $\tilde{p} = \hat{p} + 1$ и $\tilde{q} = \hat{q}$. Това означава, че при последното минаване през тялото на **while**-цикъла не е била открита триада и само p е "мръднал", а q е "останал на място". Тогава $\tilde{q} = \tilde{p}$. Това е **Възможност1**.

Да разгледаме **Възможност1**. За по-голяма яснота, нека стойността $\tilde{p} = \tilde{q}$ бъде наречена r. Да заместим и p, и q в инварианта с r. Получаваме

① за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, r-1\}$ тстк $k \in \{r+1, \ldots, n\}$.

② d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, за които е вярно, че $j \in \{i+1, \ldots, r-1\}$ и $k \in \{r+1, \ldots, n\}$.

Потенциален проблем за докателството е това, че множествата $\{i+1, \ldots, r-1\}$ и $\{r+1, \ldots, n\}$ не са разбиване на $\{i+1, \ldots, n\}$, защото $A[r]$ остава извън тях. А в крайна сметка ние искаме да покажем, че променливата d съдържа броя на **всички** триади с минимален елемент $A[i]$ (Лема 3). Сега ще се убедим, че проблем няма, понеже не може да има триади с минимален елемент $A[i]$, съдържащи $A[r]$. Да допуснем противното: поне една триада с минимален елемент $A[i]$ съдържа $A[r]$. Следните две възможности са изчерпателни.

- $A[r]$ е максималният елемент на тази триада. Тогава тя е от вида $\langle A[i], A[y], A[r]\rangle$, като $y \in \{i+1, \ldots, r-1\}$. Но това директно противоречи на ①, което казва "за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, r-1\}$ тстк $k \in \{r+1, \ldots, n\}$": щом средният елемент $A[y]$ е от $A[i+1, \ldots, r-1]$, няма как максималният елемент да не е от $A[r+1, \ldots, n]$.
- $A[r]$ е средният елемент на тази триада. Тогава тя е от вида $\langle A[i], A[r], A[z]\rangle$, като $z \in \{r+1, \ldots, n\}$. Но това директно противоречи на ①, което казва "за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, r-1\}$ тстк $k \in \{r+1, \ldots, n\}$": щом максималният елемент $A[z]$ е от $A[r+1, \ldots, n]$, няма как средният елемент да не е от $A[i+1, \ldots, r]$.

И така, $A[r]$ не може да участва в нито една триада с минимален елемент $A[i]$. Тогава ① и ② казват, че d съдържа точно броя на триадите с минимален елемент $A[i]$.

Да разгледаме **Възможност2**. За по-голяма яснота, нека стойността $\tilde{p}$ бъде наречена r. Тогава $\tilde{q}$ е $r-1$. Да заместим p с r и q с $r-1$ в инварианта. Получаваме

① за всяка триада $\langle A[i], A[j], A[k]\rangle$ е вярно, че $j \in \{i+1, \ldots, r-1\}$ тстк $k \in \{r, \ldots, n\}$.

② d съдържа броя на триадите $\langle A[i], A[j], A[k]\rangle$, за които е вярно, че $j \in \{i+1, \ldots, r-1\}$ и $k \in \{r, \ldots, n\}$.

Тъй като множествата $\{i+1, \ldots, r-1\}$ и $\{r, \ldots, n\}$ са разбиване на $\{i+1, \ldots, n\}$, инвариантът казва, че d съдържа точно броя на триадите с минимален елемент $A[i]$. ✓

С това доказателството на Лема 3 приключва. □

Връщаме се към фазата **Поддръжка** от доказателството на Инвариант 1. Съгласно Лема 3, на ред 13, d съдържа съдържа броя на триадите, чиито минимален елемент е $A[i]$. Тогава след присвояването на ред 13, c вече съдържа броя на триадите, чийто минимален елемент се намира в подмасива $A[1, \ldots, i]$. При следващото достигане на ред 2, променливата i се инкрементира. По отношение на новото i е вярно, че c съдържа броя на триадите, чийто минимален елемент се намира в подмасива $A[1, \ldots, i-1]$. Виждаме, че инвариантът се запазва. ✓

Терминация: Да разгледаме последното достигане на ред 2. Тогава $i = n-1$. Получаваме, че c съдържа броя на триадите, чийто минимален елемент се намира в $A[1, \ldots, n-2]$. Но това са всички триади! Други триади не може да има, защото всяка триада е от три различни елемента, ерго, най-малкият от тях не може да е на позиция, по-голяма от $n-2$. Тогава на ред 14 алгоритъмът Alg3SUMCount връща броя на всички триади в $A[1, \ldots, n]$. □

2.1.3.6 Една алгоритъм върху масиви

Задачата има графова интерпретация. Да си представим свързан граф G, в който има път $p = u_1, u_2, \ldots, u_k$, където $u_1, \ldots, u_k \in V(G)$, и всяко ребро от p е мост (Определение 64). Това означава, че ако изтрием от G ребрата на p, то G ще се разпадне на k свързани компоненти G_1, ..., G_k, като $u_i \in G_i$, за $1 \leqslant i \leqslant k$. Да допуснем, че $1 \leqslant i \leqslant k$ знаем ℓ_i: дължината на най-дълъг път в G_i с един край u_i. Искаме да намерим дължината на най-дълъг път q в G, единият край на който е в някое G_i, а другият край е в някое G_j, където $1 \leqslant i < j \leqslant n$. Неформално, q е път, който започва в някой G_i, после "минава по p" в смисъл, че има общ подпът с p, и после завършва в някой друг G_j. Очевидно $|q| = \ell_i + j - i + \ell_j$, защото q се състои от "слепването" на изброените три подпътя; ℓ_i и ℓ_j са дадени, а $j-i$ е броят на ребрата от p, които се намират и в q.

Ако игнорираме графовата интерпретация, задачата е, при даден масив $A[1, \ldots, n]$ да се намери максималната сума на два различни елемента и разстоянието между тях в масива, за всички двойки различни елементи. Има тривиален алгоритъм, който опитва всички двойки елементи. Тук ще направим нещо по-умно.

AlgDist($A[1, \ldots, n]$: масив от цели положителни числа, като $n \geqslant 2$)
1 Нека $B[1 \ldots n]$ е масив от цели положителни числа
2 $B[1] \leftarrow A[1]$
3 $t \leftarrow -\infty$
4 $i \leftarrow 2$

```
5   do
6       B[i] ← max(B[i − 1], A[i] − (i − 1))
7       t ← max(t, B[i − 1] + A[i] + (i − 1))
8       i++
9   while i ⩽ n
10  return t
```

Теорема 11: Коректността на алгоритъма AlgDist

AlgDist$(A[1,\ldots,n])$ връща максималната сума на два различни елемента и разстоянието между тях във входния масив, за всички двойки различни елементи.

Доказателство: Следното твърдение е инвариант за **do–while** цикъла (редове 5–9). Тъй като цикълът е с постусловие, има смисъл да "закачим" предиката за ред 9, тъй като там е управлението на цикъла.

Инвариант

При всяко достигане на ред 9:

① $B[i-1]$ съдържа $\max\{A[k]-(k-1)\mid 1\leqslant k\leqslant i-1\}$,

② t съдържа $\max\{A[p]+A[q]+(q-p)\mid 1\leqslant p<q\leqslant i-1\}$

База. При първото достигане на ред 9, i съдържа 3 заради присвояването на ред 4 и инкрементацията на ред 8, така че $B[i-1]$ е всъщност $B[2]$.

От една страна, ако четем псевдокода, $B[2]$ съдържа $\max\{A[1],A[2]-1\}$ заради присвояванията на ред 2 и ред 6; забележете, че при първото достигане на ред 6, i съдържа 2. От друга страна, по отношение на текущото i, което е 3, множеството $\{A[k]-(k-1)\mid 1\leqslant k\leqslant i-1\}$, за което говори инварианта, е $\{A[1]-0,A[2]-1\}$. Заключаваме, че ① е истина.

От една страна, ако четем псевдокода, t съдържа $A[1]+A[2]+1$ заради присвояването на ред 7; в този момент $i=2$. От друга страна, по отношение на текущото (при достигането на ред 9) $i=3$, множеството $\{A[p]+A[q]+(q-p)\mid 1\leqslant p<q\leqslant i-1\}$, за което говори инварианта, всъщност е $\{A[1]+A[2]+(2-1)\}$. Тогава ② също е истина.

Поддръжка: Нека инвариантът е истина за някое достигане на ред 9, което не е последното. Ще се убедим, че той се запазва при изпълнението на тялото на цикъла.

На ред 6, $B[i]$ получава стойност $\max\{B[i-1],A[i]-(i-1)\}$. По допускане,

$$B[i-1]=\max\{A[1],A[2]-1,A[3]-2,\ldots,A[i-1]-(i-2)\}$$

Тогава

$$B[i]=\max\{\max\{A[1],A[2]-1,A[3]-2,\ldots,A[i-1]-(i-2)\},A[i]-(i-1)\}$$

което може да запишем накратко като

$$B[i]=\max\{A[1],A[2]-1,A[3]-2,\ldots,A[i]-(i-1)\}$$

След инкрементирането на i на ред 8, изразено чрез новото i, това става

$$B[i-1]=\max\{A[1],A[2]-1,A[3]-2,\ldots,A[i-1]-(i-2)\}$$

Виждаме, че ① отново е истина.

На ред 7, t получава стойност $\max(t, B[i-1] + A[i] + (i-1))$. По допускане, старото t е

$$t_{old} = \max\{A[p] + A[q] + (q-p) \mid 1 \leqslant p < q \leqslant i-1\}$$

а $B[i-1]$ е

$$B[i-1] = \max\{A[1], A[2]-1, A[3]-2, \ldots, A[i-1]-(i-2)\}$$

Тогава $B[i-1] + A[i] + (i-1)$ е

$$\begin{aligned}&\max\{A[1], A[2]-1, A[3]-2, \ldots, A[i-1]-(i-2)\} + A[i] + (i-1) = \\ &\max\{A[1] + A[i] + (i-1), A[2]-1+A[i]+(i-1), \ldots, A[i-1]-(i-2)+A[i]+(i-1)\} = \\ &\max\{A[k] + A[i] + (i-k) \mid 1 \leqslant k \leqslant i-1\}\end{aligned}$$

И така:

- t_{old} съдържа стойността на максимална сума на два различни елемента и разстоянието между тях, по всички елементи на $A[1, \ldots, i-1]$,
- а $B[i-1] + A[i] + (i-1)$ съдържа стойността на максимална сума на два различни елемента и разстоянието между тях, по всички елементи на $A[1, \ldots, i]$, единият от които обаче е $A[i]$.

Тогава $\max(t_{old}, B[i-1] + A[i] + (i-1))$, което новото t, съдържа стойността на максимална сума на два различни елемента и разстоянието между тях, по всички елементи на $A[1, \ldots, i]$. След инкрементирането на ред 8, изразено в новото i, това става

$$t = \max\{\{A[p] + A[q] + (q-p) \mid 1 \leqslant p < q \leqslant i-1\}$$

Виждаме, че ② отново е истина.

Терминация: Разглеждаме момента, в който изпълнението е на ред 9 за последен път. Очевидно i съдържа $n+1$ в този момент. От ② получаваме, заместваики i с $n+1$, че t съдържа стойността на максимална сума на два различни елемента и разстоянието между тях, по всички елементи на $A[1, \ldots, n]$. И на ред 10 алгоритъмът връща точно това. □

2.2 Анализ на сложността

"Сложност на алгоритъм" е мярка за това, колко ресурси ползва този алгоритъм. "Сложен" в този смисъл означава "ползва много ресурси". За какви ресурси става дума?

- Ресурсът, който най-често имаме предвид, е времето. Естествено е, че искаме алгоритмите ни да работят бързо. В този смисъл, "качествен алгоритъм" е алгоритъм, който работи бързо върху всички входове. Но "бързо" е разговорен термин. Прецизните разсъждения се базират на понятието "сложност по време" (на английски е *time complexity*), което разгледаме след малко. Засега само казваме, че алгоритъм с висока сложност по време е алгоритъм, който е бавен.
- Следващият по важност ресурс е паметта, която ползва алгоритъмът. Естествено е, че искаме алгоритмите ни да ползват малко памет. В този смисъл, "качествен алгоритъм" е

алгоритъм, който ползва малко памет върху всички входове. Прецизните разсъждения се базират на понятието "сложност по памет" (на английски е *space complexity*), което разгледаме след малко. Засега само казваме, че алгоритъм с висока сложност по памет е алгоритъм, който ползва много памет.

Допълнение 11: Други ресурси освен времето и паметта

По правило учебниците по алгоритми не разглеждат други ресурси освен време и памет. Но, както е посочено в [72, стр. 156], има много други възможни ресурси, наречени там *complexity measures*, които можем да разглеждаме. От особен интерес е така наречения *ink*, което е броят на презаписвания на клетки от паметта. Този ресурс очевидно е свързан с енергията, която реален софтуер, базиран на алгоритъма, би ползвал върху реален компютър. Свързан с "мастилото", но различен ресурс, е броят на достъпите до клетките на паметта. Разликата има практическо значение дотолкова, доколкото записването в паметта е по-енергоемко от четенето от паметта. Повече информация за тези ресурси и сложността, базирана на тях, има в [43] и [40].
Има елегантна, много обща дефиниция на Blum на "complexity measure", която покрива времето, паметта и "мастилото". Интересуващият се читател може да я намери в [72, стр. 156], като оригиналната статия е [15].
При разпределено изчисление (*distributed computation*) се говори за комуникационна сложност (*communication complexity*), измерваща количеството комуникация между независими изчисляващи агенти, но това е съвсем друг изчислителен модел от този, който сме приели в тези лекции, а именно със само един изчисляващ агент.

2.2.1 Големина на входа

Големина на входа на алгоритъм, или още *размер на входа* (на английски *input size*), е ключово понятие за дефинирането на сложността на алгоритмите, защото сложността е функция на размера на входа. Прецизното определение е следното.

Определение 9: Големина на входа – определение, което няма да използваме

За произволен алгоритъм, *големина на входа* при конкретен вход е дължината на стринга, който кодира този вход в избраната система за кодиране.

Това определение е полезно за теория на сложността, която отчита и кодирането на данните. Тъй като вече приехме, че няма да разглеждаме кодирания, Определение 9 не върши работа за целите на този курс. Терминът "големина на входа" ще дефинираме непрецизно, с голяма доза условност.

Конвенция 1: Всяко число има големина на представянето единица

За целите на тези лекции приемаме, че всяко число, цяло или рационално, е записано в единица памет, независимо от стойността си.

Това не означава, че ползваме само един бит за число—това очевидно би било невъзможно—а че има константа големина на компютърна памет, предварително фискирана, такава, че всяко число се "побира" в нея, колкото и да е голямо. Ако мислим за регистрите или думите в

паметта на нашия абстрактен компютър, всеки регистър може да побере всяко число и всяка дума от паметта с произволен достъп може да побере всяко число. Да, това е математически абсурд и тривиално се опровергава с принципа на Dirichlet, но е удобно опростяване.

Определение 10: Големина на входа – непрецизно определение, което ще ползваме

Ако алгоритъмът е върху числен масив, то големината на входа е броят на числата в масива. Ако алгоритъмът е върху графи, големината на входа е сумата от броя на върховете и броя на ребрата. Това важи както за графи без тегла, така и за тегловни графи. Ако алгоритъмът е върху стрингове, големината на входа е броят на символите.

Това разбиране за големина на входа отговаря добре на модела, който вече приехме, в който числата са базови елементи, а елементарните операции върху тях стават за единица време. Практиката е показала, че това разбиране за големина на входа обикновено води до смислени резултати. Както ще видим нататък, има и изключения.

Конвенция 2: Буквата n в контекста на анализа на алгоритми

При анализа на алгоритми, по правило буквата n означава големината на входа. Изключенията са малко и ще се споменават изрично.

Сравнете Конвенция 2 с Конвенция 8 за графите.

Има едно изключение. Ако разглеждаме алгоритъм, чийто вход е квадратна матрица $n \times n$, или поне основната част от входа е такава матрица, сложността се изразява като функция на n. Всъщност броят на елементите на матрицата е n^2. Следователно, изразявайки сложността като функция на n, ние я изразяваме не чрез големината на входа, а чрез корен-квадратен от големината на входа[†]. Вижте Конвенция 7.

2.2.2 Въведение в сложността по време

Да разгледаме някакъв прост алгоритъм, например сортиращия алгоритъм Insertion Sort. Псевдокодът, който ще разгледаме, е по учебника CLR (Cormen, Leiserson, Rivest) [21].

```
Insertion Sort(A[1, 2, ... n]: масив от естествени числа)
1  for i ← 2 to n
2      key ← A[i]
3      j ← i − 1
4      while j > 0 and A[j] > key do
5          A[j + 1] ← A[j]
6          j ← j − 1
7      A[j + 1] ← key
```

Колко бързо работи той? От практическа гледна точка, този въпрос е за физическото време, което отнема работата на някаква негова програмна реализация върху истински компютър. От тази гледна точка трябва да отчитаме следните фактори:

[†]Типичен пример е традиционният алгоритъм за умножение на матрици. Ако умножаваме квадратни, $n \times n$ матрици, казваме, че сложността е $\Theta(n^3)$, но алгоритъмът не е кубичен в размера на входа! Алгоритъмът всъщност има сложност $\Theta\left(N^{\frac{3}{2}}\right)$, където N е големината на входа. За да разберете тази забележка под черта, трябва да познавате съдържанието на Подсекция 2.3.3.

- За какъв вход става дума. На свой ред, този фактор има две компоненти:
  - Колко числа има във входа. Ясно е, че колкото по-голямо е n, толкова по-бавно работи алгоритъмът.
  - При една и съща големина, какви са конкретните стойности на числата. Лесно се забелязва, че Insertion Sort ще работи по-бързо, ако входът вече е сортиран, например $[1, 2, 3, 4]$. Нататък ще видим, че този алгоритъм работи най-бавно, когато входът е сортиран обратно, например $[4, 3, 2, 1]$.
- Каква е конкретната програмна реализация. Според Skiena (вж. определението на стр. 1), алгоритъмът е идеята зад някаква програма. Но истинските програми са повече от идеите зад тях, те имат конкретна реализация на конкретен език за програмиране. Очевидно някои реализации ще са по-бързи от други, защото са по-грамотно написани. Компилаторът не може да компенсира неграмотно писане на софтуер.
- Какъв е компилаторът.
- Каква е виртуалната машина, ако има такава.
- Каква е операционната система.
- Каква е компютърната архитектура и по-точно какъв е процесорът.

Да се даде точен отговор колко бързо би работила конкретна програма—например, с точност до наносекунда—реализираща Insertion Sort, на конкретен език, върху конкретен компютър, е практически невъзможно дори за фиксиран вход[†]. Толкова прецизен отговор не е и необходим.

За да може да правим смислени изводи за бързодействието на алгоритми, правим серия от опростявания.

- Първото е, че се фокусираме именно върху алгоритмите, а не върху програмните им реализации. Разликата между бързодействието на две програми за една и съща задача, които реализират два различни алгоритъма, по правило са много по-драстични от разликите в бързодействието на две програми, реализиращи един и същи алгоритъм. **По отношение на сложните, нетривиални задачи, печелившата стратегия за бързодействието е подобрение на алгоритъма.** Печалбата от по-бързия алгоритъм е толкова по-видима, колкото по-голям е входът.
- Второто е допускането, че всички елементарни инструкции (стъпки) се изпълняват за единица време[‡], така че времето за изпълнение на алгоритъма върху някакъв вход е точно броят на елементарните инструкции, които се изпълняват по време на работата му.

Ако в този опростен модел алгоритъм A е по-бърз от алгоритъм B за една и съща задача, най-вероятно в реалния свят програмната реализация на A ще е по-бърза от тази на B, като разликата ще е толкова по-очевидна, колкото по-голям е входът.

Като пример за прилагането на тези опростяващи допускания да разгледаме задачата Сортиране. Сортиращият алгоритъм Insertion Sort, както ще видим от (2.9) на стр. 70, в

[†]Съществуват специализирани компютри, изпълняващи критични дейности в *реално време*, при които наистина са необходими твърди гаранции за бързодействието на програмите, но това е далече отвъд материала в този курс.

[‡]Инструкциите на истински процесор се изпълняват за различен брой тактове.

най-лошия случай работи във време, пропорционално на на n^2. Сортиращият алгоритъм HEAPSORT, както ще видим на стр. 181, в най-лошия случай работи във време, пропорционално на на $n \cdot \lg n$. Да разгледаме стойностите на n^2 и $n \cdot \log n$ за няколко различни n. За целите на този пример логаритъмът е с основа 10.

n	10	100	1 000	100 000	1 000 000	100 000 000
n^2	100	10 000	1 000 000	10 000 000 000	1 000 000 000 000	10 000 000 000 000 000
$n \cdot \log n$	10	200	3 000	500 000	6 000 000	800 000 000

Разбира се, числата в тази таблица не са точният брой елементарни инструкции; точният брой, както казахме, е пропорционален на тези числа. Но на практика тези коефициенти на пропорционалност не са големи. И така, при вход с размер сто милиона, огромната разлика между 10 000 000 000 000 000 и 800 000 000—**осем десетични порядъка**—води до това, че програмата, реализираща HEAPSORT, е **смазващо** по-бърза[†] от тази, която реализира INSERTION SORT, независимо от неща като използван компилатор, ниво на оптимизация на компилирането, операционна система, процесор, програмистки трикове за подобряване на скоростта и така нататък.

Току-що видяхме пример за това, че предимството, което дава по-бързият алгоритъм, доминира с много над предимството на по-бързата компютърна технология, било хардуер, било софтуер.

Съществуват много по-екстремни примери за предимствата, които дават бързите алгоритми. В Лекция 13 ще разгледаме множество интересни и важни задачи, за които наивният алгоритъм[‡] се изпълнява в най-лошия случай за време, което е пропорционално на, да кажем, 2^n, докато за същата задача има по-съвършени алгоритми, чието време за изпълнение е пропорционално на, да кажем, n^2. Разликата между нарастването на n^2 и 2^n е много по-драстична от разликата между $n \lg n$ и n^2, която вече разгледахме. За илюстрация на разликата между n^2 и 2^n, да разгледаме неголямо n, примерно $n = 200$. От една страна, $200^2 = 40\,000$, което е нищо за съвременен компютър. От друга страна, $2^{200} = 1\,606\,938\,044\,258\,990\,275\,541\,962\,092\,341\,162\,602\,522\,202\,993\,782\,792\,835\,301\,376 \approx 1.6 \times 10^{60}$ е практически безкрайност; от изложението в Допълнение 12 следва, че 2^{200} като брой инструкции, които трябва да бъдат изпълнени, е **далече отвъд възможностите на всеки истински компютър**, както в момента, така и в бъдеще. Това заслужава да бъде повторено.

Забележка

В реалния свят, програма, която трябва да изпълни 2^{200} стъпки, няма да завърши работата си **НИКОГА**, независимо от използваната хардуерна и софтуерна технология за реализация на платформата, върху която тази програма работи. Ограничението идва от фундаментални принципи на физиката и **НЕ МОЖЕ** да бъде заобиколено чрез никакви технологични иновации и подобрения.

[†] При нормално грамотна реализация на двата алгоритъма.

[‡] Наивен алгоритъм е алгоритъм, който може да бъде предложен от най-общи съображения. По правило терминът се използва в контекста на задачи, за които съществуват значително по-добри, но и по-неочевидни алгоритми.

Допълнение 12: Принцип на Landauer

Принципът на Landauer [62] казва, че има долна граница за енергията, необходима за обръщането на един бит от нула в единица или обратно, като функция на температурата, при условие, че изчислението е необратимо[a] във времето. Този резултат свързва две много различни области на познанието: термодинамиката, която е дял на физиката, и теорията на цифровите изчислителните машини. Само че става дума за реални изчислителни машини, а не за изчислителни модели като машини на Turing и така нататък. Технологично, изчислителните машини може да са конструирани по много начини: с вакуумни лампи, с дискретни транзистори, с интегрални схеми, може да работят със светлина и да са базирани на фотонни превключватели, а в миналото е имало идеи да бъдат реализирани чрез релета или дори чисто механично. Принципът на Landauer е напълно независим от технологията на изчислителната машина, което го прави универсален, а долната граница, която той налага, е незаобиколима, както са незаобиколими принципите на термодинамиката. По думите на Bennett [9]:

> In his classic paper, Rolf Landauer (1961) attempted to apply thermodynamic reasoning to digital computers. Paralleling the fruitful distinction in statistical physics between macroscopic and microscopic degrees of freedom, he noted that some of a computer's degrees of freedom are used to encode the logical state of the computation, and these information bearing degrees of freedom (IBDF) are by design sufficiently robust that, within limits, the computer's logical (i.e., digital) state evolves deterministically as a function of its initial value, regardless of small fluctuations or variations in the environment or in the computer's other non-information bearing degrees of freedom (NIBDF). While a computer as a whole (including its power supply and other parts of its environment), may be viewed as a closed system obeying reversible laws of motion (Hamiltonian or, more properly for a quantum system, unitary dynamics), Landauer noted that the logical state often evolves irreversibly, with two or more distinct logical states having a single logical successor. Therefore, because Hamiltonian/unitary dynamics conserves (fine-grained) entropy, the entropy decrease of the IBDF during a logically irreversible operation must be compensated by an equal or greater entropy increase in the NIBDF and environment. This is Landauer's principle.

Съгласно принципа на Landauer, всяко битово обръщане при необратимо във времето изчисление "иска" като енергия в джаули поне

$$E = k_B \cdot T \cdot \ln 2$$

където k_B е константата на Boltzmann, приблизително равна на 1.38×10^{-23} JK^{-1}, T е температурата на устройството в Келвинови градуси, а $\ln 2$ е, разбира се, натуралният логаритъм на двойката. По понятни причини, количеството $k_B \cdot T \cdot \ln 2$ е известно като *граница на Landauer*.

Авторът на лекционните записки признава, че познанията му в термодинамиката са нищожни, но въпреки това на най-общо ниво има нещо като разбиране за същността на принципа на Landauer. Принципът е базиран на понятието *ентропия*, която в някакъв смисъл е *мярка за неподредеността в дадена система*. Изчисление, което е необратимо във времето, както всеки процес, който е необратим във времето, води до нарастване на ентропията, според термодинамиката. Количеството $k_B \cdot \ln 2$ съответства на нараст-

ването на ентропията при обръщане на един бит, а произведението от нарастването на ентропията и температурата има смисъл на енергия.
На пръв поглед, ако температурата е нула, границата на Landauer също става нула. Но на практика това няма как да стане. Температурата не може да бъде нула. В нашата Вселена има *реликтово лъчение*, остатък от Големия взрив, изотропно електромагнитно лъчение, пронизващо пространството. Дори само заради реликтовото лъчение, температурата на тяло[б] никога няма да бъде нула, ако ще тялото да е далече от Земята, извън Слънчевата система или дори извън Млечния път.
Ако вземем нереалистично ниската стойност от един келвинов градус за температура на нашата изчисляваща машина, имаме долна граница от около 10^{-23} джаула енергия за едно битово обръщане. За 10^{60} битови обръщания тази долна граница става около 10^{37} джаула. Това е приблизително равно на енергията, която Слънцето излъчва в продължение на *хиляда години*. Не енергията от Слънцето, която попада върху Земята, а **цялата енергия**, която нашата звезда произвежда. И тъй като $10^{60} \approx 2^{200}$, то за 2^{200} битови обръщания се иска енергия **поне** колкото една звезда генерира за хиляда години. Ако увеличим малко степенния показател, да кажем 2^{250}, магнитуда на енергията, която се иска за толкова битови обръщания нараства до около 10^{52} джаула. Това е много повече от 10^{45} джаула, енергията, *която се отделя при избухване на супернова*, и е от порядъка на енергията, *отделяна при експлозии на галактични ядра*. На читателя остава да сметне за коя стойност на $\mathtt{n}$ е вярно, че $2^{\mathtt{n}}$ битови обръщания при един келвинов градус на компютъра искат енергия поне 10^{69} джаула съгласно принципа на Landauer, където 10^{69} джаула е *тоталната маса-енергия в наблюдаемата Вселена*.

Има доста автори, които са критични към принципа на Landauer, както посочва Bennett [9]. Някои автори отричат връзката между физически величини като топлина абстрактно-математически свойства като обратимост на логически операции. Други автори посочват, че принципът на Landauer е в сила само при изчисления, които са необратими във времето, а принципно е възможно да се реализират изчисления, които са обратими. Въпреки че дебатите продължават, преобладаващото мнение е, че принципът на Landauer е валиден, а освен това има и негови експериментални потвърждения [46].

По правило учебниците по алгоритми още в уводната глава показват нагледно зашеметяващото нарастване на функции като $2^{\mathtt{n}}$ и $\mathtt{n}!$ и показват убедително, че дори ако даден реален компютър изпълнява много повече елементарни операции в секунда от най-добрите днешни образци, той е безнадеждно бавен за програми, искащи от порядъка на $2^{\mathtt{n}}$ или $\mathtt{n}!$ стъпки, дори за скромни стойности на $\mathtt{n}$. Като пример за това, вижте Problem 1-1 в [21, стр. 14]. По същество, този тип аргумент казва “Няма да ви стигне времето!”. Аргументацията с принципа на Landauer казва друго: “Няма да ви стигне енергията!”.
Добре известният *закон на Moore* казва, че гъстота на гейтовете в интегралните схеми се удвоява на горе-долу 18 месеца, което се интерпретира и като “производителността на компютрите се удвоява на 18 месеца”. Дори хора, които не са виждали това твърдение формулирано в явен вид, мислят, че технологичният прогрес увеличава производителността с някаква мултипликативна константа на единица време[в] и това винаги ще е така. Ако това беше неизменно вярно, то в недалечно бъдеще щеше да има компютри, способни да извършат и 2^{200} стъпки, и 2^{2000} стъпки. Но принципът на Landauer казва, че последното е невъзможно, от което следва, че експоненциалното нарастване на производителността ще спре с достигането на ограничения, произтичащи от фун-

даменталните закони на физиката.

Читателят, интересуващ от физическите ограничения върху компютърната производителност, може да намери статията на Lloyd [64] за особено информативна, защото тя разглежда най-различни ограничения върху производителността, а не само това, произтичащо от принципа на Landauer. Примерно, там се твърди, че 10^{50} елементарни операции в секунда е горна граница, произтичаща от закона на Einstein $\mathsf{E} = \mathfrak{mc}^2$.

[а] На английски терминът за "необратимо във времето изчисление" е *time-irreversible computation*. Това означава изчисление, при което не може да определим входните стойности от резултата. Примерно, елемент тип конюнкция, на английски *AND-gate*: ако резултатът е 0, не може да кажем дали входът е бил $(0,0)$ или $(0,1)$ или $(1,0)$.

[б] Става дума за тяло, "оставено на мира". В някои лабораторни експерименти тела придобиват температура, много близка до нула келвинови градуса, но за това "изпомпване на топлина" се иска енергия, и то много.

[в] Това е същото като експоненциален растеж, защото сумата на геометрична прогресия е експоненциална функция.

2.2.3 Определение на сложността по време

Сложността по време е функция на големината на входа. Но има много входове с една и съща големина. По-лошо, в нашия опростен модел, в който всяко число има един и същи размер (единица), има безброй много входове за всяка големина на входа. Да разгледаме отново Сортиране. Големината на входа $\mathfrak{n}$ може да е всяко естествено число. Множеството от входовете с големина 1 е $\mathbb{N}$. Множеството от входовете с големина 2 е $\mathbb{N} \times \mathbb{N}$. Множеството от входовете с големина 3 е $\mathbb{N} \times \mathbb{N} \times \mathbb{N}$. И така нататък. В общия случай, множеството от входовете с големина $\mathfrak{n}$ е $\mathbb{N}^{\mathfrak{n}}$. Виждаме, че множеството от входовете е безкрайно (и по-точно, изброимо безкрайно) за всяко $\mathfrak{n}$.

Следното съображение ни позволява да разглеждаме само краен брой входове за всяка големина на входа. Съображението е по отношение на Сортиране, но лесно може да бъде пренесено и при други изчислителни задачи[†]. Разглеждаме само такива сортирания, които се базират на **директни сравнения** на числата (които биват сортирани)[‡]. Лесно се вижда, че в такъв случай сортирането на следните входове

$$(2,1,3)$$
$$(2000,1000,3000)$$
$$(2 \times 10^9, 1, 10^{10^{10}})$$

ще протече по един и същи начин, и то в много силен смисъл. Всеки сортиращ алгоритъм, базиран на директни сравнения, ще завърши работата си в същия брой стъпки за всеки от тези три входа, като на всяка стъпка ще изпълнява една и съща инструкция. Разбира се, това е при не много реалистичното допускане, че $10^{10^{10}}$ има размер единица и действията върху него стават за единица време.

[†] Всъщност е доста трудно да кажем колко са съществено различните входове за, да речем, алгоритъма на Крускал. Тук се има предвид следното – ясно е, че за всеки алгоритъм, който ще разглеждаме, безкрайното множество от входове с големина $\mathfrak{n}$ може да бъде разбито на краен брой класове на еквивалентност, като върху входовете от даден клас алгоритъмът ще работи еднакво бързо.

[‡] Съществуват и други възможности за сортиране, ако са известни някакви ограничения за входа. Например, ако сортираме само нули и единици, може просто да преброим колко са нулите.

Сега ще дефинираме *неразличими по отношение на бързодействието входове*, за произволен алгоритъм за СОРТИРАНЕ, базиран на директни сравнения. За всяка големина на входа n, тоест за всяко $n \in \mathbb{N}^+$, въвеждаме *релация на неразличимост* R_n така: за всеки два входа X и Y с големина n, $(X, Y) \in R_n$ тогава и само тогава, когато:

1. X и Y имат един и същи брой от малкия елемент, един и същи брой от следващият по големина, и така нататък, имат един и същи брой от най-големия си елемент[†]. Нека m е броят на различните големини на елементи в X и Y, където $1 \leqslant m \leqslant n$.

2. За $1 \leqslant k \leqslant m$, всички k-и по големина елементи се намират на едни и същи позиции в X и в Y.

Например, нека $n = 5$ и нека има два най-малки елемента, два по-големи и един най-голям. Тогава $m = 3$. Ако $X_1 = (6, 55, 6, 55, 7075)$ и $Y_1 = (36, 37, 36, 37, 38)$, то $(X_1, Y_1) \in R_5$, понеже двата най-малки елемента са на позиция 1 и 3 както в X_1, така и в Y_1, средните по големина два елемента са на позиции 2 и 4 както в X_1, така и в Y_1, и най-големият елемент е на позиция 5 както в X_1, така и в Y_1. От друга страна, ако $X_2 = (55, 6, 6, 55, 7075)$, то $(X_2, Y_1) \notin R_5$, защото в X_2 най-малките елементи са на позиции 2 и 3.

Тривиално се показва, че за всяко n, R_n е релация на еквиваленост. Нейните класове на еквивалентост са само краен брой за всяко n. Интересен въпрос е колко точно са тези класове на еквивалентност. Ако не допускаме повтаряне на елементи, то класовете на еквивалентост са $n!$, защото всеки клас се определя от това на коя позиция е първият елемент, на коя е вторият, и така нататък до n-ия, така че просто броим пермутациите. Ако обаче допускаме повтаряне на елементи, броенето на класовете е по-сложно, затова го отделяме от основния текст в Допълнение 13.

Допълнение 13: Колко са класовете на екв. на входа при сортиране

В по-общия случай, когато допускаме повтаряне на елементи, нека броят на уникалните елементи е m, където $1 \leqslant m \leqslant n$. Броят на начините числата a_1, a_2, ..., a_n да бъдат групирани в точно m групи, във всяка от които числата са равни, а числата от две различни групи са различни, е $\left\{ {n \atop m} \right\}$. Да си припомним, че $\left\{ {n \atop m} \right\}$ за $1 \leqslant m \leqslant n$ е броят на разбиванията на n-елементно множество на точно m подмножества; чете се "числото на Stirling от втори род n-подмножество-m".
Например, $\left\{ {4 \atop 2} \right\} = 7$, и действително има точно 7 начина a_1, a_2, a_3 и a_4 да имат точно две различни големини:

a_1, a_2 и a_3 с една и съща големина, a_4 с друга големина
a_1, a_2 и a_4 с една и съща големина, a_3 с друга големина
a_1, a_3 и a_4 с една и съща големина, a_2 с друга големина
a_2, a_3 и a_3 с една и съща големина, a_1 с друга големина
a_1 и a_2 с една и съща големина, a_3 и a_4 с друга големина
a_1 и a_3 с една и съща големина, a_2 и a_4 с друга големина
a_1 и a_4 с една и съща големина, a_2 и a_3 с друга големина

Броят на класовете на класовете на еквивалентност на R за дадено m е $m!\left\{ {n \atop m} \right\}$. Напри-

[†]Тук разглеждаме общия случай, в който във входа може да има повтарящи се елементи. Ако елементите на входа са уникални, то има точно един най-малък елемент, точно един по-голям, и така нататък, точно един най-голям елемент.

мер, при $n = 4$ и $m = 2$, има точно 14 начина за подредбата на различните големини на числата:

a_1, a_2 и a_3 с една и съща големина, a_4 с друга големина, и $a_1, a_2, a_3 < a_4$
a_1, a_2 и a_3 с една и съща големина, a_4 с друга големина, и $a_1, a_2, a_3 > a_4$
a_1, a_2 и a_4 с една и съща големина, a_3 с друга големина, и $a_1, a_2, a_4 < a_3$
a_1, a_2 и a_4 с една и съща големина, a_3 с друга големина, и $a_1, a_2, a_4 > a_3$
a_1, a_3 и a_4 с една и съща големина, a_2 с друга големина, и $a_1, a_3, a_4 < a_2$
a_1, a_3 и a_4 с една и съща големина, a_2 с друга големина, и $a_1, a_3, a_4 > a_2$
a_2, a_3 и a_3 с една и съща големина, a_1 с друга големина, и $a_2, a_3, a_4 < a_1$
a_2, a_3 и a_3 с една и съща големина, a_1 с друга големина, и $a_2, a_3, a_4 > a_1$
a_1 и a_2 с една и съща големина, a_3 и a_4 с друга големина, и $a_1, a_2 < a_3, a_4$
a_1 и a_2 с една и съща големина, a_3 и a_4 с друга големина, и $a_1, a_2 > a_3, a_4$
a_1 и a_3 с една и съща големина, a_2 и a_4 с друга големина, и $a_1, a_3 < a_2, a_4$
a_1 и a_3 с една и съща големина, a_2 и a_4 с друга големина, и $a_1, a_3 > a_2, a_4$
a_1 и a_4 с една и съща големина, a_2 и a_3 с друга големина, и $a_1, a_4 < a_2, a_3$
a_1 и a_4 с една и съща големина, a_2 и a_3 с друга големина, и $a_1, a_4 > a_2, a_3$

Броят на класовете на еквивалентост на R_n за дадено n е

$$\sum_{m=1}^{n} m! \left\{ {n \atop m} \right\}$$

Да резюмираме. В нашия модел, в който всички числа са с големина единица и елементарните операции върху всяко число стават за единица време, по отношение на произволен сортиращ алгоритъм A, базиран на директни сравнения, множеството от входове $\mathbb{N}^n$ се разбива на $\sum_{m=1}^{n} m! \{ {n \atop m} \}$ класа, като за всеки два входа X и Y от един и същи клас, $A(X)$ и $A(Y)$ работят в един и същи брой стъпки, като във всеки момент и двата изпълняват една и също инструкция. От гледна точка на работата на алгоритъма, X и Y са **напълно неразличими**.

За други изчислителни задачи разбиването на (изброимо) безкрайното множество от входовете с дадена големина на краен брой класове на еквивалентост става може да е по друг начин, но за всяка задача можем да дефинираме смислено такова разбиване, като за всеки два входа X и Y от един и същи клас, $A(X)$ и $A(Y)$ извършва в един и същи брой стъпки за всеки алгоритъм A за нея. Това ни позволява, когато разглеждаме сложността по време, да смятаме, че входовете от дадена големина са само краен брой.

Определение 11: Сложност по време

Нека Π е изчислителна задача и A е алгоритъм за нея. За всяка големина на входа $n \in \mathbb{N}^+$, нека $\mathcal{I}(n)$ е крайното множество от входовете с големина n и за всеки вход κ нека $f(\kappa)$ е броят стъпки, които се изпълняват от $A(\kappa)$. Тогава, за всяко n, *сложността по време на* A *в най-лошия случай* е

$$T_A(n) = \max\{f(\kappa) \mid \kappa \in \mathcal{I}(n)\}$$

сложността по време на A *в най-добрия случай* е

$$P_A(n) = \min\{f(\kappa) \mid \kappa \in \mathcal{I}(n)\}$$

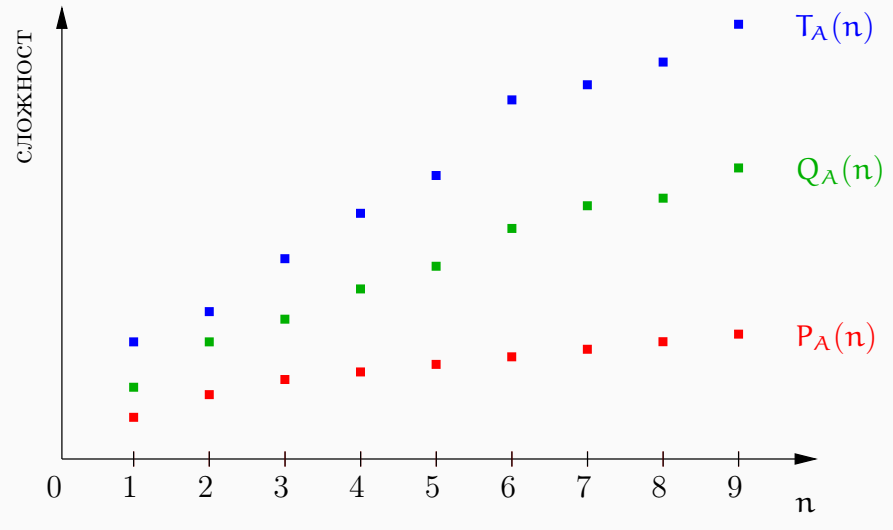
и *средната сложност по време на* A е

$$Q_A(n) = \frac{1}{|\mathcal{I}(n)|} \sum_{\kappa \in \mathcal{I}(n)} f(\kappa)$$

Забележка: използваната нотацията, например "$T_A(n)$" и т. н., не е общоприета.

Фигура 2.1 показва нагледно трите вида сложност.

Фигура 2.1 : Сложност по време в най-лошия случай $T_A(n)$, в най-добрия случай $P_A(n)$ и средна сложност $Q_A(n)$. Ординатата е сложността като брой стъпки.



Средната сложност винаги е между сложността в най-лошия и сложността в най-добрия случай. По правило функциите на сложността са строго нарастващи, защото (по правило) един и същи алгоритъм работи по-бавно върху по-голям вход, но има и изключения, както ще видим надолу в примера с Евклидовия алгоритъм.

На практика най-често използвана е сложността по време в най-лошия случай (*worst-case time complexity* на английски), по две основни причини. Първо, изследването на сложността в най-лошия случай е много по-лесно от изследването на средната сложност, в което ще се убедим в следваща лекция при анализа на QUICKSORT. Анализирането на средната сложност изисква значително по-задълбочени математически познания и значително по-сложни

техники, дори при неявното допускане в определението на $Q_A(n)$, че всички входове с дадена големина са еднакво вероятни. Втората причина е, че резултатът за най-лошия случай е твърда гаранция, че по-лошо не може да бъде.

Сложността в най-добрия случай не се ползва, тъй като практически всеки важен алгоритъм можа да бъде подобрен като скорост за само един вход (по един такъв вход за всяка големина). Например, всеки алгоритъм A за задачата Хамилтонов Път, която ще дефинираме нататък, може да бъде направен много бърз върху един конкретен вход (за всяка големина). По-точно, независимо как точно работи A, той може да бъде модифициран, като в самото му начало се добави тестване на една конкретна пермутация на върховете – дали задава хамилтонов път. Ако се окаже, че тя задава хамилтонов път, то отговорът на въпроса дали има хамилтонов път очевидно е утвърдителен и прекратяваме изпълнението. Ако се окаже, че тя не задава хамилтонов път, от това не следва, че няма такъв, и продължаваме с това, което A прави. С други думи, към всеки алоритъм, дори най-бавния, може да се пришие напълно изкуствено тестване на някакъв конкретен вход. Този трик би подобрил изкуствено сложността в най-добрия случай до теоретично оптималната, без това да ни казва нищо съществено за това, колко е бърз A. В примера с Хамилтонов Път, според преобладаващото мнение, всеки алгоритъм за тази задача е с експоненциална сложност както в най-лошия, така и в средния случай, но чрез споменатия трик може да направим кой да е алгоритъм за тази задача да работи в линейно време в най-добрия случай. Такова изкуствено увеличаване на бързодействието чрез подобрение върху само един вход е напълно безполезно.

2.2.4 Анализ на сложността по памет

Сложността по памет на даден алгоритъм A върху даден вход е броят на елементите памет, които A ползва, без да броим паметта, в която се разполага входа и паметта, в която се разполага изходът. С други думи, гледаме само **работната памет** на алгоритъма; това е съвкупността от променливите му, които се ползват, за да се получи изходът от входа. Сложност по памет в най-лошия, средния и най-добрия случай се дефинира по начин, аналогичен на сложността по време.

Една значителна принципна разлика между сложността по време и сложността по памет е, че сложността по време (било в най-лошия случай, било средната) най-често е строго растяща функция на големината на входа, докато по отношение на сложността по памет е напълно възможно даден алгоритъм да ползва само константна работна памет, за всяка големина на входа. Алгоритми, които ползват константна работна памет, се наричат *in-place*.

2.2.5 Проблеми със сложността по време, до които води нашият модел

Да разгледаме отново Евклидовия алгоритъм в модерната му формулировка от Подсекция 1.2.2, независимо дали в рекурсивния или итеративния вариант. Да се опитаме да направим груб анализ на сложността му по време в най-лошия случай. Очевидно, за някои двойки входни числа, а именно тези, за които a е кратно на b, алгоритъмът ще завършва много бързо. За други двойки входни числа алгоритъмът ще работи много повече. От [55] става ясно, че за за всяко цяло положително число m, колкото и голямо да е, има двойка (a, b), такава че Евклидовият алгоритъм с вход (a, b) извършва повече от m стъпки. Но в нашия опростен модел, всички двойки (a, b) имат един и същи размер (на представянето), а именно 2, защото се състоят от две числа, а ние приехме, че всяко число има размер единица. Тогава всеки вход има размер точно 2. Следва, че функциите на сложността, които имат аргумент-размера на входа, не могат да бъдат дефинирани смислено, защото имаме само

една големина на вход, а именно 2. За тази големина имаме входове, за които алгоритъмът работи в повече стъпки от всяко предварително избрано число. Излиза, че сложността по време е безкрайност ...

Този парадоксален извод се дължи само на допускането, че всяко число има размер единица. Ако приемем обаче по-сложния модел, в който всяко $k \in \mathbb{N}^+$ има размер, пропорционален на $\log_2 k$[†], парадоксът изчезва. Тогава Евклидовият алгоритъм работи във време, пропорционално на квадрата на размера на входа (вж. [55]).

Както вече казахме в Подсекция 1.1.5, моделите налагат някакви опростявания, иначе нямаше да са модели. Доколкото тези опростявания са смислени и полезни на практика, съответните модели също са смислени и полезни. Но в случаите, в които опростяванията и допусканията водят до безсмислени и безполезни резултати, съотвените модели стават неизползваеми и трябва да се търсят други модели, по-сложни и детайлни. Така че моделът, в който всяко число има големина единица, не е иманентно лош или сбъркан, просто в някои случаи е полезен (например, сортиращите алгоритми), в други, не е (например, Евклидовият алгоритъм).

Аналогично, пътната карта на България е модел на истинската пътна мрежа (защото съдържа само най-важната информация). Тази карта, в която градовете са отбелязани с точки, е много полезна в някои случаи и напълно безполезна в други случаи. Ако искаме да стигнем с кола от София до Варна, например, и не знаем пътя, пътната карта, в която Варна е точка, е полезна до момента, в който влезем във Варна. За отиването до конкретен адрес там, моделът, в който Варна е точка, вече не върши работа. Но това не означава, че пътната карта е излишна. Просто в някои случаи тя е полезен модел, в други, не. По същия начин, моделът, в който числата имат големина единица, е полезен в някои случаи и безполезен в други.

2.3 Асимптотични нотации

2.3.1 Опит за намиране на точната сложност по време на Selection Sort и Insertion Sort

Да разгледаме отново алгоритъма Insertion Sort, въведен на стр. 58. Въпреки че в Подсекция 2.2.2 установихме, че е безсмислено да търсим точен израз за сложността му по време като функция от големината на входа, с учебна цел сега ще се опитаме да направим точно това. Спазваме допусканията, че числата имат големина единица и че всяка елементарна инструкция отнема единица време.

Insertion Sort($A[1, 2, \ldots n]$: масив от цели числа)

```
1  for i ← 2 to n
2      key ← A[i]
3      j ← i − 1
4      while j > 0 and A[j] > key do
5          A[j + 1] ← A[j]
6          j ← j − 1
7      A[j + 1] ← key
```

[†] $\log_2 k$ е горе-долу броят на битовете, необходими за записването на n в двоична позиционна бройна система.

Колко стъпки отнема изпълнението на този алгоритъм върху n числа? Използваме следните съображения.

- В нашия опростен модел това, което е на ред 1—а именно инициализацията на управляващата променлива `i`, проверката за продължаване и инкрементирането—е една инструкция.[†] Тя се изпълнява n пъти.

  Да поясним защо ред 1 се изпълнява n пъти. По принцип, всяка инструкция от вида **for** `i` ← `a` **to** `b` се изпълнява точно $b - a + 2$ пъти (ако $a \leqslant b$), защото:

  - тялото на цикъла се изпълнява точно $b - a + 1$ пъти,
  - а **for** `i` ← `a` **to** `b` се изпълнява веднъж за всяко изпълнение на тялото цикъла и после **още веднъж**, когато условието за продължаване вече не е вярно.

  В този случай, $n - 2 + 2$ е точно n.

- Тялото на цикъла **for** на редове 1–7 се изпълнява, както казахме, $n-2+1 = n-1$ пъти. Следователно, всеки от редовете 2, 3 и 7 се изпълнява $n - 1$ пъти, и тъй като всеки от тях има по една инструкция, която се изпълнява за една стъпка, това са общо $3(n - 1)$ стъпки.

- При всяко изпълнение на **for** цикъла, а такива има $n-1$ на брой, изпълнението достига ред 4 (началото на **while** цикъла) поне веднъж. Но при дадено изпълнение на **for** цикъла, ред 4 може да бъде изпълняван повече от веднъж – това зависи от **конкретния вход**. С други думи, точно колко пъти ще се изпълни вложеният цикъл при някакво `i` зависи от сравненията между `A[j]` и `key`. Освен това, вложеният цикъл **не се изпълнява за една инструкция**. Следователно, за да оценим точно колко пъти ще бъдат изпълнени всеки от редовете 4–6, трябва да извършим по-подробен анализ.

  - Нека k_i е броят на изпълненията на ред 4 за всяко $i \in \{2, \ldots, n\}$. Очевидно $k_i \in \{1, \ldots, i\}$, защото ред 4 се изпълнява поне веднъж и най-много `i` пъти. Защо най-много `i` пъти? – защото `j` може да стане най-малко 0, бивайки инициализирано със стойност $i - 1$ на ред 3.
  - Всеки от редове 5 и 6 се изпълнява $k_i - 1$ пъти. Тогава ред 4 се изпълнява $\sum_{i=2}^{n} k_i$ пъти **общо за цялото изпълнение на алгоритъма**, а редове 5 и 6 се изпълняват по $\sum_{i=2}^{n}(k_i - 1)$ пъти. Общо изпълнението на **while** цикъла "струва" $\sum_{i=2}^{n} k_i + 2\sum_{i=2}^{n}(k_i - 1) = (3\sum_{i=2}^{n} k_i) - 2(n - 1)$ стъпки по време на цялото изпълнение на алгоритъма.

[†]При програмиране на истински компютър това не е така. Следният тривиален **for** цикъл на езика C:

```
for (i=2; i <= n; i++);
```

като фрагмент от програма се компилира върху Intel iCore3 процесор, Ubuntu Linux, 64 битов gcc компилатор без оптимизация, в следния код (адресите и отместванията, естествено, варират):

```
0x4005bb <main+30> movl $0x2,-0x4(%rbp)
0x4005c2 <main+37> jmp 0x4005c8 <main+43>
0x4005c4 <main+39> addl $0x1,-0x4(%rbp)
0x4005c8 <main+43> mov -0x8(%rbp),%eax
0x4005cb <main+46> cmp %eax,-0x4(%rbp)
0x4005ce <main+49> jle 0x4005c4 <main+39>
```

Както виждаме, при истинския компютър инкрементирането с единица `addl`, проверката за край на изпълнението на цикъла `cmp`, и условният скок `jle` в изпълнението, са отделни инструкции.

И така, в нашия опростен модел, изпълнението на Insertion Sort става в

$$n + 3(n-1) + 3\left(\sum_{i=2}^{n} k_i\right) - 2(n-1) \tag{2.6}$$

тоест,

$$2n - 1 + 3\sum_{i=2}^{n} k_i \tag{2.7}$$

стъпки. За да довършим анализа, трябва да определим минималната и максималната възможна стойност на сумата. k_i може да е най-малко 1, когато тялото на **while** цикъла не се изпълнява изобщо, и най-много i, когато тялото на **while** цикъла се изпълнява $i-1$ пъти.

Показахме, че сложността по време в най-добрия случай е

$$P(n) = 2n - 1 + 3\sum_{i=2}^{n} 1 = 2n - 1 + 3n - 3 = 5n - 4 \tag{2.8}$$

и сложността по време в най-лошия случай е

$$T(n) = 2n - 1 + 3\sum_{i=2}^{n}(i-1) = 2n - 1 + \frac{3n(n-1)}{2} = \frac{3n^2+n}{2} - 1 \tag{2.9}$$

Ще анализираме сложността на още един алгоритъм по аналогичен начин.

```
Selection Sort(A[1, 2, ... n]: масив от цели числа)
1   for i ← 1 to n − 1
2       for j ← i + 1 to n
3           if A[j] < A[i]
4               swap(A[i], A[j])
```

Ред 1 се изпълнява общо $n-1-1+2 = n$ пъти. Ред 2 се изпълнява $n-i+1$ за всяко i, общо $\sum_{i=1}^{n-1} n-i+1 = \frac{(n+2)(n-1)}{2}$. Ред 3 се изпълнява $n-i$ за всяко i, общо $\sum_{i=1}^{n-1} n-i = \frac{n(n-1)}{2}$. Ред 3 се изпълнява общо k пъти за някакво k, такова че $0 \leqslant k \leqslant \frac{n(n-1)}{2}$, в зависимост от конкретния вход. Общо сложността е

$$n + \frac{(n+2)(n-1)}{2} + \frac{n(n-1)}{2} + k = n^2 + n - 1 + k \tag{2.10}$$

Тогава сложността в най-добрия случай е

$$P'(n) = n^2 + n - 1 \tag{2.11}$$

и сложността в най-лошия случай е

$$T'(n) = n^2 + n - 1 + \frac{n(n-1)}{2} = \frac{3n^2+n}{2} - 1 \tag{2.12}$$

2.3.2 Отказ от точна оценка и търсене на приблизителна оценка на сложността

От изложението в Подсекция 2.2.2 трябва да е станало ясно, че точни изрази като (2.8), (2.9), (2.11) и (2.12) не са необходими. Нашият модел на сложността е достатъчно далече от реалността, така че събираемото -1, което имаме в (2.12), няма практически никакво значение по отношение на каквато и да е софтуерна реализация на Selection Sort. Не можем да предвидим времето за изпълнение с точност до наносекунда, така че по отношение на реално изпълнение, **събираемото -1 не ни казва нищо**. Нещо повече, цялото събираемо $\frac{1}{2}n - 1$ е безсмислено. Ако разполагаме с конкретна реализация на алгоритъма и я тестваме върху различни тестови входове, за които имаме най-лоша сложност (обратно сортирани последователности) и после анализираме данните, ще стане ясно, че бързодействието се интерполира от някаква квадратична функция, и толкова. Фактори като конкретна операционна система, компилатор, архитектура и виртуална машина, както и умението на програмиста да пише бързи програми върху конкретната машина, със сигурност ще се отразят повече на реалното бързодействие от $\frac{1}{2}n - 1$.

Това, което има значение в (2.12) е фактът, че от трите събираеми най-бързо растящото е квадратичната функция $\frac{3}{2}n^2$. Игнорираме множителя $\frac{3}{2}$ и казваме, че алгоритъмът има **квадратична сложност по време**. Никакъв избор на конкретна операционна система, компилатор, програмистки трикове и така нататък не могат да подобрят квадратичната сложност до, да речем, линейна (след малко ще дефинираме прецизно тези понятия).

И така, най-съществено за сложностите на Insertion Sort и Selection Sort е, че в най-лошия случай[†] те са квадратични функции.

В нашата поредица от допускания, които опростяват нещата и ни позволяват изобщо да правим анализ на сложността на алгоритми, следва допускането, че степента на нарастване на функцията на сложността е всичко, което ни интересува за нея. Защо? Защото се интересуваме само от тенденцията на изменение на сложността при **неограничено нарастване** на големината на входа. Никаква конкретна големина на входа не ни интересува. Никаква нейна конкретна стойност, ако ще да е $10\,000\,000^{10\,000\,000^{10\,000\,000}}$, не е достатъчно голяма, за да спрем до нея. След малко ще дадем прецизно определение на "степен на нарастване", засега ще се задоволим с примера с анализа на двата алгоритъма горе. И на двата функциите на сложността в най-лошия случай са квадратични. Ще смятаме, че сложността им е еднаква: квадратична.

И така, правим следните допускания:

Допускане 1. Ако изразът за сложността на даден алгоритъм е сума, разглеждаме най-бързо растящата функция измежду събираемите, в *асимптотичния смисъл*. А именно, ако изразът е $f_1(n) + f_2(n) + \ldots + f_k(n)$ и $\lim_{n\to\infty} \frac{f_1(n)}{f_i(n)} = \infty$ за всяко $i \in \{2, \ldots, k\}$, то $f_1(n)$ е най-бързо растящата в асимптотичния смисъл функция измежду $f_1(n)$, $\ldots$, $f_k(n)$[‡]. Ще смятаме, че най-бързо растящата в този смисъл функция доминира над останалите и тях можем да пренебрегнем.

Допускане 2. За два различни алгоритъма, ако водещите функции в горния смисъл в изразите за сложността са $f_1(n)$ и $g_1(n)$ и $0 < \lim_{n\to\infty} \frac{f_1(n)}{g_1(n)} < \infty$, ще смятаме, че тези алгоритми работят еднакво бързо.

[†]Казахме, че сложността в най-добрия случай не е особено информативна и не се ползва, но си заслужава да се отбележи, че в най-добрия случай Insertion Sort има сложност, която е линейна функция, без алгоритъмът да е модифициран чрез изкуствено пришити инструкции за само един най-добър случай.

[‡]Тук предполагаме, че всички те са положителни функции.

Допускане 1 не винаги е реалистично. Смисълът да се разглежда само тенденция на нарастването, когато n клони към безкрайност, е ясен: по-големите входове са по-интересни. Но на практика това е вярно само донякъде. На практика, наистина поведението на алгоритъм за сортиране върху вход с големина $10\,000\,000$ е по-интересно от поведението му върху вход с големина 10, но едва ли можем да твърдим, че поведението му върху вход с големина $10\,000\,000^{10\,000\,000}$ е още по-интересно, а това върху вход с големина $10\,000\,000^{10\,000\,000^{10\,000\,000}}$, дори още по-интересно. Последните две числа са напълно отвъд възможностите на всеки истински компютър, сегашен и бъдещ, така че програмата, реализираща въпросния алгоритъм, никога няма да има възможност да демонстрира бързината си върху входове с такива размери. Когато се фокусираме само върху водещото събираемо, ние твърдим, че измежду два алгоритъма, единият от които има сложност n^2+n, а другият, $n\sqrt{n}+10\,000\,000^{10\,000\,000^{10\,000\,000}}n$, вторият работи по-бързо, защото неговата водеща функция $n\sqrt{n}$ расте по-бавно от n^2, водещата функция на първия. Очевидно съществува стойност на големината на входа n_0, такава че за всяко $n \geqslant n_0$, вторият алгоритъм печели като бързодействие, но това n_0 е твърде голямо. Както вече стана ясно, то е число, което никога не е големина на вход в реалния свят.

Допускане 2, тоест игнорирането на мултипликативната константа във водещата функция, също не винаги е реалистично. Според него, два алгоритъма, единият от които има сложност $n^2 + n$, а другият има сложност $10\,000\,000^{10\,000\,000^{10\,000\,000}}n^2$, са с една и съща сложност, а именно квадратична. "Оправданието" ни е, че алгоритмите, които се използват на практика и почиват върху прагматични идеи, по правило имат сложности, в които тези мултипликативни константи не са фрапиращо големи и никога не са от порядъка на $10\,000\,000^{10\,000\,000^{10\,000\,000}}$, така че опростяването не е прекалено грубо. В теорията обаче са известни алгоритми, при които тази мултипликативна константа е кула от степени на двойката

$$2^{2^{2^{\cdot^{\cdot^{\cdot^{2}}}}}}$$

чиято височина е кула от степени на двойката, чиято височина е кула от степени на двойката, и така нататък няколко пъти [45]. Очевидно такъв алгоритъм е непрактичен дори за вход с големина единица.

Да обобщим. Когато разглеждаме тенденцията на нарастването при неограничено нарастване на аргумента n, ние просто правим поредното опростяващо допускане. Причината да го правим е, че **така е по-лесно**. Освен това, практиката показва, че често—но **не винаги**—резултатите, които ни дава това опростяване, са добре корелирани с качествата на реалните програми, реализиращи съответните алгоритми.

Ще завършим подсекцията с един цитат от книгата "Computational Complexity" на Papadimitriou [72, стр. 7].

> Any attempt, in any field of mathematics, to capture an intuitive, real-life notion (for example, that of a "smooth function" in real analysis) by a mathematical concept (such as C_∞) is bound to include certain undesirable specimens, while excluding others that arguably should be embraced.

Следователно, както и да опитваме да формализираме житейското понятие "бърз алгоритъм" или дори "бърза програма", неизбежно е според формалните ни дефиниции някои алгоритми, които не бихме нарекли бързи, да бъдат класифицирани като такива, а от друга страна, алгоритми, които смятаме за прилично бързи, да бъдат класифицирани като бавни.

2.3.3 Асимптотична нотация Тета

Използвайки асимптотичните нотации, които сега ще дефинираме, можем да изразяваме сложността на изследваните от нас алгоритми **приблизително**. Практиката е показала, че тези приближения са смислени: ако сложността на алгоритъм A, изразена в Θ-нотация, е по-малка от сложността на алгоритъм B, пак в Θ-нотация, твърде възможно е практическата реализация на алгоритъм A да работи по-бързо от практическата реализация на алгоритъм B.

Забележете, че дефинициите не използват нотацията lim.

Конвенция 3: функции от $\mathbb{R}^+$ в $\mathbb{R}^+$

Отсега нататък допускаме, че всички функции, които разглеждаме, имат домейн и кодомейн $\mathbb{R}^+$, освен ако изрично не е казано нещо друго. Въпреки това, типичното име на променливата е n, а не x.

Ако n е големина на входа на алгоритъм, то n, разбира се, е **цяло** положително число. Но в тази глава разглеждаме само математическите основи на сложността на алгоритмите и затова си позволяваме да допуснем, че $n \in \mathbb{R}^+$.

Определение 12: $\Theta(g(n))$

За всяка функция $g(n)$:

$$\Theta(g(n)) \stackrel{\text{def}}{=} \left\{ f(n) \mid \exists c_1, c_2 > 0 \; \exists n_0 \; \forall n \geqslant n_0 \; : \; 0 \leqslant c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n) \right\}$$

Формално, $\Theta(g(n))$ е безкрайно множество от функции $f(n)$, за които е в сила някакво условие. Ако искаме да кажем, че $h(n)$ е една от тях, пишем $h(n) = \Theta(g(n))$ наместо формално коректното $h(n) \in \Theta(g(n))$. Причината е историческа – прието е да се използва знакът за равенство, а не формално коректният знак за принадлежност към множество. Изразът "$h(n) = \Theta(g(n))$" се чете: "$h(n)$ е Тета-голямо от $g(n)$".

Ето пример за много подробно доказателство, че една функция е Тета-голямо от друга функция. Ще покажем, че $n^2 + 10n + 12 = \Theta(n^2)$, използвайки Определение 12. Съгласно определението, трябва да съществуват константи $c_1, c_2 > 0$ и стойност n_0 на аргумента, такива че за всяко $n \geqslant n_0$ да е вярно

$$0 \leqslant c_1 n^2 \leqslant n^2 + 10n + 12 \leqslant c_2 n^2$$

Това са всъщност три неравенства. Първото неравенство $0 \leqslant c_1 n^2$ е очевидно: щом $c_1 > 0$, то можем да вземем $n_0 = 1$ и тогава $\forall n \geqslant n_0 : 0 \leqslant c_1 n^2$. Да разгледаме второто неравенство $c_1 n^2 \leqslant n^2 + 10n + 12$. Имаме право да разделим двете страни на n^2, защото разглеждаме само положителни n:

$$c_1 n^2 \leqslant n^2 + 10n + 12 \leftrightarrow c_1 \leqslant 1 + \frac{10}{n} + \frac{12}{n^2}$$

Можем да вземем просто $c_1 = 1$, понеже $\forall n \geqslant n_0 : 1 \leqslant 1 + \frac{10}{n} + \frac{12}{n^2}$. Тук отново n_0 е 1. Да разгледаме третото неравенство $n^2 + 10n + 12 \leqslant c_2 n^2$:

$$n^2 + 10n + 12 \leqslant c_2 n^2 \leftrightarrow 1 + \frac{10}{n} + \frac{12}{n^2} \leqslant c_2$$

Ако искаме да използваме същата стойност на n_0, която използвахме в предните две неравенства, а именно $n_0 = 1$, можем да вземем някакво голямо c_2, да кажем $c_2 = 100$, и тогава лесно се вижда, че $\forall n \geqslant n_0 : 1 + \frac{10}{n} + \frac{12}{n^2} \leqslant 100$. Други стойности за c_2 и n_0 също биха свършили работа за доказателството, примерно $c_2 = 3$ и $n_0 = 12$.

Ако в първото, второто и третото неравенство сме открили различни стойности на n_0, всяка от които върши работа за съответното неравенство, то за цялото доказателство е достатъчно да вземем най-голямата от тях. В конкретния случай, ако за първото и второто неравенство $n_0 = 1$ върши работа, а за третото работа върши $n_0 = 12$, то за цялото доказателство ще вземем $n_0 = 12$.

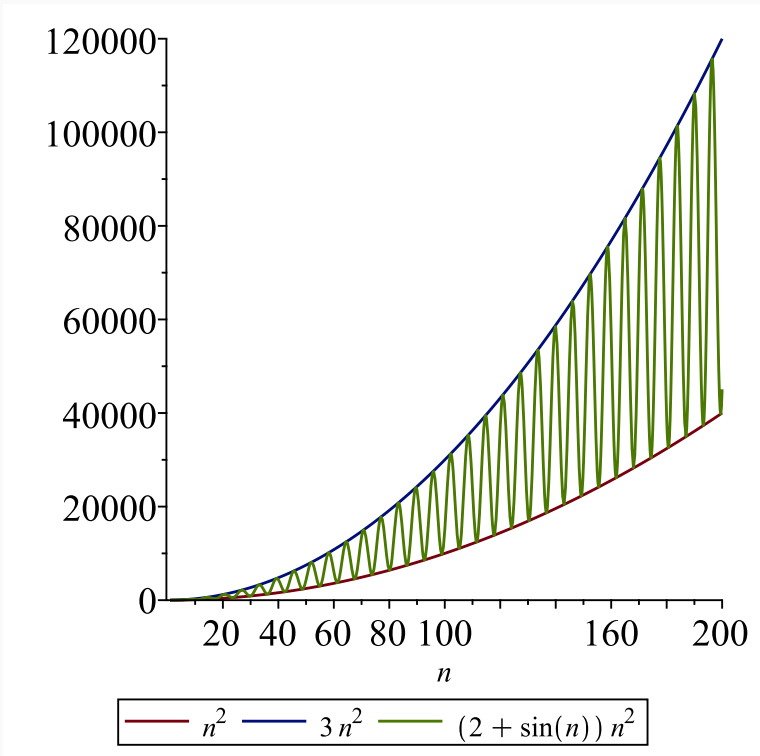
И накрая отбелязваме, че съществуват повече от една наредени тройки (c_1, c_2, n_0), които може да се използват в доказателството.

Наблюдение 4

Забележете, че "$\lim_{n\to\infty} \frac{h(n)}{g(n)}$ съществува и е равно на някакво L, такова че $0 < L < \infty$" е по-силно твърдение от "$h(n) = \Theta(g(n))$". С други думи, Тета-нотацията е по-обща от изразяването чрез граница, защото границата $\lim_{n\to\infty} \frac{h(n)}{g(n)}$ може да не съществува и въпреки това $h(n)$ да е Тета-голямо от $g(n)$.

Като пример за това да разгледаме $(2+\sin n)n^2$ и n^2. Вярно е, че $(2+\sin n)n^2 = \Theta(n^2)$. За доказателството можем да вземем константи $c_1 = 1$ и $c_2 = 3$. Обаче границата $\lim_{n\to\infty} \frac{(2+\sin n)n^2}{n^2} = \lim_{n\to\infty} 2 + \sin n$ не съществува. Разгледайте Фигура 2.2[†]. Графиката на $(2 + \sin n)n^2$ осцилира между графиките на n^2 и $3n^2$, като амплитудата на осцилирането нараства неограничено с нарастването на аргумента. Следователно, във формализма на границите не бихме могли да изразим директно, че $(2 + \sin n)n^2$ и n^2 са асимптотично "близки".

Фигура 2.2 : Графиките на n^2, $3n^2$ и $(2 + \sin n)n^2$.



[†]Фигура 2.2 е направена с Maple(TM).

Лема 4

За всички функции $f(n)$, $g(n)$ и $h(n)$:

$$f(n) = \Theta(f(n)) \tag{2.13}$$
$$f(n) = \Theta(g(n)) \rightarrow g(n) = \Theta(f(n)) \tag{2.14}$$
$$f(n) = \Theta(g(n)) \wedge g(n) = \Theta(h(n)) \rightarrow f(n) = \Theta(h(n)) \tag{2.15}$$

Доказателство: Равенствата 2.13 и 2.15 следват очевидно от рефлексивността и транзитивността на релацията "$\leqslant$" в Определение 12. Да разгледаме равенство 2.14. В термините на Определение 12, то казва:

$$\big(\exists c_1, c_2 > 0\, \exists n_0 > 0\, \forall n \geqslant n_0 : 0 \leqslant c_1.g(n) \leqslant f(n) \leqslant c_2.g(n)\big) \rightarrow$$
$$\big(\exists c_1', c_2' > 0\, \exists n_0' > 0\, \forall n \geqslant n_0' : 0 \leqslant c_1'.f(n) \leqslant g(n) \leqslant c_2'.f(n)\big)$$

Но това е очевидно вярно, ако вземем $c_2' = \frac{1}{c_1}$, $c_1' = \frac{1}{c_2}$, и $n_0' = n_0$. □

Следствие 3

От равенство 2.14 на Лема 4 веднага следва, че $f(n) = \Theta(g(n))$ тогава и само тогава, когато $g(n) = \Theta(f(n))$.

Определение 13

Линеен е всеки алгоритъм, чиято сложност по време в най-лошия случай е $\Theta(n)$. Аналогично, ако сложността по време в най-лошия случай е $\Theta(n^2)$, $\Theta(n^3)$ или $\Theta(n^4)$, казваме съответно, че алгоритъмът е *квадратичен*, *кубичен* или *квартичен*.

Използвайки Тета-нотацията, можем да запишем сложността в най-лошия случай $T(n)$ на Insertion Sort и $T'(n)$ на Selection Sort така:

$$T(n) = \Theta(n^2)$$
$$T'(n) = \Theta(n^2)$$

От гледна точка на Тета-асимптотиката, по-прецизен анализ на тези алгоритми няма. Те имат квадратична сложност по време и толкова.

Конвенция 4: Еднопосочност на знака за равенство при Θ

Записът $\Theta(n^2) = T(n)$ **е некоректен** и не се използва. Знакът за равенство в Тета-изразите е еднопосочен: Тета-нотацията може да се появява или само вдясно, което видяхме досега, или вдясно и вляво, което ще видим след малко, но никога само вляво. Примери за появата на Тета вляво и вдясно са изрази като

$$n^2 + \Theta(n) = \Theta(n^2)$$

В този случай Тета-нотациите вляво и вдясно имат съвсем различен смисъл. Тета-нотацията вдясно представлява безкрайно множество от функции, а тази вляво е *анонимна функция*. С други думи, Тета-нотацията вляво представлява не множество функции, а само една, като всичко, което знаем за нея е, че принадлежи на множеството $\Theta(n)$. Целият израз се чете така: сумата на n^2 и коя да е функция от множеството $\Theta(n)$ е функция от множеството $\Theta(n^2)$.

Допълнение 14: Друга интерпретация на появата на Θ вляво

Според Knuth [53], изрази като "$n^2+\Theta(n) = \Theta(n^2)$" се четат по следния начин. Нека A и B са множества от функции. Тогава $A+B$ е множеството от функции $\{f+g \mid f \in A, g \in B\}$. В примера "$n^2 + \Theta(n) = \Theta(n^2)$" изразът вляво се интерпретира точно по този начин: нека $A = \{n^2\}$, а B е безкрайното множество от функции $\Theta(n)$ според Определение 12; тогава "$n^2 + \Theta(n)$" е множеството от функции, всяка от които е сума на n^2 и някоя функция от $\Theta(n)$.
Тогава знакът за равенство се интерпретира не като знак за принадлежност към множество "$\in$", а като знак за **подмножество** "$\subseteq$". При тази интерпретация, "$n^2 + \Theta(n) = \Theta(n^2)$" се чете така "множеството от функции, всяка от които е сумата на n^2 и някоя функция от $\Theta(n)$, е подмножество на множеството $\Theta(n^2)$."

Конвенция 5: В Θ-нотацията се слага най-простият израз

Формално погледнато, дали ще пишем

$$10\sqrt{7}n^2 + 15.5n\sqrt{n} + 35n = \Theta(n^2)$$

или

$$n^2 = \Theta(10\sqrt{7}n^2 + 15.5n\sqrt{n} + 35n)$$

е без значение. И двете твърдения са верни. На практика вторият запис **не се ползва**. В скобите на Тета-нотацията слагаме възможно най-простия израз.

Конвенция 6: $\Theta(1)$ означава анонимна константа

В теорията на алгоритмите е прието да се ползва "$\Theta(1)$" като синоним на "анонимна константа" и "сложност $\Theta(1)$" като синоним на "константна сложност". Оттам и "n^ϵ, където ϵ е константа" в Определение 20 може да се запише кратко "$n^{\Theta(1)}$".
Единицата не е специална и на теория всяко друго число би могло да се използва за същата цел, но това не е прието. "$\Theta(2)$", "$\Theta(1000)$" или "$\Theta(\pi)$" изглеждат странно и претенциозно. Да се ползват би било лош стил.
Вместо $\Theta(1)$ понякога се пише $O(1)$—вижте Определение 14, (2.16) надолу—като смисълът е абсолютно същият; става дума за положителна анонимна константа.

Константна сложност по време при алгоритмите практически не се среща. Само най-прости алгоритми като

- алгоритъм, чийто вход има константна големина, примерно сумиране на две числа,
- или алгоритъм, който винаги връща едно и също нещо, игнорирайки входа,

имат сложност по време $\Theta(1)$. Ние такива алгоритми няма да разглеждаме, освен някои процедури, ползвани от основния разглеждан алгоритъм, примерно RELAX на стр. 450, чийто вход има константа големина. Всеки интересен самостоятелен алгоритъм има сложност по време, която е някаква растяща функция на големината на входа. Заслужава да се отбележи, че ако сложността по време на алгоритъм е $\Theta(f(n))$, където $f(n)$ е функция, която расте асимптотично по-бавно от линейна в смисъла на Определение 14, то този алгоритъм не "чете" целия вход, защото само достъпът до всеки елемент на входа иска линейно време. Пример на такъв алгоритъм е двоичното търсене на стр. 124, при който $f(n) = \lg n$; това е доказано в Подсекция 2.4.3.

От друга страна, алгоритми със сложност по памет $\Theta(1)$ се срещат често. Както вече бе казано на стр. 67, такива алгоритми се наричат in-place.

2.3.4 Други асимптотични нотации: O-голямо, Омега-голямо, o-малко, омега-малко

Както казахме, що се отнася до асимптотиката на нарастването, Тета-нотацията дава възможно най-подробната информация. Въпреки че Тета-нотацията е доста "хлабава"—примерно, $\frac{n^3}{1000}$ и $1000^{1000}n^3$ са Тета една от друга—понякога не можем да намерим дори Тета-оценката на някаква функция. За такива случаи има други нотации, по-слаби от Тета-та, които са все пак информативни за асимптотиката на нарастването.

Определение 14: $O(g(n)), \Omega(g(n)), o(g(n)), \omega(g(n))$

За всяка функция $g(n)$:

$$O(g(n)) \stackrel{\text{def}}{=} \{f(n) \mid \exists c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ 0 \leqslant f(n) \leqslant c \cdot g(n)\} \tag{2.16}$$

$$\Omega(g(n)) \stackrel{\text{def}}{=} \{f(n) \mid \exists c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ 0 \leqslant c \cdot g(n) \leqslant f(n)\} \tag{2.17}$$

$$o(g(n)) \stackrel{\text{def}}{=} \{f(n) \mid \forall c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ 0 \leqslant f(n) < c \cdot g(n)\} \tag{2.18}$$

$$\omega(g(n)) \stackrel{\text{def}}{=} \{f(n) \mid \forall c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ 0 \leqslant c \cdot g(n) < f(n)\} \tag{2.19}$$

Както и при Тета-нотацията, принадлежността към тези множества означаваме не с "$\in$", а с "$=$", примерно пишем $f(n) = O(g(n))$, $h(n) = \omega(\phi(n))$ и така нататък.

Ако $f(n) = O(g(n))$, казваме, че $g(n)$ е *асимптотична горна граница* за $f(n)$. Примерно, вярно е, че:

$$n = O(n^2) \qquad n^2 = O(n^2) \qquad n^2 + 1000n + 10000 = O(n^2) \qquad 1 = O(n^2)$$

Съответно, функцията n^2 е асимптотична горна граница за всяка от функциите n, n^2, $n^2 + 1000n + 10000$ и 1. От друга страна обаче, $\frac{1}{1\,000\,000}n^3 \neq O(n^2)$. Съответно, функцията n^2 не е асимптотична горна граница за функцията $\frac{1}{1\,000\,000}n^3$.

Забележете съществената разлика между дефинициите на $O(g(n))$ и $o(g(n))$ (същото важи и при $\Omega(g(n))$ и $\omega(g(n))$). Ако $f(n) = O(g(n))$, то за **някаква** положителна константа c, $f(n)$ не надвишава $c \cdot g(n)$[†]. Ако $f(n) = o(g(n))$, то за **всяка** положителна константа c, $f(n)$ "изостава" от $c \cdot g(n)$[‡]. Неформално говорейки, нотацията о-малко казва, че можем да "отдалечим" $f(n)$ "надолу" от $g(n)$ на каквато си искаме мултипликативна константа, стига да разглеждаме достатъчно големи стойности на аргумента. Затова, ако $f(n) = o(g(n))$, казваме, че $g(n)$ е *строга асимптотична горна граница* за $f(n)$.

Аналогично, $\Omega(\,)$ и $\omega(\,)$ задават съответно *асимптотична долна граница* и *строга асимптотична долна граница*.

В Лема 5 и Лема 6, нотациите и от двете страни на равенствата означават множества (съгласно дефинициите), а знаците за равенство означават равенства между множества (а не принадлежност към множества).

Лема 5

За всяка функция $g(n)$, $O(g(n)) \cap \Omega(g(n)) = \Theta(g(n))$.

Доказателство: Да разгледаме произволна $f(n) \in O(g(n)) \cap \Omega(g(n))$. Съгласно Определение 14(2.16) и Определение 14(2.17), това е същото като

$$(\exists c' > 0\ \exists n_0'\ \forall n \geqslant n_0'\ :\ f(n) \leqslant c' \cdot g(n)) \wedge (\exists c'' > 0\ \exists n_0''\ \forall n \geqslant n_0\ :\ c'' \cdot g(n) \leqslant f(n))$$

Но тогава очевидно съществуват положителни c', c'', такива че за всяко $n \geqslant \max\{n_0', n_0''\}$ е изпълнено

$$c'' \cdot g(n) \leqslant f(n) \leqslant c' \cdot g(n)$$

Съгласно Определение 12, $f(n) \in \Theta(g(n))$. В обратната посока, да допуснем, че $f(n) \in \Theta(g(n))$. Тогава съгласно Определение 12:

$$\exists c_1, c_2 > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ 0 \leqslant c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n)$$

Тривиално следва, че

$$(\exists c > 0\ \exists n_0\ \forall n \geqslant n_0\ : f(n) \leqslant c \cdot g(n)) \wedge (\exists c > 0\ \exists n_0\ \forall n \geqslant n_0\ : c \cdot g(n) \leqslant f(n))$$

[†]И то не непременно навсякъде, а за всяко n от **някакво** n_0 **нататък**.
[‡]И то не непременно навсякъде, а за всяко n от **някакво** n_0 **нататък**.

Съгласно Определение 14, това е същото като

$$f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n))$$

□

Лема 6

За всяка функция $g(n)$:

$$o(g(n)) \cap \omega(g(n)) = \varnothing \tag{2.20}$$
$$O(g(n)) \cap \omega(g(n)) = \varnothing \tag{2.21}$$
$$o(g(n)) \cap \Omega(g(n)) = \varnothing \tag{2.22}$$

Доказателство: Ще докажем (2.20). Да разгледаме произволна $f(n) \in o(g(n)) \cap \omega(g(n))$. Съгласно Определение 14(2.18) и Определение 14(2.19), това е същото като

$$(\forall c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ f(n) < c \cdot g(n)) \wedge (\forall c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ c \cdot g(n) < f(n))$$

Щом и двата израза започват със "за всяко c", то имаме право да вземем едно и също c' и в двата. Заключаваме, че има стойност на аргумента $\tilde{n}$, такава че за всяко $n > \tilde{n}$:

$$f(n) < c' \cdot g(n) \wedge f(n) > c' \cdot g(n)$$

Това очевидно е невъзможно.

Ще докажем (2.21). Да разгледаме произволна $f(n) \in O(g(n)) \cap \omega(g(n))$. Съгласно Определение 14(2.16) и Определение 14(2.19), това е същото като

$$(\exists c > 0\ \exists n_0\ \forall n \geqslant n_0\ :f(n) \leqslant c \cdot g(n)) \wedge (\forall c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ c \cdot g(n) < f(n))$$

Щом изразът вдясно съдържа $\forall c$, в частност той е истина за стойността на c, за която изразът вляво е истина. Заключаваме, че има $c > 0$ и има стойност на аргумента $\tilde{n}$, такава че за всяко $n > \tilde{n}$:

$$f(n) \leqslant c \cdot g(n) \wedge f(n) > c \cdot g(n)$$

Това очевидно е невъзможно.

Ще докажем (2.22). Да разгледаме произволна $f(n) \in o(g(n)) \cap \Omega(g(n))$. Съгласно Определение 14(2.18) и Определение 14(2.17), това е същото като

$$(\forall c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ f(n) < c \cdot g(n)) \wedge (\exists c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ c \cdot g(n) \leqslant f(n))$$

Щом изразът вляво съдържа $\forall c$, в частност той е истина за стойността на c, за която изразът вдясно е истина. Заключаваме, че има $c > 0$ и има стойност на аргумента $\tilde{n}$, такава че за всяко $n > \tilde{n}$:

$$f(n) > c \cdot g(n) \wedge f(n) \leqslant c \cdot g(n)$$

Това очевидно е невъзможно. □

Нотациите O и Ω са свързани. Също така и нотациите o и ω са свързани. Ето как.

Лема 7

За всички функции $f(n)$, $g(n)$:

$$f(n) = O(g(n)) \leftrightarrow g(n) = \Omega(f(n)) \tag{2.23}$$
$$f(n) = o(g(n)) \leftrightarrow g(n) = \omega(f(n)) \tag{2.24}$$

Доказателство: Ще докажем (2.23). Наистина, съгласно Определение 14(2.16), $f(n) = O(g(n))$ е същото като

$$\exists c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ f(n) \leqslant c \cdot g(n)$$

Но $f(n) \leqslant c \cdot g(n)$ е същото като $\frac{1}{c} f(n) \leqslant g(n)$; тъй като $c > 0$, $\frac{1}{c}$ е дефинирано. Заключаваме, че

$$\exists c' > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ c' \cdot f(n) \leqslant g(n)$$

а именно за $c' = \frac{1}{c}$. Но това е същото като $g(n) = \Omega(f(n))$ съгласно Определение 14(2.17).

Ще докажем (2.24). Наистина, съгласно Определение 14(2.18), $f(n) = o(g(n))$ е същото като

$$\forall c > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ f(n) < c \cdot g(n)$$

Но $f(n) < c \cdot g(n)$ е същото като $\frac{1}{c} f(n) < g(n)$; тъй като $c > 0$, $\frac{1}{c}$ е дефинирано. Нещо повече, всяко положително число може да се представи като $\frac{1}{c}$, за някое положително c. Заключаваме, че

$$\forall c' > 0\ \exists n_0\ \forall n \geqslant n_0\ :\ c' \cdot f(n) < g(n)$$

Но това е същото като $g(n) = \Omega(f(n))$ съгласно Определение 14(2.17).

□

Нотациите o и ω може да се дефинират чрез граници.

Лема 8

За всички функции $f(n)$, $g(n)$:

$$f(n) = o(g(n)) \leftrightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$
$$f(n) = \omega(g(n)) \leftrightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

Допълнение 15: Нотациите O и o в анализа

В математическия анализ нотациите O и o се ползват *от 19 век*, тоест, много преди модерната теория на алгоритмите. Да разгледаме развиването на функцията $\sin(x)$ в ред на Taylor в околността на точка 0:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} + \cdots$$

Често срещан запис на това развиване е:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} + O(x^7) \tag{2.25}$$

(2.25) е по-прецизен запис от

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!}$$

защото "$\approx$" може да означава какво ли не. (2.25) казва, че $\sin(x)$ е точно равен на сумата на $x - \frac{x^3}{3!} + \frac{x^5}{5!}$ и някаква функция, която в околността на точка 0 намалява, в асимптотичния смисъл, като x^7[a]. Очевидно нотацията "$O(x^7)$" в (2.25) има смисъл, различен от този на Определение 14(2.16). В (2.25) се има предвид не тенденция при неограничено нарастване на аргумента—както е в Определение 14—а тенденция, когато аргументът клони към определена стойност, в случая 0.
Записът "$f(x) = O(g(x))$ при $x \to a$" е кратък запис за следното:

$$\exists c > 0\, \exists \delta > 0 : |f(x)| \leqslant cg(x), \text{ ако } 0 < |x - a| < \delta$$

В (2.25) "$x \to 0$" е изпуснато, но то се подразбира.
За нотацията o нещата са аналогични. Записът "$f(x) = o(g(x))$ при $x \to a$" е кратък запис за следното:

$$\forall c > 0\, \exists \delta > 0 : |f(x)| \leqslant cg(x), \text{ ако } 0 < |x - a| < \delta$$

В заключение, нотациите O и o винаги изразяват тенденции за нарастване (или намаляване) на функции. Ако x е аргументът, тези тенденции са или при $x \to a$ за някакво фиксирано a, или при $x \to \infty$. При изследването на сложността на алгоритми се интересуваме само от тенденцията при $x \to \infty$, поради което асимптотичните нотации за целите на тези лекционни записки винаги са спрямо тенденцията на неограничено нарастване на аргумента.

[a] Тъй като аргументът е по-малък от единица, x^7 изразява намаляване, а не нарастване.

2.3.5 Релации $\asymp$, $\preceq$, $\prec$, $\succeq$ и $\succ$

Определение 15: Релация $\asymp$ над множеството на функциите

За всички функции $f(n)$ и $g(n)$, $f(n) \asymp g(n)$ тогава и само тогава, когато $f(n) = \Theta(g(n))$.

Нотацията, използваща символа "$\asymp$", е алтернатива на нотацията, използваща Θ: вместо "$f(n) = \Theta(g(n))$" можем да запишем "$f(n) \asymp g(n)$" и обратно.

Ако ползваме записа с "$\asymp$" вместо "Θ", необходимостта от Конвенция 4 изчезва по очевидни причини: в израза няма "Θ". Също така изчезва и необходимостта от Конвенция 5, понеже двете страни на знака $\asymp$ са равнопоставени. Ерго, следните два записа са еднакво приемливи:

$$10\sqrt{7}n^2 + 15.5n\sqrt{n} + 35n \asymp n^2$$
$$n^2 \asymp 10\sqrt{7}n^2 + 15.5n\sqrt{n} + 35n$$

Определение 16: Релации $\preceq$, $\prec$, $\succeq$ и $\succ$

За всички функции $f(n)$ и $g(n)$:

- $f(n) \preceq g(n)$ тогава и само тогава, когато $f(n) = O(g(n))$,
- $f(n) \prec g(n)$ тогава и само тогава, когато $f(n) = o(g(n))$,
- $f(n) \succeq g(n)$ тогава и само тогава, когато $f(n) = \Omega(g(n))$,
- $f(n) \succ g(n)$ тогава и само тогава, когато $f(n) = \omega(g(n))$.

Лема 9 се доказва тривиално с Лема 4 и Следствие 3.

Лема 9

$\asymp$ е релация на еквивалентност. За всеки две функции $f(n)$ и $g(n)$, ако $f(n) \asymp g(n)$, казваме, че $f(n)$ *и* $g(n)$ *са асимптотично еквивалентни.*

Определение 17: Релации на асимптотични сравнения

Петте релации $\asymp$, $\preceq$, $\prec$, $\succeq$ и $\succ$ са известни като *релациите на асимптотични сравнения.* За всеки две функции $f(n)$ и $g(n)$, ако поне една от тези пет релации е в сила между тях, казваме, че $f(n)$ *и* $g(n)$ *са асимптотично сравними.* В противен случай казваме, че $f(n)$ *и* $g(n)$ *са асимптотично несравними.*

От симетрията на $\asymp$ и транспонираната симетрия (вж. Следствие 6) на $\{\preceq, \succeq\}$ и на $\{\prec, \succ\}$ следва, че по отношение на асимптотичната сравнимост няма значение коя от $f(n)$ и $g(n)$ е записана вляво и коя, вдясно. С други думи, ако за някоя от петте релации, да я запишем като $\rhd$, е вярно, че $f(n) \rhd g(n)$, то съществува релация измежду петте, да я запишем като $\lhd$, такава че $g(n) \lhd f(n)$.

Съгласно Теорема 12(2.38), асимптотично несравними функции съществуват.

Определение 18: Асимптотично по-бавно и асимптотично по-бързо нарастване

Ако $f(n) \prec g(n)$, казваме, че $f(n)$ *расте асимптотично по-бавно от* $g(n)$. Ако $\phi(n) \succ \psi(n)$, казваме, че $\phi(n)$ *расте асимптотично по-бързо от* $\psi(n)$.

Заради транспонираната симетрия (вж. Следствие 6) на $\{\prec, \succ\}$ имаме право освен това да кажем, че $g(n)$ расте асимптотично по-бързо от $f(n)$ и че $\psi(n)$ расте асимптотично по-бавно от $\phi(n)$.

Лема 5 и Определение 15 влекат непосредствено Следствие 4.

Следствие 4

За всички функции $f(n)$ и $g(n)$, $f(n) \asymp g(n) \leftrightarrow f(n) \preceq g(n) \wedge f(n) \succeq g(n)$.

Лема 6 и Определение 16 влекат непосредствено Следствие 5.

Следствие 5: Несъвместими релации на асимптотични сравнения

Не съществуват функции $f(n)$ и $g(n)$, такива че:

$$f(n) \prec g(n) \wedge f(n) \succ g(n)$$
$$f(n) \preceq g(n) \wedge f(n) \succ g(n)$$
$$f(n) \prec g(n) \wedge f(n) \succeq g(n)$$

Лема 7 и Определение 16 влекат непосредствено Следствие 6.

Следствие 6: Транспонирана симетрия

За всички функции $f(n)$, $g(n)$:

$$f(n) \preceq g(n) \leftrightarrow g(n) \succeq f(n)$$
$$f(n) \prec g(n) \leftrightarrow g(n) \succ f(n)$$

Лема 10 е точно съответствие на Лема 8:

Лема 10

За всички функции $f(n)$, $g(n)$:

$$f(n) \prec g(n) \leftrightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$
$$f(n) \succ g(n) \leftrightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

Лема 11

За всеки две функции $f(n)$ и $g(n)$:

$$f(n) \prec g(n) \rightarrow f(n) \preceq g(n) \tag{2.26}$$
$$f(n) \succ g(n) \rightarrow f(n) \succeq g(n) \tag{2.27}$$

Доказателство: Доказателството е тривиално и в двата случая. За (2.26), ако за всяко положително c и всички достатъчно големи стойности на аргумента n е вярно, че $f(n) < c \cdot g(n)$, то очевидно съществува положително c, такова че за всички достатъчно големи стойности на аргумента n: $f(n) \leqslant c \cdot g(n)$. За (2.27) доказателството е аналогично. □

Лема 12

Не е вярно, че за всеки две функции $f(n)$ и $g(n)$:

$$f(n) \preceq g(n) \rightarrow f(n) \prec g(n) \tag{2.28}$$
$$f(n) \succeq g(n) \rightarrow f(n) \succ g(n) \tag{2.29}$$

Доказателство: Като контрапример и за (2.28), и за (2.29) да разгледаме $f(n) = n$ и $g(n) = n$. Очевидно $f(n) \preceq g(n)$ и $f(n) \succeq g(n)$, но $f(n) \not\prec g(n)$ и $f(n) \not\succ g(n)$. □

Строгите релации $\prec$ и $\succ$ са несъвместими съответно със $\succeq$ и $\preceq$.

Лема 13

За всеки две функции $f(n)$ и $g(n)$:

$$f(n) \succeq g(n) \rightarrow f(n) \not\prec g(n) \tag{2.30}$$
$$f(n) \prec g(n) \rightarrow f(n) \not\succeq g(n) \tag{2.31}$$
$$f(n) \preceq g(n) \rightarrow f(n) \not\succ g(n) \tag{2.32}$$
$$f(n) \succ g(n) \rightarrow f(n) \not\preceq g(n) \tag{2.33}$$

Доказателство: Всяка от тези импликации се доказва тривиално с допускане на противното. Да разгледаме само (2.30). Противното е

$$\begin{aligned}
&\neg(f(n) \succeq g(n) \rightarrow f(n) \not\prec g(n)) \equiv && \text{(* свойства на импликацията *)}\\
&\neg(\neg(f(n) \succeq g(n)) \vee \neg(f(n) \prec g(n))) \equiv && \text{(* з-н на De Morgan, з-н за дв. отриц. *)}\\
&f(n) \succeq g(n) \wedge f(n) \prec g(n)
\end{aligned}$$

Но от Следствие 5 знаем, че такива $f(n)$ и $g(n)$ няма. □

Строгите релации $\prec$ и $\succ$ са несъвместими с асимптотичната еквивалентност $\asymp$.

Лема 14

За всеки две функции $f(n)$ и $g(n)$:

$$f(n) \asymp g(n) \rightarrow f(n) \not\prec g(n) \tag{2.34}$$
$$f(n) \prec g(n) \rightarrow f(n) \not\asymp g(n) \tag{2.35}$$
$$f(n) \asymp g(n) \rightarrow f(n) \not\succ g(n) \tag{2.36}$$
$$f(n) \succ g(n) \rightarrow f(n) \not\asymp g(n) \tag{2.37}$$

Доказателство: Следват от Лема 13 и Следствие 4. Да разгледаме само (2.34). От Следствие 4 имаме $f(n) \asymp g(n) \rightarrow f(n) \succeq g(n)$. От Лема 13 имаме $f(n) \succeq g(n) \rightarrow f(n) \not\prec g(n)$. Прилагаме правилото хипотетичен силогизъм от съждителната логика $p \rightarrow q \wedge q \rightarrow r \vdash p \rightarrow r$ и получаваме желания резултат. □

Въведохме пет релации на асимптотични сравнения на функции. От най-общи комбинаторни съображения има не повече от $2^5 = 32$ възможности тези релации да са или да не са в сила за произволни функции. Теорема 12 казва, че тези възможности са точно шест.

Теорема 12: Различните възможности при асимптотично сравняване на функции

За всички функции $f(n)$ и $g(n)$ в сила е точно едно от следните:

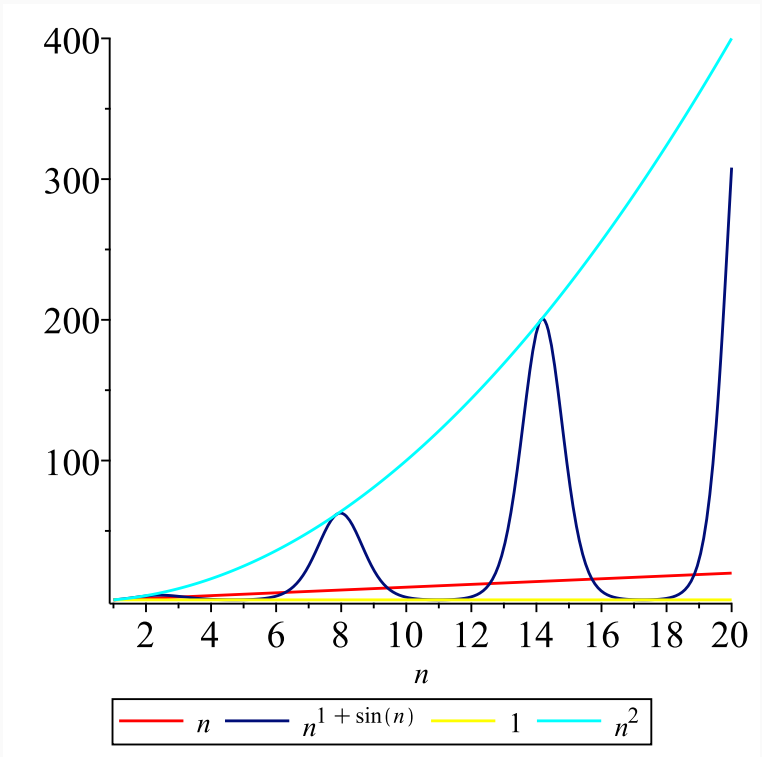
$$f(n) \not\prec g(n) \wedge f(n) \not\preceq g(n) \wedge f(n) \not\asymp g(n) \wedge f(n) \not\succ g(n) \wedge f(n) \not\succeq g(n) \tag{2.38}$$
$$f(n) \not\prec g(n) \wedge f(n) \preceq g(n) \wedge f(n) \not\asymp g(n) \wedge f(n) \not\succ g(n) \wedge f(n) \not\succeq g(n) \tag{2.39}$$
$$f(n) \prec g(n) \wedge f(n) \preceq g(n) \wedge f(n) \not\asymp g(n) \wedge f(n) \not\succ g(n) \wedge f(n) \not\succeq g(n) \tag{2.40}$$
$$f(n) \not\prec g(n) \wedge f(n) \preceq g(n) \wedge f(n) \asymp g(n) \wedge f(n) \not\succ g(n) \wedge f(n) \succeq g(n) \tag{2.41}$$
$$f(n) \not\prec g(n) \wedge f(n) \not\preceq g(n) \wedge f(n) \not\asymp g(n) \wedge f(n) \not\succ g(n) \wedge f(n) \succeq g(n) \tag{2.42}$$
$$f(n) \not\prec g(n) \wedge f(n) \not\preceq g(n) \wedge f(n) \not\asymp g(n) \wedge f(n) \succ g(n) \wedge f(n) \succeq g(n) \tag{2.43}$$

Доказателство: Първо ще покажем по една двойка функции като пример за всяка от тези шест възможности. После ще покажем, че останалите комбинации са невъзможни.

(2.38): Този случай се характеризира с това, че $f(n) \not\preceq g(n)$ и $f(n) \not\succeq g(n)$, понеже е достатъчно и $\preceq$, и $\succeq$ да не са в сила, за да не е в сила нито една от останалите три релации. Това се доказва лесно с негацията на Следствие 4 и контрапозитивните на твърденията от Лема 11.

Следният пример за такива функции е от [21, стр. 52]. Нека $f(n) = n$ и $g(n) = n^{1+\sin n}$. Разгледайте Фигура 2.3[†], която илюстрира графиките на $f(n)$ и $g(n)$, както и на функциите 1 и n^2. Тъй като $1 + \sin n$ осцилира между 0 и 2, то $n^{1+\sin n}$ осцилира между 1 и n^2[‡]. Ясно е, че за всяка константа $c > 0$ и за всяка стойност на аргумента n_0 съществува $n' > n_0$, такова че $f(n') < c \cdot g(n')$ и съществува $n'' > n_0$, такова че $f(n'') > c \cdot g(n'')$. Следователно, нито една от релациите $\preceq$ и $\succeq$ не е в сила.

Фигура 2.3 : Графиките на n, $n^{1+\sin n}$, 1 и n^2.



(2.39): Нека $f(n) = n^{1+\sin n}$ и $g(n) = n^2$ (Фигура 2.3 илюстрира и тези разсъждения). Вярно е, че $f(n) \preceq g(n)$: да вземем $n_0 = 1$ и $c = 1$.

От друга страна, $f(n) \not\prec g(n)$, понеже съществува константа $c > 0$, такава че за всяка стойност на аргумента n_0 съществува $n' > n_0$, такова че $f(n') \geqslant c \cdot g(n')$, например, $c = 1$.

Също така е вярно, че $f(n) \not\succeq g(n)$, понеже за всяка константа $c > 0$ и за всяка стойност на аргумента n_0 съществува $n' > n_0$, такова че $f(n') < c \cdot g(n')$.

От това, че $f(n) \preceq g(n)$ и $f(n) \not\succeq g(n)$ следва, че $f(n) \neq g(n)$ съгласно Следствие 4.

И накрая, $f(n) \not\succ g(n)$, което следва от това, че $f(n) \preceq g(n)$ и Лема 13.

[†]Фигура 2.3 е направена с Maple(TM).

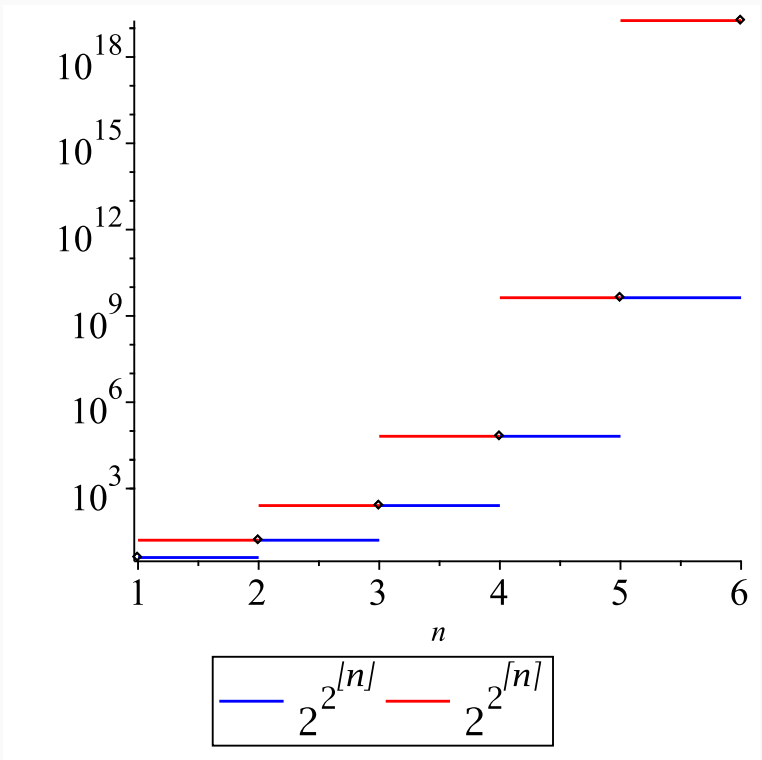
[‡]В този случай осцилирането е в степенния показател, а не в множител, както беше в примера на Фигура 2.2, така че тези функции не са Тита една от друга, за разлика от функциите, показани на Фигура 2.2.

Друга двойка функции, която е подходящ пример, е $f(n) = 2^{2^{\lfloor n \rfloor}}$ и $g(n) = 2^{2^{\lceil n \rceil}}$. Ако $n \in \mathbb{N}$, то $f(n) = g(n)$. Ако обаче $n \in \mathbb{R}^+ \backslash \mathbb{N}$, $f(n) = \sqrt{g(n)}$, понеже

$$n \in \mathbb{R}^+ \backslash \mathbb{N} \ \rightarrow \ \lfloor n \rfloor = \lceil n \rceil - 1 \ \rightarrow \ 2^{\lfloor n \rfloor} = 2^{\lceil n \rceil - 1} \ \rightarrow \ 2^{2^{\lfloor n \rfloor}} = 2^{2^{\lceil n \rceil - 1}} = 2^{\frac{1}{2} 2^{\lceil n \rceil}} = \sqrt{2^{2^{\lceil n \rceil}}}$$

Фигура 2.4[†] показва графиките на тези две функции. Функциите се изравняват върху всяко цяло число. След това, в отворения интервал до следващото цяло число, $f(n) = 2^{2^{\lfloor n \rfloor}}$ остава същата, а $2^{2^{\lceil n \rceil}}$ взема стойност, която е предната ѝ стойност, повдигната на квадрат. Фигурата е с логаритмичен мащаб по ординатата поради изключително бързото нарастване на двойната експонента.

Фигура 2.4 : Графиките на $2^{2^{\lfloor n \rfloor}}$ и $2^{2^{\lceil n \rceil}}$.



(2.40): Пример за такива функции са $f(n) = n$ и $g(n) = n^2$. Това, че $f(n) \prec g(n)$, $f(n) \preceq g(n)$, $f(n) \neq g(n)$, $f(n) \not\succ g(n)$ и $f(n) \not\succeq g(n)$, е очевидно.

(2.41): Пример за такива функции са $f(n) = n$ и $g(n) = n + 1$. Това, че $f(n) \not\prec g(n)$, $f(n) \preceq g(n)$, $f(n) \asymp g(n)$, $f(n) \not\succ g(n)$ и $f(n) \succeq g(n)$, е очевидно.

(2.42): Двата примера от (2.39), но с разменени $f(n)$ и $g(n)$, са валидни за този случай.

(2.43): Пример за такива функции са $f(n) = n^2$ и $g(n) = n$. □

Сега да разгледаме невъзможните комбинации. Всички комбинации с поне едно от следните:

$f(n) \succ g(n) \wedge f(n) \not\succeq g(n)$
$f(n) \prec g(n) \wedge f(n) \not\preceq g(n)$

[†]Фигура 2.4 е направена с Maple(TM).

са невъзможни, защото $\succ$ влече $\succeq$ и $\prec$ влече $\preceq$ (Лема 11). Всички комбинации с поне едно от следните:

$$f(n) \asymp g(n) \wedge f(n) \not\asymp g(n)$$
$$f(n) \not\asymp g(n) \wedge f(n) \asymp g(n)$$
$$f(n) \preceq g(n) \wedge f(n) \not\prec g(n) \wedge f(n) \succeq g(n)$$

са невъзможни, защото $\asymp$ е в сила тогава и само тогава, когато и $\succeq$, и $\preceq$ са в сила (Следствие 4). И накрая, всички комбинации с поне едно от следните:

$$f(n) \prec g(n) \wedge f(n) \succeq g(n)$$
$$f(n) \preceq g(n) \wedge f(n) \succ g(n)$$

са невъзможни, защото $\prec$ и $\succeq$ са несъвместими, а също така $\preceq$ и $\succ$ са несъвместими (Следствие 5). □

Теорема 13: Повдигането на степен запазва отношението $\asymp$

За всеки две функции $f(n)$ и $g(n)$ и за всяка константа $k \in \mathbb{R}^+$:

$$f(n) \asymp g(n) \leftrightarrow (f(n))^k \asymp (g(n))^k$$

Доказателство: В едната посока, нека

$$c_1 g(n) \leqslant f(n) \leqslant c_2 g(n)$$

за някакви положителни константи c_1 и c_2 и за всички $n \geqslant n_0$, за някакво $n_0 > 0$. Повдигаме двете неравенства на k-та степен (k е положително) и получаваме

$$c_1^k (g(n))^k \leqslant (f(n))^k \leqslant c_2^k (g(n))^k, \text{ за всяко } n \geqslant n_0$$

Но тогава $(f(n))^k \asymp (g(n))^k$, понеже c_1^k и c_2^k са положителни константи.

В другата посока, нека

$$c_1 (g(n))^k \leqslant (f(n))^k \leqslant c_2 (g(n))^k, \text{ за всяко } n \geqslant n_0$$

за някакви положителни константи c_1 и c_2 и за всички $n \geqslant n_0$, за някакво $n_0 > 0$. Повдигаме двете неравенства на $\frac{1}{k}$-та степен ($\frac{1}{k}$ е положително) и получаваме

$$\sqrt[k]{c_1} g(n) \leqslant f(n) \leqslant \sqrt[k]{c_2} g(n), \text{ за всяко } n \geqslant n_0$$

Но тогава $f(n) \asymp g(n)$, понеже $\sqrt[k]{c_1}$ и $\sqrt[k]{c_2}$ са положителни константи. □

Теорема 14: Водещото събираемо определя асимптотиката

Нека $f(n) = g(n) + h(n)$, където $h(n) \prec g(n)$. Тогава $f(n) \asymp g(n)$.

Доказателство: Първо ще покажем, че

$$\exists c_1 \, \exists n' \, \forall n \geqslant n' : c_1 \cdot g(n) \leqslant f(n) \tag{2.44}$$
$$\exists c_2 \, \exists n'' \, \forall n \geqslant n'' : f(n) \leqslant c_2 \cdot g(n) \tag{2.45}$$

(2.44) е лесно. Тъй като $f(n)$, $g(n)$ и $h(n)$ са положителни, то $f(n) = g(n) + h(n)$ влече $\forall n : f(n) > g(n)$, което влече $\forall n \geqslant 1 : (1 \cdot g(n) \leqslant f(n))$. Доказахме (2.44) с $c_1 = 1$ и $n' = 1$.

Разглеждаме (2.45). Използваме това, че $h(n) \prec g(n)$, което е кратък запис за

$$\forall c > 0\ \exists n_0\ \forall n > n_0 : h(n) < c \cdot g(n)$$

В частност, за $c_2 = 1$ е вярно, че има n_0, такова че $\forall n \geqslant n_0 : h(n) < 1 \cdot g(n)$. Но, тъй като $f(n) = g(n) + h(n)$, то $\forall n \geqslant n_0 : f(n) \leqslant g(n) + g(n)$. С други думи, $f(n) \leqslant 2g(n)$ за всички n, надхвърлящи n_0. Тогава за нашето доказателство вземаме $c_2 = 2$ и $n'' = n_0$. Доказахме (2.45).

От (2.44) и (2.45) следва, че

$$\exists c', c''\ \exists \hat{n}\ \forall n \geqslant \hat{n} : c' \cdot g(n) \leqslant f(n) \leqslant c'' \cdot g(n)$$

ако вземем $c' = c_1$, $c'' = c_2$ и $\hat{n} = \max\{n', n''\}$. □

С Теорема 14 веднага доказваме, че всеки полином с положителни коефициенти[†] има асимптотика, зависеща само от степента му.

Следствие 7

Нека $p(n)$ е произволен полином от степен k с положителни коефициенти. Тогава $p(n) \asymp n^k$.

Доказателство: Нека $p(n) = a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0$. Но $a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 \prec p(n)$, понеже $\lim_{n\to\infty} \frac{a_{k-1} n^{k-1} + \cdots + a_1 n + a_0}{a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0} = 0$. Сега прилагаме Теорема 14. □

Теорема 15 казва, че експоненциалната трансформация върху две функции "изостря" асимптотичните разлики в нарастването им, ако функциите са неограничено нарастващи и имат различно асимптотично нарастване.

Теорема 15: Експонирането запазва отношението $\prec$

Нека $f(n)$ и $g(n)$ са растящи и неограничени функции и нека a е константа, по-голяма от 1. Тогава

$$f(n) \prec g(n) \ \rightarrow\ a^{f(n)} \prec a^{g(n)}$$

Доказателство: Трябва да докажем, че:

$$\forall c > 0\ \exists n'\ \forall n \geqslant n' : 0 \leqslant a^{f(n)} < c.a^{g(n)} \tag{2.46}$$

Да разгледаме $k = \log_a c$. Тоест, $c = a^k$. Да препишем (2.46) така:

$$\forall k\ \exists n'\ \forall n \geqslant n' : 0 \leqslant a^{f(n)} < a^k a^{g(n)} \tag{2.47}$$

Вземаме логаритъм при основа a от двете неравенства и получаваме:

$$\forall k\ \exists n'\ \forall n \geqslant n' : 0 \leqslant f(n) < k + g(n) \tag{2.48}$$

[†]Да настояваме за положителни коефициенти е прекалено рестриктивно – достатъчно е коефициентът пред n^k да е положителен. Но, ако допуснем само това, може да имаме формални проблеми, тъй като останалата част на полинома може да е отрицателна функция и тогава, чисто формално, нямаме право да ползваме асимптотичните релации при текущите дефиниции.

Трябва да докажем (2.48). По условие имаме:

$$\forall c > 0\ \exists n_0\ \forall n \geqslant n_0 : 0 \leqslant f(n) < c.g(n)$$

Щом това е в сила за всяко $c > 0$, в частност то е в сила за $c = \frac{1}{2}$. И така:

$$\exists n_0\ \forall n \geqslant n_0 : 0 \leqslant f(n) < \frac{g(n)}{2} \tag{2.49}$$

Но $g(n)$ е растяща и неограничена. Тогава:

$$\forall k\ \exists n_1\ \forall n \geqslant n_1 : 0 < k + \frac{g(n)}{2} \tag{2.50}$$

Да препишем (2.50) така:

$$\forall k\ \exists n_1\ \forall n \geqslant n_1 : \frac{g(n)}{2} < k + g(n) \tag{2.51}$$

От (2.49) и (2.51) заключаваме, че:

$$\forall k\ \exists n''\ \forall n \geqslant n'' : 0 \leqslant f(n) < k + g(n) \tag{2.52}$$

Но (2.52) е същото като (2.48). С това доказателството е готово. □

Следствие 8

Нека $f(n)$ и $g(n)$ са растящи и неограничени функции. Тогава

$$\lg f(n) \prec \lg g(n) \ \rightarrow\ f(n) \prec g(n)$$

Теорема 16 в някакъв смисъл е конверсна на Теорема 15. Според Теорема 16, ако образите на две нарастващи и неограничени функции след експоненциална трансформация са асимптотично еквивалентни, то и самите функции са асимптотично еквивалентни.

Теорема 16: Логаритмуването запазва отношението $\asymp$

Нека $f(n)$ и $g(n)$ са растящи и неограничени функции и нека a е константа, по-голяма от 1. Тогава

$$a^{f(n)} \asymp a^{g(n)} \rightarrow f(n) \asymp g(n)$$

Доказателство: Нека

$$\exists c_1 > 0\ \exists c_2 > 0\ \exists n_0 > 0\ \forall n > n_0 : c_1 \cdot a^{g(n)} \leqslant a^{f(n)} \leqslant c_2 \cdot a^{g(n)}$$

Щом $a > 1$ и $f(n)$ и $g(n)$ са положителни, то за всички достатъчно големи n е вярно, че експонентите са строго по-големи от единица. Тогава вземаме логаритъм при основа a от двете страни и получаваме:

$$\log_a c_1 + g(n) \leqslant f(n) \leqslant \log_a c_2 + g(n)$$

Забелязваме, че щом $g(n)$ е нарастваща и неограничена, то за всяка константа k_1, такава че $0 < k_1 < 1$, в сила е $k_1 \cdot g(n) \leqslant \log_a c_1 + g(n)$, за всички достатъчно големи n. И това е така

независимо от това, дали логаритъмът е положителен или отрицателен или е нула. После забелязваме, че щом $g(n)$ е нарастваща и неограничена, то за всяка константа k_2, такава че $k_2 > 1$, в сила е $\log_a c_2 + g(n) \leqslant k_2.g(n)$ за всички достатъчно големи n. И това е така независимо от това, дали логаритъмът е положителен или отрицателен или е нула.

Заключаваме, че съществува n_1, такова че:

$$\forall n \geqslant n_1 : k_1 \cdot g(n) \leqslant f(n) \leqslant k_2 \cdot g(n)$$

Но тогава $f(n) \asymp g(n)$. □

Следствие 9

Нека $f(n)$ и $g(n)$ са растящи и неограничени функции. Тогава

$$f(n) \asymp g(n) \rightarrow \lg f(n) \asymp \lg g(n)$$

Определение 19: Полилогаритмично растяща функция

За всяка $f(n)$, такава че $f(n) \asymp (\log_a n)^k$, където $a, k \in \mathbb{R}^+$ са константи и $a > 1$, казваме, че $f(n)$ *расте полилогаритмично.*

Определение 20: Полиномиално растяща функция

За всяка $f(n)$, такава че $f(n) \asymp n^\epsilon$, където $\epsilon \in \mathbb{R}^+$ е константа, казваме, че $f(n)$ *расте полиномиално.*

Забележете, че в математиката е прието полиномите да имат цели степенни показатели, така че функцията $f(n) = n^{\frac{1}{2}}$ не е полиномиална. Но съгласно Определение 20, $f(n) = n^{\frac{1}{2}}$ също расте полиномиално.

Определение 21: Експоненциално растяща функция

За всяка $f(n)$, такава че $f(n) \asymp a^{n^k}$, където $a, k \in \mathbb{R}^+$ са константи и $a > 1$, казваме, че $f(n)$ *расте експоненциално.*

Когато чуем "експоненциална функция", обикновено си представяме 2^n, e^n и изобщо a^n за някое $a > 1$. Съгласно Определение 21, функции като 2^{n^2} и 3^{n^5} и $(\sqrt{2})^{n^{\sqrt{2}}}$ също са експоненциално растящи функции.

Теорема 17: Полиномиалното нарастване изпреварва полилогаритмичното

Нека $a, k, \epsilon \in \mathbb{R}^+$ са произволни константи, като $a > 1$. В сила е:

$$(\log_a n)^k \prec n^\epsilon$$

Доказателство:

$$\lim_{n\to\infty} \frac{n^\epsilon}{(\log_a n)^k} = \qquad // \text{ полагаме } b \leftarrow \frac{\epsilon}{k}$$
$$\lim_{n\to\infty} \frac{(n^b)^k}{(\log_a n)^k} =$$
$$\lim_{n\to\infty} \left(\frac{n^b}{\log_a n}\right)^k = \qquad // \text{ } k \text{ е положително}$$
$$\lim_{n\to\infty} \frac{n^b}{\log_a n} = \qquad // \text{ прилагаме правилото на l'Hôpital}$$
$$\lim_{n\to\infty} \frac{bn^{b-1}}{\left(\frac{1}{\ln a}\right)\left(\frac{1}{n}\right)} =$$
$$\lim_{n\to\infty} (\ln a) b\, n^b = \infty$$

□

Теорема 18: Експоненциалното нарастване изпреварва полиномиалното

Нека $a, \epsilon \in \mathbb{R}^+$ са произволни константи, като $a > 1$. В сила е:

$$n^\epsilon \prec a^n$$

Доказателство: Да вземем $\log_a$ от двете страни. Лявата страна става $\epsilon \cdot \log_a n$ и дясната страна става n. Но $\cdot \log_a n \prec n$ съгласно Теорема 17. Тогава $\epsilon \cdot \log_a n \prec n$. Прилагаме Следствие 8 и получаваме желания резултат. □

Следствие 10

Нека $a, \epsilon, k \in \mathbb{R}^+$ са произволни константи, като $a > 1$. В сила е:

$$n^\epsilon \prec a^{n^k}$$

От Теорема 17 и Следствие 10 веднага заключаваме следното.

Следствие 11

Всяка полилогаритмично растяща функция расте асимптотично по-бавно от произволна полиномиално растяща функция, а всяка полиномиално растяща функция расте асимптотично по-бавно от произволна експоненциално растяща функция.

2.3.6 Релациите $\preceq$ и $\succeq$ като преднаредби

Както видяхме в Лема 9, $\asymp$ е релация на еквивалентност, подобно на релацията на равенство върху числата. Да разгледаме $\preceq$ и $\succeq$. Те не са антисиметрични, защото може за различни функции $f(n)$ и $g(n)$ да е изпълнено $f(n) \preceq g(n)$ и $g(n) \preceq f(n)$, примерно $n^2 \preceq n^2 + n$ и $n^2 + n \preceq n^2$. Следователно $\preceq$ и $\succeq$ не са релации на частична наредба, а са по-общите **релации на преднаредба**. Освен това, както вече видяхме в доказателството на Теорема 12(2.38), $\preceq$ и $\succeq$ не са пълни, защото може за различни функции $f(n)$ и $g(n)$ да не е изпълнено нито $f(n) \preceq g(n)$, нито $g(n) \preceq f(n)$. Следният допълнителен материал съдържа достатъчно, за да бъде осмислено това.

Допълнение 16: За преднаредбите

В това допълнение ще обясним в детайли понятието *преднаредба*, което не се изучава в курса по Дискретна Математика, но е важно за осмислянето на релациите $\preceq$ и $\succeq$. Да си припомним някои дефиниции за релациите по принцип. Сравнете това изложение с изложението в [76, стр. 49–50].

Определение 22: Видови релации

Нека A е произволно множество и $R \subseteq A \times A$ е някаква бинарна релация над него. Казваме, че:

- R е *рефлексивна*, ако $\forall a \in A : aRa$.
- R е *ирефлексивна*, още наречена *стриктна*, ако $\forall a \in A : \neg aRa$.
- R е *симетрична*, ако $\forall a, b \in A : aRb \to bRa$.
- R е *антисиметрична*, ако $\forall a, b \in A : aRb \wedge bRa \to a = b$.
- R е *пълна*, ако $\forall a, b \in A : a \neq b \to aRb \vee bRa$.
- R е *транзитивна*, ако $\forall a, b, c \in A : aRb \wedge bRc \to aRc$.
- R е *преднаредба*, още наричана *квази-наредба*[a], ако е рефлексивна и транзитивна.
- R е *строга преднаредба*, ако е ирефлексивна и транзитивна.
- R е *частична наредба*, ако е антисиметрична преднаредба.
- R е *линейна наредба*, ако е пълна частична наредба.
- R е *релация на еквивалентност*, ако е симетрична преднаредба.

Казваме, че R' е *рефлексивното затваряне на* R, ако R' е най-малката релация, такава че $R \subseteq R'$ и R' е рефлексивна. Казваме, че R' е *симетричното затваряне на* R, ако R' е най-малката релация, такава че $R \subseteq R'$ и R' е симетрична. Казваме, че R' е *транзитивното затваряне на* R, ако R' е най-малката релация, такава че $R \subseteq R'$ и R' е транзитивна.

[a] На английски, *preorder* или *quasi-oder*.

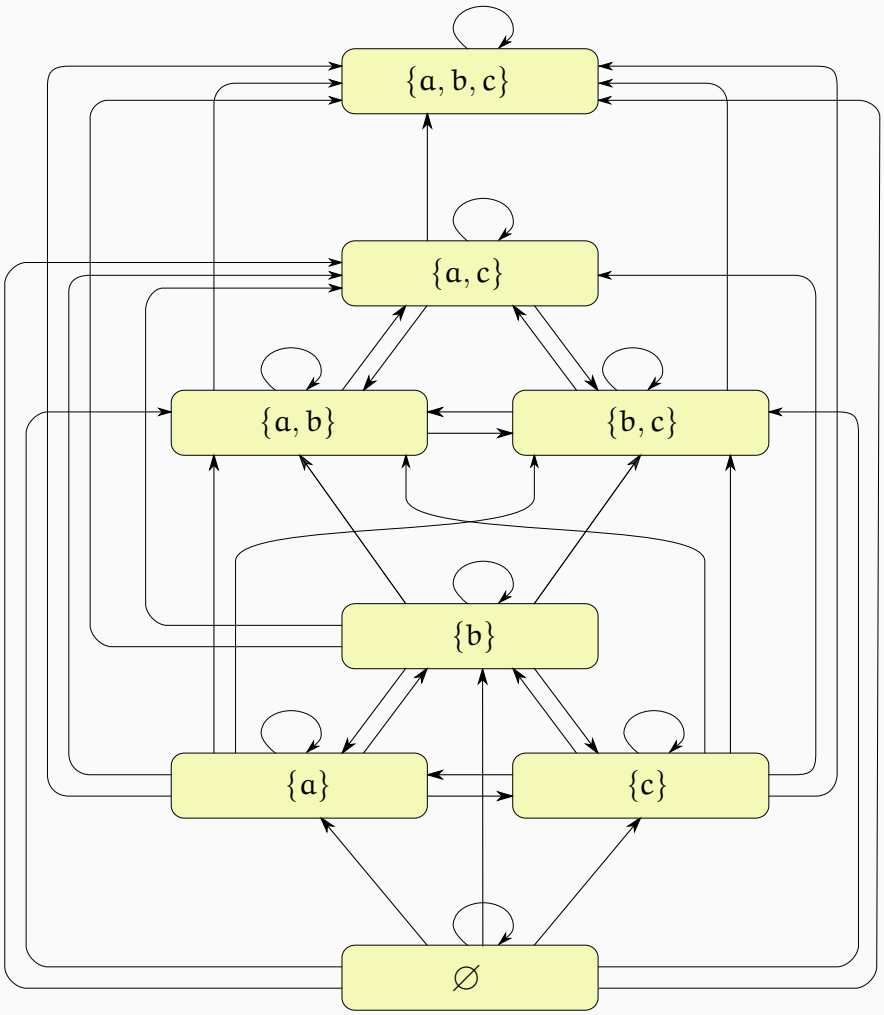
Пример за "истинска" преднаредба. Да разгледаме един пример за преднаредба, която не е нито релация на еквивалентност, нито на частична наредба; тоест, "истинска" преднаредба. Нека S е крайно множество. Нека $\mathcal{R}' \subseteq 2^S \times 2^S$ е дефинирана така:

$$\forall X \in 2^S \; \forall Y \in 2^S : (X, Y) \in \mathcal{R}' \leftrightarrow |X| \leqslant |Y| \tag{2.53}$$

Очевидно $\mathcal{R}'$ е рефлексивна и транзитивна – това следва от свойствата на релацията $\leqslant$ върху естествените числа. Забележете, че $\mathcal{R}'$ не е нито симетрична, нито антисиметрична. Нека $S = \{a, b, c\}$. Тъй като, примерно, $(\{a, b\}, \{a, b, c\}) \in \mathcal{R}'$ и $(\{a, b, c\}, \{a, b\}) \notin \mathcal{R}'$,

то $\mathcal{R}'$ не е симетрична. От друга страна, тъй като, примерно, $(\{a, b\}, \{b, c\}) \in \mathcal{R}'$ и $(\{b, c\}, \{a, b\}) \in \mathcal{R}'$, то $\mathcal{R}'$ не е антисиметрична. Така че $\mathcal{R}'$ е "истинска" преднаредба. Диаграмата на $\mathcal{R}'$ при $S = \{a, b, c\}$ е показана на Фигура 2.5. Забележете, че $\mathcal{R}'$ е и пълна релация, защото за всеки две подмножества на S, поне за едното е вярно, че има мощност поне колкото другото.

Фигура 2.5 : Диаграма на преднаредбата $\mathcal{R}'$ от (2.53).



Следното определение и лема са Lemma 1 в книгата на Birkhoff [13, стр. 21].

Определение 23: Фактор-релация

Нека A е произволно множество и $R \subseteq A \times A$ е някаква преднаредба над него. Нека $\simeq \subseteq A \times A$[а] е следната релация: $\forall a, b \in A : a \simeq b \leftrightarrow aRb \wedge bRa$. Очевидно $\simeq$ е релация на еквивалентност над A. При това, ако E и F са два класа на еквивалентност на $\simeq$, тогава или $\forall x \in E \, \forall y \in F : xRy$, или $\forall x \in E \, \forall y \in F : \neg xRy$. Нека $\mathfrak{X}$ е множеството от класовете на еквивалентност на $\simeq$. *Фактор-релацията спрямо* R[б] е релацията $\mathfrak{S} \subseteq \mathfrak{X} \times \mathfrak{X}$, дефинирана така:

$$\forall P, Q \in \mathfrak{X} : (P, Q) \in \mathfrak{S} \leftrightarrow \exists a \in P \, \exists b \in Q : aRb$$

[а] Оригиналният символ е $\sim$, а не $\simeq$. Тук ползваме $\simeq$, за да няма объркване с релацията за асимптотична еквивалентност от Подсекция 2.3.11.

[б] В книгата на Birkhoff се използва терминът *quotient set*.

Лема 15

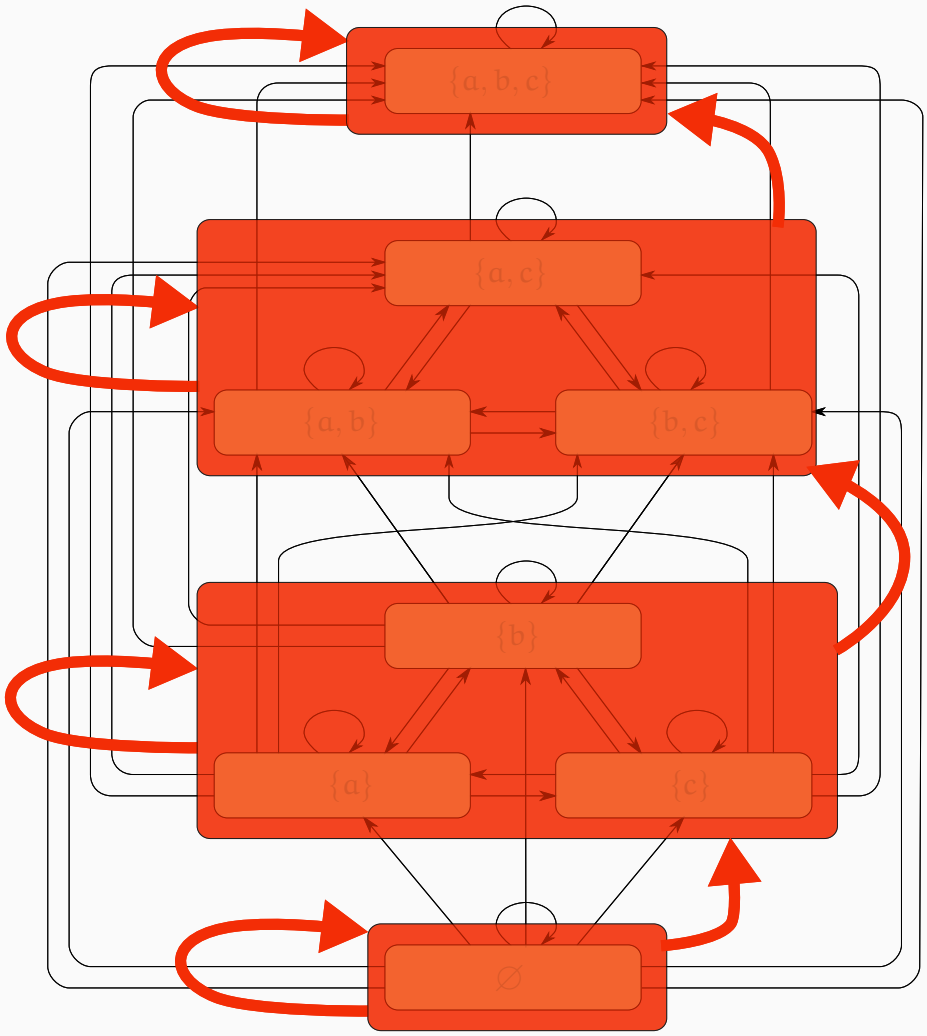
Фактор-релацията е релация на частична наредба.

Нотация 1: Фактор-релацията $R/\simeq$

Означаваме фактор-релацията с $R/\simeq$.

Лесно се вижда, че всяка преднаредба $R \subseteq A \times A$ е пълна тогава и само тогава, когато $R/\simeq$ е линейна наредба (върху класовете на еквивалентост на $\simeq$). Примерно, релацията $\mathcal{R}'$, дефинирана в (2.53) е пълна преднаредба. Фигура 2.6 показва $R'/\simeq$, която очевидно е линейна.

Фигура 2.6 : Фактор-релацията $R'/_{\simeq}$ е линейна. Класовете на еквивалентност на $\simeq$ са очертани с полупрозрачни червени кутии, а диаграмата на $R'/_{\simeq}$ е нарисувана върху тях с червени стрелки.



Пример за преднаредба, която не е пълна. Нека B е следното множество от тримерни вектори от естествени числа:

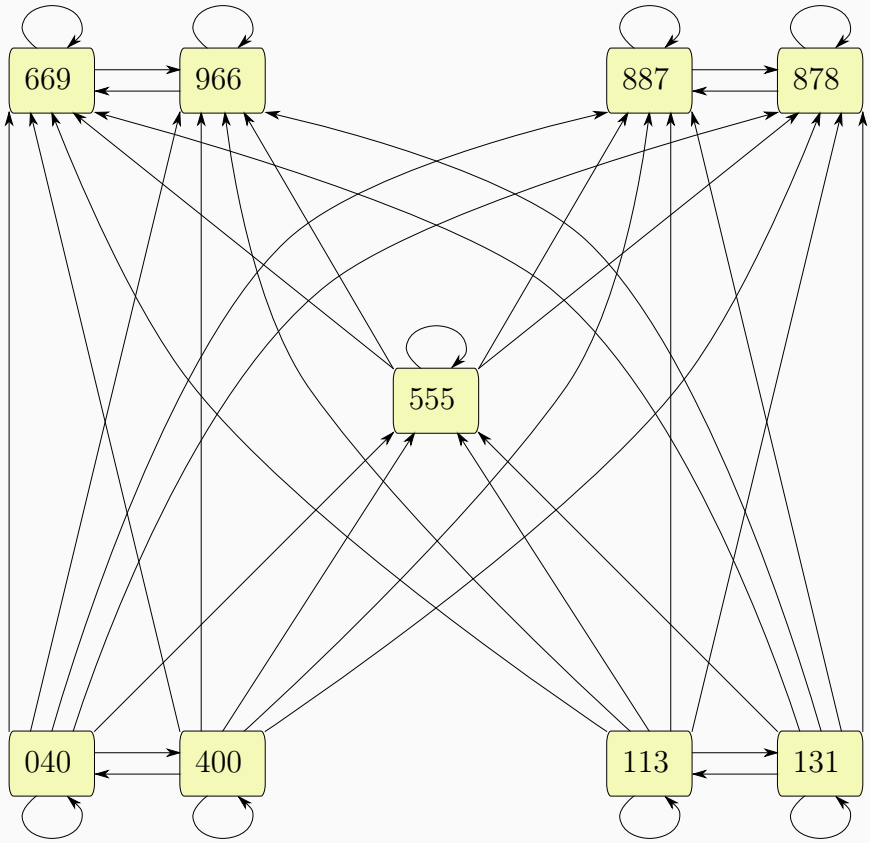
$$B = \{040, 400, 113, 131, 555, 779, 977, 886, 868\}$$

Нека Q е следната релация над B:

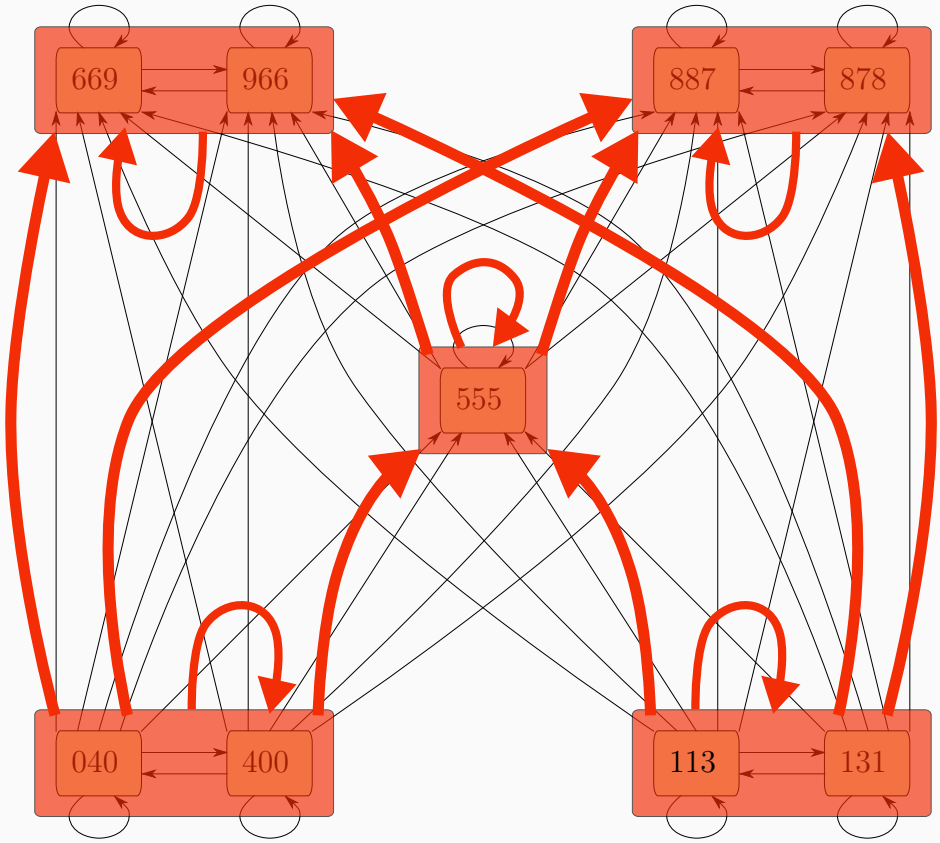
$$\forall \mathbf{x}, \mathbf{y} \in B : (\mathbf{x}, \mathbf{y}) \in Q \quad \leftrightarrow \quad \sum \mathbf{x} = \sum \mathbf{y} \text{ или } (x_i \leqslant y_i, \text{ за } i \in \{1, 2, 3\})$$

където $\sum \mathbf{x}$ означава сумата от елементите на вектора $\mathbf{x}$ (аналогично и за $\mathbf{y}$), а x_i е i-ият елемент на $\mathbf{x}$ (аналогично и за $\mathbf{y}$). Диаграмата на Q е показана на Фигура 2.7. Фигура 2.8 показва фактор-релацията.

Фигура 2.7 : Преднаредбата $\mathcal{Q}$ не е пълна.



Фигура 2.8 : Фактор-релацията $Q/_{\simeq}$ не е линейна.



Наблюдение 5

Релациите $\prec$ и $\succ$ са ирефлексивни и транзитивни, а $\preceq$ и $\succeq$ са рефлексивни и транзитивни. Тогава $\prec$ и $\succ$ са строги преднаредби, а $\preceq$ и $\succeq$ са преднаредби върху множеството на функциите.
Преднаредбите $\preceq$ и $\succeq$ не са антисиметрични.
Преднаредбите $\preceq$ и $\succeq$ не са пълни.
Тъй като $\asymp$ е релация на еквивалентност, имаме право да говорим за фактор-релациите $\preceq/\asymp$ и $\succeq/\asymp$. Техните класове на еквивалентност са (неизброимо безкрайните, неизброимо безкрайно на брой) максимални по включване подмножества от функции, които са Тета една от друга.
Фактор-релациите $\preceq/\asymp$ и $\succeq/\asymp$ са "истински" частични наредби, в които има несравними елементи.

2.3.7 Релациите на асимптотични сравнения при $n \in \mathbb{N}^+$

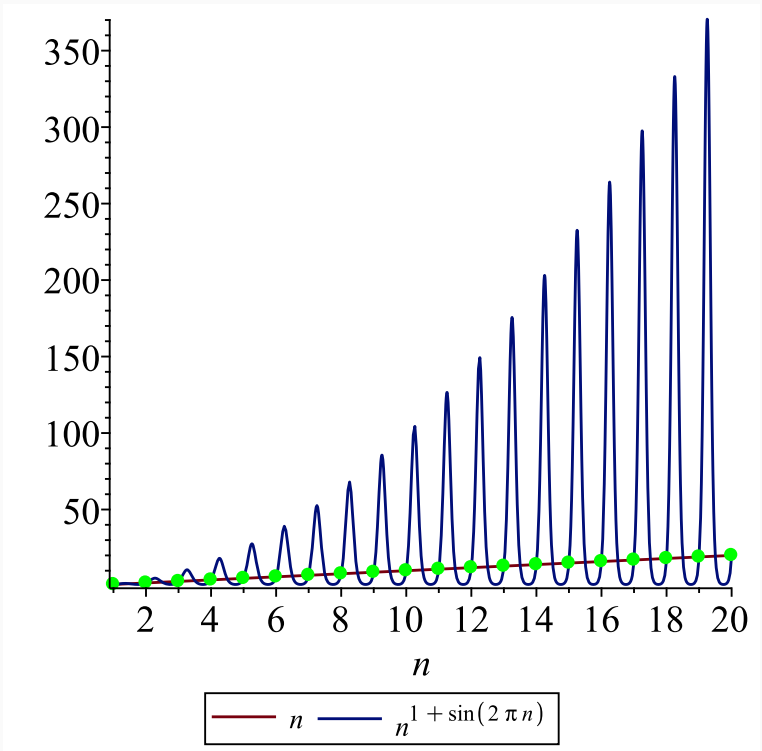
Вече приехме, че разглеждаме функции с домейн $\mathbb{R}^+$ (вж. Конвенция 3). Дали нещо ще се промени, ако преминем към домейн $\mathbb{N}^+$? От гледна точка на алгоритмичния анализ, функциите са с домейн именно $\mathbb{N}^+$, понеже големината на входа е цяло положително число.

При "добре държащи се" функции като n, n^2, 2^n и така нататък, нищо не се променя при преминаване от домейн $\mathbb{R}^+$ към домейн $\mathbb{N}^+$. Примерно, $n^2 \asymp n^2 + n$, $2^n \succ n$, $n2^n \prec 3^n$ и ако ги разглеждаме върху $\mathbb{R}^+$, и ако ги разглеждаме върху $\mathbb{N}^+$.

Дори функцията да може да връща не-цяло число при целочислен аргумент, примерно $\frac{n}{5}$, $\sqrt{n}$ или $\log_2 n$, същото остава в сила, стига функцията да е "добре държаща се". Посочените функции са именно такива, така че, примерно, $\frac{n}{5} \geq \sqrt{n}$ е изпълнено и върху $\mathbb{R}^+$, и върху $\mathbb{N}^+$.

Има обаче "лошо държащи се" функции, при които резултатът от асимптотичното сравнение е чувствителен към избора на домейн ($\mathbb{N}^+$ или $\mathbb{R}^+$). Лесно е да конструираме пример за такава функция. Да си припомним функцията $g(n) = n^{1+\sin n}$ на стр. 85. Както стана ясно там, $f(n) = n$ и $g(n) = n^{1+\sin n}$ са асимптотично несравними. Напълно аналогично, функциите n и $n^{1+\sin 2\pi n}$ са асимптотично несравними, когато $n \in \mathbb{R}^+$, понеже $1 + \sin 2\pi n$ осцилира между 0 и 2, също както $1 + \sin n$. Да разгледаме n и $n^{1+\sin 2\pi n}$, когато n е цяло число. Функцията n, която е "добре държаща се", има същото поведение при $n \in \mathbb{N}^+$. От друга страна, $n^{1+\sin 2\pi n} = n$ сега се държи по съвсем различен начин, понеже $\forall n \in \mathbb{N}^+ : \sin 2\pi n = 0$. Излиза, че върху естествените числа, функциите n и $n^{1+\sin 2\pi n}$ съвпадат. В сила е $n \asymp n^{1+\sin 2\pi n}$, ако $n \in \mathbb{N}^+$. Фигура 2.9 илюстрира всичко това – графиките съвпадат точно върху целите стойности на аргумента.

Фигура 2.9 : Графиките на n и $n^{1+\sin 2\pi n}$.



Друга двойка функции, асимптотичното сравнение на които се променя при целочислен аргумент, е $f(n) = 2^{2^{\lfloor n \rfloor}}$ и $g(n) = 2^{2^{\lceil n \rceil}}$ на стр. 86. Както видяхме там, те съвпадат при $n \in \mathbb{N}^+$.

Ако домейнът е $\mathbb{N}^+$, може да даваме примери за осцилиращи функции не чрез използване на тригонометрични функции, а просто чрез различно дефиниране върху четните и нечетните

числа. Примерно, следните две функции са асимптотично несравними:

$$f(n) = n^2$$

$$g(n) = \begin{cases} n, & \text{ако } n \text{ е четно} \\ n^3, & \text{ако } n \text{ е нечетно} \end{cases}$$

2.3.8 Асимптотични еквивалентности на често срещани функции

Теорема 19: Апроксимация на Stirling, слаба версия

В сила е

$$n! \asymp \sqrt{n}\,\frac{n^n}{e^n}$$

Доказателството на Теорема 19 е дълго и протяжно, ако се извършва без ползването на изтънчен анализ. Поради това го слагаме в Допълнение 17.

Допълнение 17: Апроксимация на Stirling

В сила е следният важен и нетривиален резултат за асимптотиката на факториела. Ползваме релацията $\sim$, дефинирана в Подсекция 2.3.11.

Теорема 20: Апроксимация на Stirling

$$n! \sim \sqrt{2\pi n}\,\frac{n^n}{e^n}$$

Доказателства на Теорема 20 има в статията на Dan Romik [77], както и в *тази експозиционна статия на Keith Conrad*, но тези доказателства изискват задълбочени познания по анализ и тук няма да ги разглеждаме.
Експозиционната статия на Byron Schmuland [79] съдържа резултат, по-слаб от Теорема 20—тя не показва, че мултипликативната константа е именно $\sqrt{2\pi}$—но с по-леко доказателство. Ако искаме само да се убедим, че $n! \asymp \sqrt{n}\frac{n^n}{e^n}$, то тази статия е напълно достатъчна. Ето нейният основен резултат.

Теорема 21: Апроксимация на Stirling, трета версия

Съществува константа c, такава че:

$$c\sqrt{n}\left(\frac{n}{e}\right)^n \leqslant n! \leqslant c\sqrt{n}\left(\frac{n}{e}\right)^n e^{\frac{1}{12n}}$$

Доказателство: Да разгледаме естествения логаритъм на факториела:

$$\ln n! = \sum_{i=1}^{n} \ln i$$

Да умножим всеки суманд с единица 1:

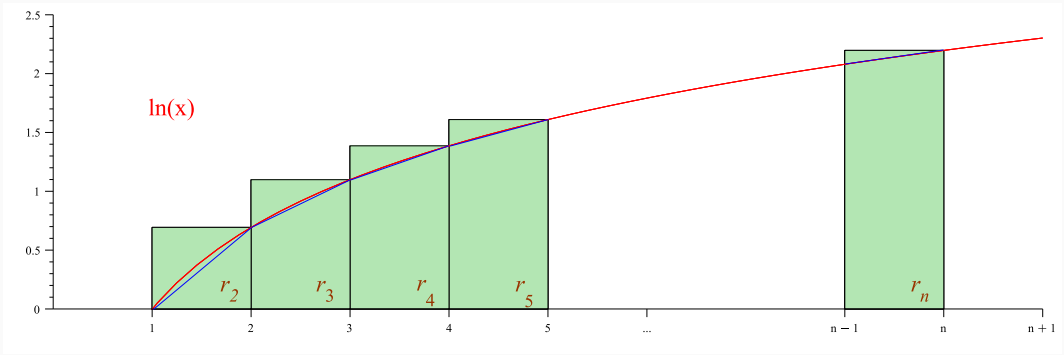
$$\sum_{i=1}^{n} \ln i = \sum_{i=1}^{n} ((\ln i) \times 1)$$

Тогава

$$\ln n! = \sum_{i=1}^{n} ((\ln i) \times 1)$$

Но $\ln n!$ е точно сумата от лицата на n на брой правоъгълници r_1, r_2, r_3, ..., r_n, показани на Фигура 2.10[a], като r_1 е изроден правоъгълник с лице 0. На фигурата е показана и графиката на функцията $\ln(x)$ и ясно се вижда, че сумата от лицата на тези правоъгълници е голямата сума на Darboux за $\ln(x)$, където x е в интервала $[0, n]$ с точки на разбиване 1, 2, ..., $n-1$.

Фигура 2.10 : $\ln n!$ е голямата сума на Darboux за $\ln(x)$ за $x \in [0,n]$.



Да означим с $S(r_i)$ лицето на r_i, за $2 \leqslant i \leqslant n$. От една страна, $S(r_i) = (\ln i) \times 1 = \ln i$. От друга страна:

$$S(r_i) = y_i + t_i - z_i$$

където

- y_i е лицето на района R_i в r_i, който лежи под кривата $\ln x$.
- t_i е лицето на триъгълника T_i с върхове $(i-1, \ln(i-1))$, $(i-1, \ln i)$ и $(i, \ln i)$.
- z_i е лицето на припокриването на R_i и T_i.

Тогава:

$$\ln i = y_i + t_i - z_i \tag{2.54}$$

Тези припокривания с лица z_2, z_3 и така нататък стават все по-малки много бързо. На фигурата се вижда ясно само първото от тях, чието лице е z_2; става дума за района в

r_2 между червената крива и синята отсечка под нея. Очевидно:

$$\begin{aligned} y_i &= \int_{i-1}^{i} \ln x \, dx \\ t_i &= \frac{\ln i - \ln(i-1)}{2} \\ z_i &= y_i - u_i - t'_i \end{aligned} \tag{2.55}$$

където u_i е лицето на правоъгълника с върхове $(i-1,0)$, $(i-1,\ln(i-1))$, $(i,\ln(i-1))$ и $(i,0)$, а t'_i е лицето на триъгълника с върхове $(i-1,\ln(i-1))$, $(i,\ln i)$ и $(i,\ln(i-1))$. Очевидно:

$$u_i = \ln(i-1) \tag{2.56}$$

$$t'_i = t_i = \frac{\ln i - \ln(i-1)}{2} \tag{2.57}$$

За y_i е вярно следното:

$$\begin{aligned} y_i = \int_{i-1}^{i} \ln x \, dx &= i \ln i - i - \big((i-1)\ln(i-1) - (i-1)\big) \\ &= \ln(i-1) - i\ln(i-1) - 1 + i\ln i \end{aligned} \tag{2.58}$$

Заместваме (2.56), (2.57) и (2.58) в (2.55) и получаваме:

$$\begin{aligned} z_i &= \underbrace{\ln(i-1) - i\ln(i-1) - 1 + i\ln i}_{y_i} - \underbrace{\ln(i-1)}_{u_i} - \underbrace{\frac{\ln i - \ln(i-1)}{2}}_{t_i} \\ &= \left(1 - i + -1 + \frac{1}{2}\right)\ln(i-1) + \left(i - \frac{1}{2}\right)\ln i - 1 \\ &= \left(i - \frac{1}{2}\right)\ln i - \left(i - \frac{1}{2}\right)\ln(i-1) - 1 \\ &= \left(i - \frac{1}{2}\right)\ln\left(\frac{i}{i-1}\right) - 1 \end{aligned} \tag{2.59}$$

Лема 16

Редът $\sum_{i=2}^{\infty} z_i$ е сходящ.

Доказателство: Да разгледаме развиването в степенен ред на $\ln(1+x)$:

$$\ln(1+x) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} x^k$$

Тогава:

$$\ln(1-x) = \ln(1+(-x)) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k}(-x)^k = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k}(-1)^k x^k$$
$$= \sum_{k=1}^{\infty} \frac{(-1)^{2k+1}}{k} x^k = -\sum_{k=1}^{\infty} \frac{x^k}{k}$$

Оттук следва, че:

$$\ln(1+x) - \ln(1-x) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} x^k + \sum_{k=1}^{\infty} \frac{x^k}{k} = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}+1}{k} x^k$$
$$= 2x + \frac{2x^3}{3} + \frac{2x^5}{5} + \frac{2x^7}{7} + \ldots$$

Изведохме, че:

$$\frac{1}{2}\ln\left(\frac{1+x}{1-x}\right) = x + \frac{x^3}{3} + \frac{x^5}{5} + \frac{x^7}{7} + \ldots$$

Заместваме x с $\frac{1}{2i-1}$ и получаваме:

$$\frac{1}{2}\ln\left(\frac{1+\frac{1}{2i-1}}{1-\frac{1}{2i-1}}\right) = \frac{1}{2i-1} + \frac{1}{3(2i-1)^3} + \frac{1}{5(2i-1)^5} + \frac{1}{7(2i-1)^7} + \ldots \quad \leftrightarrow$$
$$\frac{2i-1}{2}\ln\left(\frac{\frac{2i}{2i-1}}{\frac{2i-2}{2i-1}}\right) = 1 + \frac{1}{3(2i-1)^2} + \frac{1}{5(2i-1)^4} + \frac{1}{7(2i-1)^6} + \ldots \quad \leftrightarrow$$
$$\left(i-\frac{1}{2}\right)\ln\left(\frac{i}{i-1}\right) - 1 = \frac{1}{3(2i-1)^2} + \frac{1}{5(2i-1)^4} + \frac{1}{7(2i-1)^6} + \ldots$$

откъдето:

$$\left(i-\frac{1}{2}\right)\ln\left(\frac{i}{i-1}\right) - 1 \leqslant \frac{1}{3}\left(\frac{1}{(2i-1)^2} + \frac{1}{(2i-1)^4} + \frac{1}{(2i-1)^6} + \ldots\right)$$

Но за $i > 1$, $\frac{1}{(2i-1)^2} + \frac{1}{(2i-1)^4} + \frac{1}{(2i-1)^6} + \ldots$ е сходящ геометричен ред със сума:

$$\frac{\frac{1}{(2i-1)^2}}{1-\frac{1}{(2i-1)^2}} = \frac{\frac{1}{(2i-1)^2}}{\frac{(2i-1)^2-1}{(2i-1)^2}} = \frac{1}{4i^2-4i+1-1} = \frac{1}{4}\left(\frac{1}{i(i-1)}\right) = \frac{1}{4}\left(\frac{1}{i-1} - \frac{1}{i}\right)$$

Оттук следва, че:

$$\underbrace{\left(i-\frac{1}{2}\right)\ln\left(\frac{i}{i-1}\right) - 1}_{z_i} \leqslant \frac{1}{3}\left(\frac{1}{4}\left(\frac{1}{i-1} - \frac{1}{i}\right)\right) = \frac{1}{12}\left(\frac{1}{i-1} - \frac{1}{i}\right) \tag{2.60}$$

Лявата страна на неравенството (2.60) е z_i съгласно (2.59). Тогава:

$$z_i \leqslant \frac{1}{12}\left(\frac{1}{i-1} - \frac{1}{i}\right)$$

Тогава, за всяко $m > 1$ и за всяко $k > 0$:

$$\begin{aligned} z_m &\leqslant \frac{1}{12(m-1)} - \frac{1}{12m} \\ z_{m+1} &\leqslant \frac{1}{12m} - \frac{1}{12(m+1)} \\ &\ldots \\ z_{m+k} &\leqslant \frac{1}{12(m+k-1)} - \frac{1}{12(m+k)} \end{aligned}$$

Сумираме тези неравенства и получаваме:

$$z_m + z_{m+1} + \ldots + z_{m+k} \leqslant \frac{1}{12(m-1)} - \frac{1}{12(m+k)}$$

И така:

$$\sum_{i=m}^{\infty} z_i \leqslant \frac{1}{12(m-1)} \tag{2.61}$$

и в частност:

$$\sum_{i=2}^{\infty} z_i \leqslant \frac{1}{12} \tag{2.62}$$

Показахме, че $\sum_{i=2}^{\infty} z_i$ е сходящ. □

Сега се връщаме на уравнение (2.54). Сумираме за $2 \leqslant i \leqslant n$ и получаваме:

$$\underbrace{\sum_{i=2}^{n} S(r_i)}_{\ln n!} = \underbrace{\sum_{i=2}^{n} \int_{i-1}^{i} \ln x\,dx}_{\int_1^n \ln x\,dx = 1+n\ln n-n} + \underbrace{\sum_{i=2}^{n} \frac{\ln i - \ln(i-1)}{2}}_{\frac{\ln n}{2}} - \sum_{i=2}^{n} z_i \;\leftrightarrow$$

$$\ln n! = 1 + n\ln n - n + \frac{\ln n}{2} - \underbrace{\left(\sum_{i=2}^{\infty} z_i - \sum_{i=n+1}^{\infty} z_i\right)}_{\text{ограничено от константа съгласно (2.62)}} \tag{2.63}$$

Вземаме експонента при основа e от двете страни и получаваме:

$$\begin{aligned} n! &= e^1 \cdot e^{n\ln n} \cdot e^{-n} \cdot e^{\frac{\ln n}{2}} \cdot e^{-\sum_{i=2}^{\infty} z_i} \cdot e^{\sum_{i=n+1}^{\infty} z_i} \\ &= e\left(\frac{n}{e}\right)^n \sqrt{n} \cdot e^{-\sum_{i=2}^{\infty} z_i} \cdot e^{\sum_{i=n+1}^{\infty} z_i} = \left(e^{1-\sum_{i=2}^{\infty} z_i}\right)\sqrt{n}\left(\frac{n}{e}\right)^n\left(e^{\sum_{i=n+1}^{\infty} z_i}\right) \end{aligned}$$

Съгласно (2.62), $\sum_{i=2}^{\infty} z_i$ не надхвърля $\frac{1}{12}$, откъдето $e^{\frac{11}{12}} \leqslant e^{1-\sum_{i=2}^{\infty} z_i} \leqslant e$. От (2.61) следва,

че $e^{\sum_{i=n+1}^{\infty} z_i} \leqslant e^{\frac{1}{12n}}$. Следователно, съществува константа c, такава че $e^{\frac{11}{12}} \leqslant c \leqslant e$ и

$$c\sqrt{n}\left(\frac{n}{e}\right)^n \leqslant n! \leqslant c\sqrt{n}\left(\frac{n}{e}\right)^n e^{\frac{1}{12n}}$$

Истинската стойност на c е $\sqrt{2\pi} \approx 2.50662827463101$, $e^{\frac{11}{12}} \approx 2.50094001366213$, $e \approx 2.71828182845905$, и наистина

$$2.50094001366213 \leqslant 2.50662827463101 \leqslant 2.71828182845905$$

□

[a] Фигура 2.10 е направена с Maple(TM).

Теорема 22: Асимптотиката на $\lg n!$

В сила е

$$\lg n! \asymp n \lg n$$

Доказателство: От Теорема 19 знаем, че $n! \asymp \sqrt{n}\,\frac{n^n}{e^n}$. Прилагаме Следствие 9 и получаваме:

$$\lg n! \asymp \lg\left(\sqrt{n}\,\frac{n^n}{e^n}\right)$$

Но

$$\lg\left(\sqrt{n}\,\frac{n^n}{e^n}\right) = \frac{1}{2}\lg n + n\lg n - n\lg e$$

Забелязваме, че събираемото $n\lg n$ доминира асимптотично над останалите събираеми. Това, че $-n$ е отрицателна функция, а ние дефинираме асимптотичните нотации (в частност, релацията $\asymp$) само върху положителни функции, не е съществен проблем. Очевидно $n\ln n - n\lg e$ е положителна функция за всички достатъчно големи стойности на аргумента и $\frac{1}{2}\lg n + n\ln n - n\lg e \asymp n\ln n$. Доказахме, че $\lg n! \asymp n\lg n$. □

Теорема 23: Асимптотиката на $\sum_{k=1}^{n} k\lg k$

$\sum_{k=1}^{n} k\lg k \asymp n^2\lg n$.

Доказателство: Нека $m = \lfloor \frac{n}{2} \rfloor$, $A = \sum_{k=1}^{m-1} k\lg k$ и $B = \sum_{k=m}^{n} k\lg k$. Първо ще покажем, че $B \asymp n^2\lg n$. В сила е

$$\sum_{k=m}^{n} m\lg m \leqslant \sum_{k=m}^{n} k\lg k \leqslant \sum_{k=m}^{n} n\lg n \quad \leftrightarrow$$

$$(m\lg m)(n-m+1) \leqslant B \leqslant (n\lg n)(n-m+1) \quad \leftrightarrow$$

$$\underbrace{\left(\left\lfloor\frac{n}{2}\right\rfloor \lg\left\lfloor\frac{n}{2}\right\rfloor\right)\left(n-\left\lfloor\frac{n}{2}\right\rfloor+1\right)}_{C} \leqslant B \leqslant \underbrace{(n\lg n)\left(n-\left\lfloor\frac{n}{2}\right\rfloor+1\right)}_{D}$$

Очевидно $C \asymp n^2 \lg n$ и $D \asymp n^2 \lg n$. От това следва, че $B \asymp n^2 \lg n$.

Сега ще покажем, че $A \preceq B$. A има $\frac{n-2}{2}$ събираеми, ако n е четно, и $\frac{n-3}{2}$ събираеми, ако n е нечетно. Във всеки случай, A има по-малко събираеми от B. Нещо повече, всяко събираемо на A е по-малко от всяко събираемо на B. Ерго, $A \preceq B$. Щом $\sum_{k=1}^{n} k \lg k = A + B$, заключаваме, че $\sum_{k=1}^{n} k \lg k \asymp n^2 \lg n$. □

Важно свойство на биномния коефициент е, че средният биномен коефициент $\binom{n}{n/2}$ расте, в асимптотичния смисъл, почти като 2^n. За удобство, да допуснем, че n е четно.

Теорема 24: Асимптотиката на $\binom{n}{n/2}$

В сила е

$$\binom{n}{\frac{n}{2}} \asymp \frac{1}{\sqrt{n}} 2^n$$

Доказателство: Знаем, че:

$$\binom{n}{\frac{n}{2}} = \frac{n!}{\left(\frac{n}{2}!\right)^2}$$

Прилагаме Теорема 19 върху факториелите и получаваме

$$\frac{n!}{\left(\frac{n}{2}!\right)^2} \asymp \frac{\sqrt{2\pi n}\,\frac{n^n}{e^n}}{\left(\sqrt{2\pi\frac{n}{2}}\,\frac{\frac{n}{2}^{\frac{n}{2}}}{e^{\frac{n}{2}}}\right)^2} = \sqrt{2\pi}\sqrt{n}\,\frac{n^n}{e^n} \times \frac{e^n}{\pi\, n\, \frac{n^n}{2^n}} = \sqrt{\frac{2}{\pi}} \times \frac{1}{\sqrt{n}} \times 2^n$$

□

Нотация 2: "H_n" означава n-тата парциална сума на хармоничния ред

С "H_n" означаваме n-тата парциална сума на хармоничния ред. Иначе казано:

$$H_n = \sum_{k=1}^{n} \frac{1}{k}$$

Теорема 25: Асимптотиката на H_n

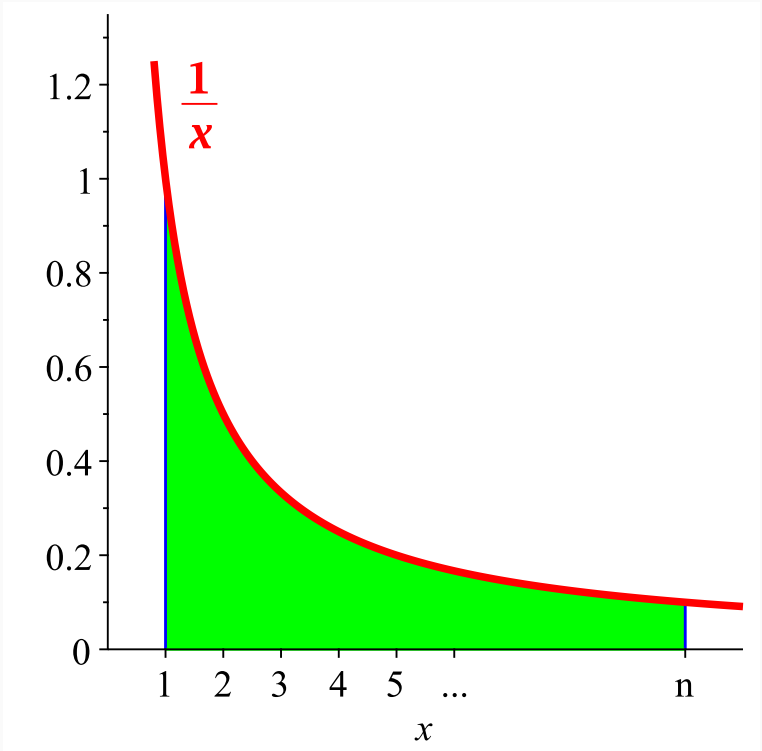
В сила е

$$H_n \asymp \lg n$$

Доказателство: Ще ограничим $\int_1^n \frac{1}{x} dx$ отгоре и отдолу със стъпаловидни функции, отговарящи на парциални суми на хармоничния ред. Знаем, че за всяко $n \in \mathbb{N}^+$, $\int_1^n \frac{1}{x} dx$ е площта на района, ограден от графиката на $\frac{1}{x}$, абсцисата и двете вертикални линии през точките 1 и n от абсцисата (вж. Фигура 2.11[†]).

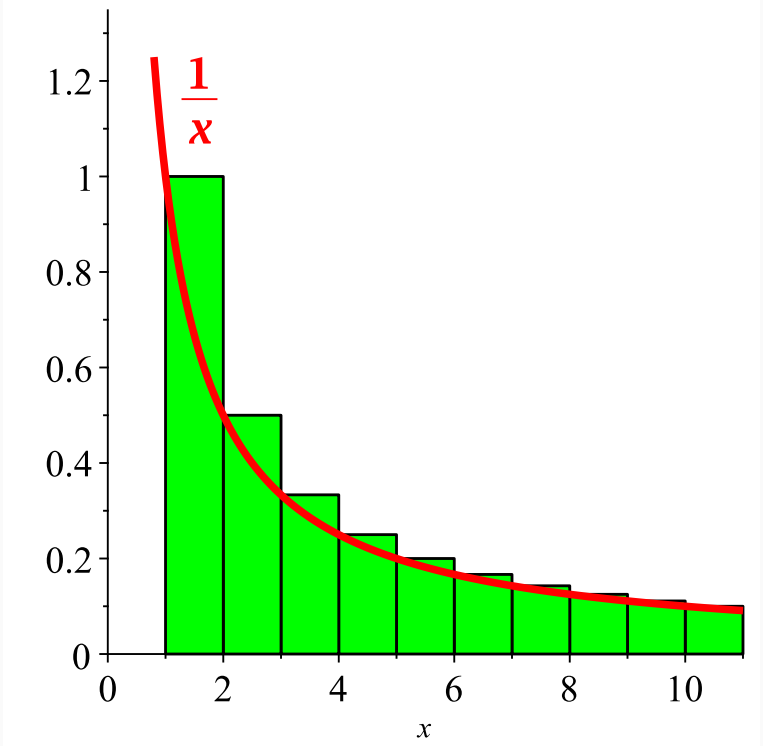
[†]Фигура 2.11 е направена с Maple(TM).

Фигура 2.11 : $\int_1^n \frac{1}{x}dx$ е площта на зеления район.



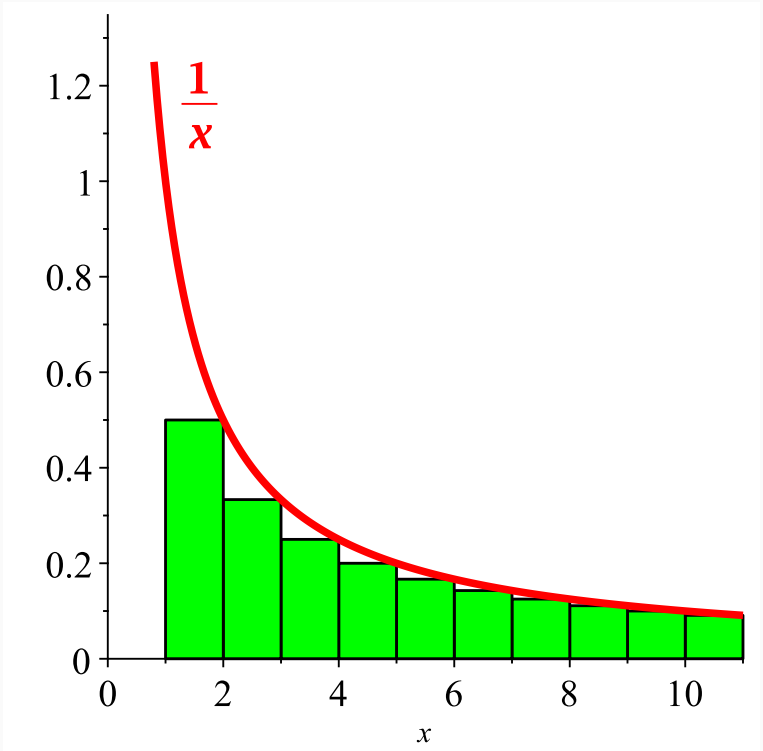
Да разгледаме стъпаловидната функция $\frac{1}{\lfloor x \rfloor}$ върху интервала $[1, \ldots, n]$. Вярно е, че $\forall x \in [1, \ldots, n] : \frac{1}{\lfloor x \rfloor} \geqslant \frac{1}{x}$, понеже $\lfloor x \rfloor \leqslant x$ за всяко x. Забелязваме, че площта на района, ограден от $\frac{1}{\lfloor x \rfloor}$, абсцисата и двете вертикални линии през точките 1 и n от абсцисата е точно $\sum_{i=1}^{n-1} \frac{1}{i}$, защото този район се разбива на $n-1$ правоъгълника, едната страна на всеки от които е с дължина единица, а другата е $\frac{1}{i}$ за някое $i \in \{1, \ldots, n-1\}$. Очевидно този район има площ, по-голяма от площта под $\frac{1}{x}$. С други думи, $\sum_{i=1}^{n-1} \frac{1}{i} > \int_1^n \frac{1}{x}dx$. Това е илюстрирано на Фигура 2.12.

Фигура 2.12 : $\sum_{i=1}^{n-1} \frac{1}{i} > \int_1^n \frac{1}{x} dx$.



Сега да разгледаме стъпаловидната функция $\frac{1}{\lceil x \rceil}$ върху интервала $[1, \ldots, n]$. $\forall x \in [1, \ldots, n] : \frac{1}{\lceil x \rceil} \leqslant \frac{1}{x}$, понеже $\lceil x \rceil \geqslant x$ за всяко x. Забелязваме, че площта на района, ограден от $\frac{1}{\lceil x \rceil}$, абсцисата и двете вертикални линии през точките 1 и n от абсцисата е точно $\sum_{i=2}^{n} \frac{1}{i}$, защото този район се разбива на $n-1$ правоъгълника, едната страна на всеки от които е с дължина единица, а другата е $\frac{1}{i}$ за някое $i \in \{2, \ldots, n\}$. Очевидно този район има площ, по-малка от площта под $\frac{1}{x}$. С други думи, $\int_1^n \frac{1}{x} dx > \sum_{i=2}^{n} \frac{1}{i}$. Това е илюстрирано на Фигура 2.13.

Фигура 2.13 : $\int_1^n \frac{1}{x}dx > \sum_{i=2}^n \frac{1}{i}$.



Показахме, че

$$\sum_{i=2}^{n} \frac{1}{i} < \int_1^n \frac{1}{x}dx < \sum_{i=1}^{n-1} \frac{1}{i}$$

Но $\sum_{i=2}^n \frac{1}{i} = \left(\sum_{i=1}^n \frac{1}{i}\right) - 1 = H_n - 1$, а $\sum_{i=1}^{n-1} \frac{1}{i} = H_{n-1}$. Тогава

$$H_n - 1 < \int_1^n \frac{1}{x}dx < H_{n-1}$$

Но $\int_1^n \frac{1}{x}dx = \ln x\Big|_1^n = \ln n - \ln 1 = \ln n - 0 = \ln n$. Тогава:

$$H_n - 1 < \ln n < H_{n-1}$$

Очевидно съществуват положителни константи c_1 и c_2 и стойност на аргумента n_0, такива че за всички $n > n_0$ е вярно

$$c_1 \cdot H_n \leqslant \ln n \leqslant c_2 \cdot H_n$$

Например, $c_1 = \frac{1}{2}$, $c_2 = 1$ и $n_0 = 1000$.

Заключаваме, че $\ln n \asymp H_n$. □

2.3.9 Асимптотичните нотации и релациите на асимптотични сравнения

На пръв поглед, релациите $\asymp$ и така нататък са доста по-удобни и прегледни от съответните асимптотични нотации като Θ и така нататък. Но, както казва Knuth в [53], има случаи, в които е много по-удобно да ползваме асимптотичните нотации, а не релационните символи. А именно, когато нотациите се появяват вътре в някакви изрази (а не вдясно от знака "="). Knuth говори за това, което ние нарекохме "анонимна функция" (вж. Конвенция 4).

Knuth в [53] дава следния пример:

$$\left(1+\frac{H_n}{n}\right)^{H_n} = e^{\frac{H_n^2}{n}+O\left(\frac{(\lg n)^3}{n^2}\right)}$$

Същото това нещо, написано не с голямо-О нотация, а с релацията $\preceq$, би било тромаво и грозно, защото, за да се използва релационният символ, всичко друго освен $\frac{(\lg n)^3}{n^2}$ би трябвало да се намира от едната му страна.

2.3.10 Асимптотични сравнения на функции на повече от една променлива

Определение 24 е обобщение на Определение 12 за функции на две променливи.

Определение 24: $\Theta(g(n,m))$

За всяка функция $g(n,m)$:

$$\Theta(g(n,m)) \stackrel{\text{def}}{=} \big\{f(n,m) \mid \exists c_1,c_2>0\ \exists n_0,m_0\ \forall n \geqslant n_0\ \forall m \geqslant m_0 : \\ 0 \leqslant c_1 \cdot g(n,m) \leqslant f(n,m) \leqslant c_2 \cdot g(n,m)\big\}$$

Очевидно е как да обобщим и другите четири асимптотични нотации за функции на две променливи. Също така е очевидно как да обобщаваме и за повече от две променливи.

2.3.11 Релация $\sim$: друга релация на асимптотична еквивалентност

Определение 25: Релация $\sim$ над множеството на функциите

За всички функции $f(n)$ и $g(n)$, $f(n) \sim g(n)$ тогава и само тогава, когато $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$.

Очевидно релацията $\sim$ е рефлексивна, симетрична и транзитивна, поради което е релация на еквивалентност над множеството на функциите, също като $\asymp$. Да си припомним Наблюдение 4. От него следва, че $\sim$ е по-силна от $\asymp$, защото:

$$\begin{aligned} f(n) \sim g(n) &\to f(n) \asymp g(n) \\ \neg(f(n) \asymp g(n)) &\to f(n) \sim g(n)) \end{aligned}$$

Поради това доказателството, че за някакви две функции $f(n)$ и $g(n)$ е в сила $f(n) \sim g(n)$, може да е по-трудно от доказателството, че $f(n) \asymp g(n)$. В тези лекции почти няма да

ползваме по-силната релация $\sim$ и ще смятаме, че ако сме показали $f(n) \asymp g(n)$, то ние сме намерили най-точната възможна асимптотична близост между $f(n)$ и $g(n)$.

Забележете, че току-що дефинираната $\sim$ и фактор-релацията $\simeq$ от Допълнение 16 (вижте Определение 23) нямат нищо общо.

2.4 Примери за изчисляване на сложност по време на итеративни алгоритми

Много грубо казано, на практика най-често сложността на даден итеративен алгоритъм се оказва $\Theta(n^k)$, където k е максималната вложеност на цикли в него. В тази секция ще видим няколко примера за изчисляване на сложност по време на итеративни алгоритми.

2.4.1 Алгоритми с единичен цикъл

Да разгледаме най-елементарно сумиране на числа от масив.

```
TRIVIAL SUMMATION(A[1,...,n]: масив от цели числа)
1  s ← 0
2  for i ← 1 to n
3      s ← s + A[i]
4  return s
```

Може да направим точна сума като (2.6) по начин, аналогичен на тамошния. Но за простота ще разсъждаваме така. Интересува ни само асимптотиката на сложността, а не точната функция. Сложността на алгоритъма е сумата от сложността на изпълнението преди цикъла, сложността на цикъла и сложността на изпълнението след цикъла. От тези сложности, първата и третата са $\Theta(1)$. Да видим каква е втората. Тялото на **for**-цикъла е $\boxed{s \leftarrow s + A[i]}$. Само по себе си, то има сложност $\Theta(1)$. Цикълът очевидно се изпълнява n пъти. Тогава сложността на цялото изпълнение на цикъла е $n \times \Theta(1)$, а за целия алгоритъм имаме

$$\underbrace{\Theta(1)}_{\text{сложността преди цикъла}} + \underbrace{n \times \Theta(1)}_{\text{сложността на цикъла}} + \underbrace{\Theta(1)}_{\text{сложността след цикъла}}$$

Елементарно се показва, че $\Theta(1) + n \times \Theta(1) + \Theta(1) = \Theta(n)$, така че сложността е $\Theta(n)$.

С напълно аналогични разсъждения извеждаме, че всеки алгоритъм по схемата

«инициализация»
for $i \leftarrow c_1$ **to** $f(n)$
    «тяло-на-цикъла»
«завършване»

където и *«инициализация»*, и *«тяло-на-цикъла»*, и *«завършване»* имат сложност $\Theta(1)$, а c_1 е някаква константа, има сложност $\Theta(f(n))$. Този резултат остава в сила дори тялото на цикъла да не е просто поредица от императивни инструкции, а да има и условни инструкции; дори да има сложна структура от вложени if-then-else, ако всеки изчислителен път през тялото има сложност $\Theta(1)$, резултатът е в сила.

Ако *«инициализация»* има сложност $\phi(n)$, *«тяло-на-цикъла»* има сложност $\sigma(n)$, а *«завършване»* има сложност $\psi(n)$, общата сложност е $\Theta(\phi(n) + \psi(n) + f(n)\sigma(n))$.

2.4.2 Алгоритми с вложени цикли

Да разгледаме алгоритъм по схемата

«инициализация»
for $i \leftarrow c_1$ **to** $f(n)$
    for $j \leftarrow c_2$ **to** $g(n)$
        «тяло-на-цикъла»
«завършване»

където и *«инициализация»*, и *«тяло-на-цикъла»*, и *«завършване»* имат сложност $\Theta(1)$, а c_1 и c_2 са някакви константи. С разсъждения, напълно аналогични на тези предната подсекция, извеждаме, че сложността е $\Theta(f(n)) \times x$, където x е сложността на вътрешния цикъл. На свой ред тя е $\Theta(g(n))$, така че общата сложност е $\Theta(f(n)g(n))$.

Ако *«инициализация»* има сложност $\phi(n)$, *«тяло-на-цикъла»* има сложност $\sigma(n)$, а *«завършване»* има сложност $\psi(n)$, общата сложност е $\Theta(\phi(n) + \psi(n) + f(n)g(n)\sigma(n))$.

Лесно се вижда, че алгоритъм по схемата

«инициализация»
for $i \leftarrow c_1$ **to** $f(n)$
    for $j \leftarrow c_2$ **to** $g(n)$
        for $j \leftarrow c_3$ **to** $h(n)$
            «тяло-на-цикъла»
«завършване»

където и *«инициализация»*, и *«тяло-на-цикъла»*, и *«завършване»* имат сложност $\Theta(1)$, а c_1, c_2 и c_3 са някакви константи, има сложност $\Theta(f(n)g(n)h(n))$. Очевидно е как се обобщава това в случай, че *«инициализация»*, *«тяло-на-цикъла»* и *«завършване»* имат сложности, зависещи от n, както и за по-голяма вложеност на циклите.

На практика често се срещат итеративни алгоритми с еднократна вложеност на цикъла, в които обаче индексната променлива на вътрешния цикъл се инициализира не с константа, каквато беше c_2 тук, а чрез някаква функция на i, индексната променлива на външния цикъл. Пример за това е SELECTION SORT на стр. 70, при който индексната променлива j на вътрешния цикъл се инициализира с $i + 1$. Ако имаме задача да анализираме такъв алгоритъм, най-сигурно е да направим сума като (2.10) – от нея асимптотиката на сложността се вижда веднага. Не можем със сигурност да твърдим, че сложността е $\Theta(n^2)$ само заради това, че имаме вложен цикъл, ако нямаме теоретичен резултат, покриващ конкретния случай.

Ще разгледаме два примера за двукратна вложеност на цикъла. В единия от тях наистина сложността се оказва $\Theta(n^3)$, което е често срещано при цикъл-в-цикъл-в-цикъл с линейни горни граници за индексните променливи, но при другия не е така.

Първи примерен алгоритъм. Този алгоритъм не прави нищо интересно и го ползваме само като пример за анализ на сложността. Входът е масив, който не се ползва от алгоритъма; в някакъв смисъл, той е запис на n в унарна бройна система.

```
AlgX(A[1,2,...n]: масив от цели числа)
1  s ← 0
2  for i ← 1 to n
3      for j ← i + 1 to n
4          for k ← i + j − 1 to n
5              s ← s + 1
6  return s
```

Ще изследваме стойността s, която връща AlgX. Ще я разгледаме като сума, ще опростим сумата до еквивалентен израз-произведение, откъдето асимптотиката ще стане очевидна, и тази асимптотика ще се окаже и сложността на AlgX по време. Тъй като извеждането е дълго, ще го обособим като лема.

Лема 17

AlgX връща

$$s = \begin{cases} \frac{k(k+1)(4k-1)}{6}, & \text{ако } n = 2k \\ \frac{k(k+1)(4k+5)}{6}, & \text{ако } n = 2k+1 \end{cases}$$

Доказателство: От кода на AlgX имаме

$$s = \sum_{i=1}^{n} \sum_{j=i+1}^{n} \sum_{k=i+j-1}^{n} 1$$

понеже при всяко изпълнение на тялото към s се добавя единица, а s се инициализира с 0.

Да оценим най-вътрешната сума $\sum_{k=i+j-1}^{n} 1$. Лесно се вижда, че долната граница $i+j-1$ може да надхвърли горната граница n. Ако се случи това, сумата е 0, защото индексната променлива взема стойности от празното множество. По-подробно написано, за всяко цяло t,

$$\sum_{i=t}^{n} 1 = \begin{cases} n - t + 1, & \text{ако } t \leqslant n \\ 0, & \text{в противен случай} \end{cases}$$

Това точно съответства на факта, че в алгоритъма най-вътрешният цикъл не се изпълнява изобщо, ако стойността, с която се инициализира k, надхвърля n. И така,

$$\sum_{k=i+j-1}^{n} 1 = \begin{cases} n - i - j + 2, & \text{ако } i + j - 1 \leqslant n, \text{ тоест } j \leqslant n - i + 1 \\ 0, & \text{в противен случай} \end{cases}$$

Тогава

$$s = \sum_{i=1}^{n} \sum_{j=i+1}^{n-i+1} (n + 2 - (i + j))$$

Но вътрешната сума е 0, ако $i+1 > n-i+1$, тоест, ако $2i > n$, тоест, ако $i > \left\lfloor \frac{n}{2} \right\rfloor$. Следователно,

максималната стойност на i, която трябва да разгледаме, е $\lfloor \frac{n}{2} \rfloor$:

$$s = \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \sum_{j=i+1}^{n-i+1} (n+2-(i+j)) =$$

$$(n+2) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \sum_{j=i+1}^{n-i+1} 1 - \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i \left(\sum_{j=i+1}^{n-i+1} 1 \right) - \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \sum_{j=i+1}^{n-i+1} j =$$

$$(n+2) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (n-i+1-(i+1)+1) - \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i(n-i+1-(i+1)+1) -$$

$$\sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \left(\sum_{j=1}^{n-i+1} j - \sum_{j=1}^{i} j \right) =$$

$$(n+2) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (n-2i+1) - \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i(n-2i+1) -$$

$$\sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \left(\frac{(n-i+1)(n-i+2)}{2} - \frac{i(i+1)}{2} \right) =$$

$$(n+2)(n+1) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} 1 - 2(n+2) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i - (n+1) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i + 2 \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i^2 -$$

$$\frac{1}{2} \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \left((n+1)(n+2) - i(2n+3) + i^2 - i^2 - i \right) =$$

$$(n+2)(n+1) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} 1 - (3n+5) \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i + 2 \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i^2 -$$

$$\frac{(n+1)(n+2)}{2} \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} 1 + \frac{(2n+4)}{2} \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i =$$

$$\left\lfloor \frac{n}{2} \right\rfloor (n+1)(n+2) - (3n+5) \frac{\lfloor \frac{n}{2} \rfloor \left(\lfloor \frac{n}{2} \rfloor + 1 \right)}{2} + 2 \frac{\lfloor \frac{n}{2} \rfloor \left(\lfloor \frac{n}{2} \rfloor + 1 \right) \left(2 \lfloor \frac{n}{2} \rfloor + 1 \right)}{6} -$$

$$\frac{1}{2} \left\lfloor \frac{n}{2} \right\rfloor (n+1)(n+2) + (n+2) \frac{\lfloor \frac{n}{2} \rfloor \left(\lfloor \frac{n}{2} \rfloor + 1 \right)}{2} =$$

$$\frac{\lfloor \frac{n}{2} \rfloor (n+1)(n+2)}{2} - \frac{\lfloor \frac{n}{2} \rfloor \left(\lfloor \frac{n}{2} \rfloor + 1 \right) (2n+3)}{2} + \frac{\lfloor \frac{n}{2} \rfloor \left(\lfloor \frac{n}{2} \rfloor + 1 \right) \left(2 \lfloor \frac{n}{2} \rfloor + 1 \right)}{3}$$

Ако n е четно, тоест $n = 2k$ за някое $k \in \mathbb{N}^+$, то $\lfloor \frac{n}{2} \rfloor = k$, така че

$$s = \frac{k(2k+1)(2k+2)}{2} - \frac{k(k+1)(4k+3)}{2} + \frac{k(k+1)(2k+1)}{3} =$$

$$\frac{k(k+1)(4k+2) - k(k+1)(4k+3)}{2} + \frac{k(k+1)(2k+1)}{3} =$$

$$k(k+1) \left(-\frac{1}{2} + \frac{2k+1}{3} \right) = \frac{k(k+1)(4k-1)}{6}$$

Ако n е нечетно, тоест $n = 2k + 1$ за някое $k \in \mathbb{N}$, то $\left\lfloor \frac{n}{2} \right\rfloor = k$, така че

$$s = \frac{k(2k+2)(2k+3)}{2} - \frac{k(k+1)(4k+5)}{2} + \frac{k(k+1)(2k+1)}{3} =$$
$$\frac{k(k+1)(4k+6) - k(k+1)(4k+5)}{2} + \frac{k(k+1)(2k+1)}{3} =$$
$$k(k+1)\left(\frac{1}{2} + \frac{2k+1}{3}\right) = \frac{k(k+1)(4k+5)}{6}$$

□

От Лема 17 веднага следва, че сложността по време на AlgX е $\Theta(n^3)$, тъй като $n \asymp k$.

Втори примерен алгоритъм. Този алгоритъм също не прави нищо интересно и също го ползваме само като пример за анализ на сложността. Отново входът не се ползва и в някакъв смисъл е запис на n в унарна бройна система.

```
AlgY(A[1,2,...n]: масив от цели числа)
1  s ← 0
2  for i ← 1 to n
3      for j ← i to n
4          for k ← n + i + j − 3 to n
5              s ← s + 1
6  return s
```

Да изследваме стойността s, която връща AlgY. Има три вложени **for**-цикли, всеки с горна граница n за съответната индексна променлива, тялото на най-вътрешния цикъл се изпълнява в $\Theta(1)$, така че със сигурност $s = O(n^3)$. Само че сега $s \neq \Theta(n^3)$, за разлика от предния алгоритъм. Защо? За всяко достатъчно голямо n е вярно, че ред 5 се изпълнява само за три стойности на наредената тройка (i, j, k), а именно

$$(i,j,k) \in \{(1,1,n-1), (1,1,n), (1,2,n-1)\}$$

по простата причина, че най-вътрешният цикъл на редове 5–4 изпълнява тялото си само при $n + i + j - 3 \leqslant n$, тоест, $i + j \leqslant 3$, тоест,

- $i = 1$ и $j = 1, 2$;
- $i = 2$ и $j = 1$.

И така, AlgY връща $s = 3$.

Дали обаче върната стойност дава асимптотиката на сложността по време? В примерите, които разгледахме дотук, това беше така. В примера с AlgY това не е така. Ред 4 съдържа проверка, която не може да стане в нула време. Трябва поне единица от дискретното време на алгоритъма, за да бъде установено, че $n + i + j - 3 > n$ и да се премине към следващото изпълнение на втория **for**-цикъл на редове 3–5. Ерго, изпълнението на AlgY не е по-бързо от изпълнението на следния алгоритъм.

```
AlgZ(A[1,2,...n]: масив от цели числа)
1  for i ← 1 to n
2      for j ← i to n
3          прави нещо в константно време
```

Както стана ясно от изложението горе, AlgZ има сложност по време $\Theta(n^2)$, което е истинската сложност на AlgY.

Тук може да има възражение, че AlgY връща 3 за всяко достатъчно голямо n, така че той е еквивалентен на алгоритъм, който просто връща 3 и работи в $\Theta(1)$. Това възражение не е валидно. Ние изследваме сложността на **дадения** алгоритъм, а не на друг, който е еквивалентен нему и е по-бърз. Това заслужава да се натърти.

Забележка 1

Когато се търси сложността на даден алгоритъм, трябва да се разглежда именно този алгоритъм, буквално, а не някаква негова еквивалентна като изход версия, но с подобрена ефикасност. Алгоритъмът е синтактичен обект и се разглежда буквално. Кодът на алгоритъма е алгоритъмът.

2.4.3 Двоично търсене, итеративен вариант

Разглеждаме алгоритъма за двоично търсене в сортиран масив в итеративния вариант, който наричаме BinSearchIter и чийто псевдокод е на стр. 124. Добре известно е, че сложността му в най-лошия случай е логаритмична и това може да се покаже полуформално с едно изречение: на всяка итерация на цикъла алгоритъмът решава един бит от индекса на key в масива—ако key изобщо се среща в масива—а индексът има размер, който е $\Theta(\lg n)$. Дали тази аргументация е задоволително формална и прецизна или не, зависи от изискванията. Сега приемаме, че не е задоволително формална. Ето по-формална аргументация.

Лема 18

Сложността по време на BinSearchIter на стр. 124 е $O(\lg n)$.

Доказателство: Следното твърдение е инвариант за **while** цикъла (редове 5–12).

Инвариант 2: Цикълът на BinSearchIter

При достигане номер t на ред 5 е вярно, че:

$$h - \ell + 1 \leqslant \frac{n}{2^{t-1}} \tag{2.64}$$

База. Нека $t = 1$. Тогава $\ell = 1$ и $h = n$ заради редове 3 и 4. В сила е

$$n - 1 + 1 \leqslant \frac{n}{2^{1-1}} \;\leftrightarrow\; n \leqslant \frac{n}{1} \qquad \checkmark$$

Поддръжка. Нека изпълнението е на ред 5 и предстои поне още едно изпълнение на **while**-цикъла. Игнорираме възможността $A[\text{mid}] = \text{key}$, защото, ако това е вярно, изпълнението стига до ред 8 и спира, поради което ред 5 няма да бъде достигнат повече. И така, разглеждаме само следните два случая:

- key $<$ A[mid]. Тогава изпълнението достига ред 10, където $\mathtt{h}$ става $\mathrm{mid}-1$, което всъщност е $\left\lfloor\frac{\ell+\mathtt{h}}{2}\right\rfloor-1$, изразено чрез старото $\mathtt{h}$. ℓ не се променя и изпълнението се връща на ред 5. Трябва да покажем, че по отношение на новите ℓ и $\mathtt{h}$ (новото ℓ е старото ℓ, но така се казва), неравенство (2.64) остава в сила. Започваме от индуктивното предположение (2.64) за старите стойности:

$$\begin{aligned}
\mathtt{h}-\ell+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}-1}} && \text{// делим на 2}\\
\frac{\mathtt{h}-\ell}{2}+\frac{1}{2} &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\frac{\mathtt{h}+\ell-2\ell}{2}+\frac{1}{2} &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\frac{\mathtt{h}+\ell}{2}-\ell+\frac{1}{2} &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\frac{\mathtt{h}+\ell}{2}-\ell+\frac{1}{2}-1+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\frac{\mathtt{h}+\ell}{2}-1-\ell+\frac{1}{2}+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\frac{\mathtt{h}+\ell}{2}-1-\ell+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}} && \text{// понеже } \frac{1}{2}>0\\
\underbrace{\left\lfloor\frac{\mathtt{h}+\ell}{2}\right\rfloor-1}_{\text{новото }\mathtt{h}}-\ell+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}} && \text{// понеже } \lfloor\mathtt{m}\rfloor\leqslant\mathtt{m},\ \forall\mathtt{m}\in\mathbb{R}^+
\end{aligned}$$

- key $>$ A[mid]. Тогава изпълнението достига ред 12, където ℓ става $\mathrm{mid}+1$, което всъщност е $\left\lfloor\frac{\ell+\mathtt{h}}{2}\right\rfloor+1$, изразено чрез старото ℓ. $\mathtt{h}$ не се променя и изпълнението се връща на ред 5. Трябва да покажем, че по отношение на новите ℓ и $\mathtt{h}$ (новото $\mathtt{h}$ е старото $\mathtt{h}$), неравенство (2.64) остава в сила. Започваме от индуктивното предположение (2.64) за старите стойности:

$$\begin{aligned}
\mathtt{h}-\ell+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}-1}} && \text{// делим на 2}\\
\frac{\mathtt{h}-\ell}{2}+\frac{1}{2} &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\frac{2\mathtt{h}-\mathtt{h}-\ell}{2}+\frac{1}{2} &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\mathtt{h}-\frac{\mathtt{h}+\ell}{2}+\frac{1}{2} &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\mathtt{h}-\frac{\ell+\mathtt{h}}{2}+\frac{1}{2}-1+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\mathtt{h}-\frac{\ell+\mathtt{h}}{2}-\frac{1}{2}+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\mathtt{h}-\left(\frac{\ell+\mathtt{h}}{2}+\frac{1}{2}\right)+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}}\\
\mathtt{h}-\underbrace{\left(\left\lfloor\frac{\ell+\mathtt{h}}{2}\right\rfloor+1\right)}_{\text{новото }\ell}+1 &\leqslant \frac{\mathtt{n}}{2^{\mathtt{t}}} && \text{// понеже } \left\lfloor\frac{\ell+\mathtt{h}}{2}\right\rfloor+1\geqslant\frac{\ell+\mathtt{h}}{2}+\frac{1}{2},\ \forall\ell,\mathtt{h}\in\mathbb{N}^+
\end{aligned}$$

Доказахме инварианта (2.64). Да наречем "$\mathtt{t}_{\max}$" стойността на $\mathtt{t}$ при достигането на ред 5,

след което тялото на цикъла се изпълнява за последен път[†]. Очевидно в този момент стойностите на ℓ и h са равни. Заместваики t с t_{max} и $h - \ell$ с 0 в инварианта (2.64), получаваме

$$0 + 1 \leqslant \frac{n}{2^{t_{max}-1}} \leftrightarrow 2^{t_{max}} \leqslant 2n \leftrightarrow t_{max} \leqslant 1 + \log_2 n$$

Щом тялото на цикъла се изпълнява $O(\lg n)$ пъти и всяко изпълнение отнема $\Theta(1)$ време, то е вярно, че сложността по време на алгоритъма е $O(\lg n)$. □

Да си припомним, че разглеждаме сложността по време в **най-лошия** случай.

Лема 19

Сложността по време на BinSearchIter на стр. 124 е $\Omega(\lg n)$.

Доказателство: Следното твърдение е инвариант за **while** цикъла (редове 5–12).

Инвариант 3: Цикълът на BinSearchIter

При достигане номер t на ред 5 е вярно, че:

$$\frac{n}{2^t} - 4 < h - \ell \tag{2.65}$$

База. Нека $t = 1$. Тогава $\ell = 1$ и $h = n$ заради редове 3 и 4. В сила е

$$\frac{n}{2^{1-1}} - 4 < n - 1 \leftrightarrow \frac{n}{1} - 4 < n - 1 \quad \checkmark$$

Поддръжка. Нека изпълнението е на ред 5 и предстои поне още едно изпълнение на **while**-цикъла. Игнорираме възможността $A[mid] = key$, защото, ако това е вярно, изпълнението стига до ред 8 и спира, поради което ред 5 няма да бъде достигнат повече. И така, разглеждаме само следните два случая:

- $key < A[mid]$. Тогава изпълнението достига ред 10, където h става $mid - 1$, което всъщност е $\left\lfloor \frac{\ell+h}{2} \right\rfloor - 1$, изразено чрез старото h. ℓ не се променя и изпълнението се връща на ред 5. Трябва да покажем, че по отношение на новите ℓ и h (новото ℓ е старото ℓ), неравенство (2.65) остава в сила. Започваме от индуктивното предположение (2.65) за старите стойности:

$$\begin{aligned}
&\frac{n}{2^t} - 4 < h - \ell \quad && \text{// делим на 2} \\
&\frac{n}{2^{t+1}} - 2 < \frac{h - \ell}{2} \\
&\frac{n}{2^{t+1}} - 2 < \frac{h + \ell - 2\ell}{2} \\
&\frac{n}{2^{t+1}} - 2 < \frac{h + \ell}{2} - \ell \\
&\frac{n}{2^{t+1}} - 4 < \frac{h + \ell}{2} - 2 - \ell \\
&\frac{n}{2^{t+1}} - 4 < \underbrace{\left\lfloor \frac{h + \ell}{2} \right\rfloor - 1}_{\text{новото } h} - \ell \quad && \text{// понеже } m - 2 \leqslant \lfloor m \rfloor - 1,\ \forall m \in \mathbb{R}^+
\end{aligned}$$

[†]Ред 5 се достига още един път.

- key > A[mid]. Тогава изпълнението достига ред 12, където ℓ става mid + 1, което всъщност е $\left\lfloor \frac{\ell+h}{2} \right\rfloor + 1$, изразено чрез старото ℓ. h не се променя и изпълнението се връща на ред 5. Трябва да покажем, че по отношение на новите ℓ и h (новото h е старото h), неравенство (2.65) остава в сила. Започваме от индуктивното предположение (2.65) за старите стойности:

$$
\begin{aligned}
\frac{n}{2^t} - 4 &< h - \ell && \text{// делим на 2} \\
\frac{n}{2^{t+1}} - 2 &< \frac{h-\ell}{2} \\
\frac{n}{2^{t+1}} - 2 &< \frac{2h-h-\ell}{2} \\
\frac{n}{2^{t+1}} - 2 &< h - \frac{h+\ell}{2} \\
\frac{n}{2^{t+1}} - 4 &< h - \frac{h+\ell}{2} - 2 \\
\frac{n}{2^{t+1}} - 4 &< h - \left(\frac{h+\ell}{2} + 2\right) \\
\frac{n}{2^{t+1}} - 4 &< h - \underbrace{\left(\left\lfloor \frac{h+\ell}{2} \right\rfloor + 1\right)}_{\text{новото } \ell} && \text{// понеже } m + 2 \geqslant \lfloor m \rfloor + 1,\ \forall m \in \mathbb{R}^+
\end{aligned}
$$

Доказахме инварианта (2.65). Да наречем "t_{max}" стойността на t при достигането на ред 5, след което тялото на цикъла се изпълнява за последен път[†]. Очевидно в този момент стойностите на ℓ и h са равни. Заместваики t с t_{max} и $h - \ell$ с 0 в Инвариант (2.65), получаваме

$$
\frac{n}{2^{t_{max}}} - 4 < 0 \ \leftrightarrow\ \frac{n}{2^{t_{max}}} < 4 \ \leftrightarrow\ n < 4 \cdot 2^{t_{max}} \ \leftrightarrow\ \log_2 n < 2 + t_{max} \ \leftrightarrow\ t_{max} > -2 + \log_2 n
$$

Очевидно сложността по време на алгоритъма в най-лошия случай е $\Omega(\lg n)$. □

Теорема 26: Сложността на двоичното търсене е логаритмична

Сложността по време на BinSearchIter на стр. 124 в най-лошия случай е $\Theta(\lg n)$.

Доказателство: Следва веднага от Лема 18 и Лема 19.

2.5 Винаги ли е лоша високата сложност

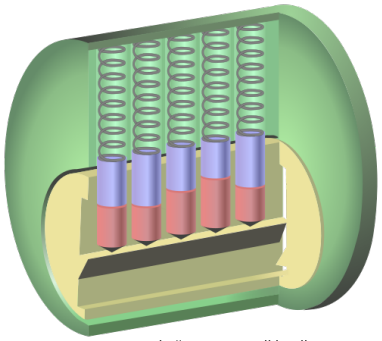
В заключение ще разискваме доколко бавно е синоним на лошо в света на алгоритмите. Наистина, ние търсим колкото е възможно по-бързи алгоритми и свикваме да мислим, че бавен алгоритъм е лош алгоритъм, а бърз алгоритъм е добър алгоритъм. Има важни изчислителни задачи, за които не са известни бързи алгоритми, а има и задачи, за които е доказано, при определени допускания, че няма бързи алгоритми. От конвенционалната гледна точка, липсата на бързи алторитми—независимо от това дали се дължи на човешкото незнание или е иманентна—е нещо лошо; ако мислим, че бърз=добър и бавен=лош, то високата сложност е нещо като проклятие.

[†]Ред 5 се достига още един път.

Има обаче друга гледна точка, от която високата сложност е благословия. Това е гледната точка на сигурността. Първо да разгледаме съвсем обикновен пример: ключ от секретна брава като този

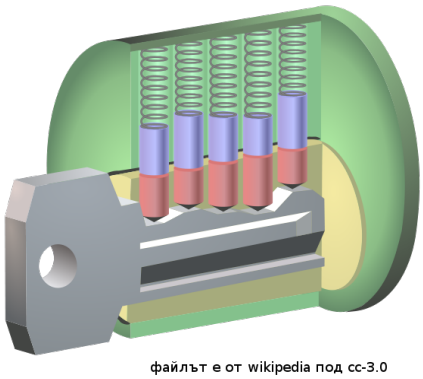


Добре известно е, че секретна брава се отваря при завъртането на цилиндъра в нейната ключалка. Цилиндърът стои плътно в гнездо в ключалката. Ако в ключалката няма ключ, този цилиндър не може да бъде завъртян, защото в него има няколко щифта, обикновено не повече от 6 на брой, сложени в странични напречни отвори, също с цилиндрична форма. Тези отвори са наредени в редица и продължават в гнездото. Всеки щифт е едновременно и в цилиндъра, и в гнездото отстрани. Всеки щифт е срязан на някаква височина, така че всъщност той се състои от два щифта, допрени плътно. Ако няма ключ в ключалката, едно от парчетата на всеки щифт стои едновременно и в цилиндъра, и в гнездото, и така щифтовене пречат на цилиндъра да се върти.



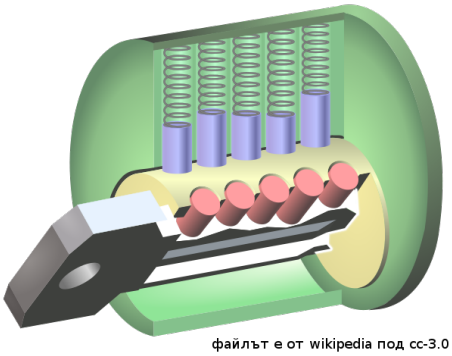
файлът е от wikipedia под cc-3.0

При вкарване на ключ в ключалката, назъбената му повърхност минава точно под щифтовете и ги избутва навън, на различни разстояния съгласно височините на зъбите. Правилният ключ размества щифтовете в отворите им така, че всички срезове на всички щифтове се оказват подравнени точно с границата между цилиндъра и гнездото



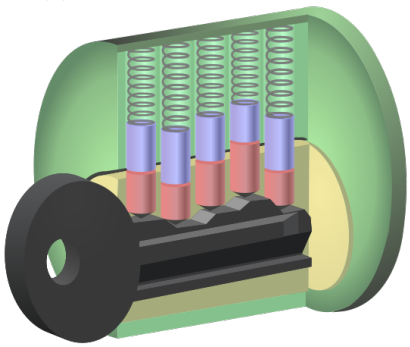
файлът е от wikipedia под cc-3.0

и вече нищо не пречи на цилиндъра да бъде завъртян, което води до издърпване на езика на бравата и вратата се отваря.



файлът е от wikipedia под cc-3.0

Ако обаче в ключалката бъде сложен неправилен ключ, поне един щифт ще пречи на завъртането на цилиндъра, понеже срезът му няма да е подравнен с границата между цилиндъра и гнездото



файлът е от wikipedia под cc-3.0

и бравата няма да бъде отворена.

Броят на щифтовете за дадена ключалка не е тайна. Всеки щифт е срязан на една измежду няколко възможни позиции, обикновено не повече от 10 различни позиции. Бройката на възможните позиции също не е тайна. Тайната на ключа е, точно на каква височина измежду всички възможни височини е срязан всеки щифт. Нека броят на щифтовете е n и има m възможности за различни височини на срязване на всеки щифт. Тогава, от информационна гледна точка, ключът е елемент от $\{1, 2, \ldots, m\}^n$. С други думи, векторът от височините, на които са срязани щифтовете, е пълно описание на ключа (заедно с информацията за профила му и за разстоянията между щифтовете, но да игнорираме тези детайли). Всички възможни ключове са най-много краен брой, а именно не повече от m^n. Например, ако $m = 10$ и $n = 6$, ключовете са най-много $1\,000\,000$.

Сигурността при физическите ключалки идва оттам, че атака срещу ключалката, която се състои в изработване на всички възможни ключове и последователното им пробване, е абсолютно непрактична. Виждаме, че в някакъв смисъл **сигурността идва от високата сложност на атаката, която се състои в опитване на всички възможности**; в случая, сложността на изработване на стотици хиляди ключове-проби и последователното им опитване.

В компютърната сигурност, високата сложност дава сигурност по доста аналогичен начин. Тази материя е далеч отвъд началния курс по алгоритми. Тук само ще спомена, че в криптографията[†], която е от огромно значение за компютърната сигурност, в съвременния си вид се основава на използване на ключове (но чисто информационни, а не метални парчета). Съвременната криптография е основана на едно допускане, известно като *Принцип*

[†]Криптография, съвсем накратко, е науката за предаване на информация в среда, в която има подслушвач, по такъв начин, че подслушвачът да не може да разбере какво се предава, въпреки че вижда ясно това, което се предава, и е наясно, че то носи важна информация.

на Kerckhoffs: противникът знае всички детайли от криптографската система—алгоритми и протоколи—и единственото, което **не знае**, е ключа[†]. Наистина, използваните в криптографията протоколи не са тайна – несериозно е да се мисли, че система, която е замислена и конструирана от множество хора и се използва с години от много повече хора, може да остане тайна за добре мотивиран противник.

Най-общо казано, криптирането на информацията работи така. Даден участник в комуникацията, която трябва да остане тайна от подслушвача, прилага определен алгоритъм с два входа: съобщението, което трябва да се предаде, и ключа. Изходът от този алгоритъм е стринг, който е неразбираем при прочитане. Процесът на превръщане на разбираемото съобщение в неразбираем стринг се нарича *шифриране*. Приемащата страна (или страни, ако съобщението се праща масово) прилага същия или друг алгоритъм с два входа: полученото неразбираемо съобщение и ключа, и изходът е първоначалното разбираемо съобщение. Процесът на превръщане на неразбираемия стринг в началното разбираемо съобщение се нарича *дешифриране*. Подслушвачът има достъп само до междинния, шифриран вариант на съобщението, от който не се научава нищо. Подслушвачът знае въпросните алгоритми, но не знае ключа. Ако подслушвачът не разполага с някаква информация за ключа и не разполага с алтернативни методи, единственото, което може да направи, е да започне да генерира на сляпо ключ след ключ и да опитва последователно да ги прилага, разчитайки, че ще налучка правилния ключ и ще разбере съобщението. Отново, **сигурността идва от високата сложност на атаката, която се състои в опитване на всички възможности**.

Важен дял от криптографията е криптографията с отворен ключ. При нея, всеки потребител има два ключа, публичен и секретен, като публичния ключ се излага—както показва и името—публично, а секретният се пази в тайна. Публичното излагане на секретния ключ би било фатално за сигурността. Ако потребител A иска да прати съобщение на потребител B, тя взема публичния ключ на B, шифрира с него съобщението си и го праща на B. На свой ред B, използвайки секретния си ключ, дешифрира съобщението. Съобщението може да се дешифрира само чрез секретния ключ на B. Подслушвачът вижда шифрирания вариант, но от него не научава нищо, защото не разполага със секретния ключ на B. Интересното е, че за всеки потребител, двата ключа, публичен и секретен, са взаимно зависими, но сложността на задачата за намиране на секретния ключ от публичния е огромна. Още веднъж, **сигурността идва от високата алгоритмична сложност на атаката**.

Криптографията с отворен ключ има решаващо значение за (почти) сигурното комуникиране в несигурна, публично достъпна среда. Ахилесовата пета на другата криптография—тази само с един ключ—е разпространението на ключа. Компютърните услуги като Интернет банкиране са немислими в масовия си вариант без криптография с публичен ключ.

Задълбочено изложение на връзката между високата сложност на дадени изчислителни задачи и криптографията с отворен ключ има в статиите на Fortnow [27] и Impaglaizzo [42]. Изключително детайлно въведение в криптографията е културната книга Applied Cryptography на Bruce Schneier [80].

[†]Аналогично, при секретните брави цялата сигурност е в информацията за височините на зъбите на ключа, при допускането, че противникът знае всичко останало като формата на профила на ключа, броя на щифтовете и т. н., и разполага с необходимата техника, за да си направи физически ключ от описанието му.

Лекция 3

Сортиране. Елементарни сортиращи алгоритми.

Резюме: Разглеждаме задачата Сортиране и нейната важност за решаването на други задачи. Въвеждаме понятието *стабилност на сортиращ алгоритъм*. Разглеждаме два елементарни сортиращи алгоритъма: Insertion Sort и Selection Sort и анализираме тяхната коректност, сложност по време и памет и стабилност. Особено внимание отделяме на коректността, която по правило доказваме чрез инвариант на цикъл.

3.1 Обща дефиниция

Понятието "преднаредба" е дефинирано и обяснено подробно в Допълнение 16.

Изч. Задача: Сортиране

пример: Редица $X = \langle a_1, a_2, \ldots, a_n \rangle$, чиито елементи са от някакво множество A, което е подходящо за сортиране и върху които е дефинирана релация на пълна преднаредба $\preccurlyeq \subseteq A \times A$.
решение: пермутация $\langle a'_1, a'_2, \ldots, a'_n \rangle$ на елементите на X, такава че $a'_1 \preccurlyeq a'_2 \preccurlyeq \cdots \preccurlyeq a'_n$.

Подчертаваме, че примерът на задачата е **редицата** $\langle a_1, a_2, \ldots, a_n \rangle$, а не (мулти)множеството $\{a_1, \ldots, a_n\}_M$.

Елементите a_1, $\ldots$, a_n не са непременно цели числа. "Елементи, подходящи за сортиране" означава елементи, за които е реалистично допускането, че имат големина единица и че може да бъдат сравнявани и местени в паметта в единица време. Може да са реални числа[†], а може да са някакви по-общи структури. На практика най-често възниква необходимостта да се сортират *записи*, всеки от които има *ключ* и някаква *сателитна информация*, като релацията $\preccurlyeq$ е дефинирана само върху множеството от ключовете. Ключовете във всеки запис може да са повече един и в такъв случай говорим за първичен ключ, вторичен ключ и така нататък.

С учебна цел ще разглеждаме основно сортиране на цели числа, а релацията $\preccurlyeq$ ще бъде добре познатата "по-малко или равно" $\leqslant$.

Съществува и алтернативна, по-абстрактна дефиниция на задачата Сортиране, в която примерът не е редица, а (мулти)множество. Това може да е много удобно при разглеждане

[†]Не е невъзможно да са дори дори комплексни числа, стига релацията $\preccurlyeq$ да е дефинирана смислено.

на долни граници (Глава 7). Примерно, в [71, стр. 6] задачата е дефинирана по този начин. Да се ограничим само върху реални числа.

Изч. Задача: Сортиране, абстрактна дефиция

пример: Мултимножество $\{a_1, a_2, \ldots, a_n\}_M$ от реални числа.
решение: За всяка двойка (i, j), такава че $1 \leqslant i < j \leqslant n$, отговор дали $a_i < a_j$.

Изглежда, че авторът на [71] смята, че е примерът е множество, тъй като, ако е мултимножество, отговорът на въпроса $a_i \overset{?}{<} a_j$ може да не е достатъчно информативен; тогава би трябвало да имаме отговорите на двата въпроса $a_i \overset{?}{<} a_j$ и $a_j \overset{?}{<} a_i$, за да установим "истинското отношение" между a_i и a_j. От друга страна, той говори за задачата Element Uniqueness [71, Problem 2] (вижте Подсекция 3.2.3) като за частен случай на Сортиране, което означава, че примерът е мултимножество – няма никакъв смисъл да питаме дали елементите на множество са уникални, защото знаем, че те са такива; уникалността на елементите е дефинираща характеристика за множествата.

Да оставим настрани въпроса дали примерът на Сортиране, абстрактна дефиниция е множество или мултимножество. Важното е друго. Наистина можем да гледаме на решението на задачата не като на редица, а като на колекция от отговорите на сравненията на двойките елементи на примера. Може дори да смятаме, че решението е пермутация, но тя не е реализирана чрез масив, а е абстрактна структура данни, която позволява да се правят запитвания (queries): при даден индекс i на елемента a_i от входа се връща броя на елементите от входа, по-малки от него, плюс едно; това е точно позицията, която би имал a_i в сортирания масив. По този начин, правейки запитвания за a_i и a_j, научаваме отговора на въпроса $a_i \overset{?}{<} a_j$. Пермутацията да бъде реализирана чрез масив—както е в "класическото" сортиране—е само една възможност за нейната реализация. Тя е добра и смислена, понеже позволява, ползвайки $\Theta(n)$ памет, да можем да отговорим в $\Theta(1)$ време на всеки от общо $\Theta(n^2)$ въпроса. Но абстрактната дефиниция е абстрактна именно защото отказваме да фиксираме реализация на пермутацията; при абстрактните типове данни реализацията е "черна кутия", в която не знаем какво има.

3.2 Защо сортирането е важно

По правило в практиката сортирането не е самоцелно, а е само първи етап от решаването на някаква задача. Има доста примери за важни задачи, за които бързото решение използва сортирането като предварителна обработка. Алгоритмичният фолклор казва, че ако съставител на алгоритми е изправен пред непозната задача, едно от първите неща, които трябва да опита, за да направи ефикасен алгоритъм, е да сортира някакви данни и да види дали от това няма да произтече нещо полезно [86, стр. 106].

Наблюдение 6

Предимството на сортираните данни над несортираните се корени в транзитивността на релацията $\lessdot$. Ако редицата $(a_1, a_2, \ldots, a_n)$ е сортирана и е известно, че $a_i \lessdot x$ за някое i и някакво x, то от транзитивността на $\lessdot$ заключаваме, че $a_j \lessdot x$ за всяко $j < i$.

3.2.1 Двоично търсене

В простия си вариант като задача за разпознаване, ТЪРСЕНЕ се дефинира така.

Изч. Задача: ТЪРСЕНЕ, ВЕРСИЯ ЗА РАЗПОЗНАВАНЕ

пример: $a_1, a_2, \ldots, a_n$, key: обекти от един и същи тип
въпрос: Дали има елемент измежду $a_1, a_2, \ldots, a_n$, който е равен на key?

На практика по-полезна е следната версия на задачата. Ще я наречем *оптимизационната версия*.

Изч. Задача: ТЪРСЕНЕ, ОПТИМИЗАЦИОННА ВЕРСИЯ

пример: $a_1, a_2, \ldots, a_n$, key: обекти от един и същи тип
решение: Ако има елемент измежду $a_1, a_2, \ldots, a_n$, който е равен на key, индексът на един такъв елемент. В противен случай индикация, че такъв елемент няма.

Ако входът на произволен алгоритъм за ТЪРСЕНЕ е произволна редица $a_1, a_2, \ldots, a_n$, то единствената възможност за действие е проверяване дали $a_i \stackrel{?}{=} \text{key}$ по всички $i \in \{1, \ldots, n\}$. Това се нарича *последователно търсене*. Най-лошият случай по отношение на сложността е, key да не е нито един от $a_1, a_2, \ldots, a_n$. Тогава всеки алгоритъм би трябвало да направи n сравнения в най-лошия случай. Очевидно е, че ако сме пропуснали да проверим дали $a_i \stackrel{?}{=} \text{key}$ за поне едно i, няма как да сме убедени, че key не се среща.

Ако обаче елементите $a_1, a_2, \ldots, a_n$ са уникални и са сортирани, можем да приложим *двоично търсене* (на английски е *binary search*), което е много по-бързо от последователното търсене.

```
BinSearchIter(A[1,...,n], key)
 1  (* итеративен вариант *)
 2  (* A е сортиран *)
 3  ℓ ← 1
 4  h ← n
 5  while ℓ ⩽ h do
 6      mid ← ⌊(ℓ+h)/2⌋
 7      if A[mid] = key
 8          return mid
 9      else if key < A[mid]
10          h ← mid − 1
11      else
12          ℓ ← mid + 1
13  return −1
```

```
BinSearchRec(A[ℓ,...,h], key)
 1  (* рекурсивен вариант *)
 2  (* A е сортиран *)
 3  if ℓ > h
 4      return −1
 5  else
 6      mid ← ⌊(ℓ+h)/2⌋
 7      if A[mid] = key
 8          return mid
 9      else if key < A[mid]
10          return
11              BinSearchRec(A[ℓ,...,mid − 1], key)
12      else
13          return
14              BinSearchRec(A[mid + 1,...,h], key)
```

Началното викане на рекурсивната версия е BinSearchRec$(A[1, \ldots, n], \text{key})$. Посочените два алгоритъма решават оптимизационната версия на ТЪРСЕНЕ. Накратко, тяхната коректност се обосновава така: ако изобщо елементът key се намира в масива A, той се намира в подмасива $A[\ell, \ldots, h]$. Това твърдение остава вярно след промените в индексите ℓ и h само защото масивът е сортиран предварително. Пълна обосновка има в Допълнение 18.

Сложността по време и на двата алгоритъма е $\Theta(\lg n)$ в най-лошия случай, което изведохме много подробно за итеративната версия в Подсекция 2.4.3. Нещо повече, точната сложност като брой сравнения $A[\text{mid}] \stackrel{?}{=} \text{key}$ е $\lfloor \lg n \rfloor + 1$ в най-лошия случай.

Разликата между последователното търсене в линейно време и двоичното търсене в логаритмично време е огромна. Ако, примерно, $n = 7\,000\,000\,000$, с не повече от 33 достъпа до масива можем да установим, че даден елемент не е във входа, ако данните са сортирани и ползваме двоично търсене. Сравнете 33 с $7\,000\,000\,000$!

Допълнение 18: Коректност на двоичното търсене

Първо ще докажем коректността на итеративната версия.

Теорема 27: Коректност на BinSearchIter

За всяко изпълнение на BinSearchIter($A[1,\ldots,n]$, key), ако key се среща в A, то BinSearchIter връща индекса на key в A, в противен случай BinSearchIter връща -1.

Доказателство: Следното твърдение е инвариант за **while**-цикъла на редове 5–12.

Инвариант 4: Цикълът на BinSearchIter

Всеки път, когато изпълнението на BinSearchIter е на ред 5, ако е вярно, че

① $A\left[\left\lfloor \frac{\ell+h}{2} \right\rfloor\right] \neq \text{key}$ и

② $\ell \leqslant h$

то, ако $\text{key} \in A[1,\ldots,n]$, то $\text{key} \in A[\ell,\ldots,h]$.

База. Разглеждаме първия път, когато изпълнението достигне ред 5. Заради присвояванията на редове 3 и 4, съждението "ако $\text{key} \in A[1,\ldots,n]$, то $\text{key} \in A[\ell,\ldots,h]$" е тавтология. Базата е вярна независимо от това дали ① е вярно или не.

Поддръжка. Да допуснем, че изпълнението е на ред 5 и ① и ② са в сила и, ако key се среща в $A[1,\ldots,n]$, то key непременно се намира в $A[\ell,\ldots,h]$; казано по друг начин, ако key се среща в $A[1,\ldots,n]$, то key не може да е нито в $A[1,\ldots,\ell-1]$, нито в $A[h+1,\ldots,n]$. Това че ① и ② са в сила влече, че тялото на цикъла ще се изпълни поне още веднъж, като изпълнението пак ще е на ред 5 и по този начин ще се убедим, че инвариантът се запазва.
На ред 6, на mid се присвоява $\left\lfloor \frac{\ell+h}{2} \right\rfloor$. Съгласно допускането ①, $A[\text{mid}] \neq \text{key}$, така че следните два случая са изчерпателни.

Случай I $\text{key} < A[\text{mid}]$. Щом $\text{key} < A[\text{mid}]$, то $\text{key} < A[j]$ за всяко $j \in \{\text{mid}+1,\ldots,n\}$; това е заради транзитивността на релацията $<$ и факта, че $A[1,\ldots,n]$ е сортиран. Но тогава key не може да се намира в $A[\text{mid},\ldots,h]$, така че, ако key се среща в A, key се намира в $A[\ell,\ldots,\text{mid}-1]$.
Очевидно условието на ред 9 е истина и на ред 10, на h се присвоява $\text{mid}-1$. Спрямо новото h е вярно, че при следващото достигане на ред 5, ако $\text{key} \in A[1,\ldots,n]$, то $\text{key} \in A[\ell,\ldots,h]$. С други думи, инвариантът се запазва.
Това, че инвариантът е импликация, не води до проблеми. Ако някое от ① или ② е лъжа, импликацията е истина, тъй като антецедентът ѝ е лъжа.

Случай II $\text{key} > A[\text{mid}]$. Щом $\text{key} > A[\text{mid}]$, то $\text{key} > A[j]$ за всяко $j \in \{\ell, \ldots, \text{mid}-1\}$; това е заради транзитивността на релацията $>$ и факта, че $A[1, \ldots, n]$ е сортиран. Но тогава key не може да се намира в $A[\ell, \ldots, \text{mid}-1]$, така че, ако key се среща в A, key се намира в $A[\text{mid}+1, \ldots, h]$.
Очевидно условието на ред 9 е лъжа и на ред 12, на ℓ се присвоява $\text{mid}+1$. Спрямо новото ℓ е вярно, че при следващото достигане на ред 5, ако $\text{key} \in A[1, \ldots, n]$, то $\text{key} \in A[\ell, \ldots, h]$. С други думи, инвариантът се запазва.

Терминация. Да допуснем, че изпълнението е на ред 5 за последен път. Има две причини тялото на цикъла да не се изпълни повече изцяло.

1. $\ell > h$. Съгласно инварианта, ако key се намира в $A[1, \ldots, n]$, то key се намира в $A[\ell, \ldots, h]$. В този случай обаче $A[\ell, \ldots, h]$ е празният масив, така че входният масив не съдържа key. От друга страна, изпълнението отива на ред 13, така че алгоритъмът коректно индикира отсъствието на key във входния масив, връщайки -1.

2. $A\left[\left\lfloor \frac{\ell+h}{2} \right\rfloor\right] = \text{key}$. Тогава очевидно изпълнението отива на ред 8 и алгоритъмът коректно връща индекса на key във входния масив. □

Сега ще докажем коректността на рекурсивната версия.

Теорема 28: Коректност на BinSearchRec

За всяко изпълнение на BinSearchRec$(A[\ell, \ldots, h], \text{key})$, ако key се среща в A, то BinSearchRec връща индекса на key в A, в противен случай BinSearchRec връща -1.

Доказателство: Доказателството е по индукция по изпълнението на алгоритъма, както подобава на доказателство за коректност на рекурсивен алгоритъм.

База. Базата на доказателството разглежда базата на рекурсията. Управляващата променлива за рекурсията е разликата $h - \ell$, така че проверката дали изпълнението е в базовия случай става на ред 3, а самата база е ред 4. Наистина, ако ℓ е по-голямо от h, масивът $A[\ell, \ldots, h]$ е празен, така че key не се среща в него и алгоритъмът връща -1 на ред 4.

Разглеждаме работата на алгоритъма на дадено ниво в рекурксията.

Индуктивно предположение. Допускаме, че ако бъде направено кое да е от рекурсивните викания на ред 10 или ред 13, то работи коректно. А именно, връща -1, ако key не се среща в съответния подмасив, или индекса на key, в противен случай.

Индуктивна стъпка. Следните три възможности са изчерпателни.

Случай I $A[\text{mid}] = \text{key}$. В този случай, от една страна е вярно, че key се среща в масива, а от друга страна на ред 8, алгоритъмът връща индекса на key.

Случай II $\text{key} < A[\text{mid}]$. В този случай, $\text{key} < A[j]$ за всяко $j \in \{\text{mid}+1, \ldots, h\}$; това е заради транзитивността на релацията $<$ и факта, че $A[1, \ldots, n]$ е сортиран. Но тогава key не може да се намира в $A[\text{mid}, \ldots, h]$, така че, ако key се среща в $A[\ell, \ldots, h]$, key се намира в $A[\ell, \ldots, \text{mid}-1]$.

Булевото условие на ред 9 е истина и рекурсивното викане на ред 10 се извършва. Съгласно индуктивното предположение, викането върху $A[\ell, \ldots, \text{mid} - 1]$ на ред 10 работи коректно, откъдето следва, че викането върху $A[\ell, \ldots, h]$ работи коректно.

Случай III $\text{key} > A[\text{mid}]$. В този случай, $\text{key} > A[j]$ за всяко $j \in \{\ell, \ldots, \text{mid} - 1\}$; това е заради транзитивността на релацията $<$ и факта, че $A[1, \ldots, n]$ е сортиран. Но тогава key не може да се намира в $A[\ell, \ldots, \text{mid}]$, така че, ако key се среща в $A[\ell, \ldots, h]$, key се намира в $A[\text{mid} + 1, \ldots, h]$.
Булевото условие на ред 9 е лъжа, поради което се извършва рекурсивното викане на ред 13. Съгласно индуктивното предположение, викането върху $A[\text{mid} + 1, \ldots, h]$ на ред 13 работи коректно, откъдето следва, че викането върху $A[\ell, \ldots, h]$ работи коректно. □

Следствие 12

Началното викане BinSearchRec$(A[1, \ldots, n], \text{key})$ работи коректно: ако key се намира в $A[1, \ldots, n]$, алгоритъмът връща индекса му, в противен случай връща -1.

Една забележка за двоичното търсене. Двоичното търсене не е приложимо, ако има елементи с еднаква стойност, които все пак различаваме някак. Те може да се намират в произволен ред един спрямо друг след сортирането. Примерно, елементите имат първичен и вторичен ключ, но са сортирани само по първичен ключ, което влече, че елементите с еднакъв първичен и различен вторичен ключ може да са подредени произволно. Ако търсим точно определена наредена двойка (x, y) от първичен и вторичен ключ, в най-лошия случай трябва да прегледаме последователно всички елементи с първичен ключ x, за да се убедим, че измежду тях нито един не е с вторичен ключ y. С други думи, търсенето се изражда в последователно търсене, защото наредбата не е линейна. И така, за да работи двоичното търсене, трябва елементите да са подредени с линейна наредба.

3.2.2 Най-близки елементи

Ето два варианта на задачата.

Изч. Задача: Най-близки Елементи, вар. 1

пример: $a_1, a_2, \ldots, a_n$: цели числа; $n \geqslant 2$
решение: $\min\{|a_i - a_j| : 1 \leqslant i < j \leqslant n\}$

Изч. Задача: Най-близки Елементи, вар. 2

пример: $a_1, a_2, \ldots, a_n$: цели числа; $n \geqslant 2$
решение: Двойка елементи (a_i, a_j), такива че $i \neq j$ и разликата $|a_i - a_j|$ е минимална.

Възможно е масивът да съдържа еднакви елементи. В такъв случай, в първата версия решението е 0, а във втората, наредена двойка (a_i, a_j) за кои да е a_i, a_j, такива че $a_i = a_j$.

Не е задължително елементите да са цели числа, но трябва да бъдат от множество, върху което е дефинирана пълна преднаредба, и трябва да е дефинирано разстояние между всеки

два елемента, което да се изчислява в константно време. В горната дефиниция разстоянието е $|a_i - a_j|$.

Наивното решение на задачата е да се тестват всички двуелементи подмножества така.

```
CLOSEST PAIR, NAÏVE(A[1,...,n])
 1  closest ← ∞
 2  for i ← 1 to n − 1
 3      for j ← i + 1 to n
 4          if |A[i] − A[j]| < closest
 5              closest ← |A[i] − A[j]|
 6              first ← A[i]
 7              second ← A[j]
 8  return (first, second)
```

Сложността по време на това решение е $\Theta(n^2)$.

По-добро решение е, първо елементите да бъдат сортирани и след това с едно единствено сканиране на сортираната последователност отляво надясно най-близката двойка ще бъде намерена. Сега не е необходимо да ползваме нотация за абсолютна стойност около $A[i+1] - A[i]$, защото след сортирането, $A[i+1] - A[i]$ е задължително неотрицателно.

```
CLOSEST PAIR, SOPHISTICATED(A[1,...,n])
 1  SORT(A)
 2  closest ← ∞
 3  for i ← 1 to n − 1
 4      if A[i + 1] − A[i] < closest
 5          closest ← |A[i + 1] − A[i]|
 6          first ← A[i]
 7          second ← A[i + 1]
 8  return (first, second)
```

Доказателството за коректност се основава на простото наблюдение, че ако минималното разстояние между кои да два елемента от входа е δ, то след сортирането задържително има съседни елементи, разстоянието между които е δ; в противен случай би имало два елемента от входа, разстоянието между които е дори по-малко.

Както ще видим в следваща лекция, сортирането може да бъде имплементирано със сложност $\Theta(n \lg n)$, така че бързото решение има сложност

$$\underbrace{\Theta(n \lg n)}_{\text{сортиране}} + \underbrace{\Theta(n)}_{\text{сканиране}} = \Theta(n \lg n),$$

което е много по-добре от $\Theta(n^2)$.

3.2.3 Уникалност на елементите

Изч. Задача: Уникалност на елементите

пример: $a_1, a_2, \ldots, a_n$: цели числа; $n \geqslant 2$
въпрос: Вярно ли е, че няма нито два еднакви елемента измежду дадените?

Напълно аналогично на НАЙ-БЛИЗКИ ЕЛЕМЕНТИ, наивното решение се състои в тестване на всички двуелементни подмножества, което означава $\Theta(n^2)$ тестове в най-лошия случай. Ако обаче първо сортираме числата, ако има еднакви елементи, всички елементи, които са два по два еднакви, задължително ще се появят в непрекъсната подредица в сортираната редица. С едно сканиране отляво надясно ще установим дали има еднакви елементи. Отново, сложността на подхода, използващ първо сортиране във време $\Theta(n \lg n)$, е $\Theta(n \lg n)$.

3.2.4 Мода

Мода е най-често срещана стойност измежду дадени стойности. Забележете, че тази стойност не е непременно уникална – ако няма повторения на елементи, всеки елемент е мода (поради което не е коректно да се казва "най-често срещана**та** стойност"). Мода може да се изчисли във време $\Theta(n \lg n)$ чрез предварително сортиране, което работи във време $\Theta(n \lg n)$. Ето едно възможно решение за намиране на мода в $\Theta(n \lg n)$ чрез сортиране, последвано от едно сканиране на сортираните данни.

```
COMPUTE MODE(A[1, 2, ..., n])
 1  SORT(A)
 2  mode ← A[1]
 3  m ← 1
 4  for i ← 2 to n
 5      if A[i − 1] = A[i]
 6          if A[i − 1] = mode
 7              m ← m + 1
 8          else
 9              s ← s + 1
10              if s > m
11                  mode ← A[i]
12                  m ← s
13      else
14          s ← 1
15  return mode
```

Допълнение 19: Коректността на COMPUTE MODE

Както вече казахме, модата не е непременно уникална. Алгоритъмът връща *най-малката мода*. За краткост, навсякъде до края на Допълнение 19, когато казваме "мода" имаме предвид най-малката мода.

Следното определение е в сила **само в рамките на това доказателство**.

Определение 26: група

Нека $A[1, 2, \ldots, n]$ е масив от числа. *Група в* A е всеки максимален по включване подмасив от еднакви елементи. *Дължината на групата* е броят на елементите ѝ. Ако групата се казва X, дължината ѝ се бележи с $|X|$. *Представител на групата* е стойността на кой да е от нейните елементи.
По отношение на даден индекс $i \in \{2, \ldots, n\}$, казваме, че $A[i]$ *разширява група*, ако $A[i] = A[i-1]$, и че $A[i]$ *започва нова група*, ако $A[i] \neq A[i-1]$. $A[i]$ *разширява група-мода*, ако $A[i]$ разширява група и $A[i-1]$ е мода на $A[1, \ldots, i-1]$. $A[i]$ *превръща група в група-мода*, ако $A[i]$ разширява група и $A[i-1]$ не е мода на $A[1, \ldots, i-1]$, но $A[i]$ е мода на $A[1, \ldots, i]$. $A[i]$ *разширява група-немода*, ако $A[i]$ разширява група, но не е мода на $A[1, \ldots, i]$. □

Забележете, че ако масивът е сортиран и $A[i]$ започва нова група, то $A[i]$ не може да е мода при $i \geqslant 2$.

Наблюдение 7

Нека $A[1, 2, \ldots, n]$ има точно t различни елемента и е сортиран. Тогава A се състои от точно t групи, чиято сумарна дължина е n и чиито представители се появяват в строго нарастващ ред.

Наблюдение 8

За всяко $i \in \{2, \ldots, n\}$, има точно четири, две по две изключващи се, възможности за $A[i]$. Ще бележим тези възможности с буквите Q_1, …, Q_4.

Q_1: $A[i]$ разширява група-мода.

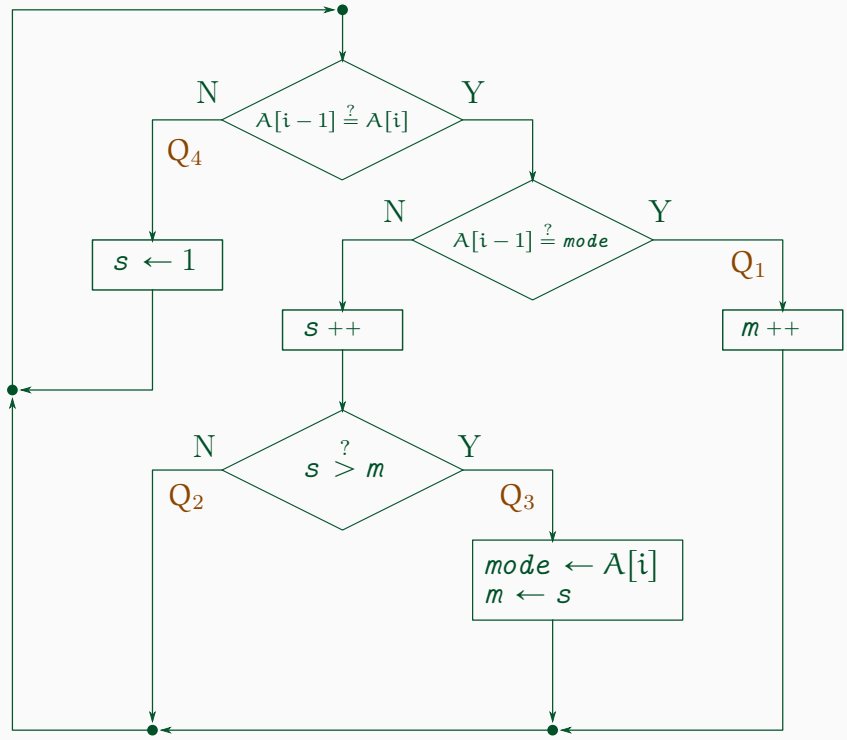
Q_2: $A[i]$ разширява група-немода.

Q_3: $A[i]$ превръща група в група-мода.

Q_4: $A[i]$ започва нова група.

Удобно е да мислим за Q_1, …, Q_4 като за *състояния*. След сортирането, алгоритъмът сканира еднократно масива отляво надясно и след прочитането на всеки пореден елемент преминава неявно в едно от четирите състояния. Неявно, защото никъде не се говори експлицитно за състояния. Но трите вложени **if**-а в някакъв смисъл ни дават именно това: четири различни състояния. Най-вътрешният **if** няма **else**, но това е без значение – можем да мислим, че има празен **else**. При всяко преминаване през тялото на цикъла, в зависимост от истинността на трите булеви условия, точно едно от четирите възможни състояние се реализира (имплицитно). Фигура 3.1 показва блок-схема на тялото на цикъла на COMPUTE MODE, на която ясно се вижда кое преминаване през тялото на цикъла на кое състояние отговаря.
Въвеждането на тези четири състояния няма отношение към доказателството за коректност – то не ги използва никъде. Въвеждаме тези четири състояния, за да обосновем конструкцията на алгоритъма. Иначе казано, те са полезни за **дизайна**, а не за **анализа** на алгоритъма.

Фигура 3.1 : Блок-схема на цикъла на Compute Mode.



Следното твърдение е инвариант на **for**-цикъла (редове 1–14).

Инвариант 5: Compute Mode

При всяко достигане на ред 4 на Compute Mode:

① *mode* съдържа стойността на модата на $A[1,\ldots,i-1]$.

② *m* съдържа броя на нейните появи в $A[1,\ldots,i-1]$.

③ Ако $A[i-1]$ не е мода на $A[1,\ldots,i-1]$, то *s* съдържа дължината на най-дясната група в $A[1,\ldots,i-1]$.

База. Нека изпълнението е на ред 4 за първи път. Тогава $i = 2$ и подмасивът $A[1,\ldots,i-1]$ е всъщност $A[1]$. Модата на $A[1]$ очевидно е $A[1]$ и броят на нейните появи е 1. Да видим дали всички съждения на инварианта са верни.

① *mode* $= A[1]$ заради присвояването на ред 2. ✓

② *m* $= 1$ и заради присвояването на ред 3. ✓

③ Това твърдение също е вярно. То е импликация, чийто антецедент е лъжа, понеже $A[i-1]$ е мода на $A[1,\ldots,i-1]$. Импликацията, чийто антецедент е лъжа, е истина. ✓

Поддръжка. Да допуснем, че инвариантът е в сила на някое изпълнение на ред 4, което не е последно.

Случай I $A[i-1] = A[i]$. Условието на ред 5 е истина и изпълнението отива на ред 6.

Случай I.1 $A[i-1]$ е мода на $A[1,\ldots,i-1]$. Този подслучай е Q_1: $A[i]$ разширява група-мода. Съгласно индуктивното предположение, изпълнено е $A[i-1] = \mathit{mode}$. Очевидно и $A[i] = \mathit{mode}$. Условието на ред 6 е истина и изпълнението отива на ред 7. Да видим дали всички съждения на инварианта са верни.

① Щом mode е мода на $A[1,\ldots,i-1]$ и $A[i] = \mathit{mode}$, то mode е мода и на $A[1,\ldots,i]$. Преди следващото достигане на ред 4, i се инкрементира. Спрямо новото i е в вярно, че mode е мода на $A[1,\ldots,i-1]$. ✓

② Изпълнява се ред 7 и m се инкрементира. Сега вече m съдържа броя на появите на модата на $A[1,\ldots,i]$. Преди следващото достигане на ред 4, i се инкрементира. Спрямо новото i е в вярно, че m съдържа броя на появите на модата на $A[1,\ldots,i-1]$. ✓

③ Знаем, че $A[i-1]$ е мода на $A[1,\ldots,i-1]$ преди инкрементирането на i. От това и от факта, че $A[i-1] = A[i]$ преди инкрементирането на i следва, че след инкрементирането на i отново е вярно, че $A[i-1]$ е мода на $A[1,\ldots,i-1]$. Следователно, антецедентът на импликацията е лъжа, а импликацията е истина. ✓

Случай I.2 $A[i-1]$ не е мода на $A[1,\ldots,i-1]$. Тогава, съгласно индуктивното предположение, $A[i-1] \neq \mathit{mode}$.
Първо ще покажем, че част ③ на инварианта се запазва и при двете възможности—лъжа или истина—за булевото условие на ред 10. Тези две възможности отговарят съответно на Q_2: $A[i]$ разширява група-немода и Q_3: $A[i]$ превръща група в група-мода. Щом $A[i-1]$ не е мода на $A[1,\ldots,i-1]$, то част ③ от индуктивното предположение казва, че s съдържа дължината на най-дясната група в $A[1,\ldots,i-1]$. Става дума за момента, в който изпълнението е на ред 8. Изпълнява се ред 9, където s се инкрементира. Тъй като $A[i-1] = A[i]$, очевидно новата стойност на s съдържа дължината на най-дясната група в $A[1,\ldots,i]$. Независимо от това дали булевото условие на ред 10 е истина или лъжа, s не се променя до следващото достигане на ред 4 (редове 11 и 12 не променят s). Точно преди следващото достигане на ред 4, i се инкрементира. Спрямо новата стойност на i е вярно, че s съдържа дължината на най-дясната група в $A[1,\ldots,i-1]$. И така, от една страна текущото $A[i]$ не е мода на текущия $A[1,\ldots,i-1]$, а от друга страна s съдържа дължината на най-дясната група в текущото $A[1,\ldots,i-1]$. Следва, че част ③ на инварианта се запазва в **Случай I.2**. ✓

Да се върнем към момента, в който изпълнението беше на ред 8. Сега ще разгледаме части ① и ② на инварианта. Помним, че променливата s, съгласно индуктивното предположение, съдържа дължината на най-дясната група на $A[1,\ldots,i-1]$. Изпълнението отива на ред 9, където s се инкрементира. Тъй като $A[i-1] = A[i]$, то новата стойност на s съдържа дължината на най-дясната група в $A[1,\ldots,i]$. Изпълнението отива на ред 10.

Случай I.2.a $A[i]$ не е мода на $A[1,\ldots,i]$. В текущия контекст, това е същото като да кажем, че най-дясната група на $A[1,\ldots,i]$, чиято дължина се съдържа в променливата s, има не по-голяма дължина от броя на появите на модата на $A[1,\ldots,i]$, който брой

се съдържа в променлива `m` поради индуктивното предположение и това, че $A[i]$ не е мода на $A[1,\ldots,i]$. С други думи, **Случай I.2.а** отговаря на Q_2: $A[i]$ разширява група-немода. Щом $s \leqslant m$, то условието на ред 10 е лъжа и изпълнението се връща на ред 4. `mode` и `m` остават непроменени. Променливата i се инкрементира. Спрямо новата ѝ стойност, `mode` съдържа модата на $A[1,\ldots,i-1]$, а `m` съдържа броя на появите ѝ. Показахме, че части ① и ② на инварианта се запазват.

Случай I.2.б $A[i]$ е мода на $A[1,\ldots,i]$. В текущия контекст, това е същото като да кажем, че най-дясната група на $A[1,\ldots,i]$, чиято дължина се съдържа в инкрементираната на ред 9 променлива `s`, има по-голяма дължина (с единица) от броя на появите на модата на $A[1,\ldots,i-1]$, който брой се съдържа в променлива `m` (от индуктивното предположение). Тогава текущото `s` съдържа броя на появите на модата на $A[1,\ldots,i]$. И така, $A[i-1]$ не е мода на $A[1,\ldots,i-1]$, но същата стойност ($A[i]$ е равна на $A[i-1]$) е мода на $A[1,\ldots,i]$. С други думи, **Случай I.2.б** отговаря на Q_3: $A[i]$ превръща група в група-мода. Щом $s > m$, то условието на ред 10 е истина и се изпълняват редове 11 и 12. Сега `mode` съдържа модата на $A[1,\ldots,i]$, а `m` съдържа броя на появите ѝ. После променливата i се инкрементира. Спрямо новата ѝ стойност, `mode` съдържа модата на $A[1,\ldots,i-1]$, а `m` съдържа броя на появите ѝ. Показахме, че части ① и ② на инварианта се запазват.

Случай II $A[i-1] \neq A[1]$ (изпълнението е на ред 5). Лесно се вижда, че в този случай $A[i]$ не може да е мода на $A[1,\ldots,i]$. Следователно, модата на $A[1,\ldots,i]$ е същата като на $A[1,\ldots,i-1]$, и броят на появите ѝ в $A[1,\ldots,i]$ е същият като в $A[1,\ldots,i-1]$. Съгласно индуктивното предположение, `mode` съдържа модата на $A[1,\ldots,i]$, а `m` съдържа броя на появите ѝ $A[1,\ldots,i]$.

Условието на ред 5 е лъжа и изпълнението отива на ред 13, след което на ред 14, променливата `s` става единица и изпълнението се връща на ред 4, като i се инкрементира. Спрямо новата стойност на i:

① `mode` е мода на $A[1,\ldots,i-1]$. ✓

② `m` съдържа броя на появите на модата на $A[1,\ldots,i-1]$. ✓

③ От една страна, $A[i-1]$ не е мода на $A[1,\ldots,i-1]$ (говорим за новата стойност на i), от друга страна най-дясната група на $A[1,\ldots,i-1]$ спрямо новата стойност на i има дължина единица, а от трета страна `s` съдържа единица. ✓

□

Модата не може да бъде изчислена във време, което не е $\Omega(n \lg n)$. Това е доказано в Подсекция 7.3.2.

3.2.5 Медиана

Нека е даден масив от n стойности. *Медиана* е стойността, от която половината стойности са по-малки или равни, а другата половина, по-големи или равни. Строго погледнато, тази дефиниция на медиана е смислена само когато броят на стойностите е нечетно число и под "половината стойности" разбираме "$\left\lfloor \frac{n}{2} \right\rfloor$ стойности". Следното определение е смислено за произволен брой на елементите.

Определение 27: Медиана

Нека $A[1, \ldots, n]$ е масив от цели числа. *Медианата* на A е елементът $A\left[\left\lfloor \frac{n+1}{2} \right\rfloor\right]$ след сортиране на A.

Съществува още по-детайлно определение на "медиана". Тъй като при четно n има два естествени кандидата за медиана, това определение казва, че масивите с четен брой елементи имат две медиани: елементът на позиция $\left\lfloor \frac{n+1}{2} \right\rfloor$ в сортирания масив е *долната медиана*, а този на позиция $\left\lceil \frac{n+1}{2} \right\rceil$ в сортирания масив е *горната медиана*. Вижте [21, стр. 213].

Медиана се пресмята лесно, ако използваме сортиране, понеже самата дефиниция на медиана използва сортиране неявно: медианата е елементът, стоящ в средата на сортираната последователност на тези елементи. В следния алгоритъм $(n/2)$ означава целочислено деление.

COMPUTE MEDIAN($A[1, 2, \ldots, n]$, n е нечетно)
1 SORT(A)
2 **return** $A[(n/2) + 1]$

Заслужава да се отбележи, че медианата може да бъде изчислена в $\Theta(n)$ чрез алгоритъма, намиращ k-тото по големина число измежду n дадени числа в *линейно време* за произволно k. Този алгоритъм може да бъде намерен в [21], а и ние ще го дискутираме в Подсекция 5.4.2.

3.2.6 2SUM, 3SUM, kSUM

Нека навсякъде в тази подсекция k е цяло число, по-голямо или равно на 2.

Дефиниция на задачата Широко разпространеното разбиране е, че това е задача за разпознаване.

Изч. Задача: kSUM, версия за разпознаване

пример: Множество от реални числа S.
въпрос: Дали има $a_1, \ldots, a_k \in S$, такива че $a_1 + \cdots + a_k = 0$?

Естествено обобщение е следната оптимизационна задача.

Изч. Задача: kSUM, оптимизационна версия

пример: Множество от реални числа S.
решение: Броят на подмножествата $\{a_1, \ldots, a_k\} \subseteq S$, такива че $a_1 + \cdots + a_k = 0$.

Задачите 2SUM и 3SUM са частни случаи при съответно $k = 2$ и $k = 3$. В тази подсекция ще се фокусираме върху 2SUM и 3SUM, като последната сякаш е най-важна на практика заради приложенията си в изчислителната геометрия.

Как помага сортирането За всяко k има наивен алгоритъм със сложност $\Theta(n^k)$[†], който просто опитва всички подмножества с мощност k. Ако обаче числата от S са предварител-

[†]Сложността на наивния алгоритъм е $\Theta(n^k)$ само ако смятаме, че k е константа. В противен случай сложността е $\Omega\left(k\binom{n}{k}\right)$, понеже подмножествата с мощност k са точно $\binom{n}{k}$, а всяко от тях бива тествано във време $\Theta(k)$.

но сортирани, то можем да "свалим" една единица от степенния показател: за 2SUM има алгоритъм със сложност $\Theta(n)$ (тук **не отчитаме** времето за сортирането!), за 3SUM има алгоритъм със сложност $\Theta(n^2)$, и така нататък.

В Подподсекция 2.1.3.4 видяхме такъв алгоритъм за 2SUM във версия за разпознаване, наречен Alg2SUM. Там се казваше, че числата са цели, но това не е съществено по никакъв начин. Нищо не се променя, ако числата са реални, при допускането, че сложността на базовите операции върху число е константна.

Alg2SUM решава вариант на задачата, в който се иска числата a_1 и a_2 да се сумират не непременно до нула, а до някакво число m, което е част от входа. Този вариант обаче не е съществено различен от "класическия" вариант на сумиране до нула. Забележете, че $a_1 + a_2 = m$ е същото като $(a_1 - m/2) + (a_2 - m/2) = 0$, така че привидно по-общият вариант се свежда до "класическия", ако предварително извадим $m/2$ от всяко число.

Напълно аналогично, алгоритъмът Alg3SUMCount от Подподсекция 2.1.3.5, който е за оптимизационната версия и в който трите числа се сумират до нула и това е заложено в кода, може да бъде използван за намиране на броя на тройките, които се сумират до каквото число m пожелаем: просто изваждаме $m/3$ от всички числа в началото и тогава

$$a_1 + a_2 + a_3 = m \leftrightarrow (a_1 - m/3) + (a_2 - m/3) + (a_3 - m/3) = 0$$

Дали става дума за различни елементи от S? Тук имаме предвид задачата за разпознаване. Възниква въпросът, дали $a_1, \ldots, a_k$ са непременно два по два различни елементи на S. Това **не е** същият въпрос като въпросът дали S е мултимножество или обикновено множество. Дори S да е обикновено множество, ако мислим за $a_1, \ldots, a_k$ като за променливи, изразът "$a_1, \ldots, a_k \in S$" казва, че те вземат стойности от S и нищо не пречи няколко или дори всички от тях да имат една и съща стойност. При тази интерпретация, ако S съдържа 0, то отговорът на въпроса от kSUM е тривиално Да, понеже, ако $a_1 = \cdots = a_k = 0$, очевидно $a_1 + \cdots + a_k = 0$. Примерно, при $k = 3$, ако настояваме $a_1 \neq a_2 \neq a_3 \neq a_1$, то отговорът за примера $\{0, 1, 2\}$ е Не, но без това ограничение отговорът е Да заради възможността $a_1 = a_2 = a_3 = 0$.

На пръв поглед е много важно да се уточни дали въпросните a_i са уникални или не, защото това променя задачата. По-внимателно разглеждане показва, че същината на задачата е при уникални a_i в смисъл, че дори да допускаме различни a_i-та да имат като стойност един и същи елемент от S, това не добавя значителна сложност към алгоритъма, който дава отговора. Примерно:

- Ако $k = 2$, да разрешим a_1 и a_2 да съдържат един и същи елемент от S може да даде възможност за различен отговор (Да вместо Не) само ако S съдържа 0. А това дали S съдържа 0 може да се провери в линейно време, така че асимптотиката на алгоритъма-решение не се променя.

- Ако $k = 3$, да разрешим a_1, a_2 и a_3 да не са уникални елементи от S може да даде възможност за различен отговор (Да вместо Не) в следните случаи:

  - S съдържа 0 и тогава отговорът е тривиално Да.
  - S съдържа $c \neq 0$ и $d \neq 0$, като $c = -d/2$. Тогава отговорът е Да, понеже за $a_1 = d$ и $a_2 = a_3 = c$ е вярно, че $a_1 + a_2 + a_3 = 0$. Наличието на елементи с обратни знаци, единият от които по абсолютна стойност е половината от другия, може да се установи в линейно време, ако числата са сортирани предварително, с алгоритъм, сходен с Alg2SUM.

Ако S е мултимножество? Тук допускаме, че $a_1, \ldots, a_k$ са два по два различни елементи от входа, но входът може да е мултимножество. Пояснение: ако, примерно, мултимножеството е $\{s_1, s_2, s_3\}_M$ и $s_1 = s_2 = 0$, а $s_3 = 1$, то s_1 и s_2 са различни елементи, въпреки че имат една и съща стойност; ерго, по отношение на 3SUM във версия за разпознаване, отговорът за примера $\{0, 0, 1\}_M$ е НЕ[†].

Възможността да има повтарящи се числа във входа е без значение за задачата във версията за разпознаване, но има отношение към оптимизационната версия. Лесно се вижда, че Alg3SUMCount от Подподсекция 2.1.3.5 не работи върху мултимножества: при всяко откриване на триада на ред 6 алгоритъмът инкрементира брояча d на ред 7, а на ред 8 "мръдва" и двата индекса p и q един към друг. Това е коректно само ако елементите от входа са уникални; ако има повторения, може да "изтървем" триади по този начин. Прочее, в екстремния случай, в който входът се състои само от нули, решението е $\binom{n}{3}$. Веднага се вижда, че Alg3SUMCount не работи коректно върху вход само от нули, защото алгоритъмът има сложност по време $\Theta(n^2)$, а решението е $\Theta(n^3)$, така че е невъзможно само с инкрементации на d с единица, които са най-много квадратичен брой, да постигнем кубично решение. По друг начин казано, всеки коректен алгоритъм за задачата 3SUM в оптимизационен вариант, който акумулира резултата в брояч, който брояч се инкрементира с единица и не се манипулира по никакъв друг начин, има сложност по време $\Omega(n^3)$, ако входът е мултимножество.

Заслужава да се отбележи, че 3SUM в оптимизационен вариант може да се реши и с квадратичен алгоритъм дори върху примери-мултимножества, ако решението е само бройката на триадите, а не самите триади. Това може да стане, ако след предварителното сортиране "кондензираме" числения масив, като наместо всички появи на едно и също число запишем само числото и броя на неговите появи. Иначе казано, кондензираният масив е от наредени двойки (x, y), където x е число от входа, а y е броят на неговите появи във входа. След това използваме идеята на Alg3SUMCount с незначителната разлика, че при откриване на триада инрементираме d така: $\boxed{d \leftarrow d + \alpha \cdot \beta \cdot \gamma}$, където α е броят срещания на A[i], β е броят срещания на A[p], а γ е броят срещания на A[q].

Този трик обаче открива само броя на триадите, състоящи се от елементи с различни стойности. Към този брой трябва да добавим още:

- Броя на триадите, състоящи се от три елемента от входа с еднаква стойност. Тази стойност може да е само нула, така че добавяме $\binom{k_0}{3}$, където k_0 е броят на нулите във входа.

- Броя на триадите, състоящи се от два елемента с една и съща стойност s и още един елемент с различна стойност t. Трябва или $s < 0$ и $t > 0$, или $s > 0$ и $t < 0$. Освен това трябва $|s| = |t|/2$. Ако броят на елементите във входа със стойност s е α, а броят на елементите във входа със стойност t е β, добавяме $\binom{\alpha}{2} \cdot \beta$ за всяка такава двойка s, t. Откриването на тези двойки може да стане в линейно време върху сортирания, кондензиран масив с идея, подобна на идеята на Alg2SUM.

Приложения на задачата 3SUM Редица важни задачи в областта на компютърната геометрия съдържат в себе си задачата 3SUM в смисъл, че съществува редукция на 3SUM до всяка една от тях. Много такива задачи са посочени в статията на Gajentaan и Overmars [30]. Понятието "редукция" ще въведем чак в Лекция 7, в Секция 7.3. Тук само ще спомена, че ако бъде открит алгоритъм за 3SUM (във версията за разпознаване) със сложност $O(n^{2-\epsilon})$, за колкото и да е малко реално положително ϵ, за всяка от тези задачи също ще има алгоритъм

[†]Ако обаче разрешавахме $a_1, \ldots, a_k$ да не са непременно два по два различни елементи от входа, то отговорът за $\{0, 0, 1\}_M$ щеше да е Да.

със сложност $O(n^{2-\epsilon})$; но ако се докаже, че няма алгоритъм за 3SUM със сложност $O(n^{2-\epsilon})$, за никое реално положително ϵ, от това следва, че за никоя от тези задачи няма алгоритъм със сложност $O(n^{2-\epsilon})$.

Иначе казано, всяка от тези задачи се решава чрез едно решаване на 3SUM, плюс "още малко". Какво ще рече това, ще видим в Секция 7.3 и Подсекция 7.3.4.

3.3 Анализ на сортиращи алгоритми. Стабилност.

Анализът на даден сортиращ алгоритъм се състои в това, което изброихме миналата лекция за анализа на произволен алгоритъм: доказателство на коректността и анализ на сложността по време и по памет. В контекста на сортиращите алгоритми има още едно свойство, което представлява интерес. То се нарича *стабилност*. Сега ще обясним в детайли какво означава сортиращ алгоритъм да бъде стабилен с един пример от култовата книга The Art of Computer Programming на Knuth [56]. Knuth означава с N броя на елементите във входа, а сортираната пермутация с "$K_{p(1)} \leqslant K_{p(2)} \leqslant \ldots \leqslant K_{p(N)}$".

Задача 3: задача 2 на стр. 5 в [56]

Да допуснем, че всеки запис R_j в даден файл съдържа два ключа, първичен ключ K_j и вторичен ключ k_j, като върху ключовете е дефинирана линейна наредба $<$. Дефинираме *лексикографска наредба* върху ключовете по стандартния начин:

$$(K_i, k_i) < (K_j, k_j) \text{ ако } K_i < K_j \text{ или } K_i = K_j \text{ и } k_i < k_j$$

Alice взема файла и го сортира първо по първичните ключове, получавайки n групи от записи с еднакви първични ключове във всяка група:

$$K_{p(1)} = \ldots = K_{p(i_1)} < K_{p(i_1+1)} = \ldots = K_{p(i_2)} < \ldots < K_{p(i_{n-1}+1)} = \ldots = K_{p(i_n)},$$

където $i_n = N$. След това тя сортира всяка от n на брой групите по вторичния ключ.
Bill взема оригиналния файл и първо го сортира по вторичния ключ, а след това, по първичния.
Chris взема оригиналния файл и прави едно единствено сортиране, като обаче ползва и двата ключа с лексикографската наредба.
Дали всеки от тях е получил непременно един и същи резултат?

Отговорът е, със сигурност Alice и Chris ще получат един и същи резултат. Резултатът на Bill ще е гарантирано същият само при условие, че неговото второ сортиране—това по първичния ключ—не променя относителния ред на записи, имащи еднакъв (първичен) ключ, който ред е създаден от вече извършеното първо сортиране. Ето малък пример: нека наредените двойки от ключовете са $(1,2)$, $(5,2)$, $(4,5)$, $(2,5)$, $(3,1)$, $(1,4)$, $(3,2)$. Сортираната с лексикографска наредба последователност е:

$$(1,2), (1,4), (2,5), (3,1), (3,2), (4,5), (5,2)$$

Това е резултатът, който ще получат Alice и Chris след своите сортирания.

Какво точно ще получи Bill след **първото** сортиране–това по вторичните ключове—зависи от сортиращия алгоритъм, който ползва. Със сигурност

- на първа позиция ще е наредената двойка $(3,1)$,

- после ще са трите наредени двойки с втори елемент 2, но не е ясно в какъв ред
- после $(1, 4)$,
- и накрая двете наредени двойки с втори елемент 5, но не е ясно в какъв ред.

Примерно, Bill може да получи това след първото сортиране:

$$(3, 1), (5, 2), (3, 2), (1, 2), (1, 4), (4, 5), (2, 5)$$

но може да получи и това:

$$(3, 1), (3, 2), (5, 2), (1, 2), (1, 4), (2, 5), (4, 5)$$

Сега Bill прави **второ** сортиране, този път по първичните ключове. В получената наредба на първите две места са наредените двойки $(1, 2)$ и $(1, 4)$. Но забележете, че при произволно сортиране по първи ключ няма как да знаем дали $(1, 2)$ ще е вляво от $(1, 4)$ или не. От гледна точка на коректна лексикографска сортировка, разбира се, искаме началото да бъде

$$(1, 2), (1, 4), \ldots$$

но може да се получи и

$$(1, 4), (1, 2), \ldots$$

тъй като при сортирането по първични ключове не "гледа" вторичните ключове. **По отношение на сортирането само по първични ключове, $(1, 2)$ и $(1, 4)$ са еднакви елементи!** Първата фаза—сортирането по вторични ключове—със сигурност слага $(1, 2)$ преди $(1, 4)$, защото $2 < 4$, но втората фаза може потенциално да сложи $(1, 4)$ вляво от $(1, 2)$, ако ги счита за еднакви.

Определение 28: Стабилно сортиране

Стабилно сортиране е сортиране, което не променя взаимното разположение на елементи с еднакви ключове.

При стабилно сортиране, за всеки два елемента с еднакви клочюве, които елементи са все пак различими (заради, примерно, вторични ключове), този, който е вляво от другия **преди** сортирането, задължително ще е вляво от него и **след** сортирането.

Наблюдение 9

Всеки нестабилен сортиращ алгоритъм може да бъде направен стабилен, ако се добави първоначалната позиция на всеки елемент като вторичен ключ[a] и се сортира по наредените двойки (първичен ключ, вторичен ключ) – като Chris от Задача 3. Ако направим това обаче, може да влошим сложността по памет, понеже $\Theta(n)$ допълнителна памет ще се използва от новите ключове, използвани за стабилизиране. Ако преди това изкуствено стабилизиране сложността по памет е била $\Theta(1)$, ще я влошим до $\Theta(n)$. Следователно, няма как да използваме този трик върху алгоритъм, който е in-place и той да остане in-place.

[a] Това е в случай, че поначало има само един ключ. Ако поначало има k ключа, добавяме още един, $(k+1)$-ви, най-младши ключ с първоначалните позиции.

3.4 Елементарни Сортиращи Алгоритми

Разделянето на сортиращите алгоритми на елементарни и други, неелементарни, е доста условно. Под елементарни сортиращи алгоритми разбираме такива, които биха хрумнали бързо на човек, който за пръв път се сблъсква със СОРТИРАНЕ. По правило те работят във време $\Theta(n^2)$ в най-лошия случай и не използват никакви изтънчени структури данни или нетривиални идеи.

3.4.1 INSERTION SORT

INSERTION SORT сортира по начин, който далечно напомня на начина, по който картоиграч сортира картите, които държи в ръка (които са в произволна наредба в началото) – плъзгайки поглед отляво надясно, той или тя разглежда последователно картите. При това всяка карта, която е обект на внимание в момента, бива сложена на правилното ѝ място в подпоследователността от картите вляво. Има една важна разлика между този начин на нареждане и работата на алгоритъма INSERTION SORT: карта може да бъде пъхната между другите карти мигновено, докато ако искаме да "пъхнем" дадено a_i някъде вляво, първо трябва да му направим място, за да не бъде затрит елементът, който вече е там.
Да си припомним псевдокода на INSERTION SORT

INSERTION SORT($A[1, 2, \ldots n]$: array of integers)
```
1  for i ← 2 to n
2      key ← A[i]
3      j ← i − 1
4      while j > 0 and A[j] > key do
5          A[j + 1] ← A[j]
6          j ← j − 1
7      A[j + 1] ← key
```

3.4.1.1 Коректност на INSERTION SORT

Доказателството за коректност ще направим чрез инвариант на цикъла. Целта е да докажем, че INSERTION SORT сортира всеки свой вход.

Допълнение 20: Неформално за доказателствата за коректност

Доказателствата за коректност на алгоритми са тежки, защото обектите, за които трябва да изказваме и доказваме твърдения, са динамични. Променливите се менят с работата на алгоритъма заради присвояванията. Ако в доказателството за коректност на INSERTION SORT говорим за $A[i]$, какво имаме предвид? Дали елемента в клетка i на входа, тоест преди началото на алгоритъма? Или елемента, който е бил в клетка i на масива в началото на текущата итерация на външния цикъл? Или имаме предвид клетка i в даден конкретен момент от изпълнението, който момент разглеждаме в доказателството? В общия случай това са **различни** елементи.

В това допълнение ще дадем интуицията зад формалното доказателство за коректността, а Лема 20 и Теорема 29 ще бъдат сухото и формално доказателство за коректност.

Най-общо казано, схемата на доказателството на коректността на INSERTION SORT е следната. Това, което искаме да докажем е, че алгоритъмът терминира върху всеки

вход и винаги, когато терминира, масивът A се състои от началните елементи, но в сортиран вид. Това е прекалено общо и е в сила за всеки сортиращ алгоритъм.

Сега да говорим конкретно за INSERTION SORT. Схемата за доказателството на коректността му е следната. Разглеждаме **само едно изпълнение** на външния цикъл (редове 1–7). По отношение на **това изпълнение** на външния цикъл, ефектът от работата на алгоритъма е следният. Ако в начало му е вярно, че:

- подмасивът $A[1,\ldots,i-1]$ се състои от същите елементи като входния подмасив $A[1,\ldots,i-1]$, но в сортиран вид
- и подмасивът $A[i,\ldots,n]$ е същият като входния $A[i,\ldots,n]$[a],

то в самия му край—това означава, точно след изпълнението на ред 7, но преди следващото инкрементиране на управляващата променлива i на ред 1—е вярно, че:

- подмасивът $A[1,\ldots,i]$ се състои от същите елементи като входния $A[1,\ldots,i]$, но в сортиран вид,
- и подмасивът $A[i+1,\ldots,n]$ е същият като входния $A[i+1,\ldots,n]$.

Разговорно казано, сортираната зона в A "върви" от левия край надясно и на всяко изпълнение на външния цикъл нараства с една клетка, но задължително се състои от същите елементи, които са били в нея в началото на алгоритъма; а частта извън сортираната зона, която пък намалява с една клетка, също се състои от същите елементи, които са били там в началото.
И всичко това формулирахме без изобщо да разглеждаме вътрешния цикъл. Дали то е приемливо за доказателство за коректност на алгоритъма или не, зависи от това, колко прецизно доказателство искаме. Посоченото доказателство е прекалено недетайлно и неформално и, ако искаме голяма прецизност, то е незадоволително – не е доказано, че ефектът от едно изпълнение на външния цикъл е такъв, какъвто тук се твърди, че е. Това не е доказателство, а схема, по която може да бъде направено детайлно доказателство.

Сега ще обясним как става нарастването на сортираната зона с една позиция. Разглеждаме в детайли **едно изпълнение на външния цикъл**. Допускаме, че в началния му момент подмасивът $A[1,\ldots,i-1]$ е сортиран и се състои точно от елементите, които са били на позиции $1,\ldots,i-1$ във входа. В този момент $A[i]$ може да "не си е на мястото" в $A[1,\ldots,i]$, защото $A[i]$ може да е по-малък от някакви елементи от $A[1,\ldots,i-1]$. В първата стъпка от изпълнението на **for**-цикъла, в променливата ***key*** се слага $A[i]$ и после се изпълнява вътрешният цикъл (редове 4–7). Ефектът от работата на вътрешния цикъл е следният. Нека t_1 е моментът на първоначалното достигане на ред 4, тоест самото начало на изпълнението на вътрешния цикъл, а t_2 е моментът на достигане на ред 7, тоест веднага след излизането от вътрешния цикъл. И така, в t_2:

- $A[1,\ldots,j]$ е същият като $A[1,\ldots,j]$ от t_1 и се състои точно от тези елементи на $A[1,\ldots,i-1]$ от t_1, които са по-малки или равни на ***key***, в сортиран вид;
- $A[j+2,\ldots,i]$ е същият като $A[j+1,\ldots,i-1]$ от t_1 и се състои точно от тези елементи на $A[1,\ldots,i-1]$ от t_1, които са по-големи от ***key***, в сортиран вид;

- клетка $A[j+1]$ може да се смята за празна, защото стойността, която е била в нея в t_1, е запазена другаде. А именно,
  - в $A[j+2]$, ако вътрешният цикъл е бил изпълнен поне веднъж, или
  - в *key*, в противен случай.

Разговорно казано, ако гледаме само $A[1,\ldots,i]$, в момента t_2, $j+1$ е "правилното място" на елемента, който се е намирал в $A[i]$ в t_1. "Правилното място" е по отношение на елементите, които са по-малки или равни на него. Те се намират в $A[1,\ldots,j]$ в t_1 и не се местят от вътрешния цикъл[б]. Вътрешният цикъл пресмята това $j+1$, като освен това премества с единица надясно точно тези елементи на $A[1,\ldots,i]$, които са по-големи от елемента $A[i]$ в t_1, без да ги размества взаимно; тоест, мести ги като блок.

Това вече е много по-детайлно обяснение, но все още не е достатъчно детайлно – все още не сме видели как действа една итерация на вътрешния цикъл. Сега разглеждаме в детайли **едно изпълнение на вътрешния цикъл**. Нека t_1 и t_2 имат същото значение, каквото имаха досега, и нека t' е моментът на някое достигане на ред 4, което не е последното, а t'' е следващото достигане на ред 4 (очевидно $t_1 < t' < t'' < t_2$). Ако в t':

- $A[j+2,\ldots,i]$ е сортиран и всеки елемент от него е по-голям от *key* и
- $A[1,\ldots,j]$ е сортиран и е същият като в t_1 и
- клетка $A[j+1]$ може да се счита за празна

то в t'':

- $A[j+1,\ldots,i]$ е сортиран и всеки елемент от него е по-голям от *key* и
- $A[1,\ldots,j-1]$ е сортиран и е същият като в t_1 и
- клетка $A[j]$ може да се счита за празна

Това вече е пълна схема на доказателство, което може да бъде извършено по индукция. Естественият ред на формалното доказателство по отношение на вложените цикли е, отвътре навън.

[а]Това е абсолютно очевидно от кода на алгоритъма, но за пълнота го формулираме отделно.

[б]Забележете, че $j+1$ е правилното място за $A[i]$ именно по отношение на елементите, които са по-малки или равни на на него. По отношение на елементите, които са по-големи от него, правилното място е j, а не $j+1$. Но това няма значение, защото сме избрали да местим елементите, които са по-големи от $A[i]$, а да оставяме на място тези, които са по-малки или равни.

Навсякъде в доказателствата долу допускаме, че $n > 1$. Използваме израза "клетка номер x на масива Y може да бъде смятана за свободна" – това означава, че съдържанието ѝ е запазено другаде и може да бъде презаписана, без първоначално намиращият се там елемент да бъде унищожен безвъзвратно.

Лема 20: Коректност на вътрешния цикъл на Insertion Sort

Спрямо **едно единствено** изпълнение на **for** цикъла на Insertion Sort (редове 1–7), нека наричаме масива A непосредствено преди изпълнението на **while** цикъла (редове 4–6) с името A'. Да допуснем, че подмасивът $A'[1,\ldots,i-1]$ е сортиран. Тогава в момента на приключване на **while** цикъла е вярно, че:

① $A[1,\ldots,j]$ се състои точно от елементите на $A'[1,\ldots,i-1]$, които са по-малки или равни на key, в сортиран вид;

② $A[j+2,\ldots,i]$ се състои точно от елементите на $A'[1,\ldots,i-1]$, които са по-големи от key, в сортиран вид;

③ клетката $A[j+1]$ е свободна;

④ $A[i+1,\ldots,n]$ е същият като $A'[i+1,\ldots,n]$.

Доказателство: Следното твърдение е инвариант за **while** цикъла.

Инвариант 6: Вътрешният цикъл на Insertion Sort

Всеки път, когато изпълнението е на ред 4:

❶ $A[1,\ldots,j] = A'[1,\ldots,j]$;

❷ $A[j+2,\ldots,i] = A'[j+1,\ldots,i-1]$;

❸ $\forall x \in A[j+2,\ldots,i] : x > key$;

❹ $A[i+1,\ldots,n] = A'[i+1,\ldots,n]$.

*С учебна цел, да повторим нещо, което вече видяхме на стр. 39. Доказателството на инварианта е доказателство по индукция. Започва с **база**, в която верността на предиката просто се проверява. В обикновените доказателства по индукция следващите две фази са: индуктивно предположение и индуктивна стъпка. Тук ги сливаме в една фаза от доказателството, която наричаме **поддръжка**. И накрая има последна фаза **терминация**, която няма еквивалент при "обикновените" доказателства по индукция, които са върху* безкрайни *индуктивни дефинирани множества. При доказателствата за коректност на алгоритми винаги доказваме твърдение върху* крайно *индуктивно генерирано множество, тъй като алгоритмите винаги терминират. Терминацията се отнася до последното достигане на условието на цикъла – когато тялото няма да се изпълни нито веднъж повече. Когато заместим стойността на управляващата променлива от този момент в инварианта, трябва да получим точно това твърдение, което ни трябва за доказателство на коректността.*

База. Ще покажем ❶. Когато изпълнението достигне до ред 4 за първи път, текущият масив A е A' по определение, така че наистина $A[1,\ldots,j] = A'[1,\ldots,j]$.

Ще покажем ❷. Когато изпълнението достигне до ред 4 за първи път, $j = i-1$ заради присвояването на ред 3. Тогава $A[j+2,\ldots,i]$ всъщност е $A[i+1,\ldots,i]$, който подмасив е празен, а $A'[j+1,\ldots,i-1]$ всъщност е $A'[i,\ldots,i-1]$, който подмасив също е празен.

❸ е изпълнено, защото $A[j+2,\ldots,i]$ е празен, а за всеки елемент на празното множество е изпълнен всеки предикат. ❹ е очевидно.

Поддръжка. Да допуснем, че твърдението е в сила в даден момент t, в който изпълнението е на ред 4 и **while** ще бъде изпълнен поне още веднъж. Последното влече, че $j > 0$ и $A[j] > \textit{key}$ в t. Нека t_{next} означава момента на следващото достигане на ред 4.

Ще покажем, че ❶ е в сила в t_{next}. Първо разглеждаме присвояването на ред 5. То не променя нищо в подмасива $A[1,\ldots,j-1]$ и той остава равен на $A'[1,\ldots,j-1]$. Сега разглеждаме декрементирането на ред 6. Непосредствено след него, спрямо новата стойност на j, същият този подмасив може да бъде означен с "$A[1,\ldots,j]$", а също така и с "$A'[1,\ldots,j]$". И така, подмасивът $A[1,\ldots,j]$ в t_{next} е същият като $A'[1,\ldots,j]$ и е сортиран.

Ще покажем, че ❷ е в сила в t_{next}. Първо разглеждаме присвояването на ред 5. От индуктивното предположение ❷ знаем, че $A[j+2,\ldots,i] = A'[j+1,\ldots,i-1]$. От индуктивното предположение ❶ следва, че $A[j] = A'[j]$. Тогава след $A[j+1] \leftarrow A[j]$ на ред 5 е вярно, че $A[j+1,j+2,\ldots,i] = A'[j,j+1,\ldots,i-1]$. Обаче подмасивът $A'[j,j+1,\ldots,i-1]$ е сортиран по допускане, така че текущият подмасив $A[j+1,j+2,\ldots,i]$ е сортиран. Сега разглеждаме декрементирането на ред 6. Непосредствено след него, спрямо новата стойност на j е вярно, че $A[j+2,\ldots,i] = A'[j+1,\ldots,i-1]$ и това е сортиран подмасив. В t_{next} това остава вярно.

Ще покажем, че ❸ е в сила в t_{next}. Допуснали сме, че $\forall x \in A[j+2,\ldots,i] : x > \textit{key}$ в t. Знаем, че $A[j] > \textit{key}$, иначе тази итерация на **while**-а не би се изпълнявала. Първо разглеждаме присвояването на ред 5. Непосредствено след него е вярно, че $\forall x \in A[j+1,\ldots,i] : x > \textit{key}$. Сега разглеждаме декрементирането на ред 6. След него, спрямо новата стойност на j е вярно, че $\forall x \in A[j+2,\ldots,i] : x > \textit{key}$ при следващото достигане на ред 4.

Това, че ❹ е в сила при следващото достигане на ред 4, е очевидно.

Терминация. Да разгледаме момента, в който изпълнението е на ред 4 и условието там е FALSE. Тогава $j \leqslant 0$ или $\textit{key} \geqslant A[j]$.

Случай 1: $j \leqslant 0$. Тъй като j намалява с единица, то не може да стане отрицателно, следователно $j = 0$. Заместваме j с 0 в инварианта и получаваме:

❶ $A[1,\ldots,0] = A'[1,\ldots,0]$;

❷ $A[2,\ldots,i] = A'[1,\ldots,i-1]$;

❸ $\forall x \in A[2,\ldots,i] : x > \textit{key}$;

❹ $A[i+1,\ldots,n] = A'[i+1,\ldots,n]$.

Очевидно в този случай в $A'[1,\ldots,i-1]$ няма елементи, по-малки или равни на $\textit{key}$. Но тогава е вярно, че

① $A[1,\ldots,0]$ се състои точно от елементите на $A'[1,\ldots,i-1]$, които са по-малки или равни на $\textit{key}$, в сортиран вид; `// става дума за празен подмасив`

② $A[2,\ldots,i]$ се състои точно от елементите на $A'[1,\ldots,i-1]$, които са по-големи от $\textit{key}$, в сортиран вид; `// става дума за целия` $A'[1,\ldots,i-1]$

③ клетката $A[1]$ е свободна;

④ $A[i+1,\ldots,n]$ е същият като $A'[i+1,\ldots,n]$.

Което е точно твърдението на лемата при $j = 0$.

Случай 1: Сега да допуснем, че $j > 0$. Тогава $\textit{key} \geqslant A[j]$. Да разгледаме отново инварианта. Той е:

❶ $A[1,\ldots,j] = A'[1,\ldots,j]$;

❷ $A[j+2,\ldots,i] = A'[j+1,\ldots,i-1]$;

❸ $\forall x \in A[j+2,\ldots,i] : x > \mathit{key}$;

❹ $A[i+1,\ldots,n] = A'[i+1,\ldots,n]$.

От ❶ следва, че $A[j] = A'[j]$. Но тогава $\mathit{key} \geqslant A'[j]$. Щом $A[1,\ldots,j] = A'[1,\ldots,j]$ и $A[1,\ldots,i-1]$ е сортиран, то $A[1,\ldots,j]$ е сортиран. Щом $A[1,\ldots,j]$ е сортиран и $\mathit{key} \geqslant A'[j]$, то $\forall x \in A[1,\ldots,j] : x \leqslant \mathit{key}$. От това и ❷ и ❸ следва, че

① $A[1,\ldots,j]$ се състои точно от елементите на $A'[1,\ldots,i-1]$, които са по-малки или равни на key, в сортиран вид.

Да разгледаме пак ❷ и ❸. От тях и ① и фактът, че $A'[1,\ldots,i-1]$ е сортиран следва, че

② $A[j+2,\ldots,i]$ се състои точно от елементите на $A'[1,\ldots,i-1]$, които са по-големи от key, в сортиран вид.

От ❶, ❷ и присвояването на ред 2 следва, че

③ клетката $A[j+1]$ е свободна.

Това, че ④ е вярно, е очевидно. Доказахме лемата. □

Теорема 29: Коректност на Insertion Sort

Insertion Sort е сортиращ алгоритъм.

Доказателство: Да наречем входа A''. Следното твърдение е инвариант за **for** цикъла:

Инвариант 7: Външният цикъл на Insertion Sort

Всеки път, когато изпълнението на Insertion Sort е на ред 1, текущият подмасив $A[1,\ldots,i-1]$ се състои точно от същите елементи като $A''[1,\ldots,i-1]$, но в сортиран вид. Освен това, текущият подмасив $A[i,\ldots,n]$ се състои от точно от същите елементи като $A''[i,\ldots,n]$.

База. Първият път, когато изпълнението достигне ред 1 е вярно, че $i = 2$. Текущият подмасив $A[1,\ldots,1]$ се състои от един единствен елемент $A''[1]$ и е тривиално сортиран. Освен това, текущият подмасив $A[2,\ldots,n]$ се състои от точно от същите елементи като $A''[2,\ldots,n]$. ✓

Поддръжка. Да допуснем, че твърдението е изпълнено при някое достигане на ред 1 и **for** цикълът ще бъде изпълнен още един път. Нека $\widetilde{A}$ е името на A при **това достигане** на ред 1. Индуктивното допускане, в изразено чрез $\widetilde{A}$, е:

> $\widetilde{A}[1,\ldots,i-1]$ се състои точно от същите елементи като $A''[1,\ldots,i-1]$, но в сортиран вид, и $\widetilde{A}[i,\ldots,n]$ се състои от точно от същите елементи като $A''[i,\ldots,n]$.

Изпълнява се ред 2 и сега key съдържа $\widetilde{A}[i]$. Съгласно индуктивното допускане, това е еквивалентно на твърдението, че сега key съдържа $A''[i]$. Следва $j \leftarrow i-1$. После се изпълнява **while**-цикълът. Изпълнението на **while**-цикъла терминира. В момента, в който **while** цикълът терминира, съгласно Лема 20 са в сила следните твърдения:

① $A[1,\ldots,j]$ се състои точно от елементите на $\tilde{A}[1,\ldots,i-1]$, които са по-малки или равни на key, в сортиран вид;

② $A[j+2,\ldots,i]$ се състои точно от елементите на $\tilde{A}[j+1,\ldots,i-1]$, които са по-големи от key, в сортиран вид;

③ клетката $A[j+1]$ е свободна;

④ $A[i+1,\ldots,n]$ е същият като $\tilde{A}[i+1,\ldots,n]$.

Имайки предвид индуктивното предположение, тези твърдения стават:

① $A[1,\ldots,j]$ се състои точно от елементите на $A''[1,\ldots,i-1]$, които са по-малки или равни на key, в сортиран вид;

② $A[j+2,\ldots,i]$ се състои точно от елементите на $A''[1,\ldots,i-1]$, които са по-големи от key, в сортиран вид;

③ клетката $A[j+1]$ е свободна;

④ $A[i+1,\ldots,n]$ е същият като $A''[i+1,\ldots,n]$.

Тъй като в key вече се намира съдържанието на $A''[i]$, можем да твърдим, че в момента след присвояването на ред 7, текущият $A[1,\ldots,i]$ се състои от същите елементи като $A''[1,\ldots,i]$, но в сортиран вид. Изпълнението отива отново на ред 1 и i се инкрементира. Спрямо новата стойност на i е вярно, че:

> текущият $A[1,\ldots,i]$ се състои от същите елементи като $\tilde{A}[1,\ldots,i]$, а $A[1,\ldots,i-1]$ се състои от същите елементи като $A''[1,\ldots,i-1]$.

И така, инвариантът се запазва.

Терминация. Да разгледаме момента, в който изпълнението е на ред 1 за последен път. Очевидно i е равно на $n+1$. Заместваме i с $n+1$ в инварианта и получаваме "текущият подмасив $A[1,\ldots,(n+1)-1]$ се състои от същите елементи като $A''[1,\ldots,(n+1)-1]$, но в сортиран вид". Накратко, "текущият подмасив $A[1,\ldots,n]$ се състои от същите елементи като $A''[1,\ldots,n]$, но в сортиран вид". □

3.4.1.2 Сложност по време на Insertion Sort

Вече показахме в (2.9) на стр. 70, че сложността по време в най-лошия случай на Insertion Sort е $\Theta(n^2)$.

3.4.1.3 Сложност по памет на Insertion Sort

Insertion Sort очевидно ползва $\Theta(1)$ допълнителна памет.

3.4.1.4 Стабилност на Insertion Sort

Insertion Sort е стабилен сортиращ алгоритъм. Прецизното доказателство остава за упражнение. Тук само ще спомена, че алгоритъмът е стабилен заради използването на строго неравенство $>$ на ред 4; **while** цикълът престава да се изпълнява за първата (максималната) стойност на j, за която $A[j]$ става равен на key и заради това всички двойки равни елементи остават в същия относителен ред, в какъвто са били в началото. Ако наместо строго равенство ползвахме $\geqslant$ на това място, алгоритъмът нямаше да е стабилен.

3.4.2 Selection Sort

Да си припомним псевдокода на Selection Sort.

```
Selection Sort(A[1, 2, ... n]: array of integers)
1   for i ← 1 to n − 1
2       for j ← i + 1 to n
3           if A[j] < A[i]
4               swap(A[i], A[j])
```

3.4.2.1 Коректност на Selection Sort

Следващите две леми обосновават коректността на Selection Sort. Това, че Selection Sort само размества елементите на входа и в края на алгоритъма елементите в масива са точно тези, които са били в началото, е очевидно: единствените промени в A стават на ред 4 чрез размени (swaps). Това отличава тази реализация на Selection Sort от реализацията на Insertion Sort, която вече разгледахме. В последната не беше очевидно, че елементите в края са точно тези, които са били в началото, защото реализацията на Insertion Sort ползва не размени, а присвоявания. Поради това при анализа на Insertion Sort се наложи да обосновем защо нито един от оригиналните елементи не се "губи", бивайки презаписан с нещо друго. При анализа на Selection Sort нямаме такива притеснения.

Лема 21

По отношение на едно изпълнение на външния **for** цикъл (редове 1–4) на Selection Sort, изпълнението на вътрешния **for** цикъл (редове 2–4) има следния ефект: $A[i]$ е най-малък елемент в $A[i, \ldots, n]$.

Доказателство:
По отношение на едно изпълнение на външния **for** цикъл, следното твърдение е инвариант за вътрешния **for** цикъл:

Инвариант 8: Вътрешният цикъл на Selection Sort

Всеки път, когато изпълнението достигне ред 2, текущият $A[i]$ е минимален елемент в $A[i, \ldots, j-1]$.

База. Първият път, когато изпълнението на вътрешния **for** цикъл е на ред 2 е вярно, че $j = i + 1$. Тогава $A[i]$ е тривиално най-малък елемент в $A[i, \ldots, (i+1)-1]$.

Поддръжка. Да допуснем, че твърдението е вярно в някой момент, когато изпълнението е на ред 2 и предстои вътрешният **for** цикъл да бъде изпълнен още веднъж. Следните два случая са изчерпателни.

Случай 1. $A[j] < A[i]$. Условието на ред 3 е True и размяната на ред 4 се случва. Съгласно допускането, $A[i]$ е минимален елемент в $A[i, \ldots, j-1]$. Тъй като $A[j] < A[i]$, съгласно транзитивността на релацията $<$, $A[j]$ е минималният елемент в подмасива $A[i, \ldots, j]$ преди размяната. Тогава $A[i]$ е минималният елемент в $A[i, \ldots, j]$ след размяната . След размяната j нараства с единица и изпълнението отива на ред 2. По отношение на новата стойност на j е вярно, че $A[i]$ е минималният елемент в $A[i, \ldots, j-1]$.

Случай 2. $A[j] \not< A[i]$. Тогава условието на ред 3 е False и размяната на ред 4 не се случва. Съгласно предположението, $A[i]$ е минимален елемент в $A[i, \ldots, j-1]$. Тъй като $A[i] \leqslant A[j]$,

очевидно $A[i]$ е минимален елемент в $A[i, \ldots, j]$. После j нараства с единица и изпълнението отива на ред 2. По отношение на новата стойност на j е вярно, че $A[i]$ е минимален елемент в $A[i, \ldots, j-1]$.

Терминация. Да разгледаме момента, в който изпълнненението е на ред 2 за последен път. Очевидно j е равно на $n+1$. Заместваме $n+1$ на мястото на j в инварианта и получаваме "текущият $A[i]$ е минимален елемент в $A[i, \ldots, (n+1)-1]$". □

Лема 22

SELECTION SORT е сортиращ алгоритъм.

Доказателство:
На наречем оригиналния масив $A'[1, \ldots, n]$. Следното е инвариант за външния **for** цикъл (редове 1–4):

Инвариант 9: Външният цикъл на SELECTION SORT

Всеки път, кгато изпълнението на SELECTION SORT е на ред 1, текущият подмасив $A[1, \ldots, i-1]$ се състои от $i-1$ на брой най-малки елементи на $A'[1, \ldots, n]$ в сортиран вид.

База. При първото достигане на ред 1 е вярно, че $i = 1$. Текущият подмасив $A[1, \ldots, i-1]$ е празен и се състои от нула на брой най-малки елементи от $A'[1, \ldots, n]$ в сортиран вид.

Поддръжка. Да допуснем, че твърдението е в сила при някакво достигане на ред 1, което не е последното. Нека наричаме масива A в този момент с името A''. Съгласно Лема 21, ефектът на вътрешния **for** цикъл е, че поставя на $i^{\text{-тата}}$ позиция най-малка стойност от $A''[i, \ldots, n]$. От друга страна, съгласно индуктивното допускане, $A''[1, \ldots, i-1]$ се състои от $i-1$ на брой най-малки елементи на $A'[1, \ldots, n]$ в сортиран вид. Заключаваме, че в края на изпълнението на външния **for** цикъл, текущият $A[1, \ldots, i]$ се състои от i на брой най-малки елемента от $A'[1, \ldots, n]$ в сортиран вид. После i нараства с единиция и изпълнението отива на ред 1. По отношение на новата стойност на i е вярно, че текущият $A[1, \ldots, i-1]$ съдържа $i-1$ на брой най-малки елемента от $A'[1, \ldots, n]$ в сортиран вид.

Терминация. Да разгледаме момента, в който изпълнението е на ред 1 за последен път. Очевидно i е равно на n. Заместваме n на мястото на i в инварианта и получаваме "текущият подмасив $A[1, \ldots, n-1]$ се състои от $n-1$ на брой най-малки елемента на $A'[1, \ldots, n]$ в сортиран вид". Но тогава текущият $A[n]$ е максимален елемент от $A'[1, \ldots, n]$. С това доказателството за коректност на SELECTION SORT приключва. □

3.4.2.2 Сложност по време на SELECTION SORT

Вече показахме в (2.12) на стр. 70, че сложността по време в най-лошия случай на SELECTION SORT е $\Theta(n^2)$.

3.4.2.3 Сложност по памет на SELECTION SORT

SELECTION SORT очевидно ползва $\Theta(1)$ допълнителна памет.

3.4.2.4 Стабилност на Selection Sort

В този си вид Selection Sort не е стабилен. За да се убедим в това, да разгледаме малък подходящ пример. Нека входът е

2, 3, 4, 2, 1

За да различаваме двойките, да ги означим така:

$2'$, 3, 4, $2''$, 1

Очевидно след първото изпълнение на външния **for** цикъл единицата ще отиде на най-лява позиция, бивайки разменена с $2'$:

1, 3, 4, $2''$, $2'$

и крайният резултат ще е

1, $2''$, $2'$, 3, 4

Лекция 4

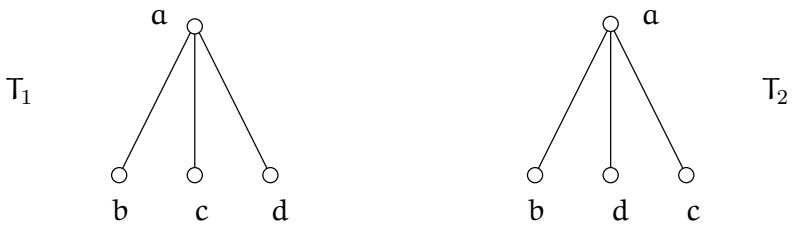
Двоична Пирамида. Heapsort. Приоритетна Опашка.

Резюме: Въвеждаме понятието попълнено двоично дърво и двоична пирамида. Показваме два начина да се строи двоична пирамида: наивен начин във време $\Theta(n \lg n)$ и бързото построяване във време $\Theta(n)$ на Floyd. Демонстрираме сортиращия алгоритъм Heapsort и го анализираме. Въвеждаме понятието приоритетна опашка като абстрактен тип данни и разискваме имплементация на приоритетна опашка чрез двоична пирамида.

4.1 Двоични дървета и пирамиди

4.1.1 Попълнено двоично дърво

Тъй като теорията на графите е сравнително нова област, няма универсално приета пълна единна терминология и всяко сериозно изложение, използващо графови понятия, трябва да започва с изчерпателни определения, за да е ясно точно какво се има предвид. Терминологията на английски ще изложим според американския Национален Институт за Стандарти и Технологии NIST. Техните определения за дървета са изложени на тази страница. Не допускаме дървета без върхове. Разглеждаме само коренови дървета, така че когато кажем "дърво" (tree), имаме предвид кореново дърво (rooted tree). В контекста на структури от данни по правило дърветата са *наредени*[†] (ordered trees), което означава, че децата на вътрешните върхове са наредени с линейна наредба. Следните две дървета T_1 и T_2 са различни като наредени дървета (наредбата е отляво надясно), но са едно и също дърво, ако ги разглеждаме като коренови дървета:



Ако T е кореново дърво с корен r, за всеки връх u в него с $T[\![u]\!]$ бележим поддървото, вкоренено в u. Формално,

- ако $u = r$, то $T[\![u]\!]$ е самото T,

[†]За разлика от "чистата" теория на графите, в която по правило кореновите дървета не са наредени.

- иначе, $\mathsf{T}[\![u]\!]$ е това от двете дървета на $\mathsf{T} - (u, \text{parent}(u))$, което съдържа u,

като $\mathsf{T}[\![u]\!]$ е кореново дърво с корен u.

Определение 29: Височина, дълбочина, нивá

Височина на връх u в кореново дърво е максималното разстояние между u и кое да е листо в $\mathsf{T}[\![u]\!]$. *Височина на дървото* е височината на корена. *Дълбочина на връх* u в кореново дърво е разстоянието между него и корена. *Ниво* в дърво е множеството от всички върхове, които са на една и съща дълбочина. *Номер на ниво* в дърво е дълбочината на кой да е връх от това ниво. *Съседни нивá* са нивá, чиито дълбочини се различават с единица.

Двоично дърво в теорията на графите е кореново дърво, наредено или не, в което всеки връх има не повече от две деца. За целите на тези лекции обаче, под "двоично дърво" ще разбираме, дърво, което задължително е наредено, но и нещо повече: различаваме ляво от дясно дете. С други думи, ако има връх с точно едно дете, това дете е или ляво, или дясно, и ние различаваме тези възможности. Следните две дървета D_1 и D_2 са различни като двоични дървета, защото детето в D_1 е ляво, а в D_2 е дясно, но са едно и също дърво, ако ги разглеждаме като подредени дървета, защото детето е само едно:



Определение 30: попълнено двоично дърво, не-индуктивно определение

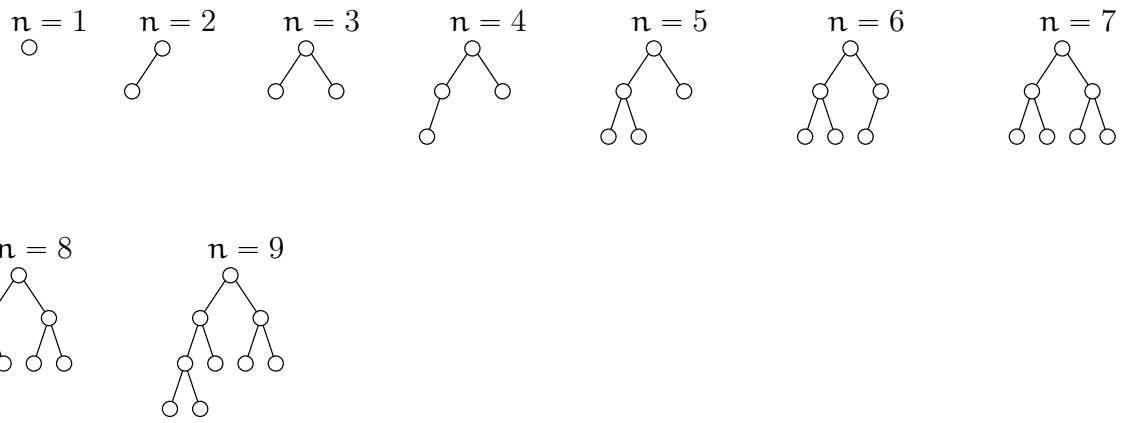
Попълнено двоично дърво (complete binary tree) е наредено двоично дърво, в което:

- всички нива, с изключение може би на последното, имат максималния възможен брой[а] върхове[б].
- ако листата са на две нива, листата на последното ниво са максимално вляво. □

[а] Ниво номер k може да има най-много 2^k върха в себе си.
[б] Следствие от това е, че всички листа са или в едно ниво, или в две нивá, които са съседни.

На английски терминът е *complete binary tree* ([69]).

За всяко $n \in \mathbb{N}^+$ има едно единствено попълнено дърво с n върха:



Съвършено двоично дърво (perfect binary tree) е попълнено двоично дърво, в което всички листа са на едно и също ниво [70].

Определение 30 е еквивалентно на Определение 31.

Определение 31: попълнено двоично дърво, индуктивно определение

База:

1. $(\{u\}, \varnothing)$ е попълнено двоично дърво с корен u, множество от листá $\{u\}$, множество от вътрешни върхове $\varnothing$ и височина 0. u няма деца.

2. $(\{u, v\}, \{(u, v)\})$ е попълнено двоично дърво с корен u, множество от листá $\{v\}$, множество от вътрешни върхове $\{u\}$ и височина 1. v е лявото дете на u, а дясното дете липсва.

Индуктивна стъпка:

3. Нека T' е съвършено кореново дърво с корен u', множество от листá W', и височина $h - 1 \geqslant 0$. Нека T'' е попълнено кореново дърво с корен u'', множество от листá W'', и височина височина $h - 1$. Нека r е връх извън T' и T''. Тогава

$$\big(V(T') \cup V(T'') \cup \{r\}, E(T') \cup E(T'') \cup \{(r, u'), (r, u'')\}\big),$$

   където u' е лявото дете на r, а u'' е дясното дете на r, е попълнено кореново дърво с корен r, множество от листá $W' \cup W''$ и височина h.

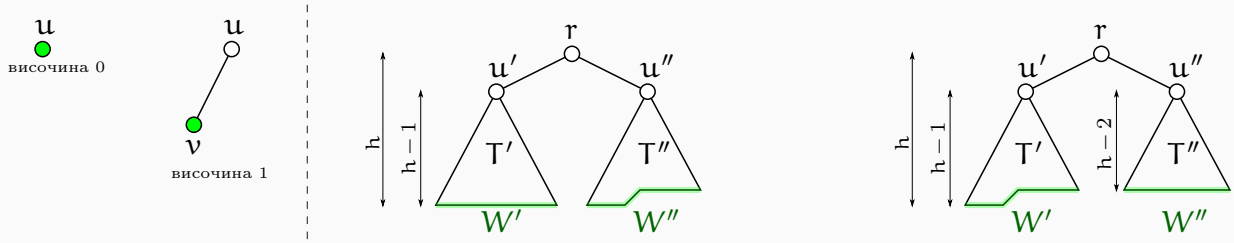
4. Нека T' е попълнено кореново дърво с корен u', множество от листá W', и височина $h - 1 \geqslant 1$. Нека T'' е съвършено кореново дърво с корен u'', множество от листá W'', и височина $h - 2$. Нека r е връх извън T' и T''. Тогава

$$\big(V(T') \cup V(T'') \cup \{r\}, E(T') \cup E(T'') \cup \{(r, u'), (r, u'')\}\big),$$

   където u' е лявото дете на r, а u'' е дясното дете на r, е попълнено кореново дърво с корен r, множество от листá $W' \cup W''$ и височина h.

Фигура 4.1 илюстрира Определение 31. Листата са в зелено.

Фигура 4.1 : Попълнено двоично дърво съгласно Определение 31.



Базата: 1 и 2. Индуктивната стъпка: 3 и 4.

Доказателството за еквивалентност на Определение 30 и Определение 31 остават за читателя.

Както обикновено, пишейки $\lg n$, имаме предвид $\log_2 n$.

Лема 23

За всяко попълнено двоично дърво с n върха, за височината h е изпълнено $h = \lfloor \lg n \rfloor$.

Доказателство: Чрез структурна индукция съгласно Определение 31. В базовите случаи 1 и 2 твърдението е вярно: имаме $0 = \lfloor \lg 1 \rfloor$ и $1 = \lfloor \lg 2 \rfloor$. ✓

В случай 3, нека T' има p върха и T'' има q върха. Съгласно индуктивното предположение, $h - 1 = \lfloor \lg p \rfloor$ и $h - 1 = \lfloor \lg q \rfloor$. Но $\lfloor x \rfloor$ е най-голямото **цяло число, ненадхвърлящо** x, за всяко x. Вярно е, че:

$$h - 1 \leqslant \lg p < h$$
$$h - 1 \leqslant \lg q < h$$

Тогава,

$$2^{h-1} \leqslant p < 2^h$$
$$2^{h-1} \leqslant q < 2^h$$

Тъй като p, q и h са цели числа, това е същото като:

$$2^{h-1} \leqslant p \leqslant 2^h - 1$$
$$2^{h-1} \leqslant q \leqslant 2^h - 1$$

Тогава

$$2 \times 2^{h-1} \leqslant p + q \leqslant 2 \times 2^h - 2 \quad \leftrightarrow$$
$$2^h \leqslant p + q \leqslant 2^{h+1} - 2 \quad \leftrightarrow$$
$$2^h < p + q + 1 < 2^{h+1}$$

Но $n = p + q + 1$. Тогава,

$$2^h < n < 2^{h+1} \quad \leftrightarrow$$
$$h < \lg n < h + 1$$

Следователно, $h = \lfloor \lg n \rfloor$. ✓

В случай 4, нека T' има $\mathtt{p}$ върха и T'' има $\mathtt{q}$ върха. Съгласно индуктивното предположение, $\mathtt{h}-1 = \lfloor \lg \mathtt{p} \rfloor$ и $\mathtt{h}-2 = \lfloor \lg \mathtt{q} \rfloor$. Но $\mathtt{q}$ е число от вида $2^{\mathtt{k}}-1$ за някое $\mathtt{k} \geqslant 0$, защото T'' е съвършено дърво. Тогава $\mathtt{q}+1 = 2^{\mathtt{k}}$. Следователно, $\lfloor \lg(\mathtt{q}+1) \rfloor = \lfloor \lg \mathtt{q} \rfloor + 1$, така че $\mathtt{h}-1 = \lfloor \lg(\mathtt{q}+1) \rfloor$. С разсъждения, аналогични на предния случай, получаваме

$$\begin{aligned} &\mathtt{h}-1 \leqslant \lg \mathtt{p} < \mathtt{h} \\ &\mathtt{h}-1 \leqslant \lg(\mathtt{q}+1) < \mathtt{h} \end{aligned}$$

Тогава,

$$\begin{aligned} &2^{\mathtt{h}-1} \leqslant \mathtt{p} < 2^{\mathtt{h}} \\ &2^{\mathtt{h}-1} \leqslant \mathtt{q}+1 < 2^{\mathtt{h}} \end{aligned}$$

Оттук имаме

$$\begin{aligned} &2^{\mathtt{h}} \leqslant \mathtt{p}+\mathtt{q}+1 < 2^{\mathtt{h}+1} \quad \leftrightarrow \\ &2^{\mathtt{h}} \leqslant \mathtt{n} < 2^{\mathtt{h}+1} \end{aligned}$$

Тогава,

$$\mathtt{h} \leqslant \lg \mathtt{n} < \mathtt{h}+1$$

Следователно, $\mathtt{h} = \lfloor \lg \mathtt{n} \rfloor$. ✓ □

Лема 24

Всяко попълнено двоично дърво T има точно $\lceil \frac{\mathtt{n}}{2} \rceil$ листá.

Доказателство: Чрез структурна индукция съгласно Определение 31. За двата базови случаи твърдението е вярно: в случай 1, $\mathtt{n}=1$ и наистина има $\lceil \frac{1}{2} \rceil = 1$ листá, а в случай 2, $\mathtt{n}=2$ и наистина има $\lceil \frac{2}{2} \rceil = 1$ листá. ✓

Нека T е конструирано чрез случай 3. Нека T' има $\mathtt{p}$ върха и T'' има $\mathtt{q}$ върха. Очевидно $\mathtt{n} = \mathtt{p}+\mathtt{q}+1$. И T', и T'' са попълнени дървета и индуктивното предположение тях казва, че те имат съответно $\lceil \frac{\mathtt{p}}{2} \rceil$ и $\lceil \frac{\mathtt{q}}{2} \rceil$ листá. Наготово използваме факта, че T' има нечетен брой върхове, понеже е съвършено двоично дърво. Нека $\mathtt{p} = 2\mathtt{k}+1$. Съгласно Определение 31, множеството от листата на T се разбива на множествата от листата на T' и T'', така че броят на листата на T е:

$$\begin{aligned} \left\lceil \frac{\mathtt{p}}{2} \right\rceil + \left\lceil \frac{\mathtt{q}}{2} \right\rceil &= \left\lceil \frac{2\mathtt{k}+1}{2} \right\rceil + \left\lceil \frac{\mathtt{q}}{2} \right\rceil = \mathtt{k}+1+\left\lceil \frac{\mathtt{q}}{2} \right\rceil = \left\lceil \mathtt{k}+1+\frac{\mathtt{q}}{2} \right\rceil = \left\lceil \frac{2\mathtt{k}+2+\mathtt{q}}{2} \right\rceil = \\ &= \left\lceil \frac{2\mathtt{k}+1+\mathtt{q}+1}{2} \right\rceil = \left\lceil \frac{\mathtt{p}+\mathtt{q}+1}{2} \right\rceil = \left\lceil \frac{\mathtt{n}}{2} \right\rceil \quad ✓ \end{aligned}$$

Нека T е конструирано чрез случай 4. Нека T' има $\mathtt{p}$ върха и T'' има $\mathtt{q}$ върха. Очевидно $\mathtt{n} = \mathtt{p}+\mathtt{q}+1$. И T', и T'' са попълнени дървета и индуктивното предположение тях казват, че те имат съответно $\lceil \frac{\mathtt{p}}{2} \rceil$ и $\lceil \frac{\mathtt{q}}{2} \rceil$ листá. Наготово използваме факта, че T'' има нечетен брой върхове, понеже е съвършено двоично дърво. Нека $\mathtt{q} = 2\mathtt{k}+1$. Съгласно Определение 31, множеството от листата на T се разбива на множествата от листата на T' и T'', така че броят

на листата на T е:

$$\left\lceil \frac{p}{2} \right\rceil + \left\lceil \frac{q}{2} \right\rceil = \left\lceil \frac{p}{2} \right\rceil + \left\lceil \frac{2k+1}{2} \right\rceil = \left\lceil \frac{p}{2} \right\rceil + k + 1 = \left\lceil \frac{p}{2} + k + 1 \right\rceil = \left\lceil \frac{p+2k+2}{2} \right\rceil =$$
$$= \left\lceil \frac{p+2k+1+1}{2} \right\rceil = \left\lceil \frac{p+q+1}{2} \right\rceil = \left\lceil \frac{n}{2} \right\rceil \quad \checkmark$$

□

Следствие 13

Вътрешните върхове на произволно попълнено двоично дърво с n върха са $\left\lfloor \frac{n}{2} \right\rfloor$.

Доказателство: Следва веднага от Лема 24 и факта, че $\left\lceil \frac{n}{2} \right\rceil + \left\lfloor \frac{n}{2} \right\rfloor = n$ за всяко цяло n. □

Лема 25: от страница 71 в [34]

Нека $f(x)$ е произволна реална непрекъсната монотонно растяща функция със свойството:

$$f(x) \text{ е цяло } \rightarrow x \text{ е цяло}$$

Тогава $\lfloor f(x) \rfloor = \lfloor f(\lfloor x \rfloor) \rfloor$ и $\lceil f(x) \rceil = \lceil f(\lceil x \rceil) \rceil$.

От Лема 25 веднага имаме тези следствия.

Следствие 14

$$\forall x \in \mathbb{R}^+ \; \forall b \in \mathbb{N}^+ : \left(\left\lfloor \frac{\lfloor x \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{b} \right\rfloor \text{ и } \left\lceil \frac{\lceil x \rceil}{b} \right\rceil = \left\lceil \frac{x}{b} \right\rceil \right)$$

Доказателство: Прилагаме Лема 25 с $f(x) = \frac{x}{b}$. □

Следствие 15

$$\forall x \in \mathbb{R}^+ \; \forall a \in \mathbb{N}^+ \; \forall b \in \mathbb{N}^+ : \left(\left\lfloor \frac{\lfloor \frac{x}{a} \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor \text{ и } \left\lceil \frac{\lceil \frac{x}{a} \rceil}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil \right)$$

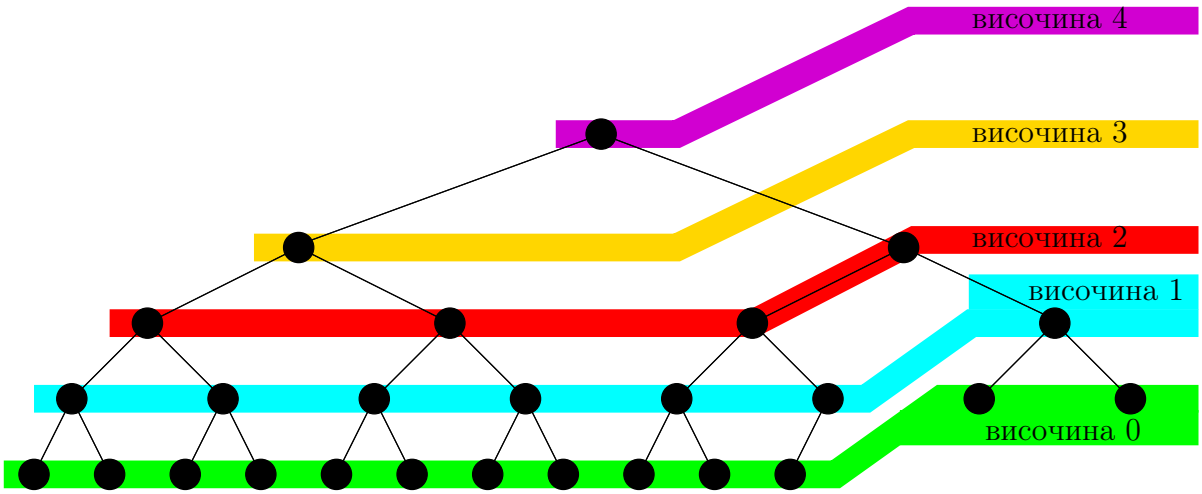
Доказателство:
Прилагаме Следствие 14 с $\frac{x}{a}$ вместо x. □

Лема 26

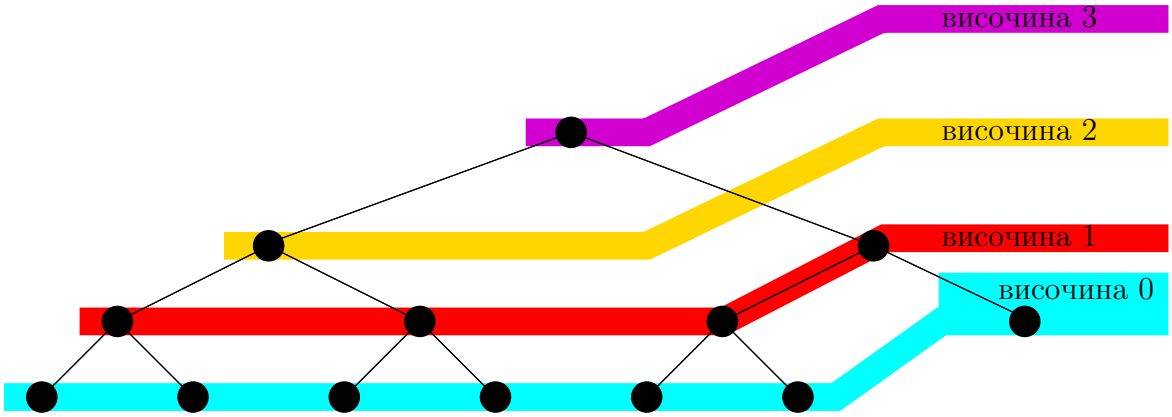
Във всяко попълнено двоично дърво T с n върха има точно $\left\lfloor \frac{n}{2^k} \right\rfloor$ върха с височина $\geqslant k$.

Доказателство: Да си припомним, че *височина на връх* u е максималното разтояние от него до кое да е листо в поддървото $T[\![u]\!]$. Пример за височините на върхове има на Фигура 4.2. Доказателството е по индукция по k.

База. $k = 0$. Върховете с височина $\geqslant 0$ са точно върховете на T, които са n на брой. Забележете, че $n = \left\lfloor \frac{n}{2^0} \right\rfloor$. ✓



Фигура 4.2: Височините на върховете в попълнено двоично дърво T.



Фигура 4.3: Попълненото дърво T', което се получава от дървото T от Фигура 4.2 след изтриване на всички върхове с височина < 1, тоест с височина 0, тоест листата. Това изтриване намалява с точно единица височините на останалите върхове.

Индуктивно предположение. Нека твърдението е в сила за някаква височина k, която не е максималната.

Индуктивна стъпка. Да изтрием всички върхове с височина $< \mathsf{k}$ от T. Нека полученото дърво е T' и нека n' е броят на неговите върхове. Листата на T', тоест върховете с височина 0 в T', очевидно са точно върховете с височина k в T. Всички върхове в T' са точно върховете с височина $\geqslant \mathsf{k}$ в T. Съгласно индуктивното предположение, $\mathsf{n}' = \left\lfloor \frac{\mathsf{n}}{2^{\mathsf{k}}} \right\rfloor$. Примерно, да разгледаме дървото T от Фигура 4.2 с $\mathsf{k} = 1$: след изтриването на всички върхове с височина < 1 получаваме дървотоT' от Фигура 4.3.

Съгласно Следствие 13, има точно $\left\lfloor \frac{\mathsf{n}'}{2} \right\rfloor$ вътрешни върхове в T'. Но вътрешните върхове на T' са точно върховете с височина $\geqslant \mathsf{k}+1$ в T. Следователно, има $\left\lfloor \frac{\left\lfloor \frac{\mathsf{n}}{2^{\mathsf{k}}} \right\rfloor}{2} \right\rfloor$ върха с височина $\geqslant \mathsf{k}+1$ в T. Съгласно Лема 25, $\left\lfloor \frac{\left\lfloor \frac{\mathsf{n}}{2^{\mathsf{k}}} \right\rfloor}{2} \right\rfloor = \left\lfloor \frac{\mathsf{n}}{2 \times 2^{\mathsf{k}}} \right\rfloor$, и очевидно $\left\lfloor \frac{\mathsf{n}}{2 \times 2^{\mathsf{k}}} \right\rfloor = \left\lfloor \frac{\mathsf{n}}{2^{\mathsf{k}+1}} \right\rfloor$. □

Теорема 30: Броят на върховете с височина k в попълнено дърво.

Във всяко попълнено двоично дърво T с n върха има точно $\left\lceil \frac{\left\lfloor \frac{n}{2^k} \right\rfloor}{2} \right\rceil$ върха с височина k.

Доказателство: Съгласно Лема 26, има точно $\left\lfloor \frac{n}{2^k} \right\rfloor$ върха с височина $\geqslant k$. Върховете с височина k са точно листата в поддървото на T, индуцирано от върховете с височина $\geqslant k$. Съгласно Лема 24, тези листа са точно $\left\lceil \frac{\left\lfloor \frac{n}{2^k} \right\rfloor}{2} \right\rceil$. □

4.1.2 Адреси в попълнено двоично дърво

Следното определение задава процедура, която дава адреси-стрингове на върховете на попълнено двоично дърво, използвайки Определение 31. С "ϵ" означаваме празния стринг.

Определение 32: Адреси на върхове в попълнено двоично дърво

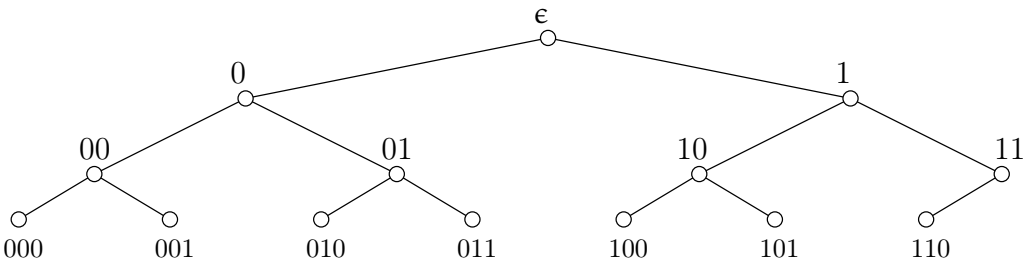
Всеки връх x в попълненото дърво има *адрес*, който бележим с $\mathrm{addr}(x)$. Адресите се конструират от следната процедура от две фази.

фаза I: В базовия случай 1, $\mathrm{addr}(u) = \epsilon$. В базовия случай 2, $\mathrm{addr}(u) = \epsilon$ и $\mathrm{addr}(v) = 0$. В случаи 3 и 4:

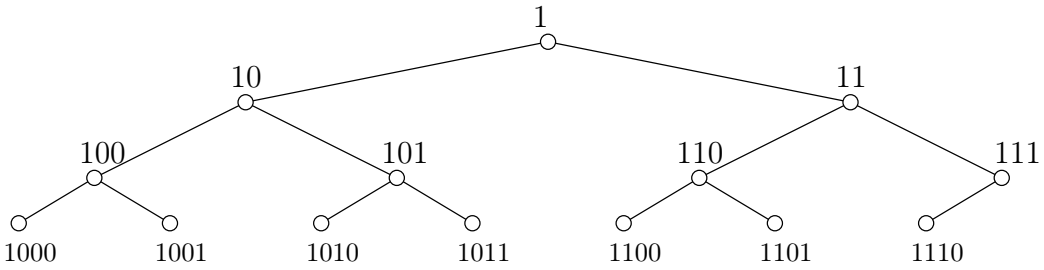
- $\mathrm{addr}(u) = \epsilon$,
- за всеки връх x от T', новият адрес на x се получава от стария с конкатенация на нула в левия край.
- за всеки връх x от T'', новият адрес на x се получава от стария с конкатенация на единица в левия край.

фаза II: За всеки връх x от T, новият адрес се получава от стария чрез конкатенация на единица в началото. □

Ето пример за конструиране на адресите от **фаза I** за попълненото дърво с 14 върха:



Продължавайки със същия пример, ето окончателните адреси след **фаза II**:



Според Определение 32 адресите на върховете в попълнените двоични дървета са стрингове. В изложението надолу понякога ще злоупотребяваме с това и ще третираме адресите като числа—а именно, числата, записани с тези стрингове.

Лема 27

Нека T е произволно попълнено двоично дърво и u е произволен връх в него. Ако u не е листо:

- ако u има две деца v и w, където v е лявото дете, в сила е $\mathrm{addr}(v) = 2 \times \mathrm{addr}(u)$ и $\mathrm{addr}(w) = 2 \times \mathrm{addr}(u) + 1$.
- ако u има точно едно дете v, в сила е $\mathrm{addr}(v) = 2 \times \mathrm{addr}(u)$.

Ако u не е корен и v е родителят на u, то $\mathrm{addr}(v) = \left\lfloor \frac{\mathrm{addr}(u)}{2} \right\rfloor$.

Доказателство, част I: Първо ще докажем твърдението за адресите на децата. Ще докажем твърдението по дълбочината на u.

Базовият случай е: u има дълбочина нула, тоест u е коренът. Ако u има две деца v и w, като v е лявото, $\mathrm{addr}(u) = 1$, $\mathrm{addr}(v) = 10$ и $\mathrm{addr}(w) = 11$, така че твърдението е вярно. Ако u има точно едно дете, очевидно става дума за дърво, построено чрез базова конструкция 2 на Определение 31, и съгласно Определение 32, $\mathrm{addr}(u) = 1$ и $\mathrm{addr}(v) = 10$, следователно твърдението пак е вярно.

Нека u има дълбочина d и u има поне едно дете. Ако u има точно две деца v и w, като v е лявото, $\mathrm{addr}(v) = \mathrm{addr}(u)0$ и $\mathrm{addr}(v) = \mathrm{addr}(u)1$, следователно твърдението е вярно, когато разглеждаме адресите като числа, записани в двоична позиционна бройна система.

Доказателство, част II: Ще докажем твърдението за адреса на родителя на всеки връх, който не е коренът. Но това твърдение е очевидно, имайки предвид **част I**. Адресът на u, изразен чрез адресите на децата си, е

$$\mathrm{addr}(u) = \frac{\mathrm{addr}(v)}{2} \quad \text{и} \quad \mathrm{addr}(u) = \frac{\mathrm{addr}(w) - 1}{2}$$

като, ако u има само едно дете, очевидно само първият израз е в сила. Тъй като $\frac{\mathrm{addr}(v)}{2}$ е задължително четно, а $\frac{\mathrm{addr}(w)}{2}$, ако съществува, е задължително нечетно, исканото твърдение следва веднага. □

За всяко $n \in \mathbb{N}^+$, $\mathsf{bin}(n)$ означава стринга-представяне на n в двоична позиционна бройна система (с водеща единица). Нека $n, k \in \mathbb{N}^+$ и $m = \lfloor \log_2 n \rfloor + 1$. Тогава $\mathsf{bin}_k(n)$ се дефинира така:

$$\mathsf{bin}_k(n) = \begin{cases} 0^{k-m}\,\mathsf{bin}(n), & \text{ако } k > m \\ \mathsf{bin}(n), & \text{ако } k = m \\ \text{суфиксът с дължина } k \text{ на } \mathsf{bin}(n), & \text{ако } k < m. \end{cases}$$

Забележете, че в случая m е дължината на представянето на n в двоична позиционна бройна система. Също така забележете, че $\mathsf{bin}(n)$ и $\mathsf{bin}_k(n)$ са стрингове над азбуката $\{0,1\}$, а не числа.

Нека x и y са произволни два върха от едно и също ниво и връх c е общият предшественик с най-малка дълбочина на x и y. Казваме, че x е вляво от y, ако x е връх от $T[\![a]\!]$ и y е връх от $T[\![b]\!]$, където a е лявото дете, а b е дясното дете на c.

Лема 28

Нека $T = (V, E)$ е произволно попълнено двоично дърво с n върха. Нека височината на T е h. Нека V_d е подмножеството от върховете на дълбочина d, за $0 \leqslant d \leqslant h$. Тогава

(а) За $0 \leqslant d \leqslant h-1$, множеството от адресите на върховете от V_d е множеството от всички булеви стрингове с дължина $d+1$ и водеща единица.

(б) Множеството от адресите на върховете от V_h е множеството от булевите стрингове с дължина $h+1$ и водеща единица от 10^h до $\mathsf{bin}(n)$ в лексикографската наредба.

(в) Нещо повече, във всяко ниво върховете са лексикографски сортирани по адреси отляво надясно в T.

Доказателство: Нека $\mathrm{addr}'(x)$ е адресът на x след **фаза I**, за всеки връх $x \in V$. В термините на $\mathrm{addr}'(x)$, твърдения (а) и (б) са еквивалентни съответно на:

(а') За $0 \leqslant d \leqslant h-1$, множеството от адресите на върховете от V_d е множеството от всички булеви стрингове с дължина d.

(б') Множеството от адресите на върховете от V_h е множеството от булевите стрингове с дължина h от 0^h до $\mathsf{bin}_h(n)$ в лексикографската наредба.

Това ще докажем чрез структурна индукция съгласно Определение 31 на попълнено дърво, използвайки Определение 32 на адреси на върховете на попълнено дърво.

База. В случай 1 на Определение 31, единственият връх има адрес ϵ. В случай 2,

- в ниво 0 единственият връх наистина има адрес ϵ, който има дължина 0,
- в последното ниво 1 наистина адресите са от 0 до $\mathsf{bin}_1(2) = 0$. ✓

Индуктивна стъпка Да разгледаме случай 3.

- На ниво 0, коренът r наистина получава адрес ϵ.
- Да разгледаме произволно ниво d, където $1 \leqslant d \leqslant h-1$. Тривиално е да се докаже, че върховете от ниво d в T са 2^d на брой и че тяхната наредба отляво надясно се състои от върховете на ниво $d-1$ в T', последвани от върховете от ниво $d-1$ на T''. Тъй като $0 \leqslant d-1 \leqslant h-2$, ниво $d-1$ не е последно ниво нито в T', нито в T''. Тогава, съгласно част (а') на индуктивното допускане, адресите на върховете от ниво $d-1$ и в T', и в T'' са булевите стрингове с дължина $d-1$, които и в T', и в T'' са сортирани отляво надясно. Очевидно след слагане на 0 вляво на всеки адрес от ниво $d-1$ в T' и на 1 вляво на всеки адрес от ниво $d-1$ в T'' получаваме—по отношение на T—всички булеви стрингове с дължина d, сортирани лексикографски.

- Да разгледаме последното ниво $\mathtt{h}$ в $\mathtt{T}$. В случай 3 то се състои от всички $2^{\mathtt{h}-1}$ върхове от ниво $\mathtt{h}-1$ на $\mathtt{T}'$, последвани вдясно от върховете от ниво $\mathtt{h}-1$ на $\mathtt{T}''$.

  Да разгледаме последното ниво $\mathtt{h}-1$ в $\mathtt{T}'$. Нека $\mathtt{T}'$ има $\mathtt{p}$ върха и $\mathtt{T}''$ има $\mathtt{q}$ върха. Съгласно част (б') на индуктивното предположение, отляво надясно адресите в ниво $\mathtt{h}-1$ на $\mathtt{T}'$ са

  $$\underbrace{00\ldots0}_{\text{дължина } \mathtt{h}-1}\ ,\quad \underbrace{00\ldots01}_{\text{дължина } \mathtt{h}-1}\ ,\ldots,\ \underbrace{\mathsf{bin}_{\mathtt{h}-1}(\mathtt{p})}_{\text{дължина } \mathtt{h}-1}$$

  Но $\mathtt{p} = 2^{\mathtt{h}-1+1}-1 = 2^{\mathtt{h}}-1$, защото $\mathtt{T}'$ е съвършено дърво, така че $\mathsf{bin}_{\mathtt{h}-1}(\mathtt{p}) = 1^{\mathtt{h}-1}$ единици. Тогава адресите в ниво $\mathtt{h}-1$ на $\mathtt{T}'$ са, отляво надясно,

  $$\underbrace{00\ldots0}_{\text{дължина } \mathtt{h}-1}\ ,\quad \underbrace{00\ldots01}_{\text{дължина } \mathtt{h}-1}\ ,\ldots,\ \underbrace{11\ldots1}_{\text{дължина } \mathtt{h}-1}$$

  тоест всички булеви стрингове с дължина $\mathtt{h}-1$.

  Да разгледаме последното ниво $\mathtt{h}-1$ в $\mathtt{T}''$. Съгласно част (а') на индуктивното предположение, отляво надясно адресите в ниво $\mathtt{h}-1$ на $\mathtt{T}''$ са

  $$\underbrace{00\ldots0}_{\text{дължина } \mathtt{h}-1}\ ,\quad \underbrace{00\ldots01}_{\text{дължина } \mathtt{h}-1}\ ,\ldots,\ \underbrace{\mathsf{bin}_{\mathtt{h}-1}(\mathtt{q})}_{\text{дължина } \mathtt{h}-1}$$

  Следователно, в цялото дърво $\mathtt{T}$, адресите на върховете от ниво $\mathtt{h}$ са, отляво надясно

  $$\underbrace{00\ldots0}_{\text{дължина } \mathtt{h}}\ ,\quad \underbrace{00\ldots01}_{\text{дължина } \mathtt{h}},\ldots,\underbrace{011\ldots1}_{\text{дължина } \mathtt{h}},\ \underbrace{10\ldots0}_{\text{дължина } \mathtt{h}}\ ,\quad \underbrace{10\ldots01}_{\text{дължина } \mathtt{h}},\ldots,\underbrace{1\,\mathsf{bin}_{\mathtt{h}-1}(\mathtt{q})}_{\text{дължина } \mathtt{h}}$$

  Тъй като $\mathtt{n} = \mathtt{p}+\mathtt{q}+1$ и $\mathtt{p} = 2^{\mathtt{h}}-1$, то $\mathtt{n} = 2^{\mathtt{h}}+\mathtt{q}$. Но тъй като $\mathtt{q} \leqslant \mathtt{p}$, дължината на $\mathsf{bin}(\mathtt{q})$ също е по-малка от $\mathtt{h}$ и $\mathsf{bin}_{\mathtt{h}-1}(\mathtt{q})$ се получава от $\mathsf{bin}(\mathtt{q})$ с добавяне на $\geqslant 0$ на брой нули вляво. Имайки предвид, че $\mathsf{bin}(2^{\mathtt{h}}) = 10^{\mathtt{h}}$, заключаваме, че стрингът $1\,\mathsf{bin}_{1-\mathtt{h}}(\mathtt{q})$ е равен на $\mathsf{bin}(\mathtt{n})$. Заключаваме, че адресите от последното ниво на $\mathtt{T}'$ отляво надясно, следвани от адресите от последното ниво на $\mathtt{T}''$ отляво надясно, представляват последователността

  $$\underbrace{00\ldots0}_{\text{дължина } \mathtt{h}}\ ,\quad \underbrace{00\ldots01}_{\text{дължина } \mathtt{h}},\ldots,\underbrace{011\ldots1}_{\text{дължина } \mathtt{h}},\ \underbrace{10\ldots0}_{\text{дължина } \mathtt{h}}\ ,\quad \underbrace{10\ldots01}_{\text{дължина } \mathtt{h}},\ldots,\ \underbrace{\mathsf{bin}(\mathtt{n})}_{\text{дължина } \mathtt{h}}$$

Да разгледаме случай 4.

- На ниво 0, коренът $\mathtt{r}$ наистина получава адрес ϵ.
- Да разгледаме произволно ниво $\mathtt{d}$ в $\mathtt{T}$, където $1 \leqslant \mathtt{d} \leqslant \mathtt{h}-1$. Аналогично на предния случай, върховете от ниво $\mathtt{d}$ в $\mathtt{T}$ са $2^{\mathtt{d}}$ на брой и тяхната наредба отляво надясно се състои от върховете на ниво $\mathtt{d}-1$ в $\mathtt{T}'$, последвани от върховете от ниво $\mathtt{d}-1$ на $\mathtt{T}''$. Тъй като $0 \leqslant \mathtt{d}-1 \leqslant \mathtt{h}-2$, ниво $\mathtt{d}-1$ не е последното ниво в $\mathtt{T}'$, но е последното ниво в съвършеното дърво $\mathtt{T}''$ при $\mathtt{d}-1 = \mathtt{h}-2$.

  Адресите на върховете от ниво $\mathtt{d}-1$ в $\mathtt{T}'$ са булевите стрингове с дължина $\mathtt{d}-1$ и са сортирани отляво надясно съгласно част (а') на индуктивното предложение. Ако $\mathtt{d}-1 < \mathtt{h}-2$, адресите на на върховете от ниво $\mathtt{d}-1$ в $\mathtt{T}''$ са булевите стрингове

с дължина $d-1$ и са сортирани отляво надясно съгласно част (а') на индуктивното предложение.

Да разгледаме случая $d-1=h-2$ по отношение на T''. Сега част (б') на индуктивното предположение е приложима. Съгласно нея, адресите на на върховете от ниво $d-1$ в T'' са булевите стрингове от 0^{h-2} до $\mathsf{bin}_{h-2}(q)$ в лексикографската наредба, където q е броят на върховете в T'', и те са сортирани отляво надясно лексикографски. Но тъй като T'' е съвършено дърво с височина $h-2$, вярно е, че $q=2^{h-2+1}-1=2^{h-1}-1$, следователно $\mathsf{bin}_{h-2}(q)=1^{d-1}$.

Заключаваме, че за $1\leqslant d\leqslant h-1$, адресите на върховете от ниво $d-1$ в T' са всички булеви стрингове с дължина $d-1$, сортирани отляво надясно, а също така адресите на върховете от ниво $d-1$ в T'' са всички булеви стрингове с дължина $d-1$ сортирани отляво надясно. Очевидно след слагане на 0 вляво на всеки адрес от ниво $d-1$ в T' и на 1 вляво на всеки адрес от ниво $d-1$ в T'' получаваме—по отношение на дървото T—всички булеви стрингове с дължина d, сортирани лексикографски отляво надясно.

- Да разгледаме последното ниво h в T. То е последното ниво $h-1$ на T', така че част (б') на индуктивното предположение е приложима. Съгласно нея, адресите на на върховете от ниво $h-1$ в T' са булевите стрингове от 0^{h-1} до $\mathsf{bin}_{h-1}(p)$, където p е броят на върховете в T', и те са сортирани отляво надясно лексикографски. Тогава във **фаза I** тези върхове получават адреси от 0^h до $0\mathsf{bin}_{h-1}(p)$, сортирани отляво надясно лексикографски.

  Да си припомним, че $n=p+q+1$, където q е броят на върховете в съвършеното дърво T''. Тъй като T'' е съвършено и с височина $h-2$, вярно е, че $q=2^{h-1}-1$, следователно $q+1=2^{h-1}$, следователно $\mathsf{bin}(q+1)=10^{h-1}$. От друга страна, $p\geqslant q+1$, така че $\mathsf{bin}(p)=1\sigma$ за някой булев стринг σ с дължина $h-1$. Тогава $\mathsf{bin}(p+q+1)=10\sigma$. С други думи, $\mathsf{bin}(n)=10\sigma$. Но тогава $\mathsf{bin}_h(n)=0\sigma=0\mathsf{bin}_{h-1}(p)$.

  Доказахме, че наистина адресите на върховете от последното ниво на T са лексикографски сортираните стрингове от 0^h до $\mathsf{bin}_h(n)$. □

4.1.3 Двоична пирамида

Сега допускаме, че всеки връх има асоциирана стойност, наречена *ключ* (key). В общия случай тази стойност е реално число, но за простота в примерите, които ще разгледаме, ключовете са цели положителни числа.

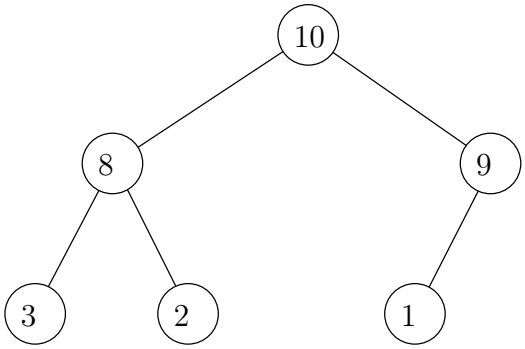
Определение 33: Двоична пирамида (binary heap, вж. [69])

Двоична пирамида е попълнено двоично дърво, в което:

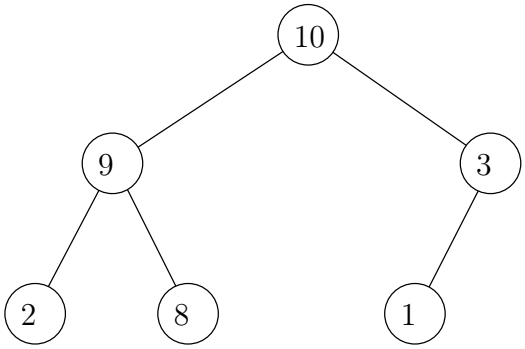
- или ключът на всеки връх по-голям или равен на ключовете на неговите деца и тогава пирамидата е *максимална пирамида* (max heap),
- или ключът на всеки връх е по-малък или равен на ключовете на неговите деца и тогава пирамидата е *минимална пирамида* (min heap).

Тук ще разглеждаме максимални пирамиди и казвайки "пирамида" ще имаме предвид "максимална пирамида". Но изложението може да бъде променено лесно, така че да се отнася за минимални пирамиди. Също така, ще изпускаме "двоични", понеже не разглеждаме пирамиди, които не са двоични.

Ето пример за пирамида:



Очевидно фòрмата на дървото е фиксирана от броя на върховете, но подредбата на ключовете не е фиксирана[†]. Най-големият ключ 10 непременно е в корена и 9 непременно е в дете на корена, но известно вариране е възможно. Примерно, това също е пирамида със тези ключове:



Наблюдение 10

Попълнено двоично дърво с ключове е пирамида тогава и само тогава, когато за всяко листо u, по уникалния път, свързващ корена с u, стойностите на ключовете са ненарастващи в посока от корена към u.

4.1.4 Реализиране на пирамида чрез масив

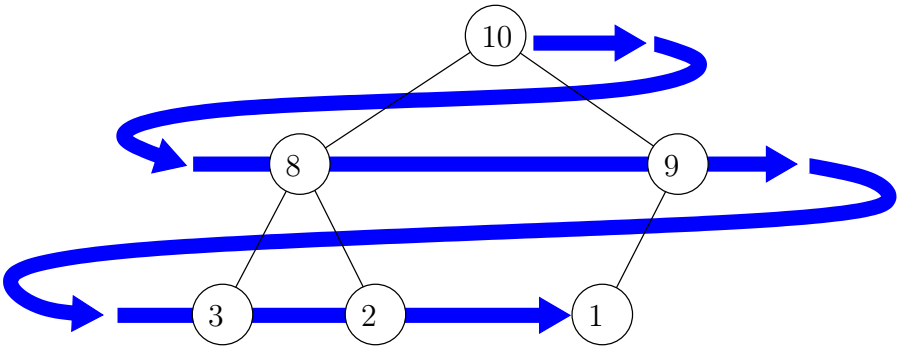
Определение 34

Нека $T = (V, E)$ е попълнено двоично дърво с n върха. *Линеаризация* на T се нарича масивът $A[1, 2, \ldots, n]$ такъв че за всяко i, $1 \leqslant i \leqslant n$, $A[i]$ е ключът на върха, чийто адрес е i.

За да съкратим изложението, често игнорираме разликата между "пирамида" и "линеаризация на пирамида" и, казвайки просто "пирамида", имаме предвид "линеаризация на пирамида". С други думи, самият масив-линеаризация бива наричан "пирамида".

Съгласно Лема 28, съществува биекция между множеството от адресите и множеството $\{1, 2, \ldots, n\}$, така че определението е смислено. И така, пирамидите може да се реализират много ефикасно чрез масиви. Линеаризацията на последния пример е $[10, 8, 9, 3, 2, 1]$. Линеаризирането можем да си представяме като обхождане на дървото по нива, във всяко ниво, отляво надясно, и последователно записване на ключовете в масива:

[†]В Допълнение 21 дискутираме броя на различните пирамиди с n върха.



Лема 28 гарантира, че в нивата няма "празнини", така че в масива няма "дупки".

При тази реализация се избягва разходът на памет, който е неизбежен при дърветата по принцип. Ако искаме да представим двоично дърво в общия случай—не непременно попълнено—трябва по някакъв начин да укажем за всеки връх кой е родителят, или кои са децата. Това ни струва допълнителна памет. Докато пирамида може да се реализира без никаква допълнителна памет, като при това обхождането е много ефикасно като сложност по време. Последното твърдение се основава на факта (виж Наблюдение 11), че ако масивът $A[1,\ldots,n]$ е линеаризация на пирамида, то за всеки елемент $A[i]$ можем в $\Theta(1)$ време да определим дали $A[i]$ е корен или не, ако не е, то кой е родителят му, дали е вътрешен връх или листо, и ако е вътрешен връх, кое е лявото му дете $A[j]$, ако такова съществува, и дясното му дете $A[k]$, ако такова съществува.

Следното наблюдение е просто приложение на Лема 27.

Наблюдение 11

Нека T е пирамида с n елемента и масивът $A[1,\ldots,n]$ е нейната линеаризация. Тогава за всеки елемент $A[i]$:

1. елементът, който отговаря на родителя на $A[i]$, е $A\left[\left\lfloor\frac{i}{2}\right\rfloor\right]$, ако върхът на T, който отговаря на $A[i]$, не е коренът.
2. елементът, който отговаря на лявото дете на $A[i]$, е $A[2\times i]$, ако върхът на T, отговарящ на $A[i]$, има ляво дете,
3. елементът, който отговаря на дясното дете на $A[i]$, е $A[2\times i+1]$, ако върхът на T, отговарящ на $A[i]$, има дясно дете. □

Наблюдение 11 ни дава основание да дефинираме следните изчислителни примитиви[†], чиито

[†]"Примитив" е, неформално казано, лесна за възприемане функция с малка сложност, която се използва често от алгоритмите, които ни интересуват.

имена са достатъчно информативни:

```
PARENT(i)
    return (if i ⩾ 2 then ⌊i/2⌋ else i)

LEFT(i)
    return (if 2i ⩽ n then 2i else UNDEFINED)

RIGHT(i)
    return (if 2i + 1 ⩽ n then 2i + 1 else UNDEFINED)

LEVEL(i)
    return ⌊log₂ i⌋

ISLEAF(i)
    return (if i > ⌊n/2⌋ then YES else NO)

ISINTERNALVERTEX(i)
    return not ISLEAF(i)

ISROOT(i)
    return (if i = 1 then YES else NO)
```

Наблюдение 12

Забележете, че PARENT(1) = 1. С други думи, коренът е родител на себе си съгласно примитива PARENT. Това не следва по никакъв начин от Наблюдение 11! Можеше да възприемем алтернативния подход, а именно, че коренът няма родител, но изложението е по-кратко и елегантно, ако всеки елемент да има родител, включително и коренът.

Освен това, за $t \geqslant 1$, дефинираме, че

$$\textsc{Parent}^t(i) = \begin{cases} i, & \text{ако } t = 0 \\ \textsc{Parent}\big(\textsc{Parent}^{t-1}(i)\big), & \text{ако } t > 0 \end{cases}$$

и

$$\textsc{Ancestors}(i) = \{\textsc{Parent}^t(i) \mid t \geqslant 1\}$$

Казваме, че $A[j]$ е *предшественик* на $A[i]$, ако $j \in \textsc{Ancestors}(i)$. По това определение никой елемент на линеаризацията не е предшественик на себе си, освен $A[1]$. Предвид последното, ясно е, че ANCESTORS(i) е крайно множество, въпреки че "$t \geqslant 1$" означава "$t \in \mathbb{N}^+$".

Определение 35: пирамидална инверсия в масив

Нека $A[1,\ldots,n]$ е масив от ключове. *Пирамидална инверсия* в A наричаме всяка наредена двойка индекси $\langle i,j\rangle$, такива че $i<j$, $A[i]$ е предшественик на $A[j]$ и $A[i]<A[j]$. Наредената двойка $\langle A[i],A[j]\rangle$ също наричаме инверсия. *Директна пирамидална инверсия*, или накратко *директна инверсия*, ако от контекста е ясно, че става дума за пирамидални инверсии, е всяка пирамидална инверсия $\langle i,j\rangle$, такава че $i=\textsc{Parent}(j)$. Пирамидални инверсии, които не са директни, се казват *индиректни*.

Например, на Фигура 4.9 е показано попълнено дърво с ключове (неговата линеаризация е на Фигура 4.10 на най-горния ред), което би било пирамида, ако не беше най-дясното листо на последното ниво, нарисувано в червено и с ключ 35. В дървото има точно три пирамидални инверсии, които са между това листо и негови предшественици; а именно, предшествениците с ключове 11, 9 и 6. Директната инверсия е с предшественика с ключ 6.

Наблюдение 13

Масивът от ключове A е пирамида тогава и само тогава, когато няма нито една пирамидална инверсия. □

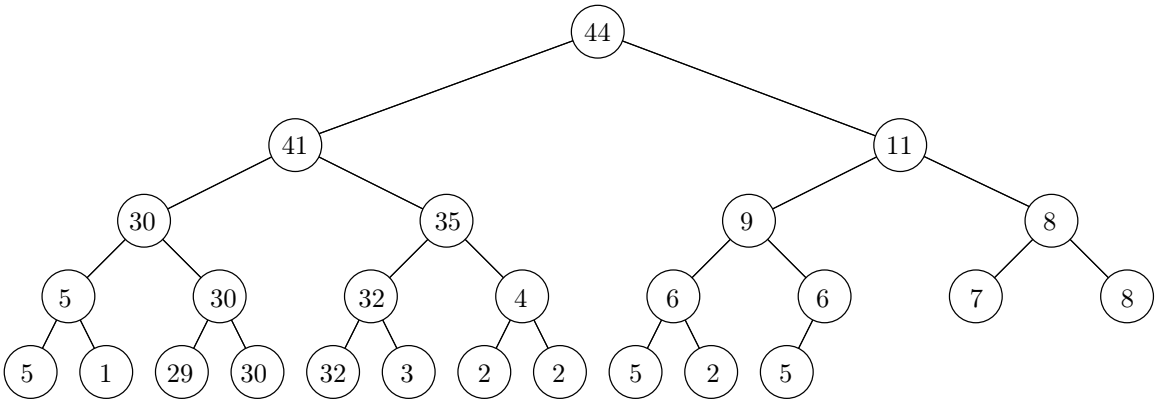
4.1.5 Подпирамида

Нека T е пирамида с n върха, реализирана с масив $A[1,2,\ldots,n]$. За всеки връх $u\in V(T)$, *подпирамидата с корен* u е $T[\![u]\!]$. По отношение на масива A, който преставя пирамидата, u е индекс на елемент. Нотацията $A[\![u]\!]$ означава реализацията на $T[\![u]\!]$ **в масива** A. Очевидно, в общия случай $A[\![u]\!]$ не е непрекъснат подмасив, а се състои от няколко подмасива—на брой колкото е височината на съответното поддърво—всеки от които е непрекъсната подпоследователност на A (вижте Фигура 4.7).

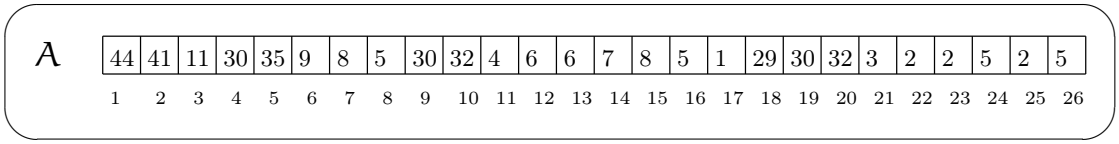
Наблюдение 14

Ако $A[1,\ldots,n]$ е пирамида, то за всяко $k\in\{1,\ldots,n\}$, $A[\![k]\!]$ е пирамида. □

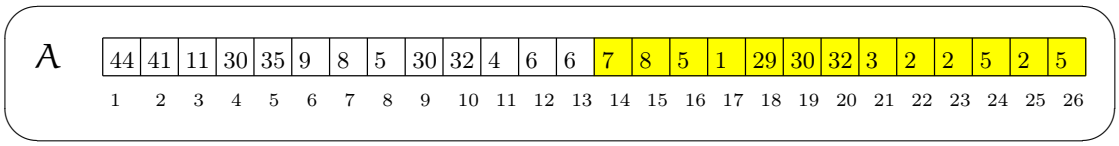
Като пример за пирамида и нейната линеаризация, да разгледаме пирамидата, показана на Фигура 4.4. Фигура 4.5 показва нейната линеаризация. Фигура 4.6 показва листата на пирамидата. На Фигура 4.7 е показана подпирамидата $A[\![3]\!]$. Връх 3 е третият връх в масива (дясното дете на корена, ако разсъждаваме в термините на дървото). Фигура 4.8 показва листата на $A[\![3]\!]$ в червено.



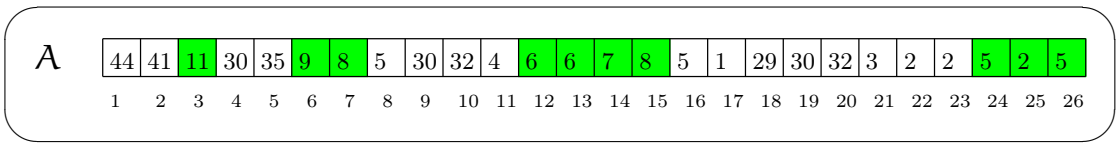
Фигура 4.4: Пирамида с 26 върха.



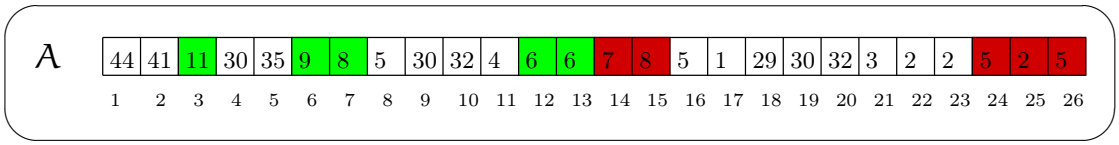
Фигура 4.5: Линеаризацията на пирамида от Фигура 4.4.



Фигура 4.6: Листата на пирамидата от Фигура 4.4 са отбелязани с жълто.



Фигура 4.7: Подпирамидата A⟦3⟧.



Фигура 4.8: Листата на A⟦3⟧ в червено.

Използвайки дефинираните примитиви, следният алгоритъм обхожда $\mathsf{T}[\![\mathtt{u}]\!]$. "Обхожда" означава "извежда ключовете на $\mathsf{T}[\![\mathtt{u}]\!]$ в *preorder*[†]" (а не по нива).

```
TRAVERSE BINARY SUBHEAP(A[1,...,n]: пирамида, i: число от {1,...n} )
1  print i
2  left ← LEFT(i)
3  right ← RIGHT(i)
4  if left ⩽ n
5      TRAVERSE BINARY SUBHEAP(A, left )
6  if right ⩽ n
7      TRAVERSE BINARY SUBHEAP(A, right)
```

4.2 Построяване на пирамида

Даден е масив $\mathsf{A}[1,\ldots,\mathtt{n}]$ от ключове. За простота допускаме, че ключовете се естествени числа. Числата са в произволен порядък. Искаме да разместим числата така, че масивът да стане пирамида. Дори числата да са две по две различни, има много начини да направим пирамида от дадени числа. За подробна дискусия на броя на пирамидите вижте Допълнение 21.

4.2.1 Наивно построяване на пирамида

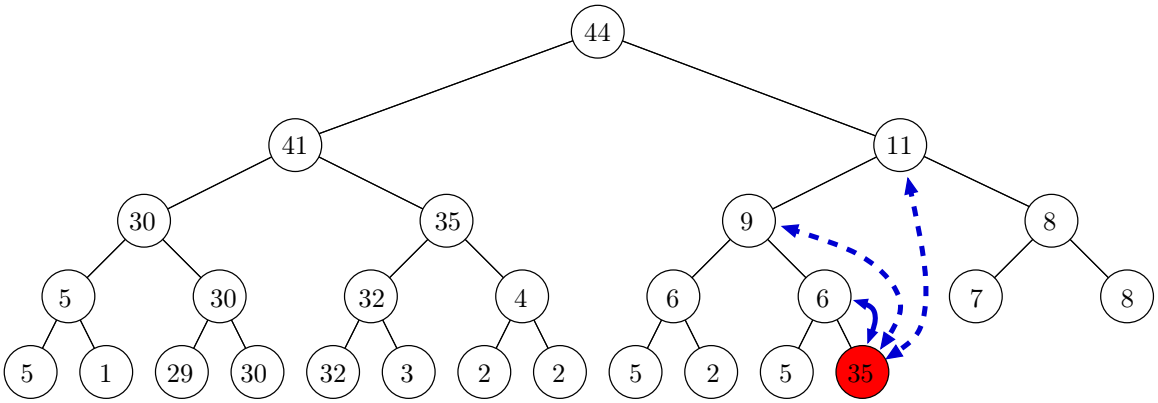
Най-естественият начин да бъде превърнат произволен масив от числа в пирамида чрез разместване е чрез последователно добавяне на елементи в частично построена пирамида.

```
NAIVE BUILD HEAP, OUTLINE(A[1,...,n]: масив от цели числа)
1  for i ← 2 to n
2      добави A[i] към досега построената пирамида A[1,...,i−1],
       така че A[1,...,i] да стане пирамида
```

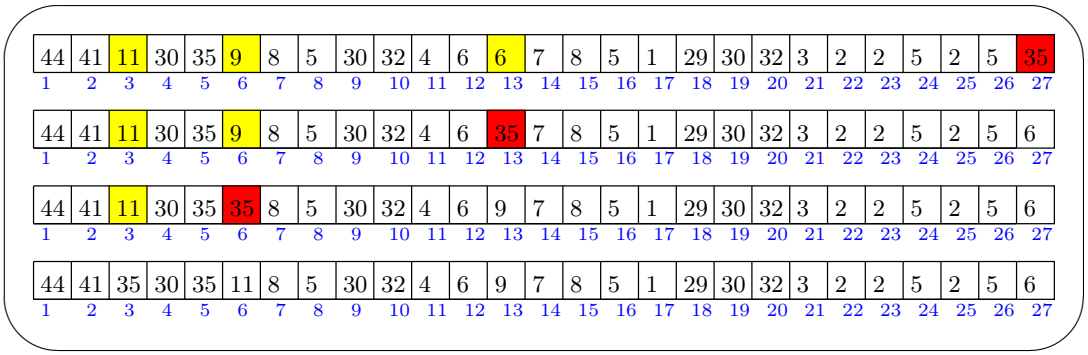
Това описание е прекалено общо, но идеята е ясна: "добави към $\mathsf{A}[1,\ldots,\mathtt{i}-1]$" не е просто увеличаване на индекса $\mathtt{i}$. Дори $\mathsf{A}[1,\ldots,\mathtt{i}-1]$ да е пирамида, $\mathsf{A}[1,\ldots,\mathtt{i}-1,\mathtt{i}]$ не е непременно пирамида, защото може да има пирамидални инверсии $\langle \textsc{Parent}^{\mathtt{t}}(\mathtt{i}), \mathtt{i}\rangle$. Това е илюстрирано на Фигура 4.9, на която структурата е показана като дърво, а не като масив. Дървото съдържа пирамидата от Фигура 4.4 плюс още един връх, оцветен в червено. Ключът на този нов връх е 35 и той участва в пирамидални инверсии с три от своите предшественици. Съществуват много начини да бъдат разместени елементите на $\mathsf{A}[1,\ldots,\mathtt{i}]$, така че да няма пирамидални инверсии. Най-естественият и прост начин е, да ликвидираме пирамидалните инверсии, в които участва новодобавеният $\mathsf{A}[\mathtt{i}]$, без да разместваме никакви елементи, които не участват в тези инверсии. Да мислим за обекта като за дърво. Тогава въпросните инверсии са между новодобавения $\mathsf{A}[\mathtt{i}]$ и върхове по пътя от него към корена, които върхове индуцират подпът (тоест, по пътя от $\mathsf{A}[\mathtt{i}]$ към корена, върховете, участващи в инверсии, са непрекъсната последователност). Очевидното решение тогава е да разменяме $\mathsf{A}[\mathtt{i}]$ с $\mathsf{A}[\textsc{Parent}(\mathtt{i})]$, докато има инверсии.

Всяка такава размяна ликвидира една инверсия и не въвежда нови инверсии, защото ключът, който застава "отгоре", е по-голям от този, който преди е бил там преди размяната,

[†]За обхождания на дървета, включително в preorder, вижте примерно [84].



Фигура 4.9: Почти-пирамидата, получена от пирамидата от Фигура 4.4 с един добавен връх (в червено). Има точно три пирамидални инверсии между новодобавения елемент и негови предшественици. Пирамидалните инверсии са показани със сини стрелки, като с непрекъсната линия е показана единствената директна пирамидална инверсия, а с прекъснати линии – двете индиректни пирамидални инверсии.



Фигура 4.10: Линеаризацията на почти-пирамидата от Фигура 4.4 е на най-горния ред. Елементът, който "не си е на мястото", е накрая в червено. Елементите, които образуват пирамидални инверсии с него, са в жълто. Следващите три реда отгоре надолу съответстват на трите изпълнения на **while**-цикъла, всяко от които "избутва" 35 наляво в масива.

следователно този елемент, който застава отгоре, остава не по-малък от другото си дете, ако има такова.

NAIVE BUILD HEAP, DETAILED($A[1,\ldots,n]$: масив от ест. числа)

```
1  for i ← 2 to n
2      k ← i
3      while k > 1 and A[PARENT(k)] < A[k] do
4          swap(A[k], A[PARENT(k)])
5          k ← PARENT(k)
```

Фигура 4.10 илюстрира едно изпълнение на **for**-цикъла на NAIVE BUILD HEAP, в **while**-цикълът разменя елемента 35 с елементи вляво, докато има нужда.

Сега ще докажем подробно коректността на наивното построяване на пирамида. Тъй като алгоритъмът е итеративен, ще направим доказателството с инвариант на цикъла. Преди това обаче ще дадем една дефиниция, която значително опростява изложението.

Определение 36: Почти-пирамида

Почти-пирамида е попълнено двоично дърво, чиито ключове удовлетворяват изискването за пирамида може би с изключение на най-дясното листо на последния ред.

Иначе казано, линеаризацията на почти-пирамидата е такава, че всички пирамидални инверсии са между последния елемент и негови предшественици. Фигура 4.9 и най-горния ред на Фигура 4.10 показват почти-пирамида и нейната линеаризация, като добре илюстрират факта, че всички инверсии са между последния елемент и негови предшественици, които са "плътно подредени" над него (в дървото). Това си заслужава да бъде формулирано в самостоятелно наблюдение.

Наблюдение 15: Инверсиите в почти-пирамида

Нека $A[1,\ldots,n]$ е линеаризация на почти-пирамида. Тогава всички пирамидални инверсии са от вида $\langle i, n\rangle$, където $i \in \text{ANCESTORS}(n)$. Нещо повече, ако $\langle i, n\rangle$ е инверсия, то за всяко k, такова че $i < k < n$ и $k \in \text{ANCESTORS}(n)$, е вярно, че $\langle k, n\rangle$ е инверсия.

Лема 29: Вътрешният цикъл на NAIVE BUILD HEAP

Спрямо едно единствено изпълнение на **for**-цикъла на NAIVE BUILD HEAP (редове 1–5), нека t е моментът непосредствено преди изпълнението на **while** цикъла (редове 3–5). Нека наричаме A' масива A в момента t. Да допуснем, че $A'[1,\ldots,i]$ реализира почти-пирамида. Тогава в момента на приключване на **while** цикъла е вярно, че текущият $A[1,\ldots,i]$ реализира пирамида.

Доказателство: В момента t е вярно, че $k = i$ заради присвояването на ред 2. Разглеждаме поотделно два случая за момента t.

Първо допускаме, че в момента t в $A'[1,\ldots,i]$ няма инверсии. Тогава $A'[1,\ldots,i]$ е пирамида. От друга страна, **while** цикълът не се изпълнява изобщо, понеже $A[\text{PARENT}(k)] \geqslant A[k]$, така че изпълнението отива на ред 1. Спрямо новото i е вярно, че текущият $A[1,\ldots,i-1]$ е същият масив като $A'[1,\ldots,i]$ спрямо предишното i. Ерго, текущият $A[1,\ldots,i-1]$ е пирамида, което трябваше да докажем.

Сега допускаме, че в момента t има инверсии. Съгласно Наблюдение 15, всички инверсии са от вида $\langle x, i\rangle$, където $x \in \text{ANCESTORS}(i)$. Дефинираме a като минималното x, такова че $\langle x, i\rangle$ е инверсия. Следното твърдение е инвариант за **while** цикъла.

Инвариант 10: Вътрешният цикъл на NAIVE BUILD HEAP

При всяко достигане на ред 3, множеството от пирамидалните инверсии в текущия $A[1,\ldots,i]$ е $\{\langle b, k\rangle \mid b \in \text{ANCESTORS}(k) \text{ и } b \geqslant a\}$.

База. Когато изпълнението достигне до ред 3 за първи път, $A'[1,\ldots,i]$ реализира почти-пирамида по допускане от условието на Лема 29. Тъй като $k = i$ заради присвояването на ред 2, твърдението на инварианта става "множеството от пирамидалните инверсии в текущия $A[1,\ldots,i]$ е $\{\langle b, i\rangle \mid b \in \text{ANCESTORS}(i) \text{ и } b \geqslant a\}$", което следва веднага от Наблюдение 15 и дефиницията на a. ✓

Поддръжка. Допускаме, че твърдението е вярно за някое достигане на ред 3, което не е последното. На ред 4 става размяна на $A[k]$ с $A[\text{PARENT}(k)]$, с което множеството от пирамидалните инверсии в текущия $A[1,\ldots,i]$ става $\{\langle b, k\rangle \mid b \in \text{ANCESTORS}(\text{PARENT}(k)) \text{ и } b \geqslant a\}$.

Но след изпълнението на ред 5, спрямо новото k, отново е вярно, че множеството от пирамидалните инверсии в текущия $A[1,\ldots,i]$ е $\{\langle b,k\rangle \mid b \in \textsc{Ancestors}(k) \text{ и } b \geqslant a\}$.

Терминация. При последното достигане на ред 3 е вярно поне едно от двете: $k \leqslant 1$ или $A[\textsc{Parent}(k)] \geqslant A[k]$.

- Ако $k \leqslant 1$, то $\textsc{Ancestors}(k) = \varnothing$, така че множеството от пирамидалните инверсии в текущия $A[1,\ldots,i]$ е празното множество. С други думи, инверсии няма.
- Нека $A[\textsc{Parent}(k)] \geqslant A[k]$. От това и транзитивността на релацията на наредба на ключовете следва, че за всяко $b \in \textsc{Ancestors}(k)$, $\langle b,k\rangle$ не е инверсия. Но съгласно инварианта, множеството от пирамидалните инверсии в текущия $A[1,\ldots,i]$ е

  $$\{\langle b,k\rangle \mid b \in \textsc{Ancestors}(k) \text{ и } b \geqslant a\}$$

  Заключаваме, че това множество е празното множество[†]. С други думи, пирамидални инверсии няма. □

Теорема 31: Коректност на наивното построяване на пирамида

За всеки вход A, Naive Build Heap построява пирамида от него.

Доказателство: Следното твърдение е инвариант на **for**-цикъла (редове 1–5).

Инвариант 11: Цикълът на Naive Build Heap

Всеки път, когато изпълнението е на ред 1, $A[1,\ldots,i]$ реализира почти-пирамида.

База. При първото достигане на ред 1 е вярно, че $i = 2$. Съгласно Определение 36, всеки двуелементен масив реализира почти-пирамида, така че инвариантът е верен. ✓

Поддръжка. Да допуснем, че твърдението е вярно за някое достигане на ред 1, което не е последното. На ред 2, k приема стойност i. Прилагаме Лема 29. Съгласно лемата, при допускането, че при влизането в **while**-а е изпълнено текущият $A[1,\ldots,i]$ да реализира почти-пирамида и k да е равно на i, вярно е, че при излизането от **while**-а, $A[1,\ldots,i]$ реализира пирамида. Тогава при следващото достигане на ред 1, променливата i се инкрементира и спрямо новото i е вярно, че $A[1,\ldots,i]$ реализира почти-пирамида.

Терминация. При последното достигане на ред 1 е вярно, че $i = n + 1$. Твърдението "$A[1,\ldots,n+1]$ реализира почти-пирамида" е формално некоректно, защото говори за несъществуващия елемент $A[n+1]$[‡]. Ако обаче игнорираме тази несъществена подробност, това твърдение ни дава желания резултат, защото казва, че в $A[1,\ldots,n]$ няма пирамидални инверсии. Тоест, $A[1,\ldots,n]$ реализира пирамида. □

Сега ще изследваме сложността по време на наивното построяване на пирамида. Външният цикъл се изпълнява $\Theta(n)$ пъти без оглед на конкретния вход. Вътрешният цикъл се изпълнява, в най-лошия случай, $\lfloor \lg i \rfloor$ пъти за всяко i, понеже височината на подпирамидата с корен

[†] А за да бъде то празно, трябва да е вярно, че за всяко $b \in \textsc{Ancestors}(k)$, $b < a$.

[‡] Тази формална непрецизност можеше да бъде избягната, ако поначало бяхме присъединили към входния $A[1,\ldots,n]$ още един елемент $A[n+1]$ със стойност $-\infty$. Този елемент би имал роля, аналогична на т. нар. *sentinel* в Mergesort, който там е $+\infty$. Вижте например ред 7 на Merge.

$A[i]$ е $\lfloor \lg i \rfloor$. Тогава сложността се определя от сумата

$$\sum_{i=2}^{n} \lfloor \lg i \rfloor = \Theta\left(\sum_{i=2}^{n} \lg i\right) = \Theta\left(\lg\left(2 \times 3 \times \ldots \times (n-1) \times n\right)\right) = \Theta\left(\lg n!\right) \tag{4.1}$$

От Теорема 22 знаем, че $\lg n! \asymp n \lg n$. Следователно, сложността на наивното построяване е $\Theta(n \lg n)$.

4.2.2 Бързо построяване на пирамида: алгоритъм Build Heap

Алгоритъмът-предмет на тази подсекция е предложен от Robert Floyd [25]. Самият алгоритъм е показан на стр. 179. Той използва функция Heapify, която показваме в два варианта: итеративен и рекурсивен.

4.2.2.1 Предварителни обяснения

Да допуснем, че е дадено попълнено двоично дърво с ключове T. Нека коренът на T е u и децата на u са v и w. За целта на това обяснение, нека ключовете са съответно $u.key$, $v.key$ и $w.key$. Нека $T[\![v]\!]$ и $T[\![w]\!]$ са пирамиди. Ако $u.key \geqslant v.key$ и $u.key \geqslant w.key$, то T е пирамида. Да допуснем, че $u.key < v.key$ или $u.key < w.key$. Без ограничение на общността, нека $u.key < v.key$ и $u.key < w.key$. Тогава в T може да има много пирамидални инверсии. Пирамидалните инверсии може да са $n-1$ на брой, където n е броят на върховете на T, ако всеки връх от $T[\![v]\!]$ и от $T[\![w]\!]$ участва в инверсия с корена u.

Ключовото наблюдение е, че с $O(\lg n)$ размени на елементи може да премахнем всички пирамидални инверсии, тоест да направим T пирамида. Интуитивно е ясно, че трябва да разменим u с този връх измежду v и w, чийто ключ е максимален[†]. Без ограничение на общността, нека $v.key < w.key$. Тогава ще разменим u с w. Със сигурност след размяната w няма да участва в инверсия с нито един връх от $T[\![v]\!]$, така че след размяната ще инверсии—ако изобщо има—само в поддървото, чийто корен сега е u. Грубо казано, броят на инверсиите ще намалее наполовина или повече.[‡]

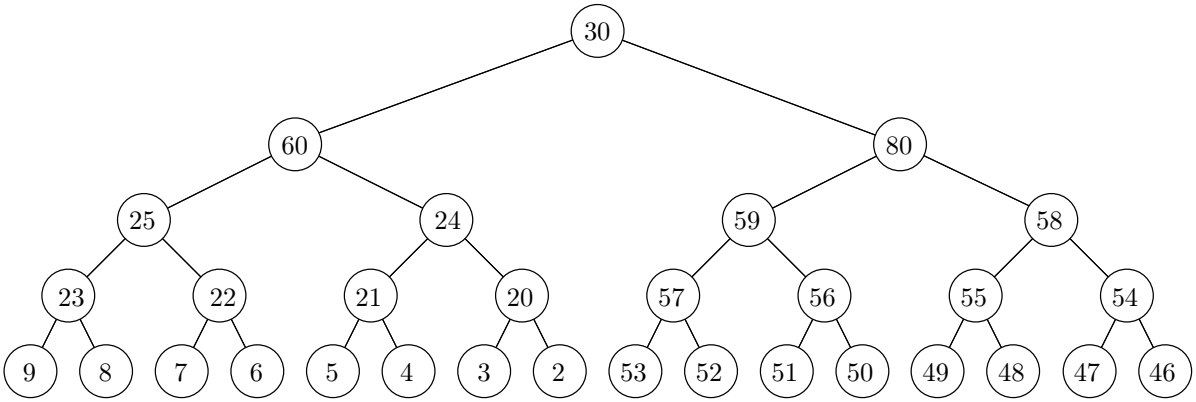
Забележете, че има примери, в които е по-изгодно да разменяме корена не с детето с по-големия ключ, а с детето с по-малкия ключ. Такова дърво е показано на Фигура 4.11. Първо да видим какво ще стане, ако върху това дърво прилагаме размяна на връх с това от двете деца, което има по-голям ключ. Фигура 4.12 показва дървото след първата размяна – елементите 30 и 80 са разменени, а с червено е очертан пътят, който ще "измине" върхът с ключ 30, движейки се надолу, докато не стане листо. На Фигура 4.13 е показано дървото след последната размяна, като е очертан пътят, който е "изминал" върхът с ключ 30. Това дърво е пирамида, получена след четири размени. За дървото от Фигура 4.11, ако разменим корена с детето с *по-малък* ключ, повече размени в лявото поддърво няма да има; ако разменим след това новия корен с ключ 60 с дясното дете с ключ 80, ще получим пирамидата, показана на Фигура 4.14, със само две размени.

Примерът, който видяхме току-що, се мащабира за всякакъв размер на дървото, така че наистина има случаи, в които разменянето с детето с по-малък ключ води до $\Theta(1)$ размени, а разменянето с детето с по-голям ключ води до $\Theta(\lg n)$ размени. Но ние се интересуваме от сложността **в най-лошия случай**. В най-лошия случай, разменянето с детето с по-малък

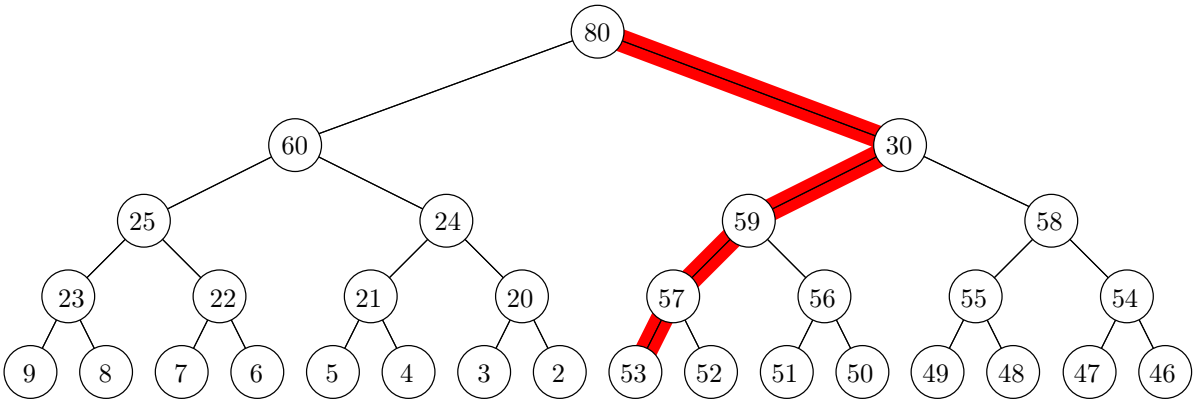
[†]Ако v и w имат еднакви ключове, ще разменим u с кой да е от тях.

[‡]Лесно е да се съобрази, че максималният брой инверсии след размяната, изразен чрез общия брой на върховете, е $\frac{2(n-2)}{3}$, където $n-2$ се дели на 3. Дори броят на инверсиите да намалява с деление на $\frac{3}{2}$ след всяка размяна, броят на размените е логаритмичен.

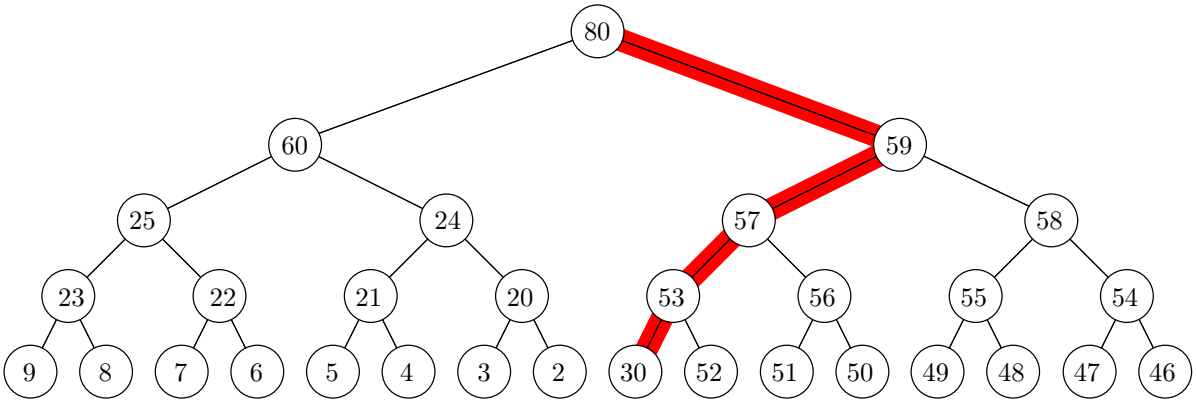
ключ може да доведе до $\Omega(n)$ размени, преди да получим пирамида, докато разменянето с детето с по-голям ключ води до $O(\lg n)$ размени **винаги**. Пример за дърво, върху което разменянето с детето с по-малък ключ води до $\Omega(n)$ размени (примерът се мащабира за безброй много n) е показан на Фигура 4.15. На Фигура 4.16 е показана пирамидата, която се получава от дървото на Фигура 4.15 след $\Omega(n)$ размени.



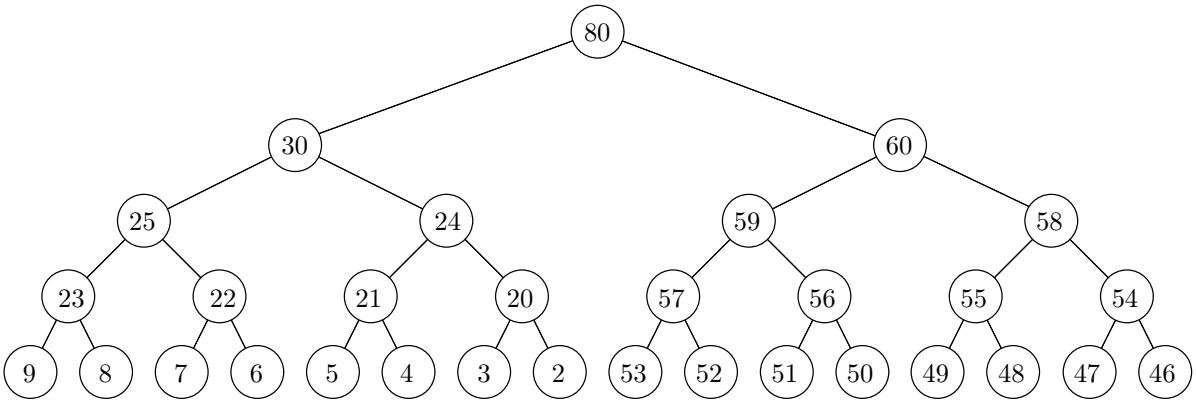
Фигура 4.11: Попълнено дърво с ключове, в което двете деца на корена са корени на поддървета-пирамиди, а ключът на корена е по-малък от ключовете на двете му деца. По-изгодно като минимален брой размени е коренът да бъде разменен с детето с по-малкия ключ.



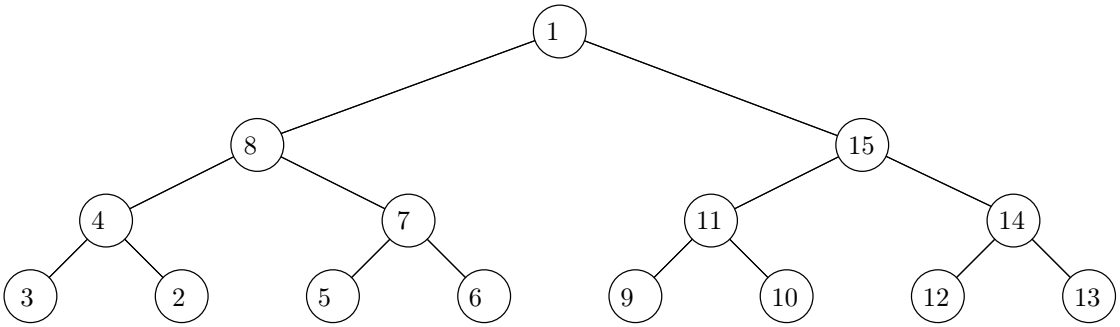
Фигура 4.12: По отношение на дървото от Фигура 4.11, ако разменим корена с детето с по-голям ключ и продължим размените по същия начин—с детето с по-голям ключ—ще направим 4 размени, докато върхът с ключ 30 не стане листо. Тук е показано дървото след първата размяна и е очертан пътят, който трябва да измине върхът с ключ 30. И така, тази фигура не показва окончателната позиция на върха с ключ 30, а само началото на "пътуването" му.



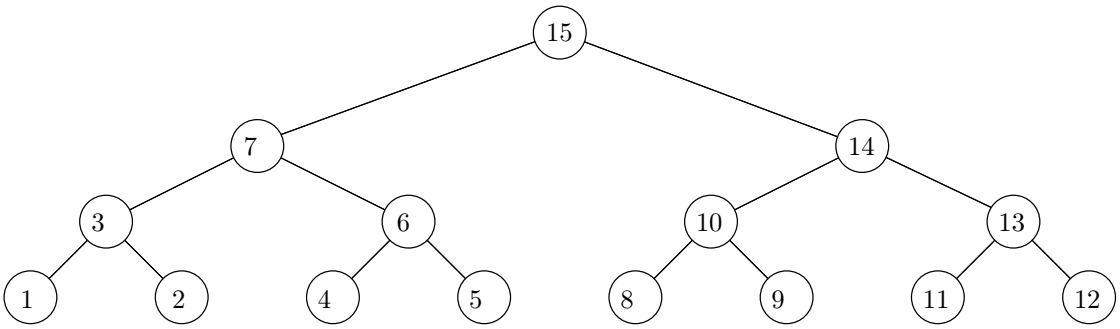
Фигура 4.13: По отношение на дървото от Фигура 4.12 е показано дървото след всички размени, където размените са от вида "разменяме корен с това дете, което има по-голям ключ". Извършени са 4 размени и дървото е пирамида. Очертан е пътят, който е "изминал" върхът с ключ 30.



Фигура 4.14: По отношение на дървото от Фигура 4.11, ако разменяме връх с корена с детето с **по-малък** ключ, ще направим само 2 размени и и ще получим показаната пирамида.



Фигура 4.15: Пример за дърво, в което, ако разменяме корена с детето с по-малък ключ (и после правим още една размяна на новия корен с детето с по-голям ключ), броят на размените е по-голям от броя на върховете.



Фигура 4.16: Ако в дървото, показано на Фигура 4.15, систематично разменяме първо корен с детето с по-малък ключ, и после новия корен с детето с по-голям ключ, ще получим пирамидата, показана тук. Броят на размените е по-голям от броя на върховете.

От изложението трябва да е станало ясно, че предложената идея—а именно, да разменяме с по-голямото дете—за конструиране на HEAPIFY би имала сложност $O(\lg n)$, ако се реализира грамотно: с $O(\lg n)$ размени на елементи унищожаваме всички пирамидални инверсии, а всяка размяна става в константно време.

4.2.2.2 Неформално въвеждане на Heapify

В изложението досега "пирамидизирахме" цялото дърво – при допускането, че само коренът има какъв да е ключ, а лявото и дясното поддърво са пирамиди. Да разгледаме по-общата възможност: разглеждаме произволен вътрешен връх u в дървото T, такъв че и двете поддървета (или само едно, в екстремния случай, в който дясно дете няма), вкоренени в неговите деца, са пирамиди, а ключът на u е произволен, и целта е да "пирамидизираме" $T[\![u]\!]$. Освен това, вече мислим за пирамидата не на високо абстрактно ниво като дърво, а като масив; тоест, разглеждаме нейната линеаризация $A[1,\ldots,n]$.

И така, даден е масив A и индекс i в A. Ако $i > \lfloor \frac{n}{2} \rfloor$, то $A[i]$ отговаря на листо и няма какво да правим, понеже $A[\![i]\!]$ е тривиална пирамида.

Нека $i \leqslant \lfloor \frac{n}{2} \rfloor$; с други думи, нека $A[i]$ отговаря на вътрешен връх. БОО, нека $2i+1 \leqslant n$. Ключовото допускане е, че $A[\![\textsc{Left}(i)]\!]$ и $A[\![\textsc{Right}(i)]\!]$ **са пирамиди**. Без това допускане, алгоритъмът Heapify е **безполезен**. Heapify размества елементите на $A[\![i]\!]$ така, че $A[\![i]\!]$ става пирамида, без да променя нищо извън $A[\![i]\!]$. Това се постига с последователни размени на елемента, който в началото е на позиция i, с това от децата му, което има по-голям ключ, докато или този елемент не стане листо, или и двете му деца (или единственото му дете, ако няма дясно дете) не се окажат по-малки или равни на него.

4.2.2.3 Heapify в итеративен вариант

```
Iterative Heapify(A[1,2,...,n]: масив от цели числа, i: индекс в A)
 1  (* A[[Left(i)]] и A[[Right(i)]], ако съществуват, са пирамиди *)
 2  j ← i
 3  while j ⩽ ⌊n/2⌋ do
 4      left ← Left(j)
 5      right ← Right(j)
 6      if A[left] > A[j]
 7          (* Няма смисъл да проверяваме дали left ⩽ n, понеже j ⩽ ⌊n/2⌋ *)
 8          largest ← left
 9      else
10          largest ← j
11      if right ⩽ n and A[right] > A[largest]
12          largest ← right
13      if largest ≠ j
14          swap(A[j], A[largest])
15          j ← largest
16      else
17          break
```

Лема 30

При допускането, че $A[1,\ldots,n]$ и i са такива, че всеки от $A[\![\textsc{Left}(i)]\!]$ и $A[\![\textsc{Right}(i)]\!]$, ако съществува, е пирамида, ефектът от Iterative Heapify е, че $A[\![i]\!]$ е пирамида при неговото приключване.

Доказателство: Следното твърдение е инвариант за **while**-цикъла (редове 2–17).

Инвариант 12: Цикълът на Iterative Heapify

Всеки път, когато изпълнението е на ред 3, единствените възможни директни инверсии в $A[\![i]\!]$ са $\langle j, \textsc{Left}(j)\rangle$ и $\langle j, \textsc{Right}(j)\rangle$[a], ако $A[\textsc{Left}(j)]$ и $A[\textsc{Right}(j)]$ съществуват.

[a]По друг начин казано, ако съществуват пирамидални инверсии в $A[\![i]\!]$, то те са в $A[\![j]\!]$.

База. $j = i$. Съгласно допусканията, всеки от $A[\![\textsc{Left}(i)]\!]$ и $A[\![\textsc{Right}(i)]\!]$, ако съществува, е пирамида, така че твърдението е в сила. ✓

Поддръжка. Допускаме, че твърдението е в сила в някой момент, в който изпълнението е на ред 3 и изпълнението ще бъде на ред 3 поне още веднъж.[§] И така, $j \leqslant \left\lfloor \frac{n}{2} \right\rfloor$ и поне едно от $\textsc{Left}(j) \leqslant n$ и $\textsc{Right}(j) \leqslant n$ е изпълнено, а именно $\textsc{Left}(j) \leqslant n$. Тогава поне $A[\textsc{Left}(j)]$ е дефинирано. Без ограничение на общността ще допуснем, че и $\textsc{Left}(j) \leqslant n$, и $\textsc{Right}(j) \leqslant n$, за да избегнем ненужни подслучаи.

Случай I: $A[\mathit{left}] > A[j]$ и $A[\mathit{right}] > A[j]$ в началото на изпълнението на цикъла, и освен това $A[\mathit{right}] > A[\mathit{left}]$. Условието на ред 6 е Истина и присвояването на ред 8 се случва. Условието на ред 11 също е Истина, така че и присвояването на ред 12 се случва. Когато изпълнението е на ред 13, $\mathit{largest}$ е равно на right. Условието на ред 13 е Истина и на ред 14, $A[j]$ и $A[\mathit{largest}]$ биват разменени. Твърдим, че единствените възможни пирамидални инверсии в $A[\![i]\!]$ след размяната са $\langle \mathit{right}, \textsc{Left}(\mathit{right})\rangle$ и $\langle \mathit{right}, \textsc{Right}(\mathit{right})\rangle$:

- нито един елемент от $A[\![i]\!]$, който е извън $A[\![j]\!]$, не е променян;
- $A[j] > A[\mathit{left}]$, така че $\langle j, \textsc{Left}(j)\rangle$ не може да е пирамидална инверсия;
- $A[j] > A[\mathit{right}]$, така че $\langle j, \textsc{Right}(j)\rangle$ не може да е пирамидална инверсия;
- $A[\![\mathit{left}]\!]$ не е бил модифициран;
- нито един от $A[\![\textsc{Left}(\mathit{right})]\!]$ и $A[\![\textsc{Right}(\mathit{right})]\!]$ не е бил модифициран.

Но на ред 15, j става равен на right. Следователно, инвариантът е в сила при следващото достигане на ред 3.

Случай II: $A[\mathit{left}] > A[j]$ и $A[\mathit{right}] > A[j]$ в началото на изпълнението на цикъла, но $A[\mathit{right}] \not> A[\mathit{left}]$. Условието на ред 6 е Истина и присвояването на ред 8 се случва. Условието на ред 11 е Лъжа, следователно присвояването на ред 12 не се случва и когато изпълнението е на ред 13, $\mathit{largest}$ е равно на left. Условието на ред 13 е Истина и на ред 14, $A[j]$ и $A[\mathit{largest}]$ биват разменени. Твърдим, че единствените възможни пирамидални инверсии в $A[\![i]\!]$ след размяната са $\langle \mathit{left}, \textsc{Left}(\mathit{left})\rangle$ и $\langle \mathit{left}, \textsc{Right}(\mathit{left})\rangle$:

- нито един елемент от $A[\![i]\!]$, който е извън $A[\![j]\!]$, не е променян;
- $A[j] > A[\mathit{left}]$, така че $\langle j, \textsc{Left}(j)\rangle$ не може да е пирамидална инверсия;
- $A[j] \geqslant A[\mathit{right}]$, така че $\langle j, \textsc{Right}(j)\rangle$ не може да е пирамидална инверсия;
- $A[\![\mathit{right}]\!]$ не е бил модифициран;
- нито един от $A[\![\textsc{Left}(\mathit{left})]\!]$ и $A[\![\textsc{Right}(\mathit{left})]\!]$ не е бил модифициран.

[§]Последното имплицира, че ред 17 няма да бъде изпълнен.

Но на ред 15, j става равен на left. Следователно, инвариантът е в сила при следващото достигане на ред 3.

Случай iii: $A[\mathit{left}] \not> A[j]$ и $A[\mathit{right}] > A[j]$ в началото на изпълнението на цикъла. Условието на ред 6 е ЛЪЖА пристояването на ред 10 се случва. Условието на ред 11 е TRUE, така че присвояването на ред 12 се случва и когато изпълнението е на ред 13, $\mathit{largest}$ е равно на right. Условието на ред 13 е ИСТИНА и на ред 14, A[j] и $A[\mathit{largest}]$ биват разменени. Твърдим, че единствените възможни пирамидални инверсии в $A[\![i]\!]$ след размяната са $\langle \mathit{right}, \text{LEFT}(\mathit{right})\rangle$ и $\langle \mathit{right}, \text{RIGHT}(\mathit{right})\rangle$. Доказателството е точно като доказателството на аналогичното твърдение в **Случай I**. Но на ред 15, j става равен на $\mathit{largest}$. Следователно, инвариантът е в сила при следващото достигане на ред 3.

Случай IV: $A[\mathit{left}] > A[j]$ и $A[\mathit{right}] \not> A[j]$ в началото на изпълнението на цикъла. Условието на ред 6 е TRUE, така че присвояването на ред 8 се случва. Условието на ред 11 е FALSE, така че присвояването на ред 12 не се случва и когато изпълнението е на ред 13, $\mathit{largest}$ е равно на left. Условието на ред 13 е TRUE и на ред 14, A[j] и $A[\mathit{largest}]$ биват разменени. Твърдим, че единствените в $A[\![i]\!]$ след размяната са $\langle \mathit{left}, \text{LEFT}(\mathit{left})\rangle$ и $\langle \mathit{left}, \text{RIGHT}(\mathit{left})\rangle$. Доказателството е точно като доказателството на аналогичното твърдение в **Случай II**. Но на ред 15, j става равен на $\mathit{largest}$. Следователно, инвариантът е в сила при следващото достигане на ред 3.

Случай V: $A[\mathit{left}] \not> A[j]$ и $A[\mathit{right}] \not> A[j]$ в началото на изпълнението на цикъла. Но това е невъзможно при текущите допускания, защото очевидно тогава ред 17 би бил достигнат и изпълнението не би се върнало повече на ред 3.

Терминация. Цикълът може да бъде напуснат по два начина: през ред 3, когато $j > \left\lfloor \frac{n}{2} \right\rfloor$, и през ред 17. Да разгледаме първата възможност. Тогава и LEFT(j), и RIGHT(j) са индекси извън $A[1, \ldots, n]$. В този случай инвариантът казва, че няма пирамидални инверсии в $A[\![i]\!]$ изобщо, понеже A[LEFT(j)] и A[RIGHT(j)] не съществуват. Следователно, $A[\![i]\!]$ е пирамида.

Да разгледаме втората възможност, а именно **while**-цикълът да бъде напуснат през ред 17. Очевидно, за да бъде достигнат ред 17, трябва да бъде вярно, че $\mathit{largest} = j$ на ред 13. А за да е вярно това, и $A[\mathit{left}] \leqslant A[j]$, и $A[\mathit{right}] \leqslant A[j]$ трябва да са в сила в началото на изпълнението на цикъла, понеже това е единственият начин ред 10 да бъде достигнат и ред 12 да не бъде достигнат. Но ако $A[\mathit{left}] \leqslant A[j]$ и $A[\mathit{right}] \leqslant A[j]$, пирамидални инверсии в $A[\![i]\!]$ няма, така че $A[\![i]\!]$ е пирамида. □

4.2.2.4 HEAPIFY в рекурсивен вариант

```
RECURSIVE HEAPIFY(A[1, 2, ..., n]: масив от цели числа, i: индекс в A)
 1  (* A[[LEFT(i)]] и A[[RIGHT(i)]], ако съществуват, са пирамиди *)
 2  left ← LEFT(i)
 3  right ← RIGHT(i)
 4  if left ⩽ n  and A[left] > A[i]
 5      largest ← left
 6  else
 7      largest ← i
 8  if right ⩽ n  and A[right] > A[largest]
 9      largest ← right
10  if largest ≠ i
11      swap(A[i], A[largest])
12      RECURSIVE HEAPIFY(A, largest)
```

Лема 31

При допускането, че $A[1,\ldots,n]$ и i са такива, че всеки от $A[\![\textsc{Left}(i)]\!]$ и $A[\![\textsc{Right}(i)]\!]$, ако съществува, е пирамида, ефектът от RECURSIVE HEAPIFY е, че $A[\![i]\!]$ е пирамида при неговото приключване.

Доказателство: По индукция по височината h на $A[\![i]\!]$.

Преди да направим доказателството, ще подчертаем следното: допусканията за $A[\![\textsc{Left}(i)]\!]$ и $A[\![\textsc{Right}(i)]\!]$—а именно, че са пирамиди—са нещо напълно различно от индуктивното предположение. Индуктивното предположение е твърдението, което доказваме, формулирано чрез някаква стойност на параметъра, на която сме дали име. Индуктивното предположение не е част от условията на лемата, докато споменатите допускания са част от условията на лемата.

База. $h = 0$. Тогава $A[\![i]\!]$ съдържа един единствен елемент A. С други думи, това е листо. Тогава и $\textsc{Left}(i)$, и $\textsc{Right}(i)$ са индекси извън A. Да проследим изпълнението на RECURSIVE HEAPIFY: условието на ред 4 е ЛЪЖА, следователно присвояването на ред 7 се случва. Условието на ред 8 също е ЛЪЖА, така че ред 9 не се изпълнява и изпълнението преминава към ред 10. Условието там е ЛЪЖА и текущото рекурсивно извикване терминира. Очевидно $A[\![i]\!]$ е пирамида при терминирането. ✓

Индуктивно предположение. Да допуснем, че за всяко $A[\![j]\!]$ от височина $\leqslant h-1$ с корен някой j, такъв че $A[j]$ е в $A[\![i]\!]$, е изпълнено, че $\textsc{Recursive Heapify}(A, j)$ превръща $A[\![j]\!]$ чрез разместванията в пирамида.

Индуктивна стъпка. Да разгледаме изпълнението на $\textsc{Recursive Heapify}(A, i)$. Без ограничение на общността, да допуснем, че $\textsc{Left}(i) \leqslant n$ и $\textsc{Right}(i) \leqslant n$, така че редове 4 и 8 са съответно

if $A[\mathit{left}] > A[i]$

и

if $A[\mathit{right}] > A[\mathit{largest}]$

Случай I: $A[\mathit{left}] > A[i]$, $A[\mathit{right}] > A[i]$ и $A[\mathit{right}] > A[\mathit{left}]$. Условието на ред 4 е ИСТИНА и присвояването на ред 5 се случва. Условието на ред 8 също е ИСТИНА, така че и присвояването на ред 9 се случва. Когато изпълнението е на ред 10, $\mathit{largest}$ е равно на right. Условието на ред 10 е ИСТИНА и на ред 11, $A[i]$ и $A[\mathit{largest}]$ биват разменени. Рекурсивното повикване на ред 12 се случва, като бившият $A[i]$ сега е на позиция right. Съгласно индуктивното предположение, това рекурсивно викане построява пирамида от от $A[\![\mathit{right}]\!]$. От друга страна, $A[\![\mathit{left}]\!]$ е пирамида, защото не е бил променян по никакъв начин. От трета страна, текущият $A[i]$, тоест първоначалният $A[\mathit{right}]$, е:

- по-голям от текущия $A[\mathit{left}]$ по допускане;
- не по-малък от текущия $A[\mathit{right}]$, по следните причини. Първоначално, $A[\![\mathit{right}]\!]$ е пирамида, така че бившият $A[\mathit{right}]$ е не по-малък от всеки друг елемент от $A[\![\mathit{right}]\!]$ в онзи момент (в началото). В края, елементите на $A[\![\mathit{right}]\!]$ се състоят от началните елементи без началния $A[\mathit{right}]$, плюс началния $A[i]$. Тъй като началният $A[\mathit{right}]$ е по-голям от началния $A[i]$ и не по-малък от всеки друг елемент от $A[\![\mathit{right}]\!]$, текущият $A[i]$ не е по-малък от текущия $A[\mathit{right}]$.

Тогава, $A[\![i]\!]$ е пирамида.

Случай II: $A[left] > A[i]$, $A[right] > A[i]$ и $A[right] \not> A[left]$. Условието на ред 4 е Истина и присвояването на ред 5 се случва. Условието на ред 8 е Лъжа, така че присвояването на ред 9 не се случва и когато изпълнението е на ред 10, $largest$ е равно на $left$. Условието на ред 10 е Истина и на ред 11, $A[i]$ и $A[largest]$ биват разменени. Рекурсивното извикване на ред 12 се случва, с бившия $A[i]$ сега на позиция $left$. Съгласно индуктивното предположение, това рекурсивно извикване построява пирамида от $A[\![left]\!]$. От друга страна, $A[\![right]\!]$ е пирамида, защото не е бил променян по никакъв начин. От трета страна, текущият $A[i]$, тоест първоначалният $A[left]$, е:

- не по-малък от текущия $A[right]$ по допускане
- не по-малък от текущия $A[left]$, по следните причини. Първоначално, $A[\![left]\!]$ е пирамида, така че бившият $A[left]$ е не по-малък от всеки друг елемент на $A[\![left]\!]$ в онзи момент (в началото). В края, елементите на $A[\![left]\!]$ се състоят от началните елементи без началния $A[left]$, плюс началния $A[i]$. Тъй като началният $A[left]$ е по-голям от началния $A[i]$ и не по-малък от всеки друг елемент от $A[\![left]\!]$, текущият $A[i]$ не е по-малък от текущия $A[left]$.

Тогава, $A[\![i]\!]$ е пирамида.

Случай III: $A[left] \not> A[i]$ и $A[right] > A[i]$. Условието на ред 4 е Лъжа и присвояването на ред 7 се случва. Условието на ред 8 е Истина, така че присвояването на ред 9 се случва и когато изпълнението е на ред 10, $largest$ е равно на $right$. Условието на ред 10 е Истина и на ред 11, $A[i]$ и $A[largest]$ биват разменени. Рекурсивното извикване на ред 12 се случва, с бившия $A[i]$ сега на позиция $right$. Съгласно индуктивното предположение, това рекурсивно извикване построява пирамида от $A[\![right]\!]$. От друга страна, $A[\![left]\!]$ е пирамида, защото не е бил променян по никакъв начин. От трета страна, текущият $A[i]$, тоест първоначалният $A[right]$, е:

- по-голям от текущия $A[left]$ заради допусканията и транзитивността на неравенствата;
- не по-малък от текущия $A[right]$ по следните причини. Първоначално, $A[\![right]\!]$ е пирамида, така че бившият $A[right]$ е не по-малък от всеки друг елемент на $A[\![right]\!]$ в онзи момент (в началото). В края, елементите на $A[\![right]\!]$ се състоят от началните елементи без началния $A[right]$, плюс началния $A[i]$. Тъй като началният $A[right]$ е по-голям от началния $A[i]$ и не по-малък от всеки друг елемент от $A[\![right]\!]$, текущият $A[i]$ не е по-малък от текущия $A[right]$.

Тогава, $A[\![i]\!]$ е пирамида.

Случай IV: $A[left] > A[i]$ и $A[right] \not> A[i]$. Условието на ред 4 е Истина и присвояването на ред 5 се случва. Условието на ред 8 е Лъжа, така че присвояването на ред 9 не се случва и когато изпълнението е на ред 10, $largest$ е равно $left$. Условието на ред 10 е Истина и на ред 11, $A[i]$ и $A[largest]$ биват разменени. Рекурсивното извикване на ред 12 се случва, с бившия $A[i]$ сега на позиция $left$. Съгласно индуктивното предположение, това рекурсивно извикване построява пирамида от $A[\![left]\!]$. От друга страна, $A[\![right]\!]$ е пирамида, защото не е бил променян по никакъв начин. От трета страна, текущият $A[i]$, тоест първоначалният $A[left]$, е:

- по-голям от текущия $A[right]$ заради допусканията и транзитивността на неравенствата;

- не по-малък от текущия $A[\mathit{left}]$ поради следните причини. Първоначално, $A[\![\mathit{left}]\!]$ е пирамида, така че бившият $A[\mathit{left}]$ е не по-малък от всеки друг елемент от $A[\![\mathit{left}]\!]$ в онзи момент (в началото). В края, елементите на $A[\![\mathit{left}]\!]$ се състоят от началните елементи без началния $A[\mathit{left}]$, плюс началния $A[i]$. Тъй като началният $A[\mathit{left}]$ е по-голям от началния $A[i]$ и не по-малък от всеки друг елемент от $A[\![\mathit{left}]\!]$, текущият $A[i]$ не е по-малък от текущия $A[\mathit{left}]$.

Тогава, $A[\![i]\!]$ е пирамида.

Случай V: $A[\mathit{left}] \not> A[i]$ и $A[\mathit{right}] \not> A[i]$. Условието на ред 4 е ЛЪЖА и присвояването на ред 7 се случва. Условието на ред 8 също е ЛЪЖА, така че присвояването на ред 9 не се случва и когато изпълнението достигне ред 10, $\mathit{largest}$ е равно на i. Условието на ред 10 е ЛЪЖА и изпълнението терминира, оставяйки $A[\![i]\!]$ непроменен. Съгласно допусканията, $A[\![\mathit{left}]\!]$ и $A[\![\mathit{right}]\!]$ са пирамиди и $A[\mathit{left}] \not> A[i]$ и $A[\mathit{right}] \not> A[i]$. Тогава, $A[\![i]\!]$ е пирамида. □

4.2.2.5 Алгоритъм Build Heap

Идеята на бързия алгоритъм за построяване на пирамида е да сканира масива отдясно наляво, иначе казано, от листата към корена, започвайки от първото не-листо. Листата са тривиални пирамиди.

За всяко не-листо $A[i]$ в указания ред, при допускането, че и двете му деца (или едното му дете, ако друго дете няма) са корени на подпирамиди, ако $A[i]$ е по-голям или равен от двете си деца (или едното си дете), то $A[\![i]\!]$ е подпирамида. Ако това не е вярно, ще направим $A[\![i]\!]$ пирамида чрез серия от размени на елементи в нея, разменяйки корен с по-голямото дете, избутвайки малкия елемент, който образува пирамидални инверсии, в посока към листата, докато той или не стане листо, или не се окаже по-голям или равен на децата си (детето си).

Това всъщност означава, че викаме Heapify върху всяко нелисто, отдясно наляво. Функцията Heapify в следния алгоритъм е или Recursive Heapify на стр. 176, или Iterative Heapify на стр. 174.

```
Build Heap(A[1, 2, ..., n]: array of integers)
1    for i ← ⌊n/2⌋ downto 1
2        Heapify(A, i)
```

Теорема 32: Коректност на бързото построяване на пирамида

За всеки вход A, Build Heap построява пирамида от него.

Доказателство: Следното твърдение е инвариант на **for**-цикъла (редове 1–2).

Инвариант 13: Цикълът на Build Heap

Всеки път, когато изпълнението е на ред 1, $A[\![i+1]\!]$, $A[\![i+2]\!]$, ..., $A[\![n]\!]$ са пирамиди.

База. При първото достигане на ред 1, i е равно на $\lfloor \frac{n}{2} \rfloor$. Тогава $A[\![\lfloor \frac{n}{2} \rfloor + 1]\!]$, $A[\![\lfloor \frac{n}{2} \rfloor + 2]\!]$, ..., $A[\![n]\!]$ са точно листата на попълненото дърво, чиято линеаризация е A. Очевидно всяко листо е пирамида. ✓

Поддръжка. Да разгледаме някое достигане на ред 1, което не е последното. И $A[\![\textsc{Left}(i)]\!]$, и $A[\![\textsc{Right}(i)]\!]$ (ако съществува) са пирамиди съгласно индуктивното предположение, понеже $i < \textsc{Left}(i)$ и $i < \textsc{Right}(i)$. Тогава използваме доказаната коректност на HEAPIFY и заключаваме, че $A[\![i]\!]$ става пирамида. След това i намалява с единица. Спрямо новата стойност на i, инвариантът е в сила. ✓

Терминация. Когато изпълнението е на ред 2 за последен път, $i = 0$. Тогава $A[\![0+1]\!]$, $A[\![0+2]\!]$, $\ldots$, $A[\![n]\!]$ са пирамиди. В частност, $A[\![1]\!]$ е пирамида. Но $A[\![1]\!]$ е A. Това доказва лемата. ✓ □

Лема 32

BUILD HEAP работи във време $\Theta(n)$.

Доказателство:
Това, че сложността по време е $\Omega(n)$, е очевидно. Ще покажем, че сложността е $O(n)$. Асимптотична горна граница за сложността на BUILD HEAP е сумата по всички височини от 1 (BUILD HEAP прескача листата) до $\lfloor \lg n \rfloor$ (височината на дървото) от произведението от броя на върховете с дадената височина и самата височина (всеки връх може да се "придвижва" надолу към листата с най-много толкова размени, колкото е височината му поначало). Съгласно Теорема 30, броят на върховете с височина k е $\left\lceil \frac{\lfloor \frac{n}{2^k} \rfloor}{2} \right\rceil$. И така, ако $T(n)$ е функцията на сложността, то в най-лошия случай,

$$T(n) = \sum_{k=1}^{\lfloor \lg n \rfloor} \left\lceil \frac{\lfloor \frac{n}{2^k} \rfloor}{2} \right\rceil \Theta(k) = \sum_{k=1}^{\lfloor \lg n \rfloor} \Theta\left(\frac{n}{2^k} k\right)$$

Да разгледаме $\sum_{k=1}^{\lfloor \lg n \rfloor} \frac{n}{2^k} k$. В сила е

$$\sum_{k=1}^{\lfloor \lg n \rfloor} \frac{n}{2^k} k \leqslant \sum_{k=1}^{\infty} \frac{n}{2^k} k = n \sum_{k=1}^{\infty} \frac{k}{2^k}$$

Тривиално се доказва, че редът $\sum_{k=1}^{\infty} \frac{k}{2^k}$ е сходящ, примерно с критерия на d'Alembert:

$$\lim_{k \to \infty} \frac{\frac{k+1}{2^{k+1}}}{\frac{k}{2^k}} = \frac{1}{2} < 1$$

Щом $\sum_{k=1}^{\infty} \frac{k}{2^k}$ е ограничен от константа, то $n \sum_{k=1}^{\infty} \frac{k}{2^k} \asymp n$ и очевидно $\sum_{k=1}^{\lfloor \lg n \rfloor} \frac{n}{2^k} k \asymp n$. Тогава $T(n) \asymp n$. □

4.3 Сортиращ алгоритъм HEAPSORT

Следният сортиращ алгоритъм е предложен от Williams [96]. Той е първият от бързите сортиращи алгоритми, които ще разгледаме. "Бърз" наричаме алгоритъм, чиято сложност по време е $O(n \lg n)$.

Нека $A.size$ относно масива $A[1, \ldots, n]$ е число, такова че $1 \leqslant A.size \leqslant n$ и HEAPIFY работи върху подмасива $A[1, \ldots, A.size]$, а **не** върху $A[1, \ldots, n]$.

HEAPSORT($A[1, 2, \ldots, n]$)
1 BUILD HEAP(A)

```
2   A.size ← n
3   for i ← n downto 2
4       swap(A[1], A[i])
5       A.size ← A.size − 1
6       Heapify(A, 1)
```

Лема 33

Heapsort е сортиращ алгоритъм.

Доказателство: Нека $A'[1,\ldots,n]$ означава първоначалния масив. Следното твърдение е инвариант на цикъла за **for**-цикъла (редове 3–6).

Инвариант 14: Цикълът на Heapsort

Всеки път, когато изпълнението на Heapsort е на ред 3:

- Текущият подмасив $A[i+1,\ldots,n]$ се състои от $n-i$ на брой най-големи елементи на $A'[1,\ldots,n]$ в сортиран вид.
- Освен това, текущият $A[1,\ldots,i]$ е пирамида.

База. При първото достигане на ред 3, $i = n$. Подмасивът $A[i+1,\ldots,n]$ е празен, следователно, в празния смисъл (*vacuously*), той се състои от нула на брой най-големи елементи от $A'[1,\ldots,n]$, в сортиран вид, следователно първата част на инвариантът е в сила. $A[1,\ldots,n]$ е пирамида съгласно Теорема 32, приложена към ред 1, така че и втората част на инвариантът е в сила. ✓

Поддръжка. Да допуснем, че твърдението е в сила при някакво достигане на ред 3, което не е последното. Нека да наричаме масива A в този момент, A''. Съгласно първата част на индуктивното предположение, $A''[i+1,\ldots,n]$ се състои от $n-i$ на брой елементи от A' в сортиран вид. Съгласно втората част на индуктивното предположение, $A''[1]$ е максимален елемент от $A''[1,\ldots,i]$. След размяната на ред 4, $A''[i,\ldots,n]$ съдържа $n-i+1$ на брой най-големи елемента на A' в сортиран вид. Спрямо новата стойност на i при следващото достигане на ред 3, първата част на инварианта е в сила.

Ще докажем, че втората част на инварианта е в сила. Да приложим Лема 30 или Лема 31 в зависимост от това дали ползваме рекурсивна или итеративна Heapify функция на ред 6. Трябва да се има предвид, че Heapify работи върху $A[1,\ldots,i-1]$, защото i е равно на A.size в момента , в който изпълнението е на ред 6, а на ред 5, A.size е получил стойност $i-1$. Заради това, на ред 6 текущият $A[i]$ е "извън обхвата" на Heapify.

Терминация. Да разгледаме момента, в който изпълнението е на ред 3 за последен път. Очевидно, $i = 1$. Да заместим 1 на мястото на i в инварианта. Получаваме "текущият подмасив $A[2,\ldots,n]$ се състои от $n-1$ на брой най-големи елементи на $A'[1,\ldots,n]$ в сортиран вид". Но тогава $A[1]$ е минимален елемент на $A'[1,\ldots,n]$. С това доказателството за коректност на Heapsort приключва. □

Да разгледаме сложността по време на Heapsort. Сложността на Build Heap е $\Theta(n)$, което доказахме в Лема 32. Сложността на **for**-цикъла (редове 3–6) $\Theta(n \lg n)$, защото за всяко i, сложността на Heapify в най-лошия случай е $\Theta(\lg i)$, а ние вече изследвахме сумата

$\sum_{i=1}^{n} \lg i$ (вж. (4.1)) и установихме, че асимптотично и́ нарастване е $\Theta(n \lg n)$. Следователно, сложността на HEAPSORT е $\Theta(n) + \Theta(n \lg n) = \Theta(n \lg n)$.

Сложността по памет на HEAPSORT е $\Theta(1)$, ако HEAPIFY е в итеративния вариант. Рекурсивният вариант RECURSIVE HEAPIFY на стр. 176, ако бъде реализиран директно по псевдокода, ползва в най-лошия случай $\Theta(\lg n)$ работна памет заради рекурсивните викания, които са $\Theta(\lg n)$ на брой в най-лошия случай.

HEAPSORT не е стабилен сортиращ алгоритъм. За да се убедим в това, достатъчно е да разгледаме работата му над вход от еднакви ключове. При дадения псевдокод, BUILD HEAP няма да промени нищо, след което още първото изпълнение на **for**-цикъла ще размени еднаквите елементи $A[1]$ и $A[n]$, като този елемент, който бива записан в $A[n]$, ще остане там до края на алгоритъма.

Дори да сложим проверка на ред 4, такава размяната да не се изпълнява, ако $A[1]$ и $A[i]$ са равни, това няма да направи алгоритъма стабилен. Да си представим вход от k ключа 2 и k ключа 1, като двойките са преди единиците. BUILD HEAP няма да промени нищо, след което още първото изпълнение на **for**-цикъла ще размени двойката в $A[1]$ с единицата в $A[n]$, при което двойката, която е била вляво от всички други двойки, ще се окаже вдясно от тях, и ще остане така до края на алгоритъма.

4.4 Приоритетни опашки

4.4.1 Абстрактен Тип Данни (АТД)

Приоритетна опашка (на английски, *priority queue*) е вид *абстрактен тип данни* (на английски, *abstract data type*, или ADT). Първо ще изясним понятието абстрактен тип данни (АТД). Според Sedgewick и Wayne [84, стр. 64]:

> An *abstract data type* (ADT) is a data type whose representation is hidden from the client.
> ...
> When using an ADT, we focus on the operations specified in the API and pay no attention to the data representation; when implementing an ADT, we focus on the data, then implement operations on that data.
>
> To specify the behavior of an abstract data type, we use an *application programming interface* (API), ...

И така, АТД е множество от данни, обикновено от един и същи тип, заедно с множество функции върху тях, което множество от функции се нарича *интерфейс*. "Абстрактен" означава, че конкретната реализация остава скрита, а цялото "общуване" с данните става през интерфейса. Конкретната реализация би могла да бъде променяна, като това остава прозрачно за софтуера, който ползва АТД-то (или поне на теория софтуерът-потребител не би трябвало да "усеща" разликата, ако няма бъгове и интерфейсът на новата реализация е изграден съгласно спецификациите). Конкретната реализация остава напълно скрита зад интерфейса и можем да си представяме АТД-то като черна кутия (*black box*) плюс интерфейс. Прости примери за АТД са *стек* и *опашка*, наричани още съответно LIFO и FIFO [84, стр. 121] (вж. Фигура 4.17 за илюстрация на черна кутия с интерфейс).

4.4.2 Приоритетна опашка: вид АТД

Приоритетните опашки имат елементи, всеки от които има две стойности: ключ и данни, свързани с този ключ. Ключовете обикновено са цели числа, макар че всеки тип данни, кой-



Фигура 4.17: АТД приоритетна опашка като черна кутия. Конкретната имплементация е скрита. Отвън се "виждат" само трите функции на интерфейса.

то може да се използва в контекста на сортирането, може да е типът данни на ключовете. Приоритетните опашки, подобно на пирамидите, са максимални и минимални. Тук ще дискутираме само максимални приоритетни опашки, като изложението за минималния тип е напълно аналогично. Казвайки "приоритетна опашка", имаме предвид максимална приоритетна опашка.

Интерфейсът на приоритетните опашки обикновено се ограничава до следните три функции. Ако приоритетната опашка се казва S, функциите от интерфейса са:

- Maximum(S): връща елемента с максимален ключ;
- Extract-Max(S): връща елемента с максимален ключ и го премахва от S;
- Insert(S, x): вмъква в S елемент x, който е от подходящ за S тип.

Написаното тук е според учебника на Cormen и др. [21, стр. 162]. Авторите дават и четвърта функция от интерфейса, а именно Increase-Key(S, x, k), която има смисъл на "увеличи ключа на x на нова стойност k, при допускането, че тя е не по-малка от стария ключ на x". Тази функция, която ще дискутираме след малко, не е част от стандартния интерфейс на приоритетните опашки. Sedgewick и Wayne [84, стр. 309] дават следния изчерпателен списък на интерфейса на реални (които се ползват на практика) приоритетни опашки. Описанието е на езиковите конвенции на Java.

- `MaxPQ()`: create a priority queue
- `MaxPQ(int max)`: create a priority queue of initial capacity `max`
- `MaxPQ(Key[] a)`: create a priority queue from the keys in `a[]`
- `void insert(Key v)`: insert a key into the priority queue
- `Key max()`: return the largest key
- `Key delMax()`: return and remove the largest key
- `boolean isEmpty()`: is the priority queue empty?
- `int size()`: number of keys in the priority queue

За целите на тази лекция, интерфейсът е според учебника на Cormen и др., като Increase-Key не е част от него, освен ако това не е казано изрично. Фигура 4.17 илюстрира приоритетна опашка като черна кутия плюс интерфейс.

Приложенията на приоритетните опашки са много. Всяка дейност, в която трябва да бъдат обработвани някакви неща, пристигащи асинхронно, като в даден момент може да се обработва само едно нещо, може да се моделира с приоритетна опашка. Нещата може да са, print jobs в операционна система, кораби за разтоварване в пристанище, самолети, кацащи на летище, спешно отделение в болница, и така нататък. С приоритетни опашки може и да се сортира: HEAPSORT можем да разглеждаме като сортиращ алгоритъм с последователно избиране на максимален елемент (какъвто е SELECTION SORT), ползващ ефикасно реализирана приоритетна опашка.

4.4.3 Реализация на приоритетни опашки с двоични пирамиди

Както ще видим след малко, двоичните пирамиди са много подходящи за реализация на приоритетни опашки, но в никакъв случай не са единствената възможна реализация. Приоритетна опашка може да бъде реализирана и чрез следните структури данни.

- Сортиран масив. На читателя остава да измисли имплементация на трите функции от интерфейса. Тук само ще спомена, че MAXIMUM(S) и EXTRACT-MAX(S) при тази реализация имат сложност $\Theta(1)$, докато INSERT(S, x) има сложност $\Theta(n)$.

- Несортиран масив. И тук остава на читателя да измисли имплементация на трите функции от интерфейса. При тази реализация, MAXIMUM(S) и EXTRACT-MAX(S) имат сложност $\Theta(n)$, докато INSERT(S, x) има сложност $\Theta(1)$.

- Пирамида на Фибоначи. Това е сложна структура данни, даваща възможност за изключително ефикасна реализация на функциите от интерфейса. Нещо повече, тази структура данни позволява интерфейс от по-голямо множество функции, които включват изтриване на елемент и смесване (merge) на две пирамиди. За съжаление, реализацията на функциите от интерфейса е ефикасна само в асимптотичния смисъл. Големите скрити константи правят пирамидите на Фибоначи неефикасни на практика, освен за наистина много големи данни. За подробности, вижте [21, стр. 506].

- Двоични дървета, реализирани чрез списъци с дървовидна структура (по два указателя към децата за всеки запис). Подробна дискусия на предимствата, недостатъците и вариациите на този подход има в третия том на културата поредица на Доналд Кнут "Изкуството на Компютърното Програмиране" (The Art of Computer Programming) [56, стр. 149]. От общи съображения е ясно, че този подход изисква значително—по-точно, линейно—повече памет от реализацията с масив. Измежду неговите предимства се откроява възможността за бързо сливане (merge) на приоритетни опашки.

Трите функции на интерфейса може да бъдат реализирани чрез двоични пирамиди по следния начин. Входът S е масив $S[1, \ldots, n]$, реализиращ пирамида. S.size е размерът на пирамидата, тоест, само $S[1, \ldots, S.\text{size}]$ е пирамида. Допускаме, че винаги $S.\text{size} < n$, така че няма да се притесняваме за препълване на масива в HEAP-INSERT. Ако пирамидата е празна, както е в самото начало при създаването ѝ, $S.\text{size} = 0$.

```
HEAP-MAXIMUM(S[1, ..., n])
1  (* Допускаме, че S.size ⩾ 1 *)
2  return S[1]
```

```
HEAP-EXTRACT-MAX(S[1, ..., n])
1  (* Допускаме, че S.size ⩾ 1 *)
```

```
2  max ← S[1]
3  S[1] ← S[S.size]
4  S.size ← S.size − 1
5  HEAPIFY(S, 1)
6  return max
```

```
HEAP-INSERT(S[1, . . . , n], x)
1  S.size ← S.size + 1
2  i ← S.size
3  S[i] ← x
4  while i > 1 and S[PARENT(i)] < S[i] do
5      swap(S[PARENT(i)], S[i])
6      i ← PARENT(i)
```

Коректността на тези функции е абсолютно очевидна след изложението в Секция 4.2. Сложността на HEAP-MAXIMUM е $\Theta(1)$, а на HEAP-EXTRACT-MAX и HEAP-INSERT е $\Theta(\lg(S.size))$. Ако не ползваме "n" за големината на масива, а за големината на самата пирамида, и изразяваме сложността чрез символа "n", то двете функции имат сложност $\Theta(\lg n)$. Имайки предвид огромната, качествена разлика между нарастванията $\Theta(n)$ и $\Theta(\lg n)$, виждаме, че реализацията с двоични пирамиди е **много по-добра** от реализацията със сортирани или несортирани масиви.

Оттук правим важен извод, който е приложим в много по-широк контекст от ефикасното реализиране на приоритетните опашки. В някои случаи (реализиране на приоритетна опашка), оптималният подход (пирамида) е **компромис** (на английски, *tradeoff*) между два екстремални подхода (сортиран масив и несортиран масив). Относно две различни изисквания (бързо изваждане на максимален елемент и бързо вмъкване на нов елемен), при единият екстремален подход имаме оптимален резултат ($\Theta(1)$) по първото изискване, но изключително лош резултат ($\Theta(n)$) по второто; а при другия подход, оптимален резултат по второто изискване, но изключително лош резултат по първото. Компромисният подход (отказ от константно време за коя да е от операциите и постигане на задоволително време и за двете операции), направен интелигентно, се оказва печеливш.

4.4.4 Функцията INCREASE-KEY

Според [21], функцията от интерфейса INCREASE-KEY може да се реализира с двоична пирамида по следния начин.

```
HEAP-INCREASE-KEY(S[1, . . . , n], i, k)
1  if k < S[i]
2      error "опит да бъде намален ключът"
3  S[i] ← k
4  while i > 1 and S[PARENT(i)] < S[i] do
5      swap(S[PARENT(i)], S[i])
6      i ← PARENT(i)
```

Очевидно, тази имплементация не може да е част от интерфейса на АТД, защото в нея е заложено, че имплементацията е именно чрез масив. В случая параметърът i е индексът на

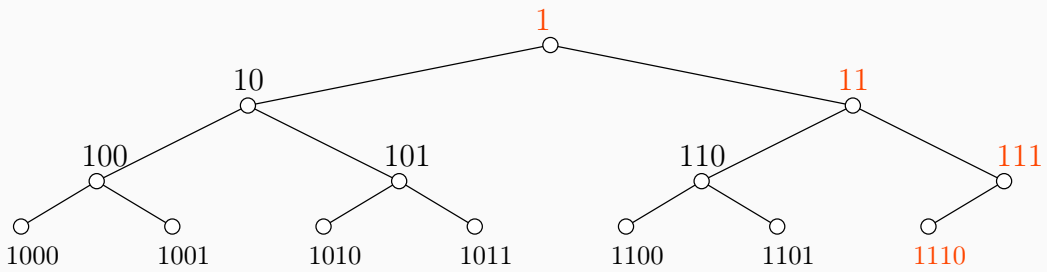
елемента, чийто ключ ще нараства. Ако имплементацията на приоритетната опашка е чрез някакви свързани списъци, този индекс няма смисъл.

Правилното викане на функцията би било INCREASE-KEY(S, x, k), където x е елемент от опашката. Но ако реализираме функцията с пирамида и дадем параметър x, който е елемент (с други думи, ако x е старата стойност на ключа, която трябва да стане k), този елемент трябва да бъде първо открит, което е операция с линейна сложност в пирамидите[†], дори ако допуснем уникални ключове. Така че, за да постигнем логаритмична сложност и на тази функция от интерфейса, се налага да "счупим" изискването за абстрактен тип данни и да заложим в нейната спецификация, че приоритетната опашка е реализирана именно чрез масив и че някак знаем индекса на елемента, чийто ключ искаме да повишим.

Допълнение 21: За броя на пирамидите на n върха

В това допълнение ще разгледаме интересния въпрос, колко от всички $n!$ пермутации на n различни числа са пирамиди, по-точно, линеаризации на пирамиди. Изложението следва изложението в [56, стр. 152].
Да разгледаме отново попълненото двоично дърво с адресите от Подсекция 4.1.2, което за удобство показваме отново на Фигура 4.18.

Фигура 4.18 : Попълнено дърво с 14 върха и техните адреси. Специалните върхове са в червено.



Дървото има 14 върха. Да запишем броевете на върховете на поддърветата, вкоренени във всички върхове на дървото, в нарастващ ред по адресите на тези върхове. Ще записваме броевете в двоична позиционна бройна система:

$$1110, 111, 110, 11, 11, 11, 10, 1, 1, 1, 1, 1, 1, 1$$

За да е напълно ясно: цялото дърво (тоест, поддървото с корен върха, чийто адрес е 1) има четиринадесет върха, което в двоична позиционна бройна система е 1110. Поддървото, вкоренено в следващия връх (с адрес 10) има седем върха, така че записваме 111. И така нататък. С червен цвят са записани броевете на върховете в тези поддървета, чиито корени лежат на пътя между корена на дървото и последния връх (чийто адрес е $\mathrm{bin}(n)$). Тези върхове са наречени в [56], *специалните върхове*, а дърветата, вкоренени в тях, *специалните поддървета*. В [56, стр. 157, упр. 20 и 21] е показано, че ако $\mathrm{bin}(n) = b_k b_{k-1} \ldots b_1 b_0$, където $k = \lfloor \lg n \rfloor$ и $b_k = 1$, то размерите[a] на специалните поддървета, записани в двоична позиционна бройна система, са

$$1b_{k-1} \ldots b_1 b_0, \quad 1b_{k-2} \ldots b_1 b_0, \quad \ldots \quad 1b_1 b_0, \quad 1b_0, \quad 1$$

[†]Примерно, най-малкият ключ може да е във всяко листо

а размерите на неспециалните поддървета числа от вида $2^m - 1$, защото те са съвършени двоични дървета, и броевете на неспециалните поддървета от всяка срещаща се големина са съответно

$$\begin{aligned}
&\left\lfloor \frac{n-1}{2} \right\rfloor \text{ числа } 1,\\
&\left\lfloor \frac{n-2}{4} \right\rfloor \text{ числа } 3,\\
&\left\lfloor \frac{n-4}{8} \right\rfloor \text{ числа } 7,\\
&\qquad \ldots\\
&\left\lfloor \frac{n-2^{k-1}}{2^k} \right\rfloor \text{ числа } 2^k - 1
\end{aligned}$$

Знаейки размерите на поддърветата, можем да изчислим H_n: броя на начините да разположим ключовете $\{1, 2, \ldots, n\}$ в пирамида, като използваме резултата от [56, стр. 67, упр. 20]:

$$H_n = \frac{n!}{s_1 \times s_2 \times \ldots \times s_n} \tag{4.2}$$

където $\{s_1, s_2, \ldots, s_n\}_M$ е мултимножеството от всички размери на поддървета (вж. Определение 40). Примерно, ако елементите са 14, каквото е дървото на Фигура 4.18, начините да сложим ключовете $\{1, 2, \ldots, 14\}$, така че дървото да е пирамида, са на брой

$$H_{14} = \frac{14!}{14 \times 6 \times 2 \times 1 \times 1^6 \times 3^3 \times 7^1} = 2\,745\,600$$

Сравнете H_{14} с $14! = 87\,178\,291\,200$.

Редицата H_n за $n \geqslant 1$ е редица `A056971` в онлайн енциклопедията на целочислените редици. Точният израз (4.2) не дава представа за асимптотиката на H_n. Асимптотиката на H_n се разглежда в [41]. Основният резултат е твърде сложен, но грубо приближение е следното. Ако $h_n = \log\left(\frac{n!}{H_n}\right)$, то $h_n \asymp n$. Оттук—това е грубо приближение—асимптотиката на H_n е приблизително $\frac{n!}{c^n}$ за някаква константна c.

[a] Това са общо $k = \lfloor \lg n \rfloor$ числа, колкото е височината на попълненото двоично дърво с n върха съгласно Лема 23.

Лекция 5

Рекурсивни Алгоритми, алгоритмична схема Разделяй-и-Владей и Рекурентни Уравнения.

Резюме: Въвеждаме понятията рекурсивен алгоритъм и рекурентно уравнение. Въвеждаме алгоритмичната схема Разделяй-и-Владей. Разискваме най-обща класификация на рекурентните уравнения. Разглеждаме шест начина за решаване на рекурентни уравнения: чрез развиване, чрез дървото на рекурсията, по индукция, чрез Мастър теоремата, чрез теоремата на Akra-Bazzi и чрез характеристични уравнения. Разглеждаме прости примери за рекурсивни алгоритми, конструирани по схемата Разделяй-и-Владей.

5.1 Рекурсивни алгоритми и Разделяй-и-Владей

Думата *рекурсия* идва от латинския глагол *curro*[†], което означава "тичам", и префикса *re-*, който означава "назад" или "отново". *Recurro* означава "тичам назад", откъдето *рекурсивен* буквално означава "тичащ назад".

Рекурсивен алгоритъм е алгоритъм, който вика себе си. Очевидно, ако детерминистичен алгоритъм извика себе си с точно същия вход, с който е бил извикан, той ще неизбежно ще вика себе си отново и отново със същия вход до безкрай, без да терминира никога. Поради това викане на себе си със същия вход е недопустимо.

Въвеждаме понятието *алгоритмична схема*. Това означава означава общ шаблон, по който са конструирани много алгоритми за много задачи. В този курс ще разгледаме три алгоритмични схеми: **Разделяй-и-Владей**, **Алчна схема** и **Динамично Програмиране**. Известни са и други схеми, например **Пълно Изчерпване** (на английски **Exhaustive Search**) и **Търсене с Връщане** (на английски **Backtracking**), които няма да разглеждаме в курса.

В тази лекция ще разгледаме схемата **Разделяй-и-Владей**, тъй като често рекурсивните алгоритми са изградени по нея. При нея алгоритъмът се състои от три *фази*:

- **Разделяй** – ако входът е достатъчно голям, то той се разделя[‡] на части и се преминава към следващата стъпка **Владей**. В противен случай, тоест ако входът е по-малък

[†]Откъдето идва, например, и думата "куриер".

[‡]Терминът "разбива" не е подходящ, защото общоприетата дефиниция на "разбива" означава "на части, които не се припокриват". При алгоритмите, изградени по схемата **Разделяй-и-Владей**, не е задължително частите на входа да не се припокриват, просто всяка трябва да е по-малка от целия вход.

от някаква предварително зададена големина, задачата върху него се решава по някакъв тривиален начин или с "брутална сила" и се преминава директно към стъпка **Комбинирай**.

- **Владей** – решаваме задачата върху всяка от по-малките части и получаваме резултатите от всяко решение.
- **Комбинирай** – използваме тези резултати, за да генерираме решение за задачата върху първоначалния вход.

Да повторим: първата фаза (на разделянето) става само докато размерът на входа не стане достатъчно малък, защото няма как да разделяме дискретен вход на по-малки части неограничено. Разделянето се случва ако и само ако размерът на входът надхвърля еди-колко; в противен случай, генерираме резултат по друг начин, например с пълно изчерпване на възможностите, който метод наричаме "с брутална сила"[†].

Както ще видим в многобройни алгоритмични примери, някои изчислителни задачи са податливи на ефикасно алгоритмично решение, което първо разделя входа на части, стига той да е достатъчно голям, решава задачата върху всяка от частите, и накрая комбинира решенията. За съжаление, този подход съвсем не е универсален. За много задачи—по правило, за всички много тежки изчислителни задачи—схемата **Разделяй-и-Владей** не е приложима[‡].

Типичен пример за рекурсивен алгоритъм е следният алгоритъм, пресмятащ факториел на число.

```
FACTORIAL(n ∈ ℕ)
1  if n = 0
2      return 1
3  else
4      return n × FACTORIAL(n − 1)
```

Да разгледаме работата му върху някакъв малък вход, например 5.

- FACTORIAL(5) проверява условието на ред 1 и понеже то е ЛЪЖА, на ред 4 изпълнява FACTORIAL(4).
- FACTORIAL(4) проверява условието на ред 1 и понеже то е ЛЪЖА, на ред 4 изпълнява FACTORIAL(3).
- FACTORIAL(3) проверява условието на ред 1 и понеже то е ЛЪЖА, на ред 4 изпълнява FACTORIAL(2).
- FACTORIAL(2) проверява условието на ред 1 и понеже то е ЛЪЖА, на ред 4 изпълнява FACTORIAL(1).

[†]Терминът "Разделяй-и-Владей" (на английски, *Divide and Conquer*) идва от латинската фраза *Divide et impera*. Според наложилия се фолклор, тази максима е била ръководен принцип на древния Рим—и кралството, и републиката, и империята—който се е справял с даден силен противник, като първо е създавал вътрешни разделения в него, а после е побеждавал всеки от получените фрагменти последователно. Легендарната битка между братята Хорации и братята Куриации е спечелена именно с прилагане на разделяй-и-владей.

[‡]Както ще видим в следваща лекция, алгоритмичната схема **Динамично Програмиране** също е свързана с рекурсия, но при онези алгоритми акцентът е върху ефикасното пресмятане, което означава умело използване на вече получени резултати на общи подподзадачи.

- Factorial(1) проверява условието на ред 1 и понеже то е Лъжа, на ред 4 изпълнява Factorial(0).
- Factorial(0) проверява условието на ред 1 и понеже то е Истина, на ред 2 връща стойност 1 на Factorial(1).
- Factorial(1) получава стойност 1 от викането на Factorial(0), след което умножава 1×1 и връща резултата, който е 1, на Factorial(2).
- Factorial(2) получава стойност 1 от викането на Factorial(1), след което умножава 2×1 и връща резултата, който е 2, на Factorial(3).
- Factorial(3) получава стойност 2 от викането на Factorial(2), след което умножава 3×2 и връща резултата, който е 6, на Factorial(4).
- Factorial(4) получава стойност 6 от викането на Factorial(3), след което умножава 4×6 и връща резултата, който е 24, на Factorial(5).
- Factorial(5) получава стойност 24 от викането на Factorial(4), след което умножава 5×4 и връща 120.

Работата на Factorial(5) се състои от два етапа. Първият етап можем да наречем *движение надолу*. През него входът намалява, докато не бъде достигната стойността 0, определена от ред 1. Тази стойност можем да наречем *спирачката на рекурсията*. Вторият етап да наречем, *движение нагоре*. Същинското изчисляване на факториела става във втория етап. По отношение на софтуерна имплементация на алгоритъма, първият етап е само попълване на подходяща структура данни, а именно стек, с подходящи стойности. Името "рекурсия", което, както казахме, буквално означава "тичане назад", без съмнение се дължи на втория етап – движението нагоре.

5.2 Рекурентни уравнения

5.2.1 Определение на рекурентно уравнение

Рекурентно уравнение е уравнение, в което вляво от знака за равенство има символ за функция на някаква променлива-естествено число, например "$\mathsf{T}(\mathfrak{n})$" или "$\mathfrak{a}_\mathfrak{k}$", а вдясно има валиден израз[†], който съдържа поне една поява на същия символ за функция, но с по-малки стойности на променливата, например "$\mathfrak{a}_{\mathfrak{k}-1}$" или "$\mathsf{T}\left(\frac{\mathfrak{n}}{2}\right)$". Алтернативна дефиниция е: уравнение, което изразява общия член на някаква редица от числа чрез предишни членове.

За една или повече *начални стойности* на променливата, има други уравнения, наречени *начални условия*, които определят въпросната функция върху началните стойности.

Можем да мислим за рекурентните уравнения като за рекурсивни алгоритми. Наистина, всяко рекурентно уравнение е рекурсивен алгоритъм, който има вход-число и изход-число и вика себе си върху достатъчно големите стойности на входа. Типичен пример е рекурентното

[†]Синтактичните правила за това, кои изрази вдясно са валидни, остават неназовани. Първо, тези правила не са универсални, и второ, би било прекален педантизъм да ги излагаме строго формално като граматики. Това само би попречило на изложението.

уравнение на числата на Fibonacci:

$$\begin{aligned} \mathsf{F}_0 &= 0 \\ \mathsf{F}_1 &= 1 \\ \mathsf{F}_n &= \mathsf{F}_{n-1} + \mathsf{F}_{n-2} \quad \text{за } n \geqslant 2 \end{aligned} \tag{5.1}$$

По правило, рекурентните уравнения са твърде елементарни алгоритми, защото нямат цикли и вложеност на `if`-овете. Важно е да различаваме алгоритъма-рекурентно уравнение от другия алгоритъм – онзи, чиято сложност се описва от уравнението (вижте Наблюдение 16).

Добре дефинирано рекурентно уравнение ще наричаме рекурентно уравнение, което задава коректен (рекурсивен) алгоритъм, който алгоритъм пресмята елементи на някаква безкрайната числова редица, да кажем $a_0, a_1, a_2, \ldots$. На читателя остава да съобрази, че следното уравнение **не е добре дефинирано** заради "недостиг" на начални условия:

$$\begin{aligned} \mathsf{T}(0) &= 1 \\ \mathsf{T}(n) &= 2\mathsf{T}(n-3) \quad \text{за } n \geqslant 1 \end{aligned}$$

Въпросът дали дадено рекурентно уравнение е добре дефинирано, в частност дали началните условия са достатъчни и смислени, може да е нетривиален. Следното рекурентно уравнение не е добре дефинирано

$$\begin{aligned} \mathsf{T}(1) &= 1 \\ \mathsf{T}(n) &= 2\mathsf{T}\left(\left\lfloor \frac{n}{2} + 17 \right\rfloor\right) + n \quad \text{за } n > 1 \end{aligned}$$

поради това, че трансформацията $n \mapsto \left\lfloor \frac{n}{2} + 17 \right\rfloor$[†], започвайки от произволно естествено число n, се "установява" на $n = 33$ или $n = 34$,[‡] така че началната стойност 1 не е достижима от никое число освен самото 1. С други думи, началното условие $\mathsf{T}(1) = 1$ в случая е безполезно! Съответната програма ще "гърми" за всеки вход, различен от 1[§].

И така, всяко добре дефинирано рекурентно уравнение определя еднозначно числова редица. Отношението между рекурентни уравнения и числови редици **не е биективно**: за една и съща числова редица има безброй много рекурентни уравнения, такива че тази редица е семантиката на всяко от тези уравнения. Рекурентни уравнения, които определят една и съща числова редица, ще наричаме *еквивалентни*. Убедете се сами, че рекурентното уравнение (5.2) е еквивалентно на (5.1), защото и на двете съответства редицата на Фибоначи 0, 1, 1, 2, 3, 5, 8, ...

$$\begin{aligned} \mathsf{T}(0) &= 0 \\ \mathsf{T}(1) &= 1 \\ \mathsf{T}(2) &= 1 \\ \mathsf{T}(n) &= 2\mathsf{T}(n-1) - \mathsf{T}(n-3) \qquad \text{за } n \geqslant 3 \end{aligned} \tag{5.2}$$

[†] Чете се, "n се изобразява в $\left\lfloor \frac{n}{2} + 17 \right\rfloor$".

[‡] Казваме, че 33 и 34 е *фиксираните точки* на тази трансформация.

[§] Става дума за препълване на стека, който операционната система осигурява на съответния процес. Опитайте следния фрагмент на `C`:

```
int recfun(int n) {if (n == 1) return 1; else return 2*recfun(floor(n/(2.0)) + 17) + n;}
```

върху $n = 1$ и $n > 1$.

Забележка

Подчертаваме, че не трябва да се бъркат двете понятия

- рекурентно уравнение, което е понятие от синтактичното ниво
- функцията, която се реализира от някакво рекурентно уравнение – това е понятие от семантичното ниво. Числовата редица, която съответства на рекурентното уравнение, е практически същото нещо като функцията, която то реализира.

5.2.2 Обобщения на "рекурентно уравнение"

Има различни обобщения на понятието "рекурентно уравнение", което току-що въведохме. Рекурентните уравнения могат да бъдат на повече от една променлива, например биномният коефициент може да се дефинира чрез рекурентното уравнение на две променливи

$$\binom{n}{k} = \begin{cases} 0, & \text{ако } k > n \\ 1, & \text{ако } k = 0 \text{ или } k = n \\ \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}, & \text{в противен случай} \end{cases}$$

или числото на Stirling от втори род може да се дефинира чрез рекурентното уравнение на две променливи

$$\left\{ {n \atop k} \right\} = \begin{cases} 0, & \text{ако } k > n \\ 0, & \text{ако } k = 0 \text{ и } n > 0 \\ 1, & \text{ако } k = n \\ \left\{ {n \atop k} \right\} = k \cdot \left\{ {n-1 \atop k} \right\} + \left\{ {n-1 \atop k-1} \right\}, & \text{в противен случай} \end{cases}$$

Разбира се, когато променливите са повече от една, няма смисъл да казваме, че рекурентното уравнение задава числова редица. Ако променливите са две, то уравнението задава двумерен безкраен обект; например, уравнението на биномния коефициент задава триъгълника на Pascal.

Може да бъдат дадени няколко взаимно зависими рекурентни уравнения – това е проявление на *взаимна рекурсия*. Пример за такива рекурентни уравнения има в [58, стр. 9] (за краткост изпускаме началните условия):

$$\begin{aligned} T(n) &= T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + 2T_1\left(\left\lceil \frac{n}{2} \right\rceil - 1\right) + 2T_1\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2 \\ T_1(n) &= T\left(\left\lfloor \frac{n}{2} + 1 \right\rfloor\right) + T_1\left(\left\lceil \frac{n}{2} \right\rceil\right) + 2T_1\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2T_2\left(\left\lceil \frac{n}{2} \right\rceil - 1\right) + 2 \\ T_2(n) &= T_1\left(\left\lceil \frac{n}{2} \right\rceil\right) + T_1\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + 2T_2\left(\left\lceil \frac{n}{2} \right\rceil - 1\right) + 2T_2\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2 \end{aligned}$$

Друг пример за взаимна рекурентност има на тази страница на специализирано списания за числа на Фибоначи, където е дадена задачата, да се докаже, че H_{n-1} и G_n са последователни

числа на Фибоначи $\forall n \geqslant 1$, ако F_n са числата на Фибоначи:

$$\begin{aligned}
G_1 &= 1 \\
H_0 &= 0 \\
G_n &= F_{n+1}G_{n-1} + F_nH_{n-2}, \quad n \geqslant 2 \\
H_n &= F_{n+1}G_n + F_nH_{n-1}, \quad n \geqslant 1
\end{aligned}$$

5.2.3 Рекурентни уравнения и рекурсивни алгоритми

Връзката между рекурентните уравнения и рекурсивните алгоритми е трикова. От една страна, всяко рекурентно уравнение **е** рекурсивен алгоритъм, който за всеки достатъчно голям вход-число връща изход-число след пресмятане, което включва рекурсия (викане на себе си върху по-малко число). От друга страна, сложността на рекурсивните алгоритми по правило се описва от рекурентни уравнения. Както вече казахме, сложността на MERGESORT (Секция 6.1) се описва от (6.1).

При изследването на сложността на рекурсивни алгоритми чрез рекурентни уравнения е важно да се прави разлика между алгоритъма, чиято сложност изследваме, и рекурентното уравнение, което описва тази сложност. На свой ред то също е рекурсивен алгоритъм, но е **съвършено различен обект** от първоначалния алгоритъм. Като пример да разгледаме следното рекурентното уравнение, където $n \in \mathbb{N}$:

$$T(n) = \begin{cases} 1, & \text{ако } n = 0 \\ nT(n-1) + 1, & \text{в противен случай} \end{cases} \tag{5.3}$$

То е много близко до алгоритъм FACTORIAL на стр. 189. Сега да разгледаме следния алгоритъм. Той не прави нищо полезно, но ще ни послужи за илюстрация.

```
AlgX(A[1,...,n])
 1  if n = 0
 2      return 1
 3  else
 4      s ← A[1]
 5      for i ← 1 to n
 6          s = s + AlgX(A[1,...,n − 1])
 7      return s
```

Сложността на AlgX се описва от (5.3). Обаче (5.3) е **съвършено различен обект** от AlgX. Ако $n = 100$, то никоя практическа имплементация AlgX$(A[1,\ldots,n])$ няма приключи работата си **никога** (вижте Допълнение 12), докато $T(100)$ ще бъде пресметнато **мигновено** върху съвременен компютър[†]. Заслужава си да обособим това наблюдение по следния начин.

[†]Стига софтуерът да може да поддържа работа с цели числа с произволна точност (на английски, *arbitrary-precision arithmetic*), за да не се стигне до препълване на променливите заради огромните числа-факториели.

Наблюдение 16

Ако е даден рекурсивен алгоритъм AlgX, чиято сложност по време се описва от рекурентно уравнение Y, то Y също е рекурсивен алгоритъм. Обаче AlgX и Y са съвършено различни обекти. Най-малкото, входът на AlgX може да не е число, а масив или друг обект, докато входът на Y—това е променливата на уравнението—е число, имащо смисъла на големина на входа на AlgX. Освен това, по правило AlgX върху вход с големина n работи несравнимо по-бавно от Y с вход числото n.

5.2.4 Класификации на рекурентните уравнения

Тук ще игнорираме началните условия и ще дискутираме "същинското" рекурентно уравнение. Рекурентните уравнения може да се класифицират по много признаци.

- **Брой на появите на функционалния символ вдясно.** Функционалният символ може да се появява точно един път вдясно, например

$$T(n) = 2T(n-1) + 1 \tag{5.4}$$

а може да се появява много пъти с различни стойности на аргумента си, например

$$T(n) = T(n-1) + T(n-3) + T(n-5) \tag{5.5}$$

или дори

$$T(n) = T(n-1) + T(n-2) + \cdots + T(1) + T(0), \tag{5.6}$$

което се нарича *рекурентно уравнение с пълна история*.

Наблюдение 17

В следното рекурентно уравнение, появите вдясно са една, а не две:

$$T(n) = T(n-1) + T(n-1) + 1$$

Тривиално е да се съобрази, че това рекурентно уравнение всъщност е (5.4). Броят на появите вдясно, за който говорим, е по отношение на **различни** аргументи – както е, например, в (5.5).

Класификацията по брой появи вдясно не е от особена важност, но обикновено рекурентните уравнения с една поява вдясно се решават по-лесно в сравнение с тези с няколко появи. Примерно, Мастър теоремата (Подсекция 5.3.4) е приложима само за рекурентни уравнения с една поява вдясно.

- **Каква е функцията на намаляването** За всяка поява на функционалния символ вдясно, *функцията на намаляването*, неформално казано, е тази математическа операция, която е записана там. Има неограничено много възможности за това.

  - Функцията на намаляването е изваждане на константа като, например, в (5.3), в (5.4) и в (5.5).

- Функцията на намаляването може да е изваждане, но не на константа. Примерно, $T(n) = T(n - \lfloor \lg n \rfloor)$. Екстремен вариант е уравнение с пълна история като (5.6).
- Функцията на намаляването може да е деление с константа, например

$$T(n) = 2T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + 4T\left(\left\lfloor \frac{n}{7} \right\rfloor\right) + 1$$

- Може да е деление, но с нарастваща функция, например

$$T(n) = 2T\left(\left\lfloor \frac{n}{\lg n} \right\rfloor\right) + 1$$

- Може да е коренуване

$$T(n) = nT(\lfloor \sqrt{n} \rfloor) + 1$$

- Може да е логаритмуване

$$T(n) = T(\lfloor \lg n \rfloor) + 1$$

- Може да е "смесен" вид намаляване, например:

$$T(n) = T(n-2) + T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + 3T(\lfloor \sqrt{n} \rfloor) + 50T(\lfloor \lg n \rfloor) - 10$$

Възможностите наистина са неограничени.

При изследването на сложността на естествени алгоритми, смесено намаляване не се среща, а функцията на намаляването е или изваждане на константа, или деление с константа. При алгоритмите, изградени по схемата **Разделяй-и-Владей**, сложността се описва с рекурентни уравнения, при които функцията на намаляването е деление с константа. Намаляване чрез изваждане се среща често (макар и не само там) в рекурентните уравнения, описващи сложността на алгоритми, изградени по схемата **Пълно изчерпване**.

Следните определения и класификации са в сила **само за рекурентни уравнения, при които функцията на намаляването е изваждане на константа**. Тези уравнения се наричат още *диференчни уравнения*, на английски *difference equations*. Задълбочено изложение за диференчните уравнения има в книгата "An Introduction to Difference Equations" на Elaydi [23].

- *Ред* (на английски, *order*) на рекурентното уравнение е най-голямото число, което се изважда от n вдясно. В следните три примера, числото, което определя реда на уравнението, е в червено. И така, това рекурентно уравнение

$$T(n) = 2T(n - 1) + 1$$

е от първи ред, това

$$T(n) = 3T(n - 1) + 4T(n - 2) + n$$

е от втори ред, а това

$$T(n) = T(n-1) + T(n-2) + \cdots + T(n-k) + n^2$$

е от k-ти ред. Понякога се казва *рекурентно уравнение с крайна история*, за да се подчертае, че k е константа, а не, да речем, $\lfloor \lg n \rfloor$.

Наблюдение 18: В някакъв смисъл, намаляването винаги е изваждане

Ако допуснем k да не е константа, а някаква нарастваща функция на n, то **за всяко** рекурентно уравнение може да твърдим, че функцията на намаляването е изваждане. Например, $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1$ може да се запише като $T(n) = T\left(n - \left\lceil \frac{n}{2} \right\rceil\right) + 1$; $S(n) = S(\lfloor \sqrt{n} \rfloor) + 1$ може да се запише като $S(n) = S(n - (n - \lfloor \sqrt{n} \rfloor)) + 1$, и така нататък. От това съображение следва, че класифицирането на рекурентните уравнения по функцията на намаляването е—говорейки теоретично—безсмислено, защото, в някакъв смисъл, тя винаги е изваждане, ако променливата е целочислена. Но на практика това класифициране е доста удобно.

Подчертаваме, че редът на рекурентното уравнение **не е** същото като броят на появите на функционалния символ вдясно. Например, следното рекурентно уравнение

$$T(n) = T(n-2) + 2T(n-5) + 1$$

е от пети ред, мака да има само две появи вдясно. За да се убедим, че е така, да мислим за уравнението като

$$T(n) = 0T(n-1) + 1T(n-2) + 0T(n-3) + 0T(n-4) + 2T(n-5) + 1$$

- *Линейно рекурентно уравнение* е рекурентно уравнение, в което дясната страна е линейна функция на функцията на уравнението. Например, следните три рекурентни уравнения са линейни:

$$A(n) = A(n-1) + 1$$
$$B(n) = nB(n-2) + n^2B(n-4) + \left\lfloor \frac{n \sin n}{\sqrt{n^3 + \lg n}} \right\rfloor$$
$$C(n) = \sum_{k=0}^{n-1} 2^k C(k)$$

докато тези четири **не са** линейни:

$$A(n) = A(n-1)A(n-2) + 1$$
$$B(n) = \left\lceil \frac{1}{1 + B(n-1)} \right\rceil$$
$$C(n) = \lfloor \sqrt{C(n-2)} \rfloor$$
$$D(n) = \lfloor \lg (D(n-1) + D(n-2)) \rfloor$$

- Различаваме рекурентни уравнения с константни коефициенти, например

$$P(n) = 1P(n-1)P(n-2) + 3P(n-3)$$
$$S(n) = \left\lfloor \sum_{k=1}^{\infty} \frac{1}{k^2} \right\rfloor S(n-2) + n$$
$$T(n) = 2T(n-1) + 5T(n-3)$$

  от рекурентни уравнения с променливи коефициенти, например

$$P(n) = \lfloor \lg n \rfloor P(n-1)$$
$$S(n) = nS(n-1)$$
$$T(n) = \left\lceil \frac{2+n^2}{3+n} \right\rceil T(n-2)$$

- Различаваме *хомогенни* и *нехомогенни* линейни рекурентни уравнения. При хомогенните, дясната страна е сума от събираеми, всяко от които е от един и същи вид, като всяко от тях задължително съдържа функцията от лявата страна. Примерно,

$$T(n) = c_1T(n-1) + c_2T(n-2) + \cdots + c_kT(n-k) \tag{5.7}$$

  където k и $c_1, \ldots, c_k$ са константи. При нехомогенните има събираеми, които са от съвсем друг вид, които не съдържат функцията от лявата страна. Примерно,

$$T(n) = c_1T(n-1) + c_2T(n-2) + \cdots + c_kT(n-k) + f(n) \tag{5.8}$$

  където k и $c_1, \ldots, c_k$ са константи, където $f(n)$ е функция, която зависи от n и не зависи от $T(n)$. Казваме, че $f(n)$ е *нехомогенната част*.

- Различаваме *дискретни* от *континуални* рекурентни уравнения (Подсекция 5.2.5).

5.2.5 Винаги ли променливата в рекурентно уравнение е целочислена

Това, че функцията-семантика на уравнението може да има стойности, които не са цели числа, е доста очевидно. Например рекурентното уравнение (вж. [83, стр. 56])

$$a_0 = 1$$
$$a_n = \frac{1}{1 + a_{n-1}}$$

определя редицата от дроби $1, \frac{1}{2}, \frac{2}{3}, \frac{3}{5}, \frac{5}{8}, \ldots$, която ясно връзка с числата на Fibonacci. Но променливата n тук е отново целочислена и пак става дума за числова редица.

Ние по правило приемаме, че променливата е целочислена, поради което я записваме с "n" или "k". Ако уравнението описва сложност на алгоритъм, това има смисъл, защото размерите на входа са естествени числа. Дали нулата е възможен размер на вход за конкретната ни задача не е съществено; ако ли не, то размерите на входа са цели положителни числа и толкова. Съществените е, че размерите на входа не са дробни, ирационални или отрицателни.

Ако обаче гледаме на уравнението като на самостоятелен обект, нищо не пречи променливата да взема "истински" реални стойности, стига да има някаква долна граница върху

тях. Като пример, в книгата на Flajolet и Sedgewick се посочва (вж. [83, стр. 46]), че методът на Newton за пресмятане на квадратен корен от дадено положително реално β е рекурентно уравнение с $a_0 = 1$:

$$a_n = \frac{1}{2}\left(a_{n-1} + \frac{\beta}{a_{n-1}}\right), \quad n > 0$$

Забележете, че и уравнението $F_n = F_{n-1} + F_{n-2}$ (сравнете с (5.1)) не е безсмислено при реално n, стига началните условия да се предефинират така, че да не могат да бъдат прескочени, от което и достатъчно голямо n да стартира процеса на изчислението назад. Една възможност е началните условия да са върху полуотворения интервал $[0, 2)$, например

$$F_n = \begin{cases} n, & \text{ако } n \in [0, 2) \\ F_{n-1} + F_{n-2}, & \text{ако } n > 2 \end{cases} \tag{5.9}$$

Математически погледнато, този алгоритъм е безукорно ефективен. Нещо повече, тъй като реалната променлива намалява с вадене на цели числа, алгоритъмът на (5.9) може да бъде имплементиран ефикасно чрез Динамично Програмиране, подобно на изчислението на числа на Fibonacci в Подсекция 13.1.2. При други видове намалявания вдясно, примерно деление на константа, идеята на динамичното програмиране не е приложима и освен това съществува опасност от загуба на точност при практическата имплементация, ако работим с числа с плаваща запетая, но, математически погледнато, алгоритъмът винаги е безукорно ефективен, ако интервалът-начално условие е удачно подбран.

Наблюдение 19

Ако променливата в рекурентно уравнение е реално число, нямаме право да казваме, че рекурентното уравнение задава числова редица; в такъв случай то задава континуум от стойности.

Защо изобщо разглеждаме рекурентни уравнения върху реални променливи, при положение, че (в тези лекционни записки) се интересуваме използването на рекурентни уравнения само за изследване на сложността на алгоритми, чиито входове са с целочислени размери? Причината е, че функцията на намаляването не е задължително изваждане на цялочислена константа. Ако функцията на намаляването е деление, дори да е с цяло число, имаме две възможности.

- Да настояваме променливата да е задължително цяло число, което налага да ползваме точна долна или точна горна граница върху резултата от делението. Примерно, рекурентното уравнение

$$T(n) = 2T\left(\frac{n}{2}\right) + n \tag{5.10}$$

описващо сложността по време на MERGESORT (Секция 6.1), може да бъде написано по-прецизно като

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + n \tag{5.11}$$

Това е смислено, защото масив с нечетна големина не може да бъде разделен на две равни части; най-доброто, на което можем да се надяваме, е едната от тези части да

е с не повече от единица по-малка от другата, което функциите $\lfloor\,\rfloor$ и $\lceil\,\rceil$ ни гарантират. Но този подход е тромав и на практика, ако $\lfloor\,\rfloor$ и $\lceil\,\rceil$ се срещат в рекурентно уравнение, което искаме да решим, първата ни работа е да се освободим от тези функции, апроксимирайки решението с друго, подходящо рекурентно уравнение без тях.

- Да допуснем, че променливата може да е дробно число. Това налага началните условия да са върху цял интервал, а не само върху няколко дискретни точки, но това е без значение. Ние не посочваме началните условия така или иначе, защото, за целите на анализа на алгоритъма, се интересуваме само от асимптотиката на решението на рекурентното уравнение. Добре известно е, че за рекурентни уравнения, които типично се появяват при изследването на сложността по време на рекурсивни алгоритми, асимптотиката на решението **не се променя** от това, че от уравнение с целочислена променлива и функции $\lfloor\,\rfloor$ и $\lceil\,\rceil$ върху резултатите от деленията минаваме към уравнение без $\lfloor\,\rfloor$ и $\lceil\,\rceil$, но с променлива, която не е непременно цяла. Говорейки за MERGESORT, уравнение (5.10) има решение с асимптотика $\Theta(n \lg n)$, също като решението на (5.11).

Според [59], при първата възможност говорим за *discrete recurrences*, а при втората възможност, за *continuous recurrences*.

Забележе, че при континуалните рекурентни уравнения е безсмислено да говорим за пълна история (вж. (5.6)).

5.2.6 Решаване на рекурентни уравнения

Нека е дадено някакво рекурентно уравнение, в което функционалният символ е T, а името на променливата е n, тоест вляво от знака за равенство има "$T(n)$". Както стана ясно, то реализира някаква функция; иначе казано, определя някаква числова редица. *Да решим* това рекурентно уравнение означава да намерим друг, еквивалентен израз (формула) за същата функция, който обаче е *затворена формула*. Например, ако рекурентното уравнение е:

$$\begin{aligned} T(0) &= 0 \\ T(n) &= T(n-1) + n \quad \text{за } n > 0 \end{aligned} \tag{5.12}$$

то *решение* е $T(n) = \frac{n(n+1)}{2}$ (вижте Задача 4). Какво точно е затворена формула зависи от това, какви операции са разрешени. Решението "$\frac{n(n+1)}{2}$" е валидно, ако са разрешени операциите събиране с единица, умножение и деление на две.

Да разгледаме друг пример:

$$\begin{aligned} T(1) &= 1 \\ T(n) &= nT(n-1) \text{ за } n > 1. \end{aligned} \tag{5.13}$$

Както е добре известно, решението е $T(n) = n!$, но това решение е валидно само ако можем да ползваме факториела. Ако обаче са разрешени само събиране, изваждане, умножение, деление, коренуване и степенуване, то уравнението (5.13), колкото и да е странно, няма решение. При изброените ограничения, $T(n) = \prod_{i=1}^{n} i$ или $T(n) = n \cdot (n-1) \cdots 2 \cdot 1$ не са валидни решения, защото не са затворени формули. Забележете, че тези два израза използват нотации, касаещи умножение, но не са затворени формули, ако символът "$\prod$" или многоточието не са разрешени.

Тук ще допускаме, че всички "нормални" функции като изброените, плюс логаритмуването и факториелът, са допустими и че решение, което ги ползва, е валидно. Също така, ре-

шението може да съдържа биномни коефициенти, числа на Стирлинг от втори род, суми и произведения. Важното е да няма рекурсия – функционалният символ не трябва да се среща вдясно.

Ползите от това да решим дадено рекурентно уравнение са поне две.

- По правило решението е по-бърз алгоритъм за изчисляване на функцията, която съответства на рекурентното уравнение, отколкото алгоритъмът, който е зададен от рекурентното уравнение. Например, $\frac{n(n+1)}{2}$ е по-бърз алгоритъм за решаване на функцията, която рекурентното уравнение (5.12) реализира, в сравнение с алгоритъма, зададен от самото (5.12). Ако $n = 1\,000\,000\,000$, директното изпълнение на (5.12) ще се извърши в около $2\,000\,000\,000$ стъпки[†], докато $\frac{1\,000\,000\,000(1\,000\,000\,000+1)}{2}$ се изчислява с една инкрементация, едно умножение и едно деление на две.

  Има обаче и изключение. Да разгледаме (5.13). Дори $T(n) = n!$ да е валидно решение, алгоритъмът, който то задава, не е съществено по-бърз от рекурсивния алгоритъм на $T(1) = 1, T(n) = nT(n-1)$ за $n > 1$. Така че в този случай е спорно дали решението ни дава (съществено) по-бърз алгоритъм.

  Друго потенциално изключение е добре известното решение

$$F_n = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^n \tag{5.14}$$

  на рекурентното уравнение на числата на Fibonacci (5.1). При нереалистичното допускане, че повдигането на n-та степен става за единица време, наистина (5.14) задава алгоритъм със сложност по време $\Theta(1)$. Обаче при по-реалистичен изчислителен модел, който отчита наличието на ирационалните "$\sqrt{5}$" в израза, повдигането на n-та степен е алгоритъм със сложност по време $\Theta(n)$, тъй като всяко двете от събираеми $\frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n$ и $\frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^n$ има на свой ред $\Theta(n)$ събираеми, които трябва да присъстват в явен вид, преди да се извършат канцелиранията, от които появите на $\sqrt{5}$ изчезват, за да остане само цяло число. Забележете, че ако апроксимираме $\sqrt{5}$ с рационално число, ще загубим точност, колкото и добра да е апроксимацията, освен ако нямаме някакви математически гаранции за не-загуба на точност.

- Ако решението е достатъчно просто, много бързо можем да преценим каква е асимптотиката на нарастването на $T(n)$. Например, в (5.12) имаме $T(n) = \Theta(n^2)$, което се вижда веднага от решението $T(n) = \frac{n(n+1)}{2}$.

  По отношение на числата на Фибоначи, от (5.1) е трудно да се прецени каква е асимптотиката на F_n, докато от (5.14) веднага се вижда, че $F_n \asymp \left(\frac{1+\sqrt{5}}{2}\right)^n$, тоест, горе-долу, $F_n \approx 1.6^n$, понеже $\left(\frac{1-\sqrt{5}}{2}\right)$ е по-малко от единица по абсолютна стойност.

Методите за решаване на рекурентни уравнение ще разискваме в детайли в Секция 5.3. Тук само ще спомена, че универсален **формален** метод за решаване на рекурентни уравнения **няма**.

[†]Един милиард за движението надолу до достигане на началното условие и още един милиард за движението нагоре, от началното условие до $T(1\,000\,000\,000)$.

5.2.7 Рекурентни уравнения в изследването на алгоритми

Рекурентните уравнения се използват широко в теорията на алгоритмите, защото сложността на даден рекурсивен алгоритъм се описва с подходящо рекурентно уравнение. Правилото за съставяне на подходящо рекурентно уравнение е просто: лявата страна е, да речем, $\mathsf{T}(n)$ и изразява работата на алгоритъма върху вход с размер n, а дясната страна има по едно T за всяко рекурсивно викане, като аргументът е функцията, с която намалява входа при даденото рекурсивно викане, плюс израз, който отразява работата на алгоритъма извън рекурсивните викания. Ще видим достатъчно примери за конструиране на рекурентни уравнения, които описват сложността на рекурсивни алгоритми.

При изследването на алгоритмите не се интересуваме от точен израз за сложността – както се каза в минала лекция, това не е нито възможно, нито е необходимо. Интересува ни асимптотиката. Съответно няма да се опитваме да решаваме рекурентните уравнения точно, а само ще търсим асимптотиката на решението. Основно наблюдение, което оставяме без доказателство е, че асимптотиката на решението не зависи от началните условия. Разбира се, има примери за обратното – асимптотиката на

$$\mathsf{T}(n) = \mathsf{T}(n-1)^{\mathsf{T}(n-2)}$$

зависи от началните условия. Но никой естествен алгоритъм няма сложност по време, която се описва от това уравнение. Алгоритмите, които ще разглеждаме в този курс, са такива, че съответните им рекурентни уравнения **не зависят** от началните стойности в смисъл, че асимптотиката е една и съща, каквито и да са началните стойности. Поради това отсега нататък ще изпускаме началните условия от описанията на рекурентните уравнения. Под "рекурентно уравнение" ще разбираме само "същината", игнорирайки началните условия.

Тъй като се интересуваме само от асимптотиката на решението, при дискретни рекурентни уравнения, чиято дясна страна съдържа функциите $\lfloor\ \rfloor$ и $\lceil\ \rceil$, ще игнорираме тези функции, по този начин преминавайки към континуални рекурентни уравнения. Това не променя асимптотиката на решението. Примерно, както вече отбелязахме в Подсекция 5.2.5, ако ни е дадено

$$\mathsf{T}(n) = \mathsf{T}\left(\left\lfloor\frac{n}{2}\right\rfloor\right) + \mathsf{T}\left(\left\lceil\frac{n}{2}\right\rceil\right) + n$$

ще решим вместо него това уравнение

$$\mathsf{T}(n) = 2\mathsf{T}\left(\frac{n}{2}\right) + n$$

чието решение има същата асимптотика.

И още едно опростяване, което не променя асимптотиката на функцията, което ще покажем с пример. Ако последното рекурентно уравнение описва работата на алгоритъм, който прави две рекурсивни викания и извършва освен това линейна работа, събираемото, които отразява тази линейна работа, би трябвало да е "$\Theta(n)$", така че рекурентното уравнение би трябвало да е:

$$\mathsf{T}(n) = 2\mathsf{T}\left(\frac{n}{2}\right) + \Theta(n)$$

Но асимптотиката на решението не се променя, ако вместо "$\Theta(n)$" напишем просто "n", тоест най-простия за записване представител на класа от функции $\Theta(n)$. И така, записът,

който ще ползваме, е:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

5.3 Методи за решаване на рекурентни уравнения

5.3.1 Чрез развиване

Средството, което можем да опитаме винаги, за да решим дадено рекурентно уравнение, е да започнем да го развиваме, търсейки някаква закономерност. Ето много прост пример с уравнение, което вече видяхме (5.12).

Задача 4

Решете рекурентното уравнение

$$\begin{aligned} T(0) &= 0 \\ T(n) &= T(n-1) + n \text{ за } n > 0 \end{aligned} \tag{5.15}$$

чрез развиване.

Решение: Разсъждаваме така. Нека n е достатъчно голямо число. Тогава е вярно, че $T(n-1) = T((n-1)-1) + (n-1)$, тоест, че $T(n-1) = T(n-2) + n - 1$. Ако заместим "$T(n-1)$" в дясната страна на (5.15), с "$T(n-2) + n - 1$", получаваме

$$T(n) = T(n-2) + (n-1) + n \tag{5.16}$$

Но за достатъчно големи n, $T(n-2) = T((n-2)-1) + (n-2)$, тоест $T(n-2) = T(n-3) + (n-2)$. Ако сега заместим дясната страна в (5.16), получаваме

$$T(n) = T(n-3) + (n-2) + (n-1) + n \tag{5.17}$$

По правило, ако след 3-4 такива последователни развивания не видим такава закономерност, няма смисъл да опитваме повече. В случая рекурентното уравнение е достатъчно просто, така че закономерност се появи. Очевидно е, че ако продължим по същия начин, след краен брой развивания ще достигнем до

$$T(n) = T(0) + 1 + 2 + \cdots + (n-2) + (n-1) + n \tag{5.18}$$

В случая това наистина е очевидно, защото аргументът намалява с единица и рано или късно ще стане нула, а събираемите вдясно, ако се пишат систематично, започват от числото, с единица по-голямо от аргумента на T вдясно и завършват с n, като нарастват с единица.

Сега прилагаме два известни факта. Първо, че $T(0) = 0$, което знаем от началното условие, и второ, че $1 + 2 + \cdots n = \frac{n(n+1)}{2}$. Замествайки в (5.18), веднага получаваме

$$T(n) = \frac{n(n+1)}{2}$$

Това е и решението. □

Методът с развиването (на английски, *unfolding*) **не е формален**, защото можем да развием началния израз не повече от константен брой пъти и неизбежно използваме интуиция, за да

напишем уравнение, в което участват началните условия. Например, при прехода от (5.17) към (5.18) ползваме интуиция. Така че, ако се иска формално решение, методът с развиването не може да се използва. Тогава трябва да ползваме някой от формалните методи като индукция (Подсекция 5.3.3), Мастър теоремата (Подсекция 5.3.4), теорема на Akra-Bazzi (Подсекция 5.3.5) или метода с характеристичното уравнение (Подсекция 5.3.6). Измежду тези методи единственият универсален е индукцията, защото всички други искат рекурентното уравнение да има определена форма. Но преди да ползваме индукция, трябва да знаем какво да докажем по индукция и за тази цел може да ползваме неформалния метод на развиването (или с дървото на рекурсията, вижте Подсекция 5.3.2).

Методът с развиването има своите ограничения. Опитайте да решите рекурентното уравнение на числата на Фибоначи (5.1) с развиване. Практически е невъзможно да получите (5.14), ако използвате само здрав разум без никаква теория. **Теоретично**, има някаква ненулева вероятност след известен брой развивания на (5.1) да забележите, че закономерността е $\frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^n$, но това иска наистина **нечовешка интуиция**, защото никъде в развиването няма видите $\sqrt{5}$ и изобщо развиването ще съдържа само цели коефициенти.

Прилагайки метода на развиването, често ползваме следния резултат, ако търсим само асимптотиката на решението, а не точното решение. Забележете, че няма нужда k да е цяло число.

Лема 34

Нека $k > 0$ е константа. Итераторът $n \mapsto n - k$ се изпълнява $\Theta(n)$ пъти, докато n не стане $\Theta(1)$.

Вземаме за очевидно, че съществува реален интервал $[a, b]$, където $0 < a < b$ и $a \leqslant b - k$, такъв че за всяко достатъчно голямо n съществува $c \in [a, b]$, такова че итерирането

$$n \mapsto n - k \mapsto n - 2k \mapsto n - 3k \mapsto \cdots$$

завършва в c. С други думи, че итерирането е

$$\underbrace{n \mapsto n - k \mapsto n - 2k \mapsto n - 3k \mapsto \cdots \mapsto n - (m-1)k}_{\text{извън } [a,b]} \mapsto \underbrace{n - mk}_{\text{в } [a,b]}$$

за някое m, такова че $n - mk \leqslant b$, но $n - (m-1) \cdot k > b$.

Твърди се, че това итериране се извършва $\Theta(n)$ пъти. Но това следва директно от очевидния факт, че итерирането се извършва точно m пъти, а

$$n \leqslant km + b$$
$$n \geqslant km - k + b$$

така че $m \asymp n$. □

Следствие 16

Нека $k > 1$ е константа.

❶ Итераторът $n \mapsto \frac{n}{k}$ се изпълнява $\Theta(\lg n)$ пъти, докато n не стане $\Theta(1)$.

❷ Итераторът $n \mapsto \sqrt[k]{n}$ се изпълнява $\Theta(\lg \lg n)$ пъти, докато n не стане $\Theta(1)$.

Ще докажем ❶. Разглеждаме итерирането

$$n \mapsto \frac{n}{k} \mapsto \frac{n}{k^2} \mapsto \frac{n}{k^3} \mapsto \cdots \mapsto c$$

където c е елемент от предварително избран, подходящ за k, реален интервал, какъвто беше $[a, b]$ в доказателството на Лема 34. Нека $\ell = \log_k n$. Заместваме n с k^ℓ. Тогава итерирането става

$$k^\ell \mapsto \frac{k^\ell}{k} \mapsto \frac{k^\ell}{k^2} \mapsto \frac{k^\ell}{k^3} \mapsto \cdots \mapsto c$$

което е същото като

$$k^\ell \mapsto k^{\ell-1} \mapsto k^{\ell-2} \mapsto k^{\ell-3} \mapsto \cdots \mapsto c$$

Сега прилагаме Лема 34 за степенните показатели и заключаваме, че итерирането се изпълнява $\Theta(\ell)$ пъти. Но знаем, че $\ell = \log_k n$, така че изпълнението става $\Theta(\lg n)$ пъти.

❷ може да се докаже чрез още едно прилагане на същата идея, но може и директно. Разглеждаме итерирането

$$n \mapsto \sqrt[k]{n} \mapsto \sqrt[k]{\sqrt[k]{n}} \mapsto \sqrt[k]{\sqrt[k]{\sqrt[k]{n}}} \mapsto \cdots \mapsto c$$

където c е елемент от предварително избран, подходящ за k, реален интервал. Нека $\ell = \log_k(\log_k(n))$. Заместваме n с k^{k^ℓ}. Тогава итерирането става

$$k^{k^\ell} \mapsto \sqrt[k]{k^{k^\ell}} \mapsto \sqrt[k]{\sqrt[k]{k^{k^\ell}}} \mapsto \sqrt[k]{\sqrt[k]{\sqrt[k]{k^{k^\ell}}}} \mapsto \cdots \mapsto c$$

което е същото като

$$k^{k^\ell} \mapsto k^{k^{\ell-1}} \mapsto k^{k^{\ell-2}} \mapsto k^{k^{\ell-3}} \mapsto \cdots \mapsto c$$

Сега прилагаме Лема 34 за степенните показатели на степенните показатели и заключаваме, че итерирането се изпълнява $\Theta(\ell)$ пъти. Но знаем, че $\ell = \log_k(\log_k(n))$, така че изпълнението става $\Theta(\lg\lg n)$ пъти. □

Следствие 17

Лема 34 и Следствие 16 ни дават директно следните резултати.

- Уравнението $\mathsf{T}(n) = \mathsf{T}(n-1) + 1$ има решение $\mathsf{T}(n) \asymp \Theta(n)$.
- Уравнението $\mathsf{T}(n) = \mathsf{T}\left(\frac{n}{2}\right) + 1$ има решение $\mathsf{T}(n) \asymp \Theta(\lg n)$.
- Уравнението $\mathsf{T}(n) = \mathsf{T}(\sqrt{n}) + 1$ има решение $\mathsf{T}(n) \asymp \Theta(\lg \lg n)$.

Ако съобразим, че току-що изведените резултати са за височините на дървета с разклоненост 1, а при умножение с 2 на функцията вдясно същите дървета стават съвършени двоични дървета със същите съответни височини, имаме следното.

- Уравнението $\mathsf{T}(n) = 2\mathsf{T}(n-1) + 1$ има решение $\mathsf{T}(n) \asymp \Theta(2^n)$.
- Уравнението $\mathsf{T}(n) = 2\mathsf{T}\left(\frac{n}{2}\right) + 1$ има решение $\mathsf{T}(n) \asymp \Theta(n)$.
- Уравнението $\mathsf{T}(n) = 2\mathsf{T}(\sqrt{n}) + 1$ има решение $\mathsf{T}(n) \asymp \Theta(\lg n)$.

Ето един значително по-сложен пример за прилагане на метода с развиването.

Задача 5

Решете уравнението

$$\mathsf{T}(n) = n^2\mathsf{T}\left(\frac{n}{2}\right) + 1$$

чрез развиване.

Решение: В сила е

$$\begin{aligned}
\mathsf{T}(n) &= n^2\,\mathsf{T}\left(\frac{n}{2}\right) + 1 \\
&= n^2\left(\frac{n^2}{4}\mathsf{T}\left(\frac{n}{4}\right) + 1\right) + 1 \\
&= \frac{n^4}{2^2}\mathsf{T}\left(\frac{n}{2^2}\right) + n^2 + 1 \\
&= \frac{n^4}{2^2}\left(\frac{n^2}{2^4}\mathsf{T}\left(\frac{n}{2^3}\right) + 1\right) + n^2 + 1 \\
&= \frac{n^6}{2^6}\mathsf{T}\left(\frac{n}{2^3}\right) + \frac{n^4}{2^2} + n^2 + 1 \\
&= \frac{n^6}{2^6}\left(\frac{n^2}{2^6}\mathsf{T}\left(\frac{n}{2^4}\right) + 1\right) + \frac{n^4}{2^2} + n^2 + 1 \\
&= \frac{n^8}{2^{12}}\mathsf{T}\left(\frac{n}{2^4}\right) + \frac{n^6}{2^6} + \frac{n^4}{2^2} + n^2 + 1 \\
&= \frac{n^8}{2^{12}}\left(\frac{n^2}{2^8}\mathsf{T}\left(\frac{n}{2^5}\right) + 1\right) + \frac{n^6}{2^6} + \frac{n^4}{2^2} + n^2 + 1 \\
&= \frac{n^{10}}{2^{20}}\mathsf{T}\left(\frac{n}{2^5}\right) + \frac{n^8}{2^{12}} + \frac{n^6}{2^6} + \frac{n^4}{2^2} + n^2 + 1 \\
&= \frac{n^{10}}{2^{20}}\mathsf{T}\left(\frac{n}{2^5}\right) + \frac{n^8}{2^{12}} + \frac{n^6}{2^6} + \frac{n^4}{2^2} + \frac{n^2}{2^0} + \frac{n^0}{2^0} \\
&= \frac{n^{10}}{2^{5.4}}\mathsf{T}\left(\frac{n}{2^5}\right) + \frac{n^8}{2^{4.3}} + \frac{n^6}{2^{3.2}} + \frac{n^4}{2^{2.1}} + \frac{n^2}{2^{1.0}} + \frac{n^0}{2^{0.(-1)}} \\
&\ldots \\
&= \underbrace{\frac{n^{2i}}{2^{i(i-1)}}\mathsf{T}\left(\frac{n}{2^i}\right)}_{\mathsf{A}} + \underbrace{\frac{n^{2(i-1)}}{2^{(i-1)(i-2)}} + \frac{n^{2(i-2)}}{2^{(i-2)(i-3)}} + \cdots + \frac{n^8}{2^{4.3}} + \frac{n^6}{2^{3.2}} + \frac{n^4}{2^{2.1}} + \frac{n^2}{2^{1.0}} + \frac{n^0}{2^{0.(-1)}}}_{\mathsf{B}:\ \text{сума с } i \text{ събираеми}}
\end{aligned}$$

Полученият израз е общият вид на дясната страна в процеса на развиването; с други думи, дясната страна след $i+1$ развивания. Максималната стойност на i е $\lg n$ съгласно Следствие 16. Изразът вдясно е сума от два израза, които означаваме с A и B. Първо ще пресметнем A с $\lg n$ наместо i, като имаме предвид, че $\mathsf{T}(1) = \Theta(1)$.

$$\mathsf{A} = \frac{n^{2\lg n}}{2^{\lg^2 n - \lg n}}\mathsf{T}(1) \asymp \frac{2^{\lg n} n^{2\lg n}}{2^{\lg^2 n}} = \frac{n.n^{2\lg n}}{2^{\lg^2 n}}$$

Но

$$2^{\lg^2 n} = 2^{\lg n.\lg n} = 2^{\lg\left(n^{\lg n}\right)} = n^{\lg n} \tag{5.19}$$

Тогава

$$\mathsf{A} \asymp \frac{n.n^{2\lg n}}{n^{\lg n}} \asymp n^{1+\lg n} \tag{5.20}$$

Сега да разгледаме B. Очевидно, можем да го представим като сума по следния начин:

$$\begin{aligned}
B &= \sum_{j=1}^{\lg n} \frac{n^{2((\lg n)-j)}}{2^{((\lg n)-j)((\lg n)-j-1)}} \\
&= \sum_{j=1}^{\lg n} \frac{1}{n^{2j}} \frac{n^{2\lg n}}{2^{(\lg^2 n - j\lg n - j\lg n + j^2 - \lg n + j)}} \\
&= \sum_{j=1}^{\lg n} \frac{1}{n^{2j}} \frac{\left(n^{2\lg n}\right)\left(2^{2j\lg n}\right)\left(2^{\lg n}\right)}{\left(2^{\lg^2 n}\right)\left(2^{j^2+j}\right)}
\end{aligned} \tag{5.21}$$

Но

$$2^{2j\lg n} = 2^{\lg\left(n^{2j}\right)} = n^{2j} \tag{5.22}$$

и

$$2^{\lg n} = n \tag{5.23}$$

Прилагаме (5.19), (5.22) и (5.23) върху (5.21) и получаваме

$$\begin{aligned}
B &= \sum_{j=1}^{\lg n} \frac{1}{n^{2j}} \frac{\left(n^{2\lg n}\right)\left(n^{2j}\right)\left(n\right)}{\left(n^{\lg n}\right)\left(2^{j^2+j}\right)} \\
&= \sum_{j=1}^{\lg n} \frac{n^{1+\lg n}}{2^{j^2+j}} \\
&= \left(n^{1+\lg n}\right) \sum_{j=1}^{\lg n} \frac{1}{2^{j^2+j}} \\
&\leqslant \left(n^{1+\lg n}\right) \sum_{j=1}^{\infty} \frac{1}{2^{j^2+j}} && \text{// тъй като знаем, че } \sum_{j=1}^{\infty} \frac{1}{2^j} = 1 \\
&\leqslant n^{1+\lg n}
\end{aligned} \tag{5.24}$$

От (5.20) и (5.24) следва, че

$$T(n) \asymp n^{1+\lg n}$$

Допълнение 22: Няколко рекурентни уравнения чрез развиване

Задача 6

Решете рекурентното уравнение

$$T(n) = T(n-2) + 2\lg n$$

чрез развиване.

Решение Да развием уравнението:

$$\begin{aligned}
\mathsf{T}(n) &= \mathsf{T}(n-2) + 2\lg n \\
&= \mathsf{T}(n-4) + 2\lg(n-2) + 2\lg n \\
&= \mathsf{T}(n-6) + 2\lg(n-4) + 2\lg(n-2) + 2\lg n \\
&= \ldots \\
&= \Theta(1) + \cdots + 2\lg(n-4) + 2\lg(n-2) + 2\lg n \\
&\asymp \lg n^2 + \lg(n-2)^2 + \lg(n-4)^2 + \cdots + \lg 3^2 + \lg 1^2 \\
&= \lg\left(n^2 \cdot (n-2)^2 \cdot (n-4)^2 \cdot \cdots \cdot 3^2 \cdot 1\right) \\
&= \lg\Big(\underbrace{n \cdot n \cdot (n-2) \cdot (n-2) \cdot (n-4) \cdot (n-4) \cdot \cdots \cdot 5 \cdot 5 \cdot 3 \cdot 3 \cdot 1}_{n \text{ множителя}}\Big)
\end{aligned}$$

Нека

$$\mathsf{T}(n) = \lg\Big(\underbrace{n \cdot n \cdot (n-2) \cdot (n-2) \cdot (n-4) \cdot (n-4) \cdot \cdots \cdot 5 \cdot 5 \cdot 3 \cdot 3 \cdot 1}_{n \text{ множителя}}\Big)$$

$$\mathsf{X}(n) = \lg\Big(\underbrace{n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdot \cdots \cdot 3 \cdot 2 \cdot 1}_{n \text{ множителя}}\Big)$$

$$\mathsf{Y}(n) = \lg\Big(\underbrace{(n+1) \cdot n \cdot (n-1) \cdot (n-2) \cdot \cdots \cdot 4 \cdot 3 \cdot 2}_{n \text{ множителя}}\Big)$$

Очевидно

$$\mathsf{X}(n) \leqslant \mathsf{T}(n) \leqslant \mathsf{Y}(n)$$

Но $\mathsf{X}(n) = \lg n!$ и $\mathsf{Y}(n) = \lg(n+1)!$. Тогава $\mathsf{X}(n) \asymp n\lg n$ и $\mathsf{Y}(n) \asymp n\lg n$. Следователно, $\mathsf{T}(n) \asymp n\lg n$. □

Задача 7

Решете уравнението

$$\mathsf{T}(n) = \frac{n}{n+1}\mathsf{T}(n-1) + 1$$

чрез развиване.

Решение В сила е

$$\begin{aligned}
T(n) &= \frac{n}{n+1}T(n-1)+1\\
&= \frac{n}{n+1}\left(\frac{n-1}{n}T(n-2)+1\right)+1\\
&= \frac{n-1}{n+1}T(n-2)+\frac{n}{n+1}+1\\
&= \frac{n-1}{n+1}\left(\frac{n-2}{n-1}T(n-3)+1\right)+\frac{n}{n+1}+1\\
&= \frac{n-2}{n+1}T(n-3)+\frac{n-1}{n+1}+\frac{n}{n+1}+1\\
&= \frac{n-2}{n+1}\left(\frac{n-3}{n-2}T(n-4)+1\right)+\frac{n-1}{n+1}+\frac{n}{n+1}+1\\
&= \frac{n-3}{n+1}T(n-4)+\frac{n-2}{n+1}+\frac{n-1}{n+1}+\frac{n}{n+1}+1 \qquad (5.25)
\end{aligned}$$

Ако продължим така до $T(1)$, развиваме (5.25) до

$$\begin{aligned}
T(n) &= \frac{2}{n+1}T(1)+\frac{3}{n+1}+\frac{4}{n+1}+\cdots+\frac{n-2}{n+1}+\frac{n-1}{n+1}+\frac{n}{n+1}+1\\
&= \frac{2}{n+1}T(1)+\frac{3}{n+1}+\frac{4}{n+1}+\cdots+\frac{n-2}{n+1}+\frac{n-1}{n+1}+\frac{n}{n+1}+\frac{n+1}{n+1}\\
&= \underbrace{\frac{\Theta(1)}{n+1}}_{O(1)}+\frac{1}{n+1}\sum_{i=3}^{n+1} i\\
&\asymp \frac{1}{n+1}\cdot\frac{(n-1)(n+4)}{2}\\
&\asymp \Theta(n)
\end{aligned}$$

И така, $T(n)\asymp n$. □

Задача 8

Решете рекурентното уравнение

$$T(n) = \frac{n}{n+1}T(n-1)+\lg n$$

чрез развиване.

Решение В сила е

$$\frac{n}{n+1}T(n-1)+\lg n=$$
$$\frac{n}{n+1}\left(\frac{n-1}{n}T(n-2)+\lg(n-1)\right)+\lg n=$$
$$\frac{n-1}{n+1}T(n-2)+\frac{n}{n+1}\lg(n-1)+\lg n=$$
$$\frac{n-1}{n+1}\left(\frac{n-2}{n-1}T(n-3)+\lg(n-2)\right)+\frac{n}{n+1}\lg(n-1)+\lg n=$$
$$\frac{n-2}{n+1}T(n-3)+\frac{n-1}{n+1}\lg(n-2)+\frac{n}{n+1}\lg(n-1)+\lg n=$$
$$\ldots=$$
$$\underbrace{\frac{2}{n+1}T(1)}_{O(1)}+\frac{3}{n+1}\lg 2+\frac{4}{n+1}\lg 3+\cdots+\frac{n-1}{n+1}\lg(n-2)+\frac{n}{n+1}\lg(n-1)+\lg n\asymp$$
$$\frac{(2+1)\lg 2}{n+1}+\frac{(3+1)\lg 3}{n+1}+\cdots+\frac{(n-1+1)\lg(n-1)}{n+1}+\frac{(n+1)\lg n}{n+1}=$$
$$\frac{1}{n+1}\left(\sum_{k=2}^{n}k\lg k+\sum_{k=2}^{n}\lg k\right)$$

Но $\sum_{k=2}^{n}k\lg k\asymp n^2\lg n$ съгласно Теорема 23, а $\sum_{k=2}^{n}\lg k=\lg n!\asymp n\lg n$ съгласно Теорема 22, откъдето $\frac{1}{n+1}\left(\sum_{k=2}^{n}k\lg k+\sum_{k=2}^{n}\lg k\right)\asymp n\lg n$, така че $T(n)\asymp n\lg n$. □

Задача 9

Решете рекурентното уравнение

$$T(n)=nT(n-1)+1$$

чрез развиване.

Решение

$$\begin{aligned}
T(n) &= nT(n-1)+1\\
&= n((n-1)T(n-2)+1)+1\\
&= n(n-1)T(n-2)+n+1\\
&= n(n-1)((n-2)T(n-3)+1)+n+1\\
&= n(n-1)(n-2)T(n-3)+n(n-1)+n+1\\
&= n(n-1)(n-2)((n-3)T(n-4)+1)+n(n-1)+n+1\\
&= n(n-1)(n-2)(n-3)T(n-4)+n(n-1)(n-2)+n(n-1)+n+1\\
&\quad\dots\\
&= \frac{n!}{(n-i)!}T(n-i)+\frac{n!}{(n-i+1)!}+\frac{n!}{(n-i+2)!}+\dots+\frac{n!}{(n-1)!}+\frac{n!}{n!}\\
&\quad\dots\\
&= \frac{n!}{1!}T(1)+\frac{n!}{2!}+\frac{n!}{3!}+\dots+\frac{n!}{(n-1)!}+\frac{n!}{n!}\\
&= n!\times\underbrace{\left(\frac{\Theta(1)}{1!}+\frac{1}{2!}+\frac{1}{3!}+\dots+\frac{1}{(n-1)!}+\frac{1}{n!}\right)}_{A}
\end{aligned}$$

Твърдим, че A е ограничена отгоре от константа. Наистина, редът $\sum_{i=1}^{\infty}\frac{1}{i!}$, тъй като геометричният ред $\sum_{i=1}^{\infty}\frac{1}{2^i}$ е сходящ, а $i!>2^i$ за всяко $i>3$. И така, $T(n)\asymp n!$. □

Задача 10

Решете рекурентното уравнение

$$T(n)=nT(n-1)+2^n$$

чрез развиване.

Решение

$$\begin{aligned}
T(n) &= nT(n-1) + 2^n \\
&= n((n-1)T(n-2) + 2^{n-1}) + 2^n \\
&= n(n-1)T(n-2) + n2^{n-1} + 2^n \\
&= n(n-1)((n-2)T(n-3) + 2^{n-2}) + n2^{n-1} + 2^n \\
&= n(n-1)(n-2)T(n-3) + n(n-1)2^{n-2} + n2^{n-1} + 2^n \\
&= n(n-1)(n-2)((n-3)T(n-4) + 2^{n-3}) + n(n-1)2^{n-2} + n2^{n-1} + 2^n \\
&= n(n-1)(n-2)(n-3)T(n-4) + n(n-1)(n-2)2^{n-3} + n(n-1)2^{n-2} + n2^{n-1} + 2^n \\
&\quad \dots \\
&= \frac{n!}{(n-i)!}T(n-i) + \frac{n!\,2^{n-i+1}}{(n-i+1)!} + \frac{n!\,2^{n-i+2}}{(n-i+2)!} + \dots + \frac{n!\,2^{n-1}}{(n-1)!} + \frac{n!\,2^n}{n!} \\
&\quad \dots \\
&= \frac{n!}{1!}T(1) + \frac{n!\,2^2}{2!} + \frac{n!\,2^3}{3!} + \dots + \frac{n!2^{n-1}}{(n-1)!} + \frac{n!\,2^n}{n!} \\
&= n! \times \left(\Theta(1) + \sum_{k=2}^{n} \frac{2^k}{k!}\right)
\end{aligned}$$

Но редът $\sum_{k=1}^{\infty} \frac{2^k}{k!}$ е сходящ, тъй като, по критерия на d'Alambert:

$$\frac{\frac{2^{k+1}}{(k+1)!}}{\frac{2^k}{k!}} = \frac{2}{k+1} < \epsilon, \text{ за всяко } \epsilon > 0 \text{ и за всяко достатъчно голямо } k.$$

Следва, че $\sum_{k=2}^{n} \frac{2^k}{k!}$ е ограничена отгоре от константа. Следва, че $T(n) \asymp n!$. □

Задача 11

Решете рекурентното уравнение

$$T(n) = 2T(n-1) + n!$$

чрез развиване.

Решение

$$\begin{aligned}
T(n) &= 2T(n-1) + n! \\
&= 2(2T(n-1) + (n-1)!) + n! \\
&= 4T(n-1) + 2(n-1)! + n! \\
&= 4(2T(n-2) + (n-2)!) + 2(n-1)! + n! \\
&= 8T(n-2) + 4(n-2)! + 2(n-1)! + n! && (5.26) \\
&= 8(2T(n-3) + (n-3)!) + 4(n-2)! + 2(n-1)! + n! \\
&= 16T(n-3) + 8(n-2)! + 4(n-2)! + 2(n-1)! + n! \\
&\quad \dots && (5.27) \\
&= \underbrace{2^{n-1}T(1) + 2^{n-2}2! + 2^{n-3}3! + \cdots 2^2(n-2)! + 2(n-1)!}_{A} + n! && (5.28)
\end{aligned}$$

Ще докажем, че асимптотиката на дясната страна е $\Theta(n!)$. За целта е достатъчно да докажем, че е $A = O(n!)$, понеже другото събираемо е $n!$. Това може да се докаже чрез Лема 35.

Лема 35

$\forall n \geqslant 2 : 2(n-1)! + 2^2(n-2)! + 2^3(n-3)! + \cdots + 2^{n-3}3! + 2^{n-2}2! + 2^{n-1}1! < 2n!$.

Доказателство: По индукция по n.

База: $n = 2$. Лявата страна е $2(2-1)! = 2$. Дясната страна е $2 \cdot 2! = 4$. ✓

Индуктивно предположение: Да допуснем, че за някое $n \geqslant 2$ е вярно, че

$$2(n-1)! + 2^2(n-2)! + 2^3(n-3)! + \cdots + 2^{n-3}3! + 2^{n-2}2! + 2^{n-1}1! < 2n!$$

Индуктивна стъпка: Ще покажем, че

$$2n! + 2^2(n-1)! + 2^3(n-2)! + \cdots + 2^{n-2}3! + 2^{n-1}2! + 2^n 1! < 2(n+1)!$$

Но това е същото като

$$2n! + 2\big(2(n-1)! + 2^2(n-2)! + \cdots + 2^{n-3}3! + 2^{n-2}2! + 2^{n-1}1!\big) < 2 \cdot (n+1) \cdot n!$$

което е същото като

$$2n! + 2\big(2(n-1)! + 2^2(n-2)! + \cdots + 2^{n-3}3! + 2^{n-2}2! + 2^{n-1}1!\big) < 2 \cdot n \cdot n! + 2n! \leftrightarrow$$
$$2\big(2(n-1)! + 2^2(n-2)! + \cdots + 2^{n-3}3! + 2^{n-2}2! + 2^{n-1}1!\big) < 2 \cdot n \cdot n! \leftrightarrow$$
$$2(n-1)! + 2^2(n-2)! + \cdots + 2^{n-3}3! + 2^{n-2}2! + 2^{n-1}1! < n \cdot n!$$

Това следва веднага от индуктивното предположение и допускането, че $n \geqslant 2$. □

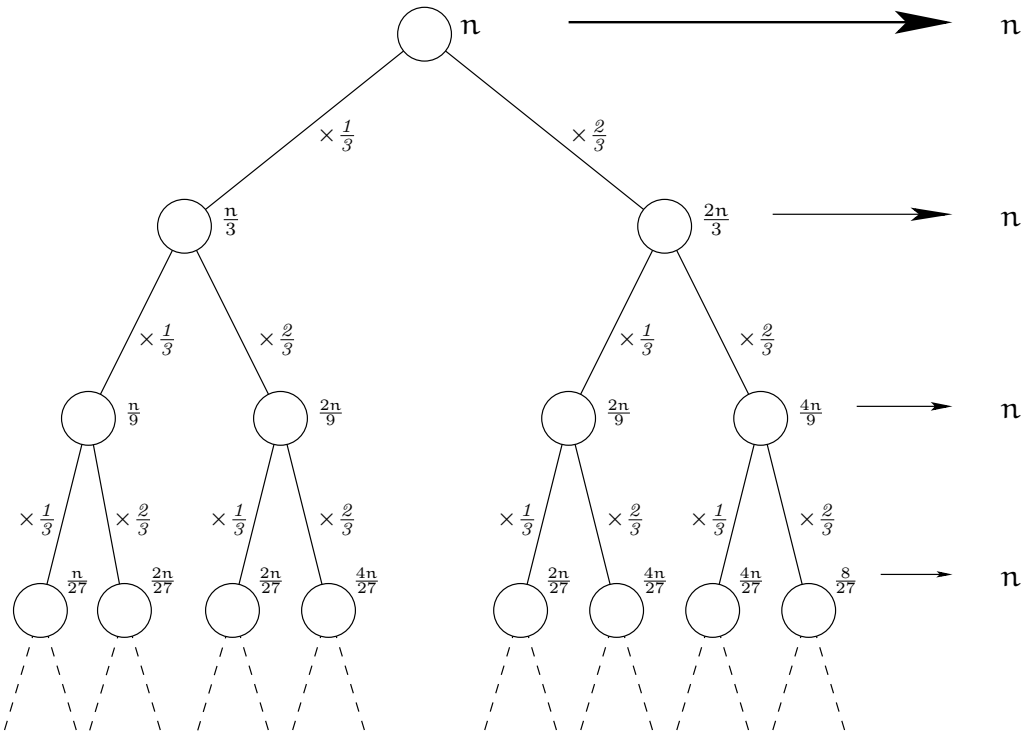
Има алтернативно, по-просто и елегантно доказателство, посочено на автора на лекционните записки от Добромир Кралчев. Представяме израза от (5.28) така:

$$n!\left(1 + \underbrace{\frac{2}{n} + \frac{2^2}{n(n-1)} + \frac{2^3}{n(n-1)(n-2)} + \cdots + \frac{2^{n-2}}{n(n-1)\cdot\cdots\cdot 3} + \frac{2^{n-1}}{n(n-1)\cdot\cdots\cdot 2}}_{B}\right)$$

Ако покажем, че $B = O(1)$, то ние сме готови с доказателството. Ще покажем, че $B = O(1)$. Но във всяко събираемо на B, множителите-двойки в числителя са точно колкото множителите в знаменателя, поради което можем да напишем B така:

$$\frac{1}{\frac{n}{2}} + \frac{1}{\frac{n}{2}\cdot\frac{n-1}{2}} + \underbrace{\frac{1}{\frac{n}{2}\cdot\frac{n-1}{2}\cdot\frac{n-2}{2}} + \cdots + \frac{1}{\frac{n}{2}\cdot\frac{n-1}{2}\cdot\cdots\cdot\frac{2}{2}}}_{C}$$

тоест, $B = \frac{1}{\frac{n}{2}} + \frac{1}{\frac{n}{2}\cdot\frac{n-1}{2}} + C$. Всяко събираемо на C е $O\left(\frac{1}{n^3}\right)$ и C има $O(n)$ събираеми, поради което $C = O\left(\frac{1}{n^2}\right)$, от където веднага следва, че $B = O(1)$.



Фигура 5.1: Дървото на рекурсията, отговарящо на $\mathsf{T}(\mathfrak{n}) = \mathsf{T}\left(\frac{\mathfrak{n}}{3}\right) + \mathsf{T}\left(\frac{2\mathfrak{n}}{3}\right) + \mathfrak{n}$. За всяко от изобразените нива, сумата от стойностите на върховете в него е $\mathfrak{n}$.

Така или иначе, показахме, че за A от (5.28) е вярно, че $A = O(\mathfrak{n}!)$. От това следва, че $\mathsf{T}(\mathfrak{n}) \asymp \mathfrak{n}!$. □

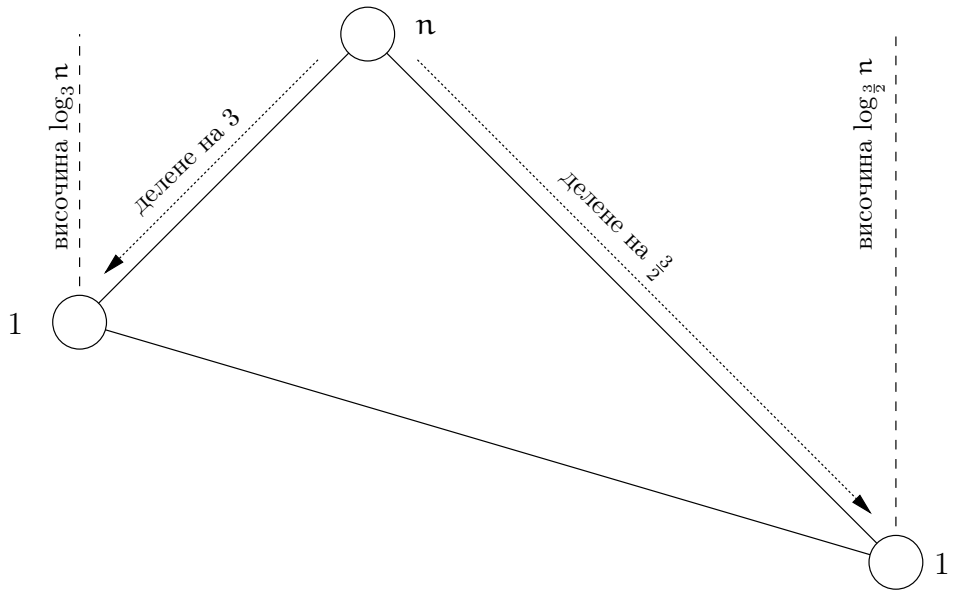
5.3.2 Чрез дървото на рекурсията

При този метод си представяме работата на рекурсивен алгоритъм, чиято сложност по време се описва от рекурентното уравнение, което изследваме. Представяме си дървото на извикванията на рекурсивния алгоритъм, съобразяваме колко работа се извършва във всеки връх от дървото (тоест, във всяко индивидуално изпълнение на рекурсивния алгоритъм), и сумираме тези количества по нива в дървото. След което търсим закономерност в сумите по нива. Виждаме, че метода с дървото е подобен на метода с развиването по това, че не е формален и за да даде резултати, трябва човекът, който го ползва, да прояви достатъчно добра интуиция. Тук ще направим илюстрация с пример. Ще решим рекурентното уравнение

$$\mathsf{T}(\mathfrak{n}) = \mathsf{T}\left(\frac{\mathfrak{n}}{3}\right) + \mathsf{T}\left(\frac{2\mathfrak{n}}{3}\right) + \mathfrak{n} \tag{5.29}$$

използвайки метода с дървото на рекурсията. Дървото на рекурсията—по-точно, само "горната" му част—е изобразено на Фигура 5.1. Всеки връх представлява едно индивидуално изпълнение на алгоритъма без рекурсивните викания. Двете ребра към децата му представляват рекурсивните викания, като те (ребрата) са маркирани с множителите, с които входът намалява. До всеки връх е написана работата, която алгоритъмът извършва в това индивидуално изпълнение извън рекурсивните викания. Върховете от едно и също ниво в дървото са нарисувани на едно и също ниво на фигурата.

Коренът е първоначалното изпълнение. В него алгоритъмът извършва работа $\mathfrak{n}$ извън



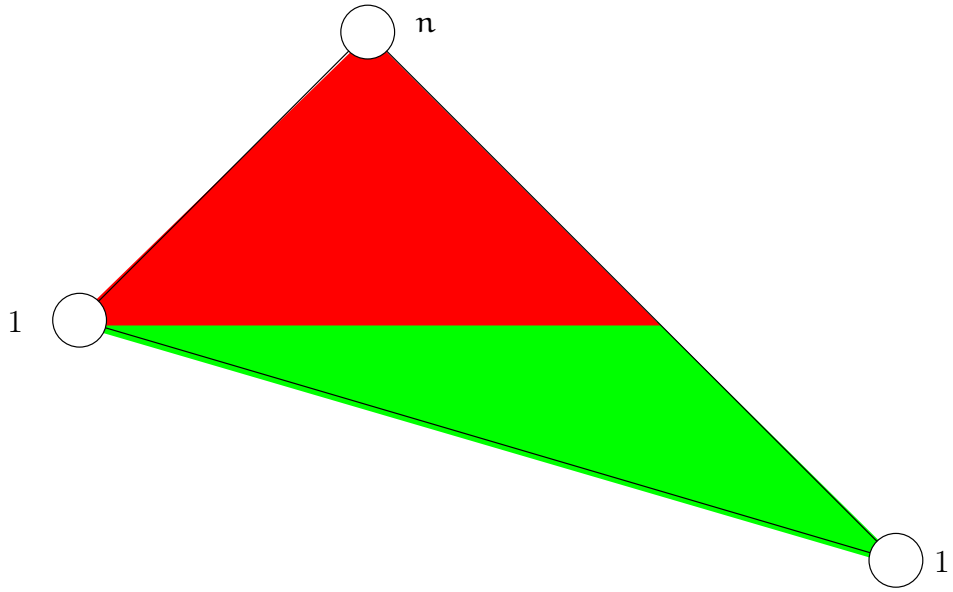
Фигура 5.2: Схема на дървото на рекурсията, отговарящо на $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$. Най-лявото листо е най-близо до корена, а най-дясното листо е най-отдалечено от корена. Височината на най-лявото листо е $\log_3 n$, а на най-дясното е $\log_{\frac{3}{2}} n$.

двете рекурсивни викания, затова вдясно от корена е написано "n". Едното рекурсивно викане се върши върху вход, който е с големина $\frac{1}{3}$ от големината на началния, а другото върху вход, който е с големина $\frac{2}{3}$ от големината на началния. Затова двете ребра, свързващи корена с децата му, са маркирани с $\frac{1}{3}$ и $\frac{2}{3}$. До всяко от двете деца е написана работата, която се извършва от съответното извикване на алгоритъма. Тази работа е същата като големината на входа за съответното викане, затова записаните стойности са $\frac{n}{3}$ и $\frac{2n}{3}$. И така нататък.

Да мислим за дървото като за подредено дърво. Припомняме, че това означава, че различаваме ляво от дясно дете на всеки вътрешен връх. Стойността, асоциирана с лявото дете на даден връх, е стойността, асоциирана с родителя, умножена по $\frac{1}{3}$. Стойността, асоциирана с дясното дете, е стойността, асоциирана с родителя, умножена по $\frac{2}{3}$. С други думи, наляво има деление на 3, а надясно, деление на $\frac{3}{2}$. Това, че разглеждаме дървото като подредено дърво, има значение единствено за по-лесното решаване на задачата. Листата на дървото са очевидно на различни нива, така че ни трябва някаква систематика в описването на дървото, за да видим закономерност. По начина, по който разглеждаме дървото, най-близкото до корена листо е краят на пътя, който започва от корена и във всеки вътрешен връх продължава наляво, а най-отдалеченото от корена листо е краят на пътя, който започва от корена и във всеки вътрешен връх продължава надясно. Образно казано, дървото има формата, показана на Фигура 5.2. Допускаме, че началната стойност е 1, така че листата са асоциирани с единици. Дървото е нарисувано по начин, който подсказва, че с деление на 3 достигаме 1 по-бързо, отколкото с деление на $\frac{3}{2}$.

Тривиално е да се докаже, че нива с номер от 0 (коренът) до $\log_3 n$ (най-лявото дете) са пълни[†] и във всяко от тях, сумата от стойностите е n. Фигура 5.3 илюстрира пълните нива и останалите: пълните нива са в червено, останалите са в зелено. Да намерим асимптотиката на рекурентното уравнение е същото като да оценим сумата от асоциираните стойности по всички върхове на дървото. За червеното поддърво (Фигура 5.3), тази сума е височината, умножена по n, защото по всяко негово ниво, сумата е n. Височината на червеното поддърво е $\log_3 n$, така че сумата на стойностите по неговите върхове е $n \log_3 n$. Това е долна граница

[†]В смисъл, че в ниво k има 2^k върха.



Фигура 5.3: Друга схема на дървото при $\mathsf{T}(n) = \mathsf{T}\left(\frac{n}{3}\right) + \mathsf{T}\left(\frac{2n}{3}\right) + n$. Червеното поддърво е съвършено, понеже нивата му са пълни. Следващите нива, нарисувани в зелено, са непълни.

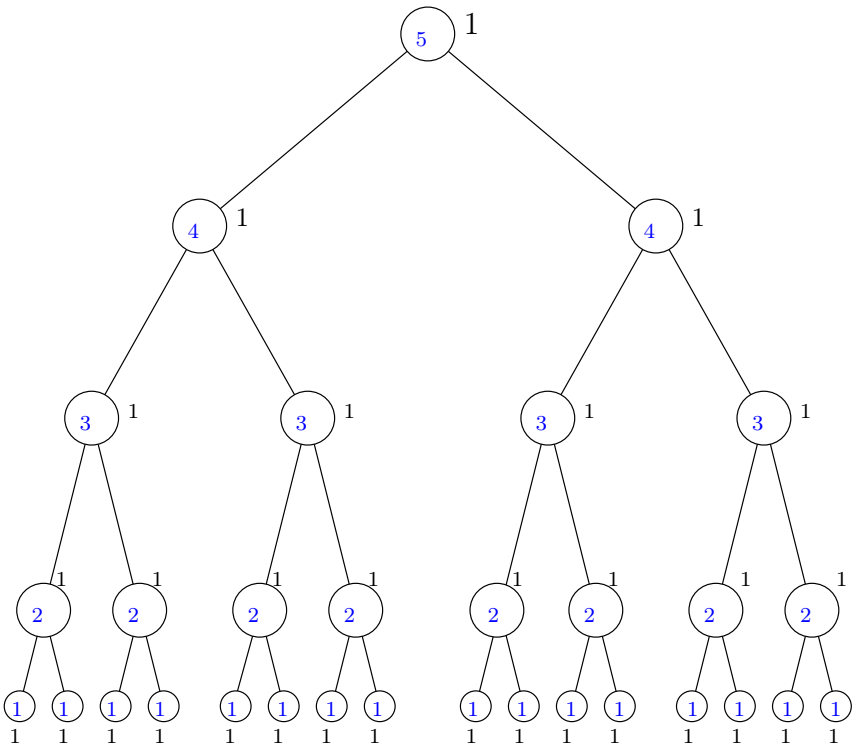
за сумата по всички върхове на дървото. Но очевидно $n \log_{\frac{3}{2}} n$ е горна граница за същата сума, понеже $\log_{\frac{3}{2}}$ е височината на цялото дърво и никое ниво няма сума повече от n. Но $n \log_3 n \asymp n \lg n$ и $n \log_{\frac{3}{2}} n \asymp n \lg n$. Щом $\mathsf{T}(n) \preceq n \lg n$ и $\mathsf{T}(n) \succeq n \lg n$, следва, че $\mathsf{T}(n) \asymp n \lg n$.

От този пример става ясно, че методът с дървото на рекурсията е подобен на метода с развиването, само че тук групираме събираемите по различен начин, търсейки закономерност. В последния пример изчислихме сумата от всички "нехомогенни части" по всички извиквания. И по-точно казано, не я изчислихме, а само я оценихме като асимптотика. Под "нехомогенни части" имаме предвид събираеми като n в (5.29). Такова събираемо отговаря на работата, която алгоритъмът извършва извън рекурсивните викания. Ако наречем такова събираемо, "нехомогенната част" по аналогия с (5.8), то, прилагайки метода с дървото на рекурсията, ние всъщност сумирахме нехомогенните части[†]. Истинската работа, която върши алгоритъмът, е сумата от тези "нехомогенни части" и броят на върховете на дървото. Но само сумата от "нехомогенните части" дава асимптотиката, така че можем да игнорираме броят на върховете и да не го добавяме към сумата от "нехомогенните части". Ако обаче трябва да решаваме хомогенно рекурентно уравнение като (5.1), трябва да намерим да намерим броя на върховете, или поне неговата асимптотика.

От друга страна, очевидно е, че всяко рекурентно уравнение, което описва сложност на алгоритъм, има **ненулева** нехомогенна част, защото алгоритъмът няма как да не върши поне константна работа извън рекурсивните викания. Така че, по отношение на изследването на алгоритми, напълно достатъчно е да сумираме въпросните "нехомогенни части". Подобно на развиването, този метод също е неформален и приложимостта му зависи от интуицията на човека, който го прилага. Ако човек не открие някаква закономерност в сумите, методът не е приложим.

Сега ще дадем пример за това, колко е важно да открием закономерност в сумите при този

[†]По-точно, сумирахме нехомогенните части и началните условия, но ние игнорираме началните условия при асимптотичния анализ.



Фигура 5.4: Дървото на рекурсията при $\mathsf{T}(1) = 1,\ \mathsf{T}(n) = 2\mathsf{T}(n-1)+1$, когато $n = 5$. Във върховете, в син цвят са написани съответните стойности на n.

метод. Да разгледаме рекурентното уравнение

$$\begin{aligned}&\mathsf{T}(1) = 1\\&\mathsf{T}(n) = 2\mathsf{T}(n-1) + 1,\ n \geqslant 2\end{aligned} \tag{5.30}$$

и съответното дърво на рекурсията на Фигура 5.4 при $n = 5$. На ниво 0 сумата е 1, на ниво 1 сумата е 2, и така нататък, на ниво 4 сумата е 16. Сумата от стойностите, асоциирани с върховете, е 31. Лесно се вижда, а и лесно се доказва формално, че решението на (5.30) е $\mathsf{T}(n) \asymp 2^n$.

Сега да преобразуваме (5.30) ето така:

$$\mathsf{T}(n) = 2\mathsf{T}(n-1) + 1 \ \leftrightarrow\ \mathsf{T}(n) = \mathsf{T}(n-1) + \mathsf{T}(n-1) + 1$$

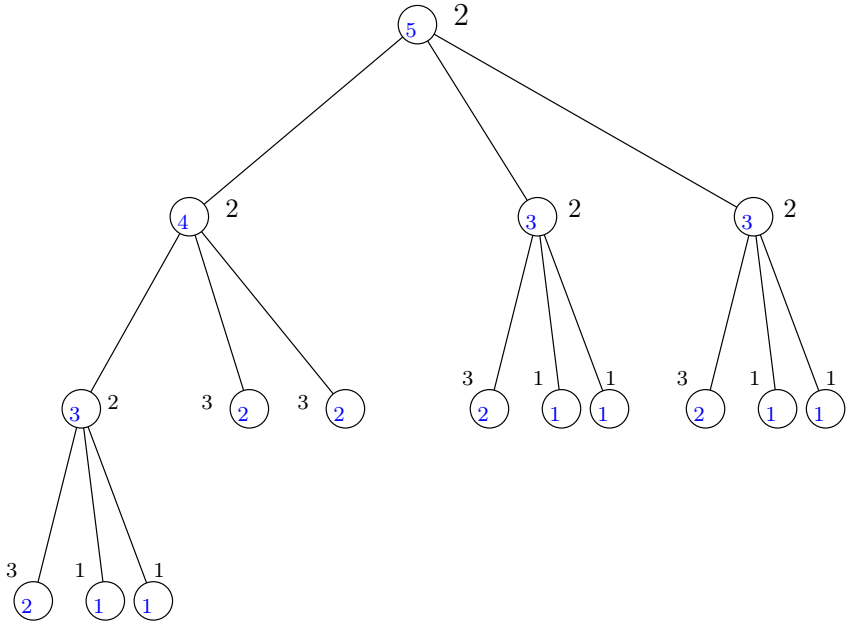
Очевидно $\mathsf{T}(n-1) = 2\mathsf{T}(n-2) + 1$

Тогава $\mathsf{T}(n) = \mathsf{T}(n-1) + 2\mathsf{T}(n-2) + 1 + 1 = \mathsf{T}(n-1) + 2\mathsf{T}(n-2) + 2$

Тъй като в случая искаме точно рекурентно уравнение с началните условия, налага се да добавим и начално условия $\mathsf{T}(2) = 3$. Рекурентното уравнение става:

$$\begin{aligned}&\mathsf{T}(1) = 1\\&\mathsf{T}(2) = 3\\&\mathsf{T}(n) = \mathsf{T}(n-1) + 2\mathsf{T}(n-2) + 2,\ n \geqslant 3\end{aligned} \tag{5.31}$$

Рекурентни уравнение (5.30) и (5.31) са еквивалентни. Но техните съответни дървета на рекурсията нито са еднакви, нито са изоморфни. Фигура 5.5 показва дървото на рекурсията, съответстващо на (5.31), за $n = 5$. Дървото от Фигура 5.5 е неподходящо за откриване на



Фигура 5.5: Дървото на рекурсията на $T(1) = 1,\ T(2) = 3,\ T(n) = T(n-1) + 2T(n-2) + 2$. Във върховете, в син цвят са написани съответните стойности на n.

закономерност както за прецизното решение $T(n) = 2^n - 1$, така и за асимптотичното решение $T(n) \asymp 2^n$.

5.3.3 По индукция

Ще демонстрираме доказването на асимптотиката на нарастването на функцията от дадено рекурентно уравнение с пример. Да разгледаме:

$$T(n) = 2T\left(\left\lceil \frac{n}{2} \right\rceil\right) + n \tag{5.32}$$

Ще докажем, че $T(n) = \Theta(n \lg n)$ по индукция по n. За да постигнем това, ще докажем поотделно, че $T(n) = O(n \lg n)$ и $T(n) = \Omega(n \lg n)$. В тези доказателство база няма, тъй като игнорираме началните условия.

Част i: Доказателство, че $T(n) = O(n \lg n)$. По дефиниция, това е същото като да докажем, че съществуват положителна константа c и положително n_0, такова че за всяко $n \geqslant n_0$:

$$T(n) \leqslant cn \lg n \tag{5.33}$$

Индуктивното предположение е, че

$$T\left(\left\lceil \frac{n}{2} \right\rceil\right) \leqslant c \left\lceil \frac{n}{2} \right\rceil \lg \left\lceil \frac{n}{2} \right\rceil \tag{5.34}$$

От (5.32) и (5.34):

$$\begin{aligned}
T(n) &\leqslant 2.c.\left\lceil\frac{n}{2}\right\rceil \lg\left\lceil\frac{n}{2}\right\rceil + n \\
&\leqslant 2.c.\left(\frac{n}{2}+1\right)\lg\left\lceil\frac{n}{2}\right\rceil + n && (5.35)\\
&\leqslant 2.c.\left(\frac{n}{2}+1\right)\lg\left(\frac{3}{4}n\right) + n \qquad // \text{ защото } \forall n \geqslant 2\left(\frac{3}{4}n \geqslant \left\lceil\frac{n}{2}\right\rceil\right) && (5.36)\\
&= c(n+2)\left(\lg n + \lg\frac{3}{4}\right) + n \\
&= cn\lg n + cn\lg\frac{3}{4} + 2c\lg n + 2c\lg\frac{3}{4} + n
\end{aligned}$$

Иска се да докажем, че полученото $cn\lg n + cn\lg\frac{3}{4} + 2c\lg n + 2c\lg\frac{3}{4} + n$ не надхвърля $cn\lg n$, за някаква константа c. Това е вярно, ако $cn\lg\frac{3}{4} + 2c\lg n + 2c\lg\frac{3}{4} + n \leqslant 0$. Да разгледаме

$$cn\lg\frac{3}{4} + 2c\lg n + 2c(\lg 3 - 2) + n = \left(c\lg\frac{3}{4} + 1\right)n + 2c\lg n + 2c\lg\frac{3}{4}$$

Асимптотиката на нарастването се определя от събираемото $\left(c\lg\frac{3}{4}+1\right)n$. Ако константата $c\lg\frac{3}{4}+1$ е отрицателна, тогава това събираемо е отрицателно за всички достатъчно големи стойности на n, откъдето веднага заключаваме, че

$$cn\lg n + cn\lg\frac{3}{4} + 2c\lg n + 2c\lg\frac{3}{4} + n \leqslant cn\lg n$$

за всички достатъчно големи стойности на n. С други думи, за по-кратко решение, няма да определяме n_0, бидейки убедени, че такава начална стойност има, а само ще определим подходящо c.

За да бъде изпълнено $c\lg\frac{3}{4}+1<0$, трябва да е вярно, че $c > \frac{1}{\lg\frac{4}{3}}$. Числено, дясната страна е по-малка от 2.41. Следователно, всяко $c > 2.41$ "върши работа" за нашето доказателство.

Наблюдение 20

В (5.35) ние заместваме $\left\lceil\frac{n}{2}\right\rceil$ с $\frac{3}{4}n$. Бихме могли да използваме всяко αn, стига да е изпълнено $\frac{1}{2} < \alpha < 1$.

- Защо трябва да е изпълнено $\frac{1}{2} < \alpha$? Ако това не е изпълнено, нямаме право да твърдим, че неравенството "$\leqslant$" между (5.35) и (5.36) е в сила.
- А защо трябва да е изпълнено $\alpha < 1$? Да допуснем, че $\alpha = 1$; с други думи, заместваме $\left\lceil\frac{n}{2}\right\rceil$ с n. Тогава (5.36) става:

  $$c(n+2)(\lg n) + n = cn\lg n + 2c\lg n + n$$

  Очевидно, това е по-голямо от $cn\lg n$ за всички достатъчно големи n, така че нямаме докателство.

Част ii: Доказателство, че $T(n) = \Omega(n\lg n)$. По дефиниция, това есъщото като да докажем, че съществуват положителна константа d и положително n_1, такова че за всяко $n \geqslant n_1$:

$$T(n) \geqslant dn\lg n \qquad (5.37)$$

Индуктивното предположение е, че

$$T\left(\left\lceil\frac{n}{2}\right\rceil\right) \geqslant d\left\lceil\frac{n}{2}\right\rceil \lg\left\lceil\frac{n}{2}\right\rceil \tag{5.38}$$

От (5.32) и (5.38):

$$\begin{aligned} T(n) &\geqslant 2.d.\left\lceil\frac{n}{2}\right\rceil \lg\left\lceil\frac{n}{2}\right\rceil + n \\ &\geqslant 2.d.\left(\frac{n}{2}\right) \lg\left(\frac{n}{2}\right) + n \\ &= dn(\lg n - 1) + n \\ &= dn\lg n + (1-d)n \\ &\geqslant dn\lg n \qquad \text{при условие, че } (1-d)n \geqslant 0 \end{aligned} \tag{5.39}$$

Очевидно всяко d, такова че $0 < d \leqslant 1$, "върши работа" за нашето доказателство.

С това доказателството е приключено. Преди да видим още примери, две предупреждения за потенциални грешки или проблеми при използването на индукция за доказване на асимптотиката на рекурентни уравнения.

- Първо, доказателството трябва да се извърши спрямо константата, с която е започнало. Да разгледаме отново следното рекурентно уравнение:

  $$T(n) = 2T(n-1) + 1$$

  Както вече знаем, решението е $T(n) = \Theta(2^n)$. Сега помислете, къде е грешката в следното доказателство, което "доказва", че $T(n) = O(n)$. Ще "докажем", че $T(n) \leqslant cn$ за някаква константа c. Индуктивното предположение е, че $T(n-1) \leqslant c(n-1)$. Тогава $T(n) \leqslant 2c(n-1) + 1$. Тогава $T(n) \leqslant 2cn - 2c + 1$. Но последният израз казва, че $T(n)$ е ограничена отгоре от константа, умножена по n. Тогава $T(n) = O(n)$, което и искахме да докажем.

  Къде е грешката? Грешката е в това, че доказателството не се извършва по отношение на константата, с която е започнато. Твърдението, което доказваме е, че **съществува положителна константа c и стойност на аргумента n_0, такива че за всяко $n \geqslant n_0$, $T(n) \leqslant cn$.** Естествено, това твърдение е лъжа, така че няма как да го докажем, но важното е, че **това** е твърдението, което доказваме. "$T(n) = O(n)$" е просто кратък запис за него. Щом избраният символ за константата е c, доказателството трябва да бъде извършено по отношение на него. $2c$ е **друга** константа. Наистина, $2c$ е константа, но за да имаме доказателство, трябва да го направим по отношение на c.

- Второ, понякога се налага да се засили индукционното предположение. Ще докажем по индукция, че решението на

  $$T(n) = 2T(n-1) + n \tag{5.40}$$

  е $T(n) = \Theta(2^n)$.

  Първо ще докажем, че $T(n) = O(2^n)$. Нарочно започваме доказателството с няколко неуспешни опита.

  Първи опит. Допускаме, че съществува константа c, такава че за всички достатъчно

големи n е вярно, че

$$T(n) \leqslant c2^n$$

От индуктивното предположение имаме

$$\begin{aligned} T(n) &\leqslant 2c2^{n-1} + n \\ &= c2^n + n \\ &\nleqslant c2^n \text{ за никоя положителна } c \end{aligned}$$

Доказателството се провали. Това не означава, че твърдението, което доказваме, непременно е грешно. Провал на доказателство **може** да означава, че твърдението не е вярно, но **може** и да означава, че твърдението е вярно, но не може да се докаже по този начин. Ще се опитаме да докажем по-силно твърдение.

Втори опит. Допускаме, че съществуват положителни константи b и c, такива че за всички достатъчно големи n е вярно, че

$$T(n) \leqslant c2^n - b$$

От индуктивното предположение имаме

$$\begin{aligned} T(n) &\leqslant 2(c2^{n-1} - b) + n \\ &= c2^n - 2b + n \\ &\nleqslant c2^n - b \text{ за никои положителни } b, c \end{aligned}$$

Ще засилим още твърдението.

Трети опит. Допускаме, че съществуват положителни константи b и c, такива че за всички достатъчно големи n е вярно, че

$$T(n) \leqslant c2^{n-b}$$

От индуктивното предположение имаме

$$\begin{aligned} T(n) &\leqslant 2(c2^{n-b-1}) + n \\ &= c2^{n-b} + n \\ &\nleqslant c2^{n-b} \text{ за никои положителни } b, c \end{aligned}$$

Ще засилим още твърдението.

Четвърти опит. Допускаме, че съществува положителна константа c, такава че за всички достатъчно големи n е вярно, че

$$T(n) \leqslant c2^n - n$$

От индуктивното предположение имаме

$$\begin{aligned} T(n) &\leqslant 2(c2^{n-1} - (n-1)) + n \\ &= c2^n - n + 2 \\ &\nleqslant c2^n - n \text{ за никоя положителна } c \end{aligned}$$

Пети опит. Допускаме, че съществуват положителни константи b и c, такива че за всички достатъчно големи n е вярно, че

$$T(n) \leqslant c2^n - bn$$

От индуктивното предположение имаме

$$\begin{aligned} T(n) &\leqslant 2(c2^{n-1} - b(n-1)) + n \\ &= c2^n - 2bn + 2b + n \\ &= c2^n - bn + (1-b)n + 2b \\ &\leqslant c2^n - bn \text{ за всеки избор на } c > 0 \text{ и } b > 1 \end{aligned}$$

Успех! Чак сега успяхме да формулираме индукционно предположение, което е доказуемо.

Сега ще докажем, че $T(n) = \Omega(n)$. С други думи, че съществува положителна константа d, такава че за всички достатъчно големи n е изпълнено

$$T(n) \geqslant d2^n$$

От индуктивното предположение имаме

$$\begin{aligned} T(n) &\geqslant 2(d2^{n-1}) + n \\ &= d2^n + n \\ &\geqslant d2^n \end{aligned}$$

Успех! Видяхме, че в този случай—а това е вярно по принцип при доказателства по индукция на решения на рекурентни уравнения—засилване на индукционното предположение се иска само за едното от O и Ω доказателствата.

Следната задача ползва термина *целочислено разбиване*, на английски *integer partition*.

Определение 37: Целочислено разбиване

Целочислено разбиване на цяло положително число k е изреждане на цели положителни числа в ненарастващ ред, които се сумират до k.

Това, че събираемите да се изреждат в ненарастващ ред, е само за по-лесно възприемане; формално, няма наредба между тях, така че две изреждания на едни и същи събираеми са едно и също разбиване; за да бъдат различни две разбивания, трябва мултимножествата от събираемите да са различни. Записът става със знака "+" между дяловете на разбиването, а не с нотация за мултимножество. Като пример, числото 4 има пет целочислени разбивания:

$$4 = 4$$
$$4 = 3 + 1$$
$$4 = 2 + 2$$
$$4 = 2 + 1 + 1$$
$$4 = 1 + 1 + 1 + 1$$

В Задача 12 доказваме само O за $T(n)$; това, че $T(n) = \Omega(n)$, е доста очевидно. А "m"

не означава брой на ребра—за каквито в тази задача не става дума—а е буква за броя на дяловете на някакво целочислено разбиване.

Задача 12

Докажете, че рекурентното уравнение

$$T(n) = \sum_{i=1}^{m} T(n_i) + \Theta(m)$$

за произволно целочислено разбиване $n_1+n_2+\cdots+n_m$ на $n-1$ има решение $T(n) = O(n)$.

Решение: С индукция по n. Да допуснем, че съществуват положителни константи b и c, такива че $T(n) \leqslant c \cdot n - b$. Да допуснем, че k е достатъчно голяма положителна константа; в частност, която е по-голяма или равна на скритата мултипликативна константа в израза "$\Theta(m)$". От индуктивното предположение:

$$\begin{aligned}
T(n) &\leqslant \sum_{i=1}^{m}(c \cdot n_i - b) + k \cdot m \\
&= c\left(\sum_{i=1}^{m} n_i\right) - b \cdot m + k \cdot m \qquad // \text{ но } n_1 + n_2 + \cdots + n_m = n - 1 \\
&= c(n-1) + m(k-b) \\
&= c \cdot n - c + m(k-b) \\
&\leqslant c \cdot n - b, \quad \text{ако } k - b < 0 \text{ и } c > b
\end{aligned}$$

□

Задача 12 е полезна при оценяване на сложността на алгоритми по схемите **Разделяй-и-Владей** или **Динамично Програмиране**, каквито ще видим в Подсекция 13.8.1 и Подсекция 13.8.2, които се изпълняват върху коренови дървета и при които сложността върху дърво с n върха е

- сумата от сложностите на виканията върху поддърветата, вкоренени в децата на корена—ние не знаем точно колко са тези деца, може да са от 1 до $n-1$ на брой, така че казваме, че има m деца, като $1 \leqslant m \leqslant n-1$—като тези деца имат $n_1, \ldots, n_m$ върха, като $n_1 + n_2 + \cdots + n_m$ се явява целочислено разбиване на $n-1$,
- плюс сложността на алгоритъма извън рекурсивните викания, за която допускаме, че е линейна в m.

Сложността извън рекурсивните викания е поне линейна в m, защото всяко от извършените m рекурсивни викания връща $\Omega(1)$ информация и всичко това трябва да бъде обработено от текущото рекурсивно викане, което дава долна граница $\Omega(m)$.

Допълнение 23: Няколко рекурентни уравнения по индукция

Следното нелеко за решаване рекурентно уравнение е от сборника на Parberry [74, pp. 40, Problem 248].

Задача 13

Решете рекурентното уравнение

$$T(n) = T\left(\left\lfloor \frac{n}{\lg n} \right\rfloor\right) + 1. \tag{5.41}$$

Решение: Уравнението е с единствена поява вдясно, поради което решението му, в асимптотичния смисъл, е броят пъти, който се изпълнява итераторът

$$n \to \left\lfloor \frac{n}{\lg n} \right\rfloor \tag{5.42}$$

така че n да стане $\Theta(1)$. Но аргументът вдясно намалява по сложен начин и изглежда невероятно да забележим закономерност, ако развиваме и развиваме.
Да опитаме да оценим броя на итерираният на око. Да разгледаме друго рекурентно уравнение $S(n)$, в което аргументът винаги бива делен на едно и също m: $S(n) = S\left(\left\lfloor \frac{n}{m} \right\rfloor\right)+1$. Ако m е константа, ние знаем от Следствие 17, че решението е $S(n) = \log_m n$. Да видим какво става, когато m не е константа. Дори тогава можем да приложим Следствие 17, стига m да е едно и също нещо при всяко итериране; тоест, винаги да делим на едно и също нещо. С други думи, ако $m = \lg N$, където N е **първоначалната стойност на** n и m не се променя при итерирането, то решението очевидно е

$$S(n) = \Theta(\log_{\lg n} n) = \Theta\left(\frac{\lg n}{\lg\lg n}\right)$$

Обаче в (5.42), n намалява при итерирането, така че и $\lg n$ намалява, така че делим на все по и по-малко нещо, което клони към константа.
От тези съображения можем да сложим следните граници за $T(n)$:

$$\frac{\lg n}{\lg\lg n} \preceq T(n) \preceq \lg n \tag{5.43}$$

Това вече е нещо. $\lg\lg n$ е доста близо до константа (макар да е неограничено растяща функция), така че (5.43) ограничава значително възможностите за $T(n)$, въпреки че не дава точно Тета-асимптотиката. Звучи невероятно $T(n) \asymp \lg n$, защото $\lg n$ е асимптотиката при деление само на константа. Да опитаме другата крайност: $T(n) \asymp \frac{\lg n}{\lg\lg n}$.

Ще покажем по индукция по n, че $T(n) \asymp \frac{\lg n}{\lg\lg n}$. За простота изпускаме нотацията "$\lfloor\ \rfloor$" от (5.41) и $T(n)$ се определя чрез

$$T(n) = T\left(\frac{n}{\lg n}\right) + 1 \tag{5.44}$$

Част i: Ще докажем, че $T(n) = O\left(\frac{\lg n}{\lg\lg n}\right)$. Тоест, че има положителна константа c и стойност на аргумента n_0, такива че за всяко $n \geqslant n_0$,

$$T(n) \leqslant c\frac{\lg n}{\lg\lg n} \tag{5.45}$$

Индуктивното предположение е

$$T\left(\frac{n}{\lg n}\right) \leqslant c\frac{\lg\left(\frac{n}{\lg n}\right)}{\lg\lg\left(\frac{n}{\lg n}\right)} \tag{5.46}$$

Заместваики (5.46) в (5.44), получаваме

$$\begin{aligned} T(n) &\leqslant c\frac{\lg\left(\frac{n}{\lg n}\right)}{\lg\lg\left(\frac{n}{\lg n}\right)} + 1 \\ &= c\frac{\lg n - \lg\lg n}{\lg\left(\lg n - \lg\lg n\right)} + 1 \end{aligned} \tag{5.47}$$

Заместваме $\lg n$ с 2^m, така че заместваме $\lg\lg n$ с m, в (5.47) и получаваме

$$\begin{aligned} &c\frac{2^m - m}{\lg\left(2^m - m\right)} + 1 \leqslant c\frac{2^m - m}{\lg 1.9^m} + 1 = c\frac{2^m - m}{m\lg 1.9} + 1 = \frac{c}{1.9}\frac{2^m - m}{m} + 1 = \\ &\frac{c}{1.9}\frac{2^m}{m} - \frac{c}{1.9} + 1 \leqslant c\frac{2^m}{m} = c\frac{\lg n}{\lg\lg n} \end{aligned} \tag{5.48}$$

От (5.47) и (5.48) заключаваме, че

$$T(n) \leqslant c\frac{\lg n}{\lg\lg n}$$

за всяко положително c, да кажем $c = 1$, и всяко достатъчно голямо n.

Част ii: Ще докажем, че $T(n) = \Omega\left(\frac{\lg n}{\lg\lg n}\right)$. Тоест, че има положителна константа d и стойност на аргумента n_0, такива че за всяко $n \geqslant n_0$,

$$T(n) \geqslant d\frac{\lg n}{\lg\lg n} \tag{5.49}$$

Индуктивното предположение е

$$T\left(\frac{n}{\lg n}\right) \geqslant d\frac{\lg\left(\frac{n}{\lg n}\right)}{\lg\lg\left(\frac{n}{\lg n}\right)} \tag{5.50}$$

Заместваики (5.50) в (5.44), получаваме

$$\begin{aligned} T(n) &\geqslant d\frac{\lg\left(\frac{n}{\lg n}\right)}{\lg\lg\left(\frac{n}{\lg n}\right)} + 1 \\ &= d\frac{\lg n - \lg\lg n}{\lg\left(\lg n - \lg\lg n\right)} + 1 \end{aligned} \tag{5.51}$$

Заместваме $\lg n$ с 2^m, така че заместваме $\lg\lg n$ с m, в (5.51) и получаваме

$$d\frac{2^m - m}{\lg(2^m - m)} + 1 \geqslant d\frac{2^m - m}{\lg 2^m} + 1 = d\frac{2^m - m}{m} + 1 = d\frac{2^m}{m} - d + 1 \geqslant d\frac{2^m}{m} \tag{5.52}$$

Това е вярно за, да кажем, $d = 1$. Връщаме се към променливата n. От (5.51) и (5.52) виждаме, че

$$T(n) \geqslant d\frac{\lg n}{\lg\lg n}$$

за $d = 1$ и всяко достатъчно голямо n. □

Ние вече решихме $T(n) = T(n-2) + 2\lg n$ в Задача 6, но това беше неформално решение чрез развиване. Сега ще покажем формално вярността на решението.

Задача 14

Докажете, че рекурентното уравнение

$$T(n) = T(n-2) + 2\lg n$$

има решение $T(n) \asymp n\lg n$.

Решение: С индукция по n.

Part i: Ще докажем, че $T(n) = O(n\lg n)$. Тоест, че има положителна константа c, такава че за всяко достатъчно голямо n,

$$T(n) \leqslant cn\lg n \tag{5.53}$$

В сила е

$$\begin{aligned} T(n) &\leqslant c(n-2)\lg(n-2) + 2\lg n \qquad // \text{ от индукционното предположение} \\ &\leqslant c(n-2)\lg n + 2\lg n \\ &= cn\lg n - 2c\lg n + 2\lg n \\ &\leqslant cn\lg n, \text{ ако } -2c\lg n + 2\lg n \leqslant 0 \leftrightarrow c \geqslant 1 \end{aligned}$$

Part ii: Ще докажем, че $T(n) = \Omega(n\lg n)$. Тоест, че има положителна константа d, такава че за всяко достатъчно голямо n,

$$T(n) \geqslant dn\lg n \tag{5.54}$$

В сила е

$$\begin{aligned} T(n) &\geqslant d(n-2)\lg(n-2) + 2\lg n \qquad // \text{ от индукционното предположение} \\ &= (dn - 2d)\lg(n-2) + 2\lg n \\ &= dn\lg(n-2) + 2(\lg n - d\lg(n-2)) \end{aligned} \tag{5.55}$$

Предвид (5.54) and (5.55), целта ни е да покажем, че

$$\begin{aligned} dn\lg(n-2)+2(\lg n-d\lg(n-2)) &\geqslant dn\lg n \;\leftrightarrow\\ dn\lg(n-2)-dn\lg n+2(\lg n-d\lg(n-2)) &\geqslant 0 \;\leftrightarrow\\ \underbrace{d\lg\left(\frac{n-2}{n}\right)^n}_{A}+\underbrace{2\lg\frac{n}{(n-2)^d}}_{B} &\geqslant 0 \end{aligned} \tag{5.56}$$

Първо ще оценим A при неограничено нарастване на n:

$$\lim_{n\to\infty} d\lg\left(\frac{n-2}{n}\right)^n = d\lim_{n\to\infty}\lg\left(1+\frac{-2}{n}\right)^n = d\lg\lim_{n\to\infty}\left(1+\frac{-2}{n}\right)^n = d\lg e^{-2} = -2d\lg e$$

Да разгледаме B при неограничено нарастване на n:

$$\lim_{n\to\infty} 2\lg\frac{n}{(n-2)^d} = 2\lg\lim_{n\to\infty}\frac{n}{(n-2)^d} \tag{5.57}$$

Забележете, че при всяко $d\in(0,1)$, (5.57) е $+\infty$. Например, за $d=\frac{1}{2}$, (5.57) става

$$\begin{aligned} &2\lg\lim_{n\to\infty}\left(n^{\frac{1}{2}}\frac{n^{\frac{1}{2}}}{(n-2)^{\frac{1}{2}}}\right) =\\ &2\lg\lim_{n\to\infty}\left(n^{\frac{1}{2}}\left(\frac{n}{n-2}\right)^{\frac{1}{2}}\right) =\\ &2\lg\left(\left(\lim_{n\to\infty}n^{\frac{1}{2}}\right)\underbrace{\left(\lim_{n\to\infty}\left(\frac{1}{1-\frac{2}{n}}\right)^{\frac{1}{2}}\right)}_{1}\right) = +\infty \end{aligned}$$

Следва, че (5.56) е вярно за всеки избор на d от $(0,1)$; примерно, $d=\frac{1}{2}$, понеже A по абсолютна стойност е ограничена от константа, а B расте неограничено. □

Наблюдение 21

Доказателството по индукция в **Част ii** е триково. Разгледайте (5.55):

$$dn\lg(n-2)+2(\lg n-d\lg(n-2))$$

Обикновено се "справяме" с логаритмите на суми или разлики като апроксимираме сумите или разликите с произведения или частни по такъв начин, че неравенството да се запази в желаната посока. Но в този случай, ако апроксимираме $n-2$ вътре в този логаритъм с αn, където α е положителна константа, трябва да е вярно, че $\alpha<1$, за да е изпълнено $\alpha n<n$. Ето какво става, ако заместим $n-2$ с αn в логаритъма отляво:

$$dn\lg\alpha n+2(\lg n-d\lg(n-2))=dn\lg n+dn\lg\alpha+2(\lg n-d\lg(n-2))$$

За да довършим доказателството, трябва да покажем, че последното е по-голямо или равно на $dn\lg n$. А за да покажем това, трябва да покажем, че събираемото $dn\lg\alpha+2(\lg n-d\lg(n-2))$ е положително. Но това не е вярно! Тъй като $d>0$ и $\alpha<1$, то $dn\lg\alpha<0$ за всяко $n>0$. И асимптотичният характер на $dn\lg\alpha+2(\lg n-d\lg(n-2))$ се определя от $dn\lg\alpha$, понеже линейната функция доминира над логаритмичната за всяко достатъчно голямо n.
И така, ако заместим n с αn, няма да получим валидно доказателство. Налага се да използваме по-изтънчена техника, базирана на математическия анализ.

5.3.4 Чрез Мастър теоремата (MT)

Този теоретичен резултат на Jon Louis Bentley, Dorothea Haken и James Saxe е от 1978 г. [11]. Преди това, рекурентни уравнения, в които функцията на намаляване е деление, са били решавани чрез *ad hoc* методи. Например, в класическия учебник по алгоритми на Aho, Hopcroft и Ullman [2] от 1974 г. има следната теорема.

Теорема 33: Теорема за решение на клас рекурентни уравнения (Theorem 2.1 в [2])

Let a, b, and c be nonnegative constants. The solution to the recurrence

$$T(n)=\begin{cases}b, & \text{for } n=1\\ aT(n/c)+bn, & \text{for } n>1\end{cases}$$

for n a power of c is

$$T(n)=\begin{cases}O(n), & \text{if } a<c,\\ O(n\lg n), & \text{if } a=c,\\ n^{\log_c a}, & \text{if } a>c.\end{cases}$$

Забележете, че авторите на [11] не говорят за "Master theorem". Гръмкото име "**Master** theorem", доколкото е известно на автора на тези лекционни записки, се дължи на Cormen,

Leiserson и Rivest – авторите на [21]. Без съмнение то идва оттам, че въпросната теорема се явява обобщение на множество теореми като Теорема 33, даващи решения на рекурентни уравнения, в които функцията на намаляване е деление.

От друга страна, няма най-обща теорема, която да дава универсален метод за решаване на всякакви рекурентни уравнение. По отношение на линейните рекурентни уравнения с константни коефициенти, при които намаляването е чрез деление с константа, каквито уравнения третира Теорема 34, е известна значително по-мощна и обща Теорема на Akra-Bazzi [4] (Подсекция 5.3.5). И така, Теорема 34, въпреки името си, е просто едно средство, като са известни много по-мощни средства от него.

Следното изложение е по учебника на Cormen и др. [21, стр. 61].

Теорема 34: Мастър теорема (Theorem 4.1 от [21, стр. 94])

Нека $a \geqslant 1$ и $b > 1$ са константи и нека $f(n)$ е положителна функция. Нека

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \tag{5.58}$$

където $\frac{n}{b}$ има смисъл или на $\left\lfloor \frac{n}{b} \right\rfloor$, или на $\left\lceil \frac{n}{b} \right\rceil$. Тогава асимптотиката на $T(n)$ е следната:

Случай 1 Ако $f(n) = O\left(n^{\log_b a}/n^{\epsilon}\right)$ за някоя положителна константа ϵ, тогава $T(n) = \Theta(n^{\log_b a})$.

Случай 2 Ако $f(n) = \Theta(n^{\log_b a})$, тогава $T(n) = \Theta\left(n^{\log_b a}.\lg n\right)$. С други думи, $T(n) = \Theta(f(n).\lg n)$.

Случай 3 Ако са изпълнени следните условия:

1. $f(n) = \Omega\left(n^{\log_b a}.n^{\epsilon}\right)$ за някоя положителна константа ϵ, и
2. $a.f\left(\frac{n}{b}\right) \leqslant c.f(n)$ за някоя положителна константа c, такава че $0 < c < 1$ за всички достатъчно големи n,

то $T(n) = \Theta(f(n))$. □

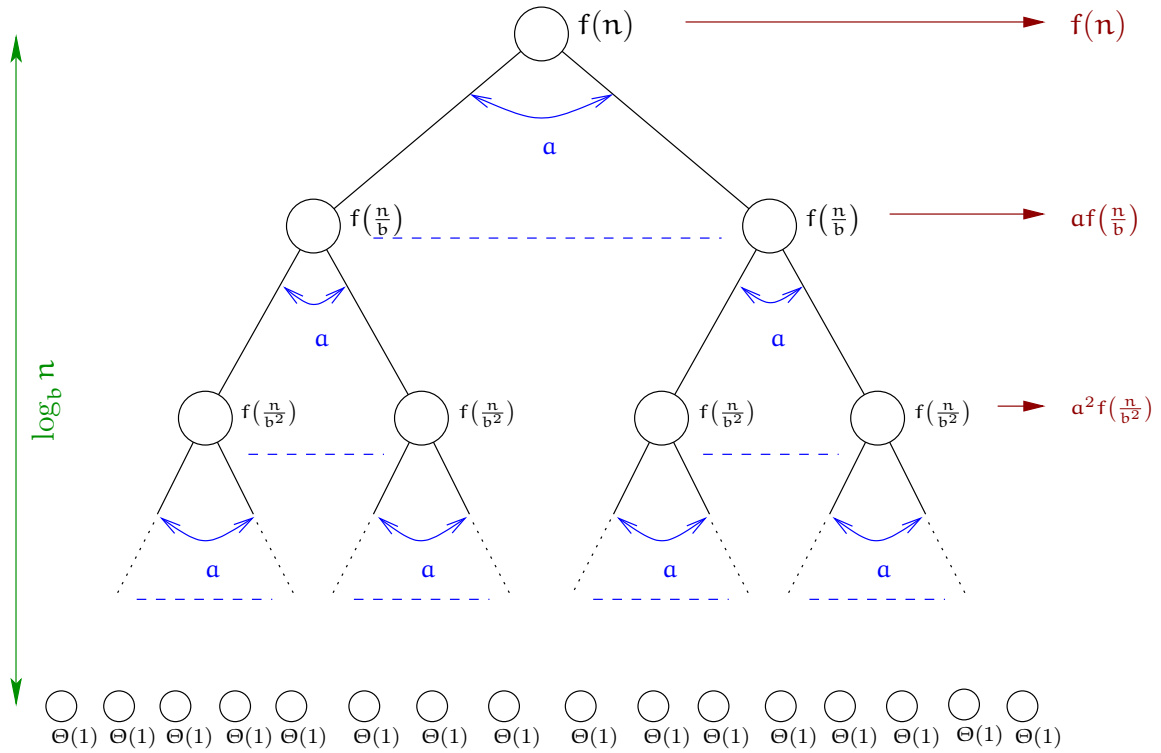
Накратко ще наричаме тази теорема "MT".

Условие 3.2 се нарича *условие за регулярност*.

Няма да даваме доказателство на MT. Доказателство, базирано на метода с характеристичното уравнение (Подсекция 5.3.6) на **ограничен вариант** на MT има в Допълнение 26. Читателят може да види пълно доказателство на MT в [21]. Доказателство има и в статията [59]. Тук само ще дадем няколко пояснения.

Пояснение 1 И в трите случая на MT сравняваме броя на листата на дървото на рекурсията с функцията $f(n)$. Да се убедим, че броят на листата на дървото наистина е $n^{\log_b a}$. Нека да мислим, че a и b са цели числа[†]. Ако a и b са цели положителни числа, като $b \geqslant 2$, то (5.58) отговаря на рекурсивен алгоритъм, направен по схемата **Разделяй-и-Владей**, който прави a рекурсивни извиквания, всяко върху вход с големина $\frac{n}{b}$, и освен това извършва $f(n)$ работа (като време) по разделянето на входа и комбинирането на резултатите от извикванията. Дървото на този алгоритъм е показано на Фигура 5.6. Всеки връх на дървото има

[†]В [21] е доказано, че същите съображения остават в сила дори когато a и b не са цели.



Фигура 5.6: Дървото на рекурсията на алгоритъм, чиято сложност по време се описва от $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, където a и b са цели положителни числа.

разклоненост a, и входът намалява с деление на b. Сумите по нивата на дървото на нехомогенните части са $f(n)$, $af\left(\frac{n}{b}\right)$, $a^2f\left(\frac{n}{b^2}\right)$, и така нататък. Височината на дървото е $\log_b n$. Броят на върховете от ниво[†] k очевидно е a^k, така че броят на листата е е $a^{\log_b n}$. Лесно се вижда, че $a^{\log_b n} = n^{\log_b a}$.

Броят на листата е пропорционален на големината на дървото, освен ако a не е 1, така че големината на дървото е $a^{\log_b n}$; при $a = 1$ има точно едно листо, а височината продължава да е $\log_b n$.

Наблюдение 22

Ако $a > 1$, МТ сравнява големината на дървото с нехомогенната част $f(n)$.

В първия случай на МТ големината на дървото доминира над $f(n)$, така че $T(n)$ е големината на дървото, в асимптотитичния смисъл. Във втория случай големината на дървото и $f(n)$ са асимптотично еквивалентни. В третия случай $f(n)$ доминира над големината на дървото, така че $f(n)$ дава директно асимптотиката, ако е изпълнено и условието за регулярност.

[†]Припомняме си, че върховете от ниво k са точно върховете на разстояние k от корена.

Наблюдение 23

Сравняването на големината на дървото с нехомогенната част има смисъл и при други видове рекурентни уравнения. Например, в Задача 9 големината на дървото е "факториелна", докато нехомогенната част е константа, и съвсем естествено решението е, в асимптотичния смисъл, същото като големината на дървото, а именно $\Theta(n!)$. В някакъв смисъл, това е аналог на първия случай на МТ.
В Задача 10 нещата са аналогични: големината на дървото пак е "факториелна", докато нехомогенната част е само експоненциална; тъй като експоненциалната функция "бледнее" пред факториела, отново големината на дървото е, в асимптотичния смисъл, същата като решението, а именно $\Theta(n!)$. И това е аналогично на първия случай на МТ.

В Задача 11 обаче големината на дървото е експоненциална, докато нехомогенната част е $n!$ и последната доминира над големината на дървото. В този случай асимптотиката на нехомогенната част е решението, а именно $n!$. В някакъв смисъл, това е аналог на третия случай на МТ.

Решението на $T(n) = 2T(n-1) + 2^n$ е $T(n) \asymp n2^n$, което се извежда елементарно с метода с характеристичното уравнение (Подсекция 5.3.6). Тук и големината на дървото, и нехомогенната част са $\Theta(2^n)$, така че решението $\Theta(n2^n)$ в някакъв смисъл е аналог на втория случай на МТ. И във втория случай на МТ, и тук решението е, асимптотично, произведението от големината на дървото и неговия логаритъм.

Пояснение 2 Трите случаи на МТ **не са разбиване** на всички възможни случаи. Както вече видяхме в Теорема 12(2.38), има функции, които не са асимптотично сравними, така че ако $n^{\log_b a}$ и $f(n)$ не са асимптотично сравними, то МТ не е приложима.

Но дори когато $n^{\log_b a}$ и $f(n)$ са асимптотично сравними, има примери за a, b и $f(n)$, в които $f(n)$ расте прекалено бързо, за да попаднем в първия случай, но прекалено бавно, за да попаднем във втория, и МТ пак не е приложима. Неформално казано, между първия и втория случай на МТ има "празнина" и ако конкретната задача е в някоя от тези "празнини", не можем да използваме МТ.

Например, нека $a = b = 2$ и $f(n) = \frac{n}{\lg n}$. Тогава $f(n) \neq O\left(n^{\log_b a}/n^{\epsilon}\right)$ за всяка положителна константа ϵ, така че рекурентното уравнение не попада в първия случай. От друга страна, $f(n) \neq \Theta(n^{\log_b a})$, така че не сме и във втория случай. Бидейки попаднали в "празнината" между първия и втория случай, налага се да ползваме други методи, за да решим задачата.

Допълнение 24: Решение на $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n}$

Ще решим $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n}$. Първо да опитаме с Теорема 34. Използвайки терминологията на Теорема 34, $a = b = 2$, следователно $\log_b a$ е $\log_2 2 = 1$, следователно $n^{\log_b a}$ е $n^1 = n$. Функцията $f(n)$ е $\frac{n}{\lg n}$. Да видим дали може да класифицираме рекурентното уравнение в един от трите случая.

Случай 1 Дали $\frac{n}{\lg n} = O(n^1/n^{\epsilon})$ за някоя константа $\epsilon > 0$? Не, защото $\forall \epsilon > 0 : n^{\epsilon} \neq O(\lg n)$. Детайлно доказателство на този факт има в сборника с решени задачи по алгоритми.

Случай 2 Дали $\frac{n}{\lg n} = \Theta(n^1)$? Не, защото $\frac{n}{\lg n} = o(n^1)$.

Случай 3 Дали $\frac{n}{\lg n} = \Omega(n^1.n^\epsilon)$ за някоя константа $\epsilon > 0$? Не, и то по същата причина: защото $\frac{n}{\lg n} = o(n^1)$.

Тогава тази задача не може да бъде решена с Теорема 34. Нещо повече: не можем да ползваме и Теорема 35 на стр. 238, защото $\forall t > 0 : \frac{n}{\lg n} \neq \Theta(n^{\log_2 2} \times \lg^t(n))$.
Ще решим задачата с развиване:

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n} \\
&= 2\left(2T\left(\frac{n}{4}\right) + \frac{\frac{n}{2}}{\lg\frac{n}{2}}\right) + \frac{n}{\lg n} \\
&= 4T\left(\frac{n}{4}\right) + \frac{n}{(\lg n) - 1} + \frac{n}{\lg n} \\
&= 4\left(2T\left(\frac{n}{8}\right) + \frac{\frac{n}{4}}{\lg\frac{n}{4}}\right) + \frac{n}{(\lg n) - 1} + \frac{n}{\lg n} \\
&= 8T\left(\frac{n}{8}\right) + \frac{n}{(\lg n) - 2} + \frac{n}{(\lg n) - 1} + \frac{n}{\lg n} \\
&\ldots \\
&= nT(1) + \frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \cdots + \frac{n}{(\lg n) - 2} + \frac{n}{(\lg n) - 1} + \frac{n}{\lg n} \\
&= nT(1) + \underbrace{n\left(\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{(\lg n) - 2} + \frac{1}{(\lg n) - 1} + \frac{1}{\lg n}\right)}_{B}
\end{aligned}$$

Забелязваме, че $B = n \times H_{\lg n}$, защото в скобите се намира $(\lg n)^{\text{тата}}$ парциална сума на хармоничния ред. Добре известно е, че $H_m \asymp \lg m$, следователно $H_{\lg n} = \Theta(\lg\lg n)$, и оттам $B = \Theta(n \lg\lg n)$. От това следва, че $T(n) = \Theta(n \lg\lg n)$.

Аналогично, има примери за a, b и $f(n)$, в които $f(n)$ расте прекалено бързо, за да попаднем във втория случай, но прекалено бавно, за да попаднем в третия. С други думи, между втория и третия случай също има "празнина", за която МТ не е приложима.

Пояснение 3 Условието $f(n) = O\left(n^{\log_b a}/n^\epsilon\right)$ в първия случай на МТ е по-силно от $f(n) = o\left(n^{\log_b a}\right)$, а условието $f(n) = \Omega\left(n^{\log_b a}.n^\epsilon\right)$ в третия случай е по-силно от $f(n) = \omega\left(n^{\log_b a}\right)$. За удобство да направим следните дефиниции.

Определение 38: асимптотично строго по-бавно/по-бързо и полиномиално по-бавно/по-бързо

Ако $\phi(n) = o(\psi(n))$, казваме, че $\phi(n)$ *расте асимптотично строго по-бавно от* $\psi(n)$.
Ако $\phi(n) = O(\psi(n)/n^\epsilon)$ за някаква положителна константа ϵ, казваме, че $\phi(n)$ расте *полиномиално по-бавно от* $\phi(n)$.
Ако $\phi(n) = \omega(\psi(n))$, казваме, че $\phi(n)$ *расте асимптотично строго по-бързо от* $\psi(n)$.
Ако $\phi(n) = O(\psi(n).n^\epsilon)$ за някаква положителна константа ϵ, казваме, че $\phi(n)$ расте *полиномиално по-бързо от* $\phi(n)$.

$\boxed{f(n) = O\left(n^{\log_b a}/n^\epsilon\right)}$, където ϵ е положителна константа, казва, че между $f(n)$ и $n^{\log_b a}$ може да бъде "вкарано" n^ϵ, като $f(n)$ е отдолу; с други думи, че $f(n)$ "изостава" от $n^{\log_b a}$ на една

полиномиално растяща функция. Това е значително по-силно от $f(n)$ да "изостава" от $n^{\log_b a}$ на повече от константа, което е $\boxed{f(n) = o\left(n^{\log_b a}\right)}$. Аналогично, $\boxed{f(n) = \Omega\left(n^{\log_b a}.n^{\epsilon}\right)}$ казва, че между $f(n)$ и $n^{\log_b a}$ може да бъде "вкарано" n^{ϵ}, като $f(n)$ е отгоре; с други думи, че $f(n)$ "изпреварва" $n^{\log_b a}$ с една полиномиално растяща функция. Това е значително по-силно от $f(n)$ да "изпреварва" $n^{\log_b a}$ на повече от константа, което е $\boxed{f(n) = \omega\left(n^{\log_b a}\right)}$.

Наблюдение 24

Ако $\phi(n)$ расте полиномиално по-бавно от $\psi(n)$, то $\phi(n)$ расте асимптотично строго по-бавно от $\psi(n)$. Конверсното не е вярно. Следователно, условието за полиномиално по-бавно нарастване е по-силно от условието за асимптотично строго по-бавно нарастване. Ако $\phi(n)$ расте полиномиално по-бързо от $\psi(n)$, то $\phi(n)$ расте асимптотично строго по-бързо от $\psi(n)$. Конверсното не е вярно. Следователно, условието за полиномиално по-бързо нарастване е по-силно от условието за асимптотично строго по-бързо нарастване.

Наблюдение 24 ни дава право да кажем следното.

$$\begin{aligned} f(n) = O(n^k/n^{\epsilon}) &\rightarrow f(n) = o(n^k) \\ f(n) = o(n^k) &\nrightarrow f(n) = O(n^k/n^{\epsilon}) \\ f(n) = \Omega(n^k.n^{\epsilon}) &\rightarrow f(n) = \omega(n^k) \\ f(n) = \omega(n^k) &\nrightarrow f(n) = \Omega(n^k.n^{\epsilon}) \end{aligned}$$

Наблюдение 25

Би било грешка да се опитаме да формулираме първия случай на MT чрез $\boxed{f(n) = o\left(n^{\log_b a}\right)}$ или третия случай чрез $\boxed{f(n) = \omega\left(n^{\log_b a}\right)}$.

Пояснение 4 Условието за регулярност в третия случай е по-силно от условието $f(n) = \Omega\left(n^{\log_b a+\epsilon}\right)$ в смисъл, че

$$\exists c, 0 < c < 1 : \left(a.f\left(\frac{n}{b}\right) \leqslant c.f(n)\right) \rightarrow \exists \epsilon \left(f(n) = \Omega\left(n^{\log_b a+\epsilon}\right)\right) \tag{5.59}$$

$$\exists \epsilon \left(f(n) = \Omega\left(n^{\log_b a+\epsilon}\right)\right) \nrightarrow \exists c, 0 < c < 1 : \left(a.f\left(\frac{n}{b}\right) \leqslant c.f(n)\right) \tag{5.60}$$

Пример за $f(n)$ като в (5.60) статията в Wikipedia за MT: да вземем $f(n) = n(2 - \cos n)$. Доказателство на (5.59) се съдържа в Допълнение 25.

Допълнение 25: Условието за регулярност в MT влече Случай 3

Задача 4.4-3 в [21] е: покажете, че Случай 3 на MT съдържа излишък в смисъл, че условието за регулярност $af\left(\frac{n}{b}\right) \leqslant cf(n)$ за някаква положителна константа $c < 1$ влече съществуването на константа $\epsilon > 0$, такава че $f(n) = \Omega(n^{\log_b a+\epsilon})$. Сега ще покажем точно това. Нека съществува константа c, такава че $0 < c < 1$ и:

$$f(n) \geqslant \frac{a}{c} f\left(\frac{n}{b}\right)$$

Тъй като $a \geqslant 1$ и $c < 1$, то $\frac{a}{c} > 1$. За краткост, ще означаваме $\frac{a}{c}$ с буквата s. Тогава в сила е:

$$f(n) \geqslant sf\left(\frac{n}{b}\right)$$

Тогава

$$\begin{aligned} f(n) &\geqslant s^2 f\left(\frac{n}{b^2}\right) \\ f(n) &\geqslant s^3 f\left(\frac{n}{b^3}\right) \\ &\dots \\ f(n) &\geqslant s^t f\left(\frac{n}{b^t}\right) \end{aligned} \tag{5.61}$$

Съществува някаква стойност n_0 на променливата n, за която този процес ще спре. Тогава $\frac{n}{b^t} = n_0$, следователно $t = \log_b\left(\frac{n}{n_0}\right) = \log_b n - \log_b n_0$. Заместваме t и $\frac{n}{b^t}$ и c в (5.61) и получаваме:

$$f(n) \geqslant s^{\log_b n - \log_b n_0} \times f(n_0) = \frac{s^{\log_b n}}{s^{\log_b n_0}} \times f(n_0)$$

Забележете, че $\frac{1}{s^{\log_b n_0}} \times f(n_0)$ е константа. За краткост, да наречем тази константа β. Тогава:

$$f(n) \geqslant s^{\log_b n} \times \beta$$

Да се върнем към началните имена a и c, като полагаме $z = \frac{1}{c}$, за да е изпълнено $z > 1$. Имаме $s = a \times z$.

$$f(n) \geqslant a^{\log_b n} \times z^{\log_b n} \times \beta$$

Имайки предвид, че $a^{\log_b n} = n^{\log_b a}$ and $z^{\log_b n} = n^{\log_b z}$, извеждаме:

$$f(n) \geqslant \beta \times n^{\log_b a} \times n^{\log_b z}$$

Тъй като $z > 1$, то $\log_b z > 0$. Нека $\epsilon = \log_b z$. Извеждаме желания резултат: за всички достатъчно големи n и някаква константа β:

$$f(n) \geqslant \beta n^{\log_b a + \epsilon} \Rightarrow f(n) = \Omega(n^{\log_b a + \epsilon})$$

□

Следователно, третият случай на MT би могъл да бъде формулиран само с условието за регулярност. Условието $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ е излишно. Дадената формулировка е избрана, без съмнение, за да има външна аналогия с формулировките на първите два случая. Това е и краят на поясненията върху MT.

Допълнение 26: Ограничена МТ следва от метода с хар-чното у-ние

Ще покажем, че МТ, но в ограничен вариант, е следствие на метода с характеристичното уравнение (Подсекция 5.3.6). Разглеждаме

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Ще направим субституция на n с b^m. Тоест, $m = \log_b n$.

$$\begin{aligned} T(n) &= aT\left(\frac{n}{b}\right) + f(n) \leftrightarrow \\ T(b^m) &= aT\left(\frac{b^m}{b}\right) + f(b^m) \leftrightarrow \\ T(b^m) &= aT\left(b^{m-1}\right) + f(b^m) \end{aligned}$$

Ще направим две субституции на функции.

- Ще заменим $T(b^m)$ с някаква $S(m)$. Имаме право да го направим, понеже $T(b^m)$ е функция на m, при положение, че b е константа. Щом $T(b^m)$ става $S(m)$, то $T(b^{m-1})$ става $S(m-1)$.
- С аналогични съображения, заменяме $f(b^m)$ с някаква $g(m)$.

Вече ползвахме тази техника (Лема 34 и Следствие 16): при деление с някаква константа правим субституция с експонента с основа тази константа, при което делението става изваждане в степенния показател.

И така, получихме

$$S(m) = aS(m-1) + g(m)$$

Но това уравнение е решимо с метода с характеристичното уравнение (Подсекция 5.3.6), ако нехомогенната част $g(m)$ е от подходящ вид, а именно

$$g(m) = p_1(m)c_1^m + \cdots + p_t(m)c_t^m \tag{5.62}$$

където p_i са полиноми, а c_i са константи, такива че $i \neq j \rightarrow c_i \neq c_j$. Да допуснем, че (5.62) е изпълнено. Тогава решението е елементарно. Хомогенната част

$$S_{\text{хом}}(m) = aS_{\text{хом}}(m-1)$$

дава мултимножество от корени $\{a\}_M$. От нехомогенната част идва мултимножество от корените

$$\{\underbrace{c_1, \ldots, c_1}_{\deg(p_1)+1}, \ldots, \underbrace{c_t, \ldots, c_t}_{\deg(p_t)+1}\}_M$$

За определяне на асимптотиката на решението нас ни интересува само най-голямата константа. БОО, нека най-голямата константа е c_1 и нека $d = \deg(p_1) + 1$ е броят на срещанията на c_1 в мултимножеството от корените. Тогава, съгласно (5.78), решението е

$$S(m) \asymp \begin{cases} a^m, & \text{ако } a > c_1 \\ m^d \cdot a^m, & \text{ако } a = c_1 \\ m^{d-1} \cdot c_1^m. & \text{ако } a < c_1 \end{cases} \tag{5.63}$$

Основанието е ясно:

- ако $a > c_1$, то най-големият елемент на мултимножеството от корените е a и той има кратност 1 и асимптотиката се дава от $m^{1-1} \cdot a^m$, което е a^m;
- ако $a = c_1$, то най-големият елемент на мултимножеството от корените е $a = c_1$ и той има кратност $d+1$, а не d, така че асимптотиката се дава от $m^{(d+1)-1} \cdot a^m$, което е $m^d \cdot a^m$;
- ако $c_1 > a$, то най-големият елемент на мултимножеството от корените е c_1 и той има кратност d, така че асимптотиката на решението се дава от $m^{d-1} \cdot c_1^m$.

Забележете, че в последния случай, $d-1$ е равно на $\deg(p_1)$, а $m^{\deg(p_1)} \cdot c_1^m$ задава асимптотиката на нехомогенната част (помним, че c_1 е най-голямата от константите-основи на експоненти в нехомогенната част). Ерго, $m^{d-1} \cdot c_1^m \asymp g(m)$. Това ни дава основание да твърдим, че

$$S(m) \asymp \begin{cases} a^m, & \text{ако } a > c_1 \\ m^d \cdot a^m, & \text{ако } a = c_1 \\ g(m). & \text{ако } a < c_1 \end{cases} \tag{5.64}$$

Да се върнем към оригиналната променлива n и функция $T(n)$, помнейки, че $m = \log_b n$, $S(m) = T(n)$ и $g(m) = f(n)$:

$$T(n) \asymp \begin{cases} a^{\log_b n}, & \text{ако } a > c_1 \\ (\log_b n)^d \cdot a^{\log_b n}, & \text{ако } a = c_1 \\ f(n). & \text{ако } a < c_1 \end{cases} \tag{5.65}$$

Предвид това, че $a^{\log_b n} = n^{\log_b a}$, можем да твърдим, че

1. $T(n) \asymp n^{\log_b a}$, ако $a > c_1$,
2. $T(n) \asymp (\log_b n)^d \cdot n^{\log_b a}$, ако $a = c_1$,
3. $T(n) \asymp f(n)$, ако $a < c_1$.

Но това е същото като MT (Теорема 34), на която вторият случай е засилен чрез обобщението на MT (Теорема 35), но **при допускането**, че $f(n)$ е такава, че

$$f(n) = p_1(\log_b n) \cdot c_1^{\log_b n} + \cdots + p_t(\log_b n) \cdot c_t^{\log_b n} \tag{5.66}$$

С други думи, ако $f(n)$ сума от функции, всяка от които е произведение от полиномиална функция на n и полилогаритмична функция на n. За да се убедим, че (5.62) и (5.66) казват едно и също, да си припомним, че $m = \log_b n$ и $g(m) = f(b^m) = f(n)$.

Предвид това, че събираемото с най-голямата константа c_i задава асимптотиката на $f(n)$, веднага виждаме, че случаите на МТ (Теорема 34) точно отговарят на тези случаи: **Случай 1** отговаря на (1), **Случай 2** отговаря на (2) и **Случай 3** отговаря на (3). Условието за регуларност задължително е изпълнено, ако $f(n)$ е от вида (5.66), така че няма нужда да го споменаваме в (3).
Нещо повече. Дори $f(n)$ да е по-бързо растяща функция от това, което (5.66) казва—с други думи, ако $f(n)$ не е от вида (5.66), но е суперполиномиална—асимптотиката на $T(n)$ пак се дава от $f(n)$. Така че резултата от Допълнение 26 покрива всяка $f(n)$, която би се срещнала на практика в изследване на рекурсивни алгоритми, чиято сложност се описва от $T(n) = aT\left(\frac{n}{b}\right) + f(n)$.

В резюме, извеждането чрез метода с характеристичното уравнение на МТ е достатъчно добро за целите на курса. Функциите $f(n)$, които хем остават извън условието, наложено от (5.66), хем не са суперполиномиални, са екзотика, която едва ли ще се срещне при намиране на сложността на алгоритъм, който е полезен на практика, а не е чисто мисловна конструкция.

Ще видим няколко примера за прилагането на МТ.

Задача 15

Решете чрез МТ $T(n) = 3T\left(\frac{n}{4}\right) + n \lg n$.

Решение: Използвайки терминологията на МТ, a е 3, b е 4, така че $\log_b a$ е $\log_4 3$, което е приблизително 0.79, и $f(n)$ е $n \lg n$. Със сигурност е вярно, че $n \lg n = \Omega(n^{\log_4 3 + \epsilon})$ за някое $\epsilon > 0$, например $\epsilon = 0.1$. Но трябва да проверим и дали условието за регулярност е в сила, за да видим дали може да приложим третия случай на МТ. Условието за регулярност е:

$$\exists c \text{ такова че } 0 < c < 1 \text{ и } 3\frac{n}{4} \lg \frac{n}{4} \leqslant cn \lg n \text{ за всички достатъчно големи } n$$

Очевидно това е вярно за, да речем, $c = \frac{3}{4}$, така че третият случай на МТ е приложим. Съгласно него, $T(n) \asymp n \lg n$. □

Задача 16

Решете чрез МТ $T(n) = T\left(\frac{2n}{3}\right) + 1$.

Решение: Преписваме условието така:

$$T(n) = 1.T\left(\frac{n}{\frac{3}{2}}\right) + 1$$

за да е по-ясно какви са a и b. Виждаме, че a е 1, b е $\frac{3}{2}$, така че $\log_b a$ е $\log_{\frac{3}{2}} 1 = 0$, следователно $n^{\log_b a}$ е $n^0 = 1$. Нехомогенната част $f(n)$ е 1. Очевидно, $1 = \Theta(n^0)$, така че вторият случай на МТ е приложим. Съгласно него, $T(n) \asymp \lg n$. □

Задача 17

Решете чрез МТ $T(n) = 4T\left(\frac{n}{2}\right) + n$.

Решение: Тук $a = 4$, $b = 2$, така че $\log_b a$ е $\log_2 4 = 2$, следователно $n^{\log_b a}$ е n^2. Нехомогенната част $f(n)$ е n. Сравняваме по асимптотично нарастване n с n^2. Очевидно, $n = O(n^2/n^\epsilon)$ за някое $\epsilon > 0$, така че първият случай на МТ е приложим. Съгласно него, $T(n) \asymp n^2$. □

Допълнение 27: Едно разширение на Теорема 34

Теорема 35: Разширение на МТ

При допусканията на Теорема 34, нека

$$f(n) = \Theta(n^{\log_b a} \lg^t n) \tag{5.67}$$

за някое $t \in \mathbb{N}^+$. Тогава $T(n) = \Theta(n^{\log_b a} \lg^{t+1} n)$.

Доказателство: Теорема 34 не е директно приложима, защото рекурентното уравнение с тази функция $f(n)$ при $t \geqslant 1$ не може да бъде класифицирано в нито един от трите случая. Решаваме задачата с развиване. За простота допускаме, че n е точна степен на b, тоест $n = b^m$ за някакво цяло число $m > 0$. За читателя остава да обобщи доказателството за всяко положително n.
Да допуснем, че логаритъмът в (5.67) е с основа b. Забелязваме, че всичко в Θ-нотацията в дясната страна на (5.67) може да се запише така:

$$n^{\log_b a} \log_b^t n = b^{(m \log_b a)} m^t = b^{(\log_b a^m)} m^t = a^m m^t \tag{5.68}$$

Тогава (5.67) е еквивалентно на

$$c_1 a^m m^t \leqslant f(b^m) \leqslant c_2 a^m m^t$$

за някакви положителни константи c_1 и c_2 и всички достатъчно големи стойности на m. За простота, ще допуснем, че

$$f(b^m) = a^m m^t \tag{5.69}$$

Не е трудно да се направи доказателството да бъде направено по същия начин и без това опростяване.

От условието на МТ имаме $T(n) = aT\left(\frac{n}{b}\right) + f(n)$. Използвайки (5.69), записваме:

$$\begin{aligned}
T(b^m) &= aT\left(\frac{b^m}{b}\right) + a^m m^t \\
&= aT(b^{m-1}) + a^m m^t \;\leftrightarrow \\
S(m) &= aS(m-1) + a^m m^t \qquad \text{заместваики } T(b^m) \text{ със } S(m) \\
&= a\big(aS(m-2) + a^{m-1}(m-1)^t\big) + a^m m^t \\
&= a^2 S(m-2) + a^m (m-1)^t + a^m m^t \\
&= a^2\big(aS(m-3) + a^{m-2}(m-2)^t\big) + a^m (m-1)^t + a^m m^t \\
&= a^3 S(m-3) + a^m (m-2)^t + a^m (m-1)^t + a^m m^t \\
&\dots \\
&= a^{m-1} S(1) + a^m 2^t + a^m 3^t + \dots + a^m (m-2)^t + a^m (m-1)^t + a^m m^t \\
&= a^{m-1} S(1) + a^m (2^t + 3^t + \dots + (m-2)^t + (m-1)^t + m^t)
\end{aligned}$$

Но $2^t + 3^t + \dots + (m-2)^t + (m-1)^t + m^t = \Theta(m^{t+1})$; за извеждането на това вижте [34, (6.98) на стр. 288]. Тогава в сила е

$$\begin{aligned}
&a^{m-1} S(1) + a^m (2^t + 3^t + \dots + (m-2)^t + (m-1)^t + m^t) = \\
&a^{m-1} S(1) + a^m \Theta(m^{t+1}) = \\
&\Theta(a^m m^{t+1})
\end{aligned}$$

Изведохме:

$$S(m) = \Theta(a^m m^{t+1}) \tag{5.70}$$

Да се върнем към началните T и n. (5.70) става

$$T(n) = \Theta\left(a^{\log_b n} (\log_b n)^{t+1}\right)$$

Имайки предвид, че $a^{\log_b n} = n^{\log_b a}$ и $\log_b n = \Theta(\lg n)$, заключаваме, че:

$$T(n) \asymp n^{\log_b a} \lg^{t+1} n$$

□

Задача 18

Решете чрез $T(n) = 2T\left(\frac{n}{2}\right) + n \lg n$.

Решение: Да опитаме с МТ. Имаме $a = 2$, $b = 2$, така че $\log_b a$ е $\log_2 2 = 1$, следователно $n^{\log_b a}$ е $n^1 = n$. Нехомогенната част е $n \lg n$. Да видим дали можем да класифицираме тази задача в някой от трите случая на МТ.

Опитваме случай 1: Дали $n \lg n = O(n^1/n^\epsilon)$ за някое $\epsilon > 0$? Не, защото $n \lg n = \omega(n^1)$.

Опитваме случай 2: Дали $n \lg n = \Theta(n^1)$? Не, защото $n \lg n = \omega(n^1)$.

Опитваме случай 3: Дали $n \lg n = \Omega(n^1.n^\epsilon)$ за някое $\epsilon > 0$? Не. Вижте Теорема 17.

Следователно, тази задача не може да се реши с МТ. Ще я решим чрез Теорема 35, съгласно която $T(n) \asymp n \lg^2 n$. □

5.3.5 Чрез теоремата на Akra-Bazzi

МТ има съществено ограничение: тя решава рекурентни уравнения с **една поява вдясно**, които описват **Разделяй-и-Владей** алгоритми, които винаги делят входа **по един и същи начин** (преди да достигнат началните условия). Но това означава, че дори прости **Разделяй-и-Владей** алгоритми, които делят по по различни начини като например SELECT в Подсекция 5.4.2, не се поддават на изследване с МТ. Рекурентното уравнение (5.114), което описва сложността на SELECT, е решено по индукция в [21, стр. 222], защото [21] не предлага друг формален метод, освен индукцията, за решаване на (5.114).

Сега ще разгледаме една теорема, която е строго по-мощна от МТ и чрез която може да се намери сложността по време на голям клас от естествени **Разделяй-и-Владей** алгоритми, включително и на SELECT. Авторите са Mohamad Akra и Louay Bazzi, а статията [4] е от 1996 г. В тези лекционни записки доказателството на теоремата е изложено в Допълнение 28, понеже е сравнително сложно. Теоремата, както и доказателството ѝ, са представени съгласно статията на Leighton [63][†]. Заслужава да се спомене непубликуваната статия на Neville Campbell [19] от над 300 (триста!) страници, която твърди, че доказателствата на Leighton съдържа дребни грешки от непрецизност, които тя (твърди, че) поправя.

Теорема 36: Теорема на Akra-Bazzi (Theorem 1 от [63])

Нека

$$T(x) = \begin{cases} \Theta(1), & \text{ако } 1 \leqslant x \leqslant x_0 \\ \sum_{i=1}^{k} a_i T(b_i x) + g(x), & \text{ако } x > x_0 \end{cases} \tag{5.71}$$

където

1. $x \geqslant 1$ е реално число,
2. x_0 е константа, такава че $x_0 \geqslant \frac{1}{b_i}$ и $x_0 \geqslant \frac{1}{1-b_i}$, за $1 \leqslant i \leqslant k$,
3. $a_i > 0$ е константа за $1 \leqslant i \leqslant k$,
4. $b_i \in (0,1)$ е константа за $1 \leqslant i \leqslant k$,
5. $k \geqslant 1$ е константа и
6. $g(x)$ е неотрицателна функция, удовлетворяваща условието за полиномиалното нарастване.
7. p е уникалното число, за което $\sum_{i=1}^{k} a_i b_i^p = 1$.

Тогава

$$T(x) \asymp x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du\right)$$

[†]Книгата на Cormen, Leiserson, Rivest [21] споменава теоремата на Akra-Bazzi, но само в края на главата за МТ (стр. 112–113), без да са решени никакви задачи с нея.

Определение 39: Условието за полиномиалното нарастване от [63]

В контекста на Теорема 36 казваме, че $g(x)$ удовлетворява *условието за полиномиалното нарастване*, ако съществуват положителни константи c_1 и c_2, такива че за всяко $x \geqslant 1$, за всяко $1 \leqslant i \leqslant k$, за всяко $u \in [b_i x, x]$:

$$c_1 g(x) \leqslant g(u) \leqslant c_2 g(x)$$

На английски е *the polynomial growth condition*.

Трудно е да се каже **точно** кои функции удовлетворяват условието за полиномиалното нарастване. Ако f е функция, удовлетворяваща условието за полиномиалното нарастване, то има константи $k \in \mathbb{N}$ и $\alpha, \beta \in \mathbb{R}^+$, такива че $\alpha x^{-k} \leqslant f(x) \leqslant \beta x^k$ [19, Lemma 2.34], но конверсното не е непременно вярно [19, стр. 58].

Следните функции:

- полиномите x^α, където α е реална константа,
- логаритмите $\log_b x$, където b е реална константа, като $b > 1$,
- $\lfloor x \rfloor$,
- $\lceil x \rceil$

са примери за функции, удовлетворяващи условието за полиномиалното нарастване [19, Lemma 4.1]. Сумата и произведението на функции, удовлетворяващи условието за полиномиалното нарастване, удовлетворяват условието за полиномиалното нарастване [19, Corollary 4.3]. Също така и частното, стига в знаменателя да има положителна функция. Композицията на функции също запазва условието за полиномиалното нарастване [19, Lemma 4.6].

Една заключителна забележка. Ако $g(x)$ расте суперполиномиално, то тя задава асимптотиката, така че $T(x) \asymp g(x)$. Но това не се получава от Теорема 36, защото в такъв случай $g(x)$ не удовлетворява условието за полиномиалното нарастване.

Допълнение 28: Доказателство на теоремата на Akra-Bazzi

Лема 36: Lemma 1 от [63]

Ако $g(x)$ е неотрицателна функция, удовлетворяваща условието за полиномиалното нарастване, то съществуват положителни константи c_3 и c_4, такива че за всяко $1 \leqslant i \leqslant k$ и всяко $x \geqslant 1$ е вярно, че

$$c_3 g(x) \leqslant x^p \int_{b_i x}^{x} \frac{g(u)}{u^{p+1}} \, du \leqslant c_4 g(x)$$

Доказателство: От условието за полиномиалното нарастване имаме:

$$\begin{aligned}\int_{b_i x}^{x} \frac{g(u)}{u^{p+1}}\,du &\leqslant x^p(x - b_i x)\frac{c_2 g(x)}{\min\{(b_i x)^{p+1}, x^{p+1}\}}\\ &= \frac{(1-b_i)c_2 g(x)}{\min\{1, b_i^{p+1}\}}\\ &\leqslant c_4 g(x)\end{aligned}$$

където c_4 е константа, такава че

$$c_4 \geqslant \frac{(1-b_i)c_2}{\min\{1, b_i^{p+1}\}}$$

за $1 \leqslant i \leqslant k$.

Аналогично,

$$\begin{aligned}\int_{b_i x}^{x} \frac{g(u)}{u^{p+1}}\,du &\geqslant x^p(x - b_i x)\frac{c_1 g(x)}{\max\{(b_i x)^{p+1}, x^{p+1}\}}\\ &= \frac{(1-b_i)c_1 g(x)}{\max\{1, b_i^{p+1}\}}\\ &\geqslant c_3 g(x)\end{aligned}$$

където c_3 е константа, такава че

$$c_3 \leqslant \frac{(1-b_i)c_1}{\min\{1, b_i^{p+1}\}}$$

за $1 \leqslant i \leqslant k$.

□

Ето отново теоремата.

Теорема: Theorem 1 от [63]

Дадено е рекурентното уравнение (5.71)

$$T(x) = \begin{cases}\Theta(1), & \text{ако } 1 \leqslant x \leqslant x_0\\ \sum_{i=1}^{k} a_i T(b_i x) + g(x), & \text{ако } x > x_0\end{cases}$$

с ограниченията и имената, посочени след (5.71). Тогава

$$T(x) \asymp x^p\left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du\right)$$

Доказателство: Разбиваме домейна, от който взема стойности x, на интервали

$$\begin{aligned}I_0 &= [1, x_0]\\ I_j &= (x_0 + j - 1, x_0 + j] \quad \text{за } j \geqslant 1\end{aligned}$$

Ако $x \in I_j$ за някое $j \geqslant 1$, то за всяко i, такова че $1 \leqslant i \leqslant k$ е вярно, че $b_i x \in I_{j'}$ за някое $j' < j$. Това е така, понеже $b_i x > b_i(x_0 + j - 1) \geqslant b_i x_0 \geqslant 1$, и освен това $b_i x \leqslant b_i(x_0 + j) \leqslant x_0 + j - (1 - b_i)x_0 \leqslant x_0 + j - 1$. Следователно, стойността на T в даден интервал след нулевия зависи само от стойностите на T от предишни интервали. Следователно, можем да опитаме да докажем теоремата със силна индукция по j.

Първо ще покажем, че има положителна константа c_5, такава че за всяко $x > x_0$ е вярно, че

$$T(x) \geqslant c_5 x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du\right)$$

Доказателството е по индукция по интервала I_j, съдържащ x. Базовият случай е $j = 0$. Там твърдението следва от факта, че $T(x) = \Theta(1)$ върху I_0. В индуктивната стъпка процедираме така:

$$\begin{aligned}
T(x) &= \sum_{i=1}^{k} a_i T(b_i x) + g(x) \quad \text{// от индуктивното предположение} \\
&\geqslant \sum_{i=1}^{k} a_i c_5 (b_i x)^p \left(1 + \int_1^{b_i x} \frac{g(u)}{u^{p+1}}\,du\right) + g(x) \\
&= c_5 x^p \sum_{i=1}^{k} a_i b_i^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du - \int_{b_i x}^x \frac{g(u)}{u^{p+1}}\,du\right) + g(x) \quad \text{// от Лема 36} \\
&\geqslant c_5 x^p \sum_{i=1}^{k} a_i b_i^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du - \frac{c_4 g(x)}{x^p}\right) + g(x) \\
&= c_5 x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du - \frac{c_4 g(x)}{x^p}\right)\left(\sum_{i=1}^{k} a_i b_i^p\right) + g(x) \quad \text{//защото } \sum_{i=1}^{k} a_i b_i^p = 1 \\
&= c_5 x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du - \frac{c_4 g(x)}{x^p}\right) + g(x) \\
&= c_5 x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du\right) + g(x)(1 - c_4 c_5) \\
&\geqslant c_5 x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du\right), \quad \text{ако } (c_4 c_5 < 1)
\end{aligned}$$

Доказателството, че има положителна константа c_6, такава че за всяко $x > x_0$ е вярно, че

$$T(x) \leqslant c_6 x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}}\,du\right),$$

е доста подобно. Трябва само да сме сигурни, че c_6 е достатъчно голяма, така че в базовия случай твърдението да е вярно, и освен това $c_3 c_6 \geqslant 1$.
Това е и края на доказателството. □

Задача 19: от [4]

Решете чрез метода на Akra-Bazzi $T(n) = \frac{4}{3}T\left(\frac{n}{2}\right) + 3T\left(\frac{n}{3}\right) + \frac{16}{3}T\left(\frac{n}{4}\right) + \Theta(n^2 \lg\lg n)$.

Решение: Първо да намерим p като решението на

$$\frac{4}{3}\left(\frac{1}{2}\right)^p + 3\left(\frac{1}{3}\right)^p + \frac{16}{3}\left(\frac{1}{4}\right)^p = 1$$

Намираме $p = 2$.

Тогава,

$$\begin{aligned} T(n) &\asymp n^2\left(1 + \int_1^n \frac{u^2 \ln\ln u}{u^3}\,du\right) \\ &= n^2\left(1 + \int_1^n \frac{\ln\ln u}{u}\,du\right) \\ &= n^2(1 + \ln n(\ln\ln n - 1)) \\ &\asymp n^2 \cdot \ln n \cdot \ln\ln n \end{aligned}$$

Задача 20: от [63]

Решете чрез метода на Akra-Bazzi $T(x) = \frac{1}{2}T\left(\frac{x}{2}\right) + \Theta\left(\frac{1}{x}\right)$.

Решение: Това уравнение не може да описва сложност на алгоритъм, но може да се реши с метода на Akra-Bazzi. Първо да намерим p като решението на

$$\frac{1}{2}\left(\frac{1}{2}\right)^p = 1$$

Очевидно $p = -1$. Тогава,

$$\begin{aligned} T(n) &\asymp x^{-1}\left(1 + \int_1^x \frac{\frac{1}{u}}{u^0}\,du\right) \\ &= \frac{1}{x}\left(1 + \int_1^x \frac{1}{u}\,du\right) \\ &= \frac{1}{x}(1 + \ln x) \\ &\asymp \frac{\ln x}{x} \end{aligned}$$

5.3.6 Чрез метода с характеристичното уравнение

Следната теорема е от учебника Увод в Дискретната Математика на Кр. Манев [65, стр. 60]. Тя е дадена там без доказателство.

Теорема 37: Решение на линейно хомогенно рекурентно уравнение

Нека редицата $\tilde{a} = (a_0, a_1, a_2, \ldots)$ е зададена с линейното рекурентно уравнение

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_{r-1} a_{n-(r-1)} + c_r a_{n-r},\ n \geqslant r \tag{5.72}$$

и $\alpha_1, \alpha_2, \ldots, \alpha_s$ са различните комплексни корени на *характеристичното уравнение*

$$x^r - c_1 x^{r-1} - c_2 x^{r-2} - \cdots - c_{r-1} x - c_r = 0 \tag{5.73}$$

като α_i е с кратност k_i и $k_1 + k_2 + \cdots + k_s = r$. Тогава

$$a_n = P_1(n)\alpha_1^n + P_2(n)\alpha_2^n + \cdots + P_s(n)\alpha_s^n \tag{5.74}$$

където $P_i(n)$ е полином на n от степен $< k_i$. Полиномите $P_i(n)$ имат общо r коефициента, които се определят еднозначно от първите r члена на редицата $\tilde{a}$. □

В текущата терминология, (5.72) изглежда така:

$$T(n) = c_1 T(n-1) + c_2 T(n-2) + \cdots + c_{r-1} T(n-(r-1)) + c_r T(n-r) \tag{5.75}$$

Както вече казахме, това е линейно хомогенно рекурентно уравнение с константни коефициенти и крайна история. Такова рекурентно уравнение не може да описва сложността на рекурсивен алгоритъм, защото алгоритъмът не може да не извършва поне константна работа извън рекурсивните викания. Понеже се интересуваме от анализ на алгоритми, налага се да разгледаме нехомогенни рекурентни уравнения. Ще разглеждаме нехомогенни рекурентни уравнения от този вид:

$$\begin{aligned} T(n) = {} & c_1 T(n-1) + c_2 T(n-2) + \cdots + c_{r-1} T(n-(r-1)) + c_r T(n-r) \\ & + \beta_1^n Q_1(n) + \beta_2^n Q_2(n) + \cdots + \beta_m^n Q_m(n) \end{aligned} \tag{5.76}$$

$\beta_1, \beta_2, \ldots, \beta_m$ са две по две различни положителни константи. $Q_1(n), Q_2(n), \ldots, Q_m(n)$ са полиноми от съответни степени $d_1, d_2, \ldots, d_m$, а m е цяла положителна константа.

Понятието "мултимножество" спокойно може да бъде избегнато, като вместо това се говори за обикновено множество и кратност на елементите, но авторът на записките намира изразяването чрез "мултимножество" за много ясно и естествено.

Определение 40: Мултимножество: нотация, кратност, обединение, кардиналност

Със нотацията "$\{\quad\}_M$" означаваме мултимножества. Например, $\{1,1,2,3,3,3\}_M$ е мултимножеството, състоящо се от две единици, една двойка и две тройки. За всеки елемент a по отношение на някакво мултимножество A, с $\#(a, A)$ означаваме броя на появите на a в A, когото наричаме *кратността на a в A*. $\#(a, A) = 0$, ако a не се среща в A. Например, $\#(1, \{1,1,2,3,3,3\}_M) = 2$. Обединението на две мултимножества A и B е:

$$A \cup B = \{x \mid (x \in A \text{ или } x \in B) \text{ и } (\#(x, A \cup B) = \#(x, A) + \#(x, B))\}_M$$

Кардиналност на мултимножеството A е сумата от броевете на появите на елементите му и се означава с $|A|$. Например, $|\{1,1,2,3,3,3\}_M| = 6$.

Теорема 38: Решение на линейно нехомогенно рекурентно уравнение

Нека мултимножеството от корените на характеристичното уравнение е A. Очевидно, $|A| = r$. Нека B е мултимножеството от всички β_i, като $\#(\beta_i, B) = d_i + 1$ за $1 \leqslant i \leqslant m$. Нека $Y = A \cup B$. Очевидно, $|Y| = r + \sum_{i=1}^{m}(d_i + 1)$. Нека преименуваме различните елементи на Y като $y_1, y_2, \ldots, y_t$ и да дефинираме, че $z_i = \#(y_i, Y)$, за $1 \leqslant i \leqslant t$. Тогава общото решение на (5.76) е:

$$\begin{aligned} T(n) = {} & \gamma_{1,1}\, y_1^n + \gamma_{1,2}\, n\, y_1^n + \gamma_{1,3}\, n^2\, y_1^n + \cdots + \gamma_{1,z_1}\, n^{z_1-1}\, y_1^n + \\ & \gamma_{2,1}\, y_2^n + \gamma_{2,2}\, n\, y_2^n + \gamma_{2,3}\, n^2\, y_2^n + \cdots + \gamma_{2,z_2}\, n^{z_2-1}\, y_2^n + \\ & \ldots \\ & \gamma_{t,1}\, y_t^n + \gamma_{t,2}\, n\, y_t^n + \gamma_{t,3}\, n^2\, y_t^n + \cdots + \gamma_{t,z_t}\, n^{z_t-1}\, y_t^n, \end{aligned} \tag{5.77}$$

където $\gamma_{i,j}$, за $1 \leqslant i \leqslant t$ и $1 \leqslant j \leqslant z_t$, са константи, общо $|Y|$ на брой. В общото решение те са неизвестни константи. Можем да ги намерим от началните условия (като добавим $|B|$ на брой нови начални условия) и тогава ще разполагаме с точното решение.

Следствие 18: Асимптотиката на общото решение от Теорема 38

Понеже се интересуваме само от асимптотичното нарастване на $T(n)$, точните стойности на константите $\gamma_{i,j}$ са без значение. Нещо повече. Дори само общото решение е прекалено подробно. Асимптотичното нарастване се определя от **точно едно събираемо** в общото решение, а именно, най-голямото y_i, умножено по най-голямата възможна степен на n. Накратко, ако $y_i = \max\{y_1, \ldots, y_t\}$, то

$$T(n) \asymp n^{z_i-1}\, y_i^n \tag{5.78}$$

Забележете, че ако $T(n)$ описва сложността на алгоритъм, най-голямото по абсолютна стойност y_i задължително е положително.

Допълнение 29: Извеждане на решението на линейните рек. у-ния

Ще изведем решението съгласно учебника *An Introduction to Difference Equations* на Elaydi [23, стр. 75]. Първо ще въведем две понятия и после ще видим как с тяхна помощ извеждаме решение.

Първо понятие: оператор. Нека n е дискретна променлива; по-точно, $n \in \mathbb{N}$. Нека $\mathcal{G}$ е множеството от функции

$$\mathcal{G} = \{g \mid g : \mathbb{N} \to \mathbb{R}\}$$

Оператор, за целите на това изложение, е функция с домейн и кодомейн $\mathcal{G}$; ерго, оператор е функция, изобразяваща функция във функция. Нека x е произволна функция от $\mathcal{G}$. Въвеждаме операторите $\boldsymbol{\Delta}$ (оператор разлика) и $\mathbf{E}$ (оператор изместване):

$$\boldsymbol{\Delta} x(n) = x(n+1) - x(n) \tag{5.79}$$

$$\mathbf{E} x(n) = x(n+1) \tag{5.80}$$

Примерно, ако $x(n) = n$, операторите резултатът от действието на двата оператора е:

n	0	1	2	3	...	n	...
$x(n)$	0	1	2	3	...	n	...
$\Delta x(n)$	1	1	1	1	...	1	...
$\mathbf{E}x(n)$	1	2	3	4	...	$n+1$	...

Въвеждаме и съответните итерираните *итерирани оператори*:

$$\Delta^k x(n) = \begin{cases} x(n), & \text{ако } k = 0 \\ \Delta(\Delta^{k-1} x(n)), & \text{ако } k > 0 \end{cases}$$

$$\mathbf{E}^k x(n) = \begin{cases} x(n), & \text{ако } k = 0 \\ \mathbf{E}(\mathbf{E}^{k-1} x(n)), & \text{ако } k > 0 \end{cases}$$

Ето как изглеждат Δ^2, Δ^3, $\mathbf{E}^2$ и $\mathbf{E}^3$ за $x(n) = n$:

n	0	1	2	3	...	n	...
$x(n)$	0	1	2	3	...	n	...
$\Delta x(n)$	1	1	1	1	...	1	...
$\Delta^2 x(n)$	0	0	0	0	...	0	...
$\Delta^3 x(n)$	0	0	0	0	...	0	...
$\mathbf{E}x(n)$	1	2	3	4	...	$n+1$	...
$\mathbf{E}^2 x(n)$	2	3	4	5	...	$n+2$	...
$\mathbf{E}^3 x(n)$	3	4	5	6	...	$n+3$	...

В някакъв смисъл, операторът Δ е аналог на оператора $\mathbf{D}$ (производна) от математическия анализ:

$$\mathbf{D}f(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Също както в континуалния анализ, ако $f(x) = x$, то $\mathbf{D}f(x) = 1$, а $\mathbf{D}^2 f(x) = 0$, в дискретния анализ е вярно, че, ако $x(n) = n$, то $\Delta x(n) = 1$, а $\Delta^2 x(n) = 0$.

И операторът разлика, и операторът изместване са линейни оператори. Ако a и b са константи, в сила са:

$$\Delta(ax(n) + by(n)) = a\Delta x(n) + b\Delta y(n)$$
$$\mathbf{E}(ax(n) + by(n)) = a\mathbf{E}x(n) + b\mathbf{E}y(n)$$

По дефиниция, $\mathbf{I}$ е операторът идентитет:

$$\mathbf{I}x = x$$

Вярно е, че

$$\mathbf{E} = \Delta + \mathbf{I}$$
$$\Delta = \mathbf{E} - \mathbf{I} \tag{5.81}$$

В общия случай, итерираният оператор изместване е лесен за разбиране. Тривиално

се показва по индукция, че

$$\forall k \in \mathbb{N} : \mathbf{E}^k x(n) = x(n+k)$$

Итерираният оператор разлика е малко по-триков. За $\boldsymbol{\Delta}^2$ имаме:

$$\begin{aligned}
\boldsymbol{\Delta}^2 x(n) &= \boldsymbol{\Delta}(\boldsymbol{\Delta} x(n)) \\
&= \boldsymbol{\Delta}(x(n+1) - x(n)) \\
&= \boldsymbol{\Delta} x(n+1) - \boldsymbol{\Delta} x(n) \\
&= x(n+2) - x(n+1) - x(n+1) + x(n) \\
&= x(n+2) - 2x(n+1) + x(n)
\end{aligned}$$

За $\boldsymbol{\Delta}^3$ имаме:

$$\begin{aligned}
\boldsymbol{\Delta}^3 x(n) &= \boldsymbol{\Delta}(\boldsymbol{\Delta}^2 x(n)) \\
&= \boldsymbol{\Delta}(x(n+2) - 2x(n+1) + x(n)) \\
&= \boldsymbol{\Delta} x(n+2) - 2\boldsymbol{\Delta} x(n+1) + \boldsymbol{\Delta} x(n) \\
&= (x(n+3) - x(n+2)) - 2(x(n+2) - x(n+1)) + (x(n+1) - x(n)) \\
&= x(n+3) - 3x(n+2) + 3x(n+1) - x(n)
\end{aligned}$$

Можем да покажем по индукция, че

$$\boldsymbol{\Delta}^k x(n) = \sum_{i=0}^{k} (-1)^i \binom{k}{i} x(n+k-i)$$

Действието на оператора $\boldsymbol{\Delta}$ върху полиноми е аналогично на действието на оператора $\mathbf{D}$ от континуалния анализ: ако полиномът е от степен $k \geqslant 1$, то еднократното прилагане на $\boldsymbol{\Delta}$ "сваля" степента с единица, k-кратното му прилагане дава $a_k k!$, където a_k е коефициентът пред старшия член, а оттам $(k+i)$-кратното му прилагане за $i \geqslant 1$ дава нула. Наистина, нека

$$p(n) = a_k n^k + a_{k-1} n^{k-1} + \cdots a_1 n + a_0$$

където $k \geqslant 1$ и $a_k, \ldots, a_0$ са константи; ерго, $\deg p(n) = k$. Тогава

$$\begin{aligned}
\Delta p(n) &= \big(a_k(n+1)^k + a_{k-1}(n+1)^{k-1} + \cdots + a_1(n+1) + a_0\big) \\
&\quad - \big(a_k n^k + a_{k-1}n^{k-1} + \cdots + a_1 n + a_0\big) = \\
&= \underbrace{a_k n^k + a_k k n^{k-1} + \cdots + a_k}_{a_k(n+1)^k} + \underbrace{a_{k-1}(n+1)^{k-1} + \cdots + a_1(n+1) + a_0}_{\text{от степен } k-1} \\
&\quad - a_k n^k - a_{k-1}n^{k-1} - \cdots - a_1 n - a_0 = \\
&= a_k k n^{k-1} + \underbrace{\cdots + a_k}_{\text{от степен } <k-1} + a_{k-1}(n+1)^{k-1} + \underbrace{\cdots + a_1(n+1) + a_0}_{\text{от степен } <k-1} \\
&\quad - a_{k-1}n^{k-1} - \underbrace{\cdots - a_1 n - a_0}_{\text{от степен } <k-1} = \\
&= a_k k n^{k-1} + \underbrace{\cdots + a_k}_{\text{от степен } <k-1} \\
&\quad + a_{k-1}n^{k-1} + \underbrace{\cdots}_{\text{от степен } <k-1} + \underbrace{\cdots + a_1(n+1) + a_0}_{\text{от степен } <k-1} \\
&\quad - a_{k-1}n^{k-1} - \underbrace{\cdots - a_1 n - a_0}_{\text{от степен } <k-1} = \\
&= a_k k n^{k-1} + \underbrace{\cdots}_{\text{от степен } <k-1}
\end{aligned}$$

И така, $\Delta p(n)$ е $a_k k n^{k-1}$ плюс полином от степен, по-малка от $k-1$. Аналогично, $\Delta^2 p(n)$ е $a_k k(k-1)n^{k-2}$ плюс полином от степен, по-малка от $k-2$, ако $k \geqslant 2$. Лесно се вижда, че

$$\Delta^k p(n) = a_k k!$$

и

$$i \geqslant 1 \;\rightarrow\; \Delta^{k+i} p(n) = 0 \tag{5.82}$$

Дефинираме *оператор полином* от степен k така. Нека k е константа и е даден следният полином от степен k на E:

$$p(E) = a_k \cdot E^k + a_{k-1} \cdot E^{k-1} + \cdots + a_1 \cdot E + a_0 \cdot I$$

Да разгледаме действието на $p(E)$ върху $b^n y(n)$, където b е константа, а $y(n)$ е някаква функция.

$$\begin{aligned}
p(E)b^n y(n) &= (a_k \cdot E^k + a_{k-1} \cdot E^{k-1} + \cdots + a_1 \cdot E + a_0 \cdot I)b^n y(n) \\
&= a_k E^k b^n y(n) + a_{k-1}E^{k-1}b^n y(n) + \cdots a_1 E b^n y(n) + a_0 b^n y(n) \\
&= b^n\big(a_0 b^k y(n+k) + a_1 b^{k-1} y(n+k-1) + \cdots + a_{k-1} b y(n+1) + a_k y(n)\big) \\
&= b^n p(b \cdot E) y(n)
\end{aligned}$$

И така,

$$\mathbf{p}(\mathbf{E})b^n y(n) = b^n \mathbf{p}(b \cdot \mathbf{E})y(n) \tag{5.83}$$

Анихилатор за функцията $y(n)$ е оператор полином $\mathbf{N}(\mathbf{E})$, такъв че за всяко n е изпълнено

$$\mathbf{N}(\mathbf{E})y(n) = 0 \tag{5.84}$$

Може да се каже, че $\mathbf{N}(\mathbf{E})$ е анихилатор за $y(n)$, ако $y(n)$ е решение на (5.84). Следват няколко примера.

- Нека $y(n) = 2^n$. Тогава $\mathbf{N}(\mathbf{E}) = \mathbf{E} - 2$ е анихилатор за $y(n)$, защото $(\mathbf{E} - 2)2^n = 2^{n+1} - 2 \cdot 2^n = 0$.
- И изобщо, ако $y(n) = a^n$, то $\mathbf{N}(\mathbf{E}) = \mathbf{E} - a$ е анихилатор за $y(n)$, защото $(\mathbf{E} - a)a^n = a^{n+1} - a \cdot a^n = 0$.
- Нека $y(n) = n2^n$. Тогава $\mathbf{N}(\mathbf{E}) = (\mathbf{E} - 2)^2$ е анихилатор за $y(n)$, защото

$$\begin{aligned}(\mathbf{E} - 2)^2 n2^n &= (\mathbf{E}^2 - 4\mathbf{E} + 4)n2^n \\ &= (n+2)2^{n+2} - 4(n+1)2^{n+1} + 4n2^n \\ &= 4n2^n + 8 \cdot 2^n - 8n2^n - 8 \cdot 2^n + 4n2^n = 0\end{aligned}$$

- И изобщо, ако $y(n) = na^n$, то $\mathbf{N}(\mathbf{E}) = (\mathbf{E} - a)^2$ е анихилатор за $y(n)$.
- Лесно се вижда, че ако $y(n) = n^2a^n$, то $\mathbf{N}(\mathbf{E}) = (\mathbf{E} - a)^3$ е анихилатор за $y(n)$.
- И изобщо, ако $y(n) = n^b a^n$, то $\mathbf{N}(\mathbf{E}) = (\mathbf{E} - a)^{b+1}$ е анихилатор за $y(n)$.
- Лесно се вижда, че ако $y(n) = n^{b_1}a_1^n + n^{b_2}a_2^n$, то $\mathbf{N}(\mathbf{E}) = (\mathbf{E} - a_1)^{b_1+1}(\mathbf{E} - a_2)^{b_2+1}$ е анихилатор за $y(n)$.
- И изобщо, ако $y(n) = n^{b_1}a_1^n + \cdots + n^{b_t}a_t^n$, то $\mathbf{N}(\mathbf{E}) = (\mathbf{E} - a_1)^{b_1+1} \cdot \cdots \cdot (\mathbf{E} - a_t)^{b_t+1}$ е анихилатор за $y(n)$.

Второ понятие: фундаментално множество от решения. Разглеждаме общи хомогенни линейни рекурентни уравнения с константни коефициенти и крайна история

$$x(n+k) + c_1x(n+k-1) + c_2x(n+k-2) + \cdots + c_{k-1}x(n+1) + c_kx(n) = 0 \tag{5.85}$$

където $c_1, \ldots, c_k$ са реални константи и $c_k \neq 0$. Казваме, че функциите $f_1(n), \ldots, f_r(n)$ са *линейно независими* за $n \geqslant n_0$, ако

$$\forall n \geqslant n_0 : a_1f_1(n) + \cdots + a_rf_r(n) = 0$$

тстк $a_1 = \cdots = a_r = 0$.

Определение 41

Всяко множество от k линейно независими решения на (5.85) се нарича *фундаментално множество от решения*.

Има сравнително ефикасен начин да се провери дали множество от решения е линейно независимо, тоест, фундаментално. За целта ще ни помогне понятието *детерминанта на Casorati*[a], на английски *Casoratian*, която е дискретния аналог на детерминантата на Wronski (на английски, *Wronskian*) от континуалната математика.

Определение 42

Детерминантата на Casorati за решенията $x_1(n), \ldots, x_r(n)$ е

$$W(n) = \det \begin{bmatrix} x_1(n) & x_2(n) & \cdots & x_r(n) \\ x_1(n+1) & x_2(n+1) & \cdots & x_r(n+1) \\ x_1(n+2) & x_2(n+2) & \cdots & x_r(n+2) \\ \vdots & \vdots & \cdots & \vdots \\ x_1(n+r-1) & x_2(n+r-1) & \cdots & x_r(n+r-1) \end{bmatrix} \tag{5.86}$$

Лема 37: Lemma 2.13 от [23, стр. 68]

Нека $x_1(n), \ldots, x_r(n)$ са решения на (5.85). Нека $W(n)$ е детерминантата на Casorati за тях. Тогава, за всяко $n \geqslant n_0$:

$$W(n) = (-1)^{k(n-n_0)} c_k^{n-n_0} W(n_0).$$

Доказателство: Ще докажем за частния случай $n = 3$. Доказателството за общия случай може да се направи аналогично.
И така, нека $n = 3$. От (5.86) имаме

$$W(n+1) = \det \begin{bmatrix} x_1(n+1) & x_2(n+1) & x_3(n+1) \\ x_1(n+2) & x_2(n+2) & x_3(n+2) \\ x_1(n+3) & x_2(n+3) & x_3(n+3) \end{bmatrix} \tag{5.87}$$

От (5.85) имаме:

$$\begin{aligned} x_1(n+3) &= -c_3 x_1(n) - (c_1 x_1(n+2) + c_2 x_1(n+1)) \\ x_2(n+3) &= -c_3 x_2(n) - (c_1 x_2(n+2) + c_2 x_2(n+1)) \\ x_3(n+3) &= -c_3 x_3(n) - (c_1 x_3(n+2) + c_2 x_3(n+1)) \end{aligned}$$

Заместваме в (5.87) и получаваме

$$W(n+1) = \det \begin{bmatrix} x_1(n+1) & x_2(n+1) & x_3(n+1) \\ x_1(n+2) & x_2(n+2) & x_3(n+2) \\ A & B & C \end{bmatrix} \tag{5.88}$$

където

$$\begin{aligned}
A &= -c_3x_1(n) - (c_2x_1(n+1) + c_1x_1(n+2)) \\
B &= -c_3x_2(n) - (c_2x_2(n+1) + c_1x_2(n+2)) \\
C &= -c_3x_3(n) - (c_2x_3(n+1) + c_1x_3(n+2))
\end{aligned}$$

Съгласно линейната алгебра, (5.88) може да се запише просто като

$$\begin{aligned}
W(n+1) &= \det \begin{bmatrix} x_1(n+1) & x_2(n+1) & x_3(n+1) \\ x_1(n+2) & x_2(n+2) & x_3(n+2) \\ -c_3x_1(n) & -c_3x_2(n) & -c_3x_3(n) \end{bmatrix} \\
&= -c_3 \det \begin{bmatrix} x_1(n+1) & x_2(n+1) & x_3(n+1) \\ x_1(n+2) & x_2(n+2) & x_3(n+2) \\ x_1(n) & x_2(n) & x_3(n) \end{bmatrix} \\
&= -c_3(-1)^2 \det \begin{bmatrix} x_1(n) & x_2(n) & x_3(n) \\ x_1(n+1) & x_2(n+1) & x_3(n+1) \\ x_1(n+2) & x_2(n+2) & x_3(n+2) \end{bmatrix}
\end{aligned}$$

Изразихме $W(n+1)$ чрез $W(n)$:

$$W(n+1) = (-1)^3 c_3 W(n) \tag{5.89}$$

което е линейно рекурентно уравнение с една поява вдясно (от първи ред). С обикновено развиване получаваме

$$W(n+1) = (-1)^{3(n-n_0)} c_3^{n-n_0} W(n_0) \tag{5.90}$$

Което е и краят на доказателството за $n = 3$. □

Следствие 19

Тъй като $c_k \neq 0$ по условие, вярно е, че детерминантата на Casorati $W(n)$ е ненулева за всяко $n \geqslant n_0$ тстк детерминантата на Casorati $W(n_0)$ е ненулева. Ерго, или $W(n)$ е тъждествено нула при $n \geqslant n_0$, или никога не е нула при $n \geqslant n_0$.

Ще покажем, че множество от k решения на (5.85) е фундаментално (тоест, линейно независимо) тстк $W(n)$ никога не е нула. Разглеждаме k решения $x_1(n)$, ..., $x_k(n)$ на (5.85). Да кажем, че за някакви константи α_1, ..., α_k е вярно, че

$$\forall n \geqslant n_0 : \alpha_1x_1(n) + \cdots + \alpha_kx_k(n) = 0$$

Тогава също така е вярно, че

$$\begin{aligned}
&\alpha_1x_1(n+1) + \cdots \quad + \alpha_kx_k(n+1) \quad = 0 \\
&\cdots \\
&\alpha_1x_1(n+k-1) + \cdots + \alpha_kx_k(n+k-1) = 0
\end{aligned}$$

Всичко това може да се запише кратко ето така:

$$X(n)\xi = \mathbf{0} \tag{5.91}$$

където

$$X(n) = \begin{bmatrix} x_1(n) & \cdots & x_k(n) \\ \vdots & \cdots & \vdots \\ x_1(n+k-1) & \cdots & x_k(n+k-1) \end{bmatrix}$$

и

$$\xi = \begin{bmatrix} \alpha_1 \\ \cdots \\ \alpha_k \end{bmatrix}$$

Очевидно $\det X(n) = W(n)$. Съгласно линейната алгебра, (5.91) има само тривиалното решение $\alpha_1 = \cdots = \alpha_k = 0$ тстк $X(n)$ е несингулярна, тоест, $\det X(n) \neq 0$, тоест, $W(n) \neq 0$ за $n \geqslant n_0$. От тези и предишните съображения лесно следва Теорема 39.

Теорема 39: Множество решения е фундаментално тстк дет. на Casorati е ненулева

Множеството $\{x_1(n), \ldots, x_k(n)\}$ от решения на (5.85) е фундаментално тстк за някое $n_0 \in \mathbb{N}^+$ е вярно, че $W(n_0) \neq 0$.

Задача 21

Покажете, че 2^n, $(-2)^n$ и $(-3)^n$ образуват множество от фундаментални решения на

$$x(n+3) + 3x(n+2) - 4x(n+1) - 12x(n) = 0 \tag{5.92}$$

Решение: Лесно може да се убедим, че и трите функции са решения, замествайки в (5.92). Примерно,

$$2^{n+3} + 3 \cdot 2^{n+2} - 4 \cdot 2^{n+1} - 12 \cdot 2^n = 8 \cdot 2^n + 12 \cdot 2^n - 8 \cdot 2^n - 12 \cdot 2^n = 0$$

Сега да се убедим, че те са фундаментални решения. Построяваме детерминантата на Casorati:

$$W(n) = \det \begin{bmatrix} 2^n & (-2)^n & (-3)^n \\ 2^{n+1} & (-2)^{n+1} & (-3)^{n+1} \\ 2^{n+2} & (-2)^{n+2} & (-3)^{n+2} \end{bmatrix}$$

Тогава

$$W(0) = \det \begin{bmatrix} 2^0 & (-2)^0 & (-3)^0 \\ 2^1 & (-2)^1 & (-3)^1 \\ 2^2 & (-2)^2 & (-3)^2 \end{bmatrix} = \det \begin{bmatrix} 1 & 1 & 1 \\ 2 & -2 & -3 \\ 4 & 4 & 9 \end{bmatrix} = -20 \neq 0$$

□

Теорема 40: Theorem 2.18 от [23, стр. 72]: съществуване на фунд. множество от решения

Уравнение (5.85) има фундаментално множество от решения за $n \geqslant 0$.

Доказателство: Очевидно (5.85) има "единични" решения $x_1(n)$, $x_2(n)$, $\ldots$, $x_k(n)$:

$$
\begin{aligned}
&x_1(0) = 1, x_1(1) = 0, x_1(2) = 0, \ldots, x_1(k-1) = 0\\
&x_2(0) = 0, x_2(1) = 1, x_2(2) = 0, \ldots, x_2(k-1) = 0\\
&x_3(0) = 0, x_3(1) = 0, x_3(2) = 1, \ldots, x_3(k-1) = 0\\
&\ldots\\
&x_k(0) = 0, x_k(1) = 0, x_k(2) = 0, \ldots, x_k(k-1) = 1
\end{aligned}
$$

Веднага се вижда, че $W(0) = 1$. Щом детерминантата на Casorati е ненулева, то от Теорема 39 следва, че множеството от "единичните" решения е фундаментално множество от решения на (5.85). □

Уравнение (5.85) има безброй много фундаментални множества от решения. Сега ще видим как се генерират фундаментални решения от известни решения.

Лема 38

Нека $x_1(n)$ и $x_2(n)$ са решения на (5.85). Тогава всяка линейна комбинация на $x_1(n)$ и $x_2(n)$ е решение на (5.85).

Доказателство: Ако за $n \geqslant k$ имаме

$$
\begin{aligned}
&x_1(n+k) + c_1x_1(n+k-1) + c_2x_1(n+k-2) + \cdots + c_{k-1}x_1(n+1) + c_kx_1(n) = 0\\
&x_2(n+k) + c_1x_2(n+k-1) + c_2x_2(n+k-2) + \cdots + c_{k-1}x_2(n+1) + c_kx_2(n) = 0
\end{aligned}
$$

то очевидно

$$
\begin{aligned}
&\big(x_1(n+k) + x_2(n+k)\big) + c_1\big(x_1(n+k-1) + x_2(n+k-1)\big) + \cdots +\\
&\qquad c_{k-1}\big(x_1(n+1) + x_2(n+1)\big) + c_k\big(x_1(n) + x_2(n)\big) = 0\\
&\alpha x_1(n+k) + \alpha c_1x_1(n+k-1) + \cdots + \alpha c_{k-1}x_2(n+1) + \alpha p_kx_2(n) = 0
\end{aligned}
$$

за произволна константа α. □

Чрез Лема 38 лесно можем да докажем следния важен резултат.

Теорема 41: Принцип на суперпозицията

Ако x_1, x_2, $\ldots$, $x_r(n)$ са решения на (5.85), то

$$x(n) = \alpha_1x_1(n) + \alpha_2x_2(n) + \cdots + \alpha_rx_r(n)$$

също е решение на на (5.85), където α_1, α_2, $\ldots$, α_r са произволни константи. □

Теорема 42: Фундаментално множество от решения и конкретно решение

Нека $\{x_1(n), \ldots, x_k(n)\}$ е фундаментално множество от решения на (5.85). Тогава за всяко конкретно решение $x(n)$ на (5.85) съществуват константи α_1, α_2, $\ldots$, α_k, такива че

$$x(n) = \sum_{i=1}^{k} \alpha_i x_i(n)$$

Доказателство: Ползвайки нотацията на (5.91), записваме

$$X(n)\xi = \widehat{x}(n)$$

където

$$\widehat{x}(n) = \begin{bmatrix} x(n) \\ x(n+1) \\ \vdots \\ x(n+k-1) \end{bmatrix}$$

Тъй като $X(n)$ е инвертируема, то

$$\xi = X^{-1}(n)\widehat{x}(n)$$

В частност,

$$\xi = X^{-1}(n_0)\widehat{x}(n_0)$$

и

$$\xi = X^{-1}(0)\widehat{x}(0)$$

□

Теорема 42 ни дава право да направим следното определение.

Определение 43: Общото решение на хомогенно линейно рекурентно уравнение

Нека $\{x_1(n), \ldots, x_k(n)\}$ е фундаментално множество от решения на (5.85). Тогава *общото решение* на (5.85) е

$$x(n) = \sum_{i=1}^{k} \alpha_i x_i(n)$$

за произволни константи α_1, α_2, $\ldots$, α_k.

Всяко (конкретно) решение на (5.85) се получава от общото с подходящ избор на константи α_1, α_2, $\ldots$, α_k.

Теорема 43: Решенията на хомогенно лин. рек. у-ние образуват линейно пространство

Нека S е множеството от всички решения на (5.85). Тогава $(S, +, \cdot)$ е линейно пространство с димесия k, където $+$ е сумиране на елементи, а $\cdot$ е умножение със скалар.

Доказателство: Следва лесно от Лема 38. Базис може да е фундаменталното множество от решения от Теорема 40. □

Извеждане на решението за хомогенните уравнения. След като въведохме необходимите понятия, да видим как се решава хомогенно линейно рекурентно уравнение от ред k с константни коефициенти. Общият вид на уравнението е (5.85). Ще намерим фундаментално решение и оттам ще намерим общото решение.

Допускаме, че решенията на (5.85) са от вида λ^n, където λ е комплексно число. Замествайки в (5.85), получаваме

$$\lambda^k + c_1\lambda^{k-1} + c_2\lambda^{k-2} + \cdots + c_{k-1}\lambda + c_k = 0 \tag{5.93}$$

Уравнение (5.93) е *характеристичното уравнение* на рекурентното уравнение (5.85). Съгласно основната теорема на алгебрата, (5.93) има k на брой комплексни, не непременно различни, корени. Нулата не е корен, тъй като $c_k \neq 0$. Тези корени са *характеристичните корени* и решението се получава чрез тях[6]. Нека характеристичните корени са $\lambda_1, \ldots, \lambda_k$.

Случай i Характеристичните корени са два по два различни. Ще покажем, че в този случай множеството $\{\lambda_1^n, \ldots, \lambda_k^n\}$ е фундаментално множество от решения. За да се убедим, че е така, да разгледаме детерминантата на Casorati $W(0)$:

$$W(0) = \det \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ \lambda_1 & \lambda_2 & \lambda_3 & \cdots & \lambda_k \\ \lambda_1^2 & \lambda_2^2 & \lambda_3^2 & \cdots & \lambda_k^2 \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \lambda_1^{k-1} & \lambda_2^{k-1} & \lambda_3^{k-1} & \cdots & \lambda_k^{k-1} \end{bmatrix} \tag{5.94}$$

Това е пример за детерминанта на Vandermonde, за която има добре известно решение. А именно,

$$W(0) = \prod_{1 \leqslant i < j \leqslant k} (\lambda_i - \lambda_j)$$

Щом $\lambda_i \neq \lambda_j$ при $i \neq j$, очевидно $W(0) \neq 0$. Следователно, $\{\lambda_1^n, \lambda_2^n, \ldots, \lambda_k^n\}$ е фундаментално множество от решения на (5.85). Оттук следва, че общото решение е

$$x(n) = \sum_{i=1}^{k} \alpha_i \lambda_i^n$$

От гледна точка на анализа на алгоритми по-точно решение не ни интересува. БОО, нека най-голямата от λ_i е λ_1, като това е и най-голямата по абсолютна стойност ламбда. Тогава решение е $T(n) \asymp \lambda_1^n$.

Случай ii По-интересният случай е случаят, в който има кратни корени. Това е и общият случай. **Случай i** е само негов подслучай, който ние разгледахме "за загравяне". Нека различните корени са $\lambda_1, \lambda_2, \ldots, \lambda_r$ с кратности съответно $m_1, m_2, \ldots, m_r$. Очевидно $\sum_{i=1}^{r} m_i = k$.
Можем да запишем (5.85) като

$$(\mathbf{E}^k + c_1\mathbf{E}^{k-1} + c_2\mathbf{E}^{k-2} + \cdots + c_{k-1}\mathbf{E} + c_k\mathbf{I})x(n) = 0$$

което можем да запишем като

$$(\mathbf{E} - \lambda_1)^{m_1}(\mathbf{E} - \lambda_2)^{m_2} \cdot \ldots \cdot (\mathbf{E} - \lambda_r)^{m_r}x(n) = 0 \tag{5.95}$$

факторизирайки $\boxed{\mathbf{E}^k + c_1\mathbf{E}^{k-1} + \cdots + c_k}$ до $\boxed{(\mathbf{E} - \lambda_1)^{m_1}(\mathbf{E} - \lambda_2)^{m_2} \cdot \ldots \cdot (\mathbf{E} - \lambda_r)^{m_r}}$.

Да разгледаме само

$$(\mathbf{E} - \lambda_i)^{m_i}x(n) = 0 \tag{5.96}$$

за кое да е $i \in \{1, \ldots, r\}$. Твърдим, че всяка функция $h(n)$, която е решение на (5.96), е решение и на (5.85). Наистина, ако $h(n)$ е такова решение, то $(\mathbf{E}-\lambda_i)^{m_i}h(n) = 0$. Тогава

$$\begin{aligned}
&(\mathbf{E} - \lambda_1)^{m_1}(\mathbf{E} - \lambda_2)^{m_2} \cdot \ldots \cdot (\mathbf{E} - \lambda_r)^{m_r}h(n) = \\
&(\mathbf{E} - \lambda_1)^{m_1} \cdot \ldots \cdot (\mathbf{E} - \lambda_{i-1})^{m_{i-1}} \\
&\quad (\mathbf{E} - \lambda_{i+1})^{m_{i+1}} \cdot \ldots \cdot (\mathbf{E} - \lambda_r)^{m_r} \underbrace{(\mathbf{E} - \lambda_i)^{m_i}h(n)}_{0} = \\
&(\mathbf{E} - \lambda_1)^{m_1} \cdot \ldots \cdot (\mathbf{E} - \lambda_{i-1})^{m_{i-1}} \\
&\quad (\mathbf{E} - \lambda_{i+1})^{m_{i+1}} \cdot \ldots \cdot (\mathbf{E} - \lambda_r)^{m_r}0 = 0
\end{aligned}$$

Да допуснем, че сме намерили множество от фундаментални решения за уравненията $(\mathbf{E} - \lambda_i)^{m_i}x(n)$, за всяко $i \in \{1, \ldots, r\}$. Ще покажем, че тяхното обединение е множество от фундаментални решения за (5.95).

Лема 39

Множеството

$$G_i = \left\{\lambda_i^n, \binom{n}{1}\lambda_i^{n-1}, \binom{n}{2}\lambda_i^{n-2}, \ldots, \binom{n}{m_i - 1}\lambda_i^{n-m_i+1}\right\}$$

е множество от фундаментални решения на (5.96).

Доказателство: Съгласно Следствие 19, достатъчно е да покажем, че $W(0) \neq 0$,

където $W(0)$ е следната детерминантата на Casorati:

$$\begin{vmatrix}
\lambda_i^0 & \binom{0}{1}\lambda_i^{0-1} & \binom{0}{2}\lambda_i^{0-2} & \cdots & \binom{0}{m_i-2}\lambda_i^{0-m_i+2} & \binom{0}{m_i-1}\lambda_i^{0-m_i+1} \\
\lambda_i^1 & \binom{1}{1}\lambda_i^{1-1} & \binom{1}{2}\lambda_i^{1-2} & \cdots & \binom{1}{m_i-2}\lambda_i^{1-m_i+2} & \binom{1}{m_i-1}\lambda_i^{1-m_i+1} \\
\lambda_i^2 & \binom{2}{1}\lambda_i^{2-1} & \binom{2}{2}\lambda_i^{2-2} & \cdots & \binom{2}{m_i-2}\lambda_i^{2-m_i+2} & \binom{2}{m_i-1}\lambda_i^{2-m_i+1} \\
\vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\
\lambda_i^{m_i-2} & \binom{m_i-2}{1}\lambda_i^{m_i-2-1} & \binom{m_i-2}{2}\lambda_i^{m_i-2-2} & \cdots & \binom{m_i-2}{m_i-2}\lambda_i^{m_i-2-m_i+2} & \binom{m_i-2}{m_i-1}\lambda_i^{m_i-2-m_i+1} \\
\lambda_i^{m_i-1} & \binom{m_i-1}{1}\lambda_i^{m_i-1-1} & \binom{m_i-1}{2}\lambda_i^{m_i-1-2} & \cdots & \binom{m_i-1}{m_i-2}\lambda_i^{m_i-1-m_i+2} & \binom{m_i-1}{m_i-1}\lambda_i^{m_i-1-m_i+1}
\end{vmatrix}$$

Припомняме си, че биномният коефициент е нула, ако долният индекс е по-голям от горния, поради което елементите над главния диагонал са нули. А елементите по главния диагонал са очевидно единици, следователно $W(0)$ е:

$$\begin{vmatrix}
1 & 0 & 0 & \cdots & 0 & 0 \\
\lambda_i^1 & 1 & 0 & \cdots & 0 & 0 \\
\lambda_i^2 & \binom{2}{1}\lambda_i^{2-1} & 1 & \cdots & 0 & 0 \\
\vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\
\lambda_i^{m_i-2} & \binom{m_i-2}{1}\lambda_i^{m_i-2-1} & \binom{m_i-2}{2}\lambda_i^{m_i-2-2} & \cdots & 1 & 0 \\
\lambda_i^{m_i-1} & \binom{m_i-1}{1}\lambda_i^{m_i-1-1} & \binom{m_i-1}{2}\lambda_i^{m_i-1-2} & \cdots & \binom{m_i-1}{m_i-2}\lambda_i^{m_i-1-m_i+2} & 1
\end{vmatrix}$$

От линейната алгебра знаем, че такава детерминанта е произведението от елементите по главния диагонал, а то е 1. И така, $W(0) = 1^6$, поради което множеството $G_i = \left\{\lambda_i^n, \binom{n}{1}\lambda_i^{n-1}, \binom{n}{2}\lambda_i^{n-2}, \ldots, \binom{n}{m_i-1}\lambda_i^{n-m_i+1}\right\}$ е фундаментално множество от решения.
Остава да покажем, че $\binom{n}{t}\lambda_i^{n-t}$ е решение на (5.96) за всяко $t \in \{0, 1, \ldots, m_i - 1\}$. Наистина, имайки предвид, че $\binom{n}{t}$ е полином на n от степен t, в сила е:

$$\begin{aligned}
&(E - \lambda_i)^{m_i}\binom{n}{t}\lambda_i^{n-t} = && \text{// съгласно (5.83)} \\
&\lambda_i^{n-t}(\lambda_i E - \lambda_i)^{m_i}\binom{n}{t} = \\
&\lambda_i^{n-t+m_i}(E - I)^{m_i}\binom{n}{t} = && \text{// съгласно (5.81)} \\
&\lambda_i^{n-t+m_i}\Delta^{m_i}\binom{n}{t} = && \text{// понеже } t < m_i \text{ и съгласно (5.82)} \\
&0
\end{aligned}$$

□

Теорема 44: Theorem 2.23 от [23, стр. 77]: фунд. м-во от решения при кратни корени

Множеството $\mathsf{G} = \bigcup_{i=1}^{r} \mathsf{G}_i$ е фундаментално множество от решения на (5.95).

Доказателство: Съгласно Лема 39, функциите от G са решения на (5.95). Да разгледаме детерминантата на Casorati $W(0)$. Тъй общият вид на $W(0)$ не е особено прегледен и информативен, ако матрицата трябва да се запише на една страница (вж. [23, стр. 77]), да разгледаме конкретен пример. Да кажем, че $k = 6$, $r = 2$, $m_1 = 4$ и $m_2 = 2$. Тогава

$$W(0) = \det \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ \lambda_1 & 1 & 0 & 0 & \lambda_2 & 1 \\ \lambda_1^2 & 2\lambda_1 & 1 & 0 & \lambda_2^2 & 2\lambda_2 \\ \lambda_1^3 & 3\lambda_1^2 & 3\lambda_1^1 & 1 & \lambda_2^3 & 3\lambda_2^2 \\ \lambda_1^4 & 4\lambda_1^3 & 6\lambda_1^2 & 4\lambda_1 & \lambda_2^4 & 4\lambda_2^3 \\ \lambda_1^5 & 5\lambda_1^4 & 10\lambda_1^3 & 10\lambda_1^2 & \lambda_2^5 & 5\lambda_2^4 \end{bmatrix} \tag{5.97}$$

Сравнете (5.97) с детерминантата от (5.94). В (5.94) на всяко λ_i съответства точно една колона, в която, отгоре надолу, са записани степените на това λ_i, започвайки от нула. В (5.97), на всяко λ_i съответстват точно m_i колони, оформени като блок; в (5.97) блоковете са очертани в червено за λ_1 и синьо за λ_2. Клетката на ред t и колона j в блока, съответстващ на λ_i, съдържа $\binom{t}{j}\lambda_i^t$, ако броенето на колоните започва от нула. Очевидно в най-лявата колона на блока са степените на λ_i, понеже долните индекси на биномните коефициенти са нули и по този начин биномните коефициенти са единици. Това е пример за *обобщена детерминанта на Vandermonde*, за която има добре известно решение [48]. А именно,

$$W(0) = \prod_{1 \leqslant i < j \leqslant r} (\lambda_i - \lambda_j)^{m_i \cdot m_j}$$

Тъй като $\lambda_i \neq \lambda_j$ при $i \neq j$, то $W(0) \neq 0$. Тогава G е множество от фундаментални решения на (5.95). □

Следствие 20 на Теорема 44 е математическа обосновка на метода с характеристичното уравнение, що се отнася до хомогенните уравнения.

Следствие 20

Общото решение на (5.95) се дава от

$$x(n) = \sum_{i=1}^{r} \left(\alpha_{i,0} + \alpha_{i,1} n + \cdots + \alpha_{i,m_i-1} n^{m_i-1}\right) \lambda_i^n$$

където $\alpha_{i,j}$ за $1 \leqslant i \leqslant r$, $0 \leqslant j \leqslant m_i - 1$ са константи.

Извеждане на решението за нехомогенните уравнения. За съжаление, Следствие 20 не е достатъчно, за да решаваме рекурентни уравнения, които описват сложност на алгоритми. Всяко рекурентно уравнение, описващо адекватно сложност на (рекурсивен) алгоритъм, неизбежно има нехомогенна част, която е поне единица, защото алгоритъмът не може да не върши работа поне $\Theta(1)$ извън рекурсивните викания. Ерго, хомогенното уравнение (5.85) не може да описва сложност на алгоритъм. Сега разглеждаме съответно нехомогенно уравнение в общ вид:

$$x(n+k) + c_1x(n+k-1) + c_2x(n+k-2) + \cdots + c_{k-1}x(n+1) + c_kx(n) = g(n) \quad (5.98)$$

където $g(n)$ е нехомогенната част. *Съответното хомогенно уравнение* е

$$x(n+k) + c_1x(n+k-1) + c_2x(n+k-2) + \cdots + c_{k-1}x(n+1) + c_kx(n) = 0 \quad (5.99)$$

Забележете, че то е същото като (5.85); тук е записано отново за прегледност.

Задача 22

Разгледайте нехомогенното уравнение

$$x(n+2) - x(n+1) - 6x(n) = 5 \cdot 3^n \quad (5.100)$$

1. Намерете общото решение на съответното хомогенно уравнение.
2. Проверете дали 3^n и $(-2)^n$ са решения на (5.100).
3. Проверете дали $x_1(n) = n3^{n-1}$ и $x_2(n) = (1+n)3^{n-1}$ са решения на (5.100).
4. Покажете, че $x_2(n) - x_1(n)$ не е решение на (5.100).
5. Покажете, че не е вярно, че $c \cdot x_1(n)$ за всяка константа c е решение на (5.100).

Решение: По 1: съответното хомогенно уравнение е

$$x(n+2) - x(n+1) - 6x(n) = 0 \quad (5.101)$$

Характеристичното уравнение е

$$\lambda^2 - \lambda - 6 = 0$$

Корените му са $\lambda_1 = 3$ и $\lambda_2 = -2$. Общото му решение, съгласно Следствие 20, е

$$x(n) = \alpha_1 3^n + \alpha_2(-2)^n$$

Естествено, 3^n и $(-2)^n$ са решения на (5.101):

$$3^{n+2} - 3^{n+1} - 6 \cdot 3^n = 9 \cdot 3^n - 3 \cdot 3^n - 6 \cdot 3^n = 0$$
$$(-2)^{n+2} - (-2)^{n+1} - 6 \cdot (-2)^n = 4 \cdot (-2)^n - (-2)(-2)^n - 6 \cdot (-2)^n = 0$$

По 2: нито 3^n, нито $(-2)^n$ са решения на (5.100). Току-що видяхме, че заместването на всяко от тях в лявата страна на (5.100) дава нула, а дясната страна на (5.100) не е нула.

По 3: $n3^{n-1}$ е решение. Заместваме $x(n)$ с $n3^{n-1}$ в лявата страна на (5.100):

$$\begin{aligned}(n+2)3^{n+1} - (n+1)3^n - 6n3^{n-1} &= 3^{n-1}\big((2+n)3^2 - (1+n)3^1 - 6n\big)\\ &= 3^{n-1}\big(18 + 9n - 3 - 3n - 6n\big)\\ &= 3^{n-1}\big(15\big)\\ &= 5 \cdot 3^n\end{aligned}$$

$(1+n)3^{n-1}$ също е решение. Пак заместваме $x(n)$ вляво, този път с $(1+n)3^{n-1}$:

$$\begin{aligned}(1+n+2)3^{n+1} - (1+n+1)3^n - 6(1+n)3^{n-1} &= 3^{n-1}\big((3+n)3^2 - (2+n)3^1 - 6(1+n)\big)\\ &= 3^{n-1}\big(27 + 9n - 6 - 3n - 6 - 6n\big)\big)\\ &= 3^{n-1}\big(15\big)\big)\\ &= 5 \cdot 3^n\end{aligned}$$

По 4: Вярно е, че $x_2(n) - x_1(n) = 3^{n-1}$. Вече показахме, че 3^n е решение на съответното хомогенно уравнение, а оттук следва, че и $3^{n-1} = \frac{1}{3}3^n$ е негово решение. Тогава няма как 3^{n-1} да е решение на нехомогенното, чиято дясна страна е $5 \cdot 3^n$.

По 5: Заместваме $x(n)$ със $cn3^{n-1}$ в лявата страна на (5.100):

$$\begin{aligned}c(n+2)3^{n+1} - c(n+1)3^n - 6cn3^{n-1} &= c3^{n-1}\big((2+n)3^2 - (1+n)3^1 - 6n\big)\\ &= c3^{n-1}\big(18 + 9n - 3 - 3n - 6n\big)\\ &= c3^{n-1}\big(15\big)\\ &= 5c3^n\end{aligned}$$

Ако $c \neq 1$, то $5c3^n$ не е същото като $5 \cdot 3^n$. □

Задача 22 показва, че, за разлика от хомогенните уравнения (вж. Теорема 43), при нехомогенните уравнения решенията не образуват линейно пространство: нито сумата на две решения е решение, нито произведението на решение с произволен скалар е решение.

Задача 22 ни навежда и на мисълта, че разликата от две решения на нехомогенното уравнение (5.98) е решение на съответното хомогенно уравнение (5.99). Ще оформим това в отделна теорема, въпреки че доказателството е твърде просто.

Теорема 45: Разликата м-у две реш. на нехомогенното е реш. на хомогенното у-ние

Разликата между всеки две решения на нехомогенно линейно рекурентно уравнение е решение на съответното хомогенно уравнение.

Доказателство: Нека $x_1(n)$ и $x_2(n)$ са решения на (5.98). Тогава

$$x_1(n+k) + c_1 x_1(n+k-1) + \cdots + c_{k-1} x_1(n+1) + c_k x_1(n) = g(n)$$
$$x_2(n+k) + c_1 x_2(n+k-1) + \cdots + c_{k-1} x_2(n+1) + c_k x_2(n) = g(n)$$

Изваждаме втория ред от първи и получаваме:

$$\big(x_1(n+k) - x_2(n+k)\big) + c_1\big(x_1(n+k-1) - x_2(n+k-1)\big) + \cdots + c_{k-1}\big(x_1(n+1) - x_2(n+1)\big) + c_k\big(x_1(n) - x_2(n)\big) = 0$$

Виждаме, че $x_1(n) - x_2(n)$ е решение на (5.99). □

Да кажем, че общото решение на хомогенното (5.99) е $x_h(n)$. Да кажем, че $x_p(n)$ означава кое е да е решение[2] на нехомогенното (5.98). Ще наричаме $x_p(n)$, "частно решение". Следната теорема ни казва как може да конструираме всички решения на нехомогенното (5.98).

Теорема 46: Theorem 2.40 от [23, стр. 84]: общото решение на нехомогенното уравнение

Всяко решение на нехомогенното (5.98) може да бъде записано като

$$x(n) = x_p(n) + \sum_{i=1}^{k} \alpha_i x_i(n)$$

където $\{x_1(n), \ldots, x_k(n)\}$ е фундаментално множество от решения на хомогенното (5.99), а $x_p(n)$ е частно решение.

Доказателство: Според Теорема 46, разликата $x(n) - x_p(n)$ е решение на хомогенното (5.99). Съгласно Теорема 42, това решение е от вида $\sum_{i=1}^{k} \alpha_i x_i(n)$, където $x_1(n)$, ..., $x_k(n)$ са елементите на някое фундаментално множество от решения на въпросното хомогенно уравнение. □

Теорема 46 ни дава право да направим следното определение.

Определение 44: Общото решение на нехомогенно линейно рекурентно уравнение

Нека $x_h(n)$ е общото решение на хомогенното уравнение (5.99). Общото решение на нехомогенното уравнение (5.98) е $x_h(n) + x_p(n)$, където $x_p(n)$ е частно решение на (5.98).

Ние вече знаем как се намира общото решение на хомогенното уравнение. Ако имаме и начин да намерим някое частно решение, то имаме начин да намерим и общото решение на нехомогенното. Методът, който се използва за намиране на частно решение, се нарича *методът на неопределените коефициенти* ([23, стр. 85]). Той не е универсален: трябва нехомогенната част да е от определен вид. А именно, да е линейна комбинация от функции от вида $n^b a^n$, където a и b са константи. Има и други видове функции, за които методът работи, но по отношение на изследването на сложността на рекурсивни

алгоритми е достатъчно да разгледаме нехомогенна част, която е линейна комбинация на функции от вида $n^b a^n$. В (5.76) нехомогенната част е точно такава.

Да запишем (5.98) така:

$$\mathbf{p}(\mathbf{E})x(n) = g(n) \tag{5.102}$$

където $\mathbf{p}(\mathbf{E}) = \mathbf{E}^k + c_1\mathbf{E}^{k-1} + c_2\mathbf{E}^{k-2} + \cdots + c_{k-1}\mathbf{E} + c_k\mathbf{I}$. Нека $\mathbf{N}(\mathbf{E})$ е анихилатор за $g(n)$ от (5.102). Тогава прилагаме анихилатора върху двете страни на уравнението и получаваме

$$\mathbf{N}(\mathbf{E})\,\mathbf{p}(\mathbf{E})\,x(n) = 0 \tag{5.103}$$

Тъй като търсим само асимптотиката на решението, а не истинското решение с точните константи, (5.103) е напълно достатъчно. В [23, стр. 85–87] акцентът е върху намиране на точното решение, откъдето идва и името "методът с неопределените коефициенти". Нас тези коефициенти-константи не ни интересуват, така че третираме (5.103) по начин, напълно аналогичен на начина, по който третирахме (5.95). Уравнение (5.95) бе получено от хомогенното (5.85), прилагайки оператора полином, факторизиран по характеристичните корени. Уравнение (5.103) бе получено от нехомогенното (5.98). Очевидно лявата страна на (5.103) може да бъде факторизирана по корените на алгебрично уравнение, което се получава от композицията на оператора полином, който "идва" от хомогенната част, и анихилатора, който "идва" от нехомогенната част. Мултимножеството от получените корени е обединението на мултимножествата от

- характеристичните корени на $\mathbf{p}(\mathbf{E})x(n) = 0$ и
- характеристичните корени на $\mathbf{N}(\mathbf{E})x(n) = 0$.

Да разгледаме отново примера (5.100). Съответното хомогенно е $x(n+2) - x(n+1) - 6x(n) = 0$ с мултимножество от корените $\{3, -2\}$, така че операторът полином е $\mathbf{p}(\mathbf{E}) = (\mathbf{E}-3)(\mathbf{E}+2)$. Анихилаторът на $5 \cdot 3^n$ е $\mathbf{E}-3$ (петицата е без значение за анихилатора и оттам, за общото решение). Тогава операторът от дясната страна на (5.103) е $(\mathbf{E}-3)^2(\mathbf{E}+2)$. Очевидно е в какъв смисъл коренът 3, "идващ" от дясната страна чрез $(\mathbf{E}-3)x(n) = 0$, "влиза" в мултимножеството от корените. Асимптотиката на решението е $\Theta(n3^n)$.

---

[a] Felice Casorati е италиански математик.

[б] Читателят може да възрази, че в общия случай алгебрични уравнения от степен $\geqslant 5$ нямат точни решения в радикали. Това е вярно, но не е съществено от гледна точка на анализа на алгоритми в практиката. Дори характеристичното уравнение да е от степен $\geqslant 5$, характеристичните корени могат да бъдат апроксимирани числено с каквато точност искаме. Тъй като всеки корен λ участва в решението като λ^n, то, ако решението е $T(n) \asymp \lambda^n$, очевидно никоя апроксимация $\widetilde{\lambda}$, която е различна от λ, няма да даде правилната асимптотика на решението, понеже или $\lambda^n \prec \widetilde{\lambda}^n$, или $\lambda^n \succ \widetilde{\lambda}^n$.

Но това е само от гледна точка на теоретичен перфекционизъм. На практика, намирането на $\widetilde{\lambda}$ с точност до четвъртия или шестия знак след десетичната точка ще ни даде достатъчно прецизна приблизителна оценка за сложността на алгоритъма, който разглеждаме. Като пример, нека водещото събираимо в израза за $T(n)$ е $3\lambda^n$ и ние пресметнем с числени методи, че $\lambda \approx 2.0667$, като четирите цифри след десетичната точка са коректни. Тогава изразът $3 \cdot 2.0667^n$ ни дава чудесна представа за това, колко дълго би работила софтуерна имплементация на този алгоритъм за разумно големи стойности на n (за които изобщо има смисъл да пускаме експоненциалния алгоритъм). Това, че 2.0667 е само апроксимация, няма значение. То "върши работа".

Нещо повече. Дори да намерим λ с точен израз с радикали, да кажем $\lambda = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)$ (намесени са числата на Fibonacci!), то, ако искаме да преценим колко дълго би работил софтуер, имплементиращ алгоритъм със сложност по време $\Theta(\lambda^n)$, ние ще вземем някаква числена апроксимация на λ.

[6]В книгата на Elaydi [23, стр. 76] има очевидна грешка в крайния етап на извеждането на $W(0)$.

[7]Субскриптът "p" в "$x_p(n)$" идва от "particular".

Ще разгледаме три примера за решаване на рекурентни уравнения с метода с характеристичното уравнение.

Задача 23

Решете рекурентното уравнение

$$T(n) = T(n-1) + 1$$

Решение: Преписваме рекурентното уравнение така: $T(n) = T(n-1) + 1^n n^0$, за да сме сигурни, че фо́рмата му удовлетворява (5.76). Характеристичното уравнение е $x - 1 = 0$ с единствен корен $x_1 = 1$. Тогава мултимножеството от корените на характеристичното уравнение е $\{1\}_M$. Използвайки конвенцията за именуване на (5.76), $m = 1$, $b_1 = 1$, и $d_1 = 0$. Така че към мултимножеството $\{1\}_M$ добавяме $b_1 = 1$ с кратност $d_1 + 1 = 1$, получавайки $\{1, 1\}_M$. Тогава $T(n) = A\,1^n + B\,n\,1^n$ за някакви константи A и B, следователно $T(n) = \Theta(n)$. □

Задача 24

Решете рекурентното уравнение

$$T(n) = 4T(n-3) + 1 \tag{5.104}$$

Решение: Характеристичното уравнение е

$$x^3 - 4 = 0$$

Корените му са

$$x_1 = \sqrt[3]{4}$$
$$x_2 = \sqrt[3]{4}\,e^{i\frac{2\pi}{3}}$$
$$x_3 = \sqrt[3]{4}\,e^{i\frac{-2\pi}{3}}$$

Имайки предвид и нехомогенната част, мултимножеството от корените е

$$\{\sqrt[3]{4}, \sqrt[3]{4}\,e^{i\frac{2\pi}{3}}, \sqrt[3]{4}\,e^{i\frac{-2\pi}{3}}, 1\}_M$$

Решението, спрямо някакви комплексни константи A, B, C и D е:

$$\begin{aligned}
T(n) &= A\left(\sqrt[3]{4}\right)^n + B\left(\sqrt[3]{4}\right)^n e^{\frac{2\pi n i}{3}} + C\left(\sqrt[3]{4}\right)^n e^{\frac{-2\pi n i}{3}} + D1^n = \\
&= A\left(\sqrt[3]{4}\right)^n + B\left(\sqrt[3]{4}\right)^n \left(\cos\left(\frac{2\pi n}{3}\right) + i\sin\left(\frac{2\pi n}{3}\right)\right) + \\
&\quad C\left(\sqrt[3]{4}\right)^n \left(\cos\left(\frac{-2\pi n}{3}\right) + i\sin\left(\frac{-2\pi n}{3}\right)\right) + D \\
&= A\left(\sqrt[3]{4}\right)^n + \left(\sqrt[3]{4}\right)^n \cos\left(\frac{2\pi n}{3}\right)(B+C) \;+\; \left(\sqrt[3]{4}\right)^n \sin\left(\frac{2\pi n}{3}\right)(B-C)i + D \qquad (5.105)
\end{aligned}$$

Гарантирано A е реално число, а константите B, C и D са такива комплексни числа, че $T(n)$ е цяло положително за всяко цяло положително n, надхвърлящо аргументите на трите неупоменати начални условия, при допускането, че началното условия задават цели положителни числа на T. Причината е много проста: няма как (5.104) да "произведе" дори едно число, което не е цяло положително, ако стартира с цяли положителни числи. И така, от (5.105) извеждаме веднага, че $T(n) \asymp \left(\sqrt[3]{4}\right)^n$. □

Допълнение 30: Когато има комплексни характеристични корени

Разсъждаваме върху точното решение на

$$T(n) = 4T(n-3) + 1$$

имайки предвид общото решение (5.105).

Ако вземем $B = C = \frac{1}{2}$, получаваме едно решение:

$$T_1(n) = A\left(\sqrt[3]{4}\right)^n + \left(\sqrt[3]{4}\right)^n \cos\left(\frac{2\pi n}{3}\right) + D$$

Ако вземем $B = -\frac{1}{2}i$ and $C = \frac{1}{2}i$, получаваме друго решение:

$$T_2(n) = A\left(\sqrt[3]{4}\right)^n + \left(\sqrt[3]{4}\right)^n \sin\left(\frac{2\pi n}{3}\right) + D$$

Съгласно принципът на суперпозицията (Лема 38), ако имаме линейно рекурентно уравнение и знаем, че някакви функции g_i, където $1 \leqslant i \leqslant k$, са решения, то всяка тяхна линейна комбинация също е решение. Тогава имаме общо решение

$$T(n) = A_1\left(\sqrt[3]{4}\right)^n + A_2\left(\sqrt[3]{4}\right)^n \cos\left(\frac{2\pi n}{3}\right) + A_3\left(\sqrt[3]{4}\right)^n \sin\left(\frac{2\pi n}{3}\right) + A_4 \qquad (5.106)$$

за някакви константи A_1, A_2, A_3 и A_4. Асимптотиката на решението е $T(n) = \Theta\left(\left(\sqrt[3]{4}\right)^n\right)$.

Точното решение на (5.104) зависи от началните условия, които не са дадени. Да кажем,

че началните условия са следните:

$$\begin{aligned}T(1) &= 1\\ T(2) &= 2\\ T(3) &= 3\\ T(4) &= 4\end{aligned}$$

Щом началните условия са цели числа, то константите $A_1, \ldots, A_4$ в (5.106) трябва да са такива, че $T(n)$ е цяло число за всяко $n \in \mathbb{N}^+$.
Първо да разгледаме общото решение (5.106) и да съобразим какви стойности вземат $\cos\left(\frac{2\pi n}{3}\right)$ и $\sin\left(\frac{2\pi n}{3}\right)$ при цели положителни стойности на n:

n	1	2	3	4	5	6	7	8	9	10
$\cos\left(\frac{2\pi n}{3}\right)$	$-\frac{1}{2}$	$-\frac{1}{2}$	1	$-\frac{1}{2}$	$-\frac{1}{2}$	1	$-\frac{1}{2}$	$-\frac{1}{2}$	1	$-\frac{1}{2}$
$\sin\left(\frac{2\pi n}{3}\right)$	$\frac{\sqrt{3}}{2}$	$-\frac{\sqrt{3}}{2}$	0	$\frac{\sqrt{3}}{2}$	$-\frac{\sqrt{3}}{2}$	0	$\frac{\sqrt{3}}{2}$	$-\frac{\sqrt{3}}{2}$	0	$\frac{\sqrt{3}}{2}$

Забележете какво означава това за общото решение (5.106). Тъй като $\cos\left(\frac{2\pi n}{3}\right)$ и $\sin\left(\frac{2\pi n}{3}\right)$ задават периодични числови редици, и двете с период 3, можем да мислим за (5.106) като за **три** израза:

$$T(n) = \begin{cases} A_1\left(\sqrt[3]{4}\right)^n - A_2\frac{\left(\sqrt[3]{4}\right)^n}{2} + A_3\frac{\left(\sqrt[3]{4}\right)^n\sqrt{3}}{2} + A_4, & \text{ако } n \equiv 1 \mod 3\\ A_1\left(\sqrt[3]{4}\right)^n - A_2\frac{\left(\sqrt[3]{4}\right)^n}{2} - A_3\frac{\left(\sqrt[3]{4}\right)^n\sqrt{3}}{2} + A_4, & \text{ако } n \equiv 2 \mod 3\\ A_1\left(\sqrt[3]{4}\right)^n + A_2\left(\sqrt[3]{4}\right)^n + A_4, & \text{ако } n \equiv 0 \mod 3\end{cases} \tag{5.107}$$

За да намерим $A_1, \ldots, A_4$, в (5.106) последователно заместваме n с 1, 2, 3 и 4, като за $T(1), \ldots, T(4)$ ползваме началните условия, и така получаваме следната система:

$$\begin{aligned}1 &= A_1\,4^{1/3} - A_2\frac{4^{1/3}}{2} + A_3\frac{4^{1/3}\sqrt{3}}{2} + A_4\\ 2 &= A_1\,4^{2/3} - A_2\frac{4^{2/3}}{2} - A_3\frac{4^{2/3}\sqrt{3}}{2} + A_4\\ 3 &= A_1 + A_2\,4 + A_4\\ 4 &= A_1\,4^{4/3} - A_2\,2\cdot 4^{1/3} + A_3\,2\sqrt{3}\,4^{1/3} + A_4\end{aligned}$$

Решението на системата е

$$\begin{aligned}A_1 &= \frac{1}{4} + \frac{4^{2/3}}{12} + \frac{4^{1/3}}{6}\\ A_2 &= -\frac{4^{2/3}}{12} - \frac{4^{1/3}}{6} + \frac{1}{2}\\ A_3 &= \frac{4^{2/3}\sqrt{3}}{12} - \frac{4^{1/3}\sqrt{3}}{6}\\ A_4 &= 0\end{aligned}$$

Тогава точното решение е

$$\begin{aligned}\mathsf{T}(n) = &\left(\frac{1}{4} + \frac{4^{2/3}}{12} + \frac{4^{1/3}}{6}\right) 4^{n/3} + \\ &\left(-\frac{4^{2/3}}{12} - \frac{4^{1/3}}{6} + \frac{1}{2}\right) 4^{n/3} \cos\left(2\pi n/3\right) + \\ &\left(\frac{4^{2/3}\sqrt{3}}{12} - \frac{4^{1/3}\sqrt{3}}{6}\right) 4^{n/3} \sin\left(2\pi n/3\right)\end{aligned} \tag{5.108}$$

Колкото и да е странно, този израз е цяло положително число за всяко $n \in \mathbb{N}^+$.

Имайки предвид, че $\cos\left(\frac{2\pi n}{3}\right)$ и $\sin\left(\frac{2\pi n}{3}\right)$ задават периодични числови редици, и двете с период 3, можем да мислим за (5.108) като за три отделни израза, всеки с константни коефициенти – също както (5.107).

Задача 25

Решете рекурентното уравнение

$$\mathsf{T}(n) = 15\mathsf{T}(n-1) - 81\mathsf{T}(n-2) + 185\mathsf{T}(n-3) - 150\mathsf{T}(n-4) + n^2 2^{n+1} + (3n+1)2^{2n} + (3n+7)2^n + 7n^3 + 17$$

Решение: Първо да препишем уравнението по такъв начин, че нехомогенната част да стане от "правилния вид". В момента тя не е от правилния вид: в степенните показатели се срещат $n+1$ и $2n$, което не е разрешено, и освен това се среща два пъти експоненциална функция с основа 2, което също не е разрешено. И така,

$$\mathsf{T}(n) = 15\mathsf{T}(n-1) - 81\mathsf{T}(n-2) + 185\mathsf{T}(n-3) - 150\mathsf{T}(n-4) + (2n^2+3n+7)2^n + (3n+1)4^n + (7n^3+17)1^n$$

Сега вече нехомогенната част е в правилния вид.

Хомогенното уравнение е

$$\mathsf{T}(n) = 15\mathsf{T}(n-1) - 81\mathsf{T}(n-2) + 185\mathsf{T}(n-3) - 150\mathsf{T}(n-4)$$

Характеристичното уравнение е

$$x^4 - 15x^3 + 81x^2 - 185x + 150 = 0$$

Мултимножеството от корените е $\{2, 3, 5, 5\}_M$. От нехомогенната част имаме мултимножество $\{1, 1, 1, 1, 2, 2, 2, 4, 4\}_M$. Обединението на мултимножествата е $\{1, 1, 1, 1, 2, 2, 2, 2, 3, 4, 4, 5, 5\}_M$. Най-големият елемент е 5 с кратност 2. Решението е $\mathsf{T}(n) \asymp n5^n$. □

5.4 Примери за ефикасни Разделяй-и-Владей алгоритми

Ще разгледаме два примера за задачи, които се решават ефикасно с алгоритми, конструирани по схемата Разделяй-и-Владей.

5.4.1 Най-близки елементи, 2D

Задачата е обобщение на Най-близки Елементи на на стр. 127. Сега са дадени не числа, а двойки от числа, и се търси най-близка двойка. Може да мислим за задачата като за

геометрична задача, където двойките са точки в Евклидовата равнина, но може да мислим и за по-общ контекст, в който точките са елементи на някакво фазово пространство.

Има качествена разлика между 1D и 2D вариантите на задачата: 1D вариантът има ефикасно решение със сортиране и премитане, което не се обобщава за повече измерения.

Оригиналната статия с решение на тази задача е от 1976 година и съдържа ефикасно решение както за двумерния случай, така и за k-мерния случай, ако k е константа [12].

Разстоянието между две двойки числа (a, b) и (c, d) най-често се дефинира като Евклидовото разстояние, което още се нарича L_2 разстоянието[†]

$$\sqrt{(a-c)^2 + (b-d)^2}$$

но може да бъде Манхатънското разстояние, което още се нарича L_1 разстоянието

$$|a-c| + |b-d|$$

или L_∞ разстоянието

$$\max\{|a-c|, |b-d|\}$$

Как точно се дефинира разстоянието не е особено важно, въпреки че някои детайли от алгоритъма, който ще разгледаме, зависят от това. Най-важното е разстоянието да може да се пресмята в $\Theta(1)$.

Изч. Задача: Най-близки Елементи, 2D, вар. 1

пример: Множество от точки $P = \{p_1, p_2, \ldots, p_n\}$, където $p_i = (a_i, b_i)$, където $a_i, b_i \in \mathbb{Q}$, за $1 \leqslant i \leqslant n$; $n \geqslant 2$
решение: $\min\{\text{dist}(p_i, p_j) : 1 \leqslant i < j \leqslant n\}$

Изч. Задача: Най-близки Елементи, 2D, вар. 2

пример: Множество от точки $P = \{p_1, p_2, \ldots, p_n\}$, където $p_i = (a_i, b_i)$, където $a_i, b_i \in \mathbb{Q}$, за $1 \leqslant i \leqslant n$; $n \geqslant 2$
решение: Двойка точки (p_i, p_j), такива че $i \neq j$ и $\text{dist}(p_i, p_j)$ е минимална.

Числата a_i и b_i са произволни рационални числа. Може две точки да имат една и съща първа координата или една и съща втора координата. Може и двете им координати да съвпадат, което означава, че това е една и съща точка в равнината. Може дори всички двойки-точки да съвпадат. Това не е непременно безсмислица. За целите на това изложение смятаме, че всяка точка се идентифицира с координатите си, но в някакъв практически контекст "точките" може да са много по-сложни данни и двете координати да са само два от многото атрибути на една точка.

Ние ще решим задачата в по-лесния първи вариант, в който се търси само оптималното разстояние. Има очевидно решение с брутална сила, опитващо всички ненаредени двойки точки, но то работи във време $\Theta(n^2)$, което е прекалено бавно. Ще построим алгоритъм, работещ в $\Theta(n \lg n)$, изграден по схемата Разделяй-и-Владей. Изложението тук повтаря в основни линии изложението в [21, стр. 1039].

[†] L_m разстоянието между (a, b) и (c, d) е $(|a-c|^m + |b-d|^m)^{\frac{1}{m}}$.

Ще направим пример с точките, показани на Фигура 5.7.

Фигура 5.7 : Точките, които ще ползваме за пример.



За удобство да въведем една нотация. Нека е дадена точка $p \in P$. Тогава "$p.x$" означава нейната първа координата, а "$p.y$" означава нейната втора координата. Например, на Фигура 5.7, $p_{11}.x = 41$, $p_8.y = 52$ и така нататък.

Предварителна обработка (preprocessing). Сортираме P веднъж по първата координата и записваме резултата в масив $A[1, \ldots, n]$, и после още веднъж по втората координата и записваме резултата в масив $B[1, \ldots, n]$. Подчертаваме две неща.

1. A и B са масиви от точки, а не масиви от числа. В нашия пример:

$$A = [p_5, p_2, p_{10}, p_{11}, p_1, p_7, p_8, p_4, p_{12}, p_6, p_9, p_3]$$
$$B = [p_7, p_9, p_{10}, p_4, p_1, p_5, p_3, p_{12}, p_{11}, p_8, p_2, p_6]$$

2. Сортирането **не е** част от Разделяй-и-Владей! Ако беше част от него, щяхме да сортираме на всяко ниво на рекурсията. Ние сортираме **само веднъж**: в предварителната обработка. По време на същинското изпълнение на рекурсивния алгоритъм не сортираме, а ползваме началната сортировка.

   Това е съществено за ефикасността на алгоритъма. Ако сортираме на всяко ниво на рекурсията (в която делим входа на две равни части и правим по едно рекурсивно викане върху всяка от тях), сложността по време ще се описва от рекурентното уравнение

$$T(n) = 2T\left(\frac{n}{2}\right) + \Omega(n \lg n)$$

   Причината да има нехомогенна част $\Omega(n \lg n)$ е, че сортирането изисква време поне $n \lg n$, в асимптотичния смисъл (вижте Подсекция 7.2.2). Но рекурентното уравнение $S(n) = 2S\left(\frac{n}{2}\right) + \Theta(n \lg n)$ има решение $S(n) = \Theta(n \lg^2 n)$ (вижте Теорема 35 в Допълнение 27). Тогава уравнението $T(n) = 2T\left(\frac{n}{2}\right) + \Omega(n \lg n)$ има решение $T(n) = \Omega(n \lg^2 n)$. Ерго, ако сортираме на всяко ниво на рекурсията, няма как да "слезем" под $\Theta(n \lg^2 n)$, така че $\Theta(n \lg n)$ би била непостижима сложност за алгоритъма.

Продължаваме изложението за предварителната обработка. Заради ефикасността на алгоритъма **има смисъл всеки p_i от B да съдържа и индекса на p_i в A**. Заради това, елементите на B всъщност са наредени двойки от вида (p_i, j), където $p_i \in P$, а j е позицията на p_i в A. Да си припомним, че в нашия пример според досегашните дефиниции имаме:

$$A = [p_5, p_2, p_{10}, p_{11}, p_1, p_7, p_8, p_4, p_{12}, p_6, p_9, p_3]$$
$$B = [p_7, p_9, p_{10}, p_4, p_1, p_5, p_3, p_{12}, p_{11}, p_8, p_2, p_6]$$

Сега казваме, че всъщност елементите на B са наредени двойки от точка и индекс. В нашия пример, искаме след предварителната обработка B да изглежда така:

$$B = [(p_7, 6), (p_9, 11), (p_{10}, 3), (p_4, 8), (p_1, 5), (p_5, 1), (p_3, 12), (p_{12}, 9), (p_{11}, 4), (p_8, 7), (p_2, 2), (p_6, 10)] \tag{5.109}$$

Смисълът от това ще стане ясен нататък.

Допълнение 31: Как да изчисляваме индексите на елементите на B

Индексите на елементите на B трябва да бъдат изчислени ефикасно. Не е ефикасна следната идея: след създаването на A и B като сортирани масиви от точки, за всеки елемент на B да търсим къде се намира той (същата точка) в A. Това би означавало предварителната обработка да има квадратична сложност, което би означавало на свой ред целият алгоритъм да има сложност $\Omega(n^2)$.
Ефикасен е следният начин за пресмятане на тези индекси. Поначало всеки от тези индекси е индексът на дадената точка в P, ако мислим за P като за масив[a]. В нашия пример, да кажем, че P е

$$P = [p_4, p_2, p_3, p_{12}, p_1, p_8, p_7, p_5, p_{10}, p_{11}, p_6, p_9]$$

Едно уточнение. "Истинските" елементи на P са наредени двойки от координати. Тези означения: "p_4", "p_2" и така нататък са съкратени записи за наше удобство. "Истинският" P е (Фигура 5.7):

$$P = [(69, 33), (29, 54), (96, 44), (75, 46), (45, 34), (62, 52), (56, 10), (21, 36), (36, 20), (41, 47), (78, 63), (92, 15)]$$

Тогава B непосредствено след сортирането си е

$$B = [(p_7, 7), (p_9, 12), (p_{10}, 9), (p_4, 1), (p_1, 5), (p_5, 8), (p_3, 3), (p_{12}, 4), (p_{11}, 10), (p_8, 6), (p_2, 2), (p_6, 11)]$$

като всъщност B е

$$B = [((56, 10), 7), ((92, 15), 12), ((36, 20), 9), ((69, 33), 1), ((45, 34), 5), ((21, 36), 8), ((96, 44), 3), ((75, 46), 4), ((41, 47), 10), ((62, 52), 6), ((29, 54), 2), ((78, 63), 11)]$$

Нека елементите на P всъщност са наредени двойки (p_i, k), като p_i е точка, а k е индексът на точката в сортирания масив A. Поначало тези индекси са недефинирани;

тоест, P поначало е

$$\mathsf{P} = [((69,33),?),((29,54),?),((96,44),?),((75,46),?),((45,34),?),((62,52),?),\\ ((56,10),?),((21,36),?),((36,20),?),((41,47),?),((78,63),?),((92,15),?)]$$

Нека елементите на A всъщност са наредени двойки (p_i, s), като p_i е точка, а s е индексът на точката p_i в P. След сортирането си A става:

$$\mathsf{A} = [(p_5,8),(p_2,2),(p_{10},9),(p_{11},10),(p_1,5),(p_7,7),\\ (p_8,6),(p_4,1),(p_{12},4),(p_6,11),(p_9,12),(p_3,3)]$$

което всъщност е

$$\mathsf{A} = [((21,36),8),((29,54),2),((36,20),9),((41,47),10),((45,34),5),((56,10),7),\\ ((62,52),6),((69,33),1),((75,46),4),((78,63),11),((92,15),12),((96,44),3)]$$

С едно сканиране на този A отляво надясно запълваме липсващите индекси в P:

$$\mathsf{P} = [((69,33),8),((29,54),2),((96,44),12),((75,46),9),((45,34),5),((62,52),7),\\ ((56,10),6),((21,36),1),((36,20),3),((41,47),4),((78,63),10),((92,15),11)]$$

Да си припомним, че в момента B е

$$\mathsf{B} = [((56,10),7),((92,15),12),((36,20),9),((69,33),1),((45,34),5),((21,36),8),\\ ((96,44),3),((75,46),4),((41,47),10),((62,52),6),((29,54),2),((78,63),11)]$$

С едно сканиране на B отляво надясно променяме индексите на елементите му по следния начин. Нека B[i] съдържа индекс j. Тогава новият индекс на B[i] е индексът, който се съдържа в P[j]. Например, B[1] има индекс 7. Тоест, j = 7. P[7] съдържа индекс 6. B[1] става $((56,10),6)$. Аналогично, B[2] става $((92,15),11)$. И така нататък. Наистина, B става

$$\mathsf{B} = [(p_7,6),(p_9,11),(p_{10},3),(p_4,8),(p_1,5),(p_5,1),\\ (p_3,12),(p_{12},9),(p_{11},4),(p_8,7),(p_2,2),(p_6,10)]$$

точно както искахме според (5.109). Очевидно е, че генерирането на тези индекси по описания начин увеличава само с $\Theta(n)$ сложността по време на предварителната обработка. Следователно, сложността по време на предварителната обработка остава $\Theta(n \lg n)$.

[a] Дори множеството P да не е реализирано чрез масив, всеки негов елемент-точка има адрес в паметта, който адрес е известен.

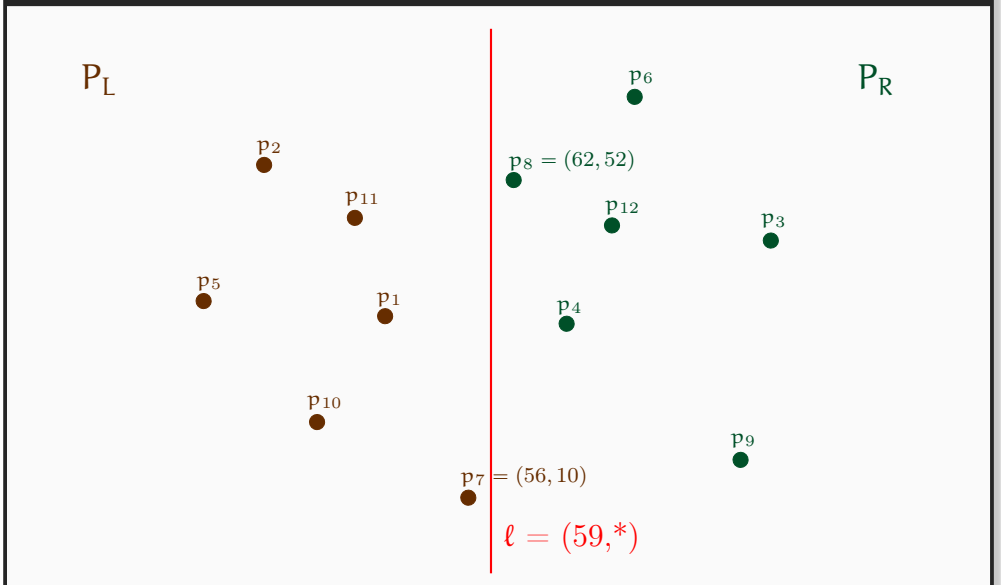
След извършването на предварителната обработка започва същинският рекурсивен алгоритъм. Базата е $n \in \{2,3\}$. Ако $n \in \{2,3\}$, то решаваме задачата с брутална сила, опитвайки всички $\binom{n}{2}$ ненаредени двойки от числа. Заслужава си да отбележим, че при $n = 1$ задачата е безсмислена (поради което в дефиницията на задачата има изискване $n \geqslant 2$), така че базата не трябва да е $n = 1$.

Да допуснем, че $n > 3$.

Фаза 1: разделяй. Разбиваме P на две подмножества с максимално близки мощности. Това действие ще наричаме *бисекция*. Разбиването става спрямо някаква права ℓ, като едното подмножество се състои от точките, които са в едната полуравнина спрямо ℓ, а другото подмножество, от точките в другата полуравнина. Правата ℓ се казва *бисектор*. Удачно е ℓ да се успоредна на някоя от координатните оси. Избираме да е успоредна на ординатата[†]. Подмножеството вляво от ℓ наричаме P_L, а това вдясно от ℓ наричаме P_R. Алгоритмично, бисекцията е елементарна. Ние вече имаме масива $A[1,\ldots,n]$ от точките, сортирани по първа координата. Нека подмасивът $A\left[1,\ldots,\left\lceil\frac{n}{2}\right\rceil\right]$ се казва A_L, а подмасивът $A\left[\left\lceil\frac{n}{2}\right\rceil+1,\ldots,n\right]$ да е A_R. Тогава P_L се състои точно от елементите на A_L, а P_R се състои точно от елементите на A_R. Бисекторът е "виртуална права" – мисловна конструкция за по-лесно обяснение на алгоритъма. В действителност бисекторът е число: щом сме избрали бисекторът да е вертикална права, то всички нейни точки имат една и съща първа координата и това число е бисекторът. Нека това число бъде средното аритметично от първите координати на средните точки (при сортиране по първа координата), а именно $\frac{1}{2}\left(A\left[\left\lceil\frac{n}{2}\right\rceil\right].x + A\left[\left\lceil\frac{n}{2}\right\rceil+1\right].x\right)$. Не е съществено важно бисекторът да е именно средното аритметично! Ако $A\left[\left\lceil\frac{n}{2}\right\rceil\right].x$ и $A\left[\left\lceil\frac{n}{2}\right\rceil+1\right].x$ са различни, всяко число от отворения интервал $\left(A\left[\left\lceil\frac{n}{2}\right\rceil\right].x, A\left[\left\lceil\frac{n}{2}\right\rceil+1\right].x\right)$ би "свършило работа" като бисектор. По отношение на разбиването на P няма значение кое число от този отворен интервал ще вземем. Но за фазата **комбинирай** точната стойност на бисектора има значение, както ще видим надолу.

Като пример вижте Фигура 5.8. $n = 12$, така че $\left\lceil\frac{n}{2}\right\rceil = 6$ и $\left\lceil\frac{n}{2}\right\rceil+1 = 7$, така че $A\left[\left\lceil\frac{n}{2}\right\rceil\right].x = 56$ и $A\left[\left\lceil\frac{n}{2}\right\rceil+1\right].x = 62$, така че $\frac{1}{2}\left(A\left[\left\lceil\frac{n}{2}\right\rceil\right].x + A\left[\left\lceil\frac{n}{2}\right\rceil+1\right].x\right) = \frac{56+62}{2} = 59$, което е и стойността на бисектора.

Фигура 5.8 : Бисекторът ℓ разбива P на P_L и P_R.



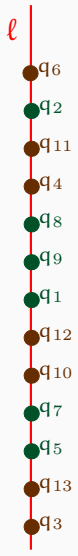
Виртуалният бисектор се състои от точките с първа координата 59. Действителният бисектор е само числото 59.

Едно уточнение. Дефиницията на задачата позволява различни точки да имат една и съща първа координата. Тъй като виртуалният бисектор е вертикална права, при съвпадения на първите координати може бисекторът да минава през много точки, стоящи една над друга или дори съвпадащи. Може дори това да са всички точки. Ако бисекторът минава през

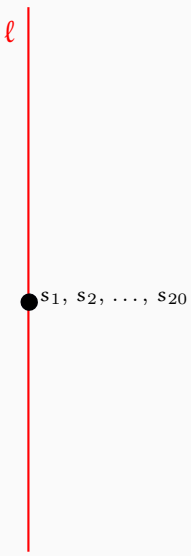
[†]Бисекторът да е вертикална права не е съществено за решението. Решение с хоризонтални бисектори има същата сложност и като описание, и като алгоритмична сложност.

повече от една точки, това не е истински бисектор от геометрична гледна точка, защото това коя от тези точки в кое от двете подмножества отива е произволно. Но това е само от геометрична гледна точка. От алгоритмична гледна точка, бисекторът е число и разбиването на сортирания масив A спрямо това число на A_L и A_R (с максимално близки мощности) е безпроблемно. Такива дегенеративни случаи са показани на Фигура 5.9.

Фигура 5.9 : Дегенеративни случаи. ℓ си остава бисектор.



Точките имат една и съща първа координата. Визуално, бисекцията $P_L = \{q_1, q_2, q_5, q_7, q_8, q_9\}$ и $P_R = \{q_3, q_4, q_6, q_{10}, q_{11}, q_{12}, q_{13}\}$ е произволна.



Всички точки съвпадат. Бисекцията е произволна и не е показана.

Както вече казахме, алгоритмичната бисекцията става много лесно:

$$A_L = A\left[1, \ldots, \left\lceil \frac{n}{2} \right\rceil\right]$$
$$A_R = A\left[\left\lceil \frac{n}{2} \right\rceil + 1, \ldots, n\right]$$

P_L се състои точно от елементите на A_L, а P_R се състои точно от елементите на A_R. По отношение на примера, показан на Фигура 5.7 и Фигура 5.8:

$$A_L = [p_5, p_2, p_{10}, p_{11}, p_1, p_7]$$
$$A_R = [p_8, p_4, p_{12}, p_6, p_9, p_3]$$

Налага се обаче да разбием и B на B_L и B_R, където B_L се състои от същите точки като A_L, но сортирани по втората координата, а B_R се състои от същите точки като A_R, но сортирани по втората координата. Да видим кои множества са B_L и B_R в примера на Фигура 5.7 и Фигура 5.8, като първо си припомним B:

$$B = [p_7, p_9, p_{10}, p_4, p_1, p_5, p_3, p_{12}, p_{11}, p_8, p_2, p_6]$$
$$B_L = [p_7, p_{10}, p_1, p_5, p_{11}, p_2]$$
$$B_R = [p_9, p_4, p_3, p_{12}, p_8, p_6]$$

Очевидно B_L и B_R не се получават като първата и втората половина на масива B (за разлика от A_L и A_R, които са именно първата и втората половина на масива A). Това си заслужава да бъде подчертано отново.

Наблюдение 26

В текущия контекст, в общия случай не е вярно, че $B_L = B\left[1, \ldots, \left\lceil \frac{n}{2} \right\rceil\right]$ и не е вярно, че $B_R = B\left[\left\lceil \frac{n}{2} \right\rceil + 1, \ldots, n\right]$. Тази разлика в конструирането на B_L и B_R спрямо A_L и A_R идва оттам, че избрахме вертикален, а не хоризонтален бисектор ℓ, с което в някакъв смисъл направихме хоризонталното направление предпочитано пред вертикалното. Естествено, можехме да изберем хоризонтален бисектор и тогава щяхме да предпочитаме вертикалното направление. Тогава B_L и B_R щяха да са съответно първата и втората половина на масива B, а A_L и A_R щяха да се конструират по-трудно.

Забележете, че не трябва да конструираме B_L като сортираме A_L по втора координата, нито да конструираме B_R като сортираме A_R по втора координата. Това би било ефективно, но неефикасно. Вече казахме, че ако искаме сложност по време $O(n \lg n)$ на алгоритъма, не трябва да сортираме на всяко ниво на рекурсията. Ще конструираме B_L и B_R от B във време $O(n)$. Да си припомним, че в предварителната обработка конструирахме масива B така, че всяка точка съдържа и индекса, който има тя самата в масива A. В нашия пример, B е

$$B = [(p_7, 6), (p_9, 11), (p_{10}, 3), (p_4, 8), (p_1, 5), (p_5, 1),$$
$$(p_3, 12), (p_{12}, 9), (p_{11}, 4), (p_8, 7), (p_2, 2), (p_6, 10)]$$

Слагаме в B_L точно тези елементи на B, чиито индекси в A са в множеството $\left\{1, \ldots, \left\lceil \frac{n}{2} \right\rceil\right\}$, а в B_R слагаме точно тези елементи на B, чиито индекси в A са в множеството $\left\{\left\lceil \frac{n}{2} \right\rceil + 1, \ldots, n\right\}$. В нашия пример, в B_L слагаме елементите на B, чиято втора компонента е в множеството $\{1, \ldots, 6\}$, а в B_R слагаме тези, чиято втора компонента е в множеството $\{7, \ldots, 12\}$. Забележете, че елементите на B_L са разположени взаимно по същия начин, по който са разположени в B, и елементите на B_R са разположени взаимно по същия начин, по който са разположени в B. Ерго, генерирането на B_L и B_R можем да постигнем с едно сканиране на B отляво надясно, като слагаме съответния $B[i]$ в B_L или B_R според индекса в A. В нашия пример, $B[1]$ отива в B_L, $B[2]$ отива в B_R, $B[3]$ отива в B_L, и така нататък, като наистина получаваме

$$B_L = [p_7, p_{10}, p_1, p_5, p_{11}, p_2]$$
$$B_R = [p_9, p_4, p_3, p_{12}, p_8, p_6]$$

Очевидно тази фаза се изпълнява в линейно време.

Фаза 2: владей. Рекурсивно изпълняваме алгоритъма върху P_L и P_R, като от тези две викания получаваме стойности d_L и d_R. d_L е минимално разстояние между точки в P_L, а d_R е минимално разстояние между точки в P_R.

Фаза 3: комбинирай. Нека t е отсечка с минимална дължина, чиито краища са две различни точки от P. Очевидно е, че точно едно от следните е истина:

- Краищата на t са в P_L. В този случай $|t| = d_L$, тъй като допускаме, че рекурсивното викане върху P_L работи коректно.
- Краищата на t са в P_R. В този случай $|t| = d_R$, тъй като допускаме, че рекурсивното викане върху P_R работи коректно.

- Единият край на t е в P_L, а другият е в P_R. В този случай казваме, че t *е прекосяваща отсечка*. Изпълнено е $|t| = d_M$, където

$$d_M = \min\{\text{dist}(x, y) \mid x \in P_L, y \in P_R\} \tag{5.110}$$

  В началото на фазата **комбинирай** ние не разполагаме с d_M и трябва тепърва да го изчислим.

От тези съображения става ясно, че задачата се свежда до намирането на d_M, след което алгоритъмът връща

$$\min\{d_L, d_R, d_M\}$$

Изразът "$\min\{\text{dist}(x, y) \mid x \in P_L, y \in P_R\}$" може да бъде превърнат директно в алгоритъм, който изчислява d_M. Този подход обаче е неефикасен. Той опитва всяка точка от P_L с всяка точка от P_R, тоест опитва всяка прекосяваща отсечка, което води до квадратична сложност по време. Естествено, ако фазата **комбинирай** работи в $\Theta(n^2)$, няма как алгоритъмът да работи в $O(n \lg n)$. Налага се да направим нещо по-умно, за да изчислим d_M. Правим две подобрения.

Първо подобрение. Нека $d = \min\{d_L, d_R\}$. Да дефинираме, че *къси прекосяващи отсечки* са тези с дължина, по-малка от d. Такива може и да няма. Ако няма къси прекосяващи отсечки, решението е d и нашият алгоритъм следва да върне d. Ако има къси прекосяващи отсечки, решението е дължината на най-къса прекосяваща отсечка и нашият алгоритъм следва да върне това число.

Наблюдение 27

Ако някакви прекосяващи отсечки не са къси, то няма смисъл да изчисляваме дължините им при изчисляването на d_M.

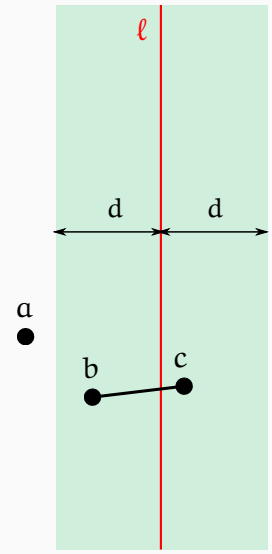
Да си представим точките от P заедно с правата-бисектор ℓ. Да дефинираме $2d$-*лентата* като областта от равнината, чиито точки са на разстояние не по-голямо от d от двете страни на ℓ. Иначе казано, ако поне единият край на някоя прекосяваща отсечка е извън $2d$-лентата, то нейната дължина е по-голяма от d и не ни интересува.

Наблюдение 28

Ако някоя точка лежи извън $2d$-лентата, то тя не е край на къса прекосяваща отсечка.

Това е илюстрирано на Фигура 5.10, на която $2d$-лентата е показана в светло синьо. Очевидно точка a не може да е край на къса прекосяваща отсечка, защото другият край на тази отсечка трябва да е от другата страна на ℓ. Обаче точка b се намира в $2d$-лентата и наистина тя е край на къса прекосяваща отсечка (другият ѝ край е c).

Фигура 5.10 : 2d-лентата.



И така, първото подобрение е следното. Съгласно Наблюдение 28 и Наблюдение 27, за изчисляването на d_M игнорираме всички точки извън 2d-лентата. Ако поне две точки от P попадат в 2d-лентата, да изчислим d_M само от точките на P, които са в 2d-лентата; а ако в 2d-лентата има нула или една точки от P, то d_M да бъде безкрайност, което е същото като да игнорираме d_M и алгоритъмът да върне d. Ако трябва да сме прецизни, разглеждайки само точките в 2d-лентата, ние не изчисляваме непременно d_M, а нещо друго, което ще наречем d'_M. Причината е, че d_M и d'_M не са непременно равни:

$$d_M = \min\{\mathrm{dist}(x,y) \mid x \in P_L, y \in P_R\} \tag{5.111}$$

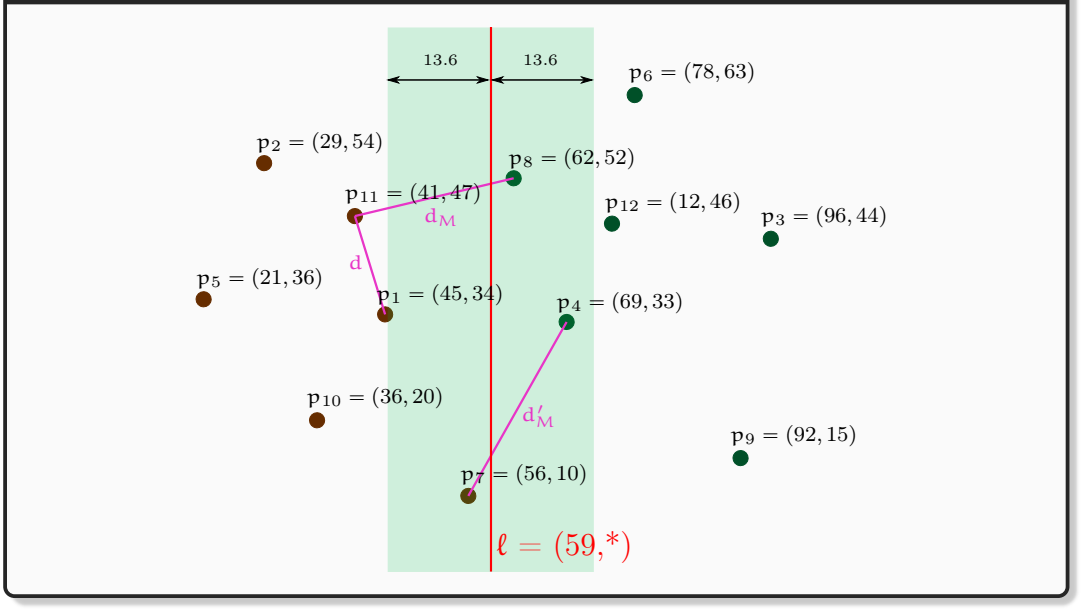
$$d'_M = \min\{\mathrm{dist}(x,y) \mid x \in P_L \text{ и } x \text{ е в 2d-лентата}, y \in P_R \text{ и } y \text{ е в 2d-лентата}\} \tag{5.112}$$

Забележете, че възможно $d'_M > d_M$, а в първото подобрение става дума именно за изчисляването на d'_M. Това обаче не е проблем, защото алгоритъмът връща $\min\{d, d'_M\}$, а ако $d'_M > d_M$, то задължително $d_M > d$[†]; ерго, ако $d'_M > d_M$, то решението е d и алгоритъмът ще върне именно d. С други думи, невъзможно е $d'_M > d > d_M$; ако това би било възможно, то би било възможно и да "изтървем" оптимално решение d_M.

Фигура 5.11 показва 2d-лентата в примера от Фигура 5.7. Минималното разстояние между точки от P_L е между $p_1 = (45, 34)$ и $p_{11} = (41, 47)$, а именно $d = \sqrt{(45-41)^2 + (34-47)^2} \approx 13.60147051$. Тогава $d_L \approx 13.6$. Минималното разстояние между точки от P_R е между p_4 и p_{12}, както и между p_8 и p_{12}, а именно $\sqrt{(75-69)^2 + (46-33)^2} = \sqrt{(62-75)^2 + (52-46)^2} \approx 14.31782106$. Тогава $d_R \approx 14.3$ и $d = d_L$. На Фигура 5.11 виждаме P_L и P_R с бисектора $\ell = (59, *)$ между тях и 2d-лентата, простираща се на около 13.6 отляво и отдясно на бисектора.

[†]Това е така, защото, ако $d'_M > d_M$, то d_M е дължина на отсечка, поне единият край на която е извън 2d-лентата и която отсечка, съгласно Наблюдение 28, не е къса, от което следва, че $d_M > d$.

Фигура 5.11 : 2d-лентата в примера от Фигура 5.7.



Само три точки от P попадат в 2d-лентата: p_7, p_4 и p_8 (p_1 е извън 2d-лентата). Тогава $d'_M = \min\{\text{dist}(x,y) \mid x \in \{p_7\}, y \in \{p_4, p_8\}\} = \text{dist}(p_4, p_7) = \sqrt{(56-69)^2 + (10-33)^2} \approx 26.41968963$. Виждаме, че в нашия пример е изпълнено $d_M < d'_M$, понеже d_M се реализира между p_{11} и p_8, като $\text{dist}(p_{11}, p_8) = \sqrt{(41-62)^2 + (47-52)^2} \approx 21.58703314$. Но, както вече отбелязахме, ако $d'_M > d_M$, какъвто е този случай, то $d_M > d$, така че за крайното решение няма значение дали сравняваме 13.6 с 26.4 или с 21.5 – решението си остава 13.6.

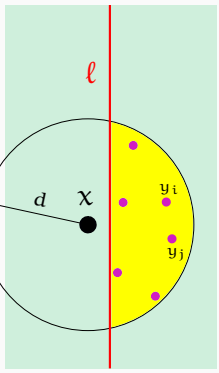
Но по отношение на най-лошия случай само първото подобрение изобщо не е подобрение. Ако всички точки от P са в 2d-лентата и търсим d_M изчерпателно, то сложността пак би била квадратична.

Второ подобрение. Забелязваме, че за всяка точка x в 2d-лентата има твърде ограничен брой други точки, такива че има смисъл да изчисляваме разстоянието между x и всяка от тях. Например, нека $x \in P_L$. Може ли да има сто точки в $y_1, y_2, \ldots, y_{100}$ в P_R, такива че във фазата **комбинирай** всяка от тях да е на разстояние, по-малко от d, от точка x? Отговорът е, че не може. За да е изпълнено това, би трябвало $y_1, \ldots, y_{100}$ да са в кръга с център x и радиус d; по-точно казано, в сечението на този кръг и дясната половина на 2d-лентата. Но тогава биха съществували различни y_i и y_j, такива че $\text{dist}(y_i, y_j) < d$, което е невъзможно, защото $d = \min\{d_L, d_R\}$ по дефиниция.

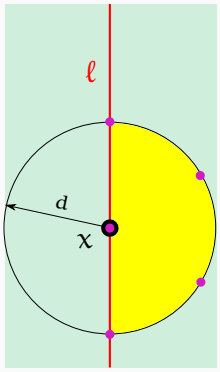
Това е илюстрирано на Фигура 5.12. Сечението на кръга с център x и радиус d с дясната половина на 2d-лентата е оцветено в жълто. Вляво разглеждаме възможността да има шест точки (в лилаво), всяка от които е "кандидат" за друг край на къса прекосяваща отсечка (единият край е x). Очевидно е, че както и да бъдат разположени шестте лилави точки в жълтия сегмент, няма как всеки две от тях да са на разстояние поне d една от друга. Например, на фигурата y_i и y_j са толкова близо една до друга, че разстоянието d_R не може да е повече от $\text{dist}(y_i, y_j)$, следователно, би трябвало $d \leqslant \text{dist}(y_i, y_j)$, в противоречие с показаното на фигурата.

Вдясно илюстрираме факта, че пет е точната горна граница за броя на въпросните кандидати. Едната от десните (лилавите) точки съвпада с x, което не е невъзможно. x лежи точно върху бисектора, така че сечението на кръга с диаметър d и център x с дясната страна на 2d-лентата е полукръг. Ако d_R е минималното разстояние между точки измежду тези пет (това е възможно), и $d = d_R$ (и това е възможно), и петте лилави точки попадат в жълтия полукръг.

Фигура 5.12 : Възможните други краища на къса прек. отсечка са малко.



Очевидно $\mathrm{dist}(y_i, y_j) < d$. Очевидно няма как да разположим шест точки в жълтия сегмент, така че всеки две от тях да са на разстояние поне d една от друга.

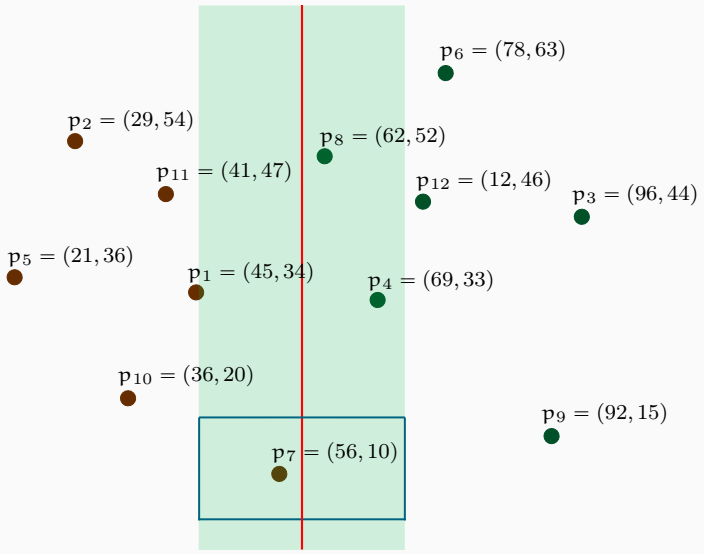


Ако жълтият сегмент е полукръг, може да разположим пет точки в него, така че всеки две да са на поне d една от друга. Едната от тези точки съвпада с x. Повече от пет точки не може.

След като се убедихме, че съществува неголяма константа c, такава че за всяка точка от $2d$-лентата трябва да разглеждаме най-много c други точки в изчисляването на d'_M, да видим как ще реализираме тази идея ефикасно. Досегашните разсъждения навеждат на мисълта да направим следното: за всяка точка x от лявата страна на $2d$-лентата да намерим всички точки от дясната страна на $2d$-лентата, които се намират в кръга с център x и радиус d, ако има такива, и да видим коя от тях е най-близо до x. Но това би довело до усложнения, които е по-добре да си спестим. Ще разгледаме подхода на [21, стр. 1040–1043], който е значително по-прост и е достатъчно ефикасен. А досегашните разсъждения за ограничения брой на точките "от другата страна", които има смисъл да разглеждаме, са само за придобиване на по-добра интуиция.

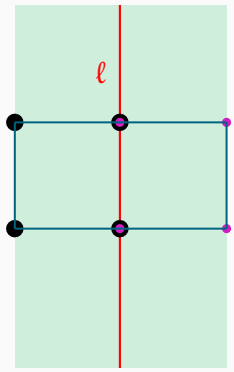
Представяме си *прозорец* – правоъгълник с размери $2d \times d$, със страни, успоредни на координатните оси, като голямата страна е успоредна на абсцисата, а центърът на прозореца лежи върху бисектора. Идеята е да плъзнем прозореца от най-долу (най-малката втора координата) до най-горе (най-голямата втора координата), като центърът му се плъзга по бисектора, а страните му остават успоредни на координатните оси. Фигура 5.13 показва прозореца, нарисуван върху примера от Фигура 5.7.

Фигура 5.13 : Прозорецът върху примера от Фигура 5.7.



На Фигура 5.13 прозорецът обхваща само една точка, а именно p_7. Колко точки от P—това означава и от P_L, и от P_R—най-много може да обхваща прозореца? Този брой е ограничен, тъй като размерите на прозореца не са произволни. Лесно се забелязва, че най-много осем точки от P може да са в прозореца. Това е демонстрирано на Фигура 5.14. Четирите черни точки са от P_L, а четирите лилави са от P_R. Черните точки са върху върховете на квадрата-лява половина на прозореца, а лилавите са върху върховете на квадрата-дясна половина на прозореца. Две от черните точкти съвпадат с две от лилавите.

Фигура 5.14 : Прозорецът може да обхваща най-много осем точки.



Ето и второто подобрение: да плъзнем прозореца отдолу нагоре в 2d-лентата, да изчислим минималното разстояние между две (различни) точки, които се намират в него в кой да е момент, едната от P_L, другата от P_R, и полученото да присвоим на d'_M.

Разбира се, прозорецът е метафора. Ние не правим компютърна геометрия и не плъзгаме правоъгълници, центрирани върху вертикални прави. В действителност правим следното и това е фазата **комбинирай**.

- Първо създаваме масив C от всички точки, намиращи се в 2d-лентата, сортирани по втора координата. За тази цел използваме масивите A и B. Да си припомним, че истинският бисектор е число, да го наречем β, а вертикалната права е виртуален бисектор.

Ерго, създаваме C като подмасивът на A, състоящ се от точно тези точки p_i, за които $|p_i.x - \beta| \leqslant d$.

Ето как изглежда това в нашия пример. Да си припомним, че

$$A = [(21,36),(29,54),(36,20),(41,47),(45,34),(56,10),\\ (62,52),(69,33),(75,46),(78,63),(92,15),(96,44)]$$

$\beta = 59$ и $d \approx 13.6$. $59 - 13.6 = 45.4$ и $59 + 13.6 = 72.6$, така че точно три точки влизат в C, а именно $(56,10)$, $(62,52)$ и $(69,33)$; нещо, което видяхме още на Фигура 5.11.

И така, използвахме A, който е сортиран по първа координата, за да отделим точките, попадащи в $2d$-лентата.

- От масива B отделяме сортираната по втора координата последователност от тези същите точки, слагайки ги в някакъв масив D в същия ред, в който са в B (тоест, сортирани по втора координата). Това можем да направим ефикасно, защото в предварителната обработка сме добавили към елементите на B техните местоположения в A, така че сега можем във време $\Theta(1)$ да определим за всеки елемент от B дали той е в C или не; с други думи, в константно време можем да определим за произволен елемент от B дали тази точка попада в $2d$-лентата или не. Ерго, с едно сканиране на B можем да копираме в D тези елементи, които попадат в $2d$-лентата, като ги копираме в същия ред, в който са в B.

- В нашия пример D е много малък:

  $$D = [(56,10),(69,33),(62,52)]$$

  но да допуснем, че D е достатъчно голям. Да кажем, че D има k елемента за някакво $k \geqslant 8$. Нека $m \leftarrow \infty$. Инициализираме $i \leftarrow 1$, $j \leftarrow 8$ и изпълняваме следното $k - j + 1$ пъти:

  - С брутална сила намираме двойка различни елементи r, s от $D[i,\ldots,j]$, които са на минимално разстояние един от друг, като единият от r, s е от P_L, а другият е от P_R. Ако има такива, $m = \min\{m, \mathrm{dist}(r,s)\}$. Това става в константно време, защото броят на ненаредените двойки от $D[i,\ldots,j]$ е $\binom{8}{2} = 28$.
  - $i{+}{+}$, $j{+}{+}$.

  След края на това, полученото m е точно търсеното d'_M.

  > *Забележете, че масивът $D[i,\ldots,j]$ не реализира точно прозореца $2d \times d$, за който стана дума във второто подобрение. Ако D е достатъчно голям, нашата алгоритмична реализация работи с $D[i,\ldots,j]$, такъв че $j - i = 7$, тоест, винаги в нашия прозорец има осем точки. Както се вижда на Фигура 5.13, ако прозорецът е наистина $2d \times d$, той може да обхваща само една точка или дори нула точки (лесно се вижда, че има такива негови положения). С други думи, осемте точки, които разглеждаме на всяка итерация, може да бъдат прекалено далече една от друга, за да се намират в рамките на $2d \times d$ прозореца.*
  >
  > *Това, че $D[i,\ldots,j]$ обхваща винаги осем точки при достатъчно голямо D, не води до проблеми за нашия алгоритъм. Важното е, че* ***гледайки не повече от осем последователни по вертикал точки в даден момент, не***

можем да изпуснем оптимална стойност на d'_M. *Иначе казано, няма смисъл да търсим* d'_M, *разглеждайки двойка точки, които не са измежду осем последователни точки в* D. *Нашата имплементация може и да опитва да търси* d'_M *върху точки, които са прекалено далече една от друга, но няма да изпусне оптимално решение и е с линейна сложност.*

И така, алгоритъмът връща

$$\min\{d_L, d_R, d'_M\}$$

Коректност. Коректността на алгоритъма е очевидна предвид разсъжденията, които направихме.

Сложност. Очевидно е, че фазата **Комбинирай**, реализирана по гореописания начин, работи във време $\Theta(n)$. Фазата **Разделяй** също работи в линейно време. Тогава сложността на алгоритъма след предварителната обработка се описва от рекурентното уравнение

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

което има решение $T(n) \asymp n \lg n$. Предварителната обработка има сложност $\Theta(n \lg n)$, доминирана от сортиранията; останалата работа в предварителната обработка е $\Theta(n)$. Като цяло, сложността на алгоритъма е $\Theta(n \lg n)$.

5.4.2 Избор на елемент по големина

Този нетривиален алгоритъм е открит от Blum, Floyd, Pratt, Rivest и Tarjan през 70-те [16], [17]. Изложението тук е по учебника на Cormen, Leiserson, Rivest и Stein [21, стр. 220–222].

Изч. Задача: Избор на елемент по големина

пример: крайно непразно множество $S \subset \mathbb{Z}$; $k \in \mathbb{N}$, такова че $0 \leqslant k \leqslant |S| - 1$
решение: Числото $x \in S$, такова че $|\{a \in S \mid a < x\}| = k$.

Една забележка: очевидно в тази изчислителна задача става дума не за k-ия, а за $(k+1)$-вия по големина елемент, като k е броят на по-малките от него елементи. Така че формулировката на задачата чрез k, а не чрез $k+1$, може да е леко подвеждаща, но в [21] задачата е дефинирана точно така.

Тази задача може да се реши тривиално във време $\Theta(n \lg n)$ чрез бързо сортиране на S, например чрез Heapsort, последвано от връщане на $(k+1)$-вия елемент. Тук ще разгледаме по-бързо решение: линеен алгоритъм, наречен PICK в [17] и Select в [21].

Алгоритъмът ползва функция Partition–Lomuto–mod, която, както е практически същата като Partition–Lomuto от алгоритъма Quicksort (вижте Подсекция 6.2.2), с две незначителни разлики:

- първо, елементът *pivot*, спрямо който се разместват елементите на масива, не е непременно най-десният елемент поначало, а е произволен елемент, който се среща; в началото той се открива и се разменя с най-десния, след което се изпълнява Partition–Lomuto;

- второ, върната стойност е $pp - 1$, а не pp, защото за целите на SELECT ни трябва не позицията на $pivot$, а броят на елементите вляво от него, в пренаредения масив.

Ефектът от работата на PARTITION–LOMUTO–MOD е следният: елементите на $A[l, \ldots, h]$ биват разместени по такъв начин, че:

- подмасивът $A[l, \ldots, pp - 1]$ се състои точно от елементите на $A[l, \ldots, h]$, по-малки от $pivot$;
- $A[pp]$ съдържа $pivot$;
- подмасивът $A[pp + 1, \ldots, h]$ се състои точно от елементите на $A[l, \ldots, h]$, по-големи от $pivot$;

Тук допускаме, че елементите на масива са два по два различни; това допускане не е в сила в Лекция 6, но в контекста на алгоритъма за избор на елемент по големина допускаме, че няма елементи с еднакви ключове.

Подчертаваме, че алгоритъмът PARTITION–LOMUTO **не сортира**: в края му, нито "малките елементи " в $A[l, \ldots, pp - 1]$ са непременно сортирани, нито "големите елементи" в $A[pp + 1, \ldots, h]$ са непременно сортирани. Следва псевдокод на PARTITION–LOMUTO–MOD. Функцията find-index връща индекса на елемента-първи аргумент в масива-втори аргумент, при допускането, че този елемент се среща точно веднъж в този масив.

```
PARTITION–LOMUTO–MOD(A[l, ..., h]: int; pivot: element from A[l, ..., h])
 1  k ← find-index(pivot, A[l, ..., h])
 2  swap(A[k], A[h])
 3  pp ← l
 4  for i ← l to h − 1
 5      if A[i] < pivot
 6          swap(A[i], A[pp])
 7          pp ← pp + 1
 8  swap(A[pp], A[h])
 9  return pp − 1
```

Следва псевдокод на SELECT. Да си припомним дефиницията на ИЗБОР НА ЕЛЕМЕНТ ПО ГОЛЕМИНА на предната страница. Множеството S е реализирано чрез масив $A[1, \ldots, n]$, чиито елементи са два по два различни. Вторият аргумент k е броят на елементите, по-малки от този, който връща алгоритъмът. Описанието на алгоритъма, взето от [21], е на по-високо ниво от обичайния псевдокод.

```
SELECT(A[1, ..., n]: int; k ∈ {0, ..., n − 1})
 1  (* A[x] ≠ A[y] за 1 ⩽ x < y ⩽ n *)
 2  if n = 1
 3      return A[1]
 4  q ← ⌈n/5⌉, r ← n mod 5
 5  if r = 0
 6      разбий A на подмасиви B_1, ..., B_q, всеки с 5 елемента
 7  else
 8      разбий A на подмасиви B_1, ..., B_{q−1}, всеки с 5 елемента, и B_q с r елемента
 9  сортирай всеки от B_1, ..., B_q и намери съответната медиана m_1, ..., m_q
```

```
10  направи масив [m_1, ..., m_q] от медианите и го наречи C
11  m ← SELECT(C, ⌊(q−1)/2⌋)
12  j ← PARTITION(A, m)
13  if k = j
14      return m
15  else
16      if k < j
17          return SELECT(A[1, ..., j], k)
18      else
19          return SELECT(A[j + 2, ..., n], k − j − 1)
```

Да разгледаме малък пример. Нека $n = 25$ и елементите на A са 1, 2, ..., 25, като

$$A = [17, 6, 8, 22, 11, 1, 7, 4, 19, 21, 3, 20, 16, 24, 10, 25, 12, 18, 15, 14, 13, 2, 23, 9, 5]$$

Нека $k = 17$. Това е входът.

Очевидно $q = 5$. Нека A бъде разбит (ред 6) на 5 подмасива по най-естествения начин:

$$B_1 = [17, 6, 8, 22, 11]$$
$$B_2 = [1, 7, 4, 19, 21]$$
$$B_3 = [3, 20, 16, 24, 10]$$
$$B_4 = [25, 12, 18, 15, 14]$$
$$B_5 = [13, 2, 23, 9, 5]$$

След сортирането на всеки от тях (ред 9), те стават:

$$B_1 = [6, 8, 11, 17, 22]$$
$$B_2 = [1, 4, 7, 19, 21]$$
$$B_3 = [3, 10, 16, 20, 24]$$
$$B_4 = [12, 14, 15, 18, 25]$$
$$B_5 = [2, 5, 9, 13, 23]$$

Тогава $m_1 = 11$, $m_2 = 7$, $m_3 = 16$, $m_4 = 15$ и $m_5 = 9$, така че $C = [11, 7, 16, 15, 9]$ (ред 10). На ред 11, $\left\lfloor \frac{5-1}{2} \right\rfloor = 2$, така че викаме SELECT с втори аргумент 2. Това викане връща 3-ия по големина елемент на C, който е 11, така че на ред 11, m получава стойност 11.

На ред 12 викаме PARTITON върху A с втори аргумент 11. На ред 12 процедурата връща $j = 10$ (което е напълно очаквано, ако елементите на A са 1, ..., 25 и *pivot* е 11). Самият A изглежда така след разместванията, направени от PARTITION:

$$A = [6, 8, 5, 1, 7, 4, 3, 10, 2, 9, 11, 20, 16, 24, 22, 25, 12, 18, 15, 14, 13, 19, 23, 21, 17$$

Тъй като $k = 17$ и $j = 10$, условието на ред 13 е лъжа, условието на ред 16 също е лъжа и изпълнението отива на ред 19. $A[j + 2, \ldots, n]$ е $A[12, \ldots, 25]$. В случая:

$$A[12, \ldots, 25] = [20, 16, 24, 22, 25, 12, 18, 15, 14, 13, 19, 23, 21, 17] \tag{5.113}$$

$k-j-1$ е 6, така че викането на ред 19 е SELECT$(A[12, \ldots, 25], 6)$. Очевидно в този пример това е коректно, понеже осемнадесетия $(17 + 1 = 18)$ по големина елемент на входния A е същият

като седмия ($6 + 1 = 7$) по големина елемент на "редуцирания" A от (5.113). Забележете, че в конкретния пример седмият по големина елемент в масива в (5.113) се намира на седма позиция (става дума за числото 18), но това е случайно; ако винаги беше така, нямаше да има нужда от рекурсивното викане на ред 19.

С това приключва разглеждането на примера.

Коректност Сега ще докажем коректността на SELECT. Твърдим, че ако е даден масив $A[1, \ldots, n]$ от две по две различни цели числа и $k \in \{0, \ldots, n-1\}$, то SELECT(A, k) връща $(k+1)$-вия по големина елемент на A. Ще покажем коректността на алгоритъма със силна индукция по n.

Спирачката на рекурсията е $n = 1$ (ред 2). Ако $n = 1$, единствената допустима стойност за k е 0, понеже $k \in \{0, \ldots, n-1\}$. Наистина, на ред 3 алгоритъмът връща $A[1]$, който наистина е $(k+1)$-вия по големина елемент при $k = 0$. ✓

Да допуснем, че $n \geqslant 2$. Редове 4–11 нямат значение на коректността, а само за ефикасността, така че тук няма да ги разглеждаме. За доказателството за коректност има значение единствено, че m на ред 11 е елемент от масива (очевидно) и че PARTITION(A, m) на ред 12 размества A по такъв начин и връща такава стойност j, че:

- всички елементи, по-малки от m, са в подмасива $A[1, \ldots, j]$;
- $A[j+1]$ съдържа m;
- всички елементи, по-големи от m, са в подмасива $A[j+2, \ldots, n]$.

Разглеждаме три случая.

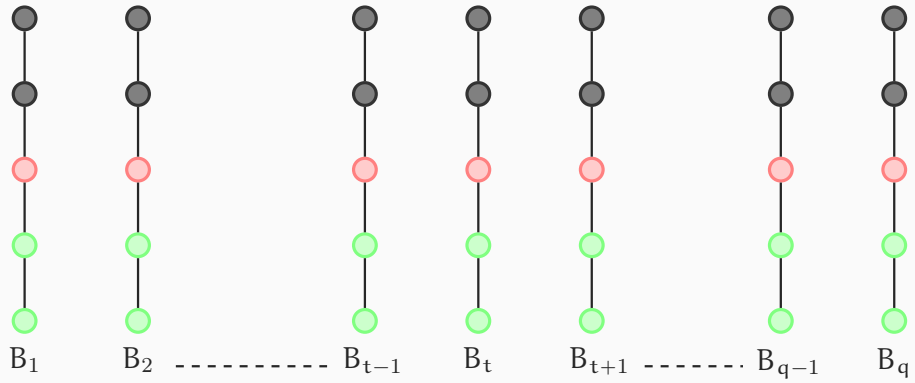
Случай 1. Ако $k = j$ (ред 13), то m е $(k+1)$-вият по големина елемент и алгоритъмът коректно връща m на ред 14.

Случай 2. Ако $k < j$ (ред 16), то $(k+1)$-вият по големина елемент на входния A е същият като $(k+1)$-вият по големина елемент на $A[1, \ldots, j]$. За да се убедим в това, да съобразим, че всички елементи, по-малки от m, са в подмасива $A[1, \ldots, j]$. Викането на процедурата SELECT$(A[1, \ldots, j], k)$ на ред 17 връща $(k+1)$-вия по големина елемент на $A[1, \ldots, j]$ съгласно индукционното предположение.

Случай 3. Ако $k > j$ (ред 19), то $(k+1)$-вият по големина елемент на входния A е същият като $(k-j)$-ият по големина елемент на $A[j+2, \ldots, n]$. За да се убедим в това, да съобразим, че преди $(k+1)$-вия в A има точно k елемента, което означава, че преди него в $A[j+2, \ldots, n]$ има $k - (j+1) = k - j - 1$ елемента, защото сега не броим $A[1]$, $A[2]$, $\ldots$, $A[j]$ и $A[j+1]$. Следователно, викането на SELECT$(A[j+2, \ldots, n], k-j-1)$ на ред 19 връща желания елемент.

Сложност по време На редове 5–8 алгоритъмът разбива A на подмасиви: или всички с големина 5, или всички без един с големина 5, а изключението е с големина, по-малка от 5. Тези подмасиви биват сортирани на ред 9, след което е тривиално за всеки от тях да се намери медианата. За удобство да допуснем, че n е кратно на 5 и всички подмасиви са с по точно 5 елемента. Тогава всяка медиана измежду m_1, $\ldots$, m_q е гарантирано по-голяма от точно два елемента и по-малка от точно два елемента от съответния подмасив измежду B_1, $\ldots$, B_q. Фигура 5.15 показва диаграма на Hasse на множеството след ред 9 на SELECT. Всеки от подмасивите B_1, $\ldots$, B_q е линейно наредено, петелементно множество, а за $i \neq j$, всеки елемент от B_i е несравним с всеки елемент от B_j. Във всеки B_i, двата елемента, по-малки от медианата, са изобразени в зелено, а двата елемента, по-големи от медианата, в тъмно сиво.

Фигура 5.15 : Диаграма на Hasse на данните след ред 9 на SELECT.



Малките елементи са в зелено, големите са в тъмно сиво, а медианите са в червено.

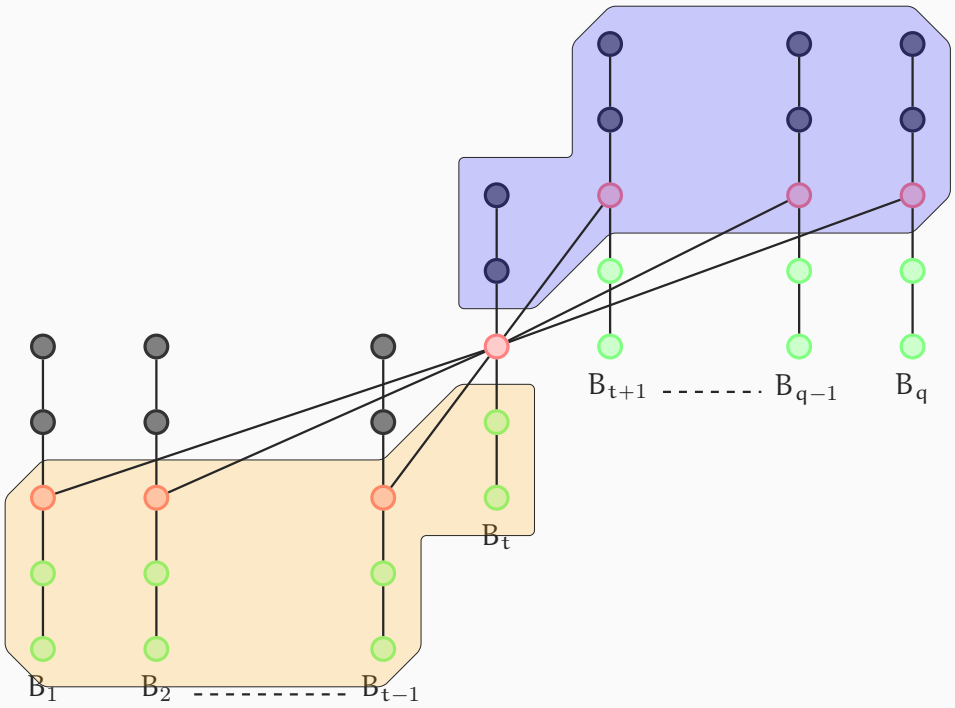
На ред 11 викаме SELECT върху масива от медианите $[m_1, \ldots, m_q]$ с втори аргумент $\lfloor \frac{q-1}{2} \rfloor$. Съгласно индуктивното предположение (да си припомним, че правим доказателството със силна индукция), на ред 11, SELECT връща този елемент на $[m_1, \ldots, m_q]$, който по големина е номер $\lfloor \frac{q-1}{2} \rfloor + 1$. Но $\lfloor \frac{q-1}{2} \rfloor + 1 = \lfloor \frac{q-1+2}{2} \rfloor = \lfloor \frac{q+1}{2} \rfloor$, а по определение елементът, който по големина е номер $\lfloor \frac{q+1}{2} \rfloor$, е медианата. Следователно, на ред 11, m получава стойността на медианата на медианите [†].

Да кажем, че медианата на медианите е медианата на B_t (Фигура 5.15), и нека без ограничение на общността медианите на B_1, ..., B_{t-1} са по-малки от нея, а медианите на B_{t+1}, ..., B_q са по-големи от нея[‡]. След като бъде установено всичко това, диаграмата на Hasse на данните престава да бъде като на Фигура 5.15 и става като на Фигура 5.16.

[†]Забележете, че медианата на медианите не е непременно медианата на целия масив A. В примера, който видяхме, медианата на медианите беше 11, а медианата на A беше 13.

[‡]От това следва, че $t = \lfloor \frac{q+1}{2} \rfloor$.

Фигура 5.16 : Диаграма на Hasse на данните след ред 11 на SELECT.



Медианата на медианите m е медианата на B_t. Елементите, гарантирано по-малки от m, са показани върху оранжев полупрозрачен фон. Елементите, гарантирано по-големи от m, са показани върху син полупрозрачен фон. Ако q е нечетно, тоест, ако m е средният по големина елемент на масива от медианите, оранжевите и сините елементи са еднакъв брой. В противен случай, сините са повече, понеже вземаме за медиана долния кандидат.

Ще намерим долна граница за броя на елементите, които са по-малки от m. На Фигура 5.16 това са елементите, показани върху оранжев полупрозрачен фон. Това са по точно три елемента от всеки от масивите B_1, ..., B_{t-1}, плюс още два елемента от B_t. Може да има и други елементи, по-малки от m—някои от зелените елементи на B_{t+1}, ..., B_q—но само за оранжевите е гарантирано, че са по-малки от m. Броят на оранжевите елементи е точно

$$3\left\lfloor\frac{q-1}{2}\right\rfloor+2=3\left\lfloor\frac{\frac{n}{5}-1}{2}\right\rfloor+2\approx\frac{3n}{10}$$

Апроксимираме сложния израз с $\frac{3n}{10}$, игнорирайки адитивната константа $+2$, нотацията $\lfloor\rfloor$ и -1 в числителя. Лесно се вижда, че това не променя асимптотиката на решението, което ще получим.

Напълно аналогично, апроксимираме броя на елементите, гарантирано по-големи от m, с $\frac{3n}{10}$.

Това количество $\frac{3n}{10}$ е от решаваща важност за намиране на сложността по време. $\frac{3n}{10}$ е минималният брой елементи, които **гарантирано няма** да бъдат част от множеството, върху което се извършва рекурсивно викане на ред 17 или ред 19. С други думи, на ред 17 или ред 19 рекурсивното викане ще бъде върху вход с размер **най-много** $\frac{7n}{10}$.

Вече можем да построим израз за сложността по време. Имайки предвид, че рекурсивното викане на ред 12 е върху вход с големина приблизително $\frac{n}{5}$, а извън рекурсивните викания,

SELECT извършва линейна по време работа, изразът е:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + n \tag{5.114}$$

Ще решим (5.114) чрез теоремата на Akra-Bazzi (Теорема 36). Имаме $a_1 = a_2 = 1$, $b_1 = \frac{1}{5}$, $b_2 = \frac{7}{10}$ и $g(n) = n$. Числото p е уникалното число, такова че

$$a_1 \cdot b_1^p + a_2 \cdot b_2^p = 1 \leftrightarrow \left(\frac{1}{5}\right)^p + \left(\frac{7}{10}\right)^p = 1$$

С Maple(TM) можем да получим апроксимация на p:

```
> evalf(solve((1/5)^p + (7/10)^p = 1, p));
memory used=3.2MB, alloc=40.3MB, time=0.11
                                0.8397803045
```

И така, $p \approx 0.83978$. Конкретната стойност няма значение – важното е, че $0 < p < 1$, така че $1 - p > 0$. Съгласно Теорема 36,

$$\begin{aligned}
T(n) &\asymp n^p \left(1 + \int_1^n \frac{g(u)}{u^{p+1}}\,du\right) = n^p \left(1 + \int_1^n \frac{du}{u^p}\right) \\
&= n^p \left(1 + \frac{u^{1-p}}{(1-p)}\Big|_1^n\right) = n^p \left(1 + \frac{n^{1-p}}{(1-p)} - \underbrace{\frac{1^{1-p}}{(1-p)}}_{\text{константа}}\right) \\
&\asymp n^p \left(1 + \frac{n^{1-p}}{(1-p)}\right) \asymp n^p + n^1 \asymp n^1
\end{aligned}$$

И така, SELECT е линеен алгоритъм.

Лекция 6

Сортиращи алгоритми Mergesort и Quicksort.

Резюме: Въвеждаме два бързи сортиращи алгоритъма, Mergesort и Quicksort, и изследваме тяхната коректност, сложност по време и сложност по памет. Изследваме средната сложност по време на Quicksort.

6.1 Mergesort

Сортиращият алгоритъм Mergesort е изобретен от John von Neumann през 1945 г. [56]. Той е типичен пример за алгоритъм, изграден по схемата Разделяй-и-Владей. Във фазата **Разделяй**, Mergesort дели входа на две равни части, всяка с размер $\frac{n}{2}$[†], без да променя наредбата на елементите. Във фазата **Владей** прави по едно рекурсивно викане върху всяка от двете части. Във фазата **Комбинирай** всяка от двете части вече е сортирана, така че алгоритъмът слива (откъдето идва и името, *сливам* в този смисъл е *to merge* на английски) двете сортирани части в една окончателна сортирана последователност. Естествено, всичко това става, когато входът е достатъчно голям. Ако входът е с размер единица или нула, алгоритъмът не прави нищо (защото празният масив и едноелементният масив са сортирани) – това е спирачката на рекурсията. Акцентът е върху третата фаза, където се извършва истинската работа по сортирането.

Първо ще дадем пример за сортиране с Mergesort. Примерът нарочно използва вход с големина, която е точна степен на двойката, но кодът, който даваме нататък, работи коректно за всяка големина. И така, нека входът е:

5 2 3 1 4 8 7 6

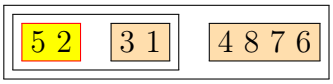
С жълт фон означаваме тази част от масива, която е вход на текущо активното рекурсивно извикване. В самото начало тази част съвпада с целия масив.

Тъй като големината на входа е повече от единица, делим входа на две равни части и правим по едно рекурсивно извикване върху всяка от тях. Тези две рекурсивни извиквания не се случват едновременно, защото нашият изчислителен модел не позволява паралелизъм. Да речем, че първо извикваме рекурсивно алгоритъма върху масива вляво.

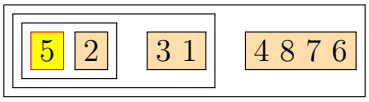
5 2 3 1 4 8 7 6

[†]В действителност разделянето е на една част с размер $\lfloor \frac{n}{2} \rfloor$ и друга част с размер $\lceil \frac{n}{2} \rceil$.

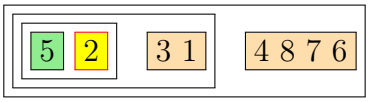
Отново, с жълт фон означаваме само тази част от началния масив, която е вход на текущото рекурсивно викане. Понеже големината на този вход е четири, което е по-голямо от едно, пак делим входа на две и правим две рекурсивни викания, първото върху частта вляво:



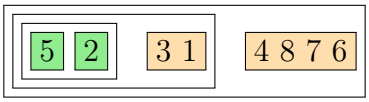
Тъй като големината на входа на текущото рекурсивно викане е две, което е по-голямо от едно, пак делим входа на две правим две рекурсивни викания, първото върху частта вляво:



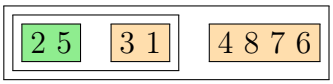
Сега вече големината на входа на текущото рекурсивно викане е едно, и алгоритъмът не прави нищо в това извикване и връща директно управлението на предното извикване, което на свой ред вика алгоритъма върху другия подмасив с големина едно:



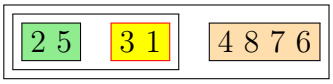
Светлозеленият фон означава, че върху този подмасив вече е достигнато дъното на рекурсията. Отново големината на текущия вход (жълт фон) е едно и алгоритъмът директно връща управлението на горното ниво, бидейки привършил и с двете викания

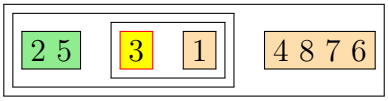


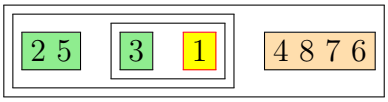
Сега алгоритъмът слива двата сортирани подмасива с големини единици в един сортиран подмасив с големина две:

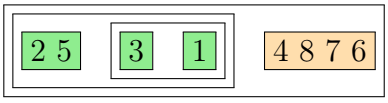


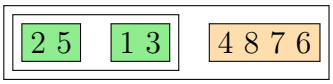
Изчислението продължава така:

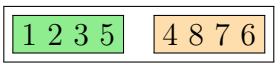




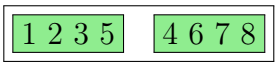




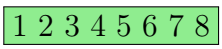




Прескачаме няколко стъпки, които съответстват на второто рекурсивно викане върху 4 8 7 6, и разглеждаме масива след приключване на това второ рекурсивно викане:



Сливането на двата сортирани подмасива води до



Сега ще дадем псевдокод са функцията, която осъществява сливането. Нейната коректност е условна: тя работи коректно **при условие**, че двете половини на входния ѝ масив са сортирани. Символът ∞ означава число, по-голямо от всяко друго число, което може да се срещне. Този символ се нарича *пазач* (на английски, *sentinel*, по терминологията на [21]). Използването му не е задължително, но с него кодът става по-лесен за четене и верификация (Определение 6).

```
MERGE(A[1,...,n]: int; l, mid, h: цели числа, такива че 1 ⩽ l < mid < h ⩽ n)
 1  (* подмасивите A[l,...,mid] и A[mid+1,...,h] са сортирани *)
 2  n1 ← mid − l + 1
 3  n2 ← h − mid
 4  създай L[1,...,n1 + 1] и R[1,...,n2 + 1]
 5  L[1,...,n1] ← A[l,...,mid]
 6  R[1,...,n2] ← A[mid + 1,...,h]
 7  L[n1 + 1] ← ∞
 8  R[n2 + 1] ← ∞
 9  i ← 1
10  j ← 1
11  for k ← l to h
12      if L[i] ⩽ R[j]
13          A[k] ← L[i]
14          i++
15      else
16          A[k] ← R[j]
17          j++
```

Лема 40

Да допуснем, че подмасивите $A[l,\ldots,mid]$ и $A[mid+1,\ldots,h]$ във входа на MERGE са сортирани. След терминирането на MERGE, масивът $A[l,\ldots,h]$ се състои точно от елементите на входните $A[l,\ldots,mid]$ и $A[mid+1,\ldots,h]$, но в сортиран вид.

Доказателство: Приемаме за очевидно, че при първото достигане на ред 11, масивите L и R са точни копия на съответно $A[l,\ldots,mid]$ и $A[mid+1,\ldots,h]$, плюс един sentinel накрая.

Инвариант 15: MERGE

част i Всеки път, когато изпълнението на MERGE е на ред 11, $A[l, \ldots, k-1]$ съдържа $k - l$ най-малки елемента на L и R, в сортиран вид.
част ii Освен това, L[i] и R[j] са най-малките елементи съответно в L и R, които още не са копирани в A.

База. Когато изпълнението е на ред 11 за пръв път е вярно, че $k = l$. Тогава подмасивът $A[l, \ldots, k-1]$ всъщност е $A[l, \ldots, l-1]$, тоест е празен. Празният масив е сортиран и съдържа $k - l = 0$ най-малки елемента на L и R, в сортиран вид. Освен това, L[1] и R[1] са най-малките елементи съответно в L и R, които все още не са копирани в A.

Поддръжка. Нека твърдението е в сила при някое достигане на ред 11, което не е последното. Има два алтернативни начина, по които изпълнението може да премине през тялото на цикъла. Ще ги разгледаме и двата. Преди да направим това обаче, ще докажем едно важно помощно твърдение, Лема 41. Забележете, че и L, и R съдържат стойност ∞, така че трябва да сме сигурни, че двете стойности ∞ не биват сравнявани, понеже сравнение на две ∞ стойности не е дефинирано.

Лема 41

В сравнението на ред 12, не може едновременно L[i] и R[j] да са ∞.

Доказателство: Да допуснем противното. Съгласно индуктивното предположение, $k - l$ най-малки елементи на L и R вече са копирани в A. Щом изпълнението е на ред 12, то $k \leqslant h$. Ерго, броят на копираните елементи е $\leqslant h - l$.

Очевидно е, че щом на ред 12 сравняваме ∞ с ∞, всички останали елементи на L и R вече са копирани. В L и R има точно $n_1 + n_2$ елемента, които не са ∞. Следователно, броят на копираните елементи е поне $n_1 + n_2 = mid - l + 1 + h - mid = h - l + 1 > h - l$. ↯

Да разгледаме сравнението на ред 12. Тъй като няма как и L[i], и R[j] да са ∞, резултатът от сравнието е дефиниран. Първо да допуснем, че $L[i] \leqslant R[j]$. Очевидно, $L[i] < \infty$. Съгласно **част ii** от индуктивното предположение и допускането, че $L[i] \leqslant R[j]$, заключаваме, че L[i] е най-малкият елемент както в L, така и в R, който все още не е копиран. Съгласно **част i** на индуктивното предположение, L[i] не е по-малък от никой елемент на $A[l, \ldots, k\text{-}1]$. Изпълнението отива на ред 13. Забелязваме, че $A[k]$ не е по-малък от нито един елемент на $A[l, \ldots, k\text{-}1]$ и заключаваме, че $A[l, \ldots, k]$ е сортиран и съдържа $k - l + 1 = (k + 1) - l$ на брой най-малки елемента на L и R. Но k бива инкрементирано при следващото достигане на ред 11. Спрямо новата стойност на k е вярно, че $A[l, \ldots, k\text{-}1]$ съдържа $k - l$ най-малки елемента на L and R, в сортиран вид. И така, **част i** на инварианта остава в сила.

Сега ще докажем и **част ii** на инварианта. По допускане, L и R са сортирани. Преди присвояването на ред 13, L[i] беше най-малък елемент от L, който все още не е копиран в A. След това присвояване, L[i + 1] е най-малък елемент от L, който все още не е копиран в A. Но променливата i бива инкрементирана при следващото достигане на ред 14. По отношение на новата стойност на i, L[i] е най-малък елемент от L който все още не е копиран в A.

Сега да допуснем, че изпълнението все още е на ред 12 и $L[i] \nleqslant R[j]$, тоест $L[i] > R[j]$. Доказателството е напълно аналогично на току-що направеното.

Терминация. Променливата k съдържа $h + 1$ при последното достигане на ред 11. Заместваме тази стойност в инварианта и получаваме "подмасивът $A[l, \ldots, h]$ съдържа $h - l + 1$ най-малки елемента на L и R, в сортиран вид". Имайки предвид факта, че L и R съдържат

точно елементите на входния $A[l, \ldots, h]$, заключаваме, че текущият $A[l, \ldots, h]$ съдържа точно елементите на входния $A[l, \ldots, h]$, но в сортиран вид. □

MERGESORT($A[1, \ldots, n]$: int; l, h: индекси в A)
```
1  if l < h
2      mid ← ⌊(l+h)/2⌋
3      MERGESORT(A, l, mid)
4      MERGESORT(A, mid + 1, h)
5      MERGE(A, l, mid, h)
```

Лема 42

Алгоритъм MERGESORT е коректен сортиращ алгоритъм, ако началното извикване е MERGESORT$(A, 1, n)$.

Доказателство: По индукция по разликата $h - l$[†]. Смятаме за очевидно, че $h - l$ може да стане най-малко нула, но не и по-малко, и че базата на нашето доказателство е $h - l = 0$.

База. Нека $h - l = 0$, тоест $h = l$. Масивът $A[l, \ldots, h]$ е едноелементен. От една страна, едноелементният масив $A[l]$ е тривиално сортиран. От друга страна, MERGESORT не прави нищо, когато $h = l$. Така че масивът е сортиран в края на алгоритъма.

Поддръжка. Допускаме, че MERGESORT сортира коректно подмасивите $A[l, \ldots, mid]$ и $A[mid + 1, \ldots, h]$ (редове 3 и 4) при всички рекурсивни викания, такива че $h > l$. От Лема 6.1 следва, че в края на работата на текущото рекурсивно извикване, целият $A[l, \ldots, h]$ е сортиран.

Терминация. Когато доказваме коректност на рекурсивни алгоритми по индукция, стъпката **Терминация** се отнася до приключването на изпълнението на началното викане. В този случай, началното извикване е MERGESORT$(A[1, \ldots, n])$. С MERGESORT, тази стъпка е тривиална: просто забелязваме, че алгоритъмът приключи работата си, $A[1, \ldots, n]$ е сортиран. □

Сложността по време на MERGESORT се определя с рекурентното уравнение:

$$T(n) = 2T\left(\frac{n}{2}\right) + n \tag{6.1}$$

Решението, съгласно втория случай на Мастър теоремата, е $T(n) \asymp n \lg n$. Сложността по памет е $\Theta(n)$, ако алгоритъмът се имплементира грамотно (имплементация, която буквално следва псевдокода горе би имала линейна сложност по време, но на практика би била твърде бавна заради честото алокиране и деалокиране на памет). MERGESORT е стабилен сортиращ алгоритъм, защото на ред 12 във функцията MERGE има нестрого неравенство. Поради това, ако биват сравнявани два елемента (от L и от R) с равни ключове, елементът от L ще окаже вляво от този от R в окончателната подредба.

[†]Забележете, че това не е доказателство с инвариант на цикъла, понеже MERGESORT не е итеративен, а е рекурсивен, алгоритъм.

Допълнение 32: Бързо намиране на броя на инверсиите в масив.

Определение 45: инверсия в масив

Нека $A[1,\ldots,n]$ е масив от цели числа. *Инверсия* в A е всяка двойка индекси $\langle i,j\rangle$, такива че $1 \leqslant i < j \leqslant n$ и $A[i] > A[j]$.

Като прост пример да разгледаме $A = [5,1,3,2,4]$. Инверсиите са $\langle 1,2\rangle$, $\langle 1,3\rangle$, $\langle 1,4\rangle$, $\langle 1,5\rangle$ и $\langle 3,4\rangle$.
Очевидно максимален брой инверсии се достига тогава и само тогава, когато елементите на масива са два по два различни и масивът е сортиран обратно. В такъв случай има точно $\frac{n(n-1)}{2}$ инверсии. Минимален брой инверсии се достига при сортиран масив. В такъв случай инверсиите са точно 0.
Забележете приликата между Определение 45 и Определение 35. Посоката на неравенството в Определение 35 е обратната: там се иска $A[i] < A[j]$, но това е, защото Определение 35 е в контекста на максимални пирамиди (вж. Определение 33). Ако ставаше дума за минимални пирамиди, неравенството в Определение 35 щеше да е $A[i] > A[j]$, също както в Определение 45. Изключвайки несъществената подробност за посоката на неравенството, забелязваме, че в някакъв смисъл Определение 35 е частен случай на Определение 45, тъй като в Определение 35 се иска $A[i]$ да е предшественик на $A[j]$, което е частен случай на изискването $i < j$ в Определение 45. И така, Наблюдение 29 е аналог на Наблюдение 13.

Наблюдение 29

Масивът от цели числа A е сортиран тогава и само тогава, когато няма нито една инверсия. □

Разглеждаме задачата, за даден масив $A[1,\ldots,n]$ да се изчисли във време $O(n \lg n)$ броят на инверсиите. Наивния алгоритъм не се признава, понеже е квадратичен.

```
COUNT-INVERSIONS-NAIVE(A[1,...,n]: int)
1  c ← 0
2  for i ← 1 to n − 1
3      for j ← i + 1 to n
4          if A[i] > A[j]
5              c++
6  return c
```

Ще решим задачата с алгоритъм, изграден по схемата Разделяй-и-Владей. Ако разбием A на два подмасива L и R, то броят на инверсиите е сумата от

- броят на инверсиите в L,
- броят на инверсиите в R и
- броят на инверсиите $\langle i,j\rangle$, такива че i е индекс в L и j е индекс в R.

Заради ефикасността на решението искаме L и R да имат колкото се може по-близки големини. Алгоритъмът INVERSIONCOUNT, който предлагаме, е малка модификация на MERGESORT. За разлика от MERGESORT той връща стойност.

INVERSIONCOUNT($A[1,\ldots,n]$: int; l, h: индекси в A)
```
1  if l ⩾ h
2      return 0
3  else
4      mid ← ⌊(l+h)/2⌋
5      x ← INVERSIONCOUNT(A, l, mid)
6      y ← INVERSIONCOUNT(A, mid + 1, h)
7      z ← MERGE-MODIFIED(A, l, mid, h)
8      return x + y + z
```

Имайки предвид горните съображения за броя на инверсиите, коректността на алгоритъма е очевидна, ако MERGE-MODIFIED връща броя на инверсиите $\langle i, j\rangle$, такива че $i \in \{l,\ldots,mid\}$ и $j \in \{mid+1,\ldots,h\}$. MERGE-MODIFIED е малка модификация на MERGE на стр. 290.

MERGE-MODIFIED($A[1,\ldots,n]$: int; l, mid, h: $1 \leqslant l < mid < h \leqslant n$)
```
1   (* подмасивите A[l,...,mid] и A[mid+1,...,h] са сортирани *)
2   n1 ← mid − l + 1
3   n2 ← h − mid
4   създай L[1,...,n1 + 1] и R[1,...,n2 + 1]
5   L[1,...,n1] ← A[l,...,mid]
6   R[1,...,n2] ← A[mid + 1,...,h]
7   L[n1 + 1] ← ∞
8   R[n2 + 1] ← ∞
9   i ← 1
10  j ← 1
11  c ← 0
12  for k ← l to h
13      if L[i] ⩽ R[j]
14          A[k] ← L[i]
15          i++
16      else
17          A[k] ← R[j]
18          c ← c + n1 − i + 1
19          j++
20  return c
```

Обосновката на коректността на MERGE-MODIFIED е обосновката на MERGE с добавена аргументация, свързана с брояча c. Преди да направим по-формална обосновка ще дадем интуитивно обяснение. Винаги при проверката на ред 13:

- Ако $L[i] \leqslant R[j]$, то елементите $L[i]$ и $R[j]$ не задават инверсия (очевидно). Очевидно инверсиите между елементи от $L[1,\ldots,i-1]$ и елементи от $R[1,\ldots,j]$ са толкова, колкото са инверсиите между елементи от $L[1,\ldots,i]$ и елементи от $R[1,\ldots,j]$].

Това, че инкрементираме $\mathtt{i}$ на ред 15, без да променяме $\mathtt{c}$, е правилно.

- Ако $\mathtt{L}[\mathtt{i}] > \mathtt{R}[\mathtt{j}]$, то елементните $\mathtt{L}[\mathtt{i}]$ и $\mathtt{R}[\mathtt{j}]$ задават инверсия (очевидно). Освен това, всеки елемент от $\mathtt{L}[\mathtt{i}+1, \ldots, n_1]$ задава инверсия с $\mathtt{R}[\mathtt{j}]$, понеже $\mathtt{L}$ е сортиран, а релацията $\leqslant$ е транзитивна. Тогава всеки елемент от $\mathtt{L}[\mathtt{i}, \ldots, n_1]$ задава инверсия с $\mathtt{R}[\mathtt{j}]$, а това са $n_1 - \mathtt{i} + 1$ елемента, което означава и $n_1 - \mathtt{i} + 1$ инверсии. Ерго, инверсиите между елементи от $\mathtt{L}[1, \ldots, \mathtt{i}]$ и $\mathtt{R}[1, \ldots, \mathtt{j}]$ са с $n_1 - \mathtt{i} + 1$ повече от инверсиите между елементи от $\mathtt{L}[1, \ldots, \mathtt{i}-1]$ и $\mathtt{R}[1, \ldots, \mathtt{j}]$. Това, че инкрементираме $\mathtt{j}$ на ред 19 и добавяме $n_1 - \mathtt{i} + 1$ към $\mathtt{c}$ на ред 18, е правилно.

6.2 QUICKSORT

QUICKSORT е открит от Sir Charles Antony Richard Hoare [38] в началото на 1960-те (вж. [36], [37], [38]). В много случаи, той е най-бързият сортиращ алгоритъм за истински изчисления—а не в асимптотичния смисъл—и оттам идва и името му. Реалните имплементации са по-сложни от псевдокода, който разглеждаме тук.

QUICKSORT е типичен за схемата Разделяй-и-Владей алгоритъм, макар да е много различен от MERGESORT. Най-общо казано, идеята е да бъде избрана някаква стойност, която се нарича *pivot*, и спрямо нея масивът да бъде пренареден така, че в лявата част (може и да е празна, това зависи от pivot) да са само елементи, по-малки или равни на pivot, а вдясно от тях, само елементи, по-големи от pivot. Каква точно е наредбата в лявата и дясната част няма значение – не се иска при това пренареждане да се постигне сортиран масив, това би било прекалено силно изискване. Сортиране ще се получи чак след края на рекурсивните викания. На този етап искаме просто вляво да са "малките" елементи, а вдясно от тях, "големите". Това пренареждане е фазата **Разделяй** на алгоритъма. След това алгоритъмът бива викан рекурсивно върху всяка от тези части, ако тя е достатъчно голяма. Това е фазата **Владей**. Фазата **Комбинирай** е празна: след приключването на извикванията, масивът е сортиран. Виждаме една основна разлика между QUICKSORT и MERGESORT. При MERGESORT фазата **Разделяй** е тривиална и същината на алгоритъма е във фазата **Комбинирай**. При QUICKSORT фазата **Разделяй** е същината на алгоритъма, докато фазата **Комбинирай** е празна.

Изборът на pivot, както ще видим, е от огромно значение за сложността по време. Не е задължително pivot да е елемент от масива. Простите имплементации избират за pivot елемент от масива, и то по прост начин, например най-левия или най-десния елемент.

Допълнение 33: За избора на pivot в QUICKSORT.

Говорейки не съвсем строго, в идеалния случай стойността на pivot е такава, че половината елементи са по-малки от него, а другата половина, по-големи от него. С други думи, ако pivot е елемент от масива, то идеалният pivot е медианата.
Нещо повече. Както ще видим след малко, много неудачен избор на pivot, по-точно серия от много неудачни избори на pivot, може да доведе до това, че сложността по време на QUICKSORT да дегенерира до $\Theta(n^2)$, което го прави драстично по-бавен от HEAPSORT и MERGESORT в най-лошия случай. Както също ще видим след малко, такава крайно неудачна серия от избори на pivot е малко вероятна и средната сложност по време на QUICKSORT е $\Theta(n \lg n)$.

Възниква въпросът – не може ли да изчисляваме медианата и да избираме нея за pivot, с което сложността по време на QUICKSORT да стане $\Theta(n \lg n)$ в най-лошия случай? Вече видяхме в Подсекция 5.4.2, че медианата може да се намери в линейно време. Говорейки **чисто теоретично** и мислейки за сложността по време **само в асимптотичния смисъл**, изчисляването на pivot в линейно време няма да увеличи сложността нито в най-лошия, нито в средния случай. Защо тогава не изчисляваме идеалния pivot – медианата?
Отговорът е, че QUICKSORT е алгоритъм от голям практически интерес. Той "бие" останалите сортиращи алгоритми, дори бързите HEAPSORT и MERGESORT. Поради това ние се интересуваме не само от асимптотичните оценки на бързодействието му, а и от реалното бързодействие на неговите софтуерни имплементации.
На практика, забавянето, до което би довело намирането на медианата, би направило QUICKSORT напълно безсмислен. HEAPSORT и MERGESORT са бързи сортиращи алгоритми. Фолклорът казва, че QUICKSORT не е повече от, горе-долу, два пъти по-бърз от кой да е от тях (вижте *тази статия на D.Abhyankar, M.Ingle* или *този проект на Rashmi Raj*). Ако променим QUICKSORT по такъв начин, че да първо да намираме медианата и после pivot да е медианата, той ще стане значително по-бавен от HEAPSORT или MERGESORT и ще престане да бъде от да е практически интерес, въпреки че сложността му в най-лошия случай, в асимптотичния смисъл, ще се подобри от $\Theta(n^2)$ на $\Theta(n \lg n)$.

6.2.1 Имплементация на фазата Разделяй чрез PARTITION–HOARE

Фазата **Разделяй** на QUICKSORT се реализира чрез функция, която е широко известна под името PARTITION. Ние вече видяхме една имплементация на тази функция на стр. 282, но за пълнота на изложението тук допускаме, че виждаме функцията за първи път сега.

Очевидно PARTITION може да се имплементира в линейно време, като това е и долна граница (защото всеки елемент от масива трябва да бъде "прегледан" при това пренареждане). Ключовото наблюдение е, че освен това, тя може да бъде реализирана само с **константна допълнителна памет**, тоест in-place. Известни са няколко имплементации на оригиналната PARTITION, която е открита от Hoare. Ето как изглежда изглежда функцията PARTITION в [22, стр. 154]. Там тя е описана с **repeat**–**until** цикъл, който се отличава от **do**–**while** цикъл само по това, че **repeat**–**until** "върти", докато постусловието е лъжа, докато **do**–**while** "върти", докато постусловието е истина; оттук постусловието "$\mathtt{A[j]} \leqslant \mathtt{pivot}$" от [22, стр. 154] е транслирано в "$\mathtt{A[j]} > \mathtt{pivot}$" тук.

PARTITION–HOARE($A[1, 2, \ldots, n]$: цели числа; l, h: индекси в A, такива че $l < h$)

```
 1  pivot ← A[l]
 2  i ← l − 1
 3  j ← h + 1
 4  while TRUE do
 5      do
 6          j ← j − 1
 7      while A[j] > pivot
 8      do
 9          i ← i + 1
10      while A[i] < pivot
```

```
11      if i < j
12          swap(A[i], A[j])
13      else
14          return j
```

Грубо казано, i и j са индекси, които "вървят" един срещу друг, съответно отляво надясно и отдясно наляво, докато не "открият" елементи от масива X и Y, които са неправилно разположени спрямо избрания pivot в смисъл, че $X \geqslant$ pivot и $Y \leqslant$ pivot, но X е вляво от Y. Тези елементи се разменят и i и j продължават да "вървят" един срещу друг, докато не се разминат, след което желаното пренареждане е постигнато.

Съгласно предложения псевдокод, ходовете на j и надолу и на i нагоре **спират**, ако достигнат елемент, равен на `pivot`. От общи съображения човек може да помисли, че ходовете надолу или нагоре трябва да **продължават** през елементи, равни на `pivot`. На пръв поглед това ще спести излишни размени на еднакви елементи. Тази "оптимизация" обаче не е добра идея, както е показано в *тази онлайн лекция в Университета Princeton*, защото върху масив от еднакви елементи тя води автоматично до квадратичен алгоритъм, а даденият псевдокод работи почти винаги във време $\Theta(n \lg n)$.

По-прецизно казано, по време на работата на алгоритъма, масивът е разбит на три зони (някои, но не всички, от които може да са празни):

- **Зелена зона** $A[l, \ldots, i]$. Там са елементи, по-малки или равни на `pivot`.
- **Сива зона** $A[i+1, \ldots, j-1]$. Това е неизследваната част от масива.
- **Жълта зона** $A[j, \ldots, h]$. Там са елементи, по-големи или равни на `pivot`.

Изборът на имена за зоните е произволен. Освен това, оцветяваме в червено двете клетки на масива, върху които "се спират" индексите j и i, вървейки съответно отгоре надолу и отдолу нагоре. Тези червени клетки са в някакъв смисъл "нарушители": клетката, върху която спира j, съдържа елемент, който е по-малък от pivot, а клетката, върху която спира i, съдържа елемент, който е по-голям от pivot. След извършването на swap-а на ред 12, червената клетка, която била е съседна на жълтата зона, преминава в зелената зона и вече я оцветяваме в зелено, и обратното, червената клетка, която е била съседна на зелената зона, преминава в жълтата зона и вече я оцветяваме в жълто.

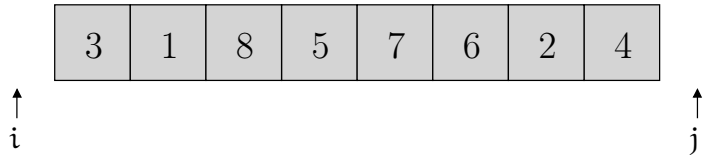
Когато и двата **do ... while** цикъла приключат, и $A[i] \geqslant$ `pivot`, и $A[j] \leqslant$ `pivot`. В случай, че $i < j$, изпълнението отива на ред 12; след swap-а, както казахме, зелената и жълтата зона нарастват с по една клетка, и отново индексите j и i тръгват, съответно надолу и нагоре. Ако $i \not< j$, няма какво повече да се прави.

Ще демонстрираме работата на PARTITION–HOARE с пример. Нека

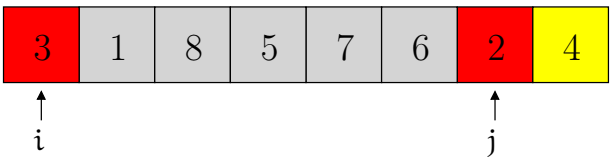
$A =$

3	1	8	5	7	6	2	4

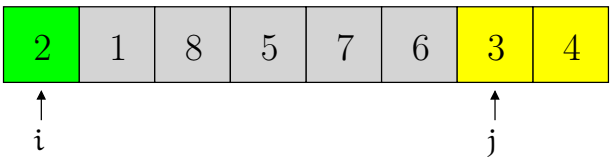
като масивът е индексиран от 1, с други думи, l е 1 и h е 8. В началото `pivot` е $A[l] = 3$. После i и j "застават" извън границите на масива, така че зелената и жълтата зона са празни:



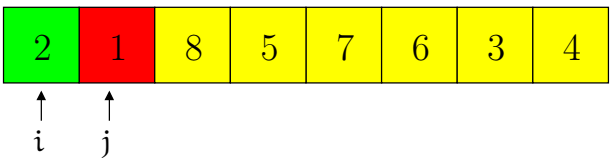
После j "слиза" надолу, първо става 8, но $A[8] \nleqslant$ `pivot`, така че j става 7 и изпълнението на първия **do ... while** цикъл (редове 5-6) спира, понеже $A[7] \leqslant$ `pivot`. Започва изпълнението на втория **do ... while** цикъл (редове 8-9). Индексът i се "качва" нагоре и става 1, но $A[1] = 3$, така че $A[1] \geqslant$ `pivot`, така че вторият **do ... while** цикъл спира. Ситуацията е следната:



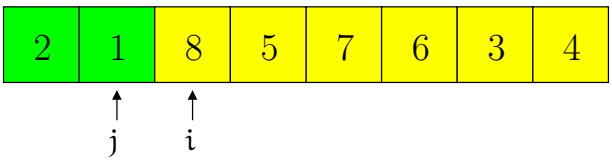
Изпълнението достига ред 11. Понеже i < j, изпълняваме размяната, която в случая е размяна на $A[1]$ с $A[7]$ получаваме:



После j "слиза" надолу и става 2, спирайки върху тази стойност:



После i се "качва" нагоре и става 3, спирайки върху тази стойност:



Вече не оцветяваме в червено клетките $A[j]$ и $A[i]$, защото това са елементи, чиято принадлежност към съответно жълтата и зелената зона вече сме установили.

Изпълнението отново е на ред 11. Сега вече $i \nless j$, така че изпълнението преминава към ред 14, като алгоритъмът връща стойност 2. И наистина, $A[1, \ldots, 2]$ се състои от елементи, които са не по-големи от `pivot`.

Формално доказателство за коректността на PARTITION–HOARE няма да даваме, но следният инвариант може да се докаже по познатия ни начин по индукция и оттам следва коректността. Няма нужда да се посочва, че елементите на A във всеки момент са точно същите като елементите на входния масив, защото единственото място в алгоритъма, на което се променя масива, е ред 12, а там променянето става чрез размяна на елементи.

Инвариант 16: PARTITION–HOARE

При всяко достигане на ред 4 на PARTITION–HOARE:

$$\forall x \in A[l, \ldots, i] : x \leqslant \texttt{pivot}$$
$$\forall x \in A[j, \ldots, h] : x \geqslant \texttt{pivot}$$

Освен това, ако $i \leqslant j$:

$$\exists x \in A[l, \ldots, j-1] : x \leqslant \texttt{pivot}$$
$$\exists x \in A[i+1, \ldots, h] : x \geqslant \texttt{pivot}$$

Известни са няколко вариации на PARTITION–HOARE, една от които е следната. Тя е описана в [94, стр. 16]. Тази функция ползва sentinel, който е $-\infty$ и се намира в $A[0]$. Числата, които ще се сортират, са в $A[1, \ldots, n]$. В оригинала вътрешните цикли са **repeat ... until**. За удобство при четене, тук те са сменени с **do ... while** цикли.

PARTITION–HOARE–ANOTHER($A[0, 1, 2, \ldots, n]$: масив; l, h: индекси в A, такива че $l < h$)
```
 1  (* Данните за сортиране са A[1, ..., n]. A[0] е −∞. *)
 2  pivot ← A[h]
 3  i ← l − 1
 4  j ← h
 5  while TRUE do
 6      do
 7          j ← j − 1
 8      while A[j] > pivot
 9      do
10          i ← i + 1
11      while A[i] < pivot
12      if i < j
13          swap(A[i], A[j])
14      else
15          break
16  swap(A[i], A[h])
17  return i
```

Разлика между PARTITION–HOARE и PARTITION–HOARE–ANOTHER е, че втората връща позицията на `pivot` и сме сигурни, че този елемент си е на мястото и може да правим следващите викания върху подмасиви, в които той не присъства.

6.2.2 Имплементация на фазата Разделяй чрез PARTITION–LOMUTO

Сега само ще покажем алтернативен псевдокод на функция, която пренарежда масива по желания начин. Идеята за този код е на Nico Lomuto, около 1984 г., и е публикувана в статията *Programming Pearls: Little Languages* в списанието Communications of the ACM [10].

PARTITION–LOMUTO($A[1, 2, \ldots, n]$: цели числа; l, h: индекси в A, такива че $l < h$)
```
1  pivot ← A[h]
2  pp ← l
```

```
3   for i ← l to h − 1
4       if A[i] < pivot
5           swap(A[i], A[pp])
6           pp ← pp + 1
7   swap(A[pp], A[h])
8   return pp
```

Името на променливата pp идва от "pivot position".

И при PARTITION–LOMUTO можем да дефинираме зелен район (малките елементи), жълт район (големите елементи) и сив район, само че взаимното им разположение е различно, сега те са зелен, жълт и сив, отляво надясно. PARTITION–LOMUTO "търкаля" жълтия район надясно, разменяйки най-левия му елемент с текущия A[i], но само в случай, че текущият A[i] е по-голям или равен на pp.

Ето пример за работата на PARTITION–LOMUTO. Нека, както и в предишния пример, A е следният масив:

A =

3	1	8	5	7	6	2	4

В началото `pivot` е 4. Индексите `i` и pp "застават" под първия елемент.

3	1	8	5	7	6	2	4

i pp

Условието на ред 4 е истина, елементът 3 се разменя със себе си, и после `i` и pp се оказват под втория елемент.

3	1	8	5	7	6	2	4

i pp

Елементът 1 се разменя със себе си и после `i` и pp се оказват под третия елемент.

3	1	8	5	7	6	2	4

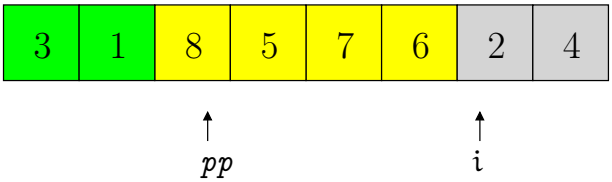
i pp

Сега вече A[i] $\not<$ `pivot` и тялото на цикъла не се изпълнява. Индексът `i` вече изпреварва pp и между тях се оформя районът от елементи, по-големи или равни на `pivot`.

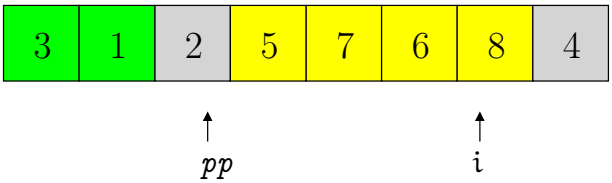
3	1	8	5	7	6	2	4

pp i

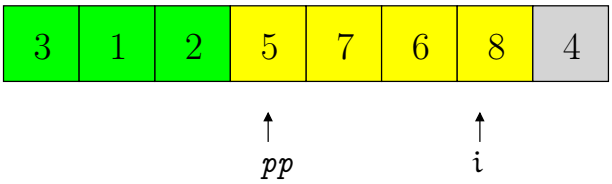
После се срещат няколко елемента, по-големи от `pivot`, които алгоритъмът "прескача", и се стига до тази ситуация:



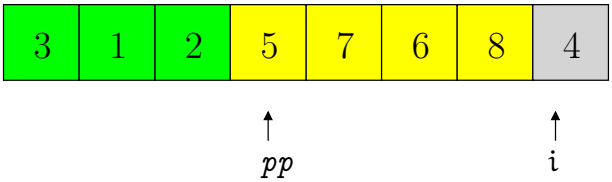
Сега условието на ред 4 е истина. Следва размяна на елементите 8 и 2:



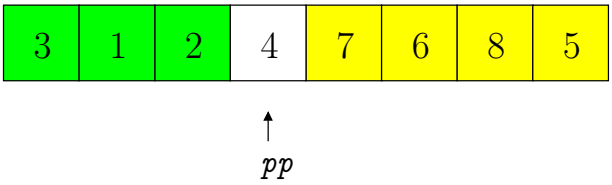
После *pp* пак се инкрементира:



Индексът `i` се инкрементира за последен път, жълтият район нараства, след което цикълът не се изпълнява повече:



После се прави размяната на ред 7:



На ред 8 алгоритъмът връща индексът на третия елемент, който е `pivot`-ът (четворката). Този елемент си е на мястото. Вляво от него са само по-малки елементи. Вдясно, само по-големи или равни.

Ще докажем формално коректността на Partition–Lomuto.

Лема 43

Ефектът от работата на PARTITION–LOMUTO е следният. В края на алгоритъма, спрямо някаква стойност pp, входният масив е пренареден така, че:

$$\forall x \in A[l, \ldots, pp-1] : x < A[pp]$$
$$\forall x \in A[pp+1, \ldots, h] : x \geqslant A[pp]$$

Освен това, алгоритъмът връща именно pp.

Доказателство:

Следното твърдение е инвариант на **for**-цикъла (редове 3–6).

Инвариант 17: PARTITION–LOMUTO

При всяко достигане на ред 3 на PARTITION–LOMUTO:

$$\forall x \in A[l, \ldots, pp-1] : x < pivot$$
$$\forall x \in A[pp, \ldots, i-1] : x \geqslant pivot$$

База. Когато изпълнението е на ред 3 за първи път, $i = l$ и $pp = l$. Инвариантът е:

$$\forall x \in A[l, \ldots, l-1] : x < pivot$$
$$\forall x \in A[l, \ldots, l-1] : x \geqslant pivot$$

И двете твърдения са верни, в празния смисъл. ✓

Поддръжка. Да допуснем, че инвариантът е верен при някое достигане на ред 3, което не е последното.

Случай I $A[i] < pivot$. Ползваме индуктивното предположение и заключаваме, че:

$$\forall x \in A[l, \ldots, pp-1] : x < pivot \wedge A[i] < pivot \tag{6.2}$$

Условието на ред 4 е истина и изпълнението отива на ред 5. Там се извършва размяна на $A[i]$ и $A[pp]$. Имайки предвид тази размяна, (6.2) става:

$$\forall x \in A[l, \ldots, pp] : x < pivot \tag{6.3}$$

След инкрементирането на pp на 6, (6.3) става:

$$\forall x \in A[l, \ldots, pp\text{-}1] : x < pivot \tag{6.4}$$

Виждаме, че първата част на инварианта се запазва. За да се убедим, че и втората част е вярна, да "върнем лентата" назад до момента, в който изпълнението беше на ред 4. От индуктивното предположение знаем, че

$$\forall x \in A[pp, \ldots, i-1] : x \geqslant pivot \tag{6.5}$$

След размяната на $A[i]$ и $A[pp]$, която става на ред 5, в сила е:

$$\forall x \in A[pp+1, \ldots, i] : x \geqslant pivot \tag{6.6}$$

След инкрементиранията на pp на 6 и неявното инкрементиране на i при следващото достигане на ред 3, спрямо новите стойности на тези две променливи, (6.6) става:

$$\forall x \in A[pp, \ldots, i-1] : x \geqslant pivot \tag{6.7}$$

Случай II $A[i] \geqslant pivot$. От индуктивното предположение знаем, че:

$$\forall x \in A[l, \ldots, pp-1] : x < pivot$$
$$\forall x \in A[pp, \ldots, i-1] : x \geqslant pivot$$

Тогава:

$$\forall x \in A[l, \ldots, pp-1] : x < pivot$$
$$\forall x \in A[pp, \ldots, i] : x \geqslant pivot$$

Но условието на ред 4 е лъжа и изпълнението отива на ред 3, като i се инкрементира неявно. Спрямо новата стойност на i е вярно, че:

$$\forall x \in A[l, \ldots, pp-1] : x < pivot$$
$$\forall x \in A[pp, \ldots, i-1] : x \geqslant pivot$$

Терминация. При последното достигане на ред 3 е вярно, че $i = h$. Тогава

$$\forall x \in A[l, \ldots, pp-1] : x < pivot$$
$$\forall x \in A[pp, \ldots, h-1] : x \geqslant pivot$$

Доказахме инварианта и видяхме ефекта от работата на цикъла. Но доказателството за коректността все още не е готово. Изпълнението отива на ред 7, където се разменят $A[pp]$ и $A[h]$), който всъщност е $pivot$. В сила вече е

$$\forall x \in A[l, \ldots, pp-1] : x < A[pp]$$
$$\forall x \in A[pp+1, \ldots, h] : x \geqslant A[pp]$$

После алгоритъмът връща pp. □

6.2.3 Самият Quicksort

Псевдокодът на самия Quicksort е съвсем кратък (функцията Partition е или Partition–Lomuto, или Partition–Hoare–another):

```
Quicksort(A[1,2,...,n]: цели числа; l, h: индекси в A)
1   if l < h
2       mid ← Partition(A, l, h)
3       Quicksort(A, l, mid − 1)
4       Quicksort(A, mid + 1, h)
```

Ако обаче искаме да ползваме Partition–Hoare на стр. 296, трябва да направим рекурсивните викания върху подмасиви, които представляват разбиване на оригиналния масив; с други думи, всеки от оригиналните елементи е елемент в някой от тях.

```
QUICKSORT–ANOTHER(A[1, 2, ..., n]: цели числа; l, h: индекси в A)
1  (* На стр. 154 в [22] *)
2  if l < h
3      mid ← PARTITION(A, l, h)
4      QUICKSORT(A, l, mid)
5      QUICKSORT(A, mid + 1, h)
```

Доказателството за коректност е напълно аналогично на доказателството за коректност на MERGESORT.

QUICKSORT не е стабилен сортиращ алгоритъм. Лесно е да се намери пример, в който PARTITION разменя взаимната наредба на два еднакви елемента. Примерът за PARTITION–LOMUTO може да се различава от примера за PARTITION–HOARE, но и за двата варианта има такива примери.

Забелязваме още една разлика между QUICKSORT и MERGESORT. Фазата **Разделяй** на QUICKSORT връща стойност, а именно индексът, който определя разделянето на масива на две части за рекурсивните викания. Това се налага съвсем естествено: за разлика от MERGESORT, сега не можем да изчислим индекса, определящ разделянето, без да сме прегледали масива.

6.2.4 Сложност по време на QUICKSORT

Сложността на QUICKSORT е изключително чувствителна към изборите на pivot. Казваме "изборите" в множествено число, защото става дума за изборите във всички рекурсивни викания. Ако във всяко отделно викане се оказва, че pivot дели съответния входен (под)масив на две равни части, сложността са описва от рекурентното уравнение

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

което, както вече видяхме при MERGESORT, има решение $T(n) \asymp n \lg n$. Ако обаче всеки избор на pivot е такъв, че този елемент дели масива на подмасив с един елемент (самият той) и друг подмасив с $n - 1$ елемента, сложността се описва от рекурентното уравнение

$$T(n) = T(n-1) + n$$

С метода с характеристичното уравнение е лесно да се покаже, че това рекурентно уравнение има решение $T(n) \asymp n^2$.

Следователно, в най-лошия случай QUICKSORT е квадратичен сортиращ алгоритъм, с линейна сложност по памет. Ако се задоволяваме само с анализ на сложността в най-лошия случай, това е всичко, което имаме да кажем за сложността на QUICKSORT. На практика обаче QUICKSORT е по-бърз от всеки друг известен сортиращ алгоритъм, говорейки осреднено. Причината е, че лошите избори на pivot са твърде малко вероятни, ако pivot е случаен елемент от масива и големините на елементите от масива са униформно разпределени. Затова, за пръв и последен път в този курс, ще направим анализ на средната сложност на QUICKSORT. Иначе би трябвало да класифицираме най-бързия на практика сортиращ алгоритъм като бавен.

И така, анализът на средната сложност се прави с рекурентно уравнение, което обаче отразява именно средния случай. Както се убедихме, това на какви части бива разделен входът

се определя от pivot елемента. Ако pivot е случаен елемент от масива, то всяко разделяне на входа на две части е равновероятно. Отчитаме само **броя на сравненията**. Неговата асимптотика е асимптотиката на самия алгоритъм. Следното рекурентно уравнение моделира казаното дотук:

$$T(n) = \frac{1}{n}\left(\sum_{k=1}^{n} T(k-1) + T(n-k)\right) + (n-1) \tag{6.8}$$

Променливата k е мястото **по големина**, а не по позиция, на pivot във входа. Казано по друг начин, k е "правилно място" за елемента pivot след разместването на елементите, направено от функцията PARTITION. И така, pivot може да се океже най-малък елемент ($k = 1$) или втори по големина ($k = 2$) или ... или най-голям елемент ($k = n$). Тогава рекурентно уравнение се чете така: работата на QUICKSORT е сумата от

- сумата по всички възможности за pivot ($k = 1 \ldots n$) от сумата от работата върху получени спрямо него двата подмасива. Единият подмасив е с големина $k-1$, а другият е с големина $n-k$, оттам е и изразът $T(k-1) + T(n-k)$. Сумата е умножена с $\frac{1}{n}$, като този множител отразява факта, че всички n възможности за относителната големина на pivot спрямо останалите елементи са равновероятни.
- и $n-1$, което е броят на сравненията за определяне на мястото на pivot. Очевидно pivot трябва да бъде сравнен с всеки друг елемент и това е достатъчно.

Сега ще решим (6.8).

$$\begin{aligned} T(n) &= \frac{1}{n}\left(\sum_{k=1}^{n} T(k-1) + T(n-k)\right) + (n-1) \\ &= \frac{1}{n}\left(\sum_{k=1}^{n} T(k-1) + \sum_{k=1}^{n} T(n-k)\right) + (n-1) \\ &= \frac{1}{n}\left(\underbrace{T(0) + T(1) + \ldots + T(n-1)}_{\sum_{k=1}^{n} T(k-1)} + \underbrace{T(n-1) + T(n-2) + \ldots + T(0)}_{\sum_{k=1}^{n} T(n-k)}\right) + (n-1) \end{aligned} \tag{6.9}$$

Очевидно двете подчертани суми в (6.9) са равни, така че

$$T(n) = \frac{2}{n}\big(T(0) + T(1) + \ldots + T(n-1)\big) + (n-1) \tag{6.10}$$

Умножаваме двете страни на (6.10) по n и получаваме

$$nT(n) = 2\big(T(0) + T(1) + \ldots + T(n-1)\big) + n(n-1) \tag{6.11}$$

Но тогава, ако n е достатъчно голямо,

$$(n-1)T(n-1) = 2\big(T(0) + T(1) + \ldots + T(n-2)\big) + (n-1)(n-2) \tag{6.12}$$

Изваждаме (6.12) от (6.11) и получаваме

$$\begin{aligned} nT(n) - (n-1)T(n-1) &= 2T(n-1) + n(n-1) - (n-1)(n-2) \\ &= 2T(n-1) + 2(n-1) \quad \leftrightarrow \\ nT(n) &= (n+1)T(n-1) + 2(n-1) \end{aligned} \tag{6.13}$$

Делим двете страни на (6.13) на $n(n+1)$ и получаваме

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)} \tag{6.14}$$

Удобно е да направим следното полагане, за да се освободим от нотация със знаменатели: $\frac{T(n)}{n+1}$ ще бъде наричано $S(n)$. Тогава очевидно $\frac{T(n-1)}{n} = S(n-1)$ и замествайки в (6.14), получаваме следното рекурентно уравнение:

$$S(n) = S(n-1) + \frac{2(n-1)}{n(n+1)} \tag{6.15}$$

То не може да бъде решено чрез метода с характеристичното уравнение, защото нехомогенната част не е от вида, полином на n по константа на n-та степен, така че ще използваме универсалното средство – развиване.

$$\begin{aligned} S(n) &= S(n-1) + \frac{2(n-1)}{n(n+1)} \\ &= S(n-2) + \frac{2(n-2)}{(n-1)n} + \frac{2(n-1)}{n(n+1)} \\ &= S(n-3) + \frac{2(n-3)}{(n-2)(n-1)} + \frac{2(n-2)}{(n-1)n} + \frac{2(n-1)}{n(n+1)} \\ &\dots \\ &= S(1) + \frac{2\cdot 1}{2\cdot 3} + \frac{2\cdot 2}{3\cdot 4} + \frac{2\cdot 3}{4\cdot 5} + \dots + \frac{2(n-3)}{(n-2)(n-1)} + \frac{2(n-2)}{(n-1)n} + \frac{2(n-1)}{n(n+1)} \\ &= S(1) + 2\sum_{k=2}^{n} \frac{k-1}{k(k+1)} \end{aligned} \tag{6.16}$$

Да разгледаме (6.16). $S(1)$ е константа, това е някакво начално условие, и асимптотиката на $S(n)$ се определя от $\sum_{k=2}^{n} \frac{k-1}{k(k+1)}$. Но

$$\sum_{k=2}^{n} \frac{k-1}{k(k+1)} = \sum_{k=2}^{n} \frac{k}{k(k+1)} - \sum_{k=2}^{n} \frac{1}{k(k+1)} = \sum_{k=2}^{n} \frac{1}{k+1} - \sum_{k=2}^{n} \frac{1}{k(k+1)}$$

Както знаем от Теорема 25, $\sum_{k=1}^{n} \frac{1}{k} \asymp \lg n$, откъдето веднага следва, че $\sum_{k=2}^{n} \frac{1}{k+1} \asymp \lg n$. От друга страна, $\sum_{k=2}^{n} \frac{1}{k(k+1)}$ е ограничена от константа, защото редът $\sum_{k=1}^{\infty} \frac{1}{k^2}$ е сходящ със сума $\frac{\pi^2}{6}$. Следва, че $\sum_{k=2}^{n} \frac{k-1}{k(k+1)} \asymp \lg n$. Тоест, $S(n) \asymp \lg n$. Връщаме се към нотацията $T(n)$ и получаваме, че

$$T(n) \asymp n \lg n$$

Доказахме, че QUICKSORT има средна сложност по време $\Theta(n \lg n)$. Оттук и средната слож-

ност по памет е логаритмична (а не линейна като в най-лошия случай), защото средната дълбочина на стека на рекурсията е $\Theta(\lg n)$, а на всяко ниво на рекурсията се ползва само константна допълнителна памет от функцията PARTITION.

Лекция 7

Долни граници върху сложност на задачи.

Резюме: Въвеждаме понятието долна граница върху сложност на задача. Доказваме долни граници върху броя на сравненията в две занимателни задачи. Въвеждаме понятието дърво на вземане на решение. Използвайки този модел, доказваме долна граница $\Omega(n \lg n)$ върху сложностите на Сортиране и Уникалност на елементи, при определени допускания.

7.1 Долни граници на сложността на изчислителни задачи

Определение 46: Долна и горна граница на сложността на задача

Нека Π е произволна изчислителна задача. *Долна граница върху сложността на* Π е всяка функция $\phi(n)$, такава че всеки алгоритъм, който решава Π, работи във време $\Omega(\phi(n))$. *Горна граница върху сложността на* Π е всяка функция $\psi(n)$, такава че поне един алгоритъм, който решава Π, работи във време $O(\psi(n))$.

Забележете разликата между долна и горна граница.

- За да покажем, че дадена функция е горна граница за дадена задача, достатъчно е да покажем един алгоритъм, чиято сложност по време има такава горна граница. Например, от това, че Insertion Sort работи във време $O(n^2)$ и това, че Insertion Sort решава задачата Сортиране, следва, че n^2 е асимптотична горна граница за Сортиране.

- От друга страна, за да покажем, че дадена функция е долна граница за дадена задача, не е достатъчно да покажем един алгоритъм, чиято сложност по време има такава долна граница. Например, от това, че Mergesort работи във време $\Omega(n \lg n)$ и това, че Mergesort решава задачата Сортиране, по никакъв начин не следва, че $\Omega(n \lg n)$ е долна граница за Сортиране. Доказателство за долна граница е аргумент за всички алгоритми, а те са безброй много, които решават задачата. Всеки известен алгоритъм за Сортиране, който е базиран на директни сравнения, работи във време $\Omega(n \lg n)$, но **от това не следва**, че няма по-бърз алгоритъм, работещ във време, примерно, $\Theta(n \lg \lg n)$ или дори $\Theta(n)$. Както ще видим, такъв наистина няма, но това трябва да се докаже.

Доказателство за долна граница на задача обезсмисля опитите да бъдат конструирани по-бързи алгоритми. Долната граница е **фундаментално ограничение**, нещо като природна даденост, с която сме длъжни да се съобразяваме, независимо дали ни харесва или не. Има далечна аналогия с фундаменталните ограничения от физиката, например *невъзможността за пътуване със скорост, по-голяма от скоростта на светлината във вакуум ("Consequently, the speed of light is a natural absolute speed limit.")*, *принципът на неопределеността от квантовата механика* или *първият и втори принцип на термодинамиката*.

Доказателствата на нетривиални долни граници върху сложността на задачи са трудни. Какво е тривиална долна граница, ще обясним с пример. За СОРТИРАНЕ, тривиална долна граница е $\Omega(n)$, защото няма как да сортираме масив, без да сме "прегледали" всеки елемент[†]. Доказателството, че $\Omega(n \lg n)$ е долна граница за СОРТИРАНЕ е значително по-трудно и затова казваме, че не е тривиално.

Допълнение 34: Долни граници, задачи и изчислителни модели

Ще разгледаме една тривиална долна граница: $\Omega(n^2)$ за сортирането чрез *транспозиции*. Транспозиция е размяна на местата на два съседни елементи от масива. В нашия псевдокод това се записва като "swap$(A[i], A[i+1])$". Сортиране чрез транспозиции е сортиране, при което единствените достъпи до масива за преместване на елементи са транспозиции. Пример за алгоритъм, който сортира чрез транспозиции, е BUBBLESORT [21, стр. 40].

Очевидно е, че в процеса на работата на алгоритъма, всяка извършена транспозиция може да намали броя на инверсиите (Определение 45) с най-много единица. И тъй като инверсиите може да са $\frac{n(n-1)}{2}$ (при обратно сортиран масив от два по два различни елементи), то само "ликвидирането" на инверсиите налага квадратичен брой транспозиции. Масивът не може да е сортиран, ако има инверсии (Наблюдение 29), и от това веднага следва, че всеки алгоритъм, сортиращ само чрез транспозиции, има долна граница за сложността $\Omega(n^2)$.

Тази долна граница обаче не е върху самата изчислителна задача СОРТИРАНЕ, а върху СОРТИРАНЕ с някакво допълнително ограничение: да се ползват само транспозиции. Ограничението не е част от дефиницията на задачата. Да си припомним Разяснение 1: задачата е функция или релация, изобразяваща примери в решения, а алгоритъм за нея е някаква приемливо детайлна реализация на тази функция или релация. Ограничението да се ползват само транспозиции е ограничение не върху задачата, а върху алгоритмите за нея.

Ключовото понятие тук е "изчислителен модел", което (бегло) споменахме в Допълнение 4. Изчислителният модел определя какво можем да правим. Очевидно, за да ограничим сортирането до такова, което ползва само транспозиции, трябва да наложим ограничения върху нашия изчислителен модел. Изследването на долни граници винаги е в контекста на даден изчислителен модел. В тези лекционни записки изчислителни модели не се разглеждат и само се споменават в допълнения, но внимателния читател ще забележи, че изследването на долни граници иска първо да е дефиниран изчислителен модел.

За ограничените цели на лекционните записки можем да избегнем разглеждането на

[†]Аналогично, както ще видим в следваща лекция, тривиална долна граница за обхождане на граф, представен със списъци на съседства, е $\Omega(n+m)$, където n е броят на върховете и m е броят на ребрата, поради това, че размерът на списъците на съседства е $\Theta(n+m)$, и няма как да обходим графа, без да сме "прегледали" всеки елемент от тези списъци.

изчислителни модели при долните граници, като кажем, че долната граница не е само върху дадена изчислителна задача, а върху задачата и някакъв клас алгоритми за нея; примерно, Сортиране и алгоритмите, ползващи само транспозиции.

И още една забележка. Това, че $\Omega(n)$ е долна граница за Сортиране и това, че $\Omega(n \lg n)$ е долна граница за Сортиране (което все още не сме доказали), не са в противоречие. Просто втората долна граница е по-висока, следователно е и по-добра (и по-интересна). Когато става дума за долни граници, интересуваме се от колкото е възможно по-високи такива. Дуално, при горните граници искаме колкото е възможно по-ниски такива: $O(n^9)$ е валидна горна граница за Сортиране, но тя е безинтересна предвид факта, че $O(n^2)$ също е горна граница, а на свой ред тя е безинтересна предвид това, че $O(n \lg n)$ е горна граница.

Доказателствата за долни граници, които ще разгледаме, се класифицират така.

- Елементарни доказателства. Примерно, Сортиране има тривиална долна граница $\Omega(n)$, защото всеки елемент трябва да бъде разгледан поне веднъж. Също така, Търсене в несортиран масив има тривиална долна граница $\Omega(n)$ – по същата причина.
- Доказателства, използващи дървета за вземане на решения (на английски, *decision trees*). Примери за такива доказателства има в тази лекция: долните граници 2 за The Balance Puzzle, 3 за The Twelve-Coin Puzzle, $\Omega(n \lg n)$ за Сортиране и $\Omega(n \lg n)$ за Уникалност на елементи използват дървета за вземане на решения.
- Доказателства, използващи редукции. Примерно, долната граница $\Omega(n \lg n)$ за Най-близка двойка елементи. Редукцията там е, Уникалност на елементи се редуцира до Най-близка двойка елементи.
- Аргументиране чрез противник (на английски, *adversary argument*).

7.2 Дървета за вземане на решения

7.2.1 Задачите The Balance Puzzle и The Twelve-Coin Puzzle

Сега ще разгледаме две занимателни задачи, решението на които ще ни даде необходимата интуиция за ключовото понятие "дърво на вземане на решения".

Задача 26: The Balance Puzzle

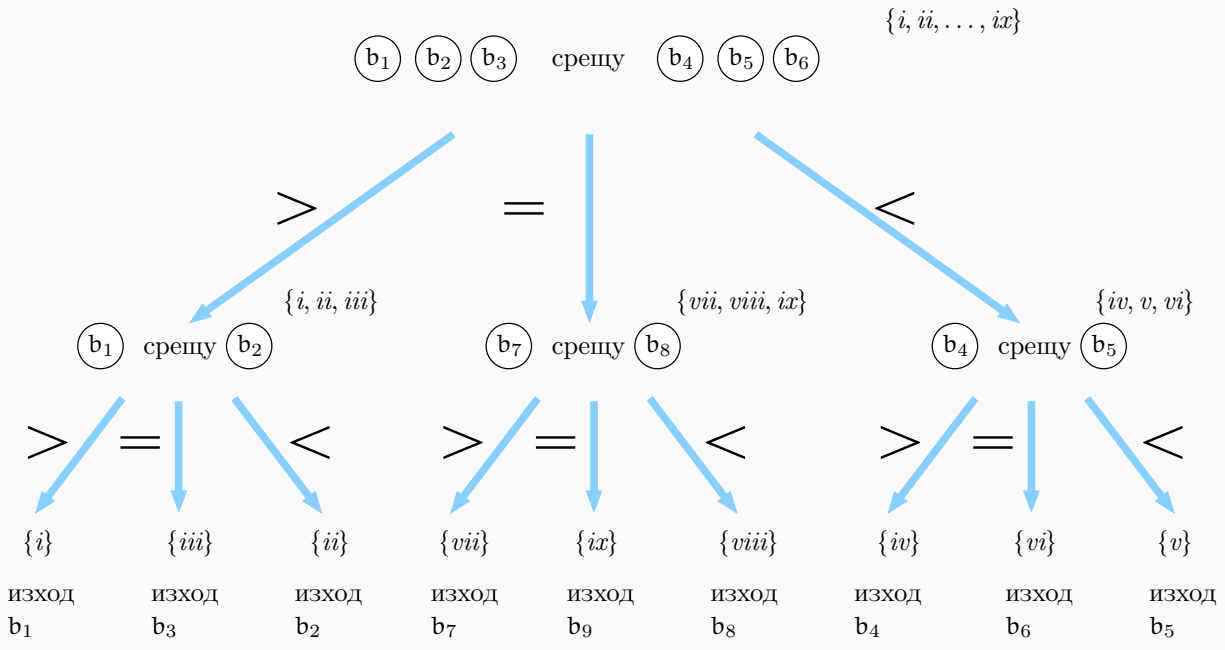
Дадени са 9 номерирани предмета, да речем топки. Осем от тях имат едно и също тегло, а една топка е по-тежка (от коя да е от осемте). Нашата цел е да идентифицираме тежката топка, като използваме везни. Везните нямат стандартни теглилки, така че можем правим единствено измервания от вида: някои топки на едното блюдо срещу други топки на другото блюдо, и да наблюдаваме резултата. Има точно три възможности за резултата: везната да се наклони наляво или везната да се наклони надясно или или везната да остане балансирана. Трябва да намерим тежката топка с колкото е възможно по-малко измервания.
Предложете схема за измервания, която ползва не повече от две измервания. Докажете, че две е долна граница за броя на измерванията.

Решение: Нека деветте топки са $b_1, \ldots, b_9$. С едно използване на везната мерим b_1, b_2 и b_3 срещу b_4, b_5 и b_6. Има точно три възможни изхода.

- Ако $\mathfrak{b}_1$, $\mathfrak{b}_2$ и $\mathfrak{b}_3$ са по-тежки от $\mathfrak{b}_4$, $\mathfrak{b}_5$ и $\mathfrak{b}_6$, използваме везната втори път с $\mathfrak{b}_1$ срещу $\mathfrak{b}_2$. Отново има точно три възможни изхода.
  - Ако $\mathfrak{b}_1$ е по-тежка от $\mathfrak{b}_2$, връщаме *“$\mathfrak{b}_1$ е тежката топка”.*
  - Ако $\mathfrak{b}_2$ е по-тежка от $\mathfrak{b}_1$, връщаме *“$\mathfrak{b}_2$ е тежката топка”.*
  - Ако $\mathfrak{b}_1$ и $\mathfrak{b}_2$ са еднакво тежки, връщаме *“$\mathfrak{b}_3$ е тежката топка”.*
- Ако $\mathfrak{b}_4$, $\mathfrak{b}_5$ и $\mathfrak{b}_6$ са по-тежки от $\mathfrak{b}_1$, $\mathfrak{b}_2$ и $\mathfrak{b}_3$, използваме везната втори път с $\mathfrak{b}_4$ срещу $\mathfrak{b}_5$. Отново има точно три възможни изхода.
  - Ако $\mathfrak{b}_4$ е по-тежка от $\mathfrak{b}_5$, връщаме *“$\mathfrak{b}_4$ е тежката топка”.*
  - Ако $\mathfrak{b}_5$ е по-тежка от $\mathfrak{b}_4$, връщаме *“$\mathfrak{b}_5$ е тежката топка”.*
  - Ако $\mathfrak{b}_4$ и $\mathfrak{b}_5$ са еднакво тежки, връщаме *“$\mathfrak{b}_6$ е тежката топка”.*
- Ако $\mathfrak{b}_1$, $\mathfrak{b}_2$ и $\mathfrak{b}_3$, от една страна, и $\mathfrak{b}_4$, $\mathfrak{b}_5$ и $\mathfrak{b}_6$, от друга страна, са еднакво тежки, използваме везната втори път с $\mathfrak{b}_7$ срещу $\mathfrak{b}_8$. Отново има точно три възможни изхода.
  - Ако $\mathfrak{b}_7$ е по-тежка от $\mathfrak{b}_8$, връщаме *“$\mathfrak{b}_7$ е тежката топка”.*q
  - Ако $\mathfrak{b}_8$ е по-тежка от $\mathfrak{b}_7$, връщаме *“$\mathfrak{b}_8$ е тежката топка”.*
  - Ако $\mathfrak{b}_7$ и $\mathfrak{b}_8$ са еднакво тежки, връщаме *“$\mathfrak{b}_9$ е тежката топка”.*

Фигура 7.1 илюстрира тази схема.

Фигура 7.1 : Схема за измервания за The Balance Puzzle.



Възможностите (състоянията) са записани с римски цифри. Да решим задачата е същото като да отговорим, коя от деветте възможности е истината.

Много полезно е да мислим в термините на възможности, или възможни състояния. Поначало има точно девет възможности: или $\mathfrak{b}_1$ е тежката топка, или $\mathfrak{b}_2$ е тежката топка, и

така нататък, или b_9 е тежката топка. Нека запишем тези девет състояния със, съответно, i, ii, ..., ix. В началото множеството от състоянията е $\{i, ii, \ldots, ix\}$. След всяко измерване, множеството от възможностите намалява. Задачата бива решена когато множеството от възможностите стане едноелементно. На Фигура 7.1 до всеки връх е записано множеството от възможностите, които са консистентни с резултатите от измерванията досега, изключвайки текущото измерване. Тоест, консистентни с резултатите от всички върхове от корена до родителя на настоящия връх.

Доказателството за долна граница е следното. Схемата трябва да е такава, че да различи една възможност от общо девет, използвайки тернарно[†] дърво на вземане на решение. Тернарно дърво с височина 1 има само три листа, така че може да различава само три възможности, което не върши работа за тази задача. Очевидно дървото трябва да има височина поне 2, за да има поне девет листа, с които да различава девет възможности.

Наблюдение 30

Броят на листата на дървото в схемата за измерване е горна граница за броят на състоянията (възможностите), които съответната схема за измерване може да различава.

Забележете, че аргументацията за долната граница няма **нищо общо** с конкретната схема, която изложихме горе. Това, че изведохме долна граница две за броя на измерванията съвсем не означава автоматично, че тази долна граница е достижима. В случая, тя **е** достижима, но това следва от показаната конкретна схема. Аргументът за долна граница казва само, че с по-малко измервания няма да стане. □

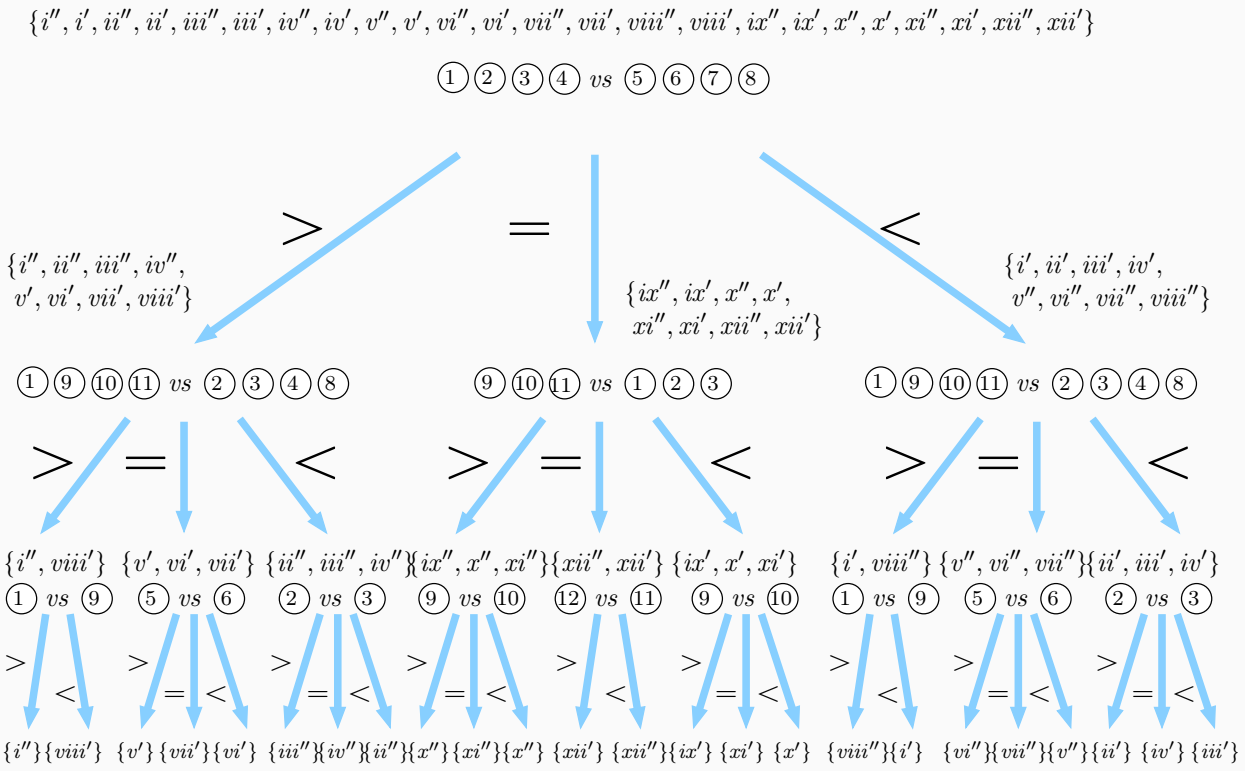
Задача 27: The Twelve-coin Puzzle

Дадени са 12 номерирани монети. Измежду тях, 11 имат едно и също тегло, а другата—да я наречем *странната монета*—е или по-лека, или по-тежка. Нашата задача е да идентифицираме странната монета, както и да определим дали е лека или тежка, използвайки везни като тези в Задача 26. Монетите са различими на външен вид, примерно са номерирани c_1, ..., c_{12}.
Предложете схема за измервания, с която идентифицираме странната монета и нейната странност с не повече от три измервания. Докажете, че три е долна граница за броя на измерванията.

Решение: Тази задача е по-сложна от предишната. Броят на възможностите сега е 24: два пъти броят на монетите. Нека i' означава "c_1 е тежка", i'' означава "c_1 е лека", ii' означава "c_2 е тежка", ii'' означава "c_2 е лека", и така нататък, xii' означава "c_{12} е тежка" и xii'' означава "c_{12} е лека". Фигура 7.2 показва схема от измервания с най-много три използвания на везните, която определя коя от 24-те възможности е истината.

[†]"Тернарно" кореново дърво е такова, в което всеки връх има най-много три деца.

Фигура 7.2 : Схема от измервания за The Twelve-coin Puzzle.



Понякога само два от трите изхода са възможни. Всеки връх в дървото е асоцииран с множеството от възможностите, които са консистентни с измерванията до този момент.

Дотук доказахме, че три измервания са достатъчни.

Сега ще докажем, че три измервания са необходими. Тъй като искаме да различим една възможност от общо 24 възможности и всяко измерване има най-много три възможни изхода, долна граница за броя на измерванията е $\lceil \log_3 24 \rceil = 3$. Защо? Защото тернарно дърво с $\geqslant 24$ листа има височина поне три.

Забележете, че долната граница, която следва от такива съображения (височината на дървото като логаритъм от броя на листата) не винаги е достижима. В случая с The Twelve-coin Puzzle, долната граница наистина е достижима, както виждаме от схемата на Фигура 7.2. Но същата задача с 13 монети **не може** да бъде решена с най-много три измервания в най-лошия случай. Ето защо:

- Няма смисъл да мерим едно срещу друго две множества от монети с различен брой монети, защото, ако блюдото с повечето монети се наклони надолу, ние не придобиваме **никаква нова информация**, в смисъл, че възможностите не намаляват, а остават същите.
- Имайки предвид предното съображение, за всеки избор на две подмножества с една и съща кардиналност (от множеството от 13-те монети), или тези множества имат кардиналност $\geqslant 5$, или множеството от останалите монети има кардиналност $\geqslant 5$.
- Множество с кардиналност $\geqslant 5$ има $\geqslant 10$ възможности за странната монета.
- Няма начин да определим една възможност измежду $\geqslant 10$ възможности, използвайки $\leqslant 2$ измервания, защото при не повече от три изхода от измерване, две последователни

измервания могат да различат най-много 9 възможности.

Показахме долна граница четири за броя на измерванията. От друга страна, $\lceil \log_3 26 \rceil = 3$. Няма противоречие между факта, че $\lceil \log_3 26 \rceil = 3$, и факта, че The Thirteen-coin Puzzle изисква поне четири измервания в най-лошия случай. Това са различни аргументации за долна граница, като едната аргументация дава долна граница три, а другата, четири. Три си остава валидна долна граница, просто при The Thirteen-coin Puzzle долната граница три не е достижима; иначе казано, долната граница три не е точна. □

7.2.2 Долна граница $\Omega(n \lg n)$ за Сортиране

Долната граница $\Omega(n \lg n)$, която ще докажем, не е в сила за всеки сортиращ алгоритъм, а **само за сортиращ алгоритъм, базиран на директни сравнения**. В [22, стр. 191] това понятие е обяснено така:

> In a comparison sort, we use only comparisons between elements to gain order information about an input sequence $\langle a_1, a_2, \ldots, a_n \rangle$. That is, given two elements a_i and a_j, we perform one of the tests $a_i < a_j$, $a_i \leqslant a_j$, $a_i = a_j$, $a_i \geqslant a_j$, or $a_i > a_j$ to determine their relative order. We may not inspect the values of the elements or gain order information about them in any other way.

Както ще се убедим от изложението в тази глава, тази дефиниция е ненужно ограничаваща. Ако променим, примерно, Insertion Sort, слагайки в самото начало код, който преброява елементите от всеки вид (това може да се направи тривиално в $O(n^2)$ време и $O(n)$ памет), и оставяйки същинското сортиране същото, то това ще остане сортиращ алгоритъм, базиран на директни сравнения, въпреки че сега "инспектира" стойностите на елементите. Съществено е не дали алгоритъмът инспектира стойности, а дали пресмята изходната сортирана последователност само въз основа на директни сравнения, или не. Поради това тук правим следното определение.

Определение 47: Сортиращ алгоритъм, базиран на директни сравнения

Сортиращ алгоритъм, базиран на директни сравнения, е всеки сортиращ алгоритъм, който извършва директни сравнения от вида $a_i \overset{?}{<} a_j$ върху входните елементи и изходът се определя еднозначно от серията извършени директни сравнения.

Образно казано, всеки алгоритъм за сортиране, базиран на директни сравнения, използва везна, подобна на везните от Подсекция 7.2.1, само че с два, а не три възможни изхода.

Всички разгледани досега сортиращи алгоритми (глави 3, 4 и 6) са базирани на директни сравнения, в контраст с Counting Sort от Глава 8, който не е базиран на директни сравнения, а брои колко елемента има от всеки вид, като броят на видовете е ограничен.

Сега ще се убедим с пример, че на всеки сортиращ алгоритъм, базиран на директни сравнения, съответства схема от въпроси и отговори, като въпросите са от вида $a_i \overset{?}{<} a_j$ или $a_i \overset{?}{\leqslant} a_j$, а отговорите са бинарни. Да си припомним Insertion Sort.

```
Insertion Sort(A[1, ..., n])
1   for i ← 2 to n
2       key ← A[i]
3       j ← i − 1
4       while j > 0 and key < A[j] do
```

```
5            A[j + 1] ← A[j]
6            j ← j − 1
7        A[j + 1] ← key
```

Да разгледаме работата на INSERTION SORT върху вход с големина три. Нека входът е

$$a_1, a_2, a_3$$

С малки букви, примерно "a_1", означаваме елементите от входа. Те са неизменни, в смисъл че който и да е от тях, да кажем a_1, е едно и също нещо във всеки момент от работата на алгоритъма. С главни букви, примерно "$A[1]$", означаваме елементите от текущия масив A. Те може да се менят по време на работата! В началото е вярно, че $A = [a_1, a_2, a_3]$, но след няколко стъпки от изпълнението може да имаме $A = [a_2, a_3, a_1]$. Да допуснем, че елементите от входа са уникални. Точно едно от следните е в сила:

$$a_1 < a_2 < a_3 \tag{7.1}$$
$$a_1 < a_3 < a_2 \tag{7.2}$$
$$a_2 < a_1 < a_3 \tag{7.3}$$
$$a_2 < a_3 < a_1 \tag{7.4}$$
$$a_3 < a_1 < a_2 \tag{7.5}$$
$$a_3 < a_2 < a_1 \tag{7.6}$$

Ще наричаме (7.1), ..., (7.6), *пермутациите*[†]. Ключовото наблюдение, без което няма истинско разбиране на понятието "дърво за вземане на решение при сортиране, базирано на директни сравнения", е следното.

Наблюдение 31

Да сортираме входа с директни сравнения в някакъв смисъл е същото като да определим, само в резултат на изходите от сравненията, коя от пермутациите е истината.

Сравненията стават само на ред 4 в $\text{key} < A[j]$. Първото сравнение е $a_2 < a_1$.

Случай I Ако $a_2 < a_1$ е TRUE, то точно тези три пермутации (от началните шест) са възможни:

$$a_2 < a_1 < a_3$$
$$a_2 < a_3 < a_1$$
$$a_3 < a_2 < a_1$$

В този случай алгоритъмът променя A по време на текущото изпълнение на **for**-цикъла, така че A става $[a_2, a_1, a_3]$. Други сравнения не се правят по време на текущото изпълнение на **for**-цикъла и започва ново негово изпълнение. Следващото сравнение е $a_3 < a_1$. Но първо да разгледаме алтернативния случай на **Случай I**.

[†]Всеки от (7.1), ..., (7.6) е логически израз – конюнкция от две неравенства. Стриктно говорейки, това не е пермутация. Казвайки "пермутация", имаме предвид съответната пермутация. Примерно, на (7.5) съответства пермутацията $a_3\, a_1\, a_2$.

Случай II Ако $a_2 < a_1$ е FALSE, то точно тези три пермутации са възможни:

$$a_1 < a_2 < a_3$$
$$a_1 < a_3 < a_2$$
$$a_3 < a_1 < a_2$$

В този случай алгоритъмът не променя A по време на текущото изпълнение на **for**-цикъла, така че A остава $[a_1, a_2, a_3]$. Други сравнения не се правят по време на текущото изпълнение на **for**-цикъла и започва ново негово изпълнение.

Случай I.1 Връщаме се към **Случай I**. Ако $a_3 < a_1$ е TRUE, точно тези две пермутации остават възможни:

$$a_2 < a_3 < a_1$$
$$a_3 < a_2 < a_1$$

A става $[a_2, a_1, a_1]$, като стойността a_3 се съхранява в променливата key. Следващото сравнение е $a_3 < a_2$.

Случай I.1.a Ако $a_3 < a_2$ е TRUE, остава една единствена възможна пермутация:

$$a_3 < a_2 < a_1$$

while-цикълът се изпълнява още веднъж и A става $[a_2, a_2, a_1]$. Тогава a_3 се записва на първа позиция в A (line 7) и A става $[a_3, a_2, a_1]$. После алгоритъмът приключва работата си.

I.1.b Ако $a_3 < a_2$ е FALSE, остава една единствена възможна пермутация:

$$a_2 < a_3 < a_1$$

while-цикълът не се изпълнява повече. a_3 се записва на втора позиция в A (line 7)) и A става $[a_2, a_3, a_1]$.

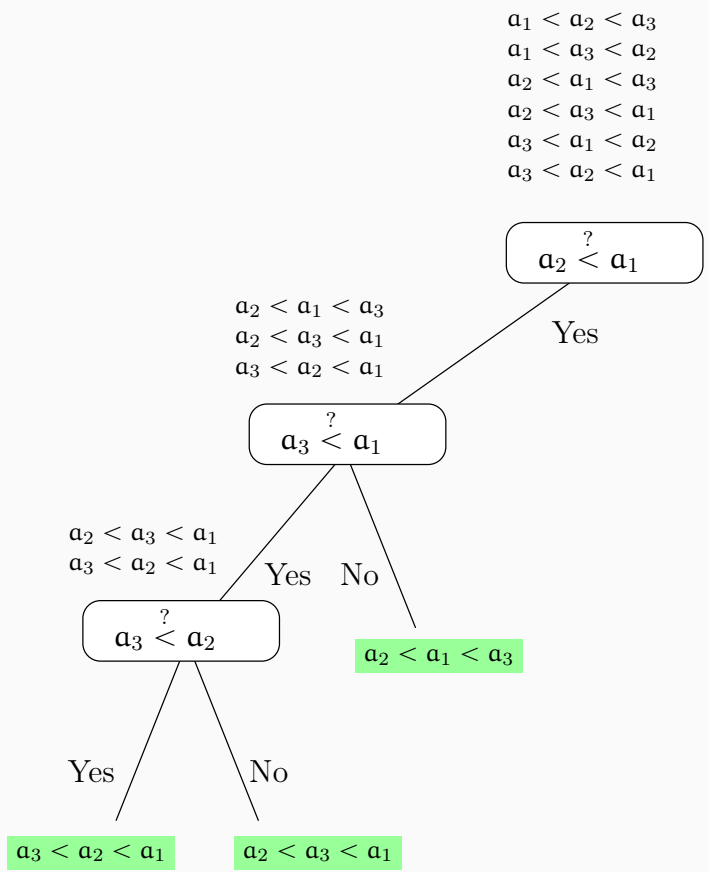
Случай I.2 Ако $a_3 < a_1$ е FALSE, остава една единствена възможна пермутация:

$$a_2 < a_1 < a_3$$

Алгоритъмът приключва работата си. A остава $[a_2, a_1, a_3]$.

Четирите подслучай на **Случай I** може да бъдат представени чрез дървовидната структура на Фигура 7.3. Има два типа върхове—върхове-сравнения, съответстващи на въпроси от вида $a_i \overset{?}{<} a_j$, и листа (в зелено). Над всеки връх-сравнение записваме множеството от пермутациите, които са консистентни с отговорите до този момент (преди това сравнение). Естествено, над корена са записани всички шест пермутации, които са възможни поначало. Листата отговарят на възможните изходи от алгоритъма.

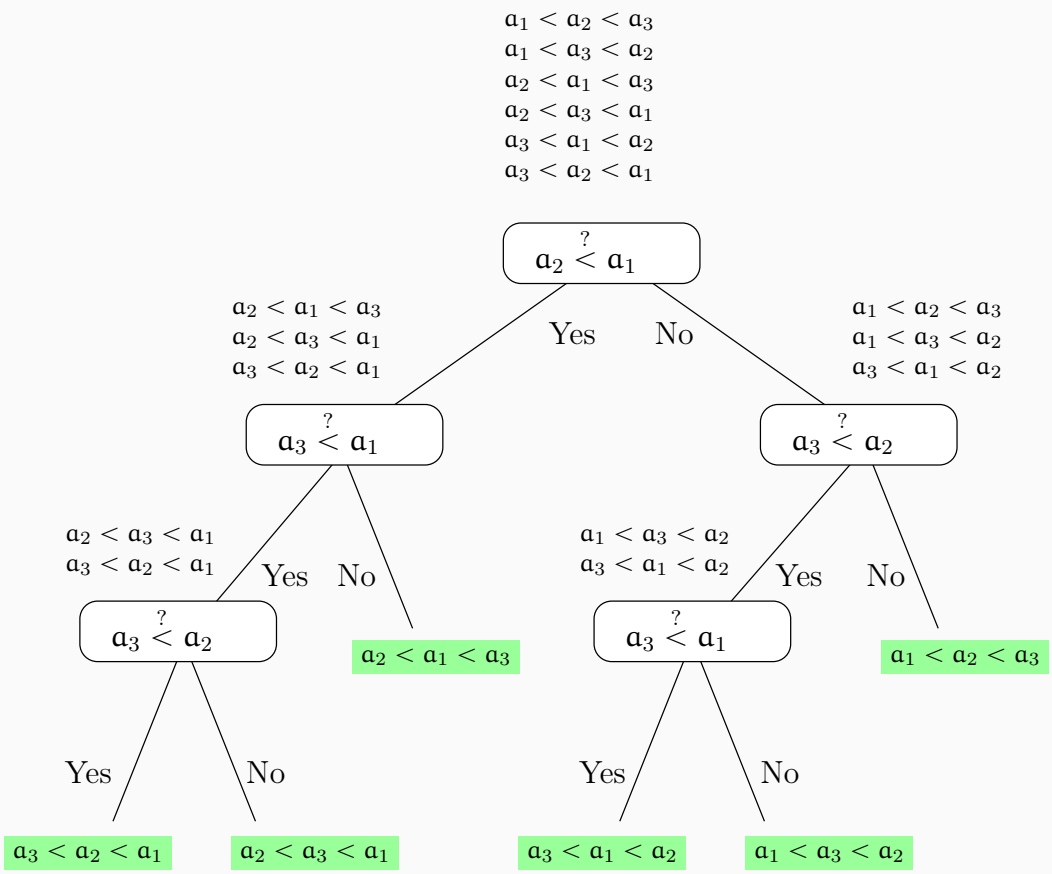
Фигура 7.3 : Половината от дървото на сравненията.



Изобразени са случаите, в които $a_2 < a_1$.

Ако направим аналогичен анализ на подслучаите на **Случай II** ще получим структурата, показана на Фигура 7.4. Дървото T е двоично, понеже има два възможни отговора при всяко сравнение.

Фигура 7.4 : Дървото на сравненията на INSERTION SORT върху $\langle a_1, a_2, a_3 \rangle$.



Дървото е схема от въпроси и отговори, които определят коя е сортираната пермутация на $\langle a_1, a_2, a_3 \rangle$.

Изведохме дървовидната структура чрез подробно симулиране на работата на INSERTION SORT върху всички възможни[†] входове с размер три. Такава дървовидна схема от въпроси и отговори се нарича *дърво на вземане на решения*, на английски *decision tree*. Определението в [21, стр. 192] е:

> *A decision tree* is a full binary tree[‡] that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size. Control, data movement, and all other aspects of the algorithm are ignored.

Дърветата на вземане на решения няма нужда да са двоични и не са ограничени до задачата СОРТИРАНЕ. Както видяхме в миналата секция, известните задачи THE BALANCE PUZZLE и THE TWELVE-COIN PUZZLE (вж. Задача 26 на стр. 310 и Задача 27 на стр. 312) са податливи на анализ чрез дървета на вземане на решение, само че тернарни.

При всяко от тези:

- THE BALANCE PUZZLE,
- THE TWELVE-COIN PUZZLE,
- INSERTION SORT върху вход с размер три

[†]Като относителни големини.

[‡]*Full binary tree* е двоично дърво, в което всеки вътрешен връх има точно две деца. *Binary tree* е дърво, в което всеки вътрешен връх има най-много две деца.

дървото на вземане на решения е едно единствено, защото става дума за една единствена големина на входа. Очевидно е, че Insertion Sort по принцип има не едно такова дърво, а безкрайно множество от дървета, по едно за всеки размер на входа[†].

Наблюдение 32

Нека A е произволен сортиращ алгоритъм, базиран на директни сравнения. A има не едно единствено дърво на вземане на решения, а **безкрайно множество** такива дървета, **по едно за всеки размер на входа**. Ако кажем просто "дървото на вземане на решения", имаме предвид общия елемент на това множество.

Допълнение 35: Decision trees са изчислителни модели

При разсъжденията за Insertion Sort ние изведохме дървото от алгоритъма. Но може да мислим за дървото като първичен обект. Дървото, в някакъв смисъл, сортира, само че не чрез местене на елементи, а определяйки коя е сортираната пермутация. И така, може да мислим, че дървото е първичният обект, а алгоритъмът е вторичен, като алгоритъмът мести елементи по такъв начин, че в правилният момент "на везната" да бъдат поставени правилните елементи. "Везната" в случая с Insertion Sort е $\text{key} < A[j]$.

Дърветата за вземане на решения са изчислителни модели, защото те смятат. Ползвайки дърво за вземане на решения, можем да пресметнем коя е тежката топка или коя е сортираната пермутация. Когато разглеждаме дърветата за вземане на решения като изчислителен модел, ние ги разглеждаме именно като първични обекти.

Казваме, че дървото на вземане на решение е *неуниформен изчислителен модел*, на английски *nonuniform computational model*, защото за всяка големина на входа изчисляващото дърво е различно. Друг пример за неуниформен изчислителен модел са логическите схеми (на английски, *logic circuits*; вижте [78, стр. 16]). В контраст на това, машината с произволен достъп [78, стр. 19] е униформен изчислителен модел, защото за всяка големина на входа тази машината е една и съща. Машината на Тюринг също е униформен изчислителен модел.

Сега ще покажем, че дърветата на вземане на решение ни дават механизъм за доказване на долни граници на задачи. Първото ще разгледаме конкретното дърво T, съответстващо на Insertion Sort с размер на входа три (Фигура 7.4). Забелязваме, че има листа с дълбочина две и листа с дълбочина три. Това означава, че върху някои входове определяме сортираната пермутация с два въпроса, а върху други, с три. Височината на дървото е максималният брой въпроси, които са ни необходими **в най-лошия случай**, по отношение на конкретната схема. Естествено, има много други възможни схеми, отговарящи на други сортиращи алгоритми. Нека читателят сам си направи дървото-схема, отговарящо на Selection Sort с размер на входа три, и ще се убеди, че то е различно от дървото-схема на Insertion Sort.

Но, каквото и да е дървото-схема, за вход с размер три, то трябва да има височина поне три. Защо? Допуснете, че има дърво T' на вземане на решения за сортиране на три елемента, което има височина най-много две. Тоест, схема от въпроси S', която с най-много два въпроса определя коя е сортираната пермутация. Но T' също е двоично дърво, защото всеки въпрос

[†] The Balance Puzzle и The Twelve-coin Puzzle също имат обобщения за n топки или монети, където n е произволно цяло положително число. Всяко от тези обобщения има безкрайно множество от дървета на вземане на решение: по едно за всеки конкретен брой топки или монети.

има два възможни отговора. Тривиално наблюдение е, че T' не може да има повече от четири листа, откъдето веднага следва, че S' не може да различи повече от четири възможности. С други думи, S' не може да различи повече от четири пермутации. Само че общо пермутациите са шест, следователно S' не е коректна схема от въпроси. Полученото противоречие показва, че такова дърво T' не съществува.

Забележете, че този анализ не разглежда конкретните особености на хипотетичното дърво T'. Със сигурност бихме могли да разгледаме едно след друго всички дървета за вземане на решение за вход с размер три и да установим, че всяко от тях има височина поне три. Само че това би било твърде отегчително, и освен това този конструктивент подход не се мащабира за произволен размер на входа. Докато не-конструктивният анализ се мащабира за **произволен** размер на входа. В основата на неконструктувния анализ е следното просто наблюдение: ако листата на хипотетичното дърво са по-малко от възможностите-пермутации, то, съгласно по принципа на Дирихле, поне едно листо е асоциирано с повече от една пермутация, от което следва, че това дърво не сортира.

Ето как се мащабира не-конструктивният анализ за вход с размер n. За всеки вход с размер n има схема от въпроси, която установява сортираната пермутация, като въпросите са от вида $a_i \overset{?}{<} a_j$. Знаем, че такива схеми има, защото съществуват коректни сортиращи алгоритми, базирани на директни сравнения, а на всеки сортиращ алгоритъм съответства безкрайно множество от дървета на вземане на решения, по едно за всяка големина на входа. Размерите на тези дървета растат експлозивно с нарастването на n, защото броят на листата е $\geqslant n!$. Следователно, не е практично да се опитваме да рисуваме такова дърво за $n > 3$. Но това не е важно. Важното е, че ако разгледаме множеството $\mathfrak{T}_n$ от всички възможни дървета на вземане на решение за размер на входа n, за произволно n, **дървото с най-малка височина от $\mathfrak{T}_n$ задава долна граница за броя на сравненията за задачата Sorting върху вход с големина n**. Тогава долна граница за височината на кое да е дърво от $\mathfrak{T}_n$ е долна граница за изчислителната задача Сортиране, базирано на директни сравнения.

И така, всяко дърво от $\mathfrak{T}_n$ трябва да има поне $n!$ листа, защото толкова са възможните пермутации. Тъй като дървото е бинарно, височината му е поне логаритмична при основа 2 в броя на листата. Тогава всяко дърво от $\mathfrak{T}_n$ има височина $\geqslant \log_2 n!$. Както вече показахме в Теорема 22 в $\log_2 n! = \Theta(n \lg n)$. Веднага следва, че всяко дърво от $\mathfrak{T}_n$ има височина $\Omega(n \lg n)$, а оттук следва долната граница $\Omega(n \lg n)$ за Сортиране, базирано на директни сравнения.

7.2.3 Долна граница $\Omega(n \lg n)$ за Уникалност на елементи

Да си припомним тази изчислителна задача, въведена в Подсекция 3.2.3 на стр. 128.

Изч. Задача: Element Uniqueness

пример: Числа $a_1, a_2, \ldots, a_n$.
въпрос: Дали всички числа са уникални?

Разглеждаме задачата само в изчислителен модел, базиран на директни сравнения. Алгоритъм за задачата Element Uniqueness е базиран на директни сравнения, ако достъпът до елементите става само по един начин: чрез изпълнения на операция на сравнения $a_i \overset{?}{<} a_j$ с бинарен изход. Изходите от тези сравнения определят и работата на алгоритъма. Наивният, очевиден алгоритъм е сравняването на всички ненаредени двойки от входа. Сложността на този подход очевидно е $\Theta(n^2)$. По-изтънчен подход е посочен в Подсекция 3.2.3: първо

сортираме входа във време $\Theta(n \lg n)$ и след това с едно "премитане" (на английски, *linear sweep*) установяваме във време $\Theta(n)$ дали няма или има повтаряния: ако има повтарящи се елементи, те задължително образуват непрекъсната последователност. Сложността по време е $\Theta(n \lg n) + \Theta(n) = \Theta(n \lg n)$.

```
Comparison-based Element Uniqueness(A[1, ..., n]: array of integers)
1  Sort(A)
2  are_unique ← True
3  for i ← 2 to n
4      if A[i − 1] = A[i]
5          are_unique ← False
6  return are_unique
```

Ще докажем, че всеки алгоритъм за Element Uniqueness, базиран на директни сравнения, има сложност $\Omega(n \lg n)$. Ще направим доказателство с дърво на вземане на решения. Както вече посочихме при сортирането, всеки алгоритъм, базиран на директни сравнения, има безкрайно множество от дървета за вземане на решения, асоциирани с него, по едно за всяка големина на входа. Ние ще разгледаме общия елемент на това множество, който е дървото, асоциирано с вход n. За да докажем желаното твърдение, достатъчно е да докажем, че листата на дървото са $\geqslant n!$ на брой и после да приложим същите разсъждения, които направихме на предишната страница за сортирането, базирано на директни сравнения.

За да докажем, че дървото има $\geqslant n!$ листа, достатъчно е да покажем, че то различава[†] всички пермутации – в смисъл, че всяко листо е асоциирано с не повече от една пермутация. По отношение на сортирането, това е тривиално: ако дървото има листо, асоциирано с повече от една пермутация, съответният алгоритъм не е коректен сортиращ алгоритъм. По отношение на уникалността на елементите, това не е толкова очевидно. Доказателството надолу следва доказателството от [84].

Теорема 47: Листата на дървото на вземане на решения имат по $\leqslant 1$ пермутация

Нека X е произволен алгоритъм за Element Uniqueness, базиран на директни сравнения. Нека неговото дърво за вземане на решения за вход с големина n е T. Всяко листо на T е асоциирано с най-много една пермутация на входните елементи.

Доказателство:

Можем да допуснем, че входните елементи са уникални, защото правим анализ на най-лошия случай, а това това ограничение се мащабира: за всяко n има вход с големина n, състоящ се от уникални елементи, и щом X е коректен алгоритъм, той трябва да работи коректно и върху този вход.

Да допуснем, че най-малката стойност във входа е b_1, втората най-малка е b_2, и така

[†]Трябва да поясним в какъв смисъл алгоритъм за уникалност на елементи различава пермутации. При сортирането е ясно, защото резултатът от процеса на сортиране е пермутация, точно една на брой. При уникалността обаче резултатът е булева стойност, а не пермутация, така че може би не е очевидно в какъв смисъл алгоритъмът различава пермутации. Но припомнете си, че алгоритъмът за уникалносте е базиран на **директни сравнения**. В самото начало всички пермутации са възможни в смисъл, че не знаем кой елемент е най-малък и т. н., но след като започнем да правим сравнения, след всяко сравнение, множеството от пермутациите, консистентни с изходите от сравненията до момента, намалява. И така, пермутациите, асоциирани с даден връх на дървото са точно тези, които са консистентни с изходите от сравненията до момента.

нататък, най-голямата стойност е b_n. С други думи,

$$b_1 < b_2 < \cdots < b_n$$

Лема 44

В текущия контекст, за всяко k, такова че $1 \leqslant k < n$, X сравнява b_k с b_{k+1} в даден момент.

Доказателство: Да допуснем противното. Тогава съществува вход $A[1, \ldots, n]$ от уникални елементи, такива че X не сравнява k-ия с $(k+1)$-вия по големина елемент. Щом X е коректен сортиращ алгоритъм и елементите на A са уникални, $X(A[1, \ldots, n])$ връща TRUE. Нека $A[i]$ е елементът от входа със стойност b_k и $A[j]$ е елементът от входа със стойност b_{k+1}. Да преобразуваме $A[1, \ldots, n]$ в $A'[1, \ldots, n]$, присвоявайки b_{k+1} на $A[i]$; с други думи, правим $A[i] \leftarrow A[j]$ и наричаме този масив A'. После да изпълним $X(A')$.

От една страна, A' има повтарящи се елементи и X е коректен алгоритъм, следователно $X(A')$ трябва е FALSE. От друга страна, работата на X върху A' е напълно идентична на работата му върху A, понеже по допускане X никога не сравнява $A[i]$ с $A[j]$, а резултатите от всички други сравнения са едни и същи и при вход A, и при вход A'. Но тогава $X(A[1, \ldots, n])$ и $X(A'[1, \ldots, n])$ връщат една и съща стойност. ↯ □

Доказахме, че X задължително сравнява всяка ненаредена двойка (има $n-1$ такива) от елементи **със съседни стойности**.

Лема 45

Нека $\mathbf{a} = \langle a_1, a_2, \ldots, a_n \rangle$ е редица от числа, две по две различни, и нека $\mathbf{a}' = \langle a'_1, a'_2, \ldots, a'_n \rangle$ е различна редица от същите числа. Нека b_1 е най-малката стойност измежду числата, нека b_2 е втората най-малка, и така нататък. Тогава съществува наредена двойка $(k, k+1)$ за някое $k \in \{1, \ldots, n-1\}$, такава че, ако a_i е елементът със стойност b_k и a_j е елементът със стойност b_{k+1} (което влече $a_i < a_j$), то $a'_i > a'_j$.

Доказателство: Твърди се, че има съседни **по големина** елементи в $\mathbf{a}$—забележете, не на съседни позиции в редицата $\mathbf{a}$, а съседни **по големина** измежду всички числа от $\mathbf{a}$—да речем a_i и a_j, като $a_i < a_j$, такива че в $\mathbf{a}'$ на същите тези позиции i и j има такива елементи, че този на позиция i е по-голям от този на позиция j.

Ето пример. Нека $n = 8$. Нека

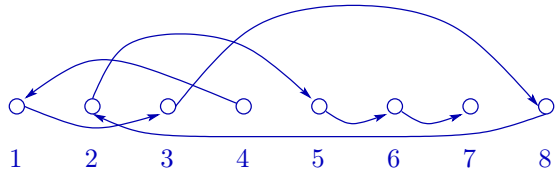
$$\mathbf{a} = \langle 4, 11, 5, 2, 14, 18, 22, 7 \rangle \tag{7.7}$$
$$\mathbf{a}' = \langle 7, 4, 11, 5, 22, 18, 2, 14 \rangle \tag{7.8}$$

Има ли в $\mathbf{a}$ двойка елементи a_i и a_j с желаните свойства? Да, разбира се, например $7 = a_8$ и $11 = a_2$. Съответните им големини са 4 и 5, защото 7 е четвъртият по големина в $\mathbf{a}$, а 11 е петият. Очевидно $a_8 < a_2$. Забележете, че $a'_8 = 14$ и $a'_2 = 4$; ерго, в $\mathbf{a}'$ отношението между осмият и вторият елемент е обратното, а именно $a'_8 > a'_2$.

Сега ще извършим доказателството чрез допускане на противното. Противното е: за всеки две позиции в $\mathbf{a}$, съдържащи елементи със съседни по големина стойности, в $\mathbf{a}'$ посоката на неравенството между елементите на тези позиции е същата. Но тогава следва, че $\mathbf{a} = \mathbf{a}'$. Защо? Всяка редица от n различни по двойки числа може да се опише еднозначно чрез:

- един ориентиран граф-път, чиито върхове са позициите и който има ребро от позиция i към позиция j тогава и само тогава, когато елементът на позиция i е непосредствен предшественик по големина на елемента на позиция j
- и множеството от самите числа.

Примерно, въпросният граф на редицата $\langle 4, 11, 5, 2, 14, 18, 22, 7\rangle$ е



Ако допуснем, че твърдението, което доказваме, не е истина, директно извеждаме, че две различни редици от едни и същи числа имат един и същи граф-път. Но тогава те са една и съща редица. □

Заслужава си да се направи едно пояснение върху Лема 45. Щом две пермутации се различават, те се различават в поне две позиции. Тоест, ако пермутациите са $\mathbf{a} = \langle a_1, a_2, \ldots, a_n\rangle$ и $\mathbf{a}' = \langle a'_1, a'_2, \ldots, a'_n\rangle$, то има някакви i и j, такива че $1 \leqslant i < j \leqslant n$ и $a_i \neq a'_i$ и $a_j \neq a'_j$. Иначе казано, различни пермутации на едни и същи елементи не може да се различават само в една позиция – това е очевидно. И така, разглеждаме пермутации, които се различават минимално, а именно в точно две позиции i и j. Лема 45 **не твърди**, че числата a_i и a_j в едната пермутация са непременно съседните по големина числа b_k и b_{k+1}! Да разгледаме пример. Пермутациите от (7.7) и (7.8) не са удачен пример, защото се различават в доста позиции. Да разгледаме

$$\mathbf{a} = \langle 4, 11, 5, 2, 14, 18, 22, 7\rangle$$
$$\mathbf{a}'' = \langle 4, 18, 5, 2, 14, 11, 22, 7\rangle$$

Наистина, $\mathbf{a}$ и $\mathbf{a}''$ се различават само в две позиции: числата 11 и 18 са разменени, останалите са на същите позиции. Числата 11 и 18 обаче не са съседни по големина, защото между тях по големина е 14. Така че съседните по големина числа, за които Лема 45 говори, са други, например 14 и 18. Числата 14 и 18 са съседни по големина. В $\mathbf{a}$ те са на позиции 5 и 6, като $a_5 < a_6$. И действително в $\mathbf{a}''$ на тези позиции има числа, за които неравенството е в обратната посока; тези числа са 14 и 11, като $a''_5 > a''_6$.

Лема 44 и Лема 45 вече са доказани. Сега ще докажем теоремата. Да допуснем, че дървото на вземане на решения T на алгоритъм за уникалност на елементи X (базиран на директни сравнения) асоциира някое листо u с поне две пермутации. Съгласно Лема 45, съществуват съседни по големина елементи в първата пермутация, такива че на техните съответни позиции i и j във втората пермутация има елементи, неравенството между които е в обратната посока (спрямо неравенството между елементите на позиции i и j в първата).

Да обясним това по-подробно. Нека числата във входа са a_1, a_2, …, a_n. Теоремата твърди, че всяко листо е асоциирано с не повече от една тяхна пермутация. Допускаме противното: съществува листо u и поне две различни пермутации на входните числа, да кажем $\mathbf{a}' = \langle a'_1, a'_2, \ldots, a'_n\rangle$ и $\mathbf{a}'' = \langle a''_1, a''_2, \ldots, a''_n\rangle$, които са асоциирани с u. Това означава, че и $\mathbf{a}'$, и $\mathbf{a}''$ са консистентни с въпросите и отговорите по пътя от корена до u.

Нека b_1, b_2, …, b_n са стойностите от входа в нарастваща големина $b_1 < b_2 < \cdots < b_n$. Лема 45 казва, че съществуват b_k и b_{k+1}, такива че, ако те са на позиции съответно i и j в

$\mathbf{a}'$ (което влече $a'_i < a'_j$), то в $\mathbf{a}''$, елементите на позиции i и j (съответно a''_i и a''_j) са такива, че $a''_i > a''_j$.

Сега си припомняме Лема 44. Тя казва, че X трябва да направи сравнението $\boxed{b_k \overset{?}{<} b_{k+1}}$. Само че X **не задава въпроси за съседни стойности b_k и b_{k+1}!**[†] X **задава въпроси от вида $\boxed{a_p \overset{?}{<} a_q}$, а Лема 44 гарантира, че рано или късно, a_i и a_j, които имат стойности съответно b_k и b_{k+1}, се оказват "на везната".**

Ключовото наблюдение е следното. По отношение на въпроса $\boxed{a_i \overset{?}{<} a_j}$, $\mathbf{a}'$ е консистентна с Да отговора, докато $\mathbf{a}''$ е консистентна с Не отговора. Тогава за вътрешния връх v, в който се задава въпроса $\boxed{a_i \overset{?}{<} a_j}$, едното дете на v, да го наречем x, е асоциирано с $\mathbf{a}'$, а другото дете, да го наречем y, е асоциирано с $\mathbf{a}''$. Очевидно u не може да е връх (листо) и в $T[\![x]\!]$, и в $T[\![y]\!]$. □

След като сме доказали Теорема 47, долната граница $\Omega(n \lg n)$ за задачата Уникалност на елементи, базирана на директни сравнения, следва от същите съображения като тази на Сортиране.

7.2.4 Долна граница $\Omega(\lg n)$ за Търсене

Вече разгледахме алгоритми за двоично търсене на стр. 124, които очевидно имат сложност по време $\Theta(\lg n)$ в най-лошия случай.

Сега твърдим, че $\Omega(\lg n)$ е асимптотична долна граница за Търсене. Забележете, че тук не говорим за двоично търсене, а за **всякакво** търсене! Ако търсенето е базирано на директни сравнения, всеки алгоритъм работи във време $\Omega(\lg n)$. Причината е много проста: има $\Theta(n)$ различни състояния по отношение на даден вход. Нека входът е масив $A[1, \ldots, n]$ (сортиран или не, няма значение) и търсената стойност е key. Да допуснем, че елементите от A са два по два различни. Това не е ограничение на общността, понеже, ако получим долна граница при това допускане, в общия случай долната граница не може да е по-ниска.

Всеки алгоритъм за Търсене, базиран на директни сравнения, трябва да може да различава (поне) следните $n + 1$ състояния:

състояние 1**:** key $= A[1]$,

състояние 2**:** key $= A[2]$,

...

състояние n**:** key $= A[n]$,

състояние $n + 1$**:** key $\notin A$.

в смисъл, че това са съществено различни ситуации и дървото на вземането на решение трябва да е такова, че всяко листо да е асоциирано с не повече от едно от тези състояния. В началото на алгоритъма всички тези състояния (ситуации, ако предпочитате) са възможни, а след всяко извършване на сравнение между key и елемент на масива, някои от тях стават невъзможни[‡]. Ако в края на алгоритъма останат две или повече възможни състояния, то

[†] Ако знаем, че два елемента са b_k и b_{k+1}, то е напълно безсмислено да ги сравняваме. Отговорът на въпроса $b_k \overset{?}{<} b_{k+1}$ е предварително известен.

[‡] Може дори да няма такива—които стават невъзможни след някое сравнение—ако сравнението е безсмислено. Това може да се случи, ако алгоритъмът е глупав, но с нищо не променя валидността на аргументацията.

алгоритъмът "не си е свършил работата" и отговорът му, какъвто и да е, е невалиден. Ерго, всеки коректен алгоритъм има такова дърво, че всяко листо е асоциирано с най-много едно от тези състояния.

Но това са $\Theta(n)$ състояния, което дава долна граница $\Omega(n)$ за броя на листата, което дава долна граница $\Omega(\lg n)$ за височината, което дава долна граница $\Omega(\lg n)$ за сложността по време на алгоритъма.

7.2.5 Лампи и ключове

Преди повече от петнадесет години авторът на тези лекционни записки чу следната задача.

> В една стая има три ключа за лампи. Лампите се намират в друга стая, като между двете стаи няма видимост. Всеки ключ пали/гаси точно една от лампите. Целта е да разберем кой ключ за коя лампа е чрез само един експеримент.

"Да направим експеримент" в този контекст означава да сложим всеки от ключовете в някакво положение и след това да отидем в стаята с лампите и да извършим наблюдение – да видим коя лампа е запалена и коя е загасена.

Първоначално авторът изпадна в когнитивен дисонанс, защото за него беше очевидно, че за повече от две лампи не е достатъчен един експеримент. С един експеримент може да определим кой ключ на коя лампа отговаря само при два ключа и две лампи. При повече ключове и лампи са необходими поне два експеримента. А задачата иска решение със само един експеримент!

Естествено, задачата мълчаливо допуска, че ключовете са *позиционни*. Това означава, че всеки ключ има две външно различими положения и ние знаем кое положение затваря електрическа верига и кое положение отваря електрическа верига. Повечето ключове за осветление са точно такива, като при правилно монтиране положението *надолу* прекъсва верига (тоест, гаси лампата), а положението *нагоре* затваря верига (тоест, пали лампата)[†]. Но има ключове, които нямат две видимо различни положения – при такъв ключ няма как да знаем дали с едно "цъкване" палим или гасим[‡].

Ако ключовете не са позиционни, очевидно няма решение дори при само два ключа и две лампи – каквото и да направим с ключовете, ако при наблюдението се окаже, че нито една от лампите не свети, или и двете светят, няма как да разберем кой ключ на коя лампа съответства без допълнителни експерименти. И така, допускаме, че ключовете са позиционни.

Но дори при позиционни ключове не изглежда да има решение с един експеримент за три лампи и три ключа. Има четири възможности за лампите: и трите светят, две светят, една свети и нито една не свети. Няма смисъл да поставяме ключовете в положения, при които нито една лампа не свети, защото тогава не ни трябва да правим наблюдение – ние знаем какво ще видим, а именно, че нито една от лампите не свети. Аналогично, няма смисъл да палим и трите лампи. Остава да запалим една лампа или две лампи. Но и в двата случая при последващото наблюдение няма да решим задачата, въпреки че ще получим някаква информация. Примерно, ако пуснем точно една лампа с ключ едно, при наблюдението ще разберем ключ едно на коя лампа съответства, но ни трябва още един експеримент, за да разберем кой от ключове две и три, на коя от останалите две лампи съответства.

[†]Тази конвенция идва от шалтерите в електротехниката. Тъй като гравитацията действа винаги, шалтерът може да падне сам, но не може да се вдигне сам. Много по-добре е непревидено падане на шалтера да прекъсне електрическата верига, което може да остави някакви хора временно без ток, отколкото да затвори веригата и да убие човек, който в момента работи по нея и разчита, че шалтерът я е прекъснал.

[‡]Някои фарове на автомобили са с такива ключове.

"Решението" на задачата се оказа дразнещо глупаво: с точно един от ключовете палим лампа (не знаем коя, понеже не сме извършили наблюдение, но знаем, че я палим, понеже ключовете са позиционни), държим е запалена известно време, след което я гасим, палим друга лампа с някой от другите два ключа и бързо отиваме в стаята с лампите. Точно една от лампите свети—това е тази, която пуснахме с второто пускане на ключа—и точно една от несветещите лампи е гореща заради това, че с първото пускане на ключ сме я били включили; по този начин разбираме и първият ключ, с който запалихме лампа (която после угасихме), на коя лампа съответства – на горещата измежду двете несветещи. Решението е глупаво, понеже "чупи" модела, който мислено сме изградили, а именно, че разрешените действия са манипулиране на ключовете и наблюдения на лампите. Има много сценарии, в които това "решение" не работи: може лампите са на десетметров таван и не можем да ги достигнем с ръка, може лампите да светят със студена светлина и да не се нагряват осезаемо при работа, може лампите да са в гореща среда, примерно гореща фурна, в която всички лампи са горещи, може коридорът между двете стаи да е толкова дълъг, че лампата да изстине, докато стигнем до нея, и така нататък.

Допълнение 36: Thinking outside the box – добро или лошо?

Посоченото решение на задачата с трите ключа и лампи, което си позволих да нарека "глупаво", е типично нестандартно решение, за чието измисляне е необходимо да "излезем от рамката", което на английски се нарича *thinking outside the box* или *thinking out of the box*. Това понятие, поначало смислено, се е превърнало в досадно клише, като битува разбиране, че излизането от рамката е непременно нещо добро и положително, така че трябва да излизаме от рамката колкото е възможно по-често, едва ли не постоянно, за да сме оригинални, креативни и неповторими.
Всъщност, излизането от рамката може да е положително, а може и да не е. **Всяка** словесна задача без изключение предполага някакъв модел, като решението на задачата е съгласно конвенциите и ограниченията на модела; решението е **в рамката на модела**, а не извън нея. По правило този модел е имплицитен и се подразбира както от автора на задачата, така и от решаващия задачата. Примерно, в основното училище решавахме задачи от типа на

> *"Влак тръгва в 10:00 ч. от град А за град Б с 80 км/ч. В 11:00 ч. от Б за А тръгва друг влак с 60 км/ч. Намерете разстоянието между А и Б, ако влаковете се срещат в 12:00 ч.*

Решението 220 км. се намира наум. Обаче, за да намерим това решение, ние се "облегнахме" на допускания, които не са споменати изрично:

- всеки от влаковете развива съответната скорост мигновено, за нула време, което в реалния свят е невъзможно;
- влаковете не спират на междинни гари;
- веднъж достигнал съответната скорост, всеки влак се движи абсолютно равномерно, което, ако говорим педантично, е невъзможно за реални влакове.

Решението 220 км. е някаква идеализация, апроксимация на някаква реална ситуация. А какво бихте казали за такова "решение":

> *Влаковете всъщност не са срещнали, защото мотрисата на първия влак се е запалила и пътниците са били прехвърлени в автобуси, а вторият влак е заседнал на междинна гара поради кражби на семафори.*

Това "решение" вече излиза напълно извън рамката, която се подразбира за задачата, въпреки че съвсем не е изключено в живота да стане именно това.
Такива драстични излизания от рамката на модела обезсмислят всяка словесна задача и изобщо всяка задача, чието решение изисква спазване/съблюдаване на правила. Като друг пример, шахматните задачи от типа "Намерете мат в четири хода за белите" се обезсмислят от решения, при които играчът с черните фигури прибягва към насилие, виждайки, че губи. Не че е невъзможно на практика да стане точно това и със сигурност историята познава такива случаи, но задачата е дадена при допускането, че се играе по правилата на шаха и се признават само решения по правилата на шаха. Аналогично, недопустимо е да се предлага решение на задачата за влаковете, което излиза от рамките на разумния имплицитен модел.

Трябва ясно да различаваме решаването на теоретична задача съгласно някакви правила, в рамките на даден модел, от решаването на житейска задача, в която наистина трябва да мислим "извън кутията", защото правила няма. Последното важи особено силно за компютърната сигурност. Често теорията разглежда модел, в който комуникират две страни, да кажем А и Б, по информационен канал, който е общодостъпен и който се следи от подслушвач. Подслушвачът следи пасивно какво минава по канала или активно прехваща връзката, подменяйки информация. Много от задачите в областта на компютърната сигурност са в рамките на този модел и решенията се основават на неговите ограничения. В реалния свят обаче нещата са много, много по-сложни и действията на противника може да не се ограничат до подслушване или прехващане на комуникацията! Ако информацията е от огромно значение и противникът е мощен и мотивиран да я получи на всяка цена, доста сигурно е, че той няма да спазва ограниченията на модела, а ще прибегне до подкупване, изнудване, проникване с взлом, откровено насилие и така нататък. Потърсете с гуугъл за *rubber hose crypto* за повече информация. В по-кротки и по-малко драматични сценарии, излизането извън модела по отношение на сигурността става не чрез прибягване към насилие, а чрез използване на, примерно, *backdoors* в хардуера.

Потенциалната възможност да се излезе от рамката на модела, който допускаме, и да започне да се действа по неочакван начин, колкото и да е житейски важна, не обезсмисля задачите, които са поставени в рамката на даден модел и чието решение трябва да е съобразено с модела. Умението да решаваме задачи в рамките на моделите е много важно и ценно:

- първо, защото е възможно житейската ситуация да се развие по, или близо до, модела; в примера с комуникацията, възможността противникът да прибегне към насилие е по-скоро изключение, а backdoors по правило се използват от малко на брой, супермощни агенти, срещу които нямаме шанс така или иначе,
- второ, защото решенията в рамката на модела показват аналитични способности, за разлика от много от "решенията", чупещи модела.

Да се върнем към задачата за ключовете и лампите. Моделът, който е нормално да допуснем, е следният: имаме право само да нагласим ключовете в някакви положения,

после да извършим наблюдение, да правим изводи от наблюдението, въз основа на тях да променим някои от ключовете, пак да правим наблюдение, и така нататък, докато не разберем кой ключ коя лампа пуска/спира. Решението с пипане на несветещите лампи, за да се разбере коя от тях е била пусната допреди малко, излиза напълно извън този модел; иначе казано, чупи модела. В някои житейски ситуации то може да е полезно, в други да е неприложимо (лампите са на висок таван, примерно), но по-важното е, че чупи модела. Ако започнем да чупим модела, може да стигнем до екстремни "решения", в които разбираме кой ключ към коя лампа е изобщо без наблюдения. Възможностите за това са неограничени:

- изкъртваме с кирка мазилката, за да проследим кабелите в стената,
- питаме електротехника (или който и да е, който знае свързването),
- подкупваме електротехника,
- изнудваме електротехника,
- намираме планове на сградата,
- и така нататък.

Започнем ли веднъж да чупим модела, спиране няма.

За да избегнем чупене на модела, може да направим условието огромно, с много ограничения, изредени експлицитно, надявайки се така да предотвратим екстравагантни решения и оставяйки като възможно само решението, което имаме предвид. Но ако решаващият задачата е решил сериозно да чупи модела (или просто да се лигави), многото ограничения в условието няма да го спрат. Колкото и прецизно и подробно да е условието на словесна задача, винаги може да се намери интерпретация, която е малко или много различна от тази, която е имал предвид този, който е измислил задачата и която интерпретация води до решение, съвсем различно от оригиналното.

В резюме, всяка словесна задача изисква изготвяне на модел и решение в рамката на този модел. При достатъчно "добро желание" винаги може да се намери интерпретация, при която се изготвя различен модел и решението в неговата рамка има малко общо с решението на този, който дава задачата. За задачите, които не изискват правене на модел, това не е в сила. Например, задачата да се реши дадено рекурентно уравнение не изисква правене на модел и при нея този, който решава, няма възможност да шикалкави: или решава уравнението, или не го решава. За разлика от това, ако е дадена словесна задача, която трябва да се моделира с рекурентно уравнение, има неограничени възможности за шикалкавене при достатъчно "добро желание".

И най-важното. За **житейски** задачи мисленето "извън кутията", тоест, чупенето на общоприетия модел, може да е изключително полезно. Но от това не следва, че решаването на **теоретични** задачи при стриктно съблюдаване на рамката на модела е безполезно или второкачествено, нито че чупенето на модела при теоретичните задачи е смислено.

И така, задачата няма решение с един експеримент при три ключа и лампи, ако съблюдаваме ограниченията на разумния модел.

Нека n е броят на ключовете/лампите. При $n = 3$ и $n = 4$ има решение с два експеримента. При $n = 5$ не изглежда да има решение с два експеримента, но има решение с три експеримента. И така нататък. Изглежда, че в общия случай има решение с $\lceil \log_2 n \rceil$ експеримента, но не с по-малко от това. Решението е много просто и ще го опишем с думи. Нека $n = 2^m$, за някое $m \in \mathbb{N}$. Ако $m = 0$, има един ключ и една лампа и този ключ пали тази лампа, което заключаваме без да правим експерименти; тоест, нула експерименти. Ако $m > 1$, нека множеството от ключовете е $K = \{k_1, \ldots, k_n\}$, а множеството от лампите е $L = \{\ell_1, , \ldots, \ell_n\}$. Разбиваме K на две равномощни множества, да кажем $K_1 = \{k_1, \ldots, k_{n/2}\}$ и $K_2 = \{k_{(n/2)+1}, \ldots, k_n\}$. Поставяме ключовете от K_1 в положение "запалено", поставяме ключовете от K_2 в положение "загасено", отиваме в стаята с лампите и наблюдаваме. Точно $n/2$ лампи светят. Нека множеството от светещите лампи е L_1, а от останалите лампи е L_2. Сега вече знаем, че ключовете от K_1 съответстват на лампите от L_1, а тези от K_2, на лампите от L_2. Решаваме задачата рекурсивно върху (K_1, L_1) и (K_2, L_2). Това решение по същество е по схемата разделяй-и-владей, само че, ако си го представяме като алгоритъм, той е паралелен алгоритъм, защото едновременно решаваме за (K_1, L_1) и (K_2, L_2).

В тази лекция обаче ние се интересуваме от долни граници, така че за нас е по-важен въпросът, защо задачата не може да се реши с по-малко от логаритмичен брой експерименти. Ще докажем асимптотична логаритмична долна граница за броя на експериментите. Ще използваме дървета за вземане на решения.

Лесно се вижда, че това, което се иска в задачата, е да се намери точно една пермутация измежду $n!$ пермутации: при фиксирано изброяване на ключовете k_1, $\ldots$, k_n, това, което се иска, е пермутация $\pi : \{1, \ldots, n\} \to \{1, \ldots, n\}$, такава че k_i отговаря на лампа $\ell_{\pi(i)}$, за $1 \leqslant i \leqslant n$. На всяка схема, която е серия от "разцъкване" на лампи и последващо наблюдение, съответства дърво на вземане на решение. Листата на дървото са поне $n!$, защото то изчислява една пермутация измежду $n!$. Да видим каква е разклонеността му. За разлика от дърветата за вземане на решение, които разгледахме досега, това дърво има разклоненост, която не е константа, а е функция на n. Разклонеността съответства на броя на различните резултати, които можем да получим от едно наблюдение. Колкото по-бързо растяща е тази функция, толкова по-малка е височината на дървото и съответно долната граница за задачата е по-ниска.

При n различни лампи, възможните резултати от наблюденията са не повече от 2^n, понеже всяка лампа може да свети или да не свети. Следователно, разклонеността на дървото е най-много 2^n. Ако $h(n)$ е височината на дървото, то

$$h(n) \geqslant \log_{2^n} n!$$

Това не е грешка – основата на логаритъма е 2^n. В сила е

$$\log_{2^n} n! = \frac{\log_2 n!}{\log_2 2^n} = \frac{\Theta(n \lg n)}{n} = \Theta(\lg n)$$

Показахме, че $h(n) = \Omega(\lg n)$. Следователно, броят на експериментите е поне логаритмичен, в асимптотичния смисъл.

7.3 Редукции

7.3.1 Долна граница $\Omega(n \lg n)$ за Най-близка двойка елементи

Изч. Задача: Closest Pair

пример: Числа $a_1, a_2, \ldots, a_n$.
решение: Минимално $|a_i - a_j|$ за $1 \leqslant i < j \leqslant n$.

Това е едномерният вариант на задачата: входът се състои от числа (а не от наредени двойки числа, примерно). Без съмнение съществува директно доказателство на долната граница $\Omega(n \lg n)$ за тази задача с дърво на вземане на решение. Но, както видяхме при задачата Уникалност на Елементи, директното доказателство може да е дълго и тежко. Тук ще извършим *редукция*. А именно, ще редуцираме задачата Уникалност на Елементи до задачата Най-близка двойка елементи. По този начин ще използваме наготово резултата за долна граница на Уникалност на елементи. Редукцията става по следния начин.

Нека AlgCP е произволен алгоритъм за задачата Най-близка двойка елементи (базиран на директни сравнения, естествено). Да разгледаме следния алгоритъм AlgX.

```
AlgX(Числа a_1, a_2, ..., a_n)
1  d ← AlgCP(a_1, a_2, ..., a_n)
2  if d = 0
3      return False
4  else
5      return True
```

Очевидно AlgX решава задачата Уникалност на Елементи. Нещо повече, неговата сложност по време е равна, в асимптотичния смисъл, на сложността на AlgCP, защото извън извикването на AlgCP на ред 1 и последващото връщане, AlgX извършва само константна допълнителна работа (редове 2–5). Да допуснем, че не е вярно, че Най-близка двойка елементи има долна граница $\Omega(n \lg n)$. Тогава за Най-близка двойка елементи има поне един алгоритъм алгоритъм, за който $\Omega(n \lg n)$ не е долна граница. Тогава[†] за този алгоритъм $o(n \lg n)$ е горна граница. Ключовото наблюдение е, че ако AlgCP имаше сложност по време $o(n \lg n)$, то и AlgX щеше има сложност по време $o(n \lg n)$, а оттук и задачата Уникалност на Елементи щеше да има сложност $o(n \lg n)$. Само че ние вече знаем, че Уникалност на Елементи има долна граница $\Omega(n \lg n)$. Следователно, невъзможно е AlgCP да работи във време $o(n \lg n)$. Оттук имаме и долна граница $\Omega(n \lg n)$ за задачата Най-близка двойка елементи.

Допълнение 37: $f(n) \neq \Omega(g(n))$ не влече $f(n) = o(g(n))$

Какво следва от факта, че $f(n) \neq \Omega(g(n))$? С други думи, какво влече $f(n) \not\geq g(n)$? Читателят може да се изкуши да помисли, че тогава непременно $f(n) \prec g(n)$, по аналогия с импликацията $x \not\geqslant y \to x < y$ върху реалните числа. Само че $\succeq$ и $\prec$ са релации върху функции и при тях нещата са по-сложни. Импликация $f(n) \neq \Omega(g(n)) \to f(n) \not\prec g(n)$

[†]Стриктно говорейки, такава импликация няма: от това, че $\Omega(f(n))$ не е долна граница за сложността на някакъв алгоритъм **не следва**, че $o(f(n))$ е горна граница за него. Виж Допълнение 37. Тук прилагаме тази импликация, понеже разглеждаме "нормално държащи се" функции като $n \lg n$.

няма.
Съгласно Теорема 12, има точно два случая, в $f(n) \not\succeq g(n)$ и $f(n) \prec g(n)$:

1. (2.38), тоест функциите са асимптотично несравними;
2. (2.39), тоест $f(n) \preceq g(n)$ и само това.

Тези случаи обаче са екзотика, що се отнася до функции, които описват алгоритмична сложност. За да бъдат функциите асимптотично несравними или едната да изостава от, и после да догонва, другата, и така до безкрай, едната или двете функции трябва да имат "подскачащ характер". Обикновено срещащите се в практиката функции, описващи алгоритмична сложност, като n, $n \lg n$, n^2, 2^n и така нататък не са такива. И така, за нормално държащите се, срещащи се на практика функции е вярно, че $f(n) \not\succeq g(n)$ влече $f(n) \prec g(n)$.

Казваме, че задачата Уникалност на Елементи *се редуцира* до задачата Най-близка двойка елементи в смисъл, че задачата Уникалност на Елементи има решение, което се състои в решение на задачата Най-близка двойка елементи, плюс пренебрежимо малко допълнителна работа. Това е доста неформално казано, но засега ще се задоволим с неформално разбиране за "редукция".

Грубо казано, от факта, че Π_1 се свежда до Π_2 правим два извода:

- Ако Π_1 е трудна задача (каквото и да значи това), то Π_2 също е трудна. Няма как някаква вътрешна, иманентна сложност да изчезне при редукцията.
- Ако Π_2 е лесна (каквото и точно да значи това), то Π_1 също е лесна. По същата причина: не може иманентната сложност да изчезне при редукцията.

Важно е да знаем, че следните изводи **не са верни**.

- Ако Π_1 е лека, то Π_2 също е лека. **Такъв извод няма**. Винаги можем да направим нещата произволно сложни.
- Ако Π_2 е трудна, то Π_1 също е трудна. **Такъв извод няма**. Причината отново е, че винаги можем да направим нещата сложни.

Наблюдение 33

Трансформирайки задача в задача, можем да добавим колкото си искаме допълнителна сложност отгоре. Това, което е невъзможно, е при такива трансформации да намалим иманентната сложност.

7.3.2 Долна граница $\Omega(n \lg n)$ за Мода

Да си припомним определението на "мода" от Подсекция 3.2.4. Формално, изчислителната задача Мода е следната.

Изч. Задача: Mode

пример: Числа $a_1, a_2, \ldots, a_n$.
решение: Най-често срещано число измежду дадените.

Ще докажем долна граница $\Omega(n \lg n)$ за Мода. Естествено, имаме предвид намиране на мода чрез директни сравнения. Нека AlgMode е произволен алгоритъм за задачата Мода, базиран на директни сравнения. Той връща стойността на някоя мода. Да разгледаме следния алгоритъм AlgY.

```
AlgY(Числа a_1, a_2, ..., a_n)
 1  m ← AlgMode(a_1, a_2, ..., a_n)
 2  c ← 0
 3  for i ← 1 to n
 4      if a_i = m
 5          c ← c + 1
 6  if c = 1
 7      return True
 8  else
 9      return False
```

Очевидно AlgY решава задачата Уникалност на Елементи. Нещо повече, неговата сложност по време е равна, в асимптотичния смисъл, на сложността на AlgMode, защото извън извикването на AlgMode на ред 1 и последващото връщане, AlgX извършва само $\Theta(n)$ допълнителна работа (редове 2–9). Ако алгоритъмът AlgMode имаше сложност по време $o(n \lg n)$[†], то и AlgY щеше има сложност по време $o(n \lg n)$, а оттук и задачата Уникалност на Елементи щеше да има сложност $o(n \lg n)$. Ние вече знаем, че Уникалност на Елементи има долна граница $\Omega(n \lg n)$. Следователно, невъзможно е AlgMode да работи във време $o(n \lg n)$. Оттук имаме и долна граница $\Omega(n \lg n)$ за задачата Мода.

7.3.3 Долна граница $\Omega(n \lg n)$ за 2SUM във версия за броене

Ето версията на 2SUM, която имаме предвид тук.

Изч. Задача: 2SUM, версия за броене

пример: Масив от положителни числа $A[1, \ldots, n]$.
решение: Броят на наредените двойки (i, j), такива че $1 \leqslant i < j \leqslant n$ и $A[i] + A[j] = 0$.

Нека Alg2SUMCount е произволен алгоритъм за задачата 2SUM във версия за броене, базиран на директни сравнения. Да разгледаме следния алгоритъм AlgZ.

```
AlgZ(A[1, ..., n]: масив от положителни числа)
 1  B = [−a_1, −a_2, ..., −a_n, a_1, a_2, ..., a_n]
 2  p ← Alg2SUMCount(B)
 3  if p = n
 4      return Да
 5  else
 6      return Не
```

[†]Съображенията на Допълнение 37 са в сила.

Ще покажем, че AlgZ решава задачата Уникалност на Елементи. Първо да допуснем, че в $A[1,\ldots,n]$ няма повторения. Тогава единствените двойки различни елементи от B, които се сумират до 0, са двойките $(-A[i], A[i])$, за $1 \leqslant i \leqslant n$. Тогава очевидна AlgZ връща Да.

Сега да допуснем, че има поне една наредена двойка (i,j), като $i < j$ и $A[i] = A[j]$. Тогава броят на двойките различни елементи от B, които се сумират до 0, е по-голям от n. Тогава очевидна AlgZ връща Не.

И така, AlgZ решава задачата Уникалност на Елементи. Но конструирането на масива B става в линейно време и неговият размер е $\Theta(n)$. Ако съществува алгоритъм за 2SUM във версия за броене, базиран на директни сравнения и работещ във време $o(n \lg n)$, то AlgZ е алгоритъм за Уникалност на Елементи, базиран на директни сравнения и със сложност $o(n \lg n)$. Ние вече знаем, че Уникалност на Елементи има долна граница $\Omega(n \lg n)$. Следователно, невъзможно е AlgZ да работи във време $o(n \lg n)$.

7.3.4 Относителна долна граница: Degeneracy Testing е 3SUM-hard

В тази подсекция ще видим пример за *условна*, или *относителна*, долна граница на задача. Задачата е Degeneracy Testing (вижте надолу). Долната граница за нея е същата като на 3SUM. Тук говорим за 3SUM във версия за разпознаване.

Дълго време се е смятало, че 3SUM има долна граница $\Omega(n^2)$. Това е доказано през 1999 г. от Jeff Erickson [24], но за доста слаб изчислителен модел. През 21 век са открити няколко алгоритъма за 3SUM със сложност $o(n^2)$ (в нормалния изчислителен модел), най-бързият от които е със сложност $O\left(n^2 \cdot \frac{\lg\lg n}{\lg n}\right)$ [32]. Текущата хипотеза (недоказана обаче!) е, че няма алгоритъм за 3SUM със сложност $O(n^{2-\epsilon})$ за никоя положителна константа ϵ в нормалния изчислителен модел.

Каква е истината за долната граница на 3SUM, ние не знаем. Но ще видим една лесна редукция на 3SUM до Degeneracy Testing. От съществуването на тази редукция следва, че всяка долна граница за 3SUM е и долна граница за Degeneracy Testing, стига да е по-висока, в асимптотичния смисъл, от $n^{2-\epsilon}$ за всяка положителна константа ϵ .

Една полезна дефиниция, която конкретизира какво разбираме под "редукция".

Определение 48: 3SUM-hard

Нека Π е изчислителна задача за разпознаване. Казваме, че Π е *3SUM-hard*, ако произволен пример с размер n на 3SUM може да бъде решен чрез

1. решаване на Π върху константен брой примери (на Π), всеки с размер $O(n)$,
2. и още $O(n^c)$ допълнителна работа, където c е константа, такава че $0 < c < 2$.

Сравнете тази дефиниция с дефиницията на "**NP**-hard" от Лекция 14. Интуитивно, "Π е 3SUM-hard" казва "Π е поне толкова трудна, колкото 3SUM".

Забележете, че ако Π е 3SUM-hard, то 3SUM се свежда до Π, но "Π е 3SUM-hard" казва нещо доста конкретно и добре дефинирано, предвид Определение 48, за разлика от неформалното и не особено конкретно "3SUM се свежда до Π".

Изч. Задача: Degeneracy Testing

пример: $p_1, p_2, \ldots, p_n$: точки в равнината
въпрос: Дали поне три от точките са колинеарни?

Ще докажем, че Degeneracy Testing е 3SUM-hard. Първо един помощен факт.

Лема 46

За всеки три реални числа a, b и c, две по две различни, е вярно, че точките (a, a^3), (b, b^3) и (c, c^3) са колинеарни тстк $a + b + c = 0$.

Доказателство: В едната посока, нека точките (a, a^3), (b, b^3) и (c, c^3) са колинеарни. Тогава те лежат върху права с уравнение $y = \alpha x + \beta$. Тогава и a, и b, и c са корени на $x^3 = \alpha x + \beta$. Но това е кубично уравнение, така че трябва да е вярно

$$x^3 - \alpha x - \beta = (x - a)(x - b)(x - c)$$

Но коефициентът пред x^2 вляво е нула. Следователно, вдясно коефициентът пред x^2 също е нула. Но вдясно коефициентът пред x^2 е $-(a + b + c)$. Тогава $a + b + c = 0$.

В обратната посока, нека $a+b+c = 0$. Да разгледаме $(x-a)(x-b)(x-c)$. Отваряйки скобите, ще получим $x^3 - \alpha x - \beta$ за някакви реални α и β, тъй като коефициентът пред x^2 е нула, което на свой ред се дължи на факта, че $a + b + c = 0$.

Тогава a, b и c са корените на уравнението $x^3 - \alpha x - \beta = 0$. Но тогава точките (a, a^3), (b, b^3) и (c, c^3) са колинеарни, понеже лежат върху правата $y = \alpha x + \beta$. □

Теорема 48: Задачата Degeneracy Testing е 3SUM-hard

Degeneracy Testing е 3SUM-hard.

Доказателство: Нека е даден пример на 3SUM: множество от реални числа $\{s_1, \ldots, s_n\}$. От него конструираме пример $\{(s_1, s_1^3), \ldots, (s_n, s_n^3)\}$ на Degeneracy Testing. Съгласно Лема 46, отговорът на въпроса "Дали има поне три точки от $\{(s_1, s_1^3), \ldots, (s_n, s_n^3)\}$, които са колинеарни?" е същият като отговорът на въпроса "Дали има три елемента на $\{s_1, \ldots, s_n\}$, които се сумират до нула?". Конструкцията работи достатъчно бързо според изискването на Определение 48, а именно във време $O(n^c)$ за $c = 1$. Броят на примери на Degeneracy Testing, които конструираме, е само един, така че изпълняваме и изискването решаването да става чрез константен брой примери на задачата, към която свеждаме. □

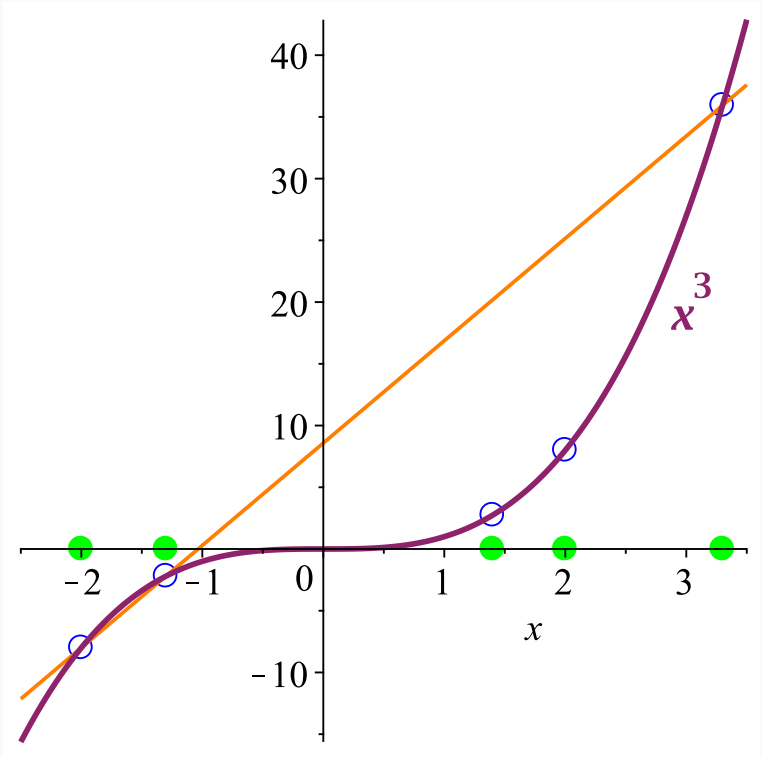
Фигура 7.5[†] илюстрира факта, че Degeneracy Testing е 3SUM-hard. Примерът на 3SUM е $\{-2, -1.3, 1.4, 2, 3.3\}$. Нему съответстват точките в зелено върху абсцисата. Той е Да-пример, понеже $-2 + (-1.3) + 3.3 = 0$. От този пример на 3SUM строим следния пример на Degeneracy Testing:

$$\{(-2, -8), (-1.3, -2.197), (1.4, 2.744), (2, 8), (3.3, 35.937)\}$$

Нему съответстват точките, нарисувани със сини кръгчета и лежащи върху тъмно лилавата крива $y = x^3$. Той е Да-пример на Degeneracy Testing, понеже трите точки $(-2, -8)$, $(-1.3, -2.197)$ и $(3.3, 35.937)$ са колинеарни. Правата в оранжево показва, че тези точки са колинеарни.

[†]Фигура 7.5 е направена с Maple(TM).

Фигура 7.5 : DEGENERACY TESTING е 3SUM-hard.



7.4 Аргументиране чрез противник (Adversary argument)

7.4.1 Неформално обяснение

Вие искате да решите някаква задача върху някакви данни. Вие обаче нямате директен достъп до данните. Данните се държат от друг субект, когото наричаме условно "дявола". Дяволът позволява да му задавате въпроси за данните, и то адаптивно: задавате въпрос, той отговаря, а какъв ще бъде следващият Ваш въпрос (може да) зависи от получения отговор. Дяволът може да знае, но може и да не знае стратегията, която следвате. Какви са въпросите, които можете да задавате, зависи от правилата на играта. Може да са въпроси с булев отговор, примерно дали измежду две числа от данните, едното е по-малко от другото. Може да не са с булев, а с числен отговор, примерно, каква е стойността на число от входа, или каква е сумата на някакви числа от входа.

Лошотията на дявола се изразява единствено в това, че той Ви кара да зададете максимално много въпроси. Всеки път, когато зададете въпрос, дяволът дава такъв отговор, който ви принуждава, по отношение на всички зададени досега въпроси и получени отговори, да зададете максимално много въпроси, за да получите решение. Дяволът е **неограничено умен**; по-академично казано, той **има неограничени изчислителни способности** и във всеки момент знае точно каква информация Ви е разкрил досега и какъв отговор да даде на следващия Ви въпрос, за да Ви затрудни максимално. Както вече казахме, дяволът може да знае стратегията, която следвате, при което той знае всеки следващ Ваш въпрос, но може и да не я знае, при което той трябва да има предвид всяка възможна поредица от следващи Ваши въпроси. Подчертаваме, че дяволът **няма магически способности** и, ако не знае Вашата стратегия за решаване на задачата, няма как да знае следващия Ви въпрос.

Дяволът трябва да е *консистентен*: той е длъжен да се ангажира с отговорите, които е дал до момента. Ако веднъж каже, примерно, че $a_5 < a_{10}$, всички негови следващи отговори трябва да са консистентни с това; той вече не може да каже $a_5 \geqslant a_{10}$, нито може да даде поредица отговори, от които следва, че $a_5 \geqslant a_{10}$. Очевидно е, че ако дяволът имаше свободата да дава неконсистентни отговори, то цялото усилие щеше да е напълно безсмислено. Забележете, че **не се твърди**, че дяволът не лъже. Това, което се твърди, че лъжите на дявола не трябва да си противоречат. Както ще стане ясно от изложението тук, дяволът дори не лъже, защото **поначало вход няма**. Дяволът създава входа в процеса на отговаряне на Вашите въпроси. Единствената му лъжа би била тази: ако в края на играта го попитате "Това ли беше входът поначало?", той би казал "Да.", а всъщност в началото вход изобщо не е имало. Дяволът е създал в процеса на отговаряне такъв вход, спрямо редицата от Вашите въпроси, който Ви е накарал да зададете максимален брой въпроси.

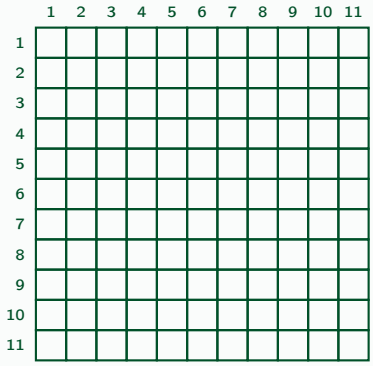
Ето пример. Нека задачата е Сортиране, и то базирано на директни сравнения. Вие задавате въпроси от вида $a_i \overset{?}{<} a_j$, за да изчислите пермутацията на входа, която представлява сортирана редица. Работата на дявола е да Ви накара да зададете максимален брой въпроси. Във всеки момент дяволът знае какви отговори е дал до момента, колко пермутации на n елемента са консистентни с досега дадените отговори и за всяко следващо запитване от вида $a_i \overset{?}{<} a_j$, дяволът отговаря ДА или НЕ въз основа на един единствен критерий – при кой от тези отговори, броят на пермутациите, които остават възможни (тоест, които са консистентни с всички досегашни отговори и този отговор) е по-голям. Ако отговор ДА дава по-голям брой пермутации, дяволът ще каже ДА. Ако отговор НЕ дава по-голям брой пермутации, дяволът ще каже НЕ. Ако отговор ДА оставя същият брой пермутации като отговор НЕ, дяволът казва ДА или НЕ произволно. В началото дяволът изобщо не си направил труда да генерира вход – вход не е имало. В края на играта, ако поискате от дявола да ви покаже входа, той ще изчисли вход, който е консистентен с всички въпроси и отговори, и ще излъже, че точно това е бил входът през цялото време.

Сега да конкретизираме примера. Нека задачата е Сортиране на три елемента, базирано на директни сравнения. Да кажем, че трите елемента са уникални. Нека дяволът не знае Вашата стратегия за получаване на решение. В този пример лошотията на дявола е в това, че ще Ви кара да зададете (поне) три въпроса. Очевидно, в някои случаи задачата може да се реши само с два въпроса, ако входът е подходящ за тях. Примерно, ако $a_1 < a_2 < a_3$ и въпросите са $a_1 \overset{?}{<} a_2$ и $a_2 \overset{?}{<} a_3$. Но при игра срещу дявола никога няма да "минете" само с два въпроса, защото дяволът ще генерира, по отношение на Вашите въпроси, вход, който е "неподходящ". Вашият първи въпрос е или $a_1 \overset{?}{<} a_2$, или $a_1 \overset{?}{<} a_3$, или $a_2 \overset{?}{<} a_3$. Кой от тези въпроси ще зададете, това дяволът не знае. Дяволът обаче знае, че има точно 6 възможни пермутации на входа и за всеки от тези въпроси, точно 3 пермутации са съвместими с положителния отговор и точно 3, с отрицателния. Така че няма значение какво ще отговори на първото запитване; да кажем, че дяволът винаги отговаря с ДА на първия въпрос. На втория въпрос обаче дяволът не отговаря произволно! Тъй като след отговора на първия въпрос има 3 възможни пермутации, консистентни с отговора, то, какъвто и втори въпрос да зададете, съществува отговор—ДА или НЕ, в зависимост от конкретиката—такъв че поне две пермутации са съвместими и с първия, и с втория отговор. Е, дяволът ще даде точно този отговор на втория въпрос и по този начин ще Ви принуди да зададе и трети въпрос. Забележете, че той ще ви принуди да зададете и трети въпрос независимо от това, какви са първия и втория Ви въпрос. След като получите решението и играта приключи, дяволът лесно може да предостави такъв вход, който е консистентен с всички отговори, които е дал, така че не може да бъде уличен в лъжа и измама.

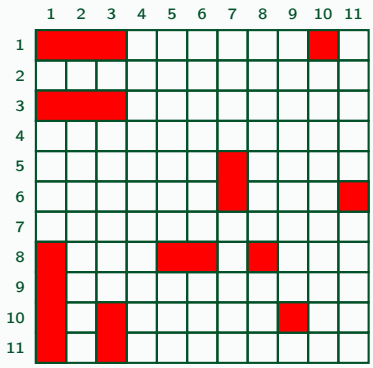
Допълнение 38: Морски шах с дявола

Няколко игри се наричат Морски шах на български. Тук ще разгледаме една от тях (на английски е Battleships), която е удачна илюстрация на основната идея на игра срещу злонамерен противник, разполагащ с неограничена изчислителна мощ.

Правилата са следните. Играят двама играчи. Всеки разполага с квадратна мрежа, да кажем 11×11, с номерирани редове и колони, наречена *море*:

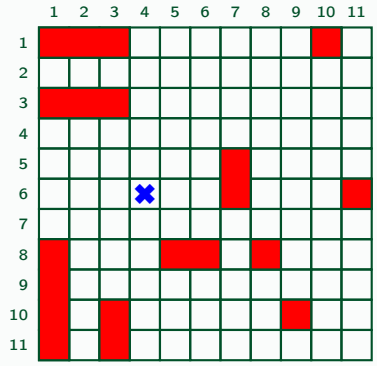


и няколко фигури, наречени *кораби*. Всеки кораб е правоъгълник, който може да бъде поставен, хоризонтално или вертикално, върху няколко съседни квадратчета от мрежата, запълвайки ги точно. Корабите са следните: един кораб 1×4, два кораба 1×3, три кораба 1×2 и четири кораба 1×1. Всеки играч крие своето море от другия играч. Преди началото на играта, всеки играч разполага своите кораби в своето море както намери за добре, но скришно от другия играч. Има ограничение два кораба да не се не се припокриват и да не се докосват, нито по хоризонтал, нито по вертикал, нито по диагонал. Ето примерно разполагане на корабите на единия играч:



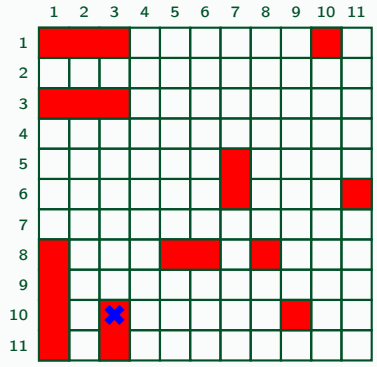
Самата игра се състои в редуващи се "изстрели" на всеки играч в морето на другия играч, като "изстрел" е просто казване на координатите на някое квадратче от морето на противника. Всеки играч записва изстрелите на противника си с хиксове върху своето море. Единият играч е първи. Той или тя казва на втория някаква наредена двойка $(\mathtt{i}, \mathtt{j})$, където $1 \leqslant \mathtt{i}, \mathtt{j} \leqslant 11$. Вторият играч прави следното.

- Ако на $(\mathtt{i}, \mathtt{j})$ няма кораб (в нашия пример, нека $(\mathtt{i}, \mathtt{j}) = (6, 4)$), вторият казва "вода", отбелязва хикс на $(\mathtt{i}, \mathtt{j})$ в своето море:

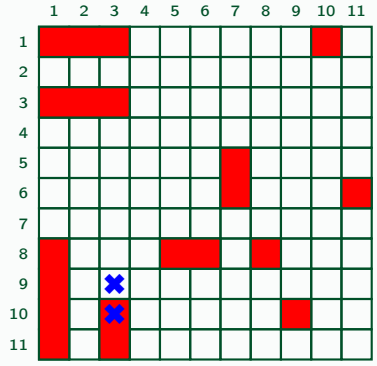


и на свой ред играе, казвайки координатите на някакво квадратче от морето на първия играч.

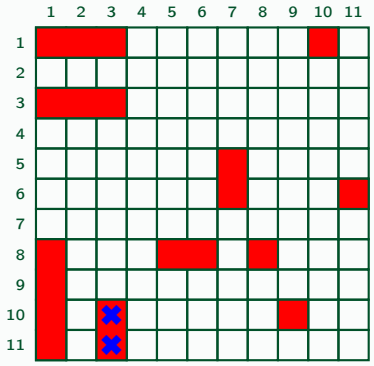
- Ако на това квадратче има кораб, който е по-голям от 1×1 и има поне едно неударено квадратче освен (i, j) (в нашия пример, нека $(i, j) = (10, 3)$), то вторият играч казва "удар" и отбелязва удара с хикс:



След това отново играе първият играч – този, който току-що нанесе удара. Първият продължава да играе, докато или не уцели вода:

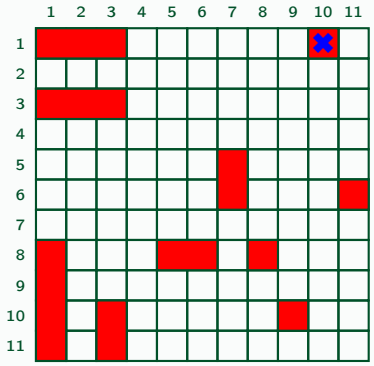


при което вече играе вторият играч, или не удари и последното досега неударено квадратче:

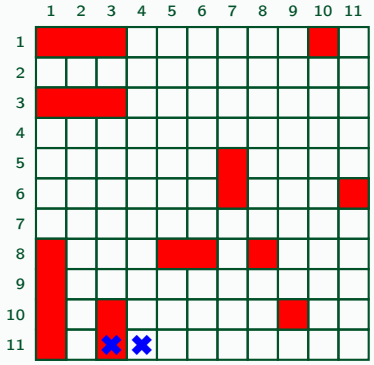


при което вторият играч казва “потопен” и първият играч продължава да играе.

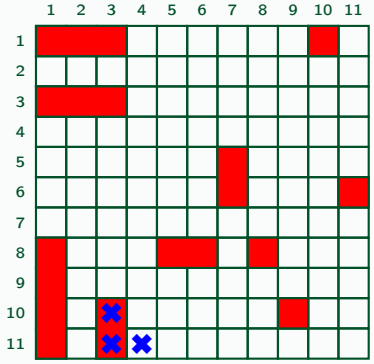
- Ако или на това квадратче има кораб 1×1—в нашия пример, нека $(i, j) = (1, 10)$:



или (i, j) е единственото неударено квадратче на кораб, по-голям от 1×1—в нашия пример, нека $(i, j) = (10, 3)$ и досега първият играч е направил тези два изстрела:

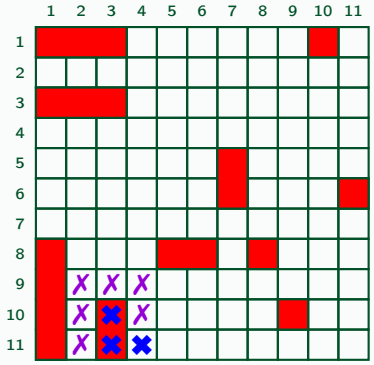


то вторият играч казва “потопен” и отбелязва мястото на удара с хикс.



После първият играч играе отново.

Печели играчът, който пръв или първа потопи всички кораби на противника си. Естествено, по време на играта няма смисъл играч да стреля върху квадратче, върху което вече е стрелял. Също така няма смисъл да стреля в района, ограждащ вече потопен кораб; ако разгледаме отново последния пример, след потапянето на кораба на $\langle(10,3),(11,3)\rangle$, всички квадратчета около него, в които досега не е стреляно, стават безсмислени като цели:



Това са правилата на играта. Сега да допуснем, че играете с дявола на морски шах. Почти сигурно ще загубите! Като добросъвестен и почтен човек, Вие поставяте корабите си в самото начало и после отговаряте вярно на изстрелите на противника, които улучват Вашите вече поставени кораби, с "удар" или "потопен". Дяволът обаче не е почтен и със сигурност на Вашите начални изстрели, където и да са те, той ще отговори с "вода". Същността на неговата стратегия е, че той изобщо не е поставил кораби. Дяволът не може да отговаря с "вода" прекалено дълго, защото съществува множество от изстрели от Ваша страна, които са така разположени, че неговите кораби нямат възможни позиции спрямо тях – но Вие с почти пълна сигурност няма да имате възможност да дадете толкова изстрели, защото ще сте загубили играта отдавна, играейки почтено.

Ако след загубата си поискате от дявола да ви покаже как са били разположени корабите му през цялото време, той ще изчисли такова разположение на корабите си, че нито един от Вашите изстрели не е попадение, и ще ви покаже него.

Имайте предвид, че дяволът е неограничено умен и ще отлага до последно първото попадение върху свой кораб. Както вече казахме, най-вероятно Вие ще сте загубили играта много преди да имате възможност да дадете толкова изстрели, че дяволът да не може повече да отлага първото попадение.

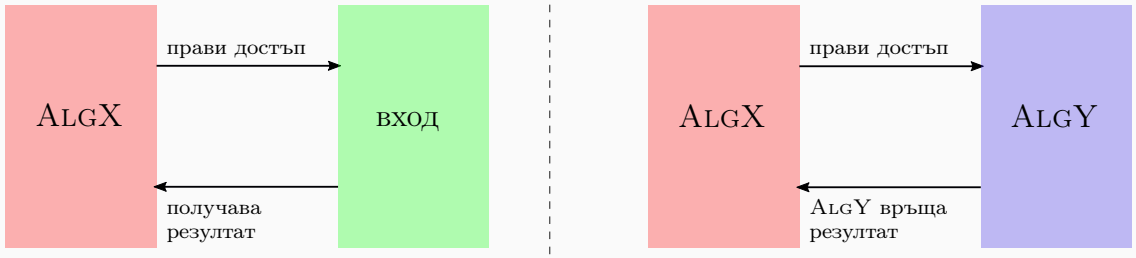
По друг начин казано: корабите общо заемат $4+6+6+4=20$ квадратчета, а морето е $11\times 11=121$ квадратчета. Вероятността при първи изстрел да бъде ударен кораб, ако се играе почтено и кораби са разположени поначало, а даващия изстрел стреля по случайно квадратче, е $\frac{20}{121}\approx\frac{1}{6}$. Ако играете достатъчно дълго с дявола и се редувате за пръв изстрел, в горе-долу $\frac{1}{6}$ от игрите неговият пръв изстрел ще е попадение (удар или потапяне). Дяволът няма магически способности и не вижда корабите Ви, така че в около $\frac{5}{6}$ от игрите първият му изстрел ще е "вода". Ще се изненадате обаче (ако не сте наясно, че играете срещу мошеник) от това, че **всеки** Ваш пръв изстрел е "вода". Защото дяволът не слага кораби поначало. Нещо повече, Вие ще загубите преди да можете да реализирате попадение, така че **всеки** Ваш изстрел по време на **цялата** игра ще е "вода".

Примерът с морския шах не е перфектна илюстрация за даване на долна граница на изчислителна задача, понеже при решаване на изчислителна задача играете само Вие (само Вие задавате въпроси), а дяволът само отговаря; в морския шах Вие се редувате с дявола за задаване на въпроси. Въпреки тази разлика, би трябвало този пример да даде добра интуиция за стратегията на опонент, който е максимално злонамерен и има неограничена изчислителна мощ.

7.4.2 Формално определение

Долна граница за някакъв предварително известен алгоритъм AlgX. Разглеждаме AlgX и друг алгоритъм AlgY, който играе ролята на противника (дявола). Всеки достъп на AlgX до входните данни всъщност е едно запитване (query) от страна на AlgX към AlgY. Може да си представяме, че AlgY прехваща всеки опит на AlgX за достъп до входа и той (AlgY) връща стойност, която, от гледна точка на AlgX, е легитимен резултат от достъпа да входа. Може още да си представяме, че AlgX не знае за съществуването на AlgY и няма как да разбере, че AlgY прехваща всеки опит за достъп до входа и резултатът от достъпа е това, което AlgY решава да предостави. И накрая, можем да мислим, че вход изобщо няма и AlgY връща, каквото намери за добре. Фигура 7.6 илюстрира всичко това.

Фигура 7.6 : Аргументация с противник: два алгоритъма.



Какво си мисли AlgX, че става. Какво става в действителност.

Основните допускания са следните.

- AlgY разполага с AlgX и оттам знае каква ще е поредицата от достъпи до входа, които ще извърши AlgX.
- AlgY е неограничено мощен. По отношение на AlgY не се интересуваме от това, какви ресурси ползва като време и памет. С други думи, не ни интересува колко ефикасен е AlgY, стига да е ефективен.
- AlgY връща такава поредица от отговори на AlgX, че AlgX да направи поне някакъв брой достъпи (queries), който брой е долната граница, която доказваме.
- AlgY трябва да е консистентен. Редицата резултати $\mathcal{R}$, които той дава на AlgX, трябва да е такава, че да има поне един вход със съответната големина, за който редицата от резултатите е точно $\mathcal{R}$.

Както казахме, AlgY играе ролята на противника (дявола). Дизайнерите на AlgY обаче сме ние. Нашата цел е да докажем долна граница за работата на AlgX в най-лошия случай. За тази цел ние конструираме противник (AlgY), който е в състояние да създаде такъв вход, който да накара AlgX да извърши поне толкова достъпи, колкото е долната граница, която доказваме. Подчертаваме, че AlgY не просто генерира някакъв вход поначало, а прави нещо много по-изтънчено и перфидно: AlgY **създава входа в процеса на достъпи и връщане на резултати**, отчитайки конкретните достъпи, извършени от AlgX.

Да кажем, че AlgY кара AlgX да направи максимален брой достъпи е прекалено амбициозно. В предната подсекция се твърдеше, че дяволът, бивайки неограничено умен и абсолютно злонамерен, връща отговори, които да ни накарат да работим максимално много. Когато конструираме противник-алгоритъм, целта ни е по-скромна, но затова пък постижима. Ние конструираме противник за доказване на някаква долна граница $\mathtt{f}(\mathtt{n})$, където $\mathtt{n}$ е големината на входа. Ако конструираме противник AlgY, който кара AlgX да извърши поне $\mathtt{f}(\mathtt{n})$ достъпа за всяко $\mathtt{n}$, и докажем това, ние сме си свършили работата. При това остава възможно да има друг противник AlgY$'$, който кара AlgX да извърши дори повече достъпи; да кажем, $\mathtt{f}'(\mathtt{n})$, където $\mathtt{f}'(\mathtt{n}) > \mathtt{f}(\mathtt{n})$ за всички достатъчно големи $\mathtt{n}$. Това, че може да има дори "по-лош" противник от този, който сме намерили и анализирали, не е в противоречие с нашето доказателство, че $\mathtt{f}(\mathtt{n})$ е долна граница.

Наблюдение 34

Да допуснем, че сме доказали, че конструираният от нас противник кара AlgX да прави поне $\mathtt{f}(\mathtt{n})$ достъпа за всяко $\mathtt{n}$. Ако освен това докажем, че $\mathtt{f}(\mathtt{n})$ е **точна долна граница** за броя на достъпите, то ние сме конструирали оптимален противник, който точно отговаря на дявола от Подсекция 7.4.1. В противен случай просто сме конструирали противник, който върши работа за дадено доказателство, оставяйки възможността да има дори по-лош противник.

Долна граница за изчислителна задача Π. Това е сравнително по-труден за конструиране и анализиране противник. Сега вече противникът AlgY не знае нищо за конкретния алгоритъм AlgX, който решава Π. От гледна точка на AlgY, AlgX е просто черна кутия, от която излизат достъпи до входа (queries). Фигура 7.6 илюстрира адекватно и тази ситуация, но сега имаме предвид, че

- AlgY не знае нищо за AlgX.
- "AlgX" не означава конкретен алгоритъм, а означава кой да е алгоритъм за задачата Π.

Допусканията за неограничена мощност, "злонамереност" и консистентност на AlgY са в сила и сега.

7.4.3 Примери за доказване на долни граници чрез противник

Ще покажем няколко добре известни доказателства за долни граници на задачи чрез противник. Това не са асимптотични долни граници, каквито видяхме при задачи като Сортиране или Уникалност на елементи. Тук ще докажем точни изрази-долни граници върху броя на сравненията за няколко лесни задачи.

7.4.3.1 $n-1$ за намиране на максимален елемент

Задачата е да се намери чрез, и само чрез, директни сравнения максимално число измежду произволни числа a_1, a_2, ..., a_n. Първо забелязваме, че всяко a_i трябва да участва в поне едно сравнение. Да допуснем противното: нека има коректен алгоритъм AlgMax за тази задача, който върху поне един вход работи така, че някой a_i не участва в нито едно сравнение. Тогава противникът манипулира входа така:

- Ако AlgMax върне a_i[†], то противникът прави a_i по-малко от всяко друго число от входа. При това положение няма как a_i да е максимално число. Не можем да уличим противника в лъжа, защото a_i не е било сравнявано изобщо, така че противникът може да му присвои—след края на работата на AlgMax—каквото поиска и да ни го покаже.

- Ако AlgMax върне a_j, където $j \neq i$, противникът прави a_i по-голям от всеки друг елемент. При това положение a_i е максималното число. Отново, не можем да уличим противника в лъжа, защото a_i не е бил сравнявано изобщо.

И в двата случая AlgMax не работи коректно, така че той не е коректен алгоритъм и допускането е опровергано.

Долната граница, която следва от съображението, че всеки елемент трябва да е бил сравнен поне веднъж с друг, е $\left\lceil \frac{n}{2} \right\rceil$. За да се убедим в това, да си представим неориентиран граф на сравненията, в който върховете са числата a_1, a_2, ..., a_n, а ребрата са ненаредените двойки числа, които са били сравнени от алгоритъма. Твърдението, че всяко число трябва да е било сравнено поне веднъж е същото като твърдението, че графът няма изолирани върхове. Лесно се вижда, че минималният брой ребра в граф без изолирани върхове е $\left\lceil \frac{n}{2} \right\rceil$.

Но долната граница $\left\lceil \frac{n}{2} \right\rceil$ е твърде слаба. Не че всичко това не е истина, но има по-висока долна граница за броя на сравненията, която сега ще докажем. Ще докажем, че ако AlgMax е коректен алгоритъм за задачата, то той прави поне $n-1$ сравнения.

Първо доказателство. Да си представим отново графа на сравненията. Твърдим, че той трябва да е свързан.

Да допуснем, че графът на сравненията не е свързан. По-подробно казано, допускаме, че има вход a_1, a_2, ..., a_n, върху който AlgMax прави такива сравнения, че графът на сравненията има поне две свързани компоненти. Освен това допускаме, че AlgMax работи коректно върху a_1, a_2, ..., a_n и числото a_i, което той връща, е максимално. Но a_i е връх в някоя свързана компонента G' от графа на сравненията. Тъй като този граф не е свързан, той има поне една друга свързана компонента G''. Противникът манипулира входа така: прави всеки елемент-връх в G'' по-голям от всеки елемент-връх от G', като относителната наредба между елементите-върхове от G'' се запазва. При това положение максимумът няма как да е от G', така че AlgMax не е работи коректно върху a_1, a_2, ..., a_n. Противникът не може да бъде уличен в лъжа постфактум, защото той ще запълни входа по такъв начин, че елементите, които са върхове в G'', имат такива стойности, за които изходите от сравненията са точно такива, каквито е получил AlgMax по време на работата си.

Ключовият факт, върху който почиват тези разсъждения, е, че ако графът не е свързан, за всеки две различни свързани компоненти G' и G'' е вярно, че не може да заключим нищо за относителните големини на елемент-връх от G' и елемент-връх от G''. Може всеки елемент от G' да е по-голям от всеки елемент от G'', може всеки елемент от G'' да е по-голям от всеки

[†]Звучи безумно алгоритъмът да върне елемент, който изобщо не е бил "прегледан", но тук правим теоретично доказателство и разглеждаме всички случаи.

елемент от G', може всички елементи от G' да са с големини между два съседни по големина елементи от G'', и така нататък – всичко е възможно за относителните им големини.

И така, доказахме, че хипотетичният AlgMax, който върху поне един вход прави такива сравнения, че техният граф не е свързан, не е коректен. Заключаваме, че графът на сравненията е свързан. Добре известен факт от теорията на графите е, че всеки свързан граф на n върха има поне $n-1$ ребра. Това е и краят на доказателството на тази долна граница.

Второ доказателство. Сега да допуснем, че елементите от входа са два по два различни. Може да аргументираме долната граница $n-1$ и със следните разсъждения. Нека казваме, че a_j *е загубило от* a_k, а a_k *е спечелило от* a_j , ако AlgMax е направил сравнение $a_j \overset{?}{<} a_k$ и резултатът от сравнението е бил ДА. Нека AlgMax връща като максимално число a_i.

Лема 47

В текущия контекст е вярно, че всяко число, различно от a_i, е загубило поне едно сравнение.

Доказателство: Ако допуснем противното, а именно, че за някое $j \neq i$, a_j не е губил сравнение, то противникът манипулира входа, присвоявайки на a_j стойност, която е по-голяма от стойността на всеки друг елемент. Сега вече a_j със сигурност е максималният елемент от масива, но резултатът от всяко сравнение, в което участва a_j, си остава същият – отново a_j не губи нито едно сравнение. Резултатите от сравненията, в които a_j не участва, очевидно не се променят от увеличаването на стойността на a_j. Но тогава AlgMax продължава да връща a_i, защото резултатите от всички сравнения са същите като тези преди увеличаването на a_j.

Накратко, ако повече от един елемент не е губил сравнение, алгоритимът няма как да е коректен, защото противникът лесно ще установи това и ще направи максимум някой от тези елементи, които алгоритъмът не връща като максимален.

В това доказателство ние пак имаме предвид граф на сравненията, но сега той е **ориентиран**. Върховете пак са a_1, a_2, …, a_n, но всяко ребро има посока. На всяко сравнение $a_p \overset{?}{<} a_q$ съответства:

- или ориентираното ребро (a_q, a_p) при отговор ДА,
- или ориентираното ребро (a_p, a_q) при отговор НЕ.

Очевидно този граф е dag – обратното би означавало, че противникът е неконсистентен, което не е разрешено по условие[†]. Елементите, които не са загубили сравнение, са точно източниците. Е, противникът може лесно да установи дали dag-ът има повече от един източник и ако е така, да манипулира стойностите на тези източници, така че да "прецака" AlgMax. AlgMax ще върне някой от тези източници, а противникът ще направи друг от тях да има по-голяма стойност от този, който AlgMax връща като максимален елемент. Няма как да уличим противника в лъжа постфактум, понеже всяка двойка източници са несравними в dag-а. □

Лема 47 ни дава желания резултат, понеже всички други елементи (освен a_i) са $n-1$ и всеки от тях трябва да е бил сравнен поне веднъж, иначе няма как да е загубил сравнение.

[†]Цикъл в графа би имало тогава и само тогава, когато противникът е казал, че един елемент е по-голям от друг, той е по-голям от трети, и така нататък, и последният е по-голям от първият.

В току-що направеното доказателство нарушихме изискването противникът да не знае нищо за работата на AlgMax, а само да решава какво да отговори на следващото сравнение на базата на отговорите на предните сравнения. Противникът някак вижда, че AlgMax връща един източник на dag-а a_i и спрямо това прави друг източник на dag-а a_j по-голям от a_i. Но това не е съществена спънка. Дори да лишим противника от възможността да вижда какво прави AlgMax, може да кажем така:

> Тъй като правим анализ на най-лошия случай, самото съществуване на възможност противника да "прецака" AlgMax е същото като противникът да "прецака" AlgMax.

Заслужава да се отбележи, че долната граница $n-1$ за броя на сравненията е точна долна граница. Следният прост алгоритъм намира максимален елемент във входа с $n-1$ сравнения.

```
MaxElement(A[1,...,n]: int)
 1  tmp ← A[1]
 2  for i ← 2 to n
 3      if A[i] > tmp
 4          tmp ← A[i]
 5  return tmp
```

7.4.3.2 $\left\lceil \frac{3n}{2} \right\rceil - 2$ за намиране на максимален и минимален елемент

Задачата е да се намери чрез, и само чрез, директни сравнения максимално число и минимално число измежду произволни числа a_1, a_2, ..., a_n, където $n \geqslant 2$. Ще допуснем, че числата са две по две различни, така че ще казваме, че търсим максимума и минимума. Очевидно може да намерим максимума и минимума поотделно с общо $(n-1)+(n-1) = 2n-2$ сравнения, но може да сторим това и по-бързо ето така.

Първо да допуснем, че n е четно. Групираме числата в $\frac{n}{2}$ двойки, във всяка двойка намираме максимума и минимума с едно сравнение и получаваме $\frac{n}{2}$ максимума, по един от всяка двойка, и още $\frac{n}{2}$ минимума, по един от всяка двойка. Глобалния максимум намираме от тези $\frac{n}{2}$ максимума чрез $\frac{n}{2}-1$ сравнения. Аналогично, глобалния минимум намираме от тези $\frac{n}{2}$ минимума чрез $\frac{n}{2}-1$ сравнения. Общият брой на използваните сравнения е

$$\frac{n}{2} + \left(\frac{n}{2}-1\right) + \left(\frac{n}{2}-1\right) = \frac{3n}{2} - 2$$

което при четно n можем да напишем като

$$\left\lceil \frac{3n}{2} \right\rceil - 2$$

Сега да допуснем, че n е нечетно. Броят на двойките е $\left\lfloor \frac{n}{2} \right\rfloor$ и едно число a_j остава извън двойките. Намираме $\left\lfloor \frac{n}{2} \right\rfloor$ максимума, по един от всяка двойка, и още $\left\lfloor \frac{n}{2} \right\rfloor$ минимума, по един от всяка двойка. После глобалният максимум се намира измежду тези максимуми и a_j с $\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) - 1 = \left\lfloor \frac{n}{2} \right\rfloor$ сравнения. После глобалният минимум се намира от тези минимуми и a_j (разглеждаме най-лошият случай, в който a_j не се е оказал глобален максимум и сме длъжни да проверим дали не е глобален минимум) пак с $\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) - 1 = \left\lfloor \frac{n}{2} \right\rfloor$ сравнения. Общият брой на сравненията е $3\left\lfloor \frac{n}{2} \right\rfloor$. При $n = 2k+1$, това е

$$3\left\lfloor \frac{2k+1}{2} \right\rfloor = 3k = (3k+2) - 2 = \frac{6k+4}{2} - 2 = \left\lceil \frac{6k+3}{2} \right\rceil - 2 = \left\lceil \frac{3(2k+1)}{2} \right\rceil - 2 = \left\lceil \frac{3n}{2} \right\rceil - 2$$

Виждаме, че и при четно, и при нечетно n тази идея води до решение с $\lceil \frac{3n}{2} \rceil - 2$ сравнения. Следният алгоритъм реализира тази идея. Алгоритъмът очевидно може да се реализира in-place, но сега сложността по памет не ни интересува, така че можем да си позволим реализация с линейна сложност по памет.

```
MaxNMin(A[1,...,n]: int, n ⩾ 2)
 1  k ← ⌊n/2⌋
 2  създай масиви MAX[1,...,k], MIN[1,...,k]
 3  for i ← 1 to k
 4      if A[2i − 1] > A[2i]
 5          MAX[i] ← A[2i − 1], MIN[i] ← A[2i]
 6      else
 7          MAX[i] ← A[2i], MIN[i] ← A[2i − 1]
 8  tmpmax ← MAX[1], tmpmin ← MIN[1]
 9  for i ← 2 to k
10      if MAX[i] > tmpmax
11          tmpmax ← MAX[i]
12      if MIN[i] < tmpmin
13          tmpmin ← MIN[i]
14  if isodd(n)
15      if A[n] > tmpmax
16          tmpmax ← A[n]
17      if A[n] < tmpmin
18          tmpmin ← A[n]
19  return (tmpmax, tmpmin)
```

Ще докажем, че този алгоритъм е оптимален по отношение на броя на сравненията. Ще докажем, че има противник, който може да манипулира входа така, че да накара всеки алгоритъм за задачата да направи поне $\lceil \frac{3n}{2} \rceil - 2$ сравнения.

Да си представим някакъв алгоритъм за тази задача. Както вече видяхме в доказателството на Лема 47, сравненията, направени от алгоритъма, индуцират ориентиран граф, чиито върхове са числата a_1, a_2, ..., a_n, а ребро (a_i, a_j) съществува тогава и само тогава, когато a_i е било сравнено с a_j и резултатът е бил $a_i > a_j$. Както видяхме там, този граф е dag, който има точно един източник. Източникът отговаря на максималното число. Напълно аналогично, този dag има точно един сифон, който отговаря на минималното число. Очевидно всяко число, което не е нито максималното, нито минималното, има indegree поне единица и outdegree поне единица. Използвайки терминологията, която се ползва от Лема 47, вярно е, че всяко число без едно е загубило поне едно сравнение (най-голямото не губи никое сравнение) и всяко число без едно е спечелило поне едно сравнение (най-малкото не печели никое сравнение).

В началото нито едно число не е било сравнено. След това алгоритъмът започва да сравнява двойки числа и в края на работата му $n-2$ числа са спечелили поне по едно сравнение и загубили поне по едно сравнение, едно число само е печелило сравнения и едно друго число само е губило сравнения. Има смисъл да въведем *състояние* на всяко от числата по време на работата на алгоритъма. Във всеки момент, всяко число x е в точно едно от следните четири състояния:

`N`, ако x още не е било сравнявано.

`W`, ако x е спечелило поне едно сравнение и не е загубило нито едно сравнение.

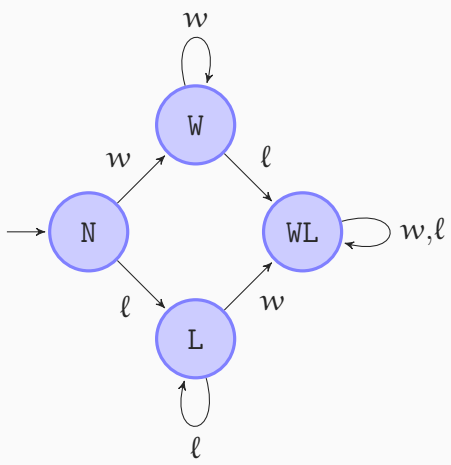
`L`, ако x е загубило поне едно сравнение и не е спечелило нито едно сравнение.

`WL`, ако x е спечелило поне едно сравнение и е загубило поне едно сравнение.

В началото всяко число е в състояние `N`. В края всички числа без две са с в състояние `WL`, точно едно е в състояние `W` и точно едно е в състояние `L`. В някакъв смисъл, работата на алгоритъма е да "вкара" числата в тези състояния – когато, и само когато, всички числа са в така описаните състояния, задачата е решена. Работата на противника е да забави колкото е възможно повече постигането на тези състояния.

Да помислим как число минава от състояние в състояние след извършване на сравнения. Началното състояние е `N`. `WL` е мъртво състояние – от него не може да се излезе. Всеки преход става след сравнение на това число с друго, което означава, че нашето число или е спечелило, или е загубило. Сравнение, в което то е спечелило, означаваме с "w", а сравнение, в което то е загубило, означаваме с "ℓ". Фигура 7.7 показва диаграмата на състоянията и преходите за произволно число.

Фигура 7.7 : Диаграма на състоянията и преходите.



За достигане на `WL` от `N` са необходими поне два прехода.

Сега ще опишем стратегия на противника при сравняване на две числа, която максимизира минималния брой на сравненията. Да кажем, че се извършва сравнение $a_i \overset{?}{<} a_j$. Всяко от a_i и a_j има състояние от $\{\texttt{N}, \texttt{W}, \texttt{L}, \texttt{WL}\}$, което е известно на противника. Да разгледаме всички възможни комбинации от състояния. Те са само $\binom{2+4-1}{2} = 10$, а не $4^2 = 16$, защото комбинациите са двуелементните мултимножества над опорното множество $\{\texttt{N}, \texttt{W}, \texttt{L}, \texttt{WL}\}$. Таблица 7.1 описва как противникът манипулира данните във всеки възможен случай.

От особен интерес са петте комбинации, в които противникът няма избор за своя отговор (оцветените редове). Примерно, да разгледаме втория ред. Състоянието на a_i е `N`, а това на a_j е `W`; тоест, a_i не е участвал в сравнения изобщо, а a_j е участвал в поне едно сравнение и спечелил всички сравнения, в които е участвал.

- Ако противникът отговори $a_i < a_j$ (както прави според таблицата), той променя състоянието само на a_i. Очевидно a_i не може да остане `N`, щом вече е участвал в едно сравнение. Но a_j си остава в `W`, защото продължава да е вярно, че само е печелил сравнения.

Съст-ние на a_i	Съст-ние на a_j	Избор на противника	Можеше ли да избере обратното?	Ново съст-ние на a_i	Ново съст-ние на a_j	Промяна в броя на съст-нията
N	N	$a_i < a_j$	да	L	W	+2
N	W	$a_i < a_j$	не	L	W	+1
N	L	$a_i > a_j$	не	W	L	+1
N	WL	$a_i < a_j$	да	L	WL	+1
W	W	$a_i < a_j$	да	WL	W	+1
W	L	$a_i > a_j$	не	W	L	0
W	WL	$a_i > a_j$	не	W	WL	0
L	L	$a_i < a_j$	да	L	WL	+1
L	WL	$a_i < a_j$	не	L	WL	0
WL	WL	$a_i < a_j$	да	WL	WL	0

Таблица 7.1: Стратегията на противника в задачата за намиране на максимален и минимален елемент.

- Ако противникът отговори $a_i > a_j$ (което той няма да направи, защото е неограничено умен и знае, че би било в негов ущърб да отговори така), то a_i би се оказал в W, спечелвайки срещу a_j, но a_j би се оказал в WL, след като е отбелязал първата си загуба. В този случай и a_i, и a_j биха променили състоянията си.

От този пример трябва да е ясно каква е играта на противника: изходът от всяко сравнение е такъв, че се промяната в състоянията **на двата участника** да е минимална. Ако се сравняват N с N (първият ред), промяната е в състоянията и на двете числа, независимо от отговора на противника (съответно, при това сравнение противникът отговаря произволно). Колкото и да не иска, противникът е принуден да даде две "парчета" информация, по едно за всяко число. Естествено, сравненията N срещу N са най-много $\left\lfloor \frac{n}{2} \right\rfloor$ на брой. Във всеки друг случай (освен N срещу N), противникът може да отговори по такъв начин, че или да не промени нито едно от състоянията (например W срещу L на ред шести), или да промени само едно от състоянията.

Да въведем понятието *макросъстояние* като векторът от състоянията на всички числа. Началното макросъстояние е $\langle \mathtt{N}, \mathtt{N}, \ldots, \mathtt{N} \rangle$. Да кажем, че *крайно макросъстояние* е всеки от векторите с един елемент W, един елемент L и останалите $n-2$ елемента WL. Както вече отбелязахме, имаме категоричен отговор на задачата тогава и само тогава, когато е достигнато крайно макросъстояние. За достигането на крайно макросъстояние от началното макросъстояние, трябва да бъдат извършени поне $2n-2$ прехода, защото:

- за всяко число, което завършва в WL са необходими поне 2 прехода (вижте Фигура 7.7), което означава поне $2(n-2) = 2n-4$ прехода общо.
- за числото, което завършва в W е необходим поне 1 преход,
- за числото, което завършва в L е необходим поне 1 преход.

И така, трябва да бъдат извършени поне $2n-4+2 = 2n-2$ прехода общо от всички числа. Това обаче не означава $2n-2$ сравнения! Както видяхме, има сравнения, при които **и двете** участващи числа извършват преход. Това са точно сравненията N срещу N. При всички останали сравнения, противникът може да манипулира входа така, че да има не повече от 1 преход на сравнение **и за двете** участващи числа.

Сравненията от вида `N` срещу `N` са най-много $\left\lfloor \frac{n}{2} \right\rfloor$ на брой, както вече отбелязахме. Това означава, че противникът винаги може да наложи

$$2n - 2 - \left\lfloor \frac{n}{2} \right\rfloor = 2n - 2 + \left\lceil -\frac{n}{2} \right\rceil = \left\lceil 2n + \left(-\frac{n}{2} \right) \right\rceil - 2 = \left\lceil \frac{3n}{2} \right\rceil - 2$$

сравнения.

7.4.3.3 $\left\lceil \frac{3n-3}{2} \right\rceil$ за намиране на медиана

7.4.3.4 $n - 2 + \lceil \lg n \rceil$ за намиране на втори максимален елемент

7.4.3.5 $2n - 1$ за търсене в сортирана по редове и колони, $n \times n$ матрица

Дадена е матрица M с резмери $n \times n$ от числа. M е сортирана по редове и колони; тоест,

$$\forall i \in \{1, \ldots, n\}\ \forall j \in \{1, \ldots, n-1\} : M[i, j] \leqslant M[i, j+1]$$
$$\forall j \in \{1, \ldots, n\}\ \forall i \in \{1, \ldots, n-1\} : M[i, j] \leqslant M[i+1, j]$$

Дадено е и число key. Пита се дали key се среща в M.

Ето пример за такава матрица при $n = 6$.

	1	2	3	4	5	6
1	1	2	3	4	5	40
2	7	8	9	10	48	56
3	19	21	25	45	75	78
4	28	29	30	65	81	88
5	29	40	58	70	91	95
6	50	55	60	74	99	100

Също като при задачата ТЪРСЕНЕ на стр. 124, и тук задачата

- може да е задача за разпознаване,
- а може да се искат координатите на key, ако key се среща в M, или индикация, че key не се среща в M, в противен случай.

В какъв вариант е задачата не е съществено.

Първо ще разгледаме една недобра идея за бърз алгоритъм, после една добра идея, и накрая ще намерим точна долна граница, в асимптотичния смисъл, с което ще покажем, че добрата идея е оптимална.

Недобра идея за бърз алгоритъм Очевидно можем да открием key, ако го има, или да сме сигурни, че го няма, ако проверим всички n^2 клетки на M. Това би било прекалено бавно. Трябва някак да се възползваме от факта, че M е сортирана по редове и колони.

Да опитаме така. Да разгледаме главния диагонал. По отношение на примерната матрица горе, това е главният диагонал:

	1	2	3	4	5	6
1	1	2	3	4	5	40
2	7	8	9	10	48	56
3	19	21	25	45	75	78
4	28	29	30	65	81	88
5	29	40	58	70	91	95
6	50	55	60	74	99	100

Да търсим key с двоично търсене по главния диагонал, което означава да го сравняваме с елементи от вида $M[i,i]$, за $i \in \{1,\ldots,n\}$. Ако при това го открием, връщаме координатите му. В противен случай, има някакъв индекс j, такъв че $M[j,j] < key$ и $M[j+1,j+1] > key$. По отношение на нашия пример, нека $key = 52$. Не откриваме key в главния диагонал, като въпросното j е 3:

	1	2	3	4	5	6
1	1 <52	2	3	4	5	40
2	7	8 <52	9	10	48	56
3	19	21	25 <52	45	75	78
4	28	29	30	65 >52	81	88
5	29	40	58	70	91 >52	95
6	50	55	60	74	99	100 >52

Точката, маркирана с червен кръст, задава четири подматрици. В две от тях, маркирани с червен фон, числото 52 не може да се среща; в другите две, маркирани със зелен фон, числото 52 може да се среща.

	1	2	3	4	5	6
1	1	2	3	4	5	40
2	7	8	9	10	48	56
3	19	21	25	45	75	78
4	28	29	30	65	81	88
5	29	40	58	70	91	95
6	50	55	60	74	99	100

Продължаваме да търсим рекурсивно в двете зелени подматрици. Спирачката на рекурсията е главен диагонал с дължина единица.

Тази идея не е съвсем тривиална за реализиране. В примера, който току-що видяхме, подматриците, върху които ще се изпълнява рекурсивно алгоритъма, са с размери 3×3; тоест, това са пак квадратни матрици, но всяка със страна $n/2$. Но може n да е не е четно, а може и точката, която дели главния диагонал, да не го дели наполовина, при което подматриците, в които търсенето продължава, са далече от квадратни. Примерно, нека числото, което търсим, е пак 52, но в тази матрица:

	1	2	3	4	5	6
1	1	2	3	4	5	40
2	7	8	9	10	48	56
3	19	21	125	145	175	178
4	28	29	130	165	181	188
5	29	40	158	170	191	195
6	50	55	160	174	199	999

Сега зелените подматрици, където трябва да продължим с търсенето, не са квадратни. И за тях можем да въведем някакъв "обобщен главен диагонал", върху който да търсим с двоично търсене, но това е безсмислено. Както ще видим, самата идея на този алгоритъм не е добра. Дори при перфектна дихотомия върху главния диагонал, както в примера, в който точката-разделител се оказа точно в средата му, рекурентното уравнение, което описва сложността по време в най-лошия случай, е

$$T(n) = 2T\left(\frac{n}{2}\right) + \lg n \tag{7.9}$$

Това е само ако на **всяко** ниво на рекурсията точката-делител на главния диагонал се пада по средата му. Събираемото $\lg n$ отразява работата на двоичното търсене върху главния диагонал, а $2T\left(\frac{n}{2}\right)$ отразява работата на двете рекурсивни викания, всяко върху матрица със страна $n/2$. Съгласно първия случай на Мастър теоремата, решението е $T(n) \asymp n$. Както предстои да видим, има много по-прост и елегантен линеен алгоритъм за тази задача, който не по схемата **Разделяй-и-Владей**.

Добра идея за бърз алгоритъм Да допуснем, че key не се среща в M, но ние не знаем това и тепърва трябва да го установим. Да сравним $M[n, 1]$ с key. Има две възможности.

1. Ако $M[n, 1] < \text{key}$, то със сигурност $M[n-1, 1] < \text{key}$, ..., $M[1, 1] < \text{key}$.
2. Ако $M[n, 1] > \text{key}$, то със сигурност $M[n, 2] > \text{key}$, ..., $M[n, n] > \text{key}$.

В първата възможност има смисъл да сравним $M[n, 2]$ с key. Във втората възможност има смисъл да сравним $M[n-1, 1]$ с key.

Това дава следната идея. Да въведем текущ елемент $M[p, q]$ от матрицата, като поначало $p = n$ и $q = 1$. Ако $M[p, q] < \text{key}$, правим "ход надясно" $q{+}{+}$. Ако $M[p, q] > \text{key}$, правим "ход нагоре" $p{-}{-}$. И правим това, докато не излезем извън матрицата; тоест, докато $p \geqslant 1$ и $q \leqslant n$. Когато излезем извън матрицата, извеждаме съобщение, че key не се среща в M.

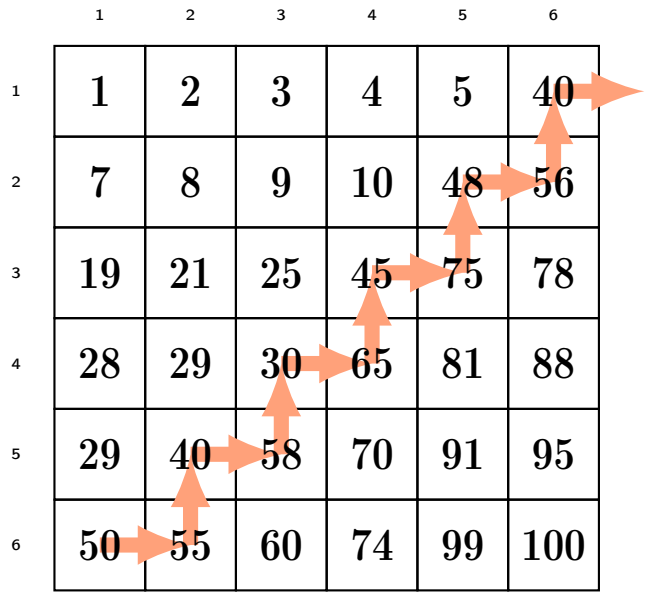
Разбира се, в общия случай key може да се среща, така че на всяка стъпка първо проверяваме дали $M[p, q] = \text{key}$. Псевдокодът е следният.

PARTIALLY-ORDERED MATRIX SEARCH($M[1, \ldots, n][1, \ldots, n]$: матрица от числа, key: число)

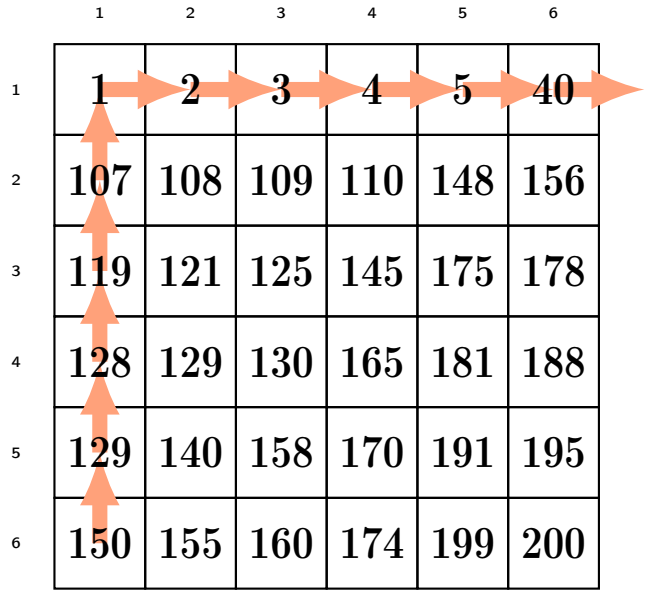
```
1  p ← n, q ← 1
2  while p ⩾ 1 and q ⩽ n do
3      if M[p, q] = key
4          return (p, q)
5      else if M[p, q] < key
6          q++
7      else
8          p--
9  return −1
```

Приемаме, че коректността е очевидна. Тъй като "придвижването" става, в общия случай, по нещо като начупена линия, която започва от долу-ляво и върви само нагоре и надясно, максималният брой клетки, които ще проверим—което е същото като максималният брой изпълнения на тялото на **while**-цикъла—е $2n - 1$. Следователно, сложността по време на този алгоритъм е $\Theta(n)$ в най-лошия случай. Читателят лесно може да се убеди, че засуканият, разделяй-и-владей алгоритъм с двоичното търсене по главния диагонал, който разгледахме преди малко, не прави нищо друго освен да схожда към въпросната начупена линия. Очевидно е много по-смислено да се движим по нея последователно.

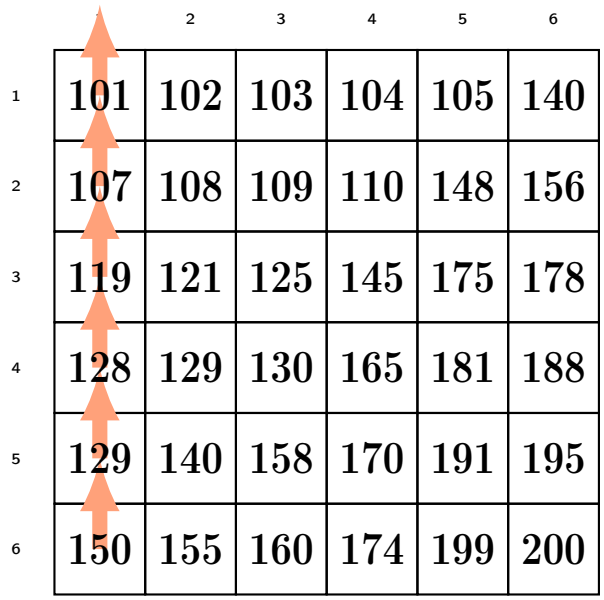
Ето илюстрация на работата на алгоритъма. Използваме матрицата, с която започнахме примера горе, а key отново е 52. Илюстрацията показва въпросната начупена линия с цветни стрелки върху матрицата и по този начин виждаме редицата от стойностите, които наредената двойка (p, q) е вземала по време на работата.



"Начупената линия" може и да не толкова начупена, и въпреки това да има дължина $2n-1$, както се вижда от следната илюстрация:



Възможно е key да не се среща и "начупената линия" да е с дължина само n:



	1	2	3	4	5	6
1	101	102	103	104	105	140
2	107	108	109	110	148	156
3	119	121	125	145	175	178
4	128	129	130	165	181	188
5	129	140	158	170	191	195
6	150	155	160	174	199	200

Възможни са и подобрения, при които "начупената линия" не започва непременно с $M[n, 1]$, а нейното начало се намира с двоично търсене върху долната страна $M[n, 1]$, ..., $M[n, n]$ или върху дясната страна $M[1, 1]$, ..., $M[1, n]$. По отношение на най-лошия случай тези подобрения нямат значение.

Долна граница $\Omega(n)$ за задачата Ще видим прост аргумент чрез противник, който дава желаната долна граница. Да видим следната илюстрация за $n = 6$. За краткост, пишем "k" вместо "key". Допуснете, че ϵ е стойност, много по-малка от k, а ∞ е стойност, много по-голяма от k.

	1	2	3	4	5	6
1	ϵ	ϵ	ϵ	ϵ	ϵ	$k + \epsilon$
2	ϵ	ϵ	ϵ	ϵ	$k - \epsilon$	$k + \epsilon$
3	ϵ	ϵ	ϵ	$k - \epsilon$	$k + \epsilon$	∞
4	ϵ	ϵ	$k - \epsilon$	$k + \epsilon$	∞	∞
5	ϵ	$k - \epsilon$	$k + \epsilon$	∞	∞	∞
6	$k - \epsilon$	$k + \epsilon$	∞	∞	∞	∞

Очевидно k не се съдържа в M. Същественото обаче е, че всеки алгоритъм ALGA, който коректно заключава това, трябва да провери всяка от оцветените клетки (които съдържат $k - \epsilon$ или $k + \epsilon$). Ако ALGA пропусне дори една от тези клетки, противникът ще промени нейното съдържание на k, но ALGA ще продължава да "твърди", че k не се среща, защото резултатите от всички достъпи до клетките на M ще е същият.

Като пример, основан на предния пример, да кажем, че алгоритъмът никога не проверява $M[4, 3]$. Очевидно алгоритъмът няма как да различи предната матрица от тази:

	1	2	3	4	5	6
1	ϵ	ϵ	ϵ	ϵ	ϵ	$k+\epsilon$
2	ϵ	ϵ	ϵ	ϵ	$k-\epsilon$	$k+\epsilon$
3	ϵ	ϵ	ϵ	$k-\epsilon$	$k+\epsilon$	∞
4	ϵ	ϵ	k	$k+\epsilon$	∞	∞
5	ϵ	$k-\epsilon$	$k+\epsilon$	∞	∞	∞
6	$k-\epsilon$	$k+\epsilon$	∞	∞	∞	∞

Сега k се съдържа, но алгоритъмът няма как да извлече този факт при положение, че не прави достъп до $M[4,3]$. Забележете, че M и сега е сортирана по редове и по колони.

И така, има вход за $n = 6$, за който се налага да бъдат проверени $2n - 1 = 11$ клетки, преди да се направи заключението, че k не се среща в матрицата. Очевидно този аргумент се мащабира за всяко n, с което наблюдение приключваме с доказателството.

В тази задача символът "n" означаваше не големината на входа, а корен-квадратен от големината на входа. Забележете, че входът е с големина n^2 (плюс още 1 заради key), което е броят на елементите на матрицата. Поради това, алгоритъмът, който разгледахме, всъщност не е линеен, а със сложност по време $\Theta(\sqrt{N})$, където N е големината на входа. И долната граница, която доказахме, е $\Omega(\sqrt{N})$, изразена чрез големината на входа.

Конвенция 7: Големина на входа и квадратни матрици

Прието е, когато входът или основната част на входа е квадратна матрица $n \times n$, сложността на алгоритъма да се изразява като функция на n. Това нарушава правилото, сложността да се изразява чрез големината на входа.

Лекция 8

Сортиране в линейно време. Counting Sort. Radix Sort.

Резюме: Демонстрираме сортиране в линейно време върху вход, чиито елементи не са произволни, а се подчиняват на определени ограничения. Въвеждаме сортиращите алгоритми Counting Sort и Radix Sort

8.1 Сортиране в линейно време

Както видяхме в предната лекция, сортирането на прозволни елементи, чиито индивидуални стойности не можем четем, а само можем да сравняваме елементите един с друг, не може да става асимптотично по-бързо от $n \lg n$. Но тази долна граница не е в сила, ако има някакви ограничения възможните стойности на елементите. Съвсем прост пример е сортирането на булев масив – очевидно е достатъчно да преброим колко са елементите от всеки вид и после да презапишем входа със съответния брой нули и единици, като нулите са преди единиците, което може да стане в $\Theta(n)$ време. Алгоритъмът, който ще разгледаме сега, се основава на идеята за пребройване на елементите от всяка големина, но както ще стане ясно, той третира елементите от входа не просто като числа, а като записи, чиито ключове са числа. Тъй като записите освен ключовете имат и някаква сателитна информация, сортирането става с местене на елементи (а не просто презаписване на входа с числа). Дори когато казваме "входът е масив от числа", имаме предвид, че входът е масив от записи с ключове-числа.

8.2 Counting Sort

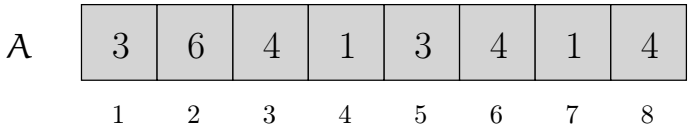
Входът е масив от числа $A[1, \ldots, n]$. Известно е, че за някакво $k \in \mathbb{N}^+$, за всяко $A[i]$ е изпълнено

$$A[i] \in \{1, 2, \ldots, k\}$$

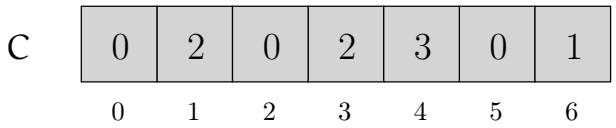
Алгоритъмът ползва работен масив $C[0, 1, \ldots, k]$ и още един масив $B[1, 2, \ldots, n]$, в който се записва резултата от сортирането.

```
1  COUNTING SORT(A[1,2,...,n]: positive integers; k: a positive integer)
2    (* k е такова, че 1 ⩽ A[i] ⩽ k за всички i *)
3    създай B[1,...,n] и C[0,1,...,k]
4    for i ← 0 to k
5        C[i] ← 0
6    for i ← 1 to n
7        C[A[i]] ← C[A[i]] + 1
8    for i ← 1 to k
9        C[i] ← C[i] + C[i − 1]
10   for i ← n downto 1
11       B[C[A[i]]] ← A[i]
12       C[A[i]] ← C[A[i]] − 1
13   return B
```

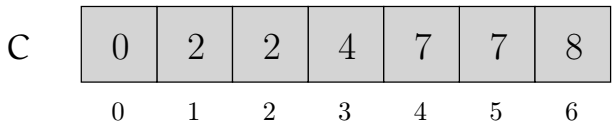
Първо ще покажем работата на алгоритъма с пример и после ще дадем формално доказателство за коректност. Нека $n = 8$, $k = 6$ и масивът A е:



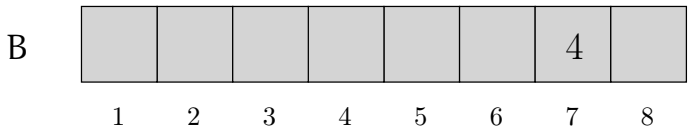
Очевидно първият **for**-цикъл (редове 3–4) просто нулира масива C. Вторият **for**-цикъл (редове 5–6) преброява по колко елемента от всяко $i \in \{1, \ldots, k\}$ има в A и записва това в C:



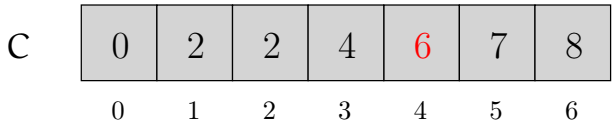
Очевидно, $\sum_{i=0}^{k} C[i] = n$. Третият **for**-цикъл (редове 7–8) присвоява на всяко $C[i]$ броя на всички елементи на A, по-малки или равни на i:



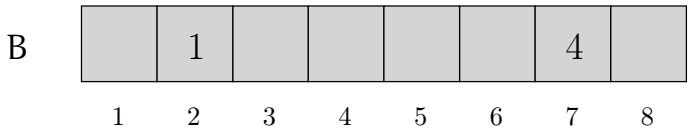
Очевидно, сега $C[k] = n$. Същината на алгоритъма е в четвъртия **for**-цикъл (редове 9–11). В началото $i = 8$, $A[8] = 4$, $C[4] = 7$. $B[7]$ става 4:



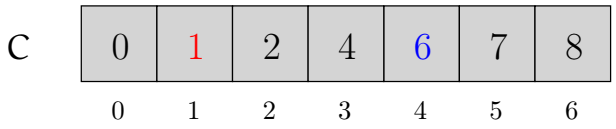
а C става:



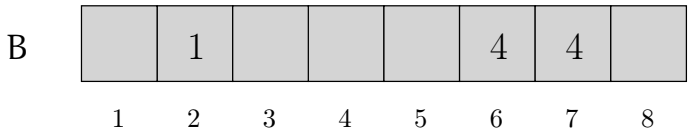
В червено е току-що намаленият елемент (заради ред 11) на C. После $i = 7$, $A[7] = 1$, $C[1] = 2$. $B[2]$ става 1:



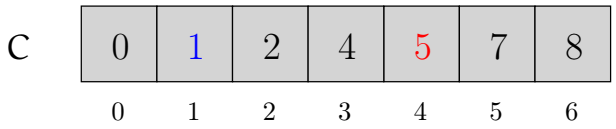
а C става:



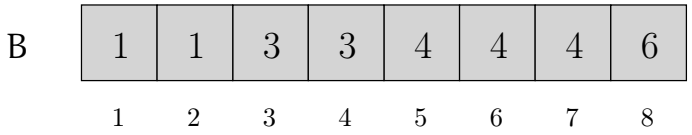
После $i = 6$, $A[6] = 4$, $C[4] = 6$. $B[6]$ става 4:



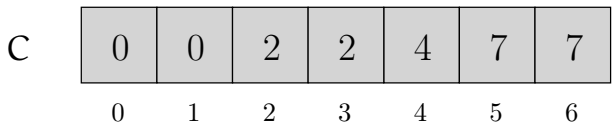
а C става:



И така нататък. В края на алгоритъма B е:



а C е:



В Определение 40 въведохме нотацията "$\#(a, A)$" за кратността на a в мултимножеството A. Тук ще използваме нотацията "$\#(x, Z[1, \ldots, j])$", за да означаваме кратността на x в мултимножеството от елементите на $Z[1, \ldots, j]$, където Z е масив, j е индекс, сочещ в него, а x е елемент, чийто тип позволява да е елемент на Z.

Сега ще докажем коректността на Counting Sort.

Лема 48

След приключването на втория **for**-цикъл (редове 5–6), за всяко j, такова че $1 \leqslant j \leqslant k$, $C[j]$ съдържа броят на елементите в масива A, които са равни на j.

Доказателство:

Инвариант 18: Вторият for-цикъл на Counting Sort

Всеки път, когато изпълнението е на ред 5, за всеки елемент $C[j]$, където $1 \leqslant j \leqslant k$, е изпълнено $C[j] = \#(j, A[1, \ldots, i-1])$.

База. При първото достигане на ред 5, всички елементи на C са нули. От друга страна, i е 1, и така $A[1,\ldots,i-1]$ е празен. Твърдението е вярно.

Поддръжка. Да допуснем, че твърдението е в сила при дадено достигане на ред 5, което не е последното. Нека стойността на $C[A[i]]$ е y в момента, в който изпълнението е на ред 6. По индуктивното предположение, $y = \#(A[i], A[1,\ldots,i-1])$. Очевидно е, че $\#(A[i], A[1,\ldots,i-1]) + 1 = \#(A[i], A[1,\ldots,i])$. След изпълнението на ред 6, $C[A[i]]$ се увеличава с единица, така че $C[A[i]]$ става равно на $\#(A[i], A[1,\ldots,i])$. Тъй като всички останали елементи на C (освен $C[A[i]]$) остават непроменени от текущото изпълнение на **for**-цикъла, е вярно, че:

- за всеки елемент $C[j]$ освен $C[A[i]]$, $C[j] = \#(j, A[1,\ldots,i-1])$, а също така $C[j] = \#(j, A[1,\ldots,i])$
- $C[A[i]] = \#(A[i], A[1,\ldots,i])$.

Като цяло, за всеки елемент $C[j]$ е в сила $C[j] = \#(j, A[1,\ldots,i])$. Това е преди инкрементирането на i. След инкрементиравнето на i, $C[j] = \#(j, A[1,\ldots,i-1])$ за всяко j, такова че $1 \leqslant j \leqslant k$.

Терминация. Да разгледаме момента, в който изпълнението е на ред 5 за последен път. Очевидно i е $n+1$. Заместваме i с $n+1$ в инварианта и получаваме "за всеки елемент $C[j]$, където $1 \leqslant j \leqslant k$, е в сила $C[j] = \#(j, A[1,\ldots,n])$." □

Лема 49

След приключването на третия **for**-цикъл (редове 7–8), за всяко j, такова че $1 \leqslant j \leqslant k$, $C[j]$ съдържа броят на елементите в масива A, които са по-малки или равни на j.

Доказателство:

Инвариант 19: Третият for-цикъл на Counting Sort

Всеки път, когато изпълнението е на ред 7, за всяко j, такова че $0 \leqslant j \leqslant i-1$, $C[j] = \sum_{t=1}^{j} \#(t, A[1,\ldots,n])$.

База. При първото достигане на ред 7, i е 1, така че твърдението е $C[0] = \sum_{t=1}^{0} \#(t, A[1,\ldots,n]) = 0$. Но $C[0]$ е наистина 0, защото то се инициализира с 0 от първия **for**-цикъл и вторият **for**-цикъл не му присвоява нищо (понеже $A[i]$ не може да е 0). ✓

Поддръжка. Да допуснем, че твърдението е в сила при някое достигане на ред 7, което не е последното. Елементът $C[i]$ не е променян (засега) от изпълнението на третия **for**-цикъл, така че съгласно Лема 48, $C[i] = \#(i, A[1,\ldots,n])$. По индуктивното предположение, $C[i-1] = \sum_{t=1}^{i-1} \#(t, A[1,\ldots,n])$. След изпълнението на ред 8 имаме $C[i] = \sum_{t=1}^{i} \#(t, A[1,\ldots,n])$. По отношение на новата стойност на i, за всяко j, такова че $0 \leqslant j \leqslant i-1$, $C[j] = \sum_{t=1}^{j} \#(t, A[1,\ldots,n])$.

Терминация. Да разгледаме момента, в който изпълнението е на ред 7 за последен път. Очевидно $i = n+1$. Заместваме i с $n+1$ в инварианта и получаваме "за всяко j, такова че $0 \leqslant j \leqslant n$, $C[j] = \sum_{t=1}^{j} \#(t, A[1,\ldots,n])$." □

Определение 49

За всяко $j \in \{1,\ldots,k\}$, j е *съществено*, ако съществува елемент на A със стойност j.

Съгласно Лема 48, ненулевите елементи на C след втория **for**-цикъл са точно елементите, чиито индекси в C са съществени.

Определение 50

За всеки x от A, *правилното място на* x е броят на елементите от A, които са по-малки от x, плюс броя на елементите равни на x, които са вляво от x, плюс едно.

С други думи, правилното място на x е позицията му в масива след изпълнението на стабилен сортиращ алгоритъм.

Теорема 49: Counting Sort е стабилен сортиращ алгоритъм.

Counting Sort е стабилен сортиращ алгоритъм.

Доказателство:

Инвариант 20: Четвъртият for-цикъл (редове 9–11)

Всеки път, когато изпълнението е на ред 9, за всяко j, такова че $1 \leqslant j \leqslant k$ и j е съществено, $C[j]$ съдържа правилното място на най-десния елемент от подмасива $A[1, \ldots, i]$, който има стойност j. Нещо повече, всички елементи от $A[i+1, \ldots, n]$ са на своите правилни места в B.

База. При първото изпълнение на ред 7, i е n. Първата част от инварианта гласи "за всяко j, такова че $1 \leqslant j \leqslant k$ и j е съществено, $C[j]$ е правилното място на най-десния елемент от подмасива $A[1, \ldots, n]$, който има стойност j". Забележете, че C все още не е променян от четвъртия цикъл, така че Лема 49 е в сила. За всяка стойност j, която се появява в A, правилното място на най-дясното j в A е равно на сумата от броевете на елементите със стойност $\leqslant j$. Според Лема 49, $C[j]$ е равно точно на тази сума.

Да разгледаме втората част от инварианта. Подмасивът $A[i+1, \ldots, n] = A[n+1, \ldots, n]$ е празен, така че твърдението е в сила. ✓

Поддръжка. Да допуснем, че твърдението е в сила при някое достигане на ред 9, което не е последното. Стойността на $A[i]$ е някое съществено цяло число j между 1 и k. Нещо повече, то е **най-десният** елемент в $A[1, \ldots, i]$ със стойност j. Съгласно индуктивното предположение, неговото правилно място е $C[A[i]]$ и алгоритъмът го копира точно там (ред 10).

Да допуснем, че има и други елементи със стойност j в $A[1, \ldots, i]$. На ред 11, $C[A[i]]$ бива декрементирано, така че то вече съдържа правилното място на j в $A[1, \ldots, i-1]$. След като i бъде декрементирано е вярно, че елементът на C, който току-що беше декрементиран съдържа правилното място на най-дясното j в $A[1, \ldots, i]$. Тъй като всички останали елементи на C са непроменени, първата част от инварианта е в сила.

Сега да допуснем, че в $A[1, \ldots, i]$ няма други елементи със стойност j. Тогава j не е съществен по отношение на останалата част на A (която предстои да бъде сканирана от четвъртия цикъл), така че стойността на $C[A[i]]$ е без значение за алгоритъма оттук нататък. Наистина, след декрементирането на ред 11, $C[A[i]]$ сочи клетка в B, която е мястото на друг елемент (а не на j). Но, както казахме, стойността на това $C[A[i]]$ няма да бъде използвана до края на алгоритъма, понеже тази стойност на $A[i]$ няма да се среща повече. Първата част от инварианта се запазва и в този случай.

Сега ще докажем втората част от инварианта. Да разгледаме момента, когато изпълнението е на ред 9 в началото на текущата итерация. Съгласно индуктивното предположение, всички елементи от $A[i+1, \ldots, n]$ са на своите правилни места в B. Току-що доказахме, че

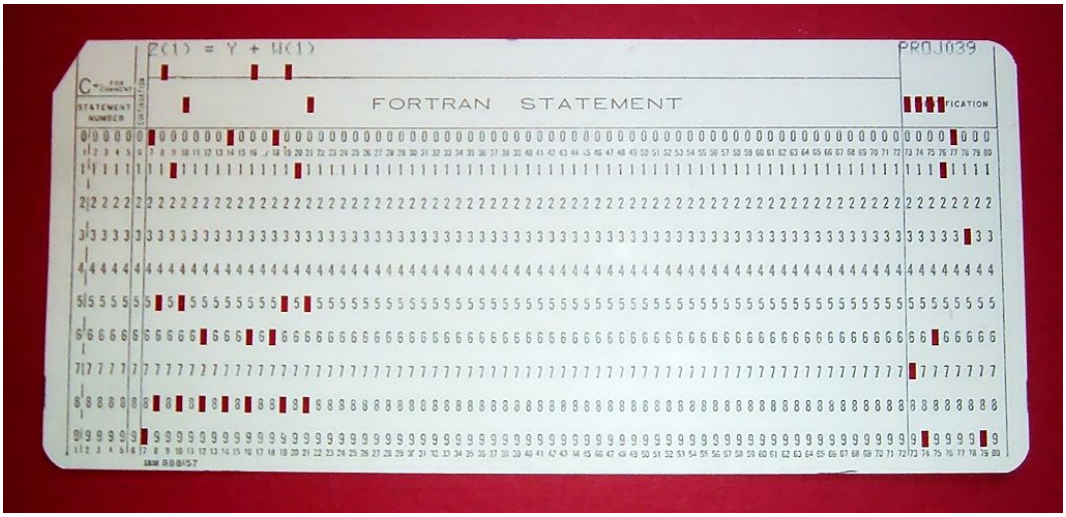
по време на това изпълнение на цикъла, елементът $A[i]$ бива копиран на правилното място. Тогава всички елементи от $A[i, \ldots, n]$ са на своите правилни места в B. По отношение на новата стойност на i, вярно е, че всички елементи от $A[i+1, \ldots, n]$ са на своите правилни места в B.

Терминация. При завършването на цикъла, $i = 0$. Заместваме i с 0 във втората част на инварианта и получаваме "всички елементи от $A[1, \ldots, n]$ са своите правилни места в B." □

Анализът на сложността е тривиален: Counting Sort работи във време $\Theta(n + k)$. Ако $k = O(n)$, както често се случва при използването на този алгоритъм, сложността по време е $\Theta(n)$. Това е значително подобрение спрямо $\Theta(n \lg n)$ на бързите сортиращи алгоритми, базирани на директни сравнения. Сложността по памет е същата: $\Theta(n + k)$, а ако $k = O(n)$, та е $\Theta(n)$. Следователно, алгоритъмът не е *in-place*.

8.3 Radix Sort

Radix Sort е алгоритъмът, използван за сортиране на перфокарти. Перфокарта, на английски *punched card* или *Hollerith card*, е цифров носител на информация, използван преди появата на реални компютри, и използван до 70те и дори 80те години на 20 век в компютрите. Работещ компютър, в който информацията се въвежда с перфокарти, в днешно време едво ли може да бъде намерен извън музея, но снимки на перфокарти се намират лесно по Интернет. Ето една такава снимка, взета от wikipedia с лиценз CC2.5; върху картата е написана в кодиран вид част от програма на Fortran:

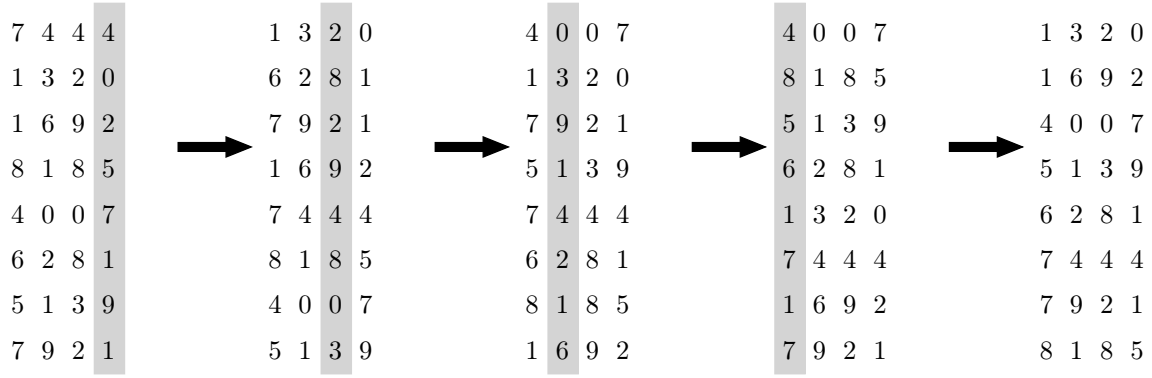


Стандартната Холерит перфокарта има 80 колони, а във всяка колона може да бъде пробита дупка на точно едно измежду дванадесет възможни места. Всяка колона представлява символ. Ако символът е (десетична) цифра, дупката е на една от десетте позиции, маркирани с цифрите. Другите две позиции са за не-цифрова информация. Ако картата записва число, то може да има най-много 80 десетични разряда, колкото са колоните.

Задачата е, при дадено множество карти, които записват числа, картите да се сортират от електро-механично устройство, така че записаните числа да са в ненамаляващ ред. Електромеханичното устройство може да чете в даден момент точно един от десетичните разряди, с други думи, една от колоните, като допира вертикално разположени игли до картата и отчита коя игла потъва (през вече направената дупка), по този начин отчитайки коя е цифрата от съответния разряд.

Първата идея за сортиращ алгоритъм, който да управлява това електро-механично устройство, е да сортираме картите по най-старшия разряд, после по следващия, и така нататък, до най-младшия разряд. Недостатъкът на този подход е, че след сортирането по най-старши разряд трябва да държим картите в десет различни купчини[†] до края, след сортирането по втори най-старши разряд купчините ще станат сто[‡] и така нататък, което не е практично. Правилният подход е разрядите (колоните) да се тълкуват като ключове: старшият разряд е първичният ключ, вторият най-старши е вторичният, и така нататък, най-младшият разряд е 80-ичния ключ. Както знаем от лекцията по сортиране, ако сортираме цялата купчина (без да я разбиваме на подкупчини и подподкупчини и т. н.) първо по 80-ичния ключ, после от 79-ичния ключ, и така нататък, и най-накрая по първичния ключ, ще имаме правилно сортирани карти. Но: **при условие, че сортираме със стабилен сортиращ алгоритъм**. Стабилността гарантира, че когато сортираме по колона $\mathtt{i}$, ако има няколко карти с една и съща цифра в колона $\mathtt{i}$, техният взаимен порядък няма да се наруши; с други думи, вече извършените сортирания по колони $\mathtt{i}+1$, $\mathtt{i}+2$ и така нататък са определили правилното им взаимно разположение.

Ще дадем пример за работата на Radix Sort. Ще сортираме числата 7444, 1320, 1692, 8185, 4007, 6281, 5139 и 7921. Представяме си, че са написани едно над друго в този ред, и започваме да сортираме от колоната на най-младшите разряди към колоната на най-старшите разряди. Колоната със сив фон отбелязва по кои разряди сортираме в момента.



Псевдокодът на Radix Sort е съвсем прост (съгласно [21]):

Radix Sort($\mathtt{A}[1,\ldots,\mathtt{n}]$: положителни числа, записани с един и същи брой цифри; $\mathtt{d}$: броят на цифрите)

```
1  (* най-младшата цифра е номер 1, най-старшата цифра е номер d *)
2  for i ← 1 to d
3      сортирай A по цифра i със стабилен сортиращ алгоритъм
```

Очевиден кандидат за стабилен сортиращ алгоритъм е Counting Sort. Коректността на Radix Sort е очевидна, имайки предвид това, което знаем за стабилните сортирания. Ако използваме Counting Sort като стабилен сортиращ алгоритъм, сложността на Radix Sort е $\Theta(\mathtt{d}(\mathtt{n}+\mathtt{k}))$, където $\mathtt{k}$ е броят на възможните цифри. Ако $\mathtt{k}$ е константа, примерно 10, то сложността по време е $\Theta(\mathtt{dn})$.

Използвайки Radix Sort, можем да решим следната задача: да се сортира в линейно време масив от $\mathtt{n}$ числа, всяко от което принадлежи на множеството $\{1,2,\ldots,\mathtt{n}^2\}$. Ако опита-

[†]Купчините биха били десет, ако и десетте цифри се появяват като най-старша цифра на някоя карта.

[‡]Ще станат сто, ако и десетте цифри се появяват като и като най-старша цифра, и като втора най-старша цифра, на някоя карта. Корекция на Емилиян Рогачев: Купчините ще станат 100, ако всяка наредена двойка цифри се среща като начални две цифри.

ме директно да я решим с Counting Sort, решението ще работи в $\Theta(n^2)$, защото работният масив на Counting Sort ще бъде с размер n^2. Но ако разгледаме записите на тези числа, примерно в двоична позиционна бройна система, ще видим, че всяко от тях се записва с $\lfloor \lg n^2 \rfloor + 1 = \lfloor 2 \lg n^2 \rfloor + 1 \approx 2 \lg n$ бита. Ако разделим всеки бинарен запис на две части, всяка с по $\approx \lg n$ бита, може да смятаме дясната част за младши ключ, а лявата, за старши ключ. Прилагайки Radix Sort (който на свой ред ползва Counting Sort) първо по младшите ключове, а после по старшите, имаме решение във време $\Theta(2n) = \Theta(n)$. Това, което ни помогна да конструираме линеен алгоритъм е, че прилагаме Counting Sort два пъти, но при всяко прилагане, работният масив е линеен, защото и младшите, и старшите ключове са с големина $\approx \lg n$, което означава, че могат да има най-много $\Theta(n)$ различни стойности.

Лекция 9

Въведение в алгоритмите върху графи. Обхождания на графи.

Резюме: Правим кратка рекапитулация на графите. Правим сравнителен анализ на най-популярните представяния на графи. Разглеждаме основната алгоритмична задача върху графи: обхождане. Разглеждаме двата най-популярни алгоритъма за обхождане на графи: BFS и DFS.

9.1 Рекапитулация на графите

Определение 51: Граф

Граф е наредена двойка (V, E), където V е непразно множество от *върховете*, а E е множество от *ребрата*. V е опорното множество (ground set), а E е подмножество на $\{\{x, y\} \mid \{x, y\} \subseteq V\}$.

Графите, дефинирани в Определение 51 се наричат *неориентирани*, тъй като техните ребра са множества—двуелементни множества от върхове, по-точно—а множествата нямат наредба. Ако кажем само "граф", ще разбираме неориентиран граф.

Определение 52: Ориентиран граф

Ориентиран граф е наредена двойка (V, E), където V е непразно множество от върховете, а E е множество от ребрата. V е опорното множество, а E е подмножество на $(V \times V) \backslash \{(v, v) \in V \times V\}$.

Определение 51 и Определение 52 не допускат *примки*; примка е ребро, на което двата края са един и същи връх. На читателя остава да прецени как да ги модифицира по такъв начин, че да се допускат примки.

Определение 53: Ориентиран мултиграф

Ориентиран мултиграф е наредена тройка (V, E, f_G), където V е непразно множество от *върхове*, E е множество от *ребра*, а f_G е *свързващата функция*. V и E са опорни множества, а $f_G : E \to V \times V$.

Оттук насетне ще описваме мултиграфите без да споменаваме свързващата функция; примерно, $G = (V, E)$.

На читателя остава да дефинира "неориентиран мултиграф". Когато кажем "мултиграф" без уточнения, ще разбираме ориентиран мултиграф. Под *обикновен граф* разбираме граф, който не е мултиграф.

Всички определения върху графи от курса по Дискретни Структури са в сила. Тук добавяме няколко определения и нотации, които не са изучавани по Дискретни Структури.

Нотация 3: "$\mathtt{SC}(\mathsf{G})$" означава релацията на силна свързаност над $\mathsf{V}(\mathsf{G})$

Нека G е ориентиран граф. "$\mathtt{SC}(\mathsf{G})$" означава релацията на силна свързаност над $\mathsf{V}(\mathsf{G})$.

Определение 54: Даг

Ако G е ориентиран ацикличен граф, ще казваме накратко, че G *е даг.*

Името идва от английския акроним "dag" (**d**irected **a**cyclic **g**raph). Авторът на тези лекционни записки не знае общоприет български термин за "ориентиран граф без цикли", нито харесва "огбц", така че ще ползва "даг".

Определение 55: арборесценция

На английски *arborescence* или *out-tree* се нарича ориентиран граф, който се получава от кореново дърво—което е неориентиран граф—с корен $\mathtt{r}$, като на ребрата се даде ориентация навън от корена. Алтернативна дефиниция е, ориентиран граф G, в който съществува връх $\mathtt{r}$, такъв че за всеки връх $\mathtt{u} \in \mathsf{V}(\mathsf{G})$ е вярно, че има един единствен ориентиран, не непременно прост, път от $\mathtt{r}$ до $\mathtt{u}$.
Връх $\mathtt{r}$ е коренът на арборесценцията.

Думата "arborescence" произхожда от латинското *arbor*, което означава дърво. Авторът на тези лекционни записки не знае общоприет български термин за "ориентирано дърво с ориентация навън от корена", така че по неволя ще използва "арборесценция". Забележете, че всеки арборесценция е даг, докато обратното не е вярно.

Конвенция 8: Буквите $\mathtt{m}$ и $\mathtt{n}$ в контекста на графите

Когато разглеждаме графи, неизменно буквата $\mathtt{n}$ ще има смисъл на броя на върховете, а $\mathtt{m}$, на броя на ребрата, освен ако изрично не е казано нещо друго.

Наблюдение 35: Ограничения за $\mathtt{m}$ при дадено $\mathtt{n}$

Ако разглеждаме обикновен неориентиран граф без примки, то $0 \leqslant \mathtt{m} \leqslant \binom{\mathtt{n}}{2}$, като границите са точни. Ако разглеждаме обикновен ориентиран граф без примки, то $0 \leqslant \mathtt{m} \leqslant \mathtt{n}(\mathtt{n}-1)$, като границите са точни. Ако разглеждаме обикновен ориентиран граф, който може да има примки, $0 \leqslant \mathtt{m} \leqslant \mathtt{n}^2$, като границите са точни. Ако разглеждаме мултиграф, няма горна граница за $\mathtt{m}$, така че $0 \leqslant \mathtt{m}$ е всичко, което знаем за $\mathtt{m}$, ако не е дадено някакво ограничение в условието.

Основните представяния на графи, които не са мултиграфи, са следните две. Нека графът е $\mathsf{G} = (\mathsf{V}, \mathsf{E})$, $\mathsf{V} = \{\mathtt{u}_1, \ldots, \mathtt{u}_\mathtt{n}\}$ и $\mathsf{E} = \{e_1, \ldots, e_\mathtt{m}\}$.

Списъци на съседство: За всеки връх е даден списък от неговите съседи, ако става дума за неориентиран граф, или списък от неговите деца, ако става дума за ориентиран граф. Списъците са точно n. Списъците **не са подредени по никакъв начин**, както един спрямо друг, така и всеки от тях вътрешно.

Матрица на съседство: Това е булева матрица $n \times n$, където клетка (i, j) съдържа 1 тогава и само тогава, когато

- има ребро **между** u_i и u_j, ако графът е неориентиран; в този случай матрицата е симетрична.
- има ребро **от** u_i **до** u_j, ако графът е ориентиран; в този случай матрицата не е непременно симетрична.

Мултиграфите се представят със списъци на съседство по гореописания начин. Има разлика в матричното представяне между обикновени графи и мултиграфи: ако искаме да представяме мултиграф с матрица, тя трябва да е не булева, а от естествени числа, като клетка (i, j) съдържа броя на ребрата от u_i до u_j.

Наблюдение 36

Ако неориентиран граф, който е мултиграф или не, е представен чрез списъци, всяко ребро (u_i, u_j), което не е примка, се появява **два пъти**: веднъж в списъка на u_i и веднъж в списъка на u_j.

Размерът на граф е $m + n$, ако е представен чрез списъци. Това е в сила и за мултиграфи, и за графи, които не са мултиграфи. От друга страна, ако е представен чрез матрица, размерът е n^2.

Конвенция 9

Тъй като използваме списъчното представяне по-често, по подразбиране размерът на граф е $m + n$. Следователно, по подразбиране, линеен алгоритъм върху графи е такъв, който работи във време $\Theta(m + n)$.

Нотация 4: "adj[x]" означава списъка на съседство на връх x

Ако G е граф и x е връх в него, "adj[x]" означава списъка на съседство на x в списъчното представяне на G.

Действия върху графи. Графите, които разглеждаме, понякога са статични, но понякога искаме да добавяме или да изтриваме техни елементи, в който случай ще казваме, че са *динамични*[†]. Естествено, в най-общия случай бихме искали да добавяме или изтриваме както върхове, така и ребра. На практика се оказва, че значително по-лесно е при **фиксирано** множество от върхове да се добавят или премахват ребра. Такива графи—с фиксирано множество от върхове, спрямо което добавяме или изтриваме ребра—се наричат *полудинамични*. Ние ще се ограничим до действията добавяне на ребра и изтриване на ребра, което значи, че ако графите, които разглеждаме, не са статични, те са полудинамични.

[†]Това няма **нищо общо** с динамичното програмиране, което ще разгледаме в Лекция 13

Таблица 9.1 показва сложността **в най-лошия случай** на основни алгоритмични примитиви и алгоритми върху графи при матрично и при списъчно представяне. Допускаме, че е даден обикновен граф $G = (V, E)$, ориентиран или не, като $V = \{u_1, \ldots, u_n\}$.

Действие \ Представяне	чрез матрица	чрез списъци
тестване дали $(u_i, u_j) \in E$	$\Theta(1)$	$\Theta(n)$
намиране на $d(u_i)$	$\Theta(n)$	$\Theta(n)$
опит за добавяне на ребро (u_i, u_j)	$\Theta(1)$	$\Theta(n)$
опит за изтриване на ребро (u_i, u_j)	$\Theta(1)$	$\Theta(n)$
преброяване на ребрата	$\Theta(n^2)$	$\Theta(m + n)$
обхождане на графа	$\Theta(n^2)$	$\Theta(m + n)$

Таблица 9.1: Сложностите на действията върху графи при двете представяния.

Забележете, че ако графът е мултиграф, опит за добавяне на ребро отнема само $\Theta(1)$ време и при списъчното представяне, защото просто вмъкваме новото ребро в началото на списъка; в такъв случай има смисъл да говорим само за "добавяне на ребро", а не за "опит за добавяне", защото няма причина да не може да добавим ново ребро между/от-до съществуващи върхове. При обикновените графи такъв опит може да не е успешен, защото такова ребро може вече да има, оттам идва и сложността $\Theta(n)$ при списъчното представяне – налага се да минем през целия списък на u_i или u_j, за да се убедим, че няма ребро (u_i, u_j).

Определение 56: Разредени графи

Нека $\mathfrak{G}$ е безкрайно множество от графи. Казваме, че $\mathfrak{G}$ *са разредени графи*, на английски *sparse graphs*, ако за всеки $G \in \mathfrak{G}$ имаме $|E(G)| = O(|V(G)|)$. По-подробно казано, съществува положителна константа c, такава че за всеки $G \in \mathfrak{G}$ имаме $|E(G)| \leqslant c|V(G)|$.

Забележете, че въпросът дали даден граф е разреден граф е **безсмислен въпрос**, говорейки формално. Съгласно Определение 56, въпросът е смислен само за безкрайни множества от графи, или *класове от графи*. Важни класове графи, които са разредени, са:

дърветата. Знаем, че за всяко дърво $m = n - 1$.

планарните графи. Знаем, че за всеки планарен граф $m \leqslant 3n - 6$.

графите, в които максималната степен на връх е ограничена от константа. Знаем, че $\sum_{u \in V} d(u) = 2m$, което влече $\Delta(G)n \geqslant 2m$, а оттук $m \leqslant \frac{\Delta(G)}{2} n$. Да си припомним, че "$\Delta(G)$" е максималната степен на връх в графа G.

Наблюдение 37: Размер на граф от клас от разредени графи

Ако $\mathfrak{G}$ е клас от разредени графи, за всеки $G \in \mathfrak{G}$ е вярно, че размерът на G е $\Theta(n)$, ако използваме списъчно представяне. Ако използваме матрично представяне, размерът е $\Theta(n^2)$. Ерго, списъчното представяне е особено удачно за разредени графи.

Дали неориентираните графи са всъщност ориентирани? Забележете, че за всеки неориентиран граф, представен чрез списъци, можем да мислим като за ориентиран граф, чиито ребра са "сдвоени" в смисъл, че всяко неориентирано ребро (u, v) **всъщност** е двойка ориентирани ребра (u, v) и (v, u). Ако възприемем тази гледна точка, можем да заключим, че **всъщност** неориентирани графи няма – всички графи са само ориентирани, а понякога на тези със "сдвоените" ребра казваме "неориентирани графи". От тази гледна точка, понятието "неориентиран граф" е само метафора, или начин за удобно изразяване, докато на ниско ниво графите са само ориентирани. Ако гледаме само представянето със списъци, това е точно така.

Ние обаче **няма** да възприемаме тази гледна точка. Това, че представяме неориентираните графи като ориентирани със сдвоени ребра, съвсем не означава, че всъщност неориентирани графи няма. Ето някои от проблемите, до които би довело решението да отречем съществуването на неориентирани графи.

- Веднага ще следва, че няма нетривиални ациклични неориентирани графи – ако има поне едно неориентирано ребро, то всъщност е двойка ребра, които образуват ориентиран цикъл. Тогава понятието "дърво" се обезсмисля. Но интуитивно е ясно, че има гигантска разлика между "истински" неориентиран цикъл и "ерзац" цикъл от сдвоени ориентирани ребра. И че "дърво" е смислено и важно понятие.

- Алгоритми като DFS, които класифицират ребрата на графа по някакъв начин, ще класифицират двете ориентирани ребра (u, v) и (v, u) в различни класове. Но—ако става дума за неориентиран граф—бихме искали това да е само едно ребро и то да бъде класифицирано в един клас.

- Някои алгоритмични задачи, които по принцип са върху ориентирани графи, може да бъдат разгледани и върху неориентирани графи в някои случаи. Например, Обхождане на граф или Най-къс път в граф[†]. Но задачите върху неориентирани графи като Минимално покриващо дърво или Върхово покриване или Доминиращо множество са задачи именно върху неориентирани графи. Те или не може да се обобщят естествено върху ориентирани графи, или обобщенията им върху ориентирани графи имат решения със съвсем друга сложност. Например, обобщението на Минимално покриващо дърво върху ориентирани графи е **NP**-пълната задача Minimum Equivalent Digraph[‡] [31, стр. 65].

9.2 Обхождания на графи

Обхождането на графи е базова изчислителна задача. На английски терминът е *graph traversal*. Понякога се използва и *graph search*, но това звучи прекалено близо до *graph searching*, което означава съвсем друга задача. Много неформално казано, обхождането на граф се извършва от някакво същество, което живее в графа; тръгвайки от **даден начален връх**, то иска да да се разходи из графа по такъв начин, че да обиколи максимална част (подграф) на графа.

Съществото обхожда както върхове, така и ребра. Формулировката на задачата, която казва, че биват обхождани върхове, без да се споменават ребра, е **некоректна**. Прочее, ако

[†]Както ще видим в Лекция 12, Най-къс път в граф върху неориентирани графи е безсмислена при наличие на отрицателни тегла, докато върху ориентирани графи има смисъл дори при отрицателни тегла, стига да няма отрицателни цикли.

[‡]MED е дефинирана за ориентирани графи, които не са тегловни. Ясно е, че ако добавим и тегла, задачата няма да стане по-лесна.

няма изолирани върхове, достатъчно е да кажем, че се обхождат ребрата на графа; при липса на изолирани върхове обхождането на ребрата влече и обхождане на върховете.

Задачата се формулира за ориентирани мултиграфи с възможни примки. Това, че ще я разглеждаме и отделно като задача за неориентирани мултиграфи, не променя факта, че общата ѝ формулировка е за ориентирани мултиграфи. Паралелните ребра и примките имат значение: те трябва да бъдат обходени, всяко/всяка поотделно.

Удачно е да мислим за графа като за *лабиринт*, състоящ се от *стаи* и *коридори*, свързващи стаи; даден коридор може да свързва стая със себе си и такива коридори отговарят на примките в графа. Коридорите са еднопосочни, ако графът е ориентиран, или двупосочни, ако е неориентиран. Съществото иска да обиколи максимална част от лабиринта, минавайки през всеки коридор и посещавайки всяка стая, без да пропуска коридори/стаи и без да "зацикля".

За да се избегне зацикляне се използва *маркировка*. Противно на житейската интуиция, според която съществото би маркирало коридорите, за да знае къде вече е било и да не зацикля, в алгоритмите-решения на тази задача се маркират **върховете**.

- В простия вариант всеки връх има две състояния, иначе казано, бива маркиран по точно един от следните два начина: **посетен** и **непосетен**. В началото всички върхове са непосетени, с изключение на стартовия връх; в неформалното обяснение с лабиринта, всички стаи в началото са непосетени с изключение на стартовата стая.

  Можем да мислим, че се използват *цветове*, като непосетените стаи/върхове са бели, а посетените, черни. В началото всички стаи без стартовата са *бели*, а стартовата е *черна*. След това, когато съществото посети бяла стая, то разбира, че не е било в нея досега, и я маркира, боядисвайки я в черно, за да знае, че вече е било в нея при повторно попадане.

- В по-изтънчените решения се ползат три цвята, тоест, всяка стая има три състояния. Белите, както и в другия вариант, са непосетените. Тези, които са посетени, но с тях не е приключено, са *сиви*. Тези, с които е приключено, са черни. Да не бъде приключено с посетена стая означава, че от нея излизат коридори, по които (може би) не сме минавали. Със стая приключваме, когато сме убедени, че от нея не излизат коридори, по които не сме минавали.

Защо маркираме върховете, а не ребрата? За да пестим памет. За да маркираме върховете ни е необходима и достатъчна $\Theta(\mathtt{n})$ памет, тъй като за всеки връх има $\Theta(1)$ различни възможности (бял, сив и черен, или само бял и черен в по-простия вариант). За да маркираме ребрата ни е необходима и достатъчна $\Theta(\mathtt{m})$ памет, дори да ползваме само един бит за ребро. Както вече видяхме, $\mathtt{m}$ може да е квадратично по-голямо от $\mathtt{n}$ при графи, които не са мултиграфи, а върху мултиграфи $\mathtt{m}$ може да е произволно по-голямо от $\mathtt{n}$.

Аналогията с лабиринта може да е подвеждаща. За реално обхождане на физически лабиринт, реализирано по BFS или DFS идеята, ще е необходимо да се измине много по-голямо разстояние (заради повтаряне на коридори), отколкото налагат нашите BFS и DFS. За пълна аналогия с обхождане на лабиринт трябва да допуснем, че съществото може да се телепортира от стая в друга стая, но само при условие, че вече е било в другата стая. Такова обхождане много точно съответства на обхождането, което следва от BFS и DFS.

В неориентираните графи има една особеност: всяко ребро се обхожда по точно два пъти, дори да е примка. Причината е, че маркираме не ребрата, а върховете. В аналогията с лабиринта, ходейки по някакъв коридор, ние няма как да знаем, че сме били в него. Това, че минаваме за втори път по даден коридор става ясно едва когато излезем от него и установим,

че сме попаднали в сива или черна стая. При ориентираните графи ребрата се обхождат точно по веднъж.

Какво обхождаме в неориентиран граф, стартирайки от даден връх. Ако графът, мултиграф или не, е неориентиран, при обхождане, започващо във връх u, обхождаме точно свързаната компонента, на която принадлежи u. Както знаем, всеки връх в неориентиран граф се намира в точно една свързана компонента, а обхождането няма как да "излезе" от нея. За да обходим целия граф се налага да рестартираме обхождането върху всяка свързана компонента.

Какво обхождаме в ориентиран граф, стартирайки от даден връх. Ако обаче графът е ориентиран, въпросът какво точно ще обходим, стартирайки от връх u, е по-триков. Ако е силно свързан, задължително ще го обходим целия. Ако всяка от слабо свързаните му компоненти е силно свързана, задължително ще обходим цялата слабо свързана компонента, на която принадлежи u. Ако обаче слабо свързаната компонента, на която принадлежи u, не е силно свързана, може:

- в един екстремен случай, да я обходим цялата,
- в друг екстремен случай, да обходим само u, без да може да излезем от него,
- и в общия случай, да обходим само част от нея. Естествено, не е възможно да се прехвърляме от слабо свързана компонента в друга слабо свързана компонента, понеже няма ребро от никой връх на едната от тях до никой връх на другата, но каква част от слабо свързаната компонента, на която принадлежи u, е силно зависещо от конкретиката на графа.

С извинение за тавтологичния изказ: стартирайки от u, ще обходим точно този подграф, който е достижим от u. Следното определение прецизира това.

Определение 57: достижим подграф

Нека G е ориентиран мултиграф с възможни примки и u е връх в него. *Подграфът на* G*, достижим от* u е подграфът на G, индуциран от $\{v \in V(G) \mid$ съществува път от u до $v\}$.

9.3 Обща схема за обхождане

Ще опишем общата схема, която следват алгоритмите за обхождане, които ще разгледаме. За ограничените цели на тази секция, върховете имат само две състояния: необходен и обходен. Белите върхове са необходените, черните са обходените. Схемата е една и съща за всички видове графи, които разглеждаме, така че допускаме, че е даден ориентиран мултиграф с възможни примки $G = (V, E)$. Даден е и стартов връх u.

Идея за обхождане. В началото всички върхове са бели с изключение на u. Поддържаме множество от върхове S. В началото $S = \{u\}$. Обхождането е итеративно, като на всяка итерация изваждаме един връх от S и го слагаме в променлива (тип връх) x. След това разглеждаме всички ребра, излизащи от x, което означава—при списъчно представяне на графа—че минаваме през списъка на x. За всяко ребро, да го наречем (x, y):

- ако връх y е бял, правим y черен и го слагаме в S,
- в противен случай прескачаме y.

Това приключва, когато S стане празно.

Самото обхождане на ребрата става при тези минавания през списъците. Ако беше даден лабиринт и трябваше да намерим нещо в коридорите му—изхода, някакво съкровище или *Минотавъра*—щяхме да го сторим по този начин.

Доказателство за коректност на идеята за обхождане. Ще покажем, не особено прецизно, че тази идея (защото това е прекалено общо описание, за да бъде алгоритъм) работи. Обхождането може да сбърка по два начина: да зацикли или да пропусне. Нашето обхождане не зацикля, защото в S влизат само бели върхове, които веднага с влизането стават черни; веднъж станали черни, те остават черни. Ще покажем, че не може и да пропусне. Без ограничение на общността, нека (мулти)графът е силно свързан[†]. Да допуснем, че поне един връх v остава бял, тоест, непосетен, след края на обхождането. Но в графа има ориентиран път p от u до v. В края на алгоритъма u е черен, защото бива направен черен поначало, а v е бял по допускане. Всеки връх от p е или бял, или черен. Щом началото на p е черен връх, а краят му е бял връх, то в p има върхове a и b, такива че a е родител на b и a е черен и b е бял. Очевидно, в даден момент от обхождането, a е бил изваден от S (щом $S = \varnothing$ накрая) и всички деца на a са били направени черни и сложени в S. Тогава няма как b да е бил пропуснат и да е останал бял. ↯

Ако множеството S бъде реализирано чрез АТД опашка[‡], тази схема за обхождане се превръща в алгоритъма BFS, който ще разгледаме първо. Ако S бъде реализирано чрез стек, получаваме алгоритъм, близък до DFS, който ще разгледаме след BFS.

9.4 BFS

Името на този алгоритъм идва от Breadth-First Search, където "search" е синоним на "traversal". При него, границата между посетените и непосетените върхове—образно и непрецизно казано—расте равномерно във всички посоки спрямо стартовия връх.

Отново допускаме, че е даден ориентиран мултиграф с възможни примки, а някои особености на алгоритъма върху неориентирани (мулти)графи ще разгледаме отделно.

Винаги е даден е стартов връх, който ще наричаме s. Но ако не е вярно, че целият граф е достижим от s, има две възможности.

1. BFS спира след като обходи подграфа, който е достижим от s.
2. След като обходи подграфа, който е достижим от s, BFS бива рестартиран с нов начален връх измежду непосетените досега върхове, и така нататък, докато целият граф бъде обходен.

Приемете, че върховете са числата $1, 2, \ldots, n$.

[†]Ако не е, аргументът се отнася само за подграфа, достижим от u.

[‡]Да си припомним Подсекция 4.4.1. Опашка е именно АТД, защото се дефинира чрез интерфейса си от Enqueue и Dequeue, а имплементацията остава скрита зад този интейрфейс. Аналогично, стек е АТД с интерфейс push и pop.

Цветове на върховете. Всеки връх на графа във всеки момент от работата на BFS има едно от следните три състояния: *непосетен*, *посетен, но неприключен*, и *приключен*, със съответните цветове бял, сив и черен. Тези цветове се реализират от масив $\mathrm{color}[1, \ldots, n]$, където $\mathrm{color}[i]$ е текущият цвят на връх i; $\mathrm{color}[i] \in \{\mathrm{white}, \mathrm{gray}, \mathrm{black}\}$.

Дърво или гора на обхождането. Ако целият граф е достижим от s или BFS спира изчерпване на достижими от s непосетени върхове (възможност 1), BFS строи *дърво на обхождането* T. Това T е арборесценция (вижте Определение 55) с корен s, чиито върхове са точно върховете, достижими от s. В T се намират точно тези ребра на G, с които BFS открива бели върхове.

Ако BFS се рестартира, докато не обходи целия граф (възможност 2), BFS строи колекция от арборесценции $T_1, \ldots, T_k$, по една за всяко рестартиране, като корените им са стартовите върхове на всяко от пусканията на BFS и всеки връх на графа е в точно една от $T_1, \ldots, T_k$. Казваме, че колекцията $\{T_1, \ldots, T_k\}$ е *гората на обхождането*.

В алгоритъма ни дървото или гората на обхождането се реализира с масив на предшествията $\pi[1, \ldots, n]$, в който за всеки връх i, който не е корен, съответният елемент $\pi[i]$ е родителят на i; ако i е корен, то $\pi[i] = \mathrm{Nil}$.

Разстояния спрямо стартовия връх. Във възможност 1, BFS може да изчисли разстоянията от s до всеки друг връх. Забележете изказа с "от-до". Става дума за разстояния в ориентирания смисъл.

Определение 58: разстояние в ориентирания смисъл

Нека G е ориентиран граф. Дали има или няма паралелни ребра или примки е без значение. За всеки два върха $u, v \in V(G)$, *разстоянието в ориентирания смисъл от* u *до* v, което означаваме с $\delta(u, v)$, е дължината на най-къс ориентиран път от u до v, ако такъв има, или ∞, ако такъв няма. Ако от контекста е ясно, че разстоянието е в ориентирания смисъл, казваме кратко *разстоянието от* u *до* v.
Ако е даден и някакъв подграф H на G, то с $\delta_H(u, v)$ означаваме разстоянието от u до v в H. Това е дължината на най-къс ориентиран път от u до v, състоящ се само от ребра от $E(H)$. Ако такъв няма, то $\delta_H(u, v) = \infty$.

Очевидно е, че $\delta(u, v) \leqslant \delta_H(u, v)$.

Съществената разлика между разстоянието в ориентирания смисъл и разстоянието в неориентирания смисъл (в неориентирани графи) е, че разстоянието в ориентирания смисъл не е непременно симетрично. Тривиално е да се измислят примери, в които $\delta(u, v) \neq \delta(v, u)$, като дори може едното да е число, а другото да е ∞. Поради това Определение 58 ползва изказа "от-до", а не "между", който ползваме при симетричните разстояния в неориентирания смисъл.

Нататък ще покажем, че BFS наистина изчислява разстоянията в ориентирания смисъл. Тук само отбелязваме, че BFS записва тези разстояния в масив $d[1, \ldots, n]$, където $d[i]$ след терминирането е равно на $\delta(s, i)$, за всеки връх i.

Опашката от сивите върхове. Ще разгледаме итеративен BFS, в който опашка Q съдържа текущите сиви върхове, а на всяка итерация се изважда един връх x от Q с $\boxed{\mathrm{dequeue}(Q)}$—стига Q да не е празна преди това—и се обхождат всички излизащи от x ребра, като при това всеки открит бял връх y, дете на x, бива направен сив и вкаран в Q с $\boxed{\mathrm{enqueue}(Q, y)}$. Алгоритъмът спира, когато Q стане празна. Очевидно интерфейсът на абстрактния тип данни опаш-

ка, за целите на нашия алгоритъм, се състои от трите функции isempty(Q), enqueue(Q, y) и dequeue(Q), като имената са индикативни за работата им.

9.4.1 Псевдокод на BFS от един стартов връх

BFS(G = (V, E): ориентиран мултиграф с възможни примки, като V = {1, . . . , n}; s ∈ V)

```
 1  for i ← 1 to n
 2      color[i] ← white
 3      d[i] ← ∞
 4      π[i] ← Nil
 5  color[s] ← gray
 6  d[s] ← 0
 7  създай опашка Q
 8  enqueue(Q, s)
 9  while not isempty(Q) do
10      x ← dequeue(Q)
11      for y ∈ adj[x]
12          if color[y] = white
13              color[y] ← gray
14              d[y] ← d[x] + 1
15              π[y] ← x
16              enqueue(Q, y)
17      color[x] ← black
```

Коректност на BFS. Това, че BFS терминира върху всеки вход, е очевидно: нови върхове влизат в опашката само ако са бели, веднъж влезли в опашката, те вече не са бели, а не-бял връх не може да стане бял; тъй като на всяка итерация се вади връх от опашката, рано или късно тя ще стане празна и BFS ще спре.

В Теорема 50 и помощните леми, нека G = (V, E) е произволен ориентиран мултиграф и s е произволен връх в него. Нека G′ е подграфът на G, достижим от s.

Лема 50

При терминирането на BFS(G, s) няма сиви върхове.

Доказателство: Следното твърдение е инвариант за **while** цикъла (редове 9–17).

Инвариант 21: Цикълът на BFS (1)

Всеки път, когато изпълнението е на ред 9, множеството от сивите върхове е точно множеството от върховете в опашката Q.

База. При първото достигане на ред 9 твърдението е вярно, понеже на ред 7 е създадена празна опашка, а на ред 8 в нея е сложен (само) връх s. От друга страна, в този момент s е единственият сив връх.

Поддръжка. Да допуснем, че твърдението е в сила в даден момент, в който изпълнението е на ред 9 и **while** ще бъде изпълнен поне още веднъж. Последното влече, че Q не е празна.

По допускане, всички сиви върхове са в Q и в Q има само сиви върхове. Един сив връх бива изваден от Q на ред 10, но този връх става черен на ред 17. По време на изпълнение на тялото на цикъла, всяко бяло дете на този връх става сиво (ред 13), но след това то влиза в опашката на ред 16. Очевидно е, че при следващото достигане на ред 9 твърдението е вярно.

Терминация. При последното достигане на ред 9 опашката Q е празна. Това означава, че няма сиви върхове. □

Доказателството на Лема 51 е практически същото като доказателството за коректност на идеята на обхождането на стр. 372.

Лема 51

При терминирането на BFS(G, s):

- за всеки $v \in V(G')$ е вярно, че $\text{color}[v] = \text{black}$ и $d[v] < \infty$;
- за всеки $v \in V \backslash V(G')$ е вярно, че $\text{color}[v] = \text{white}$ и $d[v] = \infty$.

Доказателство: Разглеждаме момента, в който BFS терминира. От Лема 50 знаем, че при терминирането на BFS сиви върхове няма. Очевидно е, че белите върхове са точно тези, чиято d-стойност е ∞, а черните са тези, чиято d-стойност не е ∞. Очевидно е също така, че всички върхове от $V \backslash V(G')$ са бели. Остава да докажем, че всички върхове от $V(G')$ са черни.

Да допуснем противното. Тогава съществува връх $u \in V(G')$, който е бял в разглеждания момент. Щом u е връх в G', съществува път p от s до u. Очевидно $u \neq s$, тъй като в този момент s е черен. Щом началото на p е черен връх, а краят му е бял връх, съществува ребро (a, b) в p, такова че a е черен и b е бял. Но щом a е черен, той е станал черен на ред 17 в някакъв момент t, в който променливата x е имала съдържание a. Преди a да стане черен, всяко негово дете y е било "прегледано" на ред 11. Но b е дете на a, щом има ребро (a, b), така че в някой момент $\hat{t} < t$ е било вярно, че $y = b$. Тъй като цветът на $y = b$ е бил бял и в момента $\hat{t}$, булевото условие на ред 12 е било TRUE и връх b трябва да е станал сив на ред 13. Веднъж престанал да бъде бял, той няма как да се окаже бял при терминиране на BFS. ↯

Лема 52

При терминирането на BFS(G, s), за всеки $v \in V$ е вярно, че $d[v] \geqslant \delta(s, v)$.

Доказателство: Ще докажем твърдението само за върховете от $V(G')$, тъй като онези от $V \backslash V(G')$ не са достижими от s, така че $\forall V \backslash V(G') : d[v] = \infty$ в края на алгоритъма, а $\delta(s, v) = \infty$ по дефиниция. Следното твърдение е инвариант за **while** цикъла (редове 9–17).

Инвариант 22: Цикълът на BFS (2)

Всеки път, когато изпълнението е на ред 9, за всеки връх $v \in V(G')$ е вярно, че $d[v] \geqslant \delta(s, v)$.

База. При първото достигане на ред 9 разглеждаме поотделно s и останалите върхове.

- Да разгледаме s. Тъй като $d[s] = 0$ (ред 6) и $\delta(s, s) = 0$ (от теорията на графите), неравенството $d[s] \geqslant \delta(s, s)$ е в сила. ✓

- Да разгледаме $V \setminus \{s\}$. За всеки $v \in V \setminus \{s\}$ е вярно, че $d[v] = \infty$ заради присвояването на ред 3. Тогава $d[v] \geqslant \delta(s, v)$ независимо от това дали v е достижим от s или не; с други думи, дали $\delta(s, v)$ е различна от или равна на ∞.

Поддръжка. Да допуснем, че твърдението е в сила в даден момент, в който изпълнението е на ред 9 и **while** ще бъде изпълнен поне още веднъж. Последното влече, че Q не е празна.

На ред 10 изваждаме връх x от непразната опашка Q. Ефектът от работата на редове 11–17 очевидно е следният:

- всяко бяло дете y на x бива "обработено" на редове 13–16. $d[y]$ получава стойност $d[x]+1$ на ред 14. По допускане, в този момент е вярно, че

  $$d[x] \geqslant \delta(s, x)$$

  Оттук

  $$d[x] + 1 \geqslant \delta(s, x) + 1$$

  което е същото като

  $$d[y] \geqslant \delta(s, x) + 1$$

  Но $\delta(s, x) + 1 \geqslant \delta(s, y)$, понеже, щом има път p от s до x, има и път от s до y с дължина $|p| + 1$[†]. Тогава

  $$d[y] \geqslant \delta(s, y)$$

  Доказахме, че за всеки връх y, който е бил "обработен" при изпълнение на текущата итерация, $d[y] \geqslant \delta(s, y)$.

- Сивите и черните деца на x не биват "обработвани" и техните d-стойности не биват променяни в тялото на цикъла (редове 13–16), така че за тях неравенството остава в сила. Аналогично, неравенството остава в сила за върховете на G', които не са деца на x.

Терминация. При последното достигане на ред 9 за всеки $v \in V(G')$ е вярно, че $d[v] \geqslant \delta(s, v)$, което и трябваше да покажем. □

Лема 53 казва, че във всеки момент от изпълнението на BFS, в опашката има върхове или с една и съща d-стойност, или с точно две различни, но съседни по големина, d-стойности. С други думи, опашката "обработва" върховете по d-стойности. Което, както ще видим в Теорема 50, е същото като да "обработва" върховете по ориентираните разстояния от s.

Лема 53: (Lemma 22.3 в [21, стр. 599])

Нека в произволен момент t от работата на $\mathrm{BFS}(G, s)$, опашката Q съдържа следната пермутация на върхове: $\langle u_1\, u_2 \cdots u_k \rangle$, като u_1 е най-отдавна сложеният връх, а u_k е най-скоро сложеният връх. Тогава

$$d[u_1] \leqslant d[u_2] \leqslant \cdots \leqslant d[u_k] \leqslant d[u_1] + 1$$

[†] В [21] това очевидно твърдение е Lemma 22.1 на стр. 598.

Доказателство: Ще докажем лемата по индукция по броя на достиганията на ред 9. Няма смисъл да формулираме инвариант, понеже не се интересуваме от крайното състояние на нещата при терминирането, а разглеждаме произволен момент от изпълнението – така че формулировката на лемата е инварианта.

Базата на индукцията е първото достигане на ред 9. В този момент опашката съдържа само s и твърдението със сигурност е вярно.

Разглеждаме произволно достигане на ред 9, което не е последното. Нека това е момент t от изпълнението на BFS. Щом това достигане на ред 9 не е последното, опашката Q не е празна в момент t. Да кажем, че Q съдържа пермутацията на върхове $\langle u_1\, u_2 \cdots u_k \rangle$. На ред 10 връх u_1 бива изваден от Q и сложен в променливата x. Вече опашката съдържа пермутацията $\langle u_2\, u_3 \cdots u_k \rangle$, така че неравенствата по отношение на върховете в опашката стават

$$d[u_2] \leqslant \cdots \leqslant d[u_k] \leqslant d[u_1] + 1$$

Имайки предвид това, че $d[u_1] \leqslant d[u_2]$, имаме $d[u_1] + 1 \leqslant d[u_2] + 1$, откъдето следва, че

$$d[u_2] \leqslant \cdots \leqslant d[u_k] \leqslant d[u_2] + 1$$

Ерго, твърдението остава в сила веднага след изваждането на връх от опашката.

Да се убедим, че твърдението остава в сила след вкарването на връх в опашката във вътрешния цикъл (редове 11–16). Разглеждаме произволен връх u', който е дете на текущия x (ред 11) и е бял. Помним, че $x = u_1$. Рано или късно, променливата y получава съдържание u' и този връх бива сложен в опашката на ред 16 в някакъв момент $\hat{t} > t$. Преди да бъде сложен в опашката, връх u' получава d-стойност на ред 14. А именно, $d[u'] \leftarrow d[u_1] + 1$. Текущият първи връх в опашката е u_2, а вече видяхме, че $d[u_1] + 1 \leqslant d[u_2] + 1$. Оттук следва, че $d[u'] \leqslant d[u_2] + 1$. Но съдържанието на опашката в момент $\hat{t}$ е $\langle u_2\, u_3 \cdots u' \rangle$, така че твърдението остава в сила.

Очевидно твърдението остава в сила дори когато не се добавят нови върхове в Q, а също така и когато Q е празна; ако Q е празна, твърдението е вярно в празния смисъл (vacuously). □

Теорема 50: Коректност на BFS (Theorem 22.5 в [21, стр. 599])

След терминирането на BFS(G, s):

1. Обходеният подграф на G е точно G'.
2. За всеки връх $v \in V(G')$, $d[v] = \delta(s, v)$.
3. Дървото на обхождането, реализирано чрез $\pi[1, \ldots, n]$, е дърво на най-къси пътища за G' с корен s.

Доказателство: Твърдение 1 следва директно от Лема 51.

Ще докажем 2. Да допуснем, че съществува връх $u \in V(G')$, такъв че $d[u] \neq \delta(s, u)$ след терминирането на BFS(G, s). Нека v е такъв връх, като освен това стойността $\delta(s, v)$ е минимална. Очевидно $v \neq s$. Съгласно Лема 52, $d[v] \geqslant \delta(s, v)$. От това и допускането, че $d[v] \neq \delta(s, v)$ следва, че $d[v] > \delta(s, v)$.

Щом $v \in V(G')$, съществува път p от s до v с дължина поне единица. БОО, нека p е път от s до v с минимална дължина. Тогава $|p| = \delta(s, v)$. Нека w е родителя на v в p. Нека подпътя на p от s до w се казва q. Забележете, че $|p| = |q| + 1$.

Известно е, че най-къс път се състои от най-къси подпътища (Теорема 66). Тогава q е най-къс път от s до w, така че $|q| = \delta(s,w)$. От това следва, че $\delta(s,v) = \delta(s,w) + 1$.

Но $\delta(s,v) > \delta(s,w)$ и v е избран като връх-нарушител с минимална δ-стойност. Тогава w не е "нарушител"; тоест, $d[w] = \delta(s,w)$. Изведените дотук факти изглеждат така на един ред:

$$d[v] > \delta(s,v) = \delta(s,w) + 1 = d[w] + 1 \tag{9.1}$$

Нека t е моментът от работата на BFS, в който w бива изваден от опашката (ред 10). С други думи, променливата x получава съдържание w в момент t. Следните възможности за цвета на v в момент t са изчерпателни.

color$[v]$ = **white** Тогава при изпълнението на **for**-цикъла (редове 11–16) в даден момент $\widehat{t} > t$, променливата y ще получи съдържание v. При присвояването на ред 14 с $y = v$ е изпълнено $d[v] = d[w] + 1$. Но това е в противоречие с (9.1). ↯

color$[v]$ = **gray** В такъв случай v е бил оцветен в сиво на някоя предишна итерация на **while**-цикъла, когато върха в променливата x е бил някой $a \in V(G')$, като v е бил y и е получил d-стойността си като $d[v] \leftarrow d[a] + 1$. Но този a влиза в опашката преди w, от което следва, съгласно Лема 53, че $d[a] \leqslant d[w]$, което е същото като $d[a] + 1 \leqslant d[w] + 1$. Тогава $d[v] \leqslant d[w] + 1$, в противоречие с (9.1). ↯

color$[v]$ = **black** Щом v е черен, той вече е бил в Q и е бил изваден оттам. Тогава v бива изваден от Q преди w, което влече, че v влиза в Q преди w. Съгласно Лема 53, $d[v] \leqslant d[w]$, в противоречие с (9.1). ↯

Ще докажем 3. Нека дървото на обхождането, реализирано чрез масива на родители $\pi[1, \ldots, n]$, се казва T. Твърди се, че за всеки връх $v \in V(G')$, $\delta(s,v) = \delta_T(s,v)$. Тривиално е да докажем това по индукция по реда на влизане на върховете в опашката.

Базата е за $v = s$ и твърдението очевидно е вярно. Нека $v \neq s$. v получава своите d- и π-стойности на някоя итерация на на **while**-цикъла, променливата y има стойност v, а променливата x има някаква стойност u, където u е родител на v в G' и е родителят на v в T. А именно, $d[v]$ става $d[u] + 1$ и $\pi[v]$ става u. Тъй като u влиза в опашката преди v, по индукционното предположение, пътят от s до u в T е най-къс път от s до u в G. Имайки предвид това и факта, че $\delta(s,v) = \delta(s,u) + 1$, виждаме, че след присвояването $\pi[v] \leftarrow u$, масивът π реализира най-къс от s до v, така че $\delta(s,v) = \delta_T(s,v)$. □

Сложност по време. Реализацията на BFS, която разгледахме, работи във време $\Theta(n + m)$, ако мултиграфът е реализиран чрез списъци на съседство. Както в много други алгоритми върху графи, които ще разгледаме, сложността по време **НЕ СЕ ОПРЕДЕЛЯ** така (допускаме, че $G' = G$):

> **while**-цикълът се изпълнява n пъти, по веднъж за всеки връх, а всяко изпълнение отнема време, в най-лошия случай, $\Theta(m)$, така че сложността е $\Theta(nm)$.

Това разсъждение в общия случай е **ГРЕШНО**. От това разсъждение следва асимптотична горна граница $O(nm)$, но, в общия случай, тя не е точна. Да, има безкрайни класове графи, за които това разсъждение е приложимо. Примерно, клас графи, в който всеки граф има един и същи брой ребра; тоест, $m = \Theta(1)$. Тогава наистина сметката е такава. Но в общия случай това не е така.

Прецизното разсъждение, което ни дава точна асимптотична оценка винаги, е да се игнорира **while**-цикълът и да се фокусираме върху вътрешния цикъл, тоест, върху **for**-цикъла. Но не върху всички изпълнения на **for**-цикъла в рамките на едно изпълнение на **while**-цикъла, а върху всички изпълнения на **for**-цикъла по време на цялата работа на BFS. Тогава виждаме, че променливата y минава систематично през всички елементи на всички списъци на съседство. Размерът на списъците на съседство е $\Theta(n + m)$, а алгоритъмът върши само $\Theta(1)$ работа за всеки списъчен елемент. Оттук веднага получаваме точна асимптотична оценка $\Theta(n + m)$ за сложността по време на BFS.

Ако G не е мултиграф, може да разсъждаваме така:

> **while**-цикълът се изпълнява n пъти, по веднъж за всеки връх, а всяко изпълнение отнема време, в най-лошия случай, $\Theta(n)$, така че сложността е $\Theta(n^2)$.

Това разсъждение вече не е грешно. Наистина, най-лошият случай, в който $m = \Theta(n)$, е точно такъв. Но $\Theta(n + m)$ остава по-прецизна оценка, тъй като функцията в израза е на две променливи и това е по-информативно за общия случай.

И така, сложността по време на BFS е $\Theta(n + m)$.

9.4.2 Псевдокод на BFS, обхождащ целия граф

Следва псевдокод на BFS, който обхожда целия граф. Отново графът на входа е ориентиран мултиграф с възможни примки. Този път стартов връх няма.

Ако мултиграфът е неориентиран, променливата count се инкрементира точно веднъж за всяка свързана компонента, така че алгоритъмът връща броя на свързаните компоненти. Тривиално е да се добави код, така че алгоритъмът да връща самите свързани компоненти, а не само броя им.

```
BFS-MOD(G = (V, E), като V = {1, ..., n})
 1  for i ← 1 to n
 2      color[i] ← white
 3      d[i] ← ∞
 4      π[i] ← Nil
 5  count ← 0
 6  for i ← 1 to n
 7      if color[i] = white
 8          count++
 9          BFS-VISIT(G, i)
10  return count
```

```
BFS-VISIT(G = (V, E); s ∈ V)
 1  color[s] ← gray
 2  d[s] ← 0
 3  създай опашка Q
 4  enqueue(Q, s)
 5  while not isempty(Q) do
 6      x ← dequeue(Q)
 7      for y ∈ adj[x]
 8          if color[y] = white
 9              color[y] ← gray
10              d[y] ← d[x] + 1
11              π[y] ← x
12              enqueue(Q, y)
13      color[x] ← black
```

Аргументация за коректност няма да правим. Очевидно сложността е $\Theta(n + m)$.

9.4.3 Приложения на BFS

Изчисляване на двуделност на неориентиран граф Нека $G = (V, E)$ е обикновен граф. БОО, нека G е свързан. Пита се дали G е двуделен; алтернативно, дали е 2-оцветим,

което е същото или почти същото нещо, в замисимост от формалните дефиниции[†]. Знаем необходимо и достатъчно условие за това: да няма нечетни цикли. Това условие обаче не се превежда директно в ефикасен алгоритъм. Циклите на графа може да са суперекспоненциално много в n, така че, дори да имаме алгоритъм, намиращ всеки цикъл във време $\Theta(1)$ (в обикновения или амортизирания смисъл), той би бил неприемливо бавен за определяне на двуделността.

Оптимално ефикасен, в асимптотичния смисъл, е алгоритъм, базиран на BFS. Ако изобщо G е 2-оцветим[‡], да кажем в зелено и червено, то има точно две възможности:

- s е червен, неговите съседи са зелени, техните съседи без s са червени, и така нататък,
- и обратно, s е зелен, неговите съседи са червени, техните съседи без s са зелени, и така нататък.

Виждаме, че разбиването на V е едно и също и при двете схеми за оцветяване – те само разменят цветовете на двата дяла. Съществото е, че двата дяла може да дефинират по отношение на разстоянието (сега говорим за обикновено симетрично разстояние, понеже G е неориентиран) между s и всеки друг връх – върховете на четно разстояние са единият дял, примерно червените, а тези на нечетно разстояние са другия дял, в случая зелените.

Това веднага дава идея за опит за оцветяване на произволен свързан граф. Започваме BFS обхождане от произволен връх s и за всяко ребро (u, v) следим дали двата края са на различно разстояние от s, или не. Забележете, че е невъзможно разстоянието между u и s да се различават на повече от единица от разстоянието между v и s. Ерго, тези разстояния или се различават на единица, което означава, че са с противоположна четност, което означава, че u и v са в различни цветове, или са равни, при което u и v са в един и същи цвят. Ако при обхождането се окаже, че съществува ребро (u, v), такова че $d[u] = d[v]$, алгоритъмът връща НЕ. В противен случай връща ДА.

Тривиално лесно е да се модифицира кода на BFS съгласно тази идея, така че сложността по време да остане $\Theta(n + m)$.

9.5 DFS

9.5.1 Неформално въведение и сравнение с BFS

DFS реализира, в някакъв смисъл, обратната идея на BFS обхождането. BFS обхождането е "предпазливо". Ако си представим обхождане на лабиринт, BFS идеята е, грубо казано, такава: има стаи, в които сме били, но с които не сме приключили; ние първо преглеждаме всички коридори, излизащи от тях, преди да продължим с новите стаи, като новите стаи са тези, които сме открили чрез въпросните коридори.

DFS обхождането е "смело". От началната стая излизаме през първия възможен коридор, от следващата също, и така нататък, като надлежно маркираме стаите, в които вече сме били, за да не зациклим. Рано или късно ще се окажем в стая, в която сме били. Тогава се връщаме колкото е възможно по-малко назад до стая, в която сме били, и опитваме същото нещо, само че излизайки от нея през врата, която не сме използвали. На английски тази идея се нарича *backtracking*[§].

[†]Някои формални дефиниции на "двуделен граф" предполагат, че графът има поне два върха.

[‡]Тези цветове нямат **нищо общо** с цветовете бял, сив и черен, които BFS използва по време на работата си!

[§]Backtracking е алгоритмична схема, също както разделяй-и-владей е алгоритмична схема. В този курс няма да разглеждаме алгоритми, използващи backtracking, освен DFS.

Допълнение 39: Разликата между BFS и DFS, илюстрирана с шах

Ще илюстрираме разликата между двете идеи—BFS и DFS—с шаха. Нека е дадена задача да открием мат в три хода в дадена позиция за белите[a]. Да подходим алгоритмично.

BFS идеята. BFS подходът е да генерираме всички възможни, съгласно правилата на шаха, позиции с един полуход на белите. Нека множеството от тези позиции е P_1. За всяка от тези позиции генерираме всички позиции с един полуход на черните, получавайки множество Q_1: това са всички позиции на един ход. Използвайки Q_1 и всички възможни полуходове на белите, генерираме множеството P_2, а от него с всички възможни полуходове на черните генерираме Q_2: множеството от всички позиции на два хода от началната. От Q_2 с всички възможни полуходове на белите генерираме P_3. Ако задачата е смислена, в една или повече от позициите на P_3 има мат на черния цар, но в P_1 и P_2 няма мат на черния цар. Нещо повече. Трябва:

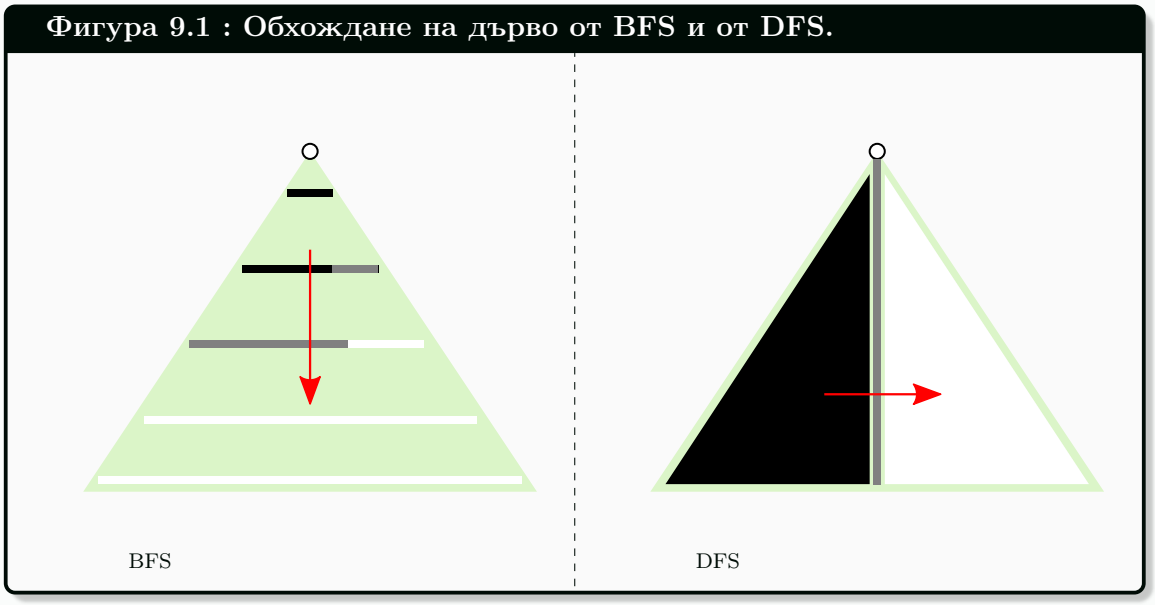
- да **съществува** полуход на белите в началната позиция, такъв че
- **за всеки** полуход на черните след него
- да **съществува** полуход на белите, такъв че
- **за всеки** полуход на черните след него
- да **съществува** полуход на белите, такъв че черният цар е мат.

В тази редица от полуходове се редуват квантори $\exists$ и $\forall$, като $\exists$ са на белите полуходове, а $\forall$, на черните. Тъй като възможните полуходове от дадена позиция може да са десетки, дори при малко фигури, говорим за дърво с разклоненост от порядъка на десетки. Височината му в полуходове е шест. Това е дървото на BFS-а, който току-що си представихме. При BFS идеята генерираме нивàта на дървото систематично от корена към листата.

DFS идеята. DFS подходът е, да опитаме един възможен полуход на белите от началната позиция, после един полуход на черните, и така нататък, докато направим трети полуход за белите. Отчитаме дали черният цар е мат или не, и се връщаме в предната позиция, която е получена при втори полуход за черните. От нея правим следващия възможен трети полуход за белите—ако в предишния трети полуход за белите не е имало мат на черния цар—и така нататък.
След като "влезе" в позиция $\mathtt{p}$, DFS генерира **една след друга** позициите, достижими от нея, като за всяка от тях, да я наречем $\mathtt{q}$, "влиза" в $\mathtt{q}$ и се връща обратно в $\mathtt{p}$ чак след като е изчерпила всички възможности "до долу", за да опита следващата след $\mathtt{q}$, ако има такава. Ако $\mathtt{p}$ не е началната позиция, трябва да се "излезе" от нея, което става чак когато са изчерпани всички възможности за следваща позиция.
Както и при BFS, така и при DFS решението на задачата е полуход на белите, такъв че за всеки полуход на черните има полуход на белите, такъв че за всеки полуход на черните има полуход на белите, в който черния цар е мат. Съществената разлика с BFS е, че BFS генерира цялото P_1, от него цялото Q_1, и така нататък, докато DFS не генерира пълните нивá. DFS генерира елементите на нивата, но последователно, а не всички наведнъж.

Ако си представим търсенето на мат като дърво с корен началната позиция, BFS генерира цялото дърво[б], вървейки вертикално, отгоре надолу, ако коренът е горе. Докато DFS поддържа само един вертикален път от корена до листата, като този път се движи хоризонтално, образно казано. Във всеки момент DFS съхранява в явен вид само позициите от този път, а следващите позиции се пресмятат от текущите. Фигура 9.1 илюстрира разликата в двата подхода. На нея цветовете бял, сив и черен имат смисъла на цветовете на върховете от BFS и DFS и нямат нищо общо с цветовете на шахматните фигури.

Фигура 9.1 : Обхождане на дърво от BFS и от DFS.



[а]Ето сайт с такива *шахматни задачи*.
[б]Почти цялото; може би без върхове от най-долното ниво.

Ако заменим в кода на BFS опашката със стек (съответно и функциите от интерфейса с push и pop), ще получим **нещо като** DFS, но не точно DFS. Нека читателя пробва да замени опшаката със стек в BFS и да се убеди върху малък пример, че полученото обхождане е, в някакъв смисъл, междинен вариант, нито BFS, нито истински DFS обхождане.

9.5.2 Псевдокод на DFS

Следният псевдокод е буквално същият като псевдокода в [21, стр. 604]. Смисълът на цветовете е същият като при BFS. Разстояния сега няма – DFS не пресмята разстояния в графа. В някои приложения на DFS въвеждаме нивá, които се пресмятат като $\mathtt{d}$-стойностите в BFS, но при DFS нивáта нямат смисъл на разстояния.

Използваме глобална променлива time, чрез която пресмятаме времето на откриване на всеки връх и времето на финализиране на всеки връх. Два масива $\mathtt{d}[1,\ldots,n]$ и $\mathtt{f}[1,\ldots,n]$ съхраняват тази информация за върховете.

Разглеждаме вариант на DFS, който задължително обхожда целия граф, подобно на модификацията на BFS на стр. 379. Тривиално лесно е да направим от него DFS, който получава стартов връх като аргумент и обхожда само подграфа, който е достижим от този стартов връх.

```
DFS(G = (V, E), като V = {1, ..., n})
1   time е глобална
2   for i ← 1 to n
3       color[i] ← white
4       π[i] ← Nil
5   time ← 0
6   for i ← 1 to n
7       if color[i] = white
8           DFS-VISIT(G, i)
```

```
DFS-VISIT(G = (V, E); x ∈ V)
1   color[x] ← gray
2   time++
3   d[x] ← time
4   foreach y ∈ adj[x]
5       if color[y] = white
6           π[y] ← x
7           DFS-VISIT(G, y)
8   color[x] ← black
9   time++
10  f[x] ← time
```

Много подробен пример за работата на DFS има в [21, стр. 605]. Забележете, че наредените двойки $(d[v], f[v])$, по всички $v \in V$, напълно определят работата на DFS върху даден граф. Конвенцията на [21] е върховете да се рисуват като елипси, а в елипсата, съответстваща на връх v, наредената двойка $(d[v], f[v])$ да се записва като "$d[v]/f[v]$". Например, в рисунката, показваща края на алгоритъма, връх u е маркиран с "1/8", което означава, че връх u е бил открит в момент 1 и е бил финализиран в момент 8.

Формално доказателство за коректността на DFS няма да правим тук. Аргументацията на стр. 372 за коректността на общата схема е достатъчна. Сложността по време на DFS е $\Theta(n+m)$. Това се извежда със същите разсъждения, с които се извежда линейната сложност на BFS.

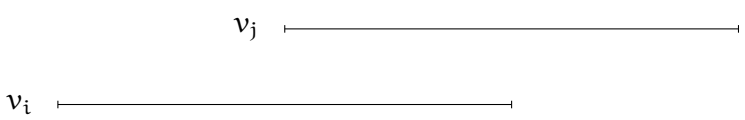
9.5.3 Свойства на DFS

Скобова структура на интервалите, генерирани от DFS. За целите на това изложение, нека върховете са $v_1, \ldots, v_n$. Очевидно е, че променливата time, която се инициализира с 0 на ред 5, приема последователно стойностите $1, 2, \ldots, 2n$, тъй като бива инкрементирана два пъти за всеки връх x в DFS-VISIT. Да си представим числовата ос с тези $2n$ стойности на t. Ако групираме стойностите по двойки за отделните върхове, получаваме n интервала $[d[v_1], f[v_1]], [d[v_2], f[v_2]], \ldots, [d[v_n], f[v_n]]$. Казваме, че това са *интервалите, генерирани от DFS*(G). Краищата на интервалите са, две по две, различни точки, така че всеки два интервала или имат празно сечение, или имат общ подинтервал с дължина поне единица. Фигура 22.5 в [21, стр. 607] илюстрира добре интервалите на DFS.

Теорема 51: Скобова структура на интервалите при DFS

В текущия контекст, ако два различни интервала имат непразно сечение, то единият се съдържа строго в другия: ⊢⊣ .

Доказателство: Твърди се, че е невъзможно да има такива интервали: ⊢⊣ . Да допуснем противното. Нека има върхове v_i и v_j, такива че $d[v_i] < d[v_j] < f[v_i] < f[v_j]$:



DSF първо открива v_i, а после v_j, като v_i не е финализиран в момента на откриването на v_j. Това означава, рекурсивното викане R_i на DFS-VISIT, в което $x = v_i$, не е приключило в момента, в който в някое следващо рекурсивно викане R_j е вярно, че $x = v_j$.

Същото нещо, по-подробно казано, звучи така.

- В рекурсивното викане R_i е вярно, че $x = v_i$ и се изпълнява цикълът на редове 4–7. $d[v_i]$ получава своята стойност на ред 3. За някое дете u' на v_i е вярно, че когато y (в R_i) стане u' се прави рекурсивно викане R' на DFS-VISIT.
- В рекурсивното викане R' е вярно, че $x = u'$ и се изпълнява цикълът на редове 4–7. За някое дете u'' на u' е вярно, че когато y (в R') стане u'' се прави рекурсивно викане R'' на DFS-VISIT.
- И така нататък ...
- В рекурсивното викане R_j е вярно, че $x = v_j$.

Ключовото наблюдение е, че в момента, в който $d[v_j]$ получава своята стойност на ред 3—това става в рекурсивното викане R_j—викането R_i още не е приключило и в R_i се изпълнява цикълът на редове 4–7, така че ред 10 в R_i не е достигнат и $f[v_i]$ все още не е получила стойност. Със сигурност $f[v_i]$ ще получи стойност, но това ще е за стойност на time, по-голяма от стойността на time в момента, в който $d[v_j]$ получава своята стойност. ↯ □

Класифициране на ребрата на ориентиран граф от DFS. Нека $G = (V, E)$ е ориентиран мултиграф с възможни примки. Нека гората на обхождането, реализирана чрез масива π, е A. Да си спомним, че A е множество от дървета. В края на DFS(G) ребрата от E се разбиват[†] на следните четири категории:

дървесни ребра. Това са точно ребрата на A. С други думи, (u, v) е дървесно ребро, ако u и v са върхове от едно и също дърво $T \in A$, като u е родителят на v в T.

ребра назад. (u, v) е ребро назад, ако u и v са върхове от едно и също дърво $T \in A$, като v е предшественик на u в T. Тоест, в T има път от v до u. Примките в G задължително са в тази категория.

ребра напред. (u, v) е ребро напред, ако u и v са върхове от едно и също дърво $T \in A$, като u е предшественик на v в T, но (u, v) не е дървесно ребро. Тоест, (u, v) не е ребро в T.

ребра настрани. (u, v) е ребро настрани, ако u и v не са в отношение на предшествие в A. Това означава, че

- или u и v са върхове от едно и също дърво от A, но нито единият не е предшественик на другия,
- или са върхове от две различни дървета от A.

В [21, стр. 609] съответните термини са *tree edges*, *back edges*, *forward edges* и *cross edges*. Примери за граф и DFS върху него, след който се срещат и четирите вида ребра, има на Фигура 22.4 в [21, стр. 605] и на Фигура 22.5 в [21, стр. 607], като на Фигура 22.5.c е показан и графът, прерисуван като дърветата на обхождането плюс останалите ребра.

[†]Някои дялове-категории на това разбиване може да са празни, така че, формално, правилно е да се каже "се разбиват на най-много четири категории.

Как да разберем по време на работата на DFS дадено ребро от кой вид е? Ребрата на G биват обхождани на ред 4 от DFS-VISIT. Нека реброто е (x, y). Първо да разгледаме възможностите за цвета на y.

- Ако y е бял, то реброто (x, y) е дървесно, понеже условието на ред 5 е TRUE и изпълнението отива на ред 6, където x става родителят на y в дървото; това означава, че (x, y) става ребро от дървото.
- Ако y е сив, то y е предшественик на x в дървото. Приемаме това за очевидно.
- Ако y е черен, то или y е наследник на x и с него вече сме приключили, или y не е в отношение на предшествие с x и с него сме приключили. Тоест, (x, y) е ребро напред или ребро настрани. Ерго, само от черния цвят на y не можем да класифицираме точно (x, y).

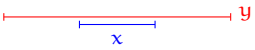
Сега да разгледаме възможностите за взамните разположения на $d[x]$, $d[y]$, $f[x]$ и $f[y]$. Предвид това, че тези са две по две различни, и че $d[x] < f[x]$ и $d[y] < f[y]$, има $3 + 2 + 1 = 6$ възможности от общи комбинаторни съображения. С по-подробен анализ, възможностите стават само три.

1.

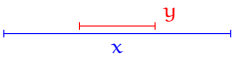
Това е невъзможно. Щом (x, y) е ребро, няма как x да бъде финализиран на ред 10 преди бялото му дете y да бъде открито на ред 5.

2. 

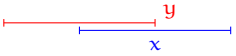
Това е невъзможно съгласно Теорема 51.

3. 

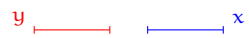
y е предшественик на x. В такъв случай y е сив и (x, y) е ребро назад. Всички примки са такива.

4. 

x е родителят на y. В такъв случай y е бял и (x, y) е дървесно ребро или y е черен и (x, y) е ребро напред.

5. 

Това е невъзможно съгласно Теорема 51.

6. 

Това е възможно. Щом y е финализиран преди откриването на x, нито x е предшественик на y, нито y е предшественик на x. В този случай y е черен и (x, y) е ребро настрани.

Сега е ясно как да различаваме дали (x, y), при черен връх y, е ребро напред или ребро настрани. Ако $d[x] < f[y]$, то (x, y) е ребро напред, ако $f[y] < d[x]$, то (x, y) е ребро настрани.

Следната лема обобщава казаното дотук. Доказателството ѝ са току-що извършените разсъждения.

Лема 54: Възможностите за $d[x]$, $d[y]$, $f[x]$ и $f[y]$ на $(x, y) \in E$ след DFS(G)

Това са всички възможности след DFS(G) за взаимното разположение на $d[x]$, $d[y]$, $f[x]$ и $f[y]$, където $(x, y) \in E$:

1. $d[x] < d[y] < f[y] < f[x]$ тстк (x, y) е дървесно ребро или ребро напред,
2. $d[y] < d[x] < f[x] < f[y]$ тстк (x, y) е ребро назад,
3. и $d[y] < f[y] < d[x] < f[x]$ тстк (x, y) е ребро настрани.

Кладифициране на ребрата на неориентиран граф от DFS. Нека $G = (V, E)$ е неориентиран мултиграф с възможни примки. Нека гората на обхождането, реализирана чрез масива π, е A. В края на DFS(G) ребрата от E се разбиват на следните две категории:

дървесни ребра. Това са точно ребрата на A.

ребра назад. (u, v) е ребро назад, ако u и v са върхове от едно и също дърво $T \in A$, като v е предшественик на u в T. Примките в G задължително са в тази категория.

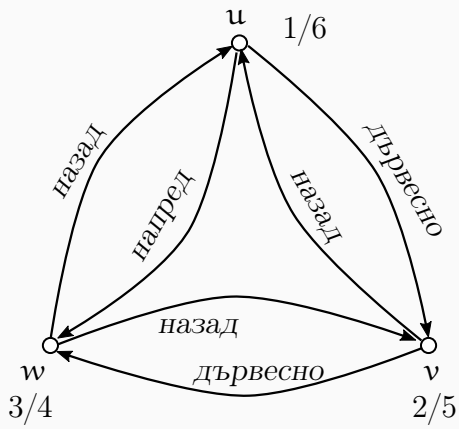
С други думи, в неориентираните графи няма ребра напред или ребра настрани. Общото за ребрата напред или настрани е, че чрез тях откриваме черни върхове. Дали е невъзможно DFS върху неориентирани графи да открива черни върхове? Естествено, че е възможно. Но винаги, когато DFS върху неориентиран граф открие черен връх—тоест, връх y на ред 5 е черен—е вярно, че неориентираното ребро (x, y) **вече** е било обходено от DFS и **вече** е било класифицирано като дървесно ребро или ребро напред. Същността на тази аргументация е в това, че

1. ако гледаме на даден граф със сдвоени (вж. дискусията на стр. 369) ориентирани ребра като на ориентиран граф, DFS може да открие ребра напред или ребра настрани, като всяко такова ребро е от двойка, другото ребро от която е дървесно ребро или ребро назад;

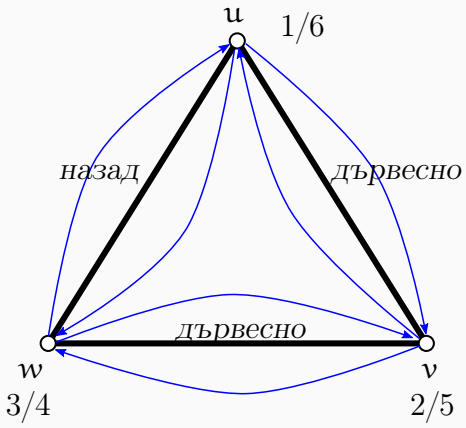
2. но ако гледаме на същия граф като на неориентиран граф, който само е представен като ориентиран—всяко неориентирано ребро (u, v) е представено чрез двойка ориентирани ребра (u, v) и (v, u)—то няма как да класифицираме едно и също неориентирано ребро по два начина, съответно за двете ориентирани ребра, които го представят. И класифицирането на реброто никога не става като ребро напред или настрани.

Разгледайте Фигура 9.2. Тя сравнява резултатите от работата на DFS върху ориентиран и неориентиран граф. Двата графа имат едни и същи списъци на съседства, но графът вдясно се смята за неориентиран и всяка двойка сдвоени ориентирани ребра в представянето (в синьо) отговаря на едно неориентирано ребро на графа (в черно), докато при графа вляво всяко ориентирано ребро в представянето отговаря на точно едно ориентирано ребро на графа. И при ориентирания, и при неориентирания граф има точно две дървесни ребра след приключването на DFS. Забележете, че реброто (u, v) в ориентирания граф е дървесно, докато реброто (v, u) е ребро назад, тоест, ребрата от двойката биват класифицирани по различен начин, докато в неориентирания граф на тях отговаря само едно ребро и то бива класифицирано като дървесно, защото първият път, в който прекосяваме едното от двете отговарящи му ориентирани ребра от представянето (в синьо), откриваме бял връх (v).

Фигура 9.2 : Ориентиран граф със сдвоени ребра и неориентиран граф.



Ориентиран граф със сдвоени ребра.



Неориентиран граф, представен чрез ориентиран граф със сдвоени ребра.

Фигура 9.2 е добра илюстрация на факта, че в неориентиран граф не може да има ребра напред. Сравнете ориентираното ребро напред (u, w) в ориентирания граф с неориентираното ребро (u, w) в неориентирания граф – очевидно е, че при сдвоени ориентирани ребра, ребро може да бъде класифицирано като ребро напред само ако другото ребро от двойката вече е било прекосено и класифицирано като ребро назад. Ако графът е неориентиран и това са само ориентираните ребра от преставянето, очевидно е, че реброто като един обект ще бъде класифицирано съгласно първото прекосяване, а именно като ребро назад. С аналогични разсъждения може да изведем, че в неориентирани графи няма ребра настрани.

Следната теорема дава формално доказателство, че при неориентираните графи ребрата са само дървесни и ребра назад.

Теорема 52: Theorem 22.10 в [21, стр. 610]

При всяка работа на DFS върху неориентиран граф G, всяко обходено ребро е дървесно или ребро назад.

Доказателство: Разглеждаме произволно ребро $(u, v) \in E(G)$. БОО, нека $d[u] < d[v]$. Тогава DFS и открива, и финализира v преди да финализира u; това е очевидно.

Да разгледаме двете възможности за обхождане на реброто (u, v):

1. Ако то бъде обходено в посока от u към v, то чрез него ще открием бял връх (v) и ще го класифицираме като дървесно.

2. Ако то бъде обходено в посока от v към u, то чрез него ще открием сив връх (u) и ще го класифицираме като ребро назад. □

9.5.4 Приложения на DFS

Изследване на цикличност на графи чрез DFS. И за ориентирани, и за неориентирани графи може да отговорим ефикасно на въпроса дали графът е цикличен, използвайки DFS.

Теорема 53: Граф е цикличен тстк DFS открива ребро назад

Нека G е ориентиран или неориентиран граф. G е цикличен тогава и само тогава, когато произволно изпълнение на DFS(G) открива поне едно ребро назад.

Доказателство: В едната посока, да допуснем, че $e = (u, v)$ е ребро назад. Ако G е неориентиран, нека обхождането става в посока от u към v. Както вече видяхме, това означава, че в момента на обхождането, u и v са сиви, като v е предшественик на u в дървото на обхождането. Тогава в дървото има път от v до u (или между v и u, в неориентирания случай), който път заедно с реброто e дава цикъл. Този аргумент е валиден дори когато e е примка; тогава въпросният път се състои от един единствен връх, а именно $u = v$.

В другата посока, нека има цикъл c. Очевидно DFS открива върховете на цикъла последователно, но не в непременно непрекъсната последователност. Нека $v \in V(c)$ е този връх на цикъла, който бива открит последен от DFS. Но v има поне един съседен връх u в c.

1. v има поне два съседни върха u, w, които са от цикъла (ако става дума за мултиграф, може $u = w$; ако c е примка, може дори $v = u = w$; всичко това е без значение за доказателството), ако G е неориентиран;

2. v има поне един съседен връх u, който е от цикъла (възможно е $u = v$, ако реброто е примка; това е без значение за доказателството), ако G е ориентиран.

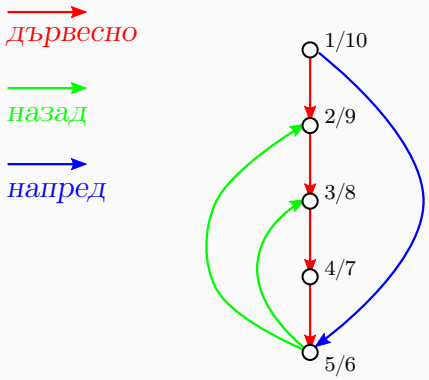
Когато DFS-VISIT минава през списъка на съседство на v, открива u като сив връх и реброто (v, u) бива класифицирано като ребро назад. □

Наблюдение 38

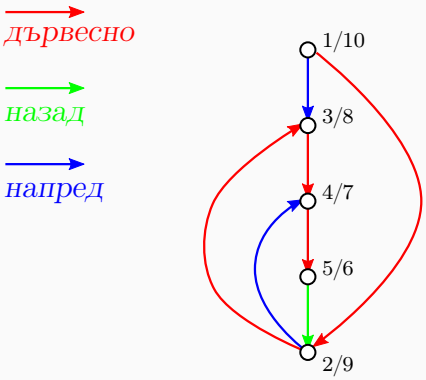
При пускане на DFS върху G от един връх s, броят на дървесните ребра, които DFS открива, зависи единствено от G, а не от представянето му чрез списъци. Нека G' е подграфът, достижим от s. Очевидно G' не зависи от представянето. Тогава броят на дървесните ребра, които DFS открива, е $|V(G')| - 1$.
Броят на ребрата от всички останали видове е $|E(G')| - (|V(G')| - 1) = |E(G')| - |V(G')| + 1$ и очевидно също не зависи от представянето на G. Ако G е неориентиран, останалите ребра са само ребра назад. Но ако G е ориентиран, останалите ребра са от (най-много) три вида и това, по колко има от всеки вид, може да зависи от представянето на G.

Илюстрация: Фигура 9.3 показва две различни изпълнения на DFS върху един и същи граф, от един и същи начален връх, но с различни представяния на графа (различни списъци на съседство). Броят на дървесните ребра е един и същи—винаги четири, защото върховете са пет—но останалите ребра се категоризират като ребра назад и ребра напред по различни начини и с различен брой ребра в тези категории. □

Фигура 9.3 : Различен брой ребра назад.



Две ребра назад.



Едно ребро назад.

Лекция 10

Топологическо сортиране. Намиране на силно свързани компоненти. Намиране на срязващи върхове и мостове.

Резюме: Разглеждаме два линейни алгоритъма за топологическо сортиране на дагове: алгоритъмът на Kahn и алгоритъмът на Tarjan. Разглеждаме линеен алгоритъм за намиране на силно свързани компоненти на ориентиран граф. Разглеждаме линейни алгоритми за намиране на срязващите върхове и мостовете на неориентиран граф.

10.1 Фундамент

Навсякъде в тази лекция, нека $\mathsf{G} = (\mathsf{V}, \mathsf{E})$ е ориентиран граф.

Определение 59: Топологическо сортиране

Топологическо сортиране на G е всяка биекция $\mathfrak{h} : \mathsf{V} \to \{1, \ldots, \mathfrak{n}\}$, такава че за всяко ребро $(\mathfrak{u}, \mathfrak{v}) \in \mathsf{E}$ е вярно, че $\mathfrak{h}(\mathfrak{u}) < \mathfrak{h}(\mathfrak{v})$.

Теорема 54: НДУ за съществуване на топологическо сортиране

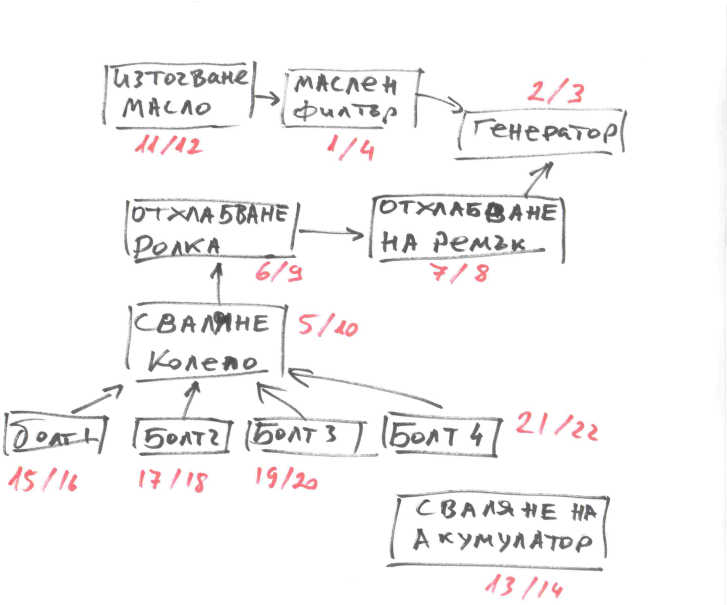
Съществува топологическо сортиране на G тогава и само тогава, когато G е даг.

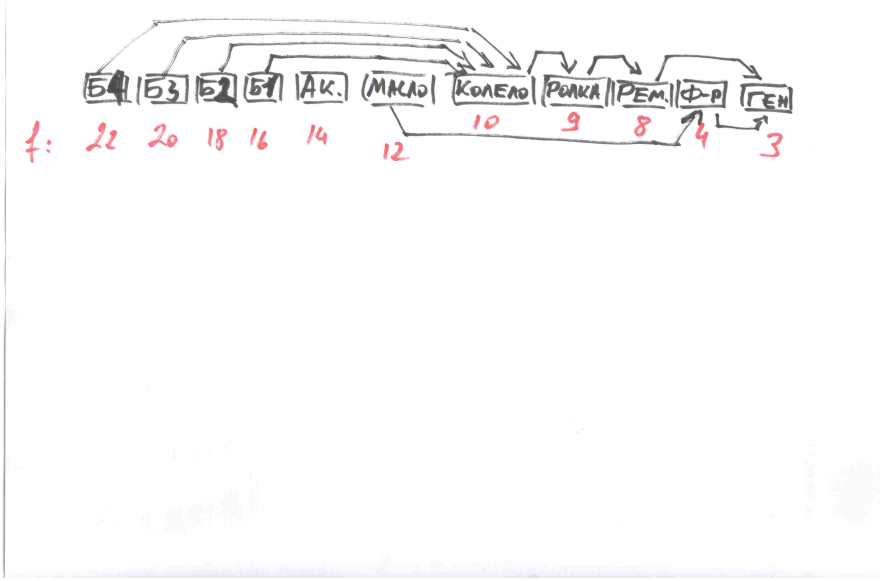
Източник в G, на английски *source*, е всеки връх в G, чиято полустепен на входа е нула. *Сифон в* G, на английски *sink*, е всеки връх в G, чиято полустепен на изхода е нула.

Теорема 55: Поне един източник и поне един сифон в даг

Във всеки даг има поне един източник и поне един сифон.

Пример за даг и негово топологическо сортиране е показан на следващата страница. Примерът идва от реална житейска ситуация – смяна на електрическия генератор на автомобил, като свалянето на генератора налага предварително да се извършат други дейности, които на свой ред налагат предварително да се извършат други дейности, и така нататък. Върховете на дага са дейностите, а ребрата съответстват на непосредствените предшествия между дейностите.


Източване масло
11/12
Маслен филтър
1/4
2/3
Генератор
Отхлабване ролка
6/9
Отхлабване на ремък
7/8
Сваляне колело
5/10
Болт 1
15/16
Болт 2
17/18
Болт 3
19/20
Болт 4
21/22
Сваляне на акумулатор
13/14


Б4
Б3
Б2
Б1
АК.
Масло
Колело
Ролка
Рем.
Ф-р
Ген
f:
22
20
18
16
14
12
10
9
8
4
3

Допълнение 40: Произход на "топологическо сортиране".

Името "топологическо сортиране", без съмнение, идва от "топология" – дял на математиката, възникнал като абстракция на геометрията. По думите на Henning Makholm в *тази дискусия за произхода на името*:

> *Graph theory was originally (and still sometimes is, depending on who you ask) considered a branch of topology.*
>
> *This may sound strange to people with a modern education, where "topology" means more or less "the part of mathematics that deals abstractly with continuity and limits, without using real numbers" – or at least without giving the real numbers any central position in the theory. However, earlier on, "topology" appears to have been a catch-all term for "the part of mathematics that isn't about numbers or geometric magnitudes". (This was before algebraists stopped pretending that algebra is necessarily about numbers). Only later did a distinction between what we now call topology and discrete mathematics become common.*
>
> *In this old usage, "topological sorting" simply means "the kind of sorting you can define without reference to comparison of numbers".*

Терминът "topological ordering" се появява в статията на Kahn от 1962 г. [47], където се казва:

> *In recent years much work has been done on the computational analysis of problems which can be formulated as networks with distinct elements. In particular, this class of problems include PERT (Program Evaluation Review Technique) which is used as management schedule tool. To a large extent the feasibility of network computations depends upon the ability to arrange the network information in topological order. A list in topological order has a special property. Simply expressed: proceeding from element to element along any path in the network, one passes through the list in one direction only. Stated another way, a list in topological order is such that no element appears in it until after all elements appearing on all paths leading to the particular element have been listed.*

"PERT" е средство от 50-те години на XX век за управление на проекти, като описанието на задачите в даден проект задава нещо като даг от зависимости (задача X преди задача Y). За подробности вижте *статията в уикипедия*.

Допълнение 41: Команда `tsort`.

Типичните UNIX инсталации имат средство за топологическо сортиране: командата `tsort`. Подробна информация за нея може да се получи с `info tsort`. Ето пример за използването ѝ. Това е примерът с ремонта на автомобил на предишната страница. Елементите, които трябва да бъдат топологически сортирани, се изписват в двойки, които означават ограниченията кое преди кое да бъде. Тези двойки са ориентираните

ребра на графа. Разполагането на двойките по редове е без значение: програмата чете списък и го разбива на двойки. Известен недостатък на представянето на графа само чрез ребрата е, че няма как да се опишат изолирани върхове; затова дейността сваляне на акумулатор не присъства.

```
$ tsort <<EOF
> източване-масло маслен-филтър
> маслен-филтър генератор
> отхлабване-ролка отхлабване-на-ремък
> отхлабване-на-ремък генератор
> сваляне-колело отхлабване-ролка
> болт-1 сваляне-колело
> болт-2 сваляне-колело
> болт-3 сваляне-колело
> болт-4 сваляне-колело
> EOF
болт-1
болт-2
болт-3
болт-4
източване-масло
сваляне-колело
маслен-филтър
отхлабване-ролка
отхлабване-на-ремък
генератор
$
```

10.2 Алгоритъм на Tarjan за топологическо сортиране

Ефикасен алгоритъм за топологическо сортиране на даг е показаната в тази секция модификация на DFS, предложена от Tarjan [89]. Същината на този алгоритъм е да се сортират върховете по **намаляващи** f-стойности, където $f[1,\ldots,n]$ е масивът от времената на финализиране, генерирани от DFS(G). Примерът на стр. 391 показва точно това. Времената на откриване и финализиране са показани в червено до върховете на дага, а след това е показана и редицата от върховете, обратно сортирани п времената на финализиране.

Сортирането на върховете по намаляващи f-стойности може да стане имплицитно в процеса на работата на DFS. Топологическата сортировка се реализира чрез масив $T[1,\ldots,n]$, който е глобално видим. Масивът се пълни с върхове отдясно наляво, като върхът с най-малка f-стойност е в най-дясна позиция и така нататък. Ето и псеводокод.

```
TSORT(Даг G = (V, E), като V = {1, ..., n})
1  T[1, ..., n], time и ℓ са глобални
2  for i ← 1 to n
3      color[i] ← white
4      π[i] ← NIL
5  time ← 0, ℓ ← n
6  for i ← 1 to n
7      if color[i] = white
8          TSORT-REC(G, i)
9  return T[1, ..., n]
```

```
TSORT-REC(G = (V, E); x ∈ V)
1   color[x] ← gray
2   time++
3   d[x] ← time
4   foreach y ∈ adj[x]
5       if color[y] = white
6           π[y] ← x
7           TSORT-REC(G, y)
8   color[x] ← black
9   time++
10  f[x] ← time
11  T[ℓ] ← x
12  ℓ--
```

Интуитивна аргументация на коректността на алгоритъма е, че най-малката f-стойност задължително е на сифон—ако графът е даг, той има поне един сифон съгласно Теорема 55—така че върхът в $T[n]$ е сифон. Махайки сифона от дага, получаваме пак даг, така че следващата по големина f-стойност е на сифон в новия даг, така че $T[n-1]$ е сифон в новия даг, и така нататък. При това не може да се появят ребра отдясно наляво, по отношение на масива T, защото от сифоните не излизат ребра.

Ето и формално доказателство на коректността. В него не се използват сифони; за сифони говорихме в неформалната обосновка.

Теорема 56: Коректност на топологическото сортиране на Tarjan

Ако G е даг, пермутацията $\pi = \langle u_1\, u_2 \cdots u_n \rangle$ на върховете на G, такава че $f[u_1] > f[u_2] > \cdots > f[u_n]$ след терминирането на DFS(G), реализира топологическо сортиране.

Доказателство: Нека (u, v) е произволно ребро на графа. Съгласно Лема 54, следните възможности са изчерпателни:

- $d[u] < d[v] < f[v] < f[u]$, ако (u, v) е дървесно ребро или ребро напред. Важното в случая е, че $f[u] > f[v]$. Това влече, че връх u е вляво от връх v в пермутацията: $\pi = \cdots u \cdots v \cdots$, така че реброто (u, v) не противоречи на топологическото сортиране.
- $d[v] < d[u] < f[u] < f[v]$, което обаче е невъзможно в текущия контекст: това би означавало, че (u, v) е ребро назад, а съгласно Теорема 53 това би означавало, че G е цикличен, в противоречие с допускането, че G е даг.
- $d[v] < f[v] < d[u] < f[u]$, ако (u, v) е ребро настрани. Отново, имаме $f[u] > f[v]$, така че u е вляво от v в пермутацията и реброто (u, v) не противоречи на топологическото сортиране. □

Важно наблюдение, което ще използваме в Секция 10.4 е, че този алгоритъм не дава съобщение за грешка, ако входният граф не е даг, а приключва работата си и генерира масива $T[1, \ldots, n]$ от върхове. Разбира се, ако G е цикличен, T не реализира топологическо сортиране.

Сложността по време на топологическото сортиране на Tarjan е $\Theta(n + m)$. Това следва от сложността по време на DFS и факта, че топологическото сортиране на Tarjan добавя само $\Theta(1)$ работа към рекурсивните викания.

10.3 Алгоритъм на Kahn за топологическо сортиране

Този алгоритъм е описан още през 1962 г. [47]. Той работи, в някакъв смисъл, обратно на алгоритъма на Tarjan. Докато алгоритъмът на Tarjan върви отдясно наляво, тоест от сифоните назад, алгоритъмът на Kahn върви отляво надясно, тоест от източниците напред. Наричат алгоритъма на Kahn "топологическо сортиране чрез BFS". За да се подчертае приликата с BFS тук ползваме цветовете бял, сив и черен за маркиране на различните от гледна точка на алгоритъма състояния на върховете. Не особено съществена разлика с BFS е, че сега множеството S се инициализира с всички източници в G, докато при BFS, неговото съответствие, опашката Q, се инициализира с точно един връх.

TSORT–KAHN(Даг $G = (V, E)$, като $V = \{1, \ldots, n\}$)

```
 1  S ← ∅
 2  for i ← 1 to n
 3      A[i] ← 0
 4  for x ← 1 to n
 5      foreach y ∈ adj[x]
 6          A[y]++
 7  for i ← 1 to n
 8      if A[i] = 0
 9          S ← S ∪ {i}
10          color[i] ← gray
11      else
12          color[i] ← white
13  k ← 0
14  while S ≠ ∅ do
15      x ← произволен елемент на S
16      S ← S\{x}
17      k++
18      T[k] ← x
19      foreach y ∈ adj[x]
20          A[y]--
21          if A[y] = 0
22              S ← S ∪ {y}
23              color[y] ← gray
24      color[x] ← black
25  if k < n
26      return "Грешка! G не е даг."
27  else
28      return T[1, ..., n]
```

Множеството S може да бъде реализирано например чрез АТД стек или опашка. Аргументацията за коректност и сложност остава валидна и в двата случая.

Коректност. Във всеки момент от работата на алгоритъма, нека B означава множеството от черните върхове, нека A_B е множеството от клетките на A, които съдържат елементите на B, и нека $A_{\overline{B}}$ е множеството от останалите клетки на A.

Да си припомним, че "$G - B$" означава графа, получен от G с изтриване на върховете от B.

Приемаме за очевиден ефектът от работата на редове 1–13, поради което го обобщаваме в следното наблюдение без доказателство.

Наблюдение 39: Инициализацията (редове 1–13) на топол. сортиране на Kahn

При първото достигане на ред 14 е вярно, че:

- $B = \varnothing$.
- S е множеството от източниците на G и се състои точно от сивите върхове.
- A съдържа полустепените на входа на върховете на G.

Теорема 57: Коректност на топологическото сортиране на Kahn

Ако входният G е даг, алгоритъмът на Kahn го сортира топологически чрез масива T. В противен случай, алгоритъмът на Kahn връща съобщение за грешка.

Доказателство: Да допуснем, че G е даг. Следното твърдение е инвариант за **while** цикъла (редове 14–24).

Инвариант 23: while-цикълът на алгоритъма на Kahn

Всеки път, когато изпълнението е на ред 14,

① S е множеството от източниците на $G - B$ и се състои точно от сивите върхове.

② B е множеството $\{T[i] \mid 1 \leqslant i \leqslant k\}$.

③ Нито един връх от B не е дете на връх от $G - B$.

④ $A_{\overline{B}}$ съдържа полустепените на входа на $G - B$.

⑤ $T[1, \ldots, k]$ съдържа топологическа сортировка на подграфа на G, индуциран от B.

База. Разглеждаме първото достигане на ред 14.

Ще покажем ①. Според Наблюдение 39, в този момент $B = \varnothing$, така че $G - B = G$ и ① става "S е множеството от източниците на G и се състои точно от сивите върхове", което е част от Наблюдение 39.

Ще покажем ②. От една страна, в този момент $B = \varnothing$. От друга страна, в този момент $k = 0$ заради ред 13, така че $T[1, \ldots, k]$ е празен.

Ще покажем ③. Тъй като $B = \varnothing$, твърдението е вярно в празния смисъл.

Ще покажем ④. Щом $B = \varnothing$, то $A_{\overline{B}} = A$ и ④ става "A съдържа полустепените на входа на върховете на G", което е част от Наблюдение 39.

Ще покажем ⑤. Щом $k = 0$, то $T[1, \ldots, k]$ е празен. От друга страна, подграфът на G, индуциран от B, е празният граф. Ерго, ⑤ е вярно в празния смисъл. ✓

Поддръжка. Да допуснем, че инвариантът е в сила в даден момент t, в който изпълнението е на ред 14 и **while** ще бъде изпълнен поне още веднъж. Последното влече, че Q не е празна. Нека t' е моментът на следващото достигане на ред 14.

Ще покажем ①. На ред 16, x престава да е елемент на S, а на ред 24, x става черен и по този начин става елемент на B. Тогава в момента t', x вече хем не е сив, хем не е връх на $G - B$.

Множеството от източниците в $G - B - x$ се състои точно от тези върхове, които в $G - B$ имат един единствен родител, който е x. При допускането, че ④ е в сила в момента t, това са точно тези деца y на x, за които $A[y] = 1$ преди да бъдат "обработени" от **for**-цикъла (редове 19–23). Тези деца получават A-стойност нула на ред 20, влизат в S на ред 22, а на ред 23 стават сиви. Но $G - B - x$ от момент t е $G - B$ в момент t' заради ред 24, така че ① остава в сила в t'.

Ще покажем ②. Но ② е в сила в момента t, след което, от една страна, точно един връх от сив става черен (връх x) и никой черен връх не си мени цвета, а от друга страна, k бива инкрементиран на ред 17 и, спрямо новото k, $T[k]$ получава стойност x.

Ще покажем ③. По допускане (①), в момента t, връх x е източник в $G - B$, понеже $x \in S$ в този момент. Това означава, че x не е дете на нито един връх от $G - B$ в момент t. След това към B се добавя един единствен връх, а именно x (ред 24). Очевидно в момент t', ③ е в сила за текущото B.

Ще покажем ④. Изпълнението на тялото на **while**-цикъла засяга $A_{\overline{B}}$ по два начина. Първо, връх x, който в момент t не е в B, "влиза" в B на ред 24; ерго, в момент t, връх x е в $A_{\overline{B}}$, но в t' вече не е в $A_{\overline{B}}$. Що се отнася само до x, твърдение ④ е истина, понеже, от една страна, x престава да е връх от $G - B$, а от друга страна, $A_{\overline{B}}$ престава да съдържа съответен нему елемент.

Второ, изпълненията на ред 20 намаляват с единица полустепента на входа на всички деца на x. Твърдим, че в момент t, $\forall y \in \mathrm{adj}[x] : A[y] > 0$. Това следва от факта, че всяко дете y на x е връх от $G - B$, който не е източник в $G - B$, и че $A[y]$ е полустепента на входа на това дете съгласно ④.

- Защо y е връх от $G - B$? Защото от ③ знаем, че в момента t, нито един черен връх не е дете на връх от $G - B$, а x в t е връх от $G - B$ и y е дете на x.
- Защо y не е източник? Защото в момент t, източниците в $G - B$ са точно върховете в S съгласно ①, а източниците не са деца по дефиниция.

И така, при всяко изпълнение на ред 20 по време на текущото изпълнение на **while**-цикъла се декрементира положително число, което след декрементирането може да е най-малко нула. Очевидно стойностите $A[y]$ на всички деца y на x в момента t' са точно полустепените на входа на тези върхове в текущия $G - B$.

Ще покажем ⑤. Очевидно $T[1, \ldots, k]$ нараства с един елемент между момент t и момент t' и този елемент е връх x от ред 15. Трябва да покажем, че в момент t' няма ребро с начало $T[k]$ и край някой $T[j]$, $1 \leqslant j \leqslant k - 1$. Но това следва от това, че

- същият връх, който е $T[k]$ в момент t', е източник в $G - B$ съгласно ①
- и това, че нито един връх от B не е дете на връх от $G - B$ в момент t.

Терминация. Приемаме за очевидно това, че при терминирането на **while**-цикъла (което е момента, в който $S = \varnothing$ съгласно ред 14), $k = n$ тстк G е даг. При текущото допускане, че G е даг, имаме $k = n$ при достигането на ред 28. Съгласно част ⑤ на инварианта, $T[1, \ldots, n]$ съдържа топологическа сортировка на подграфа на G, индуциран от B. Но от ② знаем, че в този момент $B = \{T[i] \mid 1 \leqslant i \leqslant k\}$, което очевидно е същото като множеството $V(G)$. И така, подграфа на G, индуциран от B, е самият G, така че масивът $T[1, \ldots, n]$ съдържа топологическа сортировка на G. □

Сложност по време. Алгоритъмъ на Kahn има линейна сложност по време. Намирането на полустепените на входа на редове 2–6 става във време $\Theta(n) + \Theta(n + m) = \Theta(n + m)$, защото в редове 2–6 мминаваме през всички списъци на съседства с контантна работа върху всеки елемент. Намирането на източниците в G на редове 7–12 става във време $\Theta(n)$. Цялото изпълнение на **while**-цикъла (редове 14–24) става във време $\Theta(n + m)$, защото пак се минава през всички списъци на съседство, с константна работа върху елемент. Като цяло, сложността по време е $\Theta(n + m)$.

10.4 Алгоритъм за намиране на силно свързаните компоненти

Задачата за намиране на силно свързани компоненти на ориентиран граф има изненадващо много приложения, например за решаване на задачата 2SAT в логиката [72, Theorem 9.1, стр. 184].

Нека навсякъде в тази секция G е ориентиран граф. Знаем, че релацията на силна свързаност $SC(G)$ (Нотация 3) е релация на еквивалентност и подграфите на G, индуцирани от класовете на еквивалентност на $SC(G)$, се наричат силно свързаните компоненти на G.

Забележете приликата между фактор-релация (Определение 23) и фактор-граф (Определение 60).

Определение 60: Фактор-граф

Нека $R \subseteq V(G) \times V(G)$ е релация на еквивалентност . Нека $\mathfrak{X}$ е множеството от класовете на еквивалентност на R. *Фактор-графът на* G *спрямо* R[a] е ориентираният граф G/R с множество от върхове $\mathfrak{X}$, като за всеки два различни класа $P, Q \in \mathfrak{X}$, ребро (P, Q) в G/R съществува тогава и само тогава, когато съществува връх $a \in P$ и съществува връх $b \in Q$, такива че $(a, b) \in E(G)$.

[a] На английски терминът е *quotient graph*.

Несъществена разлика между фактор-релация и фактор-граф е, че фактор-релацията е рефлексивна, докато фактор-графът няма примки.

В тези лекционни записки ще разглеждаме фактор-графи само спрямо релацията на силна свързаност. Когато кажем "фактор-графът на G", имаме предвид "фактор-графът на G спрямо $SC(G)$". Фактор-графът на G се означава с "$G/SC(G)$".

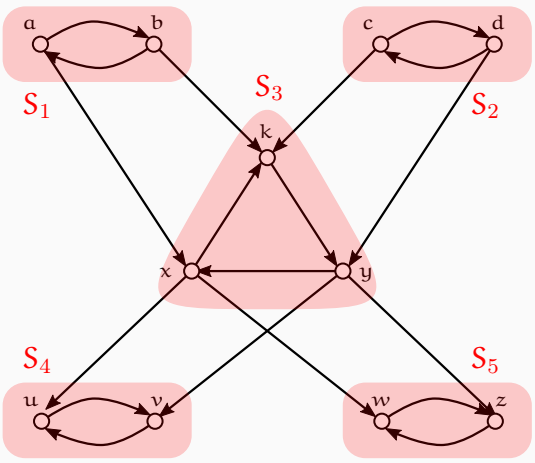
Наблюдение 40: Фактор-графът е даг.

$G/SC(G)$ е даг. Ако в $G/SC(G)$ има цикъл, за всеки два върха X и Y от цикъла—тези върхове са силно свързани компоненти в G—е вярно, че от всеки връх на X има път до всеки връх на Y в G, и от всеки връх на Y има път до всеки връх на X в G. Тогава върховете на X и Y всъщност са в една и съща силно свързана компонента на G, в противоречие с предишното допускане, според което X и Y са различни силно свързани компоненти на G.

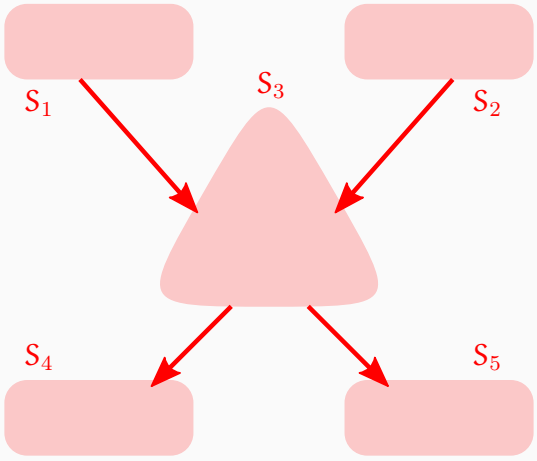
На английски за това понятие (което нарекохме "фактор-граф") понякога се *the condensed graph* или *the component graph* [21, стр. 617].

Фигура 10.1 показва ориентиран граф и неговият фактор-граф.

Фигура 10.1 : Ориентиран граф и неговият фактор-граф.



G и силно свърз. му компоненти.



$G/SC(G)$.

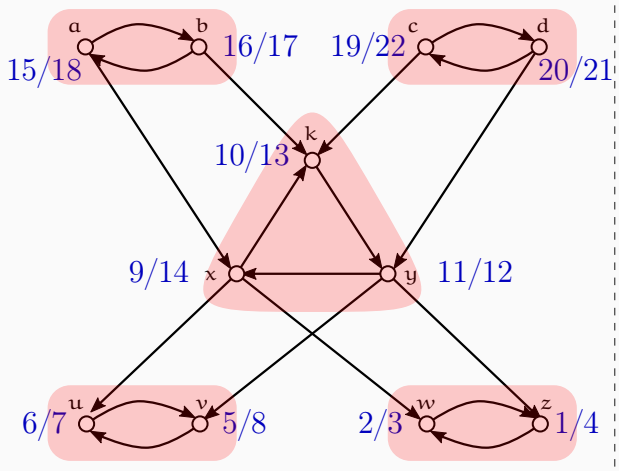
Да се опитаме да решим задачата за намиране на силно свързаните компоненти на G чрез следната малка модификация на DFS, наречена SCC-DUMMY. Както знаем, силно свързаните компоненти не са само множества от върхове, а са истински графи с върхове и ребра, но за простота генерирането на силно свързаните компоненти ще се състои в генерирането на техните множества от върхове.

```
SCC-DUMMY(G = (V, E); V = {1, ..., n})
 1  S и time са глобални
 2  for i ← 1 to n
 3      color[i] ← white
 4  time ← 0
 5  for i ← 1 to n
 6      if color[i] = white
 7          S ← ∅
 8          SCC-DUMMY-REC(G, i)
 9          print S
```

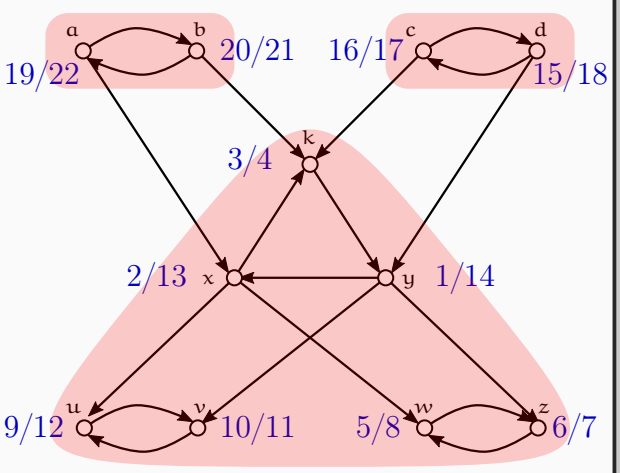
```
SCC-DUMMY-REC(G = (V, E); x ∈ V)
 1  color[x] ← gray
 2  time++
 3  d[x] ← time
 4  S ← S ∪ {x}
 5  foreach y ∈ adj[x]
 6      if color[y] = white
 7          SCC-DUMMY-REC(G, y)
 8  color[x] ← black
 9  time++
10  f[x] ← time
```

Да си представим работата на SCC-DUMMY върху графа G от Фигура 10.1. Очевидно множествата, които алгоритъмът ще принтира на ред 9, зависят от това, в какъв ред **for**-цикълът на ред 5 ще "минава" през върховете. Това е илюстрирано на Фигура 10.2. Там е показана работата на SCC-DUMMY при две различни наредби на върховете и съответните различни разбивания. Както се вижда, при наредба 1, разбиването точно отговаря на класовете на еквивалентност на $SC(G)$, тоест, то задава силно свързаните компоненти. Но при наредба 2, разбиването е само на три класа на еквивалентност, а не на пет. Ако искаме да намираме силно свързаните компоненти, би трябвало да ползваме наредба, подобна на наредба 1.

Фигура 10.2 : SCC-DUMMY при различни наредби на върховете.



наредба 1: $\langle z\,v\,u\,x\,y\,w\,k\,a\,c\,d\,b\rangle$.



наредба 2: $\langle y\,w\,u\,c\,a\,b\,d\,z\,v\,x\,k\rangle$.

Лесно се вижда какъв е проблемът при наредба 2 от гледна точка на намиране на силно свързаните компоненти. Започвайки обхождането от връх y във време 1, DFS "излиза" от силно свързаната компонента на $\{k, x, y\}$ и се "прехвърля" в две други силно свързани компоненти, а именно на $\{w, z\}$ във време 5 и на $\{u, v\}$ във време 9, преди да финишира върху z във време 14. Така с едно викане на SCC-DUMMY-REC на ред 8, алгоритъмът открива три силно свързани компоненти и слага върховете им в S, което е недопустимо, ако искаме да намерим силно свързаните компоненти на G.

Наблюдение 41

За да намерим точно една силно свързана компонента при първото викане на ред 8, наредбата на ред 5 трябва да е такава, че първият връх в нея да е от силно свързана компонента, от която не може да се излезе. Наредба 1 от Фигура 10.2 е пример за такава наредба. Първият връх в нея, а именно z, е от силно свързана компонента, от която обхождането не може да излезе.

Определение 61: Благоприятна силно свързана компонента

Силно свързана компонента, от която обхождането не може да излезе, ще наричаме *благоприятна силно свързана компонента*.

Наблюдение 42

Благоприятните силно свързани компоненти на G са сифоните на фактор-графа $G/SC(G)$. Тъй като $G/SC(G)$ е даг, той има поне един сифон съгласно Теорема 55, така че G има поне една благоприятна силно свързана компонента.

За да намерим силно свързаните компоненти чрез DFS, не е достатъчно първият връх в наредбата на ред 5 да е от благоприятна силно свързана компонента G_1. След излизането от викането на ред 8 трябва следващият бял връх, за който условието на ред 6 е истина, да е от благоприятна силно свързана компонента на $G - V(G_1)$ (за да не може DFS да излезе

от нея), и така нататък до края на **for**-цикъла на ред 5. Не е необходимо върховете на силно свързаните компоненти да са непрекъснати подредици в наредбата на върховете на **for**-цикъла. Достатъчно е в тази наредба първите върхове от силно свързаните компоненти да са подредени отляво надясно така, че всеки нов тях да е от благоприятна силно свързана компонента в графа G, от който са изтрити вече обходените силно свързани компоненти.

Определение 62: Важните ребра

Важните ребра на G са точно тези ребра, чиито начало и край са от различни силно свързани компоненти.

Например, на графа G от Фигура 10.1, важните ребра са (a, x), (b, k), (c, k), (d, y), (x, u), (y, v), (x, w) и (y, z), а останалите ребра не са важни.

Наблюдение 43

За да намерим силно свързаните компоненти на G чрез SCC-DUMMY, необходимо и достатъчно е наредбата на върховете по отношение на ред 5 да е такава, че за всяко важно ребро (p, q), краят q да е преди началото p. Разположението на важните ребра в тази наредба трябва да обратното на разположението на ребрата в топологическа сортировка: в топо-сортирането се иска всички ребра да са "вчесани" отляво надясно, а сега се иска важните ребра да са вчесани "отдясно наляво". В този смисъл, въпросната наредба, по отношение на важните ребра, трябва да е обратната на топологическата.

И така достигаме до идеята, че ако алгоритъмът получи като вход "правилна" наредба на върховете, тоест, масив от върховете $L[1, \ldots, n]$, подреден според Наблюдение 43, той ще работи коректно.

```
SCC(G = (V, E); L[1, ..., n]: масив от върховете)
1  S и time са глобални
2  for i ← 1 to n
3      color[i] ← white
4  time ← 0
5  for i ← 1 to n
6      if color[L[i]] = white
7          S ← ∅
8          SCC-REC(G, L[i])
9          print S
```

```
SCC-REC(G = (V, E); x ∈ V)
1  color[x] ← gray
2  time++
3  d[x] ← time
4  S ← S ∪ {x}
5  foreach y ∈ adj[x]
6      if color[y] = white
7          SCC-REC(G, y)
8  color[x] ← black
9  time++
10 f[x] ← time
```

Като пример да вземем пак Фигура 10.2. Ако $L = [z, v, u, x, y, w, k, a, c, d, b]$ (подфигурата вляво), алгоритъмът ще намери силно свързаните компоненти.

Ефикасен алгоритъм за намиране на силно свързаните компоненти на ориентиран граф. Следният алгоритъм решава ефикасно задачата за намиране на силно свързаните компоненти. Открит е от Rao Kosaraju през 1978 [1]. Необходимо ни е следното определение.

Определение 63: Транспониран граф

Транспонираният граф на G е графът $G^T = (V(G), \{(u, v) \mid (v, u) \in E(G)\})$.

На английски терминът е *the transpose of* G ([21, стр. 616]). Без съмнение, прилагателното "транспониран" идва оттам, че матрицата на съседство на G^T е транспонираната матрица на съседство на G.

Алгоритъмът на Kosaraju вика алгоритъмът за Tarjan на стр. 394 за топологическо сортиране. Това, че сега G може да не е даг, **няма значение**. Забележете, че алгоритъмът на Tarjan, ако получи не-даг на входа, не установява, че графът не е даг! Той подрежда върховете по намаляващи стойности на време на финализиране, което може да бъде направено за всеки граф; ако графът не е даг, то изходът няма смисъл на топологическо сортиране на графа, но това е друго нещо.

KOSARAJU(Ориентиран граф $G = (V, E)$)
1 $L[1, \ldots, n] \leftarrow$ TSORT(G)
2 намери G^T
3 SCC(G^T, L)

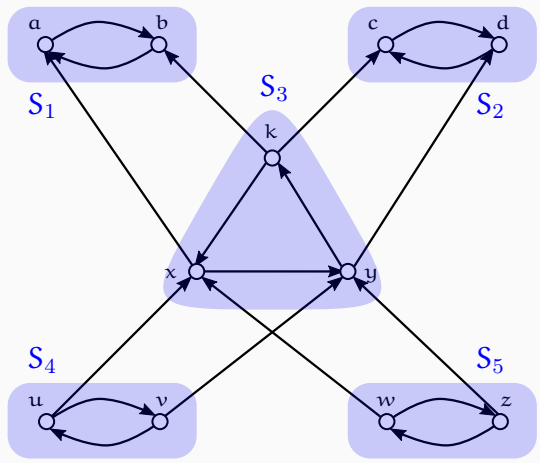
Коректност. Аргументацията за коректност се основава на следните очевидни факти.

Наблюдение 44

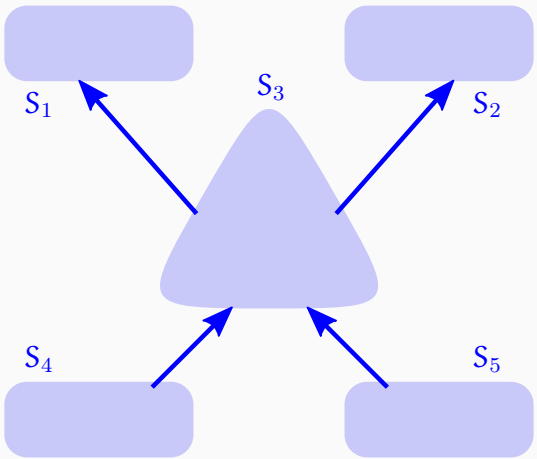
G и G^T имат практически същите силно свързани компоненти в следния смисъл: $SC(G)$ и $SC(G^T)$ съвпадат, така че разбиванията на върховете (в класове на еквивалентност на силна свързаност) в графа и в неговия транспониран граф са еднакви. Нещо повече: $\big(G/SC(G)\big)^T = G^T/SC(G^T)$. Оттук следва, че източниците в $G/SC(G)$ са точно сифоните в $G^T/SC(G^T)$ и обратното, сифоните в $G/SC(G)$ са точно източниците в $G^T/SC(G^T)$.

Фигура 10.3 илюстрира това наблюдение. Тя показва транспонирания граф на графа от Фигура 10.1, както и неговия фактор-граф, който очевидно е транспонирания граф на фактор графа от Фигура 10.1.

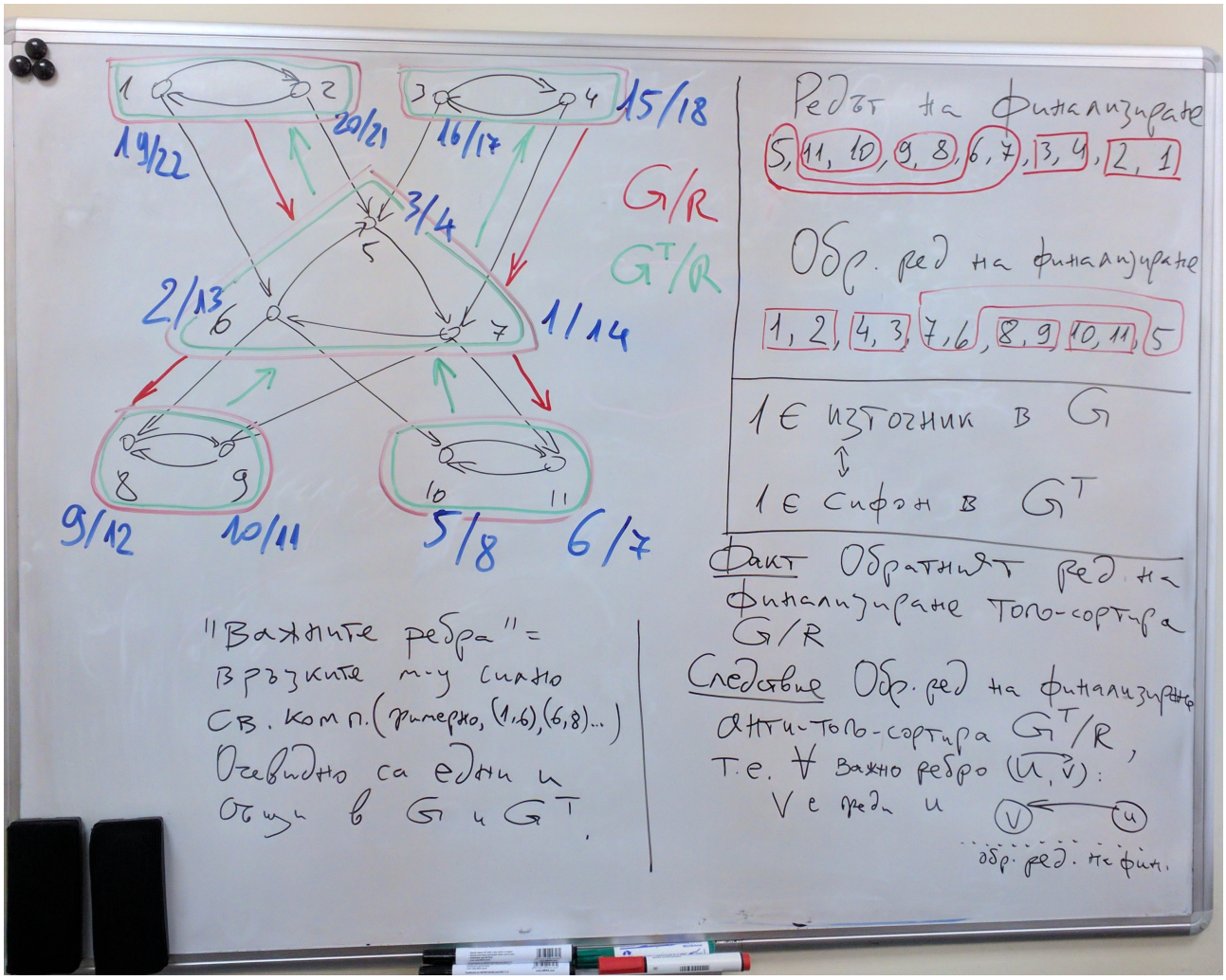
Фигура 10.3 : Транспонираният граф на графа от Фигура 10.1.



Силно свързаните компоненти са практически същите.



Фактор-графът е транспониран.



Сложност по време. Алгоритъмът на Kosaraju има линейна сложност по време. Работата на TSORT има линейна сложност – факт, в който вече се убедихме. Това, че сега графът-вход не е непременно даг, няма значение за това. Създаването на транспонирания граф също има лиейна сложност: то може да се реализира чрез едно "минаване" през списъците на съседства на G, като, щом срещнем връх v в списъка на връх u (това означава ребро (u, v) в G), слагаме u в списъка на v в представянето на G^T (това има смисъл на слагане на ребро (v, u) в G^T). И накрая, работата на SCC се извършва очевидно в линейно време.

10.5 Алгоритми за намиране на срязващите върхове и на мостовете

10.5.1 Фундамент

Сега разглеждаме само неориентирани графи. В тази секция, $G = (V, E)$ означава неориентиран свързан граф (без примки, не е мултиграф).

Определение 64: Срязващ връх и мост

Срязващ връх в G е всеки $u \in V$, такъв че $G - u$ има повече от една свързани компоненти. *Срязващо ребро*, още се нарича *мост*, е всяко $e \in E$, такова че $G - e$ има две свързани компоненти.

Определение 65: Срязващ връх и мост, алтернативно определение

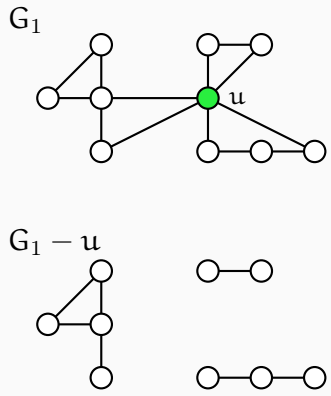
Срязващ връх в G е всеки $u \in V$, такъв че за някои различни върхове $v, w \in V$ е вярно, че всеки път между v и w съдържа u като вътрешен връх. Мост е всяко $e \in E$, такова че за някои различни върхове $v, w \in V$ е вярно, че всеки път между v и w съдържа e.

Лема 55

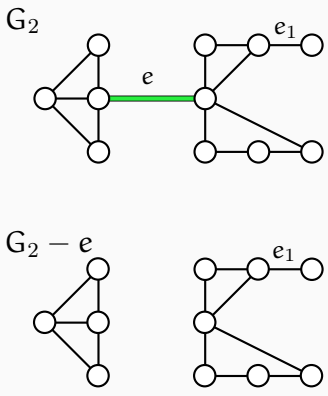
Определения 64 и 65 са еквивалентни.

Лесно се вижда, че на всеки висящ връх в свързана компонента с повече от два върха съответства точно един срязващ връх, а именно единственият му съсед. Самият висящ връх не е срязващ. Ако свързаната компонента има точно два върха, или с други думи, ако е едно единствено ребро, то в нея срязващ връх няма. На всеки висящ връх съответства точно един мост, а именно единственото ребро, инцидентно с него; това остава в сила дори свързаната компонента да се състои само от това ребро. Фигура 10.4 илюстрира Определения 64 и 65. В графа G_1, връх u е срязващ. В графа G_2, и e, и e_1 са мостове, но е показано само изтриването на e.

Фигура 10.4 : Срязващ връх и срязващо ребро.



u е срязващ връх.



e е срязващо ребро (e_1 също).

Приложение на срязващи върхове и ребра. Срязващите върхове и мостовете са важни понятия, ако графът моделира някаква свързаност. Например, ако моделира комуникационна мрежа, или пътна мрежа, или електрическа мрежа.

Нека графът моделира комуникационна мрежа от компютри и жични връзки между тях. Нека мрежата е свързана в смисъл, че всеки два компютъра X и Y могат да си "говорят"; може би не директно, а през други компютри, но между X и Y има маршрут. Тогава срязващ връх в графа съответства на компютър, чиято повреда би довела до това, че мрежата би се

разпадала на две или повече подмрежи, които не си "говорят". В разпаднатата мрежа има компютри, които не могат да комуникират, защото между тях няма маршрут. А срязващо ребро отговаря на жична връзка, чиято повреда би довела до разпадане на мрежата на две подмрежи, които не си "говорят".

И така, срязващите върхове и мостовете отговарят на някакъв критично важни компоненти на мрежата. Мрежа, чийто съответен граф има срязващи върхове или мостове е потенциално по-ненадеждна от мрежа, чийто съответен граф няма такива елементи. От гледна точка на надеждността, очевидно предпочитаме мрежи, чиито съответни графи нямат срязващи върхове или мостове.

Определение 66: Разцепване на срязващи върхове

Нека $u \in V$ е срязващ връх. Нека $G - u$ има k свързани компоненти, наречени $G_1, \ldots, G_k$. Нека $z_1, \ldots, z_k$ са k нови върхове – такива, които не са от V. Нека, за $1 \leqslant i \leqslant k$,

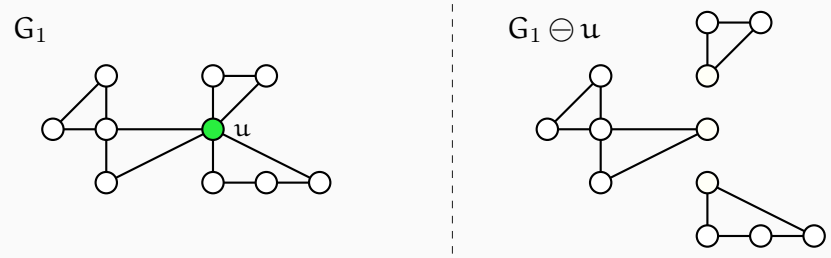
$$H_i = (V(G_i) \cup \{z_i\}, E(G_i) \cup \{(z_i, x) \mid x \in V(G_i) \cap N(u)\})$$

Да *разцепим* u означава да преобразуваме G в колекцията $H_1, \ldots, H_k$. Графът–резултат означаваме с $G \ominus u$.

Очевидно $H_1, \ldots, H_k$ са свързани графи.

Неформално казано, разцепването на срязващ връх u се състои в създаването на негови копия (самият u изчезва), на брой колкото са свързаните компоненти **спрямо него**, като всяко копие бива свързано с точно една от тези компоненти, с точно тези върхове в нея, с които u е бил свързан. Фигура 10.5 показва разцепване на връх – след разцепването на u се появяват три свързани компоненти.

Фигура 10.5 : Разцепване на връх.



u е срязващ връх. След разцепването на u.

Операцията "разцепване на срязващ връх" е асоциативна: можем да разцепваме различни върхове в каквато искаме последователност и резултатът е един и същи. Ако $u_1, \ldots, u_t$ са различни срязващи върхове в G, пишем $G \ominus u_1 \ominus \cdots \ominus u_t$. Резултатът от разцепването на всички срязващи върхове е предмет на следващата дефиниция.

Определение 67: Блокове в граф

Нека $G = (V, E)$ е граф. *Блок в* G е всеки максимален по включване свързан подграф, който не съдържа срязващи върхове.

Определение 68: Блокове в граф, алтернативна дефиниция

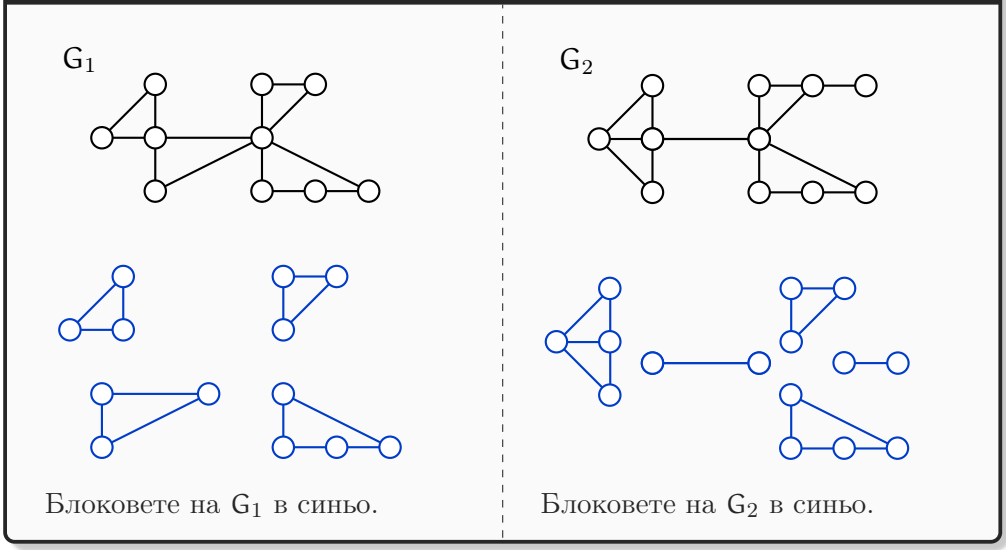
Нека $G = (V, E)$ е граф. *Блоковете на* G са тези свързани компоненти, които се получават от разцепването на всички срязващи върхове.

Наблюдение 45

Определения 67 и 68 са еквивалентни.

Фигура 10.6 илюстрира блоковете на два графа G_1 и G_2. Тя илюстрира и "разцепване на срязващи върхове".

Фигура 10.6 : Блокове в графи.



Блоковете на G_1 в синьо. Блоковете на G_2 в синьо.

Има много алгоритми върху графи—например, алгоритмите за планарност—които извършват като предварителна обработка следните декомпозиции. Първо, намират свързаните компоненти, което може да стане ефикасно с BFS или DFS. Очевидно, графът е планарен тстк всяка свързана компонента е планарна. После за всяка компонента намират срязващите върхове и блоковете. Очевидно, графът е планарен тстк блоковете са планарни.

Това е друго важно приложение на срязващите върхове: декомпозицията на граф на блокове.

10.5.2 Алгоритъм за ефикасно намиране на срязващите върхове

Ако конструираме алгоритъм за намиране на срязващите върхове на G направо от Определение 64, той би бил следният. За всеки връх u, изтриваме u от G и с BFS или DFS проверяваме дали G остава свързан; ако остава, то u не е срязващ връх, ако ли не, то u е срязващ връх и го добавяме към множеството от срязващите върхове, после връщаме u и продължаваме по същия начин. Това би бил твърде неефикасен алгоритъм, дори да можем да трием и връщаме върхове в $\Theta(1)$: сложността му в най-лошия случай би била $\Theta(n(n+m))$.

Ефикасният алгоритъм, който ще разгледаме сега, е базиран на [86, Section 5.9.2, pp. 173–177]. Това е DFS, модифициран за намиране на срязващи върхове.

Всяко изпълнение на DFS върху свързан граф $G = (V, E)$ генерира дърво на обхождането T. Знаем, че $V(T) = V$ and $E(T) = \{e \in E \mid e$ е класифицирано като дървесно ребро от DFS$\}$.

Разглеждаме три категории върхове на **графа** спрямо **дървото** и необходимите и достатъчни условия за това, връх от дадена категория да е срязващ.

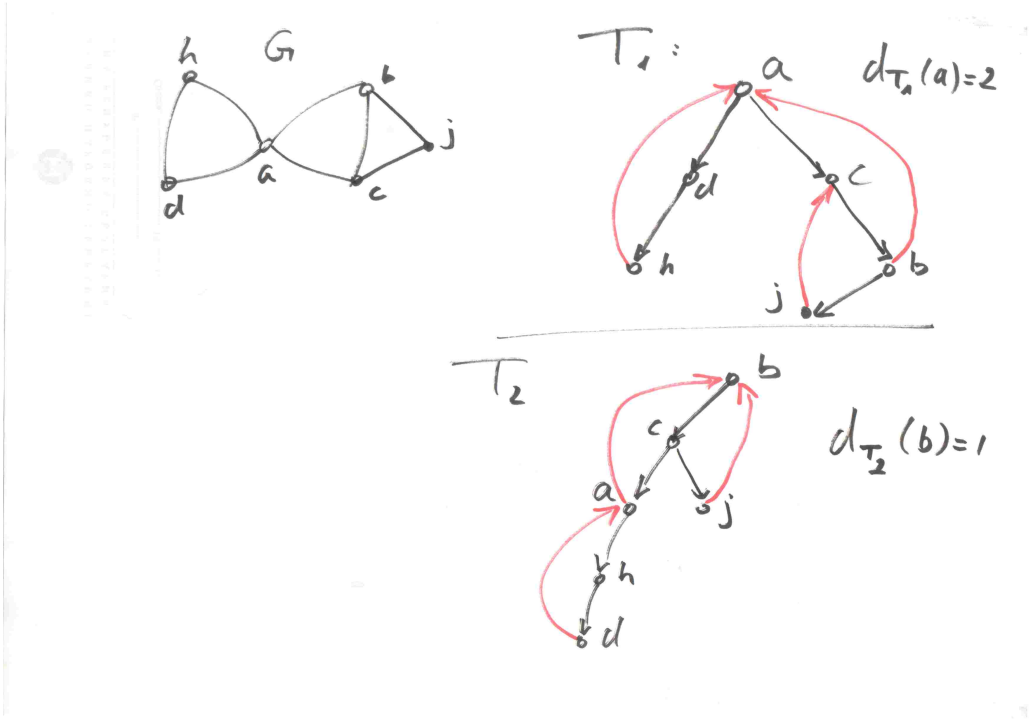
Лема 56: Листата на дървото

В текущия контекст, нито едно листо не може да е срязващ връх. Ако v е листо на T, очевидно $G - v$ има точно една свързана компонента.

Лема 57: Коренът на дървото

В текущия контекст, коренът е срязващ връх в G тстк степента на му **в дървото** е поне 2.

Доказателство:

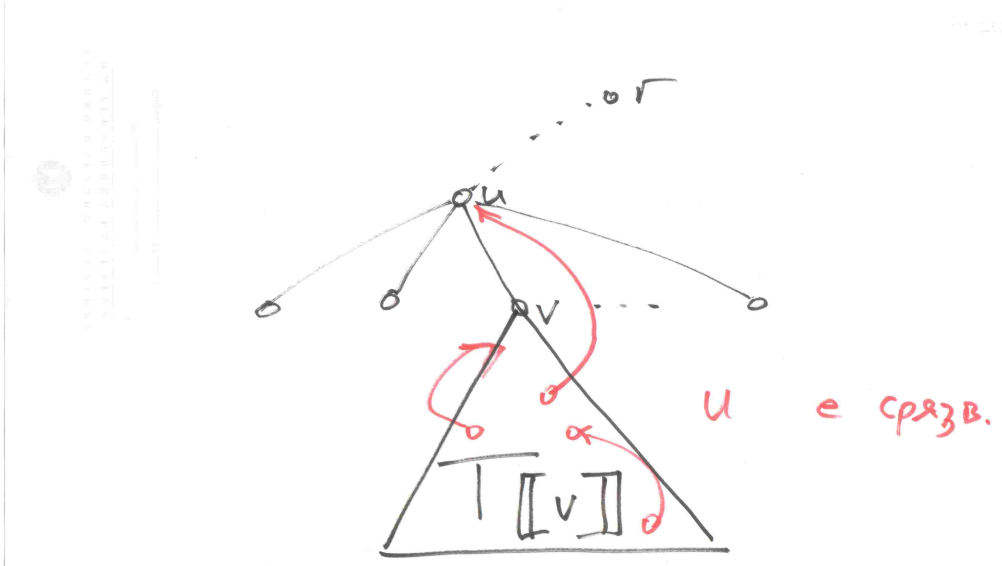


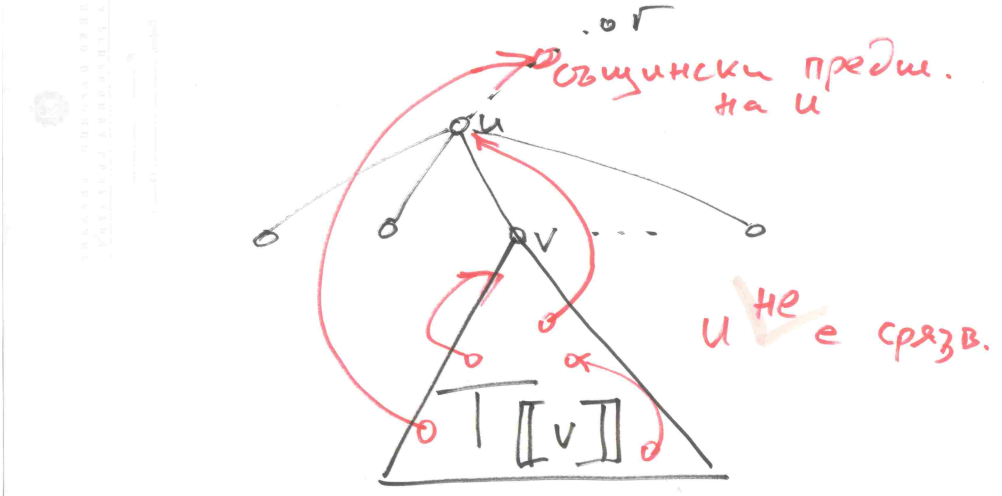
Лема 58: Останалите върхове на дървото

В текущия контекст, всеки връх u, който не е листо или коренът, е срязващ връх тстк има дете v на u в T, такова че няма нито едно ребро назад от връх на $T[\![v]\!]$ към предшественик на u в дървото.

Доказателство: Да допуснем, че има дете v на u (в дървото), такова че няма ребро назад от никой връх от $V(T[\![v]\!])$ към предшественик на u в дървото. Това влече, че за всеки $x \in V(T[\![v]\!])$ и всеки $y \in V(G) \setminus (V(T[\![v]\!]) \cup \{u\})$, за всеки път p в G между x и y е вярно, че u е вътрешен връх в p. Тогава u е срязващ връх в G.

В обратната посока, нека за всяко дете v на u (в дървото) има има ребро назад между връх $x_v \in V(T[\![v]\!])$ към предшественик y_v на u в дървото. Тогава за всеки два върха a и b в G е вярно, че в G има път $p_{a,b}$, който не съдържа u като вътрешен връх. Тогава u не е срязващ връх.





□

Следният алгоритъм имплементира тези идеи в модификация на DFS. В него няма масиви $d[1,\ldots,n]$, $f[1,\ldots,n]$ и $\pi[1,\ldots,n]$, а също така няма и променлива time. DFS-ът стартира от произволен връх u, който е и коренът на дървото на обхождането T.

Да си припомним един факт за разстоянията. Разстоянията в дървото и в графа не са непременно същите, като за всеки $v \in V$ е вярно, че $\mathrm{dist}_G(u,v) \leqslant \mathrm{dist}_T(u,v)$. Това е така, защото T е подграф на G.

Алгоритъмът поддържа масив от нива́та $\mathrm{level}[1,\ldots,n]$, като в края на работата, за всеки връх v, $\mathrm{level}[v] = \mathrm{dist}_T(u,v)$. С други думи, нивото на v е разстоянието между u и v **в**

дървото на обхождането (вижте Определение 29). Пак подчертаваме, че **не става дума за разстояния в** G; знаем, че DFS в общия случай не изчислява разстояния в графа. Очевидно, $\mathrm{level}[u] = 0$.

```
FIND CUT VERTICES(G = (V, E): неориентиран свързан граф)
 1  level[1, ..., n] е глобален
 2  foreach u ∈ V
 3      color[u] ← white
 4  u е произволен връх от V
 5  FCV-REC(G, u, 0);
```

```
FCV-REC(G = (V, E), x ∈ V, ℓ ∈ ℕ)
 1  color[x] ← gray
 2  level[x] ← ℓ
 3  minback ← level[x]
 4  if level[x] = 0
 5      isroot ← TRUE
 6  else
 7      isroot ← FALSE
 8  count ← 0
 9  iscutvertex ← FALSE
10  foreach y ∈ adj[x]
11      if color[y] = white {
12          count++
13          b ← FCV-REC(G, y, ℓ + 1)
14          if b ⩾ level[x] and ( not isroot )
15              iscutvertex ← TRUE
16          minback ← min {minback, b} }
17      if color[y] = gray {
18          if level[y] < minback and level[y] ≠ level[x] − 1
19              minback ← min {level[y], minback} }
20  if iscutvertex or ( isroot and count ⩾ 2 )
21      print x
22  color[x] ← black
23  return minback
```

Коректност. Променливата minback, както показва и името ѝ, която FCV-REC връща, има смисъл на минимално ниво (най-близо до корена), към връх от което има ребро, другият край на което ребро е връх от $T[\![x]\!]$.

Лема 56, Лема 57 и Лема 58 са достатъчна обосновка за коректността, тъй като рекурсивната функция FCV-REC работи така, че да имплементира техните необходими и достатъчно условия. Функцията очевидно не категоризира никое листо като срязващ връх, отбелязва корена като срязващ връх тстк той има поне две деца (в дървото), а за всеки друг вид връх x сравнява на ред 14 разстоянието между върха и корена $\mathrm{level}[x]$ с минималното разстояние b между корена и най-близък до корена връх, към който има ребро назад от връх от $T[\![y]\!]$, по всички деца y на x в дървото. При допускането, че рекурсивното викане на ред 13 връща стойност b с именно такъв смисъл, Лема 58 дава желаната аргументация веднага.

Има една особеност при изчисляването на minback на редове 17–19. Щом y е сив, реброто

(x, y) е ребро назад. Естествено, алгоритъмът трябва да "прегледа" всички ребра назад между x и връх, който е предшественик на x в дървото, за да намери истинската стойност на minback. Но реброто между x и родителят на x в дървото не е ребро назад, а е дървесно ребро, така че трябва да бъде прескочено в това "преглеждане". Смисълът на $\text{level}[y] \neq \text{level}[x] - 1$ на ред 18 е точно този. Забележете, че, ако x не е коренът, има един единствен негов съсед y, за който $\text{level}[y] = \text{level}[x] - 1$ и това е родителят на x в дървото.

Сложност. Очевидно сложността е $\Theta(m+n)$, което следва веднага от това, че алгоритъмът обхожда еднократно списъците на съседства и върши само $\Theta(1)$ работа за всеки елемент от тях.

10.5.3 Алгоритъм за за ефикасно намиране на мостовете

Подходът с брутална сила за намиране на мостовете би работил, в най-бруталния си вариант, в $\Theta(m(n+m))$: за всяко ребро e, изтриваме e от G и с BFS или DFS проверяваме дали G остава свързан; ако остава, то e не е мост, ако ли не, то e е мост и го добавяме към множеството от мостовете, после връщаме e и продължаваме по същия начин.

Задачата за намиране на мостовете е много подобна на задачата за намиране на срязващите върхове, откъдето е и приликата между алгоритмите Find Cut Vertices и Find Bridges.

Лема 59

В текущия контекст, за всяко ребро (u, v) е вярно, че то е мост тстк DFS(G) го категоризира като дървесно и, ако u е бащата на v в дървото на обхождането, то няма ребро назад от никой връх на $T[\![v]\!]$ към u или предшественик на u.

Доказателство: В едната посока, да допуснем, че (u, v) е дървесно ребро, като u е бащата на v, и няма ребро назад от никой връх на $T[\![v]\!]$ към u или предшественик на u. Тогава за всеки $x \in V(T[\![v]\!])$ и всеки $y \in V(G) \backslash (V(T[\![v]\!]) \cup \{u\})$, за всеки път p в G между x и y е вярно, че (u, v) е ребро в p, което означава, че (u, v) е мост.

В обратната посока, нека има ребро назад от връх $x \in V(T[\![v]\!])$ към u или предшественик на u. Тогава за всеки два върха a и b в G е вярно, че в G има път $p_{a,b}$, който не съдържа реброто (u, v). Тогава (u, v) не е мост. □

За разлика от задачата за намиране на срязващите върхове, сега не е необходимо да третираме корена, листата и останалите върхове по различни начини.

```
Find Bridges(G = (V, E): неориентиран свързан граф)
 1  level[1, ..., n] е глобален
 2  foreach u ∈ V
 3      color[u] ← white
 4  u е произволен връх от V
 5  FBR-rec(G, u, 0)


FBR-rec(G = (V, E), x ∈ V, ℓ ∈ ℕ)
 1  color[x] ← gray
 2  level[x] ← ℓ
 3  minback ← level[x]
```

```
 4   foreach y ∈ adj[x]
 5       if color[y] = white {
 6           b ← FBR-REC(G, y, ℓ + 1)
 7           if b > level[x]
 8               print (x, y)
 9           else
10               minback ← min {minback, b}
11       if color[y] = gray {
12           if level[y] < minback and level[y] ≠ level[x] − 1
13               minback ← level[y] }
14   color[x] ← BLACK
15   return minback
```

Коректност и сложност. Доказателството за коректност почива на Лема 59. Ако рекурсивното викане на ред 6 връща коректно минималния номер на ниво, към което има ребро назад от връх от $T[\![y]\!]$, съхранявайки го в b, то, съгласно Лема 59:

- ако $b > \text{level}[x]$, тоест, ако това ниво е "под x", то (x, y) е мост;
- в противен случай, (x, y) не е мост. Нещо повече, в този случай има смисъл да се опитаме да подобрим (тоест, намалим) minback на ред 10. В предишния случай няма смисъл от такъв опит.

Действията на алгоритъма върху сивите съседи на x (редове 11–13) и тяхната обосновка са точно както във Find Cut Vertices.

Сложността по време е очевидно $\Theta(m + n)$.

Лекция 11

Намиране на минимални покриващи дървета. Алгоритми на Prim и Kruskal.

Резюме: Въвеждаме тегловни графи и задачата за намиране на минимално покриващо дърво на тегловен граф. Разглеждаме два ефикасни алгоритъма за намиране на минимални покриващи дървета: алгоритмите на Prim и Kruskal. Заради ефикасността на алгоритъма на Kruskal въвеждаме и изследваме Union-Find структури данни.

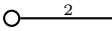
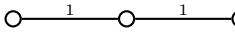
11.1 Фундамент

Определение 69: Тегловен граф

Тегловен граф е наредена двойка (G, w) от граф $\mathsf{G} = (\mathsf{V}, \mathsf{E})$ и *тегловна функция* $w : \mathsf{E} \to \mathbb{R}$.

Говорим за *тегла на ребрата*[†]. На английски терминът за "тегловен граф" е *weighted graph*, откъдето идва и *edge weights*. Освен "тегловна", тази функция понякога бива наричана *ценова функция*, откъдето може да говорим и за *цени на ребрата*, имайки предвид същото нещо като "тегла на ребрата". Използваме термините *леките/евтините ребра* и *тежките/скъпите ребра* с очевидния смисъл.

Тегловните графи имат много по-голяма моделираща мощ от обикновените, не-тегловни графи. Очевидно е, че не-тегловните графи са частен случай на тегловните: просто си представяме, че всяко ребро има тегло единица.

Има ограничени случаи, в които задача върху тегловен граф може да бъде сведена до задача върху не-тегловен граф и да използваме алгоритъм за не-тегловния граф, за да я решим ефикасно върху тегловния. Пример за това е задачата за най-къси пътища в тегловен граф, където тегловната функция е доста ограничена, да кажем $w : \mathsf{E} \to \{1, 2\}$. При това ограничение може да решим задачата не с алгоритъма на Dijkstra за най-къси пътища в тегловни графи, а с прост BFS, ако първо заменим всяко ребро с тегло две:  с последователност от две ребра с тегла единици:  . Но този подход за свеждане на задачи върху тегловни графи към задачи върху не-тегловни графи не са мащабира. Ако ограничението за теглата е $w : \mathsf{E} \to \{1, \ldots, 1\,000\,000\}$, този подход би налагал да заменим ребро с тегло един милион с път от един милион ребра, всяко с тегло единица. Това е все едно да записваме числата в унарна бройна система; големината на теглото става брой ребра. В

[†]Възможно е да се въведат тегла и на върховете, но ние ще разглеждаме тегла само на ребрата.

общия случай, този подход би довел до експоненциално нарастване на големината на примера, ако отчитаме размера на представянето на числата, или до неограничено нарастване на примера в големината на входа, ако игнорираме размера на представянето на числата. Да не говорим за проблемите на този подход при тегла, които не са цели числа или са отрицателни числа. И така, тегловните графи са строго по-мощно средство за моделиране на житейски задачи от не-тегловните графи.

Тегловни могат да бъдат както неориентирани, така и ориентирани графи. В тази лекция разглеждаме само неориентирани графи, тегловни или не.

Определение 70: Покриващ подграф

Нека $G = (V, E)$ е граф. *Покриващ подграф на* G е всеки подграф $G = (V, E')$ на G. *Покриващо дърво на* G, съкратено ПД, е покриващ подграф на G, който е дърво.

На английски термините са съответно *spanning subgraph* и *spanning tree*. Ние няма да разглеждаме други видове покриващи подграфи освен покриващи дървета, но принципно има смисъл да се обобщи "покриващо дърво" до "покриващ подграф". На читателя остава да съобрази, че покриващ цикъл е същото като Хамилтонов цикъл.

Наблюдение 46

За всеки граф G, G има поне едно покриващо дърво тстк G е свързан.

Броят на покриващите дървета е най-много n^{n-2} и тази граница е точна. Този резултат обаче е нетривиален и затова го показваме в Допълнение 42. Това, което трябва да запомним, е, че в общия случай, броят на покриващите дървета е грамаден.

Допълнение 42: За броя на покриващите дървета

Броят на покриващите дървета на граф е равен на детерминантата на една матрица, която характеризира графа. Резултатът е в сила дори за мултиграфи (наличието на примки е без значение за покриващите дървета). Този нетривиален резултат е част от алгебричната теория на графите и е открит още през 19 век от великия физик Gustav Kirchhoff. Резултатът присъства имплицитно в негова статия [52] (превод на английски има в [51]). Подробно изследване на работата на Kirchhoff има в [50]. За подробно доказателство на резултата на Kirchhoff вижте [87, глава 9, стр. 135].

С $\kappa(G)$ означаваме броя на покриващите дървета на G.

Определение 71: Матрица на Laplace на граф

Нека $G = (\{1, \ldots, n\}, E)$ е граф. *Матрицата на Laplace* на граф G е квадратна, $n \times n$, симетрична матрица L, където

$$L[i,j] = \begin{cases} d(i), & \text{ако } i = j, \\ -1, & \text{ако } i \neq j \text{ и } (i,j) \in E, \\ 0, & \text{в противен случай} \end{cases}$$

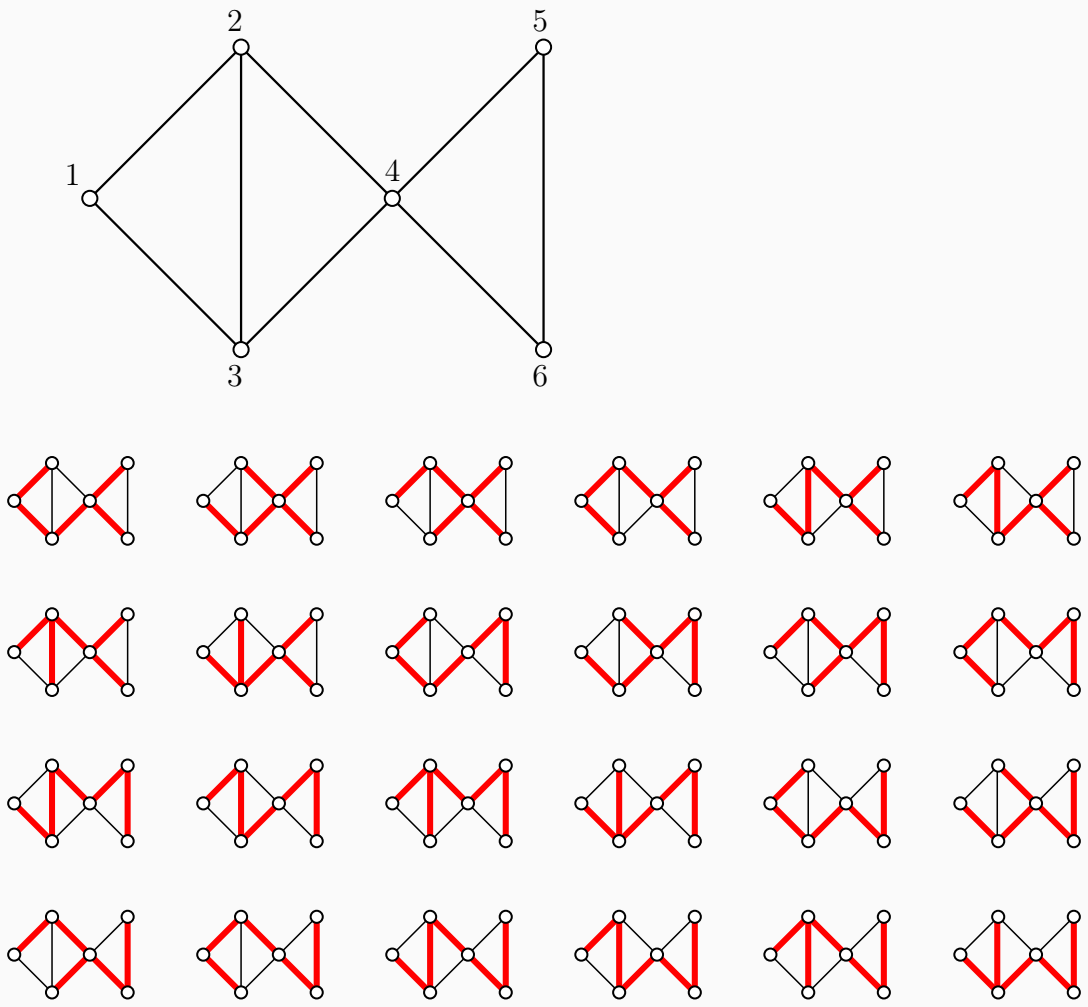
Теорема 58: The Matrix-Tree Theorem, [87, 9.8 Theorem, стр. 141]

Нека G е граф без примки с матрица на Laplace L. Нека L_0 означава L с последния ред и колона изтрити (или, по-общо казано, с i-ия ред и колона изтрити, за кое да е i). Тогава

$$\det(L_0) = \kappa(G)$$

Например, да разгледаме графа на Фигура 11.1. Той има 24 покриващи дървета, показани под него.

Фигура 11.1 : Граф и неговите покриващи дървета.



Матрицата на Laplace за този граф е

$$\begin{bmatrix} 2 & -1 & -1 & 0 & 0 & 0 \\ -1 & 3 & -1 & -1 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 & 0 \\ 0 & -1 & -1 & 4 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{bmatrix}$$

Съгласно Теорема 58, броят на покриващите го дървета е

$$\kappa(G) = \det \begin{bmatrix} 2 & -1 & -1 & 0 & 0 \\ -1 & 3 & -1 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 4 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix} = 24$$

Теорема 59: Corollary 9.10-b на стр. 142 в [87]

Нека G е свързан, d-регулярен, граф без примки с матрица на съседство A. Нека собствените стойности на матрицата на съседство A са $\lambda_1, \ldots, \lambda_{n-1}, \lambda_n$, като $\lambda_n = d$. Тогава

$$\kappa(G) = \frac{(d-\lambda_1)(d-\lambda_2)\cdots(d-\lambda_{n-1})}{n}$$

Теорема 60: Proposition 1.5 на стр. 4 в [87]

Собствените стойности на матрицата на съседство за пълния граф K_n са следните:

- -1 с кратност $n-1$ и
- $n-1$ с кратност 1.

Следствие 21

Броят на покриващите дървета на K_n е

$$\kappa(K_n) = \frac{(n-1-(-1))(n-1-(-1))\cdots(n-1-(-1))}{n} = \frac{n^{n-1}}{n} = n^{n-2}$$

Разбира се, този резултат е в сила само ако пълният граф K_n е именуван и покриващите дървета са именувани. Броят на неименуваните покриващи дървета (неизоморфните покриващи дървета) е значително по-малък.

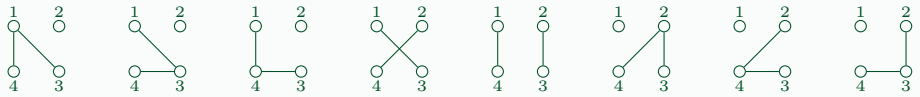
Формулата n^{n-2} за броя на покриващите дървета на K_n се нарича *формула на Cayley*. Този резултат може да се изведе и с комбинаторни, а не алгебрични средства според [3, стр. 204]. Сега мислим не за покриващите дървета на именуван K_n, а просто за възможностите да конструираме дървета над n върха.

Теорема 61: стр. 204 в [3]

Разглеждаме множество от върхове $V = \{1, \ldots, n\}$. Нека A е произволно k-елементно подмножество на V. Тогава броят на именуваните гори над V, състоящи се от точно k дървета, като всеки връх от A е в различно дърво, е:

$$T_{n,k} = \begin{cases} \sum_{i=0}^{n-k} \binom{n-k}{i} T_{n-1,k-1+i}, & \text{ако } n \geqslant k > 1 \\ 0, & \text{ако } n \geqslant 1 \text{ и } k = 0 \\ 1, & \text{ако } n = 0 \text{ и } k = 0 \end{cases} \tag{11.1}$$

Първо да видим пример. Нека $n = 4$, $k = 2$ и $A = \{1, 2\}$.



Вижда се, че $T_{4,2} = 8$.

Доказателство: Началните условия са очевидни. Ще докажем рекурсивната част от формулата. Очевидно няма значение точно кои върхове са в A; значение има само кардиналността на A. БОО, нека $A = \{1, \ldots, k\}$. Фиксираме произволен връх от A. Нека това е връх 1. Индексната променлива i има смисъл на броя съседите на връх 1. Те може да са от 0 до $n - 1 - (k - 1) = n - k$, което дава и границите на сумирането в (11.1).
Изтриваме връх 1 и получаваме гора с $n - 1$ върха и с точно $k - 1 + i$ дървета, защото едно дърво изчезва, а се появяват i нови дървета. Различните гори с $n - 1$ върха и $k - 1 + i$ дървета са $T_{n-1,k-1+i}$. Връщаме връх 1, като трябва да го направим съсед на точно i върха измежду върховете от $V \backslash A$. Но $|V \backslash A| = n - k$, следователно има точно $\binom{n-k}{i}$ начина да изберем съседите на върнатия връх 1. Всяко свързване на връх 1 с i върха измежду върховете на някоя гора (общо $T_{n-1,k-1+i}$ горй) дава различна гора, което доказва и верността на (11.1).
Това, че $T_{n,n} = 1$ следва от (11.1). □

Теорема 62: стр. 204 в [3]

В сила е

$$T_{n,k} = kn^{n-k-1} \tag{11.2}$$

Доказателство: С индукция по (11.1). Базовите случаи $n \geqslant 1, k = 0$ и $n = k = 0$ са очевидни.

В индуктивната стъпка имаме:

$$\begin{aligned}
T_{n,k} &= \sum_{i=0}^{n-k} \binom{n-k}{i} T_{n-1,k-1+i} = \sum_{i=0}^{n-k} \binom{n-k}{i} (k-1+i)(n-1)^{n-1-(k-1+i)-1} \\
&= \sum_{0 \leqslant i \leqslant n-k} \binom{n-k}{i} (k-1+i)(n-1)^{n-k-i-1} \\
&= \sum_{0 \leqslant n-k-i \leqslant n-k} \binom{n-k}{n-k-(n-k-i)} (k-1+(n-k-i))(n-1)^{n-k-(n-k-i)-1} \\
&= \sum_{0 \leqslant i \leqslant n-k} \binom{n-k}{i} (n-i-1)(n-1)^{i-1} \\
&= \sum_{i=0}^{n-k} \binom{n-k}{i} (n-1)^i - \sum_{i=0}^{n-k} \binom{n-k}{i} i(n-1)^{i-1} \\
&= \sum_{i=0}^{n-k} \binom{n-k}{i} (n-1)^i 1^{n-k-i} - \sum_{i=1}^{n-k} \binom{n-k-1}{i-1} \frac{n-k}{i} i(n-1)^{i-1} \\
&= (n-1+1)^{n-k} - (n-k) \sum_{i=0}^{n-k-1} \binom{n-k-1}{i} (n-1)^i 1^{n-k-1-i} \\
&= n^{n-k} - (n-k)(n-1+1)^{n-k-1} = n^{n-k} - (n-k)n^{n-k-1} \\
&= n^{n-k} - n^{n-k} + kn^{n-k-1} = kn^{n-k-1} \qquad (11.3)
\end{aligned}$$

□

Следствие 22

В контекста на Теорема 61, $T_{n,1}$ очевидно е броят на дърветата над V. Според Теорема 62, $T_{n,1} = 1n^{n-1-1} = n^{n-2}$. Тогава броят на дърветата е n^{n-2}.

Определение 72: Тегло на ПД. Минимално ПД.

Нека G е свързан тегловен граф с тегловна функция w. Нека T е ПД на G. *Теглото на* T е $w(T) = \sum_{e \in E(T)} w(e)$. Минимално ПД, съкратено МПД, е такова ПД T', че за всяко ПД T'' е вярно, че $w(T') \leqslant w(T'')$.

Следното определение е буквален превод на “safe edge” от [21, стр. 626].

Определение 73: Сигурно ребро.

Нека G е тегловен граф и $A \subseteq E$ е такова, че съществува МПД T, такова че $A \subseteq E(T)$. Нека e е ребро на G, което не е от A. Казваме, че e *е сигурно за* A, ако съществува МПД T', такова че $A \cup \{e\} \subseteq E(T')$.

Общата идея на алгоритмите за конструиране на МПД се съдържа в следния псевдокод ([21, стр. 626]). МПД се определя напълно от своите ребра, така че да конструираме МПД е същото като да конструираме множеството от ребрата му.

Общ вид на МПД(G: граф, w: тегловна функция)
```
1  A ← ∅
2  while A не образува ПД do
3      намери ребро e, което е сигурно за A
4      A ← A ∪ {e}
5  return A
```

Ясно е, че във всеки момент от работата на алгоритъм, изграден върху тази схема, ребрата от A дават ацикличен подграф на G; в противен случай, e не би било сигурно. Алгоритъмът спира, когато множеството от върховете-краища на ребрата от A стане V. Веднага следва, че **while**-цикълът се изпълнява точно $n-1$ пъти.

Цялата трудност тогава е, как да намираме сигурно ребро по отношение на множество от ребра. Теорема 63 дава достатъчна обосновка за намирането на сигурни ребра от алгоритмите, които ще разгледаме. Първо ни трябват няколко определения.

Определение 74: Срез в граф.

Срез в граф G е всяко разбиване на $V(G)$ на два дяла. Нека $S = \{V_1, V_2\}$ е срез в G. Ребро $e = (u, v)$ *прекосява* S, ако $u \in V_1$ и $v \in V_2$. Нека $E' \subseteq E(G)$. S е *съобразен с* E', ако нито едно ребро от E' не го прекосява. Ако G е тегловен и реброто e прекосява S, казваме, че реброто e е *леко*, ако e има минимално тегло измежду всички ребра, които прекосяват S.

Теорема 63: Theorem 23.1 от [21, стр. 627]

Нека $G = (V, E)$ е свързан тегловен граф с тегловна функция $w : E \to \mathbb{R}$. Нека $A \subseteq E$ е такова, че съществува МПД T, че $A \subseteq E(T)$. Нека $S = \{V_1, V_2\}$ е срез в G, който е съобразен с A. Нека $e = (u, v)$ е произволно леко ребро, прекосяващо S. Тогава e е сигурно за A.

Доказателство: Ако $e \in E(T)$, то ние сме готови с доказателството. Да допуснем, че $e \notin E(T)$. Ще покажем, че има МПД T', такова че $A \cup \{e\} \subseteq E(T')$.

Добавяме e към T и получаваме уницикличен граф U. Нещо повече, със сигурност e е ребро от цикъла c на U. За реброто e знаем, че прекосява среза. Със сигурност съществува ребро e' в c, различно от e, което прекосява среза. Изтриваме e' от U и получаваме ПД T', различно от T. Да сравним $w(T)$ с $w(T')$. T' се получи от T с добавяне на e и изтриване на e'. Тогава

$$w(T') = w(T) + w(e) - w(e')$$

Но e е леко по конструкция, така че $w(e) \leqslant w(e')$, следователно $w(e) - w(e') \leqslant 0$. Тогава $w(T') \leqslant w(T)$. Но T е МПД, така че $w(T) \leqslant w(T')$ по дефиниция. Тогава $w(T) = w(T')$ и T' също е МПД. Което съдържа както ребрата от A, така и e. □

11.2 Алчни алгоритми

Предложената обща идея за намиране на МПД на тази страница заедно с Теорема 63 се конкретизира до два алгоритъма, които ще разгледаме в следващите две секции. И два-

та алгоритъма са изградени по *алчна схема* (на английски, *greedy strategy*). Алгоритмите, изградени по тази схема, се наричат *алчни алгоритми* (*greedy algorithms*).

По правило, изчислителните задачи, които алчните алгоритми решават, са оптимизационни задачи (Определение 4). Като пример, задачата за намиране на МПД е оптимизационна. Всеки пример е наредена двойка от свързан граф и тегловна функция. Множеството от допустимите решения е множеството от покриващите дървета на графа, като то е задължително непразно[†]. Целевата функция изобразява допустимо решение (ПД) в число, което е сумата от теглата на ребрата му. Целта е минимизация. Оптималните решения са минималните покриващи дървета.

Друг пример за оптимизационна задача е задачата за намиране на най-лек Хамилтонов цикъл в неориентиран тегловен граф. Да кажем, че теглото на цикъл е сумата от теглата на ребрата в него. Сега множеството от допустимите решения е множеството от всички Хамилтонови цикли и то може да е празно, защото не всеки граф е Хамилтонов. Целевата функция изобразява цикъл в сумата от теглата на ребрата му. Множеството от оптималните решения се състои от Хамилтоновите цикли с минимална сума от теглата на ребрата.

Оптимизационните задачи МПД и Най-лек Хамилтонов цикъл се различават драстично по сложността на алгоритмите, които ги решават. Алгоритмите за МПД имат квадратична сложност (или сложност, не много по-голяма от квадратична, както ще видим в следващите секции), докато Най-лек Хамилтонов цикъл е **NP**-трудна, което почти сигурно означава, че за нея няма ефикасен алгоритъм. Със сигурност до момента такъв не е известен, като евентуалното откриването на ефикасен алгоритъм за нея би бил събитие, за което ще се говори в новините.

Задачата МПД се решава ефикасно с алгоритми, които работят итеративно, на всяка итерация добавяйки точно едно ребро към множеството от вече добавените ребра. Критерият, по който се избира ребро за добавяне, е много прост и ясен: добавя се леко ребро, прекосяващо някакъв срез (в алгоритъма на Kruskal: стига то да не образува цикъл с досега сложените ребра). Всеки такъв избор на леко ребро е *локален*, защото не се прави с оглед на "голямата картина". Теорема 63 гарантира, че **поредицата от локално оптимални избори** (на леки ребра) води до **глобално оптимално решение** (МПД).

Би трябвало да е ясно защо такива алгоритми се наричат алчни. Терминът "алчни" не е съвсем прецизен, понеже всеки алгоритъм, който решава оптимизационна задача, е в някакъв смисъл алчен. При алчните алгоритми става дума за някаква **късогледа алчност**, или **локална алчност**. Когато решаваме МПД, локалната алчност е печеливша стратегия. Има обаче задачи, в които проявата на локална алчност води до решение, което не е оптимално. Всички **NP**-трудни задачи са такива, но примерът с Най-лек Хамилтонов цикъл не е особено удачен за тази илюстрация, поради което ще илюстрираме с друга задача: Knapsack, или задачата за раницата (Подсекция 13.5.3). Пример за не-оптималността на очевидното алчно решение на задачата за раницата има на стр. 515.

Подробно теоретично изложение на фундамента на алчните алгоритми има в Секция 16.2 на [21, стр. 423] и Секция 16.4 на [21, стр. 437], както и [73, стр. 280]. Това е материал, който е далече отвъд обхвата на тези лекции.

Говорейки полуформално за алчни алгоритми и алчни схеми, авторът на тези записки е на мнение, че има неизбежен субективизъм/условност в това, което наричаме "локално оптимален избор" (което дефинира алчните схеми). От някаква гледна точка, всеки избор (на нов елемент, който да добавим в множеството) е локален. Ако искаме да прецизираме "локално оптимален избор", трябва да говорим за максималните ресурси (време/памет), които

[†]Това е особеност на тази задача, в други задачи множеството от допустимите решения може и да е празно.

сме склонни да отделим за него.

11.3 Алгоритъм на Prim

Алгоритъмът на Prim за намиране на МПД далечно прилича на обхождане. Той започва от един стартов връх s, който е част от входа, намира най-леко ребро, свързващо s с друг връх, слага го в дървото, после намира най-леко ребро, свързващо някой от тези два върха с друг връх, слага го в дървото, и така нататък. В терминологията на Общ вид на МПД на стр. 418, в алгоритъма на Prim, ребрата от A във всеки момент образуват едно (свързано) дърво. Естествено, то става ПД чак накрая, когато включи всички върхове, но във всеки момент от изпълнението то е едно дърво. Ето много общ псевдокод на алгоритъма на Prim.

Prim generic($G = (V, E)$: неориентиран свързан граф, w: тегловна функция върху G, s: стартов връх)
1 $V(T) \leftarrow \{s\}$
2 $A \leftarrow \varnothing$
3 **while** $V \backslash V(T) \neq \varnothing$ **do**
4 намери леко ребро $e = (x, y)$, прекосяващо среза $\{V(T), V \backslash V(T)\}$
5 нека $x \in V(T)$
6 $V(T) \leftarrow V(T) \cup \{y\}$
7 $A \leftarrow A \cup \{e\}$
8 **return** A

Коректността е очевидна и следва веднага от Теорема 63. От общи съображения обаче сложността не е добра. Очевидно **while**-цикълът се изпълнява $n - 1$ пъти. Потенциално проблематично е намирането на леко ребро (ред 4) на всяка итерация. Ако го правим с пълно изчерпване, тоест, минаваме през всички ребра на G и търсим ребро, прекосяващо среза с минимално тегло, само ред 4 би отнемал $\Theta(m)$ време на всяка итерация, поради което сложността на алгоритъма би била $\Theta(nm)$. Това е неприемливо бавно.

Известно подобрение би било на всяка итерация да не преглеждаме всички ребра на графа, а само тези, които прекосяват среза $\{V(T), V \backslash V(T)\}$. В асимптотичния смисъл обаче това не е никакво подобрение. В най-лошия случай, общият брой на прегледани ребра (за цялото изпълнение на алгоритъма) се оценява със сумата

$$\sum_{k=1}^{n-1} k(n-k) = \Theta(n^3)$$

тъй като има ребро между всеки връх от единия дял и всеки връх от другия дял, на всяка итерация (какъв граф трябва да е G, за да изпълнено това?). И така, отново имаме, в най-лошия случай, кубичен алгоритъм. Трябва да направим нещо по-умно, за да избираме ефикасно леко прекосяващо ребро.

В следващите две подсекции ще разгледаме два варианта на алгоритъма на Prim. Единият не използва изтънчени структури данни и ще го наречем "базов", а другият ползва АТД приоритетна опашка и ще го наречем "изтънчен". Върху разредени графи изтънченият вариант е по-добър, но върху графи с брой ребра, близък до максималния, базовият алгоритъм, колкото и да е странно, е по-бърз.

11.3.1 Базов вариант на алгоритъма на Prim

Този алгоритъм следва плътно PRIM GENERIC, като намирането на леко ребро на ред 4 става във време $O(n)$, като в най-лошия случай това е $\Theta(n)$. Идеята е елементарна. За да я изложим, ще въведем класификация на върховете. Нека във всеки момент от работата на алгоритъма,

- върховете от $V(T)$ са *дървесните върхове* (на английски, *tree vertices*),
- върховете от $N(V(T))$ са *граничните върхове* (на английски, *fringe vertices*),
- върховете от $V \setminus N[V(T)]$ са *неизвестните върхове* (на английски, *unseen vertices*).

Тази класификация на практика дублира класификацията от обхождането на графи на върховете на бели, сиви и черни на стр. 370, като черните върхове сега наричаме дървесни, сивите наричаме гранични, а белите са неизвестните. Причината да въвеждаме с други имена практически същата класификация е широко приетата терминология: класификацията на бели, сиви и черни е от [21], а на tree, fringe и unseen е например от [86, стр. 485].

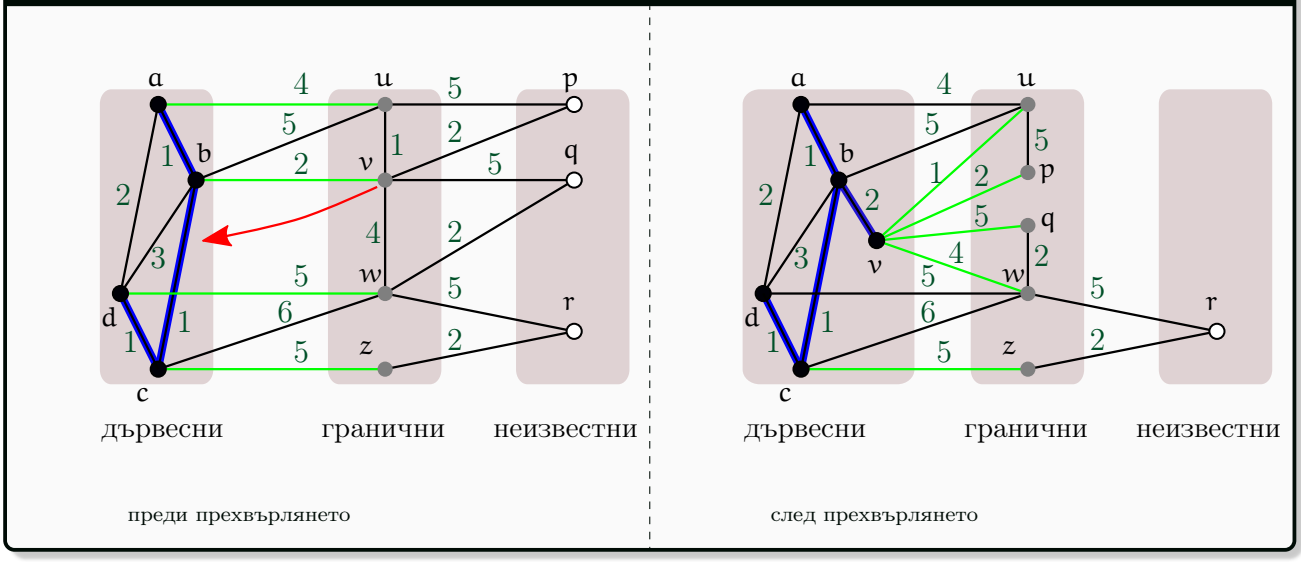
И така, сега класифицираме върховете на дървесни, гранични и неизвестни. Множеството от граничните върхове ще наричаме *границата*. На всяка итерация точно един граничен връх става дървесен. Нещо повече. Това е такъв граничен връх, който е инцидентен с леко прекосяващо ребро. В термините на PRIM GENERIC, това е връх y. За да направим избора на ред 4, достатъчно е да прегледаме всички гранични върхове; ако за всеки от тях предварително е известно теглото на най-леко инцидентно прекосяващо ребро и самото ребро, може да направим избора във време, пропорционално на размера на границата, което, в най-лошия случай е $\Theta(n)$. Иначе казано, вместо да търсим директно леко ребро, както пише на ред 4, ще търсим граничен връх, инцидентен с такова, а, когато намерим върха, в $\Theta(1)$ ще намерим и реброто, ако предварително всеки граничен връх е бил асоцииран с най-леко инцидентно прекосяващо ребро. Ако може да работим във време $\Theta(1)$ за всеки граничен връх, ще може да сведем сложността на алгоритъма до $\Theta(n^2)$.

Сега се фокусираме върху това, как да работим във време $\Theta(1)$ за всеки граничен връх. Въвеждаме понятието *кандидат-ребро*, или просто *кандидат*. По време на работата на алгоритъма, всеки граничен връх е асоцииран с точно едно кандидат-ребро и с една стойност, наречена *гранично тегло*, която е теглото на кандидат-реброто. За дървесните и неизвестните върхове кандидат-ребра и гранични тегла не се дефинират.

Всеки граничен връх може да е инцидентен с дървесни, гранични и неизвестни върхове. След избора на ред 4 прехвърляме връх от границата в дървото, като обаче това може да наложи промени в някои кандидат-ребра и гранични тегла, така че на следващата итерация, стойностите на всички кандидат-ребра и гранични тегла да са коректни и отново да можем да направим избора, извършвайки $\Theta(1)$ работа за всеки граничен връх. Това е показано на Фигура 11.2. Дървесните върхове са вляво, граничните са по средата, а неизвестните са вдясно. Ребрата в дървото са в синьо. Кандидат-ребрата са в ярко зелено. В лявата подфигура е показано състоянието на нещата в началото на някоя итерация. Граничните върхове са u, v, w и z със съответни гранични тегла 4, 2, 5 и 5. Върхът с минимално гранично тегло е v и той бива прехвърлен към дървесните върхове, както показва червената стрелка. След прехвърлянето (дясната подфигура), реброто (b, v) става част от дървото. Върховете, които са неизвестни съседи на v преди прехвърлянето (p и q) сега вече са гранични, като кандидат-ребрата им са съответно (p, v) и (q, v). Преди прехвърлянето v е съсед на граничните върхове u и w. След прехвърлянето на v, те си остават гранични, но кандидат-ребрата им се променят на съответно (u, v) и (w, v), понеже и двете имат по-малко тегло от съответните

им кандидат-ребра (a, u) и (d, w) преди прехвърлянето. И така, дясната подфигура показва състоянието на нещата в началото на следващата итерация.

Фигура 11.2 : Прехвърляне на връх от границата в дървото при Prim.



Ето псевдокод на базовия вариант на алгоритъма на Prim. Ползва се масив status$[1, \ldots, n]$, където status$[i] \in \{$дървесен, граничен, неизвестен$\}$. Този масив е аналог на масива color от обхожданията.

МПД Prim1($G = (V, E)$: неориентиран свързан граф, w: тегловна функция върху G, s: стартов връх)

```
1   V(T) ← {s}
2   A ← ∅
3   foreach u ∈ V
4       status[u] ← неизвестен
5   status[s] ← дървесен
6   foreach y ∈ adj[s]
7       status[y] ← граничен
8   x ← s
9   while V\V(T) ≠ ∅ do
10      (* x е последният сложен в дървото връх *)
11      foreach y ∈ adj[x]
12          if status[y] = граничен
13              ако w((x,y)) е по-малко от граничното тегло на y,
                        смени кандидат-реброто на y с (x,y)
14          if status[y] = неизвестен
15              status[y] ← граничен
16              кандидат-реброто на y става (x,y)
17      намери кандидат-ребро e с минимално тегло
18      x ← граничният връх на e
19      status[x] ← дървесен
20      V(T) ← V(T) ∪ {x}
21      A ← A ∪ {e}
22  return A
```

Коректност и сложност. Подробно доказателство за коректност няма да правим. Инвариантът е, при всяко достигане на управлението на ред 9, ребрата от A задават МПД на подграфа на G, индуциран от $V(T)$. Използвайки Теорема 63, лесно показваме, че реброто e на ред 17 е сигурно за A.

Сложността по време е $\Theta(n^2)$. Намирането на най-леко кандидат-ребро на ред 17 се извършва в $\Theta(n)$ в най-лошия случай. Имайки предвид, че **while**-цикълът се "извърта" $\Theta(n)$ пъти, това дава директно $\Theta(n^2)$. Знаем, че обхождането на списъците на редове 11–16 става във време $\Theta(n+m)$ за цялата работа на алгоритъма. Тъй като $\Theta(m+n) = O(n^2)$, имаме обща асимптотична оценка за сложността $\Theta(n^2)$. Забележе, че по отношение на най-лошия случай, тази сложност по време е оптимална, защото в най-лошия случай описанието на графа е с размер $\Theta(n^2)$, а за всяко n има примери за граф с n върха и тегловна функция, такива че се налага да разгледаме цялото описание на графа, за да конструираме МПД.

11.3.2 Изтънчен вариант на алгоритъма на Prim

Този вариант на алгоритъма на Prim прави бърз избор на минимално ребро, което да бъде добавено към временното дърво. Използва се АТД приоритетна опашка Q от min тип от върховете. Ключът на всеки връх в Q е минималното тегло на реброто, свързващо този връх с връх от временното дърво. Ако такова ребро няма, ключът е ∞. В Q поначало се слагат всички върхове на графа с ключове ∞ (защото в този псевдокод поначало дърво няма), а след това на всяка итерация Q съдържа точно върховете, които **не са в дървото**; тоест, Q съдържа точно граничните и неизвестните върхове. Ключовете пишем така: ако върхът е u, неговият ключ е u.key.

Самото МПД се реализира с масив на предшествията. За разлика от [21], този масив ще записваме като $\pi[1, \ldots, n]$.

МПД PRIM2($G = (V, E)$: неориентиран свързан граф, w: тегловна функция върху G, s: стартов връх)

```
 1  foreach u ∈ V
 2      u.key ← ∞
 3      π[u] ← Nil
 4  s.key ← 0
 5  създай празна приоритетна min опашка Q
 6  Q ← V
 7  while not isempty(Q) do
 8      x ← EXTRACT-MIN(Q)
 9      foreach y ∈ adj[x]
10          if y ∈ Q and w((x, y)) < y.key
11              y.key ← w((x, y))
12              π[y] ← x
13  return π[1, ..., n]
```

Този псевдокод следва плътно псевдокода от [21, стр. 634]. Две забележки по имплементацията. Първо, проверката $y \in Q$ на ред 10 може да бъде имплементирана, като всеки връх има булев флаг за принадлежност към опашката; в началото всички флагове са TRUE, а при всяко вадене на връх от опашката на ред 8, правим флага му FALSE. Второ, смяната на ключа на y на ред 11 не е просто смяна на едно число с друго, както може човек да си помисли първо. Връх y е в опашката и смяната на ключа му, тоест, намаляването на ключа

му, трябва да бъде съпроводено с изпълнение на DECREASE-KEY, която е огледален аналог на INCREASE-KEY от Подсекция 4.4.4. Това има значение за анализа на сложността.

Коректност.

Теорема 64: Коректност на МПД PRIM2

Масивът $\pi[1,\ldots,n]$, който МПД PRIM2 връща, реализира МПД на G.

Доказателство. Първо едно помощно твърдение.

Инвариант 24: while-цикълът на МПД PRIM2

При всяко достигане на ред 7:

① $Q \neq \varnothing \to (\exists u \in Q : u.key < \infty)$.

② $\forall u \in Q \backslash \{s\}$: ако $u.key < \infty$, то:

- ❶ $\pi[u] \neq Nil$
- ❷ $\pi[u] \notin Q$
- ❸ $u.key$ е теглото на най-леко ребро, инцидентно с u и прекосяващо среза $\{V \backslash Q, Q\}$.

③ $\forall u \in Q$: ако $u.key = \infty$, то: съществува $v \in Q$, такъв че $v.key < \infty$ и има път между u и v, чиито върхове са само от Q.

База. Разглеждаме първото достигане на ред 7. Тогава $Q = V$ (ред 6), така че Q не е празна и антецедентът на ① е истина. Но консеквентът също е истина, понеже има връх във V (и оттам, в Q) със стойност key, по-малка от ∞: това е връх s (ред 4). Тогава ① е в сила.

Да разгледаме ②. Тъй като единственият връх в Q със стойност key, която не е ∞, е s, антецедентът на импликацията е лъжа за всяко $u \in Q \backslash \{s\}$ и оттам, импликацията е истина за всяко $u \in Q \backslash \{s\}$.

Да разгледаме ③. Истинността следва от това, че $V = Q$, $s.key < \infty$ и G е свързан. ✓

Поддръжка. Да допуснем, че твърдението е вярно за някое достигане на ред 7, което не е последното. Нека моментът на това достигане е t, а следващото достигане е в момента t'.

Ще докажем ①. Ако след изваждането на връх от Q на ред 8, Q стане празна, антецедентът на ① става лъжа, а цялата импликация, истина. Нека след ред 8 Q не е празна. Ако в момента t съществува $z \in Q$, $z \neq x$, такъв че $z.key < \infty$, доказателството е готово. Да допуснем, че в момента t, за всеки $z \in Q$, $z \neq x$, е вярно, че $z.key = \infty$. Но част ③ от индуктивното предположение влече, че x поне един съсед z' в Q. Очевидно на ред 11 $z'.key$ ще стане $w((x,z'))$, поради което в момент t' отново е вярно, че Q съдържа връх с key стойност, по-малка от ∞.

Ще докажем ②. Нека в момента t, $Q_f = \{v \in Q \mid v.key < \infty\}$ и $Q_\infty = \{v \in Q \mid v.key = \infty\}$. Ще разгледаме три множества: $Q_f \backslash N(x)$, $Q_f \cap N(x)$ и $Q_\infty \cap N(x)$.

Върховете от $Q_f \backslash N(x)$ не биват засегнати от изваждането на x от Q и за тях ❶, ❷ и ❸ остават в сила в момента t'.

Разглеждаме произволен $v \in Q_f \cup N(x)$. Ако в момента t, $w((x,v)) \geqslant v.key$, то булевото условие на ред 10 е лъжа, когато y стане v, и редове 11–12 не се изпълняват. Тогава в

момента t', ❶, ❷ и ❸ остават в сила за v от индукционното предположение. Ако в момента t, $w((x,v)) < v.key$, то булевото условие на ред 10 е истина, когато y стане v (ред 9), и редове 11–12 се изпълняват. Тогава $\pi[v]$ става x (ред 12) и ❶ и ❷ са в сила за v. На ред 11, $v.key$ става $w((x,v))$. Очевидно сега (x,v) е най-лекото, инцидентно с v, прекосяващо ребро, така че и ❸ е в сила.

Разглеждаме произволен $v \in Q_\infty \cup N(x)$. Когато y стане v (ред 9), булевото условие на ред 10 задължително е истина, понеже $v \in Q$ и $v.key = \infty$. После на ред 12, $\pi[v]$ става x, така че ❶ и ❷ са в сила за v. На ред 11, $v.key$ става $w((x,v))$. Очевидно сега (x,v) е най-лекото (и единствено), инцидентно с v, прекосяващо ребро, така че и ❸ е в сила.

Ще докажем ③. След изваждането на x от Q, за всеки $v \in Q_\infty \cap N(x)$, $v.key$ става $w((x,v))$ на ред 11 и v престава да е елемент на Q_∞. От друга страна, за всеки $v \in Q_\infty \backslash N(x)$, твърдението остава в сила заради свързаността на графа. □

Нека във всеки момент от работата на алгоритъма, $G_{\overline{Q}}$ е подграфът на G, индуциран от върховете, които не са в Q. Твърди се, че съществува МПД T на G, спрямо което Инвариантът 25 е в сила.

Инвариант 25: while-цикълът на МПД Prim2

При всяко достигане на ред 7, $\{(u, \pi[u]) \mid u \in V \backslash (\{s\} \cup Q)\} = E(T) \cap E(G_{\overline{Q}})$.

База. Разглеждаме първото достигане на ред 7. В този момент $\{s\} \cup Q = V$, така че $V \backslash (\{s\} \cup Q) = \varnothing$ и множеството в лявата страна е празно. Но множеството в дясната страна също е празно, понеже всички върхове са в Q и $E(G_{\overline{Q}})$ е празно. Със сигурност такова МПД T съществува.

Поддръжка. Да допуснем, че инвариантът е в сила в даден момент t, в който изпълнението е на ред 7 и **while** ще бъде изпълнен поне още веднъж, което влече, че Q не е празна. Щом Q не е празна, съгласно Инвариант 24 е вярно, че $\exists u \in Q : u.key < \infty$.

Твърди се, че в момента t за всеки връх $v \in Q$, такъв че $v.key$ е минимален, е вярно, че реброто $(v, \pi[v])$ е най-леко ребро, прекосяващо среза $\{V \backslash Q, Q\}$. Но това следва директно от Инвариант 24. Съгласно Теорема 63, в момента t за всеки връх $v \in Q$, такъв че $v.key$ е минимален, е вярно, че реброто $(v, \pi[v])$ е сигурно за множеството $E(T) \cap E(G_{\overline{Q}})$, защото това множество е съобразено със среза среза $\{V \backslash Q, Q\}$.

На ред 8 върхът, който бива изваден от опашката и съхранен в променливата x е връх, който в момента t е в Q и е с минимална key стойност – при допускането, че Extract-Min работи коректно. Заключаваме, че след изваждането на този връх от Q, множеството $\{(u, \pi[u]) \mid u \in V \backslash (\{s\} \cup Q)\}$ продължава да е равно на $E(T) \cap E(G_{\overline{Q}})$.

Терминация. При последното достигане на ред 7, опашката Q е празна. Тогава $E(G_{\overline{Q}}) = E(G)$, така че $E(T) \cap E(G_{\overline{Q}}) = E(T)$; освен това, $V \backslash (\{s\} \cup Q) = V \backslash \{s\}$; заключаваме, че $\{(u, \pi[u]) \mid u \in V \backslash \{s\}\} = E(T)$. С други думи, МПД Prim2 е построил МПД чрез $\pi[1, \ldots, n]$. □

Сложност по време. Допускаме, че Q е реализирана чрез двоична пирамида. Ще приложим подхода към анализа на сложността, който ползвахме при BFS/DFS, и ще игнорираме факта, че **while**-цикъла (редове 7–12) се изпълнява $\Theta(n)$ пъти. В израза за сложността няма множител n. Наместо да броим колко пъти се "извърта" **while**-а, ще разсъждаваме колко пъти се изпълнява **for**-цикъла на редове 9–12 за **цялото изпълнение на алгоритъма**.

Както знаем, ред 9 се изпълнява $\Theta(n+m)$ пъти, защото това е просто минаване през списъците на съседство. Само че тялото на този **for** се изпълнява в най-лошия случай във време $\Theta(\lg n)$ заради имплицитното викане на DESCREASE-KEY на ред 11. Оттук имаме оценка за сложността $\Theta((n+m)\lg n)$, което при свързан граф е $\Theta(m \lg n)$. Колкото и да е странно, ако ребрата на графа са много в смисъл, че $m \asymp n^2$, изтънчения вариант на алгоритъма на Prim с приоритетна опашка е по-лош от базовия вариант, в който намираме следващото ребро с просто последователно търсене. За разредени графи обаче предимството на изтънчения вариант е сериозно: $n \lg n$ срещу n^2.

Забележете, че в това извеждане игнорирахме факта, че EXTRACT-MIN на ред 8 работи във време $\Theta(\lg n)$. Ако сумираме цялата работа на EXTRACT-MIN по време на изпълнението на алгоритъма, ще получим $\Theta(n \lg n)$. Но $\Theta(n \lg n) = O(m \lg n)$ за свързан граф, така че изразът за сложността може да бъде записан най-кратко като $\Theta(m \lg n)$.

Ако Q бъде реализирана не чрез двоична пирамида, а чрез пирамида на Fibonacci (Fibonacci heap) може сложността да се подобри, поне в асимптотичния смисъл. При пирамидите на Fibonacci, функцията DESCREASE-KEY работи в амортизирано време $\Theta(1)$. Функцията EXTRACT-MIN остава със сложност $\Theta(\lg n)$ в най-лошия случай, поради което асимптотичната сложност става $\Theta(m + n \lg n)$. Събираемото $n \lg n$ отчита работата на всички EXTRACT-MIN по време на изпълнението на алгоритъма. Пирамида на Fibonacci обаче е много сложна структура, ползването на която води до големи скрити мултипликативни константи (скрити в Θ-нотацията). Трябва данните наистина да са големи, за да има осезаема практическа полза от ползването на пирамиди на Fibonacci наместо двоични пирамиди в алгоритъма на Prim.

Пирамидите на Fibonacci имат и друго предимство пред двоичните пирамиди: те може да бъдат сливани ефикасно, в $\Theta(1)$ амортизирано време, докато двоичните пирамиди искат линейно време за сливане. Пирамидите на Fibonacci са изложени подробно в [21, стр. 505].

11.4 Алгоритъм на Kruskal

Алгоритъмът на Kruskal е ефикасен алгоритъм за намиране на МПД, който е основан на съвсем различна идея от тази на алгоритъма на Prim. Най-общо казано, първо сортираме ребрата по тегло, и после **в ненамаляващия ред** на теглата слагаме първите $n-1$ ребра, за всяко от които, в този ред, е вярно, че не образува цикъл с вече сложените ребра. Ако обаче имплементираме тази идея буквално, ще получим кубичен алгоритъм.

- Сортирането на ребрата става във, и иска, време $\Theta(m \lg m)$, което е същото като $\Theta(n^2 \lg n)$, ако $m \asymp n^2$.

- В най-лошия случай се налага да проверим всички m ребра. За да се убедим в това, нека най-тежкото ребро е мост. Всеки мост задължително участва във всяко МПД, защото всеки мост участва във всяко ПД. Ерго, в най-лошия случай може да се наложи да стигнем до края на сортираната редица от ребра.

  За всяко ребро в сортирана редица тестваме дали образува цикъл с вече сложените ребра. Тоест, за всяко ребро тестваме дали подграфът, индуциран от вече сложените ребра плюс него е цикличен. Ако тестът се направи с BFS или DFS, той става във време $\Theta(n)$ в най-лошия случай[†]. Така че тези тестове ще станат във време $O(nm)$, като може да се покаже, че границата е точна, тоест, $\Theta(nm)$.

[†] А не в $\Theta(n+m)$ – защо?

При $m \asymp n^2$, това е $\Theta(n^2 \lg n) + \Theta(n^3) = \Theta(n^3)$.

Как да подобрим сложността по време? Сортирането на ребрата отнема $\Omega(m \lg n)$ време и това е неизбежно заради долната граница на сортирането (Подсекция 7.2.2)[†]. Ерго, имаме долна граница $\Omega(m \lg n)$ за алгоритъма на Kruskal, независимо от това колко ефикасно имплементираме слагането на ребрата.

Слагането на ребрата става итеративно, като в най-лошия случай се изпълняват m итерации (ако най-тежкото ребро е мост). Имайки предвид това, целта ни е тестването за едно ребро да става във време $O(\lg n)$. Ако успеем да постигнем това, ще имаме $\Theta(m \lg n)$ имплементация на алгоритъма на Kruskal; дори да постигнем тестване за едно ребро във време $o(\lg n)$, пак ще имаме $\Theta(m \lg n)$ имплементация на алгоритъма на Kruskal заради сортирането в началото.

Ключовото наблюдение е, че ако имаме гора с повече от едно дърво и добавим ново ребро между два върха:

- ако те са от различни дървета на гората, няма да се образува цикъл, но тези две дървета плюс новото ребро ще станат едно дърво, а графът ще продължи да е гора;
- ако те са от едно и също дърво, ще се образува цикъл, а графът ще престане да е гора.

В светлината на това, за ефикасното тестване дали всяко ново ребро в наредбата образува цикъл е достатъчно да поддържаме разбиване на върховете на графа, като дяловете на разбиването отговарят на дърветата в гората. С други думи, за всяко дърво от гората ни трябва да знаем само кои са върховете му. Ребрата му не ни интересуват. Наистина, алгоритъмът строи множество от ребра и връща множество от ребра (чиято съвкупност е МПД), но по време на итерациите няма смисъл да поддържаме експлицитно отделните дървета в гората като върхове и ребра – интересуват ни само върховете им. Тестването дали ребро образува цикъл се свежда до това дали двата му края са в различни дялове на разбиването: ако не, това ребро се прескача, ако да, то се добавя, като обаче тези дялове трябва да станат един и същи дял.

Забележете промяната в терминологията, която направихме. Поначало говорехме за тестване дали двата края на новото ребро са в различни дървета, всяко от които си има върхове и ребра, и за сливане на тези дървета плюс новото ребро. Сега говорим за тестване дали двата края на новото ребро са в различни множества (само от върхове) и сливане на тези множества; "плюс новото ребро" сега няма.

Ето псевдокод на алгоритъма на Kruskal съгласно [86]. Алгоритъмът поддържа разбиване на множеството от върховете, като в началото създава разбиването на едноелементни множества; функцията component(u) връща идентификатора на дяла на разбиването, в който е връх u, а функцията identify(component(u), component(v)) слива дяловете, в които се намират u и v, при условие, че тези дялове са различни.

МПД Kruskal($G = (\{1, \ldots, n\}, E)$: неориентиран свързан граф, w: тегловна функция върху G)

```
1  A ← ∅
2  направи разбиването {{1}, {2}, ..., {n}}
3  сортирай E по тегла
4  count ← 0
5  while count < n − 1 do
```

[†]За теглата на ребрата няма ограничения, така че не може да ползваме Counting Sort.

```
 6      нека e = (u, v) е следващото ребро в сортираната редица
 7      if component(u) ≠ component(v)
 8          A ← A ∪ {e}
 9          identify(component(u), component(v))
10          count++
11  return A
```

Коректност. Коректността може да се докаже със следния инвариант: съществува МПД T, такова че при всяко достигане на ред 5 е вярно, че $A \subseteq E(T)$. Реброто e, което избираме на ред 6, има краища u и v. Ако компонентите, в които се намират u и v, да ти наречем T_u и T_v съответно, са различни, булевото условие на ред 7 е истина и изпълнението отива на ред 8. Но щом T_u и T_v са различни, то e прекосява среза $\{V(T_u), V(G) \setminus V(T_u)\}$. Тогава, съгласно Теорема 63, реброто e е сигурно за A.

Сложност по време. За да бъде ефикасен МПД Kruskal, трябва проверката дали $\boxed{\text{component}(u) \neq \text{component}(v)}$ (ред 7) и операцията $\boxed{\text{identify}(\text{component}(u), \text{component}(v))}$ (ред 9) да се вършат във време $O(\lg n)$. Ако успеем да постигнем това, **while**-цикълът (редове 5–10) ще се изпълнява във време $O(m \lg n)$ и алгоритъмът ще има сложност по време $\Theta(m \lg n)$ заради сортирането (ред 3).

Всяка компонента, или дял, на разбиването трябва да има идентификатор; тоест, свое име. Нека са дефинирани две примитивни функции: Find и Union. Ако u е елемент от опорното множество (в случая, връх на графа), Find(u) връща името на дяла на разбиването, в който е u. Ако i и j са имената на два различни дяла, Union(i, j) слива тези два дяла в един и му дава име i. Ако разполагаме с такива примитиви, можем да реализираме $\boxed{\text{component}(u) \neq \text{component}(v)}$ така:

```
component(u) ≠ component(v)
1  i ← Find(u)
2  j ← Find(v)
3  if i ≠ j
4      return True
5  else
6      return False
```

и да реализираме $\boxed{\text{identify}(\text{component}(u), \text{component}(v))}$ така:

```
identify(component(u), component(v))
1  i ← Find(u)
2  j ← Find(v)
3  Union(i, j)
```

В следващата секция ще видим как се реализират ефикасно примитивите Find и Union.

Допълнение 43: Частична наредба $\sqsubseteq$ върху разбиванията

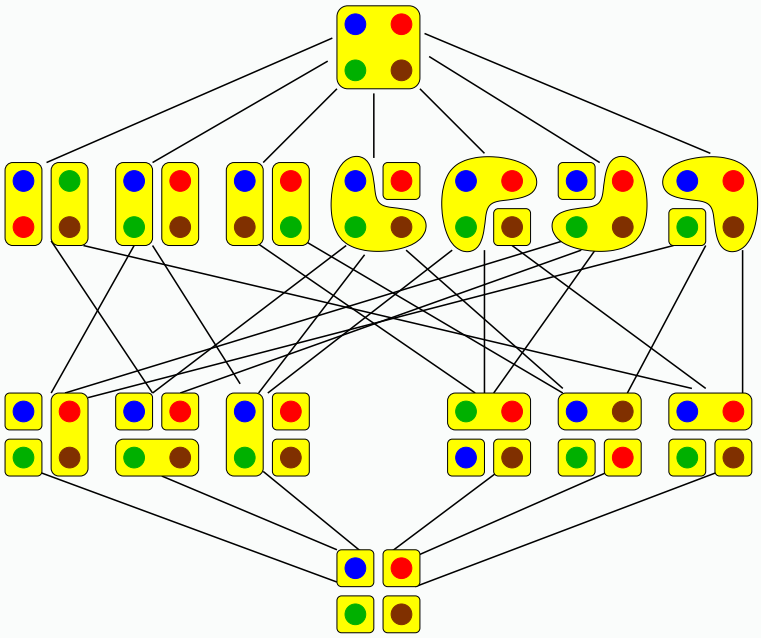
Нека S е множество. Множеството от всички разбивания на S ще бележим с $\Pi(S)$. За

всеки $P_1, P_2 \in \Pi(S)$, P_1 *рафинира* P_2, което означаваме с $P_1 \sqsubseteq P_2$, ако

$$\forall X \in P_1 \;\; \exists Y \in P_2 : X \subseteq Y$$

Конверсно, P_2 *абстрахира* P_1. Примерно, ако $S = \{a, b, c, d\}$, то е вярно, че $\{\{a, b\}, \{c\}, \{d\}\} \sqsubseteq \{\{a, b\}, \{c, d\}\}$ Очевидно релацията $\sqsubseteq$ е частична наредба.

Нареденатa двойка $(S, \sqsubseteq)$ е вид *решетка* – понятие, което няма да дефинираме тук (вижте [13] и [76] за много подробно и изчерпателно въведение). За целите ни е достатъчно да се каже, че всяка решетка има уникален минимален елемент и уникален максимален елемент. Ето как изглежда диаграмата на Hasse на $(\{a, b, c, d\}, \sqsubseteq)$ (без имената на елементите):



Минималният елемент е $\{\{a\}, \{b\}, \{c\}, \{d\}\}$: разбиването на едноелементни множества (*partition into singletons*). Максималният елемент е тривиалното разбиване $\{\{a, b, c, d\}\}$.

Връзката на всичко това с алгоритъма на Kruskal е следната. В някакъв смисъл, алгоритъмът на Kruskal се "придвижва" от минималния елемент нагоре до максималния в решетката на разбиванията, като на всяка итерация текущото разбиване абстрахира предишното. При това без да се прескачат нивá в решетката, защото се сливат два дяла в един, което означава, че броят на елементите в разбиването намалява с единица.

11.5 Union-Find структури данни

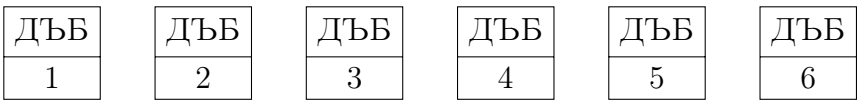
11.5.1 Два крайни подхода, които не вършат работа

Да разгледаме две екстремни възможности за реализация именуването на дяловете на разбиването. Понеже говорим за дървета, нека има дялове ДЪБ и БУК.

- Всеки елемент (връх) е запис, който има поле за името на дяла на разбиването, в който се намира.



При този начин на съхранение на имената, проверката $\boxed{\text{component}(u) \neq \text{component}(v)}$ става във време $\Theta(1)$. Но операцията $\boxed{\text{identify}(\text{component}(u), \text{component}(v))}$ е бавна. Да даваме напълно ново (което досега не е използвано) на дяла-резултат от сливането, примерно OPEX, не е добра идея. Да кажем, че едното от двете използвани имена ДЪБ и БУК става името на дяла-резултат от сливането, а другото име изчезва. Да кажем, че остава името ДЪБ. Тогава след сливането имаме:



Очевидно при този начин на съхранение на имената, сливането на дялове става в линейно, а не в логаритмично време.

- Другата екстремна възможност е името да се пази на едно място, а от всеки елемент да има указател към него.

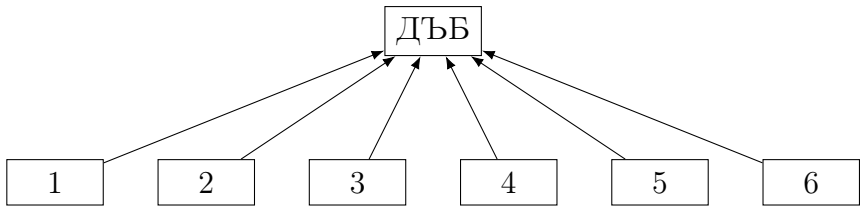


На пръв поглед, това съхранение на имената ни дава възможност да реализираме и Union, и Find в $\Theta(1)$, защото Find(u), където u е елемент, се състои един единствен *dereference* на указателя на u, а Union(i, j) можем да реализираме, като просто сменим името j с i (в примера, БУК става ДЪБ):



Тази идея, разбира се, не работи. Показаната диаграма е "снимка" на моментно състояние на някакви компоненти, което е недостижимо: поначало компонентите имат по един елемент (защото започваме с разбиването на едноелементни дялове), така че, ако следваме тази идея, всеки елемент ще носи името на дяла си (само че на един указател разстояние) до края и Union пак ще работи в линейно време.

Ако опитаме да премахваме едното хранилище на име, така щото всеки елемент от даден дял да сочи към само едно хранилище на име, пак ще получим Union, работещ в линейно време:



Печелившата идея е да се откажем от крайностите и да възприемем междинен подход: нито "всеки елемент има свое копие на името на компонентата си", нито "всички хранилища на имена на компоненти са на един указател разстояние от всеки елемент, през цялото време".

11.5.2 Реализация на разбиване чрез ориентирана гора

Ориентирана гора е множество от ориентирани дървета без общи върхове. В случая става дума за ориентирани дървета с ориентация от листата към върха (на английски, *in-trees*), точно обратно на арборесценциите от Определение 55. Върховете са еднотипни: всеки връх е запис от идентификатора на елемента в множеството и указател към родителя му. Указателите на корените сочат към себе си. Подобни дървета са дърветата на обхождането в BFS/DFS, реализирани с масив на предшествие π. Козметична разлика е, че при онези дървета указателят на корена има стойност Nil, докато тук указателят на корена е "примка" към себе си. В тази подсекция, когато кажем "дърво", разбираме ориентирано дърво с ориентация от листата към корена и указател на корена, който е примка към себе си.

Забележете това: "върховете са еднотипни". При компонентите ДЪБ и БУК от миналата подсекция, имената на компонентите бяха съвсем различни от имената на елементите в тях. Математическият подход към описанието на дадено разбиване е точно такъв – ако е дадено множество $Z = \{z_1, \ldots, z_n\}$ и разглеждаме някакво разбиване на S, типично именуване на дяловете на разбиването е, да кажем, $W_1, \ldots, W_k$. В практическата реализация на разбиване обаче е лоша идея да се ползват отделни имена за дяловете. В практическите реализации, всеки дял на разбиването се идентифицира с точно един от елементите в него. За този елемент казваме, че е *лидер* на дяла на разбиванено (в който се намира). В примера със Z от този параграф, всеки дял на разбиването би бил именуван, или идентифициран, с точно едно z_i от него.

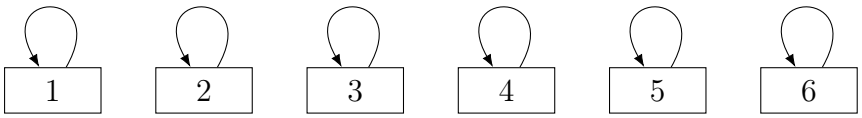
Тези лидери са важни. Функцията Find(u) връща лидера на компонентата, в която се намира u, а функцията Union(i, j) работи само върху лидери на компоненти.

Ето псевдокодът на Find. $\pi[u]$ е родителят на u.

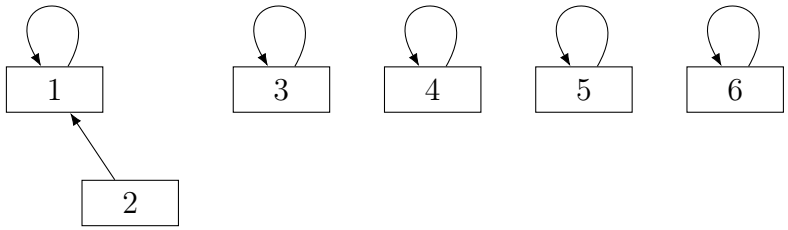
```
Find(u)
1  if u ≠ π[u]
2      return Find(π[u])
3  else
4      return u
```

При реализация на разбиване с ориентирана гора, всеки дял се представя чрез точно едно дърво от гората, а лидерът на дяла е коренът – това е естественият избор. Началното разбиване на едноелементни множества, което се има предвид на ред 2 от МПД Kruskal[†] се реализира от дървета с по един връх-корен ето така:

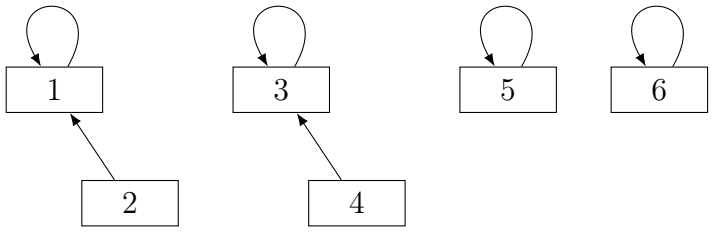
[†]С други думи, минималният елемент на решетката от Допълнение 43.



В този момент има шест компоненти, всяка с по един елемент, който е и лидерът. Find върху който и да е елемент ще върне самия него. Да кажем, че се извърши Union$(1, 2)$. След това вече има само пет дяла, като единият е от два елемента, а именно 1 и 2. Да кажем, че лидерът на двуелементния дял е 1. Тъй като имплементираме с ориентирани дървета, трябва указателят на 2 вече да сочи към 1:

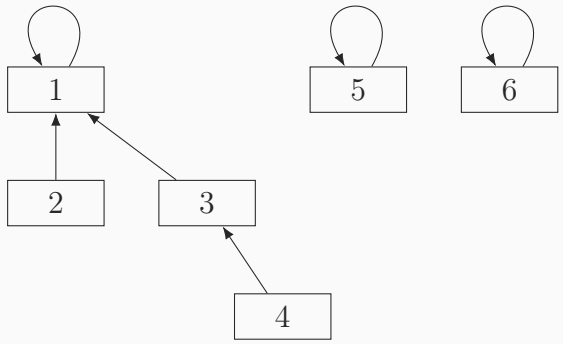


Да кажем, че следващото сливане е Union$(3, 4)$. Резултатът изглежда така:

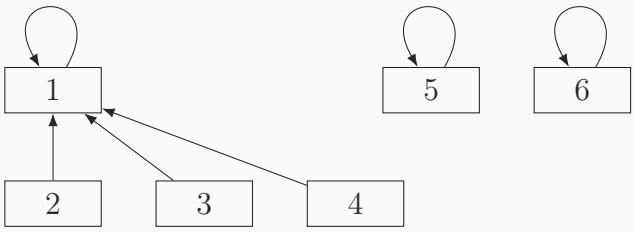


Нека следващото сливане е Union$(1, 3)$. Викането е коректно, защото и 1, и 3 са лидери (викане Union$(1, 4)$ би било некоректно, защото 4 не е лидер). Както се вижда на Фигура 11.3, има два начина да направим дървото на новата компонента.

Фигура 11.3 : Два варианта за Union.



Вариант 1.



Вариант 2.

Предимството на това, дървото с върхове 1, 2, 3 и 4 да е във Вариант 2 е, че височината му е само едно, за разлика от дървото с тези върхове във Вариант 1, което има височина две. Ако има бъдещо викане на Find(4), във Вариант 1 ще се наложи да се направят два "скока нагоре", за да се стигне до лидера, докато във Вариант 2, Find($\mathfrak{u}$) за кой да е връх $\mathfrak{u} \in \{1, 2, 3, 4\}$ иска най-много един "скок нагоре". Предимството на ниските дървета е ясно: бърза работа на Find в най-лошия случай.

Обаче ние предпочитаме Вариант 1. Фигура 11.3 показва малки дървета, но показва ясно, че ако сливаме две произволни дървета, всяко с височина едно, и резултатът трябва да е с височина едно, в най-лошия случай Union ще работи в линейно време в броя на върховете на тези две дървета, който, на свой ред, може да е $\Theta(\mathfrak{n})$. А ние искаме Union да работи в логаритмично време. От тези съображения, Вариант 2 отпада. Вариант 1 е изключително бърз: ако лидерите са известни, сливането на дърветата (компонентите) става във време $\Theta(1)$.

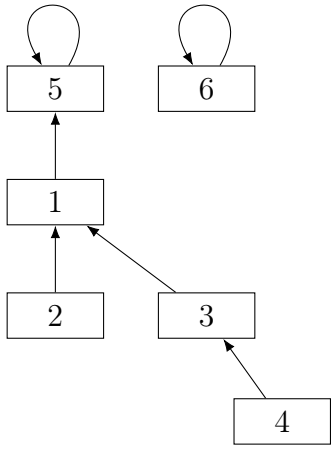
Ние обаче искаме и Union, и Find да работят в не по-лошо от логаритмично време в броя на върховете в двете компоненти, при Union, и в единствената компонента, при Find. Видяхме дори как да реализираме Union в константно време. Ако извършваме обаче Union-ите безразборно, може да получим дървета ненужно голяма височина, поради което Find да стане ненужно бавна.

Наблюдение 47

При реализацията с ориентирана гора, сложността по време на Find($\mathfrak{u}$) в най-лошия случай е $\Theta(\mathfrak{h})$, където $\mathfrak{h}$ е височината на дървото, чийто връх е $\mathfrak{u}$.

Като пример, да разгледаме пак дърветата на Фигура 11.3, Вариант 1. Нека извършим

Union(1, 5). Ако направим връх 5 лидера на тази компонента, дървото би било с височина три:

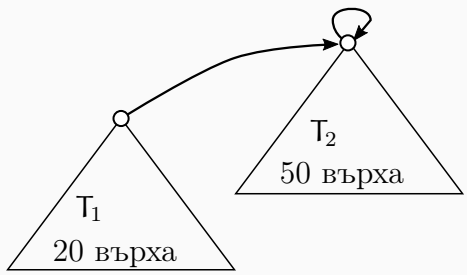


Ясно е, че изборът на 5 за нов лидер беше неудачен. Нещо повече. Серия от такива неудачни избори може да доведе до дървета с линейна височина, откъдето в най-лошия случай Find ще работи в линейно време. За да избегнем това, прилагаме две *евристики*[†].

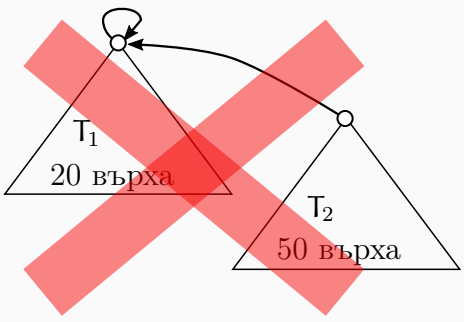
Union by rank. Винаги, когато правим Union, "наказваме" върховете на едно от дърветата с увеличаване на дълбочината им с единица. Ако извършваме Union с просто пренасочване на указателя на единия корен към другия корен, това е неизбежно. Но кое от двете дървета да накажем?

Логично е да накажем дървото с по-малко върхове. В това се състои тази евристика: всяка от компонентите съдържа и информация за броя на елементите в себе си, която е *ранг*ът на дървото. При Union се променя указателя на корена на дървото с по-малък ранг (ако са с еднакви рангове, решението се взема произволно).

Фигура 11.4 : Union by rank: правилен и неправилен..



правилен: 20 върха са наказани.



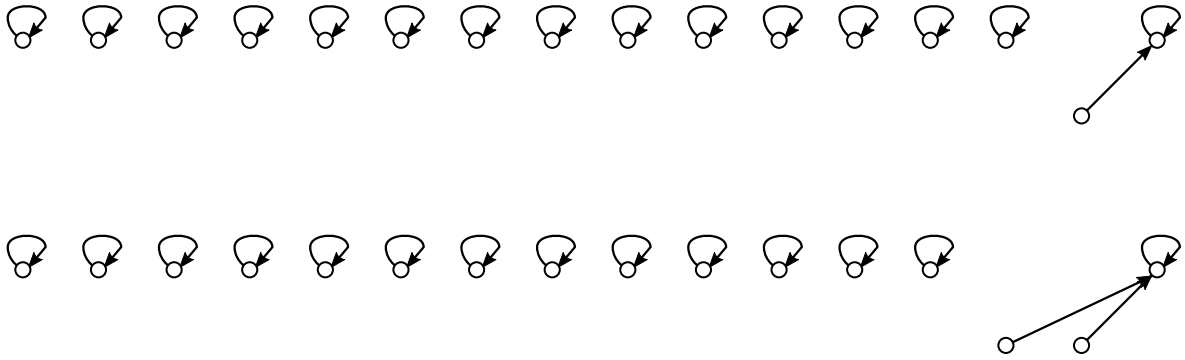
неправилен: 50 върха са наказани.

Да разгледаме примери за създаване на дървета (разбивания) чрез Union by rank. Започваме с шестнадесет върха.

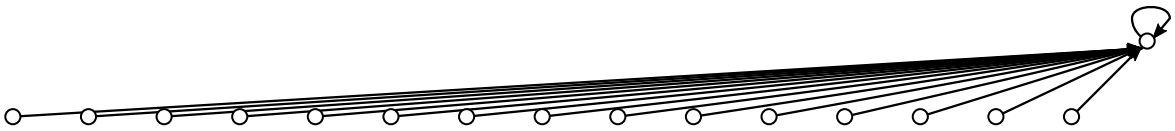
[†]На английски терминът е *heuristic*. Свободно казано, означава добра идея за решаване на някаква задача. Евристиките не гарантират оптималност на решението и дори не дават гаранция за това, колко далече от оптималното е тяхното решение.



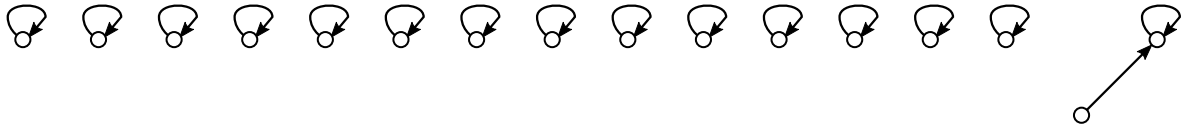
Ако сме късметлии, може сливанията да стават така:



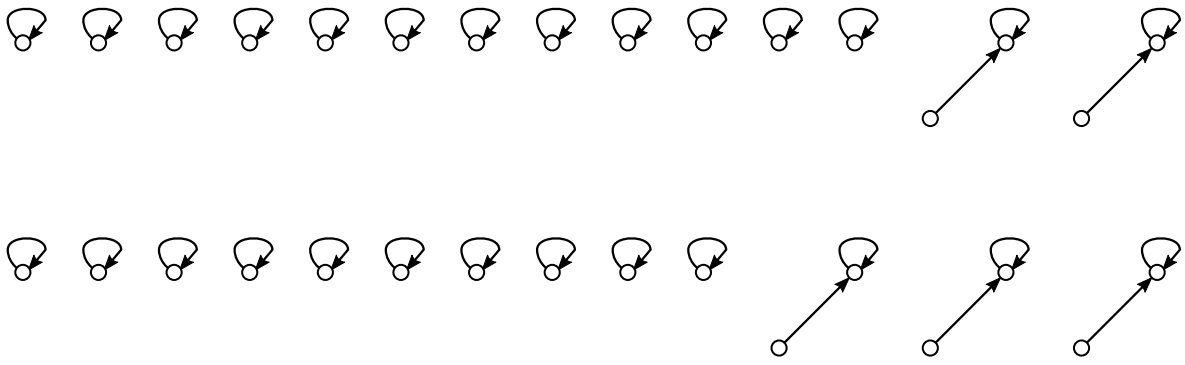
и така нататък, докато получим дърво с височина едно:



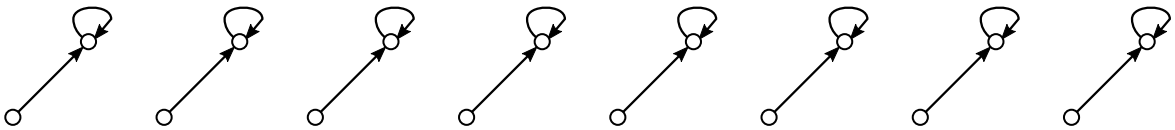
Ако не сме късметлии, може сливанията да стават по следния начин. Първото сливане отново е:



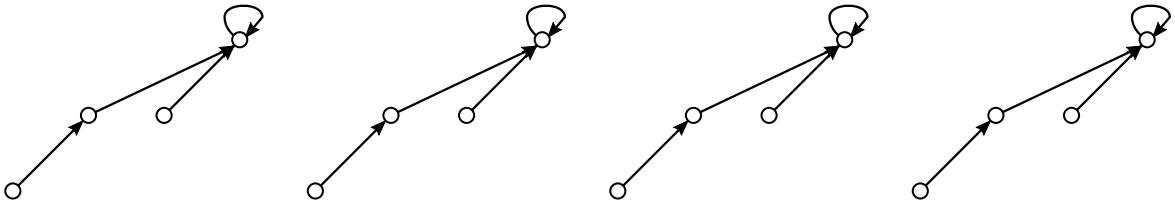
После обаче продължават така:



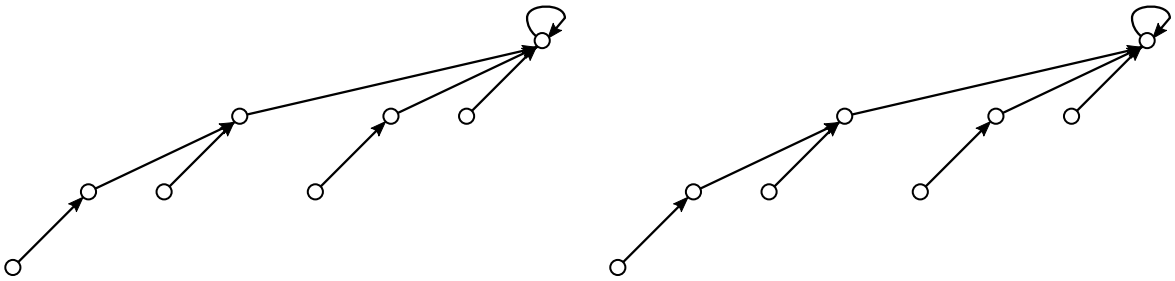
и така нататък до:



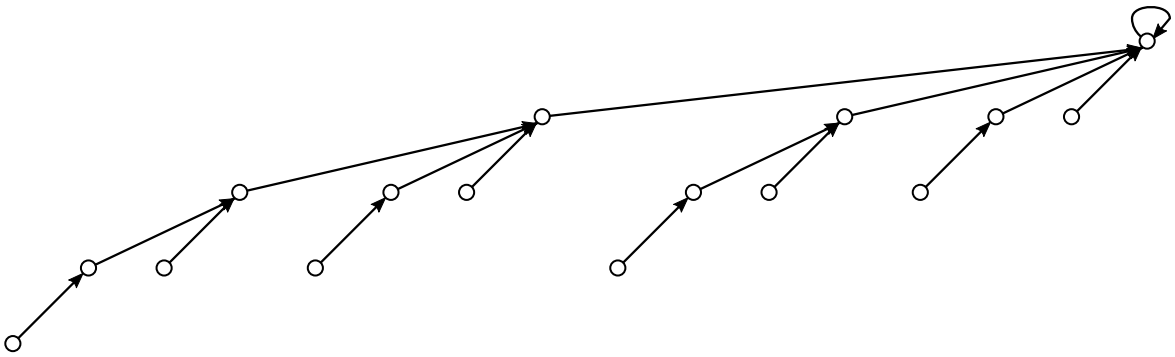
После имаме:



След това:



И накрая:



От тези илюстрации е очевидно, че чрез Union by rank може да получаваме дървета с височина и разклоненост $\lg n$. Но дали това е най-лошата (най-голямата) възможна височина? Оказва се, че да. Сега ще докажем това формално. Само че за лекота на доказателството ще вземем като ранг на дървото не броят на върховете, а височината му.

Теорема 65: Максимална височина на дърво след Union by rank

Започвайки от дървета с по един връх (разбиване на едноелементни множества), след произволна серия от изпълнения на Union by rank, за всяко дърво в колекцията от дървета е вярно, че $h = O(\lg n)$, където h е височината на дървото, а n е броят на **неговите** върхове.

Трябва да се подчертае отново, че n не е общият брой на върховете във всички дървета, а само в дървото, което разглеждаме.

Доказателство: Твърди се, че съществува константа $c > 0$, такава че $h \leqslant c \lg n$. Ще го докажем с индукция по h.

База $h = 0$. Дървото може да се състои от само един връх, ако височината му е нула, така че $n = 1$. Наистина, за всяка положителна константа е вярно, че $0 \leqslant c \cdot \lg 1$. ✓

Индукционно предположение Допускаме, че съществува константа $c > 0$, така че за някое $h \geqslant 0$, всяко дърво, генерирано от Union by rank, $h \leqslant c \lg n$.

Индукционна стъпка Разглеждаме произволно дърво с височина $h + 1$, генерирано от Union by rank, получено от две дървета с височина h. Защо две дървета с височина h? – защото искаме да видим какво става, когато минаваме към височина $h + 1$, сливайки дървета с по-малки височини, но тези по-малки височини може да са само h и h, ако искаме да получим $h + 1$. Нека тези дървета са T_1 и T_2, съответно с n_1 и n_2 върха. Предположението е, че

$$h \leqslant c \lg n_1$$
$$h \leqslant c \lg n_2$$

БОО, нека $n_1 \geqslant n_2$. Тогава

$$c \lg (n_1 + n_2) \geqslant c \lg (n_2 + n_2) = c \lg 2n_2 = c \lg n_2 + c \geqslant \quad // \text{ съгласно инд. предположение}$$
$$h + c \geqslant h + 1$$

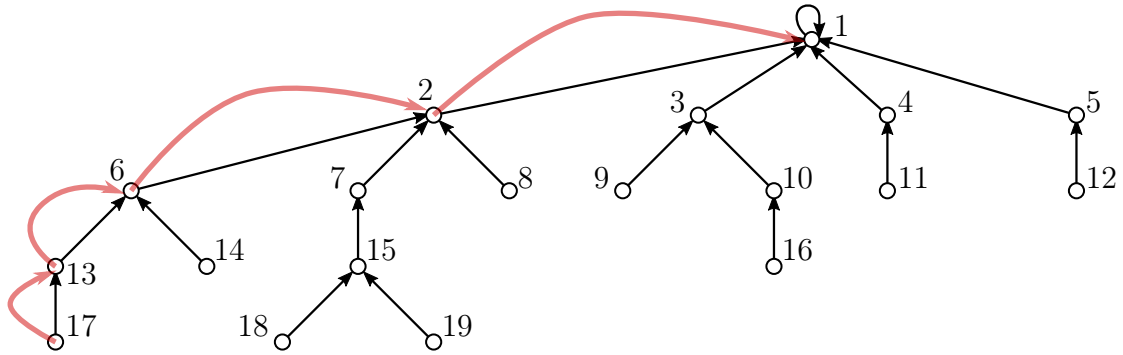
Има такава константа, да кажем $c = 1$. □

Показахме, че Union by rank генерира дърветата с най-много логаритмична в броя на върховете им височина. Това означава, че Find върху такива дървета работи в не по-лошо от логаритмично време. На свой ред, това означава, че в МПД Kruskal проверката дали $\boxed{\text{component}(u) \neq \text{component}(v)}$ (ред 7) и операцията $\boxed{\text{identify}(\text{component}(u), \text{component}(v))}$ (ред 9) се изпълняват във време $O(\lg n)$. Оттук заключаваме, че МПД Kruskal има сложност по време $\Theta(m \lg n)$, като само сортирането ред 3 дава долна граница $\Omega(m \lg n)$.

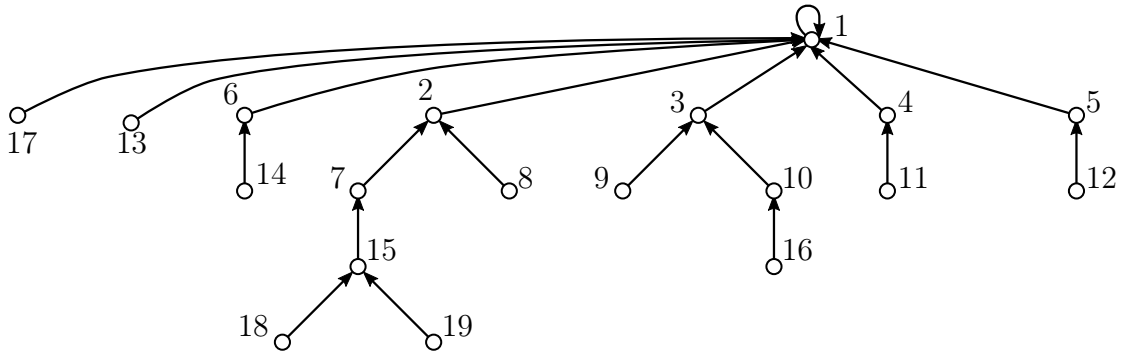
Path compression. Да си припомним текущия псевдокод на Find.

```
Find(u)
1  if u ≠ π[u]
2      return Find(π[u])
3  else
4      return u
```

Още едно подобрение (евристика), която можем да направим към Union-Find структурите, за да ги направим още по-ефикасни, е при всяко изпълнение на Find, при всяко "катерене нагоре" в дървото за достигане до корена, върховете, през които минаваме, да ги наречем $u_1, \ldots, u_k$, да бъдат направени деца на корена. Това няма да натовари значително Find, защото той така или иначе трябва да измине пътя до корена – защо да не направи дървото "по-сплескано", така че при следващи изпълнения на функцията Find върху някой от $u_1, \ldots, u_k$, тя да прави точно един "скок нагоре", за да стигне до корена? Ето пример на дърво в Union-Find структура плюс илюстрация в червено на "скоковете" на Find(17).



Правим всеки от върховете 17, 13 и 6 дете на корена 1.



Ето и псевдокод на подобрения Find.

```
Find with path compression(u)
1  if u ≠ π[u]
2      π[u] ← Find(π[u])
3  return π[u]
```

Анализът на сложността на Union и Find операциите при използване на union by rank и path compression евристиките е сложен и нетривиален. В [21, стр. 573] е показано, без стриктно формално доказателство, че при m операции Union или Find върху n елементно опорно множество, в каквато и да е последователност, сумарната сложност по време е $O(m\alpha(n))$, където

$\alpha(n)$ е **изключително бавно растяща функция**, наречена *обратната функция на функцията на Ackermann*. Теоретично говорейки, $\alpha(n)$ е неограничено нарастваща функция на n, но от практическа гледна точка, $\alpha(n) \leqslant 4$ за всяка стойност на n, която може да възникне на практика. Така че с пълно основание можем да мислим, че $\alpha(n)$ е константа на практика и че сложността на m операции е $O(m)$, което означава сложност $O(1)$ в амортизирания смисъл.

Това подобрение на сложността на Union и Find не води до подобрение на асимптотичната оценка за сложността на алгоритъма на Kruskal. Тя си остава $\Theta(m \lg n)$ заради сортирането. Но Union-Find структурите се ползват не само за реализация на гората в алгоритъма на Kruskal, а и винаги, когато трябва бързо да се установява свързаност между елементи и бързо да се сливат компоненти, така че възможността да правим всяка от тези операции в константно амортизирано време е нещо забележително и полезно.

Лекция 12

Намиране на най-къси пътища. Алгоритми на Dijkstra, Bellman-Ford и Floyd-Warshall.

Резюме: Въвеждаме задачата за намиране на най-къси пътища в ориентирани тегловни графи в няколко варианта. минимално покриващо дърво на тегловен граф. Разглеждаме проблемите, възникващи при наличие на ребра с отрицателни тегла. Разглеждаме два ефикасни алгоритъма за намиране на най-къси пътища от даден връх до всички останали: алгоритмите на Dijkstra и на Bellman-Ford. Разглеждаме ефикасни алгоритми за намиране на къси пътища от всеки до всеки връх: алгоритъм, реализиращ матрично умножение, и алгоритъмът на Floyd-Warshall.

12.1 Фундамент

12.1.1 Базови дефиниции

В тази лекция по подразбиране разглеждаме ориентирани тегловни графи. Ако в даден момент разглеждаме неориентирани графи, това ще се каже изрично. Примки не се допускат, понеже те или нямат отношение към най-късите пътища, ако теглата им са положителни, или представляват отрицателни цикли, ако теглата им са отрицателни – а отрицателните цикли са "проклятието" на задачата за най-къси пътища, както ще видим нататък. Не се допускат и паралелни ребра, тъй като от всеки сноп паралелни ребра, за най-късите пътища значение има само най-леко ребро. Така че и мултиграфи не се допускат.

По подразбиране $\mathsf{G} = (\mathsf{V}, \mathsf{E})$ е ориентиран тегловен граф с тегловна функция $w : \mathsf{E} \to \mathbb{R}$.

Конвенция 10

В тази лекция, казвайки "път", имаме предвид път, който не е непременно прост.

Нотация 5: "$u \overset{p}{\rightsquigarrow} v$" означава "$p$ е ориентиран път от u до v"

Нека $u, v \in V$. Нотацията "$u \rightsquigarrow v$" е кратък запис за "ориентиран път от u до v". Нотацията "$u \overset{p}{\rightsquigarrow} v$" е кратък запис за "$p$ е ориентиран път от u до v".

Определение 75: Тегло на път

Теглото на пътя $\mathtt{p}$ е $w(\mathtt{p}) = \sum_{e \in \mathsf{E}(\mathtt{p})} w(e)$.

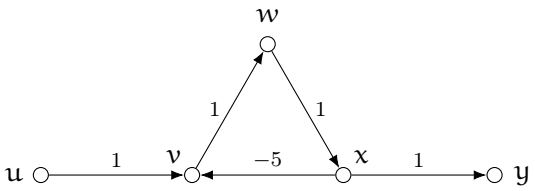
Определение 76: Отрицателен цикъл

Нека G е тегловен граф. Ако G е ориентиран, *отрицателен цикъл* е всеки прост цикъл в G, който има отрицателна сума от теглата на ребрата. Ако G е неориентиран, отрицателен цикъл е всеки прост цикъл в G, който има отрицателна сума от теглата на ребрата, както и всяко ребро с отрицателно тегло.

Наблюдение 48: Проблем при отрицателните цикли.

Ако теглата са само положителни, "най-къс път от $\mathtt{u}$ до v" е добре дефинирано понятие. Обаче за всеки отрицателен цикъл $\mathtt{c}$, за всеки път $\mathtt{p}$, който съдържа $\mathtt{c}$, съществува път $\mathtt{p}'$, такъв че $w(\mathtt{p}') < w(\mathtt{p})$, като $\mathtt{p}'$ съдържа $\mathtt{c}$ повече пъти от $\mathtt{p}$. Ерго, ако поне един път от връх $\mathtt{u}$ до връх v съдържа поне един отрицателен цикъл, не можем да дефинираме "най-къс път от $\mathtt{u}$ до v".
Ако няма отрицателни цикли, този проблем не съществува дори при наличие на отрицателни тегла.

Като пример, да разгледаме следния граф:



Ако разглеждаме само прости пътища, най-късият път (той е и единствен) $\mathtt{u} \rightsquigarrow \mathtt{y}$ е пътят $\mathtt{u}, v, w, \mathtt{x}, \mathtt{y}$ с тегло 4. Ако обаче разглеждаме пътища, които не са непременно прости, най-къс път $\mathtt{u} \rightsquigarrow \mathtt{y}$ няма, защото всяко минаване през цикъла $v, w, \mathtt{x}, v$ добавя $1 + 1 - 5 = -3$ към теглото на пътя:

$\mathtt{u}, v, w, \mathtt{x}, v, w, \mathtt{x}, \mathtt{y}$ има тегло 1
$\mathtt{u}, v, w, \mathtt{x}, v, w, \mathtt{x}, v, w, \mathtt{x}, \mathtt{y}$ има тегло -2
$\mathtt{u}, v, w, \mathtt{x}, v, w, \mathtt{x}, v, w, \mathtt{x}, v, w, \mathtt{x}, \mathtt{y}$ има тегло -5

и така нататък.

Наблюдение 49: Когато няма отрицателни цикли.

При липса на отрицателни цикли, за всеки два върха $\mathtt{u}$ и v, всеки най-къс път от $\mathtt{u}$ до v задължително е прост път.

Определение 77: Теглото на най-къс път

Теглото на най-кȣс пȣт от u *до* v е

$$\delta(u,v) = \begin{cases} \min\{w(p) \mid u \overset{p}{\rightsquigarrow} v\}, & \text{ако има поне един } u \rightsquigarrow v \\ \infty, & \text{ако няма такъв път} \end{cases}$$

Забележете, че ако съществува поне един $u \rightsquigarrow v$, който съдържа отрицателен цикъл, то, съгласно Наблюдение 48, $\min\{w(p) \mid u \overset{p}{\rightsquigarrow} v\}$ става $-\infty$. Причината да дефинираме като ∞ теглото на най-къс път при липса на поне един път е очевидна. И така, $\delta(u,v) \in \mathbb{R} \cup \{-\infty, \infty\}$.

Има леко терминологично несъответствие: щом говорим за "тегло на път", би трябвало да казваме "най-лек път" за път с минимално телго, а не "най-къс път". Но както "тегло на път", така и "най-къс път" (в тегловния смисъл) вече са широко приети както на български, така и на английски, като съответно се казва "weight of path" и "shortest path weight", така че в тези лекции ще се съобразим с това. И така, ще се придържаме към следната езикова конвенция.

Конвенция 11

В контекста на задачата, която разглеждаме, "най-къс път" е синоним на "път с най-малко тегло". Ако имаме предвид броя на ребрата в пътя, ще го кажем експлицитно.

Конвенция 12

Тегловните функции, които ще разглеждаме, или имат кодомейн $\mathbb{R}^+$, или имат кодомейн $\mathbb{R}$, но в G няма отрицателни цикли. Поради това никъде няма да дефинираме пътищата като прости пътища. Те ще се оказват прости вследствие на отсъствието на отрицателни цикли, предвид Наблюдение 49.

Задачата за най-къс път е в няколко варианта. По отношение на началото и края на най-къс път вариантите на задачата са следните.

Изч. Задача: Single Pair Shortest Path

пример: Ориентиран граф $G = (V, E)$, тегловна функция w, начален връх s, краен връх t.
решение: $\delta(s,t)$

Изч. Задача: Single Source Shortest Paths

пример: Ориентиран граф $G = (V, E)$, тегловна функция w, начален връх s.
решение: $\forall v \in V : \delta(s,v)$.

Изч. Задача: Single Destination Shortest Paths

пример: Ориентиран граф $G = (V, E)$, тегловна функция w, краен връх t.
решение: $\forall v \in V : \delta(v,t)$.

Изч. Задача: All Pairs Shortest Paths

пример: Ориентиран граф $G = (V, E)$, тегловна функция w.
решение: $\forall u, v \in V : \delta(u, v)$.

Първият вариант на задачата, а именно от даден връх s до даден връх t, изглежда алгоритмично най-лесен. Както ще се убедим обаче, в най-лошия случай, за да изчислим теглото на най-лек път $s \rightsquigarrow t$, се налага да изчислим и теглото на най-лек път $s \rightsquigarrow v$, за всеки $v \in V$. Ерго, в най-лошия случай, първият вариант е труден колкото втория.

Ние ще разгледаме подробно втория вариант, като за краткост ще го наричаме SSShP.

Третият вариант има алгоритмичната трудност на втория, защото третият вариант се превръща във втория при обръщане на посоките на ребрата; тоест, на транспониране на графа.

Четвъртият вариант ще бъде разгледан подборно. За краткост ще го наричаме APShP. Очевидно APShP се редуцира лесно до SSShP, защото, ако имаме алгоритъм за SSShP, можем да го пуснем от всеки връх и да намерим решение. Но има интересни ефикасни алгоритми за APShP, които решават само APShP в смисъл, че нямат ограничена версия, която да решава SSShP по-ефикасно. Ние ще разгледаме и изследваме такъв алгоритъм в края на лекцията.

Ще разгледаме и друга класификация на задачата за най-къси пътища по това, какво искаме за най-къс път $u \rightsquigarrow v$. За разлика от предната класификация, която е специфична за задачата за най-къси пътища, следната класификация е приложима за всяка оптимизационна задача (Определение 4).

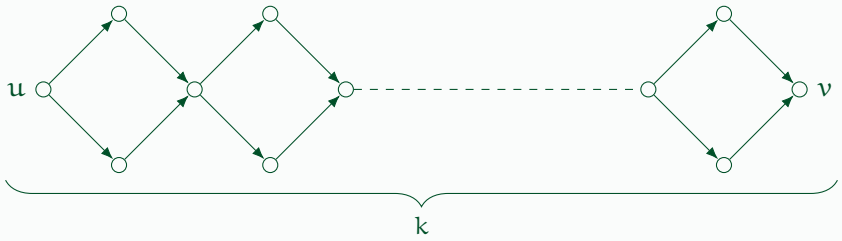
- Може да искаме само дължината, а не самия път. Това не е много полезно на практика.
- Може да искаме дължината плюс самия път, което в реални приложения е много по-полезно и информативно.
- Може да искаме броя на най-късите пътища $u \rightsquigarrow v$.
- Може да искаме множеството от всички най-къси пътища $u \rightsquigarrow v$.

Ние ще разглеждаме алгоритми, които връщат дължината на най-къс път плюс самия път.

Да обобщим: ще решаваме задачата във вариант SSShP, като за всеки връх-край на най-къс път, ще изчисляваме както дължината на пътя, така и самия път.

Допълнение 44: Генерирането на всички най-къси пътища е неефикасно.

В реални приложения може да е много полезно да бъдат получени всички най-къси пътища от връх до друг връх. Имайки всички най-къси пътища, може да изберем от тях път по някакъв друг критерий. За съжаление, в най-лошия случай, броят на най-късите пътища $u \rightsquigarrow v$ може да е експоненциален в n, в което може да се убедим с този прост пример (приемете, че теглата са единици):



Виждаме k на брой подграфи, да ги наречем *ромбовете*, като всеки ромб е  . Ромбовете са "слепени" в редица. Очевидно на фигурата има 2^k пътя от u до v, всеки от тях с дължина $2k$, защото за всеки ромб, може да минем "отгоре" или "отдолу". И тъй като е възможно $k \asymp n$, в графа може да има $\Omega(2^n)$ пътя от u до v. Следователно, няма ефикасен алгоритъм, който строи всички пътища, защото само записването им отнема време $\Omega(2^n)$ в най-лошия случай.

Следният резултат е Lemma 24.1 в [21, стр. 645].

Теорема 66: Най-къс път се състои от най-къси подпътища.

Нека G е ориентиран тегловен граф, s и t са върхове в него и p е най-къс път от s до t. Нека $u, v \in V(p)$, като u предхожда v от s към t в p, ако u и v са различни[a]. Нека подпътят на p от u до v се казва q:

$$p = \underbrace{s \rightsquigarrow u \overset{q}{\rightsquigarrow} v \rightsquigarrow t}_{p}$$

Тогава q е най-къс път от u до v.

[a] Не е необходимо s, t, u и v да са четири различни върха; в най-екстремния вариант, може $s = t = u = v$ и пак остава вярно, че най-къс път се състои от най-къси подпътища.

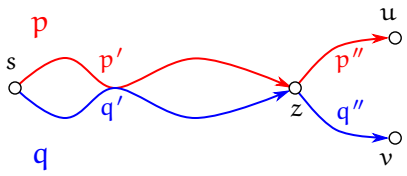
Доказателство: Ако допуснем, че има път q', такъв че $u \overset{q}{\rightsquigarrow} v$ и $w(q') < w(q)$, веднага заключаваме, че p не е най-къс път от s до t, понеже замяната на q с q' в p намалява $w(p)$ с $w(q) - w(q')$. □

В доказателството на Теорема 66 се възползваме от това, че пътищата не са непременно прости. Ако настоявахме пътищата да са прости, то вмъкването на q' на мястото на q може да е проблематично, понеже няма гаранции, че q' и p нямат други общи върхове освен u и v; ерго, след вмъкването целият път може да не е прост. Ако теглата са положителни, можем "да изрежем" общите части на p и q' и да получим път p'', който е с дори още по-малко тегло, което пак ни дава желаното противоречие. Обаче при отрицателни тегла и по-специално при наличието на отрицателни цикли, това "изрязване" може да увеличи теглото на пътя, което е проблем за доказателството.

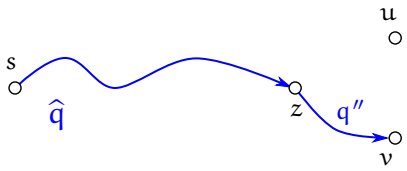
Дърво на най-късите пътища. Ако искаме да изчислим най-къс път $s \rightsquigarrow t$, паметта, която е необходима за записването на пътя е $\Theta(n)$ в най-лошия случай, защото, в най-лошия случай, дължината на пътя е $\Theta(n)$ и не можем да избегнем записването на всеки връх от него. На пръв поглед, ако искаме да запишем по един най-къс път $s \rightsquigarrow v$ за всеки $v \in V$, ще ни трябва $O(n^2)$ памет (може да се покаже, че асимптотичната горна граница $O(n^2)$ е точна, така че имаме право да пишем $\Theta(n^2)$).

Всъщност, можем "да минем" само с $\Theta(n)$ памет за всички пътища, защото можем да ги представим с ориентирано кореново дърво-арборесценция, която може да се представи с масив на предшествията $\pi[1, \ldots, n]$, също както при BFS и DFS. Ще покажем, че най-къси пътища от s могат да се представят с една арборесценция с корен s. Нека p и q са най-къси пътища от s съответно до u и v, където u и v са различни върхове. Ако единственият общ връх на p и q е s, няма какво да се показва. Нека p и q имат поне един общ връх освен s. Нека z е най-отдалеченият в p и в q връх от s, който е общ за p и q. Очевидно не може

$z = u = v$, понеже u и v са различни. Ако $z = u \neq v$ или $z = v \neq u$, няма какво повече да показваме; това е случаите, в който съответно p е част от q или q е част от p. Остава да разгледаме случая, в който $z \neq u$ и $z \neq v$. Нека подпътят на p от s до z е p', а подпътят на q от s до z е q':

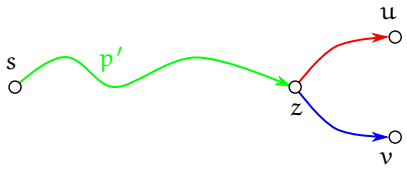


Твърдим, че $w(p') = w(q')$. Да допуснем противното. БОО, нека $w(p') < w(q')$. Тогава в q подменяме q' с p':



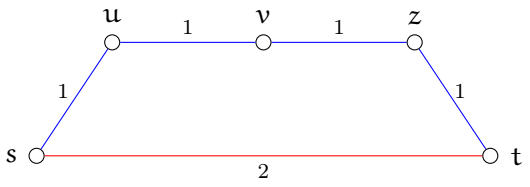
и получаваме път $\hat{q}$ от s до v, такъв че $w(\hat{q}) < w(q)$, в противоречие с допускането, че q е най-къс път от s до v.

Щом $w(p') = w(q')$, може и в p, и в q да използваме само единият от тях, да кажем p'; по този начин, получаваме най-къси пътища от s до u и от s до v, които се състоят от един общ подпът от s до z, след което се "разделят" и повече не се събират:



Ако си направим същото нещо за всеки два върха, достижими от s, ще получим арборесценция с корен s, което е дървото на най-късите пътища от s.

Задачата за най-късите пътища и задачата за МПД са различни. Естествено, те са задачи върху различни видове графи, като МПД е върху неориентирани тегловни графи, а най-късите пътища са върху ориентирани тегловни графи, но съществената разлика не е в това. Дори да решаваме задачата за най-късите пътища върху неориентирани тегловни графи, тя остава принципно различна от задачата за МПД. Ето малък пример:



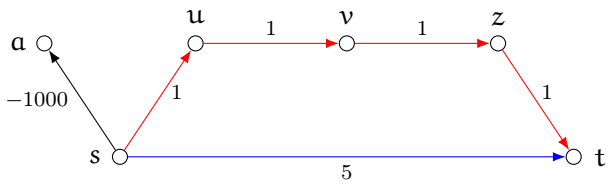
Най-късият път между s и t е реброто (s, t) с тегло 2 (в червено). Но МПД-то (то е само едно) се състои от четирите сини ребра, всяко с тегло 1. Ако си представим работата на алгоритъма на Kruskal върху този граф, сортирайки ребрата, той ще постави в началото четирите ребра с тегло 1 и след това реброто с тегло 2. Когато започне да слага ребра, той ще сложи четирите ребра с тегло 1 и ще спре, без да достигне до реброто с тегло 2, което реализира най-късия път.

Наблюдение 50

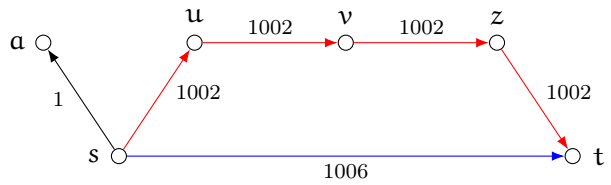
Задачите за намиране на МПД и намиране на дърво на най-късите пътища върху неориентирани тегловни графи са принципно различни. Всяко МПД, за всеки два върха s и t, дава (уникален) път p между s и t, но в общия случай p не е най-къс път в графа между s и t.

Друга важна разлика между тези две задачи е възможността да се "отървем" от отрицателни тегла. Когато дефинирахме задачата за МПД, дефинирахме теглата като реални числа, оставяйки възможността те да са отрицателни. Наистина, нищо не пречи теглата да са отрицателни в Теорема 63 или в алгоритмите на Prim и Kruskal. Ако обаче по някаква причина не искаме отрицателни тегла там, лесно може да се отървем от тях, като намерим най-малкото отрицателно тегло x и после добавим $|x| + 1$ към теглата на всички ребра. Те ще станат положителни, а МПД-тата на графа ще останат **същите**.

Този трик не работи при най-късите пътища. Ако "повдигнем" теглата достатъчно, така че всички те да станат положителни, дървото на най-късите пътища може да се промени. Ето малък пример за това, този път с ориентиран граф:



Очевидно най-късият път от s до t се състои от червените ребра и има тегло 4. Ако обаче добавим 1001 към теглото на всяко ребро, така че теглата да станат само положителни:



пътят от червените ребра става с тегло 4008, а този със синьото ребро, само 1006, така че сега той е най-късият път от s до t.

Наблюдение 51

Нека G тегловен неориентиран граф с тегловна функция $w : E \to \mathbb{R}$. Нека $w' : E \to \mathbb{R}$ е друга тегловна функция, като $\forall e \in E : w'(e) = w(e) + c$, където c е произволна реална константа. МПД-тата на G по отношение на w и по отношение на w' са едни и същи. Ако обаче G тегловен ориентиран граф с тегловна функция $w : E \to \mathbb{R}$ и $w' : E \to \mathbb{R}$ е друга тегловна функция, като $\forall e \in E : w'(e) = w(e) + c$, където c е произволна реална константа, дървото на най-късите пътища по отношение на произволен $s \in V(G)$ може да бъде различно за w и за w'.

12.1.2 Пак за отрицателните тегла

Както се убедихме, отрицателните тегла представляват значителен проблем за задачата за най-късите пътища (за разлика от задачата за МПД, където те нямат никакво значение). При наличие на отрицателни тегла ние може дори да не сме в състояние да дефинираме "най-къс път от s до t", ако поне един път от s до t съдържа отрицателен цикъл.

Това се дължи на факта, че разглеждаме пътища, които не са непременно прости, което влече, че всяко "завъртане" през отрицателен цикъл дава още по-къс път. Не може ли да постулираме, че разглеждаме само прости пътища, и по този начин да няма възможност да се "въртим" в циклите поначало? Отговорът е, че теоретично можем да го направим, но алгоритмично не можем да го имплементираме по ефикасен начин. Алгоритмите за най-къси пътища, примерно този на Dijkstra, не правят проверка дали изградените пътища са прости или не. Те се оказват прости (вижте Наблюдение 49; в алгоритъма на Dijkstra се иска теглата да са положителни) или, ако се окаже, че има повторение на върхове (алгоритъмът на Bellman-Ford "усеща" повторение на върхове) следва съобщение за грешка, тоест, алгоритъмът отказва да търси най-къси пътища след установяване на съществуване на отрицателен цикъл. В Секция 13.11 е показана схема, по която може да се построи алгоритъм за най-дълги пътища, който работи с прости пътища и предотвратява повтаряне на върхове. За съжаление, такъв алгоритъм би бил с експоненциална сложност по време.

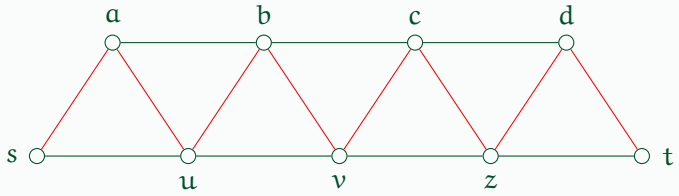
Силна улика за това, че не може да решим проблема с отрицателните цикли с елементарни средства—примерно, постулиране, че не разглеждаме други пътища освен прости—е фактът, че задачата за най-къси пътища е същата като задачата за най-дълги пътища при обръщане на знака на теглата. Иначе казано, ако G е ориентиран тегловен граф, върху който са дефинирани тегловни функции $w, w' : E \to \mathbb{R}$, като $\forall e \in E(G) : w(e) = -w'(e)$, то, за всеки $s, t \in V(G)$, p е най-къс път $s \rightsquigarrow t$ по отношение на w тстк p е най-дълъг път $s \rightsquigarrow t$ по отношжение на w'. Задачата за най-дълги пътища е **NP**-пълна, което означава, че почти сигурно за нея няма ефикасен алгоритъм. Какво означава "**NP**-пълна" и защо задачата за най-дългите пътища е такава, ще видим в Лекция 14, Лекция 15 и Лекция 16.

Допълнение 45: За структурата на най-дългите пътища

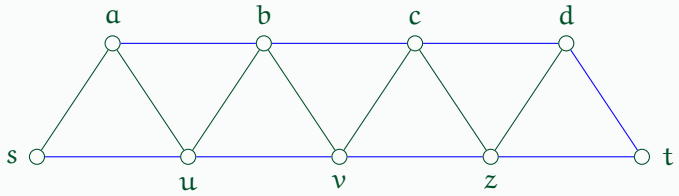
За задачата за най-дългите пътища не е вярно, че оптималната структура се състои от оптимални подструктури (сравнете с Теорема 66). Това обяснява драстичната разлика между ефикасността на най-добрите известни алгоритми за най-къси пътища и за най-дълги пътища.

Преди всичко, за да дефинираме смислено задачата за най-дългите пътища, трябва да постулираме, че разглеждаме само прости пътища. Иначе, дори в не-тегловния вариант, задачата е лошо дефинирана, понеже съществуването на цикъл в ориентиран граф или

на ребро в неориентиран граф влече съществуването на неограничено дълги, като брой ребра, пътища, които не са прости. И така, ако дефинираме задачата за най-дълги пътища по очевидния начин, настоявайки пътищата да са прости, е лесно да намерим пример, в който най-дългият път не се състои от най-дълги подпътища (допускаме, че теглата са единици):



Най-дългият път между s и t е с дължина 8 и е нарисуван с червено. В него има подпът между s и a с дължина 1. Но най-дългият път между s и a е с дължина 8, както се вижда на следната фигура (в синьо):



Очевидно не е вярно, че най-дългите пътища се състоят от най-дълги подпътища.

След всички изложени проблеми, които възникват при отрицателни тегла на ребрата, възниква въпроса, а защо не се откажем от отрицателни тегла поначало? Наистина, ако теглата моделират някакви физически величини, които по природа да положителни, примерно закъснения във времето или разстояния в реалния свят, няма как да се появят отрицателни тегла; отрицателно закъснение, например, означава пътуване назад във времето. Обаче има важни практически задачи, в които е удобно да се работи с абстракцията на отрицателните числа. Примерно, ако моделираме някакви потоци от стоки или пари, можем да кажем, че от сметката на X са прехвърлени 100 лева в сметката на Y, като кажем, че от сметката на Y са прехвърлени -100 лева в сметката на X. Така че, ако се откажем от отрицателни тегла изобщо, графите ни ще станат със строго по-малка моделираща мощ.

Допълнение 46: Намиране на арбитражи чрез отрицателни цикли

Арбитраж, на английски *arbitrage*, е термин от финансите и търговията, който означава, най-общо казано, купуване на нещо, последване от продаването на същото нещо (по всяка вероятност, не на същото място), като, естествено, купуването е на ниска цена, а продаването, на висока цена.

В случая става дума за търговия на валути. В този контекст, арбитраж се дефинира като почти едновременното купуване и продаване на ценни книжа или чуждестранна валута в различни пазари с цел печалба от разминавания в курсовете. Като пример, да разгледаме валутите евро EUR, лев BGN, унгарски форинт HUF, чешка крона CZK и монголски тугрик MNT. Дадени са обменните курсове в квадратна матрица $M[i, j]$,

като $M[i,j]$ за $i \neq j$ има смисъл на количество валута j, която можем да закупим за единица валута i. Да допуснем, че матрицата има свойството $M[i,j] = \frac{1}{M[j,i]}$ при $i \neq j$ (обратното би било арбитраж върху само две валути).
Ето примерна матрица с курсовете на тези валути от 18 май 2020 г. Курсовете са истинските с изключение на двете числа в червено.

	EUR	BGN	HUF	CZK	MNT
EUR	1	1.95583	352.604	27.6033	3 035.51
BGN	0.51129	1	180.241	14.1150	1 551.88
HUF	0.0028360	0.0055481	1	0.063403	8.60872
CZK	0.036227	0.0055481	15.77188	1	110.009
MNT	0.00032943	0.00064437	0.116161	0.0090834	1

Да кажем, че искаме да продадем и купим валути в тази последователност:

$$\text{EUR} \to \text{BGN} \to \text{MNT} \to \text{CZK} \to \text{HUF} \to \text{EUR}$$

Съгласно дадената матрица, за всяко евро, с което започваме, ще получим

$$1.95583 \times 1551.88 \times 0.0090834 \times 15.77188 \times 0.0028360 = 1.233182546$$

евро. С други думи, печалба от над 23%[a].

Задача е, при дадена матрица M, да се определи дали има редица от валути $c_{i_1}, c_{i_2}, \ldots, c_{i_k}$, такава че

$$M[i_1, i_2] \times M[i_2, i_3] \times \cdots \times M[i_{k-1}, i_k] \times M[i_k, i_1] > 1 \tag{12.1}$$

Това е Problem 24.3 в [21, стр. 679]. Тази задача има добре известно решение, което я свежда до задачата за намиране на отрицателен цикъл в граф. Логаритмуваме двете страни в (12.1) и получаваме:

$$\lg(M[i_1, i_2]) + \lg(M[i_2, i_3]) + \cdots + \lg(M[i_{k-1}, i_k]) + \lg(M[i_k, i_1]) > 0 \tag{12.2}$$

което е същото като

$$-\lg(M[i_1, i_2]) - \lg(M[i_2, i_3]) - \cdots - \lg(M[i_{k-1}, i_k]) - \lg(M[i_k, i_1]) < 0 \tag{12.3}$$

Сведохме задачата до задачата за намиране на отрицателен цикъл в пълен ориентиран граф, като върховете са валутите, а теглото на реброто (i,j) е $-\lg(M[i,j])$. Алгоритъмът на Bellman-Ford може да открива наличието на отрицателни цикли и с малка модификация може да намира отрицателен цикъл.

[a] Естествено, в реалния живот не може да се правят толкова лесни печалби. Тук това става заради нагласените числа.

12.1.3 Релаксация

Става дума за задачата за най-късите пътища в SSShP варианта. Даден е някакъв стартов връх s. Според [21, стр. 647], *релаксация* е техника за подобряване на *оценката* (*estimate* на английски), по отношение на произволен връх $u \in V(G)$, на теглото на най-къс път $s \rightsquigarrow u$. Както ще видим надолу, тази оценка е всъщност горна граница, която в алгоритмите, които ще разгледаме, започва с "∞" (тоест, нищо не знаем) и става все пó и по-добра, докато не стане точна (горна граница) и остане така до края на алгоритъма.

Тази оценка за връх u може да бъде записана като $u.d$, а може да има масив $d[1, \ldots, n]$ и оценката за u да е $d[u]$. Във всеки случай, това е реално число, което има смисъл на "най-ниската горна граница за минималното тегло на $s \overset{p}{\rightsquigarrow} u$, която можем да дадем въз основа на наличната информация".

Както вече казахме, ние искаме не просто дължините на най-къси пътища, а и самите пътища, реализирани чрез масив на предшествия. На свой ред, тези стойности (идентификаторът на родителя в дървото) може да се записват като $u.\pi$, а може да има масив $\pi[1, \ldots, n]$, като $\pi[u]$ е родителят на u в дървото (родителят на корена е Nil).

Оценките се инициализират от следната процедура, която ще наричаме кратко "ISS", което идва от Initialize-Single-Source.

```
ISS(G: ориентиран тегловен граф, s: връх в G)
1  foreach ν ∈ V(G)
2      ν.d ← ∞
3      ν.π ← Nil
4  s.d ← 0
```

Самата релаксация става така. Контекстът е, както очакваме, ориентиран тегловен граф G и тегловна функция w.

```
RELAX((x, y) ∈ E(G))
1  if y.d > x.d + w((x, y))
2      y.d ← x.d + w((x, y))
3      y.π ← x
```

Смисълът е напълно ясен: опитваме да намерим път с по-малко тегло $s \rightsquigarrow y$ от този, който е оптимален до момента. Стойността $y.d$ има смисъл на дължината на най-къс път $s \rightsquigarrow y$ според информацията до момента. Опитваме да намерим по-къс път до y, ползвайки x като предпоследен връх. Ако се окаже, че $y.d \leqslant x.d + w((x, y))$, то опитът е неуспешен и просто прескачаме. Ако обаче $y.d > x.d + w((x, y))$, наистина има такъв по-къс път—при допускането, че $x.d$ е смислена и π-стойностите реализират дърво на пътища—така че подменяме $y.d$ и $y.\pi$ по адекватен начин.

Изборът на името "релаксация" може ли изглежда странен. Чрез тази примитивна процедура ние всъщност **затягаме** горната граница (оценката) за дължината на най-къс път, а не я отпускаме. Според [21, стр. 647], произходът на името "релаксация" е следният. Резултатът от RELAX$((x, y))$ може да бъде разглеждан като релаксиране на ограничението (constraint) $y.d \leqslant x.d + w((x, y))$; съгласно неравенството на триъгълника, това ограничение трябва да е в сила, ако $y.d = \delta(s, y)$ и $x.d = \delta(s, x)$. С други думи, ако е вярно, че $y.d \leqslant x.d + w((x, y))$, то няма защо да се притесняваме за удовлетворяването на това ограничение, така че можем да го релаксираме.

Свойства на релаксацията В сила са следните твърдения. Всички те са от [21, стр. 650]

Лема 60: Неравенство на триъгълника

За всяко ребро $(x, y) \in E(G)$ е вярно, че $\delta(s, y) \leqslant \delta(s, x) + w((x, y))$.

Доказателство: Ако няма път $s \rightsquigarrow x$, то $\delta(s, x) = \infty$ и дясната страна е ∞. Неравенството остава в сила независимо от това, дали няма път $s \rightsquigarrow y$, което влече, че лявата страна е ∞, или има път $s \rightsquigarrow y$, което влече, че лявата страна е число.

Ако има път $s \rightsquigarrow x$, то има път $s \rightsquigarrow y$, който съдържа x като предпоследен връх и има тегло $\delta(s, x) + w((x, y))$. □

Лема 61: Горна граница и точна горна граница

Нека G е инициализиран с $\text{ISS}(G, s)$. Тогава, за всеки връх $v \in V(G)$, $v.d \geqslant \delta(s, v)$, след произволна поредица от релаксации на ребрата на G. Нещо повече, при достигане на равенство, то не се променя след произволна поредица от релаксации на ребрата на G.

Сравнете Лема 61 с Лема 52.

Доказателство: Ще докажем, че $v.d \geqslant \delta(s, v)$ за всеки връх v, след произволна поредица от релаксации на ребра, по индукция по броя на релаксациите.

Базата е за нула релаксации, тоест, d-стойностите са както са генерирани от ISS, а именно $s.d = 0$ и $v.d = \infty$, ако $v \neq s$. Вярно е, че $s.d \geqslant \delta(s, s)$, защото $s.d = 0$ заради ISS, а $\delta(s, s) = 0$ или $\delta(s, s) = -\infty$ в зависимост от това, дали s не е, или е, върху отрицателен цикъл. И в двата случая е вярно, че $s.d \geqslant \delta(s, s)$. Нека $v \neq s$. Вярно е, че $v.d \geqslant \delta(s, v)$, защото $v.d = \infty$ заради ISS, а $\delta(s, v) \in \mathbb{R} \cup \{-\infty, \infty\}$.

Да допуснем, че твърдението е вярно след някаква произволна серия от релаксации. Разглеждаме $\text{RELAX}((u, v))$, за някое $(u, v) \in E(G)$, веднага след тази серия. Единствената d-стойност, която може да се промени от $\text{RELAX}((u, v))$, е $v.d$. Ако тя изобщо се промени, промяната ще е такава:

$$\begin{aligned} v.d &= u.d + w((u, v)) && // \text{ защото RELAX работи така} \\ &\geqslant \delta(s, u) + w((u, v)) && // \text{ от индуктивното предположение за } u \\ &\geqslant \delta(s, v) && // \text{ от Лема 60} \end{aligned}$$

За да се убедим, че, ако веднъж се постигне равенство $v.d = \delta(s, v)$, то остава в сила във всеки момент след това, да съобразим, че

- няма как да се получи $v.d < \delta(s, v)$ заради вече доказания факт, че $v.d \geqslant \delta(s, v)$ винаги, и
- няма как да се получи $v.d > \delta(s, v)$ след $v.d = \delta(s, v)$, защото RELAX никога не увеличава d-стойността на края на релаксираното ребро. □

Лема 62: При липса на пътища

Нека v е връх, такъв че няма $s \rightsquigarrow v$. Тогава, след произволна поредица от RELAX върху ребрата на графа е вярно, че $v.d = \infty$.

Доказателство: Съгласно Лема 61, $v.d \geqslant \delta(s,v)$, след произволна поредица от релаксации на ребрата. Но $\delta(s,v) = \infty$ съгласно Определение 77. □

Лема 63: Конвергенция

Нека p е най-къс път $s \rightsquigarrow v$ и $u \neq v$ е предпоследният връх в p. Ако $u.d = \delta(s,u)$ в някакъв момент преди релаксирането на реброто (u,v), то $v.d = \delta(s,v)$ във всеки момент след това релаксиране.

Доказателство: Съгласно Лема 61, ако веднъж се постигне равенство $u.d = \delta(s,u)$, то се запазва нататък. В частност, след релаксирането на (u,v), в сила е

$$\begin{aligned} v.d &\leqslant u.d + w((u,v)) \quad // \text{ очевидно от кода на RELAX} \\ &= \delta(s,u) + w((u,v)) \quad // \text{ както вече казахме} \\ &= \delta(s,v) \quad // \text{ Теорема 66} \end{aligned}$$

Щом $v.d \leqslant u.d + w((u,v))$ и $v.d \geqslant u.d + w((u,v))$ (Лема 61), то $v.d = u.d + w((u,v))$ и това равенство остава в сила до края (пак Лема 61). □

12.2 Алгоритъмът на Dijkstra в два варианта

Ще разгледаме един от най-популярните алгоритми за намиране на най-къс път в ориентиран граф: алгоритъмът на Dijkstra. Нека тегловната функция е $w : E \to \mathbb{R}^+$. Алгоритъмът на Dijkstra не работи коректно, ако отрицателни тегла, дори да няма отрицателни цикли – намерете сами малък пример за тегловен граф с точно едно отрицателно тегло, без отрицателни цикли, и стартов връх, върху който алгоритъмът на Dijkstra "бърка". Ще разгледаме два варианта на алгоритъма на Dijkstra, базов и изтънчен[†].

SSSHP DIJKSTRA1(G: ориентиран тегловен граф, w: тегловна функция върху G, s: стартов връх)
```
1  ISS(G, s)
2  S ← ∅
3  while ∃u ∈ V\S : u.d < ∞ do
4      избери x ∈ V\S, такъв че x.d е минимална
5      S ← S ∪ {x}
6      foreach y ∈ adj[x]
7          RELAX((x, y))
```

SSSHP DIJKSTRA2(G: ориентиран тегловен граф, w: тегловна функция върху G, s: стартов връх)
```
1  ISS(G, s)
2  S ← ∅
3  създай празна приоритетна min опашка Q
4  Q ← V
```

[†]Забележете аналогията с алгоритъма на Prim, който също беше имплементиран в два варианта, със същата разлика между тях.

```
5  while not isempty(Q) do
6      x ← Extract-Min(Q)
7      S ← S ∪ {x}
8      foreach y ∈ adj[x]
9          Relax((x, y))
```

Коректност и сложност. Въпреки повърхностната прилика между алгоритъма на Prim и алгоритъма на Dijkstra, вторият има значително по-трудно доказателство за коректност, защото в неговия контекст няма аналог на Теорема 63. Ще докажем коректността само по отношение на d-стойностите. Приемаме, че от това доказателство лесно следва и коректността на π-стойностите, които реализират дървото.

Допускаме БОО, че целият G е достижим от s. "SSSHP Dijkstra" означава кой да е от двата варианта на алгоритъма.

Теорема 67: Коректност на алгоритъма на Dijkstra

За всеки ориентиран граф G, за всяка тегловна функция w над него, за всеки избор на стартов връх s е вярно, че след терминирането на SSSHP Dijkstra(G, w, s):

$$v.d = \delta(s, v),\ \forall v \in V(G)$$

Доказателство: Приемаме за очевидно, че всеки връх на графа "влиза" в S на някоя итерация на **while**-а. Ще докажем, че за всеки връх $v \in V(G)$, в момента, в който v влиза в S е вярно, че $v.d = \delta(s, v)$. Това влече верността на теоремата заради Лема 61.

Да допуснем противното: за поне един $v \in V(G)$ е вярно, че $v.d \neq \delta(s, v)$ в момента, в който v влиза в S. Нещо повече: нека v е **първият** връх, който "влиза" в S с d-стойност, различна от минималното тегло на път от s до него. Очевидно $v \neq s$, защото $s.d = 0$ и, при положителни тегла, $\delta(s, s) = 0$.

Разглеждаме тази итерация, на която в S "влиза" връх v. В началото на въпросната итерация, нека t е моментът точно преди v да "влезе" в S. Щом $v \neq s$, то $S \neq \varnothing$ при тази итерация. По допускане, съществува $s \rightsquigarrow v$. Разглеждаме най-къс $s \overset{p}{\rightsquigarrow} v$. Щом теглата са положителни, такъв има.

В момента t е вярно, че първият връх на p (това е s) е в S, а последният връх (това е v) не е в S. Дефинираме y като най-близкият до s връх в p, който в момента t не е в S. Възможно е $y = v$, възможно е и $y \neq v$. Важното е, че y е добре дефиниран. Нека върхът преди y в p се казва x (възможно е $x = s$). Забележете, че всички върхове в p от s до x включително в момента t са в S, и y е първият връх в p, в посока s към v, който не е в S:

$$p = \underbrace{s \cdots\cdots x}_{\text{в } S}\ \underbrace{y}_{\text{не е в } S} \cdots\cdots \underbrace{v}_{\text{не е в } S}$$

Ще докажем, че $y.d = \delta(s, y)$ в t. Знаем, че $x \in S$ в t и че v е първият връх, влизащ в S, за който d-стойността не е равна на претегленото разстояние от s. Връх x е бил сложен в S на някоя предишна итерация, в някакъв момент $t' < t$. Следва, че в момента t', $x.d = \delta(s, x)$, и съгласно Лема 61, във всеки момент след t' продължава да е вярно, че $x.d = \delta(s, x)$. Но на итерацията, в която x е сложен в S, реброто (x, y) е било релаксирано – вижте псевдокода. Прилагаме Лема 63 и заключаваме, че

$$y.d = \delta(s, y) \tag{12.4}$$

след това релаксиране, включително и в момента t.

Съгласно Теорема 66, $\delta(s,y) \leqslant \delta(s,v)$, тъй като теглата са положителни, а s, y и v са наредени в този ред върху най-къс път от s до v. Съгласно Лема 61, $\delta(s,v) \leqslant v.d$ по всяко време от работата на алгоритъма. Тогава

$$\delta(s,y) \leqslant \delta(s,v) \leqslant v.d \tag{12.5}$$

От (12.4) и (12.5) заключаваме, че в момента t:

$$y.d = \delta(s,y) \leqslant \delta(s,v) \leqslant v.d \tag{12.6}$$

Ключовото наблюдение е, че и y, и v са върхове извън S в момента t и причината алгоритъмът да избере v, а не y, като връх, който да влезе в S, е една единствена:

$$v.d \leqslant y.d \tag{12.7}$$

От (12.6) и (12.7) заключаваме, че

$$v.d = y.d \tag{12.8}$$

Но тогава (12.6) всъщност е верига от равенства:

$$y.d = \delta(s,y) = \delta(s,v) = v.d \tag{12.9}$$

и в частност $\delta(s,v) = v.d$. Тогава, съгласно Лема 61, това остава в сила до края на алгоритъма, в противоречие с допускането, че $\delta(s,v) \neq v.d$ в края. ↯ □

Ако не е вярно, че целият G е достижим от s, Лема 62 казва, че върховете, които не са достижими от s, са с d-стойности ∞ в края на алгоритъма. Имайки предвид Теорема 67, заключаваме, че върховете, които не са достижими от s са точно тези, които са с d-стойности ∞ в края на алгоритъма.

Сложността по време е същата, в асимптотичния смисъл, като на съответните имплементации на Prim. Базовата имплементация има сложност $\Theta(n^2)$ в най-лошия случай, а изтънчената има сложност $\Theta(m \lg n)$, ако Q е имплементирана с двоична пирамида.

12.3 Най-къси пътища в дагове

Ако G е даг, може да използваме по-ефикасен алгоритъм от алгоритъма на Dijkstra за задачата SSShP.

```
DAG SSSHP(G: даг, w: тегловна функция върху G, s: стартов връх)
1  TOPOSORT(G)
2  ISS(G, s)
3  foreach x ∈ V(G) в топологичната наредба
4      foreach y ∈ adj[x]
5          RELAX((x, y))
```

Ще покажем коректността на този алгоритъм: след терминирането, за всеки връх $v \in V(G)$ е вярно, че $v.d = \delta(s,v)$.

Инвариант

При всяко достигане на ред 3 е вярно, че $x.d = \delta(s, x)$.

Твърдението е вярно за върховете вляво от s: за всеки връх u вляво от s е вярно, че $\delta(s, u) = \infty$, понеже няма път $s \rightsquigarrow u$, а, от друга страна, във всеки момент от работата на алгоритъма е вярно, че $u.d = \infty$ заради Лема 62. ✓

Ще докажем инварианта със силна индукция по топологичната наредба за върховете вдясно от и включително s. Базата е $x = s$. Вярно е, че $\delta(s, s) = 0$ при отсъствието на отрицателни цикли (което следва от отсъствието на каквито и да е цикли), а, от друга страна, $s.d$ става 0 на ред 2 и остава така до края на алгоритъма (Лема 61). ✓

Разглеждаме произволен връх v вдясно от s. Ако няма път $s \rightsquigarrow v$, то $\delta(s, v) = \infty$, а, от друга страна, във всеки момент от работата на алгоритъма е вярно, че $v.d = \infty$ заради Лема 62. Да допуснем, че v е достижим от s. Разглеждаме момента t, в който $x = v$. Нека $z_1, \ldots, z_k$ са всички родители на v. В топологичното сортиране, те са вляво от v. Тъй като използваме силна индукция, ние допускаме, че в момента t, за всеки връх u вляво от v е вярно, че $u.d = \delta(s, u)$. В частност, в момента t е вярно, че $z_i.d = \delta(s, z_i)$ за $1 \leqslant i \leqslant k$. Нещо повече: поне един z_i е достижим от s, иначе v не бил бил достижим от s, така че за поне едно i, $z_i.d < \infty$ в момента t. Забележете, че Relax е "обработвала" връх v точно k пъти преди момента t, в смисъл че за всеки z_i, когато x е имала стойност z_i, точно веднъж y е имала стойност v. Забележете, че Relax реализира $v.d = \min_{1 \leqslant i \leqslant k} \{z_i.d + w((z_i, v))\}$. Но $\delta(s, v) = \min_{1 \leqslant i \leqslant k} \{\delta(s, z_i) + w((z_i, v))\}$ (това е в сила дори някои z_i да не са достижими от s). Показахме, че $v.d = \delta(s, v)$ в момента t.

Показахме, че за всеки връх v, $v.d = \delta(s, v)$ в момента, в който $x = v$. Съгласно Лема 61, това равенство остава в сила до края на алгоритъма. □

Да разгледаме сложността по време. Сложността на този алгоритъм е $\Theta(n + m)$, защото и топологическото сортиране, и вложеният **for**-цикъл имат такава сложност по време.

Предимствата на DAG SSSHP пред алгоритъма на Dijkstra върху даг са следните:

- DAG SSSHP е по-бърз, както на практика, така и в асимптотичния смисъл. Сложността $\Theta(n + m)$ е оптимална, в асимптотичния смисъл, и е по-добра от сложностите на Dijkstra, както в базовия вариант, така и в изтънчения вариант, дори приоритетната опашка да е реализирана с пирамида на Fibonacci.

- DAG SSSHP работи без проблеми дори ако има отрицателни тегла. В даговете няма цикли поначало, така че няма опасност от отрицателен цикъл. Както се каза, Dijkstra не е използваем при отрицателни тегла. Bellman-Ford "се справя" с отрицателни тегла, но той е драстично по-бавен от DAG SSSHP.

- DAG SSSHP може да намира и най-дълги пътища с една тривиална модификация: обръщане на посоката на неравенството в Relax. Забележете, че при даговете е вярно, че най-дълъг път се състои от най-дълги подпътища (аналогът на Теорема 66): ако заменим подпът с по-дълъг подпът, ще получим по-дълъг път без опасност от повтаряне на върхове. За да се убедим в това, достатъчно е да разгледаме дага в контекста на топологична сортировка. Dijkstra не може да бъде трансформиран в алгоритъм за намиране на най-дълги пътища с просто обръщане на посоката на неравенството в Relax и на min в max по отношение на избора на нов връх, който да влезе в S (в изтънчената реализация това би било смяна на типа на приоритетната опашка от min в max), дори графът да е даг.

12.4 Алгоритъмът на Bellman-Ford

Този алгоритъм намира най-къси пътища и в графи с отрицателни тегла, стига да няма отрицателни цикли. А ако има поне един отрицателен цикъл, алгоритъмът установява това и връща съответна индикация False. Ако върне True, то отрицателни цикли няма. С малка модификация, алгоритъмът може да връща и някой отрицателен цикъл, ако има отрицателни цикли.

Bellman-Ford(G: ориентиран тегловен граф, w: тегловна функция върху G, s: стартов връх)

```
1  ISS(G, s)
2  for i ← 1 to n − 1
3      foreach (x, y) ∈ E(G)
4          Relax((x, y))
5  foreach (x, y) ∈ E(G)
6      if y.d > x.d + w((x, y))
7          return False
8  return True
```

Коректност и сложност. Самият алгоритъм е на редове 1–4, което го прави изключително компактен. Кодът на редове 5–7 е проверка за наличие на отрицателен цикъл.

Ще направим повърхностно доказателство за коректност. БОО, нека целият G е достижим от s. Първо да допуснем, че отрицателни цикли няма. Тогава дърво на най-късите пътища с корен s е добре дефинирано. Нека T е такова дърво. T има височина най-много $n-1$, което обяснява и "$n-1$" на ред 2. Инвариант на **for**-цикъла на редове 2–4 е, при всяко достигане на ред 2, за всеки връх $v \in V(G)$, който се намира на ниво $i-1$ в T (ниво е множеството от върховете на едно и също разстояние, **като брой ребра**, в T), $v.d = \delta(s, v)$. Базата е при $i = 1$, тоест, $i - 1 = 0$, тоест, нулевото ниво, тоест, само връх s – очевидно твърдението е вярно. В индуктивната стъпка, ако е вярно за върховете от ниво k, прилагаме Лема 63 и заключаваме, че е вярно и за върховете от ниво $k + 1$.

Лесно се вижда, че ако G няма отрицателни цикли, така че най-къси пътища са дефинирани от s до всеки връх, ако продължим да изпълняваме цикъла на редове 3–4 и след $i = n - 1$, няма да има промяна в нито една d-стойност на връх. Иначе казано, нито едно ребро не може да бъде релаксирано с промяна (намаляване) на d-стойността на крайния му връх, понеже се е стигнало до състояние $\forall v \in V(G) : v.d = \delta(s, v)$ и това е окончателно съгласно Лема 61. Това показва и коректността на проверката за отрицателни цикли на редове 5–7: ако поне едно ребро (x, y) е "релаксируемо", което е същото като булевото условие на ред 6 да е истина, със сигурност има отрицателен цикъл. В такъв случай, колкото и пъти да продължим да изпълняваве редове 3–4 отвъд $i = n - 1$, ще получаваме "подобрения" на d-стойността на някой връх, за който е вярно, че от s до него има път, съдържащ отрицателен цикъл. □

Сложността по време, очевидно, е $\Theta(n(n+m))$, което означава, че Bellman-Ford е кубичен алгоритъм в най-лошия случай.

12.5 Най-къс път за всяка двойка върхове

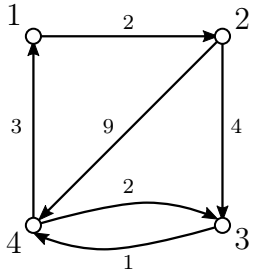
Разглеждаме задачата за най-късите пътища във варианта APShP: за всеки два върха u и v в тегловен ориентиран граф G, търсим дължината на най-къс път $u \rightsquigarrow v$, както и

един такъв път. Това можем да постигнем с алгоритъм за варианта SSShP, като например DIJKSTRA или BELLMAN-FORD (ако има отрицателни тегла), който пускаме от всеки връх. Но има интересен алгоритъм, специализиран за варианта APShP, който си заслужава да бъде разгледан подробно. Това е алгоритъмът на Floyd-Warshall, който, освен че е интересен от теоретична гледна точка, допуска отрицателни тегла и има сложност по време $\Theta(n^3)$, така че е по-бърз от n-кратното пускане на BELLMAN-FORD, което би било със сложност $\Theta(n^4)$.

В тази секция графите се представят не със списъци на съседство, а с матрици от теглата. $G = (\{1, \ldots, n\}, E)$ е графът, който разглеждаме, и той е представен с квадратна матрица $n \times n$ от теглата, наречена W, като

$$W[i,j] = \begin{cases} 0, \text{ ако } i = j, \\ \text{теглото на реброто } (i,j), \text{ ако има такова и } i \neq j \\ \infty, \text{ ако няма ребро } (i,j) \text{ и } i \neq j \end{cases}$$

Теглата са произволни реални числа. Ето пример:



	1	2	3	4
1	0	2	∞	∞
2	∞	0	4	9
3	∞	∞	0	1
4	3	∞	2	0

Разговорно казано, смисълът на $W[i,j]$ е (при липса на отрицателни цикли) "минималната цена на **директно отиване** от връх i във връх j": цената на отиване от i в i е нула, а цената от i в j за $i \neq j$ е, ако няма ребро (i,j), то тя е безкрайност, иначе е цената на реброто. Става дума именно за директно отиване—без междинни върхове—а не за най-къс път. В примера, директното отиване от връх 2 във връх 4 има цена 9 и затова $W[2,4] = 9$; това, че има път от 2 до 4 с цена 5 няма значение, тъй като този път съдържа връх 3 като вътрешен връх.

На първо време се фокусираме върху изчисляването на теглата на най-къси пътища. Когато постигнем това, лесно ще добавим код, който изчислява самите пътища. И така, изходът е квадратна матрица $n \times n$, наречена D, като $D[i,j]$ съдържа тегло на най-къс път $i \rightsquigarrow j$.

12.5.1 Алгоритъм, реализиращ матрично умножение

Според [21, стр. 706], този алгоритъм е фолклор. Това означава, че авторството му е неясно и загубено в миналото.

Това е първият алгоритъм по алгоритмичната схема Динамично Програмиране, който ще разгледаме. Тази алгоритмична схема е темата на Лекция 13, но алгоритъмът е за най-къси пътища и мястото му е в тази лекция.

Както всеки алгоритъм по схемата Динамично Програмиране, дизайна на този алгоритъм се състои от следните фази.

1. Измисляне на *характеристика на структурата на оптимално решение*; в нашия случай, на най-къс път.

2. Създаване на подходяща *рекурсивна декомпозиция* на обектите, които искаме да пресмятаме.

3. След това се решава как да се изчисляват ефикасно оптималните решения, но вече не рекурсивно, а отдолу-нагоре (bottom-up).

Първите две фази са по-скоро математически. Третата фаза е изчислителна (алгоритмична). Самият алгоритъм се съставя в третата фаза, но първите две фази са решаващо важни. Ако не разкрием смислено структурата на оптимално решение и/или не намерим смислена рекурсивна декомпозиция на оптималните решения, няма да можем да конструираме алгоритъм. Обектите, които ни интересуват (най-къси пътища в случая) може да бъдат характеризирани по различни начини, които са в основата на различни алгоритми по схемата Динамично Програмиране.

Да направим следното характеризиране на най-къс път $i \rightsquigarrow j$. Всеки път се състои от някакъв брой ребра, не повече от $n-1$ (ако е прост път). Тогава всеки най-къс път $i \rightsquigarrow j$ има някакъв брой ребра, да кажем t. *// дотук е тривиално...*
Променяме леко формулировката: за някое t, всеки най-къс път $i \rightsquigarrow j$ има не повече от t ребра. *// очевидно – е, и?*
Продължаваме: всеки най-къс път $i \rightsquigarrow j$, който има не повече от t ребра, се състои от най-къс път $i \rightsquigarrow k$, който има не повече от $t-1$ ребра, за някой връх k, плюс реброто (k,j) с тегло $W[k,j]$. *// аха, това е!*

За това характеризиране на структурата ще направим рекурсивна декомпозиция: за произволни i, j, такива че $1 \leqslant i, j \leqslant n$:

- дължината на най-къс път $i \rightsquigarrow j$ с не повече от 0 ребра е 0, ако $i = j$, или ∞, ако $i \neq j$ (това е базата на рекурсията).

- дължината на най-къс път $i \rightsquigarrow j$ с не повече от t ребра е по-малкото от тези две:

  - дължината на най-къс път $i \rightsquigarrow j$ с не повече от $t-1$ ребра
  - минимума от сумите
    * дължината на най-къс път $i \rightsquigarrow k$ с не повече от $t-1$ ребра плюс
    * теглото $W[k,j]$.

    за $k \in \{1, \ldots, n\}$.

Ето схемата за изчисление, базирана на тази рекурсивна декомпозиция. За $t \in \{0, \ldots, n-1\}$, нека $D^{(t)}$ е $n \times n$ матрица от дължините на най-къси пътищата с не повече от t ребра. Тогава $D^{(t)}[i,j]$ дължината на най-къс път с не повече от t ребра от i до j. Тогава можем да изчисляваме рекурсивно така:

$$D^{(0)}[i,j] = \begin{cases} 0, & \text{ако } i = j \\ \infty, & \text{ако } i \neq j \end{cases}$$

$$D^{(t)}[i,j] = \min\left\{D^{(t-1)}[i,j], \min\left\{D^{(t-1)}[i,k] + W[k,j] \mid 1 \leqslant k \leqslant n\right\}\right\}, \text{ за } t > 0$$

Това може да бъде опростено до

$$D^{(0)}[i,j] = \begin{cases} 0, & \text{ако } i = j \\ \infty, & \text{ако } i \neq j \end{cases} \tag{12.10}$$

$$D^{(t)}[i,j] = \min\left\{D^{(t-1)}[i,k] + W[k,j] \mid 1 \leqslant k \leqslant n\right\}, \text{ за } t > 0 \tag{12.11}$$

понеже $D^{(t-1)}[i,j]$ се среща в множеството $\{D^{(t-1)}[i,k]+W[k,j] \mid 1 \leqslant k \leqslant n\}$. За да се убедим в това, да видим, че при $k=j$, $D^{(t-1)}[i,k]+W[k,j]$ става $D^{(t-1)}[i,j]+W[j,j]$, което е $D^{(t-1)}[i,j]$, понеже $W[j,j]=0$.

Решението е матрицата $D^{(n-1)}$, тъй като простите пътища не може да имат повече от $n-1$ ребра.

Изразите (12.10) и (12.11) са на практика **ефективен** рекурсивен алгоритъм: за всеки i и j, $D^{(n-1)}[i,j]$ се пресмята чрез рекурсивни викания на $D^{(n-2)}[i,k]$, за $1 \leqslant k \leqslant n$, а стойностите $W[k,j]$ са част от входа. Този алгоритъм обаче е изключително **неефикасен**! Дървото на рекурсията му има разклоненост n и височина $n-1$, което означава, че сложността му е $\Omega(n^{n-1})$. За каквито и да е реални приложения, такъв алгоритъм е безсмислен.

Но ние можем да направим друг алгоритъм, който работи съгласно (12.10) и (12.11), без ги следва буквално. Стойностите, които трябва да бъдат изчислени общо, не са необятно много. Общо всички матрици са $D^{(0)}$, $D^{(1)}$, ..., $D^{(n-1)}$, тоест, n матрици, всяка с n^2 клетки, така че, съгласно (12.10) и (12.11), трябва да сметнем n^3 стойности. Възниква въпросът, защо има алгоритъм със сложност $\Omega(n^{n-1})$, който пресмята само n^3 неща? Както ще видим в Лекция 13, това се случва, когато **различни** рекурсивни викания пресмятат **една и съща** стойност. Тези викания са напълно отделни и "не знаят" едно за друго, поради което една стойност, веднъж пресметната, се губи и после се пресмята отново и отново.

Примерно, при $n = 300$, пресмятането на $D^{(299)}[1,300]$ ползва стойностите $D^{(298)}[1,2]$, ..., $D^{(298)}[1,300]$ ($D^{(298)}[1,1]$ е 0). На свой ред, пресмятането на всяка от $D^{(298)}[1,2]$, ..., $D^{(298)}[1,300]$ ползва $D^{(297)}[1,2]$, ..., $D^{(297)}[1,300]$. Това означава, че за да намерим $D^{(298)}[1,2]$, ние трябва да знаем $D^{(297)}[1,2]$, ..., $D^{(297)}[1,300]$. Дори да допуснем, че сме намерили $D^{(297)}[1,2]$, ..., $D^{(297)}[1,300]$ и после сме сметнали $D^{(298)}[1,2]$, когато започваме да пресмятаме $D^{(298)}[1,3]$, ще пресметнем всяко от $D^{(297)}[1,2]$, ..., $D^{(297)}[1,300]$ **отново**, ако следваме рекурсията в (12.11) буквално. Ако това разхитително пресмятане на едно и също нещо в различни върхове на дървото на рекурсията става на всяко ниво, не е чудно, че сложността на алгоритъма се оказва $\Omega(n^{n-1})$.

Печелившата идея е, веднъж пресменати, стойностите да не се губят, а се пазят и при необходимост се вземат в $\Theta(1)$ време. Така можем да направим прилично бърз алгоритъм от (12.10) и (12.11). Той вече не е рекурсивен, а итеративен, защото работи отдолу-нагоре: от $D^{(0)}$ намира $D^{(1)}$, от $D^{(1)}$ намира $D^{(2)}$, и така нататък, и накрая от $D^{(n-2)}$ намира $D^{(n-1)}$, което е решението.

Следният алгоритъм реализира тази идея. "MM" идва от "Matrix Multiplication".

SHORTEST PATH MM(W: матрица $n \times n$, представяща тегловен ориентиран граф)

```
 1  for i ← 1 to n
 2      for j ← 1 to n
 3          if i = j
 4              D^(0)[i,j] ← 0
 5          else
 6              D^(0)[i,j] ← ∞
 7  for t ← 1 to n − 1
 8      for i ← 1 to n
 9          for j ← 1 to n
10              D^(t)[i,j] ← ∞
11              for k ← 1 to n
12                  D^(t)[i,j] ← min{D^(t)[i,j], D^(t−1)[i,k] + W[k,j]}
13  return D^(n−1)
```

Коректността му е очевидна за читател, който/която е убеден/убедена в коректността на (12.10) и (12.11), тъй като той пресмята ефикасно тази рекурсия. Сложността му по време очевидно е $\Theta(n^4)$. След малко ще видим как може да подобрим значително тази сложност. Сложността му по памет привидно е $\Theta(n^3)$, защото за всяко $i \in \{0, \ldots, n-1\}$, матрицата $D^{(i)}$ е отделен обект. Всъщност, сложността по памет може лесно да се подобри до $\Theta(n^2)$, като използваме само една матрица и я презаписваме.

Да видим откъде идва "Matrix Multiplication".

Shortest Path MM реализира абстрактно матрично умножение Да разгледаме отново кода на Shortest Path MM, с намалени отстъпи в началото на редовете, и до него да разгледаме код, реализиращ повдигането на $n \times n$ числена матрица A на $(n-1)$-ва степен, започвайки от матрицата-идентитет.

```
Shortest Path MM(W)
for i ← 1 to n
  for j ← 1 to n
   if i = j
    D^(0)[i,j] ← 0
   else
    D^(0)[i,j] ← ∞
for t ← 1 to n − 1
  for i ← 1 to n
   for j ← 1 to n
    D^(t)[i,j] ← ∞
    for k ← 1 to n
     D^(t)[i,j] ← min (D^(t)[i,j],
                       D^(t−1)[i,k] + W[k,j])
return D^(n−1)
```

```
Matix Power(A)
for i ← 1 to n
  for j ← 1 to n
   if i = j
    A^(0)[i,j] ← 1
   else
    A^(0)[i,j] ← 0
for t ← 1 to n − 1
  for i ← 1 to n
   for j ← 1 to n
    A^(t)[i,j] ← 0
    for k ← 1 to n
     A^(t)[i,j] ← + (A^(t)[i,j],
                     A^(t−1)[i,k] · W[k,j])
return A^(n−1)
```

Приликата между двата алгоритъма не е повърхностна, а фундаментална. Shortest Path MM наистина реализира $n-1$ матрични умножения с входната матрица, започвайки от матрицата-идентитет, само че спрямо полупръстен[†] $\mathbb{R} \cup \{\infty\}$ с операции min (абстрактно събиране) и събиране (абстрактно умножение), като абстрактната нула (идентитетът на абстрактното събиране) е ∞, а абстрактната единица (идентитетът на абстрактното умножение) е 0. Съответствията са подчертани чрез различни цветове. Лесно може да проверим, че

1. абстрактното събиране min е асоциативно и комутативно с идентитет ∞;
2. абстрактното умножение $+$ е асоциативно и комутативно с идентитет 0;
3. абстрактното умножение е ляво и дясно дистрибутивно спрямо абстрактното събиране

$$a + \min(b, c) = \min((a + b), (a + c))$$
$$\min(a, b) + c = \min((a + c), (b + c))$$

4. абстрактното умножение $+$ с абстрактната нула ∞ дава абстрактната нула:

$$a + \infty = \infty + a = \infty$$

[†]В алгебрата, полупръстен е структура, подобна на пръстен, но без изискването за обратен елемент на абстрактното събиране. В този случай абстрактното събиране е операцията min, която наистина няма обратен елемент.

така че това наистина е полупръстен. Повече информация за този вид полупръстени и техните свойства има в [39, Глава 35].

И така, Shortest Path MM реализира матрично умножение, като

$$\begin{aligned} \mathrm{D}^{(1)} &= W \\ \mathrm{D}^{(2)} &= W^2 \\ &\dots \\ \mathrm{D}^{(n-1)} &= W^{n-1} \end{aligned}$$

Подобряване на сложността по време до $\Theta(n^3 \lg n)$ Да кажем, че се налага да извършим умножения на матрица A със себе си, така че да получим A^m. Очевидният начин е да правим така, използвайки $\Theta(m)$ матрични умножения:

$$\begin{aligned} A^2 &\leftarrow A \cdot A \\ A^3 &\leftarrow A^2 \cdot A \\ A^4 &\leftarrow A^3 \cdot A \\ &\dots \\ A^m &\leftarrow A^{m-1} \cdot A \end{aligned}$$

Може обаче да постигнем същия резултат със само $\Theta(\lg m)$ матрични умножения:

$$\begin{aligned} A^2 &\leftarrow A \cdot A \\ A^4 &\leftarrow A^2 \cdot A^2 \\ A^8 &\leftarrow A^4 \cdot A^4 \\ &\dots \\ A^{m/2} &\leftarrow A^{m/4} \cdot A^{m/4} \\ A^m &\leftarrow A^{m/2} \cdot A^{m/2} \end{aligned}$$

Тази техника се нарича *повтаряне на повдигането на квадрат*, на английски, *repeated squaring*. На пръв поглед, това работи толкова чисто и елегантно само когато m е точна степен на двойката; в противен случай, предпоследната матрица ще има в степенния показател число, по-малко от m, а последната, число, по-голямо от m.

Ключовото наблюдение е, че по отношение на повдигането на квадрат на матрицата, съдържаща дължините на най-къси пътища с горна граница върху броя на ребрата, където под "умножение на матрици" разбираме умножението от Shortest Path MM, ако входната матрица (тоест, първата степен) няма отрицателни цикли, то след краен брой умножения със себе си, резултатът ще започне да се повтаря. Причината е, че W^m има смисъл именно на дължини на най-къси пътища с не повече от m ребра. При липса на отрицателни цикли, всеки най-къс път е прост (Наблюдение 49) и съдържа най-много $n-1$ ребра. Ерго, може W^{n-2} да е равна или да не е равна на W^{n-1}, но със сигурност

$$W^{n-1} = W^n = W^{n+1} = W^{n+2} = \dots$$

Оттук следва, че ако искаме да получим W^{n-1} с повтаряне на повдигането на квадрат, няма да сбъркаме, ако получим някоя от W^n, W^{n+1}, W^{n+2} и така нататък, защото всички те са равни на W^{n-1}.

И така, търсим най-малката точна степен на двойката, по-голяма или равна на $n-1$, за да я сложим в степенния показател на последната матрица от серията. Тази степен на двойката е $2^{\lceil \lg(n-1)\rceil}$. Сега ще обосновем това. Тръгвайки от очевидните неравенства

$$\lceil p \rceil \geqslant p, \qquad \forall p \in \mathbb{R}$$
$$\lceil p \rceil - 1 < p, \forall p \in \mathbb{R}$$

имаме за $x > 0$

$$\lceil \lg x \rceil \geqslant \lg x \;\leftrightarrow\; 2^{\lceil \lg x \rceil} \geqslant 2^{\lg x} = x \;\leftrightarrow\; 2^{\lceil \lg x \rceil} \geqslant x$$
$$\lceil \lg x \rceil - 1 < \lg x \;\leftrightarrow\; 2^{\lceil \lg x \rceil - 1} < 2^{\lg x} = x \;\leftrightarrow\; 2^{\lceil \lg x \rceil - 1} < x$$

Замествайки x с $n-1$, получаваме

$$2^{\lceil \lg(n-1)\rceil} \geqslant n-1$$
$$2^{\lceil \lg(n-1)\rceil - 1} < n-1$$

И така, търсената степен, на която е необходимо и достатъчно да повдигнем W, за да получим желания резултат, е $2^{\lceil \lg(n-1)\rceil}$.

Всичко това ни позволява да подобрим сложността на намирането на най-къси пътища чрез матрично умножение до $\Theta(n^3 \lg n)$. Разбира се, ако искаме ефикасна реализация, няма да започваме от единичната матрица, за да получим от нея W; матрицата W е входът, така че ние разполагаме с нея. В псевдокода на Shortest Path MM на стр. 459 започнахме от единичната матрица $D^{(0)}$ от съображения за елегантност. Тук започваме от W от съображения за ефикасност. Друга малка оптимизация, която може да се направи, е да не инициализираме $D^{(t)}[i,j]$ (която сега наричаме $\mathrm{tmp}[i,j]$) с ∞, а директно с $W[i,1] + W[1,j]$, и да започнем най-вътрешния цикъл от $k = 2$; отново предпочитаме ефикасността пред теоретичната елегантност.

Shortest Path MM, repeated squaring(W)
```
1  for t ← 1 to ⌈lg(n − 1)⌉
2      for i ← 1 to n
3          for j ← 1 to n
4              tmp[i, j] ← W[i, 1] + W[1, j]
5              for k ← 2 to n
6                  tmp[i, j] ← min(tmp[i, j], W[i, k] + W[k, j])
7      W ← tmp
8  return W
```

Инвариант за най-външния цикъл е, всеки път, когато изпълнението е на ред 1, ако $\widehat{W}$ означава входната матрица, то $W = \widehat{W}^{2^{i-1}}$ за текущата W. Наистина, при първото достигане $i = 1$ и очевидно $W = \widehat{W}^{2^{1-1}} = \widehat{W}^{2^0} = \widehat{W}^1$ в този момент; при всяко следващото достигане степенният показател очевидно се идвоява; при последното достигане имаме $i = \lceil \lg(n-1)\rceil + 1$, така че $W = \widehat{W}^{2^{\lceil \lg(n-1)\rceil + 1 - 1}} = \widehat{W}^{2^{\lceil \lg(n-1)\rceil}}$ в този момент, и алгоритъмът връща тази матрица.

Сложността по време е $\Theta(n^3 \lg n)$, понеже външният цикъл се изпълнява $\Theta(\lg n)$ пъти. Сложността по памет е $\Theta(n^2)$.

12.5.2 Алгоритъмът на Floyd-Warshall

Да характеризираме най-къс път $i \rightsquigarrow j$ по друг начин. Всеки път има множество от вътрешни върхове (което може да е и празното множество, това няма значение). Върховете на графа са $1, \ldots, n$. Тогава всеки най-къс път $i \rightsquigarrow j$ има множество от вътрешни върхове, което може да е и празното множество. *// дотук е тривиално...*
Тогава за всеки най-къс път $i \overset{p}{\rightsquigarrow} j$ има число $k \in \{0, \ldots, n\}$, такова че множеството от вътрешните върхове е подмножество на $\{1, \ldots, k\}$; очевидно $k = 0$ показва, че множеството от вътрешните върхове е празното множество *// е, и?*
Продължаваме: за всеки най-къс път $i \overset{p}{\rightsquigarrow} j$, множеството от вътрешните върхове на който е подмножество на $\{1, \ldots, k\}$, p или съдържа k като междинен връх, или не го съдържа. В първия случай p се състои от най-къс път $i \overset{q}{\rightsquigarrow} k$, "слепен" в общия край k с най-къс път $k \overset{r}{\rightsquigarrow} j$, като q и r имат вътрешни върхове от множеството $\{1, \ldots, k-1\}$. Във втория случай множеството от вътрешните върхове на p е подмножество на $\{1, \ldots, k-1\}$. *// аха, това е!*

За това характеризиране на структурата ще направим рекурсивна декомпозиция. За произволни i, j, такива че $1 \leqslant i, j \leqslant n$, е вярно следното.

- Дължината на най-къс път $i \rightsquigarrow j$, чието множество от вътрешни върхове е празното множество, е или 0, ако $i = j$, или $w((i,j))$, ако $i \neq j$ и $(i,j) \in E(G)$, или ∞, ако $i \neq j$ и $(i,j) \notin E(G)$. С други думи, ако няма вътрешни върхове, то дължината на най-къс път $i \rightsquigarrow j$ е точно $W[i,j]$.

- Нека $i \overset{p}{\rightsquigarrow} j$ с минимално тегло и вътрешните върхове на p са от $\{1, \ldots, k\}$.

  - p може да съдържа връх $k \geqslant 1$ като вътрешен връх. Тогава $w(p)$ е сумата от дължината на най-къс път $i \rightsquigarrow k$, чиито вътрешни върхове са от $\{1, \ldots, k-1\}$, и дължината на най-къс път $k \rightsquigarrow j$, чиито вътрешни върхове са от $\{1, \ldots, k-1\}$. В този случай казваме, че k *помага*.
  - p може да не съдържа връх $k \geqslant 1$ като вътрешен връх. Тогава $w(p)$ е равна на дължината на най-къс път $i \rightsquigarrow j$, чиито вътрешни върхове са от $\{1, \ldots, k-1\}$. В този случай казваме, че k *не помага*.

Тогава можем да изчисляваме рекурсивно така:

$$D^{(0)} \leftarrow W \tag{12.12}$$

$$D^{(k)}[i,j] \leftarrow \min\{\underbrace{D^{(k-1)}[i,j]}_{k \text{ не помага}}, \underbrace{D^{(k-1)}[i,k] + D^{(k-1)}[k,j]}_{k \text{ помага}}\}, \text{ за } k > 0 \tag{12.13}$$

Забележете алгоритмичното предимство на (12.12) и (12.13) пред (12.10) и (12.11)! В (12.12) и (12.13) се пресмята минимум на **2** числа, докато в (12.10) и (12.11) се пресмята минимум на n числа.

Естествено, ако имплементираме (12.12) и (12.13) директно, ще получим алгоритъм с експоненциална сложност по време, защото при разклоненост 2 и височина $\Theta(n)$ на дървото на рекурсията сложността по време е $\Omega(2^n)$. Но, също както при предишния алгоритъм, ние няма да имплементираме (12.12) и (12.13) директно, а ще съобразим, че всъщност има само $\Theta(n^3)$ различни стойности, които се налага да изчислим, които са разпределени в матрици $D^{(0)}$, $D^{(1)}$, ..., $D^{(n)}$, всяка от които е $n \times n$ и се пресмята от предишната. Тези матрици ще запълваме отдолу-нагоре, точно както диктува схемата Динамично Програмиране.

Всичко това води до следния кубичен алгоритъм за задачата APShP, който е публикуван от Robert Floyd [26], но се базира на теорема на Stephen Warshall [93].

```
FLOYD-WARSHALL(W: матрица n × n, представяща тегловен ориентиран граф)
1  D^(0) ← W
2  for k ← 1 to n
3      for i ← 1 to n
4          for j ← 1 to n
5              D^(k)[i,j] ← min(D^(k−1)[i,j], D^(k−1)[i,k] + D^(k−1)[k,j])
6  return D^(n)
```

Коректност и сложност по време и памет. Коректността на FLOYD-WARSHALL е очевидна, ако читателят е убеден в коректността на (12.12) и (12.13). Сложността му по време е $\Theta(n^3)$. Сложността му по памет, ако бъде реализиран по точно този начин, е $\Theta(n^3)$, но може лесно да бъде подобрена до $\Theta(n^2)$, избягвайки създаването на $n+1$ матрици и използвайки само две матрици, една, от която четем стойности (тя отговаря на $D^{(k-1)}$) и една, в която пишем (тя отговаря на $D^{(k)}$).

Заслужава да се отбележи, че всъщност може да минем със само една матрица, тоест, можем да пишем директно върху матрицата, от която четем, и алгоритъмът ще работи коректно. Това е задача 25.2-4 от [21, стр. 699]. Да видим защо е така. Да кажем, че използваме само една матрица D, която се инициализира с W. Същността на алгоритъма е

$$D[i,j] \leftarrow \min(D[i,j], D[i,k] + D[k,j]) \tag{12.14}$$

Ако $k \neq i$ и $k \neq j$, няма причина това присвояване да води до некоректно изчисление на новото $D[i,j]$. Да кажем БОО, че $k = i$. Тогава (12.14) става

$$D[i,j] \leftarrow \min(D[i,j], D[i,i] + D[i,j]) \tag{12.15}$$

Но $D[i,i]$ е задължително 0 при отсъствие на отрицателни цикли, така че (12.16) става

$$D[i,j] \leftarrow \min(D[i,j], D[i,j]) \tag{12.16}$$

което е същото като $D[i,j]$ да не се променя. Но това е коректно, защото дължината на най-къс път $i \rightsquigarrow j$, съдържащ вътрешен връх не по-голям от i (в момента сме допуснали, че $k = i$), е равна на дължината на най-къс път $i \rightsquigarrow j$, съдържащ вътрешен връх не по-голям от $i-1$, защото пътищата са задължително прости (Наблюдение 49) и връх i вече се появява като начало на пътя, ерго, не може да е вътрешен връх. Но $D[i,j]$ преди изпълнението на (12.15) е именно дължината на най-къс път $i \rightsquigarrow j$, съдържащ вътрешен връх не по-голям от $i-1$. С което решаваме задача 25.2-4 от [21, стр. 699].

И още нещо за сложността по памет. Ако в началото на алгоритъма не копираме W в D, а работим **директно върху** W, ще реализираме алгоритъм с **константна сложност по памет**, иначе казано, in-place версия на **Floyd-Warshall**. Да си припомним (Подсекция 2.2.4), че при изследването на сложността по памет отчитаме само допълнителната памет, а входа игнорираме. Така че лесно може да направим версия на **Floyd-Warshall** със сложност по памет $\Theta(1)$, при условие, че сме готови да загубим входа, който ще се окаже презаписан.

Намирането на самите пътища. Дотук разгледахме алгоритмите SHORTEST PATH MM и FLOYD-WARSHALL за APShP, но само за намиране на дължините на пътищата. Сега ще видим как да изчисляваме и самите пътища. Очевидно във варианта APShP няма да минем с $\Theta(n)$ памет за описание на пътищата, както беше във варианта SSShP, а ще ни трябва $\Theta(n^2)$ памет, за да опишем най-къси пътища по отношение на всички двойки върхове. А именно, ще използваме матрица $n \times n$, наречена "Π" в [21, стр. 695], със следния смисъл на

елементите: $\Pi[i,j]$ съдържа предшественика на връх j по най-къс път от връх i. Тъй като целият i-ти ред на Π описва най-къси пътища от връх i, можем да мислим за Π като за n масива на предшествие, каквито имахме в SSShP варианта, разположени в матрица. Иначе казано, i-ият ред на Π описва дърво на най-къси пътища с корен връх i.

Всъщност, алгоритъмът конструира редица от матрици $\Pi^{(0)}$, $\Pi^{(1)}$, $\ldots$, $\Pi^{(n)}$, по една за всяка стойност на k в Floyd-Warshall, а Π е последната от тези матрици, тоест $\Pi = \Pi^{(n)}$. Смисълът на елементите е следният: $\Pi^{(k)}[i,j]$ съдържа предшественика на връх j по най-къс път от връх i, който най-къс път има вътрешни върхове от $\{1,\ldots,k\}$. Ще опишем тези матрици рекурсивно.

$$\Pi^{(0)}[i,j] = \begin{cases} \text{Nil}, & \text{ако } i = j \text{ или } W[i,j] = \infty \\ i, & \text{ако } i \neq j \text{ и } W[i,j] < \infty \end{cases}$$

$$\Pi^{(k)}[i,j] = \begin{cases} \Pi^{(k-1)}[i,j], & \text{ако } D^{(k-1)}[i,j] \leqslant D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \quad // k \text{ не помага} \\ \Pi^{(k-1)}[k,j], & \text{ако } D^{(k-1)}[i,j] > D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \quad // k \text{ помага} \end{cases}, \text{ за } k > 0$$

Следва псевдокод на Floyd-Warshall, който пресмята и пътища.

```
Floyd-Warshall paths(W: матрица n × n, представяща тегловен ориентиран граф)
 1  for i ← 1 to n
 2      for j ← 1 to n
 3          D^(0)[i,j] ← W[i,j]
 4          if i = j or W[i,j] = ∞
 5              Π^(0)[i,j] ← Nil
 6          else
 7              Π^(0)[i,j] ← i
 8  for k ← 1 to n
 9      for i ← 1 to n
10          for j ← 1 to n
11              if D^(k-1)[i,j] ⩽ D^(k-1)[i,k] + D^(k-1)[k,j]
12                  D^(k)[i,j] ← D^(k-1)[i,j]
13                  Π^(k)[i,j] ← Π^(k-1)[i,j]
14              else
15                  D^(k)[i,j] ← D^(k-1)[i,k] + D^(k-1)[k,j]
16                  Π^(k)[i,j] ← Π^(k-1)[k,j]
17  return (D^(n), Π^(n))
```

След като веднъж разполагаме с матрицата $\Pi = \Pi^{(n)}$, ето как може да получим най-късия път от i до j, който тя описва, ако има такъв, или информация, че път от i до j няма.

```
Print APShP(Π, i, j)
1  if i = j
2      print i
3  else
4      if Π[i,j] = Nil
5          print no path
6      else
7          Print APShP(Π, i, Π[i,j])
```

```
8           print j
```

Пример за работата на FLOYD-WARSHALL PATHS има в [21, стр. 696]. Там са показани всички матрици $D^{(0)}$, $\Pi^{(0)}$, ..., $D^{(n)}$, $\Pi^{(n)}$, които се получават при работата на алгоритъма върху графа на стр. 690.

Лекция 13

Динамично Програмиране.

Резюме: Започваме с прости примери: задачата на опашката пред касата и пресмятане на числа на Fibonacci. Разглеждаме основата на алгоритмичната схема Динамично Програмиране, след което разглеждаме множество примери за нейното успешно прилагане: пресмятане на биномни коефициенти, числа на Stirling и брой на целочислени разбивания, Matrix-Chain Multiplication, Minimum Triangulation, задачата за неасоциативното умножение, деривация в контекстно-свободна граматика, Set Partition, 2 Equal Sum Subsets, Knapsack, 2 Equal Sum Subsets, Interval Scheduling, Longest Common Subsequence, Minimum Vertex Cover on Trees, Minimum Dominating Set on Trees. Въвеждаме понятието "псевдополиномиална сложност" в контекста на задачите върху множества. Споменаваме мемоизацията като алтернативен подход на динамичното програмиране. Накрая стигаме до ограниченията на динамичното програмиране: неефикасен алгоритъм за **NP**-пълната задача за намиране на най-дълъг път в графи, построен по схемата Динамично Програмиране.

13.1 Въведение

13.1.1 Задачата за опашката пред касата

Авторът на записките научи тази задача от професор Красимир Манев. Дадена е каса за билети. Пред касата има опашка от n човека h_1, h_2, ..., h_n, в този ред. Известно е, че h_i би се забавил(а) d_i минути на касата, пазарувайки билетите си. Известно е, че всеки двама души h_i и h_{i+1} (очевидно съседи в опашката) може да се комбинират и да си напазаруват билетите си заедно, като това ще отнеме c_i минути. При дадени времена на индивидуално пазаруване на билети d_1, d_2, ..., d_n и времена на комбинирано пазаруване c_1, c_2, ..., c_{n-1}, да се пресметне минималното време, за което цялата опашка може да си купи билетите.

Ако нямаше възможност за комбиниране на съседи в опашката, отговорът би бил просто $\sum_{i=1}^{n} d_i$. Но при дадените възможности за комбиниране може да има по-бърз начин – примерно, ако $c_1 < d_1 + d_2$, има смисъл h_1 и h_2 да се комбинират, наместо да минават поотделно. Забележете, че не всички комбинирания са възможни: примерно, ако h_1 и h_2 са комбина и минат заедно през касата, възможността h_2 да се комбинира с h_3 отпада; и обратно, ако h_2 и h_3 са комбина, h_1 трябва да мине през касата сам(а). Следователно, ако $c_1 < d_1 + d_2$ и $c_2 < d_2 + d_3$, можем веднага да кажем, че някакво комбиниране сред първите трима души трябва да има, но *a priori* не е ясно какво: дали h_1 с h_2, или h_2 с h_3.

Задачата има решение с "груба сила". Забелязваме, че всяко решение се определя еднозначно от това, кои съседи в опашката се комбинират; останалите хора—тези извън комбините—минават през касата индивидуално с дадените си индивидуални времена. С други думи, може

да дефинираме всяко решение чрез булев (характеристичен) вектор с дължина $n-1$, да кажем $B[1,\ldots,n-1]$, като $B[i]=1$ означава, че h_i се комбинира с h_{i+1}, а $B[i]=0$ означава, че h_i не се комбинира с h_{i+1}. Този вектор не може да има съседни единици, тъй като всеки човек от опашката, който има избор от две комбини (това са всички хора без първия и последния), може да участва в най-много една от тях. Следователно, всички възможни комбинирания **не са** 2^{n-1}, а са толкова, колкото са булевите вектори с дължина $n-1$ без съседни единици. Известно е, че тези вектори са F_{n+1} на брой, където F_n е n-тото число на Fibonacci.

Допълнение 47: Колко са булевите вектори без съседни единици

Нека F_m е m-тото число на Fibonacci. Нека T_m е броят на всички булеви вектори с дължина m, в които няма съседни единици. Ще покажем, че $T_m = F_{m+2}$ за всяко $m \geqslant 1$. Очевидно $T_1 = 2 = F_3$, а $T_2 = 3 = F_4$, защото от четирите булеви вектора с дължина 2, точно 11 не отговаря на условието.
За $m > 2$, съобразяваме, че всеки такъв вектор може да започва с единица, но тогава вторият му елемент задължително е нула и следва булев вектор с дължина $m-2$ без съседни единици, или да започва с нула, като след нея има булев вектор с дължина $m-1$ без съседни единици. По принципа на разбиването, $T_m = T_{m-1} + T_{m-2}$. Доказахме, че $T_m = F_{m+2}$ за всяко $m \geqslant 1$.

Тъй като $F_n \asymp \phi^n$, където $\phi = \frac{1+\sqrt{5}}{2} \approx 1.6$, решението с груба сила е експоненциален алгоритъм, само че по отношение на експонента, чиято основа е по-малка от 2. Въпреки че основата е по-малка от 2, все пак е число, по-голямо от 1, което прави подходът неефикасен на практика.

Ефикасно решение може да се получи със следните съображения. Да си представим, че разглеждаме последователно редиците

$$\begin{aligned} s_1 &= \langle h_1 \rangle \\ s_2 &= \langle h_1, h_2 \rangle \\ s_3 &= \langle h_1, h_2, h_3 \rangle \\ &\ldots \\ s_n &= \langle h_1, h_2, \ldots, h_n \rangle \end{aligned}$$

и се опитваме да получим оптимално решение за s_k чрез оптимални решения за s_{k-1}, s_{k-2}, $\ldots$, s_1. Ако успеем в това, то оптималното решение за s_n ще е и решение на задачата.

Решението за s_1 е просто d_1, защото един човек не може да се комбинира с никого. Решението за s_2 е $\min\{d_1+d_2, c_1\}$, защото или h_1 и h_2 минават индивидуално за време d_1+d_2, или се комбинират и минават за време c_1, а ние искаме минимума от тези времена.

Ключово е ефикасното пресмятане на решението за s_3. Ако кажем, че решението за s_3 е $\min\{d_1+d_2+d_3, c_1+d_3, d_1+c_2\}$, защото

- може и тримата да минат индивидуално за време $d_1+d_2+d_3$,
- може първите двама да се комбинират, оставяйки третия сам, което означава време c_1+d_3,
- може първият да мине сам, а вторият и третият да се комбинират, което означава време d_1+c_2

ще сбъркаме. Този начин на мислене не води до ефикасен алгоритъм. Ако продължим в същия дух, решението за s_4 ще е минимума на 5 числа, за s_5 ще е минимума на 8 числа, за s_6 ще е минимума на 13 числа, и така нататък; за s_k ще търсим минимума на F_{k+1} числа. Ерго, този начин на мислене дава ефективно решение, което обаче има експоненциална сложност по време и е неефикасно. Ефикасното решение за s_3 идва от едно съвсем просто съображение: третият човек може или да мине индивидуално, или да се комбинира с втория.

- Ако h_3 мине индивидуално, решението е d_3 плюс оптимално решение за s_2. Когато сме стигнали до s_3, ние вече имаме оптимално решение за s_2. Дали оптималното решение за s_2 е $d_1 + d_2$ или е c_1, на този етап **не ни интересува**. Ние **вече** сме изчислили кое е по-малко от $d_1 + d_2$ и c_1, пресмятайки решението за s_2, така че няма смисъл да го пресмятаме отново в решението за s_3.

- Ако h_3 се комбинира с h_2, решението е c_2 плюс оптималното решение за s_1, което вече имаме.

Ако продължим по този начин, оптималното решение за s_4 ще е минимума на:

- d_4 плюс оптималното решение за s_3 и

- c_3 плюс оптималното решение за s_2.

В същия дух, оптималното решение за s_i, $3 \leqslant i \leqslant n$, е минимум от само две числа, всяко от които е достъпно във време $\Theta(1)$. Ако искаме само числената стойност на оптималното решение, следният алгоритъм решава задачата.

Нека d и c имат смисълът, с който ги използвахме досега.

```
QUEUE PROCESSING(d[1,...,n], c[1,...,n-1])
 1  създай масив opt[1,...,n]
 2  opt[1] ← d[1]
 3  opt[2] ← min(d[1] + d[2], c[1])
 4  for i ← 3 to n
 5      opt[i] ← min(d[i] + opt[i-1], c[i-1] + opt[i-2])
 6  return opt[n]
```

13.1.2 Числата на Fibonacci

Задачата е, по дадено n, да се изчисли F_n, където

$$F_n = \begin{cases} 0, & \text{ако } n = 0, \\ 1, & \text{ако } n = 1, \\ F_{n-1} + F_{n-2}, & \text{ако } n > 1 \end{cases}$$

При големи стойности на n бихме имали проблеми с представянето на F_n, ако ползваме стандартен език за програмиране като C, защото F_n ще препълва отредената му памет. Да игнорираме това допълнително усложнение, което няма общо с основната мисъл тук; или да допуснем, че ползваме библиотека, осигуряваща работа с цели числа с произволна точност.

Ако имплементираме това рекурентно уравнение като рекурсивна програма на C директно, получаваме фрагмент от този вид:

```
unsigned fib(unsigned n){
if (n == 0)
  return 0;
if (n == 1)
  return 1;
return
  fib(n-1) + fib(n-2);
}
```

Ако пуснем тази програма с вход `10`, изпълнението трае около 0.003 секунди, като резултатът е `55`. Ако пуснем тази програма с вход `45`, изпълнението трае около 8.937 секунди (това очевидно зависи от машината), като резултатът е `1 134 903 170`. Ако пуснем тази програма с вход `47`, изпълнението трае около 23.273 секунди (отново, зависи от машината), като резултатът е `2 971 215 073`. Явно това е програма с експоненциална сложност по време.

Не е трудно да се досетим защо е така. Изпълнението на `fib(45)` вика първо `fib(44)`; на свой ред, `fib(44)` вика `fib(43)` и `fib(42)`, и така нататък. След излизането от `fib(44)` се прави викане на `fib(43)`, което обаче "не знае", че тази стойност вече е била пресметната и започва да я смята наново, като вика `fib(42)` и `fib(41)`, и така нататък. Това ненужно повторно изчисление се случва навсякъде в дървото на рекурсията, а не само в корена. За `fib(45)`, началните условия се достигат точно 1 836 311 903 пъти, а редът `fib(n-1) + fib(n-2);` се достига точно 1 836 311 902 пъти. С други думи, дървото на рекурсията има 1 836 311 903 листа и 1 836 311 902 вътрешни върхове. И всичко това е само за пресмятането на 46 различни стойности $\mathsf{F}_0, \ldots, \mathsf{F}_{45}$.

Следният фрагмент изчислява числа на Fibonacci, като помни последните досега изчислени стойности и не върши излишни изчисления отново и отново:

```
A[0] = 0;
A[1] = 1;
for (i = 2; i <= n;  i ++) {
  temp = A[0] + A[1];
  A[0] = A[1];
  A[1] = temp;
}
```

Изпълнението на този фрагмент за вход `47` е светкавично: 0.002 секунди.

13.2 Фундамент

И при пресмятането на минималното време за минаване на опашката през касата, и при пресмятането на числата на Fibonacci срещаме един и същи феномен: за всеки достатъчно голям вход, решението на задачата се свежда до решения върху подзадачи, които обаче **не са независими**, а имат общи подподзадачи.

При задачата с опашката, решението за цялата опашка $\mathsf{h}_1, \ldots, \mathsf{h}_n$ се свежда до решенията за подопашките $\mathsf{h}_1, \ldots, \mathsf{h}_{n-1}$ и $\mathsf{h}_1, \ldots, \mathsf{h}_{n-2}$, съответно отговарящи на възможностите h_n да не се комбинира с h_{n-1} и h_n да се комбинира с h_{n-1}. Това обаче не е достатъчно, за да конструираме алгоритъм по схемата Динамично Програмиране. Само свеждането ни дава алгоритъм по схемата Разделяй-и-Владей, а, както се убедихме, това е алгоритъм с експоненциална сложност по време. Следващата стъпка към конструирането на ефикасен

алгоритъм се основава на наблюдението, че решението за подопашката $h_1, \ldots, h_{n-1}$ и решението за подопашката $h_1, \ldots, h_{n-2}$ **имат общи подрешения**. Това е ключово! Въз основа на това наблюдение лесно може да измислим алгоритъм, който не е рекурсивен, а работи отдолу-нагоре и съхранява решенията за общите подподзадачи, при което отпада преизчисляването на едни и същи решения отново и отново и отново. В това е същността на схемата Динамично Програмиране.

Допълнение 48: Не всеки Разделяй-и-Владей води до Дин. Прогр.

Има много примери, в които Разделяй-и-Владей не води до ефикасен алгоритъм, изграден по схемата Динамично Програмиране, защото подзадачите от фазата **Разделяй** нямат общи подподзадачи.

Последното е напълно вярно за MERGE SORT, защото сортиранията на двата подмасива нямат нищо общо, но MERGE SORT е ефикасен алгоритъм, така че този пример не е особено удачен. Удачен пример е задачата за местенията на Ханойските кули, ако бъде формулирана по подходящ начин: при дадени n диска да се генерира редицата от минимален брой ходове, която ги премества върху крайния прът. Известно е, че броят на тези ходове е $2^n - 1$. Може би трябва да се подчертае, че задачата не е да се изведе числото $2^n - 1$, а **редицата от ходовете**. Каква е сложността по време? Ако не отчитаме представянето на n, сложността по време не е дефинирана (иначе казано, е безкрайност). Ако n е представено в двоична позиционна бройна система, сложността е двойна експонента. Ако n е представено унарно, сложността е $\Theta(2^n)$, което е единична експонента. Да кажем, че n е представено унарно; примерно, дадени са самите дискове в началото – това на практика е същото като n да е записано в унарна бройна система. Знаем, че задачата за преместването на n диска се свежда до две премествания на $n-1$ диска, но двете премествания на $n-1$ диска трябва да се случат поотделно. Тези две премествания на $n-1$ диска нямат обща част в смисъл, че те **и двете** трябва да се случат в своята пълнота и никаква част от първото преместване на $n-1$ диска не може да замени част от второто преместване. Ерго, подзадачите нямат общи подподзадачи. И наистина, алгоритъмът не е по схемата Динамично Програмиране, а е чист Разделяй-и-Владей алгоритъм с експоненциална сложност по време, при казаните допускания.

Заслужава да се прочете и мнението на Steve Skiena за схемата Динамично Програмиране [86, стр. 273–274]:

> *Once you understand it, dynamic programming is probably the easiest algorithm design technique to apply in practice. In fact, I find that dynamic programming algorithms are often easier to reinvent than to try to look up in a book. That said, until you understand dynamic programming, it seems like magic. You must figure out the trick before you can use it.*
>
> *Dynamic programming is a technique for efficiently implementing a recursive algorithm by storing partial results. The trick is seeing whether the naive recursive algorithm computes the same subproblems over and over and over again. If so, storing the answer for each subproblems in a table to look up instead of recompute can lead to an efficient algorithm. Start with a recursive algorithm or definition. Only once we have a correct recursive algorithm do we worry about speeding it up by using a results matrix.*
>
> *Dynamic programming is generally the right method for optimization problems on combinatorial objects that have an inherent left to right order among components.*

> *Left-to-right objects includes: character strings, rooted trees, polygons, and integer sequences. Dynamic programming is best learned by carefully studying examples until things start to click.*

Последният параграф от цитата заслужава особено внимание. Skiena казва, че Динамично Програмиране е приложимо, когато има някакви обекти, подредени в линейна наредба, и тази наредба не се мени, а е фиксирана. Читателят ще забележи, че това е в сила за всички примери за удачно ползване на Динамично Програмиране, които ще разгледаме. Ако става дума за коренови дървета (за които пише и Skiena), линейната наредба е обхождането на върховете в postorder. Ако става дума за множество, което няма иманентна наредба, наредбата е произволна линейна наредба върху това множество (задачата KNAPSACK, примерно).

И още нещо. Както казва Skiena, трябва значителна практика, за да може дизайнерът на алгоритми да интернализира тази техника и да я прилага успешно.

Допълнение 49: За произхода на "Динамично Програмиране"

Интересен страни́чен въпрос е, защо тази схема се казва по този начин? Има ли статично програмиране, което да различаваме от динамичното? Какво изобщо означава "програмиране" тук? Отговорът е, че името е избрано от създателя на схемата Richard Bellman през 50-те години на 20 век заради привлекателността на "динамично" и прекалено общия смисъл. Bellman описва произхода на името със следния анекдот [8, стр. 59]:

> *An interesting question is, "Where did the name, dynamic programming, come from?" The 1950's were not good years for mathematical research. We had a very interesting gentleman in Washington named Wilson. He was Secretary of Defence, and he actually had a pathological fear and hatred of the word, research. I'm not using the term lightly; I'm using it precisely. His face would suffuse, he would turn red, and he would get violent if people used the term, research, in his presence. You can imagine how he felt, then, about the term, mathematical. The RAND Corporation was employed by the Air Force, and the Air Force had Wilson as its boss, essentially. Hence, I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics inside the RAND Corporation. What title, what name, could I choose? In the first place, I was interested in planning, in decision-making, in thinking. But planning, is not a good word for various reasons. I decided therefore to use the word, "programming". I wanted to get across the idea that this was dynamic, this was multistage, this was time-varying – I thought, let's kill two birds with one stone. Let's take a word which has an absolutely precise meaning, namely dynamic, in the classical physical sense. It also has a very interesting property as an adjective, and that is it's impossible to use the word, dynamic, in the pejorative sense. Try thinking of some combination which will possibly give it a pejorative meaning. It's impossible. Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to. So I used it as an umbrella for my activities.*

Статично програмиране, разбира се, няма, а "програмиране" през 50-те е означавало "систематично планиране", за разлика от съвременния смисъл.

13.3 Задачи за броене на комбинаторни структури

Много от фундаменталните комбинаторни структури като подмножества, разбивания и така нататък може да бъдат броени от алгоритми, основани на схемата Динамично Програмиране. Всеки такъв алгоритъм запълва таблица с числа съгласно някакви правила за рекурсивна декомпозиция и някакви начални условия.

По същество това са комбинаторни задачи за мощности на множества. Тези задачи може да бъдат формулирани като задачи за съставяне на алгоритми, връщащи числа и изградени по схемата Динамично Програмиране, но в същината си това са комбинаторни задачи.

Една забележка, отнасяща се до всички тези алгоритми. При тях входът се състои от две числа n и k, като $0 \leqslant k \leqslant n$. Както винаги, когато входът се състои от константен брой числа, възниква въпросът как дефинираме сложността. Очевидно не в големината на входа, която е $\Theta(1)$[†]. В случая "ефикасен" означава "приемливо бърз на практика за разумно големи стойности на n и k". Това да съдържа много субективизъм и е неформално, но не е безсмислица.

Формално прецизно би било да започнем да разглеждаме **размерите на кодирането на n и k**, но ние няма да правим това; ако мерим с този аршин и n и k са кодирани бинарно, алгоритмите биха били експоненциални.

Друга възможност е да мерим с големината на n; в такъв случаи тези алгоритми са със сложност $O(n^2)$. Това е все едно да допуснем, че n е кодирано унарно.

13.3.1 Биномни коефициенти

Задача 28: Брой на подмножества

Конструирайте ефикасен алгоритъм, който по дадени n и k връща броя на k-елементните подмножества на n-елементно множество, където $k, n \in \mathbb{N}$.

И така, иска се да се сметне $\binom{n}{k}$. Добре известно е, че

$$\binom{n}{k} = \binom{n}{n-k} = \frac{n \cdot (n-1) \cdot \cdots \cdot (n-k+1)}{k!} \tag{13.1}$$

така че биномния коефициент може да бъде изчислен, като първо намерим кое от k и $n-k$ е не по-голямо от другото и после го ползваме в тази формула.

Има обаче и друг начин да се изчисли биномния коефициент:

$$\binom{n}{k} = \begin{cases} 0, & \text{ако } k > n \\ 1, & \text{ако } k = 0 \text{ или } k = n \\ \binom{n-1}{k} + \binom{n-1}{k-1}, & \text{ако } 0 < k < n \end{cases} \tag{13.2}$$

Лесно се вижда, че тези възможности са изчерпателни и взаимно изключващи се. Началните условия се аргументират така.

- $k > n$ означава, че подмножеството има по-голяма мощност от тази на множеството; такова подмножество няма, затова там има нула;

[†]Това прави сложността по време недефинирана или безкрайност; сравнете с Подсекция 2.2.5.

- $k = 0$ означава празното множество като подмножество, а знаем, че празното множество е подмножество на всяко множество;
- $k = n$ означава, че вземаме като подмножество самото множество, а има един начин да сторим това.

Аргументацията за същинската рекурсия е, че при $0 < k < n$, фиксираме произволен елемент от n-елементното множество и разбиваме k-елементните подмножества на тези, които го съдържат, и тези, които не го съдържат; съответно $\binom{n-1}{k-1}$ и $\binom{n-1}{k}$ на брой.

(13.2) задава двумерна рекурсивна декомпозиция. Тя може да се ползва за bottom-up изчисление на биномните коефициенти, което запълва таблица, подобна на Таблица 13.1.

n \ k	0	1	2	3	4	5	6	7	8	9	10
0	1	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0	0
2	1	2	1	0	0	0	0	0	0	0	0
3	1	3	3	1	0	0	0	0	0	0	0
4	1	4	6	4	1	0	0	0	0	0	0
5	1	5	10	10	5	1	0	0	0	0	0
6	1	6	15	20	15	6	1	0	0	0	0
7	1	7	21	35	35	21	7	1	0	0	0
8	1	8	28	56	70	56	28	8	1	0	0
9	1	9	36	84	126	126	84	36	9	1	0
10	1	10	45	120	210	252	210	120	45	10	1

Таблица 13.1: Таблица за изчисляване на биномни коефициенти.

Тази таблица е част от добре известния триъгълник на Pascal. Триъгълникът на Pascal е безкраен, а таблицата показва само "върха" му. Bottom-up означава[†], че се запълва ред по ред, като запълването на ред n става чрез изчисления, ползващи стойности от ред $n - 1$, за $n > 0$. Примерно, числото 84 на ред 9, колона 3, е равно на сумата над числото над него (56) и числото над него вляво (28).

Нулите над диагонала са само с теоретично значение; няма никакъв смисъл да се запълва областта над диагонала.

Дали е смислено да се изчислява биномния коефициент по този начин на практика? За еднократно изчисляване, за каквото става дума в Задача 28, не. За еднократно изчисление е по-добре да се ползва формулата от (13.1). Ако обаче ни се налага да пресмятаме много биномни коефициенти, може да има смисъл да се създаде таблица като Таблица 13.1 с подходящи размери и после биномните коефициенти да се вземат от нея.

Допълнение 50: Имплементацията на `binomial(n,k)` в Maple(TM)

Защо на практика е по-добре да биномният коефициент да се пресмята като дроб посредством (13.1), а не чрез таблица като Таблица 13.1? Това е очевидно от общи съображения дори без детайлен анализ, но има и друг силен аргумент: имплементацията на

[†]По отношение на Таблица 13.1, изчислението, визуално говорейки, е отгоре надолу, но правилният термин е "bottom-up" въпреки това.

функцията `binomial(n,k)`, която пресмята $\binom{n}{k}$, в средата Maple(TM). Тъй като Maple е правен от корифеи в областта на математическия софтуер, може спокойно да вярваме, че тяхната имплементация е най-доброто, което може да се направи на практика.
А как да разберем как е имплементирана тази функция в Maple? Maple е писан на C и на Maple, като има ядро на C, което осигурява достатъчно изчислителни примитиви, и около него има богата и разнообразна обвивка от функции, написани на езика на Maple. Потребителят може да види сорса (написан на Maple) на такава функция чрез командата `eval`, ако преди това нивото на "вербозност" е било повишено на 2. Ето пълен екранен print-out от Maple сесия, показваща сорса на `binomial`.

```
C:\Program Files\Maple 2018\bin.X86_64_WINDOWS>cmaple
    |\^/|     Maple 2018 (X86 64 WINDOWS)
._|\|   |/|_. Copyright (c) Maplesoft, a division of Waterloo Maple Inc. 2018
 \  MAPLE  /  All rights reserved. Maple is a trademark of
 <____ ____>  Waterloo Maple Inc.
      |       Type ? for help.
> interface(verboseproc = 2):
> eval(binomial);
proc(a::algebraic, b::algebraic)
local res, s, c, i, B, AmB, A, fl_res;
global `binomial/a`, `binomial/b`, `binomial/c`;
option builtin = HFloat_binomial, `Copyright (c) 1996 Waterloo Maple Inc. All rights reserved.`;
    if nargs <> 2 then error "expecting two arguments but got %1", nargs end if;
    if type(a, 'complex(float)') or type(b, 'complex(float)') then
        fl_res := evalf('binomial(a, b)'); if type(fl_res, 'complex(float)') then return fl_res end if
    end if;
    B := normal(b);
    if `tools/type`(B, 0) then res := 1
    else
        AmB := normal(a - B);
        if `tools/type`(AmB, 0) then res := 1
        elif `tools/type`(AmB, -1) then
            try s := is(B = 0) catch: s := FAIL end try;
            if s = true then res := 1 elif s = false and coulditbe(B = 0) = false then res := 0 end if
        end if
    end if;
    if not assigned(res) then
        A := normal(a);
        if type(A, 'infinity') or type(B, 'infinity') then res := 'binomial'(a, b)
        elif type(A, 'undefined') then
            res := NumericTools:-ThrowUndefined(a, 'preserve' = `if`(type(B, 'numeric'), 'real', 'none'))
        elif type(B, 'undefined') then
            res := NumericTools:-ThrowUndefined(b, 'preserve' = `if`(type(a, 'numeric'), 'real', 'none'))
        elif hastype([A, B], 'undefined') then res := 'binomial'(a, b)
        elif `tools/type`(B - 1, 0) or `tools/type`(AmB - 1, 0) then
            if `tools/type`(A, 'negint') then return NumericEvent('invalid_operation', A) else res := A end if
        elif `tools/type`(A, 0) then res := piecewise(b = 0, 1, sin(Pi*b)/(Pi*b))
        elif type(A, 'integer') and type(B, 'integer') then
            if 0 < A then
                if B < 0 or A < B then res := 0
                elif A < 2*B then res := binomial(a, a - b)
                elif `binomial/a` = a and `binomial/b` <= b then
                    c := `binomial/c`;
                    for i from `binomial/b` + 1 to b do c := iquo((a - i + 1)*c, i) end do;
                    `binomial/b` := b;
                    `binomial/c` := c;
                    res := `binomial/c`
                elif `binomial/a` = a then
                    c := `binomial/c`;
                    for i from `binomial/b` by -1 to b + 1 do c := iquo(i*c, a - i + 1) end do;
                    `binomial/b` := b;
                    `binomial/c` := c;
                    res := `binomial/c`
                else
                     c := 1;
                     for i to b do c := iquo((a - i + 1)*c, i) end do;
                    `binomial/a` := a;
                    `binomial/b` := b;
```

```
                    ‘binomial/c‘ := c;
                    res := ‘binomial/c‘
                end if
            elif B < 0 and A < B then res := 0
            elif 0 < B then return NumericEvent(’invalid_operation’, (-1)^B*binomial(-AmB - 1, B))
            else return NumericEvent(’invalid_operation’, (-1)^AmB*binomial(-1 - B, -A - 1))
            end if
        elif ‘tools/type‘(A, ’fraction’) and ‘tools/type‘(B, ’integer’) then
            if ‘tools/type‘(B < 0) then res := 0
            elif type(B, ’integer’) then res := (AmB + 1)*binomial(A, B - 1)/B
            else res := ’binomial’(a, b)
            end if
        elif type([A, B, AmB], [’fraction’, ’fraction’, ’integer’]) then res := binomial(a, a - b)
        elif ‘tools/type‘(A, ’negint’) and type(B, ’complex(numeric)’) then
            if type(B, ’integer’) then
                ASSERT(not type(A, ’integer’)); if B < 0 and ‘tools/type‘(AmB, ’negint’) then res := 0 end if
            elif type(B, ’numeric’) then
                return NumericEvent(’real_to_complex’, NumericEvent(’division_by_zero’, infinity + infinity*I))
            else return NumericEvent(’division_by_zero’, infinity + infinity*I)
            end if
        elif type(2*B, ’odd’) and type(A, ’integer’) then
            s := A!/(GAMMA(B + 1)*GAMMA(AmB + 1));
            if has(s, ’GAMMA’) then res := ’binomial’(a, b) else res := s end if
        else
            s := signum(A);
            if member(s, [1, -1]) and ‘tools/type‘(AmB, ’negint’) then
                i := signum(0, B, 0);
                if member(i, [1, -1]) then
                    if s = 1 or s = i then res := 0
                    else
                        try if is(A, ’integer’) and is(B, ’integer’) then if i = -1 then res := 0 end if end if
                        catch:
                        end try
                    end if
                end if
            elif s = 1 and ‘tools/type‘(B, ’negint’) then res := 0
            end if
        end if
    end if;
    if not assigned(res) then
        if ‘tools/type‘(B, ’negint’) then
            if coulditbe(A, ’negint’) = false or coulditbe(AmB, ’nonnegint’) = false then res := 0 end if
        elif ‘tools/type‘(AmB, ’negint’) then
            if coulditbe(A, ’negint’) = false or coulditbe(B, ’nonnegint’) = false then res := 0 end if
        end if
    end if;
    if not assigned(res) then
        if ‘tools/type‘(A, ’negint’) and coulditbe(B, ’integer’) = false then
            if ‘tools/type‘(B, ’real’) then
                return NumericEvent(’real_to_complex’, NumericEvent(’division_by_zero’, infinity + infinity*I))
            else return NumericEvent(’division_by_zero’, infinity + infinity*I)
            end if
        end if
    end if;
    if not assigned(res) then res := ’binomial(a, b)’ end if;
    if type(res, ’nonreal’) and not hastype([args], ’nonreal’) then NumericEvent(’real_to_complex’, res)
    else res
    end if
end proc

> quit
memory used=1.0MB, alloc=8.3MB, time=0.06
```

Редовете в червено са самото изчисление на $\binom{a}{b}$. Функцията `iquo`, от **i**nteger **q**uotient, реализира целочислено деление: първият аргумент се дели на втория и функцията връща резултата. `for`-цикълът започва с `i = 1`, което е неявно, и след това извършва умножение на `b` частни. Това е по-икономично от първо намиране на числителя и знаменателя в (13.1), последвано от деление. Временният резултат `(a - i + 1)*c` се дели на `i` винаги, така че няма загуба на прецизност заради целочисленото деление.

13.3.2 Числа на Stirling от втори род

Нотация 6: "$\left\{{n \atop k}\right\}$"означава числото на Stirling от втори род n-подмножество-k

Числото на Stirling от втори род n-подмножество-k бележим с $\left\{{n \atop k}\right\}$.

Задача 29: Брой на разбивания

Конструирайте ефикасен алгоритъм, който по дадени n и k връща броя на разбиванията на n-елементно множество на k дяла, където $k, n \in \mathbb{N}$.

Иска се да се сметне числото на Stirling от втори род $\left\{{n \atop k}\right\}$. За числата на Stirling от втори род е известна следната формула:

$$\left\{{n \atop k}\right\} = \frac{1}{k!}\sum_{j=0}^{k}(-1)^j\binom{k}{j}(k-j)^n \tag{13.3}$$

Аргументацията е проста: броят на разбиванията се получава от броя на сюрекциите от n-елементен домейн в k-елементен кодомейн чрез деление на $k!$; всяка сюрекция задава някакво разбиване на елементите на домейна на k дяла съобразно това, кои елементи от домейна върху кой елемент от кодомейна се изобразяват, а делим на $k!$, защото няма значение кой от тези дялове на точно кой елемент от кодомейна съответства.

Има и друг начин да се пресмятат числата на Stirling от втори род.

$$\left\{{n \atop k}\right\} = \begin{cases} 1, & \text{ако } k = n \\ 0, & \text{ако } (k = 0 \text{ и } n > 0) \text{ или } n < k \\ k\left\{{n-1 \atop k}\right\} + \left\{{n-1 \atop k-1}\right\}, & \text{ако } 0 < k < n \end{cases} \tag{13.4}$$

Лесно се вижда, че тези възможности са изчерпателни и взаимно изключващи се. Началните условия се аргументират така.

- $k = n$ означава, че дяловете са колкото елементите на множеството; броят на тези разбивания е 1, дори когато множеството е празното множество, а броят на дяловете е (съответно) нула;
- $k = 0$, $n > 0$ означава разбиване на нула дяла на непразно множество, което е невъзможно, така че има нула такива разбивания; $n < k$ означава повече (непразни) дялове, отколкото елементи, което също е невъзможно.

Аргументацията на същинската рекурсия е, че при $0 < k < n$, фиксираме произволен елемент от n-елементното множество и разбиваме разбиванията

- на тези, в които той е сам в дял, а те са $\left\{{n-1 \atop k-1}\right\}$, защото те са колкото разбиванията на $(n-1)$-елементно множество на $k-1$ дяла;
- и на тези, в които той бива "присъединен" към някой вече съществуващ дял на разбиване $(n-1)$-елементно множество на k дяла, като броят на тези разбивания е $\left\{{n-1 \atop k}\right\}$; има k възможности за това, къде да го "присъединим".

(13.4) задава двумерна рекурсивна декомпозиция. Тя може да се ползва за bottom-up изчисление, което запълва таблица за изчисляване на числата на Stirling от втори род, подобна на Таблица 13.2.

n \ k	0	1	2	3	4	5	6	7	8	9	10
0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0	0	0	0
3	0	1	3	1	0	0	0	0	0	0	0
4	0	1	7	6	1	0	0	0	0	0	0
5	0	1	15	25	10	1	0	0	0	0	0
6	0	1	31	90	65	15	1	0	0	0	0
7	0	1	63	301	350	140	21	1	0	0	0
8	0	1	127	966	1 701	1 050	266	28	1	0	0
9	0	1	255	3 025	7 770	6 951	2 646	462	36	1	0
10	0	1	511	9 330	34 105	42 525	22 827	5 880	750	45	1

Таблица 13.2: Таблица за изчисляване на числа на Stirling от втори род.

13.3.3 Числа на Stirling от първи род

Нотация 7: "$\left[{n \atop k}\right]$"означава числото на Stirling от първи род n-цикъл-k

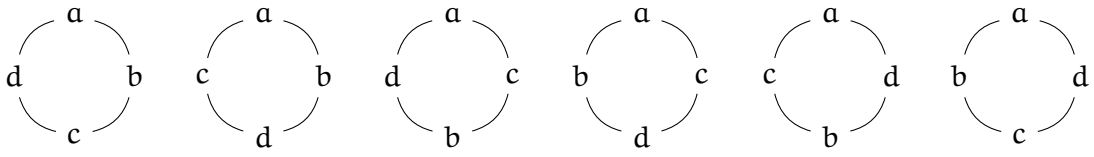
Числото на Stirling от първи род n-цикъл-k бележим с $\left[{n \atop k}\right]$.

Задача 30: Брой на циклични наредби

Конструирайте ефикасен алгоритъм, който по дадени n и k връща броя на начините да бъдат разположени елементите на n-елементно множество в k цикъла, където $k, n \in \mathbb{N}$.

Да поясним какво означава да бъдат разположени елементите в цикли. Нека A е n-елементно множество. Става дума за разбиване на елементите на A на k дяла, като обаче след разбиването подреждаме елементите на всеки дял в кръгова наредба. Различните кръгови наредби на едни и същи елементи са тези, които не могат да се получат една от друга чрез ротация (но не и чрез рефлексия).

Като пример, да разгледаме $S = \{a, b, c, d\}$. Различните кръгови наредби на S са различните начини да бъдат подредени кръгово a, b, c и d. Те са точно шест на брой. Ето ги:



Определение 78: Числа на Stirling от първи род

Броят на начините да бъдат разположени елементите на n-елементно множество в точно k цикъла се нарича *число на Stirling от първи род* n-*цикъл*-k и се бележи с $\left[\begin{smallmatrix} n \\ k \end{smallmatrix}\right]$.

Ясно е, че в общия случай $\left[\begin{smallmatrix} n \\ k \end{smallmatrix}\right]$ е много по-голямо от $\left\{\begin{smallmatrix} n \\ k \end{smallmatrix}\right\}$. Примерно, $\left\{\begin{smallmatrix} 4 \\ 1 \end{smallmatrix}\right\} = 1$, защото начините да разбием 4-елементно множество на един дял са само един (начин). Докато, както вече видяхме, $\left[\begin{smallmatrix} 4 \\ 1 \end{smallmatrix}\right] = 6$. Лесно се обобщава, че $\left[\begin{smallmatrix} n \\ 1 \end{smallmatrix}\right] = (n-1)!$ за всяко $n > 0$, докато $\left\{\begin{smallmatrix} n \\ 1 \end{smallmatrix}\right\} = 1$ за всяко $n > 0$.

За числата на Stirling от първи род е известна формулата [20, стр. 51]:

$$\left[\begin{smallmatrix} n \\ k \end{smallmatrix}\right] = \sum_{0 \leqslant j \leqslant h \leqslant n-k} (-1)^{j+h} \binom{h}{j} \binom{n-1+h}{n-k+h} \binom{2n-k}{n-k-h} \frac{(h-j)^{n-k+h}}{h!} \tag{13.5}$$

Нейното извеждане обаче е много по-сложно от (13.1) или (13.3).

Има и друг начин да се пресмятат числата на Stirling от първи род.

$$\left[\begin{smallmatrix} n \\ k \end{smallmatrix}\right] = \begin{cases} 1, & \text{ако } k = n \\ 0, & \text{ако } (k = 0 \text{ и } n > 0) \text{ или } n < k \\ (n-1)\left[\begin{smallmatrix} n-1 \\ k \end{smallmatrix}\right] + \left[\begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix}\right], & \text{ако } 0 < k < n \end{cases} \tag{13.6}$$

Лесно се вижда, че тези възможности са изчерпателни и взаимно изключващи се. Началните условия се аргументират така.

- $k = n$ означава, че циклите са колкото елементите на множеството; броят на тези разполагания в цикли е 1, дори когато множеството е празното множество, а броят на циклите е (съответно) нула;
- $k = 0$, $n > 0$ означава разполагания в нула цикъла на непразно множество, което е невъзможно, така че има нула такива разбивания; $n < k$ означава повече цикли, отколкото елементи, което също е невъзможно.

Аргументацията на същинската рекурсия е, че при $0 < k < n$, фиксираме произволен елемент от n-елементното множество и разбиваме разполаганията

- на тези, в които той е сам в цикъл, а те са $\left[\begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix}\right]$, защото те са колкото разполаганията на $(n-1)$-елементно множество в $k-1$ цикъла;
- и на тези, в които той бива "вмъкнат" в някой вече съществуващ цикъл на разполагане $(n-1)$-елементно множество в k цикъла; щом елементите са $n-1$, то има точно $n-1$ места, в които той може да бъде вмъкнат, независимо от това, колко са циклите (тоест, колко е k), оттам идва и множителят $(n-1)$.

(13.6) задава двумерна рекурсивна декомпозиция. Тя може да се ползва за bottom-up изчисление на числата на Stirling от първи род, което запълва таблица, подобна на Таблица 13.3.

Има една съществена разлика между числата на Stirling от първи род, от една страна, и биномните коефициенти и числата на Stirling от втори род, от друга страна:

n \ k	0	1	2	3	4	5	6	7	8	9	10
0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0	0	0	0
3	0	2	3	1	0	0	0	0	0	0	0
4	0	6	11	6	1	0	0	0	0	0	0
5	0	24	50	35	10	1	0	0	0	0	0
6	0	120	274	225	85	15	1	0	0	0	0
7	0	720	1 764	1 624	735	175	21	1	0	0	0
8	0	5 040	13 068	13 132	6 769	1 960	322	28	1	0	0
9	0	40 320	109 584	118 124	67 284	22 449	4 536	546	36	1	0
10	0	362 880	1 026 576	1 172 700	723 680	269 325	63 273	9 450	870	45	1

Таблица 13.3: Таблица за изчисляване на числа на Stirling от първи род.

- докато биномните коефициенти и числата на Stirling от втори род се пресмятат по-бързо чрез формулите (13.1) и (13.3), съответно, отколкото чрез алгоритмите по схемата Динамично Програмиране, основани съответно на (13.2) и (13.4),
- то числата на Stirling от първи род се пресмятат по-бързо чрез алгоритъм по схемата Динамично Програмиране, основан на (13.6), отколкото чрез засуканата формула (13.5).

Това се вижда лесно от сорсовете на тези функции в Maple(TM). Числата на Stirling от първи род се пресмятат от вградената функция `stirling1`, а тези от втори род, чрез вградената функция `stirling2`.

За разлика от функцията `binomial` обаче, тези две функции в Maple са част от пакета `combinat` и може да се ползват, ако примерно първо се зареди пакетът с `with(combinat)`.

13.3.4 Брой на целочислени разбивания

Задача 31: Брой на целочислените разбивания

Конструирайте ефикасен алгоритъм, който по дадени n и k връща броя на целочислените разбивания на n на точно k части, където $k, n \in \mathbb{N}$.

Терминът "целочислено разбиване" въведохме в Определение 37. *Частите* на дадено целочислено разбиване на числото n са числата, които се сумират до n в него. Примерно, ако $n = 4$ и разбиването е $2 + 1 + 1$, частите са 2, 1 и 1.

Не трябва да се бърка "целочислено разбиване" с просто "разбиване". Второто е по отношения на множества, първото е по отношение на естествени числа.

Традиционна нотация за броя на всички целочислени разбивания на n е "$p(n)$", а за броя на всички целочислени разбивания на n на точно k части е "$p(n, k)$" или "$p_k(n)$". Тук ще ползваме нотацията, въведена от Knuth [57, стр. 399, (38)].

Нотация 8: "$\left|{n \atop k}\right|$"означава броя на целочислените разбивания на n на k части

Броят на целочислените разбивания на n на точно k части бележим с $\left|{n \atop k}\right|$.

Както става ясно от примера на стр. 222, $\left|{4 \atop 0}\right| = 0$, $\left|{4 \atop 1}\right| = 1$, $\left|{4 \atop 2}\right| = 2$, $\left|{4 \atop 3}\right| = 1$ и $\left|{4 \atop 4}\right| = 1$. Очевидно $\sum_{k=0}^{n} \left|{n \atop k}\right| = p(n)$.

За броя на целочислените разбивания е известна формулата [57, стр. 399, (39)]:

$$\left|{n \atop k}\right| = \begin{cases} 1, & \text{ако } k = n \\ 0, & \text{ако } (k = 0 \text{ и } n > 0) \text{ или } n < k \\ \left|{n-k \atop k}\right| + \left|{n-1 \atop k-1}\right|, & \text{ако } 0 < k < n \end{cases} \tag{13.7}$$

Лесно се вижда, че тези възможности са изчерпателни и взаимно изключващи се. Началните условия се аргументират така.

- $k = n$ означава, че частите са колкото елементите на множеството; броят на тези целочислени разбивания е 1, дори когато $n = 0$ и $k = 0$;
- $k = 0$, $n > 0$ означава целочислено разбиване на положително число на нула части, което е невъзможно, така че има нула такива целочислени разбивания; $n < k$ означава повече части, отколкото е големината на числото, което също е невъзможно.

Аргументацията за същинската рекурсия е при $0 < k < n$ е следната. Разбиваме целочислените разбивания на тези, които имат най-малка част 1, и тези, чиято най-малка част е по-голяма от 1.

- Броят на тези, които имат (поне една) част 1, е $\left|{n-1 \atop k-1}\right|$, понеже съществува очевидна биекция между тях и всички целочислени разбивания на $n-1$ на точно $k-1$ части.
- Броят на тези, чиято най-малка част е поне 2, е $\left|{n-k \atop k}\right|$. Причината е, че ако извадим единица от всяка част (което означава да извадим общо k единици), ще получим k на брой части, всяка от които е поне 1, така че съществува очевидна биекция между целочислените разбивания на n на точно k части, всяка от които е поне 2, и всички целочислени разбивания на $n-k$ на точно k части.

Тъй като най-малката възможна част на всяко целочислено разбиване е единицата, можем да мислим за числото n като за множество от n на брой неразличими обекти, които можем да означаваме с точки. Тогава на всяко целочислено разбиване на n на точно k части съответства групиране на n точки в k групи, като за удобство тези групи са сортирани низходящо по големина. Това ни дава идеята за следното представяне на целочислени разбивания.

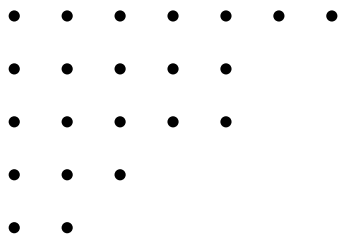
Определение 79: Диаграма на Ferrers

При дадени естествени n и k, такива че $1 \leqslant k \leqslant n$, на всяко целочислено разбиване $\alpha = \alpha_1 + \alpha_2 + \cdots + \alpha_k$ на n съответства биективно диаграма, в която на първия ред има α_1 точки, на втория ред има α_2 точки, и така нататък, на k-ия ред има α_k точки, като във всеки ред разстоянията между точките са еднакви и редовете са подравнени вляво. Такава диаграма се нарича *диаграма на Ferrers*.

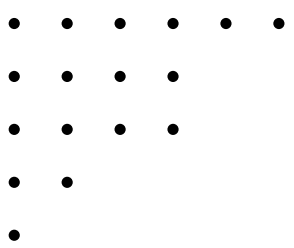
Ето пример за диаграма на Ferrers. Нека $n = 22$. Да разгледаме целочисленото разбиване:

$$\alpha = 7 + 5 + 5 + 3 + 2$$

Съответната диаграма на Ferrers е



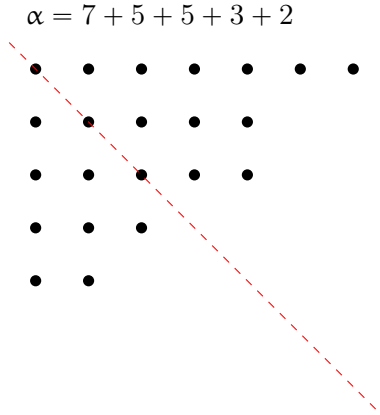
Диаграмите на Ferrers са много удобно средство за онагледяване на целочислени разбивания. Да си припомним формулата $\left|{n \atop k}\right| = \left|{n-k \atop k}\right| + \left|{n-1 \atop k-1}\right|$ при $0 < k < n$. Както вече бе отбелязано, събираемото $\left|{n-k \atop k}\right|$ отговаря на разбиванията, чиято най-малка част е по-голяма от 1. Разбиването $\alpha = 7+5+5+3+2$ е точно такова. Тук $n = 22$ и $k = 5$. И наистина, ако разгледаме горната диаграма на Ferrers и изтрием по една точка от всеки ред, получаваме диаграма на съответното целочислено разбиване на $22 - 5 = 17$ на 5 части:



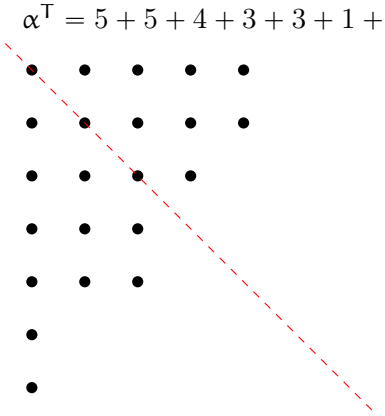
Броят на редовете очевидно се запазва, поради което долният индекс k остава непроменен, но k точки изчезват, поради което горният индекс става $n - k$.

Много полезно понятие е *спрегнатото целочислено разбиване*, на английски *conjugate integer partition* [57, стр. 394], което се илюстрира чудесно чрез диаграми. Спрегнатото разбиване е това, чиято диаграма се получава от дадената чрез рефлексия по отношение на диагонала, минаващ през горния ляв ъгъл. Наместо "спрегнато", можем да кажем "транспонирано" (сравнете с Определение 63). Като пример да разгледаме диаграмите на Ferrers на целочисленото разбиване $\alpha = 7 + 5 + 5 + 3 + 2$ на 22 и спрегнатото му целочислено разбиване $\alpha^T = 5 + 5 + 4 + 3 + 3 + 1 + 1$ (пак на 22, естествено).

$\alpha = 7 + 5 + 5 + 3 + 2$



$\alpha^T = 5 + 5 + 4 + 3 + 3 + 1 + 1$



Лесно се вижда, че най-голямата част на α, а именно 7, е равна на броя на частите на α^{T}. Конверсното също е вярно. Това очевидно може да се се обобщи за всяко целочислено разбиване. Оттук заключаваме, че за всяко n и всяко k броят на целочислените разбивания на n на точно k части е равен на броя на челочислените разбивания на n, чиято най-голяма част е точно k.

Наблюдение 52

$\left|\begin{matrix} n \\ k \end{matrix}\right|$ може алтернативно да се дефинира като броят на целочислените разбивания на n, чиято най-голяма част е точно k.

(13.7) задава двумерна рекурсивна декомпозиция. Тя може да се ползва за bottom-up изчисление на броевете на целочислените разбивания, което запълва таблица, подобна на Таблица 13.4.

n \ k	0	1	2	3	4	5	6	7	8	9	10
0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0	0	0	0
3	0	1	1	1	0	0	0	0	0	0	0
4	0	1	2	1	1	0	0	0	0	0	0
5	0	1	2	2	1	1	0	0	0	0	0
6	0	1	3	3	2	1	1	0	0	0	0
7	0	1	3	4	3	2	1	1	0	0	0
8	0	1	4	5	5	3	2	1	1	0	0
9	0	1	4	7	6	5	3	2	1	1	0
10	0	1	5	8	9	7	5	3	2	1	1

Таблица 13.4: Таблица за изчисляване на броеве на целочислени разбивания.

Незначителна разлика между Таблица 13.1, Таблица 13.2 и Таблица 13.3 от една страна, и Таблица 13.4 от друга страна е, че

- при биномните коефициенти (уравнение (13.2)) и числата на Stirling (уравненения 13.4 и 13.6) в същинската рекурсия, горният индекс вдясно винаги е $n-1$, което означава, че в съответната таблица, ред n-ти се пресмята чрез ред $(n-1)$-ви,
- докато при броя на целочислените разбивания (уравнение 13.7), в същинската рекурсия, горният индекс вдясно е или $n-1$, или $n-k$, откъдето следва, че ред n-ти се пресмята чрез ред $(n-1)$-ви и може би чрез други редове отгоре.

В Maple(TM) целочислените разбивания на числото n се намират с функцията `partition(n)`, която е част от пакета `combinat`. Функцията връща списък от самите разбивания, всяко от които на свой ред е сортиран списък от числата-части на разбиването. Ако искаме само броя на разбиванията, може да го получим с `numelems(partition(n))`.

Функцията `partition` може опционално да получи и втори аргумент, но, малко контраинтуитивно, `partition(n,k)` връща разбиванията, в които най-голямата част е **по-малка или равна** на k, а не е точно k. Ерго, ако искаме да изчислим $\left|\begin{matrix} n \\ k \end{matrix}\right|$ с Maple, може да го направим с `numelems(partition(n,k)) - numelems(partiton(n,k-1))`.

13.4 Задачи за оптимални скобувания на стринг (триъгълна таблица)

В тази секция ще разгледаме задачи, в които присъства някаква линейна наредба, която **условно** наричаме "стринг" и върху която се иска да се намери оптимална декомпозиция, която **условно** наричаме "скобуване". Всички тези задачи се решават ефикасно с алгоритми по схемата Динамично Програмиране с двумерна таблица, в която обаче се ползва (горе-долу) само половината над главния диагонал. Поради това понякога тези алгоритми се наричат и *алгоритми с триъгълна таблица*.

В някои от тези задачи директно е даден стринг, например в задачата за деривация в контекстно-свободна граматика. В други от тях обаче стринг (буквално) няма, примерно задачата за оптимално триангулиране на многоъгълник; но дори и в тях има линейна наредба на някакви обекти, която не се мени по време на работата на алгоритъма (върховете на многоъгълника са линейно наредени след игнорирането на една от страните му, което го превръща от кръгова в линейна наредба). Прочее, да си припомним, че според цитата на Skiena на стр. 471, "Dynamic programming is ... method for optimization problems on combinatorial objects that have an inherent left to right order". Ергo, в някакъв смисъл, всяка задача, податлива на ефикасно решение с алгоритъм по схемата Динамично програмиране, е задача върху стринг. Така че това, че в тази секция говорим за задачи върху стринг, не казва нищо съществено – щом текущата лекция е за Динамично Програмиране, стринг винаги има.

Същественото за тази секция е, че се иска, в някакъв смисъл, оптимално скобуване. Този израз съдържа много условност, защото, примерно при многоъгълниците, скобуване (буквално) няма. Това, което винаги има, е аналогът на главния срез от задачата за матричното умножение; това е точно една "празнина" между два символа на стринга, или един символ от стринга, спрямо която или който се извършва последното действие (каквото и да е то). Иначе казано, скобуването се състои в това, че целият стринг се факторизира на два непразни подстирнга, те на свой ред се факторизират на по два непразни подстринга, и така нататък, докато не достигнем до началните условия. Тази рекурсивна декомпозиция е налице във всички задачи от тази секция; примерно, при оптималната триангулация на многоъгълник, спрямо избраната поначало страна има точно един срещуположен връх, който заедно с нея задава уникалния триъгълник (от триангулацията), в който въпросната страна участва. Е, този уникален връх в някакъв смисъл отговаря на главния срез в стринга. В задачата за триангулацията не е точно факторизиране, защото избраният срещуположен връх участва и в двата подмногоъгълника на декомпозицията (при факторизиране на стринг, всеки символ участва в точно един подстринг-фактор), но, общо-взето, с известна непрецизност може да кажем, че във всички тези задачи има факторизиране на стринга на два подстринга, всеки от които се факторизира на свой ред, и така нататък, докато не "влезем" в начално условие.

13.4.1 Matrix-Chain Multiplication

Започваме с пример. Трябва да извършим матричното умножение $A \cdot B \cdot C \cdot D$, използвайки стандартния алгоритъм за умножение на матрици:

$$\underbrace{\begin{bmatrix}1 & -2 & 2 & -1 & 3\\ 4 & 5 & 4 & -1 & -3\end{bmatrix}}_{A} \cdot \underbrace{\begin{bmatrix}12 & -3\\ -1 & 1\\ 8 & 0\\ -6 & 9\\ 5 & 11\end{bmatrix}}_{B} \cdot \underbrace{\begin{bmatrix}19 & -3 & 0 & 0 & -15 & 8\\ -17 & -9 & 1 & 5 & 12 & -3\end{bmatrix}}_{C} \cdot \underbrace{\begin{bmatrix}4 & -9\\ 3 & 21\\ 6 & -1\\ 0 & -11\\ 3 & 2\\ 1 & -8\end{bmatrix}}_{D} \tag{13.8}$$

Един възможен начин да извършим това е:

$$A \cdot B \cdot C \cdot D = \underbrace{\begin{bmatrix}51 & 19\\ 66 & -49\end{bmatrix}}_{(A\cdot B)} \cdot \underbrace{\begin{bmatrix}19 & -3 & 0 & 0 & -15 & 8\\ -17 & -9 & 1 & 5 & 12 & -3\end{bmatrix}}_{C} \cdot \underbrace{\begin{bmatrix}4 & -9\\ 3 & 21\\ 6 & -1\\ 0 & -11\\ 3 & 2\\ 1 & -8\end{bmatrix}}_{D} =$$

$$\underbrace{\begin{bmatrix}646 & -324 & 19 & 95 & -537 & 351\\ 2087 & 243 & -49 & -245 & -1578 & 675\end{bmatrix}}_{((A\cdot B)\cdot C)} \cdot \underbrace{\begin{bmatrix}4 & -9\\ 3 & 21\\ 6 & -1\\ 0 & -11\\ 3 & 2\\ 1 & -8\end{bmatrix}}_{D} = \underbrace{\begin{bmatrix}466 & -17564\\ 4724 & -19492\end{bmatrix}}_{(((A\cdot B)\cdot C)\cdot D)}$$

Да видим колко скаларни умножения са извършени при това матрично умножение. Ако умножим матрици $M_{x\times y} \cdot N_{y\times z}$, ние извършваме $x \cdot y \cdot z$ скаларни умножение. В случая, A е 2×5, B е 5×2, така че матричното умножение $(A \cdot B)$ "струва" $2 \cdot 5 \cdot 2 = 20$ скаларни умножения. $(A \cdot B)$ е 2×2, а C е 2×6, така че матричното умножение $((A \cdot B) \cdot C)$ "струва" $2 \cdot 2 \cdot 6 = 24$ скаларни умножения. $((A \cdot B) \cdot C)$ е 2×6, а D е 6×2, така че матричното умножение $(((A \cdot B) \cdot C) \cdot D)$ "струва" $2 \cdot 6 \cdot 2 = 24$ скларни умножения. Като цяло, умножението на четирите матрици по този начин "струва" $20 + 24 + 24 = 68$ скаларни умножения.

Друг начин да умножим матриците е

$$\underbrace{\begin{bmatrix}51 & 19\\ 66 & -49\end{bmatrix}}_{(A\cdot B)} \cdot \underbrace{\begin{bmatrix}30 & -328\\ -56 & -44\end{bmatrix}}_{(C\cdot D)} = \underbrace{\begin{bmatrix}466 & -17564\\ 4724 & -19492\end{bmatrix}}_{((A\cdot B)\cdot(C\cdot D))}$$

Крайният резултат е същият. Това не е изненада, понеже умножението на матрици е асоциативно. Но да видим колко скаларни умножения ни струва този начин на умножение на матриците. $(A \cdot B)$ струва, както вече видяхме, 20 скаларни умножение. $(C \cdot D)$ струва $2 \cdot 6 \cdot 2 = 24$ скаларни умножения. $((A \cdot B) \cdot (C \cdot D))$ струва $2 \cdot 2 \cdot 2 = 8$ скаларни умножения. Като цяло, броят на скаларните умножения е $20 + 24 + 8 = 52$. Което е по-малко от 68.

Трети начин да умножим матриците е:

$$\underbrace{\begin{bmatrix}1 & -2 & 2 & -1 & 3\\ 4 & 5 & 4 & -1 & -3\end{bmatrix}}_{A}\cdot\underbrace{\begin{bmatrix}279 & -9 & -3 & -15 & -216 & 105\\ -36 & -6 & 1 & 5 & 27 & -11\\ 152 & -24 & 0 & 0 & -120 & 64\\ -267 & -63 & 9 & 45 & 198 & -75\\ -92 & -114 & 11 & 55 & 57 & 7\end{bmatrix}}_{(B\cdot C)}\cdot\underbrace{\begin{bmatrix}4 & -9\\ 3 & 21\\ 6 & -1\\ 0 & -11\\ 3 & 2\\ 1 & -8\end{bmatrix}}_{D}=$$

$$\underbrace{\begin{bmatrix}646 & -324 & 19 & 95 & -537 & 351\\ 2087 & 243 & -49 & -245 & -1578 & 675\end{bmatrix}}_{(A\cdot(B\cdot C))}\cdot\underbrace{\begin{bmatrix}4 & -9\\ 3 & 21\\ 6 & -1\\ 0 & -11\\ 3 & 2\\ 1 & -8\end{bmatrix}}_{D}=\underbrace{\begin{bmatrix}466 & -17564\\ 4724 & -19492\end{bmatrix}}_{((A\cdot(B\cdot C))\cdot D)}$$

Крайният резултат отново е същият, но да видим колко скаларни умножения са извършени. B е 5×2 и C е 2×6, така че $(B\cdot C)$ струва $5\cdot 2\cdot 6=60$ скаларни умножения. $(A\cdot(B\cdot C))$ струва $2\cdot 5\cdot 6=60$ скаларни умножения. $((A\cdot(B\cdot C))\cdot D)$ струва $2\cdot 6\cdot 2=24$ скаларни умножения. Като цяло, броят на скларните умножения е $60+60+24=144$.

Броят на скаларните умножения дава добра представа за сложността на умножението на матриците, понеже другата операция, събирането (на скаларните произведения), е по-бърза. Заключаваме, че различните начини за извършване на верижно умножение на матрици може да се различават значително като ефикасност, въпреки че крайният резултат е един и същи заради асоциативността на умножението на матрици. Иначе казано, $(((A\cdot B)\cdot C)\cdot D)$, $((A\cdot B)\cdot(C\cdot D))$ и $((A\cdot(B\cdot C))\cdot D)$ са математически неразличими, но алгоритмично (като сложност) може са са доста различни. Причината е видима и в дадения пример: сложността на верижното умножение нараства, когато се появяват междинни матрици с големи размери, като $(B\cdot C)$ в примера. Този феномен се проявява силно, когато матриците-множители са далече от квадратни. Очевидно, ако всички матрици-множители са квадратни, броят на скаларните умножения е един и същи при всеки начин на умножение.

"Начин да бъдат умножени" е същото като "начини за скобуване на веригата". При четири матрици има пет начина за това скобуване (ние разгледахме само три тях в примера):

① $(((A\cdot B)\cdot C)\cdot D)$

② $((A\cdot(B\cdot C))\cdot D)$

③ $((A\cdot B)\cdot(C\cdot D))$

④ $(A\cdot((B\cdot C)\cdot D))$

⑤ $(A\cdot(B\cdot(C\cdot D)))$

Броят на начините да бъде скобуван израз с n множители е C_{n-1}, където C_n означава n-*тото число на Catalan*. Първите няколко числа на Catalan са $C_0=1$, $C_1=1$, $C_2=2$, $C_3=5$, $C_4=14$. Числата на Catalan растат много бързо, като $C_n\asymp\frac{1}{n\sqrt{n}}4^n$. Следователно, използването на алгоритъм с груба сила за задачата, която разглеждаме, е безсмислено.

Допълнение 51: За числата на Catalan

Да дефинираме n-тото число на Catalan C_n като броят на *балансираните изрази с* $2n$ *скоби*. Балансиран израз с $2n$ скоби е стринг, съдържащ точно n отварящи скоби "(" и точно n затварящи скоби ")", като във всеки префикс, броят на отварящите скоби е по-голям или равен на броя на затварящите скоби.
Ето всички 5 балансирани израза при $n = 3$:

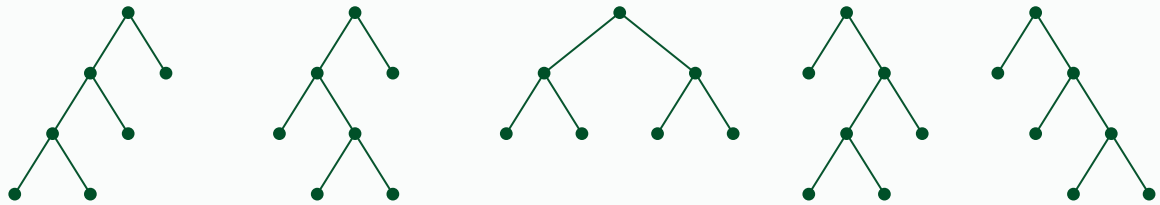
((())) ()(()) ()()() (())() (()())

Забележете, че това не е същото като скобуване на израз с n множители. Тук единствените букви са скобите – множители няма.

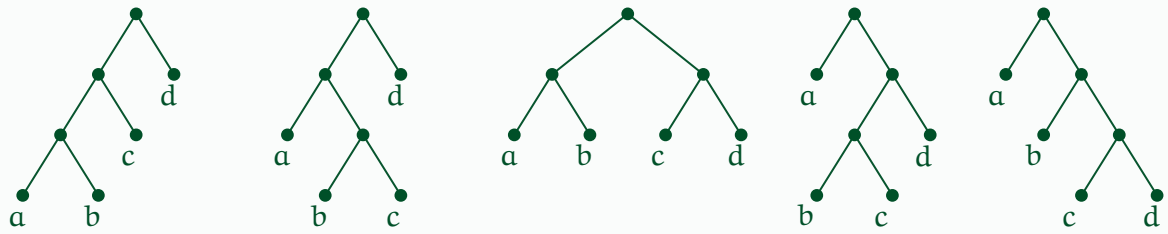
Лема 64

Съществува биекция между скобуваните изрази с $n+1$ множители и балансираните изрази с $2n$ скоби.

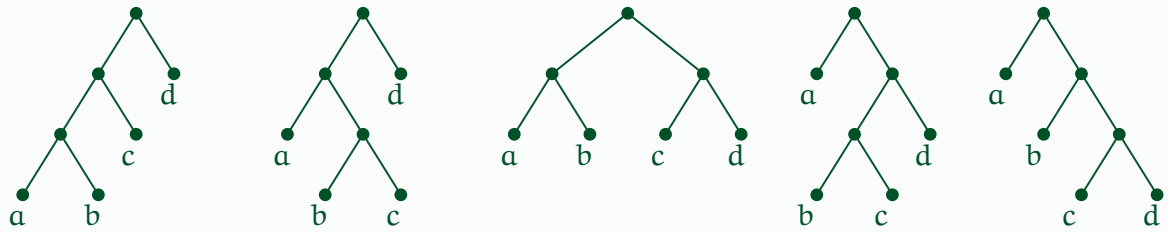
Доказателство. Първо ще покажем с пример, че съществува биекция между скобуваните изрази с $n+1$ множители и пълните подредени двоични дървета с $n+1$ листа. Нека $n = 3$. Тогава $n + 1 = 4$ и дърветата с анонимни върхове са тези:



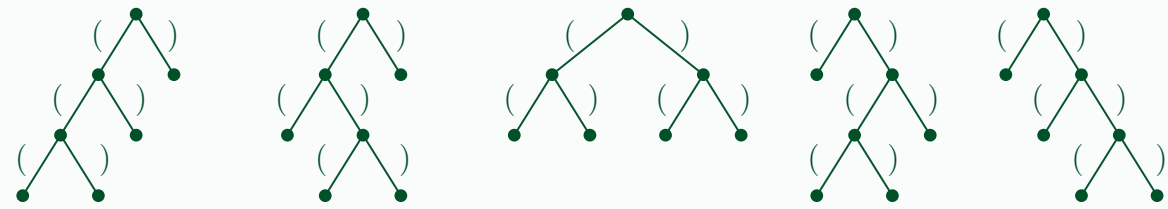
Да сложим имена на листата a, b, c и d, така че при обхождане в preorder (или postorder, няма значение), те се появяват в този ред:



Ето и съответните скобувани изрази:



Показахме с пример биекция между скобуваните изрази с $n+1$ множители и пълните подредени двоични дървета с $n+1$ листа. Върху същия пример ще покажем биекция между пълните подредени двоични дървета с $n+1$ листа и балансираните изрази с $2n$ скоби. Забележете, че при $n+1$ листа, вътрешните върхове са n на брой, така че общо върховете са $2n+1$, откъдето следва, че ребрата са точно $2n$. Ще маркираме всяко ребро или с "(", ако е ребро към ляво дете, или с ")", ако е ребро към дясно дете. Балансираните изрази са това, което би принтирал DFS, който обхожда дървото и принтира ребрата в реда на откриването, като във всеки вътрешен връх първо отива към лявото дете и след това, към дясното дете.



Това е краят на доказателството на Лема 64. □

Лема 65

Редицата от числата на Catalan е решението на следното рекурентно уравнение:

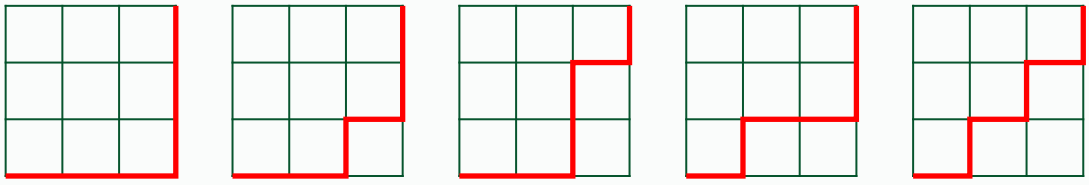
$$C_n = \begin{cases} 1, & \text{ако } n = 0, \\ \sum_{i=0}^{n-1} C_i \cdot C_{n-1-i}, & \text{ако } n > 0 \end{cases} \tag{13.9}$$

Доказателство. Ще покажем с индукция по n, че (13.9) брои скобуваните изрази с $n+1$ множители. Но съгласно Лема 64 те са колкото балансираните изрази с $2n$ скоби, откъдето желаният резултат следва веднага.
Ако $n = 0$, (13.9) дава 1, а добре скобуваният израз с $0+1$ множителя е само един: (a), ако игнорираме идентичността на множителя. ✓

Ако $n \geqslant 1$, става дума за $n+1$ множители. В линейната наредба между тях има $(n+1)-1 = n$ "празнини", във всяка от които може да попада последното умножение. Същото нещо, казано в термините на пълните двоични подредени дървета, чиито листа са множителите, е: при фиксирана наредба отляво надясно на листата, има n възможности за това, кои листа са вляво от корена (което определя кои са вдясно от корена). Нека i е номерът на "празнината", в която е последното умножение, като i взема такива стойности, че вляво от тази празнина има $i+1$ множителя. Това означава, че вдясно има $(n+1)-(i+1) = n-i$ множителя. Очевидно минималната стойност на i е 0 (1 множител вляво), а максималната е $n-1$ (n множителя вляво). За всяка позиция на празнината, умножаваме броя на скобуванията на подредицата вляво с броя на скобуванията на подредицата вдясно; съгласно индуктивното предположение, те са съответно C_i ($i+1$ множителя вляво) и C_{n-1-i} ($n-i$ множителя вдясно). Накратко, за всяко i скобуванията са $C_i \cdot C_{n-1-i}$. Прилагаме комбинаторния принцип на разбиването, сумирайки по всички i, и получаваме израза от (13.9). □

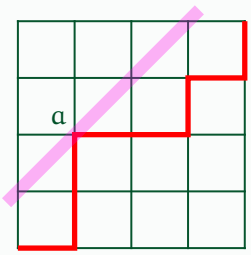
Рекурентното уравнение 13.9 не може да бъде решено нито с Мастър Теоремата, нито с метода с характеристичното уравнение. Ако имаме решение, можем да го докажем по индукция, но как да получим решение?

Ще изведем решение на уравнение 13.9 с елементарни комбинаторни съображения. Това уравнение брои балансираните изрази с $2n$ скоби. Ако отпадне изискването да са балансирани и остане само това, че са n скоби от видá "(" и още n скоби от видá ")", ние знаем колко са тези изрази: $\binom{n+n}{n} = \binom{2n}{n}$. Знаем, че те отговарят биективно на разходки в правоъгълна мрежа $n \times n$, които разходки започват долу вляво и завършват горе вдясно. Нека скобата "(" е ход надясно, а ")" е ход нагоре. Сега връщаме ограничението изразите да са балансирани. Това означава в нито един префикс броят на ")" не е по-голям от броя на "(". Иначе казано, никоя разходка не "излиза" над вторичния диагонал от долу вляво до горе вдясно. Ето петте такива разходки при $n = 3$ със съответните балансирани изрази:

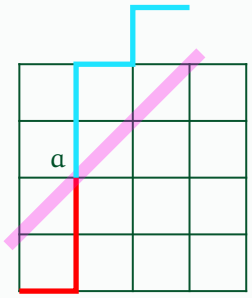


((())) (()()) (())() ()(()) ()()()

Търсим броя на разходките, които никога не излизат над вторичния диагонал, като функция на n. Този брой ще получим, като от $\binom{2n}{n}$ извадим броя на разходките-нарушители – тези, които излизат над вторичния диагонал. Следната фигура показва как броим нарушителите: всяка разходка-нарушител достига до поне една точка от диагонала над вторичния диагонал (в лилаво). От **първата** такава точка до края обръщаме разходката в смисъл, че всеки ход надясно става ход нагоре, и обратно, всеки ход нагоре става под надясно. Това ще наричаме "рефлексия след първата точка".



разходка-нарушител



рефлексия след a

Очевидно рефлектираната разходка става разходка в правоъгълна мрежа $(n-1) \times (n+1)$. Не е трудно да се види, че има биекция между разходките-нарушители в оригиналната $n \times n$ мрежа и всички разходки в новата $(n-1) \times (n+1)$ мрежа. Оттук заключаваме, че броят на разходките, които не са нарушители, е

$$\binom{2n}{n} - \binom{n-1+n+1}{n-1} = \frac{(2n)!}{n!n!} - \frac{(2n)!}{(n-1)!(n+1)!} = \frac{(2n)!}{n!(n-1)!}\left(\frac{1}{n} - \frac{1}{n+1}\right) = \frac{(2n)!}{n!(n-1)!}\left(\frac{n+1-n}{n(n+1)}\right) = \frac{(2n)!}{n!n!}\left(\frac{1}{n+1}\right) = \frac{1}{n+1}\binom{2n}{n}$$

Показахме, че $C_n = \frac{1}{n+1}\binom{2n}{n}$. Това е решението на (13.9).

Да видим асимптотиката на C_n. От Теорема 24 следва, че $\binom{2n}{n} \asymp \frac{1}{\sqrt{2n}}2^{2n}$, тоест $\binom{2n}{n} \asymp \frac{1}{\sqrt{n}}4^n$. Тогава $C_n \asymp \frac{1}{n\sqrt{n}}4^n$.

И така, нека е дадено верижно умножение на n матрици

$$A_1 \cdot A_2 \cdot \cdots \cdot A_n$$

като A_1 е $d_0 \times d_1$, A_2 е $d_1 \times d_2$, $\ldots$, A_n е $d_{n-1} \times d_n$. Самите матрици не са дадени, защото тяхното съдържание не ни интересува в тази задача. Интересуваме се от техните размери и търсим начин да бъдат умножени, който минимизира общия брой на скаларните произведения. Задачата може да се формализира така.

Изч. Задача: Matrix-Chain Multiplication

пример: Редица $\langle d_0, d_1, \ldots, d_n \rangle$ от цели положителни числа, която представлява редица от n матрици $\langle A_1, A_2, \ldots, A_n \rangle$, като A_i е с размери $d_{i-1} \times d_i$, за $1 \leqslant i \leqslant n$. Матриците не са дадени.
решение: Скобуване на редицата $\langle A_0, A_1, \ldots, A_n \rangle$, което минимизира броя на скаларните умножения за матричното умножение $A_1 \cdot A_2 \cdot \cdots \cdot A_n$.

Има ефикасен алгоритъм за Matrix-Chain Multiplication, изграден по схемата Динамично Програмиране. На първо време разглеждаме опростена версия на задачата, в която

се иска само минималният брой на скаларните умножения, а самото скобуване (което води до такъв минимален брой скаларни умножения) не се иска.

Първо ще характеризираме оптимално решение на задачата. Всяко оптимално решение има място, на което се извършва последното матрично умножение; в Допълнение 51 използвахме термина "празнина". Формално казано, има матрици A_k и A_{k+1}, за някое $k \in \{1, \ldots, n-1\}$, такова че последното матрично умножение е $B \times C$, където $B = A_1 \cdot A_2 \cdot \cdots \cdot A_k$ и $C = A_{k+1} \cdot A_{k+2} \cdot \cdots \cdot A_n$. Визуално е по-добре да си представим последното матрично умножение така:

$$\underbrace{(A_1 \cdot A_2 \cdot \cdots \cdot A_k)}_{B} \mid \underbrace{(A_{k+1} \cdot A_{k+2} \cdot \cdots \cdot A_n)}_{C}$$

В контекста на тази характеризация, да направим рекурсивна декомпозиция на оптимално решение. Броят на скаларните умножения за цялото матрично умножение е сумата от следните три събираеми:

- броят на скаларните умножения в матричното произведение $A_1 \cdot A_2 \cdot \cdots \cdot A_k$, тоест, за конструиране на матрицата B;
- броят на скаларните умножения в матричното произведение $A_{k+1} \cdot A_{k+2} \cdot \cdots \cdot A_n$, тоест, за конструиране на матрицата C;
- броят на скаларните умножения за матричното умножение $B \cdot C$.

Трябва да разгледаме всички възможни стойности на k, а именно от 1 включително до $n-1$ включително, и да вземем минимума от сумата на трите събираеми.

Тъй като дължината на всяка от веригите за умножение $A_1 \cdot A_2 \cdot \cdots \cdot A_k$ и $A_{k+1} \cdot A_{k+2} \cdot \cdots \cdot A_n$ е по-малка от n, това е истинска рекурсивна декомпозиция, само че е изказана много общо и нямаща начални условия. Началното условие е много просто: ако веригата от матрици за умножение има дължина 1, тоест, ако $n = 1$, решението е 0.

Нека осмислим добре рекурсивната декомпозиция. От написаното човек може да остане с впечатление, че при декомпозицията винаги декомпозираме на две подвериги, всяка от които "опира" в единия край на началната верига; а именно, започва с A_1 или завършва с A_n. Това, разбира се, не е вярно. Ако си представим още едно ниво на декомпозиция (означено на следната фигура със син делител), ясно се вижда, че в общия случай подверигите започват с някаква матрица A_i и завършват с някаква матрица A_j, където $1 \leqslant i \leqslant j \leqslant n$:

$$\underbrace{\Big(\underbrace{(A_1 \cdot \cdots \cdot A_\ell)}_{B_1} \mid \underbrace{(A_{\ell+1} \cdot \cdots \cdot A_k)}_{B_2}\Big)}_{B} \mid \underbrace{\Big(\underbrace{(A_{k+1} \cdot \cdots \cdot A_m)}_{C_1} \mid \underbrace{(A_{m+1} \cdot \cdots \cdot A_n)}_{C_2}\Big)}_{C}$$

Подробната рекурсивна декомпозиция е следната. Нека $M(i,j)$ означава минималния брой скаларни умножения за верижното матрично умножение $A_i \cdot \cdots \cdot A_j$, където $1 \leqslant i \leqslant j \leqslant n$.

$$M(i,j) = \begin{cases} 0, & \text{ако } i = j, \\ \min\{M(i,k) + M(k+1,j) + d_{i-1}d_k d_j \mid i \leqslant k \leqslant j-1\}, & \text{ако } i < j \end{cases} \tag{13.10}$$

$d_{i-1}d_k d_j$ е броят на скаларните умножения в матричното умножение на матрицата-произведение $A_i \cdot \cdots \cdot A_k$ (тя е с размери $(i-1) \times k$) и на матрицата-произведение $A_{k+1} \cdot \cdots \cdot A_j$ (тя е с размери $k \times j$). Управляващият параметър на тази рекурсия е разликата $j - i$. Началното

условие е за стойност 0 на $j - i$, а в рекурсивната част $M(i,j)$ се пресмята чрез някакви $M(i',j')$, където $j' - i' < j - i$.

Решението е $M(1,n)$.

Ето схемата за изчисление, базирана на тази рекурсивна декомпозиция. Нека M е квадратен масив $n \times n$. Изчислителното правило е пълен аналог на (13.10):

$$M[i,j] = \begin{cases} 0, & \text{ако } i = j, \\ \min_{i \leqslant k \leqslant j-1} (M[i,k] + M[k+1,j] + d_{i-1} d_k d_j), & \text{ако } i < j \end{cases} \tag{13.11}$$

Да съобразим как работи изчислението. Базата на рекурсията е $M[i,i] = 0$, за $1 \leqslant i \leqslant n$. С други думи, слагаме нули върху главния диагонал. Въз основа на главния диагонал и изчислителното правило от (13.11) запълваме съседния вдясно диагонал на главния. Въз основа на тези два диагонала и изчислителното правило запълваме следващия диагонал вдясно. И така нататък, като последният диагонал в тази редица от диагонали има един елемент, а именно $M[1,n]$, който е и решението. Забележете, че клетките на масива под главния диагонал не се ползват.

Ето псевдокода на алгоритъм, базиран на тази схема за изчисление.

MATRIX-CHAIN MULTIPLICATION($\langle d_0, d_1, \ldots, d_n \rangle$: цели положителни)
```
1  for i ← 1 to n
2      M[i,i] ← 0
3  for diag ← 2 to n
4      for i ← 1 to n − diag + 1
5          j ← i + diag − 1
6          M[i,j] ← M[i,i] + M[i+1,j] + d_{i−1} d_j d_j
7          for k ← i + 1 to j − 1
8              M[i,j] ← min(M[i,j], M[i,k] + M[k+1,j] + d_{i−1} d_k d_j)
9  return M[1,n]
```

Да видим пример, взет от [21, стр. 376]. Нека $n = 6$ и $\langle d_0, \ldots, d_6 \rangle = \langle 30, 35, 15, 5, 10, 20, 25 \rangle$. Двумерният масив е 6×6. Клетките под главния диагонал не се ползват. Слагаме нули върху главния диагонал.

Да видим колко е $M[1,2]$. Съгласно (13.11),

$$\begin{aligned} M[1,2] &= \min_{1 \leqslant k \leqslant 1} (M[1,k] + M[k+1,2] + d_{1-1} d_k d_2) \\ &= M[1,1] + M[2,2] + d_0 d_1 d_2 = 0 + 0 + 30 \cdot 35 \cdot 15 = 15\,750 \end{aligned}$$

	1	2	3	4	5	6
1	0	15 750				
2		0				
3			0			
4				0		
5					0	
6						0

Аналогично запълваме останалите стойности в диагонала над главния диагонал:

	1	2	3	4	5	6
1	0	15 750				
2		0	2 625			
3			0	750		
4				0	1 000	
5					0	5 000
6						0

Да видим колко е $M[1,3]$. Съгласно (13.11),

$$\begin{aligned} M[1,3] &= \min_{1 \leqslant k \leqslant 2} (M[1,k] + M[k+1,3] + d_{1-1} d_k d_3) \\ &= \min (M[1,1] + M[2,3] + d_0 d_1 d_3, M[1,2] + M[3,3] + d_0 d_2 d_3) \\ &= \min (0 + 2\,625 + 5\,250, 15\,750 + 0 + 2\,250) \\ &= \min (7\,875, 18\,000) = 7\,875 \end{aligned}$$

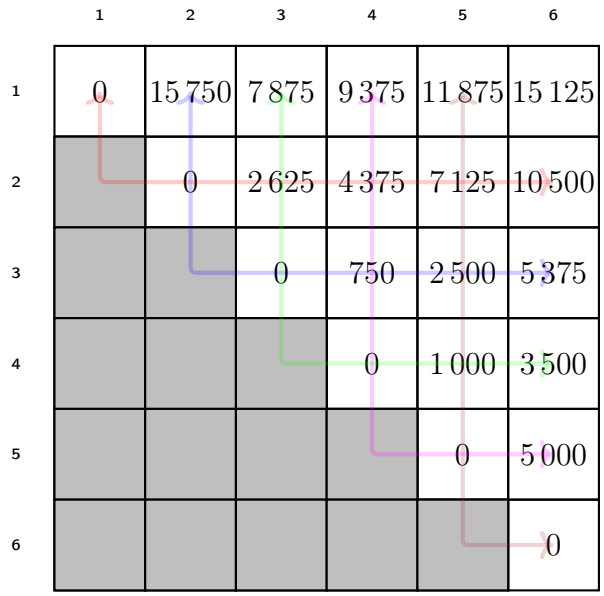
Следната фигура показва запълнената клетка $M[1,3]$, а с цветни линии е показано кои двойки елементи от предишни диагонали участват в минимума.

	1	2	3	4	5	6
1	0	15 750	7 875			
2		0	2 625			
3			0	750		
4				0	1 000	
5					0	5 000
6						0

Да кажем, че главният диагонал е *първият диагонал*, този над него е *вторият диагонал*, а следващият—който запълваме в момента—е *третият диагонал*. Изобщо, номерът на диагонала е номерът на колоната, на която той "излиза" на първия ред. Ето запълването и на третия диагонал.

	1	2	3	4	5	6
1	0	15 750	7 875			
2		0	2 625	4 375		
3			0	750	2 500	
4				0	1 000	3 500
5					0	5 000
6						0

Ето запълването на целия масив. С цветни линии е показано кои двойки елементи от предишни диагонали участват в минимума, с който се изчислява $M[1,6]$.



Това е и краят на примера. Да направим няколко пояснения върху псевдокода на MATRIX-CHAIN MULTIPLICATION. **for**-цикълът на редове 3–8 е с управляваща променлива, наречена не случайно "diag". Тя има смисъл на диагонал в направление горе-ляво – долу-дясно, чийто номер отговаря на номера на колоната, в която този диагонал пресича първия ред. Диагонал 1 е главният диагонал, който запълваме с нули на редове 1–2.

Клетките $M[i,j]$ в диагонал номер diag се отличават с това, че $j = i + \text{diag} - 1$. Примерно, за диагонал номер 2, разликата между j и i е 1: той се състои от $M[1,2], M[2,3], \ldots, M[n-1,n]$. И наистина, на ред 5, на j се присвоява точно $i + \text{diag} - 1$.

Ние запълваме всеки диагонал в посока горе-ляво – долу-дясно. Не е задължително да е така. За всяка клетка от даден диагонал, стойността се изчислява чрез стойности, които се вземат само от диагоналите с **по-малки номера** (както и от d стойностите от входа). Ерго, можем да запълваме даден диагонал в какъвто ред искаме.

Тъй като сме избрали да запълваме всеки диагонал в посока горе-ляво – долу-дясно, има смисъл да инициализираме i с 1 и да го инкрементираме, докато стане $n - (\text{diag} - 1) = n - \text{diag} + 1$, защото $\text{diag} - 1$ е броят на редовете под последния ред, съдържащ елемент на диагонал номер diag.

Коректността на алгоритъма е очевидна за всеки, който е убеден в коректността на рекурсивната декомпозиция, основаната на нея схема за изчисление и на техническите подробности на кода. Сложността по време е $\Theta(n^3)$.

Ако искаме не само оптималната стойност, но и оптималното скобуване. Ако искаме и скобуване, такова че ако умножаваме матриците съгласно него, постигаме минимален брой скаларни умножения, добавяме масив S с размери $n \times n$ (или може да е $(n-1) \times (n-1)$, защото при него дори главният диагонал не се ползва), който запълваме заедно с M. Клетка $S[i,j]$, $i < j$, има смисъл на индекса k в подверигата от матрични умножения $A_i \cdot \cdots \cdot A_j$, $i \leqslant k \leqslant j-1$, такъв че оптималното скобуване за тази подверига е

$$(A_i \cdot A_{i+1} \cdot \cdots \cdot A_k) \mid (A_{k+1} \cdot A_{k+2} \cdot \cdots \cdot A_j)$$

Псевдокодът става следният.

MATRIX-CHAIN MULTIPLICATION+($\langle d_0, d_1, \ldots, d_n \rangle$: цели положителни)
1 **for** $i \leftarrow 1$ **to** n

```
 2       M[i, i] ← 0
 3   for diag ← 2 to n
 4       for i ← 1 to n − diag + 1
 5           j ← i + diag − 1
 6           M[i, j] ← M[i, i] + M[i + 1, j] + d_{i−1} d_j d_j
 7           S[i, j] ← i
 8           for k ← i + 1 to j − 1
 9               if M[i, j] > M[i, k] + M[k + 1, j] + d_{i−1} d_k d_j
10                   M[i, j] ← M[i, k] + M[k + 1, j] + d_{i−1} d_k d_j
11                   S[i, j] ← k
12   return M[1, n], S
```

Следният рекурсивен алгоритъм генерира описание на оптимално скобуване.

```
SHOW-ORDER(S[1, ..., n, 1, ..., n], i, j)
 1  if i = j
 2      print "A_i"
 3  else
 4      k ← S[i, j]
 5      print "("
 6      SHOW-ORDER(S, i, k)
 7      print "*"
 8      SHOW-ORDER(S, k + 1, j)
 9      print ")"
```

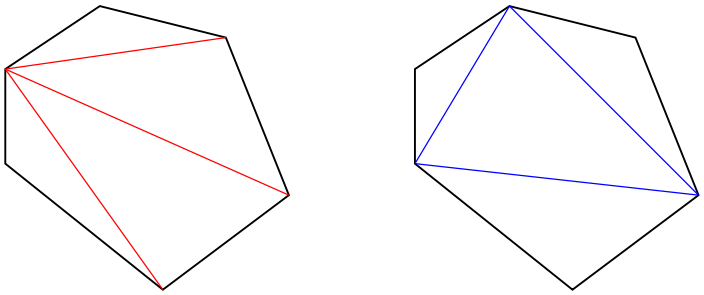
13.4.2 Minimum Weight Triangulation

Прост многоъгълник е множество от отсечки, поне три на брой, които не се пресичат две по две, освен може би в крайните точки, и чието обединение е проста затворена крива. Краищата на отсечките са *върховете на многоъгълника*. *Съседни върхове* са всеки два върха, които са краища на една от тези отсечки. Ако n е броят на отсечките (който е равен на броя на върховете), наричаме фигурата *прост* n*-ъгълник*.

Разглеждаме частен случай на прости многоъгълници, наречени *изпъкнали многоъгълници*. Изпъкнал многоъгълник е прост многоъгълник с вътрешни ъгли, по-малки от 180 градуса. Ако кажем само n*-ъгълник*, разбираме прост изпъкнал n-ъгълник.

Диагонал на прост многоъгълник е отсечка с краища върхове на многоъгълника, които не са съседни. При изпъкналите многоъгълници всички диагонали се съдържат във вътрешността на фигурата.

Триангулация на многоъгълник е множество негови диагонали, които не се пресичат по двойки, освен може би в крайните си точки, и които разбиват вътрешността на фигурата на триъгълници. Триангулирането на многоъгълници е най-важната декомпозиция на многоъгълници с големи приложения в Компютърната геометрия, CAD и други области на Компютърните науки. Ето пример със шестоъгълник и две негови различни триангулации.



Тривиално се доказва, че за всеки n-ъгълник, всяка триангулация има $n-3$ диагонала и $n-2$ триъгълника. Резултатът очевидно е в сила, ако $n=3$. За целите на задачата и алгоритъма, който ще разгледаме, има смисъл да разглеждаме индивидуални отсечки като *изродени многоъгълници*, двуъгълници в този случай, чиито триангулации имат 0 диагонала.

Тегло на триангулация е сумата от дължините на участващите в нея диагонали[†]. Примерно, на показаната фигура с двете триангулации, червената триангулация има тегло 127.61 мм, а синята, 116.86 мм (измерено в приложението, с което са генерирани).

Ако триангулацията се казва X, нейното тегло се бележи с $|X|$.

Изч. Задача: Minimum Weight Triangulation

пример: Редица от точки $\langle V_1, \ldots, V_n \rangle$ в равнината, задаваща изпъкнал n-ъгълник P.
решение: Триангулация на P с минимално тегло.

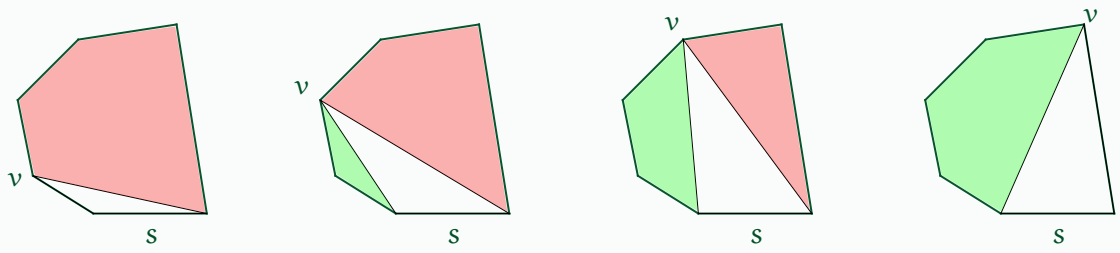
На първо време ще разгледаме по-простия вариант, в който се иска само минимално тегло.

Minimum Weight Triangulation има доста общо с Matrix-Chain Multiplication. Най-малкото, броят на триангулациите на изпъкнал n-ъгълник е C_{n-2}, тоест, $(n-2)$-рото число на Catalan; да си припомним, че броят на скобуванията на израз с n множители е C_{n-1}.

Допълнение 52: Броят на триангулациите на изпъкнал n-ъгълник

Да си представим изпъкнал n-ъгълник за $n \geqslant 3$ и произволна негова триангулация. Да си представим произволна страна s на n-ъгълника. Очевидно s е страна на точно един триъгълник на триангулацията. Този триъгълник се определя еднозначно от s, която задава два от върховете му, и от третия си връх v, който не е връх от s. За v има точно $n-2$ възможности. На следната фигура, триъгълникът е в бяло за всеки избор на v.

[†]Има и други смислени дефиниции на тегло на триангулация. Може теглото да е минималното лице на триъгълник в триангулацията или минималния ъгъл на триъгълник в триангулацията – и двете имат отношение към по-лесното визуално възприемане. При тези дефиниции на тегло, задачата става максимизационна, но това е дребна разлика.



Всеки избор на трети връх v задава един k-ъгълник (в зелено на фигурата) и един m-ъгълник (в червено на фигурата), които, заедно с триъгълника, представляват разбиване на оригиналния n-ъгълник. Забележете, че $k+m = n+1$, понеже k-ъгълникът и m-ъгълникът имат общ връх v. Минималната стойност на k е 2, при което k-ъгълникът е изроден, а m-ъгълникът е $(n-1)$-ъгълник. Максималната стойност на k е $n-1$, при което m-ъгълникът е изроден.
За всеки избор на v, броят на триангулациите е произведението от броя на триангулациите на k-ъгълника и броя на триангулациите на m-ъгълника. Предвид това, че $m = n - k + 1$, броят T_n на триангулациите на n-ъгълника е

$$T_n = \begin{cases} 1, & \text{ако } n = 2, \\ \sum_{k=2}^{n-1} T_k \cdot T_{n-k+1}, & \text{ако } n > 2 \end{cases} \tag{13.12}$$

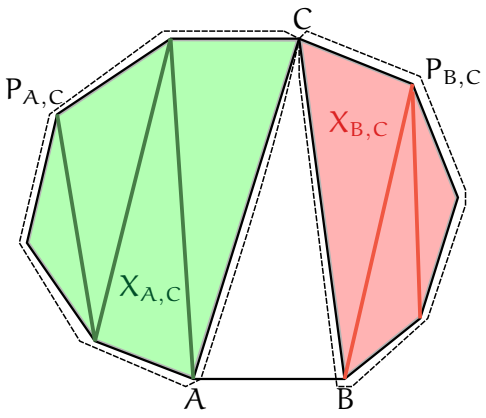
Да сравним (13.12) с (13.9). Веднага се вижда, че $T_n = C_{n-2}$.

Наблюдение 53: Тегло на триангулация, декомпозиция

Нека е даден n-ъгълник P за $n \geqslant 3$ и нека отсечката AB е негова страна. Нека е дадена произволна триангулация X на P. Тогава AB е страна в точно един триъгълник спрямо X. Нека C е третият връх на този триъгълник. Нека многоъгълникът, състоящ се от редицата от отсечките от C до A, които не съдържат B, плюс отсечката AC, се нарича $P_{A,C}$. Нека многоъгълникът, състоящ се от редицата от отсечките от C до B, които не съдържат A, плюс отсечката BC, се нарича $P_{B,C}$. Нека $X_{A,C}$ е триангулацията X, ограничена до $P_{A,C}$. Нека $X_{B,C}$ е триангулацията X, ограничена до $P_{B,C}$.
Тогава

$$|X| = |AC| + |BC| + |X_{A,C}| + |X_{B,C}| \tag{13.13}$$

Това твърдение е прекалено тривиално, за да го доказваме строго. Ето илюстрация. Имайте предвид, че теглото на триангулация е сумата от теглата на диагоналите ѝ. Това означава, че отсечките на многоъгълника не участват в тази сума. В този пример, отсечката AC не участва в $|X_{A,C}|$ и, аналогично, BC не участва в $|X_{B,C}|$. Така че (13.13) наистина брои всички диагонали точно по един път.



Нека входът е редица от точки $\langle V_1, \ldots, V_n \rangle$ в равнината, задаваща изпъкнал n-ъгълник P. Фиксираме посока в равнината, да кажем, по часовниковата стрелка. БОО, нека редицата точки $\langle V_1, \ldots, V_n \rangle$ се среща в посока по часовниковата стрелка, ако обикаляме по периферията на многоъгълника. За всеки i и j, такива че $1 \leqslant i < j \leqslant n$, $P_{i,j}$ е многоъгълникът, състоящ се от отсечките V_iV_{i+1}, $V_{i+1}V_{i+2}$, $\ldots$, $V_{j-1}V_j$ и отсечката V_iV_j, която е или страна, или диагонал на многоъгълника. Примерно, $P_{1,n} = P$, като $V_iV_j = V_1V_n$ е страна. Като други примери: $P_{1,2}$ е изроден многоъгълник, съвпадащ със страната V_1V_2, $P_{2,3}$ е изроден многоъгълник, съвпадащ със страната V_2V_3, и така нататък, $P_{n-1,n}$ е изроден многоъгълник, съвпадащ със страната $V_{n-1}V_n$. Страната V_1V_n не може да бъде изразена по този начин, понеже, както казахме, $P_{1,n}$ е целият многоъгълник.

Рекурсивната декомпозиция, базирана на Наблюдение 53, е следната. При дадени i и j, такива че $i+1 < j$, минималната триангулация на $P_{i,j}$ е минимума по всички точки $k \in \{i+1, \ldots, j-1\}$ от сумата на $|V_iV_k|$, $|V_kV_j|$ и теглата на минимални триангулации на $P_{i,k}$ и $P_{k,j}$. Базата на рекурсията е $j = i + 1$[†]: отсечките $P_{i,i+1}$ са изродени многоъгълници и техните триангулации с тегло нула дават началните условия.

Схемата за изчисление е следната. Нека $T[i,j]$ е теглото на минимална триангулация на $P_{i,j}$, където $1 \leqslant i < j \leqslant n$.

$$T[i,j] = \begin{cases} 0, & \text{ако } j = i+1 \\ \min_{i+1 \leqslant k \leqslant j-1} (T[i,k] + T[k,j] + |V_iV_k| + |V_kV_j|), & \text{ако } j > i+1 \end{cases} \tag{13.14}$$

Има една особеност: разстоянието между точки, които са съседни (иначе казано, които са краища на някоя от отсечките, дефиниращи многоъгълника), трябва да е 0. Не се твърди, че Евклидовото разстояние между тези точки е 0, а че за целите на нашето изчисление функцията, която връща разстоянието между две точки, които са краища на страна на многоъгълника, трябва да връща 0, ако те са съседни. Причината е, че за теглото на триангулацията ни трябват само дължини на диагонали, а съседни точки не са краища на диагонал.

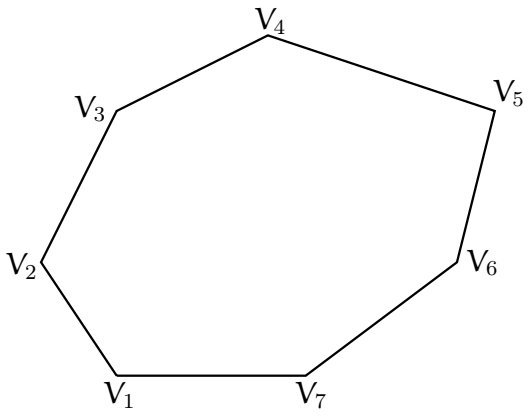
Алгоритъмът е много подобен на Matrix-Chain Multiplication. Дребна разлика е, че тук главният диагонал не се ползва, а следващите два диагонала вдясно от него съдържат само нули: първият заради началните условия (страните на многоъгълника), а вторият поради това, че триъгълниците имат триангулации с нулево тегло, нямайки диагонали.

[†]А не $j = i$. $P_{i,i}$ е индивидуална точка, а чак толкова изродени многоъгълници не разглеждаме.

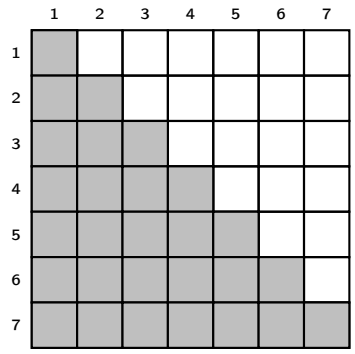
MINIMUM WEIGHT TRIANGULATION($\langle V_1, V_2, \ldots, V_n \rangle$: редица от точки, образуваща изпъкнал многоъгълник)

```
1  for i ← 1 to n − 1
2      T[i, i + 1] ← 0
3  for gap ← 2 to n − 1
4      for i ← 1 to n − gap
5          j ← i + gap
6          T[i, j] ← T[i, i + 1] + T[i + 1, j] + dist(V_i, V_{i+1}) + dist(V_{i+1}, V_j)
7          for k ← i + 2 to j − 1
8              T[i, j] ← min (T[i, j], T[i, k] + T[k, j] + dist(V_i, V_k) + dist(V_k, V_j))
9  return T[1, n]
```

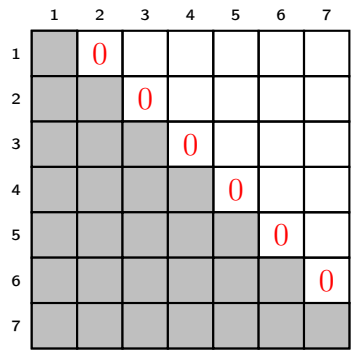
Да разгледаме малък пример със седмоъгълник:



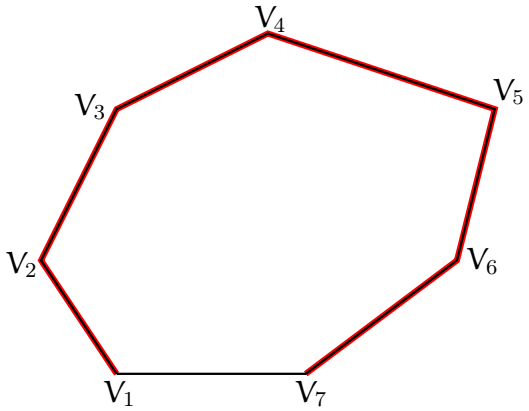
Така изглежда масивът T в началото на алгоритъма. Затъмнени са клетките, които няма да се ползват.



След изпълнението на **for**-цикъла на редове 1–2, T изглежда така:



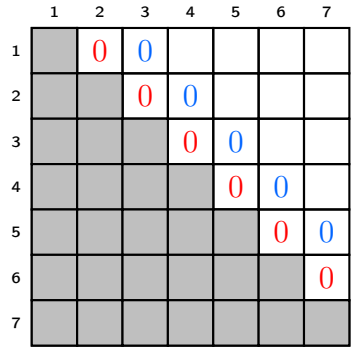
Поставянето на червените нули (това са началните условия) отговаря на триангулациите на изродените многоъгълници; тоест, страните на седмоъгълника без V_1V_7.



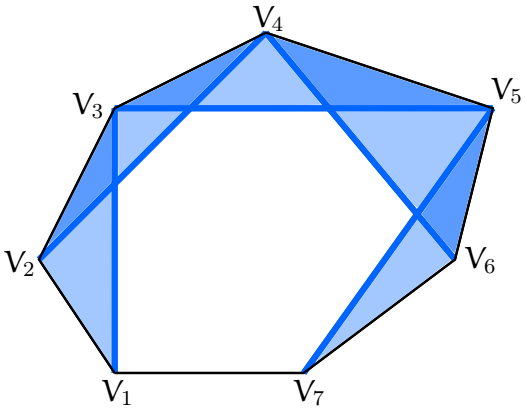
Да разгледаме запълването на следващия диагонал. Имаме gap $= 2$ от ред 3, i $= 1$ от ред 4 и j $= 3$ от ред 5. На ред 6:

$$T[1,3] \leftarrow \underbrace{T[1,2]}_{0} + \underbrace{T[2,3]}_{0} + \underbrace{\mathrm{dist}(V_1,V_2)}_{0} + \underbrace{\mathrm{dist}(V_2,V_3)}_{0}$$

така че $T[1,3]$ става 0. Аналогично, целият диагонал се запълва с нули (в синьо):



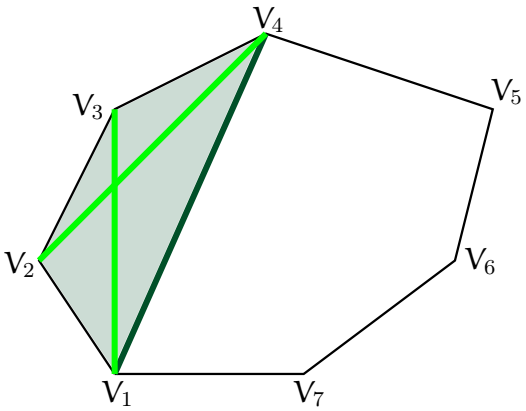
Поставянето на сините нули отговаря на триангулациите на ето тези триъгълници:



Да разгледаме запълването на следващия диагонал. Имаме gap = 3 от ред 3, i = 1 от ред 4 и j = 4 от ред 5. Редове 6–8 реализират това:

$$\begin{aligned} T[1,4] \leftarrow \min(& \underbrace{T[1,2]}_{0} + \underbrace{T[2,4]}_{0} + \underbrace{\mathrm{dist}(V_1,V_2)}_{0} + \underbrace{\mathrm{dist}(V_2,V_4)}_{42.43\ \mathrm{mm}}, \\ & \underbrace{T[1,3]}_{0} + \underbrace{T[3,4]}_{0} + \underbrace{\mathrm{dist}(V_1,V_3)}_{35\ \mathrm{mm}} + \underbrace{\mathrm{dist}(V_3,V_4)}_{0}) \end{aligned}$$

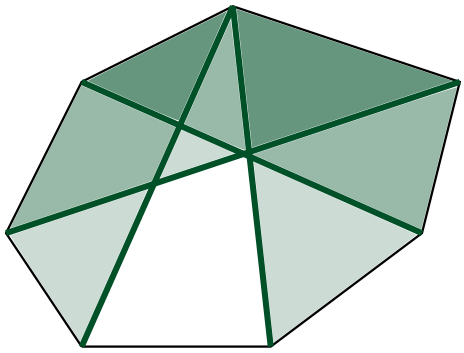
така че $T[1,4]$ става 35 mm. Във фигурата, това изчисление отговаря на минимална триангулация на зеления четириъгълник $P_{1,4}$, която е по-малкият от двата диагонала V_2V_4 и V_1V_3.



В масива записваме 35 в клетка $[1,4]$:

	1	2	3	4	5	6	7
1		0	0	35			
2			0	0			
3				0	0		
4					0	0	
5						0	0
6							0
7							

За да запълним този диагонал, пресмятаме минималните триангулации (тоест, по-малкият от двата диагонала) на тези четири четириъгълника:



След запълването на зеления диагонал, масивът изглежда така:

	1	2	3	4	5	6	7
1		0	0	35			
2			0	0	42.43		
3				0	0	39.05	
4					0	0	43.01
5						0	0
6							0
7							

С това подробната симулация на алгоритъма върху примера спира. Когато gap стане 4, търсим теглото на минимална триангулация на петоълници, три на брой; за всеки петоъгълник взмаме минимума от три стойности, отговарящи на трите междинни върха. Когато gap стане 5, търсим теглото на минимална триангулация на два шестоъгълника; за всеки от тях вземами минимума на четири стойности. И накрая, при gap = 6, намираме теглото на минимална триангулация на целия седмоъгълник като минимума от пет стойности.

Коректността на алгоритъма е очевидна. Сложността му по време е $\Theta(n^3)$.

13.4.3 Факторизация при неасоциативно умножение

Дадена е азбука Σ и стринг $x = x_1x_2\cdots x_n$ над Σ. Нека $|\Sigma| = k$. Дадена е бинарна операция $* : \Sigma^2 \to \Sigma$. Дадена е и буква $\sigma \in \Sigma$. Пита се: има ли скобуване на x, такова че, ако $*$ бъде приложена върху това скобуване, резултатът да бъде σ?

Операцията $*$ да бъде приложена върху скобуване на x означава—при някакво предварително фиксирано скобуване—тя да бъде приложена върху най-вътрешната група букви "(x_ix_{i+1})", резултатът от което е буквата $x_i * x_{i+1}$, и така нататък, докато е възможно.

Наместо строга индуктивна дефиниция ще поясним това с пример. Нека $\Sigma = \{a, b, c\}$. Нека $x = aab$, а следната таблица дефинира $*$:

$*$	a	b	c
a	a	c	c
b	a	a	b
c	b	c	a

x има две скобувания: $((ab)b)$ и $(a(bb))$. Ако приложим $*$ съгласно първото скобуване, получаваме $((a * b) * b) = (c * b) = c$. Ако приложим $*$ съгласно второто скобуване, получаваме $(a * (b * b)) = (a * a) = a$. Причината да има разлика е, че $*$ не е асоциативна.

Ако $*$ е асоциативна, задачата се обезсмисля, защото резултатът е един и същи при всяко скобуване. Но при неасоциативна операция, както видяхме, резултатът от прилагането върху различни скобувания може да е различен.

Задачата прилича много на Matrix-Chain Multiplication. В задачата Matrix-Chain Multiplication матричното умножение е асоциативно, но това няма значение, защото там гледаме не каква е матрицата-резултат, а с цената на колко скаларни умножения я постигаме, което вече не е асоциативно. Съществената разлика между тази задача, от една страна, и Matrix-Chain Multiplication и Minimum Weight Triangulation, от друга, е че в тази задача отговорът не е число. Тази задача се решава ефикасно от алгоритъм, който външно наподобява много Matrix-Chain Multiplication, пак с триъгълна таблица, която се запълва от главния диагонал навън, но сега клетките на таблицата съдържат не числа, а множества от букви. Ако таблицата се казва T, то $T[i,j]$ има следният смисъл:

$$T[i,j] = \begin{cases} x_i, & \text{ако } i = j \\ \bigcup_{k=i}^{j-1} \{a * b \mid a \in T[i,k], b \in T[k+1,j]\}, & \text{ако } i < j \end{cases}$$

Следният алгоритъм реализира ефикасно тази схема за изчисляване.

```
Nonassociative Multiplication(Σ, * : Σ² → Σ, x = x_1 ··· x_n ∈ Σ⁺, σ ∈ Σ)
 1  for i ← 1 to n
 2      T[i,i] ← x_i
 3  for diag ← 2 to n
 4      for i ← 1 to n − diag + 1
 5          j ← i + diag − 1
 6          T[i,j] ← ∅
 7          for k ← i to j − 1
 8              foreach a ∈ T[i,k]
 9                  foreach b ∈ T[k+1,j]
10                      T[i,j] ← T[i,j] ∪ {a * b}
11  if σ ∈ T[1,n]
12      return 1
13  else
14      return 0
```

Коректността на алгоритъма е очевидна предвид това, че той реализира коректна рекурсивна декомпозиция и базирана на нея изчислителна схема. Сложността му по време е $\Theta(n^3)$, ако смятаме, че $k = \Theta(1)$. В противен случай трябва да изразим сложността като функция на n и k по този начин: $\Theta(k^2 n^3)$.

Ето малък пример. Нека $\Sigma = \{1,2,3,4\}$, $x = 123412$, $*$ да е дефинирана чрез таблицата

	1	2	3	4
1	1	3	3	1
2	1	1	4	2
3	2	3	1	2
4	1	4	1	4

и нека $\sigma = 4$.

Ето имплементация на алгоритъма, но само до съставянето на цялата T, на Maple(TM), и пускането върху този пример. Очевидно 4 не се съдържа в $T[1,6]$, така че отговорът е НЕ.

```
C:\Program Files\Maple 2018\bin.X86_64_WINDOWS>cmaple
    |\^/|     Maple 2018 (X86 64 WINDOWS)
._|\|   |/|_. Copyright (c) Maplesoft, a division of Waterloo Maple Inc. 2018
 \  MAPLE  /  All rights reserved. Maple is a trademark of
 <____ ____>  Waterloo Maple Inc.
      |       Type ? for help.
> T := Array(1 .. 6, 1 .. 6, fill = {}):
> x := [1, 2, 3, 4, 1, 2]:
> myoperation := [[1, 3, 3, 1], [1, 1, 4, 2], [2, 3, 1, 2], [1, 4, 1, 4]]:
> for i to 6 do T[i, i] := {x[i]} end do:
> for diag from 2 to 6 do
>   for i from 1 to 6 - diag + 1 do
>     j := i+diag-1:
>     T[i, j] := {}:
>     for k from i to j-1 do
>       for a in op(T[i,k]) do
>         for b in op(T[k+1,j]) do
>           T[i, j] := T[i, j] union {myoperation[a, b]}:
>         end do:
>       end do:
>     end do:
>   end do:
> end do:
> T;
                 [{1}    {3}    {1}    {1, 3}    {1, 2, 3}     {1, 2, 3}  ]
                 [                                                        ]
                 [{}     {2}    {4}    {1, 4}       {1}       {1, 2, 3, 4}]
                 [                                                        ]
                 [{}     {}     {3}     {2}       {1, 2}      {1, 2, 3, 4}]
                 [                                                        ]
                 [{}     {}     {}      {4}         {1}         {1, 3}   ]
                 [                                                        ]
                 [{}     {}     {}       {}         {1}           {3}    ]
                 [                                                        ]
                 [{}     {}     {}       {}         {}            {2}    ]
> quit;
memory used=2.6MB, alloc=8.3MB, time=0.08
```

Забележете, че $T[3,6] = T[2,6] = \{1,2,3,4\}$. Това означава, че подстринговете 23412 и 3412 имат скобувания, върху които операцията дава 4. Но това, че 4 може да бъде получен от подстринг на 123412, не означава, че може да бъде получен от целия стринг 123412.

13.4.4 Деривация в контекстно-свободна граматика

Контекстно-свободна граматика (КСГ) е наредена четворка $\Gamma = \langle \Sigma, N, S, R \rangle$, където Σ е крайно множество от терминали (Σ е азбуката), N е крайно множество от нетерминали, такова че $N \cap \Sigma = \varnothing$, $S \in N$ е фиксиран нетерминал, наречен начален нетерминал, и R е множеството от правила (още се наричат продукции) от вида $A \to X$, където A е нетерминал, а $X \in (\Sigma \cup N)^*$. Формално, R е бинарна релация $R \subseteq N \times (\Sigma \cup N)^*$. Дефинираме бинарната релация $\Rightarrow \subseteq (\Sigma \cup N)^* \times (\Sigma \cup N)^*$ така:

$$x \Rightarrow z \overset{\text{def}}{\leftrightarrow} \exists A \in N \; \exists x_1, x_2 \in (\Sigma \cup N)^* \; (x = x_1 A x_2 \text{ и има правило } A \to \alpha \text{ и } z = x_1 \alpha x_2)$$

Релацията $\overset{*}{\Rightarrow}$ е рефлексивното и транзитивно затваряне на $\Rightarrow$. Ако $x \overset{*}{\Rightarrow} y$, казваме, че има *деривация от* x *до* y *в* Γ. За всеки $y \in \Sigma^*$ казваме, че y *има деривация в* Γ, ако $S \overset{*}{\Rightarrow} y$.

КСГ е в Нормална Форма на Чомски (пишем кратко КСГНФЧ), ако всички правила са от вида $A \to BC$ или $A \to a$ или $S \to \epsilon$, където A, B и C са нетерминали, a е терминал, а ϵ е празният стринг.

Предложете ефикасен алгоритъм, който по дадена КСГНФЧ $\Gamma = \langle \Sigma, N, S, R \rangle$ и $y \in \Sigma^*$ връща 1, ако y има деривация в Γ, или 0 в противен случай. Дайте съвсем кратка обосновка на коректността и сложността по време на предложения от Вас алгоритъм. Приемете, че размерът на Γ е $O(1)$.

Решение: Ще построим алгоритъм по схемата динамично програмиране. Този алгоритъм е добре известен и се нарича CYK на имената на създателите си Cocke, Younger и Kasami.

Нека $y = y_1 y_2 \cdots y_n$. БОО, нека $n \geqslant 1$. Ако за всеки $y_i \cdots y_j$, където $1 \leqslant i \leqslant j \leqslant n$, намерим множеството $X_{i,j} = \{A \in N \mid A \overset{*}{\Rightarrow} y_i \cdots y_j\}$, то ние сме готови! Отговорът е 1 тстк $S \in X_{1,n}$. Това, което позволява да направим ефикасен алгоритъм е, че при $i < j$ можем да пресмятаме ефикасно всяко $X_{i,j}$ от множествата $X_{i',j'}$, където $i \leqslant i' \leqslant j' \leqslant j$ и $j' - i' < j - i$.

Базата на рекурсията са множествата $X_{i,i}$ за $1 \leqslant i \leqslant n$:

$$\forall i \in \{1, 2, \ldots, n\} : X_{i,i} = \{A \in N \mid \text{има правило } A \to y_i\}$$

защото в КСГНФЧ, ако по време на деривацията в сентенциалната форма се появи нетерминал, той никога не изчезва в смисъл, че в крайния стринг ще има поне един съответстващ на него терминал; ерго, правилата от вида $A \to BC$ никога не могат да стартират деривация на стринг с една буква.

При $i < j$, $X_{i,j}$ е обединението, по всички нетривиални факторизации $(y_i \cdots y_k) \cdot (y_{k+1} \cdots y_j)$ на стринга $y_i \cdots y_j$ (тоест, по всички $k \in \{i, \ldots, j-1\}$) на тези нетерминали, които са левите страни в продукции от вида $A \to BC$, такива че $B \overset{*}{\Rightarrow} y_i \cdots y_k$ и $C \overset{*}{\Rightarrow} y_{k+1} \cdots y_j$. Но тези нетерминали B и C са елементите на множествата $X_{i,k}$ и $X_{k+1,j}$. При изчисляването на $X_{i,j}$ множествата $X_{i,k}$ и $X_{k+1,j}$ са вече известни.

Самият алгоритъм може да се реализира по начин, аналогичен на реализацията на алгоритъма Matrix-Chain Multiplication.

```
DERIVATION(Γ = ⟨Σ, N, S, R⟩, y ∈ Σ*)
 1  n ← |y|
 2  for i ← 1 to n
 3      M[i, i] ← {A | (A → y_i) ∈ R}
 4  for diag ← 2 to n
 5      for i ← 1 to n − diag + 1
 6          j ← i + diag − 1
 7          M[i, j] ← ∅
 8          for k ← i to j − 1
 9              for B ∈ M[i, k]
10                  for C ∈ M[k + 1, j]
11                      for P ∈ R
12                          if P = (A → BC)
13                              M[i, j] ← M[i, j] ∪ {A}
14  if S ∈ M[1, n]
15      return 1
16  else
17      return 0
```

Обосновката на коректността следва от обосновката на рекурсията. Сложността по време е $\Theta(n^3)$, ако граматиката има константна големина – тогава както множествата $M[a, b]$, така и множеството от продукциите, имат големини, ограничени от константа, от което следва, че и трите най-вътрешни цикъла "въртят" само константен брой пъти.

13.5 Задачи за оптимални подмножества

13.5.1 Set Partition

Дадено е мултимножество от цели положителни числа. Във варианта на задача за разпознаване се пита дали има подмултимножество, такова че сумата от числата в него е равна на сумата на числата извън него. Във варианта на оптимизационна задача се търси подмултимножество, такова че сумата от числата в него е равна на сумата на числата извън него.

Тази задача е частен случай на задачата KNAPSACK (вижте Подсекция 13.5.3), но решението ѝ си заслужава да бъде разгледано отделно. Ето я, формално написана, но не чрез мултимножества, а чрез обикновени множества, чиито елементи има размери (*size*, оттам и "s" за името на функцията). Разглеждаме по-лесния вариант на задача за разпознаване.

Изч. Задача: SET PARTITION

пример: Множество $A = \{a_1, a_2, \ldots, a_n\}$. Функция $s : A \to \mathbb{N}^+$.
въпрос: Дали съществува $X \subset A$, такова че $\sum_{a \in X} s(a) = \sum_{a \in A \setminus X} s(a)$?

Да разгледаме $\sum_{a \in A} s(a)$. Очевидно е, че ако $\sum_{a \in A} s(a)$ е нечетно, отговорът е НЕ. Нека $\sum_{a \in A} s(a)$ е четно и нека $B = \frac{1}{2} \sum_{a \in A} s(a)$. Разглеждаме множеството A като наредено с линейна наредба, а именно тази, която идва от имената на елементите: a_1, после a_2, ..., после a_n. Това може да изглежда странно, защото тази наредба е абсолютно произволна, а ние ще конструираме решение по отношение на нея. Всъщност няма нищо странно. Всяка линейна

наредба на A е еднакво добра за конструиране на решение. Иска се да има **някаква** линейна наредба – точно както казва Skiena за същността на схемата Динамично Програмиране. Нека A_i означава множеството $\{a_1, \ldots, a_i\}$, за всяко $i \in \{0, \ldots, n\}$. Очевидно $A_0 = \varnothing$ и $A_n = A$. БОО, в тези разсъждения допускаме, че за всяко $i \in \{1, \ldots, n\}$, $s(a_i) \leqslant B$; в противен случай отговорът би бил директно Не.

Да характеризираме пример[†] на задачата, за който отговорът е Да. Отговорът е Да тстк съществува подмножество на A, такова, че сумата от размерите на елементите му е точно B. Такова подмножество се състои от елементи. *// очевидно – е, и?*
Продължаваме: всяко непразно подмножество на A със сума B се състои от множество със сума $B - p$ и още един елемент с размер p. *// това вече е нещо.*
Продължаваме: всяко $X \subseteq A$ със сума B е подмножество на някое A_i, където $i \in \{1, \ldots, n\}$. При това,

- или $a_i \in X$, в който случай $X \backslash \{a_i\}$ е подмножество на A_{i-1} със сума $B - s(a_i)$,
- или $a_i \notin X$, в който случай X е подмножество на A_{i-1}. *// аха, това е!*

За това характеризиране на структурата ще направим рекурсивна декомпозиция. За краткост на записа въвеждаме следната нотация. Нека $P(i, j)$, където $1 \leqslant i \leqslant n$ и $0 \leqslant j \leqslant B$, е множество от предикати със следния смисъл:

$$P(i, j) \stackrel{\text{def}}{=} \exists X \subseteq A_i : \sum_{a \in X} s(a) = j$$

Тяхната истинност се определя така.

- Има точно[‡] две подмножества на A_1, чиято сума е число от $\{0, \ldots, B\}$.
  - Празното множество $\varnothing$ със сума 0.
  - Множеството $\{a_1\}$ със сума $s(a_1)$.

  Ерго, $P(1, j)$ е истина за $j = 0$ и за $j = s(a_1)$. За всички други стойности на j, $P(1, j)$ е лъжа.
- За $i > 1$, подмножествата на A_i, чиято сума е число от $\{0, \ldots, B\}$, са:
  - подмножествата на A_{i-1}, чиято сума е число от $\{0, \ldots, B\}$, и
  - подмножествата на A_{i-1}, чиято сума е число от $\{0, \ldots, B - s(a_i)\}$.

  Тези два класа може да имат непразно сечение. Ерго, $P(1, j)$ е истина тстк $P(i - 1, j)$ е истина или $P(i - 1, j - s(a_i))$ е истина и $s(a_i) \leqslant j$. Това "или" е включващо.

Очевидно търсеният отговор е $P(n, B)$.

[†]Това не е оптимизационна задача, така че не говорим за характеризиране на решение. При задачата за най-къси пътища характеризирахме решение, понеже тя е оптимизационна задача. Тук задачана е задача за разпознаване и решенията са само две: Да и Не.

[‡]Тук сме допуснали, че няма елемент a_i, такъв че $s_{a_i} > B$.

Ето схемата за изчисление, базирана на тази рекурсивна декомпозиция.

$$P[1,j] = \begin{cases} 1, & \text{ако } j = 0 \text{ или } j = s(a_1) \\ 0, & \text{в противен случай} \end{cases} \tag{13.15}$$

$$P[i,j] = \begin{cases} 1, & \text{ако } P[i-1,j] \text{ или } (s(a_i) \leqslant j \text{ и } P[i-1,j-s(a_i)]) \\ 0, & \text{в противен случай} \end{cases}, \text{ ако } i > 1 \tag{13.16}$$

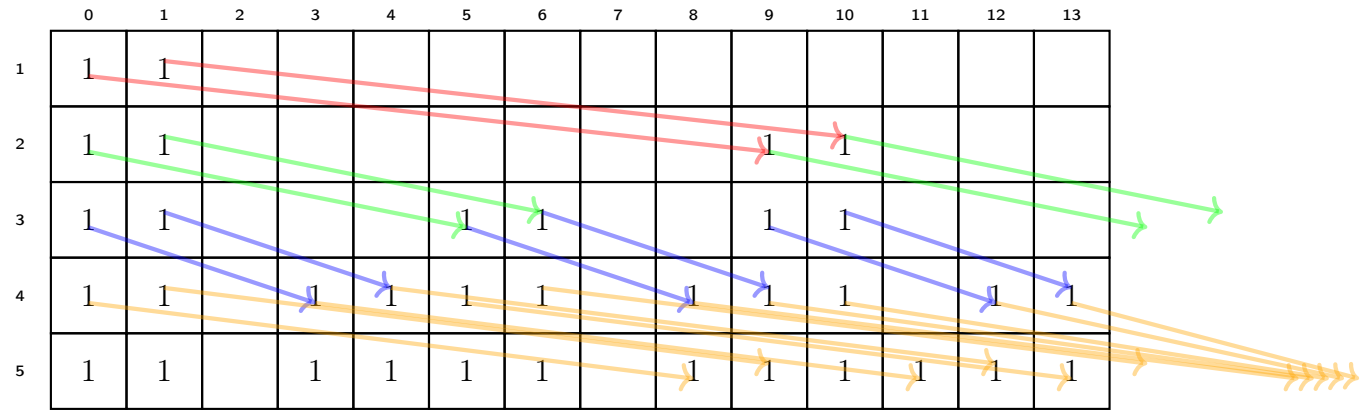
Ето алгоритъм, който извършва тези изчисления и е по схемата Динамично Програмиране.

```
SET PARTITION(A = {a_1, ..., a_n}, s : A → N^+)
 1  if isodd (∑_{i=1}^{n} s(a_i))
 2      return 0
 3  B ← (1/2) ∑_{i=1}^{n} s(a_i)
 4  if max {s(a_i) | 1 ⩽ i ⩽ n} > B
 5      return 0
 6  for j ← 0 to B
 7      if j = 0 or j = s(a_1)
 8          P[1,j] ← 1
 9      else
10          P[1,j] ← 0
11  for i ← 2 to n
12      for j ← 0 to B
13          P[i,j] ← P[i-1,j] or (s(a_i) ⩽ j and P[i-1,j-s(a_i)])
14  return P[n,B]
```

Ето пример за работата на алгоритъма. Нека $n = 5$ и $s(a_1) = 1$, $s(a_2) = 9$, $s(a_3) = 5$, $s(a_4) = 3$ и $s(a_5) = 8$. Тогава сумата от всички размери е $1 + 9 + 5 + 3 + 8 = 26$. Това е четно число и половината от него е 13. Нито един от размерите не надхвърля 13, така че продължаваме с алгоритъма. Ето таблицата, която той конструира. Това е само финалното състояние. Читателят лесто може да види как точно се запълва таблицата, при положение, че знае, че тя се запълва отгоре надолу, ред по ред. В първия ред има единици на позиции 0 (заради празното множество) и 1 (понеже $s(a_1) = 1$). Ако веднъж се появи единица в таблицата на позиция $[i,j]$:

- във всеки следващ ред в колона j има единица, тоест, тя се преписва надолу;
- във всеки следващ ред в колона $j + s(a_i)$ има единица, стига $j + s(a_i)$ да не надхвърля B.

Показани са само единиците.



Цветните стрелки показват как се "пренасят" единици от ред $\mathtt{i}-1$ на ред $\mathtt{i}$ с отместване $\mathtt{s}(\mathtt{a}_\mathtt{i})$. Тъй като $\mathtt{s}(\mathtt{a}_2)=9$, това отместване при прехода от ред 1 към ред 2 е 9, така че двете единици в $\mathtt{P}[1,0]$ и $\mathtt{P}[1,1]$ пораждат съответно единиците в $\mathtt{P}[2,9]$ и $\mathtt{P}[2,10]$. Двете единици в $\mathtt{P}[1,0]$ и $\mathtt{P}[1,1]$ пораждат също така и единиците в $\mathtt{P}[2,0]$ и $\mathtt{P}[2,1]$, но не чрез отместването 9, а чрез просто преписване надолу. Стрелките, сочещи извън таблицата, са за да напомнят на читателя, че чрез тях също можеше да има "пораждане" на единици, ако таблицата беше достатъчно дълга.

Коректността на алгоритъма е очевидна предвид (13.15) и (13.16). Интерес представлява изследването на сложността му по време. Тя е $\Theta(\mathtt{n}\mathtt{B})$. Множителят $\mathtt{n}$ е размера на входа, при допускането, че всяко от числата $\mathtt{s}(\mathtt{a}_\mathtt{i})$ има размер $\Theta(1)$. Но $\mathtt{B}$ какво е? $\mathtt{B}$ е полусумата от **големините** на числата. Тези големини може да са произволни по отношение на $\mathtt{n}$ (с други думи, не са ограничени отгоре от каквато и да е функция на $\mathtt{n}$), така че и $\mathtt{B}$ не е ограничено отгоре. Излиза, че сложността не е дефинирана (с други думи, е безкрайност). Разбира се, този парадоксален извод се появява само при допускането на нашия изчислителен модел, че числата във входа имат константни размери за всички големини; тоест, когато не отчитаме кодиранията им. В по-реалистичен модел на изчисление, който отчита и кодиранията на числата, сложността $\Theta(\mathtt{n}\mathtt{B})$ е експоненциална в големината на входа.

Накратко, в изчислителния модел, който типично използваме, Set Partition има недефинирана (безкрайна) сложност, а в по-реалистичен модел има експоненциална сложност.

Това не прави алгоритъма безполезен. Ако числата са малки, в смисъл, че имат кодирания, чиито големини можем да смятаме за константи, алгоритъмът е полезен и смислен. Забележете, че Set Partition има "истинска" полиномиална сложност по време, ако числата $\mathtt{s}(\mathtt{a}_\mathtt{i})$ са кодирани в унарна бройна система! Това не прави алгоритъмът по-бърз, дори напротив, но кодирането в унарна бройна система изкуствено "раздува" големината на входа, а ние мерим сложността като функция именно на големината на входа. Следното определение се основава на този факт.

Определение 80: Псевдополиномиална сложност

Казваме, че алгоритъм $\mathtt{A}$ има *псевдополиномиална сложност* по време, ако $\mathtt{A}$ има полиномиална сложност по време при условие, че числата във входа са кодирани в унарна бройна система.

И така, Set Partition има псевдополиномиална сложност по време.

Допълнение 53: Още за псевдополиномиалната сложност

Да разгледаме следния фрагмент от алгоритъм.

```
1  for i ← 1 to n
2      { някакъв код, работещ в константно време }
```

Каква е сложността на този фрагмент като функция на n? Отговорът е $\Theta(n)$, защото има n итерации и всяка от тях се извършва в $\Theta(1)$. А каква е сложността на целия алгоритъм, ако допуснем, че основната работа се върши от този **for**-цикъл, а извън него се върши само константна допълнителна работа? Човек може да се изкуши да каже, че пак е $\Theta(n)$, но сега това не е верен отговор. Сложността на алгоритъма е **функция от големината на входа**. А какъв е входът? Не е ясно, никъде не е казано. Докато не се знае какъв е входът, въпросът е безсмислен.

Ако входът е масив $A[1, \ldots, n]$:

```
Alg1(A[1,...,n]: цели числа)
1  s ← 0
2  for i ← 1 to n
3      s ← s + A[i]
4  return s
```

то сложността на целия алгоритъм Alg1 е $\Theta(n)$. Ако обаче входът е числото n:

```
Alg2(n ∈ ℕ⁺)
1  s ← 0
2  for i ← 1 to n
3      s ← s++
4  return s
```

то въпросът за сложността (на Alg2) става триков. Дискусиите в Подсекция 2.2.1 Подсекция 2.2.5 са напълно релевантни и тук.

- Ако не отчитаме дължината на кодирането на n, размерът на входа е единица и сложността е безкрайност.
- Ако отчитаме дължината на кодирането на n и n е кодирано в двоична позиционна бройна система, или в която и да е друга "смислена" бройна система, Alg2 е експоненциален алгоритъм.
- Ако отчитаме дължината на кодирането на n и n е кодирано в унарна позиционна бройна система, Alg2 е линеен алгоритъм.

Забележете, че броят стъпки, които изпълнява Alg2(n), по никакъв начин не намалява, ако n е записано в унарна бройна система, спрямо случая, в който n е записано в бинарна бройна система. Така че квалификациите "експоненциален" и "линеен" може да са подвеждащи. Разликата между тези два случая е *аршинът*, с който мерим! – а не броят на стъпките, които се изпълняват. Ако n е записано унарно, то аршинът, с който

мерим, е огромен, и, измерено с него, броят на стъпките е линеен. Ако n е записано бинарно, то аршинът е малък и, измерено с него, броят на стъпките е експоненциален. Забележете също така, че при ALG1($A[1,\ldots,n]$) можем да мислим, че n-те числа на масива A представят n в унарна бройна система. Наистина, и ALG1($A[1,\ldots,n]$), и ALG2(n), ако n е записано унарно, са линейни алгоритми.

13.5.2 2 Equal Sum Subsets

Тази задача е обобщение на SET PARTITION. Сега се пита дали има две непразни подмножества на A, които имат една и съща сума от размерите на елементите. БОО, тези множества имат празно сечение, защото наличието на общи елементи в тях е без значение за въпроса, на който трябва да се отговори. Накратко ще наричаме задачата (и алгоритъма) "2ESS".

Изч. Задача: 2ESS

пример: Множество $A = \{a_1, a_2, \ldots, a_n\}$. Функция $s : A \to \mathbb{N}^+$.
въпрос: Дали съществуват непразни $X, Y \subset A$, такива че $X \cap Y = \varnothing$ и $\sum_{a\in X} s(a) = \sum_{a\in Y} s(a)$?

Задачата е **NP**-пълна [97]. 2ESS е обобщение на SET PARTITION, в смисъл, че в SET PARTITION се иска и $X \cup Y = A$. Оттук насетне нека $B = \sum_{a\in A} s(a)$.

Наблюдение 54

Ако $B < 2^n - 1$, то отговорът е тривиално ДА по принципа на Dirichlet: броят на непразните подмножества на A е точно $2^n - 1$ и ако възможните стойности за сума на подмножество са по-малко, съществуват различни $C, D \subseteq A$, такива че $\sum_{a\in C} s(a) = \sum_{a\in D} s(a)$. Ако $C \cap D \neq \varnothing$, то $C' = C\backslash D$ и $D' = D\backslash C$ са непразни подмножества на A с празно сечение и $\sum_{a\in C'} s(a) = \sum_{a\in D'} s(a)$.
Това обаче е неконструктивен резултат. Ние **знаем**, че има такива подмножества C и D, но нямаме представа кои са или как да ги намерим ефикасно.

Въпреки че 2ESS е обобщение на SET PARTITION, алгоритъмът за 2ESS, който ще конструираме, не решава SET PARTITION.

Нека $\beta = \left\lfloor \frac{1}{2}B \right\rfloor$. Нека $A_i = \{a_1, \ldots, a_i\}$, за $1 \leqslant i \leqslant n$. Дефинираме $n(\beta+1)^2$ булеви променливи $Q[t,i,j]$, $1 \leqslant t \leqslant n$, $0 \leqslant i,j \leqslant \beta$, които имат следния смисъл:

$$Q[t,i,j] = 1 \text{ тстк } \exists (X,Y) \subseteq A_t \times A_t : \left(X \cap Y = \varnothing \wedge \sum_{a\in X} s(a) = i \wedge \sum_{a\in Y} s(a) = j \right)$$

Отговорът е ДА тстк съществува $k \in \{1,\ldots,\beta\}$, такова че $Q[n,k,k] = 1$.

Рекурсивната декомпозиция е следната. Базата е

$$Q[1,i,j] = \begin{cases} 1, & \text{ако } i = j = 0 \vee (i = s(a_1) \wedge j = 0) \vee (i = 0 \wedge j = s(a_1)) \\ 0, & \text{в противен случай} \end{cases}$$

Обосновката на базата е тривиална. Множеството A_1 е $\{a_1\}$. Има точно три наредени двойки от подмножества на $\{a_1\}$, които имат празно сечение, а именно $(\varnothing, \varnothing)$, $(\{a_1\}, \varnothing)$ и $(\varnothing, \{a_1\})$, със съответни променливи $Q[1,0,0]$, $Q[1, s(a_1), 0]$ и $Q[1, 0, s(a_1)]$. Наистина, в базовия случай променливите имат желания смисъл. ✓

За $2 \leqslant t \leqslant n$, променливите $Q[t, *, *]$ се пресмятат чрез $Q[t-1, *, *]$ така:

$$Q[t,i,j] = Q[t-1,i,j] \vee (i \geqslant s(a_t) \wedge Q[t-1, i-s(a_t), j]) \vee (j \geqslant s(a_t) \wedge Q[t-1, i, j-s(a_t)])$$

Да обосновем същинската част на рекурсивната декомпозиция. За всяка наредена двойка (X, Y), където X и Y са подмножества с непразно сечение на A_t, е вярно точно едно от следните две твърдения.

- Нито X, нито Y съдържат a_t. Тогава X и Y са подмножества на A_t, такива че $X \cap Y = \varnothing$. При допускането, че

Очевидно е как да направим алгоритъм от тази рекурсивна декомпозиция. Нека $n = 4$ и $s(a_1) = 2$, $s(a_2) = 3$, $s(a_3) = 4$ и $s(a_4) = 5$. Тогава $B = 14$ и $\beta = 7$. Наместо да пишем $Q[t,i,j]$, ще пишем $Q^{(t)}[i,j]$. И ще записваме само единиците в матриците $Q^{(t)}$.

За да получим $Q^{(1)}$, слагаме 1 в клетка $[0,0]$ и, тъй като $s(a_1) = 2$, още по една единица в $[0,2]$ и $[2,0]$. Тогава $Q^{(1)}$ е:

	0	1	2	3	4	5	6	7
0	1		1					
1								
2	1							
3								
4								
5								
6								
7								

За да получим $Q^{(2)}$, преписваме единиците от $Q^{(1)}$. После, тъй като $s(a_2) = 3$, слагаме по една единица на разстояние 3, по хоризонтал и вертикал, от досега сложените единици и $Q^{(2)}$ е (новите единици са в синьо):

	0	1	2	3	4	5	6	7
0	1		1	1		1		
1								
2	1			1				
3	1		1					
4								
5	1							
6								
7								

За да получим $Q^{(3)}$, преписваме единиците от $Q^{(2)}$. После, тъй като $s(a_3) = 4$, слагаме по една единица на разстояние 4, по хоризонтал и вертикал, от досега сложените единици и $Q^{(4)}$ е (новите единици са в синьо):

	0	1	2	3	4	5	6	7
0	1		1	1	1	1	1	1
1								
2	1			1	1			1
3	1		1		1		1	
4	1		1	1		1		
5	1				1			
6	1			1				
7	1		1					

Аналотично получаваме $Q^{(4)}$, като имаме предвид, че $s(a_4) = 5$:

	0	1	2	3	4	5	6	7
0	1		1	1	1	1	1	1
1								
2	1			1	1	1		1
3	1		1		1	1	1	1
4	1		1	1		1		1
5	1		1		1	1	1	1
6	1			1		1		
7	1		1	1	1	1		1

С червен фон са посочени единиците върху главния диагонал. Отговорът е Да тстк има поне една единица върху главния диагонал, така че за този пример алгоритъмът трябва да върне Да.

13.5.3 Задачата за раницата (Knapsack)

Крадец иска да открадне предмети, като ги сложи в раница. Всеки предмет има положителни тегло и цена, като те не са свързани по никакъв начин. Оптималната кражба би била да задигне всички предмети, но има максимално сумарно тегло за предметите в раницата – при по-голямо сумарно тегло тя се къса. Това максимално сумарно тегло е *капацитетът* на раницата.

В оптимизационната версия на задачата се иска да се намери подмножество от предмети с максимална сумарна цена, чието сумарно тегло не надвишава капацитета на раницата.

Ако предметите не са атомарни, а са неограничено делими, можем да кажем с известна доза условност, че е дадена *континуална* версия на задачата[†]. Примерно, нещата за крадене са златен прах, сребърен прах и бронзов прах. Континуалната версия на задачата е решима тривиално от алчен алгоритъм, който първо пресмята относителната цена на делимите предмети—да кажем, златният прах на килограм е по-скъп от сребърния, който на свой ред е по-скъп от бронзовия на килограм—и после пълни раницата с относително най-скъпото нещо (златен прах в нашия пример), докато не достигне капацитета ѝ или това нещо не се изчерпи. Ако относително най-скъпото нещо първо се изчерпи, алчният алгоритъм продължава да пълни раницата с второто относително най-скъпо нещо, и така нататък. Очевидно този алгоритъм е ефикасен и намира оптимално решение.

Ако предметите са атомарни, да кажем, че е дадена *дискретна* версия на задачата. В дискретната версия алчният подход може да доведе до решение, което не е оптимално –

[†]Още я наричат Fractional Knapsack.

вземайки предмет с най-високо отношение цена/тегло, може да се лишим от възможност да сложим други предмети, които биха били по-изгодни като цяло, въпреки че поотделно имат по-лошо отношение цена/тегло. С други думи, локалната, или късогледата, алчност сега не е печеливша стратегия; сега трябва да сме "глобално алчни", за да напълним раницата с максимално сумарно скъпи неща, без да я скъсаме.

Ето пример за това, че въпросната алчна идея не работи. Нека са дадени три предмета с цени 7, 5 и 5 и съответни тегла 6, 5 и 5, а капацитетът на раницата е 10. Най-добро отношение цена към тегло има първият предмет: $7/6 \approx 1.17$. За втория и третия предмет отношението цена тегло е $5/5 = 1$. Ерго, алчната идея слага a_1 в раницата, в раницата остава свободен капацитет $10 - 6 = 4$ и това е краят, понеже вторият и третият имат тегло 5 всеки и нито един от тях не може да бъде взет. Печалбата на крадеца е 7. А ако беше взел втория и третия предмет, които заедно се побират в раницата, щеше да е с печалба $5 + 5 = 10$.

Дискретната версия на задачата е широко известна в поне три варианта, които ще разгледаме сега.

13.5.3.1 Unbounded Knapsack

Изч. Задача: Unbounded Knapsack

пример: Множество от видове предмети $A = \{a_1, \ldots, a_n\}$, като има неограничено много предмети от всеки вид. За всеки вид a_i, $1 \leqslant i \leqslant n$, са дадени неговата стойност $v(a_i) \in \mathbb{R}^+$ и неговото тегло $w(a_i) \in \mathbb{N}^+$. Даден е капацитет $C \in \mathbb{N}^+$.
решение: $(x_1, \ldots, x_n) \in \mathbb{N}^n$, такава че

$$\sum_{i=1}^{n} x_i w(a_i) \leqslant C \tag{13.17}$$

$$\sum_{i=1}^{n} x_i v(a_i) \text{ е максимална} \tag{13.18}$$

Лесно можем да преведем това формално описание в терминологията на крадеца с раницата. C е капацитетът на раницата. Предметите за крадене са от n вида, като видовете са a_1, …, a_n, а от всеки вид има неограничено много предмети за крадене. Предметите от даден вид a_i са неразличими помежду си и имат една и съща стойност $v(a_i)$ и едно и също тегло $w(a_i)$. Под "неограничено" нямаме предвид безброй много, а нещо много по-реалистично: по отношение на дадената раница, крадецът може да я напълни с предмети от само един вид до достигане на капацитета, за кой да е вид; с други думи, никога няма да се изчерпят предметите от даден вид, запълвайки раницата. Наредената n-орка от естествени числа $(x_1, \ldots, x_n)$ има смисъл на това, по колко броя от всеки вид ще бъдат взети в раницата. (13.17) и (13.18) казват, че трябва да се вземат по толкова предмети от всеки вид, че общото тегло да не надхвърли капацитета, като тази подборка трябва да има максимална сумарна стойност.

Тук ще разгледаме олекотената версия на задачата, в която изчисляваме само максималното количество $\sum_{i=1}^{n} x_i v(a_i)$, без да изчисляваме n-торката $(x_1, \ldots, x_n)$, която казва по колко предмети от всеки вид да вземем.

Следното решение на задачата е добре известно. Правим рекурсивна декомпозиция по капацитета на раницата. Нека $M[j]$ е максималната стойност на предмети, които можем да сложим в раница с капацитет j, за всички $j \in \{0, \ldots, C\}$. Ако можем да пресмятаме $M[j]$ чрез $M[j-1]$, …, $M[0]$, ще получим решението $M[C]$. Схемата за изчисление е следната.

$$M[j] = \begin{cases} 0, & \text{ако } j = 0 \\ \max\left(M[j-1], \max_{i \in \{1,\ldots,n\} \text{ и } w(a_i) \leqslant j} \left(M[j - w(a_i)] + v(a_i)\right)\right), & \text{ако } j > 0 \end{cases}$$

Идеята е ясна. Ако раницата има нулев капацитет, в нея не можем да сложим нищо. Ако позволим капацитет $j > 0$, в нея можем да сложим най-много:

- или това, което можем да сложим при по-малкия капацитет $j - 1$, с други думи не се възползваме от целия капацитет j,
- или, възползвайки от целия капацитет j, опитваме последователно да сложим един предмет a_i от всеки вид, чието тегло не надхвърля j, слагаме този a_i, който максимизира сумарната цена. Но, за да сложим предмет от вид a_i в раница с капацитет j, трябва да има свободен капацитет $j - w(a_i)$. Максимумът, който можем да постигнем, слагайки предмет от вид a_i, е максимумът за раница с капацитет $j - w(a_i)$ плюс стойността $v(a_i)$ на предмета.

Ако искаме пълно решение на задачата, което ни дава не просто числен отгово, а освен това и по колко предмети от всеки вид да сложим в раницата, можем да записваме за всяко j, $1 \leqslant j \leqslant C$, дали $M[j]$ е получено като $M[j-1]$ или по другия начин. Ако е по другия начин, а именно като максимум по всички i, такива че $w(a_i) \leqslant j$, записваме i, за което се постига максимална стойност на $M[j - w(a_i)] + v(a_i)$. От тази информация може лесно да генерираме пълно решение във време $O(C)$.

Сложността по време е $\Theta(nC)$, а по памет е $\Theta(C)$. Това е поредният псевдополиомиален алгоритъм, който разглеждаме. Ако числата $w(a_i)$ са малки, масивът M е малък и алгоритъмът е ефикасен.

13.5.3.2 0-1 Knapsack

Изч. Задача: 0-1 Knapsack

пример: Множество от предмети $A = \{a_1, \ldots, a_n\}$. За всяко a_i, $1 \leqslant i \leqslant n$, са дадени неговата стойност $v(a_i) \in \mathbb{R}^+$ и неговото тегло $w(a_i) \in \mathbb{N}^+$. Даден е капацитет $C \in \mathbb{N}^+$.
решение: Подмножество $X \subseteq A$, такова че:

$$\sum_{a \in X} w(a) \leqslant C$$

$$\sum_{a \in X} v(a) \text{ е максимална}$$

Бихме могли да дефинираме и тази версия на задачата чрез вектор $(x_1, \ldots, x_n)$, само че булев, а не от естествени числа. Това би бил характеристичен вектор, а знаем, че характеристичните вектори определят подмножества. Така че дори с такава формулировка, това, което се има предвид, е подмножество с максимална обща стойност и сумарен размер, не по-голям от капацитета.

В тази версия на задачата—също както и в предишната—не е добра идея да правим рекурсивна декомпозиция само по подмножествата от първите i елемента на A (има се предвид $\{a_1, \ldots, a_i\}$). Читателят лесно може да измисли пример, в който оптимално решение за, да

кажем, $\{a_1, \ldots, a_6\}$ не ползва оптимално решение за $\{a_1, \ldots, a_5\}$. Ще направим двумерна рекурсивна декомпозиция, подобна на решението на Set Partition. Нека $M[i, j]$ е (стойността на) оптимално решение за раница с капацитет j, ползващо предмети $a_1, \ldots, a_i$, където $0 \leqslant i \leqslant n$ и $0 \leqslant j \leqslant C$. Решението е $M[n, C]$. Схемата за изчисление е следната.

$$
\begin{aligned}
&M[i, 0] = 0, \text{ за } 1 \leqslant i \leqslant n \quad // \text{ нулев капацитет} \\
&M[0, j] = 0, \text{ за } 1 \leqslant j \leqslant C \quad // \text{ нула предмети за слагане} \\
&M[i, j] = M[i-1, j], \text{ ако } i > 0 \text{ и } j > 0 \text{ и } w(a_i) > j \quad // \ a_i \text{ не са побира при капацитет } j \\
&M[i, j] = \max(\underbrace{M[i-1, j]}_{\text{не вземаме } a_i}, \underbrace{M[i-1, j-w(a_i)] + v(a_i)}_{\text{вземаме } a_i}), \text{ в противен случай} \qquad (13.19)
\end{aligned}
$$

Случай (13.19) е основата на декомпозицията и може би единственият, чиято обосновка не е напълно очевидна. Имаме две възможности.

- Да не вземем предмета a_i. Стойността на такова решение ни е известна и тя е $M[i-1, j]$.
- Да вземем предмета a_i. При това обаче трябва да има капацитет за него $w(a_i)$, така че гледаме колко най-много можем да сложим, оставяйки си свободен капацитет $w(a_i)$, и какво ще получим при това. Тази стойност се пресмята тривиално и тя е $M[i-1, j-w(a_i)] + v(a_i)$.

Избираме по-добрата от двете възможности, тоест, максимума.

Алгоритъм, реализиращ това изчисление, е следният.

```
0-1 Knapsack(A = {a_1, ..., a_n}, v : A → R^+, w : A → N^+, C ∈ N^+)
 1  for i ← 0 to n
 2      M[i, 0] ← 0
 3  for j ← 0 to C
 4      M[0, j] ← 0
 5  for i ← 1 to n
 6      for j ← 1 to C
 7          if w(a_i) > j
 8              M[i, j] ← M[i − 1, j]
 9          else
10              M[i, j] ← max(M[i − 1, j], M[i − 1, j − w(a_i)] + v(a_i))
11  return M[n, C]
```

Коректността на алгоритъма е очевидна. Сложността както по време, така и по памет, е $\Theta(nC)$.

13.5.3.3 Bounded Knapsack

Сега са дадени определен брой копия от всеки предмет. Тази версия на задачата е обобщение на 0-1 Knapsack, където беше дадено точно по един предмет от всеки вид. Тук ще разгледаме решение от книгата на Martello и Toth [67].

Изч. Задача: Bounded Knapsack

пример: Множество от видове предмети $A = \{a_1, \ldots, a_n\}$. За всеки вид a_i, $1 \leqslant i \leqslant n$, са дадени неговата стойност $v(a_i) \in \mathbb{R}^+$, неговото тегло $w(a_i) \in \mathbb{N}^+$ и броят предмети от него $b_i \in \mathbb{N}^+$. Даден е капацитет $C \in \mathbb{N}^+$.
решение: $(x_1, \ldots, x_n) \in \mathbb{N}^n$, такава че

$$\sum_{i=1}^{n} x_i w(a_i) \leqslant C \tag{13.20}$$

$$0 \leqslant x_i \leqslant b_i, \text{ за } 1 \leqslant i \leqslant n \tag{13.21}$$

$$\sum_{i=1}^{n} x_i v(a_i) \text{ е максимална} \tag{13.22}$$

БОО, нека $\sum_{i=1}^{n} b_i w(a_i) > C$, защото в противен случай имаме тривиално решение $x_i = b_i$ за $1 \leqslant i \leqslant n$.

Задачата се свежда до 0-1 Knapsack със следната трансформация. Нека е даден пример Π_B на Bounded Knapsack:

$$\langle \{a_1, \ldots, a_n\}, (v_1, \ldots, v_n), (w_1, \ldots, w_n), (b_1, \ldots, b_n), C \rangle$$

На него съответства пример $\Pi_{0,1}$ на 0-1 Knapsack:

$$\langle \{\widehat{a}_1, \ldots, \widehat{a}_m\}, (\widehat{v}_1, \ldots, \widehat{v}_m), (\widehat{w}_1, \ldots, \widehat{w}_m), C \rangle$$

За всеки вид a_i, $1 \leqslant i \leqslant n$, конструираме $\lceil \log_2(b_i + 1) \rceil$ предмети в $\Pi_{0,1}$ така:

- първо, $\lfloor \log_2 b_i \rfloor$ предмети, чиито наредени двойки (стойност, тегло) са

  $$(v_i, w_i), (2v_i, 2w_i), (4v_i, 4w_i), \ldots, (\lfloor \log_2 b_i \rfloor v_i, \lfloor \log_2 b_i \rfloor w_i)$$

- второ, ако b_i не е число от вида "точна степен на двойката минус единица", още един предмет със стойност

  $$b_i v_i - (v_i + 2v_i + 4v_i + \cdots \lfloor \log_2 b_i \rfloor v_i)$$

  и размер

  $$b_i w_i - (w_i + 2w_i + 4w_i + \cdots + \lfloor \log_2 b_i \rfloor w_i)$$

Очевидно броят на предметите в $\Pi_{0,1}$, съответни на a_i, е $\lceil \log_2(b_i + 1) \rceil$, като сумата от стойностите им е $b_i v_i$ и сумата от размерите им е $b_i w_i$. Общо броят на предметите в $\Pi_{0,1}$ е $\sum_{i=1}^{n} \lceil \log_2(b_i + 1) \rceil$. Накратко ще записваме това число като m

m е и броят на булевите променливи в решението на $\Pi_{0,1}$. Ерго, на всяка променлива-естествено число x_i от решението Π_B съответстват булеви променливи $\widehat{x}_{i,1}, \ldots, \widehat{x}_{i,\lceil \log_2(b_i+1) \rceil}$ от решението на $\Pi_{0,1}$. Нещо повече, за $1 \leqslant h \leqslant \lceil \log_2(b_i + 1) \rceil$, булевата променлива $\widehat{x}_{i,h}$ "дава" n_h предмети от Π_B, където

$$n_h = \begin{cases} 2^{h-1}, & \text{ако } h < \lceil \log_2(b_i + 1) \rceil \\ b_i - \sum_{j=1}^{\lceil \log_2(b_i+1) \rceil - 1} 2^{j-1}, & \text{ако } h = \lceil \log_2(b_i + 1) \rceil \end{cases}$$

Следователно $x_i = \sum_{h=1}^{\lceil \log_2(b_i+1) \rceil} n_h \hat{x}_{i,h}$ може да приеме всяка стойност от $\{0, \ldots, b_i\}$.

Сложността на алгоритъма е сложността на трансформацията плюс сложността на 0-1 KNAPSACK върху конструирания пример. Сложността на трансформацията е $\Theta(m)$, а след това 0-1 KNAPSACK работи във време $\Theta(mC)$, така че общата сложност е $\Theta(mC)$. Да се направи подробно формално доказателство на коректността би било доста дълго, така че ще разгледаме примера за трансформацията от [67, стр. 83].

Нека е даден пример на BOUNDED KNAPSACK, в който $n = 3$.

	a_1	a_2	a_3
v	10	15	11
w	1	3	5
b	6	4	2

На око се вижда, че оптималното решение е $(x_1, x_2, x_3) = (6, 1, 0)$ с обща стойност

$$6 \cdot 10 + 1 \cdot 15 + 0 \cdot 11 = 75$$

и общо тегло

$$6 \cdot 1 + 1 \cdot 3 + 0 \cdot 5 = 9$$

Да приложим трансформацията. Имаме $\lceil \log_2(a_1+1) \rceil = 3$, $\lceil \log_2(a_2+1) \rceil = 3$ и $\lceil \log_2(a_3+1) \rceil = 2$. Нека трите предмета, съответстващи на a_1, са a_1', a_1'' и a_1''', трите предмета, съответстващи на a_2, са a_2', a_2'' и a_2''' и двата предмета, съответстващи на a_3, са a_3' и a_3''. Това са осемте предмета в примера на 0-1 KNAPSACK, който строим. Стойностите и теглата са следните.

	a_1'	a_1''	a_1'''	a_2'	a_2''	a_2'''	a_3'	a_3''
v	10	20	30	15	30	15	11	11
w	1	2	3	3	6	3	5	5

$a_1' = 1 \cdot 10 = 10$, $a_1'' = 2 \cdot 10 = 20$, а $a_1''' = b(a_1) \cdot v(a_1) - (a_1' + a_1'') = 60 - 30 = 30$. Останалите стойности и теглата се получават аналогично. Ако пуснем този пример на 0-1 KNAPSACK в knapsack solver, получаваме оптимално решение 75 при

$$(x_1', x_1'', x_1''', x_2', x_2'', x_2''', x_3', x_3'') = (11110000)$$

Имената на променливите x съответстват по очевиден начин на предметите. Решението (11110000) означава вземане на a_1', a_1'' и a_1''', тоест $1 + 2 + 3 = 6$ предмета от вида a_1, и a_2', тоест 1 предмет от вида a_2: точно както в решението на око.

13.6 Задачи за планиране (Scheduling)

При тези задачи са дадени някакви дейности, всяка от които се характеризира с начало, край и може би тегло, и се иска да се намери оптимално подмножество или оптимално разбиване на множеството от дейностите.

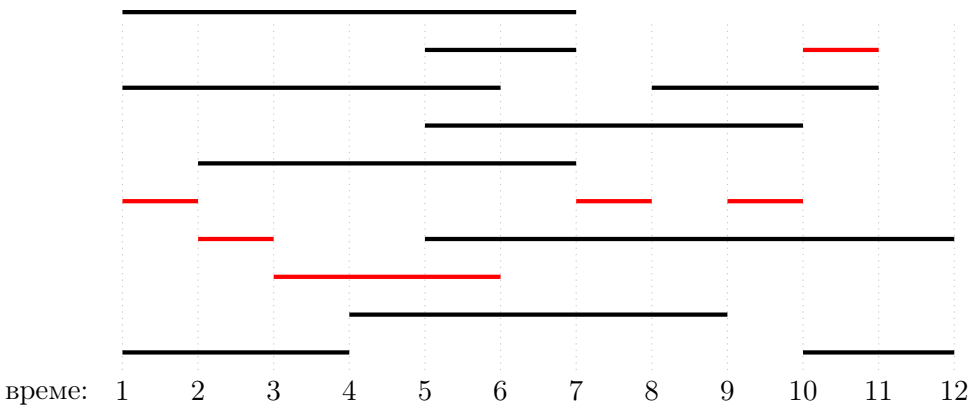
13.6.1 Задачата за безработния актьор (Interval Scheduling)

Тази задача е известна още под името "ACTIVITY-SELECTION" [21, стр. 415]. Дадено е множество отворени интервали $A = \{a_1, \ldots, a_n\}$, като $a_i = (s_i, f_i)$ и $s_i < f_i$, за $1 \leqslant i \leqslant n$. Ако мислим за интервалите като за дейности във времето, дейност a_i се характеризира с начално време s_i и крайно време f_i. Два различни интервала a_i и a_j са *съвместими*, ако имат празно сечение; тоест, ако $f_i < s_j$ или $f_j < s_i$. В противен случай те са *несъвместими*. Казваме, че подмножество от интервали е *безконфликтно*, ако всеки два интервала от него са съвместими. Забележете, че при $f_i = s_j$ интервалите a_i и a_j имат празно сечение, тъй като са отворени. Също така забележете, че напълно съвпадащи—и като начално, и като крайно време—интервали не може да има, защото идентифицираме всеки интервал с наредената двойка от началното му и крайното му време, а A е множество.

Задачата е, по дадено множество от интервали A, да се намери безконфликтно подмножество $A' \subseteq A$ с максимална мощност. Тази задача още се нарича и "задачата за безработния актьор" – актьор е без работа и получава възможност да се снима в n клипа, които са интервалите. Всеки клип има начално и крайно време, които са дадени, и заплащането за всеки клип е едно и също. Задачата на актьора е да избере максимален брой клипове, които не се "застъпват" два по два във времето, защото в даден момент той може да се снима в най-много един клип, и по този начин да получи максимално заплащане. Иначе казано, търси се максимално по мощност безконфликтно подмножество.

Още се нарича "задачата за безработния актьор", защото може да мислим, че има актьор, който си търси работа и иска да се снима в клипове, като интервалите са клиповете – снимането на всеки клип има начало и край, които задават съответния интервал. Акгьорът не може просто да бъде ангажиран във всички клипове, защото в общия случай клиповете се застъпват взаимно във времето, а в даден момент той не може да е ангажиран в повече от един клип. Ако всички клипове дават едно и също възнаграждение, актьорът би искал да намери колкото може повече взаимно незастъпващи се клипове. Ако всеки клип си има свое възнаграждение, актьорът би искал да намери подмножество клипове, взаимно незастъпващи се, за които сумарното възнаграждение е максимално. Това са съответно нетегловната и тегловната версия на задачата.

Ето пример с шестнадесет интервала. Оптимално решение е подмножеството от шестте интервала, оцветени в червено. Очевидно то е безконфликтно. Ако не е очевидно, че няма по-голямо безконфликтно множество, от изложението в Подсекция 13.6.1.1 ще стане ясно, че това наистина е така (че няма по-голямо безконфликтно множество).



13.6.1.1 Алчно решение

Ще разгледаме четири идеи, всяка от които може да е в основата на алчен алгоритъм.

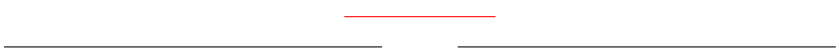
① ПЪРВИЯТ НАЛИЧЕН ИНТЕРВАЛ Актьорът взема клипа, чието начало е най-рано. След това всички клипове, които са засичат с този клип, отпадат като възможности. Измежду останалите клипове отново взема този с най-ранно начално време, и така нататък.

Тази идея не работи в смисъл, че има примери, в които дава неоптимални решения. Ето контрапример с илюстрация. Вземайки първата започваща работа (червения интервал), той ще изтърве четири други, а всички тях заплащането е едно и също.



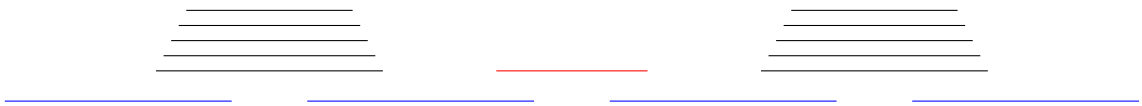
② ПЪРВО НАЙ-КЪСИЯТ ИНТЕРВАЛ Актьорът взема клипа, който е най-къс. След това всички клипове, които са засичат с този клип, отпадат като възможности. Измежду останалите клипове отново взема този, който е най-къс, и така нататък.

Тази идея също не работи. Ето контрапример с илюстрация. Вземайки най-късия във времето ангажимент (червения интервал), той ще изтърве два други.



③ ПЪРВО НАЙ-МАЛКО КОНФЛИКТИ Актьорът взема клипа, за който има най-малко други клипове, с които се засича. После тези, с които се засича, отпадат като възможности. Измежду останалите клипове отново взема този, който се засича с най-малко клипове, и така нататък.

Тази идея също не работи. Ето контрапример с илюстрация. Вземайки клипа с най-малко засичания (червения интервал; той се засича само с два други клипа), той губи възможността да вземе оптималното решение от четири клипа (в синьо).



④ НАЙ-РАННО ПРИКЛЮЧВАНЕ Актьорът взема клипа, който приключва най-рано. После тези, с които се засича, отпадат като възможности. Измежду останалите клипове отново взема този, който приключва най-рано, и така нататък.

Тази идея работи. Читателят лесно може да се убеди, че, следвайки тази идея, в примера от ① актьорът ще вземе четири клипа, в примера от ② актьорът ще вземе два клипа и в примера от ③ актьорът ще вземе четири клипа, като всички тези решения са оптимални. Но това, че тази идея работи, трябва да се докаже формално.

Теорема 68: НАЙ-РАННО ПРИКЛЮЧВАНЕ е оптимално за INTERVAL SCHEDULING

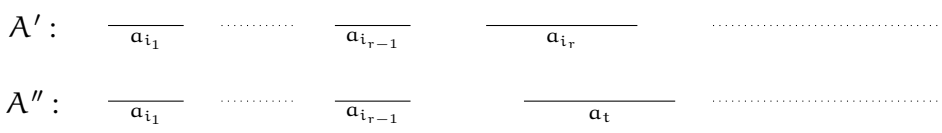
За всеки пример A на INTERVAL SCHEDULING, подмножеството $A' \subseteq A$, конструирано съгласно стратегията НАЙ-РАННО ПРИКЛЮЧВАНЕ, е максимално множество от два по два съвместими интервали.

Доказателство: Това, че стратегията НАЙ-РАННО ПРИКЛЮЧВАНЕ конструира подмножество от интервали, между които няма несъвместима двойка, е очевидно. Ще докажем,

че конструираното множество е **максимално** множество от два по два съвместими интервали. Да допуснем противното. Нека $A = \{a_1, \ldots, a_n\}$ е контрапример. Да си представим работата на алчен алгоритъм AlgIS за Interval Scheduling, изграден върху стратегията най-ранно приключване, върху вход A. Изходът A' не е максимално по мощност безконфликтно множество, съгласно допускането. Нека $A' = \{a_{i_1}, \ldots, a_{i_k}\}$, където $\{i_1, \ldots, i_k\} \subset \{1, \ldots, n\}$, и БОО нека интервалите "влизат" в A' точно в този ред: първо a_{i_1}, после a_{i_2}, и така нататък, накрая a_{i_k}.

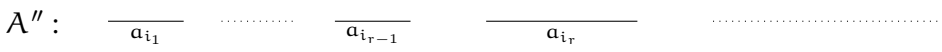
Да съобразим, че, слагайки интервалите по този начин в A', алгоритъмът AlgIS греши с поне един интервал. Какво означава това? По допускане, съществува безконфликтно множество $A'' \subseteq A$, такова че $|A''| > |A'|$. Ще докажем, че е невъзможно $A' \subset A''$. Ако допуснем, че $A' \subset A''$, трябва A'' да се състои от интервалите в A' и още поне един интервал a_m, който е или вляво от най-левия интервал на A', или някъде между интервалите на A', или вдясно от най-десния интервал на A'. И трите случая са невъзможни – лесно се вижда, че AlgIS би сложил и a_m в A'. И така, $A' \subset A''$, което е същото като да има интервал в A', който не се съдържа в A''. За всеки такъв интервал казваме че, че с него AlgIS е *сгрешил*.

Нека a_{i_r} е първият интервал, с който AlgIS е сгрешил. Тогава интервалите, сложени от AlgIS преди a_{i_r}, а именно $a_{i_1}, \ldots, a_{i_{r-1}}$, се намират и в A''. Нека интервалът в A'', който се намира след $a_{i_{r-1}}$, се нарича a_t, като очевидно $t \neq i_r$. Ключовото наблюдение е, че краят f_{i_r} на a_{i_r} се намира вляво от края f_t на a_t или съвпада с него, но не може да е вдясно от a_t. Това е напълно очевидно предвид факта, че AlgIS е изграден върху стратегията най-ранно приключване. Следната фигура илюстрира ситуацията:



Интервалите a_{i_r} и a_t може да не са разположени точно така един спрямо друг, но със сигурност $f_{i_r} \leqslant f_t$, в противен случай AlgIS щеше да сложи a_t, а не a_{i_r} в A'; забележете, че както a_{i_r}, така и a_t са съвместими с всеки от $a_{i_1}, \ldots, a_{i_{r-1}}$.

Веднага се вижда, че AlgIS не може да е сгрешил с a_{i_r}, защото, ако заменим a_t с a_{i_r} в A'', то остава валидно решение:



С други думи, A'' няма какво да загуби, получавайки a_{i_r} наместо a_t, тъй като $f_{i_r} \leqslant f_t$. □

Допълнение 54: Алчността на алчните алгоритми: глупава или умна

Нека е дадено някакво крайно непразно множество S. Знаем разликата между "максимално по мощност подмножество на S" и "максимално по включване подмножество на S спрямо свойство π"[a]. В контекста на задачата Interval Scheduling, свойството е безконфликтност. На английски съответните термини са "maximum" и "maximal".

Ако за всяко $S' \subseteq S$, изчисляването на предиката $\pi(S')$ е ефикасно, то има очевиден ефикасен алчен алгоритъм за намиране на максимално по включване подмножество U. Изграждаме решението U итеративно. В началото $U \leftarrow \varnothing$. Докато можем, правим следното: за всеки елемент от $x \in S \backslash U$, в произволен ред, пресмятаме дали $\pi(U \cup \{x\})$ е истина.

- За първия x в този ред, за който $\pi(U \cup \{x\})$ е истина, прекъсваме текущата итерация, правим $U \leftarrow U \cup \{x\}$, и итерираме отново спрямо новото U.
- Ако за никой x не получим истина, спираме итерианията.

Този процес спира след краен брой стъпки и точно едно от тези две е вярно:

- Текущото U е същинско подмножество на S, тоест, $S \backslash U$ е непразно, обаче няма елемент $x \in S \backslash U$, за който $\pi(U \cup \{x\})$ е истина.
- Текущото U съвпада с S.

И в двата случай, U е максимално по включване подмножество на S спрямо свойството π и алгоритъмът връща U.

За задачата INTERVAL SCHEDULING бихме могли да подходим по подобен начин, но полученото решение, макар и максимално по включване, в общия случай не било максимално по мощност. Подходът, за който става дума тук, е алчен, но е **алчен по глупав начин**: той избира нов интервал за добавяне напълно произволно. Алгоритъмът INTERVAL SCHEDULING, който решава задачата, е основан на евристиката НАЙ-РАННО ПРИКЛЮЧВАНЕ. Той също е алчен, но е **алчен по интелигентен начин**: критерият му за избор на нов интервал за добавяне е основан на Теорема 68.

И така, ако искаме да получим решение, което е максимално по включване—тоест, не може да бъде подобрено—можем да използваме глупаво алчния подход. За да получим решение, което е максимално по мощност, този подход (най-често) не работи и, ако има алчно решение, то е алчно по умен начин.

[a] Да си припомним, че π е свойство на подмножества, а не на елементи, на S. Ерго, ако $S' \subseteq S$, то $\pi(S')$ е или лъжа, или истина, но ако $s \in S$, то $\pi(s)$ не е нито лъжа, нито истина, а е невалиден израз.

Следният алгоритъм решава задачата, базирайки се на НАЙ-РАННО ПРИКЛЮЧВАНЕ.

INTERVAL SCHEDULING, GENERAL($A = \{a_1, \ldots, a_n\}$, където $a_i = (s_i, f_i)$ и $s_i < f_i$ за $1 \leqslant i \leqslant n$)
```
1  сортирай A по f-стойностите, резултатът е ⟨a'_1, ..., a'_n⟩
2  A' ← ∅
3  for i ← 1 to n
4      ако няма несъвместимост между a'_i и елемент от A', добави a'_i към A'
5  return A'
```

Ключово за ефикасността на алгоритъма е ефикасността на проверката на ред 4. Няма нужда да проверяваме за съвместимост между a_i, от една страна, и всеки интервал в текущия A', от друга. Напълно достатъчно е да проверяваме само дали a_i е съвместим с интервала, който е добавен последно в A'. Ако a_i е съвместим с него, то той е съвместим с всеки интервал в A' заради транзитивността на релацията $<$. Ако a_i не е съвместим с него, просто го прескачаме. И така, достатъчно е да помним в някаква променлива крайното време f^* на интервала, последно сложен в A', и да сравняваме s_i с f^*. Ето подробностите.

INTERVAL SCHEDULING, DETAILED($A = \{a_1, \ldots, a_n\}$, където $n \geqslant 1$, $a_i = (s_i, f_i)$ и $s_i < f_i$ за $1 \leqslant i \leqslant n$)
```
1  сортирай A по f-стойностите, резултатът е ⟨a'_1, ..., a'_n⟩
```

```
2   A′ ← {a′_1}
3   f* ← f′_1
4   for i ← 1 to n
5       if f* ⩽ s′_i
6           A′ ← A′ ∪ {a′_i}
7           f* ← f′_i
8   return A′
```

Коректността се обосновава с Теорема 68. Сложността по време е $\Theta(n \lg n)$ заради сортирането в началото. а сложността по памет е $\Theta(n)$.

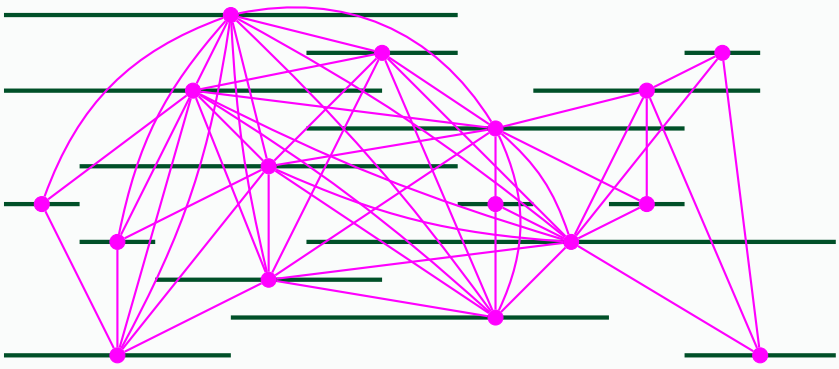
Сложността $\Theta(n \lg n)$ е оптимална. Лесно може да се направи аргумент за долна граница, защото, при определени условия, задачата Сортиране се свежда до Interval Scheduling. Нека разглеждаме само алгоритми за Interval Scheduling, които връщат не само стойността на оптималното решение, а и самото решение. И го връщат не като множество, а като **редица** от интервали, подредени по начина, по който са подредени във времето. Ако имаме такъв алгоритъм, работещ във време $o(n \lg n)$, можем да го ползваме, за да сортираме n на брой, две по две различни, **цели** числа $a_1, \ldots, a_n$ във време $o(n \lg n)$:

- Първо построяваме във време $\Theta(n)$ наредените двойки $(a_1, a_1 + \epsilon), \ldots, (a_n, a_n + \epsilon)$ за някакво достатъчно малко ϵ. Тези наредени двойки представляват интервалите.
- Върху тези интервали пускаме хипотетичния алгоритъм за Interval Scheduling.

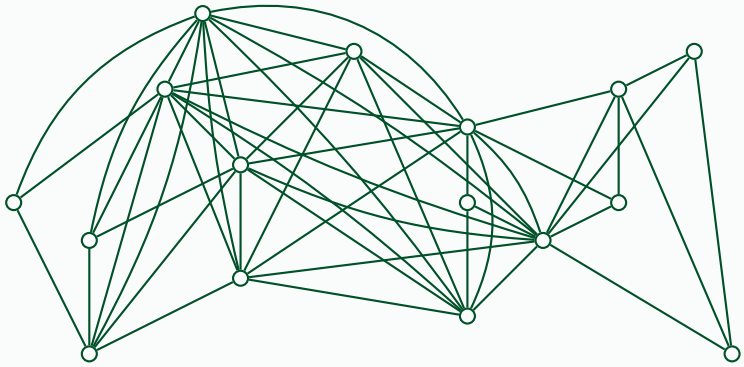
Тъй като цялото множество от тези интервали е безконфликтно, този алгоритъм връща редицата от всички интервали, сортирани по първо число, и оттук намираме сортираната редица от входните числа.

Допълнение 55: Максимална антиклика върху интервални графи

Задачата Максимална Антиклика е дефинирана на стр. 540. Тук ще покажем, че алгоритъмът Interval Scheduling в някакъв смисъл решава Максимална Антиклика. Нека е дадено множество интервали $A = \{a_1, \ldots, a_n\}$, точно както в Interval Scheduling. На всеки интервал a_i съпоставяме връх v_i, и слагаме ребро (v_i, v_j) тстк $i \neq j$ и $a_i \cap a_j \neq \varnothing$. По този начин изграждаме граф, който съответства на A. Като пример да вземем множеството от интервалите на стр. 520 и да си представим въпросния графа, нарисуван върху рисунката на интервалите.



Този граф характеризира напълно застъпванията на интервалите. Нека го разгледаме самостоятелно.



Графите, които характеризират по такъв начин застъпванията в множество от интервали, се наричат интервални графи.

Определение 81: Интервален граф

Граф $G = (V, E)$ се нарича *интервален граф*, ако съществува множество от интервали $\mathcal{I} = \{i_1, \ldots, i_n\}$, такова че съществува биекция $\phi : V \to \mathcal{I}$, такава че за всеки два различни $u, v \in V$, $(u, v) \in E$ тогава и само тогава, когато $\phi(u) \cap \phi(v) = \varnothing$.

На английски понятието е *interval graph*. Интервалните графи не са самоцелно дефинирани. Едно от първите им приложения е в моделирането на задачи от биологията през 1959 г. [33, стр. 171].

Много общо, грубо и непрецизно казано, свързаните интервални графи се състоят от клики, които са "залепени" в линейна наредба; съседни в тази наредба, максимални по включване, клики имат общи подклики. Кликите идват оттам, че, говорейки нагледно, ако вертикална права пресича k интервала, то всеки два от тях се пресичат взаимно и в графа те задават k-клика. В нашия пример има 8 интервала, които се пресичат взаимно, и наистина в графа има съответна 8-клика.

Прецизна алтернативна характеризация е: граф е интервален тогава и само тогава, когато съществува линейна наредба (редица) на максималните по включване клики, такава че в нея всеки връх се появява в непрекъсната подредица от клики [33, Theorem 8.1, стр. 172].

Далече не всеки граф е интервален. Прост пример за граф, който не е интервален, е всеки граф-цикъл с дължина поне 4. Неговите максимални по включване клики са 2-клики и те не са подредени линейно, а кръгово. Всяко дърво с поне един връх от степен поне 3, от който "излизат" поне 3 пътя с дължина поне 2 всеки, също не е интервален. Известни са линейни алгоритми, които разпознават интервалните графи [18].

Да се разгледаме алгоритъма Interval Scheduling в светлината на интервалните графи. Веднага се вижда, че, без да искаме, сме конструирали ефикасен алгоритъм за задачата Максимална Антиклика (дефиницията е на стр. 540); наистина, всяко безконфликтно множество от интервали директно отговаря на антиклика в съответния интервален граф. Добре известно е, че Максимална Антиклика е **NP**-трудна задача (Лекция 14). На пръв поглед това, че разполагаме с много ефикасен алгоритъм за задача, която е **NP**-трудна, е парадоксално. Но всъщност парадокс няма. Максимална Антиклика е **NP**-трудна върху графи **по принцип**. А ние имаме ефикасен алгоритъм за Максимална Антиклика върху **ограничен клас графи**. Структурата на интервалните графи е доста по-ограничена и в някакъв смисъл по-бедна от

структура на общите графи. По-бедната структура на интервалните графи е това, което позволява за тях да има ефикасен алгоритъм. Както ще видим в Подсекция 13.8.1, има ефикасен алгоритъм за Максимална Антиклика и върху дърветата.

И още нещо. Ние построихме алгоритъм със сложност $\Theta(n \lg n)$ за Максимална Антиклика върху интервални графи, който ползва представяне на тези графи само чрез множествата от интервали. Очевидно е, че множеството от интервалите представя графа недвусмислено. В Лекция 9 ние видяхме основните, най-често ползвани представяния на графи: списъци на съседство и матрици на съседство. Представянето чрез интервали е друг вид представяне; той очевидно е приложим само за графи, които са интервални. Ние бихме могли да конструираме списъците на съседство от интервалите. Обаче това би искало, в най-лошия случай, време, което е квадратично в броя на интервалите. За да се убедите в това, разгледайте n интервала, всеки два от които се пресичат. Съответният интервален граф е K_n, който има $\Theta(n^2)$ ребра. Само конструирането му като списъци иска време $\Omega(n^2)$. Същото е в сила и ако искаме да конструираме матрицата на съседство.
И така, това, че имаме субквадратичен алгоритъм е поради факта, че не се опитваме да строим преставяне чрез списъци или матрица, а работим директно върху интервалите. Виждаме пример за това, че графов алгоритъм с оптимална ефикасност върху ограничен клас графи е възможен само ако графът е представен по някакъв начин, специфичен за този клас; започнем ли да строим "канонично представяне", сложността по време нараства.

13.6.1.2 Решение по схемата Динамично Програмиране

Ще усложним задачата. Сега всеки интервал a_i има тегло w_i, който е реално положително число. В частност е възможно $w_i = f_i - s_i$, но в общия случай w_i и $f_i - s_i$ са независими. Решението е подмножество от интервали, два по два съвместими, които имат максимално сумарно тегло. Тази задача не може да се реши ефикасно с леко обобщение на алчния алгоритъм от предишната подсекция. Ще видим решение, конструирано по схемата Динамично Програмиране, което има същата сложност по време $\Theta(n \lg n)$ като алчното решение, но е несравнимо по-изтънчено. Естествено, решението с динамично програмиране решава и задачата без тегла, като дава едно и също тегло на всеки интервал.

Нека множеството от интервалите е $A = \{a_1, \ldots, a_n\}$, а сортираната им редица по f-стойности е $S = \langle a'_1, \ldots, a'_n \rangle$. За ефикасно решение с динамично програмиране ни е необходимо да изчисляваме бързо за всеки интервал a'_i в S, интервала с най-голям номер в S, който е вляво от a'_i и е съвместим с a'_i. Да наречем функцията, реализираща това изображение, ϕ, като $\phi : \{1, \ldots, n\} \to \{0, \ldots, n-1\}$. За всяко $i \in \{1, \ldots, n\}$, нека $C(i)$ е множеството от интервалите в S, които са вляво от i и са съвместими с i. Формалната дефиниция на ϕ е

$$\forall i \in \{1, \ldots, n\} : \phi(i) = \begin{cases} \max\{j \in \{1, \ldots, i-1\} \mid a'_j \text{ е съвместим с } a'_i\}, & \text{ако } C(i) \neq \varnothing \\ 0, & \text{ако } C(i) = \varnothing \end{cases}$$

Рекурсивната декомпозиция е по S. Оптималното решение за първите i елемента на S или ползва a'_i, или не го ползва. Ако не го ползва, то е същото като за първите $i-1$ елемента. Ако го ползва, стойността му е сумата от w'_i и оптималната стойност за първите $\phi(i)$ елемента.

Да допуснем, че имаме всички стойности на ϕ в масив $\phi[1, \ldots, n]$. Схемата за изчисление е:

$$\begin{aligned} \text{opt}[0] &= 0 \\ \text{opt}[i] &= \max(\text{opt}[i-1], w'_i + \text{opt}[\phi[i]]), \text{ ако } i > 0 \end{aligned}$$

Следният итеративен алгоритъм решава ефикасно задачата.

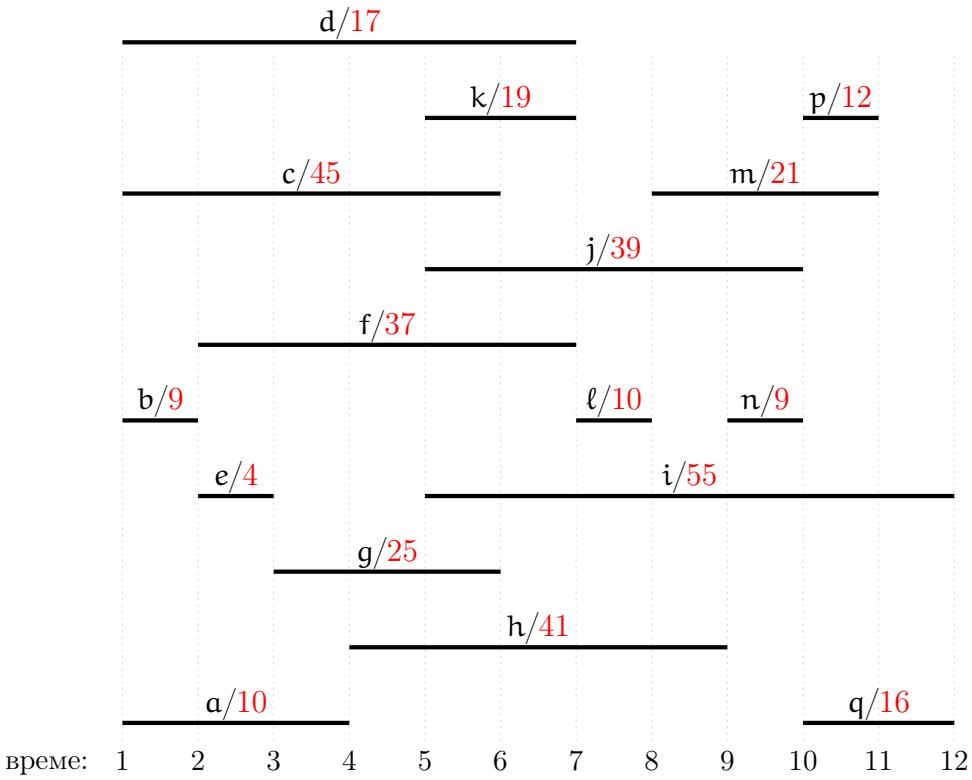
WEIGHTED INTERVAL SCHEDULING($A = \{a_1, \ldots, a_n\}$, където $n \geqslant 1$, $a_i = (s_i, f_i)$, $s_i < f_i$ и a_i има тегло w_i, за $1 \leqslant i \leqslant n$)

```
1  сортирай A по f-стойностите, резултатът е ⟨a'_1, ..., a'_n⟩
2  opt[0] ← 0
3  for i ← 1 to n
4      opt[i] ← max(opt[i − 1]), w'_i + opt[φ[i]])
5  return opt[n]
```

Коректността е очевидна, ако сме убедени в коректността на рекурсивната декомпозиция и схемата за изчисление. Да разгледаме сложността. Ако изчисляваме стойностите на масива $\phi[1, \ldots, n]$ по наивния начин, като за всяко $\phi[i]$ се връщаме назад, може би чак до началото, за да видим кой е най-десният интервал в S, съвместим с i-ия, ще получим квадратичен алгоритъм само за пресмятането на $\phi[1, \ldots, n]$ и оттам, квадратична сложност на целия алгоритъм. За щастие, стойностите на масива $\phi[1, \ldots, n]$ може да се пресметнат във време $O(n \lg n)$ така: за всяко a'_i, с двоично търсене намираме максималното j, такова че $f'_j \leqslant s'_i$. Щом четем в момента a'_i, ние разполагаме с s'_i, а това f'_j ще намерим във време $O(\lg i)$, защото разполагаме със сортираната по f-стойности редица от интервалите.

Щом можем да построим $\phi[1, \ldots, n]$ във време $O(n \lg n)$, очевидно имаме алгоритъм със сложност по време $\Theta(n \lg n)$.

Да разгледаме пример. Интервалите са именувани a, ..., q (без име “o”), а теглата са написани до имената в червено.



Можете ли да откриете оптимално решение на око? Авторът на записките признава, че не може. Да подходим алгоритмично. Ето редицата от интервалите, сортирана по крайни времена, с индексите на интервалите в нея, написани отдолу:

b	e	a	g	c	f	k	d	ℓ	h	n	j	m	p	q	i
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Ето масивът $\phi[1,\ldots,16]$:

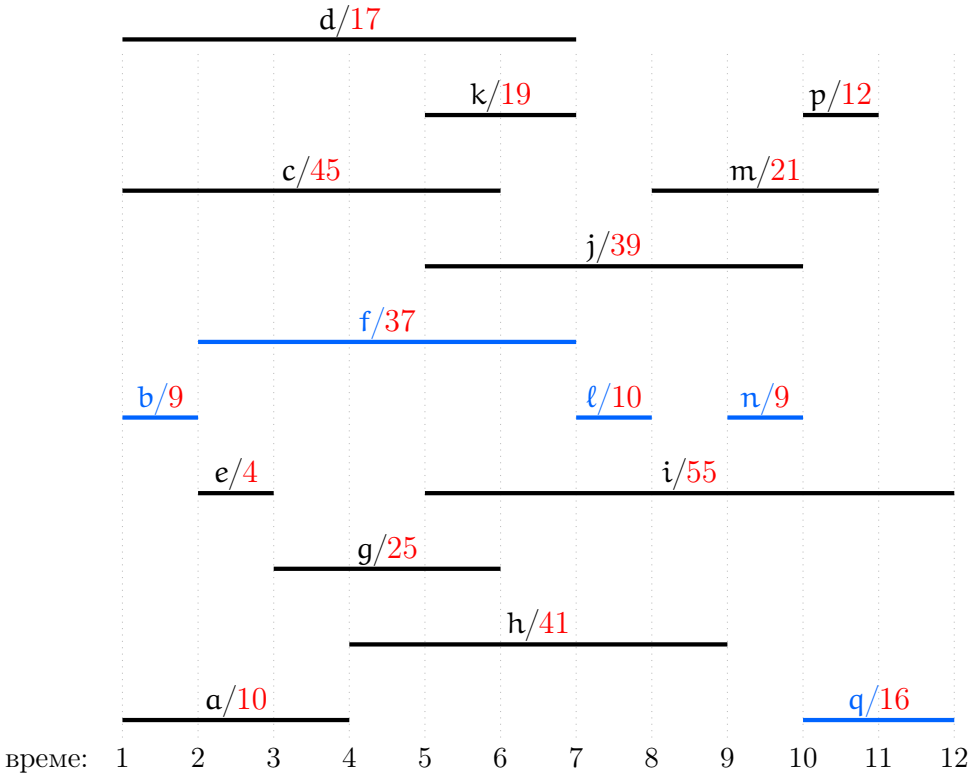
x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\phi(x)$	0	1	0	2	0	1	3	0	8	3	10	3	9	12	12	3

Ето и изчисленията за opt[16]:
$\text{opt}[1] = \max(0, 9 + \text{opt}[0]) = 9.$ ✓
$\text{opt}[2] = \max(9, 4 + \text{opt}[1]) = 13.$ ✓
$\text{opt}[3] = \max(13, 10 + \text{opt}[0]) = 13.$
$\text{opt}[4] = \max(13, 25 + \text{opt}[2]) = 38.$ ✓
$\text{opt}[5] = \max(38, 45 + \text{opt}[0]) = 45.$ ✓
$\text{opt}[6] = \max(45, 37 + \text{opt}[1]) = 46.$ ✓
$\text{opt}[7] = \max(46, 19 + \text{opt}[3]) = 46.$
$\text{opt}[8] = \max(46, 17 + \text{opt}[0]) = 46.$
$\text{opt}[9] = \max(46, 10 + \text{opt}[8]) = 56.$ ✓
$\text{opt}[10] = \max(56, 41 + \text{opt}[3]) = 56.$
$\text{opt}[11] = \max(56, 9 + \text{opt}[10]) = 65.$ ✓
$\text{opt}[12] = \max(65, 39 + \text{opt}[3]) = 65.$
$\text{opt}[13] = \max(65, 21 + \text{opt}[9]) = 77.$ ✓
$\text{opt}[14] = \max(77, 12 + \text{opt}[12]) = 77.$

$\mathrm{opt}[15] = \max(77, 16 + \mathrm{opt}[12]) = 81$. ✓
$\mathrm{opt}[16] = \max(81, 55 + \mathrm{opt}[3]) = 81$.
И така, оптималното решение има стойност 81.

Ако искаме да получим и самото оптимално решение, а не само стойността му, можем да постъпим така. В изчисленията за $\mathrm{opt}[16]$, с червен тик е отбелязан всеки ред, в който стойността е получена от втория аргумент на max, а именно $\boxed{w'_i + \mathrm{opt}[\phi[i]]}$ на ред 4. Иначе казано, това са редовете, в които има увеличение спрямо предишния ред. Тръгвайки от последния ред, връщаме се назад до първия тик, който е на $\mathrm{opt}[15]$. Виждаме, че $\mathtt{q}$ участва в оптималното решение и освен това $\mathrm{opt}[15]$ е получен като $\boxed{16 + \mathrm{opt}[12] = 81}$. Отиваме на реда на $\mathrm{opt}[12]$. Той няма тик, така че се връщаме до първия ред назад с тик, който е реда на $\mathrm{opt}[11]$. Тогава $\mathtt{n}$ участва в оптималното решение и освен това $\mathrm{opt}[11]$ е получен като $\boxed{9 + \mathrm{opt}[10] = 65}$. Отиваме на реда на $\mathrm{opt}[10]$. Той няма тик, така че се връщаме до първия ред назад с тик, който е реда на $\mathrm{opt}[9]$. Тогава ℓ участва в оптималното решение и освен това $\mathrm{opt}[9]$ е получен като $\boxed{10 + \mathrm{opt}[8] = 56}$. Отиваме на реда на $\mathrm{opt}[8]$. Той няма тик, така че се връщаме до първия ред назад с тик, който е редът на $\mathrm{opt}[6]$. Тогава $\mathtt{f}$ участва в оптималното решение и освен това $\mathrm{opt}[6]$ е получен като $\boxed{37 + \mathrm{opt}[1] = 46}$. Отиваме на реда на $\mathrm{opt}[1]$, добавяме $\mathtt{b}$ към решението и сме готови. Решението се състои от $\mathtt{q}$, $\mathtt{n}$, ℓ, $\mathtt{f}$ и $\mathtt{b}$, сумата от теглата на които наистина е $16 + 9 + 10 + 37 + 9 = 81$. Читателят лесно може да съобрази как да кодира тази идея в алгоритъма.

Ето и визуализация на оптималното решение. Интервалите от оптималното решение са в синьо.



13.7 Longest Increasing Subsequence

Дадена е числова редица $S = \langle a_1, a_2, \ldots, a_n \rangle$. БОО, нека числата са две по две различни. *Поредица в* S е всяка редица $\langle a_{i_1}, a_{i_2}, \ldots, a_{i_k} \rangle$, такава че $1 \leqslant i_1 < i_2 < \cdots < i_k \leqslant n$. *Растяща*

поредица в S е всяка поредица $\langle a_{i_1}, a_{i_2}, \ldots, a_{i_k}\rangle$, такава че $a_{i_1} < a_{i_2} < \cdots < a_{i_k}$. Задачата е, при дадена числова редица, да се намери максимална растяща поредица; в лекия вариант, да се намери само дължината на максимална растяща поредица. "Максимална" означава с максимален брой елементи.

Забележете, че "поредица в S" е редица от елементи, които **не са непременно съседни** в S. Поредица, чиито елементи са съседни—това е частен случай на поредица—наричаме *непрекъсната*, на английски *contiguous*. Разговорно казано, елементите на поредицата в общия случай са пръснати в S. Ето пример.

$$S = \langle 14, 9, -5, 11, -2, 0, 3, -4, 12, 19, 6, 8, 22, 10\rangle$$

Поредица е, примерно (в червено):

$$S = \langle 14, 9, -5, 11, -2, 0, 3, -4, 12, 19, 6, 8, 22, 10\rangle$$

Следната поредица е непрекъсната (в лилаво):

$$S = \langle 14, 9, -5, 11, -2, 0, 3, -4, 12, 19, 6, 8, 22, 10\rangle$$

Растяща поредица, максимална при това, е (в синьо):

$$S = \langle 14, 9, -5, 11, -2, 0, 3, -4, 12, 19, 6, 8, 22, 10\rangle$$

Максимална непрекъсната растяща поредица е (в зелено):

$$S = \langle 14, 9, -5, 11, -2, 0, 3, -4, 12, 19, 6, 8, 22, 10\rangle$$

Намирането на максимална непрекъсната растяща поредица е тривиална задача. Има очевидно решение в линейно време с едно премитане, но дори решението с груба сила не е невъзможно бавно, понеже непрекъснатите поредици са $\Theta(n^2)$ на брой. Намирането на максимална растяща поредица обаче е нетривиална задача. Поредиците без ограничението за непрекъснатост са много повече—а именно, $2^n - 1$, ако не броим празната поредица—и подходът с груба сила е безнадежден. В примера със синята поредица, когато разглеждаме -5 и отиваме надясно, как да разберем, че трябва да прескочим 11, но да не прескочим -2?

13.7.1 Дин. Прогр. със сложност $\Theta(n^2)$

Първо решаваме задачата в лекия вариант, в който се иска само стойността на оптимално решение. Изглежда смислено да характеризираме оптималното решение като поредица, завършваща с някакъв елемент a_k, където $1 \leqslant k \leqslant n$.

Спрямо тази характеризация, нека да мислим за рекурсивна декомпозиция. Нека

$$\begin{aligned}
S^1 &= \langle a_1\rangle \\
S^2 &= \langle a_1, a_2\rangle \\
S^3 &= \langle a_1, a_2, a_3\rangle \\
&\ldots \\
S^n &= \langle a_1, a_2, \ldots, a_n\rangle = S
\end{aligned}$$

Нека ℓ_k е дължината на максимална растяща поредица в S^k, завършваща на a_k, за $1 \leqslant k \leqslant n$. Ако имаме тези ℓ_k, то ние сме готови! Тогава отговорът е $\max\{\ell_k \mid 1 \leqslant k \leqslant n\}$, тъй като всяка

максимална растяща поредица завършва на някой елемент.

Как да получим ℓ_k от $\ell_{k-1}, \ldots, \ell_1$? Само числата $\ell_{k-1}, \ldots, \ell_1$ не са достатъчни, но комбинациите от ℓ_j и a_j, за $j \in \{1, \ldots, k-1\}$ са достатъчни. Интересуват ни само тези a_j, за които $a_j < a_k$, ако има такива. Ако има такива, a_k е максимумът от техните ℓ-стойности, плюс едно. Ако няма такива, то a_k е единица. Нека

$$\forall k \in \{1, \ldots, n\} : R_k = \{j \in \{1, \ldots, k-1\} \mid a_j < a_k\}$$

Тогава, за $1 \leqslant k \leqslant n$:

$$\ell_k = \begin{cases} \max\{\ell_j \mid j \in R_k\} + 1, & \text{ако } R_k \neq \varnothing \\ 1, & \text{ако } R_k = \varnothing \end{cases}$$

Превръщането на рекурсивната декомпозиция в алгоритъм е тривиално. Забележете, че $R_1 = \varnothing$, така че $\ell_1 = 1$ като начално условие.

Сега да помислим за по-тежкия вариант на задачата, в който се иска не просто дължината на максимална растяща поредица, а и някоя максимална растяща поредица. За всяка позиция $k \in \{1, \ldots, n\}$ записваме позицията π_k на елемента—ако има такъв—който предшества a_k в максимална растяща поредица, завършваща на a_k. Ако няма такъв, нека $\pi_k = \text{nil}$. Ето алгоритъм, който реализира тази идея.

LONGEST INCREASING SUBSEQUENCE($\langle a_1, a_2, \ldots, a_n \rangle$)
```
 1  ℓ[1] ← 1
 2  π[1] ← nil
 3  for k ← 2 to n
 4      ℓ[k] ← 1
 5      π[k] ← nil
 6      for j ← 1 to k − 1
 7          if ℓ[k] < ℓ[j] + 1 and a[k] > a[j]
 8              ℓ[k] ← ℓ[j] + 1
 9              π[k] ← j
10  return (max_{1⩽k⩽n} {ℓ[k]}, π)
```

Коректността е очевидна. Сложността е $\Theta(n^2)$. Както ще видим, има по-ефикасен алгоритъм за тази задача, но квадратичната сложност по време не е непоносима. Ето примерна работа на алгоритъма върху $S = \langle 14, 9, -5, 11, -2, 0, 3, -4, 12, 19, 6, 8, 22, 10 \rangle$.

k	1	2	3	4	5	6	7	8	9	10	11	12	13	14
S[k]	14	9	−5	11	−2	0	3	−4	12	19	6	8	22	10
ℓ[k]	1	1	1	2	2	3	4	2	5	6	5	6	7	7
π[k]	nil	nil	nil	3	3	5	6	3	7	9	7	11	10	12

Едно оптимално решение е маркирано в синьо.

13.7.2 Дин. Прогр. със сложност $\Theta(n \lg n)$

Оригиналната статия с решение на задачата LONGEST INCREASING SUBSEQUENCE на Fredman от 1972 г. [29] предлага алгоритъм със сложност по време $\Theta(n \lg n)$, но анализът на алгоритъма е само скициран.

Има множество свободно достъпни решения със сложност $\Theta(n \lg n)$ в Интернет, но повечето имат много малко обосновка или обосновката не е съгласно схемата Динамично Програмиране или авторите директно казват, че този алгоритъм не е изграден по схемата Динамично Програмиране. Представеното тук решение се базира на *онлайн ресурс на професор Geppino Pucci* от Университета на Padova, Италия. Авторът на тези лекционни записки е на мнение, че има дребна формална грешка в рекурсивната декомпозиция на професор Pucci, поради което изложението тук се различава от неговото. Също така, тук се ползва различна нотация.

В Подсекция 13.7.1 характеризирахме оптималното решение като поредица, завършваща с някакъв елемент a_k. Сега ще характеризираме по друг начин. Нека S^k има смисълът от Подсекция 13.7.1. За всяко $k \in \{1, \ldots, n\}$, измежду всички максимални растящи поредици в S^k, разглеждаме такава, която завършва с минимален елемент. Подчертаваме разликата:

- преди искахме поредицата в S^k да завършва непременно на a_k,
- а сега отпускаме това ограничение и искаме поредицата да завършва на **някакъв** $a_j \in S^k$, който е минимален измежду всички завършеци на максимални растящи поредици в S^k.

Защо искаме завършващият елемент да е минимален? Следното тривиално наблюдение дава отговора.

Наблюдение 55

Измежду всички нарастващи поредици с дадена дължина, тази с минималния завършек има най-добър шанс да бъде увеличена чрез "залепване" на елемент вдясно.

За краткост ползваме "МРПМЗ" вместо "**м**аксимална **р**астяща **п**оредица с **м**инимален **з**авършек". Като пример да разгледаме $S = \langle 14, 9, -5, 11, -2, 0, 3, -4, 12, 19, 6, 8, 22, 10\rangle$. Нека $k = 5$. Тогава $S^k = S^5 = \langle 14, 9, -5, 11, -2\rangle$. Максималните растящи поредици в S^5 са $\langle 9, 11\rangle$, $\langle -5, 11\rangle$ и $\langle -5, -2\rangle$. МРПМЗ в S^5 е $\langle -5, -2\rangle$, понеже $-2 < 11$.

Ако знаем както МРПМЗ за S^k, така и a_{k+1}, дали знаем МРПМЗ за S^{k+1}? Не непременно. Нека МРПМЗ за S^k е $\langle a_{i_1}, \ldots, a_{i_j}\rangle$.

- Ако $a_{i_j} < a_{k+1}$, със сигурност $\langle a_{i_1}, \ldots, a_{i_j}, a_{k+1}\rangle$ е МРПМЗ за S^{k+1}, понеже
  - има дължина $j+1$, а в S^k няма растяща поредица с дължина $j+1$, ерго, в S^{k+1} няма растяща поредица с дължина $> j+1$;
  - завършва на a_{k+1}, който елемент очевидно е най-малкият възможен завършек на растяща поредица с дължина $j+1$, защото е единственият завършек на растяща поредица с дължина $j+1$ в S^{k+1}.
- Ако обаче $a_{i_j} > a_{k+1}$, то $\langle a_{i_1}, \ldots, a_{i_j}, a_{k+1}\rangle$ не е растяща поредица. В този случай със сигурност в S^{k+1} няма растяща поредица с дължина $> j$, но в S^k може да има растяща поредица $\langle a_{t_1}, \ldots, a_{t_{j-1}}\rangle$, такава че $a_{t_{j-1}} < a_{k+1}$. Забележете, че $\langle a_{t_1}, \ldots, a_{t_{j-1}}\rangle$ не е максимална растяща поредица в S^k, имайки дължина само $j-1$, но в S^{k+1}, $\langle a_{t_1}, \ldots, a_{t_{j-1}}, a_{k+1}\rangle$ е растяща поредица с дължина j. Тогава и $\langle a_{i_1}, \ldots, a_{i_j}\rangle$, и $\langle a_{t_1}, \ldots, a_{t_{j-1}}, a_{k+1}\rangle$ са максимални растящи поредици в S^{k+1}, но $\langle a_{t_1}, \ldots, a_{t_{j-1}}, a_{k+1}\rangle$ е МРПМЗ, а $\langle a_{i_1}, \ldots, a_{i_j}\rangle$ не е МРПМЗ.

И така, убедихме се, че дори да знаем както МРПМЗ за S^k, така и a_{k+1}, това не е достатъчно, за да знаем МРПМЗ за S^{k+1}. Ако обаче за **всяка** дължина j знаем растяща поредица с минимален завършек в S^k с дължина j, то знаем МРПМЗ за S^{k+1}.

Да дефинираме, че за всеки $j, k \in \{1, \ldots, n\}$, L_j^k е растяща поредица с дължина j и минимален завършек в S^k, а ℓ_j^k е нейният завършек; ако няма такава, то приемаме, че $L_j^k = \langle\rangle$ (празната поредица), а $\ell_j^k = \infty$ (идентитетът на минимума).

Лема 66

В текущия контекст, нека за някое j, L_j^k и L_{j+1}^k са непразни. Нека $L_j^k = \langle a_{i_1}, \ldots, a_{i_j}\rangle$ и $L_{j+1}^k = \langle a_{t_1}, \ldots, a_{t_j}, a_{t_{j+1}}\rangle$, като $a_{i_j} = \ell_j^k$ и $a_{t_{j+1}} = \ell_{j+1}^k$. Тогава $\ell_j^k < \ell_{j+1}^k$.

Доказателство: Да допуснем, че $\ell_{j+1}^k < \ell_j^k$, тоест $a_{t_{j+1}} < a_{i_j}$. Но тогава $a_{t_j} < a_{i_j}$, понеже $a_{t_j} < a_{t_{j+1}}$. Следователно, поредицата $\langle a_{t_1}, \ldots, a_{t_j}\rangle$ е растяща поредица в S^k с дължина j и завършек, по-малък от ℓ_j^k. Но това е невъзможно, понеже по конструкция L_j^k, която има завършек ℓ_j^k, е растяща поредица в S^k с дължина j и с **минимален** завършек. ↯ □

И така, доказахме, че във всяко S^k, завършеците на непразните растящи поредици с минимални завършеци, растат с растенето на дължините на поредиците.

Да направим рекурсивна декомпозиция. Началните условия са следните:

$$L_1^k = \langle \min(S^k)\rangle, \quad \text{за } 1 \leqslant k \leqslant n \tag{13.23}$$
$$L_j^k = \langle\rangle, \quad \text{за } 1 \leqslant k < j \leqslant n \tag{13.24}$$

Коректността им е очевидна. Самата рекурсия изглежда така. При $2 \leqslant k \leqslant n$ и $2 \leqslant j \leqslant k$:

$$L_j^k = L_j^{k-1}, \quad \text{ако } \ell_j^{k-1} < a_k \tag{13.25}$$
$$L_j^k = L_j^{k-1}, \quad \text{ако } a_k < \ell_j^{k-1} \text{ и } a_k < \ell_{j-1}^{k-1} \tag{13.26}$$
$$L_j^k = \text{concat}(L_{j-1}^{k-1}, a_k), \quad \text{ако } \ell_{j-1}^{k-1} < a_k < \ell_j^{k-1} \tag{13.27}$$

Ето обосновката.

- В (13.25), ℓ_j^{k-1}, завършекът на L_j^{k-1}, е по-малък от a_k. Тогава "долепянето" на a_k като завършек не би дало растяща поредица с дължина j и по-малък завършек, така че L_j^{k-1} е растяща поредица с дължина j и минимален завършек както в S^{k-1}, така и в S^k. В (13.25), L_j^{k-1} е непразна, иначе ℓ_j^{k-1} би бил ∞, при което $\ell_j^{k-1} < a_k$ би било невъзможно.
- Да допуснем, че $a_k < \ell_j^{k-1}$. Сега е възможно L_j^{k-1} да е празна, което влече $\ell_j^{k-1} = \infty$.
  - В (13.26), $a_k < \ell_{j-1}^{k-1}$.
    - Да допуснем, че L_{j-1}^{k-1} е непразна, което влече $\ell_{j-1}^{k-1} < \infty$. В този случай, всяка растяща поредица с дължина $j-1$ в S^{k-1} има завършек-число, по-голямо от a_k. Ерго, a_k не може да се използва за "удължаване" до растяща поредица с дължина j. Ерго, $L_j^k = L_j^{k-1}$.
    - Да допуснем, че L_{j-1}^{k-1} е празна, което влече $\ell_{j-1}^{k-1} = \infty$. В този подслучай наистина $a_k < \ell_{j-1}^{k-1}$, но конкатенацията на празната L_{j-1}^{k-1} с a_k дава растящата поредица $\langle a_k\rangle$. От друга страна обаче, тъй като $j > 1$, поредицата $\langle a_k\rangle$ не може да има дължина j. Ерго, отново a_k не може да се използва за "удължаване" до растяща поредица с дължина j. Ерго, $L_j^k = L_j^{k-1}$.
  - Ако обаче $\ell_{j-1}^{k-1} < a_k < \ell_j^{k-1}$ (13.27), то a_k може да се използва за "удължаване" на растяща поредица с дължина $j-1$ до растяща поредица с дължина j, като при това новият завършек a_k е по-малък от предишния завършек ℓ_j^{k-1} на растяща поредица с дължина j. Забележете, че щом $\ell_{j-1}^{k-1} < a_k$, то L_{j-1}^{k-1} не е празна.

Нека пак разгледаме примера на стр. 530.

k	1	2	3	4	5	6	7	8	9	10	11	12	13	14
S[k]	14	9	−5	11	−2	0	3	−4	12	19	6	8	22	10

Ако следваме буквално (13.23), (13.24), (13.25), (13.26) и (13.27), получаваме следните поредици (само непразните поредици са показани):

$L_1^1 = \langle 14 \rangle$
$L_1^2 = \langle 9 \rangle$
$L_1^3 = \langle -5 \rangle$
$L_1^4 = \langle -5 \rangle$
$L_2^4 = \langle -5, 11 \rangle$
$L_1^5 = \langle -5 \rangle$
$L_2^5 = \langle -5, -2 \rangle$
$L_1^6 = \langle -5 \rangle$
$L_2^6 = \langle -5, -2 \rangle$
$L_3^6 = \langle -5, -2, 0 \rangle$
$L_1^7 = \langle -5 \rangle$
$L_2^7 = \langle -5, -2 \rangle$
$L_3^7 = \langle -5, -2, 0 \rangle$
$L_4^7 = \langle -5, -2, 0, 3 \rangle$
$L_1^8 = \langle -5 \rangle$
$L_2^8 = \langle -5, -4 \rangle$
$L_3^8 = \langle -5, -2, 0 \rangle$
$L_4^8 = \langle -5, -2, 0, 3 \rangle$

$L_1^9 = \langle -5 \rangle$
$L_2^9 = \langle -5, -4 \rangle$
$L_3^9 = \langle -5, -2, 0 \rangle$
$L_4^9 = \langle -5, -2, 0, 3 \rangle$
$L_5^9 = \langle -5, -2, 0, 3, 12 \rangle$
$L_1^{10} = \langle -5 \rangle$
$L_2^{10} = \langle -5, -4 \rangle$
$L_3^{10} = \langle -5, -2, 0 \rangle$
$L_4^{10} = \langle -5, -2, 0, 3 \rangle$
$L_5^{10} = \langle -5, -2, 0, 3, 12 \rangle$
$L_6^{10} = \langle -5, -2, 0, 3, 12, 19 \rangle$
$L_1^{11} = \langle -5 \rangle$
$L_2^{11} = \langle -5, -4 \rangle$
$L_3^{11} = \langle -5, -2, 0 \rangle$
$L_4^{11} = \langle -5, -2, 0, 3 \rangle$
$L_5^{11} = \langle -5, -2, 0, 3, 6 \rangle$
$L_6^{11} = \langle -5, -2, 0, 3, 12, 19 \rangle$

$L_1^{12} = \langle -5 \rangle$
$L_2^{12} = \langle -5, -4 \rangle$
$L_4^{12} = \langle -5, -2, 0, 3 \rangle$
$L_5^{12} = \langle -5, -2, 0, 3, 6 \rangle$
$L_6^{12} = \langle -5, -2, 0, 3, 6, 8 \rangle$
$L_1^{13} = \langle -5 \rangle$
$L_2^{13} = \langle -5, -4 \rangle$
$L_3^{13} = \langle -5, -2, 0 \rangle$
$L_4^{13} = \langle -5, -2, 0, 3 \rangle$
$L_5^{13} = \langle -5, -2, 0, 3, 6 \rangle$
$L_6^{13} = \langle -5, -2, 0, 3, 6, 8 \rangle$
$L_7^{13} = \langle -5, -2, 0, 3, 6, 8, 22 \rangle$
$L_1^{14} = \langle -5 \rangle$
$L_2^{14} = \langle -5, -4 \rangle$
$L_3^{14} = \langle -5, -2, 0 \rangle$
$L_4^{14} = \langle -5, -2, 0, 3 \rangle$
$L_5^{14} = \langle -5, -2, 0, 3, 6 \rangle$
$L_6^{14} = \langle -5, -2, 0, 3, 6, 8 \rangle$
$L_7^{14} = \langle -5, -2, 0, 3, 6, 8, 10 \rangle$

Виждаме добра илюстрация на Лема 66: по отношение на фиксирано k, завършеците на поредиците L_j^k нарастват с нарастването на j. Примерно, при $k = 10$, завършеците са, в този ред:

$$-5, -4, 0, 3, 12, 19$$

Сега да си представим за всяко $k \in \{1, \ldots, 14\}$, за всяко $j \in \{1, \ldots, 14\}$, **всички** поредици L_j^k в нашия пример. Това са 196 поредици и няма да ги записваме явно, но да си представим

нещата за различните k:

$$
\begin{aligned}
k = 1: &\quad L_1^1 = \langle 14 \rangle,\ L_2^1 = \langle\,\rangle,\ L_3^1 = \langle\,\rangle,\ \ L_4^1 = \langle\,\rangle, \ldots,\ L_{14}^1 = \langle\,\rangle \\
k = 2: &\quad L_1^2 = \langle 9 \rangle,\ L_2^2 = \langle\,\rangle,\ L_3^2 = \langle\,\rangle,\ \ L_4^2 = \langle\,\rangle, \ldots,\ L_{14}^2 = \langle\,\rangle \\
k = 3: &\quad L_1^3 = \langle -5 \rangle,\ L_2^3 = \langle\,\rangle,\ L_3^3 = \langle\,\rangle,\ \ L_4^3 = \langle\,\rangle, \ldots,\ L_{14}^3 = \langle\,\rangle \\
k = 4: &\quad L_1^4 = \langle -5 \rangle,\ L_2^4 = \langle -5, 11 \rangle,\ L_3^4 = \langle\,\rangle,\ \ L_4^4 = \langle\,\rangle, \ldots,\ L_{14}^4 = \langle\,\rangle \\
k = 5: &\quad L_1^5 = \langle -5 \rangle,\ L_2^5 = \langle -5, -2 \rangle,\ L_3^5 = \langle\,\rangle,\ \ L_4^5 = \langle\,\rangle, \ldots,\ L_{14}^5 = \langle\,\rangle \\
k = 6: &\quad L_1^6 = \langle -5 \rangle,\ L_2^6 = \langle -5, -2 \rangle,\ L_3^6 = \langle -5, -2, 0 \rangle,\ L_4^6 = \langle\,\rangle,\ \ldots,\ L_{14}^6 = \langle\,\rangle \\
&\quad \ldots \\
k = 13: &\quad L_1^{13} = \langle -5 \rangle,\ \ldots,\ L_7^{13} = \langle -5, -2, 0, 3, 6, 8, 22 \rangle,\ L_8^{13} = \langle\,\rangle,\ \ldots,\ L_{14}^{13} = \langle\,\rangle \\
k = 14: &\quad L_1^{14} = \langle -5 \rangle,\ \ldots,\ L_7^{14} = \langle -5, -2, 0, 3, 6, 8, 10 \rangle,\ L_8^{14} = \langle\,\rangle,\ \ldots,\ L_{14}^{14} = \langle\,\rangle
\end{aligned}
$$

Забележе, че за всяко релевантно k, при прехода от $k-1$ към k, точно за едно j поредицата става различна, а за останалите j поредиците са едни и същи. Примерно:

- при прехода от $k = 1$ към $k = 2$ е вярно, $L_1^1 = \langle 14 \rangle$ и $L_1^2 = \langle 9 \rangle$ се различават, а за $j \in \{2, \ldots, 14\}$ имаме $L_j^1 = L_j^2 = \langle\,\rangle$,
- при прехода от $k = 13$ към $k = 14$ е вярно, че L_7^{13} и L_7^{14} се различават, а за $j \in \{1, \ldots, 6, 8, \ldots, 14\}$ имаме $L_j^{13} = L_j^{14}$.

Това не е случайно. В *ресурса на професор Geppino Pucci*, на страница 4 се казва (написано в текущата терминология):

> *... тъй като завършеците са в нарастващ ред съгласно Лема 66, то когато превключваме от един префикс S^{k-1} към S^k, само за едно единствено j, подзадачата за това j в контекста на $k-1$ може да има различно решение от подзадачата за това j в контекста на k.*

При това терминът "може" е слаб: не просто може, а задължително има (точно едно) j, за което решението е различно.

Сега ще докажем това твърдение строго.

Теорема 69: Обосновка на бързия алг. за Longest Increasing Sequence

За всяко изпълнение на рекурсивния алгоритъм, определен от (13.23), (13.24), (13.25), (13.26) и (13.27), за всяко $k \in \{2, \ldots, n\}$ съществува точно едно $j \in \{1, \ldots, n\}$, такова че $L_j^{k-1} \neq L_j^k$.

Доказателство: Да разгледаме отново (13.25), (13.26) и (13.27). Тъй като редицата

$$\langle \ell_1^{k-1}, \ell_2^{k-1}, \ldots, \ell_k^{k-1} \rangle$$

е сортирана съгласно Лема 66 и освен това числата в S са две по две различни, наистина има точно едно j, такова че a_k попада между ℓ_{j-1}^{k-1} и ℓ_j^{k-1}. Това е в сила

- и когато $\ell_j^{k-1} < \infty$, тоест, a_k е по-малък от поне един завършек
-

когато $\ell_j^{k-1} = \infty$, тоест, когато a_k е по-голямо число от всички крайни завършеци ℓ_t^{k-1}, където $t < j$. За това j, съгласно (13.27), завършекът на L_j^k става a_k, а a_k е различен (по-малък, ако трябва да сме точни) от завършека на L_j^{k-1}, който е именно ℓ_j^{k-1}. За всички $t \neq j$ е вярно, че $L_t^k = L_t^{k-1}$ съгласно (13.25) или (13.26). □

Например да разгледаме списъка от непразните поредици на стр. 535.

Това, че съществува такова j, лесно следва от факта, че числата в S са две по две различни. А че това j не е повече от едно—което е същността на наблюдението—следва лесно от факта, че новото число S_{k+1} попада на едно единствено място в досегашната сортирана редица от завършеци. "Попада на място в редицата" означава, че в тази (сортирана) редица има една-единствена позиция, чийто елемент е най-малкият елемент, по-голям от новото S_{k+1}, и S_{k+1} го заменя; ако новото S_{k+1} е по-голямо от всеки в редицата, то елементът, който бива заменен, е най-лявата ∞.

За нашия пример, при $k = 1$, редицата от завършеците е

$$14,\ \infty,\ \ldots,\ \infty$$

Минавайки към $k = 2$, имаме $S_2 = 9$, което попада върху 14, и редицата от завършеците е

$$9,\ \infty,\ \ldots,\ \infty$$

Минавайки към $k = 3$, имаме $S_3 = -5$, което попада върху 9, и редицата от завършеците е

$$-5,\ \infty,\ \ldots,\ \infty$$

Минавайки към $k = 4$, имаме $S_4 = 11$, което попада върху най-лявата ∞, и редицата от завършеците е

$$-5,\ 11, \infty,\ \ldots,\ \infty$$

Минавайки към $k = 5$, имаме $S_5 = -2$, което попада върху 11, и редицата от завършеците е

$$-5,\ -2, \infty,\ \ldots,\ \infty$$

Минавайки към $k = 6$, имаме $S_6 = 0$, което попада върху най-лявата ∞, и редицата от завършеците е

$$-5,\ -2,\ 0,\ \infty,\ \ldots,\ \infty$$

И така нататък.

не е довършено...

13.7.3 Patiencesort

Статията на Aldous и Diaconis [5] съдържа много подробна информация за теоретичната основа на тази сортировка, както и сложни и нетривиални резултати за асимптотиките на максималните растящи поредици.

13.7.4 Building Bridges

13.8 NP-трудни задачи върху дървета

Лекция 14 разглежда феномена на алгоритмична неподатливост в детайли. Тук само ще кажем, че **NP**-трудна задача е оптимизационна задача, за която

- не е известен ефикасен алгоритъм, който намира решение "от нищото", тоест, разполага единствено с входа (всички досега разгледани алгоритми са такива),

- но има ефикасен алгоритъм, такъв, че ако му са даде кандидат-решение, той може да провери в полиномиално време, че това наистина е решение. Откъде се взема кандидат-решението е въпрос, който не разглеждаме. **Ако** му се даде такова, **то** той може да провери бързо, че то наистина е решение.

Много от **NP**-трудните задачи са върху графи, било ориентирани, било неориентирани. По правило, **NP**-трудните задачи върху неориентирани графи стават ефикасно решими, ако графите са дървета. Авторът се сеща за само една естествена графова задача, а именно Bandwidth, която остава **NP**-трудна върху дървета.

В тази секция ще разгледаме ефикасни алгоритми по схемата Динамично Програмиране за две важни графови задачи, ако примерите бъдат ограничени до дървета.

Това, че изключително трудни задачи стават лесни, ако наложим сериозни ограничения върху примерите, не е странно. Нещо такова вече видяхме в Допълнение 55. Но кое ограничение е достатъчно сериозно, така че да превърне **NP**-трудна задача в ефикасно решима задача? На този важен въпрос няма (засега) смислен отговор. Не всяко ограничение върху примерите "облекчава" задачата; повечето **NP**-трудни задачи върху неориентирани графи остават **NP**-трудни, ако ограничим примерите до планарни графи.

13.8.1 Minimum Vertex Cover on trees

Върхово покриване, на английски *vertex cover*, на (обикновен) граф G е всяко подмножество от върхове, чието изтриване води до подграф на без ребра. Намиране на какво да е върхово покриване е тривиално: самото $V(G)$ е върхово покриване, а също така $V(G)\setminus\{u\}$, за кой да е връх u, е върхово покриване. Трудно е да се намери върхово покриване с малка мощност. Задачата намира приложение, примерно, когато графът моделира някакви несъвместимости и искаме да намерим малко множество обекти (върхове), чието отстраняване елиминира несъвместимостите; с други думи, останалите обекти са съвместими по двойки.

Формалното определение е следното.

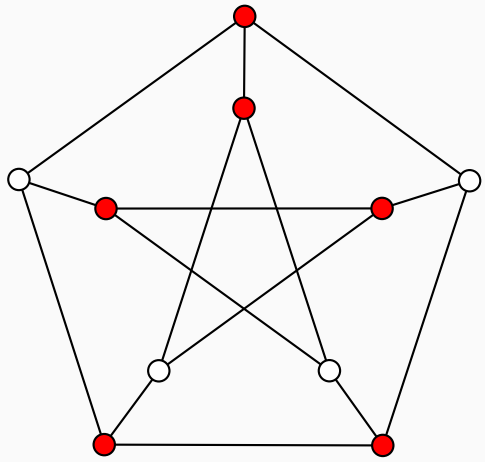
Изч. Задача: Minimum Vertex Cover

пример: Неориентиран граф $G = (V, E)$.
решение: Подмножество $U \subset V$, такова че $\forall(u, v) \in E : (u \in U$ или $v \in U)$ и $|U|$ е минимално.

Мощността на минимално върхово покриване на G се бележи с $\tau(G)$.

Пример за минимално върхово покриване е показан на Фигура 13.1. Графът на Petersen, да го наречем P, не може да бъде покрит с по-малко от 6 върха: само външният 5-цикъл иска поне 3 върха, а независимо от това, вътрешният 5-цикъл (вътрешните пет диагонала) иска също поне 3 върха, така че 6 е долна граница за мощността на върховото покриване и върховото покриване на Фигура 13.1 е оптимално. И така, $\tau(P) = 6$.

Фигура 13.1 : Минимално върхово покриване в графа на Petersen.



Задачата Максимална Антиклика, на английски Maximum Independent Set, е много близка до Минимално Върхово Покриване. *Антиклика* в граф G е всяко подмножество на $V(G)$, между нито два върха на което няма ребро.

Изч. Задача: Maximum Independent Set

пример: Неориентиран граф $G = (V, E)$.
решение: Подмножество $U \subset V$, такова че U е антиклика и $|U|$ е максимално.

Минимално Върхово Покриване и Максимална Антиклика са дуални в смисъл, че върху един и същи граф G, U е върхово покриване тстк $V(G) \backslash U$ е антиклика. Това се илюстрира добре от Фигура 13.1: четирите неоцветени върха са антиклика, и по-голяма антиклика няма (ако имаше, щеше да има по-малко върхово покриване).

Минимално Върхово Покриване е трудна изчислителна задача. Във варианта на задача за разпознаване, тя е една от класическите **NP**-пълни задачи [31]: 3SAT се свежда до нея в смисъл, че за всеки пример $\mathcal{I}$ на 3SAT бързо можем да конструираме пример $\mathcal{I}'$ на Vertex Cover, като $\mathcal{I}$ е Да-пример на 3SAT тстк $\mathcal{I}'$ е Да-пример на Vertex Cover.

Върху дървета обаче, задачата Vertex Cover е решима ефикасно. Дърветата имат много по-бедна структура от графите по принцип и това позволява да има ефикасно решение. Прочее, почти всички от добре известните **NP**-пълни задачи задачи върху неориентирани графи са решими ефикасно върху дървета; изключенията са твърде малко. В тази секция ще разгледаме линеен алгоритъм за Vertex Cover върху дървета, изграден върху схемата **Динамично Програмиране**.

Преди това обаче ще скицираме алчен алгоритъм за Vertex Cover върху дървета, който може да се имплементира с линейна сложност по време. Нека T е произволно дърво с повече от два върха. Нека U е антикликата, която ще изградим. В началото, $U = \varnothing$. Нека L_1 е множеството от висящите върхове на T. Нека L_2 е множеството от техните съседи. Твърдим, че не можем да сбъркаме, добавяйки L_2 към U. Наистина, всяко ребро между висящ връх и негов съсед трябва да бъде покрито в смисъл, че поне единият му край трябва да влезе в U. Нищо няма да загубим, ако вкараме в U връх v от L_2:

- ако v е съсед на точно един w от L_1, то поне един от $\{v, w\}$ трябва да влезе в U,

- ако v е съсед на повече от един върха $w_1, \ldots, w_k$, то, ако не сложим v в U, трябва да сложим всички $w_1, \ldots, w_k$; а ако сложим v, можем да забравим за $w_1, \ldots, w_k$, понеже v покрива ребрата $(v, w_1), \ldots, (v, w_k)$.

И така, първо изтриваме върховете от L_1 от T и после правим $U \leftarrow U \cup L_2$ и изтриваме и L_2 от T. Ако не остане нищо от T, или ако останат дървета всяко с един връх[†], то няма какво повече да правим – всички ребра от оригиналното T са покрити от L_2. В противен случай, продължаваме така върху дърветата на гората $T - L_1 - L_2$: ако дадено дърво от гората е с едно ребро, добавяме към U кой да е край на реброто, в противен случай идентифицираме висящите върхове, слагаме техните съседи в U, изтриваме и тези висящи върхове, и въпросните съседи, и така нататък. Ефикасната имплементация може да е трикова, но идеята е ясна: "белим" дървото "отвън навътре", като изхвърляме висящите върхове и вземаме техните съседи, и така докато можем. Ако искаме да решим Максимална Антиклика, можем да походим дуално: пак "белим" отвън навътре, но сега вземаме висящите върхове, а изхвърляме съседите им.

Забележете, че идеята:

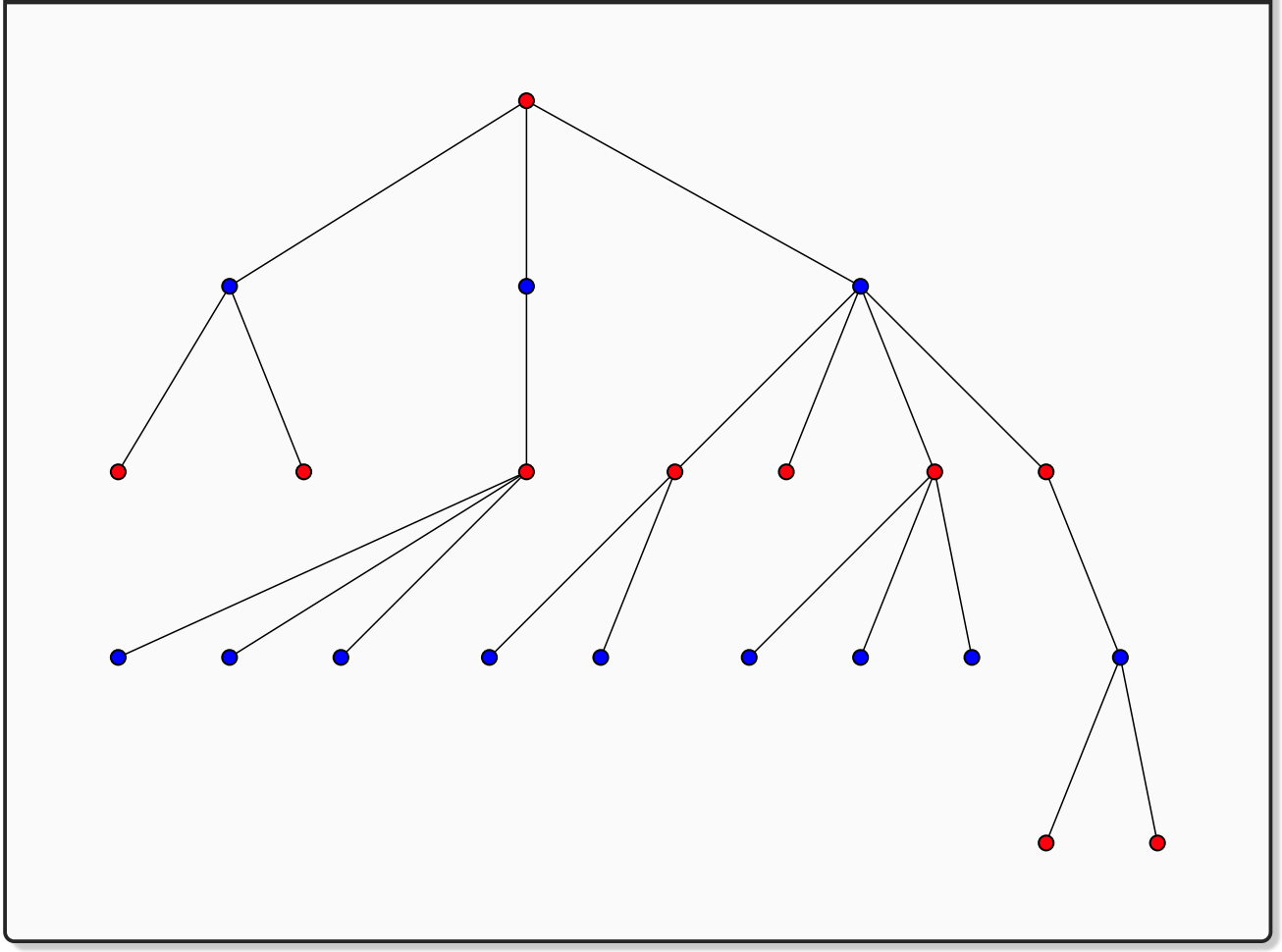
- да пуснем BFS[‡] върху дървото от кой да е връх s,
- да разбием множеството от върховете на $V(T)$ на две подмножества: V_e, тези на четно разстояние от s, и V_o, тези на нечетно разстояние от s,
- и да вземем като решение това от V_e, V_o, което е не по-голямо от другото,

не работи. Наистина, всяко от V_e и V_o е върхово покриване, понеже за всяко ребро (v, w), единият край е във V_e, а другият във V_o, но резултатът може да не е оптимален. Фигура 13.2 показва дърво, на което нито V_e (червените върхове), нито V_o (сините върхове) е минимално върхово покриване. Очевидно $|V_e| = 10$ и $|V_o| = 11$, $\min\{10, 11\} = 10$ и опитът за решение чрез редуване на слоеве спрямо корена дава решение с тегло 10. Но Фигура 13.3 показва същото дърво след изпълнението на алгоритъма, изграден по схемата динамично програмиране. Онзи пример е за дуалната задача Максимална Антиклика, но щом максималната антиклика е с големина 15, то минималното върхово покриване е с големина 7 (при положение, че броят на върховете е 22).

[†]Забележете, че изтриването на $L_1 \cup L_2$ може да доведе до несвързан граф.

[‡]Или DFS. Върху дървета, DFS също пресмята разстояния, защото всеки два върха са свързани с един единствен път.

Фигура 13.2 : Редуване на слоеве не дава мин. върхово покриване на дърво.



Ефекасно решение върху дървета Ето ефикасно решение за Максимална Антиклика върху дървета, изградено по схемата **Динамично Програмиране**. Решаваме дуалната задача Максимална Антиклика. Намираме само мощността на максимална антиклика, но с малка модификация, без да нарушаваме асимптотичната сложност по време, можем да получим и оптимално решение. Нека дървото е T. Първо правим дървото кореново, вкоренявайки в произволен връх r. За всеки връх $v \in V(T)$ поддържаме наредена двойка числа (v^+, v^-), където

- v^+ е максималната мощност на антиклика в $T[\![v]\!]$, която **съдържа** връх v,
- а v^- е максималната мощност на антиклика в $T[\![v]\!]$, която **не съдържа** връх v.

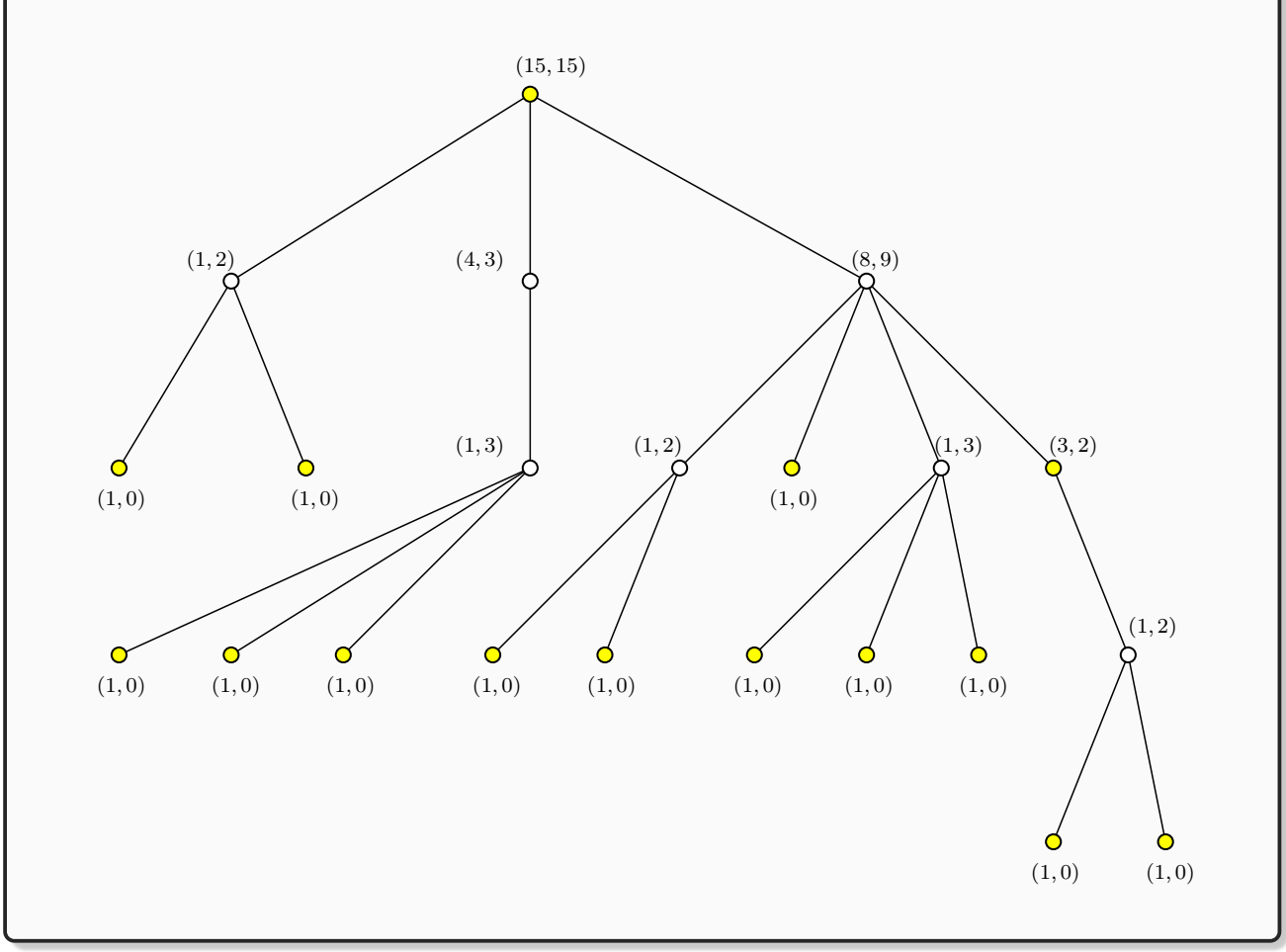
Ако v е листо, то очевидно $v^+ = 1$ и $v^- = 0$; това е дъното на рекурсията. Ако v е нелистo, то v има поне едно дете; в такъв случай (v^+, v^-) може да се пресметне ефикасно от наредените двойки на децата на v. В крайна сметка, отговорът е $\max\{r^+, r^-\}$, защото всяко оптимално решение за цялото дърво или съдържа, или не съдържа корена. А как да пресмятаме наредените двойки за не-листата? Ето как. Нека v има деца $w_1, \ldots, w_k$. Тогава

$$v^+ = 1 + \sum_{i=1}^{k} w_i^- \tag{13.28}$$

$$v^- = \sum_{i=1}^{k} \max\{w_i^-, w_i^+\} \tag{13.29}$$

Фигура 13.3 показва примерно кореново дърво с 22 върха и работата на алгоритъм, изграден върху рекурсивната декомпозиция (13.28) и (13.29). Наредената двойка $(15, 15)$ в корена означава, че има антиклика с размер 15, съдържаща корена, и антиклика с размер 15, несъдържаща корена.

Фигура 13.3 : Намиране на максимална антиклика в дърво с 22 върха.



Петнадесетте върха в жълто са оптимално решение, съдържащо корена. Очевидно размерът на минималното върхово покриване е $22 - 15 = 7$ и белите върхове са едно такова минимално върхово покриване.

Коректността на рекурсивната декомпозиция е очевидна. Сложността по време е линейна, предвид това, че алгоритъмът може да се имплементира като модифициран DFS с добавена само константа работа във всяко рекурсивно викане.

Алгоритъмът, изграден по схемата **Динамично Програмиране**, има значително предимство пред алчния алгоритъм. Този по **Динамично Програмиране** може да се модифицира тривиално така, че да решава Максимална Антиклика в тегловен вариант. Тук говорим за тегла на върховете, не на ребрата. Лесно може да си представим вариант на задачата, в който се иска антликлика с максимална сума от теглата (на своите върхове). Рекурсивната декомпозиция се променя много лесно: в базата и в рекурсивната част, наместо 1, в изчислението ще участва теглото на съответния връх. Докато алчният алгоритъм няма очевидно обобщение, което решава тегловния вариант на задачата.

13.8.2 Minimum Dominating Set on trees

Доминиращо множество, на английски *dominating set*, на (обикновен) граф G е всяко подмножество от върхове, такова че за всеки връх е вярно, че се намира в това множество или има съсед в него. Тук имаме предвид включващо "или". Задачата има многобройни приложения, например ако графът моделира пътна мрежа от градове-върхове и пътища-ребра, които свързват градове, и се иска да се намери малко множество градове, такова че всеки град да е съсед на някои от тези или самият той да е един от тях. Намирането на какво да е доминиращо множество е тривиално: очевидно $V(G)$ е доминиращо множество. Трудно е да се намери малко доминиращо множество. В примера с градовете, ако се иска да се построят болници, но не във всеки град (защото би било скъпо), а всеки град да има съседен град с болница или самият той да има болница, и да се намери минимално множество градове, с което това може да стане, всъщност се иска минимално доминиращо множество.

Дефинициите на "върхово покриване" и "доминиращо множество" си приличат повърхностно, но всъщност са доста различни. Като екстремен пример да разгледаме пълния граф K_n. Той няма върхово покриване с размер, по-малък от $n-1$, защото очевидно трябва да изтрием $n-1$ върха, за да не останат ребра. От друга страна, той има доминиращо множество с размер 1, защото всеки негов връх е съсед на всички останали. Изолираните върхове се "държат" различно спрямо тези две задачи: за минималното върхово покриване те са без значение (все едно ги няма), докато всяко доминиращо множество трябва да съдържа всички изолирани върхове, понеже изолиран връх не може да бъде доминиран от друг връх. Лесно се вижда, че при липса на изолирани върхове, всяко върхово покриване е и доминиращо множество, но конверсното не е вярно. Както вече видяхме, минималниите мощности на върхово покриване и доминиращо множество може да се различават с $\Theta(n)$.

Формалното определение е следното.

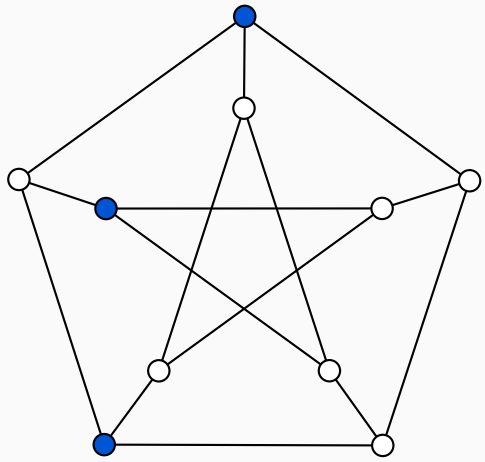
Изч. Задача: Minimum Dominating Set

пример: Неориентиран граф $G = (V, E)$.
решение: Подмножество $U \subset V$, такова че $\forall u \in V(u \in U \vee \exists v \in N(u)(v \in U)$ и $|U|$ е минимално.

Мощността на минимално по мощност доминиращо множество на G се бележи с $\gamma(G)$.

Пример за доминиращо множество е показан на Фигура 13.4. Графът на Petersen не може да бъде доминиран от по-малко от 3 върха: той е 3-регулярен граф, а във всеки 3-регулярен граф, един връх може да доминира най-много четири върха (сам себе си и трите си съседа), така че два върха не могат да доминират десетте върха на P. Ерго, 3 е долна граница за мощността на доминиращото множество и доминиращото множество на Фигура 13.1 е оптимално. И така, $\gamma(P) = 3$.

Фигура 13.4 : Минимално доминиращо множество в графа на Petersen.



Ефекасно решение върху дървета Ето ефикасно решение за Минимално Доминиращо Множество върху дървета, изградено по схемата **Динамично Програмиране**. Тъй като обосновката на рекурсията е значително по-неочевидна от тази на Максимална Антиклика върху дървета, ще я направим бавно и внимателно, като ще ще получим оптимално решение, а не просто цена на оптимално решение, както беше в (13.28) и (13.29). За целта ни трябва едно понятие: функцията argmin. На английски терминът е "*argument minimum*". Да разгледаме някаква функция $f : X \to \mathbb{R}$, където X е произволен домейн. Да допуснем, че f има и минимум, и максимум върху X. Пишейки "$\min\{f(x) \mid x \in X\}$", имаме предвид минималната функционална стойност върху множеството X. Ако обаче искаме да изразим това x, за което се достига минималната функционална стойност, този израз не върши работа. Прието е да се пише

$$\operatorname{argmin}\{f(x) \mid x \in X\}$$

което изразява точно това: елементът $x \in X$, за който f има минимална стойност. "argmax" се дефинира напълно аналогично.

За удобство да въведем следните нотации. Нека H е произволно кореново дърво. "$D(H)$" означава оптимално доминиращо множество за H. "$D^+(H)$" означава оптимално доминиращо множество за H, което съдържа корена. "$D^-(H)$" означава оптимално доминиращо множество за H, което не съдържа корена.

Очевидно

$$D(H) = \operatorname{argmin}\{|D^+(H)|, |D^-(H)|\} \tag{13.30}$$

Нека дървото, върху което решавам задачата, е $T = (V, E)$. БОО, нека T има повече от един връх. Правим T кореново дърво, вкоренявайки го в произволен връх r. Нека децата на r са w_1, $\ldots$, w_k. Започваме да разсъждаваме по начина, по който разсъждавахме за Максимална Антиклика върху дървета. Търсим $D(T)$. Коренът r или е в $D(T)$, или не е в $D(T)$.

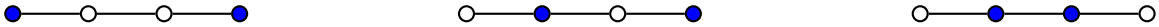
- Да допуснем, че $r \in D(T)$. Търсим $D^+(T)$. Очевидно $\{r\} \cup \bigcup_{i=1}^{k} D(T[\![w_i]\!])$ е доминиращо множество за дървото T, съдържащо корена. Но дали е оптимално решение за T? Не

непременно. Това, че $\mathtt{r}$ задължително участва в доминиращо множество ни дава свободата за всяко $\mathtt{T}[\![w_\mathtt{i}]\!]$ да вземаме множество, което доминира всеки връх, **може би без** $w_\mathtt{i}$. Причината е проста: всички $w_\mathtt{i}$ биват доминирани от $\mathtt{r}$ така или иначе, ерго, може да си позволим за поддърветата, вкоренени в децата на $\mathtt{r}$, да вземаме "почти доминиращи" множества – такива, в които коренът (той е дете на $\mathtt{r}$) може да не е доминиран.

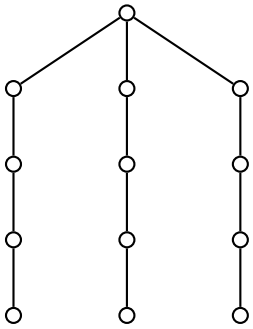
Ето прост пример. Първо забелязваме, че прост път с дължина 3 не може да бъде доминиран от един връх: който и връх да сложим в доминиращото множество, поне един връх остава недоминиран. БОО, достатъчно е да разгледаме следните два случая. Върхът, чрез който опитваме да доминираме пътя, е в синьо. Върховете, които остават недоминирани, са в сиво.



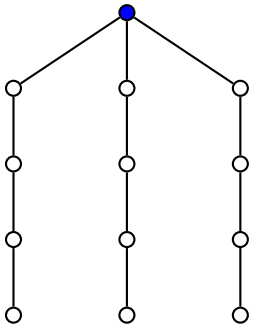
Второ, забелязваме, че два върха могат да доминират пътя по някой от тези три начина:



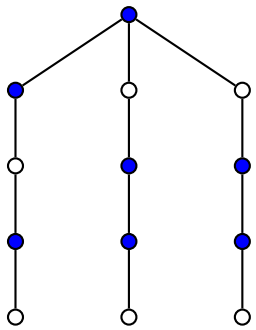
Трето, конструираме самия пример. Както обикновено, коренът е нарисуван най-горе.



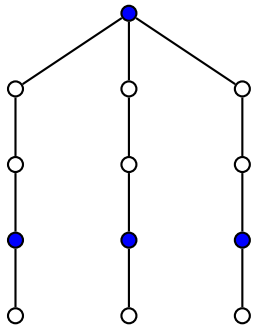
Разглеждаме случая, в който коренът е в доминиращото множество.



Поддърветата, вкоренени в децата на корена, са пътища с дължина 3. Както вече видяхме, всяко от тези поддървета има число на доминиране 2. Ако вземем решение за цялото дърво, което се състои от корена и по едно решение за всяко дете, получаваме решение с мощност 7, като например това:



Всъщност, има решение с мощност 4, съдържащо корена:



Това е краят на примера. Видяхме, че, ако коренът е в решението, за поддърветата, вкоренени в децата, можем да си позволим решения, които не доминират техните корени, понеже същите се оказват доминирани от корена на цялото дърво. Накратко, **не е непременно вярно**, че $|D^+(T)| = 1 + \sum_{i=1}^{k} |D(T[\![w_i]\!])|$, поради което $\{r\} \cup \bigcup_{i=1}^{k} D(T[\![w_i]\!])$ не е непременно оптимално решение, съдържащо корена..

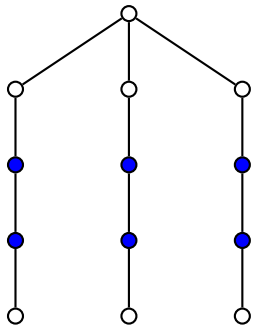
Ще въведем още една нотация. Нека H е произволно кореново дърво. "$\widetilde{D}(H)$" означава множество с минимална мощност, което доминира всеки връх на H, може би с изключение на корена.

Забележете, че $|\widetilde{D}(H)| \leqslant |D(H)|$, като е възможно строго неравенство, пример за което видяхме преди малко (дървото е път, вкоренен в единия си край).

И така,

$$D^+(T) = \{r\} \cup \bigcup_{i=1}^{k} \widetilde{D}(T[\![w_i]\!]) \tag{13.31}$$

- Да допуснем, че $r \notin D(T)$. Човек би се изкушил да каже, че решението е $\bigcup_{i=1}^{k} D(T[\![w_i]\!])$, но това множество може да остави корена недоминиран, както се вижда от следния пример.



Коренът може да бъде доминиран само от някое от децата си, при положение, че той самият не е в доминиращото множество. Ерго, налага се да искаме поне едно от децата на корена да е задължително в доминиращото множество. Тогава ще ползваме D^+ за точно едно w_i и D за останалите деца. А за кое i да ползваме $\mathsf{D}^+(\mathsf{T}[\![w_i]\!])$? За това, за което получаваме оптимално решение. Накратко,

$$\mathsf{D}^-(\mathsf{T}) = \operatorname{argmin}\left\{\left|\mathsf{D}^+(\mathsf{T}[\![w_i]\!]) \cup \bigcup_{j\in\{1,\ldots,k\}\setminus\{i\}} \mathsf{D}(\mathsf{T}[\![w_j]\!])\right| \;:\; 1 \leqslant i \leqslant k\right\} \tag{13.32}$$

Получихме изрази за $\mathsf{D}^+(\mathsf{T})$ и $\mathsf{D}^-(\mathsf{T})$, съответно (13.31) и (13.32), но с цената на въвеждането на $\widetilde{\mathsf{D}}(\mathsf{T})$. Налага са да изразим и него. Припомняме си, че $\widetilde{\mathsf{D}}(\mathsf{T})$ е минимално по мощност множество от върхове, което доминира дървото, но може би с изключение на корена. Фразата "може би" е ключова, както ще видим.

От общи съображение, човек би казал, че $\widetilde{\mathsf{D}}(\mathsf{T}) = \bigcup_{i=1}^{k} \mathsf{D}(\mathsf{T}[\![w_i]\!])$. И би сбъркал. Със сигурност множеството $\bigcup_{i=1}^{k} \mathsf{D}(\mathsf{T}[\![w_i]\!])$ доминира всички върхове, евентуално без корена. Обаче това множество може да не е оптимално! То е оптимално, ако коренът **не е** в решението. Но нищо не забранява коренът **да е** в решението. Разглеждаме множество $\widetilde{\mathsf{D}}(\mathsf{T})$, което съдържа корена. Това, че коренът се съдържа, ни "развързва ръцете" за децата му $w_1, \ldots, w_k$ да вземаме не D, а $\widetilde{\mathsf{D}}$. С други думи, ако коренът е в $\widetilde{\mathsf{D}}(\mathsf{T})$, можем да си позволим за поддърветата, вкоренени в децата, да вземаме решения, които може да не съдържат техните корени. Ето пример за това.

- Вляво е минимално решение, което не съдържа корена, но поддърветата, вкоренени в децата на корена, са доминирани напълно от подрешенията, индуцирани от поддърветата, вкоренени в децата на корена.
- Вдясно е минимално решение, което съдържа корена, и в което поддърветата, вкоренени в децата на корена, не са напълно доминирани от подрешенията, индуцирани от поддърветата, вкоренени в децата на корена.



Очевидно дясното решение, което има мощност 4, е по-добро от лявото решение, което има мощност 6. И за двете решения имаме право да кажем, че са решения, в които коренът (на цялото дърво) не е непременно доминиран. Това, че вдясно той е доминиран, и не просто е доминиран, а се намира в решението, не представлява противоречие.

И така,

$$\widetilde{\mathsf{D}}(\mathsf{T}) = \operatorname{argmin}\left\{\left|\bigcup_{i=1}^{k} \mathsf{D}(\mathsf{T}[\![w_i]\!])\right|, \left|\{r\} \cup \bigcup_{i=1}^{k} \widetilde{\mathsf{D}}(\mathsf{T}[\![w_i]\!])\right|\right\} \tag{13.33}$$

Съставихме четири уравнения с взаимна рекурсия. Да напишем цялата система (засега изпускаме началните условия). Ще използваме малко променена нотация: ако кореновото дърво е H, с "$c(H)$" означаваме множеството от децата на корена.

$$D(T) = \operatorname{argmin}\{|D^+(T)|, |D^-(T)|\}$$
$$D^+(T) = \{r(T)\} \cup \bigcup_{w \in c(T)} \widetilde{D}(T[\![w]\!])$$
$$D^-(T) = \operatorname{argmin}\left\{ \left| D^+(T[\![w]\!]) \cup \bigcup_{z \in c(H)\setminus\{w\}} D(T[\![w]\!]) \right| : w \in c(H) \right\}$$
$$\widetilde{D}(T) = \operatorname{argmin}\left\{ \left| \bigcup_{i=1}^{k} D(T[\![w_i]\!]) \right|, \left| \{r\} \cup \bigcup_{i=1}^{k} \widetilde{D}(T[\![w_i]\!]) \right| \right\}$$

Тази система може да се опрости значително. D се изразява чрез D^+ и D^-, но

- D^+ се изразява чрез $\widetilde{D}$,
- D^- се изразява чрез D^+ и D,
- $\widetilde{D}$ се изразява чрез D и $\widetilde{D}$.

Очевидно можем да изразим всичко само чрез D и $\widetilde{D}$ по този начин:

$$D(T) = \operatorname{argmin}\left\{ \left| \{r\} \cup \bigcup_{w \in c(T)} \widetilde{D}(T[\![w]\!]) \right|, \right. \tag{13.34}$$
$$\left. \operatorname{argmin}\left\{ \left| \{w\} \cup \bigcup_{z \in c(T[\![w]\!])} \widetilde{D}(T[\![z]\!]) \cup \bigcup_{z \in c(H)\setminus\{w\}} D(T[\![w]\!]) \right| : w \in c(H) \right\} \right\}$$
$$\widetilde{D}(T) = \operatorname{argmin}\left\{ \left| \bigcup_{w \in c(T)} D(T[\![w]\!]) \right|, \left| \{r\} \cup \bigcup_{w \in c(T)} \widetilde{D}(T[\![w]\!]) \right| \right\} \tag{13.35}$$

С червено и синьо е показано как D^+ и D^- биват заместени, за да изчезнат и да останат само D и $\widetilde{D}$.

В светлината на опростените изрази, искаме началните условия само за D и $\widetilde{D}$. И така, ако $T = (\{r\}, \varnothing)$, то

$$D(T) = r, \widetilde{D}(T) = \varnothing \tag{13.36}$$

В Интернет има добре известно решение за Dominating Set върху дървета, което дава само стойността на оптимално доминиращо множество, а не самото самото множество. Това решение е горе-долу следното.

DS on trees($T = (V, E)$: дърво)
1 $A[1, \ldots, n]$, $B[1, \ldots, n]$: масиви от цели числа
2 нека r е произволен връх от V
3 направи T кореново дърво с корен r
4 работи от листата нагоре по следния начин

```
5       за всяко листо u
6           A[u] = 1
7           B[u] = 0
8       за всяко не-листо u
9           A[u] = 1 + min { Σ_{v∈c(u)} B[v], min_{v∈c(u)} { Σ_{w∈c(v)} B[w] + Σ_{z∈c(u)\{v}} A[z] } }
10          B[u] = min { 1 + Σ_{v∈c(u)} B[v], Σ_{v∈c(u)} A[v] }
11  return A[r]
```

$A[u]$ има смисъл на минимално доминиращо множество в поддървото, вкоренено в u. $B[u]$ има смисъл на множество с минимална мощност в поддървото, вкоренено в u, което множество доминира всеки връх, може би с изключение на u.

Забележете, че (13.34) и (13.35) са същото нещо, само че те дават множества, а не числа. По-точно, 9 отговаря на (13.34), а 10 отговаря на (13.35). Това, че 10 отговаря на (13.35), е очевидно. Че 9 отговаря на (13.34) може би не е толкова очевидно; за да видите съответствието, забележете, че множествата $\{r\}$ и $\{w\}$ дават по едно $+1$, когато минем от изрази за множества към изрази за мощности на множества, и тези две $+1$ може да бъдат изнесени пред min, както става на ред 9.

Задачата във вариант с тегла Рекурсивната декомпозиция (13.34), (13.35) и (13.36) може да се ползва и за тегловния вариант на задачата Минимално Доминиращо Множество върху дървета. В тегловния вариант, върховете (а не ребрата) имат тегла, и решение е множество върхове, което доминира графа, като сумата от теглата на елементите му е минимална. Решението е практически същото, с малката разлика, че в argmin функциите трябва да са не мощности на множества, а сумите от теглата на елементите на тези множества, а в началните условия, $D(T)$ е теглото на единствения връх.

13.9 Игри

В тази секция разглеждаме игри, в които играят два двама противника, които се редуват. На английски такава игра се нарича *turn based game*. Много настолни игри са такива, например игрите табла, шах и Го. Играта морски шах от Допълнение 38 също е turn based game, въпреки че не е настолна игра и при нея всеки играч разполага само с частична информация, за разлика от шаха, таблата и играта Го, в които всеки от играчите вижда цялата позиция[†].

Подчертаваме, че не става дума за компютърни игри, които често не са turn based, понеже играчите играят едновременно.

Да кажем, че противниците се наричат Албена и Борис, като Албена играе първа. Кой играе последен или последна обикновено зависи от развитието на играта и не е дефинирано в правилата.

Какво означава да се спечели играта зависи от правилата. В някои игри печалбата е "булева": единият играч печели, а другият губи, и няма други възможности. Такава е The Daisy Petals

[†]Авторът не се наема да каже дали настолни игри с много играчи като монопол, или игрите с карти, са turn based games, но това няма значение за тази лекция.

Game (Подсекция 13.9.1), в която все някой ще откъсне последен или последна.. Тези игри приличат на задачите за разпознаване (Определение 2).

В по-сложни игри като шаха или единият печели и другият губи, или завършват наравно[†]. В някакъв смисъл може да кажем, че и шахът прилича на задача за разпознаване, защото за конкретна позиция има смисъл да зададем въпроса с булев отговор "Има ли печеливша стратегия за белите в n хода?".

Има обаче игри, в които печалбата има числено изражение и те приличат на оптимизационните задачи (Определение 4). Пример за такава игра е играта със събирането на монети в Подсекция 13.9.2. В тях въпросът не е **дали** даден играч печели или не, а **колко** най-много може да спечели, ако играта е максимизационна, или колко най-малко може да спечели, ако играта е минимизационна.

От особена важност е да дефинираме какво означава "оптимална стратегия" на даден играч. Оптималната стратегия, както и името подсказва, е набор от правила (може да кажем, че е следване на алгоритъм), при които даден играч постига възможно най-много **дори при най-добра игра на противника**. Авторът се изкушава да каже "**за всяка игра на противника**", но това може е подвеждащо.

Да кажем, че играта е "булева" и равенство е невъзможно, като в играта с венчелистчетата. Тогава "оптимална стратегия" за даден играч се дефинира, ако за нея или него винаги има начин да спечели, както и да играе противникът. Още може да кажем "печели задължително". Това има точно определен смисъл, който може да се изрази чрез езика на предикатната логика, редувайки екзистенциални и универсални квантори. Албена печели задължително тстк

- съществува ход на Албена, такъв че
- за всеки ход на Борис
- съществува ход на Албена, такъв че
- за всеки ход на Борис
- ...
- за всеки ход на Борис
- съществува ход на Албена, с който тя печели.

Вторият играч Борис печели задължително тстк

- за всеки ход на Албена
- съществува ход на Борис, такъв че
- за всеки ход на Албена
- съществува ход на Борис, такъв че

[†]В шаха може противниците да се съгласят на реми, може да се стигне до пат, може да се стигне до трикратно повторение на позиция или поредица от 50 хода, в които не е местена пешка и не е взета фигура, може и двете страни да останат с фигури, които теоретично не могат да дадат мат, например цар и два коня. В някои разновидности на таблата е възможно, макар и много малко вероятно, равенство; авторът има предвид тапата, но може да има и други видове табла, в които е възможно равенство. В играта Го също може да се стигне до равенство.

- ...
- за всеки ход на Албена
- съществува ход на Борис, с който Борис печели.

Очевидно второто се явява негация на първото. Оттам и тези двете са несъвместими: няма как Албена да печели задължително и Борис да печели задължително. В играта с пълна маргаритка, както ще видим, Борис печели задължително. Може също да кажем, че Албена губи задължително (отново: не абсолютно винаги, а при оптимална игра на Борис).

Ако даден играч губи задължително при оптимална игра на противника—какъвто е случаят с Албена в играта с пълна маргаритка—няма смисъл да говорим за оптимална статегия за него или нея.

При игри с количествен краен резултат, каквато е играта със събирането на монети, "оптимална стратегия" за даден играч означава нещо различно, а именно колко най-много[†] може да реализира даден играч при оптимална игра на противника. При тези игри има има смисъл да се говори за оптимална стратегия и за двата противника.

13.9.1 The Daisy Petals Game

Дадена е маргаритка като тази:

За краткост да наречем играчите А и Б. Играчите се редуват да късат едно или две венчелистчета, но ако са две, задължително трябва да са съседни. Печели този, който откъсне последен. Ето едно възможно развитие на играта, при което първият печели (отляво надясно):

А къса 2, Б на ход



Б къса 1, А на ход



А къса 1, Б на ход



Б къса 1, А на ход



А къса 2, Б на ход



Б къса 1, А на ход



А къса 2, Б на ход



Б къса 1, А на ход



А къса 1, А печели

Задачата е да се каже дали А печели задължително, или Б печели задължително.

[†]"Колко най-много" казваме, когато целта на играта е максимизация; ако целта беше минимизация, щеше да е "колко най-малко".

Лесният вариант: "пълна" маргаритка Доколкото е известно на автора на тези записки, задачата е популярна във варианта, в който се започва от маргаритка, от която не е късано нищо ("пълна" маргаритка).

Ако броят на венчелистчетата е четен и по-голям от 2, както е в примера, който видяхме (бяха 12 поначало), има елементарна стратегия за Б, с която Б печели задължително: след всеки ход на А,

- ако А е откъснала едно венчелистче, то Б къса противоположното венчелистче,
- ако А е откъснала две венчелистчета, то Б къса двете противоположни венчелистчета.

В някакъв смисъл, Б играе "огледално" на А. Гарантирано Б откъсва последно.

И така, ако стартираме от пълна маргаритка с четен брой венчелистчета, поне 4, при оптимална игра вторият играч печели задължително. В примера, който видяхме, А спечели заради грешки на Б.

По-труден вариант: "непълна" маргаритка Възниква въпрос, ако началната конфигурация не е "пълна" маргаритка, а някакъв общ случай, в който се допуска някои венчелистчета да липсват, примерно така: , дали А печели задължително, или Б печели задължително?

Сега началната конфигурация се описва не просто с броя на венчелистчетата. Сега има значение и как са групирани заедно; тоест, къде са празнините. Колко са големи празнините (тоест, във всяка празнина колко венчелистчета е имало поначало) няма значение. Важното е колко групи (максимални по включване области без липсващи венчелистчета) има и колко венчелистчета има във всяка група. В последния пример групите са 3 и могат да бъдат описани с мултимножеството $\{2, 2, 3\}_{\mathsf{M}}$ – очевидно е, че няма значение точно как са разположени една спрямо друга. БОО да допуснем, че елементите на мултимножеството са написани отляво надясно в ненарастващ ред и тогава можем да го опишем като вектор $\langle 3, 2, 2\rangle$. И така, всяка конфигурация е вектор от цели положителни числа в ненарастващ ред.

За всяка конфигурация да въведем понятията *печеливша* и *губеща*. Дадена конфигурация, тоест, вектор от цели положителни числа, е печеливша тстк А, стартирайки от нея, печели задължително; в противен случай е губеща. Ключовото наблюдение е следното: конфигурация C е печеливша тстк

- или една от $\langle 1\rangle$ или $\langle 2\rangle$, при което А печели директно с един ход,
- или има ход на А, от който се получава конфигурация C', която е губеща за Б; тук А печели индиректно, понеже Б е длъжен да играе.

Губеща конфигурация е, примерно, $\langle 1, 1\rangle$. Картинно, конфигурацията може да бъде нарисувана така или така  , това няма значение. Очевидно е, че този, който е на ход, може да направи само едно нещо—да откъсне единственото венчелистче от едната група—след което другият печели директно.

Конфигурациите $\langle 1\rangle$ и $\langle 2\rangle$ са базовите конфигурации. Както отбелязахме, те са печеливши. Всяка конфигурация C с общо $\mathtt{n}$ венчелистчета се получава

- или от конфигурация с C_1 с общо $n+1$ венчелистчета чрез откъсване на венчелистче,
- или от конфигурация с C_2 с общо $n+2$ венчелистчета чрез откъсване на две съседни венчелистчета.

Ако мислим в обратната посока, конфигурация C с n венчелистчета е печеливша тстк съществува конфигурация C', която се получава от C чрез откъсване на едно или две венчелистчета, такава че C' е губеща[†].

На практика вече имаме рекурсивна декомпозиция по броя на венчелистчетата. Нека n е броят на венчелистчетата в задачата. Можем да пресметнем рекурсията по схемата **Динамично Програмиране**, като систематично, за $k = 1, 2, \ldots, n$ намерим всички конфигурации с k венчелистчета и за всяка от тях, която не е базова, пресметнем дали е печеливша или губеща от конфигурациите с $k-1$ и $k-2$ венчелистчета. Всяка конфигурация, за дадена стойност на k, идентифицираме с вектор от цели положителни числа, в ненарастващ ред, сумиращи се до k.

Има една особеност. За някои n се налага да различаваме маргаритка с n венчелистчета, от която нищо не е късано, от маргаритка с една единствена група от n венчелистчета, от която обаче е късано. И в двата случая векторът е $\langle n \rangle$, но е възможно едната конфигурация да е печеливша, а другата конфигурация да е губеща. Като пример при $n = 3$:

- тази конфигурация  е губеща, понеже каквото и да откъсне А, след това Б откъсва останалите едно или две венчелистчета и печели,
- а тази конфигурация е печеливша, защото А трябва само да откъсне средното венчелистче, за да "вкара" Б в губещата конфигурация .

И двете конфигурации се описват от вектора $\langle 3 \rangle$. При вектори с една единствена компонента се налага да различаваме някак ситуация с нула "празнини" (маргаритка, от която не е късано) от ситуация с една "празнина" (има късани венчелистчета).

За едно или две венчелистчета няма значение дали има празнина, или няма[‡]. За $n = 3$, както видяхме, е губеща, а е печеливша. За да различаваме някак $\langle 3 \rangle$ в двата случая, ако конфигурацията е без празнина, ще пишем $\langle \widetilde{n} \rangle$, примерно $\langle \widetilde{3} \rangle$ описва , докато при точно една празнина ще пишем $\langle n \rangle$, примерно $\langle 3 \rangle$ описва .

Изключвайки тази особеност, ясно е, че всяка конфигурация на n венчелистчета е едно целочислено разбиване на n (Определение 37), така че можем да мислим в термините на

[†]Логическата негация на последното е, конфигурация C с n венчелистчета е губеща тстк за всяка конфигурация C', която се получава от C чрез откъсване на едно или две венчелистчета, C' е печеливша.

[‡]Трудно е да си представим цвете, което има точно едно венчелистче по природа, но математически може да разгледаме и такова.

целочислени разбивания. Вместо да говорим за конфигурации от венчелистчета, ще говорим за целочислени разбивания. Можем да изградим триъгълна таблица с информация за разбиванията до каквато стойнст на n пожелаем. Таблица 13.5 показва разбиванията до $n = 7$. Тези, които са написани със зелен цвят, са печеливши, а тези с червен цвят са губещи. За по-компактен запис не ползваме никакъв разделител между компонентите на дадено разбиване. Тъй като числата са едноцифрени, двусмислица не може да има.

n	целочислените разбивания на n															
1	$\widetilde{1}$	1														
2	$\widetilde{2}$	2	11													
3	$\widetilde{3}$	3	21	111												
4	$\widetilde{4}$	4	31	22	211	1111										
5	$\widetilde{5}$	5	41	32	311	221	2111	11111								
6	$\widetilde{6}$	6	51	42	33	411	321	222	2211	3111	21111	111111				
7	$\widetilde{7}$	7	61	52	43	511	421	331	322	4111	3211	2221	31111	22111	211111	1111111

Таблица 13.5: Печеливши и губещи целочислени разбивания.

Тъй като при разглеждането на задачата от произволна начална конфигурация започнахме с примера , да го разгледаме докрай. Това е 322, което е печеливша конфигурация съгласно таблицата. Наистина, от нея с откъсване на едно венчелистче получаваме , която е 321. Това е губеща конфигурация, тъй като от нея стигаме до, и само до, следните конфигурации, всяка от които е печеливша. Ето достижимите с едно късане конфигурации:

- , или 32, е печеливша,
- , или 311, е печеливша,
- , или 31, е печеливша,
- , или 221, е печеливша,
- , или 211, е печеливша,

- , или 2111, е печеливша.

Би било прекалено голямо усилие да разгледаме всяка от тях, така че да видим само , или 32. Тя наистина е печеливша, защото с едно късане става , или 22. А 22 на свой ред наистина е губеща, защото от нея може да се отиде единствено в очевидно печелившите 2 и 21.

Лесно може да се дефинират с математическа прецизност обръщанията към предни редове в таблицата – за всеки вектор със сума $\mathtt{n}$, точно тези вектори със сума $\mathtt{n}-1$ или $\mathtt{n}-2$, които може да се получат от него. По този начин таблицата може да бъде попълвана систематично.

Забележете връзката между Таблица 13.5 и Таблица 13.4. В някакъв смисъл, Таблица 13.4 е кондензиран вариант на Таблица 13.5: ако игнорираме колоната с тилдите в Таблица 13.5, останалата част на Таблица 13.5 съдържа самите целочислени разбивания, чиито бройки се съдържат в клетките на Таблица 13.4. Примерно, да разгледаме реда с $\mathtt{n}=7$ в Таблица 13.4. Сумата от числата в него е $1+3+4+3+2+1+1=15$, и наистина в Таблица 13.5 има петнадесет целочислени разбивания за $\mathtt{n}=7$, експлицитно посочени. От тези петнадесет целочислени разбивания на седмицата, точно три имат по точно две части (това са 61, 52 и 43), а от друга страна в Таблица 13.4, елементът, отговарящ на $\mathtt{n}=7$ и $\mathtt{k}=2$, е 3. И така нататък.

Асимптотично говорейки, този алгоритъм за решаване на задачата с маргаритките не е ефикасен. Броят на целочислените разбивания на $\mathtt{n}$ расте *доста бързо*, като асимптотиката е, грубо казано, експонента от $\sqrt{\mathtt{n}}$. Но за разумни стойности на $\mathtt{n}$ нещата не са толкова черни: за $\mathtt{n}=20,40,60$ бройките са съответно 627, 37 338 и 966 467[†]. *Известни са* ефикасни алгоритми за генерирането на самите целочислени разбивания, които вършат работа само $\Theta(1)$ за разбиване, в амортизирания смисъл. За маргаритки, които се срещат в реалния свят, този подход води до успех.

13.9.2 The Coins in a Line Game

Дадени са $\mathtt{n}$ монети, разположени една до друга в редица. $\mathtt{n}$ е четно число. Монетите са от най-различни деноминации. За целите на задачата, деноминациите са произволни цели положителни числа. Всеки играч, когато е неговият или нейният ред, взема точно една монета или от левия, или от десния край на редицата. Играта продължава до изчерпването на монетите в редицата (което означава, че монетите са разпределени между играчите). След играта всеки задържа монетите, които е събрал(а). Иска се ефикасен алгоритъм, който намира максималната възможна сума за Албена.

Може да мислим за състоянието в даден момент от играта като за вектор от цели положителни числа. Тук наредбата е от ключово значение, така че не може да я пренебрегнем и да разглеждаме мултимножеството от числата. Да кажем, че началната наредба е $\langle 1,2,100,2\rangle$.

[†]С други думи, ако продължим Таблица 13.5 надолу, дължината на $\mathtt{n}$-ия ред ще е от порядъка на експонента от $\sqrt{\mathtt{n}}$, като, примерно, за $\mathtt{n}=60$, редът ще има 966 467 колони за самите целочислени разбивания плюс още една за $\widetilde{60}$.

Съгласно правилата, за Албена са достъпни само единицата вляво и двойката вдясно, така че нейният избор е само между тези две монети. Очевидната алчна стратегия "избери тази от двете достъпни монети, която е не по-малка от другата", не работи; ако следва тази стратегия, Албена ще вземе двойката вдясно, конфигурацията ще стане $\langle 1, 2, 100\rangle$ и Борис ще вземе стотицата. Забележете, че при начална конфигурация $\langle 1, 2, 100, 2\rangle$, Албена може да вземе стотицата, ако първо вземе единицата вдясно, с което Борис получава $\langle 2, 100, 2\rangle$ и, тъй като е длъжен да играе, взема една от двойките, предоставяйки на Албена или $\langle 100, 2\rangle$, или $\langle 2, 100\rangle$, при което тя взема стотицата.

И така, очевидната алчна стратегия не работи. Има и по-изтънчена алчна идея. Да си припомним, че n е четно. Да разбием монетите на тези, които са четни позиции, и тези които са на нечетни. Условно да ги наречем "червените" и "сините". Примерно,

$$1, 3, 6, 3, 1, 3$$

Играейки първа, Албена лесно може да събере монетите от даден цвят (и само тях). Така че тя може първо да сметне кое множество има не по-малка сума от другото и след това да събере монетите му, принуждавайки Борис да събере останалите. Тази идея гарантира на Албена, че ще вземе не по-малко от Борис, но не ѝ гарантира, че ще вземе максимална сума. В примера: и червеното множество има сума 8, а синьото, 9, така че Албена ще вземе монети на стойност 9, следвайки тази идея. Обаче максималната печалба за Албена всъщност е 10. Да се убедим в това. Албена започва с тройката вдясно, оставяйки на Борис избор от две единици:

$$1, 3, 6, 3, 1$$

Която и единица да вземе Борис, Албена взема другата единица, след което Борис трябва да избира между две тройки:

$$3, 6, 3$$

Която и тройка да вземе Борис, Албена взема шестицата, с което тя взема общо $3+1+6 = 10$.

Има ефикасен алгоритъм по схемата **Динамично Програмиране**, който гарантира максимална печалба за Албена. Да кажем, че $\mathrm{opt}(i, j)$ е оптималната сума, която Албена може да събере, ако има пред себе си подредицата от i-тата монета включително до j-тата монета включително, за $1 \leqslant i < j \leqslant n$. Забележете, че няма смисъл да допускаме $i = j$, защото при четно n, Албена в своя последен ход играе върху редица от две монети. Последното вземане задължително се прави от Борис, докато алгоритъмът, който строим, е за Албена.

Ако Албена може да изчислява тази функция ефикасно, тя разполага и с ефикасно решение, защото ѝ трябва $\mathrm{opt}(1, n)$. Тук разглеждаме само намирането на оптималната печалба, но от това решение лесно може да получим решение, което дава и самите оптимални ходове.

Ще измислим подходяща рекурсия за изчислението на $\mathrm{opt}(i, j)$ чрез някакви $\mathrm{opt}(i', j')$, където $j' - i' < j - i$. С други думи, управляващият параметър е разликата между втория и първия аргумент. Тази функция ще се изчислява чрез запълване на (част от) двумерен масив $\mathrm{opt}[1, \ldots, n][1, \ldots, n]$. Решението е $\mathrm{opt}[1][n]$.

Нека масив $M[1, \ldots, n]$ съдържа редицата от монети; тоест, $M[i]$ е стойността на i-тата монета отляво, за $1 \leqslant i \leqslant n$.

Дъното на рекурсията се реализира чрез

$$\mathrm{opt}[i, i+1] = \max\{M[i], M[i+1]\}$$

за $1 \leqslant i < n$. Идеята е проста: ако Албена играе върху двуелементната редица от монети $M[i], M[i+1]$, тя взема тази, която е не по-малка от другата, после Борис взема другата и играта приключва.

Същината е изчислението на $\mathrm{opt}[i,j]$ за $j > i+1$. Албена избира дали да вземе $M[i]$, или да вземе $M[j]$. Други възможности няма.

- Ако Албена вземе $M[i]$, на ход е Борис, който или която избира между $M[i+1]$ и $M[j]$. Тъй като допускаме, че Борис играе оптимално, той ще направи избора, който му дава максимална печалба, и за Албена ще остане по-лошото (по-малкото) от $\mathrm{opt}[i+1, j-1]$ и $\mathrm{opt}[i+2, j]$. Така че Албена ще вземе най-много, при оптимална игра на Борис,

$$M[i] + \min\{\mathrm{opt}[i+1, j-1], \mathrm{opt}[i+2, j]\}$$

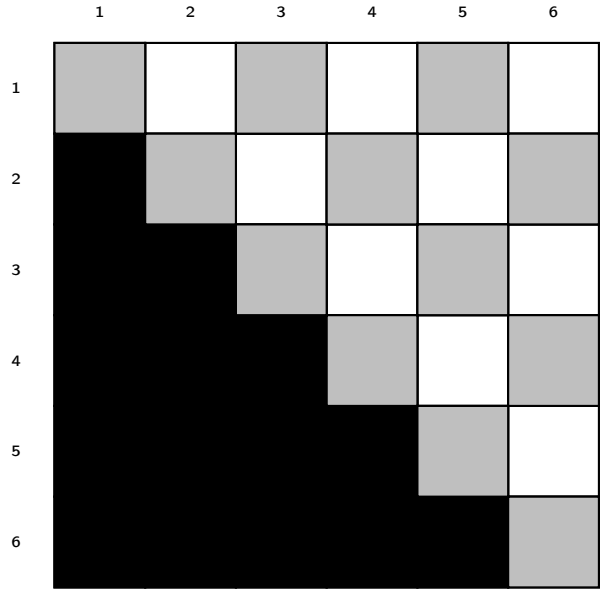
- Ако Албена вземе $M[j]$, на ход е Борис, който избира между $M[i]$ и $M[j-1]$. Тъй като допускаме, че Борис играе оптимално, той ще направи изборът, който му дава максимална печалба, и за Албена ще остане по-лошото от $\mathrm{opt}[i, j-2]$ и $\mathrm{opt}[i+1, j-1]$. Така че Албена ще вземе най-много, при оптимална игра на Борис,

$$M[j] + \min\{\mathrm{opt}[i, j-2], \mathrm{opt}[i+1, j-1]\}$$

Албена трябва да избере максимума от тези. И така:

$$\mathrm{opt}[i,j] = \max\left\{M[i] + \min\{\mathrm{opt}[i+1, j-1], \mathrm{opt}[i+2, j]\}, M[j] + \min\{\mathrm{opt}[i, j-2], \mathrm{opt}[i+1, j-1]\}\right\}$$

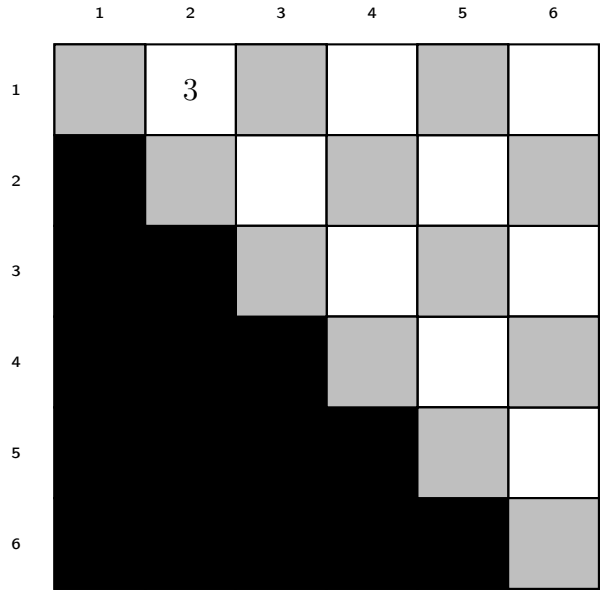
Да илюстрираме работата на алгоритъма, изграден върху тази рекурсивна декомпозиция. Нека входът отново е $\langle 1, 3, 6, 3, 1, 3\rangle$. Работната матрица е квадратна, като клетките под главния диагонал не се ползват по очевидни причини (те са в черно на илюстрацията), а в останалата част се ползват само клетките $[i,j]$, за които $j-i$ е нечетно; тези, за които $j-i$ е четно (те са в сиво на илюстрацията), отговарят на ходове на Борис, които са нерелевантни за алгоритъма за Албена.



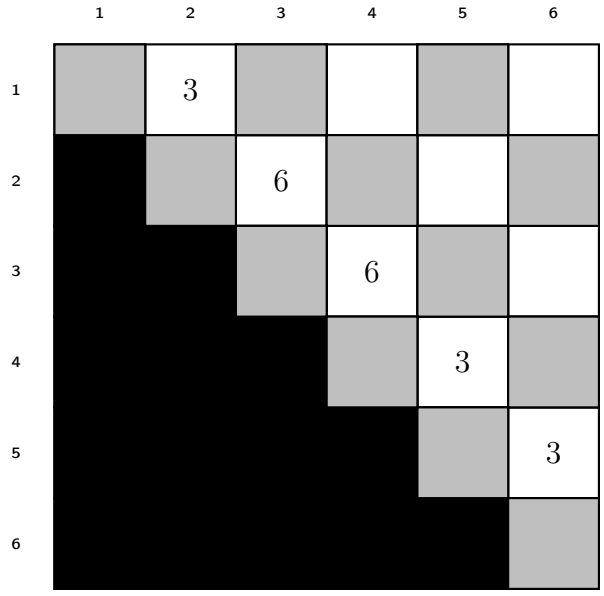
Да разгледаме началните условия

$$\mathrm{opt}[i, i+1] = \max\{M[i], M[i+1]\}$$

за $1 \leqslant \mathtt{i} \leqslant 5$. Тогава $\mathrm{opt}[1,2] = \max\{1,3\} = 3$, така че имаме:



Аналогично, $\mathrm{opt}[2,3] = \max\{3,6\} = 6$, $\mathrm{opt}[3,4] = \max\{6,3\} = 6$, $\mathrm{opt}[4,5] = \max\{3,1\} = 3$ и $\mathrm{opt}[5,6] = \max\{1,3\} = 3$, така че в диагонала над главния имаме:



С това приключихме с началните условия.

За клетка $[1,4]$, стойността се получава като

$$\begin{aligned}\mathrm{opt}[1,4] &= \max\big\{1 + \min\{\mathrm{opt}[2,3], \mathrm{opt}[3,4]\}, 3 + \min\{\mathrm{opt}[1,2], \mathrm{opt}[2,3]\}\big\} \\ &= \max\big\{1 + \min\{6,6\}, 3 + \min\{3,6\}\big\} \\ &= \max\big\{1+6, 3+6\big\} \\ &= 7\end{aligned}$$

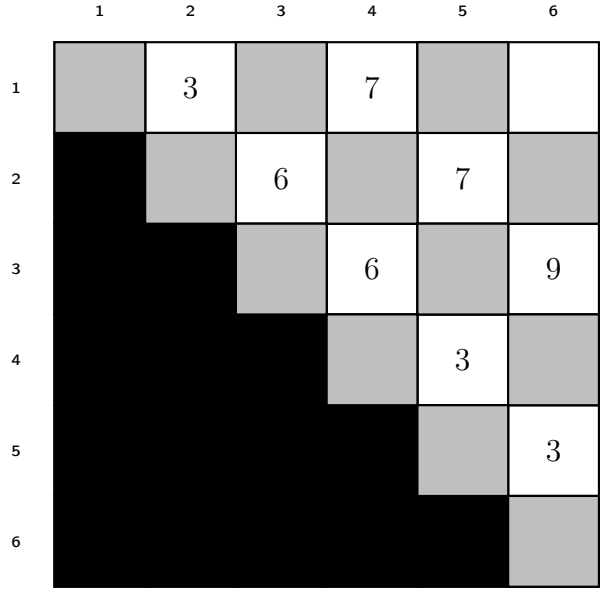
Наистина, върху вход $\langle 1, 3, 6, 3 \rangle$, Албена може да вземе най-много 7. Аналогично,

$$\begin{aligned}
\mathrm{opt}[2,5] &= \max\left\{3 + \min\left\{\mathrm{opt}[3,4], \mathrm{opt}[4,5]\right\}, 1 + \min\left\{\mathrm{opt}[2,3], \mathrm{opt}[3,4]\right\}\right\} \\
&= \max\left\{3 + \min\left\{6,3\right\}, 1 + \min\left\{6,6\right\}\right\} \\
&= \max\left\{3+3, 1+6\right\} \\
&= 7
\end{aligned}$$

и

$$\begin{aligned}
\mathrm{opt}[3,6] &= \max\left\{6 + \min\left\{\mathrm{opt}[4,5], \mathrm{opt}[5,6]\right\}, 3 + \min\left\{\mathrm{opt}[3,4], \mathrm{opt}[4,5]\right\}\right\} \\
&= \max\left\{6 + \min\left\{3,3\right\}, 3 + \min\left\{6,3\right\}\right\} \\
&= \max\left\{6+3, 3+3\right\} \\
&= 9
\end{aligned}$$

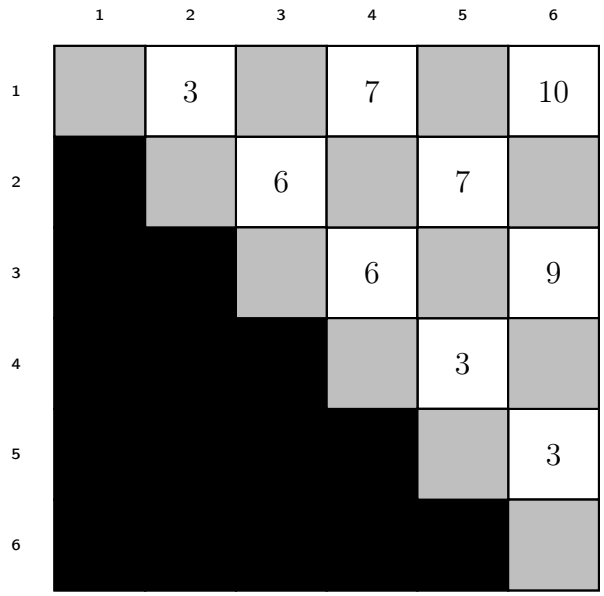
Имаме



Окончателно,

$$\begin{aligned}
\mathrm{opt}[1,6] &= \max\left\{1 + \min\left\{\mathrm{opt}[2,5], \mathrm{opt}[3,6]\right\}, 3 + \min\left\{\mathrm{opt}[1,4], \mathrm{opt}[2,5]\right\}\right\} \\
&= \max\left\{1 + \min\left\{7,9\right\}, 3 + \min\left\{7,7\right\}\right\} \\
&= \max\left\{1+7, 3+7\right\} \\
&= 10
\end{aligned}$$

Таблицата изглежда така:



Решението е 10.

13.10 Мемоизация

Името на тази секция не е сгрешено. Не става дума за мемо**р**изация, което е транслитерация на *memorisation* (по американския правопис е *memorization*) и означава запомняне в разговорния смисъл. *Memoization*, за което авторът на лекционните записки не разполага с по-подходяща българска дума от "мемоизация", е техника за ефикасна имплементация на алгоритмични идеи. Понятието е въведено от британския компютърен учен Donald Michie, работил през 60-те години на 20 век в областта на изкуствения интелект. В *статия от 1968 г.* той описва метод, с който "компютрите се учат от опита си". В секцията "Recursively Defined Functions", Michie описва изчисление на рекурсивни функции, при което резултатите от викания на дадена функция се съхраняват в таблица. При всяко викане на рекурсивната функция първо се проверява дали желаната стойност вече е била изчислена.

- Ако да, стойността се взема от таблицата.
- Ако не, започва изчисляването на стойността, но при виканията върху по-малки стойности на аргумента отново се проверява дали за тях стойността е била вече изчислена.

Лесно се вижда, че на фундаментално ниво идеята е същата като идеята на схемата Динамично Програмиране: подзадачите имат общи подподзадачи, и резултатите от работата върху подподзадачите се съхраняват, за да не бъдат пресмятани отново и отново и отново. Съществена разлика между мемоизацията и схемата Динамично Програмиране е посоката на изчислението. Мемоизацията е истинска рекурсия с "ход надолу" и "ход нагоре", само че резултатите от виканията се пазят. Динамичното програмиране е само втората част от рекурсията, "ходът нагоре".

Пример с пресмятането на биномни коефициенти Да разгледаме пресмятане на биномни коефициенти чрез рекурсия съгласно (13.2). Като среда ще ползваме Maple (TM).

Нека изчисляваме $\binom{30}{15}$, който е 155 117 520. Съгласно (13.2), рекурсията първо пресмята $\binom{29}{15}$, после пресмята $\binom{29}{14}$ и после ги сумира. За да се сметне $\binom{29}{15}$ се смятат $\binom{28}{15}$ и $\binom{28}{14}$, а за се сметне $\binom{29}{14}$ се смятат $\binom{28}{14}$ и $\binom{28}{13}$. Проблемът—аналогично на Секция 13.1.2—е в това, че

при връщането на стойността на $\binom{29}{15}$ изчислението е "забравило", че вече е разполагало със стойността на $\binom{28}{14}$, и започва да я смята наново, за да получи $\binom{29}{14}$, и това явление се появява **навсякъде в дървото на рекурсията**.

Може да избегнем този проблем като пълним таблица, подобна на Таблица 13.1, но може да използваме подхода на Michie. При него изчислението е истинска рекурсия, но всеки биномен коефициент, веднъж получен, се съхранява. Рекурсивната функция е имплементирана така, че когато бъде извикана с `n` и `k`, които искат рекурсивни викания, първо се извършва проверка дали вече разполагаме с желаната стойност. Само ако не разполагаме с тази стойност се извършват рекурсивните викания.

За да бъде имплементирана тази идея лесно на практика, трябва програмната среда да осигурява тези проверки **прозрачно** за нас. В противен случай—примерно, ако пишем на `C`—трябва ние да се погрижим за създаването на таблицата и проверяването в нея. Едно средство за програмиране, което поддържа мемоизация, е Maple(TM). Рекурсия с мемоизация в Maple(TM) се постига изключително лесно: дава се опция `remember` в дефиницията на рекурсивната процедура и след това Maple(TM) се грижи за таблицата и проверяването в нея. Следният пример с $\binom{30}{15}$ показва драстично повишаване на ефикасността на изчисленията с мемоизация в Maple.

Стандартният начин за измерване на чистото време за изчисление в Maple (TM) не е с гледане на часовника, а с използване на функцията `profile` с аргумент-името на функцията или процедурата, която искаме да профилираме. После с `showprofile` се извежда подробна информация за изразходваното изчислително време и памет. Но в примера с биномните коефициенти, това профилиране дава чисто изчислително време, което е в пъти над времето, което Maple използва, за да изчислява без профилиране (става дума за рекурсия без мемоизация; тази с мемоизация ползва, грубо казано, нула време за тези стойности на входа). За да измерим времето без ползване на `profile`, ползваме трика с функцията `time`.

```
Z:\algs> "c:\Program Files\Maple 2018\bin.X86_64_WINDOWS\cmaple.exe"
    |\^/|     Maple 2018 (X86 64 WINDOWS)
._|\|   |/|_. Copyright (c) Maplesoft, a division of Waterloo Maple Inc. 2018
 \  MAPLE  /  All rights reserved. Maple is a trademark of
 <____ ____>  Waterloo Maple Inc.
      |       Type ? for help.
> kernelopts(printbytes=false):
> mybinomial1 := proc(n::nonnegint, k::nonnegint)
> if k > n then 0
> else if k = 0 or k = n then 1
> else mybinomial1(n-1, k) + mybinomial1(n-1, k-1)
> fi; fi; end proc:
> mybinomial2 := proc(n::nonnegint, k::nonnegint) option remember;
> if k > n then 0
> else if k = 0 or k = n then 1
> else mybinomial2(n-1, k) + mybinomial2(n-1, k-1)
> fi; fi; end proc:
> mytime := time():
> mybinomial1(30,15);
                                                    155117520

> time() - mytime;
                                                     315.453
```

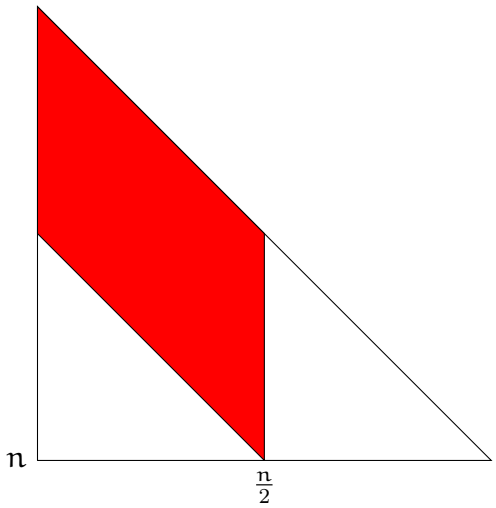
```
> mytime := time():
> mybinomial2(30,15);
                                                155117520

> time() - mytime;
                                                    0.
```

И така, `mybinomial1(30,15)` приключва работата си за 315.453 секунди, а `mybinomial2(30,15)` приключва работата си толкова бързо, че този начин на отчитане отчита нула секунди. Единствената разлика между тях е, че `mybinomial2` ползва опцията `remember`.

С мемоизацията трябва да се внимава. Ако изчислението просто трупа данни в таблици, които са огромни, в някакъв момент може да започне да не достига памет. Сякаш този потенциален проблем е по-голям при мемоизацията в сравнение с динамичното програмиране, защото при динамичното разработчикът манипулира таблиците явно и е напълно наясно за тяхното съществуване и възможни последици от не-освобождаването на паметта, използвана за тях. От друга страна, това, че мемоизацията става прозрачно за разработчика, може да го/я изкуши да забрави за нея.

Едно потенциално предимство на мемоизацията пред динамичното програмиране е, че за някои изчисления не ни трябват всички стойности за по-малки аргументи, а само някои. Образно, ако смятаме $\binom{n}{n/2}$, изчислението с мемоизация запълва само тази част от таблицата, която е оцветена в червено:



Докато динамичното програмиране би "напълнило" цялата таблица до ред n включително.

13.11 За границите на възможностите на динамичното програмиране

Алгоритмичната схема Динамично Програмиране е чудесно средство за алгоритмичен дизайн. Вече видяхме немалко примери за задачи, които на пръв поглед са невъзможно трудни, но умелото прилагане на схемата Динамично Програмиране дава ефикасни алгоритми, които са и лесни за разбиране, ако човек разбере рекурсията. За новаци в областта на алгоритмичния дизайн конструирането на такива алгоритми е нещо като магия.

За съжаление или за щастие, тази магия работи само донякъде. Тя има своите ограничения. Тук ще разгледаме един пример от книгата "The Algorithm Design Manual" на Skiena (вж. [86, стр. 303–304]) за алгоритъм по схемата Динамично Програмиране, който има експоненциална сложност. В което няма нищо странно, понеже задачата е **NP**-трудна; странно би било да се намери ефикасен алгоритъм за нея. Изводът е, че алгоритмичната неподатливост отказва да изчезне при магическите думи "динамично програмиране" и "динамично програмиране" не е непременно синоним на "ефикасно решение".

Задачата е Най-Дълъг Път. Става дума за тегловни графи. Тегло на път е, както винаги, сумата от теглата на ребрата.

Изч. Задача: Longest Path

пример: Ориентиран граф $G = (V, E)$, тегловна функция w, начален връх s, краен връх t.
решение: Дължина на най-дълъг прост път от s до t в G.

Да разгледаме следната рекурсия. Нека $L(i, j)$ означава дъжината на най-дълъг прост път от i до j. Тъй като говорим за прости пътища, има смисъл да искаме $i \neq j$. Всеки такъв най-дълъг път има някакъв предпоследен връх x, поради което можем да се изкушим да кажем:

$$L(i, j) = \max_{(x,j)\in E} \{L(i, x) + w(x, j)\}$$

Тази рекурсия обаче има два проблема, всеки от които я прави безполезна. Първо, нищо в нея не пречи да бъдат повтаряни върхове, тоест, пътят, чиято дължина се има предвид, да не е прост. Второ, не е ясно в какъв ред върви изчислението, поради което практическата ѝ имплементация може да зацикли.

Коректна рекурсия е следната. Нека $LP(i, j, P)$ означава дължината на най-дълъг път от i до j, в който P е редицата от междинните върхове. Щом пътят е прост, P не трябва да съдържа нито i, нито j, нито в P може да има повтаряне на върхове.

$$LP(i, j, P \oplus x) = \max_{(x,j)\in E, j\notin P} \{L(i, x, P) + w(x, j)\}$$

"$P \oplus x$" означава редицата P с "долепен" x в края. $L(i, x, P)$ вдясно изключва възможността $x \in P$, така че е достатъчно да сме сигурни, че $j \notin P$, което присъства като ограничение в максимума.

Тази рекурсия е очевидно коректна, но нейната алгоритмична имплементация би била твърде неефикасна. От общи съображения може да кажем, че има не повече от $(n-3)!$ възможности за P[†], поради което таблицата на динамичното програмиране би била непрактично огромна дори за n от порядъка на няколко десетки.

Малко подобрение може да се получи, ако не помним пермутации, а множества. Всъщност, P се използва само за да се избегне повтаряне на върхове. Ако въведем друга рекурсия $\widetilde{L}(i, j, S)$ със смисъл на дължина на най-къс път от i до j, в който S е множеството от междинните върхове, може да кажем:

[†] P не може да има повторения и не съдържа нито i, нито j, нито x.

$$\widetilde{L}(i, j, S \cup \{x\}) = \max_{(x,j)\in E, j\notin S} \{\widetilde{L}(i, x, S) + w(x, j)\}$$

Пространството от състоянията става от порядъка на 2^n, което е известно подобрение спрямо факториела. Въпреки това, алгоритъмът, съответен на рекурсията, продължава да е неефикасен.

Наблюдение 56

Схемата Динамично Програмиране не е всемогъща. Тя има своите естествени ограничения. За задачачи от класа **NP**-hard, конструирането на алгоритъм по тази схема не ни дава ефикасно решение, понеже пространството от състоянията е необозримо грамадно.

 © Минко Марков, 2021г.

Лекция 14

Алгоритмична неподатливост. NP-пълнота и NP-трудност.

 © Минко Марков, 2021г.

Лекция 15

SAT и теоремата на Cook.

Лекция 16

Основни NP-пълни задачи.

Лекция 17

Заобикаляне на неподатливостта: апроксимиране и параметризиране.

Благодарности

Благодарности на **Добрин Цветков** за откритите и коригирани грешки. Благодарности на **Емилиян Рогачев** за детайлното четене плюс откриване и коригиране на купища грешки. Благодарности на **Теодор Грозданов** за откритата и коригирана грешка в алгоритъма Dummy–Goldbach на стр. 29. Благодарности на **Борис Дишов** за откритата и коригирана грешка в имплементацията на Quick Sort. Благодарности на **Йордан Петров** за откритата и коригирана грешка в доказателството за коректност на алгоритъма Нарастване с единица на стр. 31, както и на грешки във формулировките на Лема 8 и Лема 11. Благодарности на **Костадин Гаров** за откритата грешка в доказателството на Лема 23. Благодарности на **Ясен Алексиев** за откритата грешка в доказателството на Теорема 29. Благодарности на **доктор Добромир Кралчев** за посочването на факта, че отсъствието на малко-о отношение между две функции не влече голямо-Омега отношение между тях (вж. Допълнение 37). Благодарности на **Иван-Асен Чакъров** за откритите грешки в рекапитулацията на графите. Благодарности на **Ива Петрова Караджова** за откритите грешки във формулировката на Наблюдение 38. Благодарности на **Радослав Георгиев** за откритата грешка в дефиницията на "дървесно ребро". Благодарности на **доктор Добромир Кралчев** за посочването на по-елегантно и илюминиращо доказателство на Лема 35. Благодарности на **Николай Пашов** за откритата и коригирана грешка в заключението на доказателството на Теорема 23. Благодарности на **Бойко Красимиров Борисов** за открит неверен резултат с невалидно "доказателство" в Допълнение 23. Благодарности на **Христо Трендафилов** за откритата груба грешка във формулировката и доказателството на Лема 29. Благодарности на **Цветан Алексиев** за откритата грешка в алгоритъма Partition от Подсекция 5.4.2. Благодаря на **Ивайло Панайтов** за откритата и коригирана грешка в записа на Лема 25. Благодарности на **доцент Стефан Герджиков**, който посочи на автора, че на български се казва "инвариант" в мъжки род, а не "инварианта" в женски род – който термин авторът поначало използваше.

Библиография

[1] Alfred V. Aho, John E. Hopcroft и Jeffrey Ullman. *Data Structures and Algorithms*. 1st. USA: Addison-Wesley Longman Publishing Co., Inc., 1983. ISBN: 0201000237.

[2] Alfred V. Aho, John E. Hopcroft и Jeffrey Ullman. *The Design and Analysis of Computer Algorithms*. 1st. USA: Addison-Wesley Longman Publishing Co., Inc., 1974. ISBN: 0201000296.

[3] Martin Aigner и Günter M. Ziegler. *Proofs from THE BOOK*. 4th. Springer Publishing Company, Incorporated, 2009. ISBN: 3642008550.

[4] Mohamad Akra и Louay Bazzi. "On the Solution of Linear Recurrence Equations". В: *Computational Optimization and Applications* 10.2 (1998), с. 195—210. ISSN: 0926-6003.

[5] David Aldous и Persi Diaconis. "Longest increasing subsequences: from patience sorting to the Baik-Deift-Johansson theorem". В: *BULL. AMER. MATH. SOC. (N.S* (1999). Достъпна онлайн на https://www.ams.org/journals/bull/1999-36-04/S0273-0979-99-00796-X/S0273-0979-99-00796-X.pdf, с. 413—432.

[6] Giorgio Ausiello и др. *Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties*. 1st. Berlin, Heidelberg: Springer-Verlag, 1999. ISBN: 3540654313.

[7] Edward G. Belaga и Maurice Mignotte. "Walking Cautiously Into the Collatz Wilderness: Algorithmically, Number Theoretically, Randomly". В: *Discrete Mathematics & Theoretical Computer Science* DMTCS Proceedings vol. AG, Fourth Colloquium on Mathematics and Computer Science Algorithms, Trees, Combinatorics and Probabilities (ян. 2006). Достъпна онлайн на https://dmtcs.episciences.org/3512.

[8] Richard Bellman. *Eye of the Hurricane: An Autobiography*. Series in modern applied mathematics. World Scientific, 1984. ISBN: 9789971966010.

[9] Charles H. Bennett. "Notes on Landauer's principle, reversible computation, and Maxwell's Demon". В: *Studies in History and Philosophy of Science Part B: Studies in History and Philosophy of Modern Physics* 34.3 (2003). Достъпна онлайн на https://www.sciencedirect.com/science/article/pii/S135521980300039X, с. 501—510. ISSN: 1355-2198. DOI: https://doi.org/10.1016/S1355-2198(03)00039-X.

[10] Jon Bentley. "Programming Pearls: Little Languages". В: *Commun. ACM* 29.8 (авг. 1986), с. 711—721. ISSN: 0001-0782. DOI: 10.1145/6424.315691. URL: http://doi.acm.org/10.1145/6424.315691.

[11] Jon Louis Bentley, Dorothea Haken и James B. Saxe. "A general method for solving divide-and-conquer recurrences". В: 12 (1980). Достъпна онлайн на https://apps.dtic.mil/dtic/tr/fulltext/u2/a064294.pdf, с. 36—44.

[12] Jon Louis Bentley и Michael Ian Shamos. “Divide-and-Conquer in Multidimensional Space”. В: *Proceedings of the Eighth Annual ACM Symposium on Theory of Computing*. STOC ’76. Достъпна онлайн на http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.366.9611&rep=rep1&type=pdf. Hershey, Pennsylvania, USA: Association for Computing Machinery, 1976, с. 220—230. ISBN: 9781450374149. DOI: 10.1145/800113.803652. URL: https://doi.org/10.1145/800113.803652.

[13] Garrett Birkhoff. *Lattice Theory*. 3rd. American Mathematical Society, 1967.

[14] Andreas Blass и Yuri Gurevich. “Algorithms: A quest for absolute definitions”. В: *Bulletin of the European Association for Theoretical Computer Science* (2003). Достъпна онлайн на http://research.microsoft.com/en-us/um/people/gurevich/Opera/164.pdf.

[15] Manuel Blum. “A Machine-Independent Theory of the Complexity of Recursive Functions”. В: *J. ACM* 14.2 (апр. 1967), с. 322—336. ISSN: 0004-5411. DOI: 10.1145/321386.321395. URL: https://doi.org/10.1145/321386.321395.

[16] Manuel Blum и др. “Linear Time Bounds for Median Computations”. В: *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*. STOC ’72. Достъпна онлайн на http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.309.9712&rep=rep1&type=pdf. Denver, Colorado, USA: Association for Computing Machinery, 1972, с. 119—124. ISBN: 9781450374576. DOI: 10.1145/800152.804904.

[17] Manuel Blum и др. “Time bounds for selection”. В: *Journal of Computer and System Sciences* 7.4 (1973). Достъпна онлайн на https://www.sciencedirect.com/science/article/pii/S0022000073800339, с. 448—461. ISSN: 0022-0000. DOI: https://doi.org/10.1016/S0022-0000(73)80033-9.

[18] Kellogg S. Booth и George S. Lueker. “Linear Algorithms to Recognize Interval Graphs and Test for the Consecutive Ones Property”. В: *Proceedings of the Seventh Annual ACM Symposium on Theory of Computing*. STOC ’75. Достъпна онлайн на https://doi.org/10.1145/800116.803776. Albuquerque, New Mexico, USA: Association for Computing Machinery, 1975, с. 255—265. ISBN: 9781450374194. DOI: 10.1145/800116.803776.

[19] Neville Campbell. Непубликувана, достъпна онлайн на https://nevillecampbell.com/Recurrences.pdf.

[20] Louis Comtet. *Analyse Combinatoire Tome II*. Presses Universitaires de France, 1970. ISBN: 978-2130309819.

[21] Thomas H. Cormen и др. *Introduction to Algorithms, Third Edition*. 3rd. The MIT Press, 2009. ISBN: 9780262033848.

[22] Thomas T. Cormen, Charles E. Leiserson и Ronald L. Rivest. *Introduction to Algorithms*. 1st edition. Cambridge, MA, USA: MIT Press, 1990. ISBN: 0-262-03141-8.

[23] Saber N. Elaydi. *An Introduction to Difference Equations*. 3rd. Springer-Verlag New York, 2005. ISBN: 978-0-387-23059-7. DOI: 10.1007/0-387-27602-5.

[24] Jeff Erickson. “Lower Bounds for Linear Satisfiability Problems”. В: *Proceedings of the Sixth Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA ’95. Достъпна онлайн на https://jeffe.cs.illinois.edu/pubs/pdf/linsat.pdf. San Francisco, California, USA: Society for Industrial и Applied Mathematics, 1995, с. 388—395. ISBN: 0898713498.

[25] Robert W. Floyd. “Algorithm 245: Treesort”. В: *Communications of the ACM* 7.12 (дек. 1964). Достъпна онлайн на http://doi.acm.org/10.1145/355588.365103, с. 701—. ISSN: 0001-0782. DOI: 10.1145/355588.365103.

[26] Robert W. Floyd. “Algorithm 97: Shortest Path”. В: *Commun. ACM* 5.6 (юни 1962). Достъпна онлайн на https://dl.acm.org/doi/10.1145/367766.368168, с. 345. ISSN: 0001-0782. DOI: 10.1145/367766.368168.

[27] Lance Fortnow. “The Status of the P Versus NP Problem”. В: *Commun. ACM* 52.9 (септ. 2009). Достъпна онлайн на http://dl.acm.org/citation.cfm?id=1562186, с. 78—86. ISSN: 0001-0782. DOI: 10.1145/1562164.1562186. URL: http://doi.acm.org/10.1145/1562164.1562186.

[28] J.B. Fraleigh, R.A. Beauregard и V.J. Katz. *Linear Algebra*. 3rd. Addison-Wesley, 1995. ISBN: 0-201-52675-1. URL: https://books.google.bg/books?id=ViqBvgAACAAJ.

[29] Michael L. Fredman. “On computing the length of longest increasing subsequences”. В: *Discrete Mathematics* 11.1 (1975). Достъпна онлайн на https://www.sciencedirect.com/science/article/pii/0012365X7590103X, с. 29—35. ISSN: 0012-365X. DOI: https://doi.org/10.1016/0012-365X(75)90103-X.

[30] Anka Gajentaan и Mark H. Overmars. “On a class of O(n2) problems in computational geometry”. В: *Computational Geometry* 45.4 (2012). Достъпна онлайн на https://www.sciencedirect.com/science/article/pii/S0925772111000927/pdfft?md5=32a2f8abc82822ff176112364be58329&pid=1-s2.0-S0925772111000927-main.pdf, с. 140—152. ISSN: 0925-7721. DOI: https://doi.org/10.1016/j.comgeo.2011.11.006.

[31] M. R. Garey и D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness (Series of Books in the Mathematical Sciences)*. First Edition. W. H. Freeman, 1979. ISBN: 0716710455. URL: http://www.amazon.com/Computers-Intractability-NP-Completeness-Mathematical-Sciences/dp/0716710455.

[32] Omer Gold и Micha Sharir. “Improved Bounds for 3SUM, k-SUM, and Linear Degeneracy”. В: *25th Annual European Symposium on Algorithms (ESA 2017)*. Под ред. на Kirk Pruhs и Christian Sohler. Т. 87. Leibniz International Proceedings in Informatics (LIPIcs). Достъпна онлайн на https://arxiv.org/abs/1512.05279. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017, 42:1—42:13. ISBN: 978-3-95977-049-1. DOI: 10.4230/LIPIcs.ESA.2017.42.

[33] M.C. Golumbic. *Algorithmic Graph Theory and Perfect Graphs*. ISSN. Elsevier Science, 2004. ISBN: 9780080526966. URL: https://books.google.bg/books?id=8xo-VrWo5%5C_QC.

[34] Ronald L. Graham, Donald E. Knuth и Oren Patashnik. *Concrete Mathematics*. second. Addison-Wesley, 1994. ISBN: 0-201-55802-5.

[35] T.L. Heath и J.L. Heiberg. *Books 3-9*. The Thirteen Books of Euclid’s Elements. Достъпна онлайн на http://www.wilbourhall.org/pdfs/Heath_Euclid_II.pdf. The University Press, 1908.

[36] C. A. R. Hoare. “Algorithm 63: Partition”. В: *Commun. ACM* 4.7 (юли 1961), с. 321—. ISSN: 0001-0782. DOI: 10.1145/366622.366642. URL: http://doi.acm.org/10.1145/366622.366642.

[37] C. A. R. Hoare. “Algorithm 64: Quicksort”. В: *Commun. ACM* 4.7 (юли 1961), с. 321—. ISSN: 0001-0782. DOI: 10.1145/366622.366644. URL: http://doi.acm.org/10.1145/366622.366644.

[38] C. A. R. Hoare. “Quicksort”. В: *The Computer Journal* 5.1 (1962), с. 10—16. DOI: 10.1093/comjnl/5.1.10. eprint: http://comjnl.oxfordjournals.org/cgi/reprint/5/1/10.pdf. URL: http://comjnl.oxfordjournals.org/cgi/content/abstract/5/1/10.

[39] L. Hogben. *Handbook of Linear Algebra, Second Edition*. Discrete Mathematics and Its Applications. Taylor & Francis, 2013. ISBN: 9781466507289.

[40] Hagen Höpfner и Christian Bunse. "Towards an Energy-Consumption Based Complexity Classification for Resource Substitution Strategies". В: *Proceedings of the 22nd Workshop "Grundlagen von Datenbanken 2010", Bad Helmstedt, Germany, May 25-28, 2010*. Под ред. на Wolf-Tilo Balke и Christoph Lofi. Т. 581. CEUR Workshop Proceedings. Достъпна онлайн на https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.228.9932&rep=rep1&type=pdf. CEUR-WS.org, 2010.

[41] Hsien-Kuei Hwang и Jean-Marc Steyaert. *On the Number of Heaps and the Cost of Heap Construction*. Достъпна онлайн на http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.13.9150&rep=rep1&type=pdf. 2001.

[42] R. Impagliazzo. "A Personal View of Average-case Complexity". В: *Proceedings of the 10th Annual Structure in Complexity Theory Conference (SCT'95)*. SCT '95. Достъпна онлайн на http://www.cs.ucsd.edu/users/russell/average.ps. Washington, DC, USA: IEEE Computer Society, 1995, с. 134—. ISBN: 0-8186-7052-5. URL: http://dl.acm.org/citation.cfm?id=829497.829786.

[43] Ravi Jain, David Molnar и Zulfikar Ramzan. "Towards a Model of Energy Complexity for Algorithms". В: *IEEE Wireless Communications & Networking Conference* (2005).

[44] Alan S. Henry John N. Crossley. "Thus spake al-Khwārizmī: A translation of the text of Cambridge University Library Ms. Ii.vi.5". В: *Historia Mathematica* 17 (2 1990). Достъпна онлайн на http://www.sciencedirect.com/science/article/pii/031508609090048I, с. 103—131.

[45] David S Johnson. "The NP-completeness column: An ongoing guide". В: *Journal of Algorithms* 8.2 (1987), с. 285—303. ISSN: 0196-6774. DOI: https://doi.org/10.1016/0196-6774(87)90043-5. URL: http://www.sciencedirect.com/science/article/pii/0196677487900435.

[46] Yonggun Jun, Momčilo Gavrilov и John Bechhoefer. "High-Precision Test of Landauer's Principle in a Feedback Trap". В: *Physical Review Letters* 113.19 (ноем. 2014). ISSN: 1079-7114. DOI: 10.1103/physrevlett.113.190601. URL: http://dx.doi.org/10.1103/PhysRevLett.113.190601.

[47] A. B. Kahn. "Topological Sorting of Large Networks". В: *Commun. ACM* 5.11 (ноем. 1962). Статията е свободно достъпна на https://dl.acm.org/doi/pdf/10.1145/368996.369025?download=true, с. 558—562. ISSN: 0001-0782. DOI: 10.1145/368996.369025.

[48] Dan Kalman. "The Generalized Vandermonde Matrix". В: *Mathematics Magazine* 57.1 (1984). Достъпна онлайн на https://www.jstor.org/stable/2690290, с. 15—21. ISSN: 0025570X, 19300980.

[49] Richard M. Karp. "On-Line Algorithms Versus Off-Line Algorithms: How Much is It Worth to Know the Future?" В: *Proceedings of the IFIP 12th World Computer Congress on Algorithms, Software, Architecture - Information Processing '92, Volume 1 - Volume I*. Достъпна онлайн на http://www.icsi.berkeley.edu/pubs/techreports/TR-92-044.pdf. NLD: North-Holland Publishing Co., 1992, с. 416—429. ISBN: 044489747X.

[50] Edward C. Kirby и др. "What Kirchhoff Actually did Concerning Spanning Trees in Electrical Networks and its Relationship to Modern Graph-Theoretical Work". В: *Croatica Chemica Acta* 89.4 (2016). Достъпна онлайн на https://hrcak.srce.hr/file/255139, с. 403—417. DOI: 10.5562/cca2995.

[51] G. Kirchhoff. “On the Solution of the Equations Obtained from the Investigation of the Linear Distribution of Galvanic Currents”. В: *IRE Transactions on Circuit Theory* 5.1 (1958), с. 4—7.

[52] G. Kirchhoff. “Ueber die Auflösung der Gleichungen, auf welche man bei der Untersuchung der linearen Vertheilung galvanischer Ströme geführt wird”. В: *Annalen der Physik* 148.12 (1847). Достъпна чрез платен достъп или оторизирана институция на https://onlinelibrary.wiley.com/doi/abs/10.1002/andp.18471481202, с. 497—508. DOI: 10.1002/andp.18471481202.

[53] Donald E. Knuth. “Big Omicron and big Omega and big Theta”. В: *SIGACT News* 8.2 (1976). Достъпна онлайн на http://www.phil.uu.nl/datastructuren/10-11/knuth_big_omicron.pdf, с. 18—24. URL: http://doi.acm.org/10.1145/1008328.1008329.

[54] Donald E. Knuth. *The Art of Computer Programming, Volume 1 (3rd Ed.): Fundamental Algorithms*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1997. ISBN: 0-201-89683-4.

[55] Donald E. Knuth. *The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1997. ISBN: 0-201-89684-2.

[56] Donald E. Knuth. *The Art of Computer Programming, Volume 3 (2Nd Ed.): Sorting and Searching*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0-201-89685-0.

[57] Donald E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*. 1st. Addison-Wesley Professional, 2011. ISBN: 0201038048.

[58] Mark Korenblit и Vadim E. Levit. “A One-Vertex Decomposition Algorithm for Generating Algebraic Expressions of Square Rhomboids”. В: *CoRR* abs/1211.1661 (2012). Достъпна онлайн на http://arxiv.org/pdf/1211.1661v1.

[59] William Kuszmaul и Charles E. Leiserson. “Floors and Ceilings in Divide-and-Conquer Recurrences”. В: *Symposium on Simplicity in Algorithms (SOSA)*. Достъпна онлайн на https://epubs.siam.org/doi/abs/10.1137/1.9781611976496.15, с. 133—141. DOI: 10.1137/1.9781611976496.15.

[60] J C Lagarias. “The 3x+1 problem: An annotated bibliography (1963–1999)”. Достъпна онлайн на https://arxiv.org/abs/math/0608208. 2012.

[61] J C Lagarias. “The 3x+1 Problem: An Annotated Bibliography, II (2000-2009)”. Достъпна онлайн на https://arxiv.org/abs/math/0309224. 2011.

[62] Rolf Landauer. “Irreversibility and heat generation in the computing process”. В: *IBM Journal of Research and Development* 5.3 (1961). Достъпна, но не свободно, онлайн на https://ieeexplore.ieee.org/document/5392446, с. 183—191. DOI: 10.1147/rd.53.0183.

[63] Leighton. *Note on Better Master Theorems for Divide-and-Conquer Recurrences*. Достъпна онлайн http://courses.csail.mit.edu/6.046/spring04/handouts/akrabazzi.pdf. 1996.

[64] Seth Lloyd. “Ultimate physical limits to computation”. В: *Nature* 406 (6799 авг. 2000). Достъпна онлайн на https://arxiv.org/abs/quant-ph/9908043, с. 1047—1054. DOI: 10.1038/35023282.

[65] Krasimir Manev. *Uvod v Diskretnata Matematika*. fifth. KLMN – Krasimir Manev, 2012.

[66] Zohar Manna. *Mathematical Theory of Computation*. Dover Publications, Incorporated, 2003.

[67] Silvano Martello и Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. USA: John Wiley & Sons, Inc., 1990. ISBN: 0471924202.

[68] E.F. Moore. *The Shortest Path Through a Maze*. Bell Telephone System. Technical publications. monograph. Bell Telephone System., 1959. URL: https://books.google.com/books?id=IVZBHAAACAAJ.

[69] *tree (data structure)*. National Institute of Standards and Technology's web site. Достъпна онлайн на http://xlinux.nist.gov/dads/HTML/tree.html.

[70] k-*ary tree*. National Institute of Standards and Technology's web site. Достъпна онлайн на https://xlinux.nist.gov/dads/HTML/perfectKaryTree.html.

[71] Aurélien Ooms. "Algorithms and Data Structures for 3SUM and Friends". Достъпна онлайн на https://dipot.ulb.ac.be/dspace/bitstream/2013/293549/5/digital.pdf. Докт. ... дис. Université libre de Bruxelles, 2019.

[72] Christos H. Papadimitriou. *Computational Complexity*. Chichester, UK: John Wiley и Sons Ltd., с. 260—265. ISBN: 0-470-86412-5.

[73] Christos H. Papadimitriou и Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. USA: Prentice-Hall, Inc., 1982. ISBN: 0131524623.

[74] Ian Parberry. *Problems on Algorithms*. Второто издание е достъпно със специален лиценз на http://larc.unt.edu/ian/books/free/license.html. Prentice Hall PTR, 1995. ISBN: 9780134335582.

[75] Heinz-Otto Peitgen, Hartmut Jürgens и Dietmar Saupe. *Chaos and fractals - new frontiers of science (2. ed.)*. Springer, 2004, с. I—XIII, 1—864. ISBN: 978-0-387-20229-7.

[76] M. Pirlot и P. Vincke. *Semiorders: Properties, Representations, Applications*. Theory and Decision Library B. Springer Netherlands, 1997. ISBN: 9780792346173. URL: https://books.google.com/books?id=rXroQjbQHn0C.

[77] Dan Romik. "Stirling's Approximation for n!: The Ultimate Short Proof?" В: *The American Mathematical Monthly* 107.6 (2000). Достъпна онлайн на https://www.math.ucdavis.edu/~romik/data/uploads/papers/stirling.pdf, с. 556—557.

[78] John E. Savage. *Models of Computation: Exploring the Power of Computing*. 1st. USA: Addison-Wesley Longman Publishing Co., Inc., 1997. ISBN: 0201895390.

[79] Byron Schmuland. *Shouting Factorials!* Достъпна онлайн на http://www.math.ualberta.ca/pi/issue7/page10-12.pdf.

[80] Bruce Schneier. *Applied Cryptography (2Nd Ed.): Protocols, Algorithms, and Source Code in C*. New York, NY, USA: John Wiley & Sons, Inc., 1995. ISBN: 0-471-11709-9.

[81] A. Schrijver. *Combinatorial Optimization - Polyhedra and Efficiency*. Springer, 2003.

[82] Alexander Schrijver. *On the History of the Shortest Path Problem*. Достъпна онлайн на https://www.math.uni-bielefeld.de/documenta/vol-ismp/32_schrijver-alexander-sp.pdf. 2012.

[83] Robert Sedgewick и Philippe Flajolet. *An introduction to the analysis of algorithms*. Addison-Wesley-Longman, 1996, с. I—XV, 1—492. ISBN: 978-0-201-40009-0.

[84] Robert Sedgewick и Kevin Wayne. *Algorithms, 4th Edition*. Addison-Wesley, 2011, с. I—XII, 1—955. ISBN: 978-0-321-57351-3.

[85] Michael Sipser. *Introduction to the theory of computation: second edition*. 2-е изд. Boston: PWS Pub., 2006. ISBN: 978-0534950972.

[86] Steven S. Skiena. *The Algorithm Design Manual*. 2nd. Springer Publishing Company, Incorporated, 2008. ISBN: 9781848000698.

[87] R.P. Stanley. *Algebraic Combinatorics: Walks, Trees, Tableaux, and More*. Undergraduate Texts in Mathematics. Авторът е предоставил свободно копие на книгата, без упражненията, на адрес http://www-math.mit.edu/~rstan/algcomb/algcomb.pdf. Springer New York, 2013. ISBN: 9781461469988.

[88] Williard Stone. "Abacists versus Algorists". В: *Journal of Accounting Research Journal of Accounting Research* 10.2 (1972). Достъпна онлайн на https://www.jstor.org/stable/2490013, с. 345—350. DOI: 10.2307/2490013.

[89] Robert Endre Tarjan. "Edge-Disjoint Spanning Trees and Depth-First Search". В: *Acta Informatica* 6.2 (юни 1976). По-стара версия на статията е свободно достъпна като technical report на http://i.stanford.edu/pub/cstr/reports/cs/tr/74/455/CS-TR-74-455.pdf, с. 171—185. ISSN: 0001-5903. DOI: 10.1007/BF00268499.

[90] Alan M. Turing. "On Computable Numbers, with an application to the Entscheidungsproblem". В: *Proceedings of the London Mathematical Society*. 2-я пор. 42 (1936). Достъпна онлайн на http://www.dna.caltech.edu/courses/cs129/caltech_restricted/Turing_1936_IBID.pdf, с. 230—265.

[91] Hui Wang и др. "Boson Sampling with 20 Input Photons and a 60-Mode Interferometer in a 10^{14} state spaces". В: *Physical Review Letters* 123.25 (дек. 2019). Достъпна онлайн на https://arxiv.org/abs/1910.09930. ISSN: 1079-7114. DOI: 10.1103/physrevlett.123.250503.

[92] Y. Wang. *The Goldbach Conjecture*. Series in pure mathematics. World Scientific, 2002. ISBN: 9789812776600. URL: https://books.google.com/books?id=VAY9nTreXkcC.

[93] Stephen Warshall. "A Theorem on Boolean Matrices". В: *J. ACM* 9.1 (ян. 1962). Достъпна онлайн на https://dl.acm.org/doi/10.1145/321105.321107, с. 11—12. ISSN: 0004-5411. DOI: 10.1145/321105.321107.

[94] Lutz M. Wegner. *Sorting – The Turku Lectures*. Достъпна онлайн на http://tucs.fi/publications/attachment.php?fname=bWegner_LutzMx14a.full.pdf. 2014.

[95] Andrew John Wiles. "Modular Elliptic Curves and Fermat's Last Theorem". В: *ANNALS OF MATH* 141 (1995). Достъпна онлайн на http://math.stanford.edu/%7Elekheng/flt/wiles-small.pdf, с. 141.

[96] J. W. J. Williams. "Algorithm 232: Heapsort". В: *Communications of the ACM* 7.6 (1964), с. 347—348.

[97] Gerhard J. Woeginger и Zhongliang Yu. "On the Equal-Subset-Sum Problem". В: *Inf. Process. Lett.* 42.6 (юли 1992), с. 299—302. ISSN: 0020-0190. DOI: 10.1016/0020-0190(92)90226-L.